

All Cpp Solutions

Count: 3495

1. 1.cpp

Path: LeetCode-main/solutions/1. Two Sum/1.cpp

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> numToIndex;

        for (int i = 0; i < nums.size(); ++i) {
            if (const auto it = numToIndex.find(target - nums[i]);
                it != numToIndex.cend())
                return {it->second, i};
            numToIndex[nums[i]] = i;
        }

        throw;
    }
};
```

2. 10.cpp

Path: LeetCode-main/solutions/10. Regular Expression Matching/10.cpp

```
class Solution {
public:
    bool isMatch(string s, string p) {
        const int m = s.length();
        const int n = p.length();
        // dp[i][j] := true if s[0..i) matches p[0..j)
        vector<vector<bool>> dp(m + 1, vector<bool>(n + 1));
        dp[0][0] = true;

        auto isMatch = [&](int i, int j) -> bool {
            return j >= 0 && p[j] == '.' || s[i] == p[j];
        };

        for (int j = 0; j < p.length(); ++j)
            if (p[j] == '*' && dp[0][j - 1])
                dp[0][j + 1] = true;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (p[j] == '*') {
                    // The minimum index of '*' is 1.
                    const bool noRepeat = dp[i + 1][j - 1];
                    const bool doRepeat = isMatch(i, j - 1) && dp[i][j + 1];
                    dp[i + 1][j + 1] = noRepeat || doRepeat;
                } else if (isMatch(i, j)) {
                    dp[i + 1][j + 1] = dp[i][j];
                }

        return dp[m][n];
    }
};
```

3. 100.cpp

Path: LeetCode-main/solutions/100. Same Tree/100.cpp

```
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (!p || !q)
            return p == q;
        return p->val == q->val && //
            isSameTree(p->left, q->left) && //
            isSameTree(p->right, q->right);
    }
};
```

4. 1000-2.cpp

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000-2.cpp

```
class Solution {
public:
    int mergeStones(vector<int>& stones, int K) {
        const int n = stones.size();
        if ((n - 1) % (K - 1))
            return -1;

        constexpr int kMax = 1'000'000'000;
        // dp[i][j][k] := the minimum cost to merge stones[i..j] into k piles
        vector<vector<vector<int>>>> dp(
            n, vector<vector<int>>(n, vector<int>(K + 1, kMax)));
        vector<int> prefix(n + 1);

        for (int i = 0; i < n; ++i)
            dp[i][i][1] = 0;

        partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                for (int k = 2; k <= K; ++k)
                    for (int m = i; m < j; m += K - 1)
                        dp[i][j][k] = min(dp[i][j][k], dp[i][m][1] + dp[m + 1][j][k - 1]);
                dp[i][j][1] = dp[i][j][K] + prefix[j + 1] - prefix[i];
            }

        return dp[0][n - 1][1];
    }
};
```

5. 1000-3.cpp

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000-3.cpp

```
class Solution {
public:
    int mergeStones(vector<int>& stones, int k) {
        const int n = stones.size();
        if ((n - 1) % (k - 1))
            return -1;

        vector<vector<int>> mem(n, vector<int>(n, kMax));
        vector<int> prefix(n + 1);

        partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);
        const int cost = mergeStones(stones, 0, n - 1, k, prefix, mem);
        return cost == kMax ? -1 : cost;
    }

private:
    static constexpr int kMax = 1'000'000'000;

    // Returns the minimum cost to merge stones[i..j].
    int mergeStones(const vector<int>& stones, int i, int j, int k,
                    const vector<int>& prefix, vector<vector<int>>& mem) {
        if (j - i + 1 < k)
            return 0;
        if (mem[i][j] != kMax)
            return mem[i][j];

        for (int m = i; m < j; m += k - 1)
            mem[i][j] =
                min(mem[i][j], mergeStones(stones, i, m, k, prefix, mem) +
                    mergeStones(stones, m + 1, j, k, prefix, mem));

        if ((j - i) % (k - 1) == 0)
            mem[i][j] += prefix[j + 1] - prefix[i];

        return mem[i][j];
    }
};
```

6. 1000-4.cpp

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000-4.cpp

```
class Solution {
public:
    int mergeStones(vector<int>& stones, int K) {
        const int n = stones.size();
        if ((n - 1) % (K - 1))
            return -1;

        constexpr int kMax = 1'000'000'000;

        // dp[i][j] := the minimum cost to merge stones[i..j]
        vector<vector<int>> dp(n, vector<int>(n, kMax));
        vector<int> prefix(n + 1);

        for (int i = 0; i < n; ++i)
            dp[i][i] = 0;

        partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                for (int m = i; m < j; m += K - 1)
                    dp[i][j] = min(dp[i][j], dp[i][m] + dp[m + 1][j]);
                if (d % (K - 1) == 0)
                    dp[i][j] += prefix[j + 1] - prefix[i];
            }

        return dp[0][n - 1];
    }
};
```

7. 1000.cpp

Path: LeetCode-main/solutions/1000. Minimum Cost to Merge Stones/1000.cpp

```
class Solution {
public:
    int mergeStones(vector<int>& stones, int K) {
        const int n = stones.size();
        vector<vector<vector<int>>>> mem(
            n, vector<vector<int>>>(n, vector<int>(K + 1, kMax)));
        vector<int> prefix(n + 1);
        partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);
        const int cost = mergeStones(stones, 0, n - 1, 1, K, prefix, mem);
        return cost == kMax ? -1 : cost;
    }

private:
    static constexpr int kMax = 1'000'000'000;

    // Returns the minimum cost to merge stones[i..j] into k piles.
    int mergeStones(const vector<int>& stones, int i, int j, int k, int K,
        const vector<int>& prefix, vector<vector<vector<int>>>& mem) {
        if ((j - i + 1 - k) % (K - 1))
            return kMax;
        if (i == j)
            return k == 1 ? 0 : kMax;
        if (mem[i][j][k] != kMax)
            return mem[i][j][k];
        if (k == 1)
            return mem[i][j][k] = mergeStones(stones, i, j, K, K, prefix, mem) +
                prefix[j + 1] - prefix[i];

        for (int m = i; m < j; m += K - 1)
            mem[i][j][k] =
                min(mem[i][j][k],
                    mergeStones(stones, i, m, 1, K, prefix, mem) +
                    mergeStones(stones, m + 1, j, k - 1, K, prefix, mem));

        return mem[i][j][k];
    }
};
```

8. 1001.cpp

Path: LeetCode-main/solutions/1001. Grid Illumination/1001.cpp

```
class Solution {
public:
    vector<int> gridIllumination(int n, vector<vector<int>>& lamps,
                                vector<vector<int>>& queries) {
        vector<int> ans;
        unordered_map<int, int> rows;
        unordered_map<int, int> cols;
        unordered_map<int, int> diag1;
        unordered_map<int, int> diag2;
        unordered_set<pair<int, int>, PairHash> lampsSet;

        for (const vector<int>& lamp : lamps) {
            const int i = lamp[0];
            const int j = lamp[1];
            if (lampsSet.insert({i, j}).second) {
                ++rows[i];
                ++cols[j];
                ++diag1[i + j];
                ++diag2[i - j];
            }
        }

        for (const vector<int>& query : queries) {
            const int i = query[0];
            const int j = query[1];
            if (rows[i] || cols[j] || diag1[i + j] || diag2[i - j]) {
                ans.push_back(1);
                for (int y = max(0, i - 1); y < min(n, i + 2); ++y)
                    for (int x = max(0, j - 1); x < min(n, j + 2); ++x)
                        if (lampsSet.erase({y, x})) {
                            --rows[y];
                            --cols[x];
                            --diag1[y + x];
                            --diag2[y - x];
                        }
            } else {
                ans.push_back(0);
            }
        }

        return ans;
    }

private:
    struct PairHash {
        size_t operator()(const pair<int, int>& p) const {
            return p.first ^ p.second;
        }
    };
};
```


9. 1002.cpp

Path: LeetCode-main/solutions/1002. Find Common Characters/1002.cpp

```
class Solution {
public:
    vector<string> commonChars(vector<string>& words) {
        vector<string> ans;
        vector<int> commonCount(26, INT_MAX);

        for (const string& word : words) {
            vector<int> count(26);
            for (const char c : word)
                ++count[c - 'a'];
            for (int i = 0; i < 26; ++i)
                commonCount[i] = min(commonCount[i], count[i]);
        }

        for (char c = 'a'; c <= 'z'; ++c)
            for (int i = 0; i < commonCount[c - 'a']; ++i)
                ans.push_back(string(1, c));

        return ans;
    }
};
```

10. 1003.cpp

Path: LeetCode-main/solutions/1003. Check If Word Is Valid After Substitutions/1003.cpp

```
class Solution {
public:
    bool isValid(string s) {
        stack<char> stack;

        for (const char c : s)
            if (c == 'c') {
                if (stack.size() < 2)
                    return false;
                if (stack.top() != 'b')
                    return false;
                stack.pop();
                if (stack.top() != 'a')
                    return false;
                stack.pop();
            } else {
                stack.push(c);
            }

        return stack.empty();
    }
};
```

11. 1004.cpp

Path: LeetCode-main/solutions/1004. Max Consecutive Ones III/1004.cpp

```
class Solution {
public:
    int longestOnes(vector<int>& nums, int k) {
        int ans = 0;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            if (nums[r] == 0)
                --k;
            while (k < 0)
                if (nums[l++] == 0)
                    ++k;
            ans = max(ans, r - l + 1);
        }
        return ans;
    }
};
```

12. 1005.cpp

Path: LeetCode-main/solutions/1005. Maximize Sum Of Array After K Negations/1005.cpp

```
class Solution {
public:
    int largestSumAfterKNegations(vector<int>& nums, int k) {
        ranges::sort(nums);

        for (int i = 0; i < nums.size(); ++i) {
            if (nums[i] > 0 || k == 0)
                break;
            nums[i] = -nums[i];
            --k;
        }

        return accumulate(nums.begin(), nums.end(), 0) -
            (k % 2) * ranges::min(nums) * 2;
    }
};
```

13. 1006.cpp

Path: LeetCode-main/solutions/1006. Clumsy Factorial/1006.cpp

```
class Solution {
public:
    int clumsy(int n) {
        if (n == 1)
            return 1;
        if (n == 2)
            return 2;
        if (n == 3)
            return 6;
        if (n == 4)
            return 7;
        if (n % 4 == 1)
            return n + 2;
        if (n % 4 == 2)
            return n + 2;
        if (n % 4 == 3)
            return n - 1;
        return n + 1;
    }
};
```

14. 1007.cpp

Path: LeetCode-main/solutions/1007. Minimum Domino Rotations For Equal Row/1007.cpp

```
class Solution {
public:
    int minDominoRotations(vector<int>& tops, vector<int>& bottoms) {
        const int n = tops.size();
        vector<int> countTops(7);
        vector<int> countBottoms(7);
        vector<int> countBoth(7);

        for (int i = 0; i < n; ++i) {
            ++countTops[tops[i]];
            ++countBottoms[bottoms[i]];
            if (tops[i] == bottoms[i])
                ++countBoth[tops[i]];
        }

        for (int i = 1; i <= 6; ++i)
            if (countTops[i] + countBottoms[i] - countBoth[i] == n)
                return n - max(countTops[i], countBottoms[i]);

        return -1;
    }
};
```

15. 1008.cpp

Path: LeetCode-main/solutions/1008. Construct Binary Search Tree from Preorder Traversal/1008.cpp

```
class Solution {
public:
    TreeNode* bstFromPreorder(vector<int>& preorder) {
        TreeNode* root = new TreeNode(preorder[0]);
        stack<TreeNode*> stack{{root}};

        for (int i = 1; i < preorder.size(); ++i) {
            TreeNode* parent = stack.top();
            TreeNode* child = new TreeNode(preorder[i]);
            // Adjust the parent.
            while (!stack.empty() && stack.top()->val < child->val)
                parent = stack.top(), stack.pop();
            // Create parent-child link according to BST property.
            if (parent->val > child->val)
                parent->left = child;
            else
                parent->right = child;
            stack.push(child);
        }

        return root;
    }
};
```

16. 1009.cpp

Path: LeetCode-main/solutions/1009. Complement of Base 10 Integer/1009.cpp

```
class Solution {
public:
    int bitwiseComplement(int n) {
        int mask = 1;
        while (mask < n)
            mask = (mask << 1) + 1;
        return mask ^ n;
    }
};
```


17. 101.cpp

Path: LeetCode-main/solutions/101. Symmetric Tree/101.cpp

```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        return isSymmetric(root, root);
    }

private:
    bool isSymmetric(TreeNode* p, TreeNode* q) {
        if (!p || !q)
            return p == q;

        return p->val == q->val && //
            isSymmetric(p->left, q->right) && //
            isSymmetric(p->right, q->left);
    }
};
```

18. 1010.cpp

Path: LeetCode-main/solutions/1010. Pairs of Songs With Total Durations Divisible by 60/1010.cpp

```
class Solution {
public:
    int numPairsDivisibleBy60(vector<int>& time) {
        int ans = 0;
        vector<int> count(60);

        for (int t : time) {
            t %= 60;
            ans += count[(60 - t) % 60];
            ++count[t];
        }

        return ans;
    }
};
```

19. 1011.cpp

Path: LeetCode-main/solutions/1011. Capacity To Ship Packages Within D Days/1011.cpp

```
class Solution {
public:
    int shipWithinDays(vector<int>& weights, int days) {
        int l = ranges::max(weights);
        int r = accumulate(weights.begin(), weights.end(), 0);

        while (l < r) {
            const int m = (l + r) / 2;
            if (shipDays(weights, m) <= days)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    int shipDays(const vector<int>& weights, int shipCapacity) {
        int days = 1;
        int capacity = 0;
        for (const int weight : weights)
            if (capacity + weight > shipCapacity) {
                ++days;
                capacity = weight;
            } else {
                capacity += weight;
            }
        return days;
    }
};
```

20. 1012.cpp

Path: LeetCode-main/solutions/1012. Numbers With Repeated Digits/1012.cpp

```
class Solution {
public:
    int numDupDigitsAtMostN(int n) {
        return n - countSpecialNumbers(n);
    }

private:
    // Same as 2376. Count Special Integers
    int countSpecialNumbers(int n) {
        const int digitSize = log10(n) + 1;
        vector<vector<vector<int>>> mem(
            digitSize + 1, vector<vector<int>>>(1 << 10, vector<int>(2, -1)));
        return count(to_string(n), 0, 0, true, mem) - 1; // - 0;
    }

private:
    // Returns the number of special integers, considering the i-th digit, where
    // `used` is the bitmask of the used digits, and `tight` indicates if the
    // current digit is tightly bound.
    int count(const string& s, int i, int used, bool tight,
              vector<vector<vector<int>>>& mem) {
        if (i == s.length())
            return 1;
        if (mem[i][used][tight] != -1)
            return mem[i][used][tight];

        int res = 0;
        const int maxDigit = tight ? s[i] - '0' : 9;

        for (int d = 0; d <= maxDigit; ++d) {
            // `d` is used.
            if (used >> d & 1)
                continue;
            // Use `d` now.
            const bool nextTight = tight && (d == maxDigit);
            if (used == 0 && d == 0) // Don't count leading 0s as used.
                res += count(s, i + 1, used, nextTight, mem);
            else
                res += count(s, i + 1, used | 1 << d, nextTight, mem);
        }

        return mem[i][used][tight] = res;
    }
};
```

21. 1013.cpp

Path: LeetCode-main/solutions/1013. Partition Array Into Three Parts With Equal Sum/1013.cpp

```
class Solution {
public:
    bool canThreePartsEqualSum(vector<int>& arr) {
        const int sum = accumulate(arr.begin(), arr.end(), 0);
        if (sum % 3 != 0)
            return false;

        const int average = sum / 3;
        int partCount = 0;
        int partSum = 0;

        for (const int a : arr) {
            partSum += a;
            if (partSum == average) {
                ++partCount;
                partSum = 0;
            }
        }

        // edge case: arr = [0, 0, 0, 0] -> partCount = 4.
        return partCount >= 3;
    }
};
```

22. 1014.cpp

Path: LeetCode-main/solutions/1014. Best Sightseeing Pair/1014.cpp

```
class Solution {
public:
    int maxScoreSightseeingPair(vector<int>& values) {
        int ans = 0;
        int bestPrev = 0;

        for (const int value : values) {
            ans = max(ans, value + bestPrev);
            bestPrev = max(bestPrev, value) - 1;
        }

        return ans;
    }
};
```

23. 1015.cpp

Path: LeetCode-main/solutions/1015. Smallest Integer Divisible by K/1015.cpp

```
class Solution {
public:
    int smallestRepunitDivByK(int k) {
        if (k % 10 != 1 && k % 10 != 3 && k % 10 != 7 && k % 10 != 9)
            return -1;

        unordered_set<int> seen;
        int n = 0;

        for (int length = 1; length <= k; ++length) {
            n = (n * 10 + 1) % k;
            if (n == 0)
                return length;
            if (seen.contains(n))
                return -1;
            seen.insert(n);
        }

        return -1;
    }
};
```

24. 1016.cpp

Path: LeetCode-main/solutions/1016. Binary String With Substrings Representing 1 To N/1016.cpp

```
class Solution {
public:
    bool queryString(string s, int n) {
        if (n > 1511)
            return false;

        for (int i = n; i > n / 2; --i) {
            string binary = bitset<32>(i).to_string();
            binary = binary.substr(binary.find("1"));
            if (s.find(binary) == string::npos)
                return false;
        }

        return true;
    }
};
```


25. 1017.cpp

Path: LeetCode-main/solutions/1017. Convert to Base -2/1017.cpp

```
class Solution {
public:
    string baseNeg2(int n) {
        string ans;

        while (n != 0) {
            ans += to_string(n % 2);
            n = -(n >> 1);
        }

        return ans.empty() ? "0" : string{ans.rbegin(), ans.rend()};
    }
};
```

26. 1018.cpp

Path: LeetCode-main/solutions/1018. Binary Prefix Divisible By 5/1018.cpp

```
class Solution {
public:
    vector<bool> prefixesDivBy5(vector<int>& nums) {
        vector<bool> ans;
        int curr = 0;

        for (const int num : nums) {
            curr = (curr * 2 + num) % 5;
            ans.push_back(curr % 5 == 0);
        }

        return ans;
    }
};
```

27. 1019.cpp

Path: LeetCode-main/solutions/1019. Next Greater Node In Linked List/1019.cpp

```
class Solution {
public:
    vector<int> nextLargerNodes(ListNode* head) {
        vector<int> ans;
        stack<int> stack;

        for (; head; head = head->next) {
            while (!stack.empty() && head->val > ans[stack.top()]) {
                int index = stack.top();
                stack.pop();
                ans[index] = head->val;
            }
            stack.push(ans.size());
            ans.push_back(head->val);
        }

        for (; !stack.empty(); stack.pop())
            ans[stack.top()] = 0;

        return ans;
    }
};
```

28. 102.cpp

Path: LeetCode-main/solutions/102. Binary Tree Level Order Traversal/102.cpp

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<vector<int>> ans;
        queue<TreeNode*> q{{root}};

        while (!q.empty()) {
            vector<int> currLevel;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode* node = q.front();
                q.pop();
                currLevel.push_back(node->val);
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
            ans.push_back(currLevel);
        }

        return ans;
    }
};
```

29. 1020.cpp

Path: LeetCode-main/solutions/1020. Number of Enclaves/1020.cpp

```
class Solution {
public:
    int numEnclaves(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();

        // Remove the lands connected to the edge.
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    if (grid[i][j] == 1)
                        dfs(grid, i, j);

        return accumulate(grid.begin(), grid.end(), 0,
            [](int acc, const vector<int>& row) {
                return acc + ranges::count(row, 1);
            });
    }

private:
    void dfs(vector<vector<int>>& grid, int i, int j) {
        if (i < 0 || i == grid.size() || j < 0 || j == grid[0].size())
            return;
        if (grid[i][j] == 0)
            return;
        grid[i][j] = 0;
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    };
};
```

30. 1021.cpp

Path: LeetCode-main/solutions/1021. Remove Outermost Parentheses/1021.cpp

```
class Solution {
public:
    string removeOuterParentheses(string s) {
        string ans;
        int opened = 0;

        for (const char c : s)
            if (c == '(') {
                if (++opened > 1)
                    ans += c;
            } else if (--opened > 0) { // c == ')'
                ans += c;
            }

        return ans;
    }
};
```

31. 1022.cpp

Path: LeetCode-main/solutions/1022. Sum of Root To Leaf Binary Numbers/1022.cpp

```
class Solution {
public:
    int sumRootToLeaf(TreeNode* root) {
        int ans = 0;
        dfs(root, 0, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, int val, int& ans) {
        if (root == nullptr)
            return;
        val = val * 2 + root->val;
        if (root->left == nullptr && root->right == nullptr)
            ans += val;
        dfs(root->left, val, ans);
        dfs(root->right, val, ans);
    }
};
```

32. 1023.cpp

Path: LeetCode-main/solutions/1023. Camelcase Matching/1023.cpp

```
class Solution {
public:
    vector<bool> camelMatch(vector<string>& queries, string pattern) {
        vector<bool> ans;
        for (const string& query : queries)
            ans.push_back(isMatch(query, pattern));
        return ans;
    }

private:
    bool isMatch(const string& query, const string& pattern) {
        int j = 0;
        for (const char c : query)
            if (j < pattern.length() && c == pattern[j])
                ++j;
            else if (isupper(c))
                return false;
        return j == pattern.length();
    }
};
```


33. 1024.cpp

Path: LeetCode-main/solutions/1024. Video Stitching/1024.cpp

```
class Solution {
public:
    int videoStitching(vector<vector<int>>& clips, int time) {
        int ans = 0;
        int end = 0;
        int farthest = 0;

        ranges::sort(clips);

        int i = 0;
        while (farthest < time) {
            while (i < clips.size() && clips[i][0] <= end)
                farthest = max(farthest, clips[i++][1]);
            if (end == farthest)
                return -1;
            ++ans;
            end = farthest;
        }
        return ans;
    }
};
```

34. 1025.cpp

Path: LeetCode-main/solutions/1025. Divisor Game/1025.cpp

```
class Solution {  
public:  
    bool divisorGame(int n) {  
        return n % 2 == 0;  
    }  
};
```

35. 1026.cpp

Path: LeetCode-main/solutions/1026. Maximum Difference Between Node and Ancestor/1026.cpp

```
class Solution {
public:
    int maxAncestorDiff(TreeNode* root) {
        return maxAncestorDiff(root, root->val, root->val);
    }

private:
    // Returns |the maximum - the minimum| of the tree.
    int maxAncestorDiff(TreeNode* root, int mn, int mx) {
        if (root == nullptr)
            return 0;
        mn = min(mn, root->val);
        mx = max(mx, root->val);
        const int l = maxAncestorDiff(root->left, mn, mx);
        const int r = maxAncestorDiff(root->right, mn, mx);
        return max({mx - mn, l, r});
    }
};
```

36. 1027.cpp

Path: LeetCode-main/solutions/1027. Longest Arithmetic Subsequence/1027.cpp

```
class Solution {
public:
    int longestArithSeqLength(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        // dp[i][k] := the length of the longest arithmetic subsequence of
        // nums[0..i] with k = diff + 500
        vector<vector<int>> dp(n, vector<int>(1001));

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < i; ++j) {
                const int k = nums[i] - nums[j] + 500;
                dp[i][k] = max(2, dp[j][k] + 1);
                ans = max(ans, dp[i][k]);
            }

        return ans;
    }
};
```

37. 1028.cpp

Path: LeetCode-main/solutions/1028. Recover a Tree From Preorder Traversal/1028.cpp

```
class Solution {
public:
    TreeNode* recoverFromPreorder(string traversal) {
        int i = 0;
        return recoverFromPreorder(traversal, 0, i);
    }

private:
    TreeNode* recoverFromPreorder(const string& traversal, int depth, int& i) {
        int nDashes = 0;
        while (i + nDashes < traversal.length() && traversal[i + nDashes] == '-')
            ++nDashes;
        if (nDashes != depth)
            return nullptr;

        i += depth;
        const int start = i;
        while (i < traversal.length() && isdigit(traversal[i]))
            ++i;

        return new TreeNode(stoi(traversal.substr(start, i - start)),
                           recoverFromPreorder(traversal, depth + 1, i),
                           recoverFromPreorder(traversal, depth + 1, i));
    }
};
```

38. 1029.cpp

Path: LeetCode-main/solutions/1029. Two City Scheduling/1029.cpp

```
class Solution {
public:
    int twoCitySchedCost(vector<vector<int>>& costs) {
        const int n = costs.size() / 2;
        int ans = 0;

        // How much money can we save if we fly a person to A instead of B?
        // To save money, we should
        // 1. Fly the person with the maximum saving to A.
        // 2. Fly the person with the minimum saving to B.

        // Sort `costs` in ascending order by the money saved if we fly a person to
        // B instead of A.
        ranges::sort(costs, ranges::less{},
            [](const vector<int>& cost) { return cost[0] - cost[1]; });

        for (int i = 0; i < n; ++i)
            ans += costs[i][0] + costs[i + n][1];

        return ans;
    }
};
```

39. 103-2.cpp

Path: LeetCode-main/solutions/103. Binary Tree Zigzag Level Order Traversal/103-2.cpp

```
class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<vector<int>> ans;
        queue<TreeNode*> q{{root}};
        bool isLeftToRight = true;

        while (!q.empty()) {
            const int size = q.size();
            vector<int> currLevel(size);
            for (int i = 0; i < size; ++i) {
                TreeNode* node = q.front();
                q.pop();
                const int index = isLeftToRight ? i : size - i - 1;
                currLevel[index] = node->val;
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
            ans.push_back(currLevel);
            isLeftToRight = !isLeftToRight;
        }

        return ans;
    }
};
```

40. 103.cpp

Path: LeetCode-main/solutions/103. Binary Tree Zigzag Level Order Traversal/103.cpp

```
class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<vector<int>> ans;
        deque<TreeNode*> dq{{root}};
        bool isLeftToRight = true;

        while (!dq.empty()) {
            vector<int> currLevel;
            for (int sz = dq.size(); sz > 0; --sz)
                if (isLeftToRight) {
                    TreeNode* node = dq.front();
                    dq.pop_front();
                    currLevel.push_back(node->val);
                    if (node->left)
                        dq.push_back(node->left);
                    if (node->right)
                        dq.push_back(node->right);
                } else {
                    TreeNode* node = dq.back();
                    dq.pop_back();
                    currLevel.push_back(node->val);
                    if (node->right)
                        dq.push_front(node->right);
                    if (node->left)
                        dq.push_front(node->left);
                }
            ans.push_back(currLevel);
            isLeftToRight = !isLeftToRight;
        }

        return ans;
    }
};
```


41. 1030.cpp

Path: LeetCode-main/solutions/1030. Matrix Cells in Distance Order/1030.cpp

```
class Solution {
public:
    vector<vector<int>> allCellsDistOrder(int rows, int cols, int rCenter,
                                       int cCenter) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        vector<vector<int>> ans;
        vector<vector<int>> seen(rows, vector<int>(cols));
        queue<pair<int, int>> q{{{rCenter, cCenter}}};
        seen[rCenter][cCenter] = true;

        while (!q.empty()) {
            const auto [i, j] = q.front();
            q.pop();
            ans.push_back({i, j});
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == rows || y < 0 || y == cols)
                    continue;
                if (seen[x][y])
                    continue;
                seen[x][y] = true;
                q.emplace(x, y);
            }
        }

        return ans;
    }
};
```

42. 1031.cpp

Path: LeetCode-main/solutions/1031. Maximum Sum of Two Non-Overlapping Subarrays/1031.cpp

```
class Solution {
public:
    int maxSumTwoNoOverlap(vector<int>& nums, int firstLen, int secondLen) {
        return max(helper(nums, firstLen, secondLen),
                    helper(nums, secondLen, firstLen));
    }

private:
    int helper(vector<int>& nums, int l, int r) {
        const int n = nums.size();
        vector<int> left(n);
        int sum = 0;

        for (int i = 0; i < n; ++i) {
            sum += nums[i];
            if (i >= l)
                sum -= nums[i - l];
            if (i >= l - 1)
                left[i] = i > 0 ? max(left[i - 1], sum) : sum;
        }

        vector<int> right(n);
        sum = 0;

        for (int i = n - 1; i >= 0; --i) {
            sum += nums[i];
            if (i <= n - r - 1)
                sum -= nums[i + r];
            if (i <= n - r)
                right[i] = i < n - 1 ? max(right[i + 1], sum) : sum;
        }

        int ans = 0;

        for (int i = 0; i < n - 1; ++i)
            ans = max(ans, left[i] + right[i + 1]);

        return ans;
    }
};
```

43. 1032.cpp

Path: LeetCode-main/solutions/1032. Stream of Characters/1032.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    bool isWord = false;
    TrieNode() : children(26) {}
};

class StreamChecker {
public:
    StreamChecker(vector<string>& words) {
        for (const string& word : words)
            insert(word);
    }

    bool query(char letter) {
        letters += letter;
        shared_ptr<TrieNode> node = root;

        for (int i = letters.length() - 1; i >= 0; --i) {
            const int index = letters[i] - 'a';
            if (node->children[index] == nullptr)
                return false;
            node = node->children[index];
            if (node->isWord)
                return true;
        }

        return false;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();
    string letters;

    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (int i = word.length() - 1; i >= 0; --i) {
            const int index = word[i] - 'a';
            if (node->children[index] == nullptr)
                node->children[index] = make_shared<TrieNode>();
            node = node->children[index];
        }
        node->isWord = true;
    }
};
```

44. 1033.cpp

Path: LeetCode-main/solutions/1033. Moving Stones Until Consecutive/1033.cpp

```
class Solution {
public:
    vector<int> numMovesStones(int a, int b, int c) {
        vector<int> nums = {a, b, c};

        ranges::sort(nums);

        if (nums[2] - nums[0] == 2)
            return {0, 0};
        return {min(nums[1] - nums[0], nums[2] - nums[1]) <= 2 ? 1 : 2,
                nums[2] - nums[0] - 2};
    }
};
```

45. 1034.cpp

Path: LeetCode-main/solutions/1034. Coloring A Border/1034.cpp

```
class Solution {
public:
    vector<vector<int>> colorBorder(vector<vector<int>>& grid, int r0, int c0,
                                  int color) {
        dfs(grid, r0, c0, grid[r0][c0]);

        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] < 0)
                    grid[i][j] = color;

        return grid;
    }

private:
    void dfs(vector<vector<int>>& grid, int i, int j, int startColor) {
        if (i < 0 || i == grid.size() || j < 0 || j == grid[0].size())
            return;
        if (grid[i][j] != startColor)
            return;

        grid[i][j] = -startColor;
        dfs(grid, i + 1, j, startColor);
        dfs(grid, i - 1, j, startColor);
        dfs(grid, i, j + 1, startColor);
        dfs(grid, i, j - 1, startColor);

        // If this cell is already on the boarder, it must be painted later.
        if (i == 0 || i == grid.size() - 1 || j == 0 || j == grid[0].size() - 1)
            return;

        if (abs(grid[i + 1][j]) == startColor && //
            abs(grid[i - 1][j]) == startColor && //
            abs(grid[i][j + 1]) == startColor && //
            abs(grid[i][j - 1]) == startColor)
            grid[i][j] = startColor;
    }
};
```

46. 1035.cpp

Path: LeetCode-main/solutions/1035. Uncrossed Lines/1035.cpp

```
class Solution {
public:
    int maxUncrossedLines(vector<int>& nums1, vector<int>& nums2) {
        const int m = nums1.size();
        const int n = nums2.size();
        vector<vector<int>> dp(m + 1, vector<int>(n + 1));

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                dp[i][j] = nums1[i - 1] == nums2[j - 1]
                    ? dp[i - 1][j - 1] + 1
                    : max(dp[i - 1][j], dp[i][j - 1]);

        return dp[m][n];
    }
};
```

47. 1036.cpp

Path: LeetCode-main/solutions/1036. Escape a Large Maze/1036.cpp

```
class Solution {
public:
    bool isEscapePossible(vector<vector<int>>& blocked, vector<int>& source,
                        vector<int>& target) {
        unordered_set<long> blockedSet;
        for (const vector<int>& b : blocked)
            blockedSet.insert(hash(b[0], b[1]));

        return dfs(blockedSet, source[0], source[1], hash(target[0], target[1]),
                    {}) &&
            dfs(blockedSet, target[0], target[1], hash(source[0], source[1]),
                {});
    }

private:
    bool dfs(unordered_set<long>& blockedSet, int i, int j, long target,
            unordered_set<long>&& seen) {
        if (i < 0 || i >= 1'000'000 || j < 0 || j >= 1'000'000 ||
            blockedSet.contains(hash(i, j)) || seen.contains(hash(i, j)))
            return false;

        seen.insert(hash(i, j));
        if (seen.size() > (1 + 199) * 199 / 2 || hash(i, j) == target)
            return true;
        return dfs(blockedSet, i + 1, j, target, std::move(seen)) ||
            dfs(blockedSet, i - 1, j, target, std::move(seen)) ||
            dfs(blockedSet, i, j + 1, target, std::move(seen)) ||
            dfs(blockedSet, i, j - 1, target, std::move(seen));
    }

    long hash(int i, int j) {
        return (static_cast<long>(i) << 16) + j;
    }
};
```

48. 1037.cpp

Path: LeetCode-main/solutions/1037. Valid Boomerang/1037.cpp

```
class Solution {
public:
    bool isBoomerang(vector<vector<int>>& points) {
        return (points[1][0] - points[0][0]) * (points[2][1] - points[1][1]) !=
            (points[1][1] - points[0][1]) * (points[2][0] - points[1][0]);
    }
};
```


49. 1038.cpp

Path: LeetCode-main/solutions/1038. Binary Search Tree to Greater Sum Tree/1038.cpp

```
class Solution {
public:
    TreeNode* bstToGst(TreeNode* root) {
        int prefix = 0;

        function<void(TreeNode*)> reversedInorder = [&](TreeNode* root) {
            if (root == nullptr)
                return;

            reversedInorder(root->right);

            root->val += prefix;
            prefix = root->val;

            reversedInorder(root->left);
        };

        reversedInorder(root);
        return root;
    }
};
```

50. 1039.cpp

Path: LeetCode-main/solutions/1039. Minimum Score Triangulation of Polygon/1039.cpp

```
class Solution {
public:
    int minScoreTriangulation(vector<int>& values) {
        const int n = values.size();
        vector<vector<int>> dp(n, vector<int>(n));

        for (int j = 2; j < n; ++j)
            for (int i = j - 2; i >= 0; --i) {
                dp[i][j] = INT_MAX;
                for (int k = i + 1; k < j; ++k)
                    dp[i][j] =
                        min(dp[i][j],
                           dp[i][k] + values[i] * values[k] * values[j] + dp[k][j]);
            }
        return dp[0][n - 1];
    }
};
```

51. 104.cpp

Path: LeetCode-main/solutions/104. Maximum Depth of Binary Tree/104.cpp

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == nullptr)
            return 0;
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

52. 1040.cpp

Path: LeetCode-main/solutions/1040. Moving Stones Until Consecutive II/1040.cpp

```
class Solution {
public:
    vector<int> numMovesStonesII(vector<int>& stones) {
        const int n = stones.size();
        int minMoves = n;

        ranges::sort(stones);

        for (int l = 0, r = 0; r < n; ++r) {
            while (stones[r] - stones[l] + 1 > n)
                ++l;
            int alreadyStored = r - l + 1;
            if (alreadyStored == n - 1 && stones[r] - stones[l] + 1 == n - 1)
                minMoves = min(minMoves, 2);
            else
                minMoves = min(minMoves, n - alreadyStored);
        }

        return {minMoves, max(stones[n - 1] - stones[1] - n + 2,
                             stones[n - 2] - stones[0] - n + 2)};
    }
};
```

53. 1041.cpp

Path: LeetCode-main/solutions/1041. Robot Bounded In Circle/1041.cpp

```
class Solution {
public:
    bool isRobotBounded(string instructions) {
        int x = 0;
        int y = 0;
        int d = 0;
        vector<vector<int>> directions{{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

        for (char instruction : instructions) {
            if (instruction == 'G') {
                x += directions[d][0];
                y += directions[d][1];
            } else if (instruction == 'L')
                d = (d + 3) % 4;
            else
                d = (d + 1) % 4;
        }

        return x == 0 && y == 0 || d > 0;
    }
};
```

54. 1042.cpp

Path: LeetCode-main/solutions/1042. Flower Planting With No Adjacent/1042.cpp

```
class Solution {
public:
    vector<int> gardenNoAdj(int n, vector<vector<int>>& paths) {
        vector<int> ans(n); // ans[i] := 1, 2, 3, or 4
        vector<vector<int>> graph(n);

        for (const vector<int>& path : paths) {
            const int u = path[0] - 1;
            const int v = path[1] - 1;
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        for (int u = 0; u < n; ++u) {
            int used = 0;
            for (const int v : graph[u])
                used |= 1 << ans[v];
            ans[u] = getFirstUnusedType(used);
        }

        return ans;
    }

private:
    int getFirstUnusedType(int used) {
        for (int type = 1; type <= 4; ++type)
            if ((used >> type & 1) == 0)
                return type;
        throw;
    }
};
```

55. 1043.cpp

Path: LeetCode-main/solutions/1043. Partition Array for Maximum Sum/1043.cpp

```
class Solution {
public:
    int maxSumAfterPartitioning(vector<int>& arr, int k) {
        const int n = arr.size();
        vector<int> dp(n + 1);

        for (int i = 1; i <= n; ++i) {
            int mx = INT_MIN;
            for (int j = 1; j <= min(i, k); ++j) {
                mx = max(mx, arr[i - j]);
                dp[i] = max(dp[i], dp[i - j] + mx * j);
            }
        }

        return dp[n];
    }
};
```

56. 1044.cpp

Path: LeetCode-main/solutions/1044. Longest Duplicate Substring/1044.cpp

```
class Solution {
public:
    string longestDupSubstring(string s) {
        const int n = s.length();
        vector<int> pows(n, 1);
        int bestStart = -1;
        int l = 1;
        int r = n;

        for (int i = 1; i < n; ++i)
            pows[i] = pows[i - 1] * kBase % kHash;

        while (l < r) {
            const int m = (l + r) / 2;
            const int start = getStart(s, m, pows);
            if (start == -1) {
                r = m;
            } else {
                bestStart = start;
                l = m + 1;
            }
        }

        if (bestStart == -1)
            return "";
        if (getStart(s, l, pows) == -1)
            return s.substr(bestStart, l - 1);
        return s.substr(bestStart, l);
    }

private:
    static constexpr long kBase = 26;
    static constexpr int kHash = 1'000'000'007;

    static constexpr int val(char c) {
        return c - 'a';
    }

    // k := the length of the substring to be hashed
    int getStart(const string& s, int k, const vector<int>& pows) {
        unordered_map<int, vector<int>> hashToStarts;
        long h = 0;

        // Compute the hash value of s[:k].
        for (int i = 0; i < k; ++i)
            h = (h * kBase % kHash + val(s[i])) % kHash;
        hashToStarts[h].push_back(0);

        // Compute the rolling hash by Rabin Karp.
        for (int i = k; i < s.length(); ++i) {
            const int startIndex = i - k + 1;
            h = ((h - static_cast<long>(pows[k - 1]) * val(s[i - k])) % kHash +
                kHash) %
                kHash;
            h = (h * kBase + val(s[i])) % kHash;
            if (const auto it = hashToStarts.find(h); it != hashToStarts.cend()) {
                const string currSub = s.substr(startIndex, k);
                for (const int start : it->second)
                    if (s.substr(start, k) == currSub)
                        return start;
            }
            hashToStarts[h].push_back(startIndex);
        }

        return -1;
    }
};
```


57. 1046.cpp

Path: LeetCode-main/solutions/1046. Last Stone Weight/1046.cpp

```
class Solution {
public:
    int lastStoneweight(vector<int>& stones) {
        priority_queue<int> pq{stones.begin(), stones.end()};

        while (pq.size() >= 2) {
            const int n1 = pq.top();
            pq.pop();
            const int n2 = pq.top();
            pq.pop();
            if (n1 != n2)
                pq.push(n1 - n2);
        }

        return pq.empty() ? 0 : pq.top();
    }
};
```

58. 1047.cpp

Path: LeetCode-main/solutions/1047. Remove All Adjacent Duplicates In String/1047.cpp

```
class Solution {
public:
    string removeDuplicates(const string& S) {
        string ans;

        for (const char c : S)
            if (!ans.empty() && ans.back() == c)
                ans.pop_back();
            else
                ans.push_back(c);

        return ans;
    }
};
```

59. 1048-2.cpp

Path: LeetCode-main/solutions/1048. Longest String Chain/1048-2.cpp

```
class Solution {
public:
    int longestStrChain(vector<string>& words) {
        int ans = 0;
        // dp[s] := the longest string chain, where s is the last word
        unordered_map<string, int> dp;

        ranges::sort(words, ranges::less{},
            [](const string& word) { return word.length(); });

        for (const string& word : words) {
            for (int i = 0; i < word.length(); ++i) {
                const string pred = word.substr(0, i) + word.substr(i + 1);
                dp[word] = max(dp[word], (dp.contains(pred) ? dp[pred] : 0) + 1);
            }
            ans = max(ans, dp[word]);
        }

        return ans;
    }
};
```

60. 1048.cpp

Path: LeetCode-main/solutions/1048. Longest String Chain/1048.cpp

```
class Solution {
public:
    int longestStrChain(vector<string>& words) {
        const unordered_set<string> wordsSet{words.begin(), words.end()};
        unordered_map<string, int> mem;
        return accumulate(words.begin(), words.end(), 0,
            [&](int acc, const string& word) {
                return max(acc, longestStrChain(word, wordsSet, mem));
            });
    }

private:
    // Returns the longest string chain, where s is the last word.
    int longestStrChain(const string& s, const unordered_set<string>& wordsSet,
        unordered_map<string, int>& mem) {
        if (const auto it = mem.find(s); it != mem.cend())
            return it->second;

        int res = 1;

        for (int i = 0; i < s.length(); ++i) {
            const string pred = s.substr(0, i) + s.substr(i + 1);
            if (wordsSet.contains(pred))
                res = max(res, longestStrChain(pred, wordsSet, mem) + 1);
        }

        return mem[s] = res;
    }
};
```

61. 1049.cpp

Path: LeetCode-main/solutions/1049. Last Stone Weight II/1049.cpp

```
class Solution {
public:
    int lastStoneweightII(vector<int>& stones) {
        const int sum = accumulate(stones.begin(), stones.end(), 0);
        vector<bool> dp(sum + 1);
        dp[0] = true;
        int s = 0;

        for (int stone : stones)
            for (int w = sum / 2; w > 0; --w) {
                if (w >= stone)
                    dp[w] = dp[w] || dp[w - stone];
                if (dp[w])
                    s = max(s, w);
            }

        return sum - 2 * s;
    }
};
```

62. 105.cpp

Path: LeetCode-main/solutions/105. Construct Binary Tree from Preorder and Inorder Traversal/105.cpp

```
class Solution {
public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        unordered_map<int, int> inToIndex;

        for (int i = 0; i < inorder.size(); ++i)
            inToIndex[inorder[i]] = i;

        return build(preorder, 0, preorder.size() - 1, inorder, 0,
                    inorder.size() - 1, inToIndex);
    }

private:
    TreeNode* build(const vector<int>& preorder, int preStart, int preEnd,
                    const vector<int>& inorder, int inStart, int inEnd,
                    const unordered_map<int, int>& inToIndex) {
        if (preStart > preEnd)
            return nullptr;

        const int rootVal = preorder[preStart];
        const int rootInIndex = inToIndex.at(rootVal);
        const int leftSize = rootInIndex - inStart;

        TreeNode* root = new TreeNode(rootVal);
        root->left = build(preorder, preStart + 1, preStart + leftSize, inorder,
                        inStart, rootInIndex - 1, inToIndex);
        root->right = build(preorder, preStart + leftSize + 1, preEnd, inorder,
                        rootInIndex + 1, inEnd, inToIndex);
        return root;
    }
};
```

63. 1051.cpp

Path: LeetCode-main/solutions/1051. Height Checker/1051.cpp

```
class Solution {
public:
    int heightChecker(vector<int>& heights) {
        int ans = 0;
        int currentHeight = 1;
        vector<int> count(101);

        for (int height : heights)
            ++count[height];

        for (int height : heights) {
            while (count[currentHeight] == 0)
                ++currentHeight;
            if (height != currentHeight)
                ++ans;
            --count[currentHeight];
        }

        return ans;
    }
};
```

64. 1052.cpp

Path: LeetCode-main/solutions/1052. Grumpy Bookstore Owner/1052.cpp

```
class Solution {
public:
    int maxSatisfied(vector<int>& customers, vector<int>& grumpy, int X) {
        int satisfied = 0;
        int madeSatisfied = 0;
        int windowSatisfied = 0;

        for (int i = 0; i < customers.size(); ++i) {
            if (grumpy[i] == 0)
                satisfied += customers[i];
            else
                windowSatisfied += customers[i];
            if (i >= X && grumpy[i - X] == 1)
                windowSatisfied -= customers[i - X];
            madeSatisfied = max(madeSatisfied, windowSatisfied);
        }

        return satisfied + madeSatisfied;
    }
};
```


65. 1053.cpp

Path: LeetCode-main/solutions/1053. Previous Permutation With One Swap/1053.cpp

```
class Solution {
public:
    vector<int> prevPermOpt1(vector<int>& arr) {
        const int n = arr.size();
        int l = n - 2;
        int r = n - 1;

        while (l >= 0 && arr[l] <= arr[l + 1])
            l--;
        if (l < 0)
            return arr;
        while (arr[r] >= arr[l] || arr[r] == arr[r - 1])
            r--;
        swap(arr[l], arr[r]);

        return arr;
    }
};
```

66. 1054.cpp

Path: LeetCode-main/solutions/1054. Distant Barcodes/1054.cpp

```
class Solution {
public:
    vector<int> rearrangeBarcodes(vector<int>& barcodes) {
        vector<int> ans(barcodes.size());
        vector<int> count(10001);
        int i = 0; // ans' index

        for (const int b : barcodes)
            ++count[b];

        const auto maxIt = ranges::max_element(count);
        const int maxNum = maxIt - count.begin();

        auto fillAns = [&](int num) {
            while (count[num]-- > 0) {
                ans[i] = num;
                i = i + 2 < barcodes.size() ? i + 2 : 1;
            }
        };

        fillAns(maxNum);
        for (int num = 1; num < 10001; ++num)
            fillAns(num);

        return ans;
    }
};
```

67. 1055-2.cpp

Path: LeetCode-main/solutions/1055. Shortest Way to Form String/1055-2.cpp

```
class Solution {
public:
    int shortestWay(string source, string target) {
        const int m = source.length();
        const int n = target.length();
        // dp[i][c] := the earliest index >= i s.t. source[index] = c
        // dp[i][c] := -1 if c isn't in the source
        vector<vector<int>> dp(m, vector<int>(26, -1));

        dp[m - 1][source[m - 1] - 'a'] = m - 1;
        for (int i = m - 2; i >= 0; --i) {
            dp[i] = dp[i + 1];
            dp[i][source[i] - 'a'] = i;
        }

        int ans = 0;
        int i = 0; // source's index

        for (const char c : target) {
            if (dp[0][c - 'a'] == -1)
                return -1;
            // If there are no c's left in source that occur more than i times but
            // there are c's from earlier in the subsequence, add 1 to subsequence
            // count and reset source's index to 0.
            if (dp[i][c - 'a'] == -1) {
                ++ans;
                i = 0;
            }
            // Continue taking letters from the subsequence.
            i = dp[i][c - 'a'] + 1;
            if (i == m) {
                ++ans;
                i = 0;
            }
        }

        return ans + (i == 0 ? 0 : 1);
    }
};
```

68. 1055.cpp

Path: LeetCode-main/solutions/1055. Shortest Way to Form String/1055.cpp

```
class Solution {
public:
    int shortestWay(string source, string target) {
        int ans = 0;

        for (int i = 0; i < target.length(); i) {
            const int prevIndex = i;
            for (int j = 0; j < source.length(); ++j)
                if (i < target.length() && source[j] == target[i])
                    ++i;
            // All chars in source didn't match target[i].
            if (i == prevIndex)
                return -1;
            ++ans;
        }
        return ans;
    }
};
```

69. 1056.cpp

Path: LeetCode-main/solutions/1056. Confusing Number/1056.cpp

```
class Solution {
public:
    bool confusingNumber(int n) {
        const string s = to_string(n);
        const vector<char> rotated{'0', '1', 'x', 'x', 'x',
                                   'x', '9', 'x', '8', '6'};
        string rotatedNum;

        for (int i = s.length() - 1; i >= 0; --i) {
            if (rotated[s[i] - '0'] == 'x')
                return false;
            rotatedNum += rotated[s[i] - '0'];
        }

        return rotatedNum != s;
    }
};
```

70. 1057.cpp

Path: LeetCode-main/solutions/1057. Campus Bikes/1057.cpp

```
class Solution {
public:
    vector<int> assignBikes(vector<vector<int>>& workers,
                           vector<vector<int>>& bikes) {
        const int n = workers.size();
        const int m = bikes.size();
        vector<int> ans(n, -1);
        vector<bool> usedBikes(m);
        // buckets[k] := (i, j), where k = dist(workers[i], bikes[j])
        vector<vector<pair<int, int>>> buckets(2001);

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                buckets[dist(workers[i], bikes[j])].emplace_back(i, j);

        for (int k = 0; k < 2001; ++k)
            for (const auto& [i, j] : buckets[k])
                if (ans[i] == -1 && !usedBikes[j]) {
                    ans[i] = j;
                    usedBikes[j] = true;
                }

        return ans;
    }

private:
    int dist(const vector<int>& p1, const vector<int>& p2) {
        return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1]);
    }
};
```

71. 1058.cpp

Path: LeetCode-main/solutions/1058. Minimize Rounding Error to Meet Target/1058.cpp

```
class Solution {
public:
    string minimizeError(vector<string>& prices, int target) {
        // A[i] := (costCeil - costFloor, costCeil, costFloor)
        // The lower the costCeil - costFloor is, the cheaper to ceil it.
        vector<tuple<double, double, double>> A;
        int sumFloored = 0;
        int sumCeiled = 0;

        for (const string& p : prices) {
            const double price = stod(p);
            const int floored = floor(price);
            const int ceiled = ceil(price);
            sumFloored += floored;
            sumCeiled += ceiled;
            const double costFloor = price - static_cast<double>(floored);
            const double costCeil = static_cast<double>(ceiled) - price;
            A.emplace_back(costCeil - costFloor, costCeil, costFloor);
        }

        if (sumFloored > target || sumCeiled < target)
            return "-1";

        ranges::sort(A);

        double sumError = 0.0;
        const int nCeiled = target - sumFloored;
        for (int i = 0; i < nCeiled; ++i)
            sumError += get<1>(A[i]);
        for (int i = nCeiled; i < A.size(); ++i)
            sumError += get<2>(A[i]);

        stringstream ss;
        ss << std::fixed << std::setprecision(3) << sumError;
        return ss.str();
    }
};
```

72. 1059.cpp

Path: LeetCode-main/solutions/1059. All Paths from Source Lead to Destination/1059.cpp

```
enum class State { kInit, kVisiting, kVisited };

class Solution {
public:
    bool leadsToDestination(int n, vector<vector<int>>& edges, int source,
                           int destination) {
        vector<vector<int>> graph(n);
        vector<State> states(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
        }

        return acyclic(graph, source, destination, states);
    }

private:
    bool acyclic(const vector<vector<int>>& graph, int u, int dest,
                vector<State>& states) {
        if (graph[u].empty())
            return u == dest;
        if (states[u] == State::kVisiting)
            return false;
        if (states[u] == State::kVisited)
            return true;

        states[u] = State::kVisiting;
        for (const int v : graph[u])
            if (!acyclic(graph, v, dest, states))
                return false;
        states[u] = State::kVisited;

        return true;
    }
};
```


73. 106.cpp

Path: LeetCode-main/solutions/106. Construct Binary Tree from Inorder and Postorder Traversal/106.cpp

```
class Solution {
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        unordered_map<int, int> inToIndex;

        for (int i = 0; i < inorder.size(); ++i)
            inToIndex[inorder[i]] = i;

        return build(inorder, 0, inorder.size() - 1, postorder, 0,
                    postorder.size() - 1, inToIndex);
    }

private:
    TreeNode* build(const vector<int>& inorder, int inStart, int inEnd,
                    const vector<int>& postorder, int postStart, int postEnd,
                    const unordered_map<int, int>& inToIndex) {
        if (inStart > inEnd)
            return nullptr;

        const int rootVal = postorder[postEnd];
        const int rootInIndex = inToIndex.at(rootVal);
        const int leftSize = rootInIndex - inStart;

        TreeNode* root = new TreeNode(rootVal);
        root->left = build(inorder, inStart, rootInIndex - 1, postorder, postStart,
                        postStart + leftSize - 1, inToIndex);
        root->right = build(inorder, rootInIndex + 1, inEnd, postorder,
                        postStart + leftSize, postEnd - 1, inToIndex);
        return root;
    }
};
```

74. 1060.cpp

Path: LeetCode-main/solutions/1060. Missing Element in Sorted Array/1060.cpp

```
class Solution {
public:
    int missingElement(vector<int>& nums, int k) {
        int l = 0;
        int r = nums.size();

        // the number of missing numbers in [nums[0], nums[i]]
        auto nMissing = [&](int i) { return nums[i] - nums[0] - i; };

        // Find the first index l s.t. nMissing(l) >= k
        while (l < r) {
            const int m = (l + r) / 2;
            if (nMissing(m) >= k)
                r = m;
            else
                l = m + 1;
        }

        return nums[l - 1] + (k - nMissing(l - 1));
    }
};
```

75. 1061.cpp

Path: LeetCode-main/solutions/1061. Lexicographically Smallest Equivalent String/1061.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n) {
        iota(id.begin(), id.end(), 0);
    }

    void union_(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i > j)
            id[i] = j;
        else
            id[j] = i;
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
};

class Solution {
public:
    string smallestEquivalentString(string s1, string s2, string baseStr) {
        string ans;
        UnionFind uf(26);

        for (int i = 0; i < s1.length(); ++i)
            uf.union_(s1[i] - 'a', s2[i] - 'a');

        for (const char c : baseStr)
            ans += 'a' + uf.find(c - 'a');

        return ans;
    }
};
```

76. 1062.cpp

Path: LeetCode-main/solutions/1062. Longest Repeating Substring/1062.cpp

```
class Solution {
public:
    int longestRepeatingSubstring(string s) {
        const int n = s.length();
        int ans = 0;
        // dp[i][j] := the number of repeating characters of s[0..i) and s[0..j)
        vector<vector<int>> dp(n + 1, vector<int>(n + 1));

        for (int i = 1; i <= n; ++i)
            for (int j = i + 1; j <= n; ++j)
                if (s[i - 1] == s[j - 1]) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                    ans = max(ans, dp[i][j]);
                }

        return ans;
    }
};
```

77. 1063.cpp

Path: LeetCode-main/solutions/1063. Number of Valid Subarrays/1063.cpp

```
class Solution {
public:
    int validSubarrays(vector<int>& nums) {
        // For each `num` in `nums`, each element x in the stack can be the leftmost
        // element s.t. [x, num] forms a valid subarray, so the size of the stack is
        // the number of valid subarrays ending in the current number.
        //
        // e.g. nums = [1, 3, 2]
        // num = 1, stack = [1] -> valid subarray is [1]
        // num = 3, stack = [1, 3] -> valid subarrays are [1, 3], [3]
        // num = 2, stack = [1, 2] -> valid subarrays are [1, 3, 2], [2]
        int ans = 0;
        stack<int> stack;

        for (const int num : nums) {
            while (!stack.empty() && stack.top() > num)
                stack.pop();
            stack.push(num);
            ans += stack.size();
        }

        return ans;
    }
};
```

78. 1064.cpp

Path: LeetCode-main/solutions/1064. Fixed Point/1064.cpp

```
class Solution {
public:
    int fixedPoint(vector<int>& arr) {
        int l = 0;
        int r = arr.size() - 1;

        // Since arr[i] is strictly increasing, arr[i] - i will also be increasing.
        // Therefore, binary search `arr` for the first arr[i] - i = 0.
        while (l < r) {
            const int m = (l + r) / 2;
            if (arr[m] - m >= 0)
                r = m;
            else
                l = m + 1;
        }

        return arr[l] == l ? l : -1;
    }
};
```

79. 1065.cpp

Path: LeetCode-main/solutions/1065. Index Pairs of a String/1065.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    bool isWord = false;
    TrieNode() : children(26) {}
};

class Solution {
public:
    vector<vector<int>> indexPairs(string text, vector<string>& words) {
        vector<vector<int>> ans;
        shared_ptr<TrieNode> root = make_shared<TrieNode>();

        for (const string& word : words) {
            shared_ptr<TrieNode> node = root;
            for (const char c : word) {
                const int i = c - 'a';
                if (node->children[i] == nullptr)
                    node->children[i] = make_shared<TrieNode>();
                node = node->children[i];
            }
            node->isWord = true;
        }

        for (int i = 0; i < text.length(); ++i) {
            shared_ptr<TrieNode> node = root;
            for (int j = i; j < text.length(); j++) {
                const int index = text[j] - 'a';
                if (node->children[index] == nullptr)
                    break;
                node = node->children[index];
                if (node->isWord)
                    ans.push_back({i, j});
            }
        }

        return ans;
    }
};
```

80. 1066.cpp

Path: LeetCode-main/solutions/1066. Campus Bikes II/1066.cpp

```
class Solution {
public:
    int assignBikes(vector<vector<int>>& workers, vector<vector<int>>& bikes) {
        vector<int> mem(1 << bikes.size(), INT_MAX);
        return assignBikes(workers, bikes, 0, 0, mem);
    }

private:
    // Returns the minimum Manhattan distances to assign bikes to
    // workers[workerIndex..n), where `used` is the bitmask of the used bikes.
    int assignBikes(const vector<vector<int>>& workers,
                    const vector<vector<int>>& bikes, int workerIndex, int used,
                    vector<int>& mem) {
        if (workerIndex == workers.size())
            return 0;
        if (mem[used] != INT_MAX)
            return mem[used];

        for (int bikeIndex = 0; bikeIndex < bikes.size(); ++bikeIndex)
            if ((used >> bikeIndex & 1) == 0)
                mem[used] =
                    min(mem[used], dist(workers[workerIndex], bikes[bikeIndex]) +
                        assignBikes(workers, bikes, workerIndex + 1,
                                    used | 1 << bikeIndex, mem));

        return mem[used];
    }

    int dist(const vector<int>& p1, const vector<int>& p2) {
        return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1]);
    }
};
```


81. 1067.cpp

Path: LeetCode-main/solutions/1067. Digit Count in Range/1067.cpp

```
class Solution {
public:
    int digitsCount(int d, int low, int high) {
        return countDigit(high, d) - countDigit(low - 1, d);
    }

private:
    int countDigit(int n, int d) {
        int count = 0;

        for (int pow10 = 1; pow10 <= n; pow10 *= 10) {
            const int divisor = pow10 * 10;
            const int quotient = n / divisor;
            const int remainder = n % divisor;
            if (quotient > 0)
                count += quotient * pow10;
            if (d == 0)
                count -= pow10;
            if (remainder >= d * pow10)
                count += min(remainder - d * pow10 + 1, pow10);
        }

        return count;
    }
};
```

82. 107.cpp

Path: LeetCode-main/solutions/107. Binary Tree Level Order Traversal II/107.cpp

```
class Solution {
public:
    vector<vector<int>> levelOrderBottom(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<vector<int>> ans;
        queue<TreeNode*> q{{root}};

        while (!q.empty()) {
            vector<int> currLevel;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode* node = q.front();
                q.pop();
                currLevel.push_back(node->val);
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
            ans.push_back(currLevel);
        }

        ranges::reverse(ans);
        return ans;
    }
};
```

83. 1071-2.cpp

Path: LeetCode-main/solutions/1071. Greatest Common Divisor of Strings/1071-2.cpp

```
class Solution {
public:
    string gcdOfStrings(string str1, string str2) {
        if (str1 + str2 != str2 + str1)
            return "";
        const int g = gcd(str1.length(), str2.length());
        return str1.substr(0, g);
    }
};
```

84. 1071.cpp

Path: LeetCode-main/solutions/1071. Greatest Common Divisor of Strings/1071.cpp

```
class Solution {
public:
    string gcdOfStrings(string str1, string str2) {
        for (int sz = min(str1.length(), str2.length()); sz > 0; --sz)
            if (isDivisible(str1, str2, sz))
                return str1.substr(0, sz);
        return "";
    }

private:
    // Returns true if str1 and str2 are divisible by str1[0..sz).
    bool isDivisible(const string& str1, const string& str2, int sz) {
        if (str1.length() % sz > 0 || str2.length() % sz > 0)
            return false;
        const string gcd = str1.substr(0, sz);
        return str1 == repeat(gcd, str1.length() / sz) &&
            str2 == repeat(gcd, str2.length() / sz);
    }

    string repeat(const string& s, int n) {
        string res;
        for (int i = 0; i < n; ++i)
            res += s;
        return res;
    }
};
```

85. 1072.cpp

Path: LeetCode-main/solutions/1072. Flip Columns For Maximum Number of Equal Rows/1072.cpp

```
class Solution {
public:
    int maxEqualRowsAfterFlips(vector<vector<int>>& matrix) {
        const int m = matrix.size();
        const int n = matrix[0].size();
        int ans = 0;
        vector<int> flip(n);
        unordered_set<int> seen;

        for (int i = 0; i < m; ++i) {
            if (seen.contains(i))
                continue;
            int count = 0;
            for (int j = 0; j < n; ++j)
                flip[j] = 1 ^ matrix[i][j];
            for (int k = 0; k < m; ++k)
                if (matrix[k] == matrix[i] || matrix[k] == flip) {
                    seen.insert(k);
                    ++count;
                }
            ans = max(ans, count);
        }

        return ans;
    }
};
```

86. 1073.cpp

Path: LeetCode-main/solutions/1073. Adding Two Negabinary Numbers/1073.cpp

```
class Solution {
public:
    vector<int> addNegabinary(vector<int>& arr1, vector<int>& arr2) {
        deque<int> ans;
        int carry = 0;
        int i = arr1.size() - 1;
        int j = arr2.size() - 1;

        while (carry != 0 || i >= 0 || j >= 0) {
            if (i >= 0)
                carry += arr1[i--];
            if (j >= 0)
                carry += arr2[j--];
            ans.push_front(carry & 1);
            carry = -(carry >> 1);
        }

        while (ans.size() > 1 && ans.front() == 0)
            ans.pop_front();

        return {ans.begin(), ans.end()};
    }
};
```

87. 1074.cpp

Path: LeetCode-main/solutions/1074. Number of Submatrices That Sum to Target/1074.cpp

```
class Solution {
public:
    int numSubmatrixSumTarget(vector<vector<int>>& matrix, int target) {
        const int m = matrix.size();
        const int n = matrix[0].size();
        int ans = 0;

        // Transfer each row in the matrix to the prefix sum.
        for (auto& row : matrix)
            for (int i = 1; i < n; ++i)
                row[i] += row[i - 1];

        for (int baseCol = 0; baseCol < n; ++baseCol)
            for (int j = baseCol; j < n; ++j) {
                unordered_map<int, int> prefixCount{{0, 1}};
                int sum = 0;
                for (int i = 0; i < m; ++i) {
                    if (baseCol > 0)
                        sum -= matrix[i][baseCol - 1];
                    sum += matrix[i][j];
                    if (const auto it = prefixCount.find(sum - target);
                        it != prefixCount.cend())
                        ans += it->second;
                    ++prefixCount[sum];
                }
            }
        return ans;
    }
};
```

88. 1078.cpp

Path: LeetCode-main/solutions/1078. Occurrences After Bigram/1078.cpp

```
class Solution {
public:
    vector<string> findOcurrences(string text, string first, string second) {
        vector<string> ans;
        stringstream ss(text);

        for (string prev2, prev, word; ss >> word;) {
            if (prev2 == first && prev == second)
                ans.push_back(word);
            prev2 = prev;
            prev = word;
        }

        return ans;
    }
};
```


89. 1079.cpp

Path: LeetCode-main/solutions/1079. Letter Tile Possibilities/1079.cpp

```
class Solution {
public:
    int numTilePossibilities(string tiles) {
        vector<int> count(26);

        for (const char t : tiles)
            ++count[t - 'A'];

        return dfs(count);
    }

private:
    int dfs(vector<int>& count) {
        int possibleSequences = 0;

        for (int& c : count) {
            if (c == 0)
                continue;
            // Put c in the current position. We only care about the number of
            // possible sequences of letters but don't care about the actual
            // combination.
            --c;
            possibleSequences += 1 + dfs(count);
            ++c;
        }

        return possibleSequences;
    }
};
```

90. 108.cpp

Path: LeetCode-main/solutions/108. Convert Sorted Array to Binary Search Tree/108.cpp

```
class Solution {
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        return build(nums, 0, nums.size() - 1);
    }

private:
    TreeNode* build(const vector<int>& nums, int l, int r) {
        if (l > r)
            return nullptr;
        const int m = (l + r) / 2;
        return new TreeNode(nums[m], build(nums, l, m - 1), build(nums, m + 1, r));
    }
};
```

91. 1080.cpp

Path: LeetCode-main/solutions/1080. Insufficient Nodes in Root to Leaf Paths/1080.cpp

```
class Solution {
public:
    TreeNode* sufficientSubset(TreeNode* root, int limit) {
        if (root == nullptr)
            return nullptr;
        if (root->left == nullptr && root->right == nullptr)
            return root->val < limit ? nullptr : root;
        root->left = sufficientSubset(root->left, limit - root->val);
        root->right = sufficientSubset(root->right, limit - root->val);
        return root->left == nullptr && root->right == nullptr ? nullptr : root;
    }
};
```

92. 1081.cpp

Path: LeetCode-main/solutions/1081. Smallest Subsequence of Distinct Characters/1081.cpp

```
class Solution {
public:
    string smallestSubsequence(string text) {
        string ans;
        vector<int> count(128);
        vector<bool> used(128);

        for (const char c : text)
            ++count[c];

        for (const char c : text) {
            --count[c];
            if (used[c])
                continue;
            while (!ans.empty() && ans.back() > c && count[ans.back()] > 0) {
                used[ans.back()] = false;
                ans.pop_back();
            }
            used[c] = true;
            ans.push_back(c);
        }

        return ans;
    }
};
```

93. 1085.cpp

Path: LeetCode-main/solutions/1085. Sum of Digits in the Minimum Number/1085.cpp

```
class Solution {
public:
    int sumOfDigits(vector<int>& nums) {
        int mn = ranges::min(nums);
        int sum = 0;

        while (mn > 0) {
            sum += mn % 10;
            mn /= 10;
        }

        return sum & 1 ^ 1;
    }
};
```

94. 1086.cpp

Path: LeetCode-main/solutions/1086. High Five/1086.cpp

```
class Solution {
public:
    vector<vector<int>> highFive(vector<vector<int>>& items) {
        vector<vector<int>> ans;
        map<int, priority_queue<int, vector<int>, greater<int>>> idToScores;

        for (const vector<int>& item : items) {
            const int id = item[0];
            const int score = item[1];
            auto& scores = idToScores[id];
            scores.push(score);
            if (scores.size() > 5)
                scores.pop();
        }

        for (auto& [id, scores] : idToScores) {
            int sum = 0;
            while (!scores.empty())
                sum += scores.top(), scores.pop();
            ans.push_back({id, sum / 5});
        }

        return ans;
    }
};
```

95. 1087.cpp

Path: LeetCode-main/solutions/1087. Brace Expansion/1087.cpp

```
class Solution {
public:
    vector<string> expand(string s) {
        vector<string> ans;
        string path;
        dfs(s, 0, path, ans);
        return ans;
    }

private:
    void dfs(const string& s, int i, string& path, vector<string>& ans) {
        if (i == s.length()) {
            ans.push_back(path);
            return;
        }
        if (s[i] == '{') {
            const int nextRightBraceIndex = s.find_first_of('}', i);
            for (const char c :
                split(s.substr(i + 1, nextRightBraceIndex - i - 1), ',')) {
                path.push_back(c);
                dfs(s, nextRightBraceIndex + 1, path, ans);
                path.pop_back();
            }
        } else { // s[i] != '{'
            path.push_back(s[i]);
            dfs(s, i + 1, path, ans);
            path.pop_back();
        }
    }

    vector<char> split(const string& s, char delimiter) {
        vector<char> splitted;
        for (const char c : s)
            if (c != delimiter)
                splitted.push_back(c);
        return splitted;
    }
};
```

96. 1088.cpp

Path: LeetCode-main/solutions/1088. Confusing Number II/1088.cpp

```
class Solution {
public:
    int confusingNumberII(int n) {
        return dfs(n, 0, 0, 1);
    }

private:
    vector<pair<int, int>> digitToRotated{{0, 0}, {1, 1}, {6, 9}, {8, 8}, {9, 6}};

    int dfs(int n, long num, long rotatedNum, long unit) {
        int ans = num != rotatedNum;
        // Add one more digit
        for (const auto& [digit, rotated] : digitToRotated) {
            if (digit == 0 && num == 0)
                continue;
            const long nextNum = num * 10 + digit;
            if (nextNum > n)
                break;
            ans += dfs(n, nextNum, rotated * unit + rotatedNum, unit * 10);
        }
        return ans;
    }
};
```


98. 109-2.cpp

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109-2.cpp

```
class Solution {
public:
    TreeNode* sortedListToBST(ListNode* head) {
        vector<int> arr;

        // Construct the array.
        for (ListNode* curr = head; curr; curr = curr->next)
            arr.push_back(curr->val);

        return helper(arr, 0, arr.size() - 1);
    }

private:
    TreeNode* helper(const vector<int>& arr, int l, int r) {
        if (l > r)
            return nullptr;
        const int m = (l + r) / 2;
        TreeNode* root = new TreeNode(arr[m]);
        root->left = helper(arr, l, m - 1);
        root->right = helper(arr, m + 1, r);
        return root;
    }
};
```

99. 109-3.cpp

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109-3.cpp

```
class Solution {
public:
    TreeNode* sortedListToBST(ListNode* head) {
        this->head = head;
        return helper(0, getLength(head) - 1);
    }

private:
    ListNode* head;

    TreeNode* helper(int l, int r) {
        if (l > r)
            return nullptr;

        const int m = (l + r) / 2;

        // Simulate inorder traversal: recursively form the left half.
        TreeNode* left = helper(l, m - 1);

        // Once the left half is traversed, process the current node.
        TreeNode* root = new TreeNode(head->val);
        root->left = left;

        // Maintain the invariance.
        head = head->next;

        // Simulate inorder traversal: recursively form the right half.
        root->right = helper(m + 1, r);
        return root;
    }

    int getLength(ListNode* head) {
        int length = 0;
        for (ListNode* curr = head; curr; curr = curr->next)
            ++length;
        return length;
    }
};
```

100. 109.cpp

Path: LeetCode-main/solutions/109. Convert Sorted List to Binary Search Tree/109.cpp

```
class Solution {
public:
    TreeNode* sortedListToBST(ListNode* head) {
        if (head == nullptr)
            return nullptr;
        if (!head->next)
            return new TreeNode(head->val);

        ListNode* mid = findMid(head);
        TreeNode* root = new TreeNode(mid->val);
        root->left = sortedListToBST(head);
        root->right = sortedListToBST(mid->next);
        return root;
    }

private:
    ListNode* findMid(ListNode* head) {
        ListNode* prev = nullptr;
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            prev = slow;
            slow = slow->next;
            fast = fast->next->next;
        }
        prev->next = nullptr;

        return slow;
    }
};
```

101. 1090.cpp

Path: LeetCode-main/solutions/1090. Largest Values From Labels/1090.cpp

```
class Solution {
public:
    int largestValsFromLabels(vector<int>& values, vector<int>& labels,
                             int numWanted, int useLimit) {
        const int n = values.size();
        int ans = 0;
        vector<pair<int, int>> items;
        unordered_map<int, int> labelsUsed;

        for (int i = 0; i < n; ++i)
            items.emplace_back(values[i], labels[i]);

        ranges::sort(items, ranges::greater{},
                    [](const pair<int, int>& item) { return item.first; });

        for (const auto& [value, label] : items)
            if (labelsUsed[label] < useLimit) {
                ans += value;
                ++labelsUsed[label];
                if (--numWanted == 0)
                    break;
            }

        return ans;
    }
};
```

102. 1091.cpp

Path: LeetCode-main/solutions/1091. Shortest Path in Binary Matrix/1091.cpp

```
class Solution {
public:
    int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
        const int n = grid.size();
        if (grid[0][0] == 0 && n == 1)
            return 1;
        if (grid[0][0] == 1 || grid.back().back() == 1)
            return -1;

        constexpr int kDirs[8][2] = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1},
                                       {0, 1}, {1, -1}, {1, 0}, {1, 1}};
        queue<pair<int, int>> q{{{0, 0}}};
        vector<vector<bool>> seen(n, vector<bool>(n));
        seen[0][0] = true;

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == n || y < 0 || y == n)
                        continue;
                    if (grid[x][y] != 0 || seen[x][y])
                        continue;
                    if (x == n - 1 && y == n - 1)
                        return step + 1;
                    q.emplace(x, y);
                    seen[x][y] = true;
                }
            }
        return -1;
    }
};
```

103. 1092.cpp

Path: LeetCode-main/solutions/1092. Shortest Common Supersequence/1092.cpp

```
class Solution {
public:
    string shortestCommonSupersequence(string str1, string str2) {
        string ans;
        int i = 0; // str1's index
        int j = 0; // str2's index

        for (const char c : lcs(str1, str2)) {
            // Append the letters that are not part of the LCS.
            while (str1[i] != c)
                ans += str1[i++];
            while (str2[j] != c)
                ans += str2[j++];
            // Append the letter of the LCS and match it with str1 and str2.
            ans += c;
            ++i;
            ++j;
        }

        // Append the remaining letters.
        return ans + str1.substr(i) + str2.substr(j);
    }

private:
    string lcs(const string& a, const string& b) {
        const int m = a.length();
        const int n = b.length();
        // dp[i][j] := the length of LCS(a[0..i), b[0..j))
        vector<vector<string>> dp(m + 1, vector<string>(n + 1));

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                if (a[i - 1] == b[j - 1])
                    dp[i][j] = dp[i - 1][j - 1] + a[i - 1];
                else
                    dp[i][j] = dp[i - 1][j].length() > dp[i][j - 1].length()
                        ? dp[i - 1][j]
                        : dp[i][j - 1];

        return dp[m][n];
    }
};
```

104. 1093.cpp

Path: LeetCode-main/solutions/1093. Statistics from a Large Sample/1093.cpp

```
class Solution {
public:
    vector<double> sampleStats(vector<int>& count) {
        const int n = accumulate(count.begin(), count.end(), 0);
        const double mode = ranges::max_element(count) - count.begin();
        return {
            getMinimum(count),
            getMaximum(count),
            getMean(count, n),
            (getLeftMedian(count, n) + getRightMedian(count, n)) / 2.0,
            mode,
        };
    }

private:
    double getMinimum(const vector<int>& count) {
        for (int i = 0; i < count.size(); ++i)
            if (count[i])
                return i;
        return -1;
    }

    double getMaximum(const vector<int>& count) {
        for (int i = count.size() - 1; i >= 0; --i)
            if (count[i])
                return i;
        return -1;
    }

    double getMean(const vector<int>& count, double n) {
        double mean = 0;
        for (long i = 0; i < count.size(); ++i)
            mean += (i * count[i]) / n;
        return mean;
    }

    double getLeftMedian(const vector<int>& count, double n) {
        int numCount = 0;
        for (int i = 0; i < count.size(); ++i) {
            numCount += count[i];
            if (numCount >= n / 2)
                return i;
        }
        return -1;
    }

    double getRightMedian(const vector<int>& count, double n) {
        int numCount = 0;
        for (int i = count.size() - 1; i >= 0; --i) {
            numCount += count[i];
            if (numCount >= n / 2)
                return i;
        }
        return -1;
    }
};
```


105. 1094-2.cpp

Path: LeetCode-main/solutions/1094. Car Pooling/1094-2.cpp

```
class Solution {
public:
    bool carPooling(vector<vector<int>>& trips, int capacity) {
        int currentPassengers = 0;
        vector<int> line(1001);

        for (const vector<int>& trip : trips) {
            const int nPassengers = trip[0];
            const int start = trip[1];
            const int end = trip[2];
            line[start] += nPassengers;
            line[end] -= nPassengers;
        }

        for (const int passengerChange : line) {
            currentPassengers += passengerChange;
            if (currentPassengers > capacity)
                return false;
        }

        return true;
    }
};
```

106. 1094.cpp

Path: LeetCode-main/solutions/1094. Car Pooling/1094.cpp

```
class Solution {
public:
    bool carPooling(vector<vector<int>>& trips, int capacity) {
        int currentPassengers = 0;
        map<int, int> line;

        for (const vector<int>& trip : trips) {
            const int nPassengers = trip[0];
            const int start = trip[1];
            const int end = trip[2];
            line[start] += nPassengers;
            line[end] -= nPassengers;
        }

        for (const auto [_, passengerChange] : line) {
            currentPassengers += passengerChange;
            if (currentPassengers > capacity)
                return false;
        }

        return true;
    }
};
```

107. 1095.cpp

Path: LeetCode-main/solutions/1095. Find in Mountain Array/1095.cpp

```
/**
 * // This is the MountainArray's API interface.
 * // You should not implement it, or speculate about its implementation
 * class MountainArray {
 * public:
 *     int get(int index);
 *     int length();
 * };
 */

class Solution {
public:
    int findInMountainArray(int target, MountainArray& mountainArr) {
        const int n = mountainArr.length();
        const int peakIndex = peakIndexInMountainArray(mountainArr, 0, n - 1);

        const int leftIndex = searchLeft(mountainArr, target, 0, peakIndex);
        if (mountainArr.get(leftIndex) == target)
            return leftIndex;

        const int rightIndex =
            searchRight(mountainArr, target, peakIndex + 1, n - 1);
        if (mountainArr.get(rightIndex) == target)
            return rightIndex;

        return -1;
    }

private:
    // 852. Peak Index in a Mountain Array
    int peakIndexInMountainArray(MountainArray& A, int l, int r) {
        while (l < r) {
            const int m = (l + r) / 2;
            if (A.get(m) < A.get(m + 1))
                l = m + 1;
            else
                r = m;
        }
        return l;
    }

    int searchLeft(MountainArray& A, int target, int l, int r) {
        while (l < r) {
            const int m = (l + r) / 2;
            if (A.get(m) < target)
                l = m + 1;
            else
                r = m;
        }
        return l;
    }

    int searchRight(MountainArray& A, int target, int l, int r) {
        while (l < r) {
            const int m = (l + r) / 2;
            if (A.get(m) > target)
                l = m + 1;
            else
                r = m;
        }
        return l;
    }
};
```

108. 1096.cpp

Path: LeetCode-main/solutions/1096. Brace Expansion II/1096.cpp

```
class Solution {
public:
    vector<string> braceExpansionII(string expression) {
        return dfs(expression, 0, expression.length() - 1);
    }

private:
    vector<string> dfs(const string& expression, int s, int e) {
        set<string> ans;
        vector<vector<string>> groups{{}};
        int layer = 0;
        int left = 0;

        for (int i = s; i <= e; ++i)
            if (expression[i] == '{' && ++layer == 1)
                left = i + 1;
            else if (expression[i] == '}' && --layer == 0)
                merge(groups, dfs(expression, left, i - 1));
            else if (expression[i] == ',' && layer == 0)
                groups.push_back({});
            else if (layer == 0)
                merge(groups, {string(1, expression[i])});

        for (const vector<string>& group : groups)
            for (const string& word : group)
                ans.insert(word);

        return {ans.begin(), ans.end()};
    }

    void merge(vector<vector<string>>& groups, const vector<string> group) {
        if (groups.back().empty()) {
            groups[groups.size() - 1] = group;
            return;
        }

        vector<string> mergedGroup;

        for (auto& word1 : groups.back())
            for (auto& word2 : group)
                mergedGroup.push_back(word1 + word2);

        groups[groups.size() - 1] = mergedGroup;
    }
};
```

109. 1099.cpp

Path: LeetCode-main/solutions/1099. Two Sum Less Than K/1099.cpp

```
class Solution {
public:
    int twoSumLessThanK(vector<int>& nums, int k) {
        if (nums.size() < 2)
            return -1;

        int ans = -1; // Note the constraint that nums[i] > 0.
        int l = 0;
        int r = nums.size() - 1;

        ranges::sort(nums);

        while (l < r)
            if (nums[l] + nums[r] < k) {
                ans = max(ans, nums[l] + nums[r]);
                ++l;
            } else {
                --r;
            }

        return ans;
    }
};
```

110. 11.cpp

Path: LeetCode-main/solutions/11. Container With Most Water/11.cpp

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int ans = 0;
        int l = 0;
        int r = height.size() - 1;

        while (l < r) {
            const int minHeight = min(height[l], height[r]);
            ans = max(ans, minHeight * (r - l));
            if (height[l] < height[r])
                ++l;
            else
                --r;
        }

        return ans;
    }
};
```

111. 110-2.cpp

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110-2.cpp

```
class Solution {
public:
    bool isBalanced(TreeNode* root) {
        bool ans = true;
        maxDepth(root, ans);
        return ans;
    }

    // Returns the height of root and sets ans to false if root unbalanced.
    int maxDepth(TreeNode* root, bool& ans) {
        if (root == nullptr || !ans)
            return 0;
        const int left = maxDepth(root->left, ans);
        const int right = maxDepth(root->right, ans);
        if (abs(left - right) > 1)
            ans = false;
        return max(left, right) + 1;
    }
};
```

112. 110-3.cpp

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110-3.cpp

```
class Solution {
public:
    bool isBalanced(TreeNode* root) {
        return maxDepth(root) != -1;
    }

private:
    // Returns the height of root if root is balanced; otherwise, returns -1.
    int maxDepth(TreeNode* root) {
        if (root == nullptr)
            return 0;

        const int left = maxDepth(root->left);
        if (left == -1)
            return -1;
        const int right = maxDepth(root->right);
        if (right == -1)
            return -1;
        if (abs(left - right) > 1)
            return -1;

        return 1 + max(left, right);
    }
};
```


113. 110.cpp

Path: LeetCode-main/solutions/110. Balanced Binary Tree/110.cpp

```
class Solution {
public:
    bool isBalanced(TreeNode* root) {
        if (root == nullptr)
            return true;
        return abs(maxDepth(root->left) - maxDepth(root->right)) <= 1 &&
            isBalanced(root->left) && isBalanced(root->right);
    }

private:
    int maxDepth(TreeNode* root) {
        if (root == nullptr)
            return 0;
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

114. 1100.cpp

Path: LeetCode-main/solutions/1100. Find K-Length Substrings With No Repeated Characters/1100.cpp

```
class Solution {
public:
    int numKLenSubstrNoRepeats(string s, int k) {
        int ans = 0;
        int unique = 0;
        vector<int> count(26);

        for (int i = 0; i < s.length(); ++i) {
            if (++count[s[i] - 'a'] == 1)
                ++unique;
            if (i >= k && --count[s[i - k] - 'a'] == 0)
                --unique;
            if (unique == k)
                ++ans;
        }

        return ans;
    }
};
```

115. 1101.cpp

Path: LeetCode-main/solutions/1101. The Earliest Moment When Everyone Become Friends/1101.cpp

```
class UnionFind {
public:
    UnionFind(int n) : count(n), id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        --count;
    }

    int getCount() const {
        return count;
    }

private:
    int count;
    vector<int> id;
    vector<int> rank;

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }
};

class Solution {
public:
    int earliestAcq(vector<vector<int>>& logs, int n) {
        UnionFind uf(n);

        // Sort `logs` by timestamp.
        ranges::sort(logs, ranges::less{},
            [](const vector<int>& log) { return log[0]; });

        for (const vector<int>& log : logs) {
            const int timestamp = log[0];
            const int x = log[1];
            const int y = log[2];
            uf.unionByRank(x, y);
            if (uf.getCount() == 1)
                return timestamp;
        }

        return -1;
    }
};
```

116. 1102.cpp

Path: LeetCode-main/solutions/1102. Path With Maximum Minimum Value/1102.cpp

```
struct T {
    int i;
    int j;
    int val;
};

class Solution {
public:
    int maximumMinimumPath(vector<vector<int>>& grid) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = grid.size();
        const int n = grid[0].size();
        int ans = grid[0][0];
        vector<vector<bool>> seen(m, vector<bool>(n));
        auto compare = [](const T& a, const T& b) { return a.val < b.val; };
        priority_queue<T, vector<T>, decltype(compare)> maxHeap(compare);

        maxHeap.emplace(0, 0, grid[0][0]);

        while (!maxHeap.empty()) {
            const auto [i, j, val] = maxHeap.top();
            maxHeap.pop();
            ans = min(ans, val);
            if (i == m - 1 && j == n - 1)
                return ans;
            seen[i][j] = true;
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                maxHeap.emplace(x, y, grid[x][y]);
            }
        }

        throw;
    }
};
```

117. 1103.cpp

Path: LeetCode-main/solutions/1103. Distribute Candies to People/1103.cpp

```
class Solution {
public:
    vector<int> distributeCandies(int candies, long n) {
        vector<int> ans(n);
        int rows = (-n + sqrt(n * n + 8 * n * n * candies)) / (2 * n * n);
        int accumN = rows * (rows - 1) * n / 2;

        for (int i = 0; i < n; ++i)
            ans[i] = accumN + rows * (i + 1);

        int givenCandies = (n * n * rows * rows + n * rows) / 2;
        candies -= givenCandies;

        for (int i = 0, lastGiven = rows * n + 1; candies > 0; ++i, ++lastGiven) {
            int actualGiven = min(lastGiven, candies);
            candies -= actualGiven;
            ans[i] += actualGiven;
        }

        return ans;
    }
};
```

118. 1104.cpp

Path: LeetCode-main/solutions/1104. Path In Zigzag Labelled Binary Tree/1104.cpp

```
class Solution {
public:
    vector<int> pathInZigZagTree(int label) {
        deque<int> ans;
        int level;

        for (int l = 0; l < 21; ++l)
            if (pow(2, l) > label) {
                level = l - 1;
                break;
            }

        if (level % 2 == 1)
            label = boundarySum(level) - label;

        for (int l = level; l >= 0; --l) {
            ans.push_front(l % 2 == 0 ? label : boundarySum(l) - label);
            label /= 2;
        }

        return {ans.begin(), ans.end()};
    }

private:
    int boundarySum(int level) {
        return pow(2, level) + pow(2, level + 1) - 1;
    }
};
```

119. 1105.cpp

Path: LeetCode-main/solutions/1105. Filling Bookcase Shelves/1105.cpp

```
class Solution {
public:
    int minHeightShelves(vector<vector<int>>& books, int shelfWidth) {
        // dp[i] := the minimum height to place the first i books
        vector<int> dp(books.size() + 1, INT_MAX);
        dp[0] = 0;

        for (int i = 0; i < books.size(); ++i) {
            int sumThickness = 0;
            int maxHeight = 0;
            // Place books[j..i] on a new shelf.
            for (int j = i; j >= 0; --j) {
                const int thickness = books[j][0];
                const int height = books[j][1];
                sumThickness += thickness;
                if (sumThickness > shelfWidth)
                    break;
                maxHeight = max(maxHeight, height);
                dp[i + 1] = min(dp[i + 1], dp[j] + maxHeight);
            }
        }
        return dp.back();
    }
};
```

120. 1106.cpp

Path: LeetCode-main/solutions/1106. Parsing A Boolean Expression/1106.cpp

```
class Solution {
public:
    bool parseBoolExpr(string expression) {
        int i = 0;
        return parse(expression, i);
    }

private:
    bool parse(const string& exp, int& i) {
        if (exp[i] == 't') {
            ++i;
            return true;
        }
        if (exp[i] == 'f') {
            ++i;
            return false;
        }
        if (exp[i] == '!') {
            i += 2;
            bool ans = !parse(exp, i);
            ++i;
            return ans;
        }

        bool isAnd = exp[i] == '&';
        bool ans = isAnd;
        i += 2;
        while (exp[i] != ')') {
            bool parsed = parse(exp, i);
            if (isAnd)
                ans &= parsed;
            else
                ans |= parsed;
            if (exp[i] == ',')
                ++i;
        }
        ++i;
        return ans;
    }
};
```


121. 1108.cpp

Path: LeetCode-main/solutions/1108. Defanging an IP Address/1108.cpp

```
class Solution {
public:
    string defangIPaddr(string address) {
        return regex_replace(address, regex("[.]"), "[.]");
    }
};
```

122. 1109.cpp

Path: LeetCode-main/solutions/1109. Corporate Flight Bookings/1109.cpp

```
class Solution {
public:
    vector<int> corpFlightBookings(vector<vector<int>>& bookings, int n) {
        vector<int> ans(n);

        for (vector<int>& booking : bookings) {
            ans[booking[0] - 1] += booking[2];
            if (booking[1] < n)
                ans[booking[1]] -= booking[2];
        }

        for (int i = 1; i < n; ++i)
            ans[i] += ans[i - 1];

        return ans;
    }
};
```

123. 111-2.cpp

Path: LeetCode-main/solutions/111. Minimum Depth of Binary Tree/111-2.cpp

```
class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == nullptr)
            return 0;

        queue<TreeNode*> q{{root}};

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode* node = q.front();
                q.pop();
                if (node->left == nullptr && node->right == nullptr)
                    return step;
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }

        throw;
    }
};
```

124. 111.cpp

Path: LeetCode-main/solutions/111. Minimum Depth of Binary Tree/111.cpp

```
class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == nullptr)
            return 0;
        if (root->left == nullptr)
            return minDepth(root->right) + 1;
        if (root->right == nullptr)
            return minDepth(root->left) + 1;
        return min(minDepth(root->left), minDepth(root->right)) + 1;
    }
};
```

125. 1110.cpp

Path: LeetCode-main/solutions/1110. Delete Nodes And Return Forest/1110.cpp

```
class Solution {
public:
    vector<TreeNode*> delNodes(TreeNode* root, vector<int>& to_delete) {
        vector<TreeNode*> ans;
        dfs(root, {to_delete.begin(), to_delete.end()}, true, ans);
        return ans;
    }

private:
    TreeNode* dfs(TreeNode*& root, const unordered_set<int>&& toDeleteSet,
                  bool isRoot, vector<TreeNode*>& ans) {
        if (root == nullptr)
            return nullptr;

        const bool deleted = toDeleteSet.contains(root->val);
        if (isRoot && !deleted)
            ans.push_back(root);

        // If root is deleted, both children have the possibility to be a new root
        root->left = dfs(root->left, std::move(toDeleteSet), deleted, ans);
        root->right = dfs(root->right, std::move(toDeleteSet), deleted, ans);
        return deleted ? nullptr : root;
    }
};
```

126. 1111.cpp

Path: LeetCode-main/solutions/1111. Maximum Nesting Depth of Two Valid Parentheses Strings/1111.cpp

```
class Solution {
public:
    vector<int> maxDepthAfterSplit(string seq) {
        vector<int> ans;
        int depth = 1;

        // Put all odd-depth parentheses in one group and even-depth parentheses in
        // the other group.
        for (const char c : seq)
            if (c == '(')
                ans.push_back(++depth % 2);
            else
                ans.push_back(depth-- % 2);

        return ans;
    }
};
```

127. 1114.cpp

Path: LeetCode-main/solutions/1114. Print in Order/1114.cpp

```
class Foo {
public:
    Foo() {
        firstDone.lock();
        secondDone.lock();
    }

    void first(function<void()> printFirst) {
        printFirst();
        firstDone.unlock();
    }

    void second(function<void()> printSecond) {
        firstDone.lock();
        printSecond();
        secondDone.unlock();
    }

    void third(function<void()> printThird) {
        secondDone.lock();
        printThird();
    }

private:
    mutex firstDone;
    mutex secondDone;
};
```

128. 1115.cpp

Path: LeetCode-main/solutions/1115. Print FooBar Alternately/1115.cpp

```
// LeetCode doesn't support C++20 yet, so we don't have std::counting_semaphore
// or binary_semaphore.
#include <semaphore.h>

class FooBar {
public:
    FooBar(int n) : n(n) {
        sem_init(&fooSemaphore, /*pshared=*/0, /*value=*/1);
        sem_init(&barSemaphore, /*pshared=*/0, /*value=*/0);
    }

    ~FooBar() {
        sem_destroy(&fooSemaphore);
        sem_destroy(&barSemaphore);
    }

    void foo(function<void()> printFoo) {
        for (int i = 0; i < n; ++i) {
            sem_wait(&fooSemaphore);
            printFoo();
            sem_post(&barSemaphore);
        }
    }

    void bar(function<void()> printBar) {
        for (int i = 0; i < n; ++i) {
            sem_wait(&barSemaphore);
            printBar();
            sem_post(&fooSemaphore);
        }
    }

private:
    const int n;
    sem_t fooSemaphore;
    sem_t barSemaphore;
};
```


129. 1116.cpp

Path: LeetCode-main/solutions/1116. Print Zero Even Odd/1116.cpp

```
// LeetCode doesn't support C++20 yet, so we don't have std::counting_semaphore
// or binary_semaphore.
#include <semaphore.h>

class ZeroEvenOdd {
public:
    ZeroEvenOdd(int n) : n(n) {
        sem_init(&zeroSemaphore, /*pshared=*/0, /*value=*/1);
        sem_init(&evenSemaphore, /*pshared=*/0, /*value=*/0);
        sem_init(&oddSemaphore, /*pshared=*/0, /*value=*/0);
    }

    ~ZeroEvenOdd() {
        sem_destroy(&zeroSemaphore);
        sem_destroy(&evenSemaphore);
        sem_destroy(&oddSemaphore);
    }

    // printNumber(x) outputs "x", where x is an integer.
    void zero(function<void(int)> printNumber) {
        for (int i = 0; i < n; ++i) {
            sem_wait(&zeroSemaphore);
            printNumber(0);
            sem_post(&(i % 2 == 0 ? oddSemaphore : evenSemaphore));
        }
    }

    void even(function<void(int)> printNumber) {
        for (int i = 2; i <= n; i += 2) {
            sem_wait(&evenSemaphore);
            printNumber(i);
            sem_post(&zeroSemaphore);
        }
    }

    void odd(function<void(int)> printNumber) {
        for (int i = 1; i <= n; i += 2) {
            sem_wait(&oddSemaphore);
            printNumber(i);
            sem_post(&zeroSemaphore);
        }
    }

private:
    const int n;
    sem_t zeroSemaphore;
    sem_t evenSemaphore;
    sem_t oddSemaphore;
};
```

130. 1117.cpp

Path: LeetCode-main/solutions/1117. Building H2O/1117.cpp

```
// LeetCode doesn't support C++20 yet, so we don't have std::counting_semaphore
// or binary_semaphore.
#include <semaphore.h>

class H2O {
public:
    H2O() {
        sem_init(&hSemaphore, /*pshared=*/0, /*value=*/1);
        sem_init(&oSemaphore, /*pshared=*/0, /*value=*/0);
    }

    ~H2O() {
        sem_destroy(&hSemaphore);
        sem_destroy(&oSemaphore);
    }

    void hydrogen(function<void()> releaseHydrogen) {
        sem_wait(&hSemaphore);
        ++h;
        // releaseHydrogen() outputs "H". Do not change or remove this line.
        releaseHydrogen();
        if (h % 2 == 0)
            sem_post(&oSemaphore);
        else
            sem_post(&hSemaphore);
    }

    void oxygen(function<void()> releaseOxygen) {
        sem_wait(&oSemaphore);
        // releaseOxygen() outputs "O". Do not change or remove this line.
        releaseOxygen();
        sem_post(&hSemaphore);
    }

private:
    sem_t hSemaphore;
    sem_t oSemaphore;
    int h = 0;
};
```

131. 1118.cpp

Path: LeetCode-main/solutions/1118. Number of Days in a Month/1118.cpp

```
class Solution {
public:
    int numberOfDays(int year, int month) {
        const vector<int> days = {0, 31, 28, 31, 30, 31, 30,
                                   31, 31, 30, 31, 30, 31};
        return month == 2 && isLeapYear(year) ? 29 : days[month];
    }

private:
    bool isLeapYear(int year) {
        return year % 4 == 0 && year % 100 != 0 || year % 400 == 0;
    };
};
```

132. 1119.cpp

Path: LeetCode-main/solutions/1119. Remove Vowels from a String/1119.cpp

```
class Solution {
public:
    string removeVowels(string s) {
        s.erase(remove_if(s.begin(), s.end(), [](char c) { return isVowel(c); }),
                s.end());
        return s;
    }

private:
    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiou";
        return kVowels.find(c) != string_view::npos;
    }
};
```

133. 112.cpp

Path: LeetCode-main/solutions/112. Path Sum/112.cpp

```
class Solution {
public:
    bool hasPathSum(TreeNode* root, int sum) {
        if (root == nullptr)
            return false;
        if (root->val == sum && root->left == nullptr && root->right == nullptr)
            return true;
        return hasPathSum(root->left, sum - root->val) ||
            hasPathSum(root->right, sum - root->val);
    }
};
```

134. 1120.cpp

Path: LeetCode-main/solutions/1120. Maximum Average Subtree/1120.cpp

```
struct T {
    int sum;
    int count;
    double maxAverage;
};

class Solution {
public:
    double maximumAverageSubtree(TreeNode* root) {
        return maximumAverage(root).maxAverage;
    }

private:
    T maximumAverage(TreeNode* root) {
        if (root == nullptr)
            return {0, 0, 0.0};

        const T left = maximumAverage(root->left);
        const T right = maximumAverage(root->right);

        const int sum = root->val + left.sum + right.sum;
        const int count = 1 + left.count + right.count;
        const double maxAverage =
            max({sum / (double)count, left.maxAverage, right.maxAverage});
        return {sum, count, maxAverage};
    }
};
```

135. 1121.cpp

Path: LeetCode-main/solutions/1121. Divide Array Into Increasing Sequences/1121.cpp

```
class Solution {
public:
    bool canDivideIntoSubsequences(vector<int>& nums, int k) {
        // Find the number with the maxFreq, we need at least maxFreq * k elements
        // e.g. nums = [1, 2, 2, 3, 4], we have maxFreq = 2 (two 2s), so we have to
        // Split nums into two subsequences say k = 3, the minimum length of nums is
        // 2 x 3 = 6, which is impossible if nums.size() = 5
        const int n = nums.size();
        int freq = 1;
        int maxFreq = 1;

        for (int i = 1; i < n; ++i) {
            freq = nums[i - 1] < nums[i] ? 1 : ++freq;
            maxFreq = max(maxFreq, freq);
        }

        return n >= maxFreq * k;
    }
};
```

136. 1122.cpp

Path: LeetCode-main/solutions/1122. Relative Sort Array/1122.cpp

```
class Solution {
public:
    vector<int> relativeSortArray(vector<int>& arr1, vector<int>& arr2) {
        vector<int> ans;
        vector<int> count(1001);

        for (int a : arr1)
            ++count[a];

        for (int a : arr2)
            while (count[a]-- > 0)
                ans.push_back(a);

        for (int num = 0; num < 1001; ++num)
            while (count[num]-- > 0)
                ans.push_back(num);

        return ans;
    }
};
```


137. 1123.cpp

Path: LeetCode-main/solutions/1123. Lowest Common Ancestor of Deepest Leaves/1123.cpp

```
struct T {
    TreeNode* lca;
    int depth;
};

class Solution {
public:
    TreeNode* lcaDeepestLeaves(TreeNode* root) {
        return dfs(root).lca;
    }

private:
    T dfs(TreeNode* root) {
        if (root == nullptr)
            return {nullptr, 0};
        const T left = dfs(root->left);
        const T right = dfs(root->right);
        if (left.depth > right.depth)
            return {left.lca, left.depth + 1};
        if (left.depth < right.depth)
            return {right.lca, right.depth + 1};
        return {root, left.depth + 1};
    }
};
```

138. 1124.cpp

Path: LeetCode-main/solutions/1124. Longest Well-Performing Interval/1124.cpp

```
class Solution {
public:
    int longestWPI(vector<int>& hours) {
        int ans = 0;
        int prefix = 0;
        unordered_map<int, int> map;

        for (int i = 0; i < hours.size(); ++i) {
            prefix += hours[i] > 8 ? 1 : -1;
            if (prefix > 0) {
                ans = i + 1;
            } else {
                if (!map.contains(prefix))
                    map[prefix] = i;
                if (const auto it = map.find(prefix - 1); it != map.cend())
                    ans = max(ans, i - it->second);
            }
        }

        return ans;
    }
};
```

139. 1125.cpp

Path: LeetCode-main/solutions/1125. Smallest Sufficient Team/1125.cpp

```
class Solution {
public:
    vector<int> smallestSufficientTeam(vector<string>& req_skills,
                                     vector<vector<string>>& people) {
        const int n = req_skills.size();
        const int nSkills = 1 << n;
        unordered_map<string, int> skillToId;
        // dp[i] := the minimum people's indices to cover skillset of mask i
        unordered_map<int, vector<int>> dp;
        dp.reserve(nSkills); // Avoid rehash.
        dp[0] = {};

        for (int i = 0; i < n; ++i)
            skillToId[req_skills[i]] = i;

        auto getSkill = [&](const vector<string>& person) {
            int mask = 0;
            for (const string& skill : person)
                if (const auto it = skillToId.find(skill); it != skillToId.cend())
                    mask |= 1 << it->second;
            return mask;
        };

        for (int i = 0; i < people.size(); ++i) {
            const int currSkill = getSkill(people[i]);
            for (const auto& [mask, indices] : dp) {
                const int newSkillSet = mask | currSkill;
                if (newSkillSet == mask) // Adding people[i] doesn't increase skill set
                    continue;
                if (!dp.contains(newSkillSet) ||
                    dp[newSkillSet].size() > indices.size() + 1) {
                    dp[newSkillSet] = indices;
                    dp[newSkillSet].push_back(i);
                }
            }
        }

        return dp[nSkills - 1];
    }
};
```

140. 1128.cpp

Path: LeetCode-main/solutions/1128. Number of Equivalent Domino Pairs/1128.cpp

```
class Solution {
public:
    int numEquivDominoPairs(vector<vector<int>>& dominoes) {
        int ans = 0;
        unordered_map<int, int> count;

        for (vector<int>& domino : dominoes) {
            int key = min(domino[0], domino[1]) * 10 + max(domino[0], domino[1]);
            ans += count[key];
            ++count[key];
        }

        return ans;
    }
};
```

141. 1129.cpp

Path: LeetCode-main/solutions/1129. Shortest Path with Alternating Colors/1129.cpp

```
enum class Color { kInit, kRed, kBlue };

class Solution {
public:
    vector<int> shortestAlternatingPaths(int n, vector<vector<int>>& redEdges,
                                       vector<vector<int>>& blueEdges) {
        vector<int> ans(n, -1);
        vector<vector<pair<int, Color>>> graph(n); // graph[u] := [(v, edgeColor)]
        queue<pair<int, Color>> q{{{0, Color::kInit}}}; // [(u, prevColor)]

        for (const vector<int>& edge : redEdges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].emplace_back(v, Color::kRed);
        }

        for (const vector<int>& edge : blueEdges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].emplace_back(v, Color::kBlue);
        }

        for (int step = 0; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [u, prevColor] = q.front();
                q.pop();
                ans[u] = ans[u] == -1 ? step : ans[u];
                for (auto& [v, edgeColor] : graph[u]) {
                    if (v == -1 || edgeColor == prevColor)
                        continue;
                    q.emplace(v, edgeColor);
                    v = -1; // Mark (u, v) as used.
                }
            }

        return ans;
    }
};
```

142. 113.cpp

Path: LeetCode-main/solutions/113. Path Sum II/113.cpp

```
class Solution {
public:
    vector<vector<int>> pathSum(TreeNode* root, int sum) {
        vector<vector<int>> ans;
        dfs(root, sum, {}, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, int sum, vector<int>&& path,
              vector<vector<int>>& ans) {
        if (root == nullptr)
            return;
        if (root->val == sum && root->left == nullptr && root->right == nullptr) {
            path.push_back(root->val);
            ans.push_back(path);
            path.pop_back();
            return;
        }

        path.push_back(root->val);
        dfs(root->left, sum - root->val, std::move(path), ans);
        dfs(root->right, sum - root->val, std::move(path), ans);
        path.pop_back();
    }
};
```

143. 1130-2.cpp

Path: LeetCode-main/solutions/1130. Minimum Cost Tree From Leaf Values/1130-2.cpp

```
class Solution {
public:
    int mctFromLeafValues(vector<int>& arr) {
        int ans = 0;
        vector<int> stack{INT_MAX};

        for (const int a : arr) {
            while (stack.back() <= a) {
                const int mid = stack.back();
                stack.pop_back();
                // Multiply mid with next greater element in the array,
                // On the left (stack[-1]) or on the right (current number a)
                ans += min(stack.back(), a) * mid;
            }
            stack.push_back(a);
        }

        for (int i = 2; i < stack.size(); ++i)
            ans += stack[i] * stack[i - 1];

        return ans;
    }
};
```

144. 1130.cpp

Path: LeetCode-main/solutions/1130. Minimum Cost Tree From Leaf Values/1130.cpp

```
class Solution {
public:
    int mctFromLeafValues(vector<int>& arr) {
        const int n = arr.size();
        // dp[i][j] := the minimum cost of arr[i..j]
        vector<vector<int>> dp(n, vector<int>(n));
        // maxVal[i][j] := the maximum value of arr[i..j]
        vector<vector<int>> maxVal(n, vector<int>(n));

        for (int i = 0; i < n; ++i)
            maxVal[i][i] = arr[i];

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                maxVal[i][j] = max(maxVal[i][j - 1], maxVal[i + 1][j]);
            }

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                dp[i][j] = INT_MAX;
                for (int k = i; k < j; ++k)
                    dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j] +
                                   maxVal[i][k] * maxVal[k + 1][j]);
            }

        return dp[0].back();
    }
};
```


145. 1131.cpp

Path: LeetCode-main/solutions/1131. Maximum of Absolute Value Expression/1131.cpp

```
class Solution {
public:
    int maxAbsValExpr(vector<int>& arr1, vector<int>& arr2) {
        const int n = arr1.size();
        vector<int> a(n);
        vector<int> b(n);
        vector<int> c(n);
        vector<int> d(n);

        for (int i = 0; i < n; ++i) {
            a[i] = arr1[i] + arr2[i] + i;
            b[i] = arr1[i] + arr2[i] - i;
            c[i] = arr1[i] - arr2[i] + i;
            d[i] = arr1[i] - arr2[i] - i;
        }

        return max(max(diff(a), diff(b)), max(diff(c), diff(d)));
    }

private:
    int diff(vector<int>& nums) {
        return ranges::max(nums) - ranges::min(nums);
    }
};
```

146. 1133.cpp

Path: LeetCode-main/solutions/1133. Largest Unique Number/1133.cpp

```
class Solution {
public:
    int largestUniqueNumber(vector<int>& nums) {
        constexpr int kMax = 1000;
        vector<short> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        for (int num = kMax; num >= 0; --num)
            if (count[num] == 1)
                return num;

        return -1;
    }
};
```

147. 1134.cpp

Path: LeetCode-main/solutions/1134. Armstrong Number/1134.cpp

```
class Solution {
public:
    bool isArmstrong(int n) {
        const string s = to_string(n);
        const int k = s.length();
        for (const char c : s)
            n -= pow(c - '0', k);
        return n == 0;
    }
};
```

148. 1135.cpp

Path: LeetCode-main/solutions/1135. Connecting Cities With Minimum Cost/1135.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    int minimumCost(int n, vector<vector<int>>& connections) {
        int ans = 0;
        UnionFind uf(n + 1);

        // Sort by cost.
        ranges::sort(connections, ranges::less{},
            [](const vector<int>& connection) { return connection[2]; });

        for (const vector<int>& connection : connections) {
            const int u = connection[0];
            const int v = connection[1];
            const int cost = connection[2];
            if (uf.find(u) == uf.find(v))
                continue;
            uf.unionByRank(u, v);
            ans += cost;
        }

        const int root = uf.find(1);
        for (int i = 1; i <= n; ++i)
            if (uf.find(i) != root)
                return -1;

        return ans;
    }
};
```

149. 1136-2.cpp

Path: LeetCode-main/solutions/1136. Parallel Courses/1136-2.cpp

```
class Solution {
public:
    int minimumSemesters(int n, vector<vector<int>>& relations) {
        vector<vector<int>> graph(n);
        vector<int> inDegrees(n);
        queue<int> q;

        // Build the graph.
        for (const vector<int>& relation : relations) {
            const int u = relation[0] - 1;
            const int v = relation[1] - 1;
            graph[u].push_back(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        int step = 0;
        for (; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const int u = q.front();
                q.pop();
                --n;
                for (const int v : graph[u])
                    if (--inDegrees[v] == 0)
                        q.push(v);
            }

        return n == 0 ? step : -1;
    }
};
```

150. 1136.cpp

Path: LeetCode-main/solutions/1136. Parallel Courses/1136.cpp

```
enum class State { kInit, kVisiting, kVisited };

class Solution {
public:
    int minimumSemesters(int n, vector<vector<int>>& relations) {
        vector<vector<int>> graph(n);
        vector<State> states(n);
        vector<int> depth(n, 1);

        for (const vector<int>& relation : relations) {
            const int u = relation[0] - 1;
            const int v = relation[1] - 1;
            graph[u].push_back(v);
        }

        for (int i = 0; i < n; ++i)
            if (hasCycle(graph, i, states, depth))
                return -1;

        return ranges::max(depth);
    }

private:
    bool hasCycle(const vector<vector<int>>& graph, int u, vector<State>& states,
                  vector<int>& depth) {
        if (states[u] == State::kVisiting)
            return true;
        if (states[u] == State::kVisited)
            return false;
        states[u] = State::kVisiting;
        for (const int v : graph[u]) {
            if (hasCycle(graph, v, states, depth))
                return true;
            depth[u] = max(depth[u], 1 + depth[v]);
        }
        states[u] = State::kVisited;
        return false;
    }
};
```

151. 1137-2.cpp

Path: LeetCode-main/solutions/1137. N-th Tribonacci Number/1137-2.cpp

```
class Solution {
public:
    int tribonacci(int n) {
        vector<int> dp{0, 1, 1};

        for (int i = 3; i <= n; ++i)
            dp[i % 3] = dp[0] + dp[1] + dp[2];

        return dp[n % 3];
    }
};
```

152. 1137.cpp

Path: LeetCode-main/solutions/1137. N-th Tribonacci Number/1137.cpp

```
class Solution {
public:
    int tribonacci(int n) {
        if (n < 2)
            return n;

        vector<int> dp{0, 1, 1};

        for (int i = 3; i <= n; ++i) {
            const int next = dp[0] + dp[1] + dp[2];
            dp[0] = dp[1];
            dp[1] = dp[2];
            dp[2] = next;
        }

        return dp[2];
    }
};
```


153. 1138.cpp

Path: LeetCode-main/solutions/1138. Alphabet Board Path/1138.cpp

```
class Solution {
public:
    string alphabetBoardPath(string target) {
        string ans;
        int x = 0;
        int y = 0;

        for (char c : target) {
            int newX = (c - 'a') % 5;
            int newY = (c - 'a') / 5;
            ans += string(max(0, y - newY), 'U') + string(max(0, newX - x), 'R') +
                string(max(0, x - newX), 'L') + string(max(0, newY - y), 'D') +
                '!';
            x = newX;
            y = newY;
        }
        return ans;
    }
};
```

154. 1139.cpp

Path: LeetCode-main/solutions/1139. Largest 1-Bordered Square/1139.cpp

```
class Solution {
public:
    int largest1BorderedSquare(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();

        // leftOnes[i][j] := consecutive 1s in the left of grid[i][j]
        vector<vector<int>> leftOnes(m, vector<int>(n));
        // topOnes[i][j] := consecutive 1s in the top of grid[i][j]
        vector<vector<int>> topOnes(m, vector<int>(n));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1) {
                    leftOnes[i][j] = j == 0 ? 1 : 1 + leftOnes[i][j - 1];
                    topOnes[i][j] = i == 0 ? 1 : 1 + topOnes[i - 1][j];
                }

        for (int sz = min(m, n); sz > 0; --sz)
            for (int i = 0; i + sz - 1 < m; ++i)
                for (int j = 0; j + sz - 1 < n; ++j) {
                    const int x = i + sz - 1;
                    const int y = j + sz - 1;
                    // If grid[i..x][j..y] has all 1s on its border.
                    if (min(leftOnes[i][y], leftOnes[x][y]) >= sz &&
                        min(topOnes[x][j], topOnes[x][y]) >= sz)
                        return sz * sz;
                }

        return 0;
    }
};
```

155. 114-2.cpp

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114-2.cpp

```
class Solution {
public:
    void flatten(TreeNode* root) {
        if (root == nullptr)
            return;

        stack<TreeNode*> stack{{root}};

        while (!stack.empty()) {
            root = stack.top(), stack.pop();
            if (root->right)
                stack.push(root->right);
            if (root->left)
                stack.push(root->left);
            if (!stack.empty())
                root->right = stack.top();
            root->left = nullptr;
        }
    }
};
```

156. 114-3.cpp

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114-3.cpp

```
class Solution {
public:
    void flatten(TreeNode* root) {
        if (root == nullptr)
            return;

        while (root != nullptr) {
            if (root->left != nullptr) {
                // Find the rightmost root.
                TreeNode* rightmost = root->left;
                while (rightmost->right != nullptr)
                    rightmost = rightmost->right;
                // Rewire the connections.
                rightmost->right = root->right;
                root->right = root->left;
                root->left = nullptr;
            }
            // Move on to the right side of the tree.
            root = root->right;
        }
    }
};
```

157. 114.cpp

Path: LeetCode-main/solutions/114. Flatten Binary Tree to Linked List/114.cpp

```
class Solution {
public:
    void flatten(TreeNode* root) {
        if (root == nullptr)
            return;

        flatten(root->left);
        flatten(root->right);

        TreeNode* const left = root->left;    // flattened left
        TreeNode* const right = root->right;  // flattened right

        root->left = nullptr;
        root->right = left;

        // Connect the original right subtree to the end of the new right subtree.
        TreeNode* rightmost = root;
        while (rightmost->right != nullptr)
            rightmost = rightmost->right;
        rightmost->right = right;
    }
};
```

158. 1140.cpp

Path: LeetCode-main/solutions/1140. Stone Game II/1140.cpp

```
class Solution {
public:
    int stoneGameII(vector<int>& piles) {
        const int n = piles.size();
        vector<vector<int>>> mem(n, vector<int>(n));
        vector<int> suffix = piles; // suffix[i] := sum(piles[i..n))
        for (int i = n - 2; i >= 0; --i)
            suffix[i] += suffix[i + 1];
        return stoneGameII(suffix, 0, 1, mem);
    }

private:
    // Returns the maximum number of stones Alice can get from piles[i..n) with M.
    int stoneGameII(const vector<int>& suffix, int i, int M,
                    vector<vector<int>>>& mem) {
        if (i + 2 * M >= mem.size())
            return suffix[i];
        if (mem[i][M] > 0)
            return mem[i][M];

        int opponent = suffix[i];
        for (int X = 1; X <= 2 * M; ++X)
            opponent = min(opponent, stoneGameII(suffix, i + X, max(M, X), mem));

        return mem[i][M] = suffix[i] - opponent;
    }
};
```

159. 1143.cpp

Path: LeetCode-main/solutions/1143. Longest Common Subsequence/1143.cpp

```
class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        const int m = text1.length();
        const int n = text2.length();
        // dp[i][j] := the length of LCS(text1[0..i), text2[0..j))
        vector<vector<int>> dp(m + 1, vector<int>(n + 1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                dp[i + 1][j + 1] = text1[i] == text2[j]
                    ? 1 + dp[i][j]
                    : max(dp[i][j + 1], dp[i + 1][j]);

        return dp[m][n];
    }
};
```

160. 1144.cpp

Path: LeetCode-main/solutions/1144. Decrease Elements To Make Array Zigzag/1144.cpp

```
class Solution {
public:
    int movesToMakeZigzag(vector<int>& nums) {
        vector<int> decreasing(2);

        for (int i = 0; i < nums.size(); ++i) {
            int l = i > 0 ? nums[i - 1] : 1001;
            int r = i + 1 < nums.size() ? nums[i + 1] : 1001;
            decreasing[i % 2] += max(0, nums[i] - min(l, r) + 1);
        }

        return min(decreasing[0], decreasing[1]);
    }
};
```


161. 1145.cpp

Path: LeetCode-main/solutions/1145. Binary Tree Coloring Game/1145.cpp

```
class Solution {
public:
    bool btreeGameWinningMove(TreeNode* root, int n, int x) {
        count(root, x);
        return max({leftCount, rightCount, n - leftCount - rightCount - 1}) > n / 2;
    }

private:
    int leftCount;    // the number of nodes of n's left
    int rightCount;   // the number of nodes of n's right

    int count(TreeNode* root, int x) {
        if (root == nullptr)
            return 0;
        const int l = count(root->left, x);
        const int r = count(root->right, x);
        if (root->val == x) {
            leftCount = l;
            rightCount = r;
        }
        return 1 + l + r;
    }
};
```

162. 1146.cpp

Path: LeetCode-main/solutions/1146. Snapshot Array/1146.cpp

```
class SnapshotArray {
public:
    SnapshotArray(int length) : snaps(length) {
        for (auto& snap : snaps)
            snap.emplace_back(0, 0);
    }

    void set(int index, int val) {
        auto& [lastSnapId, lastVal] = snaps[index].back();
        if (lastSnapId == snap_id)
            lastVal = val;
        else
            snaps[index].emplace_back(snap_id, val);
    }

    int snap() {
        return snap_id++;
    }

    int get(int index, int snap_id) {
        const vector<pair<int, int>>& snap = snaps[index];
        auto it = ranges::lower_bound(snap, pair<int, int>{snap_id + 1, 0});
        return prev(it)->second;
    }

private:
    // snaps[i] := [(snap_id, val)]
    vector<vector<pair<int, int>>> snaps;
    int snap_id = 0;
};
```

163. 1147.cpp

Path: LeetCode-main/solutions/1147. Longest Chunked Palindrome Decomposition/1147.cpp

```
class Solution {
public:
    int longestDecomposition(string text) {
        int count = 0;
        int l = 0;

        for (int r = 1; 2 * r <= text.length(); ++r)
            if (equal(text.begin() + l, text.begin() + r, text.end() - r)) {
                count += 2;
                l = r;
            }

        return count + (2 * l < text.length());
    }
};
```

164. 115-2.cpp

Path: LeetCode-main/solutions/115. Distinct Subsequences/115-2.cpp

```
class Solution {
public:
    int numDistinct(string s, string t) {
        const int m = s.length();
        const int n = t.length();
        vector<unsigned long> dp(n + 1);
        dp[0] = 1;

        for (int i = 1; i <= m; ++i)
            for (int j = n; j >= 1; --j)
                if (s[i - 1] == t[j - 1])
                    dp[j] += dp[j - 1];

        return dp[n];
    }
};
```

165. 115.cpp

Path: LeetCode-main/solutions/115. Distinct Subsequences/115.cpp

```
class Solution {
public:
    int numDistinct(string s, string t) {
        const int m = s.length();
        const int n = t.length();
        vector<vector<unsigned long>> dp(m + 1, vector<unsigned long>(n + 1));

        for (int i = 0; i <= m; ++i)
            dp[i][0] = 1;

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                if (s[i - 1] == t[j - 1])
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
                else
                    dp[i][j] = dp[i - 1][j];

        return dp[m][n];
    }
};
```

166. 1150.cpp

Path: LeetCode-main/solutions/1150. Check If a Number Is Majority Element in a Sorted Array/1150.cpp

```
class Solution {
public:
    bool isMajorityElement(vector<int>& nums, int target) {
        const int n = nums.size();
        const int i = ranges::lower_bound(nums, target) - nums.begin();
        return i + n / 2 < n && nums[i + n / 2] == target;
    }
};
```

167. 1151.cpp

Path: LeetCode-main/solutions/1151. Minimum Swaps to Group All 1's Together/1151.cpp

```
class Solution {
public:
    int minSwaps(vector<int>& data) {
        const int k = ranges::count(data, 1);
        int ones = 0;    // the number of ones in the window
        int maxOnes = 0; // the maximum number of ones in the window

        for (int i = 0; i < data.size(); ++i) {
            if (i >= k && data[i - k])
                --ones;
            if (data[i])
                ++ones;
            maxOnes = max(maxOnes, ones);
        }
        return k - maxOnes;
    }
};
```

168. 1153.cpp

Path: LeetCode-main/solutions/1153. String Transforms Into Another String/1153.cpp

```
class Solution {
public:
    bool canConvert(string str1, string str2) {
        if (str1 == str2)
            return true;

        vector<int> mappings(128);

        for (int i = 0; i < str1.length(); ++i) {
            const int a = str1[i];
            const int b = str2[i];
            if (mappings[a] != 0 && mappings[a] != b)
                return false;
            mappings[a] = b;
        }

        // No letter in the str1 maps to > 1 letter in the str2 and there is at
        // least one temporary letter can break any loops.
        return unordered_set<char>(str2.begin(), str2.end()).size() < 26;
    }
};
```


169. 1154.cpp

Path: LeetCode-main/solutions/1154. Day of the Year/1154.cpp

```
class Solution {
public:
    int dayOfYear(string date) {
        int year = stoi(date.substr(0, 4));
        int month = stoi(date.substr(5, 2));
        int day = stoi(date.substr(8));
        vector<int> days = {
            31, isLeapYear(year) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        return accumulate(days.begin(), days.begin() + month - 1, 0) + day;
    }

private:
    bool isLeapYear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    }
};
```

170. 1155.cpp

Path: LeetCode-main/solutions/1155. Number of Dice Rolls With Target Sum/1155.cpp

```
class Solution {
public:
    int numRollsToTarget(int n, int k, int target) {
        constexpr int kMod = 1'000'000'007;
        vector<int> dp(target + 1);
        dp[0] = 1;

        while (n-- > 0) { // n dices
            vector<int> newDp(target + 1);
            for (int i = 1; i <= k; ++i) // numbers 1, 2, ..., k
                for (int t = i; t <= target; ++t) { // all the possible targets
                    newDp[t] += dp[t - i];
                    newDp[t] %= kMod;
                }
            dp = std::move(newDp);
        }

        return dp[target];
    }
};
```

171. 1156.cpp

Path: LeetCode-main/solutions/1156. Swap For Longest Repeated Character Substring/1156.cpp

```
class Solution {
public:
    int maxRepOpt1(string text) {
        int ans = 0;
        vector<int> count(26);
        vector<pair<char, int>> groups{{text[0], 1}};

        for (const char c : text)
            ++count[c - 'a'];

        for (int i = 1; i < text.length(); ++i)
            if (text[i] == text[i - 1])
                ++groups[groups.size() - 1].second;
            else
                groups.emplace_back(text[i], 1);

        for (const auto& [c, length] : groups)
            ans = max(ans, min(length + 1, count[c - 'a']));

        for (int i = 1; i + 1 < groups.size(); ++i)
            if (groups[i - 1].first == groups[i + 1].first && groups[i].second == 1)
                ans = max(ans, min(groups[i - 1].second + groups[i + 1].second + 1,
                                    count[groups[i - 1].first - 'a']));

        return ans;
    }
};
```

172. 1157.cpp

Path: LeetCode-main/solutions/1157. Online Majority Element In Subarray/1157.cpp

```
class MajorityChecker {
public:
    MajorityChecker(vector<int>& arr) : arr(arr) {
        for (int i = 0; i < arr.size(); ++i)
            numToIndices[arr[i]].push_back(i);
    }

    int query(int left, int right, int threshold) {
        for (int i = 0; i < kTimes; ++i) {
            const int randIndex = rand() % (right - left + 1) + left;
            const int num = arr[randIndex];
            const vector<int>& indices = numToIndices[num];
            const auto lit = ranges::lower_bound(indices, left);
            const auto rit = ranges::upper_bound(indices, right);
            if (rit - lit >= threshold)
                return num;
        }
        return -1;
    }

private:
    const vector<int> arr;
    static constexpr int kTimes = 20; // 2^kTimes >> |arr| = n
    unordered_map<int, vector<int>> numToIndices;
};
```

173. 116-2.cpp

Path: LeetCode-main/solutions/116. Populating Next Right Pointers in Each Node/116-2.cpp

```
class Solution {
public:
    Node* connect(Node* root) {
        Node* node = root; // the node that is above the current needling

        while (node && node->left) {
            Node dummy(0); // the dummy node before needling
            // Needle the children of the node.
            for (Node* needle = &dummy; node; node = node->next) {
                needle->next = node->left;
                needle = needle->next;
                needle->next = node->right;
                needle = needle->next;
            }
            node = dummy.next; // Move the node to the next level.
        }

        return root;
    }
};
```

174. 116.cpp

Path: LeetCode-main/solutions/116. Populating Next Right Pointers in Each Node/116.cpp

```
class Solution {
public:
    Node* connect(Node* root) {
        if (root == nullptr)
            return nullptr;
        connectTwoNodes(root->left, root->right);
        return root;
    }

private:
    void connectTwoNodes(Node* p, Node* q) {
        if (p == nullptr)
            return;
        p->next = q;
        connectTwoNodes(p->left, p->right);
        connectTwoNodes(q->left, q->right);
        connectTwoNodes(p->right, q->left);
    }
};
```

175. 1160.cpp

Path: LeetCode-main/solutions/1160. Find Words That Can Be Formed by Characters/1160.cpp

```
class Solution {
public:
    int countCharacters(vector<string>& words, string chars) {
        int ans = 0;
        vector<int> count(26);

        for (const char c : chars)
            ++count[c - 'a'];

        for (const string& word : words) {
            vector<int> tempCount(count);
            for (const char c : word)
                if (--tempCount[c - 'a'] < 0) {
                    ans -= word.length();
                    break;
                }
            ans += word.length();
        }

        return ans;
    }
};
```

176. 1161-2.cpp

Path: LeetCode-main/solutions/1161. Maximum Level Sum of a Binary Tree/1161-2.cpp

```
class Solution {
public:
    int maxLevelSum(TreeNode* root) {
        // levelSums[i] := the sum of level (i + 1) (1-indexed)
        vector<int> levelSums;
        dfs(root, 0, levelSums);
        return 1 + ranges::max_element(levelSums) - levelSums.begin();
    }

private:
    void dfs(TreeNode* root, int level, vector<int>& levelSums) {
        if (root == nullptr)
            return;
        if (levelSums.size() == level)
            levelSums.push_back(0);
        levelSums[level] += root->val;
        dfs(root->left, level + 1, levelSums);
        dfs(root->right, level + 1, levelSums);
    }
};
```


177. 1161.cpp

Path: LeetCode-main/solutions/1161. Maximum Level Sum of a Binary Tree/1161.cpp

```
class Solution {
public:
    int maxLevelSum(TreeNode* root) {
        int ans = 0;
        int maxLevelSum = INT_MIN;
        queue<TreeNode*> q{{root}};

        for (int level = 1; !q.empty(); ++level) {
            int levelSum = 0;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode* node = q.front();
                q.pop();
                levelSum += node->val;
                if (node->left != nullptr)
                    q.push(node->left);
                if (node->right != nullptr)
                    q.push(node->right);
            }
            if (levelSum > maxLevelSum) {
                maxLevelSum = levelSum;
                ans = level;
            }
        }

        return ans;
    }
};
```

178. 1162.cpp

Path: LeetCode-main/solutions/1162. As Far from Land as Possible/1162.cpp

```
class Solution {
public:
    int maxDistance(vector<vector<int>>& grid) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = grid.size();
        const int n = grid[0].size();
        queue<pair<int, int>> q;
        int water = 0;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 0)
                    ++water;
                else
                    q.emplace(i, j);

        if (water == 0 || water == m * n)
            return -1;

        int ans = 0;

        for (int d = 0; !q.empty(); ++d)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                ans = d;
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (grid[x][y] > 0)
                        continue;
                    q.emplace(x, y);
                    grid[x][y] = 2; // Mark as visited.
                }
            }

        return ans;
    }
};
```

179. 1163.cpp

Path: LeetCode-main/solutions/1163. Last Substring in Lexicographical Order/1163.cpp

```
class Solution {
public:
    string lastSubstring(string s) {
        int i = 0;
        int j = 1;
        int k = 0; // the number of the same letters of s[i..n) and s[j..n)

        while (j + k < s.length())
            if (s[i + k] == s[j + k]) {
                ++k;
            } else if (s[i + k] > s[j + k]) {
                // Skip s[j..j + k) and advance to s[j + k + 1] to find a possible
                // lexicographically larger substring since s[i..i + k) == s[j..j + k)
                // and s[i + k] > s[j + k].
                j = j + k + 1;
                k = 0;
            } else {
                // Skip s[i..i + k) and advance to s[i + k + 1] or s[j] to find a
                // possible lexicographically larger substring since
                // s[i..i + k) == s[j..j + k) and s[i + k] < s[j + k].
                // Note that it's unnecessary to explore s[i + k + 1..j) if
                // i + k + 1 < j since they are already explored by j.
                i = max(i + k + 1, j);
                j = i + 1;
                k = 0;
            }

        return s.substr(i);
    }
};
```

180. 1165.cpp

Path: LeetCode-main/solutions/1165. Single-Row Keyboard/1165.cpp

```
class Solution {
public:
    int calculateTime(string keyboard, string word) {
        int ans = 0;
        int prevIndex = 0;
        vector<int> letterToIndex(26);

        for (int i = 0; i < keyboard.length(); ++i)
            letterToIndex[keyboard[i] - 'a'] = i;

        for (const char c : word) {
            const int currIndex = letterToIndex[c - 'a'];
            ans += abs(currIndex - prevIndex);
            prevIndex = currIndex;
        }

        return ans;
    }
};
```

181. 1166.cpp

Path: LeetCode-main/solutions/1166. Design File System/1166.cpp

```
struct TrieNode {
    unordered_map<string, shared_ptr<TrieNode>> children;
    int value;
    TrieNode(int value) : value(value) {}
};

class FileSystem {
public:
    bool createPath(string path, int value) {
        const vector<string> subpaths = getSubpaths(path);
        shared_ptr<TrieNode> node = root;

        for (int i = 0; i < subpaths.size() - 1; ++i) {
            auto it = node->children.find(subpaths[i]);
            if (it == node->children.end())
                return false;
            node = it->second;
        }

        if (node->children.contains(subpaths.back()))
            return false;
        node->children[subpaths.back()] = make_shared<TrieNode>(value);
        return true;
    }

    int get(string path) {
        shared_ptr<TrieNode> node = root;
        for (const string& subpath : getSubpaths(path)) {
            auto it = node->children.find(subpath);
            if (it == node->children.end())
                return -1;
            node = it->second;
        }
        return node->value;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>(0);

    vector<string> getSubpaths(const string& path) {
        vector<string> subpaths;
        istringstream iss(path);
        for (string subpath; getline(iss, subpath, '/');)
            if (!subpath.empty())
                subpaths.push_back(subpath);
        return subpaths;
    }
};
```

182. 1167.cpp

Path: LeetCode-main/solutions/1167. Minimum Cost to Connect Sticks/1167.cpp

```
class Solution {
public:
    int connectSticks(vector<int>& sticks) {
        int ans = 0;
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const int stick : sticks)
            minHeap.push(stick);

        while (minHeap.size() > 1) {
            const int x = minHeap.top();
            minHeap.pop();
            const int y = minHeap.top();
            minHeap.pop();
            ans += x + y;
            minHeap.push(x + y);
        }

        return ans;
    }
};
```

183. 1168.cpp

Path: LeetCode-main/solutions/1168. Optimize Water Distribution in a Village/1168.cpp

```
class Solution {
public:
    int minCostToSupplyWater(int n, vector<int>& wells,
                             vector<vector<int>>& pipes) {
        int ans = 0;
        using P = pair<int, int>;
        vector<vector<P>> graph(n + 1);
        priority_queue<P, vector<P>, greater<>> minHeap; // (d, u)

        for (const vector<int>& pipe : pipes) {
            const int u = pipe[0];
            const int v = pipe[1];
            const int w = pipe[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        // Connect virtual 0 with nodes 1 to n.
        for (int i = 0; i < n; ++i) {
            graph[0].emplace_back(i + 1, wells[i]);
            minHeap.emplace(wells[i], i + 1);
        }

        unordered_set<int> mst{{0}};

        while (mst.size() < n + 1) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (mst.contains(u))
                continue;
            // Add the new vertex.
            mst.insert(u);
            ans += d;
            // Expand if possible.
            for (const auto [v, w] : graph[u])
                if (!mst.contains(v))
                    minHeap.emplace(w, v);
        }

        return ans;
    }
};
```

184. 1169.cpp

Path: LeetCode-main/solutions/1169. Invalid Transactions/1169.cpp

```
struct Transaction {
    string name;
    int time;
    int amount;
    string city;
};

class Solution {
public:
    vector<string> invalidTransactions(vector<string>& transactions) {
        vector<string> ans;
        unordered_map<string, vector<Transaction>> nameToTransactions;

        for (const string& t : transactions) {
            const Transaction transaction = getTransaction(t);
            nameToTransactions[transaction.name].push_back(transaction);
        }

        for (const string& t : transactions) {
            const Transaction currTransaction = getTransaction(t);
            if (currTransaction.amount > 1000) {
                ans.push_back(t);
            } else if (const auto it = nameToTransactions.find(currTransaction.name);
                       it != nameToTransactions.cend()) {
                // Iterate through all the transactions with the same name, check if
                // they're within 60 minutes in a different city.
                for (Transaction transaction : it->second)
                    if (abs(transaction.time - currTransaction.time) <= 60 &&
                        transaction.city != currTransaction.city) {
                        ans.push_back(t);
                        break;
                    }
            }
        }

        return ans;
    }

private:
    Transaction getTransaction(const string& t) {
        istringstream iss(t);
        vector<string> s(4, "");
        for (int i = 0; getline(iss, s[i++], ',');)
            ;
        return {s[0], stoi(s[1]), stoi(s[2]), s[3]}; // [name, time, amount, city]
    }
};
```


185. 117.cpp

Path: LeetCode-main/solutions/117. Populating Next Right Pointers in Each Node II/117.cpp

```
class Solution {
public:
    Node* connect(Node* root) {
        Node* node = root; // the node that is above the current needling

        while (node != nullptr) {
            Node dummy(0); // the dummy node before needling
            // Needle the children of the node.
            for (Node* needle = &dummy; node; node = node->next) {
                if (node->left != nullptr) { // Needle the left child.
                    needle->next = node->left;
                    needle = needle->next;
                }
                if (node->right != nullptr) { // Needle the right child.
                    needle->next = node->right;
                    needle = needle->next;
                }
            }
            node = dummy.next; // Move the node to the next level.
        }

        return root;
    }
};
```

186. 1170.cpp

Path: LeetCode-main/solutions/1170. Compare Strings by Frequency of the Smallest Character/1170.cpp

```
class Solution {
public:
    vector<int> numSmallerByFrequency(vector<string>& queries,
                                     vector<string>& words) {
        vector<int> ans;
        vector<int> wordsFreq;

        for (const string& word : words)
            wordsFreq.push_back(f(word));
        ranges::sort(wordsFreq);

        for (const string& query : queries) {
            const int freq = f(query);
            ans.push_back(wordsFreq.end() - ranges::upper_bound(wordsFreq, freq));
        }

        return ans;
    }

private:
    int f(const string& word) {
        int count = 0;
        char currentChar = 'z' + 1;

        for (const char c : word)
            if (c < currentChar) {
                currentChar = c;
                count = 1;
            } else if (c == currentChar) {
                ++count;
            }

        return count;
    }
};
```

187. 1171.cpp

Path: LeetCode-main/solutions/1171. Remove Zero Sum Consecutive Nodes from Linked List/1171.cpp

```
class Solution {
public:
    ListNode* removeZeroSumSublists(ListNode* head) {
        ListNode dummy(0, head);
        int prefix = 0;
        unordered_map<int, ListNode*> prefixToNode;
        prefixToNode[0] = &dummy;

        for (; head; head = head->next) {
            prefix += head->val;
            prefixToNode[prefix] = head;
        }

        prefix = 0;

        for (head = &dummy; head; head = head->next) {
            prefix += head->val;
            head->next = prefixToNode[prefix]->next;
        }

        return dummy.next;
    }
};
```

188. 1172.cpp

Path: LeetCode-main/solutions/1172. Dinner Plate Stacks/1172.cpp

```
class DinnerPlates {
public:
    DinnerPlates(int capacity) : capacity(capacity) {
        minHeap.push(0);
    }

    void push(int val) {
        const int index = minHeap.top();
        // Add a new stack on demand.
        if (index == stacks.size())
            stacks.push_back({});
        // Push the new value.
        stacks[index].push(val);
        // If the stack pushed is full, remove its candidacy from `minHeap`.
        if (stacks[index].size() == capacity) {
            minHeap.pop();
            // If `minHeap` is empty, the next available stack index is |stacks|.
            if (minHeap.empty())
                minHeap.push(stacks.size());
        }
    }

    int pop() {
        // Remove empty stacks from the back.
        while (!stacks.empty() && stacks.back().empty())
            stacks.pop_back();
        if (stacks.empty())
            return -1;
        return popAtStack(stacks.size() - 1);
    }

    int popAtStack(int index) {
        if (index >= stacks.size() || stacks[index].empty())
            return -1;
        // If the stack is going to have space, add its candidacy to `minHeap`.
        if (stacks[index].size() == capacity)
            minHeap.push(index);
        const int val = stacks[index].top();
        stacks[index].pop();
        return val;
    }

private:
    const int capacity;
    vector<stack<int>> stacks;
    // the minimum indices of the stacks to push
    priority_queue<int, vector<int>, greater<>> minHeap;
};
```

189. 1175.cpp

Path: LeetCode-main/solutions/1175. Prime Arrangements/1175.cpp

```
class Solution {
public:
    int numPrimeArrangements(int n) {
        const int count = countPrimes(n);
        return factorial(count) * factorial(n - count) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    int countPrimes(int n) {
        vector<bool> prime(n + 1, true);
        prime[0] = false;
        prime[1] = false;
        for (int i = 2; i <= sqrt(n); ++i)
            if (prime[i])
                for (int j = i * i; j <= n; j += i)
                    prime[j] = false;
        return ranges::count(prime, true);
    }

    long factorial(int n) {
        long res = 1;
        for (int i = 2; i <= n; ++i)
            res = res * i % kMod;
        return res;
    }
};
```

190. 1176.cpp

Path: LeetCode-main/solutions/1176. Diet Plan Performance/1176.cpp

```
class Solution {
public:
    int dietPlanPerformance(vector<int>& calories, int k, int lower, int upper) {
        int ans = 0;
        int sum = 0;

        for (int i = 0; i < calories.size(); ++i) {
            sum += calories[i];
            if (i < k - 1)
                continue;
            if (i >= k)
                sum -= calories[i - k];
            if (sum < lower)
                --ans;
            else if (sum > upper)
                ++ans;
        }

        return ans;
    }
};
```

191. 1177.cpp

Path: LeetCode-main/solutions/1177. Can Make Palindrome from Substring/1177.cpp

```
class Solution {
public:
    vector<bool> canMakePaliQueries(string s, vector<vector<int>>& queries) {
        vector<bool> ans;
        vector<unsigned int> dp(s.length() + 1);

        for (int i = 1; i <= s.length(); ++i)
            dp[i] = dp[i - 1] ^ 1 << s[i - 1] - 'a';

        for (const vector<int>& query : queries) {
            const int odds = popcount(dp[query[1] + 1] ^ dp[query[0]]);
            ans.push_back(odds / 2 <= query[2]);
        }

        return ans;
    }
};
```

192. 1178.cpp

Path: LeetCode-main/solutions/1178. Number of Valid Words for Each Puzzle/1178.cpp

```
class Solution {
public:
    vector<int> findNumOfValidWords(vector<string>& words,
                                   vector<string>& puzzles) {
        vector<int> ans;
        unordered_map<int, int> binaryCount;

        for (const string& word : words) {
            int mask = 0;
            for (char c : word)
                mask |= 1 << c - 'a';
            ++binaryCount[mask];
        }

        for (const string& puzzle : puzzles) {
            int valid = 0;
            const int n = puzzle.length() - 1;
            for (int i = 0; i < (1 << n); ++i) {
                int mask = 1 << puzzle[0] - 'a';
                for (int j = 0; j < n; ++j)
                    if (i >> j & 1)
                        mask |= 1 << puzzle[j + 1] - 'a';
                if (const auto it = binaryCount.find(mask); it != binaryCount.cend())
                    valid += it->second;
            }
            ans.push_back(valid);
        }

        return ans;
    }
};
```


193. 118.cpp

Path: LeetCode-main/solutions/118. Pascal's Triangle/118.cpp

```
class Solution {
public:
    vector<vector<int>> generate(int numRows) {
        vector<vector<int>> ans;

        for (int i = 0; i < numRows; ++i)
            ans.push_back(vector<int>(i + 1, 1));

        for (int i = 2; i < numRows; ++i)
            for (int j = 1; j < ans[i].size() - 1; ++j)
                ans[i][j] = ans[i - 1][j - 1] + ans[i - 1][j];

        return ans;
    }
};
```

194. 1180.cpp

Path: LeetCode-main/solutions/1180. Count Substrings with Only One Distinct Letter/1180.cpp

```
class Solution {
public:
    int countLetters(string s) {
        int ans = 0;
        int dp = 0;    // the length of the running letter
        char letter = '@'; // the running letter

        for (const char c : s) {
            if (c == letter) {
                ++dp;
            } else {
                dp = 1;
                letter = c;
            }
            // Add the number of substrings ending in the current letter.
            ans += dp;
        }
        return ans;
    }
};
```

195. 1181.cpp

Path: LeetCode-main/solutions/1181. Before and After Puzzle/1181.cpp

```
class Solution {
public:
    vector<string> beforeAndAfterPuzzles(vector<string>& phrases) {
        set<string> ans;
        unordered_map<string, set<string>> firstWordToLasts;
        unordered_map<string, set<string>> lastWordToFirsts;

        for (const string& phrase : phrases) {
            const int firstSpaceIndex = phrase.find(' ');
            const int lastSpaceIndex = phrase.rfind(' ');
            // the index after the firstWord
            const int i =
                firstSpaceIndex == string::npos ? phrase.length() : firstSpaceIndex;
            // the index of the lastWord
            const int j = lastSpaceIndex == string::npos ? 0 : lastSpaceIndex + 1;
            const string firstWord = phrase.substr(0, i);
            const string lastWord = phrase.substr(j);

            // Concatenate `phrase` with `last` having the same `firstWord`.
            if (const auto it = firstWordToLasts.find(lastWord);
                it != firstWordToLasts.cend())
                for (const string& last : it->second)
                    ans.insert(phrase + last);

            // Concatenate `first` having the same `lastWord` with `phrase`.
            if (const auto it = lastWordToFirsts.find(firstWord);
                it != lastWordToFirsts.cend())
                for (const string& first : it->second)
                    ans.insert(first + phrase);

            // e.g. "a b c" -> {"a": " b c"}
            //      "a" -> {"a": ""}
            firstWordToLasts[firstWord].insert(phrase.substr(i));

            // e.g. "a b c" -> {"c": "a b "}
            //      "a" -> {"a": ""}
            lastWordToFirsts[lastWord].insert(phrase.substr(0, j));
        }

        return {ans.begin(), ans.end()};
    }
};
```

196. 1182.cpp

Path: LeetCode-main/solutions/1182. Shortest Distance to Target Color/1182.cpp

```
class Solution {
public:
    vector<int> shortestDistanceColor(vector<int>& colors,
                                     vector<vector<int>>& queries) {
        constexpr int kNumColor = 3;
        const int n = colors.size();
        vector<int> ans;
        // left[i][c] := the closest index of color c in index i to the left
        vector<vector<int>> left(n, vector<int>(kNumColor + 1));
        // right[i][c] := the closest index of color c in index i to the right
        vector<vector<int>> right(n, vector<int>(kNumColor + 1));

        vector<int> colorToClosestIndex{0, -1, -1, -1}; // 0-indexed, -1 := N/A
        for (int i = 0; i < n; ++i) {
            colorToClosestIndex[colors[i]] = i;
            for (int c = 1; c <= kNumColor; ++c)
                left[i][c] = colorToClosestIndex[c];
        }

        colorToClosestIndex = {0, -1, -1, -1}; // Reset.
        for (int i = n - 1; i >= 0; --i) {
            colorToClosestIndex[colors[i]] = i;
            for (int c = 1; c <= kNumColor; ++c)
                right[i][c] = colorToClosestIndex[c];
        }

        for (const vector<int>& query : queries) {
            const int i = query[0];
            const int c = query[1];
            const int leftDist = left[i][c] == -1 ? INT_MAX : i - left[i][c];
            const int rightDist = right[i][c] == -1 ? INT_MAX : right[i][c] - i;
            const int minDist = min(leftDist, rightDist);
            ans.push_back(minDist == INT_MAX ? -1 : minDist);
        }

        return ans;
    }
};
```

197. 1183-2.cpp

Path: LeetCode-main/solutions/1183. Maximum Number of Ones/1183-2.cpp

```
class Solution {
public:
    int maximumNumberOfOnes(int width, int height, int sideLength, int maxOnes) {
        vector<int> subCount;

        auto getCount = [sideLength](int length, int index) -> int {
            return (length - index - 1) / sideLength + 1;
        };

        for (int i = 0; i < sideLength; ++i)
            for (int j = 0; j < sideLength; ++j)
                subCount.push_back(getCount(width, i) * getCount(height, j));

        ranges::sort(subCount, greater<>());
        return accumulate(subCount.begin(), subCount.begin() + maxOnes, 0);
    }
};
```

198. 1183.cpp

Path: LeetCode-main/solutions/1183. Maximum Number of Ones/1183.cpp

```
class Solution {
public:
    int maximumNumberOfOnes(int width, int height, int sideLength, int maxOnes) {
        int ans = 0;
        vector<vector<int>> submatrix(sideLength, vector<int>(sideLength));
        priority_queue<int> maxHeap;

        for (int i = 0; i < width; ++i)
            for (int j = 0; j < height; ++j)
                ++submatrix[i % sideLength][j % sideLength];

        for (const vector<int>& row : submatrix)
            for (const int a : row)
                maxHeap.push(a);

        for (int i = 0; i < maxOnes; ++i)
            ans += maxHeap.top(), maxHeap.pop();

        return ans;
    }
};
```

199. 1184.cpp

Path: LeetCode-main/solutions/1184. Distance Between Bus Stops/1184.cpp

```
class Solution {
public:
    int distanceBetweenBusStops(vector<int>& distance, int start,
                                int destination) {
        int clockwise = 0;
        int counterclockwise = 0;

        if (start > destination)
            swap(start, destination);

        for (int i = 0; i < distance.size(); ++i) {
            if (i >= start && i < destination)
                clockwise += distance[i];
            else
                counterclockwise += distance[i];
        }

        return min(clockwise, counterclockwise);
    }
};
```

200. 1185.cpp

Path: LeetCode-main/solutions/1185. Day of the Week/1185.cpp

```
class Solution {
public:
    string dayOfTheWeek(int day, int month, int year) {
        vector<string> week = {"Sunday", "Monday", "Tuesday", "Wednesday",
                               "Thursday", "Friday", "Saturday"};

        vector<int> days = {
            31, isLeapYear(year) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        int count = 0;

        for (int i = 1971; i < year; ++i)
            count += i % 4 == 0 ? 366 : 365;
        for (int i = 0; i < month - 1; ++i)
            count += days[i];
        count += day;

        return week[(count + 4) % 7];
    }

private:
    bool isLeapYear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    }
};
```


201. 1186-2.cpp

Path: LeetCode-main/solutions/1186. Maximum Subarray Sum with One Deletion/1186-2.cpp

```
class Solution {
public:
    // Similar to 53. Maximum Subarray
    int maximumSum(vector<int>& arr) {
        constexpr int kMin = INT_MIN / 2;
        int ans = kMin;
        int zero = kMin; // no deletion
        int one = kMin;  // <= 1 deletion

        for (const int a : arr) {
            one = max({a, one + a, zero /*delete a*/});
            zero = max(a, zero + a);
            ans = max(ans, one);
        }

        return ans;
    }
};
```

202. 1186.cpp

Path: LeetCode-main/solutions/1186. Maximum Subarray Sum with One Deletion/1186.cpp

```
class Solution {
public:
    // Similar to 53. Maximum Subarray
    int maximumSum(vector<int>& arr) {
        // dp[0][i] := the maximum sum subarray ending in i (no deletion)
        // dp[1][i] := the maximum sum subarray ending in i (at most 1 deletion)
        vector<vector<int>> dp(2, vector<int>(arr.size()));

        dp[0][0] = arr[0];
        dp[1][0] = arr[0];
        for (int i = 1; i < arr.size(); ++i) {
            dp[0][i] = max(arr[i], dp[0][i - 1] + arr[i]);
            dp[1][i] =
                max({arr[i], dp[1][i - 1] + arr[i], dp[0][i - 1] /*delete arr[i]*/});
        }

        return ranges::max(dp[1]);
    }
};
```

203. 1187.cpp

Path: LeetCode-main/solutions/1187. Make Array Strictly Increasing/1187.cpp

```
class Solution {
public:
    int makeArrayIncreasing(vector<int>& arr1, vector<int>& arr2) {
        // dp[i] := the minimum steps to reach i at previous round
        unordered_map<int, int> dp{{-1, 0}};

        ranges::sort(arr2);

        for (const int a : arr1) {
            unordered_map<int, int> newDp;
            for (const auto& [val, steps] : dp) {
                // It's possible to use the value in the arr1.
                if (a > val)
                    newDp[a] = min(newDp.contains(a) ? newDp[a] : INT_MAX, steps);
                // Also try the value in the arr2.
                if (const auto it = ranges::upper_bound(arr2, val); it != arr2.cend())
                    newDp[*it] =
                        min(newDp.contains(*it) ? newDp[*it] : INT_MAX, steps + 1);
            }
            if (newDp.empty())
                return -1;
            dp = std::move(newDp);
        }

        int ans = INT_MAX;
        for (const auto& [_, steps] : dp)
            ans = min(ans, steps);
        return ans;
    }
};
```

204. 1188.cpp

Path: LeetCode-main/solutions/1188. Design Bounded Blocking Queue/1188.cpp

```
// LeetCode doesn't support C++20 yet, so we don't have std::counting_semaphore
// or binary_semaphore.
#include <semaphore.h>

class BoundedBlockingQueue {
public:
    BoundedBlockingQueue(int capacity) {
        sem_init(&enqueueSemaphore, /*pshared=*/0, /*value=*/capacity);
        sem_init(&dequeueSemaphore, /*pshared=*/0, /*value=*/0);
    }

    ~BoundedBlockingQueue() {
        sem_destroy(&enqueueSemaphore);
        sem_destroy(&dequeueSemaphore);
    }

    void enqueue(int element) {
        sem_wait(&enqueueSemaphore);
        q.push(element);
        sem_post(&dequeueSemaphore);
    }

    int dequeue() {
        sem_wait(&dequeueSemaphore);
        const int element = q.front();
        q.pop();
        sem_post(&enqueueSemaphore);
        return element;
    }

    int size() {
        return q.size();
    }

private:
    queue<int> q;
    sem_t enqueueSemaphore;
    sem_t dequeueSemaphore;
};
```

205. 1189.cpp

Path: LeetCode-main/solutions/1189. Maximum Number of Balloons/1189.cpp

```
class Solution {
public:
    int maxNumberOfBalloons(string text) {
        int ans = INT_MAX;
        vector<int> count(26);

        for (char c : text)
            ++count[c - 'a'];

        for (char c : string("ban"))
            ans = min(ans, count[c - 'a']);

        for (char c : string("lo"))
            ans = min(ans, count[c - 'a'] / 2);

        return ans;
    }
};
```

206. 119.cpp

Path: LeetCode-main/solutions/119. Pascal's Triangle II/119.cpp

```
class Solution {
public:
    vector<int> getRow(int rowIndex) {
        vector<int> ans(rowIndex + 1, 1);

        for (int i = 2; i < rowIndex + 1; ++i)
            for (int j = 1; j < i; ++j)
                ans[i - j] += ans[i - j - 1];

        return ans;
    }
};
```

207. 1190-2.cpp

Path: LeetCode-main/solutions/1190. Reverse Substrings Between Each Pair of Parentheses/1190-2.cpp

```
class Solution {
public:
    string reverseParentheses(string s) {
        string ans;
        stack<int> stack;
        unordered_map<int, int> pair;

        for (int i = 0; i < s.length(); ++i)
            if (s[i] == '(') {
                stack.push(i);
            } else if (s[i] == ')') {
                const int j = stack.top();
                stack.pop();
                pair[i] = j;
                pair[j] = i;
            }

        for (int i = 0, d = 1; i < s.length(); i += d)
            if (s[i] == '(' || s[i] == ')') {
                i = pair[i];
                d = -d;
            } else {
                ans += s[i];
            }

        return ans;
    }
};
```

208. 1190.cpp

Path: LeetCode-main/solutions/1190. Reverse Substrings Between Each Pair of Parentheses/1190.cpp

```
class Solution {
public:
    string reverseParentheses(string s) {
        stack<int> stack;
        string ans;

        for (const char c : s)
            if (c == '(') {
                stack.push(ans.length());
            } else if (c == ')') {
                // Reverse the corresponding substring between ().
                const int j = stack.top();
                stack.pop();
                reverse(ans.begin() + j, ans.end());
            } else {
                ans += c;
            }

        return ans;
    }
};
```


209. 1191.cpp

Path: LeetCode-main/solutions/1191. K-Concatenation Maximum Sum/1191.cpp

```
class Solution {
public:
    int kConcatenationMaxSum(vector<int>& arr, int k) {
        constexpr int kMod = 1'000'000'007;
        const int sz = arr.size() * (k == 1 ? 1 : 2);
        const int sum = accumulate(arr.begin(), arr.end(), 0);
        // The concatenated array will be [arr1, arr2, ..., arrk].
        // If sum(arr) > 0 and k > 2, then arr2, ..., arr(k - 1) should be included.
        // Equivalently, maxSubarraySum is from arr1 and arrk.
        return (sum > 0 && k > 2
                ? kadane(arr, sz) + static_cast<long>(sum) * (k - 2)
                : kadane(arr, sz)) %
            kMod;
    }

private:
    int kadane(const vector<int>& arr, int sz) {
        int ans = 0;
        int sum = 0;
        for (int i = 0; i < sz; ++i) {
            const int a = arr[i % arr.size()];
            sum = max(a, sum + a);
            ans = max(ans, sum);
        }
        return ans;
    }
};
```

210. 1192.cpp

Path: LeetCode-main/solutions/1192. Critical Connections in a Network/1192.cpp

```
class Solution {
public:
    vector<vector<int>> criticalConnections(int n,
                                         vector<vector<int>>& connections) {
        vector<vector<int>> ans;
        vector<vector<int>> graph(n);

        for (const vector<int>& connection : connections) {
            const int u = connection[0];
            const int v = connection[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        // rank[i] := the minimum node that node i can reach with forward edges
        // Initialize with NO_RANK = -2 to indicate not visited.
        getRank(graph, 0, 0, vector<int>(n, NO_RANK), ans);
        return ans;
    }

private:
    static constexpr int NO_RANK = -2;

    // Gets the minimum rank that u can reach with forward edges.
    int getRank(const vector<vector<int>>& graph, int u, int currRank,
               vector<int>&& rank, vector<vector<int>>& ans) {
        if (rank[u] != NO_RANK) // The rank is already determined.
            return rank[u];

        rank[u] = currRank;
        int minRank = currRank;

        for (const int v : graph[u]) {
            // visited || parent (that's why NO_RANK = -2 instead of -1)
            if (rank[u] == rank.size() || rank[v] == currRank - 1)
                continue;
            const int nextRank =
                getRank(graph, v, currRank + 1, std::move(rank), ans);
            // (u, v) is the only way for u go to v.
            if (nextRank == currRank + 1)
                ans.push_back({u, v});
            minRank = min(minRank, nextRank);
        }

        rank[u] = rank.size(); // Mark as visited.
        return minRank;
    }
};
```

211. 1195.cpp

Path: LeetCode-main/solutions/1195. Fizz Buzz Multithreaded/1195.cpp

```
// LeetCode doesn't support C++20 yet, so we don't have std::counting_semaphore
// or binary_semaphore.
#include <semaphore.h>

class FizzBuzz {
public:
    FizzBuzz(int n) : n(n) {
        sem_init(&fizzSemaphore, /*pshared=*/0, /*value=*/0);
        sem_init(&buzzSemaphore, /*pshared=*/0, /*value=*/0);
        sem_init(&fizzbuzzSemaphore, /*pshared=*/0, /*value=*/0);
        sem_init(&numberSemaphore, /*pshared=*/0, /*value=*/1);
    }

    ~FizzBuzz() {
        sem_destroy(&fizzSemaphore);
        sem_destroy(&buzzSemaphore);
        sem_destroy(&fizzbuzzSemaphore);
        sem_destroy(&numberSemaphore);
    }

    // printFizz() outputs "fizz".
    void fizz(function<void()> printFizz) {
        for (int i = 1; i <= n; ++i)
            if (i % 3 == 0 && i % 15 != 0) {
                sem_wait(&fizzSemaphore);
                printFizz();
                sem_post(&numberSemaphore);
            }
    }

    // printBuzz() outputs "buzz".
    void buzz(function<void()> printBuzz) {
        for (int i = 1; i <= n; ++i)
            if (i % 5 == 0 && i % 15 != 0) {
                sem_wait(&buzzSemaphore);
                printBuzz();
                sem_post(&numberSemaphore);
            }
    }

    // printFizzBuzz() outputs "fizzbuzz".
    void fizzbuzz(function<void()> printFizzBuzz) {
        for (int i = 1; i <= n; ++i)
            if (i % 15 == 0) {
                sem_wait(&fizzbuzzSemaphore);
                printFizzBuzz();
                sem_post(&numberSemaphore);
            }
    }

    // printNumber(x) outputs "x", where x is an integer.
    void number(function<void(int)> printNumber) {
        for (int i = 1; i <= n; ++i) {
            sem_wait(&numberSemaphore);
            if (i % 15 == 0)
                sem_post(&fizzbuzzSemaphore);
            else if (i % 3 == 0)
                sem_post(&fizzSemaphore);
            else if (i % 5 == 0)
                sem_post(&buzzSemaphore);
            else {
                printNumber(i);
                sem_post(&numberSemaphore);
            }
        }
    }

private:
    const int n;
    sem_t fizzSemaphore;
    sem_t buzzSemaphore;
    sem_t fizzbuzzSemaphore;
    sem_t numberSemaphore;
};
```

212. 1196.cpp

Path: LeetCode-main/solutions/1196. How Many Apples Can You Put into the Basket/1196.cpp

```
class Solution {
public:
    int maxNumberOfApples(vector<int>& weight) {
        int sum = 0;

        ranges::sort(weight);

        for (int i = 0; i < weight.size(); ++i) {
            sum += weight[i];
            if (sum > 5000)
                return i;
        }

        return weight.size();
    }
};
```

213. 1197.cpp

Path: LeetCode-main/solutions/1197. Minimum Knight Moves/1197.cpp

```
class Solution {  
public:  
    int minKnightMoves(int x, int y) {  
        return dp(abs(x), abs(y));  
    }  
  
private:  
    struct PairHash {  
        size_t operator()(const pair<int, int>& p) const {  
            return p.first ^ p.second;  
        }  
    };  
  
    unordered_map<pair<int, int>, int, PairHash> mem;  
  
    int dp(int x, int y) {  
        if (x + y == 0) // (0, 0)  
            return 0;  
        if (x + y == 2) // (0, 2), (1, 1), (2, 0)  
            return 2;  
        if (const auto it = mem.find({x, y}); it != mem.cend())  
            return it->second;  
  
        return mem[{x, y}] = 1 + min(dp(abs(x - 2), abs(y - 1)), //  
                                     dp(abs(x - 1), abs(y - 2)));  
    }  
};
```


215. 1199.cpp

Path: LeetCode-main/solutions/1199. Minimum Time to Build Blocks/1199.cpp

```
class Solution {
public:
    int minBuildTime(vector<int>& blocks, int split) {
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const int block : blocks)
            minHeap.push(block);

        while (minHeap.size() > 1) {
            minHeap.pop();           // the minimum
            const int x = minHeap.top(); // the second minimum
            minHeap.pop();
            minHeap.push(x + split);
        }

        return minHeap.top();
    }
};
```

216. 12-2.cpp

Path: LeetCode-main/solutions/12. Integer to Roman/12-2.cpp

```
class Solution {
public:
    string intToRoman(int num) {
        const vector<string> M{"", "M", "MM", "MMM"};
        const vector<string> C{"", "C", "CC", "CCC", "CD",
                                "D", "DC", "DCC", "DCCC", "CM"};
        const vector<string> X{"", "X", "XX", "XXX", "XL",
                                "L", "LX", "LXX", "LXXX", "XC"};
        const vector<string> I{"", "I", "II", "III", "IV",
                                "V", "VI", "VII", "VIII", "IX"};
        return M[num / 1000] + C[num % 1000 / 100] + X[num % 100 / 10] +
            I[num % 10];
    }
};
```


217. 12.cpp

Path: LeetCode-main/solutions/12. Integer to Roman/12.cpp

```
class Solution {
public:
    string intToRoman(int num) {
        const vector<pair<int, string>> valueSymbols{
            {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"}, {100, "C"},
            {90, "XC"}, {50, "L"}, {40, "XL"}, {10, "X"}, {9, "IX"},
            {5, "V"}, {4, "IV"}, {1, "I"}};
        string ans;

        for (const auto& [value, symbol] : valueSymbols) {
            if (num == 0)
                break;
            while (num >= value) {
                num -= value;
                ans += symbol;
            }
        }
        return ans;
    }
};
```

218. 120.cpp

Path: LeetCode-main/solutions/120. Triangle/120.cpp

```
class Solution {
public:
    int minimumTotal(vector<vector<int>>& triangle) {
        for (int i = triangle.size() - 2; i >= 0; --i)
            for (int j = 0; j <= i; ++j)
                triangle[i][j] += min(triangle[i + 1][j], triangle[i + 1][j + 1]);
        return triangle[0][0];
    }
};
```

219. 1200.cpp

Path: LeetCode-main/solutions/1200. Minimum Absolute Difference/1200.cpp

```
class Solution {
public:
    vector<vector<int>> minimumAbsDifference(vector<int>& arr) {
        vector<vector<int>> ans;
        int mn = INT_MAX;

        ranges::sort(arr);

        for (int i = 1; i < arr.size(); ++i) {
            const int diff = arr[i] - arr[i - 1];
            if (diff < mn) {
                mn = diff;
                ans.clear();
            }
            if (diff == mn)
                ans.push_back({arr[i - 1], arr[i]});
        }

        return ans;
    }
};
```

220. 1201.cpp

Path: LeetCode-main/solutions/1201. Ugly Number III/1201.cpp

```
class Solution {
public:
    int nthUglyNumber(int n, long a, long b, long c) {
        const long ab = a * b / __gcd(a, b);
        const long ac = a * c / __gcd(a, c);
        const long bc = b * c / __gcd(b, c);
        const long abc = a * bc / __gcd(a, bc);
        int l = 1;
        int r = 2'000'000'000;

        while (l < r) {
            const int m = l + (r - l) / 2;
            if (m / a + m / b + m / c - m / ab - m / ac - m / bc + m / abc >= n)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }
};
```

221. 1202.cpp

Path: LeetCode-main/solutions/1202. Smallest String With Swaps/1202.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    string smallestStringWithSwaps(string s, vector<vector<int>>& pairs) {
        string ans;
        UnionFind uf(s.length());
        unordered_map<int, priority_queue<char, vector<char>, greater<>>>
            indexToLetters;

        for (const vector<int>& pair : pairs) {
            const int a = pair[0];
            const int b = pair[1];
            uf.unionByRank(a, b);
        }

        for (int i = 0; i < s.length(); ++i)
            indexToLetters[uf.find(i)].push(s[i]);

        for (int i = 0; i < s.length(); ++i)
            ans += indexToLetters[uf.find(i)].top(), indexToLetters[uf.find(i)].pop();

        return ans;
    }
};
```

222. 1203.cpp

Path: LeetCode-main/solutions/1203. Sort Items by Groups Respecting Dependencies/1203.cpp

```
class Solution {
public:
    vector<int> sortItems(int n, int m, vector<int>& group,
                        vector<vector<int>>& beforeItems) {
        vector<vector<int>> graph(n + m);
        vector<int> inDegrees(n + m);

        // Build the graph by remapping the k-th group to k + n imaginary node.
        for (int i = 0; i < group.size(); ++i) {
            if (group[i] == -1)
                continue;
            graph[group[i] + n].push_back(i);
            ++inDegrees[i];
        }

        for (int i = 0; i < beforeItems.size(); ++i)
            for (const int b : beforeItems[i]) {
                const int u = group[b] == -1 ? b : group[b] + n;
                const int v = group[i] == -1 ? i : group[i] + n;
                if (u == v) { // u and v are already in the same group.
                    graph[b].push_back(i);
                    ++inDegrees[i];
                } else {
                    graph[u].push_back(v);
                    ++inDegrees[v];
                }
            }

        // Perform topological sorting.
        vector<int> ans;

        for (int i = 0; i < n + m; ++i)
            if (inDegrees[i] == 0) // inDegrees[i] == -1 means visited.
                dfs(graph, i, inDegrees, n, ans);

        return ans.size() == n ? ans : vector<int>();
    }

private:
    void dfs(const vector<vector<int>>& graph, int u, vector<int>& inDegrees,
            int n, vector<int>& ans) {
        if (u < n)
            ans.push_back(u);

        inDegrees[u] = -1; // Mark as visited.

        for (const int v : graph[u])
            if (--inDegrees[v] == 0)
                dfs(graph, v, inDegrees, n, ans);
    }
};
```

223. 1206.cpp

Path: LeetCode-main/solutions/1206. Design Skiplist/1206.cpp

```
struct Node {
    int val = -1;
    shared_ptr<Node> next;
    shared_ptr<Node> down;
};

class Skiplist {
public:
    bool search(int target) {
        for (shared_ptr<Node> node = dummy; node; node = node->down) {
            advance(node, target);
            if (node->next && node->next->val == target)
                return true;
        }
        return false;
    }

    void add(int num) {
        // Collect nodes that are before the insertion point.
        stack<shared_ptr<Node>> nodes;
        for (shared_ptr<Node> node = dummy; node; node = node->down) {
            advance(node, num);
            nodes.push(node);
        }

        shared_ptr<Node> down;
        bool shouldInsert = true;
        while (shouldInsert && !nodes.empty()) {
            shared_ptr<Node> prev = nodes.top();
            nodes.pop();
            prev->next = make_shared<Node>(num, prev->next, down);
            down = prev->next;
            shouldInsert = rand() % 2 == 1;
        }

        // Create a topmost new level dummy that points to the existing dummy.
        if (shouldInsert)
            dummy = make_shared<Node>(-1, nullptr, dummy);
    }

    bool erase(int num) {
        bool found = false;
        for (shared_ptr<Node> node = dummy; node; node = node->down) {
            advance(node, num);
            if (node->next && node->next->val == num) {
                node->next = node->next->next;
                found = true;
            }
        }
        return found;
    }

private:
    shared_ptr<Node> dummy = make_shared<Node>(-1);

    void advance(shared_ptr<Node>& node, int target) {
        while (node->next && node->next->val < target)
            node = node->next;
    }
};
```

224. 1207.cpp

Path: LeetCode-main/solutions/1207. Unique Number of Occurrences/1207.cpp

```
class Solution {
public:
    bool uniqueOccurrences(vector<int>& arr) {
        unordered_map<int, int> count;
        unordered_set<int> occurrences;

        for (const int a : arr)
            ++count[a];

        for (const auto& [_, value] : count)
            if (!occurrences.insert(value).second)
                return false;

        return true;
    }
};
```


225. 1208.cpp

Path: LeetCode-main/solutions/1208. Get Equal Substrings Within Budget/1208.cpp

```
class Solution {
public:
    int equalSubstring(string s, string t, int maxCost) {
        int j = 0;
        for (int i = 0; i < s.length(); ++i) {
            maxCost -= abs(s[i] - t[i]);
            if (maxCost < 0)
                maxCost += abs(s[j] - t[j++]);
        }
        return s.length() - j;
    }
};
```

226. 1209.cpp

Path: LeetCode-main/solutions/1209. Remove All Adjacent Duplicates in String II/1209.cpp

```
class Solution {
public:
    string removeDuplicates(const string& s, int k) {
        string ans;
        vector<pair<char, int>> stack;

        for (const char c : s)
            if (stack.empty() || stack.back().first != c)
                stack.emplace_back(c, 1);
            else if (++stack.back().second == k) // stack[-1] == c
                stack.pop_back();

        for (const auto& [c, count] : stack)
            ans.append(count, c);

        return ans;
    }
};
```

227. 121.cpp

Path: LeetCode-main/solutions/121. Best Time to Buy and Sell Stock/121.cpp

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int sellOne = 0;
        int holdOne = INT_MIN;

        for (const int price : prices) {
            sellOne = max(sellOne, holdOne + price);
            holdOne = max(holdOne, -price);
        }

        return sellOne;
    }
};
```

228. 1210.cpp

Path: LeetCode-main/solutions/1210. Minimum Moves to Reach Target with Rotations/1210.cpp

```
enum class Pos { kHorizontal, kVertical };

class Solution {
public:
    int minimumMoves(vector<vector<int>>& grid) {
        const int n = grid.size();
        queue<tuple<int, int, Pos>> q{{0, 0, Pos::kHorizontal}};
        vector<vector<vector<bool>>> seen(n, vector<vector<bool>>(n, vector<bool>(2)));
        seen[0][0][static_cast<int>(Pos::kHorizontal)] = true;

        auto canMoveRight = [&](int x, int y, Pos pos) -> bool {
            if (pos == Pos::kHorizontal)
                return y + 2 < n && !grid[x][y + 2];
            return y + 1 < n && !grid[x][y + 1] && !grid[x + 1][y + 1];
        };

        auto canMoveDown = [&](int x, int y, Pos pos) -> bool {
            if (pos == Pos::kVertical)
                return x + 2 < n && !grid[x + 2][y];
            return x + 1 < n && !grid[x + 1][y] && !grid[x + 1][y + 1];
        };

        auto canRotateClockwise = [&](int x, int y, Pos pos) -> bool {
            return pos == Pos::kHorizontal && x + 1 < n && !grid[x + 1][y + 1] &&
                !grid[x + 1][y];
        };

        auto canRotateCounterclockwise = [&](int x, int y, Pos pos) -> bool {
            return pos == Pos::kVertical && y + 1 < n && !grid[x + 1][y + 1] &&
                !grid[x][y + 1];
        };

        for (int step = 0; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [x, y, pos] = q.front();
                q.pop();
                if (x == n - 1 && y == n - 2 && pos == Pos::kHorizontal)
                    return step;
                if (canMoveRight(x, y, pos) && !seen[x][y + 1][static_cast<int>(pos)]) {
                    q.emplace(x, y + 1, pos);
                    seen[x][y + 1][static_cast<int>(pos)] = true;
                }
                if (canMoveDown(x, y, pos) && !seen[x + 1][y][static_cast<int>(pos)]) {
                    q.emplace(x + 1, y, pos);
                    seen[x + 1][y][static_cast<int>(pos)] = true;
                }
                const Pos newPos =
                    pos == Pos::kHorizontal ? Pos::kVertical : Pos::kHorizontal;
                if ((canRotateClockwise(x, y, pos) ||
                    canRotateCounterclockwise(x, y, pos)) &&
                    !seen[x][y][static_cast<int>(newPos)]) {
                    q.emplace(x, y, newPos);
                    seen[x][y][static_cast<int>(newPos)] = true;
                }
            }
        return -1;
    }
};
```

229. 1213.cpp

Path: LeetCode-main/solutions/1213. Intersection of Three Sorted Arrays/1213.cpp

```
class Solution {
public:
    vector<int> arraysIntersection(vector<int>& arr1, vector<int>& arr2,
                                   vector<int>& arr3) {
        vector<int> ans;
        int i = 0;
        int j = 0;
        int k = 0;

        while (i < arr1.size() && j < arr2.size() && k < arr3.size()) {
            const int mn = min({arr1[i], arr2[j], arr3[k]});
            if (arr1[i] == mn && arr2[j] == mn && arr3[k] == mn) {
                ans.push_back(mn);
                ++i;
                ++j;
                ++k;
            } else if (arr1[i] == mn) {
                ++i;
            } else if (arr2[j] == mn) {
                ++j;
            } else {
                ++k;
            }
        }

        return ans;
    }
};
```

230. 1214.cpp

Path: LeetCode-main/solutions/1214. Two Sum BSTs/1214.cpp

```
class BSTIterator {
public:
    BSTIterator(TreeNode* root, bool leftToRight) : leftToRight(leftToRight) {
        pushUntilNull(root);
    }

    bool hasNext() {
        return !stack.empty();
    }

    int next() {
        TreeNode* root = stack.top();
        stack.pop();
        pushUntilNull(leftToRight ? root->right : root->left);
        return root->val;
    }

private:
    stack<TreeNode*> stack;
    bool leftToRight;

    void pushUntilNull(TreeNode* root) {
        while (root != nullptr) {
            stack.push(root);
            root = leftToRight ? root->left : root->right;
        }
    }
};

class Solution {
public:
    bool twoSumBSTs(TreeNode* root1, TreeNode* root2, int target) {
        BSTIterator bst1(root1, true);
        BSTIterator bst2(root2, false);

        for (int l = bst1.next(), r = bst2.next(); true;) {
            const int sum = l + r;
            if (sum == target)
                return true;
            if (sum < target) {
                if (!bst1.hasNext())
                    return false;
                l = bst1.next();
            } else {
                if (!bst2.hasNext())
                    return false;
                r = bst2.next();
            }
        }
    }
};
```

231. 1215-2.cpp

Path: LeetCode-main/solutions/1215. Stepping Numbers/1215-2.cpp

```
class Solution {
public:
    vector<int> countSteppingNumbers(int low, int high) {
        vector<int> ans;
        if (low == 0)
            ans.push_back(0);

        for (int i = 1; i <= 9; ++i)
            dfs(i, low, high, ans);

        ranges::sort(ans);
        return ans;
    }

private:
    void dfs(long curr, int low, int high, vector<int>& ans) {
        if (curr > high)
            return;
        if (curr >= low)
            ans.push_back(curr);

        const int lastDigit = curr % 10;
        if (lastDigit > 0)
            dfs(curr * 10 + lastDigit - 1, low, high, ans);
        if (lastDigit < 9)
            dfs(curr * 10 + lastDigit + 1, low, high, ans);
    }
};
```

232. 1215.cpp

Path: LeetCode-main/solutions/1215. Stepping Numbers/1215.cpp

```
class Solution {
public:
    vector<int> countSteppingNumbers(int low, int high) {
        vector<int> ans;
        if (low == 0)
            ans.push_back(0);

        queue<long> q;

        for (int i = 1; i <= 9; ++i)
            q.push(i);

        while (!q.empty()) {
            const long curr = q.front();
            q.pop();
            if (curr > high)
                continue;
            if (curr >= low)
                ans.push_back(curr);
            const int lastDigit = curr % 10;
            if (lastDigit > 0)
                q.push(curr * 10 + lastDigit - 1);
            if (lastDigit < 9)
                q.push(curr * 10 + lastDigit + 1);
        }

        return ans;
    }
};
```


233. 1216.cpp

Path: LeetCode-main/solutions/1216. Valid Palindrome III/1216.cpp

```
class Solution {
public:
    bool isValidPalindrome(string s, int k) {
        return s.length() - longestPalindromeSubseq(s) <= k;
    }

private:
    // Same as 516. Longest Palindromic Subsequence
    int longestPalindromeSubseq(const string& s) {
        const int n = s.length();
        // dp[i][j] := the length of LPS(s[i..j])
        vector<vector<int>>> dp(n, vector<int>(n));

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                if (s[i] == s[j])
                    dp[i][j] = 2 + dp[i + 1][j - 1];
                else
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
            }

        return dp[0][n - 1];
    }
};
```

234. 1217.cpp

Path: LeetCode-main/solutions/1217. Play with Chips/1217.cpp

```
class Solution {  
public:  
    int minCostToMoveChips(vector<int>& position) {  
        vector<int> count(2);  
        for (const int p : position)  
            ++count[p % 2];  
        return min(count[0], count[1]);  
    }  
};
```

235. 1218.cpp

Path: LeetCode-main/solutions/1218. Longest Arithmetic Subsequence of Given Difference/1218.cpp

```
class Solution {
public:
    int longestSubsequence(vector<int>& arr, int difference) {
        int ans = 0;
        unordered_map<int, int> lengthAt;

        for (const int a : arr) {
            if (const auto it = lengthAt.find(a - difference); it != lengthAt.cend())
                lengthAt[a] = it->second + 1;
            else
                lengthAt[a] = 1;
            ans = max(ans, lengthAt[a]);
        }

        return ans;
    }
};
```

236. 1219.cpp

Path: LeetCode-main/solutions/1219. Path with Maximum Gold/1219.cpp

```
class Solution {
public:
    int getMaximumGold(vector<vector<int>>& grid) {
        int ans = 0;

        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                ans = max(ans, dfs(grid, i, j));

        return ans;
    }

private:
    int dfs(vector<vector<int>>& grid, int i, int j) {
        if (i < 0 || j < 0 || i == grid.size() || j == grid[0].size())
            return 0;
        if (grid[i][j] == 0)
            return 0;

        const int gold = grid[i][j];
        grid[i][j] = 0; // Mark as visited.
        const int maxPath = max({dfs(grid, i + 1, j), dfs(grid, i - 1, j),
                                dfs(grid, i, j + 1), dfs(grid, i, j - 1)});
        grid[i][j] = gold;
        return gold + maxPath;
    }
};
```

237. 122.cpp

Path: LeetCode-main/solutions/122. Best Time to Buy and Sell Stock II/122.cpp

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int sell = 0;
        int hold = INT_MIN;

        for (const int price : prices) {
            sell = max(sell, hold + price);
            hold = max(hold, sell - price);
        }

        return sell;
    }
};
```

238. 1221.cpp

Path: LeetCode-main/solutions/1221. Split a String in Balanced Strings/1221.cpp

```
class Solution {
public:
    int balancedStringSplit(string s) {
        int ans = 0;
        int count = 0;

        for (const char c : s) {
            count += c == 'L' ? 1 : -1;
            if (count == 0)
                ++ans;
        }

        return ans;
    }
};
```

239. 1222.cpp

Path: LeetCode-main/solutions/1222. Queens That Can Attack the King/1222.cpp

```
class Solution {
public:
    vector<vector<int>> queensAttacktheKing(vector<vector<int>>& queens,
                                           vector<int>& king) {
        vector<vector<int>> ans;
        unordered_set<int> queensSet;

        for (vector<int>& queen : queens)
            queensSet.insert(hash(queen[0], queen[1]));

        vector<vector<int>> directions = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1},
                                         {0, 1}, {1, -1}, {1, 0}, {1, 1}};
        for (vector<int> d : directions)
            for (int i = king[0] + d[0], j = king[1] + d[1];
                 0 <= i && i < 8 && 0 <= j && j < 8; i += d[0], j += d[1])
                if (queensSet.contains(hash(i, j))) {
                    ans.push_back({i, j});
                    break;
                }

        return ans;
    }

private:
    int hash(int i, int j) {
        return i * 8 + j;
    }
};
```

240. 1224.cpp

Path: LeetCode-main/solutions/1224. Maximum Equal Frequency/1224.cpp

```
class Solution {
public:
    int maxEqualFreq(vector<int>& nums) {
        int ans = 0;
        int maxFreq = 0;
        unordered_map<int, int> count;
        unordered_map<int, int> freq;

        for (int i = 0; i < nums.size(); ++i) {
            const int num = nums[i];
            --freq[count[num]];
            ++count[num];
            ++freq[count[num]];
            maxFreq = max(maxFreq, count[num]);
            if (maxFreq == 1 || maxFreq * freq[maxFreq] == i ||
                (maxFreq - 1) * (freq[maxFreq - 1] + 1) == i)
                ans = i + 1;
        }

        return ans;
    }
};
```


241. 1226.cpp

Path: LeetCode-main/solutions/1226. The Dining Philosophers/1226.cpp

```
class DiningPhilosophers {
    std::mutex mutex;

public:
    void wantsToEat(int philosopher, function<void()> pickLeftFork,
                    function<void()> pickRightFork, function<void()> eat,
                    function<void()> putLeftFork, function<void()> putRightFork) {
        mutex.lock();
        pickLeftFork();
        pickRightFork();
        eat();
        putLeftFork();
        putRightFork();
        mutex.unlock();
    }
};
```

242. 1227.cpp

Path: LeetCode-main/solutions/1227. Airplane Seat Assignment Probability/1227.cpp

```
class Solution {
public:
    double nthPersonGetsNthSeat(int n) {
        return n == 1 ? 1 : 0.5;
    }
};
```

243. 1228.cpp

Path: LeetCode-main/solutions/1228. Missing Number In Arithmetic Progression/1228.cpp

```
class Solution {
public:
    int missingNumber(vector<int>& arr) {
        const int n = arr.size();
        const int delta = (arr.back() - arr.front()) / n;
        int l = 0;
        int r = n - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (arr[m] == arr[0] + m * delta)
                l = m + 1;
            else
                r = m;
        }

        return arr[0] + l * delta;
    }
};
```

244. 1229.cpp

Path: LeetCode-main/solutions/1229. Meeting Scheduler/1229.cpp

```
class Solution {
public:
    vector<int> minAvailableDuration(vector<vector<int>>& slots1,
                                    vector<vector<int>>& slots2, int duration) {
        ranges::sort(slots1);
        ranges::sort(slots2);

        int i = 0; // slots1's index
        int j = 0; // slots2's index

        while (i < slots1.size() && j < slots2.size()) {
            const int start = max(slots1[i][0], slots2[j][0]);
            const int end = min(slots1[i][1], slots2[j][1]);
            if (start + duration <= end)
                return {start, start + duration};
            if (slots1[i][1] < slots2[j][1])
                ++i;
            else
                ++j;
        }
        return {};
    }
};
```

245. 123.cpp

Path: LeetCode-main/solutions/123. Best Time to Buy and Sell Stock III/123.cpp

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int sellTwo = 0;
        int holdTwo = INT_MIN;
        int sellOne = 0;
        int holdOne = INT_MIN;

        for (const int price : prices) {
            sellTwo = max(sellTwo, holdTwo + price);
            holdTwo = max(holdTwo, sellOne - price);
            sellOne = max(sellOne, holdOne + price);
            holdOne = max(holdOne, -price);
        }

        return sellTwo;
    }
};
```

246. 1230-2.cpp

Path: LeetCode-main/solutions/1230. Toss Strange Coins/1230-2.cpp

```
class Solution {
public:
    double probabilityOfHeads(vector<double>& prob, int target) {
        // dp[j] := the probability of tossing the coins so far with j heads
        vector<double> dp(target + 1);
        dp[0] = 1.0;

        for (const double p : prob)
            for (int j = target; j >= 0; --j)
                dp[j] = (j > 0 ? dp[j - 1] * p : 0) + dp[j] * (1 - p);

        return dp[target];
    }
};
```

247. 1230.cpp

Path: LeetCode-main/solutions/1230. Toss Strange Coins/1230.cpp

```
class Solution {
public:
    double probabilityOfHeads(vector<double>& prob, int target) {
        // dp[i][j] := the probability of tossing the first i coins with j heads
        vector<vector<double>> dp(prob.size() + 1, vector<double>(target + 1));
        dp[0][0] = 1.0;

        for (int i = 1; i <= prob.size(); ++i)
            for (int j = 0; j <= target; ++j)
                dp[i][j] = (j > 0 ? dp[i - 1][j - 1] * prob[i - 1] : 0) +
                    dp[i - 1][j] * (1 - prob[i - 1]);

        return dp[prob.size()][target];
    }
};
```

248. 1231.cpp

Path: LeetCode-main/solutions/1231. Divide Chocolate/1231.cpp

```
class Solution {
public:
    int maximizeSweetness(vector<int>& sweetness, int k) {
        int l = sweetness.size() / (k + 1);
        int r = accumulate(sweetness.begin(), sweetness.end(), 0) / (k + 1);

        while (l < r) {
            const int m = (l + r) / 2;
            if (canEat(sweetness, k, m))
                l = m + 1;
            else
                r = m;
        }

        return canEat(sweetness, k, l) ? l : l - 1;
    }

private:
    // Returns true if can eat m sweetness (the minimum sweetness of each piece).
    bool canEat(const vector<int>& sweetness, int k, int m) {
        int pieces = 0;
        int sum = 0; // the running sum

        for (const int s : sweetness) {
            sum += s;
            if (sum >= m) {
                if (++pieces > k)
                    return true;
                sum = 0;
            }
        }

        return false;
    };
};
```


249. 1232.cpp

Path: LeetCode-main/solutions/1232. Check If It Is a Straight Line/1232.cpp

```
class Solution {
public:
    bool checkStraightLine(vector<vector<int>>& coordinates) {
        int x0 = coordinates[0][0];
        int y0 = coordinates[0][1];
        int x1 = coordinates[1][0];
        int y1 = coordinates[1][1];
        int dx = x1 - x0;
        int dy = y1 - y0;

        for (int i = 2; i < coordinates.size(); ++i) {
            int x = coordinates[i][0];
            int y = coordinates[i][1];
            if ((x - x0) * dy != (y - y0) * dx)
                return false;
        }

        return true;
    }
};
```

250. 1233.cpp

Path: LeetCode-main/solutions/1233. Remove Sub-Folders from the Filesystem/1233.cpp

```
class Solution {
public:
    vector<string> removeSubfolders(vector<string>& folder) {
        vector<string> ans;
        string prev;

        ranges::sort(folder);

        for (const string& f : folder) {
            if (!prev.empty() && f.find(prev) == 0 && f[prev.length()] == '/')
                continue;
            ans.push_back(f);
            prev = f;
        }

        return ans;
    }
};
```

251. 1234.cpp

Path: LeetCode-main/solutions/1234. Replace the Substring for Balanced String/1234.cpp

```
class Solution {
public:
    int balancedString(string s) {
        const int n = s.length();
        const int k = n / 4;
        int ans = n;
        vector<int> count(128);

        for (const char c : s)
            ++count[c];

        for (int i = 0, j = 0; i < n; ++i) {
            --count[s[i]];
            while (j < n && count['Q'] <= k && count['W'] <= k && count['E'] <= k &&
                  count['R'] <= k) {
                ans = min(ans, i - j + 1);
                ++count[s[j]];
                ++j;
            }
        }

        return ans;
    }
};
```

252. 1235-2.cpp

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235-2.cpp

```
struct Job {
    int startTime;
    int endTime;
    int profit;
};

class Solution {
public:
    int jobScheduling(vector<int>& startTime, vector<int>& endTime,
                     vector<int>& profit) {
        const int n = startTime.size();
        // dp[i] := the maximum profit to schedule jobs[i..n)
        vector<int> dp(n + 1);
        vector<Job> jobs;

        for (int i = 0; i < n; ++i)
            jobs.emplace_back(startTime[i], endTime[i], profit[i]);

        ranges::sort(jobs, ranges::less{},
                    [](const Job& job) { return job.startTime; });

        for (int i = 0; i < n; ++i)
            startTime[i] = jobs[i].startTime;

        for (int i = n - 1; i >= 0; --i) {
            const int j = firstGreaterEqual(startTime, i + 1, jobs[i].endTime);
            const int pick = jobs[i].profit + dp[j];
            const int skip = dp[i + 1];
            dp[i] = max(pick, skip);
        }

        return dp[0];
    }

    int firstGreaterEqual(const vector<int>& arr, int startFrom, int target) {
        return lower_bound(arr.begin() + startFrom, arr.end(), target) -
            arr.begin();
    }
};
```

253. 1235-3.cpp

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235-3.cpp

```
struct Job {
    int startTime;
    int endTime;
    int profit;
};

class Solution {
public:
    int jobScheduling(vector<int>& startTime, vector<int>& endTime,
                     vector<int>& profit) {
        vector<Job> jobs;

        for (int i = 0; i < startTime.size(); ++i)
            jobs.emplace_back(startTime[i], endTime[i], profit[i]);

        ranges::sort(jobs, ranges::less{},
                     [](const Job& job) { return job.startTime; });

        return getMaxProfit(jobs);
    }

private:
    int getMaxProfit(const vector<Job>& jobs) {
        int maxProfit = 0;
        auto compare = [](const Job& a, const Job& b) {
            return a.endTime > b.endTime;
        };
        priority_queue<Job, vector<Job>, decltype(compare)> minHeap(compare);

        for (const auto& [s, e, p] : jobs) {
            while (!minHeap.empty() && s >= minHeap.top().endTime)
                maxProfit = max(maxProfit, minHeap.top().profit), minHeap.pop();
            minHeap.emplace(s, e, p + maxProfit);
        }

        while (!minHeap.empty())
            maxProfit = max(maxProfit, minHeap.top().profit), minHeap.pop();

        return maxProfit;
    }
};
```

254. 1235.cpp

Path: LeetCode-main/solutions/1235. Maximum Profit in Job Scheduling/1235.cpp

```
struct Job {
    int startTime;
    int endTime;
    int profit;
};

class Solution {
public:
    int jobScheduling(vector<int>& startTime, vector<int>& endTime,
                     vector<int>& profit) {
        const int n = startTime.size();
        vector<int> mem(n + 1);
        vector<Job> jobs;

        for (int i = 0; i < n; ++i)
            jobs.emplace_back(startTime[i], endTime[i], profit[i]);

        ranges::sort(jobs, ranges::less{},
                    [](const Job& job) { return job.startTime; });

        // Will use binary search to find the first available start time.
        for (int i = 0; i < n; ++i)
            startTime[i] = jobs[i].startTime;

        return jobScheduling(jobs, startTime, 0, mem);
    }

private:
    // Returns the maximum profit to schedule jobs[i..n).
    int jobScheduling(const vector<Job>& jobs, const vector<int>& startTime,
                    int i, vector<int>& mem) {
        if (i == jobs.size())
            return 0;
        if (mem[i] > 0)
            return mem[i];

        const int j = firstGreaterEqual(startTime, i + 1, jobs[i].endTime);
        const int pick = jobs[i].profit + jobScheduling(jobs, startTime, j, mem);
        const int skip = jobScheduling(jobs, startTime, i + 1, mem);
        return mem[i] = max(pick, skip);
    }

    int firstGreaterEqual(const vector<int>& arr, int startFrom, int target) {
        return lower_bound(arr.begin() + startFrom, arr.end(), target) -
            arr.begin();
    }
};
```

255. 1236.cpp

Path: LeetCode-main/solutions/1236. Web Crawler/1236.cpp

```
/**
 * // This is the HtmlParser's API interface.
 * // You should not implement it, or speculate about its implementation
 * class HtmlParser {
 * public:
 *     vector<string> getUrls(string url);
 * };
 */

class Solution {
public:
    vector<string> crawl(string startUrl, HtmlParser htmlParser) {
        queue<string> q{{startUrl}};
        unordered_set<string> seen{{startUrl}};
        const string& hostname = getHostname(startUrl);

        while (!q.empty()) {
            const string currUrl = q.front();
            q.pop();
            for (const string& url : htmlParser.getUrls(currUrl)) {
                if (seen.contains(url))
                    continue;
                if (url.find(hostname) != string::npos) {
                    q.push(url);
                    seen.insert(url);
                }
            }
        }

        return {seen.begin(), seen.end()};
    }

private:
    string getHostname(const string& url) {
        const int firstSlash = url.find_first_of('/');
        const int thirdSlash = url.find_first_of('/', firstSlash + 2);
        return url.substr(firstSlash + 2, thirdSlash - firstSlash - 2);
    }
};
```

256. 1237.cpp

Path: LeetCode-main/solutions/1237. Find Positive Integer Solution for a Given Equation/1237.cpp

```
class Solution {
public:
    vector<vector<int>> findSolution(CustomFunction& customfunction, int z) {
        vector<vector<int>> ans;
        int x = 1;
        int y = 1000;

        while (x <= 1000 && y >= 1) {
            int f = customfunction.f(x, y);
            if (f < z)
                ++x;
            else if (f > z)
                --y;
            else
                ans.push_back({x++, y--});
        }

        return ans;
    }
};
```


257. 1238.cpp

Path: LeetCode-main/solutions/1238. Circular Permutation in Binary Representation/1238.cpp

```
class Solution {
public:
    vector<int> circularPermutation(int n, int start) {
        vector<int> ans;

        for (int i = 0; i < 1 << n; ++i)
            ans.push_back(start ^ i ^ i >> 1);

        return ans;
    }
};
```

258. 1239-2.cpp

Path: LeetCode-main/solutions/1239. Maximum Length of a Concatenated String with Unique Characters/1239-2.cpp

```
class Solution {
public:
    int maxLength(vector<string>& arr) {
        vector<bitset<26>> masks;

        for (const string& s : arr) {
            const bitset<26> mask = getMask(s);
            if (mask.contains() == s.length())
                masks.push_back(mask);
        }

        return dfs(masks, 0, /*used=*/bitset<26>());
    }

private:
    int dfs(const vector<bitset<26>>& masks, int s, bitset<26> used) {
        int res = used.contains();
        for (int i = s; i < masks.size(); ++i)
            if (!(used & masks[i]).any())
                res = max(res, dfs(masks, i + 1, used | masks[i]));
        return res;
    }

    bitset<26> getMask(const string& s) {
        bitset<26> mask;
        for (const char c : s)
            mask.set(c - 'a');
        return mask;
    }
};
```

259. 1239-3.cpp

Path: LeetCode-main/solutions/1239. Maximum Length of a Concatenated String with Unique Characters/1239-3.cpp

```
class Solution {
public:
    int maxLength(vector<string>& arr) {
        vector<unsigned> masks;

        for (const string& s : arr) {
            const int mask = getMask(s);
            if (mask != -1)
                masks.push_back(mask);
        }

        return dfs(masks, 0, /*used=*/0);
    }

private:
    int dfs(const vector<unsigned>& masks, int i, int used) {
        if (i == masks.size())
            return 0;
        const int pick =
            (masks[i] & used) == 0
            ? popcount(masks[i]) + dfs(masks, i + 1, used | masks[i])
            : 0;
        const int skip = dfs(masks, i + 1, used);
        return max(pick, skip);
    }

    int getMask(const string& s) {
        int mask = 0;
        for (const char c : s) {
            const int i = c - 'a';
            if ((mask & (1 << i)) != 0)
                return -1;
            mask |= 1 << i;
        }
        return mask;
    }
};
```

260. 1239.cpp

Path: LeetCode-main/solutions/1239. Maximum Length of a Concatenated String with Unique Characters/1239.cpp

```
class Solution {
public:
    int maxLength(vector<string>& arr) {
        vector<int> masks;

        for (const string& s : arr) {
            const int mask = getMask(s);
            if (mask != -1)
                masks.push_back(mask);
        }

        return dfs(masks, 0, /*used=*/0);
    }

private:
    int dfs(const vector<int>& masks, int s, unsigned used) {
        int res = popcount(used);
        for (int i = s; i < masks.size(); ++i)
            if ((used & masks[i]) == 0)
                res = max(res, dfs(masks, i + 1, used | masks[i]));
        return res;
    }

    int getMask(const string& s) {
        int mask = 0;
        for (const char c : s) {
            const int i = c - 'a';
            if ((mask & (1 << i)) != 0)
                return -1;
            mask |= 1 << i;
        }
        return mask;
    }
};
```

261. 124.cpp

Path: LeetCode-main/solutions/124. Binary Tree Maximum Path Sum/124.cpp

```
class Solution {
public:
    int maxPathSum(TreeNode* root) {
        int ans = INT_MIN;
        maxPathSumDownFrom(root, ans);
        return ans;
    }

private:
    // Returns the maximum path sum starting from the current root, where
    // root->val is always included.
    int maxPathSumDownFrom(TreeNode* root, int& ans) {
        if (root == nullptr)
            return 0;

        const int l = max(0, maxPathSumDownFrom(root->left, ans));
        const int r = max(0, maxPathSumDownFrom(root->right, ans));
        ans = max(ans, root->val + l + r);
        return root->val + max(l, r);
    }
};
```

262. 1240.cpp

Path: LeetCode-main/solutions/1240. Tiling a Rectangle with the Fewest Squares/1240.cpp

```
class Solution {
public:
    int tilingRectangle(int n, int m) {
        unordered_map<long, int> mem;
        return tilingRectangle(n, m, 0, /*heights=*/vector<int>(m), mem);
    }

private:
    static constexpr int kBase = 13;

    int tilingRectangle(int n, int m, long hashedHeights, vector<int>&& heights,
        unordered_map<long, int>& mem) {
        if (const auto it = mem.find(hashedHeights); it != mem.cend())
            return it->second;

        const auto it = ranges::min_element(heights);
        const int minHeight = *it;
        if (minHeight == n) // All filled.
            return 0;

        int ans = m * n;
        const int start = it - heights.begin();
        // Try to put square of different size that doesn't exceed the width/height.
        for (int sz = 1; sz <= min(m - start, n - minHeight); ++sz) {
            // heights[start..start + sz) must have the same height.
            if (heights[start + sz - 1] != minHeight)
                break;
            // Put a square of size `sz` to cover heights[start..start + sz).
            for (int i = start; i < start + sz; ++i)
                heights[i] += sz;
            ans = min(ans,
                tilingRectangle(n, m, hash(heights), std::move(heights), mem));
            for (int i = start; i < start + sz; ++i)
                heights[i] -= sz;
        }

        return mem[hashedHeights] = 1 + ans;
    }

    long hash(const vector<int>& heights) {
        long hashed = 0;
        for (int i = heights.size() - 1; i >= 0; --i)
            hashed = hashed * kBase + heights[i];
        return hashed;
    }
};
```

263. 1242.cpp

Path: LeetCode-main/solutions/1242. Web Crawler Multithreaded/1242.cpp

```
/**
 * // This is the HtmlParser's API interface.
 * // You should not implement it, or speculate about its implementation
 * class HtmlParser {
 * public:
 *     vector<string> getUrls(string url);
 * };
 */

class Solution {
public:
    vector<string> crawl(string startUrl, HtmlParser htmlParser) {
        queue<string> q{{startUrl}};
        unordered_set<string> seen{{startUrl}};
        const string& hostname = getHostname(startUrl);
        const int nThreads = std::thread::hardware_concurrency();
        vector<thread> threads;
        std::mutex mtx;
        std::condition_variable cv;

        auto t = [&]() {
            while (true) {
                unique_lock<mutex> lock(mtx);
                cv.wait_for(lock, 30ms, [&]() { return q.size(); });
                if (q.empty())
                    return;
                auto cur = q.front();
                q.pop();
                lock.unlock();
                const vector<string> urls = htmlParser.getUrls(cur);
                lock.lock();
                for (const string& url : urls) {
                    if (seen.contains(url))
                        continue;
                    if (url.find(hostname) != string::npos) {
                        q.push(url);
                        seen.insert(url);
                    }
                }
                lock.unlock();
                cv.notify_all();
            }
        };

        for (int i = 0; i < nThreads; ++i)
            threads.emplace_back(t);

        for (std::thread& t : threads)
            t.join();

        return {seen.begin(), seen.end()};
    }

private:
    string getHostname(const string& url) {
        const int firstSlash = url.find_first_of('/');
        const int thirdSlash = url.find_first_of('/', firstSlash + 2);
        return url.substr(firstSlash + 2, thirdSlash - firstSlash - 2);
    }
};
```

264. 1243.cpp

Path: LeetCode-main/solutions/1243. Array Transformation/1243.cpp

```
class Solution {
public:
    vector<int> transformArray(vector<int>& arr) {
        if (arr.size() < 3)
            return arr;

        vector<int> ans(arr.size());

        while (ans != arr) {
            ans = arr;
            for (int i = 1; i < arr.size() - 1; ++i) {
                if (ans[i - 1] > ans[i] && ans[i] < ans[i + 1])
                    ++arr[i];
                else if (ans[i - 1] < ans[i] && ans[i] > ans[i + 1])
                    --arr[i];
            }
        }
        return ans;
    }
};
```


265. 1244.cpp

Path: LeetCode-main/solutions/1244. Design A Leaderboard/1244.cpp

```
class Leaderboard {
public:
    void addScore(int playerId, int score) {
        idToScore[playerId] += score;
    }

    int top(int K) {
        int ans = 0;
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const auto& [id, score] : idToScore) {
            minHeap.push(score);
            if (minHeap.size() > K)
                minHeap.pop();
        }

        while (!minHeap.empty())
            ans += minHeap.top(), minHeap.pop();

        return ans;
    }

    void reset(int playerId) {
        idToScore.erase(playerId);
    }

private:
    unordered_map<int, int> idToScore;
};
```

266. 1245.cpp

Path: LeetCode-main/solutions/1245. Tree Diameter/1245.cpp

```
class Solution {
public:
    int treeDiameter(vector<vector<int>>& edges) {
        const int n = edges.size();
        int ans = 0;
        vector<vector<int>> tree(n + 1);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        maxDepth(tree, 0, -1, ans);
        return ans;
    }

private:
    int maxDepth(const vector<vector<int>>& tree, int u, int prev, int& ans) {
        int maxDepth1 = 0; // the maximum depth
        int maxDepth2 = -1; // the second maximum depth

        for (const int v : tree[u]) {
            if (v == prev)
                continue;
            const int depth = maxDepth(tree, v, u, ans);
            if (depth > maxDepth1) {
                maxDepth2 = maxDepth1;
                maxDepth1 = depth;
            } else if (depth > maxDepth2) {
                maxDepth2 = depth;
            }
        }

        ans = max(ans, maxDepth1 + maxDepth2);
        return 1 + maxDepth1;
    }
};
```

267. 1246.cpp

Path: LeetCode-main/solutions/1246. Palindrome Removal/1246.cpp

```
class Solution {
public:
    int minimumMoves(vector<int>& arr) {
        const int n = arr.size();
        // dp[i][j] := the minimum number of moves to remove all numbers from
        // arr[i..j]
        vector<vector<int>> dp(n, vector<int>(n, n));

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int i = 0; i + 1 < n; ++i)
            dp[i][i + 1] = arr[i] == arr[i + 1] ? 1 : 2;

        for (int d = 2; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                // Remove arr[i] and arr[j] within the move of removing
                // arr[i + 1..j - 1]
                if (arr[i] == arr[j])
                    dp[i][j] = dp[i + 1][j - 1];
                // Try all the possible partitions.
                for (int k = i; k < j; ++k)
                    dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j]);
            }

        return dp[0][n - 1];
    }
};
```

268. 1247.cpp

Path: LeetCode-main/solutions/1247. Minimum Swaps to Make Strings Equal/1247.cpp

```
class Solution {
public:
    int minimumSwap(string s1, string s2) {
        // ("xx", "yy") = (2 "xy"s) -> 1 swap
        // ("yy", "xx") = (2 "yx"s) -> 1 swap
        // ("xy", "yx") = (1 "xy" and 1 "yx") -> 2 swaps
        int xy = 0; // the number of indices i's s.t. s1[i] = 'x' and s2[i] 'y'
        int yx = 0; // the number of indices i's s.t. s1[i] = 'y' and s2[i] 'x'

        for (int i = 0; i < s1.length(); ++i) {
            if (s1[i] == s2[i])
                continue;
            if (s1[i] == 'x')
                ++xy;
            else
                ++yx;
        }

        if ((xy + yx) % 2 == 1)
            return -1;
        return xy / 2 + yx / 2 + (xy % 2 == 0 ? 0 : 2);
    }
};
```

269. 1248.cpp

Path: LeetCode-main/solutions/1248. Count Number of Nice Subarrays/1248.cpp

```
class Solution {
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        return numberOfSubarraysAtMost(nums, k) -
            numberOfSubarraysAtMost(nums, k - 1);
    }

private:
    int numberOfSubarraysAtMost(vector<int>& nums, int k) {
        int ans = 0;

        for (int l = 0, r = 0; r <= nums.size(); )
            if (k >= 0) {
                ans += r - l;
                if (r == nums.size())
                    break;
                if (nums[r] % 2 == 1)
                    --k;
                ++r;
            } else {
                if (nums[l] % 2 == 1)
                    ++k;
                ++l;
            }

        return ans;
    }
};
```

270. 1249.cpp

Path: LeetCode-main/solutions/1249. Minimum Remove to Make Valid Parentheses/1249.cpp

```
class Solution {
public:
    string minRemoveToMakeValid(string s) {
        stack<int> stack; // unpaired '(' indices

        for (int i = 0; i < s.length(); ++i)
            if (s[i] == '(') {
                stack.push(i); // Record the unpaired '(' index.
            } else if (s[i] == ')') {
                if (stack.empty())
                    s[i] = '*'; // Mark the unpaired ')' as '*'.
                else
                    stack.pop(); // Find a pair!
            }

        // Mark the unpaired '(' as '*'.
        while (!stack.empty())
            s[stack.top()] = '*', stack.pop();

        std::erase(s, '*');
        return s;
    }
};
```

271. 125.cpp

Path: LeetCode-main/solutions/125. Valid Palindrome/125.cpp

```
class Solution {
public:
    bool isPalindrome(string s) {
        int l = 0;
        int r = s.length() - 1;

        while (l < r) {
            while (l < r && !isalnum(s[l]))
                ++l;
            while (l < r && !isalnum(s[r]))
                --r;
            if (tolower(s[l]) != tolower(s[r]))
                return false;
            ++l;
            --r;
        }

        return true;
    }
};
```

272. 1250.cpp

Path: LeetCode-main/solutions/1250. Check If It Is a Good Array/1250.cpp

```
class Solution {
public:
    bool isGoodArray(vector<int>& nums) {
        int g = nums[0];

        for (const int num : nums)
            g = __gcd(g, num);

        return g == 1;
    }
};
```


273. 1252-2.cpp

Path: LeetCode-main/solutions/1252. Cells with Odd Values in a Matrix/1252-2.cpp

```
class Solution {
public:
    int oddCells(int m, int n, vector<vector<int>>& indices) {
        // rows[i] and cols[i] :=
        // 1. true (flipped even times)
        // 2. false (flipped odd times)
        vector<bool> rows(m);
        vector<bool> cols(n);

        for (const vector<int>& index : indices) {
            rows[index[0]] = rows[index[0]] ^ true;
            cols[index[1]] = cols[index[1]] ^ true;
        }

        const int oddRowCount = ranges::count(rows, true);
        const int oddColCount = ranges::count(cols, true);
        const int evenRowCount = m - oddRowCount;
        const int evenColCount = n - oddColCount;
        return oddRowCount * evenColCount + oddColCount * evenRowCount;
    }
};
```

274. 1252-3.cpp

Path: LeetCode-main/solutions/1252. Cells with Odd Values in a Matrix/1252-3.cpp

```
class Solution {
public:
    int oddCells(int m, int n, vector<vector<int>>& indices) {
        unordered_set<int> oddRows;
        unordered_set<int> oddCols;

        for (const vector<int>& index : indices) {
            const int r = index[0];
            const int c = index[1];
            if (!oddRows.insert(r).second)
                oddRows.erase(r);
            if (!oddCols.insert(c).second)
                oddCols.erase(c);
        }

        return oddRows.size() * (n - oddCols.size()) +
               oddCols.size() * (m - oddRows.size());
    }
};
```

275. 1252.cpp

Path: LeetCode-main/solutions/1252. Cells with Odd Values in a Matrix/1252.cpp

```
class Solution {
public:
    int oddCells(int m, int n, vector<vector<int>>& indices) {
        int ans = 0;
        // rows[i] and cols[i] :=
        // true (flipped even times) / false (flipped odd times)
        vector<bool> rows(m);
        vector<bool> cols(n);

        for (const vector<int>& index : indices) {
            rows[index[0]] = rows[index[0]] ^ true;
            cols[index[1]] = cols[index[1]] ^ true;
        }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                ans += rows[i] ^ cols[j];

        return ans;
    }
};
```

276. 1253.cpp

Path: LeetCode-main/solutions/1253. Reconstruct a 2-Row Binary Matrix/1253.cpp

```
class Solution {
public:
    vector<vector<int>> reconstructMatrix(int upper, int lower,
                                         vector<int>& colsum) {
        if (upper + lower != accumulate(colsum.begin(), colsum.end(), 0))
            return {};
        if (min(upper, lower) <
            ranges::count_if(colsum, [](int c) { return c == 2; }))
            return {};

        vector<vector<int>> ans(2, vector<int>(colsum.size()));

        for (int j = 0; j < colsum.size(); ++j)
            if (colsum[j] == 2) {
                ans[0][j] = 1;
                ans[1][j] = 1;
                --upper;
                --lower;
            }

        for (int j = 0; j < colsum.size(); ++j) {
            if (colsum[j] == 1 && upper > 0) {
                ans[0][j] = 1;
                --colsum[j];
                --upper;
            }

            if (colsum[j] == 1 && lower > 0) {
                ans[1][j] = 1;
                --lower;
            }
        }

        return ans;
    }
};
```

277. 1254.cpp

Path: LeetCode-main/solutions/1254. Number of Closed Islands/1254.cpp

```
class Solution {
public:
    int closedIsland(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();

        // Remove the lands connected to the edge.
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    if (grid[i][j] == 0)
                        dfs(grid, i, j);

        int ans = 0;

        // Reduce to 200. Number of Islands
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 0) {
                    dfs(grid, i, j);
                    ++ans;
                }

        return ans;
    }

private:
    void dfs(vector<vector<int>>& grid, int i, int j) {
        if (i < 0 || i == grid.size() || j < 0 || j == grid[0].size())
            return;
        if (grid[i][j] == 1)
            return;
        grid[i][j] = 1;
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    };
};
```

278. 1255.cpp

Path: LeetCode-main/solutions/1255. Maximum Score Words Formed by Letters/1255.cpp

```
class Solution {
public:
    int maxScoreWords(vector<string>& words, vector<char>& letters,
                     vector<int>& score) {
        vector<int> count(26);
        for (const char c : letters)
            ++count[c - 'a'];
        return dfs(words, 0, count, score);
    }

private:
    // Returns the maximum score you can get from words[s..n).
    int dfs(const vector<string>& words, int s, vector<int>& count,
           const vector<int>& score) {
        int ans = 0;
        for (int i = s; i < words.size(); ++i) {
            const int earned = useWord(words, i, count, score);
            if (earned > 0)
                ans = max(ans, earned + dfs(words, i + 1, count, score));
            unuseWord(words, i, count);
        }
        return ans;
    }

    int useWord(const vector<string>& words, int i, vector<int>& count,
               const vector<int>& score) {
        bool isValid = true;
        int earned = 0;
        for (const char c : words[i]) {
            if (--count[c - 'a'] < 0)
                isValid = false;
            earned += score[c - 'a'];
        }
        return isValid ? earned : -1;
    }

    void unuseWord(const vector<string>& words, int i, vector<int>& count) {
        for (const char c : words[i])
            ++count[c - 'a'];
    }
};
```

279. 1256.cpp

Path: LeetCode-main/solutions/1256. Encode Number/1256.cpp

```
class Solution {
public:
    string encode(int num) {
        string binaryStr = bitset<30>(num + 1).to_string();
        binaryStr.erase(0, binaryStr.find_first_not_of('0'));
        return binaryStr.substr(1);
    }
};
```

280. 1257.cpp

Path: LeetCode-main/solutions/1257. Smallest Common Region/1257.cpp

```
class Solution {
public:
    string findSmallestRegion(vector<vector<string>>& regions, string region1,
                             string region2) {
        unordered_map<string, string> parent;
        unordered_set<string> ancestors; // region1's ancestors

        for (const vector<string>& region : regions)
            for (int i = 1; i < region.size(); ++i)
                parent[region[i]] = region[0];

        // Add all the region1's ancestors.
        while (region1 != "") {
            ancestors.insert(region1);
            region1 = parent[region1]; // Region1 becomes "" in the end
        }

        // Go up from region2 until meet any of region1's ancestors.
        while (!ancestors.contains(region2))
            region2 = parent[region2];

        return region2;
    }
};
```


281. 1258.cpp

Path: LeetCode-main/solutions/1258. Synonymous Sentences/1258.cpp

```
class Solution {
public:
    vector<string> generateSentences(vector<vector<string>>& synonyms,
                                    string text) {
        set<string> ans;
        unordered_map<string, vector<string>> graph;
        queue<string> q{{text}};

        for (const vector<string>& synonym : synonyms) {
            const string& s = synonym[0];
            const string& t = synonym[1];
            graph[s].push_back(t);
            graph[t].push_back(s);
        }

        while (!q.empty()) {
            const string u = q.front();
            q.pop();
            ans.insert(u);
            vector<string> words = split(u);
            for (string& word : words) {
                const auto it = graph.find(word);
                if (it == graph.cend())
                    continue;
                for (const string& synonym : it->second) {
                    // Replace words[i] with its synonym.
                    word = synonym;
                    const string newText = join(words, ' ');
                    if (!ans.contains(newText))
                        q.push(newText);
                }
            }
        }

        return {ans.begin(), ans.end()};
    }

private:
    vector<string> split(const string& s) {
        vector<string> words;
        istringstream iss(s);
        for (string token; iss >> token;)
            words.push_back(token);
        return words;
    }

    string join(const vector<string>& words, char c) {
        string joined;
        for (int i = 0; i < words.size(); ++i) {
            joined += words[i];
            if (i != words.size() - 1)
                joined += c;
        }
        return joined;
    }
};
```

282. 1259.cpp

Path: LeetCode-main/solutions/1259. Handshakes That Don't Cross/1259.cpp

```
class Solution {
public:
    int numberOfWays(int numPeople) {
        constexpr int kMod = 1'000'000'007;
        // dp[i] := the number of ways i handshakes could occur s.t. none of the
        // handshakes cross
        vector<long> dp(numPeople / 2 + 1);
        dp[0] = 1;

        for (int i = 1; i <= numPeople / 2; ++i)
            for (int j = 0; j < i; ++j) {
                dp[i] += dp[j] * dp[i - 1 - j];
                dp[i] %= kMod;
            }

        return dp[numPeople / 2];
    }
};
```

283. 126-2.cpp

Path: LeetCode-main/solutions/126. Word Ladder II/126-2.cpp

```
class Solution {
public:
    vector<vector<string>> findLadders(string beginWord, string endWord,
                                     vector<string>& wordList) {
        unordered_set<string> wordSet{wordList.begin(), wordList.end()};
        if (!wordSet.contains(endWord))
            return {};

        // {"hit": ["hot"], "hot": ["dot", "lot"], ...}
        unordered_map<string, vector<string>> graph;

        // Build the graph from the beginWord to the endWord.
        if (!bfs(beginWord, endWord, wordSet, graph))
            return {};

        vector<vector<string>> ans;
        dfs(graph, beginWord, endWord, {beginWord}, ans);
        return ans;
    }

private:
    bool bfs(const string& beginWord, const string& endWord,
             unordered_set<string>& wordSet,
             unordered_map<string, vector<string>>& graph) {
        unordered_set<string> forwardWords{beginWord};
        unordered_set<string> backwardWords{endWord};
        bool backward = false;

        while (!forwardWords.empty() && !backwardWords.empty()) {
            for (const string& word : forwardWords)
                wordSet.erase(word);
            for (const string& word : backwardWords)
                wordSet.erase(word);
            // Always expand the smaller queue.
            if (forwardWords.size() > backwardWords.size()) {
                swap(forwardWords, backwardWords);
                backward = !backward;
            }
            unordered_set<string> nextLevelWords;
            bool reachEndWord = false;
            for (const string& parent : forwardWords) {
                for (const string& child :
                     getChildren(parent, wordSet, backwardWords)) {
                    // Should check if `child` is in `backwardWords` since we erase them
                    // at the beginning of each while loop.
                    if (wordSet.contains(child) || backwardWords.contains(child)) {
                        nextLevelWords.insert(child);
                        if (backward)
                            graph[child].push_back(parent);
                        else
                            graph[parent].push_back(child);
                    }
                    // We've reached the end word since there's a word in the
                    // `forwardWords` connecting to a word in `backwardWords`. Note that
                    // we can't return here since we need to completely explore this
                    // level.
                    if (backwardWords.contains(child))
                        reachEndWord = true;
                }
            }
            if (reachEndWord)
                return true;
            forwardWords = std::move(nextLevelWords);
        }

        return true;
    }

    vector<string> getChildren(const string& parent,
                             const unordered_set<string>& wordSet,
                             const unordered_set<string>& backwardWords) {
        vector<string> children;
        string s(parent);

        for (int i = 0; i < s.length(); ++i) {
            const char cache = s[i];
            for (char c = 'a'; c <= 'z'; ++c) {
                if (c == cache)
                    continue;
                s[i] = c; // Now is `child`
            }
        }
    }
};
```

```

        if (wordSet.contains(s) || backwardWords.contains(s))
            children.push_back(s);
    }
    s[i] = cache;
}

return children;
}

void dfs(const unordered_map<string, vector<string>>& graph,
        const string& word, const string& endWord, vector<string>&& path,
        vector<vector<string>>& ans) {
    if (word == endWord) {
        ans.push_back(path);
        return;
    }
    if (!graph.contains(word))
        return;

    for (const string& child : graph.at(word)) {
        path.push_back(child);
        dfs(graph, child, endWord, std::move(path), ans);
        path.pop_back();
    }
}
};

```

284. 126-3.cpp

Path: LeetCode-main/solutions/126. Word Ladder II/126-3.cpp

```
class Solution {
public:
    vector<vector<string>> findLadders(string beginWord, string endWord,
                                     vector<string>& wordList) {
        vector<vector<string>> ans;
        unordered_set<string> wordSet(wordList.begin(), wordList.end());
        queue<vector<string>> paths{{{beginWord}}}; // {"hit"}

        while (!paths.empty()) {
            unordered_set<string> currentLevelVisited;
            for (int sz = paths.size(); sz > 0; --sz) {
                vector<string> path = paths.front();
                paths.pop(); // {"hit"}
                string lastWord = path.back(); // "hit"
                for (int i = 0; i < lastWord.length(); ++i) {
                    char cache = lastWord[i]; // Cache = 'i'
                    for (char c = 'a'; c <= 'z'; ++c) {
                        lastWord[i] = c; // "hit" -> "hot" (temporarily)
                        if (wordSet.contains(lastWord)) { // Find "hot" in the wordSet.
                            currentLevelVisited.insert(lastWord); // Mark "hot" as visited.
                            vector<string> nextPath(path);
                            nextPath.push_back(lastWord); // nextPath = {"hit", "hot"}
                            if (lastWord == endWord)
                                ans.push_back(nextPath);
                            else
                                paths.push(nextPath);
                        }
                    }
                    lastWord[i] = cache; // Convert "hot" back to "hit".
                }
            }
            for (const string& word : currentLevelVisited)
                wordSet.erase(word);
        }

        return ans;
    }
};
```

285. 126-4.cpp

Path: LeetCode-main/solutions/126. Word Ladder II/126-4.cpp

```
class Solution {
public:
    vector<vector<string>> findLadders(string beginWord, string endWord,
                                     vector<string>& wordList) {
        unordered_set<string> wordSet{wordList.begin(), wordList.end()};
        if (!wordSet.contains(endWord))
            return {};

        // Build the adjacency list.
        // e.g. {"hit": ["hot"], "hot": ["dot", "lot"], ...}
        if (!wordSet.contains(beginWord))
            wordList.push_back(beginWord);
        unordered_map<string, vector<string>> graph = getGraph(wordList);

        // childToParents[child] := the fastest parents to reach `child`
        unordered_map<string, vector<string>> childToParents;
        if (!bfs(beginWord, endWord, graph, childToParents))
            return {};

        vector<vector<string>> ans;
        // Find the paths from the endWord to the beginWord.
        dfs(childToParents, endWord, beginWord, {endWord}, ans);
        return ans;
    }

private:
    unordered_map<string, vector<string>> getGraph(
        const vector<string>& wordList) {
        unordered_map<string, vector<string>> graph;
        for (int i = 1; i < wordList.size(); ++i)
            for (int j = 0; j < i; ++j) {
                const string& u = wordList[i];
                const string& v = wordList[j];
                if (isConnect(u, v)) {
                    graph[u].push_back(v);
                    graph[v].push_back(u);
                }
            }
        return graph;
    }

    bool isConnect(const string& s, const string& t) {
        int count = 0;
        for (int i = 0; i < s.length(); ++i)
            if (s[i] != t[i])
                ++count;
        return count == 1;
    }

    bool bfs(const string& beginWord, const string& endWord,
            const unordered_map<string, vector<string>>& graph,
            unordered_map<string, vector<string>>& childToParents) {
        // dist[u] := the minimum distance to reach `u`
        unordered_map<string, int> dist{{beginWord, 0}};
        queue<string> q{{beginWord}};
        while (!q.empty()) {
            const string u = q.front();
            q.pop();
            if (u == endWord)
                return true;
            if (const auto it = graph.find(u); it != graph.cend())
                for (const string& v : it->second)
                    if (!dist.contains(v) || dist[u] + 1 < dist[v]) {
                        dist[v] = dist[u] + 1;
                        q.push(v);
                        // Clear childToParents[v] since they take longer to reach v.
                        childToParents[v] = {u};
                    } else if (dist[u] + 1 == dist[v]) {
                        // No need to q.push(v) since v is already in the queue.
                        childToParents[v].push_back(u);
                    }
        }
        return false;
    }

    void dfs(const unordered_map<string, vector<string>>& childToParents,
            const string& word, const string& beginWord, vector<string>&& path,
            vector<vector<string>>& ans) {
        if (word == beginWord) {
            ans.push_back({path.rbegin(), path.rend()});
        }
    }
}
```

```
        return;
    }
    if (const auto it = childToParents.find(word); it != childToParents.cend())
        for (const string& parent : it->second) {
            path.push_back(parent);
            dfs(childToParents, parent, beginWord, std::move(path), ans);
            path.pop_back();
        }
    }
};
```

286. 126-5.cpp

Path: LeetCode-main/solutions/126. Word Ladder II/126-5.cpp

```
class Solution {
public:
    vector<vector<string>> findLadders(string beginWord, string endWord,
                                     vector<string>& wordList) {
        unordered_set<string> wordSet{wordList.begin(), wordList.end()};
        if (!wordSet.contains(endWord))
            return {};

        unordered_map<string, int> distFromBeginWord{{beginWord, 0}};
        if (!bfs(beginWord, endWord, wordSet, distFromBeginWord))
            return {};

        vector<vector<string>> ans;
        wordSet.insert(beginWord);
        dfs(endWord, beginWord, distFromBeginWord, wordSet, {endWord}, ans);
        return ans;
    }

private:
    // Uses BFS to update the minimum steps to reach `endWord` from `beginWord` by
    // using the words in `wordSet`.
    bool bfs(const string& beginWord, const string& endWord,
             unordered_set<string>& wordSet,
             unordered_map<string, int>& distFromBeginWord) {
        queue<string> q{{beginWord}};
        while (!q.empty()) {
            for (int sz = q.size(); sz > 0; --sz) {
                const string parent = q.front();
                q.pop();
                if (parent == endWord)
                    return true;
                for (const string& child : getChildren(parent, wordSet)) {
                    if (distFromBeginWord.contains(child))
                        continue;
                    distFromBeginWord[child] = distFromBeginWord[parent] + 1;
                    q.push(child);
                }
            }
        }
        return false;
    }

    void dfs(const string& word, const string& beginWord,
            const unordered_map<string, int>& distFromBeginWord,
            const unordered_set<string>& wordSet, vector<string>&& path,
            vector<vector<string>>& ans) {
        if (word == beginWord) {
            ans.push_back({path.rbegin(), path.rend()});
            return;
        }

        const int currDist = distFromBeginWord.at(word);

        for (const string& child : getChildren(word, wordSet)) {
            if (const auto it = distFromBeginWord.find(child);
                it != distFromBeginWord.cend() && it->second == currDist - 1) {
                path.push_back(child);
                dfs(child, beginWord, distFromBeginWord, wordSet, std::move(path), ans);
                path.pop_back();
            }
        }
    }

    vector<string> getChildren(const string& parent,
                             const unordered_set<string>& wordSet) {
        vector<string> children;
        string child(parent);
        for (int i = 0; i < child.length(); ++i) {
            const char cache = child[i];
            for (char c = 'a'; c <= 'z'; ++c) {
                if (c == cache)
                    continue;
                child[i] = c;
                if (wordSet.contains(child))
                    children.push_back(child);
            }
            child[i] = cache;
        }
        return children;
    }
}
```


287. 126.cpp

Path: LeetCode-main/solutions/126. Word Ladder II/126.cpp

```
class Solution {
public:
    vector<vector<string>> findLadders(string beginWord, string endWord,
                                     vector<string>& wordList) {
        unordered_set<string> wordSet{wordList.begin(), wordList.end()};
        if (!wordSet.contains(endWord))
            return {};

        // {"hit": ["hot"], "hot": ["dot", "lot"], ...}
        unordered_map<string, vector<string>> graph;

        // Build the graph from the beginWord to the endWord.
        if (!bfs(beginWord, endWord, wordSet, graph))
            return {};

        vector<vector<string>> ans;
        dfs(graph, beginWord, endWord, {beginWord}, ans);
        return ans;
    }

private:
    bool bfs(const string& beginWord, const string& endWord,
             unordered_set<string>& wordSet,
             unordered_map<string, vector<string>>& graph) {
        unordered_set<string> currentLevelWords{beginWord};

        while (!currentLevelWords.empty()) {
            for (const string& word : currentLevelWords)
                wordSet.erase(word);
            unordered_set<string> nextLevelWords;
            bool reachEndWord = false;
            for (const string& parent : currentLevelWords) {
                vector<string> children;
                getChildren(parent, wordSet, children);
                for (const string& child : children) {
                    if (wordSet.contains(child)) {
                        nextLevelWords.insert(child);
                        graph[parent].push_back(child);
                    }
                    if (child == endWord)
                        reachEndWord = true;
                }
            }
            if (reachEndWord)
                return true;
            currentLevelWords = std::move(nextLevelWords);
        }

        return false;
    }

    void getChildren(const string& parent, const unordered_set<string>& wordSet,
                   vector<string>& children) {
        string s(parent);

        for (int i = 0; i < s.length(); ++i) {
            const char cache = s[i];
            for (char c = 'a'; c <= 'z'; ++c) {
                if (c == cache)
                    continue;
                s[i] = c;
                if (wordSet.contains(s))
                    children.push_back(s);
            }
            s[i] = cache;
        }
    }

    void dfs(const unordered_map<string, vector<string>>& graph,
            const string& word, const string& endWord, vector<string>&& path,
            vector<vector<string>>& ans) {
        if (word == endWord) {
            ans.push_back(path);
            return;
        }
        if (!graph.contains(word))
            return;

        for (const string& child : graph.at(word)) {
            path.push_back(child);
        }
    }
}
```

```
        dfs(graph, child, endWord, std::move(path), ans);
        path.pop_back();
    }
};
```

288. 1260.cpp

Path: LeetCode-main/solutions/1260. Shift 2D Grid/1260.cpp

```
class Solution {
public:
    vector<vector<int>> shiftGrid(vector<vector<int>>& grid, int k) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<int>> ans(m, vector<int>(n));

        k %= m * n;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                const int index = (i * n + j + k) % (m * n);
                const int x = index / n;
                const int y = index % n;
                ans[x][y] = grid[i][j];
            }

        return ans;
    }
};
```

289. 1261.cpp

Path: LeetCode-main/solutions/1261. Find Elements in a Contaminated Binary Tree/1261.cpp

```
class FindElements {
public:
    FindElements(TreeNode* root) {
        dfs(root, 0);
    }

    bool find(int target) {
        return vals.contains(target);
    }

private:
    unordered_set<int> vals;

    void dfs(TreeNode* root, int val) {
        if (root == nullptr)
            return;

        root->val = val;
        vals.insert(val);
        dfs(root->left, val * 2 + 1);
        dfs(root->right, val * 2 + 2);
    }
};
```

290. 1262.cpp

Path: LeetCode-main/solutions/1262. Greatest Sum Divisible by Three/1262.cpp

```
class Solution {
public:
    int maxSumDivThree(vector<int>& nums) {
        vector<int> dp(3); // dp[i] := the maximum sum so far s.t. sum % 3 == i

        for (const int num : nums)
            for (const int sum : vector<int>(dp))
                dp[(sum + num) % 3] = max(dp[(sum + num) % 3], sum + num);

        return dp[0];
    }
};
```

291. 1263.cpp

Path: LeetCode-main/solutions/1263. Minimum Moves to Move a Box to Their Target Location/1263.cpp

```
class Solution {
public:
    int minPushBox(vector<vector<char>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<int> box;
        vector<int> player;
        vector<int> target;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 'B')
                    box = {i, j};
                else if (grid[i][j] == 'S')
                    player = {i, j};
                else if (grid[i][j] == 'T')
                    target = {i, j};

        // (boxX, boxY, playerX, playerY)
        queue<tuple<int, int, int, int>> q{
            {{box[0], box[1], player[0], player[1]}}};
        vector<vector<vector<vector<bool>>>> seen(
            m, vector<vector<vector<bool>>>(
                n, vector<vector<bool>>(m, vector<bool>(n))));
        seen[box[0]][box[1]][player[0]][player[1]] = true;

        for (int step = 0; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [boxX, boxY, playerX, playerY] = q.front();
                q.pop();
                if (boxX == target[0] && boxY == target[1])
                    return step;
                for (int k = 0; k < 4; ++k) {
                    const int nextBoxX = boxX + kDirs[k % 4][0];
                    const int nextBoxY = boxY + kDirs[k % 4][1];
                    if (isInvalid(grid, nextBoxX, nextBoxY))
                        continue;
                    if (seen[nextBoxX][nextBoxY][boxX][boxY])
                        continue;
                    const int fromX = boxX + kDirs[(k + 2) % 4][0];
                    const int fromY = boxY + kDirs[(k + 2) % 4][1];
                    if (isInvalid(grid, fromX, fromY))
                        continue;
                    if (canGoTo(grid, playerX, playerY, fromX, fromY, boxX, boxY)) {
                        seen[nextBoxX][nextBoxY][boxX][boxY] = true;
                        q.emplace(nextBoxX, nextBoxY, boxX, boxY);
                    }
                }
            }

        return -1;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns true if (playerX, playerY) can go to (fromX, fromY).
    bool canGoTo(const vector<vector<char>>& grid, int playerX, int playerY,
        int fromX, int fromY, int boxX, int boxY) {
        queue<pair<int, int>> q{{{playerX, playerY}}};
        vector<vector<bool>> seen(grid.size(), vector<bool>(grid[0].size()));
        seen[playerX][playerY] = true;

        while (!q.empty()) {
            const auto [i, j] = q.front();
            q.pop();
            if (i == fromX && j == fromY)
                return true;
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (isInvalid(grid, x, y))
                    continue;
                if (seen[x][y])
                    continue;
                if (x == boxX && y == boxY)
                    continue;
                q.emplace(x, y);
                seen[x][y] = true;
            }
        }
    }
};
```

```
    }  
    return false;  
}  
bool isValid(const vector<vector<char>>& grid, int playerX, int playerY) {  
    return playerX < 0 || playerX == grid.size() || playerY < 0 ||  
           playerY == grid[0].size() || grid[playerX][playerY] == '#';  
}  
};
```


292. 1265-2.cpp

Path: LeetCode-main/solutions/1265. Print Immutable Linked List in Reverse/1265-2.cpp

```
/**
 * // This is the ImmutableListNode's API interface.
 * // You should not implement it, or speculate about its implementation.
 * class ImmutableListNode {
 * public:
 *     void printValue(); // Print the value of the node.
 *     ImmutableListNode* getNext(); // Returns the next node.
 * };
 */

class Solution {
public:
    void printLinkedListInReverse(ImmutableListNode* head) {
        stack<ImmutableListNode*> stack;

        while (head != nullptr) {
            stack.push(head);
            head = head->getNext();
        }

        while (!stack.empty())
            stack.top()->printValue(), stack.pop();
    }
};
```

293. 1265.cpp

Path: LeetCode-main/solutions/1265. Print Immutable Linked List in Reverse/1265.cpp

```
/**
 * // This is the ImmutableListNode's API interface.
 * // You should not implement it, or speculate about its implementation.
 * class ImmutableListNode {
 * public:
 *     void printValue(); // Print the value of the node.
 *     ImmutableListNode* getNext(); // Returns the next node.
 * };
 */

class Solution {
public:
    void printLinkedListInReverse(ImmutableListNode* head) {
        if (head == nullptr)
            return;

        printLinkedListInReverse(head->getNext());
        head->printValue();
    }
};
```

294. 1266.cpp

Path: LeetCode-main/solutions/1266. Minimum Time Visiting All Points/1266.cpp

```
class Solution {
public:
    int minTimeToVisitAllPoints(vector<vector<int>>& points) {
        int ans = 0;

        for (int i = 1; i < points.size(); ++i)
            ans += max(abs(points[i][0] - points[i - 1][0]),
                      abs(points[i][1] - points[i - 1][1]));

        return ans;
    }
};
```


296. 1268.cpp

Path: LeetCode-main/solutions/1268. Search Suggestions System/1268.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    const string* word = nullptr;
    TrieNode() : children(26) {}
};

class Solution {
public:
    vector<vector<string>> suggestedProducts(vector<string>& products,
                                           string searchWord) {
        vector<vector<string>> ans;

        for (const string& product : products)
            insert(product);

        shared_ptr<TrieNode> node = root;

        for (const char c : searchWord) {
            if (node == nullptr || node->children[c - 'a'] == nullptr) {
                node = nullptr;
                ans.push_back({});
                continue;
            }
            node = node->children[c - 'a'];
            ans.push_back(search(node));
        }

        return ans;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
        node->word = &word;
    }

    vector<string> search(shared_ptr<TrieNode> node) {
        vector<string> res;
        dfs(node, res);
        return res;
    }

    void dfs(shared_ptr<TrieNode> node, vector<string>& ans) {
        if (ans.size() == 3)
            return;
        if (node == nullptr)
            return;
        if (node->word != nullptr)
            ans.push_back(*node->word);
        for (shared_ptr<TrieNode> child : node->children)
            dfs(child, ans);
    }
};
```

297. 1269.cpp

Path: LeetCode-main/solutions/1269. Number of Ways to Stay in the Same Place After Some Steps/1269.cpp

```
class Solution {
public:
    int numWays(int steps, int arrLen) {
        constexpr int kMod = 1'000'000'007;
        const int n = min(arrLen, steps / 2 + 1);
        // dp[i] := the number of ways to stay at index i
        vector<long> dp(n);
        dp[0] = 1;

        while (steps-- > 0) {
            vector<long> newDp(n);
            for (int i = 0; i < n; ++i) {
                newDp[i] = dp[i];
                if (i - 1 >= 0)
                    newDp[i] += dp[i - 1];
                if (i + 1 < n)
                    newDp[i] += dp[i + 1];
                newDp[i] %= kMod;
            }
            dp = std::move(newDp);
        }

        return dp[0];
    }
};
```

298. 127.cpp

Path: LeetCode-main/solutions/127. Word Ladder/127.cpp

```
class Solution {
public:
    int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
        unordered_set<string> wordSet(wordList.begin(), wordList.end());
        if (!wordSet.contains(endWord))
            return 0;

        queue<string> q{{beginWord}};

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                string word = q.front();
                q.pop();
                for (int i = 0; i < word.length(); ++i) {
                    const char cache = word[i];
                    for (char c = 'a'; c <= 'z'; ++c) {
                        word[i] = c;
                        if (word == endWord)
                            return step + 1;
                        if (wordSet.contains(word)) {
                            q.push(word);
                            wordSet.erase(word);
                        }
                    }
                    word[i] = cache;
                }
            }

        return 0;
    }
};
```

299. 1271.cpp

Path: LeetCode-main/solutions/1271. Hexspeak/1271.cpp

```
class Solution {
public:
    string toHexspeak(string num) {
        const long n = stol(num);
        stringstream ss;
        ss << hex << uppercase << n;
        string ans(ss.str());

        for (char& c : ans) {
            if (c > '1' && c <= '9')
                return "ERROR";
            if (c == '0')
                c = 'O';
            else if (c == '1')
                c = 'I';
        }

        return ans;
    }
};
```


300. 1272.cpp

Path: LeetCode-main/solutions/1272. Remove Interval/1272.cpp

```
class Solution {
public:
    vector<vector<int>> removeInterval(vector<vector<int>>& intervals,
                                     vector<int>& toBeRemoved) {

        vector<vector<int>> ans;

        for (const vector<int>& interval : intervals) {
            const int a = interval[0];
            const int b = interval[1];
            if (a >= toBeRemoved[1] || b <= toBeRemoved[0]) {
                ans.push_back(interval);
            } else { // a < toBeRemoved[1] && b > toBeRemoved[0]
                if (a < toBeRemoved[0])
                    ans.push_back({a, toBeRemoved[0]});
                if (b > toBeRemoved[1])
                    ans.push_back({toBeRemoved[1], b});
            }
        }

        return ans;
    }
};
```

301. 1273.cpp

Path: LeetCode-main/solutions/1273. Delete Tree Nodes/1273.cpp

```
struct T {
    int sum;
    int count;
};

class Solution {
public:
    int deleteTreeNodes(int nodes, vector<int>& parent, vector<int>& value) {
        vector<vector<int>> tree(nodes);

        for (int i = 1; i < parent.size(); ++i)
            tree[parent[i]].push_back(i);

        return dfs(tree, 0, value).count;
    }

private:
    T dfs(const vector<vector<int>>& tree, int u, const vector<int>& value) {
        int sum = value[u]; // the root value
        int count = 1;      // this root

        for (const int v : tree[u]) {
            const T t = dfs(tree, v, value);
            sum += t.sum;
            count += t.count;
        }

        if (sum == 0) // Delete this root.
            return {0, 0}; // So, its count = 0.
        return {sum, count};
    }
};
```

302. 1274.cpp

Path: LeetCode-main/solutions/1274. Number of Ships in a Rectangle/1274.cpp

```
/**
 * // This is Sea's API interface.
 * // You should not implement it, or speculate about its implementation
 * class Sea {
 * public:
 *     bool hasShips(vector<int> topRight, vector<int> bottomLeft);
 * };
 */

class Solution {
public:
    int countShips(Sea sea, vector<int> topRight, vector<int> bottomLeft) {
        if (topRight[0] < bottomLeft[0] || topRight[1] < bottomLeft[1])
            return 0;
        if (!sea.hasShips(topRight, bottomLeft))
            return 0;

        // sea.hasShips(topRight, bottomLeft) == true
        if (topRight[0] == bottomLeft[0] && topRight[1] == bottomLeft[1])
            return 1;

        const int mx = (topRight[0] + bottomLeft[0]) / 2;
        const int my = (topRight[1] + bottomLeft[1]) / 2;
        int ans = 0;
        // the top-right
        ans += countShips(sea, topRight, {mx + 1, my + 1});
        // the bottom-right
        ans += countShips(sea, {topRight[0], my}, {mx + 1, bottomLeft[1]});
        // the top-left
        ans += countShips(sea, {mx, topRight[1]}, {bottomLeft[0], my + 1});
        // the bottom-left
        ans += countShips(sea, {mx, my}, bottomLeft);
        return ans;
    }
};
```

303. 1275.cpp

Path: LeetCode-main/solutions/1275. Find Winner on a Tic Tac Toe Game/1275.cpp

```
class Solution {
public:
    string tictactoe(vector<vector<int>>& moves) {
        vector<vector<int>> row(2, vector<int>(3));
        vector<vector<int>> col(2, vector<int>(3));
        vector<int> diag1(2);
        vector<int> diag2(2);

        for (int i = 0; i < moves.size(); ++i) {
            const int r = moves[i][0];
            const int c = moves[i][1];
            const int j = i & 1;
            if (++row[j][r] == 3 || ++col[j][c] == 3 || r == c && ++diag1[j] == 3 ||
                r + c == 2 && ++diag2[j] == 3)
                return j == 0 ? "A" : "B";
        }

        return moves.size() == 9 ? "Draw" : "Pending";
    }
};
```

304. 1276.cpp

Path: LeetCode-main/solutions/1276. Number of Burgers with No Waste of Ingredients/1276.cpp

```
class Solution {
public:
    vector<int> numOfBurgers(int tomatoSlices, int cheeseSlices) {
        if (tomatoSlices % 2 == 1 || tomatoSlices < 2 * cheeseSlices ||
            tomatoSlices > cheeseSlices * 4)
            return {};

        int jumboBurgers = (tomatoSlices - 2 * cheeseSlices) / 2;

        return {jumboBurgers, cheeseSlices - jumboBurgers};
    }
};
```

305. 1277.cpp

Path: LeetCode-main/solutions/1277. Count Square Submatrices with All Ones/1277.cpp

```
class Solution {
public:
    int countSquares(vector<vector<int>>& matrix) {
        for (int i = 0; i < matrix.size(); ++i)
            for (int j = 0; j < matrix[0].size(); ++j)
                if (matrix[i][j] == 1 && i > 0 && j > 0)
                    matrix[i][j] +=
                        min({matrix[i - 1][j - 1], matrix[i - 1][j], matrix[i][j - 1]});
        return accumulate(matrix.begin(), matrix.end(), 0,
            [](int subtotal, const vector<int>& row) {
                return subtotal + accumulate(row.begin(), row.end(), 0);
            });
    }
};
```

306. 1278-2.cpp

Path: LeetCode-main/solutions/1278. Palindrome Partitioning III/1278-2.cpp

```
class Solution {
public:
    int palindromePartition(string s, int K) {
        const int n = s.length();
        // dp[i][k] := the minimum cost to make k palindromes by s[0..i)
        vector<vector<int>>> dp(n + 1, vector<int>(K + 1, n));
        // cost[i][j] := the minimum cost to make s[i..j] palindrome
        vector<vector<int>>> cost(n, vector<int>(n));

        for (int d = 1; d < n; ++d)
            for (int i = 0, j = d; j < n; ++i, ++j)
                cost[i][j] = (s[i] != s[j]) + cost[i + 1][j - 1];

        for (int i = 1; i <= n; ++i)
            dp[i][1] = cost[0][i - 1];

        for (int k = 2; k <= K; ++k)
            for (int i = k; i <= n; ++i)
                for (int j = k - 1; j < i; ++j)
                    dp[i][k] = min(dp[i][k], dp[j][k - 1] + cost[j][i - 1]);

        return dp[n][K];
    }
};
```

307. 1278.cpp

Path: LeetCode-main/solutions/1278. Palindrome Partitioning III/1278.cpp

```
class Solution {
public:
    int palindromePartition(string s, int k) {
        const int n = s.length();
        vector<vector<int>> mem(n + 1, vector<int>(k + 1, n));
        // cost[i][j] := the minimum cost to make s[i..j] palindrome
        vector<vector<int>> cost(n, vector<int>(n));

        for (int d = 1; d < n; ++d)
            for (int i = 0, j = d; j < n; ++i, ++j)
                cost[i][j] = (s[i] != s[j]) + cost[i + 1][j - 1];

        return palindromePartition(n, k, cost, mem);
    }

private:
    // Returns the minimum cost to make k palindromes by s[0..i].
    int palindromePartition(int n, int k, const vector<vector<int>>& cost,
                           vector<vector<int>>& mem) {
        if (k == 1)
            return cost[0][n - 1];
        if (mem[n][k] < n)
            return mem[n][k];

        // Try all the possible partitions.
        for (int i = k - 1; i < n; ++i)
            mem[n][k] = min(
                mem[n][k], palindromePartition(i, k - 1, cost, mem) + cost[i][n - 1]);

        return mem[n][k];
    }
};
```


308. 1279.cpp

Path: LeetCode-main/solutions/1279. Traffic Light Controlled Intersection/1279.cpp

```
class TrafficLight {
public:
    void carArrived(
        // ID of the car
        int carId,
        // ID of the road the car travels on. Can be 1 (road A) or 2 (road B).
        int roadId,
        // direction of the car
        int direction,
        // Use turnGreen() to turn light to green on current road.
        function<void()> turnGreen,
        // Use crossCar() to make car cross the intersection.
        function<void()> crossCar
    ) {
        std::lock_guard<std::mutex> lock(mutex);
        if (canPassRoadId != roadId) {
            canPassRoadId = roadId;
            turnGreen();
        }
        crossCar();
    }

private:
    int canPassRoadId = 1; // 1 := road A, 2 := road B
    std::mutex mutex;
};
```

309. 128.cpp

Path: LeetCode-main/solutions/128. Longest Consecutive Sequence/128.cpp

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        int ans = 0;
        unordered_set<int> seen{nums.begin(), nums.end()};

        for (int num : seen) {
            // `num` is the start of a sequence.
            if (seen.contains(num - 1))
                continue;
            int length = 1;
            while (seen.contains(++num))
                ++length;
            ans = max(ans, length);
        }

        return ans;
    }
};
```

310. 1281.cpp

Path: LeetCode-main/solutions/1281. Subtract the Product and Sum of Digits of an Integer/1281.cpp

```
class Solution {
public:
    int subtractProductAndSum(int n) {
        int prod = 1;
        int summ = 0;

        for (; n > 0; n /= 10) {
            prod *= n % 10;
            summ += n % 10;
        }

        return prod - summ;
    }
};
```

311. 1282.cpp

Path: LeetCode-main/solutions/1282. Group the People Given the Group Size They Belong To/1282.cpp

```
class Solution {
public:
    vector<vector<int>> groupThePeople(vector<int>& groupSizes) {
        vector<vector<int>> ans;
        unordered_map<int, vector<int>> groupSizeToIndices;

        for (int i = 0; i < groupSizes.size(); ++i)
            groupSizeToIndices[groupSizes[i]].push_back(i);

        for (const auto& [groupSize, indices] : groupSizeToIndices) {
            vector<int> groupIndices;
            for (const int index : indices) {
                groupIndices.push_back(index);
                if (groupIndices.size() == groupSize) {
                    ans.push_back(groupIndices);
                    groupIndices.clear();
                }
            }
        }

        return ans;
    }
};
```

312. 1283.cpp

Path: LeetCode-main/solutions/1283. Find the Smallest Divisor Given a Threshold/1283.cpp

```
class Solution {
public:
    int smallestDivisor(vector<int>& nums, int threshold) {
        int l = 1;
        int r = ranges::max(nums);

        while (l < r) {
            const int m = (l + r) / 2;
            if (sumDivision(nums, m) <= threshold)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    int sumDivision(const vector<int>& nums, int m) {
        int sum = 0;
        for (const int num : nums)
            sum += (num - 1) / m + 1;
        return sum;
    }
};
```

313. 1284.cpp

Path: LeetCode-main/solutions/1284. Minimum Number of Flips to Convert Binary Matrix to Zero Matrix/1284.cpp

```
class Solution {
public:
    int minFlips(vector<vector<int>>& mat) {
        const int m = mat.size();
        const int n = mat[0].size();
        const int hash = getHash(mat, m, n);
        if (hash == 0)
            return 0;

        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        queue<int> q{{hash}};
        unordered_set<int> seen{hash};

        for (int step = 1; !q.empty(); ++step) {
            for (int sz = q.size(); sz > 0; --sz) {
                const int curr = q.front();
                q.pop();
                for (int i = 0; i < m; ++i) {
                    for (int j = 0; j < n; ++j) {
                        int next = curr ^ 1 << (i * n + j);
                        // Flie the four neighbors.
                        for (const auto& [dx, dy] : kDirs) {
                            const int x = i + dx;
                            const int y = j + dy;
                            if (x < 0 || x == m || y < 0 || y == n)
                                continue;
                            next ^= 1 << (x * n + y);
                        }
                        if (next == 0)
                            return step;
                        if (seen.contains(next))
                            continue;
                        q.push(next);
                        seen.insert(next);
                    }
                }
            }
        }

        return -1;
    }

private:
    int getHash(const vector<vector<int>>& mat, int m, int n) {
        int hash = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (mat[i][j])
                    hash |= 1 << (i * n + j);
        return hash;
    }
};
```

314. 1287.cpp

Path: LeetCode-main/solutions/1287. Element Appearing More Than 25% In Sorted Array/1287.cpp

```
class Solution {
public:
    int findSpecialInteger(vector<int>& arr) {
        const int n = arr.size();
        const int quarter = n / 4;

        for (int i = 0; i < n - quarter; ++i)
            if (arr[i] == arr[i + quarter])
                return arr[i];

        throw;
    }
};
```

315. 1288.cpp

Path: LeetCode-main/solutions/1288. Remove Covered Intervals/1288.cpp

```
class Solution {
public:
    int removeCoveredIntervals(vector<vector<int>>& intervals) {
        ranges::sort(intervals, ranges::less{}, [](const vector<int>& interval) {
            const int l = interval[0];
            const int r = interval[1];
            return pair<int, int>{l, -r};
        });

        int ans = 0;
        int prevEnd = 0;

        for (const vector<int>& interval : intervals)
            // The current interval is not covered by the previous one.
            if (prevEnd < interval[1]) {
                prevEnd = interval[1];
                ++ans;
            }

        return ans;
    }
};
```


316. 1289.cpp

Path: LeetCode-main/solutions/1289. Minimum Falling Path Sum II/1289.cpp

```
class Solution {
public:
    int minFallingPathSum(vector<vector<int>>& grid) {
        const int n = grid.size();

        for (int i = 1; i < n; ++i) {
            const vector<pair<int, int>> twoMinNumAndIndexs =
                getTwoMinNumAndIndexs(grid[i - 1]);
            const auto& [firstMinNum, firstMinIndex] = twoMinNumAndIndexs[0];
            const auto& [secondMinNum, _] = twoMinNumAndIndexs[1];
            for (int j = 0; j < n; ++j)
                if (j == firstMinIndex)
                    grid[i][j] += secondMinNum;
                else
                    grid[i][j] += firstMinNum;
        }

        return ranges::min(grid.back());
    }

private:
    vector<pair<int, int>> getTwoMinNumAndIndexs(const vector<int>& A) {
        vector<pair<int, int>> numAndIndexs;
        for (int i = 0; i < A.size(); ++i)
            numAndIndexs.emplace_back(A[i], i);
        ranges::sort(numAndIndexs);
        return {numAndIndexs[0], numAndIndexs[1]};
    }
};
```

317. 129.cpp

Path: LeetCode-main/solutions/129. Sum Root to Leaf Numbers/129.cpp

```
class Solution {
public:
    int sumNumbers(TreeNode* root) {
        int ans = 0;
        dfs(root, 0, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, int path, int& ans) {
        if (root == nullptr)
            return;
        if (root->left == nullptr && root->right == nullptr) {
            ans += path * 10 + root->val;
            return;
        }

        dfs(root->left, path * 10 + root->val, ans);
        dfs(root->right, path * 10 + root->val, ans);
    }
};
```

318. 1290.cpp

Path: LeetCode-main/solutions/1290. Convert Binary Number in a Linked List to Integer/1290.cpp

```
class Solution {
public:
    int getDecimalValue(ListNode* head) {
        int ans = 0;

        for (; head; head = head->next)
            ans = ans * 2 + head->val;

        return ans;
    }
};
```

319. 1291.cpp

Path: LeetCode-main/solutions/1291. Sequential Digits/1291.cpp

```
class Solution {
public:
    vector<int> sequentialDigits(int low, int high) {
        vector<int> ans;
        queue<int> q{{1, 2, 3, 4, 5, 6, 7, 8, 9}};

        while (!q.empty()) {
            const int num = q.front();
            q.pop();
            if (num > high)
                return ans;
            if (low <= num && num <= high)
                ans.push_back(num);
            const int lastDigit = num % 10;
            if (lastDigit < 9)
                q.push(num * 10 + lastDigit + 1);
        }

        return ans;
    }
};
```

320. 1292.cpp

Path: LeetCode-main/solutions/1292. Maximum Side Length of a Square with Sum Less than or Equal to Threshold/1292.cpp

```
class Solution {
public:
    int maxSideLength(vector<vector<int>>& mat, int threshold) {
        const int m = mat.size();
        const int n = mat[0].size();
        int ans = 0;
        vector<vector<int>> prefix(m + 1, vector<int>(n + 1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                prefix[i + 1][j + 1] =
                    mat[i][j] + prefix[i][j + 1] + prefix[i + 1][j] - prefix[i][j];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int length = ans; length < min(m - i, n - j); ++length) {
                    if (squareSum(prefix, i, j, i + length, j + length) > threshold)
                        break;
                    ans = max(ans, length + 1);
                }

        return ans;
    }

private:
    int squareSum(vector<vector<int>>& prefix, int r1, int c1, int r2, int c2) {
        return prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1] - prefix[r2 + 1][c1] +
            prefix[r1][c1];
    }
};
```

321. 1293.cpp

Path: LeetCode-main/solutions/1293. Shortest Path in a Grid with Obstacles Elimination/1293.cpp

```
class Solution {
public:
    int shortestPath(vector<vector<int>>& grid, int k) {
        const int m = grid.size();
        const int n = grid[0].size();
        if (m == 1 && n == 1)
            return 0;

        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        queue<tuple<int, int, int>> q{{{0, 0, k}}}; // (i, j, eliminate)
        vector<vector<vector<bool>>> seen(
            m, vector<vector<bool>>(n, vector<bool>(k + 1)));
        seen[0][0][k] = true;

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j, eliminate] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (x == m - 1 && y == n - 1)
                        return step;
                    if (grid[x][y] == 1 && eliminate == 0)
                        continue;
                    const int newEliminate = eliminate - grid[x][y];
                    if (seen[x][y][newEliminate])
                        continue;
                    q.emplace(x, y, newEliminate);
                    seen[x][y][newEliminate] = true;
                }
            }
        return -1;
    }
};
```

322. 1295.cpp

Path: LeetCode-main/solutions/1295. Find Numbers with Even Number of Digits/1295.cpp

```
class Solution {
public:
    int findNumbers(vector<int>& nums) {
        int ans = 0;

        for (int num : nums)
            if (9 < num && num < 100 || 999 < num && num < 10000 || num == 100000)
                ++ans;

        return ans;
    }
};
```

323. 1296.cpp

Path: LeetCode-main/solutions/1296. Divide Array in Sets of K Consecutive Numbers/1296.cpp

```
class Solution {
public:
    bool isPossibleDivide(vector<int>& nums, int k) {
        map<int, int> count;

        for (const int num : nums)
            ++count[num];

        for (const auto& [start, _] : count) {
            const int value = count[start];
            if (value > 0)
                for (int i = start; i < start + k; ++i) {
                    count[i] -= value;
                    if (count[i] < 0)
                        return false;
                }
        }

        return true;
    }
};
```


324. 1297.cpp

Path: LeetCode-main/solutions/1297. Maximum Number of Occurrences of a Substring/1297.cpp

```
class Solution {
public:
    int maxFreq(string s, int maxLetters, int minSize, int maxSize) {
        // Greedily consider strings with `minSize`, so ignore `maxSize`.
        int ans = 0;
        int letters = 0;
        vector<int> count(26);
        unordered_map<string, int> substringCount;

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s[r] - 'a'] == 1)
                ++letters;
            while (letters > maxLetters || r - l + 1 > minSize)
                if (--count[s[l++]] == 0)
                    --letters;
            if (r - l + 1 == minSize)
                ans = max(ans, ++substringCount[s.substr(l, minSize)]);
        }

        return ans;
    }
};
```

325. 1298.cpp

Path: LeetCode-main/solutions/1298. Maximum Candies You Can Get from Boxes/1298.cpp

```
class Solution {
public:
    int maxCandies(vector<int>& status, vector<int>& candies,
                  vector<vector<int>>& keys, vector<vector<int>>& containedBoxes,
                  vector<int>& initialBoxes) {
        int ans = 0;
        queue<int> q;
        vector<bool> reachedClosedBoxes(status.size());

        auto pushBoxesIfPossible = [&status, &q,
                                   &reachedClosedBoxes](const vector<int>& boxes) {
            for (const int box : boxes)
                if (status[box])
                    q.push(box);
                else
                    reachedClosedBoxes[box] = true;
        };

        pushBoxesIfPossible(initialBoxes);

        while (!q.empty()) {
            const int currBox = q.front();
            q.pop();

            // Add the candies.
            ans += candies[currBox];

            // Push `reachedClosedBoxes` by `key` obtained in this turn and change
            // their statuses.
            for (const int key : keys[currBox]) {
                if (!status[key] && reachedClosedBoxes[key])
                    q.push(key);
                status[key] = 1; // boxes[key] is now open.
            }

            // Push the boxes contained in `currBox`.
            pushBoxesIfPossible(containedBoxes[currBox]);
        }

        return ans;
    }
};
```

326. 1299.cpp

Path: LeetCode-main/solutions/1299. Replace Elements with Greatest Element on Right Side/1299.cpp

```
class Solution {
public:
    vector<int> replaceElements(vector<int>& arr) {
        int maxOfRight = -1;
        for (int i = arr.size() - 1; i >= 0; --i)
            maxOfRight = max(maxOfRight, exchange(arr[i], maxOfRight));
        return arr;
    }
};
```

327. 13.cpp

Path: LeetCode-main/solutions/13. Roman to Integer/13.cpp

```
class Solution {
public:
    int romanToInt(string s) {
        int ans = 0;
        vector<int> roman(128);

        roman['I'] = 1;
        roman['V'] = 5;
        roman['X'] = 10;
        roman['L'] = 50;
        roman['C'] = 100;
        roman['D'] = 500;
        roman['M'] = 1000;

        for (int i = 0; i + 1 < s.length(); ++i)
            if (roman[s[i]] < roman[s[i + 1]])
                ans -= roman[s[i]];
            else
                ans += roman[s[i]];

        return ans + roman[s.back()];
    }
};
```

328. 130-2.cpp

Path: LeetCode-main/solutions/130. Surrounded Regions/130-2.cpp

```
class Solution {
public:
    void solve(vector<vector<char>>& board) {
        if (board.empty())
            return;

        const int m = board.size();
        const int n = board[0].size();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    dfs(board, i, j);

        for (vector<char>& row : board)
            for (char& c : row)
                if (c == '*')
                    c = 'O';
                else if (c == 'O')
                    c = 'X';
    }

private:
    // Marks the grids with 'O' that stretch from the four sides to '*'.
    void dfs(vector<vector<char>>& board, int i, int j) {
        if (i < 0 || i == board.size() || j < 0 || j == board[0].size())
            return;
        if (board[i][j] != 'O')
            return;
        board[i][j] = '*';
        dfs(board, i + 1, j);
        dfs(board, i - 1, j);
        dfs(board, i, j + 1);
        dfs(board, i, j - 1);
    }
};
```

329. 130.cpp

Path: LeetCode-main/solutions/130. Surrounded Regions/130.cpp

```
class Solution {
public:
    void solve(vector<vector<char>>& board) {
        if (board.empty())
            return;

        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = board.size();
        const int n = board[0].size();

        queue<pair<int, int>> q;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i * j == 0 || i == m - 1 || j == n - 1)
                    if (board[i][j] == 'O') {
                        q.emplace(i, j);
                        board[i][j] = '*';
                    }

        // Mark the grids that stretch from the four sides with '*'.
        while (!q.empty()) {
            const auto [i, j] = q.front();
            q.pop();
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (board[x][y] != 'O')
                    continue;
                q.emplace(x, y);
                board[x][y] = '*';
            }
        }

        for (vector<char>& row : board)
            for (char& c : row)
                if (c == '*')
                    c = 'O';
                else if (c == 'O')
                    c = 'X';
    }
};
```

330. 1300.cpp

Path: LeetCode-main/solutions/1300. Sum of Mutated Array Closest to Target/1300.cpp

```
class Solution {
public:
    int findBestValue(vector<int>& arr, int target) {
        const int n = arr.size();
        const double err = 1e-9;

        int prefix = 0;

        ranges::sort(arr);

        for (int i = 0; i < n; ++i) {
            int ans = round((target - prefix - err) / (double)(n - i));
            if (ans <= arr[i])
                return ans;
            prefix += arr[i];
        }

        return arr.back();
    }
};
```

331. 1301.cpp

Path: LeetCode-main/solutions/1301. Number of Paths with Max Score/1301.cpp

```
class Solution {
public:
    vector<int> pathsWithMaxScore(vector<string>& board) {
        constexpr int kMod = 1'000'000'007;
        constexpr int kDirs[3][2] = {{0, 1}, {1, 0}, {1, 1}};
        const int n = board.size();
        // dp[i][j] := the maximum sum from (n - 1, n - 1) to (i, j)
        vector<vector<int>> dp(n + 1, vector<int>(n + 1, -1));
        // count[i][j] := the number of paths to get dp[i][j] from (n - 1, n - 1) to
        // (i, j)
        vector<vector<int>> count(n + 1, vector<int>(n + 1));

        dp[0][0] = 0;
        dp[n - 1][n - 1] = 0;
        count[n - 1][n - 1] = 1;

        for (int i = n - 1; i >= 0; --i)
            for (int j = n - 1; j >= 0; --j) {
                if (board[i][j] == 'S' || board[i][j] == 'X')
                    continue;
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (dp[i][j] < dp[x][y]) {
                        dp[i][j] = dp[x][y];
                        count[i][j] = count[x][y];
                    } else if (dp[i][j] == dp[x][y]) {
                        count[i][j] += count[x][y];
                        count[i][j] %= kMod;
                    }
                }
                // If there's path(s) from 'S' to (i, j) and the cell is not 'E'.
                if (dp[i][j] != -1 && board[i][j] != 'E') {
                    dp[i][j] += board[i][j] - '0';
                    dp[i][j] %= kMod;
                }
            }

        return {dp[0][0], count[0][0]};
    }
};
```


332. 1302.cpp

Path: LeetCode-main/solutions/1302. Deepest Leaves Sum/1302.cpp

```
class Solution {
public:
    int deepestLeavesSum(TreeNode* root) {
        int ans = 0;
        queue<TreeNode*> q{{root}};

        while (!q.empty()) {
            ans = 0;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode* node = q.front();
                q.pop();
                ans += node->val;
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
        }
        return ans;
    }
};
```

333. 1304.cpp

Path: LeetCode-main/solutions/1304. Find N Unique Integers Sum up to Zero/1304.cpp

```
class Solution {
public:
    vector<int> sumZero(int n) {
        vector<int> ans(n);

        for (int i = 0; i < n; ++i)
            ans[i] = i * 2 - n + 1;

        return ans;
    }
};
```

334. 1305.cpp

Path: LeetCode-main/solutions/1305. All Elements in Two Binary Search Trees/1305.cpp

```
class BSTIterator {
public:
    BSTIterator(TreeNode* root) {
        pushLeftsUntilNull(root);
    }

    int peek() {
        return stack.top()->val;
    }

    void next() {
        TreeNode* node = stack.top();
        stack.pop();
        pushLeftsUntilNull(node->right);
    }

    bool hasNext() {
        return !stack.empty();
    }

private:
    stack<TreeNode*> stack;

    void pushLeftsUntilNull(TreeNode* node) {
        while (node != nullptr) {
            stack.push(node);
            node = node->left;
        }
    }
};

class Solution {
public:
    vector<int> getAllElements(TreeNode* root1, TreeNode* root2) {
        vector<int> ans;
        BSTIterator bstIterator1(root1);
        BSTIterator bstIterator2(root2);

        while (bstIterator1.hasNext() && bstIterator2.hasNext())
            if (bstIterator1.peek() < bstIterator2.peek()) {
                ans.push_back(bstIterator1.peek());
                bstIterator1.next();
            } else {
                ans.push_back(bstIterator2.peek());
                bstIterator2.next();
            }

        while (bstIterator1.hasNext()) {
            ans.push_back(bstIterator1.peek());
            bstIterator1.next();
        }

        while (bstIterator2.hasNext()) {
            ans.push_back(bstIterator2.peek());
            bstIterator2.next();
        }

        return ans;
    }
};
```

335. 1306-2.cpp

Path: LeetCode-main/solutions/1306. Jump Game III/1306-2.cpp

```
class Solution {
public:
    bool canReach(vector<int>& arr, int start) {
        return canReach(arr, start, vector<bool>(arr.size()));
    }

private:
    bool canReach(const vector<int>& arr, int node, vector<bool>&& seen) {
        if (node < 0 || node >= arr.size())
            return false;
        if (seen[node])
            return false;
        if (arr[node] == 0)
            return true;
        seen[node] = true;
        return canReach(arr, node + arr[node], std::move(seen)) ||
            canReach(arr, node - arr[node], std::move(seen));
    }
};
```

336. 1306.cpp

Path: LeetCode-main/solutions/1306. Jump Game III/1306.cpp

```
class Solution {
public:
    bool canReach(vector<int>& arr, int start) {
        const int n = arr.size();
        queue<int> q{{start}};
        vector<bool> seen(n);

        while (!q.empty()) {
            const int node = q.front();
            q.pop();
            if (arr[node] == 0)
                return true;
            if (seen[node])
                continue;
            // Check the available next steps.
            if (node - arr[node] >= 0)
                q.push(node - arr[node]);
            if (node + arr[node] < n)
                q.push(node + arr[node]);
            seen[node] = true;
        }

        return false;
    }
};
```

337. 1307.cpp

Path: LeetCode-main/solutions/1307. Verbal Arithmetic Puzzle/1307.cpp

```
class Solution {
public:
    bool isSolvable(vector<string>& words, string result) {
        usedDigit = vector<bool>(10);
        words.push_back(result);
        rows = words.size();
        for (const string& word : words)
            cols = max(cols, static_cast<int>(word.length()));
        return dfs(words, 0, 0, 0);
    }

private:
    unordered_map<char, int> letterToDigit;
    vector<bool> usedDigit;
    int rows;
    int cols;

    bool dfs(vector<string>& words, int row, int col, int sum) {
        if (col == cols)
            return sum == 0;
        if (row == rows)
            return sum % 10 == 0 && dfs(words, 0, col + 1, sum / 10);

        string word = words[row];
        if (col >= word.length())
            return dfs(words, row + 1, col, sum);

        char letter = word[word.length() - col - 1];
        int sign = row == rows - 1 ? -1 : 1;

        if (const auto it = letterToDigit.find(letter);
            it != letterToDigit.cend() &&
            (it->second > 0 || col < word.length() - 1))
            return dfs(words, row + 1, col, sum + sign * letterToDigit[letter]);

        for (int digit = 0; digit < 10; ++digit)
            if (!usedDigit[digit] && (digit > 0 || col + 1 < word.length())) {
                letterToDigit[letter] = digit;
                usedDigit[digit] = true;
                if (dfs(words, row + 1, col, sum + sign * digit))
                    return true;
                usedDigit[digit] = false;
                letterToDigit.erase(letter);
            }

        return false;
    }
};
```

338. 1309.cpp

Path: LeetCode-main/solutions/1309. Decrypt String from Alphabet to Integer Mapping/1309.cpp

```
class Solution {
public:
    string freqAlphabets(string s) {
        string ans;

        for (int i = 0; i < s.length(); i) {
            if (i + 2 < s.length() && s[i + 2] == '#') {
                ans += stoi(s.substr(i, 2)) + 'a' - 1;
                i += 3;
            } else {
                ans += (s[i] - '0') + 'a' - 1;
                i += 1;
            }
        }

        return ans;
    }
};
```

339. 131.cpp

Path: LeetCode-main/solutions/131. Palindrome Partitioning/131.cpp

```
class Solution {
public:
    vector<vector<string>> partition(string s) {
        vector<vector<string>> ans;
        dfs(s, 0, {}, ans);
        return ans;
    }

private:
    void dfs(const string& s, int start, vector<string>&& path,
            vector<vector<string>>& ans) {
        if (start == s.length()) {
            ans.push_back(path);
            return;
        }

        for (int i = start; i < s.length(); ++i)
            if (isPalindrome(s, start, i)) {
                path.push_back(s.substr(start, i - start + 1));
                dfs(s, i + 1, std::move(path), ans);
                path.pop_back();
            }
    }

    bool isPalindrome(const string& s, int l, int r) {
        while (l < r)
            if (s[l++] != s[r--])
                return false;
        return true;
    }
};
```


340. 1310.cpp

Path: LeetCode-main/solutions/1310. XOR Queries of a Subarray/1310.cpp

```
class Solution {
public:
    vector<int> xorQueries(vector<int>& arr, vector<vector<int>>& queries) {
        vector<int> ans;
        vector<int> xors(arr.size() + 1);

        for (int i = 0; i < arr.size(); ++i)
            xors[i + 1] = xors[i] ^ arr[i];

        for (const vector<int>& query : queries) {
            const int left = query[0];
            const int right = query[1];
            ans.push_back(xors[left] ^ xors[right + 1]);
        }

        return ans;
    }
};
```

341. 1311.cpp

Path: LeetCode-main/solutions/1311. Get Watched Videos by Your Friends/1311.cpp

```
class Solution {
public:
    vector<string> watchedVideosByFriends(vector<vector<string>>& watchedVideos,
                                         vector<vector<int>>& friends, int id,
                                         int level) {

        vector<string> ans;
        queue<int> q{{id}};
        vector<bool> seen(friends.size());
        seen[id] = true;
        unordered_map<string, int> count;
        set<pair<int, string>> freqAndVideo;

        for (int i = 0; i < level; ++i)
            for (int sz = q.size(); sz > 0; --sz) {
                for (const int friend_ : friends[q.front()])
                    if (!seen[friend_]) {
                        seen[friend_] = true;
                        q.push(friend_);
                    }
                q.pop();
            }

        for (int i = q.size(); i > 0; --i) {
            for (const string& video : watchedVideos[q.front()])
                ++count[video];
            q.pop();
        }

        for (const auto& [video, freq] : count)
            freqAndVideo.insert({freq, video});

        for (const auto& [_, video] : freqAndVideo)
            ans.push_back(video);

        return ans;
    }
};
```

342. 1312.cpp

Path: LeetCode-main/solutions/1312. Minimum Insertion Steps to Make a String Palindrome/1312.cpp

```
class Solution {
public:
    int minInsertions(string s) {
        return s.length() - longestPalindromeSubseq(s);
    }

private:
    // Same as 516. Longest Palindromic Subsequence
    int longestPalindromeSubseq(const string& s) {
        const int n = s.length();
        // dp[i][j] := the length of LPS(s[i..j])
        vector<vector<int>>> dp(n, vector<int>(n));

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                if (s[i] == s[j])
                    dp[i][j] = 2 + dp[i + 1][j - 1];
                else
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
            }

        return dp[0][n - 1];
    }
};
```

343. 1313.cpp

Path: LeetCode-main/solutions/1313. Decompress Run-Length Encoded List/1313.cpp

```
class Solution {
public:
    vector<int> decompressRLElist(vector<int>& nums) {
        vector<int> ans;

        for (int i = 0; i < nums.size(); i += 2)
            ans.insert(ans.end(), nums[i], nums[i + 1]);

        return ans;
    }
};
```

344. 1314.cpp

Path: LeetCode-main/solutions/1314. Matrix Block Sum/1314.cpp

```
class Solution {
public:
    vector<vector<int>> matrixBlockSum(vector<vector<int>>& mat, int k) {
        const int m = mat.size();
        const int n = mat[0].size();
        vector<vector<int>> ans(m, vector<int>(n));
        vector<vector<int>> prefix(m + 1, vector<int>(n + 1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                prefix[i + 1][j + 1] =
                    mat[i][j] + prefix[i][j + 1] + prefix[i + 1][j] - prefix[i][j];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                const int r1 = max(0, i - k) + 1;
                const int c1 = max(0, j - k) + 1;
                const int r2 = min(m - 1, i + k) + 1;
                const int c2 = min(n - 1, j + k) + 1;
                ans[i][j] = prefix[r2][c2] - prefix[r2][c1 - 1] - prefix[r1 - 1][c2] +
                    prefix[r1 - 1][c1 - 1];
            }

        return ans;
    }
};
```

345. 1315.cpp

Path: LeetCode-main/solutions/1315. Sum of Nodes with Even-Valued Grandparent/1315.cpp

```
class Solution {
public:
    int sumEvenGrandparent(TreeNode* root) {
        return dfs(root, 1, 1); // The parent and the grandparent are odd at first.
    }

private:
    int dfs(TreeNode* root, int p, int gp) {
        if (root == nullptr)
            return 0;
        return (gp % 2 == 0 ? root->val : 0) + //
            dfs(root->left, root->val, p) + //
            dfs(root->right, root->val, p);
    }
};
```

346. 1316.cpp

Path: LeetCode-main/solutions/1316. Distinct Echo Substrings/1316.cpp

```
class Solution {
public:
    int distinctEchoSubstrings(string text) {
        unordered_set<string> seen;

        for (int k = 1; k <= text.length() / 2; ++k) { // the target length
            int same = 0;
            for (int l = 0, r = k; r < text.length(); ++l, ++r) {
                if (text[l] == text[r])
                    ++same;
                else
                    same = 0;
                if (same == k) {
                    seen.insert(text.substr(l - k + 1, k));
                    // Move the window thus leaving a letter behind, so we need to
                    // decrease the counter.
                    --same;
                }
            }
        }

        return seen.size();
    }
};
```

347. 1317.cpp

Path: LeetCode-main/solutions/1317. Convert Integer to the Sum of Two No-Zero Integers/1317.cpp

```
class Solution {
public:
    vector<int> getNoZeroIntegers(int n) {
        for (int A = 1; A < n; ++A) {
            int B = n - A;
            if (to_string(A).find('0') == string::npos &&
                to_string(B).find('0') == string::npos)
                return {A, B};
        }
        throw;
    }
};
```


348. 1318.cpp

Path: LeetCode-main/solutions/1318. Minimum Flips to Make a OR b Equal to c/1318.cpp

```
class Solution {
public:
    int minFlips(int a, int b, int c) {
        constexpr int kMaxBit = 30;
        int ans = 0;

        for (int i = 0; i < kMaxBit; ++i)
            if (c >> i & 1)
                ans += (a >> i & 1) == 0 && (b >> i & 1) == 0;
            else // (c >> i & 1) == 0
                ans += (a >> i & 1) + (b >> i & 1);

        return ans;
    }
};
```

349. 1319.cpp

Path: LeetCode-main/solutions/1319. Number of Operations to Make Network Connected/1319.cpp

```
class Solution {
public:
    int makeConnected(int n, vector<vector<int>>& connections) {
        // To connect n nodes, we need at least n - 1 edges
        if (connections.size() < n - 1)
            return -1;

        int numOfConnected = 0;
        vector<vector<int>> graph(n);
        unordered_set<int> seen;

        for (const vector<int>& connection : connections) {
            const int u = connection[0];
            const int v = connection[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        for (int i = 0; i < n; ++i)
            if (seen.insert(i).second) {
                dfs(graph, i, seen);
                ++numOfConnected;
            }

        return numOfConnected - 1;
    }

private:
    void dfs(const vector<vector<int>>& graph, int u, unordered_set<int>& seen) {
        for (const int v : graph[u])
            if (seen.insert(v).second)
                dfs(graph, v, seen);
    }
};
```

350. 132.cpp

Path: LeetCode-main/solutions/132. Palindrome Partitioning II/132.cpp

```
class Solution {
public:
    int minCut(string s) {
        const int n = s.length();
        // isPalindrome[i][j] := true if s[i..j] is a palindrome
        vector<vector<bool>> isPalindrome(n, vector<bool>(n, true));
        // dp[i] := the minimum cuts needed for a palindrome partitioning of s[0..i]
        vector<int> dp(n, n);

        for (int l = 2; l <= n; ++l)
            for (int i = 0, j = l - 1; j < n; ++i, ++j)
                isPalindrome[i][j] = s[i] == s[j] && isPalindrome[i + 1][j - 1];

        for (int i = 0; i < n; ++i) {
            if (isPalindrome[0][i]) {
                dp[i] = 0;
                continue;
            }

            // Try all the possible partitions.
            for (int j = 0; j < i; ++j)
                if (isPalindrome[j + 1][i])
                    dp[i] = min(dp[i], dp[j] + 1);
        }

        return dp.back();
    }
};
```

351. 1320.cpp

Path: LeetCode-main/solutions/1320. Minimum Distance to Type a Word Using Two Fingers/1320.cpp

```
class Solution {
public:
    int minimumDistance(string word) {
        vector<vector<vector<int>>> mem(
            27, vector<vector<int>>(27, vector<int>(word.length(), -1)));
        return minimumDistance(word, 26, 26, 0, mem);
    }

private:
    // Returns the minimum distance to type `word`, where the left finger is on
    // the i-th letter, the right finger is on the j-th letter, and word[0..k)
    // have been written.
    int minimumDistance(const string& word, int i, int j, int k,
        vector<vector<vector<int>>>& mem) {
        if (k == word.length())
            return 0;
        if (mem[i][j][k] != -1)
            return mem[i][j][k];
        const int c = word[k] - 'A';
        const int moveLeft = dist(i, c) + minimumDistance(word, c, j, k + 1, mem);
        const int moveRight = dist(j, c) + minimumDistance(word, i, c, k + 1, mem);
        return mem[i][j][k] = min(moveLeft, moveRight);
    }

    int dist(int a, int b) {
        if (a == 26) // the first hovering state
            return 0;
        const int x1 = a / 6;
        const int y1 = a % 6;
        const int x2 = b / 6;
        const int y2 = b % 6;
        return abs(x1 - x2) + abs(y1 - y2);
    }
};
```

352. 1323.cpp

Path: LeetCode-main/solutions/1323. Maximum 69 Number/1323.cpp

```
class Solution {
public:
    int maximum69Number(int num) {
        string ans = to_string(num);

        for (char& c : ans)
            if (c == '6') {
                c = '9';
                break;
            }

        return stoi(ans);
    }
};
```

353. 1324.cpp

Path: LeetCode-main/solutions/1324. Print Words Vertically/1324.cpp

```
class Solution {
public:
    vector<string> printVertically(string s) {
        vector<string> ans;
        vector<string> words = split(s);
        size_t maxLength = 0;

        for (const string& word : words)
            maxLength = max(maxLength, word.length());

        for (size_t i = 0; i < maxLength; ++i) {
            string row;
            for (const string& word : words)
                row += i < word.length() ? word[i] : ' ';
            while (row.back() == ' ')
                row.pop_back();
            ans.push_back(row);
        }

        return ans;
    }

private:
    vector<string> split(const string& s) {
        vector<string> words;
        istringstream iss(s);
        for (string token; iss >> token;)
            words.push_back(token);
        return words;
    }
};
```

354. 1325.cpp

Path: LeetCode-main/solutions/1325. Delete Leaves With a Given Value/1325.cpp

```
class Solution {
public:
    TreeNode* removeLeafNodes(TreeNode* root, int target) {
        if (root == nullptr)
            return nullptr;
        root->left = removeLeafNodes(root->left, target);
        root->right = removeLeafNodes(root->right, target);
        return isLeaf(root) && root->val == target ? nullptr : root;
    }

private:
    bool isLeaf(TreeNode* root) {
        return root->left == nullptr && root->right == nullptr;
    }
};
```

355. 1326.cpp

Path: LeetCode-main/solutions/1326. Minimum Number of Taps to Open to Water a Garden/1326.cpp

```
class Solution {
public:
    int minTaps(int n, vector<int>& ranges) {
        vector<int> nums(n + 1);

        for (int i = 0; i <= n; ++i) {
            int l = max(0, i - ranges[i]);
            int r = min(n, i + ranges[i]);
            nums[l] = max(nums[l], r - l);
        }

        int ans = 0;
        int end = 0;
        int farthest = 0;

        for (int i = 0; i < n; i++) {
            farthest = max(farthest, i + nums[i]);
            if (i == end) {
                ++ans;
                end = farthest;
            }
        }

        return end == n ? ans : -1;
    }
};
```


356. 1328.cpp

Path: LeetCode-main/solutions/1328. Break a Palindrome/1328.cpp

```
class Solution {
public:
    string breakPalindrome(string palindrome) {
        if (palindrome.length() == 1)
            return "";

        for (int i = 0; i < palindrome.length() / 2; ++i)
            if (palindrome[i] != 'a') {
                palindrome[i] = 'a';
                return palindrome;
            }

        palindrome.back() = 'b';
        return palindrome;
    }
};
```

357. 1329.cpp

Path: LeetCode-main/solutions/1329. Sort the Matrix Diagonally/1329.cpp

```
class Solution {
public:
    vector<vector<int>> diagonalSort(vector<vector<int>>& mat) {
        const int m = mat.size();
        const int n = mat[0].size();

        unordered_map<int, priority_queue<int>> count;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                count[i - j].push(mat[i][j]);

        for (int i = m - 1; i >= 0; --i)
            for (int j = n - 1; j >= 0; --j)
                mat[i][j] = count[i - j].top(), count[i - j].pop();

        return mat;
    }
};
```

358. 133-2.cpp

Path: LeetCode-main/solutions/133. Clone Graph/133-2.cpp

```
class Solution {
public:
    Node* cloneGraph(Node* node) {
        if (node == nullptr)
            return nullptr;
        if (const auto it = map.find(node); it != map.cend())
            return it->second;

        Node* newNode = new Node(node->val);
        map[node] = newNode;

        for (Node* neighbor : node->neighbors)
            newNode->neighbors.push_back(cloneGraph(neighbor));

        return newNode;
    }

private:
    unordered_map<Node*, Node*> map;
};
```

359. 133.cpp

Path: LeetCode-main/solutions/133. Clone Graph/133.cpp

```
class Solution {
public:
    Node* cloneGraph(Node* node) {
        if (node == nullptr)
            return nullptr;

        queue<Node*> q{{node}};
        unordered_map<Node*, Node*> map{{node, new Node(node->val)}};

        while (!q.empty()) {
            Node* u = q.front();
            q.pop();
            for (Node* v : u->neighbors) {
                if (!map.contains(v)) {
                    map[v] = new Node(v->val);
                    q.push(v);
                }
                map[u]->neighbors.push_back(map[v]);
            }
        }

        return map[node];
    }
};
```

360. 1330.cpp

Path: LeetCode-main/solutions/1330. Reverse Subarray To Maximize Array Value/1330.cpp

```
class Solution {
public:
    int maxValueAfterReverse(vector<int>& nums) {
        int total = 0;
        int mn = INT_MAX;
        int mx = INT_MIN;

        for (int i = 0; i + 1 < nums.size(); ++i) {
            const int a = nums[i];
            const int b = nums[i + 1];
            total += abs(a - b);
            mn = min(mn, max(a, b));
            mx = max(mx, min(a, b));
        }
        int diff = max(0, (mx - mn) * 2);

        for (int i = 0; i + 1 < nums.size(); ++i) {
            const int a = nums[i];
            const int b = nums[i + 1];
            const int headDiff = -abs(a - b) + abs(nums.front() - b);
            const int tailDiff = -abs(a - b) + abs(nums.back() - a);
            diff = max({diff, headDiff, tailDiff});
        }

        return total + diff;
    }
};
```

361. 1331.cpp

Path: LeetCode-main/solutions/1331. Rank Transform of an Array/1331.cpp

```
class Solution {
public:
    vector<int> arrayRankTransform(vector<int>& arr) {
        vector<int> sortedArr(arr);
        unordered_map<int, int> rank;

        ranges::sort(sortedArr);

        for (const int a : sortedArr)
            if (!rank.contains(a))
                rank[a] = rank.size() + 1;

        for (int& a : arr)
            a = rank[a];

        return arr;
    }
};
```

362. 1332.cpp

Path: LeetCode-main/solutions/1332. Remove Palindromic Subsequences/1332.cpp

```
class Solution {
public:
    int removePalindromeSub(string s) {
        return equal(s.begin(), s.end(), s.rbegin()) ? 1 : 2;
    }
};
```

363. 1333.cpp

Path: LeetCode-main/solutions/1333. Filter Restaurants by Vegan-Friendly, Price and Distance/1333.cpp

```
class Solution {
public:
    vector<int> filterRestaurants(vector<vector<int>>& restaurants,
                                  int veganFriendly, int maxPrice,
                                  int maxDistance) {
        vector<int> ans;
        vector<vector<int>> filteredRestaurants;

        for (const vector<int>& restaurant : restaurants)
            if (restaurant[2] >= veganFriendly && restaurant[3] <= maxPrice &&
                restaurant[4] <= maxDistance)
                filteredRestaurants.push_back(restaurant);

        ranges::sort(filteredRestaurants, ranges::less{},
                      [](const vector<int>& restaurant) {
                          const int rating = restaurant[1];
                          const int id = restaurant[0];
                          return pair<int, int>{-rating, -id};
                      });

        for (const vector<int>& restaurant : filteredRestaurants)
            ans.push_back(restaurant[0]);

        return ans;
    }
};
```


364. 1334.cpp

Path: LeetCode-main/solutions/1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance/1334.cpp

```
class Solution {
public:
    int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {
        int ans = -1;
        int minCitiesCount = n;
        const vector<vector<int>> dist = floydWarshall(n, edges, distanceThreshold);

        for (int i = 0; i < n; ++i) {
            int citiesCount = 0;
            for (int j = 0; j < n; ++j)
                if (dist[i][j] <= distanceThreshold)
                    ++citiesCount;
            if (citiesCount <= minCitiesCount) {
                ans = i;
                minCitiesCount = citiesCount;
            }
        }

        return ans;
    }

private:
    vector<vector<int>> floydWarshall(int n, const vector<vector<int>>& edges,
                                     int distanceThreshold) {
        vector<vector<int>> dist(n, vector<int>(n, distanceThreshold + 1));

        for (int i = 0; i < n; ++i)
            dist[i][i] = 0;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int w = edge[2];
            dist[u][v] = w;
            dist[v][u] = w;
        }

        for (int k = 0; k < n; ++k)
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < n; ++j)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

        return dist;
    }
};
```

365. 1335.cpp

Path: LeetCode-main/solutions/1335. Minimum Difficulty of a Job Schedule/1335.cpp

```
class Solution {
public:
    int minDifficulty(vector<int>& jobDifficulty, int d) {
        const int n = jobDifficulty.size();
        if (n < d)
            return -1;

        // dp[i][k] := the minimum difficulty to schedule the first i jobs in k days
        vector<vector<int>> dp(n + 1, vector<int>(d + 1, INT_MAX / 2));
        dp[0][0] = 0;

        for (int i = 1; i <= n; ++i)
            for (int k = 1; k <= d; ++k) {
                int maxDifficulty = 0; // max(job[j + 1..i])
                for (int j = i - 1; j >= k - 1; --j) { // 1-based
                    maxDifficulty = max(maxDifficulty, jobDifficulty[j]); // 0-based
                    dp[i][k] = min(dp[i][k], dp[j][k - 1] + maxDifficulty);
                }
            }

        return dp[n][d];
    }
};
```

366. 1337.cpp

Path: LeetCode-main/solutions/1337. The K Weakest Rows in a Matrix/1337.cpp

```
class Solution {
public:
    vector<int> kWeakestRows(vector<vector<int>>& mat, int k) {
        vector<int> ans;
        vector<pair<int, int>> rowSums; // (sum(mat[i]), i)

        for (int i = 0; i < mat.size(); ++i)
            rowSums.emplace_back(accumulate(mat[i].begin(), mat[i].end(), 0), i);

        ranges::sort(rowSums, ranges::less{},
                    [](const pair<int, int>& rowSum) { return rowSum; });

        for (int i = 0; i < k; ++i)
            ans.push_back(rowSums[i].second);

        return ans;
    }
};
```

367. 1338.cpp

Path: LeetCode-main/solutions/1338. Reduce Array Size to The Half/1338.cpp

```
class Solution {
public:
    int minSetSize(vector<int>& arr) {
        const int n = arr.size();
        int sum = 0;
        unordered_map<int, int> count;
        vector<pair<int, int>> numAndFreqs;

        for (const int a : arr)
            ++count[a];

        for (const auto& [a, freq] : count)
            numAndFreqs.emplace_back(a, freq);

        ranges::sort(
            numAndFreqs, ranges::greater{},
            [](const pair<int, int>& numAndFreq) { return numAndFreq.second; });

        for (int i = 0; i < numAndFreqs.size(); ++i) {
            sum += numAndFreqs[i].second;
            if (sum >= n / 2)
                return i + 1;
        }

        throw;
    }
};
```

368. 1339.cpp

Path: LeetCode-main/solutions/1339. Maximum Product of Splitted Binary Tree/1339.cpp

```
class Solution {
public:
    int maxProduct(TreeNode* root) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        vector<int> allSums;
        const long totalSum = treeSum(root, allSums);

        for (const long sum : allSums)
            ans = max(ans, sum * (totalSum - sum));

        return ans % kMod;
    }

private:
    int treeSum(TreeNode* root, vector<int>& allSums) {
        if (root == nullptr)
            return 0;

        const int leftSum = treeSum(root->left, allSums);
        const int rightSum = treeSum(root->right, allSums);
        const int sum = root->val + leftSum + rightSum;
        allSums.push_back(sum);
        return sum;
    }
};
```

369. 134.cpp

Path: LeetCode-main/solutions/134. Gas Station/134.cpp

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        const int gasSum = accumulate(gas.begin(), gas.end(), 0);
        const int costSum = accumulate(cost.begin(), cost.end(), 0);
        if (gasSum - costSum < 0)
            return -1;

        int ans = 0;
        int sum = 0;

        // Try to start from each index.
        for (int i = 0; i < gas.size(); ++i) {
            sum += gas[i] - cost[i];
            if (sum < 0) {
                sum = 0;
                ans = i + 1; // Start from the next index.
            }
        }

        return ans;
    }
};
```

370. 1340.cpp

Path: LeetCode-main/solutions/1340. Jump Game V/1340.cpp

```
class Solution {
public:
    int maxJumps(vector<int>& arr, int d) {
        const int n = arr.size();
        // dp[i] := the maximum jumps starting from arr[i]
        vector<int> dp(n, 1);
        // a decreasing stack that stores indices
        stack<int> stack;

        for (int i = 0; i <= n; ++i) {
            while (!stack.empty() && (i == n || arr[stack.top()] < arr[i])) {
                vector<int> indices{stack.top()};
                stack.pop();
                while (!stack.empty() && arr[stack.top()] == arr[indices[0]])
                    indices.push_back(stack.top()), stack.pop();
                for (const int j : indices) {
                    if (i < n && i - j <= d)
                        // Can jump from i to j.
                        dp[i] = max(dp[i], dp[j] + 1);
                    if (!stack.empty() && j - stack.top() <= d)
                        // Can jump from stack[-1] to j.
                        dp[stack.top()] = max(dp[stack.top()], dp[j] + 1);
                }
            }
            stack.push(i);
        }

        return ranges::max(dp);
    }
};
```

371. 1342.cpp

Path: LeetCode-main/solutions/1342. Number of Steps to Reduce a Number to Zero/1342.cpp

```
class Solution {
public:
    int numberOfSteps(unsigned num) {
        if (num == 0)
            return 0;
        const int subtractSteps = popcount(num);
        const int divideSteps = 31 - __builtin_clz(num);
        return subtractSteps + divideSteps;
    }
};
```


372. 1343.cpp

Path: LeetCode-main/solutions/1343. Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold/1343.cpp

```
class Solution {
public:
    int numOfSubarrays(vector<int>& arr, int k, int threshold) {
        int ans = 0;
        int windowSum = 0;

        for (int i = 0; i < arr.size(); ++i) {
            windowSum += arr[i];
            if (i >= k)
                windowSum -= arr[i - k];
            if (i >= k - 1 && windowSum / k >= threshold)
                ++ans;
        }

        return ans;
    }
};
```

373. 1344.cpp

Path: LeetCode-main/solutions/1344. Angle Between Hands of a Clock/1344.cpp

```
class Solution {
public:
    double angleClock(int hour, int minutes) {
        const double hourHand = (hour % 12 + minutes / 60.0) * 30;
        const double minuteHand = minutes * 6;
        const double diff = abs(hourHand - minuteHand);
        return min(diff, 360 - diff);
    }
};
```

374. 1345.cpp

Path: LeetCode-main/solutions/1345. Jump Game IV/1345.cpp

```
class Solution {
public:
    int minJumps(vector<int>& arr) {
        const int n = arr.size();
        // {a: indices}
        unordered_map<int, vector<int>>> graph;
        queue<int> q{{0}};
        vector<bool> seen(n);
        seen[0] = true;

        for (int i = 0; i < n; ++i)
            graph[arr[i]].push_back(i);

        for (int step = 0; !q.empty(); ++step) {
            for (int sz = q.size(); sz > 0; --sz) {
                const int i = q.front();
                q.pop();
                if (i == n - 1)
                    return step;
                seen[i] = true;
                const int u = arr[i];
                if (i + 1 < n)
                    graph[u].push_back(i + 1);
                if (i - 1 >= 0)
                    graph[u].push_back(i - 1);
                for (const int v : graph[u]) {
                    if (seen[v])
                        continue;
                    q.push(v);
                }
                graph[u].clear();
            }
        }
        throw;
    }
};
```

375. 1346.cpp

Path: LeetCode-main/solutions/1346. Check If N and Its Double Exist/1346.cpp

```
class Solution {
public:
    bool checkIfExist(vector<int>& arr) {
        unordered_set<int> seen;

        for (const int a : arr) {
            if (seen.contains(a * 2) || a % 2 == 0 && seen.contains(a / 2))
                return true;
            seen.insert(a);
        }

        return false;
    }
};
```

376. 1347.cpp

Path: LeetCode-main/solutions/1347. Minimum Number of Steps to Make Two Strings Anagram/1347.cpp

```
class Solution {
public:
    int minSteps(string s, string t) {
        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        for (const char c : t)
            --count[c - 'a'];

        return accumulate(count.begin(), count.end(), 0, [](int acc, int c) {
            return acc + abs(c);
        }) / 2;
    }
};
```

377. 1348.cpp

Path: LeetCode-main/solutions/1348. Tweet Counts Per Frequency/1348.cpp

```
class TweetCounts {
public:
    void recordTweet(string tweetName, int time) {
        ++tweetNameToTimeCount[tweetName][time];
    }

    vector<int> getTweetCountsPerFrequency(string freq, string tweetName,
                                         int startTime, int endTime) {
        const int chunkSize = freq == "minute" ? 60 : freq == "hour" ? 3600 : 86400;
        vector<int> counts((endTime - startTime) / chunkSize + 1);
        const map<int, int>& timeCount = tweetNameToTimeCount[tweetName];
        const auto lo = timeCount.lower_bound(startTime);
        const auto hi = timeCount.upper_bound(endTime);

        for (auto it = lo; it != hi; ++it) {
            const int index = (it->first - startTime) / chunkSize;
            counts[index] += it->second;
        }

        return counts;
    }

private:
    unordered_map<string, map<int, int>> tweetNameToTimeCount;
};
```

378. 1349.cpp

Path: LeetCode-main/solutions/1349. Maximum Students Taking Exam/1349.cpp

```
class Solution {
public:
    int maxStudents(vector<vector<char>>& seats) {
        return accumulate(seats.begin(), seats.end(), 0,
            [&](int acc, const auto& seat) {
                return acc + ranges::count(seat, '.');
            }) - hungarian(seats);
    }

private:
    static constexpr int kDirs[6][2] = {{-1, -1}, {0, -1}, {1, -1},
                                          {-1, 1}, {0, 1}, {1, 1}};

    int hungarian(const vector<vector<char>>& seats) {
        const int m = seats.size();
        const int n = seats[0].size();
        int count = 0;
        vector<vector<int>> seen(m, vector<int>(n));
        vector<vector<int>> match(m, vector<int>(n, -1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (seats[i][j] == '.' && match[i][j] == -1) {
                    const int sessionId = i * n + j;
                    seen[i][j] = sessionId;
                    count += dfs(seats, i, j, sessionId, seen, match);
                }

        return count;
    }

    int dfs(const vector<vector<char>>& seats, int i, int j, int sessionId,
            vector<vector<int>>& seen, vector<vector<int>>& match) {
        const int m = seats.size();
        const int n = seats[0].size();

        for (const auto& [dx, dy] : kDirs) {
            const int x = i + dx;
            const int y = j + dy;
            if (x < 0 || x == m || y < 0 || y == n)
                continue;
            if (seats[x][y] != '.' || seen[x][y] == sessionId)
                continue;
            seen[x][y] = sessionId;
            if (match[x][y] == -1 || dfs(seats, match[x][y] / n, match[x][y] % n,
                                        sessionId, seen, match)) {
                match[x][y] = i * n + j;
                match[i][j] = x * n + y;
                return 1;
            }
        }

        return 0;
    }
};
```

379. 135.cpp

Path: LeetCode-main/solutions/135. Candy/135.cpp

```
class Solution {
public:
    int candy(vector<int>& ratings) {
        const int n = ratings.size();
        int ans = 0;
        vector<int> l(n, 1);
        vector<int> r(n, 1);

        for (int i = 1; i < n; ++i)
            if (ratings[i] > ratings[i - 1])
                l[i] = l[i - 1] + 1;

        for (int i = n - 2; i >= 0; --i)
            if (ratings[i] > ratings[i + 1])
                r[i] = r[i + 1] + 1;

        for (int i = 0; i < n; ++i)
            ans += max(l[i], r[i]);

        return ans;
    }
};
```


380. 1351.cpp

Path: LeetCode-main/solutions/1351. Count Negative Numbers in a Sorted Matrix/1351.cpp

```
class Solution {
public:
    int countNegatives(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        int ans = 0;
        int i = m - 1;
        int j = 0;

        while (i >= 0 && j < n) {
            if (grid[i][j] < 0) {
                ans += n - j;
                --i;
            } else {
                ++j;
            }
        }

        return ans;
    }
};
```

381. 1352.cpp

Path: LeetCode-main/solutions/1352. Product of the Last K Numbers/1352.cpp

```
class ProductOfNumbers {
public:
    ProductOfNumbers() : prefix{1} {}

    void add(int num) {
        if (num == 0)
            prefix = {1};
        else
            prefix.push_back(prefix.back() * num);
    }

    int getProduct(int k) {
        return k >= prefix.size() ? 0
            : prefix.back() / prefix[prefix.size() - k - 1];
    }

private:
    vector<int> prefix;
};
```

382. 1353.cpp

Path: LeetCode-main/solutions/1353. Maximum Number of Events That Can Be Attended/1353.cpp

```
class Solution {
public:
    int maxEvents(vector<vector<int>>& events) {
        int ans = 0;
        int d = 0; // the current day
        int i = 0; // events' index
        priority_queue<int, vector<int>, greater<>> minHeap;

        ranges::sort(events);

        while (!minHeap.empty() || i < events.size()) {
            // If no events are available to attend today, let time flies to the next
            // available event.
            if (minHeap.empty())
                d = events[i][0];
            // All the events starting from today are newly available.
            while (i < events.size() && events[i][0] == d)
                minHeap.push(events[i++][1]);
            // Greedily attend the event that'll end the earliest since it has higher
            // chance can't be attended in the future.
            minHeap.pop();
            ++ans;
            ++d;
            // Pop the events that can't be attended.
            while (!minHeap.empty() && minHeap.top() < d)
                minHeap.pop();
        }

        return ans;
    }
};
```

383. 1354.cpp

Path: LeetCode-main/solutions/1354. Construct Target Array With Multiple Sums/1354.cpp

```
class Solution {
public:
    bool isPossible(vector<int>& target) {
        if (target.size() == 1)
            return target[0] == 1;

        long sum = accumulate(target.begin(), target.end(), 0L);
        priority_queue<int> maxHeap;

        for (const int num : target)
            maxHeap.push(num);

        while (maxHeap.top() > 1) {
            const long mx = maxHeap.top();
            maxHeap.pop();
            const long restSum = sum - mx;
            // Only occurs if n == 2.
            if (restSum == 1)
                return true;
            const long updated = mx % restSum;
            // updated == 0 (invalid) or didn't change.
            if (updated == 0 || updated == mx)
                return false;
            maxHeap.push(updated);
            sum = sum - mx + updated;
        }

        return true;
    }
};
```

384. 1356.cpp

Path: LeetCode-main/solutions/1356. Sort Integers by The Number of 1 Bits/1356.cpp

```
class Solution {
public:
    vector<int> sortByBits(vector<int>& arr) {
        ranges::sort(arr, ranges::less{}, [](const int a) {
            const int bitCount = bitset<32>(a).count();
            return pair<int, int>{bitCount, a};
        });
        return arr;
    }
};
```

385. 1357.cpp

Path: LeetCode-main/solutions/1357. Apply Discount Every n Orders/1357.cpp

```
class Cashier {
public:
    Cashier(int n, int discount, vector<int>& products, vector<int>& prices)
        : n(n), discount(discount) {
        for (int i = 0; i < products.size(); ++i)
            productToPrice[products[i]] = prices[i];
    }

    double getBill(vector<int> product, vector<int> amount) {
        ++count;
        int total = 0;
        for (int i = 0; i < product.size(); ++i)
            total += productToPrice[product[i]] * amount[i];
        return count % n == 0 ? total * (1 - discount / 100.0) : total;
    }

private:
    const int n;
    const int discount;
    unordered_map<int, int> productToPrice;
    int count = 0;
};
```

386. 1358-2.cpp

Path: LeetCode-main/solutions/1358. Number of Substrings Containing All Three Characters/1358-2.cpp

```
class Solution {
public:
    // Similar to 3. Longest Substring Without Repeating Characters
    int numberOfSubstrings(string s) {
        int ans = 0;
        // lastSeen[c] := the index of the last time c appeared
        vector<int> lastSeen(3, -1);

        for (int i = 0; i < s.length(); ++i) {
            lastSeen[s[i] - 'a'] = i;
            // s[0..i], s[1..i], s[min(lastSeen)..i] are satisfied strings.
            ans += 1 + ranges::min(lastSeen);
        }

        return ans;
    }
};
```

387. 1358.cpp

Path: LeetCode-main/solutions/1358. Number of Substrings Containing All Three Characters/1358.cpp

```
class Solution {
public:
    // Similar to 3. Longest Substring Without Repeating Characters
    int numberOfSubstrings(string s) {
        int ans = 0;
        vector<int> count(3);

        int l = 0;
        for (const char c : s) {
            ++count[c - 'a'];
            while (count[0] > 0 && count[1] > 0 && count[2] > 0)
                --count[s[l++] - 'a'];
            // s[0..r], s[1..r], ..., s[l - 1..r] are satisfied strings.
            ans += l;
        }

        return ans;
    }
};
```


388. 1359.cpp

Path: LeetCode-main/solutions/1359. Count All Valid Pickup and Delivery Options/1359.cpp

```
class Solution {
public:
    int countOrders(int n) {
        constexpr int kMod = 1'000'000'007;
        long ans = 1;

        for (int i = 1; i <= n; ++i)
            ans = ans * i * (i * 2 - 1) % kMod;

        return ans;
    }
};
```

389. 136.cpp

Path: LeetCode-main/solutions/136. Single Number/136.cpp

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans = 0;

        for (const int num : nums)
            ans ^= num;

        return ans;
    }
};
```

390. 1360.cpp

Path: LeetCode-main/solutions/1360. Number of Days Between Two Dates/1360.cpp

```
class Solution {
public:
    int daysBetweenDates(string date1, string date2) {
        return abs(daysFrom1971(date1) - daysFrom1971(date2));
    }

private:
    const vector<int> days{0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    int daysFrom1971(const string& date) {
        const int year = stoi(date.substr(0, 4));
        const int month = stoi(date.substr(5, 2));
        const int day = stoi(date.substr(8, 2));
        int res = 0;
        for (int i = 1971; i < year; ++i)
            res += isLeapYear(i) ? 366 : 365;
        for (int i = 0; i < month; ++i)
            res += days[i];
        if (month > 2 && isLeapYear(year))
            ++res;
        res += day;
        return res;
    }

    bool isLeapYear(int year) {
        return year % 4 == 0 && year % 100 != 0 || year % 400 == 0;
    };
};
```

391. 1361.cpp

Path: LeetCode-main/solutions/1361. Validate Binary Tree Nodes/1361.cpp

```
class Solution {
public:
    bool validateBinaryTreeNodes(int n, vector<int>& leftChild,
                                vector<int>& rightChild) {
        vector<int> inDegrees(n);
        int root = -1;

        // If the in-degree of any node > 1, return false.
        for (const int child : leftChild)
            if (child != -1 && ++inDegrees[child] == 2)
                return false;

        for (const int child : rightChild)
            if (child != -1 && ++inDegrees[child] == 2)
                return false;

        // Find the root (the node with in-degree == 0).
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                if (root == -1)
                    root = i;
                else
                    return false; // There're multiple roots.

        // Didn't find the root.
        if (root == -1)
            return false;

        return countNodes(root, leftChild, rightChild) == n;
    }

private:
    int countNodes(int root, const vector<int>& leftChild,
                  const vector<int>& rightChild) {
        if (root == -1)
            return 0;
        return 1 + //
            countNodes(leftChild[root], leftChild, rightChild) +
            countNodes(rightChild[root], leftChild, rightChild);
    }
};
```

392. 1362.cpp

Path: LeetCode-main/solutions/1362. Closest Divisors/1362.cpp

```
class Solution {
public:
    vector<int> closestDivisors(int num) {
        for (int root = sqrt(num + 2); root > 0; --root)
            for (int cand : {num + 1, num + 2})
                if (cand % root == 0)
                    return {root, cand / root};

        throw;
    }
};
```

393. 1363.cpp

Path: LeetCode-main/solutions/1363. Largest Multiple of Three/1363.cpp

```
class Solution {
public:
    string largestMultipleOfThree(vector<int>& digits) {
        string ans;
        vector<int> mod1{1, 4, 7, 2, 5, 8};
        vector<int> mod2{2, 5, 8, 1, 4, 7};
        vector<int> count(10);
        int sum = accumulate(digits.begin(), digits.end(), 0);

        for (const int digit : digits)
            ++count[digit];

        while (sum % 3 != 0)
            for (int i : sum % 3 == 1 ? mod1 : mod2)
                if (count[i]) {
                    --count[i];
                    sum -= i;
                    break;
                }

        for (int digit = 9; digit >= 0; --digit)
            ans += string(count[digit], '0' + digit);

        return ans.size() && ans[0] == '0' ? "0" : ans;
    }
};
```

394. 1365.cpp

Path: LeetCode-main/solutions/1365. How Many Numbers Are Smaller Than the Current Number/1365.cpp

```
class Solution {
public:
    vector<int> smallerNumbersThanCurrent(vector<int>& nums) {
        constexpr int kMax = 100;
        vector<int> ans;
        vector<int> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        for (int i = 1; i <= kMax; ++i)
            count[i] += count[i - 1];

        for (const int num : nums)
            ans.push_back(num == 0 ? 0 : count[num - 1]);

        return ans;
    }
};
```

395. 1366.cpp

Path: LeetCode-main/solutions/1366. Rank Teams by Votes/1366.cpp

```
struct Team {
    char name;
    vector<int> rank;
    Team(char name, int teamSize) : name(name), rank(teamSize) {}
};

class Solution {
public:
    string rankTeams(vector<string>& votes) {
        const int teamSize = votes[0].size();
        string ans;
        vector<Team> teams;

        for (int i = 0; i < 26; ++i)
            teams.push_back(Team('A' + i, teamSize));

        for (const string& vote : votes)
            for (int i = 0; i < teamSize; ++i)
                ++teams[vote[i] - 'A'].rank[i];

        ranges::sort(teams, ranges::greater{}, [](const Team& team) {
            return pair<vector<int>, char>{team.rank, -team.name};
        });

        for (int i = 0; i < teamSize; ++i)
            ans += teams[i].name;

        return ans;
    }
};
```


396. 1367.cpp

Path: LeetCode-main/solutions/1367. Linked List in Binary Tree/1367.cpp

```
class Solution {
public:
    bool isSubPath(ListNode* head, TreeNode* root) {
        if (root == nullptr)
            return false;
        return isContinuousSubPath(head, root) || isSubPath(head, root->left) ||
            isSubPath(head, root->right);
    }

private:
    bool isContinuousSubPath(ListNode* head, TreeNode* root) {
        if (head == nullptr)
            return true;
        if (root == nullptr)
            return false;
        return head->val == root->val &&
            (isContinuousSubPath(head->next, root->left) ||
             isContinuousSubPath(head->next, root->right));
    }
};
```

397. 1368.cpp

Path: LeetCode-main/solutions/1368. Minimum Cost to Make at Least One Valid Path in a Grid/1368.cpp

```
class Solution {
public:
    int minCost(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<int>> mem(m, vector<int>(n, -1));
        queue<pair<int, int>> q;

        dfs(grid, 0, 0, /*cost=*/0, q, mem);

        for (int cost = 1; !q.empty(); ++cost)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs)
                    dfs(grid, i + dx, j + dy, cost, q, mem);
            }

        return mem.back().back();
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    void dfs(const vector<vector<int>>& grid, int i, int j, int cost,
             queue<pair<int, int>>& q, vector<vector<int>>& mem) {
        if (i < 0 || i == grid.size() || j < 0 || j == grid[0].size())
            return;
        if (mem[i][j] != -1)
            return;

        mem[i][j] = cost;
        q.emplace(i, j);
        const auto& [dx, dy] = kDirs[grid[i][j] - 1];
        dfs(grid, i + dx, j + dy, cost, q, mem);
    }
};
```

398. 137-2.cpp

Path: LeetCode-main/solutions/137. Single Number II/137-2.cpp

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ones = 0;
        int twos = 0;

        for (const int num : nums) {
            ones ^= (num & ~twos);
            twos ^= (num & ~ones);
        }

        return ones;
    }
};
```

399. 137.cpp

Path: LeetCode-main/solutions/137. Single Number II/137.cpp

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans = 0;

        for (int i = 0; i < 32; ++i) {
            int sum = 0;
            for (const int num : nums)
                sum += num >> i & 1;
            sum %= 3;
            ans |= sum << i;
        }

        return ans;
    }
};
```

400. 1370.cpp

Path: LeetCode-main/solutions/1370. Increasing Decreasing String/1370.cpp

```
class Solution {
public:
    string sortString(string s) {
        string ans;
        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        while (ans.length() < s.size()) {
            for (int i = 0; i < 26; ++i) {
                if (count[i] == 0)
                    continue;
                ans += 'a' + i;
                --count[i];
            }
            for (int i = 25; i >= 0; --i) {
                if (count[i] == 0)
                    continue;
                ans += 'a' + i;
                --count[i];
            }
        }
        return ans;
    }
};
```

401. 1371.cpp

Path: LeetCode-main/solutions/1371. Find the Longest Substring Containing Vowels in Even Counts/1371.cpp

```
class Solution {
public:
    int findTheLongestSubstring(string s) {
        constexpr string_view kVowels = "aeiou";
        int ans = 0;
        int prefix = 0; // the binary prefix
        unordered_map<int, int> prefixToIndex{{0, -1}};

        for (int i = 0; i < s.length(); ++i) {
            const int index = kVowels.find(s[i]);
            if (index != -1)
                prefix ^= 1 << index;
            if (!prefixToIndex.contains(prefix))
                prefixToIndex[prefix] = i;
            ans = max(ans, i - prefixToIndex[prefix]);
        }

        return ans;
    }
};
```

402. 1372.cpp

Path: LeetCode-main/solutions/1372. Longest ZigZag Path in a Binary Tree/1372.cpp

```
struct T {
    int leftMax;
    int rightMax;
    int subtreeMax;
};

class Solution {
public:
    int longestZigZag(TreeNode* root) {
        return dfs(root).subtreeMax;
    }

private:
    T dfs(TreeNode* root) {
        if (root == nullptr)
            return {-1, -1, -1};
        const T left = dfs(root->left);
        const T right = dfs(root->right);
        const int leftZigZag = left.rightMax + 1;
        const int rightZigZag = right.leftMax + 1;
        const int subtreeMax =
            max({leftZigZag, rightZigZag, left.subtreeMax, right.subtreeMax});
        return {leftZigZag, rightZigZag, subtreeMax};
    }
};
```

403. 1373.cpp

Path: LeetCode-main/solutions/1373. Maximum Sum BST in Binary Tree/1373.cpp

```
struct T {
    bool isBST = false;
    int mx;
    int mn;
    int sum;
};

class Solution {
public:
    int maxSumBST(TreeNode* root) {
        int ans = 0;
        traverse(root, ans);
        return ans;
    }

private:
    T traverse(TreeNode* root, int& ans) {
        if (root == nullptr)
            return T(true, INT_MIN, INT_MAX, 0);

        const T left = traverse(root->left, ans);
        const T right = traverse(root->right, ans);

        if (!left.isBST || !right.isBST)
            return T();
        if (root->val <= left.mx || root->val >= right.mn)
            return T();

        // The `root` is a valid BST.
        const int sum = root->val + left.sum + right.sum;
        ans = max(ans, sum);
        return T(true, max(root->val, right.mx), min(root->val, left.mn), sum);
    }
};
```


404. 1374.cpp

Path: LeetCode-main/solutions/1374. Generate a String With Characters That Have Odd Counts/1374.cpp

```
class Solution {
public:
    string generateTheString(int n) {
        string s(n, 'a');
        if (n % 2 == 0)
            s.back() = 'b';
        return s;
    }
};
```

405. 1375.cpp

Path: LeetCode-main/solutions/1375. Number of Times Binary String Is Prefix-Aligned/1375.cpp

```
class Solution {
public:
    int numTimesAllBlue(vector<int>& flips) {
        int ans = 0;
        int rightmost = 0;

        for (int i = 0; i < flips.size(); ++i) {
            rightmost = max(rightmost, flips[i]);
            // max(flips[0..i]) = rightmost = i + 1,
            // so flips[0..i] is a permutation of 1, 2, ..., i + 1.
            if (rightmost == i + 1)
                ++ans;
        }

        return ans;
    }
};
```

406. 1376.cpp

Path: LeetCode-main/solutions/1376. Time Needed to Inform All Employees/1376.cpp

```
class Solution {
public:
    int numOfMinutes(int n, int headID, vector<int>& manager,
                     vector<int>& informTime) {
        int ans = 0;

        for (int i = 0; i < n; ++i)
            ans = max(ans, dfs(i, headID, manager, informTime, {}));

        return ans;
    }

private:
    int dfs(int i, int headID, const vector<int>& manager,
            const vector<int>& informTime, unordered_map<int, int>&& mem) {
        if (const auto it = mem.find(i); it != mem.cend())
            return it->second;
        if (i == headID)
            return 0;

        const int parent = manager[i];
        return mem[i] = informTime[parent] +
            dfs(parent, headID, manager, informTime, std::move(mem));
    }
};
```

407. 1377.cpp

Path: LeetCode-main/solutions/1377. Frog Position After T Seconds/1377.cpp

```
class Solution {
public:
    double frogPosition(int n, vector<vector<int>>& edges, int t, int target) {
        vector<vector<int>> tree(n + 1);
        queue<int> q{{1}};
        vector<bool> seen(n + 1);
        vector<double> prob(n + 1);

        seen[1] = true;
        prob[1] = 1.0;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        while (!q.empty() && t-- > 0)
            for (int sz = q.size(); sz > 0; --sz) {
                const int a = q.front();
                q.pop();
                const int nChildren =
                    ranges::count_if(tree[a], [&seen](int b) { return !seen[b]; });
                for (const int b : tree[a]) {
                    if (seen[b])
                        continue;
                    seen[b] = true;
                    prob[b] = prob[a] / nChildren;
                    q.push(b);
                }
                if (nChildren > 0)
                    prob[a] = 0.0;
            }

        return prob[target];
    }
};
```

408. 1379.cpp

Path: LeetCode-main/solutions/1379. Find a Corresponding Node of a Binary Tree in a Clone of That Tree/1379.cpp

```
class Solution {
public:
    TreeNode* getTargetCopy(TreeNode* original, TreeNode* cloned,
                           TreeNode* target) {
        TreeNode* ans = nullptr;
        dfs(original, cloned, target, ans);
        return ans;
    }

private:
    void dfs(TreeNode* original, TreeNode* cloned, TreeNode* target,
             TreeNode*& ans) {
        if (ans != nullptr)
            return;
        if (original == nullptr)
            return;
        if (original == target) {
            ans = cloned;
            return;
        }
        dfs(original->left, cloned->left, target, ans);
        dfs(original->right, cloned->right, target, ans);
    }
};
```

409. 138.cpp

Path: LeetCode-main/solutions/138. Copy List with Random Pointer/138.cpp

```
class Solution {
public:
    Node* copyRandomList(Node* head) {
        if (head == nullptr)
            return nullptr;
        if (const auto it = map.find(head); it != map.cend())
            return it->second;

        Node* newNode = new Node(head->val);
        map[head] = newNode;
        newNode->next = copyRandomList(head->next);
        newNode->random = copyRandomList(head->random);
        return newNode;
    }

private:
    unordered_map<Node*, Node*> map;
};
```

410. 1380.cpp

Path: LeetCode-main/solutions/1380. Lucky Numbers in a Matrix/1380.cpp

```
class Solution {
public:
    vector<int> luckyNumbers(vector<vector<int>>& matrix) {
        for (const vector<int>& row : matrix) {
            const int minIndex = distance(row.begin(), ranges::min_element(row));
            if (row[minIndex] == maxNumOfColumn(matrix, minIndex))
                return {row[minIndex]};
        }
        return {};
    }

private:
    int maxNumOfColumn(const vector<vector<int>>& matrix, int j) {
        int res = 0;
        for (int i = 0; i < matrix.size(); ++i)
            res = max(res, matrix[i][j]);
        return res;
    }
};
```

411. 1381.cpp

Path: LeetCode-main/solutions/1381. Design a Stack With Increment Operation/1381.cpp

```
class CustomStack {
public:
    CustomStack(int maxSize) : maxSize(maxSize) {}

    void push(int x) {
        if (stack.size() == maxSize)
            return;
        stack.push(x);
        pendingIncrements.push_back(0);
    }

    int pop() {
        if (stack.empty())
            return -1;
        const int i = stack.size() - 1;
        if (i > 0)
            pendingIncrements[i - 1] += pendingIncrements[i];
        const int val = stack.top() + pendingIncrements[i];
        stack.pop();
        pendingIncrements.pop_back();
        return val;
    }

    void increment(int k, int val) {
        if (stack.empty())
            return;
        const int i = min(k - 1, static_cast<int>(stack.size()) - 1);
        pendingIncrements[i] += val;
    }

private:
    const int maxSize;
    stack<int> stack;
    // pendingIncrements[i] := the pending increment for stack[0..i].
    vector<int> pendingIncrements;
};
```


412. 1382.cpp

Path: LeetCode-main/solutions/1382. Balance a Binary Search Tree/1382.cpp

```
class Solution {
public:
    TreeNode* balanceBST(TreeNode* root) {
        vector<int> nums;
        inorder(root, nums);
        return build(nums, 0, nums.size() - 1);
    }

private:
    void inorder(TreeNode* root, vector<int>& nums) {
        if (root == nullptr)
            return;
        inorder(root->left, nums);
        nums.push_back(root->val);
        inorder(root->right, nums);
    }

    // Same as 108. Convert Sorted Array to Binary Search Tree
    TreeNode* build(const vector<int>& nums, int l, int r) {
        if (l > r)
            return nullptr;
        const int m = (l + r) / 2;
        return new TreeNode(nums[m], build(nums, l, m - 1), build(nums, m + 1, r));
    }
};
```

413. 1383.cpp

Path: LeetCode-main/solutions/1383. Maximum Performance of a Team/1383.cpp

```
class Solution {
public:
    // Similar to 857. Minimum Cost to Hire K Workers
    int maxPerformance(int n, vector<int>& speed, vector<int>& efficiency,
                      int k) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        long speedSum = 0;
        // (efficiency[i], speed[i]) sorted by efficiency[i] in descending order
        vector<pair<int, int>> A;
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (int i = 0; i < n; ++i)
            A.emplace_back(efficiency[i], speed[i]);

        ranges::sort(A, greater<>());

        for (const auto& [e, s] : A) {
            minHeap.push(s);
            speedSum += s;
            if (minHeap.size() > k)
                speedSum -= minHeap.top(), minHeap.pop();
            ans = max(ans, speedSum * e);
        }

        return ans % kMod;
    }
};
```

414. 1385-2.cpp

Path: LeetCode-main/solutions/1385. Find the Distance Value Between Two Arrays/1385-2.cpp

```
class Solution {
public:
    int findTheDistanceValue(vector<int>& arr1, vector<int>& arr2, int d) {
        int ans = 0;

        ranges::sort(arr2);

        for (const int a : arr1)
            if (ranges::upper_bound(arr2, a + d) == ranges::lower_bound(arr2, a - d))
                ++ans;

        return ans;
    }
};
```

415. 1385.cpp

Path: LeetCode-main/solutions/1385. Find the Distance Value Between Two Arrays/1385.cpp

```
class Solution {
public:
    int findTheDistanceValue(vector<int>& arr1, vector<int>& arr2, int d) {
        int ans = 0;

        ranges::sort(arr2);

        for (const int a : arr1) {
            const auto it = lower_bound(begin(arr2), end(arr2), a);
            if ((it == arr2.end() || *it - a > d) &&
                (it == arr2.begin() || a - *prev(it) > d))
                ++ans;
        }

        return ans;
    }
};
```

416. 1386.cpp

Path: LeetCode-main/solutions/1386. Cinema Seat Allocation/1386.cpp

```
class Solution {
public:
    int maxNumberOfFamilies(int n, vector<vector<int>>& reservedSeats) {
        int ans = 0;
        unordered_map<int, int> rowToSeats;

        for (const vector<int>& reservedSeat : reservedSeats) {
            const int row = reservedSeat[0];
            const int seat = reservedSeat[1];
            rowToSeats[row] |= 1 << (seat - 1);
        }

        for (const auto& [_ , seats] : rowToSeats)
            if ((seats & 0b0111111110) == 0)
                // Can fit 2 four-person groups.
                ans += 2;
            else if ((seats & 0b0111100000) == 0 || // The left is not occupied.
                    (seats & 0b0001111000) == 0 || // The middle is not occupied.
                    (seats & 0b0000011110) == 0) // The right is not occupied.
                // Can fit 1 four-person group.
                ans += 1;

        // Any empty row can fit 2 four-person groups.
        return ans + (n - rowToSeats.size()) * 2;
    }
};
```

417. 1387.cpp

Path: LeetCode-main/solutions/1387. Sort Integers by The Power Value/1387.cpp

```
class Solution {
public:
    int getKth(int lo, int hi, int k) {
        vector<pair<int, int>> powAndVals; // (pow, val)

        for (int i = lo; i <= hi; ++i)
            powAndVals.emplace_back(getPow(i), i);

        nth_element(powAndVals.begin(), powAndVals.begin() + k - 1,
                    powAndVals.end());
        return powAndVals[k - 1].second;
    }

private:
    int getPow(int n) {
        if (n == 1)
            return 0;
        return 1 + (n % 2 == 0 ? getPow(n / 2) : getPow(n * 3 + 1));
    }
};
```

418. 1389.cpp

Path: LeetCode-main/solutions/1389. Create Target Array in the Given Order/1389.cpp

```
class Solution {
public:
    vector<int> createTargetArray(vector<int>& nums, vector<int>& index) {
        vector<int> ans;
        for (int i = 0; i < nums.size(); ++i)
            ans.insert(ans.begin() + index[i], nums[i]);
        return ans;
    }
};
```

419. 139-2.cpp

Path: LeetCode-main/solutions/139. Word Break/139-2.cpp

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        return wordBreak(s, 0, {wordDict.begin(), wordDict.end()},
                        vector<int>(s.length(), -1));
    }

private:
    // Returns true if s[i..n) can be segmented.
    bool wordBreak(const string& s, int i, const unordered_set<string>&& wordSet,
                  vector<int>&& mem) {
        if (i == s.length())
            return true;
        if (mem[i] != -1)
            return mem[i];

        for (int j = i + 1; j <= s.length(); ++j)
            if (wordSet.contains(s.substr(i, j - i)) &&
                wordBreak(s, j, std::move(wordSet), std::move(mem)))
                return mem[i] = 1;

        return mem[i] = 0;
    }
};
```


420. 139-3.cpp

Path: LeetCode-main/solutions/139. Word Break/139-3.cpp

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        const int n = s.length();
        const unordered_set<string> wordSet{wordDict.begin(), wordDict.end()};
        // dp[i] := true if s[0..i) can be segmented
        vector<bool> dp(n + 1);
        dp[0] = true;

        for (int i = 1; i <= n; ++i)
            for (int j = 0; j < i; ++j)
                // s[0..j) can be segmented and s[j..i) is in `wordSet`, so s[0..i) can
                // be segmented.
                if (dp[j] && wordSet.contains(s.substr(j, i - j))) {
                    dp[i] = true;
                    break;
                }

        return dp[n];
    }
};
```

421. 139-4.cpp

Path: LeetCode-main/solutions/139. Word Break/139-4.cpp

```

class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        const int n = s.length();
        const int maxLength = getMaxLength(wordDict);
        const unordered_set<string> wordSet{wordDict.begin(), wordDict.end()};
        // dp[i] := true if s[0..i] can be segmented
        vector<int> dp(n + 1);
        dp[0] = true;

        for (int i = 1; i <= n; ++i)
            for (int j = i - 1; j >= 0; --j) {
                if (i - j > maxLength)
                    break;
                // s[0..j] can be segmented and s[j..i] is in the wordSet, so s[0..i]
                // can be segmented.
                if (dp[j] && wordSet.contains(s.substr(j, i - j))) {
                    dp[i] = true;
                    break;
                }
            }

        return dp[n];
    }

private:
    int getMaxLength(const vector<string>& wordDict) {
        return ranges::max_element(wordDict, ranges::less{},
                                   [](const string& word) {
                    return word.length();
                })->length();
    }
};

```

422. 139.cpp

Path: LeetCode-main/solutions/139. Word Break/139.cpp

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        return wordBreak(s, {wordDict.begin(), wordDict.end()}, {});
    }

private:
    bool wordBreak(const string& s, const unordered_set<string>&& wordSet,
                   unordered_map<string, bool>&& mem) {
        if (wordSet.contains(s))
            return true;
        if (const auto it = mem.find(s); it != mem.cend())
            return it->second;

        // 1 <= prefix.length() < s.length()
        for (int i = 1; i < s.length(); ++i) {
            const string& prefix = s.substr(0, i);
            const string& suffix = s.substr(i);
            if (wordSet.contains(prefix) &&
                wordBreak(suffix, std::move(wordSet), std::move(mem)))
                return mem[s] = true;
        }

        return mem[s] = false;
    }
};
```

423. 1390.cpp

Path: LeetCode-main/solutions/1390. Four Divisors/1390.cpp

```
class Solution {
public:
    int sumFourDivisors(vector<int>& nums) {
        int ans = 0;

        for (int num : nums) {
            int divisor = 0;
            for (int i = 2; i * i <= num; ++i)
                if (num % i == 0) {
                    if (divisor == 0)
                        divisor = i;
                    else {
                        divisor = 0;
                        break;
                    }
                }
            if (divisor > 0 && divisor * divisor < num)
                ans += 1 + num + divisor + num / divisor;
        }

        return ans;
    }
};
```

424. 1391.cpp

Path: LeetCode-main/solutions/1391. Check if There is a Valid Path in a Grid/1391.cpp

```
class Solution {
public:
    bool hasValidPath(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        // g := upscaled grid
        vector<vector<bool>> g(m * 3, vector<bool>(n * 3));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                switch (grid[i][j]) {
                    case 1:
                        g[i * 3 + 1][j * 3 + 0] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 2] = true;
                        break;
                    case 2:
                        g[i * 3 + 0][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 2][j * 3 + 1] = true;
                        break;
                    case 3:
                        g[i * 3 + 1][j * 3 + 0] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 2][j * 3 + 1] = true;
                        break;
                    case 4:
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 2] = true;
                        g[i * 3 + 2][j * 3 + 1] = true;
                        break;
                    case 5:
                        g[i * 3 + 0][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 0] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        break;
                    case 6:
                        g[i * 3 + 0][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 1] = true;
                        g[i * 3 + 1][j * 3 + 2] = true;
                        break;
                }

        return dfs(g, 1, 1);
    }

private:
    bool dfs(vector<vector<bool>>& g, int i, int j) {
        if (i < 0 || i == g.size() || j < 0 || j == g[0].size())
            return false;
        if (!g[i][j]) // There's no path here.
            return false;
        if (i == g.size() - 2 && j == g[0].size() - 2)
            return true;

        g[i][j] = false; // Mark as visited.
        return dfs(g, i + 1, j) || dfs(g, i - 1, j) || dfs(g, i, j + 1) ||
            dfs(g, i, j - 1);
    }
};
```

425. 1392.cpp

Path: LeetCode-main/solutions/1392. Longest Happy Prefix/1392.cpp

```
class Solution {
public:
    string longestPrefix(string s) {
        constexpr int kBase = 26;
        constexpr long kHash = 8'417'508'174'513;

        const int n = s.length();
        int maxLength = 0;
        long pow = 1;
        long prefixHash = 0; // the hash of s[0..i]
        long suffixHash = 0; // the hash of s[j..n)

        for (int i = 0, j = n - 1; i < n - 1; ++i, --j) {
            prefixHash = (prefixHash * kBase + val(s[i])) % kHash;
            suffixHash = (val(s[j]) * pow + suffixHash) % kHash;
            pow = pow * kBase % kHash;
            if (prefixHash == suffixHash)
                maxLength = i + 1;
        }

        return s.substr(0, maxLength);
    }

private:
    constexpr int val(char c) {
        return c - 'a';
    }
};
```

426. 1394.cpp

Path: LeetCode-main/solutions/1394. Find Lucky Integer in an Array/1394.cpp

```
class Solution {
public:
    int findLucky(vector<int>& arr) {
        vector<int> count(arr.size() + 1);

        for (const int a : arr)
            if (a <= arr.size())
                ++count[a];

        for (int i = arr.size(); i >= 1; --i)
            if (count[i] == i)
                return i;

        return -1;
    }
};
```

427. 1395.cpp

Path: LeetCode-main/solutions/1395. Count Number of Teams/1395.cpp

```
class Solution {
public:
    int numTeams(vector<int>& rating) {
        int ans = 0;

        for (int i = 1; i < rating.size() - 1; ++i) {
            // Calculate soldiers on the left with less/greater ratings.
            int leftLess = 0;
            int leftGreater = 0;
            for (int j = 0; j < i; ++j)
                if (rating[j] < rating[i])
                    ++leftLess;
                else if (rating[j] > rating[i])
                    ++leftGreater;
            // Calculate soldiers on the right with less/greater ratings.
            int rightLess = 0;
            int rightGreater = 0;
            for (int j = i + 1; j < rating.size(); ++j)
                if (rating[j] < rating[i])
                    ++rightLess;
                else if (rating[j] > rating[i])
                    ++rightGreater;
            ans += leftLess * rightGreater + leftGreater * rightLess;
        }

        return ans;
    }
};
```


428. 1396.cpp

Path: LeetCode-main/solutions/1396. Design Underground System/1396.cpp

```
struct CheckIn {
    string stationName;
    int time;
};

struct CheckOut {
    int numTrips;
    int totalTime;
};

class UndergroundSystem {
public:
    void checkIn(int id, string stationName, int t) {
        checkIns[id] = {stationName, t};
    }

    void checkOut(int id, string stationName, int t) {
        const auto [startStation, startTime] = checkIns[id];
        checkIns.erase(id);
        const string& route = startStation + "->" + stationName;
        ++checkOuts[route].numTrips;
        checkOuts[route].totalTime += t - startTime;
    }

    double getAverageTime(string startStation, string endStation) {
        const auto& [numTrips, totalTime] =
            checkOuts[startStation + "->" + endStation];
        return totalTime / (double)numTrips;
    }

private:
    unordered_map<int, CheckIn> checkIns; // {id: (stationName, time)}
    unordered_map<string, CheckOut> checkOuts; // {route: (numTrips, totalTime)}
};
```

429. 1397.cpp

Path: LeetCode-main/solutions/1397. Find All Good Strings/1397.cpp

```
class Solution {
public:
    int findGoodStrings(int n, string s1, string s2, string evil) {
        vector<vector<vector<vector<int>>>>> mem(
            n, vector<vector<vector<int>>>>(
                evil.length(), vector<vector<int>>>(2, vector<int>(2, -1))));
        // nextMatchedCount[i][j] := the number of next matched evil count, where
        // there're i matches with `evil` and the current letter is ('a' + j)
        vector<vector<int>>> nextMatchedCount(evil.length(), vector<int>(26, -1));
        return count(s1, s2, evil, 0, 0, true, true, getLPS(evil), nextMatchedCount,
            mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of good strings for s[i..n), where there're j matches
    // with `evil`, `isS1Prefix` indicates if the current letter is tightly bound
    // for `s1` and `isS2Prefix` indicates if the current letter is tightly bound
    // for `s2`.
    int count(const string& s1, const string& s2, const string& evil, int i,
        int matchedEvilCount, bool isS1Prefix, bool isS2Prefix,
        const vector<int>& evilLPS, vector<vector<int>>& nextMatchedCount,
        vector<vector<vector<vector<int>>>>& mem) {
        // s[0..i) contains `evil`, so don't consider any ongoing strings.
        if (matchedEvilCount == evil.length())
            return 0;
        // Run out of strings, so contribute one.
        if (i == s1.length())
            return 1;
        int& res = mem[i][matchedEvilCount][isS1Prefix][isS2Prefix];
        if (res != -1)
            return res;
        res = 0;
        const char minLetter = isS1Prefix ? s1[i] : 'a';
        const char maxLetter = isS2Prefix ? s2[i] : 'z';
        for (char c = minLetter; c <= maxLetter; ++c) {
            const int nextMatchedEvilCount = getNextMatchedEvilCount(
                nextMatchedCount, evil, matchedEvilCount, c, evilLPS);
            res += count(s1, s2, evil, i + 1, nextMatchedEvilCount,
                isS1Prefix && c == s1[i], isS2Prefix && c == s2[i], evilLPS,
                nextMatchedCount, mem);
            res %= kMod;
        }
        return res;
    }

    // Returns the lps array, where lps[i] is the length of the longest prefix of
    // pattern[0..i] which is also a suffix of this substring.
    vector<int> getLPS(const string& pattern) {
        vector<int> lps(pattern.length());
        for (int i = 1, j = 0; i < pattern.length(); ++i) {
            while (j > 0 && pattern[j] != pattern[i])
                j = lps[j - 1];
            if (pattern[i] == pattern[j])
                lps[i] = ++j;
        }
        return lps;
    }

    // j := the next index we're trying to match with `currLetter`
    int getNextMatchedEvilCount(vector<vector<int>>& nextMatchedCount,
        const string& evil, int j, char currLetter,
        const vector<int>& evilLPS) {
        int& res = nextMatchedCount[j][currLetter - 'a'];
        if (res != -1)
            return res;
        while (j > 0 && evil[j] != currLetter)
            j = evilLPS[j - 1];
        return res = (evil[j] == currLetter ? j + 1 : j);
    }
};
```

430. 1399.cpp

Path: LeetCode-main/solutions/1399. Count Largest Group/1399.cpp

```
class Solution {
public:
    int countLargestGroup(int n) {
        vector<int> count(9 * 4 + 1);
        for (int i = 1; i <= n; ++i)
            ++count[getDigitSum(i)];
        return ranges::count(count, ranges::max(count));
    }

private:
    int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
};
```

431. 14.cpp

Path: LeetCode-main/solutions/14. Longest Common Prefix/14.cpp

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {
        if (strs.empty())
            return "";

        for (int i = 0; i < strs[0].length(); ++i)
            for (int j = 1; j < strs.size(); ++j)
                if (i == strs[j].length() || strs[j][i] != strs[0][i])
                    return strs[0].substr(0, i);

        return strs[0];
    }
};
```

432. 140.cpp

Path: LeetCode-main/solutions/140. Word Break II/140.cpp

```
class Solution {
public:
    vector<string> wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> wordSet{wordDict.begin(), wordDict.end()};
        unordered_map<string, vector<string>> mem;
        return wordBreak(s, wordSet, mem);
    }

private:
    vector<string> wordBreak(const string& s,
                           const unordered_set<string>& wordSet,
                           unordered_map<string, vector<string>>& mem) {
        if (const auto it = mem.find(s); it != mem.cend())
            return it->second;

        vector<string> ans;

        // 1 <= prefix.length() < s.length()
        for (int i = 1; i < s.length(); ++i) {
            const string& prefix = s.substr(0, i);
            const string& suffix = s.substr(i);
            if (wordSet.contains(prefix))
                for (const string& word : wordBreak(suffix, wordSet, mem))
                    ans.push_back(prefix + " " + word);
        }

        // `wordSet` contains the whole string s, so don't add any space.
        if (wordSet.contains(s))
            ans.push_back(s);

        return mem[s] = ans;
    }
};
```

433. 1400.cpp

Path: LeetCode-main/solutions/1400. Construct K Palindrome Strings/1400.cpp

```
class Solution {
public:
    bool canConstruct(string s, int k) {
        // If |s| < k, we cannot construct k strings from the s.
        if (s.length() < k)
            return false;

        bitset<26> odd;

        for (const char c : s)
            odd.flip(c - 'a');

        // If the number of letters that have odd counts > k, the minimum number of
        // palindromic strings we can construct is > k.
        return odd.count() <= k;
    }
};
```

434. 1401.cpp

Path: LeetCode-main/solutions/1401. Circle and Rectangle Overlapping/1401.cpp

```
class Solution {
public:
    bool checkOverlap(int radius, int x_center, int y_center, int x1, int y1,
                      int x2, int y2) {
        auto clamp = [&](int center, int mn, int mx) {
            return max(mn, min(mx, center));
        };

        // the closest point to the circle within the rectangle
        int closestX = clamp(x_center, x1, x2);
        int closestY = clamp(y_center, y1, y2);

        // the distance between the circle's center and its closest point
        int distanceX = x_center - closestX;
        int distanceY = y_center - closestY;

        // If the distance < the circle's radius, an intersection occurs.
        return (distanceX * distanceX) + (distanceY * distanceY) <=
            (radius * radius);
    }
};
```

435. 1402.cpp

Path: LeetCode-main/solutions/1402. Reducing Dishes/1402.cpp

```
class Solution {
public:
    int maxSatisfaction(vector<int>& satisfaction) {
        int ans = 0;
        int sumSatisfaction = 0;

        ranges::sort(satisfaction, greater<>());

        for (const int s : satisfaction) {
            sumSatisfaction += s;
            if (sumSatisfaction <= 0)
                return ans;
            ans += sumSatisfaction;
        }
        return ans;
    }
};
```


436. 1403.cpp

Path: LeetCode-main/solutions/1403. Minimum Subsequence in Non-Increasing Order/1403.cpp

```
class Solution {
public:
    vector<int> minSubsequence(vector<int>& nums) {
        vector<int> ans;
        priority_queue<int> maxHeap(nums.begin(), nums.end());
        int half = accumulate(nums.begin(), nums.end(), 0) / 2;

        while (half >= 0) {
            ans.push_back(maxHeap.top());
            half -= maxHeap.top(), maxHeap.pop();
        }

        return ans;
    }
};
```

437. 1404.cpp

Path: LeetCode-main/solutions/1404. Number of Steps to Reduce a Number in Binary Representation to One/1404.cpp

```
class Solution {
public:
    int numSteps(string s) {
        int ans = 0;

        // All the trailing 0s can be popped by 1 step.
        while (s.back() == '0') {
            s.pop_back();
            ++ans;
        }

        if (s == "1")
            return ans;

        // `s` is now odd, so add 1 to `s` and cost 1 step.
        ++ans;

        // All the 1s will become 0s and can be popped by 1 step.
        // All the 0s will become 1s and can be popped by 2 steps (adding 1 then
        // dividing by 2).
        for (const char c : s)
            ans += c == '1' ? 1 : 2;

        return ans;
    }
};
```

438. 1405.cpp

Path: LeetCode-main/solutions/1405. Longest Happy String/1405.cpp

```
class Solution {
public:
    string longestDiverseString(int a, int b, int c, char A = 'a', char B = 'b',
                               char C = 'c') {
        if (a < b)
            return longestDiverseString(b, a, c, B, A, C);
        if (b < c)
            return longestDiverseString(a, c, b, A, C, B);
        if (b == 0)
            return string(min(a, 2), A);

        const int useA = min(a, 2);
        const int useB = (a - useA >= b) ? 1 : 0;
        return string(useA, A) + string(useB, B) +
            longestDiverseString(a - useA, b - useB, c, A, B, C);
    }
};
```

439. 1406-2.cpp

Path: LeetCode-main/solutions/1406. Stone Game III/1406-2.cpp

```
class Solution {
public:
    string stoneGameIII(vector<int>& stoneValue) {
        const int n = stoneValue.size();
        // dp[i] := the maximum relative score Alice can make with stoneValue[i..n)
        vector<int> dp(n + 1, INT_MIN / 2);
        dp.back() = 0;

        for (int i = n - 1; i >= 0; --i) {
            int sum = 0;
            for (int j = i; j < i + 3 && j < n; ++j) {
                sum += stoneValue[j];
                dp[i] = max(dp[i], sum - dp[j + 1]);
            }
        }

        const int score = dp[0];
        return score > 0 ? "Alice" : score < 0 ? "Bob" : "Tie";
    }
};
```

440. 1406.cpp

Path: LeetCode-main/solutions/1406. Stone Game III/1406.cpp

```
class Solution {
public:
    string stoneGameIII(vector<int>& stoneValue) {
        vector<int> mem(stoneValue.size(), INT_MIN);
        const int score = stoneGameIII(stoneValue, 0, mem);
        return score > 0 ? "Alice" : score < 0 ? "Bob" : "Tie";
    }

private:
    // Returns the maximum relative score Alice can make from stoneValue[i..n).
    int stoneGameIII(const vector<int>& stoneValue, int i, vector<int>& mem) {
        if (i == stoneValue.size())
            return 0;
        if (mem[i] > INT_MIN)
            return mem[i];

        int sum = 0;
        for (int j = i; j < i + 3 && j < stoneValue.size(); ++j) {
            sum += stoneValue[j];
            mem[i] = max(mem[i], sum - stoneGameIII(stoneValue, j + 1, mem));
        }

        return mem[i];
    };
};
```

441. 1408-2.cpp

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408-2.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    TrieNode() : children(26) {}
};

class Trie {
public:
    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
    }

    bool search(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                return false;
            node = node->children[i];
        }
        return true;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();
};

class Solution {
public:
    vector<string> stringMatching(vector<string>& words) {
        vector<string> ans;
        Trie trie;

        ranges::sort(words, ranges::greater{},
            [](const string& word) { return word.length(); });

        for (const string& word : words) {
            if (trie.search(word))
                ans.push_back(word);
            for (int i = 0; i < word.length(); ++i)
                trie.insert(word.substr(i));
        }

        return ans;
    }
};
```

442. 1408-3.cpp

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408-3.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    int count = 0;
    TrieNode() : children(26) {}
};

class Trie {
public:
    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
            ++node->count;
        }
    }

    bool search(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                return false;
            node = node->children[i];
        }
        return node->count > 1;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();
};

class Solution {
public:
    vector<string> stringMatching(vector<string>& words) {
        vector<string> ans;
        Trie trie;

        for (const string& word : words)
            for (int i = 0; i < word.length(); ++i)
                trie.insert(word.substr(i));

        for (const string& word : words)
            if (trie.search(word))
                ans.push_back(word);

        return ans;
    }
};
```

443. 1408.cpp

Path: LeetCode-main/solutions/1408. String Matching in an Array/1408.cpp

```
class Solution {
public:
    vector<string> stringMatching(vector<string>& words) {
        vector<string> ans;
        for (const string& a : words)
            for (const string& b : words)
                if (a.length() < b.length() && b.find(a) != string::npos) {
                    ans.push_back(a);
                    break;
                }
        return ans;
    }
};
```


444. 1409.cpp

Path: LeetCode-main/solutions/1409. Queries on a Permutation With Key/1409.cpp

```
class FenwickTree {
public:
    FenwickTree(int n) : sums(n + 1) {}

    void add(int i, int delta) {
        while (i < sums.size()) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    int get(int i) const {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

private:
    vector<int> sums;

    static inline int lowbit(int i) {
        return i & -i;
    }
};

class Solution {
public:
    vector<int> processQueries(vector<int>& queries, int m) {
        vector<int> ans;
        // Map [-m, m] to [0, 2 * m].
        FenwickTree tree(2 * m + 1);
        unordered_map<int, int> numToIndex;

        for (int num = 1; num <= m; ++num) {
            numToIndex[num] = num + m;
            tree.add(num + m, 1);
        }

        int nextEmptyIndex = m; // Map 0 to m.

        for (const int query : queries) {
            const int index = numToIndex[query];
            ans.push_back(tree.get(index - 1));
            // Move `query` from `index` to `nextEmptyIndex`.
            tree.add(index, -1);
            tree.add(nextEmptyIndex, 1);
            numToIndex[query] = nextEmptyIndex--;
        }

        return ans;
    }
};
```

445. 141.cpp

Path: LeetCode-main/solutions/141. Linked List Cycle/141.cpp

```
class Solution {
public:
    bool hasCycle(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
            if (slow == fast)
                return true;
        }

        return false;
    }
};
```

446. 1410.cpp

Path: LeetCode-main/solutions/1410. HTML Entity Parser/1410.cpp

```
class Solution {
public:
    string entityParser(string text) {
        const unordered_map<string, char> entityToChar{
            {"&quot;", "'"}, {"&apos;", "'"}, {"&amp;", '&'},
            {"&gt;", '>'}, {"&lt;", '<'}, {"&frasl;", '/'};
        }
        string ans;
        int j = 0; // text[j..ampersandIndex - 1] is the pending substring.
        int ampersandIndex = -1;

        for (int i = 0; i < text.length(); ++i)
            if (text[i] == '&') {
                ampersandIndex = i;
            } else if (text[i] == ';' && ampersandIndex >= j) {
                const string sub = text.substr(ampersandIndex, i - ampersandIndex + 1);
                ans += text.substr(j, ampersandIndex - j);
                ans += getCharIfMatched(text, sub, entityToChar);
                j = i + 1;
            }

        return ans + text.substr(j);
    }

private:
    string getCharIfMatched(const string& text, const string& sub,
                           const unordered_map<string, char>& entityToChar) {
        for (const auto& [entity, c] : entityToChar)
            if (entity == sub)
                return string(1, c);
        return sub;
    }
};
```

447. 1411.cpp

Path: LeetCode-main/solutions/1411. Number of Ways to Paint N × 3 Grid/1411.cpp

```
class Solution {
public:
    int numOfWays(int n) {
        constexpr int kMod = 1'000'000'007;
        long color2 = 6; // 121, 131, 212, 232, 313, 323
        long color3 = 6; // 123, 132, 213, 231, 312, 321

        for (int i = 1; i < n; ++i) {
            const long nextColor2 = color2 * 3 + color3 * 2;
            const long nextColor3 = color2 * 2 + color3 * 2;
            color2 = nextColor2 % kMod;
            color3 = nextColor3 % kMod;
        }

        return (color2 + color3) % kMod;
    }
};
```

448. 1413.cpp

Path: LeetCode-main/solutions/1413. Minimum Value to Get Positive Step by Step Sum/1413.cpp

```
class Solution {
public:
    int minStartValue(vector<int>& nums) {
        int sum = 0;
        int minSum = 0;

        for (const int num : nums)
            minSum = min(minSum, sum += num);

        return 1 - minSum;
    }
};
```

449. 1414-2.cpp

Path: LeetCode-main/solutions/1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K/1414-2.cpp

```
class Solution {
public:
    int findMinFibonacciNumbers(int k) {
        int ans = 0;
        int a = 1; // F_1
        int b = 1; // F_2

        while (b <= k) {
            // a, b = F_{i + 1}, F_{i + 2}
            // -> a, b = F_{i + 2}, F_{i + 3}
            const int temp = a;
            a = b;
            b = a + temp;
        }

        while (a > 0) {
            if (a <= k) {
                k -= a;
                ++ans;
            }
            // a, b = F_{i + 2}, F_{i + 3}
            // -> a, b = F_{i + 1}, F_{i + 2}
            const int temp = a;
            a = b - a;
            b = temp;
        }

        return ans;
    }
};
```

450. 1414.cpp

Path: LeetCode-main/solutions/1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K/1414.cpp

```
class Solution {
public:
    int findMinFibonacciNumbers(int k) {
        if (k < 2) // k == 0 || k == 1
            return k;

        int a = 1; // F_1
        int b = 1; // F_2

        while (b <= k) {
            // a, b = F_{i + 1}, F_{i + 2}
            // -> a, b = F_{i + 2}, F_{i + 3}
            const int temp = a;
            a = b;
            b = a + temp;
        }

        return 1 + findMinFibonacciNumbers(k - a);
    }
};
```

451. 1415.cpp

Path: LeetCode-main/solutions/1415. The k-th Lexicographical String of All Happy Strings of Length n/1415.cpp

```
class Solution {
public:
    string getHappyString(int n, int k) {
        const unordered_map<char, string> nextLetters{
            {'a', "bc"}, {'b', "ac"}, {'c', "ab"}};
        queue<string> q{{"a", "b", "c"}};

        while (q.front().length() != n) {
            const string u = q.front();
            q.pop();
            for (const char nextLetter : nextLetters.at(u.back()))
                q.push(u + nextLetter);
        }

        if (q.size() < k)
            return "";

        for (int i = 0; i < k - 1; ++i)
            q.pop();
        return q.front();
    }
};
```


452. 1416-2.cpp

Path: LeetCode-main/solutions/1416. Restore The Array/1416-2.cpp

```
class Solution {
public:
    int numberOfArrays(string s, int k) {
        constexpr int kMod = 1'000'000'007;
        const int n = s.length();
        // dp[i] := the number of arrays to restore s[i..n) with k
        vector<int> dp(n + 1);
        dp.back() = 1; // an empty string ""

        for (int i = n - 1; i >= 0; --i) {
            if (s[i] == '0')
                continue; // a leading zero
            long num = 0;
            for (int j = i; j < n; ++j) {
                num = num * 10 + (s[j] - '0');
                if (num > k)
                    break;
                dp[i] = (dp[i] + dp[j + 1]) % kMod;
            }
        }

        return dp[0];
    }
};
```

453. 1416.cpp

Path: LeetCode-main/solutions/1416. Restore The Array/1416.cpp

```
class Solution {
public:
    int numberOfArrays(string s, int k) {
        vector<int> mem(s.length(), -1);
        return numberOfArrays(s, 0, k, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of arrays to restore s[i..n) with k.
    int numberOfArrays(const string& s, int i, int k, vector<int>& mem) {
        if (i == s.length())
            return 1; // an empty string ""
        if (s[i] == '0')
            return 0; // a leading zero
        if (mem[i] != -1)
            return mem[i];

        int res = 0;
        long num = 0;

        for (int j = i; j < s.length(); ++j) {
            num = num * 10 + (s[j] - '0');
            if (num > k)
                break;
            res = (res + numberOfArrays(s, j + 1, k, mem)) % kMod;
        }

        return mem[i] = res;
    }
};
```

454. 1417-2.cpp

Path: LeetCode-main/solutions/1417. Reformat The String/1417-2.cpp

```
class Solution {
public:
    string reformat(string s) {
        const int countAlpha =
            ranges::count_if(s, [](char c) { return isalpha(c); });
        const int countDigit = s.size() - countAlpha;
        if (abs(countAlpha - countDigit) >= 2)
            return "";

        // Initialize the starting index. e.g. "a0a0a" or "0a0a0".
        const int alphaStartingIndex = countAlpha >= countDigit ? 0 : 1;

        // Place all alphas in the indices 0, 2, 4, 6, ... or 1, 3, 5, 7, ....
        for (int i = 0, j = alphaStartingIndex; i < s.length(); ++i) {
            if (i < j && i % 2 == alphaStartingIndex)
                continue; // The `s[i]` has been set.
            if (!isalpha(s[i]))
                continue;
            swap(s[i], s[j]);
            j += 2;
            --i;
        }

        return s;
    }
};
```

455. 1417.cpp

Path: LeetCode-main/solutions/1417. Reformat The String/1417.cpp

```
class Solution {
public:
    string reformat(string s) {
        vector<char> A;
        vector<char> B;

        for (const char c : s)
            isalpha(c) ? A.push_back(c) : B.push_back(c);

        if (A.size() < B.size())
            swap(A, B);
        if (A.size() - B.size() > 1)
            return "";

        string ans;

        for (int i = 0; i < B.size(); ++i)
            ans += string{A[i], B[i]};

        if (A.size() > B.size())
            ans += A.back();
        return ans;
    }
};
```

456. 1418.cpp

Path: LeetCode-main/solutions/1418. Display Table of Food Orders in a Restaurant/1418.cpp

```
class Solution {
public:
    vector<vector<string>> displayTable(vector<vector<string>>& orders) {
        vector<vector<string>> ans{"Table"};
        unordered_map<string, int> tableNumberToRowIndex;
        unordered_map<string, int> foodItemToColIndex;

        // Create the first row and column of ans.
        for (const vector<string>& order : orders) {
            const string& tableNumber = order[1];
            const string& foodItem = order[2];
            // Initialize indices with 0s.
            tableNumberToRowIndex[tableNumber] = 0;
            foodItemToColIndex[foodItem] = 0;
        }
        for (const auto& [tableNumber, _] : tableNumberToRowIndex)
            ans.push_back({tableNumber});
        for (const auto& [foodItem, _] : foodItemToColIndex)
            ans[0].push_back(foodItem);

        // Sort the first row and the column, except ans[0][0].
        sort(ans[0].begin() + 1, ans[0].end());
        ranges::sort(ans.begin() + 1, ans.end(), ranges::less{},
            [](const vector<string>& cols) { return stoi(cols[0]); });

        // Set the indices after sorting.
        for (int i = 0; i < tableNumberToRowIndex.size(); ++i)
            tableNumberToRowIndex[ans[i + 1][0]] = i;
        for (int i = 0; i < foodItemToColIndex.size(); ++i)
            foodItemToColIndex[ans[0][i + 1]] = i;

        // Get all the counts of each food item in each table.
        vector<vector<int>> count;
        for (int i = 0; i < tableNumberToRowIndex.size(); ++i)
            count.push_back(vector<int>(foodItemToColIndex.size()));
        for (const vector<string>& order : orders) {
            const string& tableNumber = order[1];
            const string& foodItem = order[2];
            const int rowIndex = tableNumberToRowIndex[tableNumber];
            const int colIndex = foodItemToColIndex[foodItem];
            ++count[rowIndex][colIndex];
        }

        // Set the counts to `ans`.
        for (int i = 0; i < tableNumberToRowIndex.size(); ++i)
            for (int j = 0; j < foodItemToColIndex.size(); ++j)
                ans[i + 1].push_back(to_string(count[i][j]));

        return ans;
    }
};
```

457. 1419.cpp

Path: LeetCode-main/solutions/1419. Minimum Number of Frogs Croaking/1419.cpp

```
class Solution {
public:
    int minNumberOfFrogs(string croakOfFrogs) {
        const string kCroak = "croak";
        int ans = 0;
        int frogs = 0;
        vector<int> count(5);

        for (const char c : croakOfFrogs) {
            ++count[kCroak.find(c)];
            for (int i = 1; i < 5; ++i)
                if (count[i] > count[i - 1])
                    return -1;
            if (c == 'c')
                ++frogs;
            else if (c == 'k')
                --frogs;
            ans = max(ans, frogs);
        }

        return frogs == 0 ? ans : -1;
    }
};
```

458. 142.cpp

Path: LeetCode-main/solutions/142. Linked List Cycle II/142.cpp

```
class Solution {
public:
    ListNode* detectCycle(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
            if (slow == fast) {
                slow = head;
                while (slow != fast) {
                    slow = slow->next;
                    fast = fast->next;
                }
                return slow;
            }
        }

        return nullptr;
    }
};
```

459. 1420-2.cpp

Path: LeetCode-main/solutions/1420. Build Array Where You Can Find The Maximum Exactly K Comparisons/1420-2.cpp

```
class Solution {
public:
    int numOfArrays(int n, int m, int k) {
        constexpr int kMod = 1'000'000'007;
        // dp[i][j][k] := the number of ways to build an array of length i, where j
        // is the maximum number and k is the `search_cost`
        vector<vector<vector<int>>> dp(
            n + 1, vector<vector<int>>>(m + 1, vector<int>(k + 1)));
        vector<vector<vector<int>>> prefix(
            n + 1, vector<vector<int>>>(m + 1, vector<int>(k + 1)));

        for (int j = 1; j <= m; ++j) {
            dp[1][j][1] = 1;
            prefix[1][j][1] = j;
        }

        for (int i = 2; i <= n; ++i) // for each length
            for (int j = 1; j <= m; ++j) // for each max value
                for (int cost = 1; cost <= k; ++cost) { // for each cost
                    // 1. Appending any of [1, j] in the i-th position doesn't change the
                    //    maximum and cost.
                    // 2. Appending j in the i-th position makes j the new max and
                    //    cost 1.
                    dp[i][j][cost] = (static_cast<long>(j) * dp[i - 1][j][cost] +
                                        prefix[i - 1][j - 1][cost - 1]) %
                                        kMod;
                    prefix[i][j][cost] = (dp[i][j][cost] + prefix[i][j - 1][cost]) % kMod;
                }

        int ans = 0;
        for (int j = 1; j <= m; ++j) {
            ans += dp[n][j][k];
            ans %= kMod;
        }
        return ans;
    }
};
```


460. 1420.cpp

Path: LeetCode-main/solutions/1420. Build Array Where You Can Find The Maximum Exactly K Comparisons/1420.cpp

```
class Solution {
public:
    int numOfArrays(int n, int m, int k) {
        constexpr int kMod = 1'000'000'007;
        // dp[i][j][k] := the number of ways to build an array of length i, where j
        // is the maximum number and k is `search_cost`
        vector<vector<vector<int>>> dp(
            n + 1, vector<vector<int>>>(m + 1, vector<int>(k + 1)));

        for (int j = 1; j <= m; ++j)
            dp[1][j][1] = 1;

        for (int i = 2; i <= n; ++i)                // for each length
            for (int j = 1; j <= m; ++j)            // for each max value
                for (int cost = 1; cost <= k; ++cost) { // for each cost
                    // 1. Appending any of [1, j] in the i-th position doesn't change the
                    //    maximum and cost.
                    dp[i][j][cost] = static_cast<long>(j) * dp[i - 1][j][cost] % kMod;
                    // 2. Appending j in the i-th position makes j the new max and cost 1.
                    for (int prevMax = 1; prevMax < j; ++prevMax) {
                        dp[i][j][cost] += dp[i - 1][prevMax][cost - 1];
                        dp[i][j][cost] %= kMod;
                    }
                }

        int ans = 0;
        for (int j = 1; j <= m; ++j) {
            ans += dp[n][j][k];
            ans %= kMod;
        }
        return ans;
    }
};
```

461. 1422.cpp

Path: LeetCode-main/solutions/1422. Maximum Score After Splitting a String/1422.cpp

```
class Solution {
public:
    int maxScore(string s) {
        int ans = 0;
        int zeros = 0;
        int ones = ranges::count(s, '1');

        for (int i = 0; i + 1 < s.size(); ++i) {
            if (s[i] == '0')
                ++zeros;
            else
                --ones;
            ans = max(ans, zeros + ones);
        }

        return ans;
    }
};
```

462. 1423.cpp

Path: LeetCode-main/solutions/1423. Maximum Points You Can Obtain from Cards/1423.cpp

```
class Solution {
public:
    int maxScore(vector<int>& cardPoints, int k) {
        const int n = cardPoints.size();
        const int sum = accumulate(cardPoints.begin(), cardPoints.end(), 0);
        int windowSum =
            accumulate(cardPoints.begin(), cardPoints.begin() + n - k, 0);
        int ans = sum - windowSum;

        for (int i = 0; i < k; ++i) {
            windowSum -= cardPoints[i];
            windowSum += cardPoints[i + n - k];
            ans = max(ans, sum - windowSum);
        }

        return ans;
    }
};
```

463. 1424.cpp

Path: LeetCode-main/solutions/1424. Diagonal Traverse II/1424.cpp

```
class Solution {
public:
    vector<int> findDiagonalOrder(vector<vector<int>>& nums) {
        vector<int> ans;
        unordered_map<int, vector<int>> keyToNums; // key := row + column
        int maxKey = 0;

        for (int i = 0; i < nums.size(); ++i)
            for (int j = 0; j < nums[i].size(); ++j) {
                const int key = i + j;
                keyToNums[key].push_back(nums[i][j]);
                maxKey = max(maxKey, key);
            }

        for (int i = 0; i <= maxKey; ++i)
            for (auto it = keyToNums[i].rbegin(); it != keyToNums[i].rend(); ++it)
                ans.push_back(*it);

        return ans;
    }
};
```

464. 1425.cpp

Path: LeetCode-main/solutions/1425. Constrained Subsequence Sum/1425.cpp

```
class Solution {
public:
    int constrainedSubsetSum(vector<int>& nums, int k) {
        // dp[i] := the maximum the sum of non-empty subsequences in nums[0..i]
        vector<int> dp(nums.size());
        // dq stores dp[i - k], dp[i - k + 1], ..., dp[i - 1] whose values are > 0
        // in decreasing order.
        deque<int> dq;

        for (int i = 0; i < nums.size(); ++i) {
            if (dq.empty())
                dp[i] = nums[i];
            else
                dp[i] = max(dq.front(), 0) + nums[i];
            while (!dq.empty() && dq.back() < dp[i])
                dq.pop_back();
            dq.push_back(dp[i]);
            if (i >= k && dp[i - k] == dq.front())
                dq.pop_front();
        }

        return ranges::max(dp);
    }
};
```

465. 1426.cpp

Path: LeetCode-main/solutions/1426. Counting Elements/1426.cpp

```
class Solution {
public:
    int countElements(vector<int>& arr) {
        unordered_set<int> arrSet{arr.begin(), arr.end()};
        return ranges::count_if(
            arr, [&arrSet](int a) { return arrSet.contains(a + 1); });
    }
};
```

466. 1427.cpp

Path: LeetCode-main/solutions/1427. Perform String Shifts/1427.cpp

```
class Solution {
public:
    string stringShift(string s, vector<vector<int>>& shift) {
        const int n = s.length();
        int move = 0;

        for (const vector<int>& pair : shift) {
            const int direction = pair[0];
            const int amount = pair[1];
            if (direction == 0)
                move -= amount;
            else
                move += amount;
        }

        move = ((move % n) + n) % n;
        return s.substr(n - move) + s.substr(0, n - move);
    }
};
```

467. 1428-2.cpp

Path: LeetCode-main/solutions/1428. Leftmost Column with at Least a One/1428-2.cpp

```
/**
 * // This is the BinaryMatrix's API interface.
 * // You should not implement it, or speculate about its implementation
 * class BinaryMatrix {
 * public:
 *     int get(int row, int col);
 *     vector<int> dimensions();
 * };
 */

class Solution {
public:
    int leftMostColumnWithOne(BinaryMatrix& binaryMatrix) {
        const vector<int> dimensions = binaryMatrix.dimensions();
        const int m = dimensions[0];
        const int n = dimensions[1];
        int ans = -1;
        int i = 0;
        int j = n - 1;

        while (i < m && j >= 0)
            if (binaryMatrix.get(i, j) == 1)
                ans = j--;
            else
                ++i;

        return ans;
    }
};
```


468. 1428.cpp

Path: LeetCode-main/solutions/1428. Leftmost Column with at Least a One/1428.cpp

```
/**
 * // This is the BinaryMatrix's API interface.
 * // You should not implement it, or speculate about its implementation
 * class BinaryMatrix {
 * public:
 *     int get(int row, int col);
 *     vector<int> dimensions();
 * };
 */

class Solution {
public:
    int leftMostColumnWithOne(BinaryMatrix& binaryMatrix) {
        const vector<int> dimensions = binaryMatrix.dimensions();
        const int m = dimensions[0];
        const int n = dimensions[1];
        int ans = -1;
        int l = 0;
        int r = n - 1;

        while (l <= r) {
            const int mid = (l + r) / 2;
            if (existOne(binaryMatrix, m, mid)) {
                ans = mid;
                r = mid - 1;
            } else {
                l = mid + 1;
            }
        }

        return ans;
    }

private:
    bool existOne(BinaryMatrix& binaryMatrix, int m, int col) {
        for (int i = 0; i < m; ++i)
            if (binaryMatrix.get(i, col) == 1)
                return true;
        return false;
    }
};
```

469. 1429.cpp

Path: LeetCode-main/solutions/1429. First Unique Number/1429.cpp

```
class FirstUnique {
public:
    FirstUnique(vector<int>& nums) {
        for (const int num : nums)
            add(num);
    }

    int showFirstUnique() {
        return unique.empty() ? -1 : unique.front();
    }

    void add(int value) {
        const auto it = valueToIterator.find(value);
        if (it == valueToIterator.cend()) {
            unique.push_back(value);
            valueToIterator[value] = prev(unique.end());
        } else if (it->second != unique.end()) {
            // We have added this value before, and this is the second time we're
            // adding it. So, erase the value from `unique` and mark it as erased.
            unique.erase(it->second);
            valueToIterator[value] = unique.end();
        }
    }

private:
    list<int> unique;
    unordered_map<int, list<int>::iterator> valueToIterator;
};
```

470. 143.cpp

Path: LeetCode-main/solutions/143. Reorder List/143.cpp

```
class Solution {
public:
    void reorderList(ListNode* head) {
        if (!head || !head->next)
            return;

        ListNode* mid = findMid(head);
        ListNode* reversed = reverse(mid);
        merge(head, reversed);
    }

private:
    ListNode* findMid(ListNode* head) {
        ListNode* prev = nullptr;
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            prev = slow;
            slow = slow->next;
            fast = fast->next->next;
        }
        prev->next = nullptr;

        return slow;
    }

    ListNode* reverse(ListNode* head) {
        ListNode* prev = nullptr;
        ListNode* curr = head;

        while (curr != nullptr) {
            ListNode* next = curr->next;
            curr->next = prev;
            prev = curr;
            curr = next;
        }

        return prev;
    }

    void merge(ListNode* l1, ListNode* l2) {
        while (l2) {
            ListNode* next = l1->next;
            l1->next = l2;
            l1 = l2;
            l2 = next;
        }
    }
};
```

471. 1430.cpp

Path: LeetCode-main/solutions/1430. Check If a String Is a Valid Sequence from Root to Leaves Path in a Binary Tree/1430.cpp

```
class Solution {
public:
    bool isValidSequence(TreeNode* root, vector<int>& arr) {
        return isValidSequence(root, arr, 0);
    }

private:
    bool isValidSequence(TreeNode* root, const vector<int>& arr, int i) {
        if (root == nullptr)
            return false;
        if (i == arr.size() - 1)
            return root->val == arr[i] && root->left == nullptr &&
                root->right == nullptr;
        return root->val == arr[i] && (isValidSequence(root->left, arr, i + 1) ||
                                       isValidSequence(root->right, arr, i + 1));
    }
};
```

472. 1431.cpp

Path: LeetCode-main/solutions/1431. Kids With the Greatest Number of Candies/1431.cpp

```
class Solution {
public:
    vector<bool> kidsWithCandies(vector<int>& candies, int extraCandies) {
        vector<bool> ans;
        const int maxCandy = ranges::max(candies);

        for (const int candy : candies)
            ans.push_back(candy + extraCandies >= maxCandy);

        return ans;
    }
};
```

473. 1432.cpp

Path: LeetCode-main/solutions/1432. Max Difference You Can Get From Changing an Integer/1432.cpp

```
class Solution {
public:
    int maxDiff(int num) {
        const string s = to_string(num);
        int firstNot9 = s.find_first_not_of('9');
        int firstNot01 = s.find_first_not_of("01");
        if (firstNot9 == string::npos)
            firstNot9 = 0;
        if (firstNot01 == string::npos)
            firstNot01 = 0;

        string a = s;
        string b = s;
        replace(a.begin(), a.end(), s[firstNot9], '9');
        replace(b.begin(), b.end(), s[firstNot01], firstNot01 == 0 ? '1' : '0');
        return stoi(a) - stoi(b);
    }
};
```

474. 1433-2.cpp

Path: LeetCode-main/solutions/1433. Check If a String Can Break Another String/1433-2.cpp

```
class Solution {
public:
    bool checkIfCanBreak(string s1, string s2) {
        vector<int> count(26);

        for (const char c : s1)
            ++count[c - 'a'];

        for (const char c : s2)
            --count[c - 'a'];

        for (int i = 1; i < 26; ++i)
            count[i] += count[i - 1];

        return ranges::all_of(count, [](int c) { return c <= 0; }) ||
            ranges::all_of(count, [](int c) { return c >= 0; });
    }
};
```

475. 1433.cpp

Path: LeetCode-main/solutions/1433. Check If a String Can Break Another String/1433.cpp

```
class Solution {
public:
    bool checkIfCanBreak(string s1, string s2) {
        vector<int> count1(26);
        vector<int> count2(26);

        for (const char c : s1)
            ++count1[c - 'a'];

        for (const char c : s2)
            ++count2[c - 'a'];

        return canBreak(count1, count2) || canBreak(count2, count1);
    }

private:
    // Returns true if count1 can break count2.
    bool canBreak(const vector<int>& count1, const vector<int>& count2) {
        int diff = 0;
        for (int i = 0; i < 26; ++i) {
            diff += count2[i] - count1[i];
            // count2 is alphabetically greater than count1.
            if (diff < 0)
                return false;
        }
        return true;
    }
};
```


476. 1434-2.cpp

Path: LeetCode-main/solutions/1434. Number of Ways to Wear Different Hats to Each Other/1434-2.cpp

```
class Solution {
public:
    int numberWays(vector<vector<int>>& hats) {
        constexpr int kMod = 1'000'000'007;
        constexpr int nHats = 40;
        const int nPeople = hats.size();
        const int nAssignments = 1 << nPeople;
        vector<vector<int>> hatToPeople(nHats + 1);
        // dp[i][j] := the number of ways to assign hats 1, 2, ..., i to people,
        // where j is the bitmask of the current assignment
        vector<vector<int>> dp(nHats + 1, vector<int>(nAssignments));
        dp[0][0] = 1;

        for (int i = 0; i < nPeople; ++i)
            for (const int hat : hats[i])
                hatToPeople[hat].push_back(i);

        for (int h = 1; h <= nHats; ++h)
            for (int j = 0; j < nAssignments; ++j) {
                // We can cover the assignment j in 2 ways:
                // 1. Assign the first h - 1 hats to people without using the hat `h`.
                dp[h][j] += dp[h - 1][j];
                dp[h][j] %= kMod;
                for (const int p : hatToPeople[h])
                    if (j >> p & 1) {
                        // 2. Assign the first h - 1 hats to people without the person `p`
                        // and assign the hat `h` to the person `p`.
                        dp[h][j] += dp[h - 1][j ^ 1 << p];
                        dp[h][j] %= kMod;
                    }
            }

        return dp[nHats][nAssignments - 1];
    }
};
```

477. 1434-3.cpp

Path: LeetCode-main/solutions/1434. Number of Ways to Wear Different Hats to Each Other/1434-3.cpp

```
class Solution {
public:
    int numberWays(vector<vector<int>>& hats) {
        constexpr int kMod = 1'000'000'007;
        constexpr int nHats = 40;
        const int nPeople = hats.size();
        const int nAssignments = 1 << nPeople;
        vector<vector<int>> hatToPeople(nHats + 1);
        // dp[i] := the number of ways to assign the hats so far to people, where i
        // is the bitmask of the current assignment
        vector<int> dp(nAssignments);
        dp[0] = 1;

        for (int i = 0; i < nPeople; ++i)
            for (const int hat : hats[i])
                hatToPeople[hat].push_back(i);

        for (int h = 1; h <= nHats; ++h)
            for (int j = nAssignments - 1; j >= 0; --j)
                for (const int p : hatToPeople[h])
                    if (j >> p & 1) {
                        dp[j] += dp[j ^ 1 << p];
                        dp[j] %= kMod;
                    }

        return dp[nAssignments - 1];
    }
};
```

478. 1434.cpp

Path: LeetCode-main/solutions/1434. Number of Ways to Wear Different Hats to Each Other/1434.cpp

```
class Solution {
public:
    int numberWays(vector<vector<int>>& hats) {
        constexpr int nHats = 40;
        const int nPeople = hats.size();
        vector<vector<int>> hatToPeople(nHats + 1);
        vector<vector<int>> mem(nHats + 1, vector<int>(1 << nPeople, -1));

        for (int i = 0; i < nPeople; ++i)
            for (const int hat : hats[i])
                hatToPeople[hat].push_back(i);

        return numberWays(hats, 0, 1, hatToPeople, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of ways to assign hats 1, 2, ..., h to people, where
    // `assignment` is the bitmask of the current assignment.
    int numberWays(const vector<vector<int>>& hats, int assignment, int h,
                   const vector<vector<int>>& hatToPeople,
                   vector<vector<int>>& mem) {
        // All the people are assigned.
        if (assignment == (1 << hats.size()) - 1)
            return 1;
        if (h > 40)
            return 0;
        if (mem[h][assignment] != -1)
            return mem[h][assignment];

        // Don't wear the hat `h`.
        mem[h][assignment] = numberWays(hats, assignment, h + 1, hatToPeople, mem);

        for (const int p : hatToPeople[h]) {
            // The person `p` was assigned the hat `h` before.
            if (assignment >> p & 1)
                continue;
            // Assign the hat `h` to the person `p`.
            mem[h][assignment] +=
                numberWays(hats, assignment | 1 << p, h + 1, hatToPeople, mem);
            mem[h][assignment] %= kMod;
        }

        return mem[h][assignment];
    }
};
```

479. 1438.cpp

Path: LeetCode-main/solutions/1438. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit/1438.cpp

```
class Solution {
public:
    int longestSubarray(vector<int>& nums, int limit) {
        int ans = 1;
        deque<int> minQ;
        deque<int> maxQ;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            while (!minQ.empty() && minQ.back() > nums[r])
                minQ.pop_back();
            minQ.push_back(nums[r]);
            while (!maxQ.empty() && maxQ.back() < nums[r])
                maxQ.pop_back();
            maxQ.push_back(nums[r]);
            while (maxQ.front() - minQ.front() > limit) {
                if (minQ.front() == nums[l])
                    minQ.pop_front();
                if (maxQ.front() == nums[l])
                    maxQ.pop_front();
                ++l;
            }
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

480. 1439.cpp

Path: LeetCode-main/solutions/1439. Find the Kth Smallest Sum of a Matrix With Sorted Rows/1439.cpp

```
struct T {
    int i;
    int j;
    int sum; // nums1[i] + nums2[j]
};

class Solution {
public:
    int kthSmallest(vector<vector<int>>& mat, int k) {
        vector<int> row = mat[0];

        for (int i = 1; i < mat.size(); ++i)
            row = kSmallestPairSums(row, mat[i], k);

        return row.back();
    }

private:
    // Similar to 373. Find K Pairs with Smallest Sums
    vector<int> kSmallestPairSums(vector<int>& nums1, vector<int>& nums2, int k) {
        vector<int> ans;
        auto compare = [&](const T& a, const T& b) { return a.sum > b.sum; };
        priority_queue<T, vector<T>, decltype(compare)> minHeap(compare);

        for (int i = 0; i < k && i < nums1.size(); ++i)
            minHeap.emplace(i, 0, nums1[i] + nums2[0]);

        while (!minHeap.empty() && ans.size() < k) {
            const auto [i, j, _] = minHeap.top();
            minHeap.pop();
            ans.push_back(nums1[i] + nums2[j]);
            if (j + 1 < nums2.size())
                minHeap.emplace(i, j + 1, nums1[i] + nums2[j + 1]);
        }

        return ans;
    }
};
```

481. 144-2.cpp

Path: LeetCode-main/solutions/144. Binary Tree Preorder Traversal/144-2.cpp

```
class Solution {
public:
    vector<int> preorderTraversal(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<int> ans;
        stack<TreeNode*> stack{{root}};

        while (!stack.empty()) {
            root = stack.top(), stack.pop();
            ans.push_back(root->val);
            if (root->right)
                stack.push(root->right);
            if (root->left)
                stack.push(root->left);
        }

        return ans;
    }
};
```

482. 144.cpp

Path: LeetCode-main/solutions/144. Binary Tree Preorder Traversal/144.cpp

```
class Solution {
public:
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int> ans;
        preorder(root, ans);
        return ans;
    }

private:
    void preorder(TreeNode* root, vector<int>& ans) {
        if (root == nullptr)
            return;

        ans.push_back(root->val);
        preorder(root->left, ans);
        preorder(root->right, ans);
    }
};
```

483. 1443.cpp

Path: LeetCode-main/solutions/1443. Minimum Time to Collect All Apples in a Tree/1443.cpp

```
class Solution {
public:
    int minTime(int n, vector<vector<int>>& edges, vector<bool>& hasApple) {
        vector<vector<int>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        return dfs(graph, 0, vector<bool>(n), hasApple);
    }

private:
    int dfs(const vector<vector<int>>& graph, int u, vector<bool>&& seen,
            const vector<bool>& hasApple) {
        seen[u] = true;
        int totalCost = 0;

        for (const int v : graph[u]) {
            if (seen[v])
                continue;
            const int cost = dfs(graph, v, std::move(seen), hasApple);
            if (cost > 0 || hasApple[v])
                totalCost += cost + 2;
        }

        return totalCost;
    }
};
```


484. 1444.cpp

Path: LeetCode-main/solutions/1444. Number of Ways of Cutting a Pizza/1444.cpp

```
class Solution {
public:
    int ways(vector<string>& pizza, int k) {
        const int M = pizza.size();
        const int N = pizza[0].size();
        vector<vector<vector<int>>> mem(M,
                                     vector<vector<int>>(N, vector<int>(k, -1)));
        vector<vector<int>> prefix(M + 1, vector<int>(N + 1));

        for (int i = 0; i < M; ++i)
            for (int j = 0; j < N; ++j)
                prefix[i + 1][j + 1] = (pizza[i][j] == 'A') + prefix[i][j + 1] +
                                       prefix[i + 1][j] - prefix[i][j];

        return ways(0, 0, k - 1, M, N, prefix, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of ways to cut pizza[m..M][n..N] with k cuts.
    int ways(int m, int n, int k, const int M, const int N,
             const vector<vector<int>>& prefix,
             vector<vector<vector<int>>>& mem) {
        if (k == 0)
            return hasApple(prefix, m, M, n, N) ? 1 : 0;
        if (mem[m][n][k] != -1)
            return mem[m][n][k];

        mem[m][n][k] = 0;

        for (int i = m + 1; i < M; ++i) // Cut horizontally.
            if (hasApple(prefix, m, i, n, N) && hasApple(prefix, i, M, n, N)) {
                mem[m][n][k] += ways(i, n, k - 1, M, N, prefix, mem);
                mem[m][n][k] %= kMod;
            }

        for (int j = n + 1; j < N; ++j) // Cut vertically.
            if (hasApple(prefix, m, M, n, j) && hasApple(prefix, m, M, j, N)) {
                mem[m][n][k] += ways(m, j, k - 1, M, N, prefix, mem);
                mem[m][n][k] %= kMod;
            }

        return mem[m][n][k];
    }

    // Returns true if pizza[row1..row2][col1..col2] has apple.
    bool hasApple(const vector<vector<int>>& prefix, int row1, int row2, int col1,
                  int col2) {
        return (prefix[row2][col2] - prefix[row1][col2] - //
                prefix[row2][col1] + prefix[row1][col1]) > 0;
    };
};
```

485. 1446.cpp

Path: LeetCode-main/solutions/1446. Consecutive Characters/1446.cpp

```
class Solution {
public:
    int maxPower(string s) {
        int ans = 1;
        int count = 1;

        for (int i = 1; i < s.length(); ++i) {
            count = s[i] == s[i - 1] ? count + 1 : 1;
            ans = max(ans, count);
        }

        return ans;
    }
};
```

486. 1447.cpp

Path: LeetCode-main/solutions/1447. Simplified Fractions/1447.cpp

```
class Solution {
public:
    vector<string> simplifiedFractions(int n) {
        vector<string> ans;
        for (int denominator = 2; denominator <= n; ++denominator)
            for (int numerator = 1; numerator < denominator; ++numerator)
                if (__gcd(denominator, numerator) == 1)
                    ans.push_back(to_string(numerator) + "/" + to_string(denominator));
        return ans;
    }
};
```

487. 1448.cpp

Path: LeetCode-main/solutions/1448. Count Good Nodes in Binary Tree/1448.cpp

```
class Solution {
public:
    int goodNodes(TreeNode* root, int mx = INT_MIN) {
        if (root == nullptr)
            return 0;
        const int newMax = max(mx, root->val);
        return (root->val >= mx) + //
            goodNodes(root->left, newMax) + //
            goodNodes(root->right, newMax);
    }
};
```

488. 1449.cpp

Path: LeetCode-main/solutions/1449. Form Largest Integer With Digits That Add up to Target/1449.cpp

```
class Solution {
public:
    string largestNumber(vector<int>& cost, int target) {
        // dp[i] := the maximum length that cost i can achieve
        vector<int> dp(target + 1, INT_MIN);
        dp[0] = 0; // If cost = 0, the best choice is the empty string "".

        for (int i = 1; i <= target; ++i)
            for (int d = 0; d < 9; ++d)
                if (i >= cost[d])
                    dp[i] = max(dp[i], dp[i - cost[d]] + 1);

        if (dp[target] < 0)
            return "";

        string ans;

        // Greedily build the ans string.
        for (int d = 8; d >= 0; --d)
            while (target >= cost[d] && dp[target] == dp[target - cost[d]] + 1) {
                target -= cost[d];
                ans += '1' + d;
            }

        return ans;
    }
};
```

489. 145-2.cpp

Path: LeetCode-main/solutions/145. Binary Tree Postorder Traversal/145-2.cpp

```
class Solution {
public:
    vector<int> postorderTraversal(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<int> ans;
        stack<TreeNode*> stack{{root}};

        while (!stack.empty()) {
            root = stack.top(), stack.pop();
            ans.push_back(root->val);
            if (root->left)
                stack.push(root->left);
            if (root->right)
                stack.push(root->right);
        }

        ranges::reverse(ans);
        return ans;
    }
};
```

490. 145.cpp

Path: LeetCode-main/solutions/145. Binary Tree Postorder Traversal/145.cpp

```
class Solution {
public:
    vector<int> postorderTraversal(TreeNode* root) {
        vector<int> ans;
        postorder(root, ans);
        return ans;
    }

private:
    void postorder(TreeNode* root, vector<int>& ans) {
        if (root == nullptr)
            return;

        postorder(root->left, ans);
        postorder(root->right, ans);
        ans.push_back(root->val);
    }
};
```

491. 1450.cpp

Path: LeetCode-main/solutions/1450. Number of Students Doing Homework at a Given Time/1450.cpp

```
class Solution {
public:
    int busyStudent(vector<int>& startTime, vector<int>& endTime, int queryTime) {
        const int n = startTime.size();
        int ans = 0;

        for (int i = 0; i < n; ++i)
            if (startTime[i] <= queryTime && queryTime <= endTime[i])
                ++ans;

        return ans;
    }
};
```


492. 1453.cpp

Path: LeetCode-main/solutions/1453. Maximum Number of Darts Inside of a Circular Dartboard/1453.cpp

```
struct Point {
    double x;
    double y;
    Point(double x, double y) : x(x), y(y) {}
};

class Solution {
public:
    int numPoints(vector<vector<int>>& darts, int r) {
        int ans = 1;
        vector<Point> points = convertToPoints(darts);

        for (int i = 0; i < points.size(); ++i)
            for (int j = i + 1; j < points.size(); ++j)
                for (const Point& c : getCircles(points[i], points[j], r)) {
                    int count = 0;
                    for (const Point& point : points)
                        if (dist(c, point) - r <= kErr)
                            ++count;
                    ans = max(ans, count);
                }

        return ans;
    }

private:
    static constexpr double kErr = 1e-6;

    vector<Point> convertToPoints(const vector<vector<int>>& darts) {
        vector<Point> points;
        for (const vector<int>& dart : darts)
            points.emplace_back(dart[0], dart[1]);
        return points;
    }

    vector<Point> getCircles(const Point& p, const Point& q, int r) {
        if (dist(p, q) - 2.0 * r > kErr)
            return {};
        const Point m{(p.x + q.x) / 2, (p.y + q.y) / 2};
        const double distCM = sqrt(pow(r, 2) - pow(dist(p, q) / 2, 2));
        const double alpha = atan2(p.y - q.y, q.x - p.x);
        return {Point{m.x - distCM * sin(alpha), m.y - distCM * cos(alpha)},
                Point{m.x + distCM * sin(alpha), m.y + distCM * cos(alpha)}};
    }

    double dist(const Point& p, const Point& q) {
        return sqrt(pow(p.x - q.x, 2) + pow(p.y - q.y, 2));
    }
};
```

493. 1457.cpp

Path: LeetCode-main/solutions/1457. Pseudo-Palindromic Paths in a Binary Tree/1457.cpp

```
class Solution {
public:
    int pseudoPalindromicPaths(TreeNode* root) {
        int ans = 0;
        dfs(root, 0, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, int path, int& ans) {
        if (root == nullptr)
            return;
        if (root->left == nullptr && root->right == nullptr) {
            path ^= 1 << root->val;
            if ((path & (path - 1)) == 0)
                ++ans;
            return;
        }
        dfs(root->left, path ^ 1 << root->val, ans);
        dfs(root->right, path ^ 1 << root->val, ans);
    }
};
```

494. 1458.cpp

Path: LeetCode-main/solutions/1458. Max Dot Product of Two Subsequences/1458.cpp

```
class Solution {
public:
    int maxDotProduct(vector<int>& nums1, vector<int>& nums2) {
        const int m = nums1.size();
        const int n = nums2.size();
        // dp[i][j] := the maximum dot product of the two subsequences nums[0..i)
        // and nums2[0..j)
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, INT_MIN));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                dp[i + 1][j + 1] = max({dp[i][j + 1], dp[i + 1][j],
                                         max(0, dp[i][j]) + nums1[i] * nums2[j]});

        return dp[m][n];
    }
};
```

495. 146-2.cpp

Path: LeetCode-main/solutions/146. LRU Cache/146-2.cpp

```
struct Node {
    int key;
    int value;
};

class LRUCache {
public:
    LRUCache(int capacity) : capacity(capacity) {}

    int get(int key) {
        const auto it = keyToIterator.find(key);
        if (it == keyToIterator.cend())
            return -1;

        const auto& listIt = it->second;
        // Move it to the front.
        cache.splice(cache.begin(), cache, listIt);
        return listIt->value;
    }

    void put(int key, int value) {
        // There's no capacity issue, so just update the value.
        if (const auto it = keyToIterator.find(key); it != keyToIterator.cend()) {
            const auto& listIt = it->second;
            // Move it to the front.
            cache.splice(cache.begin(), cache, listIt);
            listIt->value = value;
            return;
        }

        // Check the capacity.
        if (cache.size() == capacity) {
            const Node& lastNode = cache.back();
            // That's why we store `key` in `Node`.
            keyToIterator.erase(lastNode.key);
            cache.pop_back();
        }

        cache.emplace_front(key, value);
        keyToIterator[key] = cache.begin();
    }

private:
    const int capacity;
    list<Node> cache;
    unordered_map<int, list<Node>::iterator> keyToIterator;
};
```

496. 146.cpp

Path: LeetCode-main/solutions/146. LRU Cache/146.cpp

```
struct Node {
    int key;
    int value;
    shared_ptr<Node> prev;
    shared_ptr<Node> next;
};

class LRUCache {
public:
    LRUCache(int capacity) : capacity(capacity) {
        join(head, tail);
    }

    int get(int key) {
        const auto it = keyToNode.find(key);
        if (it == keyToNode.cend())
            return -1;

        shared_ptr<Node> node = it->second;
        remove(node);
        moveToHead(node);
        return node->value;
    }

    void put(int key, int value) {
        if (const auto it = keyToNode.find(key); it != keyToNode.cend()) {
            shared_ptr<Node> node = it->second;
            node->value = value;
            remove(node);
            moveToHead(node);
            return;
        }

        if (keyToNode.size() == capacity) {
            shared_ptr<Node> lastNode = tail->prev;
            keyToNode.erase(lastNode->key);
            remove(lastNode);
        }

        moveToHead(make_shared<Node>(key, value));
        keyToNode[key] = head->next;
    }

private:
    const int capacity;
    unordered_map<int, shared_ptr<Node>> keyToNode;
    shared_ptr<Node> head = make_shared<Node>(-1, -1);
    shared_ptr<Node> tail = make_shared<Node>(-1, -1);

    void join(shared_ptr<Node> node1, shared_ptr<Node> node2) {
        node1->next = node2;
        node2->prev = node1;
    }

    void moveToHead(shared_ptr<Node> node) {
        join(node, head->next);
        join(head, node);
    }

    void remove(shared_ptr<Node> node) {
        join(node->prev, node->next);
    }
};
```

497. 1460.cpp

Path: LeetCode-main/solutions/1460. Make Two Arrays Equal by Reversing Subarrays/1460.cpp

```
class Solution {
public:
    bool canBeEqual(vector<int>& target, vector<int>& arr) {
        return unordered_multiset<int>(arr.begin(), arr.end()) ==
            unordered_multiset<int>(target.begin(), target.end());
    }
};
```

498. 1461.cpp

Path: LeetCode-main/solutions/1461. Check If a String Contains All Binary Codes of Size K/1461.cpp

```
class Solution {
public:
    bool hasAllCodes(string s, int k) {
        const int n = 1 << k;
        if (s.length() < n)
            return false;

        // used[i] := true if i is a substring of `s`
        vector<bool> used(n);

        int window = k == 1 ? 0 : stoi(s.substr(0, k - 1), nullptr, 2);
        for (int i = k - 1; i < s.length(); ++i) {
            // Include the s[i].
            window = (window << 1) + (s[i] - '0');
            // Discard the s[i - k].
            window &= n - 1;
            used[window] = true;
        }

        return ranges::all_of(used, [](bool u) { return u; });
    }
};
```

499. 1462-2.cpp

Path: LeetCode-main/solutions/1462. Course Schedule IV/1462-2.cpp

```
class Solution {
public:
    vector<bool> checkIfPrerequisite(int numCourses,
                                    vector<vector<int>>& prerequisites,
                                    vector<vector<int>>& queries) {

        vector<bool> ans;
        // isPrerequisite[i][j] := true if course i is a prerequisite of course j
        vector<vector<bool>> isPrerequisite(numCourses, vector<bool>(numCourses));

        for (const vector<int>& prerequisite : prerequisites) {
            const int u = prerequisite[0];
            const int v = prerequisite[1];
            isPrerequisite[u][v] = true;
        }

        for (int k = 0; k < numCourses; ++k)
            for (int i = 0; i < numCourses; ++i)
                for (int j = 0; j < numCourses; ++j)
                    isPrerequisite[i][j] = isPrerequisite[i][j] ||
                                            (isPrerequisite[i][k] && isPrerequisite[k][j]);

        for (const vector<int>& query : queries) {
            const int u = query[0];
            const int v = query[1];
            ans.push_back(isPrerequisite[u][v]);
        }

        return ans;
    }
};
```


500. 1462.cpp

Path: LeetCode-main/solutions/1462. Course Schedule IV/1462.cpp

```
class Solution {
public:
    vector<bool> checkIfPrerequisite(int numCourses,
                                    vector<vector<int>>& prerequisites,
                                    vector<vector<int>>& queries) {

        vector<bool> ans;
        vector<vector<int>> graph(numCourses);
        // isPrerequisite[i][j] := true if course i is a prerequisite of course j
        vector<vector<bool>> isPrerequisite(numCourses, vector<bool>(numCourses));

        for (const vector<int>& prerequisite : prerequisites) {
            const int u = prerequisite[0];
            const int v = prerequisite[1];
            graph[u].push_back(v);
        }

        // DFS from every course.
        for (int i = 0; i < numCourses; ++i)
            dfs(graph, i, isPrerequisite[i]);

        for (const vector<int>& query : queries) {
            const int u = query[0];
            const int v = query[1];
            ans.push_back(isPrerequisite[u][v]);
        }

        return ans;
    }

    void dfs(const vector<vector<int>>& graph, int u, vector<bool>& used) {
        for (const int v : graph[u]) {
            if (used[v])
                continue;
            used[v] = true;
            dfs(graph, v, used);
        }
    }
};
```

501. 1463-2.cpp

Path: LeetCode-main/solutions/1463. Cherry Pickup II/1463-2.cpp

```
class Solution {
public:
    int cherryPickup(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        // dp[x][y1][y2] := the maximum cherries we can collect, where the robot #1
        // is on (x, y1) and the robot #2 is on (x, y2)
        vector<vector<vector<int>>> dp(m + 1,
                                     vector<vector<int>>(n, vector<int>(n)));

        for (int x = m - 1; x >= 0; --x)
            for (int y1 = 0; y1 < n; ++y1)
                for (int y2 = 0; y2 < n; ++y2) {
                    const int currRow = grid[x][y1] + (y1 != y2) * grid[x][y2];
                    for (int d1 = max(0, y1 - 1); d1 < min(n, y1 + 2); ++d1)
                        for (int d2 = max(0, y2 - 1); d2 < min(n, y2 + 2); ++d2)
                            dp[x][y1][y2] = max(dp[x][y1][y2], currRow + dp[x + 1][d1][d2]);
                }

        return dp[0][0][n - 1];
    }
};
```

502. 1463.cpp

Path: LeetCode-main/solutions/1463. Cherry Pickup II/1463.cpp

```
class Solution {
public:
    int cherryPickup(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<vector<int>>> mem(m,
                                     vector<vector<int>>(n, vector<int>(n, -1)));
        return cherryPickup(grid, 0, 0, n - 1, mem);
    }

private:
    // Returns the maximum cherries we can collect, where the robot #1 is on
    // (x, y1) and the robot #2 is on (x, y2).
    int cherryPickup(const vector<vector<int>>& grid, int x, int y1, int y2,
                    vector<vector<vector<int>>>& mem) {
        if (x == grid.size())
            return 0;
        if (y1 < 0 || y1 == grid[0].size() || y2 < 0 || y2 == grid[0].size())
            return 0;
        if (mem[x][y1][y2] != -1)
            return mem[x][y1][y2];

        const int currRow = grid[x][y1] + (y1 == y2 ? 0 : 1) * grid[x][y2];

        for (int d1 = -1; d1 <= 1; ++d1)
            for (int d2 = -1; d2 <= 1; ++d2)
                mem[x][y1][y2] =
                    max(mem[x][y1][y2],
                       currRow + cherryPickup(grid, x + 1, y1 + d1, y2 + d2, mem));

        return mem[x][y1][y2];
    }
};
```

503. 1464-2.cpp

Path: LeetCode-main/solutions/1464. Maximum Product of Two Elements in an Array/1464-2.cpp

```
class Solution {
public:
    int maxProduct(vector<int>& nums) {
        const auto maxIt1 = ranges::max_element(nums);
        *maxIt1 *= -1;
        const int max2 = ranges::max(nums);
        *maxIt1 *= -1;
        return (*maxIt1 - 1) * (max2 - 1);
    }
};
```

504. 1464.cpp

Path: LeetCode-main/solutions/1464. Maximum Product of Two Elements in an Array/1464.cpp

```
class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int max1 = 0;
        int max2 = 0;

        for (const int num : nums)
            if (num > max1)
                max2 = std::exchange(max1, num);
            else if (num > max2)
                max2 = num;

        return (max1 - 1) * (max2 - 1);
    }
};
```

505. 1465.cpp

Path: LeetCode-main/solutions/1465. Maximum Area of a Piece of Cake After Horizontal and Vertical Cuts/1465.cpp

```
class Solution {
public:
    int maxArea(int h, int w, vector<int>& horizontalCuts,
                vector<int>& verticalCuts) {
        constexpr int kMod = 1'000'000'007;
        ranges::sort(horizontalCuts);
        ranges::sort(verticalCuts);

        // the maximum gap of each direction
        int maxGapX = max(verticalCuts[0], w - verticalCuts.back());
        int maxGapY = max(horizontalCuts[0], h - horizontalCuts.back());

        for (int i = 1; i < verticalCuts.size(); ++i)
            maxGapX = max(maxGapX, verticalCuts[i] - verticalCuts[i - 1]);

        for (int i = 1; i < horizontalCuts.size(); ++i)
            maxGapY = max(maxGapY, horizontalCuts[i] - horizontalCuts[i - 1]);

        return static_cast<long>(maxGapX) * maxGapY % kMod;
    }
};
```

506. 1466.cpp

Path: LeetCode-main/solutions/1466. Reorder Routes to Make All Paths Lead to the City Zero/1466.cpp

```
class Solution {
public:
    int minReorder(int n, vector<vector<int>>& connections) {
        vector<vector<int>> graph(n);

        for (const vector<int>& connection : connections) {
            const int u = connection[0];
            const int v = connection[1];
            graph[u].push_back(v);
            graph[v].push_back(-u); // - := u -> v
        }

        return dfs(graph, 0, -1);
    }

private:
    int dfs(const vector<vector<int>>& graph, int u, int prev) {
        int change = 0;

        for (const int v : graph[u]) {
            if (abs(v) == prev)
                continue;
            if (v > 0)
                ++change;
            change += dfs(graph, abs(v), u);
        }

        return change;
    }
};
```

507. 1467.cpp

Path: LeetCode-main/solutions/1467. Probability of a Two Boxes Having The Same Number of Distinct Balls/1467.cpp

```
enum class BoxCase { kEqualBalls, kEqualDistantBalls };

class Solution {
public:
    double getProbability(vector<int>& balls) {
        const int n = accumulate(balls.begin(), balls.end(), 0) / 2;
        return cases(balls, 0, 0, 0, 0, 0, n, BoxCase::kEqualDistantBalls) /
            cases(balls, 0, 0, 0, 0, 0, n, BoxCase::kEqualBalls);
    }

private:
    const vector<int> fact{1, 1, 2, 6, 24, 120, 720};

    // Assume we have two boxes A and B.
    double cases(const vector<int>& balls, int i, int ballsCountA,
        int ballsCountB, int colorsCountA, int colorsCountB, int n,
        BoxCase boxCase) {
        if (ballsCountA > n || ballsCountB > n)
            return 0;
        if (i == balls.size())
            return boxCase == BoxCase::kEqualBalls ? 1 : colorsCountA == colorsCountB;

        double ans = 0;

        // balls taken from A for `balls[i]`
        for (int ballsTakenA = 0; ballsTakenA <= balls[i]; ++ballsTakenA) {
            const int ballsTakenB = balls[i] - ballsTakenA;
            const int newcolorsCountA = colorsCountA + (ballsTakenA > 0);
            const int newcolorsCountB = colorsCountB + (ballsTakenB > 0);
            ans += cases(balls, i + 1, ballsCountA + ballsTakenA,
                ballsCountB + ballsTakenB, newcolorsCountA, newcolorsCountB,
                n, boxCase) /
                (fact[ballsTakenA] * fact[ballsTakenB]);
        }

        return ans;
    }
};
```


508. 1469.cpp

Path: LeetCode-main/solutions/1469. Find All The Lonely Nodes/1469.cpp

```
class Solution {
public:
    vector<int> getLonelyNodes(TreeNode* root) {
        vector<int> ans;

        dfs(root, false, ans);

        return ans;
    }

private:
    void dfs(TreeNode* root, bool isLonely, vector<int>& ans) {
        if (root == nullptr)
            return;
        if (isLonely)
            ans.push_back(root->val);

        dfs(root->left, root->right == nullptr, ans);
        dfs(root->right, root->left == nullptr, ans);
    }
};
```

509. 147.cpp

Path: LeetCode-main/solutions/147. Insertion Sort List/147.cpp

```
class Solution {
public:
    ListNode* insertionSortList(ListNode* head) {
        ListNode dummy(0);
        ListNode* prev = &dummy; // the last and thus largest of the sorted list

        while (head != nullptr) { // the current inserting node
            ListNode* next = head->next; // Cache the next inserting node.
            if (prev->val >= head->val)
                prev = &dummy; // Move `prev` to the front.
            while (prev->next && prev->next->val < head->val)
                prev = prev->next;
            head->next = prev->next;
            prev->next = head;
            head = next; // Update the current inserting node.
        }

        return dummy.next;
    }
};
```

510. 1470.cpp

Path: LeetCode-main/solutions/1470. Shuffle the Array/1470.cpp

```
class Solution {
public:
    vector<int> shuffle(vector<int>& nums, int n) {
        vector<int> ans;
        for (int i = 0; i < n; ++i) {
            ans.push_back(nums[i]);
            ans.push_back(nums[i + n]);
        }
        return ans;
    }
};
```

511. 1471-2.cpp

Path: LeetCode-main/solutions/1471. The k Strongest Values in an Array/1471-2.cpp

```
class Solution {
public:
    vector<int> getStrongest(vector<int>& arr, int k) {
        const int midIndex = (arr.size() - 1) / 2;
        nth_element(arr.begin(), arr.begin() + midIndex, arr.end());
        const int median = arr[midIndex];

        nth_element(arr.begin(), arr.begin() + k, arr.end(), [&](int a, int b) {
            const int da = abs(a - median);
            const int db = abs(b - median);
            return da == db ? a > b : da > db;
        });

        arr.resize(k);
        return arr;
    }
};
```

512. 1471.cpp

Path: LeetCode-main/solutions/1471. The k Strongest Values in an Array/1471.cpp

```
class Solution {
public:
    vector<int> getStrongest(vector<int>& arr, int k) {
        ranges::sort(arr);

        const int n = arr.size();
        const int median = arr[(n - 1) / 2];
        vector<int> ans;

        for (int l = 0, r = n - 1; k > 0; --k)
            if (median - arr[l] > arr[r] - median)
                ans.push_back(arr[l++]);
            else
                ans.push_back(arr[r--]);

        return ans;
    }
};
```

513. 1472-2.cpp

Path: LeetCode-main/solutions/1472. Design Browser History/1472-2.cpp

```
class BrowserHistory {
public:
    BrowserHistory(string homepage) {
        visit(homepage);
    }

    void visit(string url) {
        history.push(url);
        future = stack<string>();
    }

    string back(int steps) {
        while (history.size() > 1 && steps-- > 0)
            future.push(history.top()), history.pop();
        return history.top();
    }

    string forward(int steps) {
        while (!future.empty() && steps-- > 0)
            history.push(future.top()), future.pop();
        return history.top();
    }

private:
    stack<string> history;
    stack<string> future;
};
```

514. 1472-3.cpp

Path: LeetCode-main/solutions/1472. Design Browser History/1472-3.cpp

```
struct Node {
    Node(const string& url) : url(url) {}
    Node* prev = nullptr;
    Node* next = nullptr;
    const string url;
};

class BrowserHistory {
public:
    BrowserHistory(string homepage) {
        curr = new Node(homepage);
    }

    void visit(string url) {
        curr->next = new Node(url);
        curr->next->prev = curr;
        curr = curr->next;
    }

    string back(int steps) {
        while (curr->prev && steps-- > 0)
            curr = curr->prev;
        return curr->url;
    }

    string forward(int steps) {
        while (curr->next && steps-- > 0)
            curr = curr->next;
        return curr->url;
    }

private:
    Node* curr = nullptr;
};
```

515. 1472.cpp

Path: LeetCode-main/solutions/1472. Design Browser History/1472.cpp

```
class BrowserHistory {
public:
    BrowserHistory(string homepage) {
        visit(homepage);
    }

    void visit(string url) {
        if (++index < urls.size())
            urls[index] = url;
        else
            urls.push_back(url);
        lastIndex = index;
    }

    string back(int steps) {
        index = max(0, index - steps);
        return urls[index];
    }

    string forward(int steps) {
        index = min(lastIndex, index + steps);
        return urls[index];
    }

private:
    vector<string> urls;
    int index = -1;
    int lastIndex = -1;
};
```


516. 1473.cpp

Path: LeetCode-main/solutions/1473. Paint House III/1473.cpp

```
class Solution {
public:
    int minCost(vector<int>& houses, vector<vector<int>>& cost, int m, int n,
                int target) {
        vector<vector<vector<int>>> mem(target + 1,
                                       vector<vector<int>>(m, vector<int>(n + 1)));
        // Initialize `prevColor` to 0 (the virtual neighbor).
        const int c = minCost(houses, cost, m, n, target, 0, 0, mem);
        return c == kMax ? -1 : c;
    }

private:
    static constexpr int kMax = 1'000'001;

    // Returns the minimum cost to paint houses[i..m) into k neighborhoods, where
    // there are houses[i - 1] colors = prevColor.
    int minCost(const vector<int>& houses, const vector<vector<int>>& cost,
                const int& m, const int& n, int k, int i, int prevColor,
                vector<vector<vector<int>>>& mem) {
        if (i == m || k < 0)
            return k == 0 ? 0 : kMax;
        if (mem[k][i][prevColor] > 0)
            return mem[k][i][prevColor];
        if (houses[i] > 0) // The house was painted last year.
            return minCost(houses, cost, m, n, k - (prevColor != houses[i]), i + 1,
                           houses[i], mem);

        int res = kMax;

        // Try to paint the houses[i] with each color in 1..n.
        for (int color = 1; color <= n; ++color)
            res = min(res, cost[i][color - 1] + minCost(houses, cost, m, n,
                                                         k - (prevColor != color),
                                                         i + 1, color, mem));

        return mem[k][i][prevColor] = res;
    }
};
```

517. 1474.cpp

Path: LeetCode-main/solutions/1474. Delete N Nodes After M Nodes of a Linked List/1474.cpp

```
class Solution {
public:
    ListNode* deleteNodes(ListNode* head, int m, int n) {
        ListNode* curr = head;
        ListNode* prev = nullptr; // prev.next == curr

        while (curr != nullptr) {
            // Set the m-th node as `prev`.
            for (int i = 0; i < m & curr != nullptr; ++i) {
                prev = curr;
                curr = curr->next;
            }
            // Set the (m + n + 1)-th node as `curr`.
            for (int i = 0; i < n && curr != nullptr; ++i)
                curr = curr->next;
            // Delete the nodes [m + 1..n - 1].
            prev->next = curr;
        }

        return head;
    }
};
```

518. 1475.cpp

Path: LeetCode-main/solutions/1475. Final Prices With a Special Discount in a Shop/1475.cpp

```
class Solution {
public:
    vector<int> finalPrices(vector<int>& prices) {
        vector<int> ans{prices};
        stack<int> stack;

        for (int j = 0; j < prices.size(); ++j) {
            // stack[-1] := i in the problem description.
            while (!stack.empty() && prices[j] <= prices[stack.top()])
                ans[stack.top()] -= prices[j], stack.pop();
            stack.push(j);
        }

        return ans;
    }
};
```

519. 1476.cpp

Path: LeetCode-main/solutions/1476. Subrectangle Queries/1476.cpp

```
class SubrectangleQueries {
public:
    SubrectangleQueries(vector<vector<int>>& rectangle) : rectangle(rectangle) {}

    void updateSubrectangle(int row1, int col1, int row2, int col2,
                           int newValue) {
        updates.push_back({row1, col1, row2, col2, newValue});
    }

    int getValue(int row, int col) {
        for (int i = updates.size() - 1; i >= 0; --i) {
            auto [r1, c1, r2, c2, v] = updates[i];
            if (r1 <= row && row <= r2 && c1 <= col && col <= c2)
                return v;
        }
        return rectangle[row][col];
    }

private:
    const vector<vector<int>>& rectangle;
    vector<array<int, 5>> updates; // [r1, c1, r2, c2, v]
};
```

520. 1477-2.cpp

Path: LeetCode-main/solutions/1477. Find Two Non-overlapping Sub-arrays Each With Target Sum/1477-2.cpp

```
class Solution {
public:
    int minSumOfLengths(vector<int>& arr, int target) {
        int ans = INT_MAX;
        int sum = 0; // the window sum
        // best[i] := the minimum length of subarrays in arr[0..i] that have
        // sum = target
        vector<int> best(arr.size(), INT_MAX);

        for (int l = 0, r = 0; r < arr.size(); ++r) {
            sum += arr[r]; // Expand the window.
            while (sum > target)
                sum -= arr[l++]; // Shrink the window.
            if (sum == target) {
                if (l > 0 && best[l - 1] != INT_MAX)
                    ans = min(ans, best[l - 1] + r - l + 1);
                best[r] = min(best[r], r - l + 1);
            }
            if (r > 0)
                best[r] = min(best[r], best[r - 1]);
        }

        return ans == INT_MAX ? -1 : ans;
    }
};
```

521. 1477.cpp

Path: LeetCode-main/solutions/1477. Find Two Non-overlapping Sub-arrays Each With Target Sum/1477.cpp

```
class Solution {
public:
    int minSumOfLengths(vector<int>& arr, int target) {
        int ans = INT_MAX;
        int leftLength = INT_MAX;
        int prefix = 0;
        unordered_map<int, int> prefixToIndex{{0, -1}};

        for (int i = 0; i < arr.size(); ++i) {
            prefix += arr[i];
            prefixToIndex[prefix] = i;
        }

        prefix = 0;

        for (int i = 0; i < arr.size(); ++i) {
            prefix += arr[i];
            if (const auto it = prefixToIndex.find(prefix - target);
                it != prefixToIndex.cend())
                leftLength = min(leftLength, i - it->second);
            if (leftLength < INT_MAX)
                if (const auto it = prefixToIndex.find(prefix + target);
                    it != prefixToIndex.cend())
                    ans = min(ans, leftLength + it->second - i);
        }

        return ans == INT_MAX ? -1 : ans;
    }
};
```

522. 1478.cpp

Path: LeetCode-main/solutions/1478. Allocate Mailboxes/1478.cpp

```
class Solution {
public:
    int minDistance(vector<int>& houses, int k) {
        const int n = houses.size();
        vector<vector<int>> mem(n, vector<int>(k + 1, INT_MAX));
        // cost[i][j] := the minimum cost to allocate mailboxes between houses[i]
        // and houses[j]
        vector<vector<int>> cost(n, vector<int>(n));

        ranges::sort(houses);

        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j) {
                const int median = houses[(i + j) / 2];
                for (int x = i; x <= j; ++x)
                    cost[i][j] += abs(houses[x] - median);
            }

        return minDistance(houses, 0, k, cost, mem);
    }

private:
    static constexpr int kMax = 1'000'000;

    // Returns the minimum distance to allocate k mailboxes for houses[i..n).
    int minDistance(const vector<int>& houses, int i, int k,
                    const vector<vector<int>>& cost, vector<vector<int>>& mem) {
        if (i == houses.size() && k == 0)
            return 0;
        if (i == houses.size() || k == 0)
            return kMax;
        if (mem[i][k] != INT_MAX)
            return mem[i][k];

        for (int j = i; j < houses.size(); ++j)
            mem[i][k] = min(
                mem[i][k], cost[i][j] + minDistance(houses, j + 1, k - 1, cost, mem));

        return mem[i][k];
    }
};
```

523. 148.cpp

Path: LeetCode-main/solutions/148. Sort List/148.cpp

```
class Solution {
public:
    ListNode* sortList(ListNode* head) {
        const int length = getLength(head);
        ListNode dummy(0, head);

        for (int k = 1; k < length; k *= 2) {
            ListNode* curr = dummy.next;
            ListNode* tail = &dummy;
            while (curr != nullptr) {
                ListNode* l = curr;
                ListNode* r = split(l, k);
                curr = split(r, k);
                auto [mergedHead, mergedTail] = merge(l, r);
                tail->next = mergedHead;
                tail = mergedTail;
            }
        }

        return dummy.next;
    }

private:
    int getLength(ListNode* head) {
        int length = 0;
        for (ListNode* curr = head; curr; curr = curr->next)
            ++length;
        return length;
    }

    ListNode* split(ListNode* head, int k) {
        while (--k && head)
            head = head->next;
        ListNode* rest = head ? head->next : nullptr;
        if (head != nullptr)
            head->next = nullptr;
        return rest;
    }

    pair<ListNode*, ListNode*> merge(ListNode* l1, ListNode* l2) {
        ListNode dummy(0);
        ListNode* tail = &dummy;

        while (l1 && l2) {
            if (l1->val > l2->val)
                swap(l1, l2);
            tail->next = l1;
            l1 = l1->next;
            tail = tail->next;
        }
        tail->next = l1 ? l1 : l2;
        while (tail->next != nullptr)
            tail = tail->next;

        return {dummy.next, tail};
    }
};
```


524. 1480.cpp

Path: LeetCode-main/solutions/1480. Running Sum of 1d Array/1480.cpp

```
class Solution {
public:
    vector<int> runningSum(vector<int>& nums) {
        vector<int> ans;
        int sum = 0;
        for (const int num : nums)
            ans.push_back(sum += num);
        return ans;
    }
};
```

525. 1481.cpp

Path: LeetCode-main/solutions/1481. Least Number of Unique Integers after K Removals/1481.cpp

```
class Solution {
public:
    int findLeastNumOfUniqueInts(vector<int>& arr, int k) {
        unordered_map<int, int> count;
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const int a : arr)
            ++count[a];

        for (const auto& [_, freq] : count)
            minHeap.push(freq);

        // Greedily remove the k least frequent numbers to have the least number of
        // unique integers.
        while (k > 0)
            k -= minHeap.top(), minHeap.pop();

        return minHeap.size() + (k < 0 ? 1 : 0);
    }
};
```

526. 1482.cpp

Path: LeetCode-main/solutions/1482. Minimum Number of Days to Make m Bouquets/1482.cpp

```
class Solution {
public:
    int minDays(vector<int>& bloomDay, int m, int k) {
        if (bloomDay.size() < static_cast<long>(m) * k)
            return -1;

        int l = ranges::min(bloomDay);
        int r = ranges::max(bloomDay);

        while (l < r) {
            const int mid = (l + r) / 2;
            if (getBouquetCount(bloomDay, k, mid) >= m)
                r = mid;
            else
                l = mid + 1;
        }

        return l;
    }

private:
    // Returns the number of bouquets (k flowers needed) can be made after the
    // `waitingDays`..
    int getBouquetCount(const vector<int>& bloomDay, int k, int waitingDays) {
        int bouquetCount = 0;
        int requiredFlowers = k;
        for (const int day : bloomDay)
            if (day > waitingDays) {
                // Reset `requiredFlowers` since there was not enough adjacent flowers.
                requiredFlowers = k;
            } else if (--requiredFlowers == 0) {
                // Use k adjacent flowers to make a bouquet.
                ++bouquetCount;
                requiredFlowers = k;
            }
        return bouquetCount;
    }
};
```

527. 1483.cpp

Path: LeetCode-main/solutions/1483. Kth Ancestor of a Tree Node/1483.cpp

```
class TreeAncestor {
public:
    TreeAncestor(int n, vector<int>& parent)
        : maxLevel(32 - __builtin_clz(n)), dp(n, vector<int>(maxLevel)) {
        // Node i's 2^0 ancestor is its direct parent.
        for (int i = 0; i < n; ++i)
            dp[i][0] = parent[i];

        for (int j = 1; j < maxLevel; ++j)
            for (int i = 0; i < n; ++i)
                if (dp[i][j - 1] == -1) // There's no such ancestor.
                    dp[i][j] = -1;
                else // A(i, 2^j) = A(A(i, 2^{j - 1}), 2^{j - 1})
                    dp[i][j] = dp[dp[i][j - 1]][j - 1];
    }

    int getKthAncestor(int node, int k) {
        for (int j = 0; j < maxLevel && node != -1; ++j)
            if (k >> j & 1)
                node = dp[node][j];
        return node;
    }

private:
    const int maxLevel;
    vector<vector<int>> dp; // dp[i][j] := node i's 2^j-th ancestor
};
```

528. 1485.cpp

Path: LeetCode-main/solutions/1485. Clone Binary Tree With Random Pointer/1485.cpp

```
class Solution {
public:
    NodeCopy* copyRandomBinaryTree(Node* root) {
        if (root == nullptr)
            return nullptr;
        if (const auto it = map.find(root); it != map.cend())
            return it->second;

        NodeCopy* newNode = new NodeCopy(root->val);
        map[root] = newNode;

        newNode->left = copyRandomBinaryTree(root->left);
        newNode->right = copyRandomBinaryTree(root->right);
        newNode->random = copyRandomBinaryTree(root->random);
        return newNode;
    }

private:
    unordered_map<Node*, NodeCopy*> map;
};
```

529. 1486.cpp

Path: LeetCode-main/solutions/1486. XOR Operation in an Array/1486.cpp

```
class Solution {  
public:  
    int xorOperation(int n, int start) {  
        int ans = 0;  
        for (int i = 0; i < n; ++i)  
            ans ^= start + 2 * i;  
        return ans;  
    }  
};
```

530. 1487.cpp

Path: LeetCode-main/solutions/1487. Making File Names Unique/1487.cpp

```
class Solution {
public:
    vector<string> getFolderNames(vector<string>& names) {
        vector<string> ans;
        unordered_map<string, int> nameToSuffix;

        for (const string& name : names)
            if (const auto it = nameToSuffix.find(name); it != nameToSuffix.cend()) {
                int suffix = it->second;
                string newName = getName(name, ++suffix);
                while (nameToSuffix.contains(newName))
                    newName = getName(name, ++suffix);
                nameToSuffix[name] = suffix;
                nameToSuffix[newName] = 0;
                ans.push_back(newName);
            } else {
                nameToSuffix[name] = 0;
                ans.push_back(name);
            }

        return ans;
    }

private:
    string getName(const string& name, int suffix) {
        return name + "(" + to_string(suffix) + ")";
    }
};
```

531. 1488.cpp

Path: LeetCode-main/solutions/1488. Avoid Flood in The City/1488.cpp

```
class Solution {
public:
    vector<int> avoidFlood(vector<int>& rains) {
        vector<int> ans(rains.size(), -1);
        unordered_map<int, int> lakeIdToFullDay;
        set<int> emptyDays; // indices of rains[i] == 0

        for (int i = 0; i < rains.size(); ++i) {
            const int lakeId = rains[i];
            if (lakeId == 0) {
                emptyDays.insert(i);
                continue;
            }
            if (const auto itFullDay = lakeIdToFullDay.find(lakeId);
                itFullDay != lakeIdToFullDay.cend()) {
                // The lake was full in a previous day. Greedily find the closest day
                // to make the lake empty.
                const auto itEmptyDay = emptyDays.upper_bound(itFullDay->second);
                if (itEmptyDay == emptyDays.cend()) // Not found.
                    return {};
                // Empty the lake at this day.
                ans[*itEmptyDay] = lakeId;
                emptyDays.erase(itEmptyDay);
            }
            // The lake with `lakeId` becomes full at the day `i`.
            lakeIdToFullDay[lakeId] = i;
        }

        // Empty an arbitrary lake if there are remaining empty days.
        for (const int emptyDay : emptyDays)
            ans[emptyDay] = 1;

        return ans;
    }
};
```


532. 1489.cpp

Path: LeetCode-main/solutions/1489. Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree/1489.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    vector<vector<int>> findCriticalAndPseudoCriticalEdges(
        int n, vector<vector<int>>& edges) {
        vector<int> criticalEdges;
        vector<int> pseudoCriticalEdges;

        // Record the index information, so edges[i] := (u, v, weight, index).
        for (int i = 0; i < edges.size(); ++i)
            edges[i].push_back(i);

        // Sort by the weight.
        ranges::sort(edges, ranges::less{},
            [](const vector<int>& edge) { return edge[2]; });

        const int mstWeight = getMSTWeight(n, edges, {}, -1);

        for (const vector<int>& edge : edges) {
            const int index = edge[3];
            // Deleting the `edge` increases the MST's weight or makes the MST
            // invalid.
            if (getMSTWeight(n, edges, {}, index) > mstWeight)
                criticalEdges.push_back(index);
            // If an edge can be in any MST, we can always add `edge` to the edge set.
            else if (getMSTWeight(n, edges, edge, -1) == mstWeight)
                pseudoCriticalEdges.push_back(index);
        }

        return {criticalEdges, pseudoCriticalEdges};
    }

private:
    int getMSTWeight(int n, const vector<vector<int>>& edges,
        const vector<int>& firstEdge, int deletedEdgeIndex) {
        int mstWeight = 0;
        UnionFind uf(n);

        if (!firstEdge.empty()) {
            uf.unionByRank(firstEdge[0], firstEdge[1]);
            mstWeight += firstEdge[2];
        }

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int weight = edge[2];
            const int index = edge[3];
            if (index == deletedEdgeIndex)
                continue;
            if (uf.find(u) == uf.find(v))
                continue;
            uf.unionByRank(u, v);
            mstWeight += weight;
        }

        return mstWeight;
    }
};
```

```
        continue;
        uf.unionByRank(u, v);
        mstWeight += weight;
    }

    const int root = uf.find(0);
    for (int i = 0; i < n; ++i)
        if (uf.find(i) != root)
            return INT_MAX;

    return mstWeight;
}
};
```

533. 149.cpp

Path: LeetCode-main/solutions/149. Max Points on a Line/149.cpp

```
class Solution {
public:
    int maxPoints(vector<vector<int>>& points) {
        int ans = 0;

        for (int i = 0; i < points.size(); ++i) {
            unordered_map<pair<int, int>, int, PairHash> slopeCount;
            const vector<int> p1{points[i]};
            int samePoints = 1;
            int maxPoints = 0; // the maximum number of points with the same slope
            for (int j = i + 1; j < points.size(); ++j) {
                const vector<int> p2{points[j]};
                if (p1 == p2)
                    ++samePoints;
                else
                    maxPoints = max(maxPoints, ++slopeCount[getSlope(p1, p2)]);
            }
            ans = max(ans, samePoints + maxPoints);
        }

        return ans;
    }

private:
    pair<int, int> getSlope(const vector<int>& p, const vector<int>& q) {
        const int dx = p[0] - q[0];
        const int dy = p[1] - q[1];
        if (dx == 0)
            return {0, p[0]};
        if (dy == 0)
            return {p[1], 0};
        const int d = __gcd(dx, dy);
        return {dx / d, dy / d};
    }

    struct PairHash {
        size_t operator()(const pair<int, int>& p) const {
            return p.first ^ p.second;
        }
    };
};
```

534. 1490.cpp

Path: LeetCode-main/solutions/1490. Clone N-ary Tree/1490.cpp

```
class Solution {
public:
    Node* cloneTree(Node* root) {
        if (root == nullptr)
            return nullptr;
        if (const auto it = map.find(root); it != map.cend())
            return it->second;

        Node* newNode = new Node(root->val);
        map[root] = newNode;

        for (Node* child : root->children)
            newNode->children.push_back(cloneTree(child));

        return newNode;
    }

private:
    unordered_map<Node*, Node*> map;
};
```

535. 1491.cpp

Path: LeetCode-main/solutions/1491. Average Salary Excluding the Minimum and Maximum Salary/1491.cpp

```
class Solution {
public:
    double average(vector<int>& salary) {
        const double sum = accumulate(salary.begin(), salary.end(), 0.0);
        const int mx = ranges::max(salary);
        const int mn = ranges::min(salary);
        return (sum - mx - mn) / (salary.size() - 2);
    }
};
```

536. 1492.cpp

Path: LeetCode-main/solutions/1492. The kth Factor of n/1492.cpp

```
class Solution {
public:
    int kthFactor(int n, int k) {
        // If i is a divisor of n, then n / i is also a divisor of n. So, we can
        // find all the divisors of n by processing the numbers <= sqrt(n).
        int factor = 1;
        int i = 0; // the i-th factor

        for (; factor * factor < n; ++factor)
            if (n % factor == 0 && ++i == k)
                return factor;

        for (factor = n / factor; factor >= 1; --factor)
            if (n % factor == 0 && ++i == k)
                return n / factor;

        return -1;
    }
};
```

537. 1493-2.cpp

Path: LeetCode-main/solutions/1493. Longest Subarray of 1's After Deleting One Element/1493-2.cpp

```
class Solution {
public:
    int longestSubarray(vector<int>& nums) {
        int l = 0;
        int zeros = 0;

        for (const int num : nums) {
            if (num == 0)
                ++zeros;
            if (zeros > 1 && nums[l++] == 0)
                --zeros;
        }

        return nums.size() - l - 1;
    }
};
```

538. 1493.cpp

Path: LeetCode-main/solutions/1493. Longest Subarray of 1's After Deleting One Element/1493.cpp

```
class Solution {
public:
    int longestSubarray(vector<int>& nums) {
        int ans = 0;
        int zeros = 0;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            if (nums[r] == 0)
                ++zeros;
            while (zeros == 2)
                if (nums[l++] == 0)
                    --zeros;
            ans = max(ans, r - l);
        }
        return ans;
    }
};
```


539. 1494.cpp

Path: LeetCode-main/solutions/1494. Parallel Courses II/1494.cpp

```
class Solution {
public:
    int minNumberOfSemesters(int n, vector<vector<int>>& relations, int k) {
        // dp[i] := the minimum number of semesters to take the courses, where i is
        // the bitmask of the taken courses
        vector<int> dp(1 << n, n);
        // prereq[i] := the bitmask of all the dependencies of the i-th course
        vector<int> prereq(n);

        for (const vector<int>& relation : relations) {
            const int prevCourse = relation[0] - 1;
            const int nextCourse = relation[1] - 1;
            prereq[nextCourse] |= 1 << prevCourse;
        }

        dp[0] = 0; // Don't need time to finish 0 course.

        for (int i = 0; i < dp.size(); ++i) {
            // the bitmask of all the courses can be taken
            int coursesCanBeTaken = 0;
            // Can take the j-th course if i contains all of j's prerequisites.
            for (int j = 0; j < n; ++j)
                if ((i & prereq[j]) == prereq[j])
                    coursesCanBeTaken |= 1 << j;
            // Don't take any course which is already taken.
            // (i represents set of courses that are already taken)
            coursesCanBeTaken &= ~i;
            // Enumerate every bitmask subset of `coursesCanBeTaken`.
            for (unsigned s = coursesCanBeTaken; s > 0;
                 s = (s - 1) & coursesCanBeTaken)
                if (popcount(s) <= k)
                    // Any combination of courses (if <= k) can be taken now.
                    // i | s := combining courses taken with courses can be taken.
                    dp[i | s] = min(dp[i | s], dp[i] + 1);
        }

        return dp.back();
    }
};
```

540. 1496.cpp

Path: LeetCode-main/solutions/1496. Path Crossing/1496.cpp

```
class Solution {
public:
    bool isPathCrossing(string path) {
        set<int> seen;

        seen.insert(0);

        int x = 0;
        int y = 0;

        for (const char c : path) {
            switch (c) {
                case 'N':
                    ++y;
                    break;
                case 'S':
                    --y;
                    break;
                case 'E':
                    ++x;
                    break;
                case 'W':
                    --x;
                    break;
            }
            const int key = x * 20001 + y;
            if (seen.contains(key))
                return true;
            seen.insert(key);
        }
        return false;
    }
};
```

541. 1497.cpp

Path: LeetCode-main/solutions/1497. Check If Array Pairs Are Divisible by k/1497.cpp

```
class Solution {
public:
    bool canArrange(vector<int>& arr, int k) {
        vector<int> count(k);

        for (int a : arr) {
            a %= k;
            ++count[a < 0 ? a + k : a];
        }

        if (count[0] % 2 != 0)
            return false;

        for (int i = 1; i <= k / 2; ++i)
            if (count[i] != count[k - i])
                return false;

        return true;
    }
};
```

542. 1498.cpp

Path: LeetCode-main/solutions/1498. Number of Subsequences That Satisfy the Given Sum Condition/1498.cpp

```
class Solution {
public:
    int numSubseq(vector<int>& nums, int target) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        int ans = 0;
        vector<int> pows(n, 1); // pows[i] = 2^i % kMod

        for (int i = 1; i < n; ++i)
            pows[i] = pows[i - 1] * 2 % kMod;

        ranges::sort(nums);

        for (int l = 0, r = n - 1; l <= r;)
            if (nums[l] + nums[r] <= target) {
                ans += pows[r - l];
                ans %= kMod;
                ++l;
            } else {
                --r;
            }

        return ans;
    }
};
```

543. 1499-2.cpp

Path: LeetCode-main/solutions/1499. Max Value of Equation/1499-2.cpp

```
class Solution {
public:
    int findMaxValueOfEquation(vector<vector<int>>& points, int k) {
        int ans = INT_MIN;
        deque<pair<int, int>> maxQ; // (y - x, x) max queue

        for (const vector<int>& point : points) {
            const int x = point[0];
            const int y = point[1];
            // Remove the invalid points, xj - xi > k.
            while (!maxQ.empty() && x - maxQ.front().second > k)
                maxQ.pop_front();
            if (!maxQ.empty())
                ans = max(ans, x + y + maxQ.front().first);
            // Remove the points that contribute less value and have a bigger x.
            while (!maxQ.empty() && y - x >= maxQ.back().first)
                maxQ.pop_back();
            maxQ.emplace_back(y - x, x);
        }

        return ans;
    }
};
```

544. 1499.cpp

Path: LeetCode-main/solutions/1499. Max Value of Equation/1499.cpp

```
class Solution {
public:
    int findMaxValueOfEquation(vector<vector<int>>& points, int k) {
        int ans = INT_MIN;
        priority_queue<pair<int, int>> maxHeap; // (y - x, x)

        for (const vector<int>& p : points) {
            const int x = p[0];
            const int y = p[1];
            while (!maxHeap.empty() && x - maxHeap.top().second > k)
                maxHeap.pop();
            if (!maxHeap.empty())
                ans = max(ans, x + y + maxHeap.top().first);
            maxHeap.emplace(y - x, x);
        }

        return ans;
    }
};
```

545. 15.cpp

Path: LeetCode-main/solutions/15. 3Sum/15.cpp

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        if (nums.size() < 3)
            return {};

        vector<vector<int>> ans;

        ranges::sort(nums);

        for (int i = 0; i + 2 < nums.size(); ++i) {
            if (i > 0 && nums[i] == nums[i - 1])
                continue;
            // Choose nums[i] as the first number in the triplet, then search the
            // remaining numbers in [i + 1, n - 1].
            int l = i + 1;
            int r = nums.size() - 1;
            while (l < r) {
                const int sum = nums[i] + nums[l] + nums[r];
                if (sum == 0) {
                    ans.push_back({nums[i], nums[l++], nums[r--]});
                    while (l < r && nums[l] == nums[l - 1])
                        ++l;
                    while (l < r && nums[r] == nums[r + 1])
                        --r;
                } else if (sum < 0) {
                    ++l;
                } else {
                    --r;
                }
            }
        }

        return ans;
    }
};
```

546. 150.cpp

Path: LeetCode-main/solutions/150. Evaluate Reverse Polish Notation/150.cpp

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<long> stack;
        const unordered_map<string, function<long(long, long)>> op{
            {"+", std::plus<long>()},
            {"-", std::minus<long>()},
            {"*", std::multiplies<long>()},
            {"/", std::divides<long>()}};

        for (const string& token : tokens)
            if (op.contains(token)) {
                const long b = stack.top();
                stack.pop();
                const long a = stack.top();
                stack.pop();
                stack.push(op.at(token)(a, b));
            } else {
                stack.push(stoi(token));
            }

        return stack.top();
    }
};
```


547. 1500.cpp

Path: LeetCode-main/solutions/1500. Design a File Sharing System/1500.cpp

```
class FileSharing {
public:
    FileSharing(int m) {}

    int join(vector<int> ownedChunks) {
        const int userId = getMinUserId();
        userToChunks[userId] = {ownedChunks.begin(), ownedChunks.end()};
        for (const int chunk : ownedChunks)
            chunkToUsers[chunk].insert(userId);
        return userId;
    }

    void leave(int userID) {
        for (const int chunk : userToChunks[userID]) {
            chunkToUsers[chunk].erase(userID);
            if (chunkToUsers[chunk].empty())
                chunkToUsers.erase(chunk);
        }
        userToChunks.erase(userID);
        availableUserIds.push(userID);
    }

    vector<int> request(int userID, int chunkID) {
        const auto it = chunkToUsers.find(chunkID);
        if (it == chunkToUsers.end())
            return {};
        vector<int> userIds{it->second.begin(), it->second.end()};
        userToChunks[userID].insert(chunkID);
        chunkToUsers[chunkID].insert(userID);
        return userIds;
    }

private:
    unordered_map<int, set<int>> userToChunks;
    unordered_map<int, set<int>> chunkToUsers;
    priority_queue<int, vector<int>, greater<>> availableUserIds;

    int getMinUserId() {
        if (availableUserIds.empty())
            return userToChunks.size() + 1;
        const int minUserId = availableUserIds.top();
        availableUserIds.pop();
        return minUserId;
    }
};
```

548. 1502-2.cpp

Path: LeetCode-main/solutions/1502. Can Make Arithmetic Progression From Sequence/1502-2.cpp

```
class Solution {
public:
    bool canMakeArithmeticProgression(vector<int>& arr) {
        const int n = arr.size();
        const int mx = ranges::max(arr);
        const int mn = ranges::min(arr);
        const int range = mx - mn;
        if (range % (n - 1) != 0)
            return false;
        const int diff = range / (n - 1);
        if (diff == 0)
            return true;

        for (int i = 0; i < n; i++) {
            const int gap = arr[i] - mn;
            if (gap % diff != 0)
                return false;
            if (gap == i * diff) {
                ++i;
            } else {
                const int rightIndex = gap / diff;
                swap(arr[i], arr[rightIndex]);
            }
        }

        return true;
    }
};
```

549. 1502.cpp

Path: LeetCode-main/solutions/1502. Can Make Arithmetic Progression From Sequence/1502.cpp

```
class Solution {
public:
    bool canMakeArithmeticProgression(vector<int>& arr) {
        const int n = arr.size();
        const int mx = ranges::max(arr);
        const int mn = ranges::min(arr);
        const int range = mx - mn;
        if (range % (n - 1) != 0)
            return false;
        const int diff = range / (n - 1);
        if (diff == 0)
            return true;

        unordered_set<int> seen;

        for (const int a : arr) {
            if ((a - min) % diff != 0)
                return false;
            if (!seen.insert(a).second)
                return false;
        }

        return true;
    }
};
```

550. 1503.cpp

Path: LeetCode-main/solutions/1503. Last Moment Before All Ants Fall Out of a Plank/1503.cpp

```
class Solution {
public:
    int getLastMoment(int n, vector<int>& left, vector<int>& right) {
        const int maxLeft = left.empty() ? 0 : ranges::max(left);
        const int minRight = right.empty() ? n : ranges::min(right);
        return max(maxLeft, n - minRight);
    }
};
```

551. 1504-2.cpp

Path: LeetCode-main/solutions/1504. Count Submatrices With All Ones/1504-2.cpp

```
class Solution {
public:
    int numSubmat(vector<vector<int>>& mat) {
        int ans = 0;
        vector<int> hist(mat[0].size());

        for (const vector<int>& row : mat) {
            for (int i = 0; i < row.size(); ++i)
                hist[i] = row[i] == 0 ? 0 : hist[i] + 1;
            ans += count(hist);
        }

        return ans;
    }

private:
    int count(const vector<int>& hist) {
        // submatrices[i] := the number of submatrices, where the i-th column is the
        // right border
        vector<int> submatrices(hist.size());
        stack<int> stack;

        for (int i = 0; i < hist.size(); ++i) {
            while (!stack.empty() && hist[stack.top()] >= hist[i])
                stack.pop();
            if (!stack.empty()) {
                const int prevIndex = stack.top();
                submatrices[i] = submatrices[prevIndex] + hist[i] * (i - prevIndex);
            } else {
                submatrices[i] = hist[i] * (i + 1);
            }
            stack.push(i);
        }

        return accumulate(submatrices.begin(), submatrices.end(), 0);
    }
};
```

552. 1504.cpp

Path: LeetCode-main/solutions/1504. Count Submatrices With All Ones/1504.cpp

```
class Solution {
public:
    int numSubmat(vector<vector<int>>& mat) {
        const int m = mat.size();
        const int n = mat[0].size();
        int ans = 0;

        for (int baseRow = 0; baseRow < m; ++baseRow) {
            vector<int> row(n, 1);
            for (int i = baseRow; i < m; ++i) {
                for (int j = 0; j < n; ++j)
                    row[j] &= mat[i][j];
                ans += count(row);
            }
        }

        return ans;
    }

private:
    int count(vector<int>& row) {
        int res = 0;
        int length = 0;
        for (const int num : row) {
            length = num == 0 ? 0 : length + 1;
            res += length;
        }
        return res;
    }
};
```

553. 1505.cpp

Path: LeetCode-main/solutions/1505. Minimum Possible Integer After at Most K Adjacent Swaps On Digits/1505.cpp

```
class FenwickTree {
public:
    FenwickTree(int n) : sums(n + 1) {}

    void add(int i, int delta) {
        while (i < sums.size()) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    int get(int i) const {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

private:
    vector<int> sums;

    static inline int lowbit(int i) {
        return i & -i;
    }
};

class Solution {
public:
    string minInteger(string num, int k) {
        const int n = num.length();
        string ans;
        FenwickTree tree(n);
        vector<bool> used(n);
        vector<queue<int>> numToIndices(10);

        for (int i = 0; i < n; ++i)
            numToIndices[num[i] - '0'].push(i);

        while (k > 0 && ans.length() < n)
            for (int d = 0; d < 10; ++d) {
                if (numToIndices[d].empty())
                    continue;
                const int i = numToIndices[d].front();
                const int cost = i - tree.get(i);
                if (cost > k)
                    continue;
                k -= cost;
                ans += '0' + d;
                used[i] = true;
                tree.add(i + 1, 1);
                numToIndices[d].pop();
                break; // Scan from 0 -> 9 again.
            }

        for (int i = 0; i < n; ++i)
            if (!used[i])
                ans += num[i];

        return ans;
    }
};
```

554. 1506.cpp

Path: LeetCode-main/solutions/1506. Find Root of N-Ary Tree/1506.cpp

```
class Solution {
public:
    Node* findRoot(vector<Node*> tree) {
        int sum = 0;

        for (Node* node : tree) {
            sum ^= node->val;
            for (Node* child : node->children)
                sum ^= child->val;
        }

        for (Node* node : tree)
            if (node->val == sum)
                return node;

        throw;
    }
};
```


555. 1507.cpp

Path: LeetCode-main/solutions/1507. Reformat Date/1507.cpp

```
class Solution {
public:
    string reformatDate(string date) {
        const unordered_map<string, string> monthToNumString{
            {"Jan", "01"}, {"Feb", "02"}, {"Mar", "03"}, {"Apr", "04"},
            {"May", "05"}, {"Jun", "06"}, {"Jul", "07"}, {"Aug", "08"},
            {"Sep", "09"}, {"Oct", "10"}, {"Nov", "11"}, {"Dec", "12"},
        };
        const int index1 = date.find_first_of(' ');
        const int index2 = date.find_first_of(' ', index1 + 1);
        const string day = index1 == 4 ? date.substr(0, 2) : string("0") + date[0];
        const string month =
            monthToNumString.at(date.substr(index1 + 1, index2 - (index1 + 1)));
        const string year = date.substr(index2 + 1);
        return year + "-" + month + "-" + day;
    }
};
```

556. 1508.cpp

Path: LeetCode-main/solutions/1508. Range Sum of Sorted Subarray Sums/1508.cpp

```
class Solution {
public:
    int rangeSum(vector<int>& nums, int n, int left, int right) {
        constexpr int kMod = 1'000'000'007;

        auto subarraysAndSumNoGreaterThan = [&](int m) -> pair<int, long> {
            int count = 0;    // the number of subarrays <= m
            long total = 0;    // sum(subarrays)
            int sum = 0;       // the current sum
            int window = 0;    // the window sum

            for (int i = 0, j = 0; j < n; ++j) {
                sum += nums[j] * (j - i + 1);
                window += nums[j];    // Extend each subarray that ends in j.
                while (window > m) {
                    sum -= window;
                    window -= nums[i++];    // Shrink the window.
                }
                count += j - i + 1;
                total += sum;
            }

            return {count, total};
        };

        // [L, R] is the possible range of the sum of any subarray.
        const int L = ranges::min(nums);
        const int R = accumulate(nums.begin(), nums.end(), 0);

        auto firstKSubarraysSum = [&](int k) -> long {
            int l = L;
            int r = R;

            while (l < r) {
                const int m = l + (r - l) / 2;
                if (subarraysAndSumNoGreaterThan(m).first < k)
                    l = m + 1;
                else
                    r = m;
            }

            const auto& [count, total] = subarraysAndSumNoGreaterThan(l);
            // If count != k, there're subarray(s) have the same sum as l.
            return total - l * (count - k);
        };

        return (firstKSubarraysSum(right) - firstKSubarraysSum(left - 1)) % kMod;
    }
};
```

557. 1509.cpp

Path: LeetCode-main/solutions/1509. Minimum Difference Between Largest and Smallest Value in Three Moves/1509.cpp

```
class Solution {
public:
    int minDifference(vector<int>& nums) {
        const int n = nums.size();
        if (n < 5)
            return 0;

        int ans = INT_MAX;

        ranges::sort(nums);

        // 1. Change nums[0..i) to nums[i].
        // 2. Change nums[n - 3 + i..n) to nums[n - 4 + i].
        for (int i = 0; i <= 3; ++i)
            ans = min(ans, nums[n - 4 + i] - nums[i]);

        return ans;
    }
};
```

558. 151.cpp

Path: LeetCode-main/solutions/151. Reverse Words in a String/151.cpp

```
class Solution {
public:
    string reverseWords(string s) {
        ranges::reverse(s);           // Reverse the whole string.
        reverseWords(s, s.length()); // Reverse each word.
        return cleanSpaces(s, s.length()); // Clean up the spaces.
    }

private:
    void reverseWords(string& s, int n) {
        int i = 0;
        int j = 0;

        while (i < n) {
            while (i < j || i < n && s[i] == ' ') // Skip the spaces.
                ++i;
            while (j < i || j < n && s[j] != ' ') // Skip the spaces.
                ++j;
            reverse(s.begin() + i, s.begin() + j); // Reverse the word.
        }

        // Trim leading, trailing, and middle spaces
        string cleanSpaces(string& s, int n) {
            int i = 0;
            int j = 0;

            while (j < n) {
                while (j < n && s[j] == ' ') // Skip the spaces.
                    ++j;
                while (j < n && s[j] != ' ') // Keep non spaces
                    s[i++] = s[j++];
                while (j < n && s[j] == ' ') // Skip the spaces.
                    ++j;
                if (j < n) // Keep only one space.
                    s[i++] = ' ';
            }

            return s.substr(0, i);
        }
    }
};
```

559. 1510.cpp

Path: LeetCode-main/solutions/1510. Stone Game IV/1510.cpp

```
class Solution {
public:
    bool winnerSquareGame(int n) {
        // dp[i] := the winning result for n = i
        vector<bool> dp(n + 1);

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j * j <= i; ++j)
                if (!dp[i - j * j]) { // Removing j^2 stones make the opponent lose.
                    dp[i] = true;    // So, we win.
                    break;
                }

        return dp[n];
    }
};
```

560. 1512.cpp

Path: LeetCode-main/solutions/1512. Number of Good Pairs/1512.cpp

```
class Solution {
public:
    int numIdenticalPairs(vector<int>& nums) {
        int ans = 0;
        vector<int> count(101);

        for (const int num : nums)
            ans += count[num]++;

        return ans;
    }
};
```

561. 1513.cpp

Path: LeetCode-main/solutions/1513. Number of Substrings With Only 1s/1513.cpp

```
class Solution {
public:
    int numSub(string s) {
        constexpr int kMod = 1'000'000'007;

        int ans = 0;
        int l = -1;

        for (int i = 0; i < s.length(); ++i) {
            if (s[i] == '0')
                l = i; // Handle the reset value.
            ans = (ans + i - l) % kMod;
        }

        return ans;
    }
};
```

562. 1514.cpp

Path: LeetCode-main/solutions/1514. Path with Maximum Probability/1514.cpp

```
class Solution {
public:
    double maxProbability(int n, vector<vector<int>>& edges,
                          vector<double>& succProb, int start, int end) {
        // {a: [(b, probability_ab)]}
        vector<vector<pair<int, double>>> graph(n);
        // (the probability to reach u, u)
        priority_queue<pair<double, int>> maxHeap;
        maxHeap.emplace(1.0, start);
        vector<bool> seen(n);

        for (int i = 0; i < edges.size(); ++i) {
            const int u = edges[i][0];
            const int v = edges[i][1];
            const double prob = succProb[i];
            graph[u].emplace_back(v, prob);
            graph[v].emplace_back(u, prob);
        }

        while (!maxHeap.empty()) {
            const auto [prob, u] = maxHeap.top();
            maxHeap.pop();
            if (u == end)
                return prob;
            if (seen[u])
                continue;
            seen[u] = true;
            for (const auto& [nextNode, edgeProb] : graph[u]) {
                if (seen[nextNode])
                    continue;
                maxHeap.emplace(prob * edgeProb, nextNode);
            }
        }

        return 0;
    }
};
```


563. 1515.cpp

Path: LeetCode-main/solutions/1515. Best Position for a Service Centre/1515.cpp

```
class Solution {
public:
    double getMinDistSum(vector<vector<int>>& positions) {
        constexpr double kErr = 1e-6;
        double currX = 50;
        double currY = 50;
        double ans = distSum(positions, currX, currY);
        double step = 1;

        while (step > kErr) {
            bool shouldDecreaseStep = true;
            for (const auto& [dx, dy] : vector<pair<double, double>>{
                {0, step}, {0, -step}, {step, 0}, {-step, 0}}) {
                const double x = currX + dx;
                const double y = currY + dy;
                const double newDistSum = distSum(positions, x, y);
                if (newDistSum < ans) {
                    ans = newDistSum;
                    currX = x;
                    currY = y;
                    shouldDecreaseStep = false;
                }
            }
            if (shouldDecreaseStep)
                step /= 10;
        }

        return ans;
    }

private:
    double distSum(const vector<vector<int>>& positions, double a, double b) {
        double sum = 0;
        for (const vector<int>& p : positions)
            sum += sqrt(pow(a - p[0], 2) + pow(b - p[1], 2));
        return sum;
    }
};
```

564. 1516.cpp

Path: LeetCode-main/solutions/1516. Move Sub-Tree of N-Ary Tree/1516.cpp

```
class Solution {
public:
    Node* moveSubTree(Node* root, Node* p, Node* q) {
        if (find(q->children.begin(), q->children.end(), p) != q->children.end())
            return root;

        // Create a dummy node for the case when root == p.
        Node* dummy = new Node(0, {root});

        // Get each parent of p and q.
        Node* pParent = getParent(dummy, p);
        Node* qParent = getParent(p, q);

        // Get p's original index in p's parent.
        vector<Node*>& pSiblings = pParent->children;
        const int pIndex =
            find(pSiblings.begin(), pSiblings.end(), p) - pSiblings.begin();
        pSiblings.erase(pSiblings.begin() + pIndex);

        q->children.push_back(p);

        // If q is in p's subtree, qParent != nullptr.
        if (qParent != nullptr) {
            vector<Node*>& qSiblings = qParent->children;
            std::erase(qSiblings, q);
            pSiblings.insert(pSiblings.begin() + pIndex, q);
        }

        return dummy->children[0];
    }

private:
    Node* getParent(Node* root, Node* target) {
        for (Node* child : root->children) {
            if (child == target)
                return root;
            Node* res = getParent(child, target);
            if (res != nullptr)
                return res;
        }
        return nullptr;
    }
};
```

565. 1518-2.cpp

Path: LeetCode-main/solutions/1518. Water Bottles/1518-2.cpp

```
class Solution {
public:
    int numWaterBottles(int numBottles, int numExchange) {
        return numBottles + (numBottles - 1) / (numExchange - 1);
    }
};
```

566. 1518.cpp

Path: LeetCode-main/solutions/1518. Water Bottles/1518.cpp

```
class Solution {
public:
    int numWaterBottles(int numBottles, int numExchange) {
        int ans = numBottles;

        while (numBottles >= numExchange) {
            ans += numBottles / numExchange;
            numBottles = numBottles / numExchange + numBottles % numExchange;
        }

        return ans;
    }
};
```

567. 1519.cpp

Path: LeetCode-main/solutions/1519. Number of Nodes in the Sub-Tree With the Same Label/1519.cpp

```
class Solution {
public:
    vector<int> countSubTrees(int n, vector<vector<int>>& edges, string labels) {
        vector<int> ans(n);
        vector<vector<int>> tree(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        dfs(tree, 0, -1, labels, ans);
        return ans;
    }

private:
    vector<int> dfs(const vector<vector<int>>& tree, int u, int prev,
                   const string& labels, vector<int>& ans) {
        // count[i] := the number of letters down from ('a' + i)
        vector<int> count(26);

        for (const int v : tree[u]) {
            if (v == prev)
                continue;
            vector<int> childCount = dfs(tree, v, u, labels, ans);
            for (int i = 0; i < 26; ++i)
                count[i] += childCount[i];
        }

        ans[u] = ++count[labels[u] - 'a']; // the u itself
        return count;
    }
};
```

568. 152.cpp

Path: LeetCode-main/solutions/152. Maximum Product Subarray/152.cpp

```
class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int ans = nums[0];
        int dpMin = nums[0]; // the minimum so far
        int dpMax = nums[0]; // the maximum so far

        for (int i = 1; i < nums.size(); ++i) {
            const int num = nums[i];
            const int prevMin = dpMin; // dpMin[i - 1]
            const int prevMax = dpMax; // dpMax[i - 1]
            if (num < 0) {
                dpMin = min(prevMax * num, num);
                dpMax = max(prevMin * num, num);
            } else {
                dpMin = min(prevMin * num, num);
                dpMax = max(prevMax * num, num);
            }
            ans = max(ans, dpMax);
        }

        return ans;
    }
};
```

569. 1520.cpp

Path: LeetCode-main/solutions/1520. Maximum Number of Non-Overlapping Substrings/1520.cpp

```
class Solution {
public:
    vector<string> maxNumOfSubstrings(string s) {
        const int n = s.length();
        vector<string> ans;
        // leftmost[i] := the leftmost index of ('a' + i)
        vector<int> leftmost(26, n);
        // rightmost[i] := the rightmost index of ('a' + i)
        vector<int> rightmost(26, -1);

        for (int i = 0; i < n; ++i) {
            leftmost[s[i] - 'a'] = min(leftmost[s[i] - 'a'], i);
            rightmost[s[i] - 'a'] = i;
        }

        auto getNewRight = [&](int i) {
            int right = rightmost[s[i] - 'a'];
            for (int j = i; j <= right; ++j) {
                if (leftmost[s[j] - 'a'] < i) // Find a letter's leftmost index < i.
                    return -1;
                // Expand the right dynamically.
                right = max(right, rightmost[s[j] - 'a']);
            }
            return right;
        };

        int right = -1; // the rightmost index of the last substring
        for (int i = 0; i < n; ++i) {
            // the current index is the first appearance
            if (i == leftmost[s[i] - 'a']) {
                const int newRight = getNewRight(i);
                if (newRight == -1)
                    continue; // Find a letter's leftmost index < i.
                if (i <= right && !ans.empty())
                    ans.back() = s.substr(i, newRight - i + 1);
                else
                    ans.push_back(s.substr(i, newRight - i + 1));
                right = newRight;
            }
        }

        return ans;
    }
};
```

570. 1521.cpp

Path: LeetCode-main/solutions/1521. Find a Value of a Mysterious Function Closest to Target/1521.cpp

```
class Solution {
public:
    int closestToTarget(vector<int>& arr, int target) {
        int ans = INT_MAX;
        // all the values of subarrays that end in the previous number
        unordered_set<int> prev;

        for (const int num : arr) {
            unordered_set<int> curr{num};
            // Extend each subarray that ends in the previous number. Due to
            // monotonicity of the AND operation, the size of `curr` will be at most
            // num.bit_count() + 1.
            for (const int val : prev)
                curr.insert(val & num);
            for (const int val : curr)
                ans = min(ans, abs(target - val));
            prev = std::move(curr);
        }

        return ans;
    }
};
```


571. 1522.cpp

Path: LeetCode-main/solutions/1522. Diameter of N-Ary Tree/1522.cpp

```
class Solution {
public:
    int diameter(Node* root) {
        int ans = 0;
        maxDepth(root, ans);
        return ans;
    }

private:
    // Returns the maximum depth of the subtree rooted at `root`.
    int maxDepth(Node* root, int& ans) {
        int maxSubDepth1 = 0;
        int maxSubDepth2 = 0;
        for (Node* child : root->children) {
            const int maxSubDepth = maxDepth(child, ans);
            if (maxSubDepth > maxSubDepth1) {
                maxSubDepth2 = maxSubDepth1;
                maxSubDepth1 = maxSubDepth;
            } else if (maxSubDepth > maxSubDepth2) {
                maxSubDepth2 = maxSubDepth;
            }
        }
        ans = max(ans, maxSubDepth1 + maxSubDepth2);
        return 1 + maxSubDepth1;
    }
};
```

572. 1523.cpp

Path: LeetCode-main/solutions/1523. Count Odd Numbers in an Interval Range/1523.cpp

```
class Solution {  
public:  
    int countOdds(int low, int high) {  
        return (high + 1) / 2 - low / 2;  
    }  
};
```

573. 1524-2.cpp

Path: LeetCode-main/solutions/1524. Number of Sub-arrays With Odd Sum/1524-2.cpp

```
class Solution {
public:
    int numOfSubarrays(vector<int>& arr) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        long dp0 = 0;
        long dp1 = 0;

        for (const int a : arr) {
            if (a % 2 == 1) {
                const int cache = dp0;
                dp0 = dp1;
                dp1 = cache + 1;
            } else {
                ++dp0;
            }
            ans = (ans + dp1) % kMod;
        }

        return ans;
    }
};
```

574. 1524.cpp

Path: LeetCode-main/solutions/1524. Number of Sub-arrays With Odd Sum/1524.cpp

```
class Solution {
public:
    int numOfSubarrays(vector<int>& arr) {
        constexpr int kMod = 1'000'000'007;
        const int n = arr.size();
        long ans = 0;
        // dp0[i] := the number of subarrays that end in arr[i - 1] with an even sum
        vector<int> dp0(n + 1);
        // dp1[i] := the number of subarrays that end in arr[i - 1] with an odd sum
        vector<int> dp1(n + 1);

        for (int i = 1; i <= n; ++i) {
            if (arr[i - 1] % 2 == 1) {
                dp0[i] = dp1[i - 1];
                dp1[i] = dp0[i - 1] + 1;
            } else {
                dp0[i] = dp0[i - 1] + 1;
                dp1[i] = dp1[i - 1];
            }
            ans = (ans + dp1[i]) % kMod;
        }

        return ans;
    }
};
```

575. 1525.cpp

Path: LeetCode-main/solutions/1525. Number of Good Ways to Split a String/1525.cpp

```
class Solution {
public:
    int numSplits(string s) {
        const int n = s.length();
        int ans = 0;
        // prefix[i] := the number of unique letters in s[0..i]
        vector<int> prefix(n);
        // suffix[i] := of unique letters in s[i..n)
        vector<int> suffix(n);
        unordered_set<int> seen;

        for (int i = 0; i < n; ++i) {
            seen.insert(s[i]);
            prefix[i] = seen.size();
        }

        seen.clear();

        for (int i = n - 1; i >= 0; --i) {
            seen.insert(s[i]);
            suffix[i] = seen.size();
        }

        for (int i = 0; i + 1 < n; ++i)
            if (prefix[i] == suffix[i + 1])
                ++ans;

        return ans;
    }
};
```

576. 1526.cpp

Path: LeetCode-main/solutions/1526. Minimum Number of Increments on Subarrays to Form a Target Array/1526.cpp

```
class Solution {
public:
    int minNumberOperations(vector<int>& target) {
        int ans = target.front();

        for (int i = 1; i < target.size(); ++i)
            if (target[i] > target[i - 1])
                ans += target[i] - target[i - 1];

        return ans;
    }
};
```

577. 1528.cpp

Path: LeetCode-main/solutions/1528. Shuffle String/1528.cpp

```
class Solution {
public:
    string restoreString(string s, vector<int>& indices) {
        string ans(s.length(), '.');

        for (int i = 0; i < indices.size(); ++i)
            ans[indices[i]] = s[i];

        return ans;
    }
};
```

578. 1529.cpp

Path: LeetCode-main/solutions/1529. Minimum Suffix Flips/1529.cpp

```
class Solution {
public:
    int minFlips(string target) {
        int ans = 0;
        int state = 0;

        for (const char c : target)
            if (c - '0' != state) {
                state = c - '0';
                ++ans;
            }

        return ans;
    }
};
```


579. 153.cpp

Path: LeetCode-main/solutions/153. Find Minimum in Rotated Sorted Array/153.cpp

```
class Solution {
public:
    int findMin(vector<int>& nums) {
        int l = 0;
        int r = nums.size() - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (nums[m] < nums[r])
                r = m;
            else
                l = m + 1;
        }
        return nums[l];
    }
};
```

580. 1530.cpp

Path: LeetCode-main/solutions/1530. Number of Good Leaf Nodes Pairs/1530.cpp

```
class Solution {
public:
    int countPairs(TreeNode* root, int distance) {
        int ans = 0;

        dfs(root, distance, ans);

        return ans;
    }

private:
    vector<int> dfs(TreeNode* root, int distance, int& ans) {
        vector<int> d(distance + 1); // {distance: the number of leaf nodes}
        if (root == nullptr)
            return d;
        if (root->left == nullptr && root->right == nullptr) {
            d[0] = 1;
            return d;
        }

        const vector<int> dl = dfs(root->left, distance, ans);
        const vector<int> dr = dfs(root->right, distance, ans);

        for (int i = 0; i < distance; ++i)
            for (int j = 0; j < distance; ++j)
                if (i + j + 2 <= distance)
                    ans += dl[i] * dr[j];

        for (int i = 0; i < distance; ++i)
            d[i + 1] = dl[i] + dr[i];

        return d;
    }
};
```

581. 1531.cpp

Path: LeetCode-main/solutions/1531. String Compression II/1531.cpp

```
class Solution {
public:
    int getLengthOfOptimalCompression(string s, int k) {
        vector<vector<int>> mem(s.length(), vector<int>(k + 1, kMax));
        return compression(s, 0, k, mem);
    }

private:
    static constexpr int kMax = 101;

    // Returns the length of the optimal compression of s[i..n) with at most k
    // deletion.
    int compression(const string& s, int i, int k, vector<vector<int>>& mem) {
        if (k < 0)
            return kMax;
        if (i == s.length() || s.length() - i <= k)
            return 0;
        if (mem[i][k] != kMax)
            return mem[i][k];

        int maxFreq = 0; // the maximum frequency in s[i..j]
        vector<int> count(128);

        // Make letters in s[i..j] be the same.
        // Keep the letter that has the maximum frequency in this range and remove
        // the other letters.
        for (int j = i; j < s.length(); ++j) {
            maxFreq = max(maxFreq, ++count[s[j]]);
            mem[i][k] = min( //
                mem[i][k], //
                getLength(maxFreq) +
                compression(s, j + 1, k - (j - i + 1 - maxFreq), mem));
        }

        return mem[i][k];
    }

    // Returns the length to compress `maxFreq`.
    int getLength(int maxFreq) {
        if (maxFreq == 1)
            return 1; // c
        if (maxFreq < 10)
            return 2; // [1-9]c
        if (maxFreq < 100)
            return 3; // [1-9][0-9]c
        return 4; // [1-9][0-9][0-9]c
    }
};
```

582. 1533-2.cpp

Path: LeetCode-main/solutions/1533. Find the Index of the Large Integer/1533-2.cpp

```
/**
 * // This is the ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * class ArrayReader {
 * public:
 *     // Compares the sum of arr[l..r] with the sum of arr[x..y]
 *     // return 1 if sum(arr[l..r]) > sum(arr[x..y])
 *     // return 0 if sum(arr[l..r]) == sum(arr[x..y])
 *     // return -1 if sum(arr[l..r]) < sum(arr[x..y])
 *     int compareSub(int l, int r, int x, int y);
 *
 *     // Returns the length of the array
 *     int length();
 * };
 */

class Solution {
public:
    int getIndex(ArrayReader& reader) {
        int l = 0;
        int r = reader.length() - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            const int res = (r - l + 1) % 2 == 0 ? reader.compareSub(l, m, m + 1, r)
                : reader.compareSub(l, m, m, r);

            if (res == -1)
                l = m + 1;
            else // res == 1 || res == 0
                r = m;
        }

        return l;
    }
};
```

583. 1533.cpp

Path: LeetCode-main/solutions/1533. Find the Index of the Large Integer/1533.cpp

```
/**
 * // This is the ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * class ArrayReader {
 * public:
 *     // Compares the sum of arr[l..r] with the sum of arr[x..y]
 *     // return 1 if sum(arr[l..r]) > sum(arr[x..y])
 *     // return 0 if sum(arr[l..r]) == sum(arr[x..y])
 *     // return -1 if sum(arr[l..r]) < sum(arr[x..y])
 *     int compareSub(int l, int r, int x, int y);
 *
 *     // Returns the length of the array
 *     int length();
 * };
 */

class Solution {
public:
    int getIndex(ArrayReader& reader) {
        int l = 0;
        int r = reader.length() - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if ((r - l) % 2 == 0) {
                const int res = reader.compareSub(l, m - 1, m + 1, r);
                if (res == 0)
                    return m;
                if (res == 1) {
                    r = m - 1;
                } else { // res == -1
                    l = m + 1;
                }
            } else {
                const int res = reader.compareSub(l, m, m + 1, r);
                // res is either 1 or -1.
                if (res == 1) {
                    r = m;
                } else { // res == -1
                    l = m + 1;
                }
            }
        }

        return l;
    }
};
```


585. 1535.cpp

Path: LeetCode-main/solutions/1535. Find the Winner of an Array Game/1535.cpp

```
class Solution {
public:
    int getWinner(vector<int>& arr, int k) {
        int ans = arr[0];
        int wins = 0;

        for (int i = 1; i < arr.size() && wins < k; ++i)
            if (arr[i] > ans) {
                ans = arr[i];
                wins = 1;
            } else {
                ++wins;
            }

        return ans;
    }
};
```

586. 1536.cpp

Path: LeetCode-main/solutions/1536. Minimum Swaps to Arrange a Binary Grid/1536.cpp

```
class Solution {
public:
    int minSwaps(vector<vector<int>>& grid) {
        const int n = grid.size();
        int ans = 0;
        // suffixZeros[i] := the number of suffix zeros in the i-th row
        vector<int> suffixZeros;

        for (const vector<int> row : grid) {
            const auto itLastOne = find(row.rbegin(), row.rend(), 1);
            const int suffixZeroCount = distance(row.rbegin(), itLastOne);
            suffixZeros.push_back(suffixZeroCount);
        }

        for (int i = 0; i < n; ++i) {
            const int neededZeros = n - 1 - i;
            // Get the first row with suffix zeros >= `neededZeros` in
            // suffixZeros[i..n).
            const auto it = find_if(suffixZeros.begin() + i, suffixZeros.end(),
                                   [&](int count) { return count >= neededZeros; });
            if (it == suffixZeros.end())
                return -1;
            const int j = distance(suffixZeros.begin(), it);
            // Move the rows[j] to the rows[i].
            for (int k = j; k > i; --k)
                suffixZeros[k] = suffixZeros[k - 1];
            ans += j - i;
        }

        return ans;
    }
};
```


587. 1537.cpp

Path: LeetCode-main/solutions/1537. Get the Maximum Score/1537.cpp

```
class Solution {
public:
    int maxSum(vector<int>& nums1, vector<int>& nums2) {
        constexpr int kMod = 1'000'000'007;
        // Keep the running the sum of `nums1` and `nums2` before the next
        // rendezvous. Since `nums1` and `nums2` are increasing, move forward on the
        // smaller one to ensure we don't miss any rendezvous. When meet rendezvous,
        // choose the better path.
        long ans = 0;
        // sum(nums1) in (the prevoious rendezvous, the next rendezvous)
        long sum1 = 0;
        // sum(nums2) in (the prevoious rendezvous, the next rendezvous)
        long sum2 = 0;
        int i = 0; // nums1's index
        int j = 0; // nums2's index

        while (i < nums1.size() && j < nums2.size())
            if (nums1[i] < nums2[j]) {
                sum1 += nums1[i++];
            } else if (nums1[i] > nums2[j]) {
                sum2 += nums2[j++];
            } else { // An rendezvous happens.
                ans += max(sum1, sum2) + nums1[i];
                sum1 = 0;
                sum2 = 0;
                ++i;
                ++j;
            }

        while (i < nums1.size())
            sum1 += nums1[i++];

        while (j < nums2.size())
            sum2 += nums2[j++];

        return (ans + max(sum1, sum2)) % kMod;
    }
};
```

588. 1538.cpp

Path: LeetCode-main/solutions/1538. Guess the Majority in a Hidden Array/1538.cpp

```
/**
 * // This is the ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * class ArrayReader {
 * public:
 *     // Compares 4 different elements in the array
 *     // Returns 4 if the values of the 4 elements are the same (0 or 1).
 *     // Returns 2 if three elements have a value equal to 0 and one element has
 *     //         value equal to 1 or vice versa.
 *     // Returns 0 if two element have a value equal to 0 and two elements have
 *     //         a value equal to 1.
 *     int query(int a, int b, int c, int d);
 *
 *     // Returns the length of the array
 *     int length();
 * };
 */

class Solution {
public:
    int guessMajority(ArrayReader& reader) {
        const int n = reader.length();
        const int query0123 = reader.query(0, 1, 2, 3);
        const int query1234 = reader.query(1, 2, 3, 4);
        // the number of numbers that are same as `nums[0]`
        int zeros = 1;
        // the number of numbers that are different from `nums[0]`
        int nonZeros = 0;
        // any index i s.t. nums[i] != nums[0]
        int indexNot0 = -1;

        // Find which group nums[1..3] belong to.
        for (int i = 1; i <= 3; ++i) {
            vector<int> abcd = getABCD(i);
            if (reader.query(abcd[0], abcd[1], abcd[2], abcd[3]) == query1234) {
                ++zeros;
            } else {
                ++nonZeros;
                indexNot0 = i;
            }
        }

        // Find which group nums[4..n] belong to.
        for (int i = 4; i < n; ++i)
            if (reader.query(1, 2, 3, i) == query0123) {
                ++zeros;
            } else {
                ++nonZeros;
                indexNot0 = i;
            }

        if (zeros == nonZeros)
            return -1;
        if (zeros > nonZeros)
            return 0;
        return indexNot0;
    }

private:
    // Returns [0..4] except i.
    vector<int> getABCD(int i) {
        vector<int> abcd{0};
        for (int j = 1; j <= 3; ++j)
            if (j != i)
                abcd.push_back(j);
        abcd.push_back(4);
        return abcd;
    }
};
```

589. 1539.cpp

Path: LeetCode-main/solutions/1539. Kth Missing Positive Number/1539.cpp

```
class Solution {
public:
    int findKthPositive(vector<int>& arr, int k) {
        int l = 0;
        int r = arr.size();

        // Find the first index l s.t. nMissing(l) = A[l] - l - 1 >= k.
        while (l < r) {
            const int m = (l + r) / 2;
            if (arr[m] - m - 1 >= k)
                r = m;
            else
                l = m + 1;
        }

        // The k-th missing positive
        // = A[l - 1] + k - nMissing(l - 1)
        // = A[l - 1] + k - (A[l - 1] - (l - 1) - 1)
        // = A[l - 1] + k - (A[l - 1] - l)
        // = l + k
        return l + k;
    }
};
```

590. 154.cpp

Path: LeetCode-main/solutions/154. Find Minimum in Rotated Sorted Array II/154.cpp

```
class Solution {
public:
    int findMin(vector<int>& nums) {
        int l = 0;
        int r = nums.size() - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (nums[m] == nums[r])
                --r;
            else if (nums[m] < nums[r])
                r = m;
            else
                l = m + 1;
        }

        return nums[l];
    }
};
```

591. 1540-2.cpp

Path: LeetCode-main/solutions/1540. Can Convert String in K Moves/1540-2.cpp

```
class Solution {
public:
    bool canConvertString(string s, string t, int k) {
        if (s.length() != t.length())
            return false;

        // e.g. s = "aab", t = "bbc", so shiftCount[1] = 3.
        // 1. a -> b, need 1 move.
        // 2. a -> b, need 1 + 26 moves.
        // 3. b -> c, need 1 + 26 * 2 moves.
        vector<int> shiftCount(26);

        for (int i = 0; i < s.length(); ++i) {
            const int shift = (t[i] - s[i] + 26) % 26;
            if (shift == 0)
                continue;
            if (shift + 26 * shiftCount[shift] > k)
                return false;
            ++shiftCount[shift];
        }

        return true;
    }
};
```

592. 1540.cpp

Path: LeetCode-main/solutions/1540. Can Convert String in K Moves/1540.cpp

```
class Solution {
public:
    bool canConvertString(string s, string t, int k) {
        if (s.length() != t.length())
            return false;

        // e.g. s = "aab", t = "bbc", so shiftCount[1] = 3
        // 1. a -> b, need 1 move.
        // 2. a -> b, need 1 + 26 moves.
        // 3. b -> c, need 1 + 26 * 2 moves.
        vector<int> shiftCount(26);

        for (int i = 0; i < s.length(); ++i)
            ++shiftCount[(t[i] - s[i] + 26) % 26];

        for (int shift = 1; shift < 26; ++shift)
            if (shift + 26 * (shiftCount[shift] - 1) > k)
                return false;

        return true;
    }
};
```

593. 1541.cpp

Path: LeetCode-main/solutions/1541. Minimum Insertions to Balance a Parentheses String/1541.cpp

```
class Solution {
public:
    int minInsertions(string s) {
        int neededRight = 0; // Increment by 2 for each '('.
        int missingLeft = 0; // Increment by 1 for each missing '('.
        int missingRight = 0; // Increment by 1 for each missing ')'.

        for (const char c : s)
            if (c == '(') {
                if (neededRight % 2 == 1) {
                    // e.g. "()(..."
                    ++missingRight;
                    --neededRight;
                }
                neededRight += 2;
            } else if (--neededRight < 0) { // c == ')'
                // e.g. "()))..."
                ++missingLeft;
                neededRight += 2;
            }

        return neededRight + missingLeft + missingRight;
    }
};
```

594. 1542.cpp

Path: LeetCode-main/solutions/1542. Find Longest Awesome Substring/1542.cpp

```
class Solution {
public:
    int longestAwesome(string s) {
        int ans = 0;
        int prefix = 0; // the binary prefix
        vector<int> prefixToIndex(1024, s.length());
        prefixToIndex[0] = -1;

        for (int i = 0; i < s.length(); ++i) {
            prefix ^= 1 << s[i] - '0';
            ans = max(ans, i - prefixToIndex[prefix]);
            for (int j = 0; j < 10; ++j)
                ans = max(ans, i - prefixToIndex[prefix ^ 1 << j]);
            prefixToIndex[prefix] = min(prefixToIndex[prefix], i);
        }

        return ans;
    }
};
```


595. 1544.cpp

Path: LeetCode-main/solutions/1544. Make The String Great/1544.cpp

```
class Solution {
public:
    string makeGood(string s) {
        string ans;
        for (const char c : s)
            if (!ans.empty() && isBadPair(ans.back(), c))
                ans.pop_back();
            else
                ans.push_back(c);
        return ans;
    }

    bool isBadPair(char a, char b) {
        return a != b && tolower(a) == tolower(b);
    }
};
```

596. 1545.cpp

Path: LeetCode-main/solutions/1545. Find Kth Bit in Nth Binary String/1545.cpp

```
class Solution {
public:
    char findKthBit(int n, int k) {
        if (n == 1)
            return '0';
        const int midIndex = pow(2, n - 1); // 1-indexed
        if (k == midIndex)
            return '1';
        if (k < midIndex)
            return findKthBit(n - 1, k);
        return findKthBit(n - 1, midIndex * 2 - k) == '0' ? '1' : '0';
    }
};
```

597. 1546.cpp

Path: LeetCode-main/solutions/1546. Maximum Number of Non-Overlapping Subarrays With Sum Equals Target/1546.cpp

```
class Solution {
public:
    int maxNonOverlapping(vector<int>& nums, int target) {
        // Ending the subarray ASAP always has a better result.
        int ans = 0;
        int prefix = 0;
        unordered_set<int> prefixes{0};

        // Greedily find the subarrays that equal to the target.
        for (const int num : nums) {
            // Check if there is a subarray ends in here and equals to the target.
            prefix += num;
            if (prefixes.contains(prefix - target)) {
                // Find one and discard all the prefixes that have been used.
                ++ans;
                prefix = 0;
                prefixes = {0};
            } else {
                prefixes.insert(prefix);
            }
        }

        return ans;
    }
};
```

598. 1547-2.cpp

Path: LeetCode-main/solutions/1547. Minimum Cost to Cut a Stick/1547-2.cpp

```
class Solution {
public:
    int minCost(int n, vector<int>& cuts) {
        cuts.push_back(0);
        cuts.push_back(n);
        ranges::sort(cuts);

        // dp[i][j] := minCost(cuts[i..j])
        vector<vector<int>> dp(cuts.size(), vector<int>(cuts.size()));

        for (int d = 2; d < cuts.size(); ++d)
            for (int i = 0; i + d < cuts.size(); ++i) {
                const int j = i + d;
                dp[i][j] = INT_MAX;
                for (int k = i + 1; k < j; ++k)
                    dp[i][j] = min(dp[i][j], cuts[j] - cuts[i] + dp[i][k] + dp[k][j]);
            }

        return dp[0][cuts.size() - 1];
    }
};
```

599. 1547.cpp

Path: LeetCode-main/solutions/1547. Minimum Cost to Cut a Stick/1547.cpp

```
class Solution {
public:
    int minCost(int n, vector<int>& cuts) {
        cuts.push_back(0);
        cuts.push_back(n);
        ranges::sort(cuts);
        vector<vector<int>> mem(cuts.size(), vector<int>(cuts.size(), INT_MAX));
        return minCost(cuts, 0, cuts.size() - 1, mem);
    }

private:
    // Returns minCost(cuts[i..j]).
    int minCost(const vector<int>& cuts, int i, int j, vector<vector<int>>& mem) {
        if (j - i <= 1)
            return 0;
        if (mem[i][j] != INT_MAX)
            return mem[i][j];

        for (int k = i + 1; k < j; ++k)
            mem[i][j] = min(mem[i][j], cuts[j] - cuts[i] + minCost(cuts, i, k, mem) +
                            minCost(cuts, k, j, mem));

        return mem[i][j];
    }
};
```

600. 1548.cpp

Path: LeetCode-main/solutions/1548. The Most Similar Path in a Graph/1548.cpp

```
class Solution {
public:
    vector<int> mostSimilar(int n, vector<vector<int>>& roads,
                           vector<string>& names, vector<string>& targetPath) {
        this->names = names;
        this->targetPath = targetPath;
        // cost[i][j] := the minimum cost to start from names[i] in path[j]
        this->cost.resize(names.size(), vector<int>(targetPath.size(), -1));
        // next[i][j] := the best next of names[i] in path[j]
        this->next.resize(names.size(), vector<int>(targetPath.size()));
        this->graph.resize(n);

        for (const vector<int>& road : roads) {
            graph[road[0]].push_back(road[1]);
            graph[road[1]].push_back(road[0]);
        }

        int minDist = INT_MAX;
        int start = 0;

        for (int i = 0; i < n; ++i) {
            const int dist = dfs(i, 0);
            if (dist < minDist) {
                minDist = dist;
                start = i;
            }
        }

        vector<int> ans;

        while (ans.size() < targetPath.size()) {
            ans.push_back(start);
            start = next[start][ans.size() - 1];
        }

        return ans;
    }

private:
    vector<string> names;
    vector<string> targetPath;
    vector<vector<int>> cost;
    vector<vector<int>> next;
    vector<vector<int>> graph;

    int dfs(int nameIndex, int pathIndex) {
        if (cost[nameIndex][pathIndex] != -1)
            return cost[nameIndex][pathIndex];

        const int editDist = names[nameIndex] != targetPath[pathIndex];
        if (pathIndex == targetPath.size() - 1)
            return editDist;

        int minDist = INT_MAX;

        for (const int v : graph[nameIndex]) {
            const int dist = dfs(v, pathIndex + 1);
            if (dist < minDist) {
                minDist = dist;
                next[nameIndex][pathIndex] = v;
            }
        }

        return cost[nameIndex][pathIndex] = editDist + minDist;
    }
};
```

601. 155.cpp

Path: LeetCode-main/solutions/155. Min Stack/155.cpp

```
class MinStack {
public:
    void push(int x) {
        if (stack.empty())
            stack.emplace(x, x);
        else
            stack.emplace(x, min(x, stack.top().second));
    }

    void pop() {
        stack.pop();
    }

    int top() {
        return stack.top().first;
    }

    int getMin() {
        return stack.top().second;
    }

private:
    stack<pair<int, int>> stack; // {x, min}
};
```

602. 1550.cpp

Path: LeetCode-main/solutions/1550. Three Consecutive Odds/1550.cpp

```
class Solution {
public:
    bool threeConsecutiveOdds(vector<int>& arr) {
        int count = 0;
        for (const int a : arr) {
            count = a % 2 == 0 ? 0 : count + 1;
            if (count == 3)
                return true;
        }
        return false;
    }
};
```


603. 1551.cpp

Path: LeetCode-main/solutions/1551. Minimum Operations to Make Array Equal/1551.cpp

```
class Solution {
public:
    int minOperations(int n) {
        //      median := median of arr
        //      diffs[i] := median - arr[i] where 1 <= i <= n / 2
        //      ans := sum(diffs)
        // e.g.
        // n = 5, arr = [1, 3, 5, 7, 9], diffs = [4, 2]
        //      ans = (4 + 2) * 2 / 2 = 6
        // n = 6, arr = [1, 3, 5, 7, 9, 11], diffs = [5, 3, 1]
        //      ans = (5 + 1) * 3 / 2 = 9
        const int halfSize = n / 2;
        const int median = (arr(n) + arr(1)) / 2;
        const int firstDiff = median - arr(1);
        const int lastDiff = median - arr(halfSize);
        return (firstDiff + lastDiff) * halfSize / 2;
    }

private:
    // Returns the i-th element of `arr`, where 1 <= i <= n.
    int arr(int i) {
        return (i - 1) * 2 + 1;
    }
};
```

604. 1552.cpp

Path: LeetCode-main/solutions/1552. Magnetic Force Between Two Balls/1552.cpp

```
class Solution {
public:
    int maxDistance(vector<int>& position, int m) {
        ranges::sort(position);

        int l = 1;
        int r = position.back() - position.front();

        while (l < r) {
            const int mid = r - (r - l) / 2;
            if (numBalls(position, mid) >= m) // There're too many balls.
                l = mid;
            else // There're too few balls.
                r = mid - 1;
        }

        return l;
    }

private:
    int numBalls(const vector<int>& position, int force) {
        int balls = 0;
        int prevPosition = -force;
        for (const int pos : position)
            if (pos - prevPosition >= force) {
                ++balls;
                prevPosition = pos;
            }
        return balls;
    }
};
```

605. 1553.cpp

Path: LeetCode-main/solutions/1553. Minimum Number of Days to Eat N Oranges/1553.cpp

```
class Solution {
public:
    int minDays(int n) {
        if (n <= 1)
            return n;
        if (const auto it = mem.find(n); it != mem.cend())
            return it->second;
        return mem[n] = 1 + min(minDays(n / 3) + n % 3, //
                               minDays(n / 2) + n % 2);
    }

private:
    unordered_map<int, int> mem;
};
```

606. 1554-2.cpp

Path: LeetCode-main/solutions/1554. Strings Differ by One Character/1554-2.cpp

```
class Solution {
public:
    bool differByOne(vector<string>& dict) {
        const int m = dict[0].length();
        vector<int> wordToHash;

        for (const string& word : dict)
            wordToHash.push_back(getHash(word));

        // Compute the hash without each letter.
        // e.g. hash of "abc" = 26^2 * 'a' + 26 * 'b' + 'c'
        // newHash of "a*c" = hash - 26 * 'b'
        int coefficient = 1;
        for (int j = m - 1; j >= 0; --j) {
            unordered_map<int, vector<int>> newHashToIndices;
            for (int i = 0; i < dict.size(); ++i) {
                const string& word = dict[i];
                const int newHash =
                    (wordToHash[i] -
                     static_cast<long>(coefficient) * val(word[j]) % kHash + kHash) %
                    kHash;
                if (const auto it = newHashToIndices.find(newHash);
                    it != newHashToIndices.cend())
                    for (const int index : it->second)
                        // word[0..j) == dict[index][0..j) &&
                        // word[j + 1..n) == dict[index][j + 1..n)
                        if (equal(word.begin(), word.begin() + j, dict[index].begin()) &&
                            equal(word.begin() + j + 1, word.end(),
                                dict[index].begin() + j + 1))
                            return true;
                newHashToIndices[newHash].push_back(i);
            }
            coefficient = coefficient * kBase % kHash;
        }

        return false;
    }

private:
    static constexpr long kBase = 26;
    static constexpr int kHash = 1'000'000'007;

    static constexpr int val(char c) {
        return c - 'a';
    }

    // Returns the hash of `s`. Assume the length of `s` is m.
    // e.g. getHash(s) = 26^(m - 1) * s[0] + 26^(m - 2) * s[1] + ... + s[m - 1].
    int getHash(const string& s) {
        int hash = 0;
        for (const char c : s)
            hash = (hash * kBase + val(c)) % kHash;
        return hash;
    }
};
```

607. 1554.cpp

Path: LeetCode-main/solutions/1554. Strings Differ by One Character/1554.cpp

```
class Solution {
public:
    bool differByOne(vector<string>& dict) {
        unordered_set<string> seen;

        for (string& word : dict) {
            for (char& c : word) {
                const char cache = c;
                for (char next = 'a'; next <= 'z'; ++next) {
                    if (next == cache)
                        continue;
                    c = next;
                    if (seen.contains(word))
                        return true;
                }
                c = cache;
            }
            seen.insert(word);
        }

        return false;
    }
};
```

608. 1556.cpp

Path: LeetCode-main/solutions/1556. Thousand Separator/1556.cpp

```
class Solution {
public:
    string thousandSeparator(int n) {
        const string s = to_string(n);
        string ans;

        for (int i = 0; i < s.length(); ++i) {
            if (i > 0 && (s.length() - i) % 3 == 0)
                ans += '.';
            ans += s[i];
        }

        return ans;
    }
};
```

609. 1557.cpp

Path: LeetCode-main/solutions/1557. Minimum Number of Vertices to Reach All Nodes/1557.cpp

```
class Solution {
public:
    vector<int> findSmallestSetOfVertices(int n, vector<vector<int>>& edges) {
        vector<int> ans;
        vector<int> inDegrees(n);

        for (const vector<int>& edge : edges)
            ++inDegrees[edge[1]];

        for (int i = 0; i < inDegrees.size(); ++i)
            if (inDegrees[i] == 0)
                ans.push_back(i);

        return ans;
    }
};
```

610. 1558.cpp

Path: LeetCode-main/solutions/1558. Minimum Numbers of Function Calls to Make Target Array/1558.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums) {
        const int mx = ranges::max(nums);
        return accumulate(nums.begin(), nums.end(), 0, [](int acc, unsigned num) {
            return acc + popcount(num);
        }) + (mx == 0 ? 0 : static_cast<int>(log2(mx)));
    }
};
```


611. 1559.cpp

Path: LeetCode-main/solutions/1559. Detect Cycles in 2D Grid/1559.cpp

```
class Solution {
public:
    bool containsCycle(vector<vector<char>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<bool>> seen(m, vector<bool>(n));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (seen[i][j])
                    continue;
                if (dfs(grid, i, j, -1, -1, grid[i][j], seen))
                    return true;
            }

        return false;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    bool dfs(const vector<vector<char>>& grid, int i, int j, int prevI, int prevJ,
             char c, vector<vector<bool>>& seen) {
        seen[i][j] = true;

        for (const auto& [dx, dy] : kDirs) {
            const int x = i + dx;
            const int y = j + dy;
            if (x < 0 || x == grid.size() || y < 0 || y == grid[0].size())
                continue;
            if (x == prevI && y == prevJ)
                continue;
            if (grid[x][y] != c)
                continue;
            if (seen[x][y])
                return true;
            if (dfs(grid, x, y, i, j, c, seen))
                return true;
        }

        return false;
    }
};
```

612. 156-2.cpp

Path: LeetCode-main/solutions/156. Binary Tree Upside Down/156-2.cpp

```
class Solution {
public:
    TreeNode* upsideDownBinaryTree(TreeNode* root) {
        TreeNode* prevRoot = nullptr;
        TreeNode* prevRightChild = nullptr;

        while (root != nullptr) {
            TreeNode* nextRoot = root->left; // Cache the next root.
            root->left = prevRightChild;
            prevRightChild = root->right;
            root->right = prevRoot;
            prevRoot = root; // Record the previous root.
            root = nextRoot; // Update the root.
        }

        return prevRoot;
    }
};
```

613. 156.cpp

Path: LeetCode-main/solutions/156. Binary Tree Upside Down/156.cpp

```
class Solution {
public:
    TreeNode* upsideDownBinaryTree(TreeNode* root) {
        if (root == nullptr || root->left == nullptr)
            return root;

        TreeNode* newRoot = upsideDownBinaryTree(root->left);
        root->left->left = root->right; // 2's left = 3 (root's right)
        root->left->right = root;      // 2's right = 1 (root)
        root->left = nullptr;
        root->right = nullptr;
        return newRoot;
    }
};
```

614. 1560.cpp

Path: LeetCode-main/solutions/1560. Most Visited Sector in a Circular Track/1560.cpp

```
class Solution {
public:
    vector<int> mostVisited(int n, vector<int>& rounds) {
        // 1. If start <= end, [start, end] is the most visited.
        //
        //      s ----- n
        // 1 ----- n
        // 1 ----- e
        //
        // 2. If start > end, [1, end] and [start, n] are the most visited.
        //
        //      s -- n
        // 1 ----- n
        // 1 ----- e
        const int start = rounds.front();
        const int end = rounds.back();
        vector<int> ans;
        for (int i = 1; i <= n; ++i)
            if (start <= end) {
                if (start <= i && i <= end)
                    ans.push_back(i);
            } else { // start > end
                if (i >= start || i <= end)
                    ans.push_back(i);
            }
        return ans;
    }
};
```

615. 1561.cpp

Path: LeetCode-main/solutions/1561. Maximum Number of Coins You Can Get/1561.cpp

```
class Solution {
public:
    int maxCoins(vector<int>& piles) {
        ranges::sort(piles);

        int ans = 0;

        // piles = [S S M L M L], pick all the M.
        for (int i = piles.size() / 3; i < piles.size(); i += 2)
            ans += piles[i];

        return ans;
    }
};
```

616. 1562.cpp

Path: LeetCode-main/solutions/1562. Find Latest Group of Size M/1562.cpp

```
class Solution {
public:
    int findLatestStep(vector<int>& arr, int m) {
        if (arr.size() == m)
            return arr.size();

        int ans = -1;
        int step = 0;
        // sizes[i] := the size of the group starting from i or ending in i
        // (1-indexed)
        vector<int> sizes(arr.size() + 2);

        for (const int i : arr) {
            ++step;
            // In the previous step, there exists a group with a size of m.
            if (sizes[i - 1] == m || sizes[i + 1] == m)
                ans = step - 1;
            const int head = i - sizes[i - 1];
            const int tail = i + sizes[i + 1];
            sizes[head] = tail - head + 1;
            sizes[tail] = tail - head + 1;
        }

        return ans;
    }
};
```

617. 1563.cpp

Path: LeetCode-main/solutions/1563. Stone Game V/1563.cpp

```
class Solution {
public:
    int stoneGameV(vector<int>& stoneValue) {
        const int n = stoneValue.size();
        vector<vector<int>>> mem(n, vector<int>(n, INT_MIN));
        vector<int> prefix(n + 1);
        partial_sum(stoneValue.begin(), stoneValue.end(), prefix.begin() + 1);
        return stoneGameV(stoneValue, 0, n - 1, prefix, mem);
    }

private:
    // Returns the maximum score that Alice can obtain from stoneValue[i..j].
    int stoneGameV(const vector<int>& stoneValue, int i, int j,
                   const vector<int>& prefix, vector<vector<int>>>& mem) {
        if (i == j)
            return 0;
        if (mem[i][j] > 0)
            return mem[i][j];

        // Try all the possible partitions.
        for (int p = i; p < j; ++p) {
            // sum(stoneValue[i..p])
            const int leftSum = prefix[p + 1] - prefix[i];
            const int throwRight =
                leftSum + stoneGameV(stoneValue, i, p, prefix, mem);
            // sum(stoneValue[p + 1..j])
            const int rightSum = prefix[j + 1] - prefix[p + 1];
            const int throwLeft =
                rightSum + stoneGameV(stoneValue, p + 1, j, prefix, mem);
            if (leftSum < rightSum) // Bob throws the right row.
                mem[i][j] = max(mem[i][j], throwRight);
            else if (leftSum > rightSum) // Bob throws the left row.
                mem[i][j] = max(mem[i][j], throwLeft);
            else // Alice decides which row to throw.
                mem[i][j] = max({mem[i][j], throwLeft, throwRight});
        }

        return mem[i][j];
    }
};
```

618. 1564-2.cpp

Path: LeetCode-main/solutions/1564. Put Boxes Into the Warehouse I/1564-2.cpp

```
class Solution {
public:
    int maxBoxesInWarehouse(vector<int>& boxes, vector<int>& warehouse) {
        int i = 0; // warehouse's index

        ranges::sort(boxes, greater<>());

        for (const int box : boxes)
            if (i < warehouse.size() && warehouse[i] >= box)
                ++i;

        return i;
    }
};
```


619. 1564.cpp

Path: LeetCode-main/solutions/1564. Put Boxes Into the Warehouse I/1564.cpp

```
class Solution {
public:
    int maxBoxesInWarehouse(vector<int>& boxes, vector<int>& warehouse) {
        vector<int> realWarehouse{warehouse[0]};

        for (int i = 1; i < warehouse.size(); ++i)
            realWarehouse.push_back(min(realWarehouse.back(), warehouse[i]));

        ranges::sort(boxes);
        int i = 0; // boxes' index
        for (int j = realWarehouse.size() - 1; j >= 0; j--)
            if (i < boxes.size() && boxes[i] <= realWarehouse[j])
                ++i;

        return i;
    }
};
```

620. 1566.cpp

Path: LeetCode-main/solutions/1566. Detect Pattern of Length M Repeated K or More Times/1566.cpp

```
class Solution {
public:
    bool containsPattern(vector<int>& arr, int m, int k) {
        int count = 0;
        for (int i = m; i < arr.size(); ++i) {
            count = arr[i] == arr[i - m] ? count + 1 : 0;
            if (count == m * k - m)
                return true;
        }
        return false;
    }
};
```

621. 1567.cpp

Path: LeetCode-main/solutions/1567. Maximum Length of Subarray With Positive Product/1567.cpp

```
class Solution {
public:
    int getMaxLen(vector<int>& nums) {
        int ans = 0;
        // the maximum length of subarrays ending in `num` with a negative product
        int neg = 0;
        // the maximum length of subarrays ending in `num` with a positive product
        int pos = 0;

        for (const int num : nums) {
            pos = num == 0 ? 0 : pos + 1;
            neg = num == 0 || neg == 0 ? 0 : neg + 1;
            if (num < 0)
                swap(pos, neg);
            ans = max(ans, pos);
        }

        return ans;
    }
};
```

622. 1568.cpp

Path: LeetCode-main/solutions/1568. Minimum Number of Days to Disconnect Island/1568.cpp

```
class Solution {
public:
    int minDays(vector<vector<int>>& grid) {
        if (disconnected(grid))
            return 0;

        // Try to remove 1 land.
        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] == 1) {
                    grid[i][j] = 0;
                    if (disconnected(grid))
                        return 1;
                    grid[i][j] = 1;
                }

        // Remove 2 lands.
        return 2;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    bool disconnected(const vector<vector<int>>& grid) {
        int islandsCount = 0;
        vector<vector<bool>> seen(grid.size(), vector<bool>(grid[0].size()));
        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j) {
                if (grid[i][j] == 0 || seen[i][j])
                    continue;
                if (++islandsCount > 1)
                    return true;
                dfs(grid, i, j, seen);
            }
        return islandsCount != 1;
    }

    void dfs(const vector<vector<int>>& grid, int i, int j,
            vector<vector<bool>>& seen) {
        seen[i][j] = true;
        for (const auto& [dx, dy] : kDirs) {
            const int x = i + dx;
            const int y = j + dy;
            if (x < 0 || x == grid.size() || y < 0 || y == grid[0].size())
                continue;
            if (grid[x][y] == 0 || seen[x][y])
                continue;
            dfs(grid, x, y, seen);
        }
    }
};
```

623. 1569.cpp

Path: LeetCode-main/solutions/1569. Number of Ways to Reorder Array to Get Same BST/1569.cpp

```
class Solution {
public:
    int numOfWays(vector<int>& nums) {
        comb = generate(nums.size() + 1);
        return ways(nums) - 1;
    }

private:
    static constexpr int kMod = 1'000'000'007;
    // comb[n][k] := C(n, k)
    vector<vector<int>> comb;

    int ways(const vector<int>& nums) {
        if (nums.size() <= 2)
            return 1;

        vector<int> left;
        vector<int> right;

        for (int i = 1; i < nums.size(); ++i)
            if (nums[i] < nums[0])
                left.push_back(nums[i]);
            else
                right.push_back(nums[i]);

        long ans = comb[nums.size() - 1][left.size()];
        ans = (ans * ways(left)) % kMod;
        ans = (ans * ways(right)) % kMod;
        return ans;
    }

    // Same as 118. Pascal's Triangle
    vector<vector<int>> generate(int numRows) {
        vector<vector<int>> comb;

        for (int i = 0; i < numRows; ++i)
            comb.push_back(vector<int>(i + 1, 1));

        for (int i = 2; i < numRows; ++i)
            for (int j = 1; j < comb[i].size() - 1; ++j)
                comb[i][j] = (comb[i - 1][j - 1] + comb[i - 1][j]) % kMod;

        return comb;
    }
};
```

624. 157-2.cpp

Path: LeetCode-main/solutions/157. Read N Characters Given Read4/157-2.cpp

```
/**
 * The read4 API is defined in the parent class Reader4.
 *   int read4(char *buf4);
 */

class Solution {
public:
    /**
     * @param buf Destination buffer
     * @param n    Number of characters to read
     * @return     The number of actual characters read
     */
    int read(char* buf, int n) {
        char* buf4 = new char[4];
        int i4 = 0; // buf4's index
        int n4 = 0; // buf4's size
        int i = 0;  // buf's index

        // While we're not reaching the tail (< 4 characters).
        while (i + 4 < n) {
            const int k = read4(buf4 + i); // Directly write to the buf.
            if (k == 0)                     // Reach the EOF.
                return i;
            i += k;
        }

        while (i < n) {
            if (i4 == n4) {                // All the characters in the buf4 are consumed.
                i4 = 0;                    // Reset the buf4's index.
                n4 = read4(buf4);          // Read <= 4 characters from the file to the buf4.
                if (n4 == 0)                // Reach the EOF.
                    return i;
            }
            buf[i++] = buf4[i4++];
        }

        return i;
    }
};
```

625. 157.cpp

Path: LeetCode-main/solutions/157. Read N Characters Given Read4/157.cpp

```
/**
 * The read4 API is defined in the parent class Reader4.
 *   int read4(char *buf4);
 */

class Solution {
public:
    /**
     * @param buf Destination buffer
     * @param n    Number of characters to read
     * @return    The number of actual characters read
     */
    int read(char* buf, int n) {
        char* buf4 = new char[4];
        int i4 = 0; // buf4's index
        int n4 = 0; // buf4's size
        int i = 0;  // buf's index

        while (i < n) {
            if (i4 == n4) { // All the characters in the buf4 are consumed.
                i4 = 0;    // Reset the buf4's index.
                n4 = read4(buf4); // Read <= 4 characters from the file to the buf4.
                if (n4 == 0) // Reach the EOF.
                    return i;
            }
            buf[i++] = buf4[i4++];
        }

        return i;
    }
};
```

626. 1570-2.cpp

Path: LeetCode-main/solutions/1570. Dot Product of Two Sparse Vectors/1570-2.cpp

```
class SparseVector {
public:
    SparseVector(vector<int>& nums) {
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i])
                v.push_back({i, nums[i]});
    }

    // Returns the dot product of the two sparse vectors.
    int dotProduct(SparseVector& vec) {
        int ans = 0;

        for (int i = 0, j = 0; i < v.size() && j < vec.v.size(); )
            if (v[i].first == vec.v[j].first)
                ans += v[i++].second * vec.v[j++].second;
            else if (v[i].first < vec.v[j].first)
                ++i;
            else
                ++j;

        return ans;
    }

private:
    vector<pair<int, int>> v; // [(index, num)]
};
```


627. 1570.cpp

Path: LeetCode-main/solutions/1570. Dot Product of Two Sparse Vectors/1570.cpp

```
class SparseVector {
public:
    SparseVector(vector<int>& nums) {
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i])
                indexToNum[i] = nums[i];
    }

    // Returns the dot product of the two sparse vectors.
    int dotProduct(SparseVector& vec) {
        if (indexToNum.size() < vec.indexToNum.size())
            return vec.dotProduct(*this);

        int ans = 0;

        for (const auto& [index, num] : vec.indexToNum)
            if (const auto it = indexToNum.find(index); it != indexToNum.cend())
                ans += num * it->second;

        return ans;
    }

private:
    unordered_map<int, int> indexToNum; // {index: num}
};
```

628. 1572.cpp

Path: LeetCode-main/solutions/1572. Matrix Diagonal Sum/1572.cpp

```
class Solution {
public:
    int diagonalSum(vector<vector<int>>& mat) {
        const int n = mat.size();
        int ans = 0;

        for (int i = 0; i < n; ++i) {
            ans += mat[i][i];
            ans += mat[n - 1 - i][i];
        }

        return n % 2 == 0 ? ans : ans - mat[n / 2][n / 2];
    }
};
```

629. 1573.cpp

Path: LeetCode-main/solutions/1573. Number of Ways to Split a String/1573.cpp

```
class Solution {
public:
    int numWays(string s) {
        constexpr int kMod = 1'000'000'007;
        const int ones = ranges::count(s, '1');
        if (ones % 3 != 0)
            return 0;
        if (ones == 0) {
            const long n = s.size();
            return (n - 1) * (n - 2) / 2 % kMod;
        }

        int s1End = -1;
        int s2Start = -1;
        int s2End = -1;
        int s3Start = -1;
        int onesSoFar = 0;

        for (int i = 0; i < s.length(); ++i) {
            if (s[i] == '1')
                ++onesSoFar;
            if (s1End == -1 && onesSoFar == ones / 3)
                s1End = i;
            else if (s2Start == -1 && onesSoFar == ones / 3 + 1)
                s2Start = i;
            if (s2End == -1 && onesSoFar == ones / 3 * 2)
                s2End = i;
            else if (s3Start == -1 && onesSoFar == ones / 3 * 2 + 1)
                s3Start = i;
        }

        return static_cast<long>(s2Start - s1End) * (s3Start - s2End) % kMod;
    }
};
```

630. 1574.cpp

Path: LeetCode-main/solutions/1574. Shortest Subarray to be Removed to Make Array Sorted/1574.cpp

```
class Solution {
public:
    int findLengthOfShortestSubarray(vector<int>& arr) {
        const int n = arr.size();
        int l = 0;
        int r = n - 1;

        // arr[0..l] is non-decreasing.
        while (l < n - 1 && arr[l + 1] >= arr[l])
            ++l;
        // arr[r..n - 1] is non-decreasing.
        while (r > 0 && arr[r - 1] <= arr[r])
            --r;
        // Remove arr[l + 1..n - 1] or arr[0..r - 1].
        int ans = min(n - 1 - l, r);

        // Since arr[0..l] and arr[r..n - 1] are non-decreasing, we place pointers
        // at the rightmost indices, l and n - 1, and greedily shrink them toward
        // the leftmost indices, 0 and r, respectively. By removing arr[i + 1..j],
        // we ensure that `arr` becomes non-decreasing.
        int i = l;
        int j = n - 1;
        while (i >= 0 && j >= r && j > i) {
            if (arr[i] <= arr[j])
                --j;
            else
                --i;
            ans = min(ans, j - i);
        }

        return ans;
    }
};
```

631. 1575-2.cpp

Path: LeetCode-main/solutions/1575. Count All Possible Routes/1575-2.cpp

```
class Solution {
public:
    int countRoutes(vector<int>& locations, int start, int finish, int fuel) {
        constexpr int kMod = 1'000'000'007;
        const int n = locations.size();
        // dp[i][j] := the number of ways to reach the `finish` city from the i-th
        // city with `j` fuel
        vector<vector<int>> dp(n, vector<int>(fuel + 1));

        for (int f = 0; f <= fuel; ++f)
            dp[finish][f] = 1;

        for (int f = 0; f <= fuel; ++f)
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < n; ++j) {
                    if (i == j)
                        continue;
                    const int requiredFuel = abs(locations[i] - locations[j]);
                    if (requiredFuel <= f) {
                        dp[i][f] += dp[j][f - requiredFuel];
                        dp[i][f] %= kMod;
                    }
                }

        return dp[start][fuel];
    }
};
```

632. 1575.cpp

Path: LeetCode-main/solutions/1575. Count All Possible Routes/1575.cpp

```
class Solution {
public:
    int countRoutes(vector<int>& locations, int start, int finish, int fuel) {
        vector<vector<int>> mem(locations.size(), vector<int>(fuel + 1, -1));
        return countRoutes(locations, start, finish, fuel, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of ways to reach the `finish` city from the i-th city
    // with `fuel` fuel.
    int countRoutes(const vector<int>& locations, int i, int finish, int fuel,
                    vector<vector<int>>& mem) {
        if (fuel < 0)
            return 0;
        if (mem[i][fuel] != -1)
            return mem[i][fuel];

        int res = i == finish ? 1 : 0;
        for (int j = 0; j < locations.size(); ++j) {
            if (j == i)
                continue;
            res += countRoutes(locations, j, finish,
                               fuel - abs(locations[i] - locations[j]), mem);
            res %= kMod;
        }

        return mem[i][fuel] = res;
    }
};
```

633. 1576.cpp

Path: LeetCode-main/solutions/1576. Replace All ?'s to Avoid Consecutive Repeating Characters/1576.cpp

```
class Solution {
public:
    string modifyString(string s) {
        string ans;

        for (int i = 0; i < s.length(); ++i)
            if (s[i] == '?')
                ans += nextAvailable(ans, s, i);
            else
                ans += s[i];

        return ans;
    }

private:
    char nextAvailable(const string& ans, const string& s, int i) {
        char c = 'a';
        while ((i > 0 && ans[i - 1] == c) || //
              (i + 1 < s.size() && c == s[i + 1]))
            ++c;
        return c;
    }
};
```

634. 1577.cpp

Path: LeetCode-main/solutions/1577. Number of Ways Where Square of Number Is Equal to Product of Two Numbers/1577.cpp

```
class Solution {
public:
    int numTriplets(vector<int>& nums1, vector<int>& nums2) {
        return countTriplets(nums1, nums2) + countTriplets(nums2, nums1);
    }

private:
    // Returns the number of triplet (i, j, k) if  $A[i]^2 == B[j] * B[k]$ .
    int countTriplets(const vector<int>& A, const vector<int>& B) {
        int res = 0;
        unordered_map<int, int> count;

        for (const int b : B)
            ++count[b];

        for (const int a : A) {
            const long target = static_cast<long>(a) * a;
            for (const auto& [b, freq] : count) {
                if (target % b > 0 || !count.contains(target / b))
                    continue;
                if (target / b == b)
                    res += freq * (freq - 1);
                else
                    res += freq * count[target / b];
            }
        }

        return res / 2;
    }
};
```


635. 1578.cpp

Path: LeetCode-main/solutions/1578. Minimum Time to Make Rope Colorful/1578.cpp

```
class Solution {
public:
    int minCost(string colors, vector<int>& neededTime) {
        int ans = 0;
        int maxNeededTime = neededTime[0];

        for (int i = 1; i < colors.length(); ++i)
            if (colors[i] == colors[i - 1]) {
                ans += min(maxNeededTime, neededTime[i]);
                // For each continuous group, Bob needs to remove every balloon except
                // the one with the maximum `neededTime`. So, he should hold the balloon
                // with the highest `neededTime` in his hand.
                maxNeededTime = max(maxNeededTime, neededTime[i]);
            } else {
                // If the current balloon is different from the previous one, discard
                // the balloon from the previous group and hold the new one in hand.
                maxNeededTime = neededTime[i];
            }

        return ans;
    }
};
```

636. 1579.cpp

Path: LeetCode-main/solutions/1579. Remove Max Number of Edges to Keep Graph Fully Traversable/1579.cpp

```
class UnionFind {
public:
    UnionFind(int n) : count(n), id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    bool unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return false;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        --count;
        return true;
    }

    int getCount() const {
        return count;
    }

private:
    int count;
    vector<int> id;
    vector<int> rank;

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }
};

class Solution {
public:
    int maxNumEdgesToRemove(int n, vector<vector<int>>& edges) {
        UnionFind alice(n);
        UnionFind bob(n);
        int requiredEdges = 0;

        // Greedily put type 3 edges in the front.
        ranges::sort(edges, ranges::greater{},
            [](const vector<int>& edge) { return edge[0]; });

        for (const vector<int>& edge : edges) {
            const int type = edge[0];
            const int u = edge[1] - 1;
            const int v = edge[2] - 1;
            switch (type) {
                case 3: // Can be traversed by Alice and Bob.
                    // Note that we should use | instead of || because if the first
                    // expression is true, short-circuiting will skip the second
                    // expression.
                    if (alice.unionByRank(u, v) | bob.unionByRank(u, v))
                        ++requiredEdges;
                    break;
                case 2: // Can be traversed by Bob.
                    if (bob.unionByRank(u, v))
                        ++requiredEdges;
                    break;
                case 1: // Can be traversed by Alice.
                    if (alice.unionByRank(u, v))
                        ++requiredEdges;
                    break;
            }
        }

        return alice.getCount() == 1 && bob.getCount() == 1
            ? edges.size() - requiredEdges
            : -1;
    }
};
```

637. 158-2.cpp

Path: LeetCode-main/solutions/158. Read N Characters Given Read4 II - Call multiple times/158-2.cpp

```
/**
 * The read4 API is defined in the parent class Reader4.
 *   int read4(char *buf4);
 */

class Solution {
public:
    /**
     * @param buf Destination buffer
     * @param n    Number of characters to read
     * @return     The number of actual characters read
     */
    int read(char* buf, int n) {
        int i = 0; // buf's index

        // Put the remaining characteres in the buf4 to the buf.
        while (i4 < n4 && i < n)
            buf[i++] = buf4[i4++];

        // While we're not reaching the tail (< 4 characters).
        while (i + 4 < n) {
            const int k = read4(buf + i); // Directly write to the buf.
            if (k == 0) // Reach the EOF.
                return i;
            i += k;
        }

        // Reach the tail.
        while (i < n) {
            if (i4 == n4) { // All the characters in the buf4 are consumed.
                i4 = 0; // Reset the buf4's index.
                n4 = read4(buf4); // Read <= 4 characters from the file to the buf4.
                if (n4 == 0) // Reach the EOF.
                    return i;
            }
            buf[i++] = buf4[i4++];
        }

        return i;
    }

private:
    char* buf4 = new char[4];
    int i4 = 0; // buf4's index
    int n4 = 0; // buf4's size
};
```

638. 158.cpp

Path: LeetCode-main/solutions/158. Read N Characters Given Read4 II - Call multiple times/158.cpp

```
/**
 * The read4 API is defined in the parent class Reader4.
 *   int read4(char *buf4);
 */

class Solution {
public:
    /**
     * @param buf Destination buffer
     * @param n    Number of characters to read
     * @return     The number of actual characters read
     */
    int read(char* buf, int n) {
        int i = 0; // buf's index

        while (i < n) {
            if (i4 == n4) { // All the characters in the buf4 are consumed.
                i4 = 0; // Reset the buf4's index.
                n4 = read4(buf4); // Read <= 4 characters from the file to the buf4.
                if (n4 == 0) // Reach the EOF.
                    return i;
            }
            buf[i++] = buf4[i4++];
        }

        return i;
    }

private:
    char* buf4 = new char[4];
    int i4 = 0; // buf4's index
    int n4 = 0; // buf4's size
};
```

639. 1580.cpp

Path: LeetCode-main/solutions/1580. Put Boxes Into the Warehouse II/1580.cpp

```
class Solution {
public:
    int maxBoxesInWarehouse(vector<int>& boxes, vector<int>& warehouse) {
        int l = 0;
        int r = warehouse.size() - 1;

        ranges::sort(boxes, greater<>());

        for (const int box : boxes) {
            if (l > r)
                return warehouse.size();
            if (box <= warehouse[l])
                ++l;
            else if (box <= warehouse[r])
                --r;
        }

        return l + (warehouse.size() - r - 1);
    }
};
```


641. 1583.cpp

Path: LeetCode-main/solutions/1583. Count Unhappy Friends/1583.cpp

```
class Solution {
public:
    int unhappyFriends(int n, vector<vector<int>>& preferences,
                      vector<vector<int>>& pairs) {
        int ans = 0;
        vector<int> matches(n);
        vector<unordered_map<int, int>> prefer(n);

        for (const vector<int>& pair : pairs) {
            const int x = pair[0];
            const int y = pair[1];
            matches[x] = y;
            matches[y] = x;
        }

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n - 1; ++j)
                prefer[i][preferences[i][j]] = j;

        for (int x = 0; x < n; ++x)
            for (const auto& [u, _] : prefer[x]) {
                const int y = matches[x];
                const int v = matches[u];
                if (prefer[x][u] < prefer[x][y] && prefer[u][x] < prefer[u][v]) {
                    ++ans;
                    break;
                }
            }

        return ans;
    }
};
```

642. 1584.cpp

Path: LeetCode-main/solutions/1584. Min Cost to Connect All Points/1584.cpp

```
class Solution {
public:
    int minCostConnectPoints(vector<vector<int>>& points) {
        // dist[i] := the minimum distance to connect the points[i]
        vector<int> dist(points.size(), INT_MAX);
        int ans = 0;

        for (int i = 0; i < points.size() - 1; ++i) {
            for (int j = i + 1; j < points.size(); ++j) {
                // Try to connect the points[i] with the points[j].
                dist[j] = min(dist[j], abs(points[i][0] - points[j][0]) +
                             abs(points[i][1] - points[j][1]));
                // Swap the points[j] (the point with the minimum distance) with the
                // points[i + 1].
                if (dist[j] < dist[i + 1]) {
                    swap(points[j], points[i + 1]);
                    swap(dist[j], dist[i + 1]);
                }
            }
            ans += dist[i + 1];
        }

        return ans;
    }
};
```


643. 1585.cpp

Path: LeetCode-main/solutions/1585. Check If String Is Transformable With Substring Sort Operations/1585.cpp

```
class Solution {
public:
    bool isTransformable(string s, string t) {
        if (getCount(s) != getCount(t))
            return false;

        vector<queue<int>> positions(10);

        for (int i = 0; i < s.length(); ++i)
            positions[s[i] - '0'].push(i);

        // For each digit in `t`, check if we can put this digit in `s` at the same
        // position as `t`. Ensure that all the left digits are equal to or greater
        // than it. This is because the only operation we can perform is sorting in
        // ascending order. If there is a digit to the left that is smaller than it,
        // we can never move it to the same position as in `t`. However, if all the
        // digits to its left are equal to or greater than it, we can move it one
        // position to the left until it reaches the same position as in `t`.
        for (const char c : t) {
            const int d = c - '0';
            const int front = positions[d].front();
            positions[d].pop();
            for (int smaller = 0; smaller < d; ++smaller)
                if (!positions[smaller].empty() && positions[smaller].front() < front)
                    return false;
        }
        return true;
    }

private:
    vector<int> getCount(const string& s) {
        vector<int> count(10);
        for (const char c : s)
            ++count[c - '0'];
        return count;
    }
};
```

644. 1586.cpp

Path: LeetCode-main/solutions/1586. Binary Search Tree Iterator II/1586.cpp

```
class BSTIterator {
public:
    BSTIterator(TreeNode* root) {
        pushLeftsUntilNull(root);
    }

    bool hasNext() {
        return !nexts.empty();
    }

    int next() {
        auto [root, fromNext] = nexts.top();
        nexts.pop();
        if (fromNext)
            pushLeftsUntilNull(root->right);
        prevsAndCurr.push(root);
        return root->val;
    }

    bool hasPrev() {
        return prevsAndCurr.size() > 1;
    }

    int prev() {
        nexts.push({prevsAndCurr.top(), /*fromNext=*/false});
        prevsAndCurr.pop();
        return prevsAndCurr.top()->val;
    }

private:
    stack<TreeNode*> prevsAndCurr;
    stack<pair<TreeNode*, /*fromNext=*/bool>> nexts;

    void pushLeftsUntilNull(TreeNode* root) {
        while (root != nullptr) {
            nexts.push({root, /*fromtNext=*/true});
            root = root->left;
        }
    }
};
```

645. 1588.cpp

Path: LeetCode-main/solutions/1588. Sum of All Odd Length Subarrays/1588.cpp

```
class Solution {
public:
    int sumOddLengthSubarrays(vector<int>& arr) {
        int ans = 0;
        // Maintain two sums of subarrays ending in the previous index.
        // Each time we meet a new number, we'll consider "how many times" it should
        // contribute to the newly built subarrays by calculating the number of
        // previous even/odd-length subarrays.
        int prevEvenSum = 0; // the sum of even-length subarrays
        int prevOddSum = 0;  // the sum of odd-length subarrays

        for (int i = 0; i < arr.size(); ++i) {
            // (i + 1) / 2 := the number of previous odd-length subarrays.
            const int currEvenSum = prevOddSum + ((i + 1) / 2) * arr[i];
            // i / 2 + 1 := the number of previous even-length subarrays
            // (including 0).
            const int currOddSum = prevEvenSum + (i / 2 + 1) * arr[i];
            ans += currOddSum;
            prevEvenSum = currEvenSum;
            prevOddSum = currOddSum;
        }

        return ans;
    }
};
```

646. 1589.cpp

Path: LeetCode-main/solutions/1589. Maximum Sum Obtained of Any Permutation/1589.cpp

```
class Solution {
public:
    int maxSumRangeQuery(vector<int>& nums, vector<vector<int>>& requests) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        // count[i] := the number of times nums[i] has been requested
        vector<int> count(nums.size());

        for (const vector<int>& request : requests) {
            const int start = request[0];
            const int end = request[1];
            ++count[start];
            if (end + 1 < nums.size())
                --count[end + 1];
        }

        for (int i = 1; i < nums.size(); ++i)
            count[i] += count[i - 1];

        ranges::sort(count);
        ranges::sort(nums);

        for (int i = 0; i < nums.size(); ++i) {
            ans += static_cast<long>(nums[i]) * count[i];
            ans %= kMod;
        }

        return ans;
    }
};
```

647. 159.cpp

Path: LeetCode-main/solutions/159. Longest Substring with At Most Two Distinct Characters/159.cpp

```
class Solution {
public:
    int lengthOfLongestSubstringTwoDistinct(string s) {
        int ans = 0;
        int distinct = 0;
        vector<int> count(128);

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s[r]] == 1)
                ++distinct;
            while (distinct == 3)
                if (--count[s[l++]] == 0)
                    --distinct;
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

648. 1590.cpp

Path: LeetCode-main/solutions/1590. Make Sum Divisible by P/1590.cpp

```
class Solution {
public:
    int minSubarray(vector<int>& nums, int p) {
        const long sum = accumulate(nums.begin(), nums.end(), 0L);
        const int remainder = sum % p;
        if (remainder == 0)
            return 0;

        unordered_map<int, int> prefixToIndex{{0, -1}};
        int ans = nums.size();
        int prefix = 0;

        for (int i = 0; i < nums.size(); ++i) {
            prefix += nums[i];
            prefix %= p;
            const int target = (prefix - remainder + p) % p;
            if (const auto it = prefixToIndex.find(target);
                it != prefixToIndex.cend())
                ans = min(ans, i - it->second);
            prefixToIndex[prefix] = i;
        }

        return ans == nums.size() ? -1 : ans;
    }
};
```

649. 1591.cpp

Path: LeetCode-main/solutions/1591. Strange Printer II/1591.cpp

```
enum class State { kInit, kVisiting, kVisited };

class Solution {
public:
    bool isPrintable(vector<vector<int>>& targetGrid) {
        constexpr int kMaxColor = 60;
        const int m = targetGrid.size();
        const int n = targetGrid[0].size();
        // graph[u] := {v1, v2} means v1 and v2 cover u
        vector<unordered_set<int>> graph(kMaxColor + 1);

        for (int color = 1; color <= kMaxColor; ++color) {
            // Get the rectangle of the current color.
            int minI = m;
            int minJ = n;
            int maxI = -1;
            int maxJ = -1;
            for (int i = 0; i < m; ++i)
                for (int j = 0; j < n; ++j)
                    if (targetGrid[i][j] == color) {
                        minI = min(minI, i);
                        minJ = min(minJ, j);
                        maxI = max(maxI, i);
                        maxJ = max(maxJ, j);
                    }
            // Add any color covering the current as the children.
            for (int i = minI; i <= maxI; ++i)
                for (int j = minJ; j <= maxJ; ++j)
                    if (targetGrid[i][j] != color)
                        graph[color].insert(targetGrid[i][j]);
        }

        vector<State> states(kMaxColor + 1);

        for (int color = 1; color <= kMaxColor; ++color)
            if (hasCycle(graph, color, states))
                return false;

        return true;
    }

private:
    bool hasCycle(const vector<unordered_set<int>>& graph, int u,
                  vector<State>& states) {
        if (states[u] == State::kVisiting)
            return true;
        if (states[u] == State::kVisited)
            return false;
        states[u] = State::kVisiting;
        for (const int v : graph[u])
            if (hasCycle(graph, v, states))
                return true;
        states[u] = State::kVisited;
        return false;
    }
};
```

650. 1592.cpp

Path: LeetCode-main/solutions/1592. Rearrange Spaces Between Words/1592.cpp

```
class Solution {
public:
    string reorderSpaces(string text) {
        const int spaces = ranges::count(text, ' ');
        string ans;
        vector<string> words;

        istringstream iss(text);
        string word;

        while (iss >> word)
            words.push_back(word);

        if (words.size() == 1)
            return word + string(spaces, ' ');

        const int gapSize = spaces / (words.size() - 1);
        const int remains = spaces % (words.size() - 1);

        for (int i = 0; i < words.size() - 1; ++i)
            ans += words[i] + string(gapSize, ' ');
        ans += words.back() + string(remains, ' ');

        return ans;
    }
};
```


651. 1593.cpp

Path: LeetCode-main/solutions/1593. Split a String Into the Max Number of Unique Substrings/1593.cpp

```
class Solution {
public:
    int maxUniqueSplit(string s) {
        size_t ans = 0;
        dfs(s, 0, {}, ans);
        return ans;
    }

private:
    void dfs(const string& s, int start, unordered_set<string>&& seen,
              size_t& ans) {
        if (start == s.length()) {
            ans = max(ans, seen.size());
            return;
        }

        for (int i = 1; start + i <= s.length(); ++i) {
            const string cand = s.substr(start, i);
            if (seen.contains(cand))
                continue;
            seen.insert(cand);
            dfs(s, start + i, std::move(seen), ans);
            seen.erase(cand);
        }
    }
};
```

652. 1594.cpp

Path: LeetCode-main/solutions/1594. Maximum Non Negative Product in a Matrix/1594.cpp

```
class Solution {
public:
    int maxProductPath(vector<vector<int>>& grid) {
        constexpr int kMod = 1'000'000'007;
        const int m = grid.size();
        const int n = grid[0].size();
        // dpMin[i][j] := the minimum product from (0, 0) to (i, j)
        // dpMax[i][j] := the maximum product from (0, 0) to (i, j)
        vector<vector<long>> dpMin(m, vector<long>(n));
        vector<vector<long>> dpMax(m, vector<long>(n));

        dpMin[0][0] = dpMax[0][0] = grid[0][0];

        for (int i = 1; i < m; ++i)
            dpMin[i][0] = dpMax[i][0] = dpMin[i - 1][0] * grid[i][0];

        for (int j = 1; j < n; ++j)
            dpMin[0][j] = dpMax[0][j] = dpMin[0][j - 1] * grid[0][j];

        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                if (grid[i][j] < 0) {
                    dpMin[i][j] = max(dpMax[i - 1][j], dpMax[i][j - 1]) * grid[i][j];
                    dpMax[i][j] = min(dpMin[i - 1][j], dpMin[i][j - 1]) * grid[i][j];
                } else {
                    dpMin[i][j] = min(dpMin[i - 1][j], dpMin[i][j - 1]) * grid[i][j];
                    dpMax[i][j] = max(dpMax[i - 1][j], dpMax[i][j - 1]) * grid[i][j];
                }

        const long mx = max(dpMin.back().back(), dpMax.back().back());
        return mx < 0 ? -1 : mx % kMod;
    }
};
```

653. 1595.cpp

Path: LeetCode-main/solutions/1595. Minimum Cost to Connect Two Groups of Points/1595.cpp

```
class Solution {
public:
    int connectTwoGroups(vector<vector<int>>& cost) {
        const int m = cost.size();
        const int n = cost[0].size();
        vector<vector<int>> mem(m, vector<int>(1 << n, INT_MAX));
        // minCosts[j] := the minimum cost of connecting group2's point j
        vector<int> minCosts(n);

        for (int j = 0; j < n; ++j) {
            int minCostIndex = 0;
            for (int i = 1; i < m; ++i)
                if (cost[i][j] < cost[minCostIndex][j])
                    minCostIndex = i;
            minCosts[j] = cost[minCostIndex][j];
        }

        return connectTwoGroups(cost, 0, 0, minCosts, mem);
    }

private:
    // Returns the minimum cost to connect group1's points[i..n) with group2's
    // points, where `mask` is the bitmask of the connected points in group2.
    int connectTwoGroups(const vector<vector<int>>& cost, int i, int mask,
                        const vector<int>& minCosts, vector<vector<int>>& mem) {
        if (i == cost.size()) {
            // All the points in group 1 are connected, so greedily assign the
            // minimum cost for the unconnected points of group2.
            int res = 0;
            for (int j = 0; j < cost[0].size(); ++j)
                if ((mask >> j & 1) == 0)
                    res += minCosts[j];
            return res;
        }
        if (mem[i][mask] != INT_MAX)
            return mem[i][mask];

        for (int j = 0; j < cost[0].size(); ++j)
            mem[i][mask] =
                min(mem[i][mask],
                    cost[i][j] +
                    connectTwoGroups(cost, i + 1, mask | 1 << j, minCosts, mem));

        return mem[i][mask];
    }
};
```

654. 1597.cpp

Path: LeetCode-main/solutions/1597. Build Binary Expression Tree From Infix Expression/1597.cpp

```
class Solution {
public:
    Node* expTree(string s) {
        stack<Node*> nodes;
        stack<char> ops; // [operators | parentheses]

        for (const char c : s)
            if (isdigit(c)) {
                nodes.push(new Node(c));
            } else if (c == '(') {
                ops.push(c);
            } else if (c == ')') {
                while (ops.top() != '(')
                    nodes.push(buildNode(pop(ops), pop(nodes), pop(nodes)));
                ops.pop(); // Remove '('
            } else if (c == '+' || c == '-' || c == '*' || c == '/') {
                while (!ops.empty() && compare(ops.top(), c))
                    nodes.push(buildNode(pop(ops), pop(nodes), pop(nodes)));
                ops.push(c);
            }

        while (!ops.empty())
            nodes.push(buildNode(pop(ops), pop(nodes), pop(nodes)));

        return nodes.top();
    }

private:
    Node* buildNode(char op, Node* right, Node* left) {
        return new Node(op, left, right);
    }

    // Returns true if op1 is a operator and priority(op1) >= priority(op2).
    bool compare(char op1, char op2) {
        if (op1 == '(' || op1 == ')')
            return false;
        return op1 == '*' || op1 == '/' || op2 == '+' || op2 == '-';
    }

    char pop(stack<char>& ops) {
        const char op = ops.top();
        ops.pop();
        return op;
    }

    Node* pop(stack<Node*>& nodes) {
        Node* node = nodes.top();
        nodes.pop();
        return node;
    }
};
```

655. 1598.cpp

Path: LeetCode-main/solutions/1598. Crawler Log Folder/1598.cpp

```
class Solution {
public:
    int minOperations(vector<string>& logs) {
        int ans = 0;

        for (const string& log : logs) {
            if (log == "../")
                continue;
            if (log == "./")
                ans = max(0, ans - 1);
            else
                ++ans;
        }

        return ans;
    }
};
```

656. 1599.cpp

Path: LeetCode-main/solutions/1599. Maximum Profit of Operating a Centennial Wheel/1599.cpp

```
class Solution {
public:
    int minOperationsMaxProfit(vector<int>& customers, int boardingCost,
                              int runningCost) {
        int waiting = 0;
        int profit = 0;
        int maxProfit = 0;
        int rotate = 0;
        int maxRotate = -1;
        int i = 0;

        while (waiting > 0 || i < customers.size()) {
            if (i < customers.size())
                waiting += customers[i++];
            // Onboard new customers.
            const int newOnboard = min(waiting, 4);
            waiting -= newOnboard;
            profit += newOnboard * boardingCost - runningCost;
            ++rotate;
            if (profit > maxProfit) {
                maxProfit = profit;
                maxRotate = rotate;
            }
        }

        return maxRotate;
    }
};
```

657. 16.cpp

Path: LeetCode-main/solutions/16. 3Sum Closest/16.cpp

```
class Solution {
public:
    int threeSumClosest(vector<int>& nums, int target) {
        int ans = nums[0] + nums[1] + nums[2];

        ranges::sort(nums);

        for (int i = 0; i + 2 < nums.size(); ++i) {
            if (i > 0 && nums[i] == nums[i - 1])
                continue;
            // Choose nums[i] as the first number in the triplet, then search the
            // remaining numbers in [i + 1, n - 1].
            int l = i + 1;
            int r = nums.size() - 1;
            while (l < r) {
                const int sum = nums[i] + nums[l] + nums[r];
                if (sum == target)
                    return sum;
                if (abs(sum - target) < abs(ans - target))
                    ans = sum;
                if (sum < target)
                    ++l;
                else
                    --r;
            }
        }

        return ans;
    }
};
```

658. 160.cpp

Path: LeetCode-main/solutions/160. Intersection of Two Linked Lists/160.cpp

```
class Solution {
public:
    ListNode* getIntersectionNode(ListNode* headA, ListNode* headB) {
        ListNode* a = headA;
        ListNode* b = headB;

        while (a != b) {
            a = a == nullptr ? headB : a->next;
            b = b == nullptr ? headA : b->next;
        }

        return a;
    }
};
```


659. 1600.cpp

Path: LeetCode-main/solutions/1600. Throne Inheritance/1600.cpp

```
class ThroneInheritance {
public:
    ThroneInheritance(string kingName) : kingName(kingName) {}

    void birth(string parentName, string childName) {
        family[parentName].push_back(childName);
    }

    void death(string name) {
        dead.insert(name);
    }

    vector<string> getInheritanceOrder() {
        vector<string> ans;
        dfs(kingName, ans);
        return ans;
    }

private:
    unordered_set<string> dead;
    unordered_map<string, vector<string>> family;
    string kingName;

    void dfs(const string& name, vector<string>& ans) {
        if (!dead.contains(name))
            ans.push_back(name);
        if (!family.contains(name))
            return;

        for (const string& child : family[name])
            dfs(child, ans);
    }
};
```

660. 1601.cpp

Path: LeetCode-main/solutions/1601. Maximum Number of Achievable Transfer Requests/1601.cpp

```
class Solution {
public:
    int maximumRequests(int n, vector<vector<int>>& requests) {
        int ans = 0;
        vector<int> degrees(n); // degrees[i] := degrees of the i-th building

        function<void(int, int)> dfs = [&](int i, int processedReqs) {
            if (i == requests.size()) {
                if (ranges::all_of(degrees, [](int d) { return d == 0; }))
                    ans = max(ans, processedReqs);
                return;
            }

            // Skip the requests[i].
            dfs(i + 1, processedReqs);

            // Process the requests[i].
            --degrees[requests[i][0]];
            ++degrees[requests[i][1]];
            dfs(i + 1, processedReqs + 1);
            --degrees[requests[i][1]];
            ++degrees[requests[i][0]];
        };

        dfs(0, 0);

        return ans;
    }
};
```

661. 1602.cpp

Path: LeetCode-main/solutions/1602. Find Nearest Right Node in Binary Tree/1602.cpp

```
class Solution {
public:
    TreeNode* findNearestRightNode(TreeNode* root, TreeNode* u) {
        TreeNode* ans = nullptr;
        int targetDepth = -1;
        dfs(root, u, 0, targetDepth, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, TreeNode* u, int depth, int& targetDepth,
              TreeNode*& ans) {
        if (root == nullptr)
            return;
        if (root == u) {
            targetDepth = depth;
            return;
        }
        if (depth == targetDepth && ans == nullptr) {
            ans = root;
            return;
        }
        dfs(root->left, u, depth + 1, targetDepth, ans);
        dfs(root->right, u, depth + 1, targetDepth, ans);
    }
};
```

662. 1603.cpp

Path: LeetCode-main/solutions/1603. Design Parking System/1603.cpp

```
class ParkingSystem {
public:
    ParkingSystem(int big, int medium, int small) {
        count = {big, medium, small};
    }

    bool addCar(int carType) {
        return count[carType - 1]-- > 0;
    }

private:
    vector<int> count;
};
```

663. 1604.cpp

Path: LeetCode-main/solutions/1604. Alert Using Same Key-Card Three or More Times in a One Hour Period/1604.cpp

```
class Solution {
public:
    vector<string> alertNames(vector<string>& keyName, vector<string>& keyTime) {
        vector<string> ans;
        unordered_map<string, vector<int>>> nameToMinutes;

        for (int i = 0; i < keyName.size(); ++i) {
            const int minutes = getMinutes(keyTime[i]);
            nameToMinutes[keyName[i]].push_back(minutes);
        }

        for (auto& [name, minutes] : nameToMinutes)
            if (hasAlert(minutes))
                ans.push_back(name);

        ranges::sort(ans);
        return ans;
    }

private:
    // Returns true if any worker uses the key-card three or more times in an
    // one-hour period.
    bool hasAlert(vector<int>& minutes) {
        if (minutes.size() > 70)
            return true;
        ranges::sort(minutes);
        for (int i = 2; i < minutes.size(); ++i)
            if (minutes[i - 2] + 60 >= minutes[i])
                return true;
        return false;
    }

    int getMinutes(const string& time) {
        const int h = stoi(time.substr(0, 2));
        const int m = stoi(time.substr(3));
        return 60 * h + m;
    }
};
```

664. 1605.cpp

Path: LeetCode-main/solutions/1605. Find Valid Matrix Given Row and Column Sums/1605.cpp

```
class Solution {
public:
    vector<vector<int>> restoreMatrix(vector<int>& rowSum, vector<int>& colSum) {
        const int m = rowSum.size();
        const int n = colSum.size();
        vector<vector<int>> ans(m, vector<int>(n));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                ans[i][j] = min(rowSum[i], colSum[j]);
                rowSum[i] -= ans[i][j];
                colSum[j] -= ans[i][j];
            }

        return ans;
    }
};
```

665. 1606.cpp

Path: LeetCode-main/solutions/1606. Find Servers That Handled Most Number of Requests/1606.cpp

```
class Solution {
public:
    vector<int> busiestServers(int k, vector<int>& arrival, vector<int>& load) {
        vector<int> ans;
        vector<int> times(k);
        set<int> idleServers;
        // (endTime, server)
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> minHeap;

        for (int i = 0; i < k; ++i)
            idleServers.insert(i);

        for (int i = 0; i < arrival.size(); ++i) {
            // Pop all the servers that are available now.
            while (!minHeap.empty() && minHeap.top().first <= arrival[i]) {
                idleServers.insert(minHeap.top().second);
                minHeap.pop();
            }
            // Get the next available server.
            const int server = getNextAvailableServer(idleServers, i, k);
            if (server == -1)
                continue;
            ++times[server];
            minHeap.emplace(arrival[i] + load[i], server);
            idleServers.erase(server);
        }

        const int busiest = ranges::max(times);
        for (int i = 0; i < k; ++i)
            if (times[i] == busiest)
                ans.push_back(i);
        return ans;
    }

private:
    int getNextAvailableServer(const set<int>& idleServers, int ithRequest,
                              int k) {
        if (idleServers.empty())
            return -1;
        const auto it = idleServers.lower_bound(ithRequest % k);
        return it == idleServers.cend() ? *idleServers.begin() : *it;
    }
};
```

666. 1608.cpp

Path: LeetCode-main/solutions/1608. Special Array With X Elements Greater Than or Equal X/1608.cpp

```
class Solution {
public:
    int specialArray(vector<int>& nums) {
        ranges::sort(nums);

        if (nums[0] >= nums.size())
            return nums.size();

        for (int i = 1; i < nums.size(); ++i) {
            const int count = nums.size() - i;
            if (nums[i - 1] < count && nums[i] >= count)
                return count;
        }

        return -1;
    }
};
```


667. 1609.cpp

Path: LeetCode-main/solutions/1609. Even Odd Tree/1609.cpp

```
class Solution {
public:
    bool isEvenOddTree(TreeNode* root) {
        queue<TreeNode*> q{{root}};
        bool isEven = true;

        for (; !q.empty(); isEven = !isEven) {
            int prevVal = isEven ? INT_MIN : INT_MAX;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode* node = q.front();
                q.pop();
                if (isEven && (node->val % 2 == 0 || node->val <= prevVal))
                    return false; // invalid case on even level
                if (!isEven && (node->val % 2 == 1 || node->val >= prevVal))
                    return false; // invalid case on odd level
                prevVal = node->val;
                if (node->left != nullptr)
                    q.push(node->left);
                if (node->right != nullptr)
                    q.push(node->right);
            }
        }
        return true;
    }
};
```

668. 161.cpp

Path: LeetCode-main/solutions/161. One Edit Distance/161.cpp

```
class Solution {
public:
    bool isOneEditDistance(string s, string t) {
        const int m = s.length();
        const int n = t.length();
        if (m > n) // Make sure that |s| <= |t|.
            return isOneEditDistance(t, s);

        for (int i = 0; i < m; ++i)
            if (s[i] != t[i]) {
                if (m == n)
                    return s.substr(i + 1) == t.substr(i + 1); // Replace s[i] with t[i].
                return s.substr(i) == t.substr(i + 1); // Delete t[i].
            }

        return m + 1 == n; // Delete t[-1].
    }
};
```

669. 1610.cpp

Path: LeetCode-main/solutions/1610. Maximum Number of Visible Points/1610.cpp

```
class Solution {
public:
    int visiblePoints(vector<vector<int>>& points, int angle,
                     vector<int>& location) {
        const int posX = location[0];
        const int posY = location[1];
        int maxVisible = 0;
        int same = 0;
        vector<double> pointAngles;

        for (const vector<int>& p : points) {
            const int x = p[0];
            const int y = p[1];
            if (x == posX && y == posY)
                ++same;
            else
                pointAngles.push_back(getAngle(y - posY, x - posX));
        }

        ranges::sort(pointAngles);

        const int n = pointAngles.size();
        for (int i = 0; i < n; ++i)
            pointAngles.push_back(pointAngles[i] + 360);

        for (int l = 0, r = 0; r < pointAngles.size(); ++r) {
            while (pointAngles[r] - pointAngles[l] > angle)
                ++l;
            maxVisible = max(maxVisible, r - l + 1);
        }

        return maxVisible + same;
    }

private:
    double getAngle(int dy, int dx) {
        return atan2(dy, dx) * 180 / M_PI;
    }
};
```

670. 1611.cpp

Path: LeetCode-main/solutions/1611. Minimum One Bit Operations to Make Integers Zero/1611.cpp

```
class Solution {
public:
    int minimumOneBitOperations(int n) {
        // Observation: e.g. n = 2^2
        //          100 (2^2 needs 2^3 - 1 ops)
        // op1 -> 101
        // op2 -> 111
        // op1 -> 110
        // op2 -> 010 (2^1 needs 2^2 - 1 ops)
        // op1 -> 011
        // op2 -> 001 (2^0 needs 2^1 - 1 ops)
        // op1 -> 000
        //
        // So 2^k needs 2^(k + 1) - 1 ops. Note this is reversible, i.e., 0 -> 2^k
        // also takes 2^(k + 1) - 1 ops.

        // e.g. n = 1XXX, our first goal is to change 1XXX -> 1100.
        // - If the second bit is 1, you only need to consider the cost of turning
        //   the last 2 bits to 0.
        // - If the second bit is 0, you need to add up the cost of flipping the
        //   second bit from 0 to 1.
        // XOR determines the cost minimumOneBitOperations(1XXX^1100) accordingly.
        // Then, 1100 -> 0100 needs 1 op. Finally, 0100 -> 0 needs 2^3 - 1 ops.
        if (n == 0)
            return 0;
        // x is the largest 2^k <= n.
        // x | x >> 1 -> x >> 1 needs 1 op.
        // x >> 1 -> 0 needs x = 2^k - 1 ops.
        int x = 1;
        while (x * 2 <= n)
            x <<= 1;
        return minimumOneBitOperations(n ^ (x | x >> 1)) + 1 + x - 1;
    }
};
```

671. 1612.cpp

Path: LeetCode-main/solutions/1612. Check If Two Expression Trees are Equivalent/1612.cpp

```
class Solution {
public:
    bool checkEquivalence(Node* root1, Node* root2) {
        vector<int> count(26);
        dfs(root1, count, 1);
        dfs(root2, count, -1);
        return ranges::count_if(count, [](int c) { return c != 0; }) == 0;
    }

private:
    void dfs(Node* root, vector<int>& count, int add) {
        if (root == nullptr)
            return;
        if ('a' <= root->val && root->val <= 'z')
            count[root->val - 'a'] += add;
        dfs(root->left, count, add);
        dfs(root->right, count, add);
    }
};
```

672. 1614.cpp

Path: LeetCode-main/solutions/1614. Maximum Nesting Depth of the Parentheses/1614.cpp

```
class Solution {
public:
    int maxDepth(string s) {
        int ans = 0;
        int opened = 0;

        for (const char c : s)
            if (c == '(')
                ans = max(ans, ++opened);
            else if (c == ')')
                --opened;

        return ans;
    }
};
```

673. 1615.cpp

Path: LeetCode-main/solutions/1615. Maximal Network Rank/1615.cpp

```
class Solution {
public:
    int maximalNetworkRank(int n, vector<vector<int>>& roads) {
        vector<int> degrees(n);

        for (const vector<int>& road : roads) {
            const int u = road[0];
            const int v = road[1];
            ++degrees[u];
            ++degrees[v];
        }

        // Find the first maximum and the second maximum degrees.
        int maxDegree1 = 0;
        int maxDegree2 = 0;
        for (const int degree : degrees) {
            if (degree > maxDegree1) {
                maxDegree2 = maxDegree1;
                maxDegree1 = degree;
            } else if (degree > maxDegree2) {
                maxDegree2 = degree;
            }
        }

        // There can be multiple nodes with `maxDegree1` or `maxDegree2`.
        // Find the counts of such nodes.
        int countMaxDegree1 = 0;
        int countMaxDegree2 = 0;
        for (const int degree : degrees)
            if (degree == maxDegree1)
                ++countMaxDegree1;
            else if (degree == maxDegree2)
                ++countMaxDegree2;

        if (countMaxDegree1 == 1) {
            // 1. If there is only one node with degree = `maxDegree1`, then we'll
            // need to use the node with degree = `maxDegree2`. The answer in general
            // will be (maxDegree1 + maxDegree2), but if the two nodes that we're
            // considering are connected, then we'll have to subtract 1.
            const int edgeCount =
                getEdgeCount(roads, degrees, maxDegree1, maxDegree2) +
                getEdgeCount(roads, degrees, maxDegree2, maxDegree1);
            return maxDegree1 + maxDegree2 - (countMaxDegree2 == edgeCount ? 1 : 0);
        } else {
            // 2. If there are more than one node with degree = `maxDegree1`, then we
            // can consider `maxDegree1` twice, and we don't need to use `maxDegree2`.
            // The answer in general will be 2 * maxDegree1, but if the two nodes that
            // we're considering are connected, then we'll have to subtract 1.
            const int edgeCount =
                getEdgeCount(roads, degrees, maxDegree1, maxDegree1);
            const int maxPossibleEdgeCount =
                countMaxDegree1 * (countMaxDegree1 - 1) / 2;
            return 2 * maxDegree1 - (maxPossibleEdgeCount == edgeCount ? 1 : 0);
        }
    }

private:
    // Returns the number of edges (u, v) where degrees[u] == degreeU and
    // degrees[v] == degreeV.
    int getEdgeCount(const vector<vector<int>>& roads, const vector<int>& degrees,
                     int degreeU, int degreeV) {
        int edgeCount = 0;
        for (const vector<int>& road : roads) {
            const int u = road[0];
            const int v = road[1];
            if (degrees[u] == degreeU && degrees[v] == degreeV)
                ++edgeCount;
        }
        return edgeCount;
    }
};
```

674. 1616.cpp

Path: LeetCode-main/solutions/1616. Split Two Strings to Make Palindrome/1616.cpp

```
class Solution {
public:
    bool checkPalindromeFormation(string a, string b) {
        return check(a, b) || check(b, a);
    }

private:
    bool check(const string& a, const string& b) {
        for (int i = 0, j = a.length() - 1; i < j; ++i, --j)
            if (a[i] != b[j])
                // a[0:i] + a[i..j] + b[j + 1:] or
                // a[0:i] + b[i..j] + b[j + 1:]
                return isPalindrome(a, i, j) || isPalindrome(b, i, j);
        return true;
    }

    bool isPalindrome(const string& s, int i, int j) {
        while (i < j)
            if (s[i++] != s[j--])
                return false;
        return true;
    }
};
```


675. 1617.cpp

Path: LeetCode-main/solutions/1617. Count Subtrees With Max Distance Between Cities/1617.cpp

```
class Solution {
public:
    vector<int> countSubgraphsForEachDiameter(int n, vector<vector<int>>& edges) {
        const int maxMask = 1 << n;
        const vector<vector<int>> dist = floydWarshall(n, edges);
        vector<int> ans(n - 1);

        // mask := the subset of the cities
        for (int mask = 0; mask < maxMask; ++mask) {
            const int maxDist = getMaxDist(mask, dist, n);
            if (maxDist > 0)
                ++ans[maxDist - 1];
        }

        return ans;
    }

private:
    vector<vector<int>> floydWarshall(int n, const vector<vector<int>>& edges) {
        vector<vector<int>> dist(n, vector<int>(n, 0));

        for (int i = 0; i < n; ++i)
            dist[i][i] = 0;

        for (const vector<int>& edge : edges) {
            const int u = edge[0] - 1;
            const int v = edge[1] - 1;
            dist[u][v] = 1;
            dist[v][u] = 1;
        }

        for (int k = 0; k < n; ++k)
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < n; ++j)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

        return dist;
    }

    int getMaxDist(int mask, const vector<vector<int>>& dist, int n) {
        int maxDist = 0;
        int edgeCount = 0;
        int cityCount = 0;
        for (int u = 0; u < n; ++u) {
            if ((mask >> u & 1) == 0) // u is not in the subset.
                continue;
            ++cityCount;
            for (int v = u + 1; v < n; ++v) {
                if ((mask >> v & 1) == 0) // v is not in the subset.
                    continue;
                if (dist[u][v] == 1) // u and v are connected.
                    ++edgeCount;
                maxDist = max(maxDist, dist[u][v]);
            }
        }
        return edgeCount == cityCount - 1 ? maxDist : 0;
    }
};
```

676. 1618.cpp

Path: LeetCode-main/solutions/1618. Maximum Font to Fit a Sentence in a Screen/1618.cpp

```
/**
 * // This is the FontInfo's API interface.
 * // You should not implement it, or speculate about its implementation
 * class FontInfo {
 * public:
 *     // Return the width of char ch when fontSize is used.
 *     int getWidth(int fontSize, char ch);
 *
 *     // Return Height of any char when fontSize is used.
 *     int getHeight(int fontSize)
 * };
 */
class Solution {
public:
    int maxFont(string text, int w, int h, vector<int>& fonts,
                FontInfo fontInfo) {
        vector<int> count(26);

        for (const char c : text)
            ++count[c - 'a'];

        int l = 0;
        int r = fonts.size() - 1;

        while (l < r) {
            const int m = (l + r + 1) / 2;
            if (fontInfo.getHeight(fonts[m]) <= h &&
                getWidthSum(count, fonts[m], fontInfo) <= w)
                l = m;
            else
                r = m - 1;
        }

        return getWidthSum(count, fonts[l], fontInfo) <= w ? fonts[l] : -1;
    }

private:
    int getWidthSum(const vector<int>& count, int font, FontInfo& fontInfo) {
        int width = 0;
        for (int i = 0; i < 26; ++i)
            width += count[i] * fontInfo.getWidth(font, (char)('a' + i));
        return width;
    }
};
```

677. 1619.cpp

Path: LeetCode-main/solutions/1619. Mean of Array After Removing Some Elements/1619.cpp

```
class Solution {
public:
    double trimMean(vector<int>& arr) {
        int offset = arr.size() / 20;
        nth_element(arr.begin(), arr.begin() + offset, arr.end());
        nth_element(arr.begin() + offset, arr.end() - offset, arr.end());
        double sum = accumulate(arr.begin() + offset, arr.end() - offset, 0.0);
        return sum / (arr.size() - offset * 2);
    }
};
```

678. 162.cpp

Path: LeetCode-main/solutions/162. Find Peak Element/162.cpp

```
class Solution {
public:
    int findPeakElement(vector<int>& nums) {
        int l = 0;
        int r = nums.size() - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (nums[m] >= nums[m + 1])
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
};
```

679. 1620.cpp

Path: LeetCode-main/solutions/1620. Coordinate With Maximum Network Quality/1620.cpp

```
class Solution {
public:
    vector<int> bestCoordinate(vector<vector<int>>& towers, int radius) {
        constexpr int kMax = 50;
        const int n = towers.size();
        vector<int> ans(2);
        int maxQuality = 0;

        for (int i = 0; i <= kMax; ++i)
            for (int j = 0; j <= kMax; ++j) {
                int qualitySum = 0;
                for (const vector<int>& tower : towers) {
                    const double d = dist(tower, i, j);
                    if (d <= radius) {
                        const int q = tower[2];
                        qualitySum += static_cast<int>(q / (1 + d));
                    }
                }
                if (qualitySum > maxQuality) {
                    maxQuality = qualitySum;
                    ans = {i, j};
                }
            }

        return ans;
    }

private:
    // Returns the distance between the tower and the coordinate.
    double dist(const vector<int>& tower, int i, int j) {
        return sqrt(pow(tower[0] - i, 2) + pow(tower[1] - j, 2));
    }
};
```

680. 1621.cpp

Path: LeetCode-main/solutions/1621. Number of Sets of K Non-Overlapping Line Segments/1621.cpp

```
class Solution {
public:
    int numberOfSets(int n, int k) {
        vector<vector<vector<int>>> mem(
            n, vector<vector<int>>(k + 1, vector<int>(2, -1)));
        return numberOfSets(0, k, /*drawing=*/false, n, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    int numberOfSets(int i, int k, bool drawing, int n,
        vector<vector<vector<int>>>& mem) {
        if (k == 0) // Find a way to draw k segments.
            return 1;
        if (i == n) // Reach the end.
            return 0;
        if (mem[i][k][drawing] != -1)
            return mem[i][k][drawing];
        if (drawing)
            // 1. Keep drawing at i and move to i + 1.
            // 2. Stop at i so decrease k. We can start from i for the next segment.
            return mem[i][k][drawing] = (numberOfSets(i + 1, k, true, n, mem) +
                numberOfSets(i, k - 1, false, n, mem)) %
                kMod;
        // 1. Skip i and move to i + 1.
        // 2. Start at i and move to i + 1.
        return mem[i][k][drawing] = (numberOfSets(i + 1, k, false, n, mem) +
            numberOfSets(i + 1, k, true, n, mem)) %
            kMod;
    }
};
```

681. 1622.cpp

Path: LeetCode-main/solutions/1622. Fancy Sequence/1622.cpp

```
class Fancy {
public:
    // To undo  $a * val + b$  and get the original value, we append  $(val - b) / a$ .
    // By Fermat's little theorem:
    //  $a^{(p - 1)} \equiv 1 \pmod{p}$ 
    //  $a^{(p - 2)} \equiv a^{(-1)} \pmod{p}$ 
    // So,  $(val - b) / a \equiv (val - b) * a^{(p - 2)} \pmod{p}$ 
    void append(int val) {
        const long x = (val - b + kMod) % kMod;
        vals.push_back(x * modPow(a, kMod - 2) % kMod);
    }

    // If the value is  $a * val + b$ , then the value after adding by `inc` will be
    //  $a * val + b + inc$ . So, we adjust b to  $b + inc$ .
    void addAll(int inc) {
        b = (b + inc) % kMod;
    }

    // If the value is  $a * val + b$ , then the value after multiplying by `m` will
    // be  $a * m * val + b * m$ . So, we adjust a to  $a * m$  and b to  $b * m$ .
    void multAll(int m) {
        a = (a * m) % kMod;
        b = (b * m) % kMod;
    }

    int getIndex(int idx) {
        return idx >= vals.size() ? -1 : (a * vals[idx] + b) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;
    // For each `val` in `vals`, it actually represents  $a * val + b$ .
    vector<unsigned long> vals;
    unsigned long a = 1;
    unsigned long b = 0;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

682. 1624.cpp

Path: LeetCode-main/solutions/1624. Largest Substring Between Two Equal Characters/1624.cpp

```
class Solution {
public:
    int maxLengthBetweenEqualCharacters(string s) {
        int ans = -1;
        vector<int> lastSeen(26, -1);

        for (int i = 0; i < s.length(); ++i) {
            const int c = s[i] - 'a';
            if (lastSeen[c] == -1)
                lastSeen[c] = i;
            else
                ans = max(ans, i - lastSeen[c] - 1);
        }

        return ans;
    }
};
```


683. 1625.cpp

Path: LeetCode-main/solutions/1625. Lexicographically Smallest String After Applying Operations/1625.cpp

```
class Solution {
public:
    string findLexSmallestString(string s, int a, int b) {
        string ans = s;

        dfs(s, a, b, {}, ans);

        return ans;
    }

private:
    void dfs(string s, int a, int b, unordered_set<string>&& seen, string& ans) {
        if (seen.contains(s))
            return;

        seen.insert(s);
        ans = min(ans, s);

        dfs(add(s, a), a, b, std::move(seen), ans);
        dfs(rotate(s, b), a, b, std::move(seen), ans);
    }

    string add(string& s, int a) {
        for (int i = 1; i < s.length(); i += 2)
            s[i] = '0' + (s[i] - '0' + a) % 10;
        return s;
    }

    string rotate(const string& s, int b) {
        const int n = s.length();
        return s.substr(n - b, b) + s.substr(0, n - b);
    }
};
```

684. 1626.cpp

Path: LeetCode-main/solutions/1626. Best Team With No Conflicts/1626.cpp

```
struct Player {
    int age;
    int score;
    Player(int age, int score) : age(age), score(score) {}
};

class Solution {
public:
    int bestTeamScore(vector<int>& scores, vector<int>& ages) {
        const int n = scores.size();
        vector<Player> players;
        // dp[i] := the maximum score of choosing the players[0..i] with the
        // players[i] being selected
        vector<int> dp(n);

        for (int i = 0; i < n; ++i)
            players.emplace_back(ages[i], scores[i]);

        ranges::sort(players, ranges::greater{}, [](const Player& player) {
            return pair<int, int>{player.age, player.score};
        });

        for (int i = 0; i < n; ++i) {
            // For each player, choose it first.
            dp[i] = players[i].score;
            // players[j].age >= players[i].age since we sort in descending order.
            // So, we only have to check that players[j].score >= players[i].score.
            for (int j = 0; j < i; ++j)
                if (players[j].score >= players[i].score)
                    dp[i] = max(dp[i], dp[j] + players[i].score);
        }

        return ranges::max(dp);
    }
};
```

685. 1627.cpp

Path: LeetCode-main/solutions/1627. Graph Connectivity With Threshold/1627.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    bool unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return false;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        return true;
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    vector<bool> areConnected(int n, int threshold,
                             vector<vector<int>>& queries) {
        vector<bool> ans;
        UnionFind uf(n + 1);

        for (int z = threshold + 1; z <= n; ++z)
            for (int x = z * 2; x <= n; x += z)
                uf.unionByRank(z, x);

        for (const vector<int>& query : queries) {
            const int a = query[0];
            const int b = query[1];
            ans.push_back(uf.find(a) == uf.find(b));
        }

        return ans;
    }
};
```

686. 1628.cpp

Path: LeetCode-main/solutions/1628. Design an Expression Tree With Evaluate Function/1628.cpp

```
/**
 * This is the interface for the expression tree Node.
 * You should not remove it, and you can define some classes to implement it.
 */

class Node {
public:
    virtual ~Node() {};
    virtual int evaluate() const = 0;

protected:
    // define your fields here
};

class ExpNode : public Node {
public:
    ExpNode(const string& val, ExpNode* left, ExpNode* right)
        : val(val), left(left), right(right) {}

    int evaluate() const override {
        return left == nullptr && right == nullptr
            ? stoi(val)
            : op.at(val)(left->evaluate(), right->evaluate());
    }

private:
    static const inline unordered_map<string, function<long(long, long)>> op{
        {"+", std::plus<long>()},
        {"-", std::minus<long>()},
        {"*", std::multiplies<long>()},
        {"/", std::divides<long>()}};
    const string val;
    const ExpNode* const left;
    const ExpNode* const right;
};

/**
 * This is the TreeBuilder class.
 * You can treat it as the driver code that takes the postfix input
 * and returns the expression tree representing it as a Node.
 */

class TreeBuilder {
public:
    Node* buildTree(vector<string>& postfix) {
        stack<ExpNode*> stack;

        for (const string& val : postfix)
            if (val == "+" || val == "-" || val == "*" || val == "/") {
                ExpNode* right = stack.top();
                stack.pop();
                ExpNode* left = stack.top();
                stack.pop();
                stack.push(new ExpNode(val, left, right));
            } else {
                stack.push(new ExpNode(val, nullptr, nullptr));
            }

        return stack.top();
    }
};
```

687. 1629.cpp

Path: LeetCode-main/solutions/1629. Slowest Key/1629.cpp

```
class Solution {
public:
    char slowestKey(vector<int>& releaseTimes, string keysPressed) {
        char ans = keysPressed[0];
        int maxDuration = releaseTimes[0];

        for (int i = 1; i < keysPressed.length(); ++i) {
            const int duration = releaseTimes[i] - releaseTimes[i - 1];
            if (duration > maxDuration ||
                (duration == maxDuration && keysPressed[i] > ans)) {
                ans = keysPressed[i];
                maxDuration = duration;
            }
        }

        return ans;
    }
};
```

688. 163.cpp

Path: LeetCode-main/solutions/163. Missing Ranges/163.cpp

```
class Solution {
public:
    vector<vector<int>> findMissingRanges(vector<int>& nums, int lower,
                                         int upper) {
        if (nums.empty())
            return {getRange(lower, upper)};

        vector<vector<int>> ans;

        if (nums.front() > lower)
            ans.push_back(getRange(lower, nums.front() - 1));

        for (int i = 1; i < nums.size(); ++i)
            if (nums[i] > nums[i - 1] + 1)
                ans.push_back(getRange(nums[i - 1] + 1, nums[i] - 1));

        if (nums.back() < upper)
            ans.push_back(getRange(nums.back() + 1, upper));

        return ans;
    }

private:
    vector<int> getRange(int lo, int hi) {
        if (lo == hi)
            return vector<int>{lo, lo};
        return vector<int>{lo, hi};
    }
};
```

689. 1630.cpp

Path: LeetCode-main/solutions/1630. Arithmetic Subarrays/1630.cpp

```
class Solution {
public:
    vector<bool> checkArithmeticSubarrays(vector<int>& nums, vector<int>& l,
                                         vector<int>& r) {
        vector<bool> ans;

        for (int i = 0; i < l.size(); ++i)
            ans.push_back(isArithmetic(nums, l[i], r[i]));

        return ans;
    }

private:
    bool isArithmetic(vector<int>& nums, int l, int r) {
        if (r - l < 2)
            return true;

        unordered_set<int> numsSet;
        int mn = INT_MAX;
        int mx = INT_MIN;

        for (int i = l; i <= r; ++i) {
            mn = min(mn, nums[i]);
            mx = max(mx, nums[i]);
            numsSet.insert(nums[i]);
        }

        if ((mx - mn) % (r - l) != 0)
            return false;

        const int interval = (mx - mn) / (r - l);

        for (int k = 1; k <= r - l; ++k)
            if (!numsSet.contains(mn + k * interval))
                return false;

        return true;
    }
};
```

690. 1631.cpp

Path: LeetCode-main/solutions/1631. Path With Minimum Effort/1631.cpp

```
struct T {
    int i;
    int j;
    int d; // the maximum difference of (i, j) and its neighbors
};

class Solution {
public:
    int minimumEffortPath(vector<vector<int>>& heights) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = heights.size();
        const int n = heights[0].size();
        auto compare = [](const T& a, const T& b) { return a.d > b.d; };
        priority_queue<T, vector<T>, decltype(compare)> minHeap(compare);
        // diff[i][j] := the maximum absolute difference to reach (i, j)
        vector<vector<int>> diff(m, vector<int>(n, INT_MAX));
        vector<vector<bool>> seen(m, vector<bool>(n));

        minHeap.emplace(0, 0, 0);
        diff[0][0] = 0;

        while (!minHeap.empty()) {
            const auto [i, j, d] = minHeap.top();
            minHeap.pop();
            if (i == m - 1 && j == n - 1)
                return d;
            seen[i][j] = true;
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                const int newDiff = abs(heights[i][j] - heights[x][y]);
                const int maxDiff = max(diff[i][j], newDiff);
                if (diff[x][y] > maxDiff) {
                    diff[x][y] = maxDiff;
                    minHeap.emplace(x, y, maxDiff);
                }
            }
        }

        throw;
    }
};
```


691. 1632.cpp

Path: LeetCode-main/solutions/1632. Rank Transform of a Matrix/1632.cpp

```
class UnionFind {
public:
    void union_(int u, int v) {
        if (!id.contains(u))
            id[u] = u;
        if (!id.contains(v))
            id[v] = v;
        const int i = find(u);
        const int j = find(v);
        if (i != j)
            id[i] = j;
    }

    unordered_map<int, vector<int>> getGroupIdToValues() {
        unordered_map<int, vector<int>> groupIdToValues;
        for (const auto& [u, _] : id)
            groupIdToValues[find(u)].push_back(u);
        return groupIdToValues;
    }

private:
    unordered_map<int, int> id;

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }
};

class Solution {
public:
    vector<vector<int>> matrixRankTransform(vector<vector<int>>& matrix) {
        const int m = matrix.size();
        const int n = matrix[0].size();
        vector<vector<int>> ans(m, vector<int>(n));
        // {val: [(i, j)]}
        map<int, vector<pair<int, int>>> valToGrids;
        // rank[i] := the maximum rank of the row or column so far
        vector<int> maxRankSoFar(m + n);

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                valToGrids[matrix[i][j]].emplace_back(i, j);

        for (const auto& [val, grids] : valToGrids) {
            UnionFind uf;
            for (const auto& [i, j] : grids)
                // Union i-th row with j-th col.
                uf.union_(i, j + m);
            for (const auto& [groupId, values] : uf.getGroupIdToValues()) {
                // Get the maximum rank of all the included rows and columns.
                int maxRank = 0;
                for (const int i : values)
                    maxRank = max(maxRank, maxRankSoFar[i]);
                // Update all the rows and columns to maxRank + 1.
                for (const int i : values)
                    maxRankSoFar[i] = maxRank + 1;
            }
            for (const auto& [i, j] : grids)
                ans[i][j] = maxRankSoFar[i];
        }

        return ans;
    }
};
```

692. 1634.cpp

Path: LeetCode-main/solutions/1634. Add Two Polynomials Represented as Linked Lists/1634.cpp

```
/**
 * Definition for polynomial singly-linked list.
 * struct PolyNode {
 *   int coefficient, power;
 *   PolyNode *next;
 *   PolyNode(): coefficient(0), power(0), next(nullptr) {};
 *   PolyNode(int x, int y): coefficient(x), power(y), next(nullptr) {};
 *   PolyNode(int x, int y, PolyNode* next): coefficient(x), power(y),
 *                                           next(next) {};
 * };
 */

class Solution {
public:
    PolyNode* addPoly(PolyNode* poly1, PolyNode* poly2) {
        PolyNode* dummy = new PolyNode();
        PolyNode* curr = dummy;
        PolyNode* p = poly1; // poly1's pointer
        PolyNode* q = poly2; // poly2's pointer

        while (p != nullptr && q != nullptr) {
            if (p->power > q->power) {
                curr->next = new PolyNode(p->coefficient, p->power);
                curr = curr->next;
                p = p->next;
            } else if (p->power < q->power) {
                curr->next = new PolyNode(q->coefficient, q->power);
                curr = curr->next;
                q = q->next;
            } else { // p->power == q->power
                const int sumCoefficient = p->coefficient + q->coefficient;
                if (sumCoefficient != 0) {
                    curr->next = new PolyNode(sumCoefficient, p->power);
                    curr = curr->next;
                }
                p = p->next;
                q = q->next;
            }
        }

        while (p != nullptr) {
            curr->next = new PolyNode(p->coefficient, p->power);
            curr = curr->next;
            p = p->next;
        }

        while (q != nullptr) {
            curr->next = new PolyNode(q->coefficient, q->power);
            curr = curr->next;
            q = q->next;
        }

        return dummy->next;
    }
};
```

693. 1636.cpp

Path: LeetCode-main/solutions/1636. Sort Array by Increasing Frequency/1636.cpp

```
struct T {
    int num;
    int freq;
};

class Solution {
public:
    vector<int> frequencySort(vector<int>& nums) {
        vector<int> ans;
        auto compare = [](const T& a, const T& b) {
            return a.freq == b.freq ? a.num < b.num : a.freq > b.freq;
        };
        priority_queue<T, vector<T>, decltype(compare)> heap(compare);
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        for (const auto& [num, freq] : count)
            heap.emplace(num, freq);

        while (!heap.empty()) {
            const auto [num, freq] = heap.top();
            heap.pop();
            for (int i = 0; i < freq; ++i)
                ans.push_back(num);
        }

        return ans;
    }
};
```

694. 1637.cpp

Path: LeetCode-main/solutions/1637. Widest Vertical Area Between Two Points Containing No Points/1637.cpp

```
class Solution {
public:
    int maxWidthOfVerticalArea(vector<vector<int>>& points) {
        int ans = 0;
        vector<int> xs;

        for (const vector<int>& point : points) {
            const int x = point[0];
            xs.push_back(x);
        }

        ranges::sort(xs);

        for (int i = 1; i < xs.size(); ++i)
            ans = max(ans, xs[i] - xs[i - 1]);

        return ans;
    }
};
```

695. 1638.cpp

Path: LeetCode-main/solutions/1638. Count Substrings That Differ by One Character/1638.cpp

```
class Solution {
public:
    int countSubstrings(string s, string t) {
        int ans = 0;

        for (int i = 0; i < s.length(); ++i)
            ans += count(s, t, i, 0);

        for (int j = 1; j < t.length(); ++j)
            ans += count(s, t, 0, j);

        return ans;
    }

private:
    // Returns the number of substrings of s[i..n) and t[j..n) that differ by one
    // letter.
    int count(const string& s, const string& t, int i, int j) {
        int res = 0;
        // the number of substrings starting at s[i] and t[j] ending in the current
        // index with zero different letter
        int dp0 = 0;
        // the number of substrings starting at s[i] and t[j] ending in the current
        // index with one different letter
        int dp1 = 0;

        for (; i < s.length() && j < t.length(); ++i, ++j) {
            if (s[i] == t[j]) {
                ++dp0;
            } else {
                dp1 = dp0 + 1;
                dp0 = 0;
            }
            res += dp1;
        }

        return res;
    }
};
```

696. 1639-2.cpp

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639-2.cpp

```
class Solution {
public:
    int numWays(vector<string>& words, string target) {
        constexpr int kMod = 1'000'000'007;
        const int wordLength = words[0].length();
        // dp[i][j] := the number of ways to form the first i characters of the
        // `target` using the j first characters in each word
        vector<vector<int>> dp(target.length() + 1, vector<int>(wordLength + 1));
        // counts[j] := the count map of words[i][j], where 0 <= i < |words|
        vector<vector<int>> counts(wordLength, vector<int>(26));

        for (int i = 0; i < wordLength; ++i)
            for (const string& word : words)
                ++counts[i][word[i] - 'a'];

        dp[0][0] = 1;

        for (int i = 0; i <= target.length(); ++i)
            for (int j = 0; j < wordLength; ++j) {
                if (i < target.length())
                    // Pick the character target[i] from word[j].
                    dp[i + 1][j + 1] =
                        static_cast<long>(dp[i][j]) * counts[j][target[i] - 'a'] % kMod;
                // Skip the word[j].
                dp[i][j + 1] = (dp[i][j + 1] + dp[i][j]) % kMod;
            }

        return dp[target.length()][wordLength];
    };
};
```

697. 1639-3.cpp

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639-3.cpp

```
class Solution {
public:
    int numWays(vector<string>& words, string target) {
        constexpr int kMod = 1'000'000'007;
        const int wordLength = words[0].length();
        // dp[i] := the number of ways to form the first i characters of the
        // `target`
        vector<long> dp(target.size() + 1);
        dp[0] = 1;

        for (int j = 0; j < wordLength; ++j) {
            vector<int> count(26);
            for (const string& word : words)
                ++count[word[j] - 'a'];
            for (int i = target.size(); i > 0; --i) {
                dp[i] += dp[i - 1] * count[target[i - 1] - 'a'];
                dp[i] %= kMod;
            }
        }

        return dp[target.length()];
    };
};
```

698. 1639.cpp

Path: LeetCode-main/solutions/1639. Number of Ways to Form a Target String Given a Dictionary/1639.cpp

```
class Solution {
public:
    int numWays(vector<string>& words, string target) {
        const int wordLength = words[0].length();
        vector<vector<int>> mem(target.length(), vector<int>(wordLength, -1));
        // counts[j] := the count map of words[i][j], where 0 <= i < |words|
        vector<vector<int>> counts(wordLength, vector<int>(26));

        for (int i = 0; i < wordLength; ++i)
            for (const string& word : words)
                ++counts[i][word[i] - 'a'];

        return numWays(target, 0, 0, counts, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of ways to form target[i..n) using word[j..n).
    int numWays(const string& target, int i, int j,
                const vector<vector<int>>& counts, vector<vector<int>>& mem) {
        if (i == target.length())
            return 1;
        if (j == counts.size())
            return 0;
        if (mem[i][j] != -1)
            return mem[i][j];
        return mem[i][j] =
            (static_cast<long>(numWays(target, i + 1, j + 1, counts, mem)) *
             counts[j][target[i] - 'a'] +
             numWays(target, i, j + 1, counts, mem)) %
            kMod;
    };
};
```


699. 164.cpp

Path: LeetCode-main/solutions/164. Maximum Gap/164.cpp

```
struct Bucket {
    int mn;
    int mx;
};

class Solution {
public:
    int maximumGap(vector<int>& nums) {
        if (nums.size() < 2)
            return 0;

        const int mn = ranges::min(nums);
        const int mx = ranges::max(nums);
        if (mn == mx)
            return 0;

        const int gap = ceil((mx - mn) / (double)(nums.size() - 1));
        const int bucketSize = (mx - mn) / gap + 1;
        vector<Bucket> buckets(bucketSize, {INT_MAX, INT_MIN});

        for (const int num : nums) {
            const int i = (num - mn) / gap;
            buckets[i].mn = min(buckets[i].mn, num);
            buckets[i].mx = max(buckets[i].mx, num);
        }

        int ans = 0;
        int prevMax = mn;

        for (const Bucket& bucket : buckets) {
            if (bucket.mn == INT_MAX)
                continue; // empty bucket
            ans = max(ans, bucket.mn - prevMax);
            prevMax = bucket.mx;
        }

        return ans;
    }
};
```

700. 1640.cpp

Path: LeetCode-main/solutions/1640. Check Array Formation Through Concatenation/1640.cpp

```
class Solution {
public:
    bool canFormArray(vector<int>& arr, vector<vector<int>>& pieces) {
        vector<int> concatenated;
        unordered_map<int, vector<int>> startToPiece;

        for (const vector<int>& piece : pieces)
            startToPiece[piece[0]] = piece;

        for (const int a : arr)
            if (startToPiece.contains(a))
                for (const int num : startToPiece[a])
                    concatenated.push_back(num);

        return concatenated == arr;
    }
};
```

701. 1641-2.cpp

Path: LeetCode-main/solutions/1641. Count Sorted Vowel Strings/1641-2.cpp

```
class Solution {
public:
    int countVowelStrings(int n) {
        return (n + 4) * (n + 3) * (n + 2) * (n + 1) / 24;
    }
};
```

702. 1641.cpp

Path: LeetCode-main/solutions/1641. Count Sorted Vowel Strings/1641.cpp

```
class Solution {
public:
    int countVowelStrings(int n) {
        // dp[0] := the number of lexicographically sorted string that end in 'a'
        // dp[1] := the number of lexicographically sorted string that end in 'e'
        // dp[2] := the number of lexicographically sorted string that end in 'i'
        // dp[3] := the number of lexicographically sorted string that end in 'o'
        // dp[4] := the number of lexicographically sorted string that end in 'u'
        vector<int> dp(5, 1);

        for (int i = 2; i <= n; ++i) {
            vector<int> newDp(5);
            for (int j = 0; j < 5; ++j)
                for (int k = 0; k <= j; ++k)
                    newDp[j] += dp[k];
            dp = std::move(newDp);
        }

        return accumulate(dp.begin(), dp.end(), 0);
    }
};
```

703. 1642.cpp

Path: LeetCode-main/solutions/1642. Furthest Building You Can Reach/1642.cpp

```
class Solution {
public:
    int furthestBuilding(vector<int>& heights, int bricks, int ladders) {
        priority_queue<int, vector<int>, greater<int>> minHeap;

        for (int i = 1; i < heights.size(); ++i) {
            const int diff = heights[i] - heights[i - 1];
            if (diff <= 0)
                continue;
            minHeap.push(diff);
            // If we run out of ladders, greedily use as less bricks as possible.
            if (minHeap.size() > ladders)
                bricks -= minHeap.top(), minHeap.pop();
            if (bricks < 0)
                return i - 1;
        }
        return heights.size() - 1;
    }
};
```

704. 1643.cpp

Path: LeetCode-main/solutions/1643. Kth Smallest Instructions/1643.cpp

```
class Solution {
public:
    string kthSmallestPath(vector<int>& destination, int k) {
        string ans;
        int v = destination[0];
        int h = destination[1];
        const int totalSteps = v + h;
        const vector<vector<int>>> comb = getComb(totalSteps - 1, v);

        for (int i = 0; i < totalSteps; ++i) {
            // If 'H' is picked, we can reach ranks [1, availableRank].
            const int availableRank = comb[h + v - 1][v];
            if (availableRank >= k) { // Should pick 'H'.
                ans += 'H';
                --h;
            } else { // Should pick 'V'.
                k -= availableRank;
                ans += 'V';
                --v;
            }
        }

        return ans;
    }

private:
    // C(n, k) = C(n - 1, k) + C(n - 1, k - 1)
    vector<vector<int>>> getComb(int n, int k) {
        vector<vector<int>>> comb(n + 1, vector<int>(k + 1));
        for (int i = 0; i <= n; ++i)
            comb[i][0] = 1;
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= k; ++j)
                comb[i][j] = comb[i - 1][j] + comb[i - 1][j - 1];
        return comb;
    }
};
```

705. 1644-2.cpp

Path: LeetCode-main/solutions/1644. Lowest Common Ancestor of a Binary Tree II/1644-2.cpp

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        TreeNode* ans = getLCA(root, p, q);
        if (ans == p) // Search q in the subtree rooted at p.
            return getLCA(p, q, q) == nullptr ? nullptr : ans;
        if (ans == q) // Search p in the subtree rooted at q.
            return getLCA(q, p, p) == nullptr ? nullptr : ans;
        return ans; // (ans != p && ans != q) || ans == nullptr
    }

private:
    TreeNode* getLCA(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root == nullptr || root == p || root == q)
            return root;
        TreeNode* left = getLCA(root->left, p, q);
        TreeNode* right = getLCA(root->right, p, q);
        if (left != nullptr && right != nullptr)
            return root;
        return left == nullptr ? right : left;
    }
};
```

706. 1644.cpp

Path: LeetCode-main/solutions/1644. Lowest Common Ancestor of a Binary Tree II/1644.cpp

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        bool seenP = false;
        bool seenQ = false;
        TreeNode* lca = getLCA(root, p, q, seenP, seenQ);
        return seenP && seenQ ? lca : nullptr;
    }

private:
    TreeNode* getLCA(TreeNode* root, TreeNode* p, TreeNode* q, bool& seenP,
                    bool& seenQ) {
        if (root == nullptr)
            return nullptr;
        // Need to traverse the entire tree to update `seenP` and `seenQ`.
        TreeNode* left = getLCA(root->left, p, q, seenP, seenQ);
        TreeNode* right = getLCA(root->right, p, q, seenP, seenQ);
        if (root == p) {
            seenP = true;
            return root;
        }
        if (root == q) {
            seenQ = true;
            return root;
        }
        if (left != nullptr && right != nullptr)
            return root;
        return left == nullptr ? right : left;
    }
};
```


707. 1646.cpp

Path: LeetCode-main/solutions/1646. Get Maximum in Generated Array/1646.cpp

```
class Solution {
public:
    int getMaximumGenerated(int n) {
        if (n == 0)
            return 0;
        if (n == 1)
            return 1;

        vector<int> nums(n + 1);
        nums[1] = 1;

        for (int i = 1; (2 * i + 1) <= n; ++i) {
            nums[2 * i] = nums[i];
            nums[2 * i + 1] = nums[i] + nums[i + 1];
        }

        return ranges::max(nums);
    }
};
```

708. 1647.cpp

Path: LeetCode-main/solutions/1647. Minimum Deletions to Make Character Frequencies Unique/1647.cpp

```
class Solution {
public:
    int minDeletions(string s) {
        int ans = 0;
        vector<int> count(26);
        unordered_set<int> usedFreq;

        for (const char c : s)
            ++count[c - 'a'];

        for (int freq : count)
            while (freq > 0 && !usedFreq.insert(freq).second) {
                --freq; // Delete ('a' + i).
                ++ans;
            }

        return ans;
    }
};
```

709. 1648.cpp

Path: LeetCode-main/solutions/1648. Sell Diminishing-Valued Colored Balls/1648.cpp

```
class Solution {
public:
    int maxProfit(vector<int>& inventory, int orders) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        long largestCount = 1;

        ranges::sort(inventory, greater<>());

        for (int i = 0; i < inventory.size(); ++i, ++largestCount)
            if (i == inventory.size() - 1 || inventory[i] > inventory[i + 1]) {
                // If we are at the last inventory, or inventory[i] > inventory[i + 1].
                // In either case, we will pick inventory[i - largestCount + 1..i].
                const int pick = (i == inventory.size() - 1)
                    ? inventory[i]
                    : inventory[i] - inventory[i + 1];
                if (largestCount * pick >= orders) {
                    // We have run out of orders, so we need to recalculate the number of
                    // balls that we actually pick for inventory[i - largestCount + 1..i].
                    const int actualPick = orders / largestCount;
                    const int remaining = orders % largestCount;
                    return (ans +
                        largestCount *
                            trapezoid(inventory[i], inventory[i] - actualPick + 1) +
                        static_cast<long>(remaining) * (inventory[i] - actualPick)) %
                        kMod;
                }
                ans += largestCount * trapezoid(inventory[i], inventory[i] - pick + 1);
                ans %= kMod;
                orders -= largestCount * pick;
            }

        throw;
    }

private:
    long trapezoid(long a, long b) {
        return (a + b) * (a - b + 1) / 2;
    }
};
```

710. 1649-2.cpp

Path: LeetCode-main/solutions/1649. Create Sorted Array through Instructions/1649-2.cpp

```
template <typename NodeType, typename ValueType>
class SegmentTree {
public:
    explicit SegmentTree(const int n, const NodeType& defaultNode)
        : n(n), defaultNode(defaultNode), tree(4 * n) {}

    // Adds nums[i] to val equivalently.
    void add(int i, ValueType val) {
        add(0, 0, n - 1, i, val);
    }

    // Returns the result of the range query from nums[i..j].
    NodeType query(int i, int j) const {
        return query(0, 0, n - 1, i, j);
    }

protected:
    // Merges the result of the left node and the right node.
    virtual NodeType merge(const NodeType& a, const NodeType& b) const = 0;
    virtual NodeType makeLeafNode(ValueType val) const = 0;

private:
    const int n; // the size of the input array
    const NodeType defaultNode; // default node value for non-overlapping queries
    vector<NodeType> tree; // the segment tree

    void add(int treeIndex, int lo, int hi, int i, ValueType val) {
        if (lo == hi) {
            tree[treeIndex] += makeLeafNode(val);
            return;
        }
        const int mid = (lo + hi) / 2;
        if (i <= mid)
            add(2 * treeIndex + 1, lo, mid, i, val);
        else
            add(2 * treeIndex + 2, mid + 1, hi, i, val);
        tree[treeIndex] = merge(tree[2 * treeIndex + 1], tree[2 * treeIndex + 2]);
    }

    NodeType query(int treeIndex, int lo, int hi, int i, int j) const {
        if (i <= lo && hi <= j) // [lo, hi] lies completely inside [i, j].
            return tree[treeIndex];
        if (j < lo || hi < i) // [lo, hi] lies completely outside [i, j].
            return defaultNode;
        const int mid = (lo + hi) / 2;
        return merge(query(2 * treeIndex + 1, lo, mid, i, j),
                     query(2 * treeIndex + 2, mid + 1, hi, i, j));
    }
};

class SumSegmentTree : public SegmentTree<int, int> {
public:
    explicit SumSegmentTree(int n) : SegmentTree(n, 0) {}

protected:
    int merge(const int& a, const int& b) const override {
        return a + b;
    }

    int makeLeafNode(int val) const override {
        return val;
    }
};

class Solution {
public:
    int createSortedArray(vector<int>& instructions) {
        constexpr int kMod = 1'000'000'007;
        const int mx = ranges::max(instructions);
        int ans = 0;
        SumSegmentTree tree(mx + 1);

        for (const int i : instructions) {
            ans += min(tree.query(0, i - 1), tree.query(i + 1, mx));
            ans %= kMod;
            tree.add(i, 1);
        }

        return ans;
    }
};
```


711. 1649-3.cpp

Path: LeetCode-main/solutions/1649. Create Sorted Array through Instructions/1649-3.cpp

```
struct Item {
    int num;
    int index;
};

class Solution {
public:
    int createSortedArray(vector<int>& instructions) {
        constexpr int kMod = 1'000'000'007;
        const int n = instructions.size();
        int ans = 0;
        vector<Item> items;
        // lesser[i] := the number of numbers < instructions[i]
        vector<int> lesser(n);
        // greater[i] := the number of numbers > instructions[i]
        vector<int> greater(n);

        for (int i = 0; i < n; ++i)
            items.push_back({instructions[i], i});

        mergeSort(items, 0, n - 1, lesser, greater);

        for (int i = 0; i < n; ++i) {
            ans += min(lesser[i], greater[i]);
            ans %= kMod;
        }

        return ans;
    }

private:
    void mergeSort(vector<Item>& items, int l, int r, vector<int>& lesser,
                  vector<int>& greater) {
        if (l >= r)
            return;

        const int m = (l + r) / 2;
        mergeSort(items, l, m, lesser, greater);
        mergeSort(items, m + 1, r, lesser, greater);
        merge(items, l, m, r, lesser, greater);
    }

    void merge(vector<Item>& items, int l, int m, int r, vector<int>& lesser,
              vector<int>& greater) {
        int lo = l; // the first index s.t. items[lo].num >= items[i].num
        int hi = l; // the first index s.t. items[hi].num > items[i].num

        for (int i = m + 1; i <= r; ++i) {
            while (lo <= m && items[lo].num < items[i].num)
                ++lo;
            while (hi <= m && items[hi].num <= items[i].num)
                ++hi;
            lesser[items[i].index] += lo - l;
            greater[items[i].index] += m - hi + 1;
        }

        vector<Item> sorted(r - l + 1);
        int k = 0; // sorted's index
        int i = l; // left's index
        int j = m + 1; // right's index

        while (i <= m && j <= r)
            if (items[i].num < items[j].num)
                sorted[k++] = items[i++];
            else
                sorted[k++] = items[j++];

        // Put the possible remaining left part into the sorted array.
        while (i <= m)
            sorted[k++] = items[i++];

        // Put the possible remaining right part into the sorted array.
        while (j <= r)
            sorted[k++] = items[j++];

        copy(sorted.begin(), sorted.end(), items.begin() + l);
    }
};
```

712. 1649.cpp

Path: LeetCode-main/solutions/1649. Create Sorted Array through Instructions/1649.cpp

```
class FenwickTree {
public:
    FenwickTree(int n) : sums(n + 1) {}

    void add(int i, int delta) {
        while (i < sums.size()) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    int get(int i) const {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

private:
    vector<int> sums;

    static inline int lowbit(int i) {
        return i & -i;
    }
};

class Solution {
public:
    int createSortedArray(vector<int>& instructions) {
        constexpr int kMod = 1'000'000'007;
        const int mx = ranges::max(instructions);
        int ans = 0;
        FenwickTree tree(mx);

        for (int i = 0; i < instructions.size(); ++i) {
            ans += min(tree.get(instructions[i] - 1), i - tree.get(instructions[i]));
            ans %= kMod;
            tree.add(instructions[i], 1);
        }

        return ans;
    }
};
```

713. 165.cpp

Path: LeetCode-main/solutions/165. Compare Version Numbers/165.cpp

```
class Solution {
public:
    int compareVersion(string version1, string version2) {
        istringstream iss1(version1);
        istringstream iss2(version2);
        int v1;
        int v2;
        char dotChar;

        while (bool(iss1 >> v1) + bool(iss2 >> v2)) {
            if (v1 < v2)
                return -1;
            if (v1 > v2)
                return 1;
            iss1 >> dotChar;
            iss2 >> dotChar;
            v1 = 0;
            v2 = 0;
        }

        return 0;
    };
};
```


714. 1650.cpp

Path: LeetCode-main/solutions/1650. Lowest Common Ancestor of a Binary Tree III/1650.cpp

```
class Solution {
public:
    // Same as 160. Intersection of Two Linked Lists
    Node* lowestCommonAncestor(Node* p, Node* q) {
        Node* a = p;
        Node* b = q;

        while (a != b) {
            a = a == nullptr ? q : a->parent;
            b = b == nullptr ? p : b->parent;
        }

        return a;
    }
};
```

715. 1652.cpp

Path: LeetCode-main/solutions/1652. Defuse the Bomb/1652.cpp

```
class Solution {
public:
    vector<int> decrypt(vector<int>& code, int k) {
        const int n = code.size();
        vector<int> ans(n);
        if (k == 0)
            return ans;

        int sum = 0;
        int start = k > 0 ? 1 : n + k; // the start of the next k numbers
        int end = k > 0 ? k : n - 1;  // the end of the next k numbers

        for (int i = start; i <= end; ++i)
            sum += code[i];

        for (int i = 0; i < n; ++i) {
            ans[i] = sum;
            sum -= code[start++ % n];
            sum += code[++end % n];
        }

        return ans;
    }
};
```

716. 1653.cpp

Path: LeetCode-main/solutions/1653. Minimum Deletions to Make String Balanced/1653.cpp

```
class Solution {
public:
    // Same as 926. Flip String to Monotone Increasing
    int minimumDeletions(string s) {
        // the number of characters to be deleted to make the substring so far
        // balanced
        int dp = 0;
        int countB = 0;

        for (const char c : s)
            if (c == 'a')
                // 1. Delete 'a'.
                // 2. Keep 'a' and delete the previous 'b's.
                dp = min(dp + 1, countB);
            else
                ++countB;

        return dp;
    }
};
```

717. 1654.cpp

Path: LeetCode-main/solutions/1654. Minimum Jumps to Reach Home/1654.cpp

```
enum class Direction { kForward, kBackward };

class Solution {
public:
    int minimumJumps(vector<int>& forbidden, int a, int b, int x) {
        int furthest = x + a + b;
        unordered_set<int> seenForward;
        unordered_set<int> seenBackward;

        for (const int pos : forbidden) {
            seenForward.insert(pos);
            seenBackward.insert(pos);
            furthest = max(furthest, pos + a + b);
        }

        // (direction, position)
        queue<pair<Direction, int>> q{{{Direction::kForward, 0}}};

        for (int ans = 0; !q.empty(); ++ans)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [dir, pos] = q.front();
                q.pop();
                if (pos == x)
                    return ans;
                const int forward = pos + a;
                const int backward = pos - b;
                if (forward <= furthest && seenForward.insert(forward).second)
                    q.emplace(Direction::kForward, forward);
                // It cannot jump backward twice in a row.
                if (dir == Direction::kForward && backward >= 0 &&
                    seenBackward.insert(backward).second)
                    q.emplace(Direction::kBackward, backward);
            }

        return -1;
    }
};
```

718. 1655.cpp

Path: LeetCode-main/solutions/1655. Distribute Repeating Integers/1655.cpp

```
class Solution {
public:
    bool canDistribute(vector<int>& nums, vector<int>& quantity) {
        // validDistribution[i][j] := true if it's possible to distribute the i-th
        // freq into a subset of quantity represented by the bitmask j
        const vector<int> freqs = getFreqs(nums);
        const vector<vector<bool>> validDistribution =
            getValidDistribution(freqs, quantity);
        const int n = freqs.size();
        const int m = quantity.size();
        const int maxMask = 1 << m;
        // dp[i][j] := true if it's possible to distribute freqs[i..n), where j is
        // the bitmask of the selected quantity
        vector<vector<bool>> dp(n + 1, vector<bool>(maxMask));
        dp[n][maxMask - 1] = true;

        for (int i = n - 1; i >= 0; --i)
            for (int mask = 0; mask < maxMask; ++mask) {
                dp[i][mask] = dp[i + 1][mask];
                const int availableMask = ~mask & (maxMask - 1);
                for (int submask = availableMask; submask > 0;
                     submask = (submask - 1) & availableMask)
                    if (validDistribution[i][submask])
                        dp[i][mask] = dp[i][mask] || dp[i + 1][mask | submask];
            }

        return dp[0][0];
    }

private:
    vector<int> getFreqs(const vector<int>& nums) {
        vector<int> freqs;
        unordered_map<int, int> count;
        for (const int num : nums)
            ++count[num];
        for (const auto& [_, freq] : count)
            freqs.push_back(freq);
        return freqs;
    }

    vector<vector<bool>> getValidDistribution(const vector<int>& freqs,
                                           const vector<int>& quantity) {
        const int maxMask = 1 << quantity.size();
        vector<vector<bool>> validDistribution(freqs.size(), vector<bool>(maxMask));
        for (int i = 0; i < freqs.size(); ++i)
            for (int mask = 0; mask < maxMask; ++mask)
                if (freqs[i] >= getQuantitySum(quantity, mask))
                    validDistribution[i][mask] = true;
        return validDistribution;
    }

    // Returns the sum of the selected quantity represented by `mask`.
    int getQuantitySum(const vector<int>& quantity, int mask) {
        int sum = 0;
        for (int i = 0; i < quantity.size(); ++i)
            if (mask >> i & 1)
                sum += quantity[i];
        return sum;
    }
};
```

719. 1656.cpp

Path: LeetCode-main/solutions/1656. Design an Ordered Stream/1656.cpp

```
class OrderedStream {
public:
    OrderedStream(int n) : values(n) {}

    vector<string> insert(int idKey, string value) {
        --idKey; // Converts to 0-indexed.
        values[idKey] = value;
        if (idKey > i)
            return {};
        while (i < values.size() && !values[i].empty())
            ++i;
        return vector<string>{values.begin() + idKey, values.begin() + i};
    }

private:
    vector<string> values;
    int i = 0; // values' index (0-indexed)
};
```

720. 1657.cpp

Path: LeetCode-main/solutions/1657. Determine if Two Strings Are Close/1657.cpp

```
class Solution {
public:
    bool closeStrings(string word1, string word2) {
        if (word1.length() != word2.length())
            return false;

        unordered_map<char, int> count1;
        unordered_map<char, int> count2;
        string s1;           // Unique chars in word1
        string s2;           // Unique chars in word2
        vector<int> freqs1;   // Freqs of unique chars in word1
        vector<int> freqs2;   // Freqs of unique chars in word2

        for (const char c : word1)
            ++count1[c];

        for (const char c : word2)
            ++count2[c];

        for (const auto& [c, freq] : count1) {
            s1 += c;
            freqs1.push_back(freq);
        }

        for (const auto& [c, freq] : count2) {
            s2 += c;
            freqs2.push_back(freq);
        }

        ranges::sort(s1);
        ranges::sort(s2);

        if (s1 != s2)
            return false;

        ranges::sort(freqs1);
        ranges::sort(freqs2);
        return freqs1 == freqs2;
    }
};
```

721. 1658.cpp

Path: LeetCode-main/solutions/1658. Minimum Operations to Reduce X to Zero/1658.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums, int x) {
        const int targetSum = accumulate(nums.begin(), nums.end(), 0) - x;
        if (targetSum == 0)
            return nums.size();
        const int maxLen = maxSubArrayLen(nums, targetSum);
        return maxLen == -1 ? -1 : nums.size() - maxLen;
    }

private:
    // Same as 325. Maximum Size Subarray Sum Equals k
    int maxSubArrayLen(vector<int>& nums, int k) {
        int res = -1;
        int prefix = 0;
        unordered_map<int, int> prefixToIndex{{0, -1}};

        for (int i = 0; i < nums.size(); ++i) {
            prefix += nums[i];
            const int target = prefix - k;
            if (const auto it = prefixToIndex.find(target);
                it != prefixToIndex.cend())
                res = max(res, i - it->second);
            // No need to check the existence of the prefix since it's unique.
            prefixToIndex[prefix] = i;
        }

        return res;
    }
};
```


722. 1659.cpp

Path: LeetCode-main/solutions/1659. Maximize Grid Happiness/1659.cpp

```
class Solution {
public:
    int getMaxGridHappiness(int m, int n, int introvertsCount,
                           int extrovertsCount) {
        const int twoToThePowerOfN = pow(2, n);
        vector<vector<vector<vector<vector<int>>>>>>> mem(
            m * n, vector<vector<vector<vector<int>>>>>(
                twoToThePowerOfN,
                vector<vector<vector<int>>>>(
                    twoToThePowerOfN,
                    vector<vector<int>>>(introvertsCount + 1,
                                       vector<int>(extrovertsCount + 1))))));
        return getMaxGridHappiness(m, n, 0, 0, 0, introvertsCount, extrovertsCount,
                                   mem);
    }

private:
    // Calculates the cost based on left and up neighbors.
    //
    // The `diff` parameter represents the happiness change due to the current
    // placed person in (i, j). We add `diff` each time we encounter a neighbor
    // (left or up) who is already placed.
    //
    // 1. If the neighbor is an introvert, we subtract 30 from cost.
    // 2. If the neighbor is an extrovert, we add 20 to from cost.
    int getPlacementCost(int n, int i, int j, int inMask, int exMask, int diff) {
        int cost = 0;
        if (i > 0) {
            if ((1 << (n - 1)) & inMask)
                cost += diff - 30;
            if ((1 << (n - 1)) & exMask)
                cost += diff + 20;
        }
        if (j > 0) {
            if (1 & inMask)
                cost += diff - 30;
            if (1 & exMask)
                cost += diff + 20;
        }
        return cost;
    }

    int getMaxGridHappiness(int m, int n, int pos, int inMask, int exMask,
                           int inCount, int exCount,
                           vector<vector<vector<vector<vector<int>>>>>>> mem) {
        // `inMask` is the placement of introvert people in the last n cells.
        // e.g. if we have m = 2, n = 3, i = 1, j = 1, then inMask = 0b101 means
        //
        // ? 1 0
        // 1 x ? (x := current position)
        const int i = pos / n;
        const int j = pos % n;
        if (i == m)
            return 0;
        if (mem[pos][inMask][exMask][inCount][exCount] > 0)
            return mem[pos][inMask][exMask][inCount][exCount];

        const int shiftedInMask = (inMask << 1) & ((1 << n) - 1);
        const int shiftedExMask = (exMask << 1) & ((1 << n) - 1);

        const int skip = getMaxGridHappiness(m, n, pos + 1, shiftedInMask,
                                             shiftedExMask, inCount, exCount, mem);
        const int placeIntrovert =
            inCount > 0
            ? 120 + getPlacementCost(n, i, j, inMask, exMask, -30) +
              getMaxGridHappiness(m, n, pos + 1, shiftedInMask | 1,
                                  shiftedExMask, inCount - 1, exCount, mem)
            : INT_MIN;
        const int placeExtrovert =
            exCount > 0 ? 40 + getPlacementCost(n, i, j, inMask, exMask, 20) +
              getMaxGridHappiness(m, n, pos + 1, shiftedInMask,
                                  shiftedExMask | 1, inCount,
                                  exCount - 1, mem)
            : INT_MIN;
        return mem[pos][inMask][exMask][inCount][exCount] =
            max({skip, placeIntrovert, placeExtrovert});
    }
};
```

723. 166.cpp

Path: LeetCode-main/solutions/166. Fraction to Recurring Decimal/166.cpp

```
class Solution {
public:
    string fractionToDecimal(int numerator, int denominator) {
        if (numerator == 0)
            return "0";

        string ans;

        if (numerator < 0 ^ denominator < 0)
            ans += "-";

        long n = labs(numerator);
        long d = labs(denominator);
        ans += to_string(n / d);

        if (n % d == 0)
            return ans;

        ans += '.';
        unordered_map<int, int> seen;

        for (long r = n % d; r; r %= d) {
            if (const auto it = seen.find(r); it != seen.cend()) {
                ans.insert(it->second, 1, '(');
                ans += ')';
                break;
            }
            seen[r] = ans.size();
            r *= 10;
            ans += to_string(r / d);
        }

        return ans;
    }
};
```

724. 1660.cpp

Path: LeetCode-main/solutions/1660. Correct a Binary Tree/1660.cpp

```
class Solution {
public:
    TreeNode* correctBinaryTree(TreeNode* root) {
        if (root == nullptr)
            return nullptr;
        if (root->right != nullptr && seen.contains(root->right->val))
            return nullptr;
        seen.insert(root->val);
        root->right = correctBinaryTree(root->right);
        root->left = correctBinaryTree(root->left);
        return root;
    }

private:
    unordered_set<int> seen;
};
```

725. 1662.cpp

Path: LeetCode-main/solutions/1662. Check If Two String Arrays are Equivalent/1662.cpp

```
class Solution {
public:
    bool arrayStringsAreEqual(std::vector<std::string>& word1,
                             std::vector<std::string>& word2) {
        int i = 0; // word1's index
        int j = 0; // word2's index
        int a = 0; // word1[i]'s index
        int b = 0; // word2[j]'s index

        while (i < word1.size() && j < word2.size()) {
            if (word1[i][a] != word2[j][b])
                return false;
            if (++a == word1[i].size()) {
                ++i;
                a = 0;
            }
            if (++b == word2[j].size()) {
                ++j;
                b = 0;
            }
        }

        return i == word1.size() && j == word2.size();
    }
};
```

726. 1663.cpp

Path: LeetCode-main/solutions/1663. Smallest String With A Given Numeric Value/1663.cpp

```
class Solution {
public:
    string getSmallestString(int n, int k) {
        string ans;

        for (int i = 0; i < n; ++i) {
            const int remainingLetters = n - 1 - i;
            const int rank = max(1, k - remainingLetters * 26);
            ans += 'a' + rank - 1;
            k -= rank;
        }

        return ans;
    }
};
```

727. 1664-2.cpp

Path: LeetCode-main/solutions/1664. Ways to Make a Fair Array/1664-2.cpp

```
class Solution {
public:
    int waysToMakeFair(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        // l[0] := the sum of even-indexed nums[0..i)
        // l[1] := the sum of odd-indexed nums[0..i)
        // r[0] := the sum of even-indexed nums[i + 1..n)
        // r[1] := the sum of odd-indexed nums[i + 1..n)
        vector<int> l(2);
        vector<int> r(2);

        for (int i = 0; i < n; ++i)
            r[i % 2] += nums[i];

        for (int i = 0; i < n; ++i) {
            r[i % 2] -= nums[i];
            if (l[0] + r[1] == l[1] + r[0])
                ++ans;
            l[i % 2] += nums[i];
        }

        return ans;
    }
};
```

728. 1664.cpp

Path: LeetCode-main/solutions/1664. Ways to Make a Fair Array/1664.cpp

```
class Solution {
public:
    int waysToMakeFair(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        vector<int> even(n + 1); // the sum of even-indexed nums[0..i)
        vector<int> odd(n + 1);  // the sum of odd-indexed nums[0..i)

        for (int i = 1; i <= n; ++i) {
            odd[i] = odd[i - 1];
            even[i] = even[i - 1];
            if (i % 2 == 0)
                even[i] += nums[i - 1];
            else
                odd[i] += nums[i - 1];
        }

        const int sum = even.back() + odd.back();

        for (int i = 0; i < n; ++i) {
            const int evenSum = even[i] + odd.back() - odd[i + 1];
            const int oddSum = sum - nums[i] - evenSum;
            if (evenSum == oddSum)
                ++ans;
        }

        return ans;
    }
};
```

729. 1665.cpp

Path: LeetCode-main/solutions/1665. Minimum Initial Energy to Finish Tasks/1665.cpp

```
class Solution {
public:
    int minimumEffort(vector<vector<int>>& tasks) {
        int ans = 0;
        int prevSaved = 0;

        ranges::sort(tasks, ranges::greater{},
            [](const vector<int>& task) { return task[1] - task[0]; });

        for (const vector<int>& task : tasks) {
            const int actual = task[0];
            const int minimum = task[1];
            if (prevSaved < minimum) {
                ans += minimum - prevSaved;
                prevSaved = minimum - actual;
            } else {
                prevSaved -= actual;
            }
        }

        return ans;
    }
};
```


730. 1666.cpp

Path: LeetCode-main/solutions/1666. Change the Root of a Binary Tree/1666.cpp

```
class Solution {
public:
    Node* flipBinaryTree(Node* root, Node* leaf) {
        return reroot(root, leaf, nullptr);
    }

private:
    Node* reroot(Node* root, Node* node, Node* newParent) {
        Node* oldParent = node->parent;
        node->parent = newParent;

        // Clean up the child if it's the new parent.
        if (node->left == newParent)
            node->left = nullptr;
        if (node->right == newParent)
            node->right = nullptr;

        // We meet the original root, so we're done.
        if (node == root)
            return node;

        if (node->left)
            node->right = node->left;
        node->left = reroot(root, oldParent, node);

        return node;
    }
};
```

731. 1668.cpp

Path: LeetCode-main/solutions/1668. Maximum Repeating Substring/1668.cpp

```
class Solution {
public:
    int maxRepeating(string sequence, string word) {
        int ans = 1;
        string repeating = word;
        while (sequence.find(repeating) != string::npos) {
            ++ans;
            repeating += word;
        }
        return ans - 1;
    }
};
```

732. 1669.cpp

Path: LeetCode-main/solutions/1669. Merge In Between Linked Lists/1669.cpp

```
class Solution {
public:
    ListNode* mergeInBetween(ListNode* list1, int a, int b, ListNode* list2) {
        ListNode* nodeBeforeA = list1;
        for (int i = 0; i < a - 1; ++i)
            nodeBeforeA = nodeBeforeA->next;

        ListNode* nodeB = nodeBeforeA->next;
        for (int i = 0; i < b - a; ++i)
            nodeB = nodeB->next;

        nodeBeforeA->next = list2;
        ListNode* lastNodeInList2 = list2;

        while (lastNodeInList2->next != nullptr)
            lastNodeInList2 = lastNodeInList2->next;

        lastNodeInList2->next = nodeB->next;
        nodeB->next = nullptr;
        return list1;
    }
};
```

733. 167.cpp

Path: LeetCode-main/solutions/167. Two Sum II - Input array is sorted/167.cpp

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int l = 0;
        int r = numbers.size() - 1;

        while (numbers[l] + numbers[r] != target)
            if (numbers[l] + numbers[r] < target)
                ++l;
            else
                --r;

        return {l + 1, r + 1};
    }
};
```

734. 1670.cpp

Path: LeetCode-main/solutions/1670. Design Front Middle Back Queue/1670.cpp

```
class FrontMiddleBackQueue {
public:
    void pushFront(int val) {
        frontQueue.push_front(val);
        moveFrontToBackIfNeeded();
    }

    void pushMiddle(int val) {
        if (| frontQueue | == | backQueue |)
            backQueue.push_front(val);
        else
            frontQueue.push_back(val);
    }

    void pushBack(int val) {
        backQueue.push_back(val);
        moveBackToFrontIfNeeded();
    }

    int popFront() {
        if (!frontQueue.empty()) {
            const int x = frontQueue.front();
            frontQueue.pop_front();
            moveBackToFrontIfNeeded();
            return x;
        }
        if (!backQueue.empty()) {
            const int x = backQueue.front();
            backQueue.pop_front();
            moveFrontToBackIfNeeded();
            return x;
        }
        return -1;
    }

    int popMiddle() {
        if (frontQueue.empty() && backQueue.empty())
            return -1;
        if (frontQueue.size() + 1 == backQueue.size()) {
            const int x = backQueue.front();
            backQueue.pop_front();
            return x;
        } else { // |frontQueue| == |backQueue|
            const int x = frontQueue.back();
            frontQueue.pop_back();
            return x;
        }
    }

    int popBack() {
        if (backQueue.empty())
            return -1;
        const int x = backQueue.back();
        backQueue.pop_back();
        moveFrontToBackIfNeeded();
        return x;
    }

private:
    // |frontQueue| = |backQueue| or
    // |frontQueue| = |backQueue| - 1
    deque<int> frontQueue;
    deque<int> backQueue;

    void moveFrontToBackIfNeeded() {
        if (frontQueue.size() - 1 == backQueue.size()) {
            const int x = frontQueue.back();
            frontQueue.pop_back();
            backQueue.push_front(x);
        }
    }

    void moveBackToFrontIfNeeded() {
        if (frontQueue.size() + 2 == backQueue.size()) {
            const int x = backQueue.front();
            backQueue.pop_front();
            frontQueue.push_back(x);
        }
    }
};
```

735. 1671.cpp

Path: LeetCode-main/solutions/1671. Minimum Number of Removals to Make Mountain Array/1671.cpp

```
class Solution {
public:
    int minimumMountainRemovals(vector<int>& nums) {
        const vector<int> l = lengthOfLIS(nums);
        const vector<int> r = reversed(lengthOfLIS(reversed(nums)));
        int maxMountainSeq = 0;

        for (int i = 0; i < nums.size(); ++i)
            if (l[i] > 1 && r[i] > 1)
                maxMountainSeq = max(maxMountainSeq, l[i] + r[i] - 1);

        return nums.size() - maxMountainSeq;
    }

private:
    // Similar to 300. Longest Increasing Subsequence
    vector<int> lengthOfLIS(vector<int> nums) {
        // tails[i] := the minimum tail of all the increasing subsequences having
        // length i + 1
        vector<int> tails;
        // dp[i] := the length of LIS ending in nums[i]
        vector<int> dp;
        for (const int num : nums) {
            if (tails.empty() || num > tails.back())
                tails.push_back(num);
            else
                tails[firstGreaterEqual(tails, num)] = num;
            dp.push_back(tails.size());
        }
        return dp;
    }

    int firstGreaterEqual(const vector<int>& arr, int target) {
        return ranges::lower_bound(arr, target) - arr.begin();
    }

    vector<int> reversed(const vector<int>& nums) {
        return {nums.rbegin(), nums.rend()};
    }
};
```

736. 1672.cpp

Path: LeetCode-main/solutions/1672. Richest Customer Wealth/1672.cpp

```
class Solution {
public:
    int maximumWealth(vector<vector<int>>& accounts) {
        return accumulate(accounts.begin(), accounts.end(), 0,
            [](int acc, const vector<int>& account) {
                return max(acc, accumulate(account.begin(), account.end(), 0));
            });
    }
};
```

737. 1673.cpp

Path: LeetCode-main/solutions/1673. Find the Most Competitive Subsequence/1673.cpp

```
class Solution {
public:
    vector<int> mostCompetitive(vector<int>& nums, int k) {
        vector<int> ans;

        for (int i = 0; i < nums.size(); ++i) {
            // If |ans| - 1 + |nums[i..n]| >= k, then it means we still have enough
            // numbers, and we can safely pop an element from ans.
            while (!ans.empty() && ans.back() > nums[i] &&
                   ans.size() - 1 + nums.size() - i >= k)
                ans.pop_back();
            if (ans.size() < k)
                ans.push_back(nums[i]);
        }

        return ans;
    }
};
```


738. 1674.cpp

Path: LeetCode-main/solutions/1674. Minimum Moves to Make Array Complementary/1674.cpp

```
class Solution {
public:
    int minMoves(vector<int>& nums, int limit) {
        const int n = nums.size();
        int ans = n;
        // delta[i] := the number of moves needed when target goes from i - 1 to i
        vector<int> delta(limit * 2 + 2);

        for (int i = 0; i < n / 2; ++i) {
            const int a = nums[i];
            const int b = nums[n - 1 - i];
            --delta[min(a, b) + 1];
            --delta[a + b];
            ++delta[a + b + 1];
            ++delta[max(a, b) + limit + 1];
        }

        // Initially, we need `moves` when the target is 2.
        for (int i = 2, moves = n; i <= limit * 2; ++i) {
            moves += delta[i];
            ans = min(ans, moves);
        }

        return ans;
    }
};
```

739. 1675.cpp

Path: LeetCode-main/solutions/1675. Minimize Deviation in Array/1675.cpp

```
class Solution {
public:
    int minimumDeviation(vector<int>& nums) {
        int ans = INT_MAX;
        int mn = INT_MAX;
        priority_queue<int> maxHeap;

        for (const int num : nums) {
            const int evenNum = num % 2 == 0 ? num : num * 2;
            mn = min(mn, evenNum);
            maxHeap.push(evenNum);
        }

        while (maxHeap.top() % 2 == 0) {
            const int mx = maxHeap.top();
            maxHeap.pop();
            ans = min(ans, mx - mn);
            mn = min(mn, mx / 2);
            maxHeap.push(mx / 2);
        }

        return min(ans, maxHeap.top() - mn);
    }
};
```

740. 1676.cpp

Path: LeetCode-main/solutions/1676. Lowest Common Ancestor of a Binary Tree IV/1676.cpp

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, vector<TreeNode*>& nodes) {
        unordered_set<TreeNode*> nodesSet{nodes.begin(), nodes.end()};
        return lca(root, nodesSet);
    }

private:
    TreeNode* lca(TreeNode* root, unordered_set<TreeNode*>& nodesSet) {
        if (root == nullptr)
            return nullptr;
        if (nodesSet.contains(root))
            return root;
        TreeNode* left = lca(root->left, nodesSet);
        TreeNode* right = lca(root->right, nodesSet);
        if (left != nullptr && right != nullptr)
            return root;
        return left == nullptr ? right : left;
    }
};
```

741. 1678.cpp

Path: LeetCode-main/solutions/1678. Goal Parser Interpretation/1678.cpp

```
class Solution {
public:
    string interpret(string command) {
        string ans;
        for (int i = 0; i < command.size(); i++)
            if (command[i] == 'G') {
                ans += "G";
            } else if (command[i] == '(') {
                ans += "o";
                i++;
            } else if (command[i] == ')') {
                ans += "al";
                i += 2;
            }
        return ans;
    }
};
```

742. 1679.cpp

Path: LeetCode-main/solutions/1679. Max Number of K-Sum Pairs/1679.cpp

```
class Solution {
public:
    int maxOperations(vector<int>& nums, int k) {
        int ans = 0;
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        for (const auto& [num, freq] : count)
            if (const auto it = count.find(k - num); it != count.end())
                ans += min(freq, it->second);

        return ans / 2;
    }
};
```

743. 168.cpp

Path: LeetCode-main/solutions/168. Excel Sheet Column Title/168.cpp

```
class Solution {
public:
    string convertToTitle(int n) {
        return n == 0 ? ""
            : convertToTitle((n - 1) / 26) + (char)('A' + ((n - 1) % 26));
    }
};
```

744. 1680-2.cpp

Path: LeetCode-main/solutions/1680. Concatenation of Consecutive Binary Numbers/1680-2.cpp

```
class Solution {
public:
    int concatenatedBinary(int n) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        int numberOfBits = 0;

        for (unsigned i = 1; i <= n; ++i) {
            if (popcount(i) == 1)
                ++numberOfBits;
            ans = ((ans << numberOfBits) % kMod + i) % kMod;
        }

        return ans;
    }
};
```

745. 1680.cpp

Path: LeetCode-main/solutions/1680. Concatenation of Consecutive Binary Numbers/1680.cpp

```
class Solution {
public:
    int concatenatedBinary(int n) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;

        for (int i = 1; i <= n; ++i)
            ans = ((ans << numberOfBits(i)) % kMod + i) % kMod;

        return ans;
    }

private:
    int numberOfBits(int n) {
        return log2(n) + 1;
    }
};
```


746. 1681.cpp

Path: LeetCode-main/solutions/1681. Minimum Incompatibility/1681.cpp

```
class Solution {
public:
    int minimumIncompatibility(vector<int>& nums, int k) {
        constexpr int kMaxCompatibility = (16 - 1) * (16 / 2);
        const int n = nums.size();
        const int subsetSize = n / k;
        const int maxMask = 1 << n;
        const vector<int> incompatibilities =
            getIncompatibilities(nums, subsetSize);
        // dp[i] := the minimum possible sum of incompatibilities of the subset
        // of numbers represented by the bitmask i
        vector<int> dp(maxMask, kMaxCompatibility);
        dp[0] = 0;

        for (unsigned mask = 1; mask < maxMask; ++mask) {
            // The number of 1s in `mask` isn't a multiple of `subsetSize`.
            if (popcount(mask) % subsetSize != 0)
                continue;
            // https://cp-algorithms.com/algebra/all-submasks.html
            for (int submask = mask; submask > 0; submask = (submask - 1) & mask)
                if (incompatibilities[submask] != -1) // valid subset
                    dp[mask] =
                        min(dp[mask], dp[mask - submask] + incompatibilities[submask]);
        }

        return dp.back() == kMaxCompatibility ? -1 : dp.back();
    }

private:
    static constexpr int kMaxNum = 16;

    // Returns an incompatibilities array where
    // * incompatibilities[i] := the incompatibility of the subset of numbers
    //   represented by the bitmask i
    // * incompatibilities[i] := -1 if the number of 1s in the bitmask i is not
    //   `subsetSize`
    vector<int> getIncompatibilities(const vector<int>& nums, int subsetSize) {
        const int maxMask = 1 << nums.size();
        vector<int> incompatibilities(maxMask, -1);
        for (unsigned mask = 0; mask < maxMask; ++mask)
            if (popcount(mask) == subsetSize && isUnique(nums, mask, subsetSize))
                incompatibilities[mask] = getIncompatibility(nums, mask);
        return incompatibilities;
    }

    // Returns true if the numbers selected by `mask` are unique.
    //
    // e.g. If we call isUnique(0b1010, 2, [1, 2, 1, 4]), `used` variable
    // will be 0b1, which only has one 1 (less than `subsetSize`). In this case,
    // we should return false.
    bool isUnique(const vector<int>& nums, int mask, int subsetSize) {
        unsigned used = 0;
        for (int i = 0; i < nums.size(); ++i)
            if (mask >> i & 1)
                used |= 1 << nums[i];
        return popcount(used) == subsetSize;
    }

    // Returns the incompatibility of the selected numbers represented by the
    // `mask`.
    int getIncompatibility(const vector<int>& nums, int mask) {
        int mn = kMaxNum;
        int mx = 0;
        for (int i = 0; i < nums.size(); ++i)
            if (mask >> i & 1) {
                mx = max(mx, nums[i]);
                mn = min(mn, nums[i]);
            }
        return mx - mn;
    }
};
```

747. 1682-2.cpp

Path: LeetCode-main/solutions/1682. Longest Palindromic Subsequence II/1682-2.cpp

```
class Solution {
public:
    int longestPalindromeSubseq(string s) {
        const int n = s.length();
        // dp[i][j][k] := the length of LPS(s[i..j]), where the previous letter is
        // ('a' + k).
        vector<vector<vector<int>>> dp(n, vector<vector<int>>(n, vector<int>(27)));

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i)
                for (int k = 0; k <= 26; ++k) {
                    const int j = i + d;
                    if (s[i] == s[j] && s[i] != 'a' + k)
                        dp[i][j][k] = dp[i + 1][j - 1][s[i] - 'a'] + 2;
                    else
                        dp[i][j][k] = max(dp[i + 1][j][k], dp[i][j - 1][k]);
                }

        return dp[0][n - 1][26];
    }
};
```

748. 1682.cpp

Path: LeetCode-main/solutions/1682. Longest Palindromic Subsequence II/1682.cpp

```
class Solution {
public:
    int longestPalindromeSubseq(string s) {
        const int n = s.length();
        vector<vector<vector<int>>> mem(n, vector<vector<int>>(n, vector<int>(27)));
        return lps(s, 0, n - 1, 26, mem);
    }

private:
    // Returns the length of LPS(s[i..j]), where the previous letter is ('a' + k).
    int lps(const string& s, int i, int j, int k,
            vector<vector<vector<int>>>& mem) {
        if (i >= j)
            return 0;
        if (mem[i][j][k] > 0)
            return mem[i][j][k];
        if (s[i] == s[j] && s[i] != 'a' + k)
            return mem[i][j][k] = lps(s, i + 1, j - 1, s[i] - 'a', mem) + 2;
        return mem[i][j][k] =
            max(lps(s, i + 1, j, k, mem), lps(s, i, j - 1, k, mem));
    }
};
```

749. 1684.cpp

Path: LeetCode-main/solutions/1684. Count the Number of Consistent Strings/1684.cpp

```
class Solution {
public:
    int countConsistentStrings(string allowed, vector<string>& words) {
        return accumulate(words.begin(), words.end(), 0,
            [&allowed](int acc, const string& word) {
                return acc + ranges::all_of(word, [&allowed](char c) {
                    return allowed.find(c) != string::npos;
                });
            });
    }
};
```

750. 1685.cpp

Path: LeetCode-main/solutions/1685. Sum of Absolute Differences in a Sorted Array/1685.cpp

```
class Solution {
public:
    vector<int> getSumAbsoluteDifferences(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans;
        // prefix[i] := sum(nums[0..i])
        vector<int> prefix(n);
        // suffix[i] := sum(nums[i..n - 1])
        vector<int> suffix(n);

        prefix[0] = nums[0];
        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i - 1] + nums[i];

        suffix[n - 1] = nums[n - 1];
        for (int i = n - 2; i >= 0; --i)
            suffix[i] = suffix[i + 1] + nums[i];

        for (int i = 0; i < nums.size(); ++i) {
            const int left = prefix[i] * (i + 1) - prefix[i];
            const int right = suffix[i] - nums[i] * (n - i);
            ans.push_back(left + right);
        }

        return ans;
    }
};
```

751. 1686.cpp

Path: LeetCode-main/solutions/1686. Stone Game VI/1686.cpp

```
class Solution {
public:
    int stoneGameVI(vector<int>& aliceValues, vector<int>& bobValues) {
        const int n = aliceValues.size();
        vector<vector<int>> values;
        int a = 0;
        int b = 0;

        for (int i = 0; i < n; ++i)
            values.push_back({aliceValues[i], bobValues[i]});

        ranges::sort(values, ranges::greater{},
            [](const vector<int>& value) { return value[0] + value[1]; });

        for (int i = 0; i < n; ++i)
            if (i % 2 == 0)
                a += values[i][0];
            else
                b += values[i][1];

        return a > b ? 1 : a < b ? -1 : 0;
    }
};
```

752. 1687.cpp

Path: LeetCode-main/solutions/1687. Delivering Boxes from Storage to Ports/1687.cpp

```
class Solution {
public:
    int boxDelivering(vector<vector<int>>& boxes, int portsCount, int maxBoxes,
                     int maxWeight) {
        const int n = boxes.size();
        // dp[i] := the minimum trips to deliver boxes[0..i) and return to the
        // storage
        vector<int> dp(n + 1);
        int trips = 2;
        int weight = 0;

        for (int l = 0, r = 0; r < n; ++r) {
            weight += boxes[r][1];

            // The current box is different from the previous one, need to make one
            // more trip.
            if (r > 0 && boxes[r][0] != boxes[r - 1][0])
                ++trips;

            while (r - l + 1 > maxBoxes || weight > maxWeight ||
                  // Loading boxes[l] in the previous turn is always no bad than
                  // loading it in this turn.
                  (l < r && dp[l + 1] == dp[l])) {
                weight -= boxes[l][1];
                if (boxes[l][0] != boxes[l + 1][0])
                    --trips;
                ++l;
            }

            // min trips to deliver boxes[0..r]
            // = min trips to deliver boxes[0..l) + trips to deliver boxes[l..r]
            dp[r + 1] = dp[l] + trips;
        }

        return dp[n];
    }
};
```

753. 1688.cpp

Path: LeetCode-main/solutions/1688. Count of Matches in Tournament/1688.cpp

```
class Solution {  
public:  
    int numberOfMatches(int n) {  
        return n - 1;  
    }  
};
```


754. 1689.cpp

Path: LeetCode-main/solutions/1689. Partitioning Into Minimum Number Of Deci-Binary Numbers/1689.cpp

```
class Solution {  
public:  
    int minPartitions(string n) {  
        return ranges::max(n) - '0';  
    }  
};
```

755. 169.cpp

Path: LeetCode-main/solutions/169. Majority Element/169.cpp

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int ans;
        int count = 0;

        for (const int num : nums) {
            if (count == 0)
                ans = num;
            count += num == ans ? 1 : -1;
        }

        return ans;
    }
};
```

756. 1690-2.cpp

Path: LeetCode-main/solutions/1690. Stone Game VII/1690-2.cpp

```
class Solution {
public:
    int stoneGameVII(vector<int>& stones) {
        const int n = stones.size();
        // dp[i][j] := the maximum score you can get more than your opponent in
        // stones[i..j]
        vector<vector<int>> dp(n, vector<int>(n));
        vector<int> prefix(n + 1);

        partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                dp[i][j] = max(prefix[j + 1] - prefix[i + 1] - dp[i + 1][j],
                               prefix[j] - prefix[i] - dp[i][j - 1]);
            }
        return dp[0][n - 1];
    }
};
```

757. 1690.cpp

Path: LeetCode-main/solutions/1690. Stone Game VII/1690.cpp

```
class Solution {
public:
    int stoneGameVII(vector<int>& stones) {
        const int n = stones.size();
        vector<vector<int>>> mem(n, vector<int>(n));
        vector<int> prefix(n + 1);
        partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);
        return stoneGameVII(stones, 0, n - 1, prefix, mem);
    }

private:
    // Returns the maximum score you can get more than your opponent in
    // stones[i..j].
    int stoneGameVII(const vector<int>& stones, int i, int j,
                     const vector<int>& prefix, vector<vector<int>>>& mem) {
        if (i == j)
            return 0;
        if (mem[i][j] > 0)
            return mem[i][j];
        // Remove stones[i] to get sum(stones[i + 1..j]).
        const int removeLeft = prefix[j + 1] - prefix[i + 1] -
                                stoneGameVII(stones, i + 1, j, prefix, mem);
        // Remove stones[j] to get sum(stones[i..j - 1]).
        const int removeRight = prefix[j] - prefix[i] - //
                                stoneGameVII(stones, i, j - 1, prefix, mem);
        return mem[i][j] = max(removeLeft, removeRight);
    }
};
```

758. 1691.cpp

Path: LeetCode-main/solutions/1691. Maximum Height by Stacking Cuboids/1691.cpp

```
class Solution {
public:
    int maxHeight(vector<vector<int>>& cuboids) {
        // For each cuboid, sort it so that c[0] <= c[1] <= c[2].
        for (vector<int>& cuboid : cuboids)
            ranges::sort(cuboid);

        ranges::sort(cuboids);

        // dp[i] := the maximum height with cuboids[i] in the bottom
        vector<int> dp(cuboids.size());

        for (int i = 0; i < cuboids.size(); ++i)
            dp[i] = cuboids[i][2];

        for (int i = 1; i < cuboids.size(); ++i)
            for (int j = 0; j < i; ++j)
                if (cuboids[j][0] <= cuboids[i][0] && //
                    cuboids[j][1] <= cuboids[i][1] && //
                    cuboids[j][2] <= cuboids[i][2])
                    dp[i] = max(dp[i], dp[j] + cuboids[i][2]);

        return ranges::max(dp);
    }
};
```

759. 1692-2.cpp

Path: LeetCode-main/solutions/1692. Count Ways to Distribute Candies/1692-2.cpp

```
class Solution {
public:
    int waysToDistribute(int n, int k) {
        constexpr int kMod = 1'000'000'007;
        vector<vector<long>> dp(k + 1, vector<long>(n + 1));

        for (int i = 0; i <= k; ++i)
            dp[i][i] = 1;

        for (int i = 1; i <= k; ++i)
            for (int j = i + 1; j <= n; ++j)
                dp[i][j] = (dp[i - 1][j - 1] + i * dp[i][j - 1]) % kMod;

        return dp[k][n];
    }
};
```

760. 1692.cpp

Path: LeetCode-main/solutions/1692. Count Ways to Distribute Candies/1692.cpp

```
class Solution {
public:
    int waysToDistribute(int n, int k) {
        constexpr int kMod = 1'000'000'007;
        // dp[i][j] := the number of ways to distribute i candies to j bags
        // dp[i][j] = 0, if i < j
        //             = 1, if i == j
        //             = dp[i - 1][j - 1] (the new candy occupies a bag)
        //             + dp[i - 1][j] * j (the new candy is in any of the j bags)
        vector<vector<long>> dp(n + 1, vector<long>(k + 1));

        for (int i = 0; i <= k; ++i)
            dp[i][i] = 1;

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= k; ++j)
                dp[i][j] = (dp[i - 1][j - 1] + dp[i - 1][j] * j) % kMod;

        return dp[n][k];
    }
};
```

761. 1694.cpp

Path: LeetCode-main/solutions/1694. Reformat Phone Number/1694.cpp

```
class Solution {
public:
    string reformatNumber(string number) {
        string ans;

        std::erase(number, '-');
        std::erase(number, ' ');

        int i = 0; // number's index
        for (i = 0; i + 4 < number.length(); i += 3)
            ans += number.substr(i, 3) + '-';

        const int countFinalDigits = number.length() - i;
        if (countFinalDigits < 4)
            ans += number.substr(i);
        else // countFinalDigits == 4
            ans += number.substr(i, 2) + '-' + number.substr(i + 2);

        return ans;
    }
};
```


762. 1695.cpp

Path: LeetCode-main/solutions/1695. Maximum Erasure Value/1695.cpp

```
class Solution {
public:
    int maximumUniqueSubarray(vector<int>& nums) {
        int ans = 0;
        int score = 0;
        unordered_set<int> seen;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            while (!seen.insert(nums[r]).second) {
                score -= nums[l];
                seen.erase(nums[l++]);
            }
            score += nums[r];
            ans = max(ans, score);
        }

        return ans;
    }
};
```

763. 1696.cpp

Path: LeetCode-main/solutions/1696. Jump Game VI/1696.cpp

```
class Solution {
public:
    int maxResult(vector<int>& nums, int k) {
        // Stores dp[i] within the bounds.
        deque<int> maxQ{0};
        // dp[i] := the maximum score to consider nums[0..i]
        vector<int> dp(nums.size());
        dp[0] = nums[0];

        for (int i = 1; i < nums.size(); ++i) {
            // Pop the index if it's out-of-bounds.
            if (maxQ.front() + k < i)
                maxQ.pop_front();
            dp[i] = dp[maxQ.front()] + nums[i];
            // Pop indices that won't be chosen in the future.
            while (!maxQ.empty() && dp[maxQ.back()] <= dp[i])
                maxQ.pop_back();
            maxQ.push_back(i);
        }

        return dp.back();
    }
};
```

764. 1697.cpp

Path: LeetCode-main/solutions/1697. Checking Existence of Edge Length Limited Paths/1697.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    vector<bool> distanceLimitedPathsExist(int n, vector<vector<int>>& edgeList,
                                           vector<vector<int>>& queries) {
        vector<bool> ans(queries.size());
        UnionFind uf(n);

        for (int i = 0; i < queries.size(); ++i)
            queries[i].push_back(i);

        ranges::sort(queries, ranges::less{},
                    [](const vector<int>& query) { return query[2]; });
        ranges::sort(edgeList, ranges::less{},
                    [](const vector<int>& edge) { return edge[2]; });

        int i = 0; // i := edgeList's index
        for (const vector<int>& query : queries) {
            const int p = query[0];
            const int q = query[1];
            const int limit = query[2];
            // Union edges whose distances < limit.
            while (i < edgeList.size() && edgeList[i][2] < limit)
                uf.unionByRank(edgeList[i][0], edgeList[i++][1]);
            if (uf.find(p) == uf.find(q))
                ans[query.back()] = true;
        }

        return ans;
    }
};
```

765. 1698.cpp

Path: LeetCode-main/solutions/1698. Number of Distinct Substrings in a String/1698.cpp

```
class Solution {
public:
    int countDistinct(string s) {
        const int n = s.length();
        int ans = 0;
        vector<long> pow(n + 1);    // pow[i] := kBase^i
        vector<long> hashes(n + 1); // hashes[i] := the hash of s[0..i)

        pow[0] = 1;
        for (int i = 1; i <= n; ++i) {
            pow[i] = pow[i - 1] * kBase % kHash;
            hashes[i] = (hashes[i - 1] * kBase + val(s[i - 1])) % kHash;
        }

        for (int length = 1; length <= n; ++length) {
            unordered_set<int> seen;
            for (int i = 0; i + length <= n; ++i)
                seen.insert(getHash(i, i + length, hashes, pow));
            ans += seen.size();
        }

        return ans;
    }

private:
    static constexpr int kBase = 26;
    static constexpr int kHash = 1'000'000'007;

    static constexpr int val(char c) {
        return c - 'a';
    }

    // Returns the hash of s[l..r).
    long getHash(int l, int r, const vector<long>& hashes,
                 const vector<long>& pow) {
        const long hash = (hashes[r] - hashes[l] * pow[r - l]) % kHash;
        return hash < 0 ? hash + kHash : hash;
    }
};
```

766. 17-2.cpp

Path: LeetCode-main/solutions/17. Letter Combinations of a Phone Number/17-2.cpp

```
class Solution {
public:
    vector<string> letterCombinations(string digits) {
        if (digits.empty())
            return {};

        vector<string> ans{""};
        const vector<string> digitToLetters{"", "", "abc", "def", "ghi",
                                             "jkl", "mno", "pqrs", "tuv", "wxyz"};

        for (const char d : digits) {
            vector<string> temp;
            for (const string& s : ans)
                for (const char c : digitToLetters[d - '0'])
                    temp.push_back(s + c);
            ans = std::move(temp);
        }

        return ans;
    }
};
```

767. 17.cpp

Path: LeetCode-main/solutions/17. Letter Combinations of a Phone Number/17.cpp

```
class Solution {
public:
    vector<string> letterCombinations(string digits) {
        if (digits.empty())
            return {};

        vector<string> ans;

        dfs(digits, 0, "", ans);
        return ans;
    }

private:
    const vector<string> digitToLetters{"", "", "abc", "def", "ghi",
                                         "jkl", "mno", "pqrs", "tuv", "wxyz"};

    void dfs(const string& digits, int i, string&& path, vector<string>& ans) {
        if (i == digits.length()) {
            ans.push_back(path);
            return;
        }

        for (const char letter : digitToLetters[digits[i] - '0']) {
            path.push_back(letter);
            dfs(digits, i + 1, std::move(path), ans);
            path.pop_back();
        }
    }
};
```

768. 170.cpp

Path: LeetCode-main/solutions/170. Two Sum III - Data structure design/170.cpp

```
class TwoSum {
public:
    void add(int number) {
        ++count[number];
    }

    bool find(int value) {
        for (const auto& [key, freq] : count) {
            const int remain = value - key;
            if (key == remain && freq > 1)
                return true;
            if (key != remain && count.contains(remain))
                return true;
        }
        return false;
    }

private:
    unordered_map<int, int> count;
};
```

769. 1700.cpp

Path: LeetCode-main/solutions/1700. Number of Students Unable to Eat Lunch/1700.cpp

```
class Solution {
public:
    int countStudents(vector<int>& students, vector<int>& sandwiches) {
        vector<int> count(2);

        for (const int student : students)
            ++count[student];

        for (int i = 0; i < sandwiches.size(); ++i) {
            if (count[sandwiches[i]] == 0)
                return sandwiches.size() - i;
            --count[sandwiches[i]];
        }

        return 0;
    }
};
```


770. 1701.cpp

Path: LeetCode-main/solutions/1701. Average Waiting Time/1701.cpp

```
class Solution {
public:
    double averageWaitingTime(vector<vector<int>>& customers) {
        double wait = 0;
        double curr = 0;

        for (const vector<int>& c : customers) {
            curr = max(curr, 1.0 * c[0]) + c[1];
            wait += curr - c[0];
        }

        return 1.0 * wait / customers.size();
    }
};
```

771. 1702.cpp

Path: LeetCode-main/solutions/1702. Maximum Binary String After Change/1702.cpp

```
class Solution {
public:
    string maximumBinaryString(string binary) {
        // e.g. binary = "100110"
        // Do Operation 2 -> "100011"
        // Do Operation 1 -> "111011"
        // So, the index of the only '0' is prefixOnes + zeros - 1.
        const int zeros = ranges::count(binary, '0');
        const int prefixOnes = binary.find('0');

        // Make the entire string as 1s.
        binary.assign(binary.length(), '1');

        // Make the only '0' if necessary.
        if (prefixOnes != string::npos)
            binary[prefixOnes + zeros - 1] = '0';
        return binary;
    }
};
```

772. 1703.cpp

Path: LeetCode-main/solutions/1703. Minimum Adjacent Swaps for K Consecutive Ones/1703.cpp

```
class Solution {
public:
    int minMoves(vector<int>& nums, int k) {
        vector<int> ones;

        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == 1)
                ones.push_back(i);

        // Returns the median index of [i..i + k).
        auto getMedIndex = [&](int i) { return (i + (i + k - 1)) / 2; };

        // Calculate the first group: window[0] = A[0..k).
        const int median = ones[getMedIndex(0)];
        int moves = 0;
        for (int i = 0; i < k; ++i)
            moves += abs(ones[i] - median);

        int ans = moves;

        for (int i = 1; i <= ones.size() - k; ++i) {
            const int oldMedianIndex = ones[getMedIndex(i - 1)];
            const int newMedianIndex = ones[getMedIndex(i)];
            if (k % 2 == 1)
                moves += newMedianIndex - oldMedianIndex;
            moves -= newMedianIndex - ones[i - 1];
            moves += ones[i + k - 1] - newMedianIndex;
            ans = min(ans, moves);
        }

        auto nThSum = [&](int n) { return n * (n + 1) / 2; };
        return ans - nThSum((k - 1) / 2) - nThSum(k / 2);
    }
};
```

773. 1704.cpp

Path: LeetCode-main/solutions/1704. Determine if String Halves Are Alike/1704.cpp

```
class Solution {
public:
    bool halvesAreAlike(string s) {
        const string a = s.substr(0, s.length() / 2);
        const string b = s.substr(s.length() / 2);
        const int aVowelsCount =
            ranges::count_if(a, [this](char c) { return isVowel(c); });
        const int bVowelsCount =
            ranges::count_if(b, [this](char c) { return isVowel(c); });
        return aVowelsCount == bVowelsCount;
    }

private:
    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiouAEIOU";
        return kVowels.find(c) != string_view::npos;
    }
};
```

774. 1705.cpp

Path: LeetCode-main/solutions/1705. Maximum Number of Eaten Apples/1705.cpp

```
class Solution {
public:
    int eatenApples(vector<int>& apples, vector<int>& days) {
        const int n = apples.size();
        int ans = 0;
        using P = pair<int, int>; // (the rotten day, the number of apples)
        priority_queue<P, vector<P>, greater<>> minHeap;

        for (int i = 0; i < n || !minHeap.empty(); ++i) { // i := day
            // Remove the rotten apples.
            while (!minHeap.empty() && minHeap.top().first <= i)
                minHeap.pop();
            // Add today's apples.
            if (i < n && apples[i] > 0)
                minHeap.emplace(i + days[i], apples[i]);
            // Eat one apple today.
            if (!minHeap.empty()) {
                const auto [rottenDay, numApples] = minHeap.top();
                minHeap.pop();
                if (numApples > 1)
                    minHeap.emplace(rottenDay, numApples - 1);
                ++ans;
            }
        }

        return ans;
    }
};
```

775. 1706.cpp

Path: LeetCode-main/solutions/1706. Where Will the Ball Fall/1706.cpp

```
class Solution {
public:
    vector<int> findBall(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        // dp[i] := status of the i-th column
        // -1 := empty, 0 := b0, 1 := b1, ...
        vector<int> dp(n);
        // ans[i] := the i-th ball's final position
        vector<int> ans(n, -1);

        iota(dp.begin(), dp.end(), 0);

        for (int i = 0; i < m; ++i) {
            vector<int> newDp(n, -1);
            for (int j = 0; j < n; ++j) {
                // out-of-bounds
                if (j + grid[i][j] < 0 || j + grid[i][j] == n)
                    continue;
                // Stuck
                if (grid[i][j] == 1 && grid[i][j + 1] == -1 ||
                    grid[i][j] == -1 && grid[i][j - 1] == 1)
                    continue;
                newDp[j + grid[i][j]] = dp[j];
            }
            dp = std::move(newDp);
        }

        for (int i = 0; i < n; ++i)
            if (dp[i] != -1)
                ans[dp[i]] = i;

        return ans;
    }
};
```

776. 1707.cpp

Path: LeetCode-main/solutions/1707. Maximum XOR With an Element From Array/1707.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    TrieNode() : children(2) {}
};

class BitTrie {
public:
    BitTrie(int maxBit) : maxBit(maxBit) {}

    void insert(int num) {
        shared_ptr<TrieNode> node = root;
        for (int i = maxBit; i >= 0; --i) {
            const int bit = num >> i & 1;
            if (node->children[bit] == nullptr)
                node->children[bit] = make_shared<TrieNode>();
            node = node->children[bit];
        }
    }

    int getMaxXor(int num) {
        int maxXor = 0;
        shared_ptr<TrieNode> node = root;
        for (int i = maxBit; i >= 0; --i) {
            const int bit = num >> i & 1;
            const int toggleBit = bit ^ 1;
            if (node->children[toggleBit] != nullptr) {
                maxXor = maxXor | 1 << i;
                node = node->children[toggleBit];
            } else if (node->children[bit] != nullptr) {
                node = node->children[bit];
            } else { // There's nothing in the Bit Trie.
                return 0;
            }
        }
        return maxXor;
    }

private:
    const int maxBit;
    shared_ptr<TrieNode> root = make_shared<TrieNode>();
};

struct IndexedQuery {
    int queryIndex;
    int x;
    int m;
};

class Solution {
public:
    vector<int> maximizeXor(vector<int>& nums, vector<vector<int>>& queries) {
        vector<int> ans(queries.size(), -1);
        const int maxNumInNums = ranges::max(nums);
        const int maxNumInQuery = ranges::max_element(queries, ranges::less{},
                                                        [])(const vector<int>& query) {
            return query[0];
        }->at(0);
        const int maxBit = static_cast<int>(log2(max(maxNumInNums, maxNumInQuery)));
        BitTrie bitTrie(maxBit);

        ranges::sort(nums);

        int i = 0; // nums' index
        for (const auto& [queryIndex, x, m] : getIndexedQueries(queries)) {
            while (i < nums.size() && nums[i] <= m)
                bitTrie.insert(nums[i++]);
            if (i > 0 && nums[i - 1] <= m)
                ans[queryIndex] = bitTrie.getMaxXor(x);
        }

        return ans;
    }

private:
    vector<IndexedQuery> getIndexedQueries(const vector<vector<int>>& queries) {
        vector<IndexedQuery> indexedQueries;
        for (int i = 0; i < queries.size(); ++i)
            indexedQueries.emplace_back(i, queries[i][0], queries[i][1]);
        ranges::sort(
            indexedQueries, ranges::less{},

```

```
        [](const IndexedQuery& indexedQuery) { return indexedQuery.m; });  
    return indexedQueries;  
}  
};
```


777. 1708.cpp

Path: LeetCode-main/solutions/1708. Largest Subarray Length K/1708.cpp

```
class Solution {
public:
    vector<int> largestSubarray(vector<int>& nums, int k) {
        const auto it = max_element(nums.begin(), nums.end() - k + 1);
        return std::vector<int>(it, it + k);
    }
};
```

778. 171.cpp

Path: LeetCode-main/solutions/171. Excel Sheet Column Number/171.cpp

```
class Solution {
public:
    int titleToNumber(string columnTitle) {
        return accumulate(columnTitle.begin(), columnTitle.end(), 0,
            [](int acc, char c) { return acc * 26 + (c - 'A' + 1); });
    }
};
```

779. 1710.cpp

Path: LeetCode-main/solutions/1710. Maximum Units on a Truck/1710.cpp

```
class Solution {
public:
    int maximumUnits(vector<vector<int>>& boxTypes, int truckSize) {
        int ans = 0;

        ranges::sort(boxTypes, ranges::greater{},
            [](const vector<int>& boxType) { return boxType[1]; });

        for (const vector<int>& boxType : boxTypes) {
            const int boxes = boxType[0];
            const int units = boxType[1];
            if (boxes >= truckSize)
                return ans + truckSize * units;
            ans += boxes * units;
            truckSize -= boxes;
        }

        return ans;
    }
};
```

780. 1711.cpp

Path: LeetCode-main/solutions/1711. Count Good Meals/1711.cpp

```
class Solution {
public:
    int countPairs(vector<int>& deliciousness) {
        constexpr int kMod = 1'000'000'007;
        constexpr int kMaxBit = 20 + 1;
        const int kMaxPower = pow(2, kMaxBit);
        int ans = 0;
        unordered_map<int, int> count;

        for (const int d : deliciousness) {
            for (int power = 1; power <= kMaxPower; power *= 2)
                if (const auto it = count.find(power - d); it != count.cend()) {
                    ans += it->second;
                    ans %= kMod;
                }
            ++count[d];
        }

        return ans;
    }
};
```

781. 1712-2.cpp

Path: LeetCode-main/solutions/1712. Ways to Split Array Into Three Subarrays/1712-2.cpp

```
class Solution {
public:
    int waysToSplit(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        int ans = 0;
        vector<int> prefix(n);

        partial_sum(nums.begin(), nums.end(), prefix.begin());

        for (int i = 0, j = 0, k = 0; i < n - 2; ++i) {
            // Find the first index j s.t.
            // left = prefix[i] <= mid = prefix[j] - prefix[i]
            j = max(j, i + 1);
            while (j < n - 1 && prefix[i] > prefix[j] - prefix[i])
                ++j;
            // Find the first index k s.t.
            // mid = prefix[k] - prefix[i] > right = prefix[-1] - prefix[k]
            k = max(k, j);
            while (k < n - 1 && prefix[k] - prefix[i] <= prefix.back() - prefix[k])
                ++k;
            ans += k - j;
            ans %= kMod;
        }

        return ans;
    }
};
```

782. 1712.cpp

Path: LeetCode-main/solutions/1712. Ways to Split Array Into Three Subarrays/1712.cpp

```
class Solution {
public:
    int waysToSplit(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        int ans = 0;
        vector<int> prefix(n);

        partial_sum(nums.begin(), nums.end(), prefix.begin());

        // Finds the first index j s.t.
        // mid = prefix[j] - prefix[i] >= left = prefix[i].
        auto firstGreaterEqual = [&](int i) {
            int l = i + 1;
            int r = n - 1;
            while (l < r) {
                const int m = (l + r) / 2;
                if (prefix[m] - prefix[i] >= prefix[i])
                    r = m;
                else
                    l = m + 1;
            }
            return l;
        };

        // Finds the first index k s.t.
        // mid = prefix[k] - prefix[i] > right = prefix[-1] - prefix[k].
        auto firstGreater = [&](int i) {
            int l = i + 1;
            int r = n - 1;
            while (l < r) {
                const int m = (l + r) / 2;
                if (prefix[m] - prefix[i] > prefix.back() - prefix[m])
                    r = m;
                else
                    l = m + 1;
            }
            return l;
        };

        for (int i = 0; i < n - 2; ++i) {
            const int j = firstGreaterEqual(i);
            if (j == n - 1)
                break;
            const int mid = prefix[j] - prefix[i];
            const int right = prefix.back() - prefix[j];
            if (mid > right)
                continue;
            const int k = firstGreater(i);
            ans = (ans + k - j) % kMod;
        }

        return ans;
    }
};
```

783. 1713.cpp

Path: LeetCode-main/solutions/1713. Minimum Operations to Make a Subsequence/1713.cpp

```
class Solution {
public:
    int minOperations(vector<int>& target, vector<int>& arr) {
        vector<int> indices;
        unordered_map<int, int> numToIndex;

        for (int i = 0; i < target.size(); ++i)
            numToIndex[target[i]] = i;

        for (const int a : arr)
            if (const auto it = numToIndex.find(a); it != numToIndex.end())
                indices.push_back(it->second);

        return target.size() - lengthOfLIS(indices);
    }

private:
    // Same as 300. Longest Increasing Subsequence
    int lengthOfLIS(vector<int>& nums) {
        // tails[i] := the minimum tail of all the increasing subsequences having
        // length i + 1
        vector<int> tails;
        for (const int num : nums)
            if (tails.empty() || num > tails.back())
                tails.push_back(num);
            else
                tails[firstGreaterEqual(tails, num)] = num;
        return tails.size();
    }

private:
    int firstGreaterEqual(const vector<int>& arr, int target) {
        return ranges::lower_bound(arr, target) - arr.begin();
    }
};
```

784. 1714.cpp

Path: LeetCode-main/solutions/1714. Sum Of Special Evenly-Spaced Elements In Array/1714.cpp

```
class Solution {
public:
    vector<int> solve(vector<int>& nums, vector<vector<int>>& queries) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        const int sqrtN = static_cast<int>(sqrt(n));
        vector<int> ans;
        // prefix[x][y] = sum(nums[x + ay]), where a >= 0 and x + ay < n
        vector<vector<int>> prefix(n, vector<int>(sqrtN));

        // Set prefix[i][j] to nums[i] to indicate the sequence starts with nums[i].
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < sqrtN; ++j)
                prefix[i][j] = nums[i];

        for (int x = n - 1; x >= 0; --x)
            for (int y = 1; y < sqrtN; ++y)
                if (x + y < n) {
                    prefix[x][y] += prefix[x + y][y];
                    prefix[x][y] %= kMod;
                }

        for (const vector<int>& query : queries) {
            const int x = query[0];
            const int y = query[1];
            if (y < sqrtN) {
                ans.push_back(prefix[x][y]);
            } else {
                int sum = 0;
                for (int i = x; i < n; i += y)
                    sum = (sum + nums[i]) % kMod;
                ans.push_back(sum);
            }
        }

        return ans;
    }
};
```


785. 1716.cpp

Path: LeetCode-main/solutions/1716. Calculate Money in Leetcode Bank/1716.cpp

```
class Solution {
public:
    int totalMoney(int n) {
        const int weeks = n / 7;
        const int firstWeek = trapezoid(1, 7);
        const int lastFullWeek = trapezoid(1 + weeks - 1, 7 + weeks - 1);
        const int remainingDays = trapezoid(1 + weeks, n % 7 + weeks);
        return (firstWeek + lastFullWeek) * weeks / 2 + remainingDays;
    }

    // Returns sum(a..b).
    int trapezoid(int a, int b) {
        return (a + b) * (b - a + 1) / 2;
    }
};
```

786. 1717.cpp

Path: LeetCode-main/solutions/1717. Maximum Score From Removing Substrings/1717.cpp

```
class Solution {
public:
    int maximumGain(string s, int x, int y) {
        // The assumption that gain("ab") > gain("ba") while removing "ba" first is
        // optimal is contradicted. Only "b(ab)a" satisfies the condition of
        // preventing two "ba" removals, but after removing "ab", we can still
        // remove one "ba", resulting in a higher gain. Thus, removing "ba" first is
        // not optimal.
        return x > y ? gain(s, "ab", x, "ba", y) : gain(s, "ba", y, "ab", x);
    }

private:
    // Returns the points gained by first removing sub1 ("ab" | "ba") from s with
    // point1, then removing sub2 ("ab" | "ba") from s with point2.
    int gain(const string& s, const string& sub1, int point1, const string& sub2,
             int point2) {
        int points = 0;
        vector<char> stack1;
        vector<char> stack2;

        // Remove "sub1" from s with point1 gain.
        for (const char c : s)
            if (!stack1.empty() && stack1.back() == sub1[0] && c == sub1[1]) {
                stack1.pop_back();
                points += point1;
            } else {
                stack1.push_back(c);
            }

        // Remove "sub2" from s with point2 gain.
        for (const char c : stack1)
            if (!stack2.empty() && stack2.back() == sub2[0] && c == sub2[1]) {
                stack2.pop_back();
                points += point2;
            } else {
                stack2.push_back(c);
            }

        return points;
    }
};
```

787. 1718.cpp

Path: LeetCode-main/solutions/1718. Construct the Lexicographically Largest Valid Sequence/1718.cpp

```
class Solution {
public:
    vector<int> constructDistancedSequence(int n) {
        vector<int> ans(2 * n - 1);
        dfs(n, 0, 0, ans);
        return ans;
    }

private:
    bool dfs(int n, int i, int mask, vector<int>& ans) {
        if (i == ans.size())
            return true;
        if (ans[i] > 0)
            return dfs(n, i + 1, mask, ans);

        // Greedily fill in `ans` in descending order.
        for (int num = n; num >= 1; --num) {
            if (mask >> num & 1)
                continue;
            if (num == 1) {
                ans[i] = num;
                if (dfs(n, i + 1, mask | 1 << num, ans))
                    return true;
                ans[i] = 0;
            } else { // num in [2, n]
                if (i + num >= ans.size() || ans[i + num] > 0)
                    continue;
                ans[i] = num;
                ans[i + num] = num;
                if (dfs(n, i + 1, mask | 1 << num, ans))
                    return true;
                ans[i + num] = 0;
                ans[i] = 0;
            }
        }
        return false;
    }
};
```

788. 1719.cpp

Path: LeetCode-main/solutions/1719. Number Of Ways To Reconstruct A Tree/1719.cpp

```
class Solution {
public:
    int checkWays(vector<vector<int>>& pairs) {
        constexpr int kMax = 501;
        unordered_map<int, vector<int>> graph;
        vector<int> degrees(kMax);
        vector<vector<bool>> connected(kMax, vector<bool>(kMax));

        for (const vector<int>& pair : pairs) {
            const int u = pair[0];
            const int v = pair[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
            ++degrees[u];
            ++degrees[v];
            connected[u][v] = true;
            connected[v][u] = true;
        }

        // For each node, sort its children by degrees in descending order.
        for (auto& [_, children] : graph)
            ranges::sort(children, ranges::greater{},
                [&degrees](int child) { return degrees[child]; });

        const int root = getRoot(degrees, graph.size());
        if (root == -1)
            return 0;
        if (!dfs(graph, root, degrees, connected, {}, vector<bool>(kMax)))
            return 0;
        return hasMoreThanOneWay ? 2 : 1;
    }

private:
    bool hasMoreThanOneWay = false;

    // Returns the root by finding the node with a degree that equals to n - 1.
    int getRoot(const vector<int>& degrees, int n) {
        for (int i = 1; i < degrees.size(); ++i)
            if (degrees[i] == n - 1)
                return i;
        return -1;
    }

    // Returns true if each node rooted at u is connected to all of its ancestors.
    bool dfs(const unordered_map<int, vector<int>>& graph, int u,
        vector<int>& degrees, vector<vector<bool>>& connected,
        vector<int>&& ancestors, vector<bool>&& seen) {
        seen[u] = true;

        for (const int ancestor : ancestors)
            if (!connected[u][ancestor])
                return false;

        ancestors.push_back(u);

        for (const int v : graph.at(u)) {
            if (seen[v])
                continue;
            // We can swap u with v, so there are more than one way.
            if (degrees[v] == degrees[u])
                hasMoreThanOneWay = true;
            if (!dfs(graph, v, degrees, connected, std::move(ancestors),
                std::move(seen)))
                return false;
        }

        ancestors.pop_back();
        return true;
    }
};
```

789. 172.cpp

Path: LeetCode-main/solutions/172. Factorial Trailing Zeroes/172.cpp

```
class Solution {  
public:  
    int trailingZeroes(int n) {  
        return n == 0 ? 0 : n / 5 + trailingZeroes(n / 5);  
    }  
};
```

790. 1720.cpp

Path: LeetCode-main/solutions/1720. Decode XORed Array/1720.cpp

```
class Solution {
public:
    vector<int> decode(vector<int>& encoded, int first) {
        vector<int> ans(encoded.size() + 1);
        ans[0] = first;

        for (int i = 0; i < encoded.size(); ++i)
            ans[i + 1] = ans[i] ^ encoded[i];

        return ans;
    }
};
```

791. 1721.cpp

Path: LeetCode-main/solutions/1721. Swapping Nodes in a Linked List/1721.cpp

```
class Solution {
public:
    ListNode* swapNodes(ListNode* head, int k) {
        ListNode* p = nullptr; // Points the k-th node from the beginning.
        ListNode* q = nullptr; // Points the k-th node from the end.

        for (ListNode* curr = head; curr != nullptr; curr = curr->next) {
            if (q != nullptr)
                q = q->next;
            if (--k == 0) {
                p = curr;
                q = head;
            }
        }

        swap(p->val, q->val);
        return head;
    }
};
```

792. 1722.cpp

Path: LeetCode-main/solutions/1722. Minimize Hamming Distance After Swap Operations/1722.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    int minimumHammingDistance(vector<int>& source, vector<int>& target,
                               vector<vector<int>>& allowedSwaps) {
        const int n = source.size();
        int ans = 0;
        UnionFind uf(n);
        vector<unordered_map<int, int>> groupIdToCount(n);

        for (const vector<int>& allowedSwap : allowedSwaps) {
            const int a = allowedSwap[0];
            const int b = allowedSwap[1];
            uf.unionByRank(a, b);
        }

        for (int i = 0; i < n; ++i)
            ++groupIdToCount[uf.find(i)][source[i]];

        for (int i = 0; i < n; ++i) {
            const int groupId = uf.find(i);
            unordered_map<int, int>& count = groupIdToCount[groupId];
            if (!count.contains(target[i]))
                ++ans;
            else if (--count[target[i]] == 0)
                count.erase(target[i]);
        }

        return ans;
    }
};
```


793. 1723.cpp

Path: LeetCode-main/solutions/1723. Find Minimum Time to Finish All Jobs/1723.cpp

```
class Solution {
public:
    int minimumTimeRequired(std::vector<int>& jobs, int k) {
        int ans = accumulate(jobs.begin(), jobs.end(), 0);
        vector<int> times(k);

        ranges::sort(jobs, greater<>());
        dfs(jobs, 0, times, ans);
        return ans;
    }

private:
    void dfs(const vector<int>& jobs, int s, vector<int>& times, int& ans) {
        if (s == jobs.size()) {
            ans = min(ans, ranges::max(times));
            return;
        }
        for (int i = 0; i < times.size(); ++i) {
            if (times[i] + jobs[s] >= ans)
                continue;
            times[i] += jobs[s];
            dfs(jobs, s + 1, times, ans);
            times[i] -= jobs[s];
            if (times[i] == 0)
                return;
        }
    };
};
```

794. 1724.cpp

Path: LeetCode-main/solutions/1724. Checking Existence of Edge Length Limited Paths II/1724.cpp

```
class UnionFind {
public:
    UnionFind() {}
    UnionFind(int n) {
        id.resize(n);

        for (int i = 0; i < n; ++i)
            id[i][0] = i;
    }

    void union_(int u, int v, int limit) {
        const int i = find(u, limit);
        const int j = find(v, limit);
        if (i == j)
            return;
        id[i][limit] = j;
    }

    int find(int u, int limit) {
        // the minimum iterator of id[u] > limit
        const auto it = id[u].upper_bound(limit);
        const int i = prev(it)->second;
        if (i == u)
            return u;
        // Recursively find i's id.
        const int j = find(i, limit);
        id[u][limit] = j;
        return j;
    }

private:
    // id[i]'s (key, value) := (limit, id of node i <= limit)
    vector<map<int, int>> id;
};

class DistanceLimitedPathsExist {
public:
    DistanceLimitedPathsExist(int n, vector<vector<int>>& edgeList) {
        uf = UnionFind(n);

        ranges::sort(edgeList, ranges::less{},
            [](const vector<int>& edge) { return edge[2]; });

        for (const vector<int>& edge : edgeList) {
            const int u = edge[0];
            const int v = edge[1];
            const int d = edge[2];
            uf.union_(u, v, d);
        }

        bool query(int p, int q, int limit) {
            return uf.find(p, limit - 1) == uf.find(q, limit - 1);
        }

private:
    UnionFind uf;
};
```

795. 1725.cpp

Path: LeetCode-main/solutions/1725. Number Of Rectangles That Can Form The Largest Square/1725.cpp

```
class Solution {
public:
    int countGoodRectangles(vector<vector<int>>& rectangles) {
        vector<int> minSides;

        for (const vector<int>& rectangle : rectangles) {
            const int x = rectangle[0];
            const int y = rectangle[1];
            minSides.push_back(min(x, y));
        }

        const int maxLen = ranges::max(minSides);
        return ranges::count(minSides, maxLen);
    }
};
```

796. 1726.cpp

Path: LeetCode-main/solutions/1726. Tuple with Same Product/1726.cpp

```
class Solution {
public:
    int tupleSameProduct(vector<int>& nums) {
        int ans = 0;
        unordered_map<int, int> count;

        for (int i = 0; i < nums.size(); ++i)
            for (int j = 0; j < i; ++j)
                ans += count[nums[i] * nums[j]]++ * 8;

        return ans;
    }
};
```

797. 1727.cpp

Path: LeetCode-main/solutions/1727. Largest Submatrix With Rearrangements/1727.cpp

```
class Solution {
public:
    int largestSubmatrix(vector<vector<int>>& matrix) {
        const int n = matrix[0].size();
        int ans = 0;
        vector<int> hist(n);

        for (const vector<int>& row : matrix) {
            // Accumulate the histogram if possible.
            for (int i = 0; i < n; ++i)
                hist[i] = row[i] == 0 ? 0 : hist[i] + 1;

            // Get the sorted histogram.
            vector<int> sortedHist(hist);
            ranges::sort(sortedHist);

            // Greedily calculate the answer.
            for (int i = 0; i < n; ++i)
                ans = max(ans, sortedHist[i] * (n - i));
        }

        return ans;
    }
};
```

798. 1728.cpp

Path: LeetCode-main/solutions/1728. Cat and Mouse II/1728.cpp

```
class Solution {
public:
    bool canMouseWin(vector<string>& grid, int catJump, int mouseJump) {
        const int m = grid.size();
        const int n = grid[0].size();
        int nFloors = 0;
        int cat;    // cat's position
        int mouse;  // mouse's position

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (grid[i][j] != '#')
                    ++nFloors;
                if (grid[i][j] == 'C')
                    cat = hash(i, j, n);
                else if (grid[i][j] == 'M')
                    mouse = hash(i, j, n);
            }

        vector<vector<vector<int>>> mem(
            m * n, vector<vector<int>>(m * n, vector<int>(nFloors * 2, -1)));
        return canMouseWin(grid, cat, mouse, 0, catJump, mouseJump, m, n, nFloors,
            mem);
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns true if the mouse can win, where the cat is on (i / 8, i % 8), the
    // mouse is on (j / 8, j % 8), and the turns is k.
    bool canMouseWin(const vector<string>& grid, int cat, int mouse, int turn,
        const int& catJump, const int& mouseJump, const int& m,
        const int& n, const int& nFloors,
        vector<vector<vector<int>>>& mem) {
        // We already search the whole touchable grid.
        if (turn == nFloors * 2)
            return false;
        if (mem[cat][mouse][turn] != -1)
            return mem[cat][mouse][turn];

        if (turn % 2 == 0) {
            // the mouse's turn
            const int i = mouse / n;
            const int j = mouse % n;
            for (const auto& [dx, dy] : kDirs) {
                for (int jump = 0; jump <= mouseJump; ++jump) {
                    const int x = i + dx * jump;
                    const int y = j + dy * jump;
                    if (x < 0 || x == m || y < 0 || y == n)
                        break;
                    if (grid[x][y] == '#')
                        break;
                    // The mouse eats the food, so the mouse wins.
                    if (grid[x][y] == 'F')
                        return mem[cat][mouse][turn] = true;
                    if (canMouseWin(grid, cat, hash(x, y, n), turn + 1, catJump,
                        mouseJump, m, n, nFloors, mem))
                        return mem[cat][mouse][turn] = true;
                }
            }
        }
        // The mouse can't win, so the mouse loses.
        return mem[cat][mouse][turn] = false;
    } else {
        // the cat's turn
        const int i = cat / n;
        const int j = cat % n;
        for (const auto& [dx, dy] : kDirs) {
            for (int jump = 0; jump <= catJump; ++jump) {
                const int x = i + dx * jump;
                const int y = j + dy * jump;
                if (x < 0 || x == m || y < 0 || y == n)
                    break;
                if (grid[x][y] == '#')
                    break;
                // The cat eats the food, so the mouse loses.
                if (grid[x][y] == 'F')
                    return mem[cat][mouse][turn] = false;
                const int nextCat = hash(x, y, n);
                // The cat catches the mouse, so the mouse loses.
                if (nextCat == mouse)
                    return mem[cat][mouse][turn] = false;
            }
        }
    }
}
```

```

        return mem[cat][mouse][turn] = false;
    if (!canMouseWin(grid, nextCat, mouse, turn + 1, catJump, mouseJump,
        m, n, nFloors, mem))
        return mem[cat][mouse][turn] = false;
    }
}
// The cat can't win, so the mouse wins.
return mem[cat][mouse][turn] = true;
}
}

int hash(int i, int j, int n) {
    return i * n + j;
}
};

```

799. 173-2.cpp

Path: LeetCode-main/solutions/173. Binary Search Tree Iterator/173-2.cpp

```
class BSTIterator {
public:
    BSTIterator(TreeNode* root) {
        pushLeftsUntilNull(root);
    }

    int next() {
        TreeNode* root = stack.top();
        stack.pop();
        pushLeftsUntilNull(root->right);
        return root->val;
    }

    bool hasNext() {
        return !stack.empty();
    }

private:
    stack<TreeNode*> stack;

    void pushLeftsUntilNull(TreeNode* root) {
        while (root != nullptr) {
            stack.push(root);
            root = root->left;
        }
    }
};
```


800. 173.cpp

Path: LeetCode-main/solutions/173. Binary Search Tree Iterator/173.cpp

```
class BSTIterator {
public:
    BSTIterator(TreeNode* root) {
        inorder(root);
    }

    int next() {
        return vals[i++];
    }

    bool hasNext() {
        return i < vals.size();
    }

private:
    int i = 0;
    vector<int> vals;

    void inorder(TreeNode* root) {
        if (root == nullptr)
            return;
        inorder(root->left);
        vals.push_back(root->val);
        inorder(root->right);
    }
};
```

801. 1730.cpp

Path: LeetCode-main/solutions/1730. Shortest Path to Get Food/1730.cpp

```

class Solution {
public:
    int getFood(vector<vector<char>>& grid) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = grid.size();
        const int n = grid[0].size();
        queue<pair<int, int>> q{{getStartLocation(grid)}};

        for (int ans = 0; !q.empty(); ++ans)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (grid[x][y] == 'X')
                        continue;
                    if (grid[x][y] == '#')
                        return ans + 1;
                    q.emplace(x, y);
                    grid[x][y] = 'X'; // Mark as visited.
                }
            }

        return -1;
    }

private:
    pair<int, int> getStartLocation(const vector<vector<char>>& grid) {
        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] == '*')
                    return {i, j};
        throw;
    }
};

```

802. 1732.cpp

Path: LeetCode-main/solutions/1732. Find the Highest Altitude/1732.cpp

```
class Solution {
public:
    int largestAltitude(vector<int>& gain) {
        int ans = 0;
        int currAltitude = 0;
        for (const int g : gain) {
            currAltitude += g;
            ans = max(ans, currAltitude);
        }
        return ans;
    }
};
```

803. 1733.cpp

Path: LeetCode-main/solutions/1733. Minimum Number of People to Teach/1733.cpp

```
class Solution {
public:
    int minimumTeachings(int n, vector<vector<int>>& languages,
                        vector<vector<int>>& friendships) {
        vector<unordered_set<int>> languageSets;
        unordered_set<int> needTeach;
        unordered_map<int, int> languageCount;

        for (const vector<int>& language : languages)
            languageSets.push_back({language.begin(), language.end()});

        // Find friends that can't communicate.
        for (const vector<int>& friendship : friendships) {
            const int u = friendship[0] - 1;
            const int v = friendship[1] - 1;
            if (cantTalk(languageSets, u, v)) {
                needTeach.insert(u);
                needTeach.insert(v);
            }
        }

        // Find the most popular language.
        for (const int u : needTeach)
            for (const int language : languageSets[u])
                ++languageCount[language];

        // Teach the most popular language to people who don't understand.
        int maxCount = 0;
        for (const auto& [_, freq] : languageCount)
            maxCount = max(maxCount, freq);

        return needTeach.size() - maxCount;
    }

private:
    // Returns true if u can't talk with v.
    bool cantTalk(const vector<unordered_set<int>>& languageSets, int u, int v) {
        for (const int language : languageSets[u])
            if (languageSets[v].contains(language))
                return false;
        return true;
    }
};
```

804. 1734.cpp

Path: LeetCode-main/solutions/1734. Decode XORed Permutation/1734.cpp

```
class Solution {
public:
    vector<int> decode(vector<int>& encoded) {
        // Our goal is to find the value of a1, which will allow us to decode a2,
        // a3, ..., an. This can be achieved by performing XOR operation between
        // each element in `encoded` and a1.
        //
        // e.g. n = 3, perm = [a1, a2, a3] is a permutation of [1, 2, 3]
        //      encoded = [a1^a2, a2^a3]
        //      accumulatedEncoded = [a1^a2, a1^a3]
        //      a1 = (a1^a2)^(a1^a3)^(a1^a2^a3)
        //      a2 = a1^(a1^a2)
        //      a3 = a2^(a2^a3)
        const int n = encoded.size() + 1;
        int nXors = 0;
        for (int i = 1; i <= n; i++)
            nXors ^= i;

        // Instead of constructing the array, we can track of the running XOR value
        // of `accumulatedEncoded`.
        int runningXors = 0;
        int xors = 0; // xors(accumulatedEncoded)

        for (const int encode : encoded) {
            runningXors ^= encode;
            xors ^= runningXors;
        }

        vector<int> ans{xors ^ nXors};

        for (const int encode : encoded)
            ans.push_back(ans.back() ^ encode);

        return ans;
    }
};
```

805. 1735.cpp

Path: LeetCode-main/solutions/1735. Count Ways to Make Array With Product/1735.cpp

```
class Solution {
public:
    vector<int> waysToFillArray(vector<vector<int>>& queries) {
        constexpr int kMax = 10000;
        constexpr int kMaxFreq = 13; // 2^13 = 8192 < kMax
        const vector<int> minPrimeFactors = sieveEratosthenes(kMax + 1);
        const auto [fact, invFact] = getFactAndInvFact(kMax + kMaxFreq - 1);
        vector<int> ans;

        for (const vector<int>& query : queries) {
            const int n = query[0];
            const int k = query[1];
            int res = 1;
            for (const auto& [_, freq] : getPrimeFactorsCount(k, minPrimeFactors))
                res = static_cast<long>(res) * nCk(n - 1 + freq, freq, fact, invFact) %
                    kMod;
            ans.push_back(res);
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Gets the minimum prime factor of i, where 1 < i <= n.
    vector<int> sieveEratosthenes(int n) {
        vector<int> minPrimeFactors(n + 1);
        iota(minPrimeFactors.begin() + 2, minPrimeFactors.end(), 2);
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    unordered_map<int, int> getPrimeFactorsCount(
        int num, const vector<int>& minPrimeFactors) {
        unordered_map<int, int> count;
        while (num > 1) {
            const int divisor = minPrimeFactors[num];
            while (num % divisor == 0) {
                num /= divisor;
                ++count[divisor];
            }
        }
        return count;
    }

    pair<vector<long>, vector<long>> getFactAndInvFact(int n) {
        vector<long> fact(n + 1);
        vector<long> invFact(n + 1);
        vector<long> inv(n + 1);
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = kMod - kMod / i * inv[kMod % i] % kMod;
            fact[i] = fact[i - 1] * i % kMod;
            invFact[i] = invFact[i - 1] * inv[i] % kMod;
        }
        return {fact, invFact};
    }

    int nCk(int n, int k, const vector<long>& fact, const vector<long>& invFact) {
        return fact[n] * invFact[k] % kMod * invFact[n - k] % kMod;
    }
};
```

806. 1736.cpp

Path: LeetCode-main/solutions/1736. Latest Time by Replacing Hidden Digits/1736.cpp

```
class Solution {
public:
    string maximumTime(string time) {
        string ans = time;
        if (time[0] == '?')
            ans[0] = time[1] == '?' || time[1] < '4' ? '2' : '1';
        if (time[1] == '?')
            ans[1] = ans[0] == '2' ? '3' : '9';
        if (time[3] == '?')
            ans[3] = '5';
        if (time[4] == '?')
            ans[4] = '9';
        return ans;
    }
};
```

807. 1737.cpp

Path: LeetCode-main/solutions/1737. Change Minimum Characters to Satisfy One of Three Conditions/1737.cpp

```
class Solution {
public:
    int minCharacters(string a, string b) {
        const int m = a.length();
        const int n = b.length();
        vector<int> countA(26);
        vector<int> countB(26);

        for (const char c : a)
            ++countA[c - 'a'];

        for (const char c : b)
            ++countB[c - 'a'];

        int ans = INT_MAX;
        int prevA = 0; // the number of characters in a <= c
        int prevB = 0; // the number of characters in b <= c

        for (char c = 'a'; c <= 'z'; ++c) {
            // the condition 3
            ans = min(ans, m + n - countA[c - 'a'] - countB[c - 'a']);
            // the conditions 1 and 2
            if (c > 'a')
                ans = min({ans, m - prevA + prevB, n - prevB + prevA});
            prevA += countA[c - 'a'];
            prevB += countB[c - 'a'];
        }

        return ans;
    }
};
```


808. 1738.cpp

Path: LeetCode-main/solutions/1738. Find Kth Largest XOR Coordinate Value/1738.cpp

```
class Solution {
public:
    int kthLargestValue(vector<vector<int>>& matrix, int k) {
        const int m = matrix.size();
        const int n = matrix[0].size();
        vector<vector<int>> xors(m + 1, vector<int>(n + 1));
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j) {
                xors[i][j] = xors[i - 1][j] ^ xors[i][j - 1] ^ xors[i - 1][j - 1] ^
                    matrix[i - 1][j - 1];
                minHeap.push(xors[i][j]);
                if (minHeap.size() > k)
                    minHeap.pop();
            }

        return minHeap.top();
    }
};
```

809. 1739.cpp

Path: LeetCode-main/solutions/1739. Building Boxes/1739.cpp

```
class Solution {
public:
    int minimumBoxes(int n) {
        int nBoxes = 0;
        int nextTouchings = 0;    // j
        int currLevelBoxes = 0;    // 1 + 2 + ... + j

        // Find the minimum j s.t. `nBoxes` = 1 + (1 + 2) + ... + (1 + 2 + ... + j)
        // >= n.
        while (nBoxes < n) {
            ++nextTouchings;
            currLevelBoxes += nextTouchings;
            nBoxes += currLevelBoxes;
        }

        // If nBoxes = n, the answer is `currLevelBoxes` = 1 + 2 + ... + j.
        if (nBoxes == n)
            return currLevelBoxes;

        // Otherwise, need to remove the boxes in the current level and rebuild it.
        nBoxes -= currLevelBoxes;
        currLevelBoxes -= nextTouchings;
        nextTouchings = 0;

        while (nBoxes < n) {
            ++nextTouchings;
            nBoxes += nextTouchings;
        }

        return currLevelBoxes + nextTouchings;
    }
};
```

810. 174.cpp

Path: LeetCode-main/solutions/174. Dungeon Game/174.cpp

```
class Solution {
public:
    int calculateMinimumHP(vector<vector<int>>& dungeon) {
        const int m = dungeon.size();
        const int n = dungeon[0].size();
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, INT_MAX));
        dp[m][n - 1] = 1;
        dp[m - 1][n] = 1;

        for (int i = m - 1; i >= 0; --i)
            for (int j = n - 1; j >= 0; --j) {
                dp[i][j] = min(dp[i + 1][j], dp[i][j + 1]) - dungeon[i][j];
                dp[i][j] = max(dp[i][j], 1);
            }

        return dp[0][0];
    }
};
```

811. 1740.cpp

Path: LeetCode-main/solutions/1740. Find Distance in a Binary Tree/1740.cpp

```
class Solution {
public:
    int findDistance(TreeNode* root, int p, int q) {
        TreeNode* lca = getLCA(root, p, q);
        return dist(lca, p) + dist(lca, q);
    }

private:
    TreeNode* getLCA(TreeNode* root, int p, int q) {
        if (root == nullptr || root->val == p || root->val == q)
            return root;
        TreeNode* left = getLCA(root->left, p, q);
        TreeNode* right = getLCA(root->right, p, q);
        if (left != nullptr && right != nullptr)
            return root;
        return left == nullptr ? right : left;
    }

    int dist(TreeNode* lca, int target) {
        if (lca == nullptr)
            return 10000;
        if (lca->val == target)
            return 0;
        return 1 + min(dist(lca->left, target), dist(lca->right, target));
    }
};
```

812. 1742.cpp

Path: LeetCode-main/solutions/1742. Maximum Number of Balls in a Box/1742.cpp

```
class Solution {
public:
    int countBalls(int lowLimit, int highLimit) {
        const int maxDigitSum = 9 * 5; // 99999
        int ans = 0;
        vector<int> count(maxDigitSum + 1);

        for (int num = lowLimit; num <= highLimit; ++num)
            ans = max(ans, ++count[getDigitSum(num)]);

        return ans;
    }

private:
    int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
};
```

813. 1743.cpp

Path: LeetCode-main/solutions/1743. Restore the Array From Adjacent Pairs/1743.cpp

```
class Solution {
public:
    vector<int> restoreArray(vector<vector<int>>& adjacentPairs) {
        vector<int> ans;
        unordered_map<int, vector<int>> numToAdjs;

        for (const vector<int>& pair : adjacentPairs) {
            const int u = pair[0];
            const int v = pair[1];
            numToAdjs[u].push_back(v);
            numToAdjs[v].push_back(u);
        }

        for (const auto& [num, adjs] : numToAdjs)
            if (adjs.size() == 1) {
                ans.push_back(num);
                ans.push_back(adjs[0]);
                break;
            }

        while (ans.size() < adjacentPairs.size() + 1) {
            const int tail = ans.back();
            const int prev = ans[ans.size() - 2];
            const vector<int>& adjs = numToAdjs[tail];
            if (adjs[0] == prev)
                ans.push_back(adjs[1]);
            else
                ans.push_back(adjs[0]);
        }

        return ans;
    }
};
```

814. 1744.cpp

Path: LeetCode-main/solutions/1744. Can You Eat Your Favorite Candy on Your Favorite Day?/1744.cpp

```
class Solution {
public:
    vector<bool> canEat(vector<int>& candiesCount, vector<vector<int>>& queries) {
        const int n = candiesCount.size();
        vector<bool> ans;
        vector<long> prefix{0};

        for (int i = 0; i < n; ++i)
            prefix.push_back(prefix.back() + candiesCount[i]);

        for (const vector<int>& query : queries) {
            const int type = query[0];
            const int day = query[1];
            const int cap = query[2];
            // the minimum day required to eat
            const long minDay = prefix[type] / cap;
            // the maximum day required to eat
            const long maxDay = prefix[type + 1] - 1;
            ans.push_back(minDay <= day && day <= maxDay);
        }

        return ans;
    }
};
```

815. 1745-2.cpp

Path: LeetCode-main/solutions/1745. Palindrome Partitioning IV/1745-2.cpp

```
class Solution {
public:
    bool checkPartitioning(string s) {
        const int n = s.length();
        // dp[i][j] := true if s[i..j] is a palindrome
        vector<vector<bool>> dp(n + 1, vector<bool>(n + 1));

        for (int i = 0; i < n; ++i)
            dp[i][i] = true;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                if (s[i] == s[j])
                    dp[i][j] = i + 1 > j - 1 || dp[i + 1][j - 1];
            }

        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                if (dp[0][i] && dp[i + 1][j] && dp[j + 1][n - 1])
                    return true;

        return false;
    }
};
```


816. 1745.cpp

Path: LeetCode-main/solutions/1745. Palindrome Partitioning IV/1745.cpp

```
class Solution {
public:
    bool checkPartitioning(string s) {
        const int n = s.length();
        vector<vector<int>> mem(n, vector<int>(n, -1));

        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                if (isPalindrome(s, 0, i, mem) && //
                    isPalindrome(s, i + 1, j, mem) && //
                    isPalindrome(s, j + 1, n - 1, mem))
                    return true;

        return false;
    }

private:
    // Returns true if s[i..j] is a palindrome.
    // Returns false if s[i..j] is not a palindrome.
    bool isPalindrome(const string& s, int i, int j, vector<vector<int>>& mem) {
        if (i > j)
            return true;
        if (mem[i][j] != -1)
            return mem[i][j];
        if (s[i] == s[j])
            return mem[i][j] = isPalindrome(s, i + 1, j - 1, mem);
        return mem[i][j] = false;
    }
};
```

817. 1746.cpp

Path: LeetCode-main/solutions/1746. Maximum Subarray Sum After One Operation/1746.cpp

```
class Solution {
public:
    int maxSumAfterOperation(vector<int>& nums) {
        int ans = INT_MIN;
        int regular = 0;
        int squared = 0;

        for (const int num : nums) {
            squared = max({num * num, regular + num * num, squared + num});
            regular = max(num, regular + num);
            ans = max(ans, squared);
        }

        return ans;
    }
};
```

818. 1748.cpp

Path: LeetCode-main/solutions/1748. Sum of Unique Elements/1748.cpp

```
class Solution {
public:
    int sumOfUnique(vector<int>& nums) {
        constexpr int kMax = 100;
        int ans = 0;
        vector<int> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        for (int i = 1; i <= kMax; ++i)
            if (count[i] == 1)
                ans += i;

        return ans;
    }
};
```

819. 1749-2.cpp

Path: LeetCode-main/solutions/1749. Maximum Absolute Sum of Any Subarray/1749-2.cpp

```
class Solution {
public:
    int maxAbsoluteSum(vector<int>& nums) {
        int sum = 0;
        int maxPrefix = 0;
        int minPrefix = 0;

        for (const int num : nums) {
            sum += num;
            maxPrefix = max(maxPrefix, sum);
            minPrefix = min(minPrefix, sum);
        }

        return maxPrefix - minPrefix;
    }
};
```

820. 1749.cpp

Path: LeetCode-main/solutions/1749. Maximum Absolute Sum of Any Subarray/1749.cpp

```
class Solution {
public:
    int maxAbsoluteSum(vector<int>& nums) {
        int ans = INT_MIN;
        int maxSum = 0;
        int minSum = 0;

        for (const int num : nums) {
            maxSum = max(num, maxSum + num);
            minSum = min(num, minSum + num);
            ans = max({ans, maxSum, -minSum});
        }

        return ans;
    }
};
```

821. 1750.cpp

Path: LeetCode-main/solutions/1750. Minimum Length of String After Deleting Similar Ends/1750.cpp

```
class Solution {
public:
    int minimumLength(string s) {
        int i = 0;
        int j = s.length() - 1;

        while (i < j && s[i] == s[j]) {
            const char c = s[i];
            while (i <= j && s[i] == c)
                ++i;
            while (i <= j && s[j] == c)
                --j;
        }

        return j - i + 1;
    }
};
```

822. 1751.cpp

Path: LeetCode-main/solutions/1751. Maximum Number of Events That Can Be Attended II/1751.cpp

```
class Solution {
public:
    int maxValue(vector<vector<int>>& events, int k) {
        vector<vector<int>> mem(events.size(), vector<int>(k + 1, -1));
        ranges::sort(events);
        return maxValue(events, 0, k, mem);
    }

private:
    // Returns the maximum sum of values that you can receive by attending
    // events[i..n), where k is the maximum number of attendance events.
    int maxValue(const vector<vector<int>>& events, int i, int k,
                vector<vector<int>>& mem) {
        if (k == 0 || i == events.size())
            return 0;
        if (mem[i][k] != -1)
            return mem[i][k];

        // Binary search `events` to find the first index j
        // s.t. events[j][0] > events[i][1].
        const auto it = upper_bound(
            events.begin() + i, events.end(), events[i][1],
            [](int end, const vector<int>& event) { return event[0] > end; });
        const int j = distance(events.begin(), it);
        return mem[i][k] = max(events[i][2] + maxValue(events, j, k - 1, mem),
                               maxValue(events, i + 1, k, mem));
    }
};
```

823. 1752.cpp

Path: LeetCode-main/solutions/1752. Check if Array Is Sorted and Rotated/1752.cpp

```
class Solution {
public:
    bool check(vector<int>& nums) {
        const int n = nums.size();
        int rotates = 0;

        for (int i = 0; i < n; ++i)
            if (nums[i] > nums[(i + 1) % n] && ++rotates > 1)
                return false;

        return true;
    }
};
```


824. 1753.cpp

Path: LeetCode-main/solutions/1753. Maximum Score From Removing Stones/1753.cpp

```
class Solution {
public:
    int maximumScore(int a, int b, int c) {
        // the maximum <= the minimum + the middle
        const int x = (a + b + c) / 2;
        // the maximum > the minimum + the middle
        const int y = a + b + c - max({a, b, c});
        return min(x, y);
    }
};
```

825. 1754.cpp

Path: LeetCode-main/solutions/1754. Largest Merge Of Two Strings/1754.cpp

```
class Solution {
public:
    string largestMerge(string word1, string word2) {
        if (word1.empty())
            return word2;
        if (word2.empty())
            return word1;
        return word1 > word2 ? word1[0] + largestMerge(word1.substr(1), word2)
                             : word2[0] + largestMerge(word1, word2.substr(1));
    }
};
```

826. 1755.cpp

Path: LeetCode-main/solutions/1755. Closest Subsequence Sum/1755.cpp

```
class Solution {
public:
    int minAbsDifference(vector<int>& nums, int goal) {
        const int n = nums.size() / 2;
        const vector<int> lNums(nums.begin(), nums.begin() + n);
        const vector<int> rNums(nums.begin() + n, nums.end());
        int ans = INT_MAX;
        vector<int> lSums;
        vector<int> rSums;

        dfs(lNums, 0, 0, lSums);
        dfs(rNums, 0, 0, rSums);
        ranges::sort(rSums);

        for (const int lSum : lSums) {
            const int i = firstGreaterEqual(rSums, goal - lSum);
            if (i < rSums.size()) // 2^n
                ans = min(ans, abs(goal - lSum - rSums[i]));
            if (i > 0)
                ans = min(ans, abs(goal - lSum - rSums[i - 1]));
        }

        return ans;
    }

private:
    void dfs(const vector<int>& arr, int i, int path, vector<int>& sums) {
        if (i == arr.size()) {
            sums.push_back(path);
            return;
        }
        dfs(arr, i + 1, path + arr[i], sums);
        dfs(arr, i + 1, path, sums);
    }

    int firstGreaterEqual(const vector<int>& arr, int target) {
        return ranges::lower_bound(arr, target) - arr.begin();
    }
};
```

827. 1758.cpp

Path: LeetCode-main/solutions/1758. Minimum Changes To Make Alternating Binary String/1758.cpp

```
class Solution {
public:
    int minOperations(string s) {
        int cost10; // the cost to make s "1010"

        for (int i = 0; i < s.length(); ++i)
            if (s[i] - '0' == i % 2)
                ++cost10;

        const int cost01 = s.length() - cost10; // the cost to make s "0101"
        return min(cost10, cost01);
    }
};
```

828. 1759.cpp

Path: LeetCode-main/solutions/1759. Count Number of Homogenous Substrings/1759.cpp

```
class Solution {
public:
    int countHomogenous(string s) {
        constexpr int kMod = 1'000'000'007;
        int ans = 0;
        int count = 0;
        char currentChar = '@';

        for (const char c : s) {
            count = c == currentChar ? count + 1 : 1;
            currentChar = c;
            ans += count;
            ans %= kMod;
        }

        return ans;
    }
};
```

829. 1760.cpp

Path: LeetCode-main/solutions/1760. Minimum Limit of Balls in a Bag/1760.cpp

```
class Solution {
public:
    int minimumSize(vector<int>& nums, int maxOperations) {
        int l = 1;
        int r = ranges::max(nums);

        while (l < r) {
            const int m = (l + r) / 2;
            if (numOperations(nums, m) <= maxOperations)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    // Returns the number of operations required to make m penalty.
    int numOperations(const vector<int>& nums, int m) {
        int operations = 0;
        for (const int num : nums)
            operations += (num - 1) / m;
        return operations;
    }
};
```

830. 1761.cpp

Path: LeetCode-main/solutions/1761. Minimum Degree of a Connected Trio in a Graph/1761.cpp

```
class Solution {
public:
    int minTrioDegree(int n, vector<vector<int>>& edges) {
        int ans = INT_MAX;
        vector<unordered_set<int>> graph(n);
        vector<int> degrees(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0] - 1;
            const int v = edge[1] - 1;
            // Store the mapping from `min(u, v)` to `max(u, v)` to speed up.
            graph[min(u, v)].insert(max(u, v));
            ++degrees[u];
            ++degrees[v];
        }

        for (int u = 0; u < n; ++u)
            for (const int v : graph[u])
                for (const int w : graph[u])
                    if (graph[v].contains(w))
                        ans = min(ans, degrees[u] + degrees[v] + degrees[w] - 6);

        return ans == INT_MAX ? -1 : ans;
    }
};
```

831. 1762.cpp

Path: LeetCode-main/solutions/1762. Buildings With an Ocean View/1762.cpp

```
class Solution {
public:
    vector<int> findBuildings(vector<int>& heights) {
        vector<int> stack;

        for (int i = 0; i < heights.size(); ++i) {
            while (!stack.empty() && heights[stack.back()] <= heights[i])
                stack.pop_back();
            stack.push_back(i);
        }

        return stack;
    }
};
```


832. 1763.cpp

Path: LeetCode-main/solutions/1763. Longest Nice Substring/1763.cpp

```
class Solution {
public:
    string longestNiceSubstring(string s) {
        if (s.length() < 2)
            return "";

        unordered_set<char> seen{s.begin(), s.end()};

        for (int i = 0; i < s.size(); ++i)
            // If both upper and lower case letters exists in the string, keep moving,
            // else take the erroneous character as a partition and check for its left
            // and right parts to be nice strings.
            if (!seen.contains(toggleCase(s[i]))) {
                const string prefix = longestNiceSubstring(s.substr(0, i));
                const string suffix = longestNiceSubstring(s.substr(i + 1));
                return prefix.length() >= suffix.length() ? prefix : suffix;
            }

        return s;
    }

private:
    char toggleCase(char c) {
        return islower(c) ? toupper(c) : tolower(c);
    }
};
```

833. 1764.cpp

Path: LeetCode-main/solutions/1764. Form Array by Concatenating Subarrays of Another Array/1764.cpp

```
class Solution {
public:
    bool canChoose(vector<vector<int>>& groups, vector<int>& nums) {
        int i = 0; // groups' index
        int j = 0; // nums' index

        while (i < groups.size() && j < nums.size())
            if (isMatch(groups[i], nums, j)) {
                j += groups[i].size();
                ++i;
            } else {
                ++j;
            }

        return i == groups.size();
    }

private:
    // Returns true if group == nums[j..j + |group|].
    bool isMatch(const vector<int>& group, const vector<int>& nums, int j) {
        if (j + group.size() > nums.size())
            return false;
        for (int i = 0; i < group.size(); ++i)
            if (group[i] != nums[j + i])
                return false;
        return true;
    }
};
```

834. 1765.cpp

Path: LeetCode-main/solutions/1765. Map of Highest Peak/1765.cpp

```
class Solution {
public:
    vector<vector<int>> highestPeak(vector<vector<int>>& isWater) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = isWater.size();
        const int n = isWater[0].size();
        vector<vector<int>> ans(m, vector<int>(n, -1));
        queue<pair<int, int>> q;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (isWater[i][j] == 1) {
                    q.emplace(i, j);
                    ans[i][j] = 0;
                }

        while (!q.empty()) {
            const auto [i, j] = q.front();
            q.pop();
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (ans[x][y] != -1)
                    continue;
                ans[x][y] = ans[i][j] + 1;
                q.emplace(x, y);
            }
        }

        return ans;
    }
};
```

835. 1766.cpp

Path: LeetCode-main/solutions/1766. Tree of Coprimes/1766.cpp

```
class Solution {
public:
    vector<int> getCoprimes(vector<int>& nums, vector<vector<int>>& edges) {
        vector<int> ans(nums.size(), -1);
        vector<vector<int>> tree(nums.size());
        // stacks[i] := (node, depth)s of nodes with value i
        vector<stack<pair<int, int>>> stacks(kMax + 1);

        for (const vector<int> edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        dfs(tree, 0, /*prev=*/-1, /*depth=*/0, nums, stacks, ans);
        return ans;
    }

private:
    static constexpr int kMax = 50;

    void dfs(const vector<vector<int>>& tree, int u, int prev, int depth,
             const vector<int>& nums, vector<stack<pair<int, int>>& stacks,
             vector<int>& ans) {
        ans[u] = getAncestor(u, stacks, nums);
        stacks[nums[u]].push({u, depth});

        for (const int v : tree[u])
            if (v != prev)
                dfs(tree, v, u, depth + 1, nums, stacks, ans);

        stacks[nums[u]].pop();
    }

    int getAncestor(int u, const vector<stack<pair<int, int>>& stacks,
                    const vector<int>& nums) {
        int maxNode = -1;
        int maxDepth = -1;
        for (int i = 1; i <= kMax; ++i)
            if (!stacks[i].empty() && stacks[i].top().second > maxDepth &&
                __gcd(nums[u], i) == 1) {
                maxNode = stacks[i].top().first;
                maxDepth = stacks[i].top().second;
            }
        return maxNode;
    }
};
```

836. 1768.cpp

Path: LeetCode-main/solutions/1768. Merge Strings Alternately/1768.cpp

```
class Solution {
public:
    string mergeAlternately(string word1, string word2) {
        const int n = min(word1.length(), word2.length());
        string prefix;

        for (int i = 0; i < n; ++i) {
            prefix += word1[i];
            prefix += word2[i];
        }

        return prefix + word1.substr(n) + word2.substr(n);
    }
};
```

837. 1769.cpp

Path: LeetCode-main/solutions/1769. Minimum Number of Operations to Move All Balls to Each Box/1769.cpp

```
class Solution {
public:
    vector<int> minOperations(string boxes) {
        vector<int> ans(boxes.length());

        for (int i = 0, count = 0, moves = 0; i < boxes.length(); ++i) {
            ans[i] += moves;
            count += boxes[i] - '0';
            moves += count;
        }

        for (int i = boxes.length() - 1, count = 0, moves = 0; i >= 0; --i) {
            ans[i] += moves;
            count += boxes[i] - '0';
            moves += count;
        }

        return ans;
    }
};
```

838. 1770.cpp

Path: LeetCode-main/solutions/1770. Maximum Score from Performing Multiplication Operations/1770.cpp

```
class Solution {
public:
    int maximumScore(vector<int>& nums, vector<int>& multipliers) {
        vector<vector<int>> mem(multipliers.size(),
                               vector<int>(multipliers.size(), -1));
        return maximumScore(nums, 0, multipliers, 0, mem);
    }

private:
    // Returns the maximum score of nums[s..e] and multipliers[i].
    int maximumScore(const vector<int>& nums, int s,
                    const vector<int>& multipliers, int i,
                    vector<vector<int>>& mem) {
        if (i == multipliers.size())
            return 0;
        if (mem[s][i] != -1)
            return mem[s][i];

        // The number of nums picked on the start side is s.
        // The number of nums picked on the end side is i - s.
        // So, e = n - (i - s) - 1.
        const int e = nums.size() - (i - s) - 1;
        const int pickStart = nums[s] * multipliers[i] +
                               maximumScore(nums, s + 1, multipliers, i + 1, mem);
        const int pickEnd = nums[e] * multipliers[i] +
                               maximumScore(nums, s, multipliers, i + 1, mem);
        return mem[s][i] = max(pickStart, pickEnd);
    }
};
```

839. 1771.cpp

Path: LeetCode-main/solutions/1771. Maximize Palindrome Length From Subsequences/1771.cpp

```
class Solution {
public:
    int longestPalindrome(string word1, string word2) {
        const string& s = word1 + word2;
        const int n = s.length();
        int ans = 0;
        // dp[i][j] := the length of LPS(s[i..j])
        vector<vector<int>> dp(n, vector<int>(n));

        for (int i = 0; i < n; ++i)
            dp[i][i] = 1;

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                if (s[i] == s[j]) {
                    dp[i][j] = 2 + dp[i + 1][j - 1];
                    if (i < word1.length() && j >= word1.length())
                        ans = max(ans, dp[i][j]);
                } else {
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
                }
            }

        return ans;
    }
};
```


840. 1772.cpp

Path: LeetCode-main/solutions/1772. Sort Features by Popularity/1772.cpp

```
class Solution {
public:
    vector<string> sortFeatures(vector<string>& features,
                               vector<string>& responses) {
        vector<string> ans;
        vector<pair<int, int>> featCount; // {i: count[features[i]]}
        unordered_map<string, int> count;

        for (const string& res : responses) {
            istringstream iss(res);
            unordered_set<string> seen;
            for (string token; getline(iss, token, ' ');)
                seen.insert(token);
            for (const string& token : seen)
                ++count[token];
        }

        for (int i = 0; i < features.size(); ++i)
            featCount.emplace_back(i, count[features[i]]);

        ranges::sort(featCount, ranges::less{}, [](const pair<int, int>& a) {
            const auto& [i, count] = a;
            return pair<int, int>{-count, i};
        });

        for (const auto& [i, count] : featCount)
            ans.push_back(features[i]);

        return ans;
    }
};
```

841. 1773.cpp

Path: LeetCode-main/solutions/1773. Count Items Matching a Rule/1773.cpp

```
class Solution {
public:
    int countMatches(vector<vector<string>>& items, string ruleKey,
                    string ruleValue) {
        const int index = ruleKey == "type" ? 0 : ruleKey == "color" ? 1 : 2;
        return ranges::count_if(items, [index, &ruleValue](const auto& item) {
            return item[index] == ruleValue;
        });
    }
};
```

842. 1774.cpp

Path: LeetCode-main/solutions/1774. Closest Dessert Cost/1774.cpp

```
class Solution {
public:
    int closestCost(vector<int>& baseCosts, vector<int>& toppingCosts,
                    int target) {
        int ans = INT_MAX;

        for (const int baseCost : baseCosts)
            dfs(toppingCosts, 0, target, baseCost, ans);

        return ans;
    }

private:
    void dfs(const vector<int>& toppingCosts, int i, int target, int currCost,
              int& ans) {
        if (abs(currCost - target) < abs(ans - target))
            ans = currCost;
        if (i == toppingCosts.size() || currCost >= target)
            return;

        for (int k = 0; k < 3; ++k)
            dfs(toppingCosts, i + 1, target, currCost + k * toppingCosts[i], ans);
    }
};
```

843. 1775.cpp

Path: LeetCode-main/solutions/1775. Equal Sum Arrays With Minimum Number of Operations/1775.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums1, vector<int>& nums2) {
        if (nums1.size() * 6 < nums2.size() || nums2.size() * 6 < nums1.size())
            return -1;

        int sum1 = accumulate(nums1.begin(), nums1.end(), 0);
        int sum2 = accumulate(nums2.begin(), nums2.end(), 0);
        if (sum1 > sum2)
            return minOperations(nums2, nums1);

        int ans = 0;
        // increasing in `nums1` and decreasing in `nums2`
        vector<int> count(6);

        for (const int num : nums1)
            ++count[6 - num];

        for (const int num : nums2)
            ++count[num - 1];

        for (int i = 5; sum2 > sum1;) {
            while (count[i] == 0)
                --i;
            sum1 += i;
            --count[i];
            ++ans;
        }

        return ans;
    }
};
```

844. 1776.cpp

Path: LeetCode-main/solutions/1776. Car Fleet II/1776.cpp

```
struct Car {
    int pos;
    int speed;
    double collisionTime;
    Car(int pos, int speed, double collisionTime)
        : pos(pos), speed(speed), collisionTime(collisionTime) {}
};

class Solution {
public:
    vector<double> getCollisionTimes(vector<vector<int>>& cars) {
        vector<double> ans(cars.size());
        stack<Car> stack;

        for (int i = cars.size() - 1; i >= 0; --i) {
            const int pos = cars[i][0];
            const int speed = cars[i][1];
            while (!stack.empty() && (speed <= stack.top().speed ||
                                     getCollisionTime(stack.top(), pos, speed) >=
                                     stack.top().collisionTime))
                stack.pop();
            if (stack.empty()) {
                stack.emplace(pos, speed, INT_MAX);
                ans[i] = -1;
            } else {
                const double collisionTime = getCollisionTime(stack.top(), pos, speed);
                stack.emplace(pos, speed, collisionTime);
                ans[i] = collisionTime;
            }
        }

        return ans;
    }

private:
    double getCollisionTime(const Car& car, int pos, int speed) {
        return (car.pos - pos) / (double)(speed - car.speed);
    }
};
```

845. 1778.cpp

Path: LeetCode-main/solutions/1778. Shortest Path in a Hidden Grid/1778.cpp

```
/**
 * // This is the GridMaster's API interface.
 * // You should not implement it, or speculate about its implementation
 * class GridMaster {
 * public:
 *     bool canMove(char direction);
 *     void std::move(char direction);
 *     boolean isTarget();
 * };
 */

enum class Grid { kUnvisited, kStart, kTarget, kBlocked, kEmpty };

class Solution {
public:
    int findShortestPath(GridMaster& master) {
        constexpr int m = 501;
        constexpr int startX = m;
        constexpr int startY = m;
        vector<vector<Grid>> grid(m * 2, vector<Grid>(m * 2, Grid::kUnvisited));

        // Build the grid information by DFS.
        dfs(master, grid, startX, startY);

        queue<pair<int, int>> q{{{startX, startY}}};
        grid[startX][startY] = Grid::kBlocked;

        // Find the steps by BFS.
        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (grid[x][y] == Grid::kTarget)
                        return step;
                    if (grid[x][y] == Grid::kBlocked)
                        continue;
                    grid[x][y] = Grid::kBlocked;
                    q.emplace(x, y);
                }
            }

        return -1;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
    static constexpr char charTable[4] = {'R', 'D', 'L', 'U'};

    void dfs(GridMaster& master, vector<vector<Grid>>& grid, int i, int j) {
        if (grid[i][j] != Grid::kUnvisited)
            return;
        if (master.isTarget())
            grid[i][j] = Grid::kTarget;
        else
            grid[i][j] = Grid::kEmpty;

        for (int k = 0; k < 4; ++k) {
            const int x = i + kDirs[k][0];
            const int y = j + kDirs[k][1];
            const char d = charTable[k];
            const char undoD = charTable[(k + 2) % 4];
            if (master.canMove(d)) {
                master.move(d);
                dfs(master, grid, x, y);
                master.move(undoD);
            } else {
                grid[x][y] = Grid::kBlocked;
            }
        }
    }
};
```

846. 1779.cpp

Path: LeetCode-main/solutions/1779. Find Nearest Point That Has the Same X or Y Coordinate/1779.cpp

```
class Solution {
public:
    int nearestValidPoint(int x, int y, vector<vector<int>>& points) {
        int ans = -1;
        int minDist = INT_MAX;

        for (int i = 0; i < points.size(); ++i) {
            const int dx = x - points[i][0];
            const int dy = y - points[i][1];
            if (dx == 0 || dy == 0) {
                const int dist = abs(dx + dy);
                if (dist < minDist) {
                    minDist = dist;
                    ans = i;
                }
            }
        }

        return ans;
    }
};
```

847. 1780.cpp

Path: LeetCode-main/solutions/1780. Check if Number is a Sum of Powers of Three/1780.cpp

```
class Solution {
public:
    bool checkPowersOfThree(int n) {
        while (n > 1) {
            const int r = n % 3;
            if (r == 2)
                return false;
            n /= 3;
        }
        return true;
    }
};
```


848. 1781.cpp

Path: LeetCode-main/solutions/1781. Sum of Beauty of All Substrings/1781.cpp

```
class Solution {
public:
    int beautySum(string s) {
        int ans = 0;

        for (int i = 0; i < s.length(); ++i) {
            vector<int> count(26);
            for (int j = i; j < s.length(); ++j) {
                ++count[s[j] - 'a'];
                ans += ranges::max(count) - getMinFreq(count);
            }
        }

        return ans;
    }

private:
    // Returns the minimum frequency > 0.
    int getMinFreq(const vector<int>& count) {
        int minFreq = INT_MAX;
        for (const int freq : count)
            if (freq > 0)
                minFreq = min(minFreq, freq);
        return minFreq;
    }
};
```

849. 1782.cpp

Path: LeetCode-main/solutions/1782. Count Pairs Of Nodes/1782.cpp

```
class Solution {
public:
    vector<int> countPairs(int n, vector<vector<int>>& edges,
                          vector<int>& queries) {
        vector<int> ans(queries.size());

        // count[i] := the number of edges of node i
        vector<int> count(n + 1);

        // shared[i][j] := the number of edges incident to i or j, where i < j
        vector<unordered_map<int, int>> shared(n + 1);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            ++count[u];
            ++count[v];
            ++shared[min(u, v)][max(u, v)];
        }

        vector<int> sortedCount(count);
        ranges::sort(sortedCount);

        int k = 0;
        for (const int query : queries) {
            for (int i = 1, j = n; i < j;)
                if (sortedCount[i] + sortedCount[j] > query)
                    // sortedCount[i] + sortedCount[j] > query
                    // sortedCount[i + 1] + sortedCount[j] > query
                    // ...
                    // sortedCount[j - 1] + sortedCount[j] > query
                    // So, there are (j - 1) - i + 1 = j - i pairs > query
                    ans[k] += (j--) - i;
                else
                    ++i;
            for (int i = 1; i <= n; ++i)
                for (const auto& [j, sh] : shared[i])
                    if (count[i] + count[j] > query && count[i] + count[j] - sh <= query)
                        --ans[k];
            ++k;
        }
        return ans;
    }
};
```

850. 1784.cpp

Path: LeetCode-main/solutions/1784. Check if Binary String Has at Most One Segment of Ones/1784.cpp

```
class Solution {  
public:  
    bool checkOnesSegment(string s) {  
        return s.find("01") == string::npos;  
    }  
};
```

851. 1785.cpp

Path: LeetCode-main/solutions/1785. Minimum Elements to Add to Form a Given Sum/1785.cpp

```
class Solution {
public:
    int minElements(vector<int>& nums, int limit, int goal) {
        const long sum = accumulate(nums.begin(), nums.end(), 0L);
        const double diff = abs(goal - sum);
        return ceil(diff / limit);
    }
};
```

852. 1786.cpp

Path: LeetCode-main/solutions/1786. Number of Restricted Paths From First to Last Node/1786.cpp

```
class Solution {
public:
    int countRestrictedPaths(int n, vector<vector<int>>& edges) {
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0] - 1;
            const int v = edge[1] - 1;
            const int w = edge[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        return dijkstra(graph, 0, n - 1);
    }

private:
    int dijkstra(const vector<vector<pair<int, int>>>& graph, int src, int dst) {
        constexpr int kMod = 1'000'000'007;
        // ways[i] := the number of restricted path from i to n
        vector<long> ways(graph.size());
        // dist[i] := the distance to the last node of i
        vector<long> dist(graph.size(), LONG_MAX);

        ways[dst] = 1;
        dist[dst] = 0;
        using P = pair<long, int>; // (d, u)
        priority_queue<P, vector<P>, greater<>> minHeap;
        minHeap.emplace(dist[dst], dst);

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (d > dist[u])
                continue;
            for (const auto& [v, w] : graph[u]) {
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.emplace(dist[v], v);
                }
                if (dist[v] < dist[u]) {
                    ways[u] += ways[v];
                    ways[u] %= kMod;
                }
            }
        }

        return ways[src];
    }
};
```

853. 1787.cpp

Path: LeetCode-main/solutions/1787. Make the XOR of All Segments Equal to Zero/1787.cpp

```
class Solution {
public:
    int minChanges(vector<int>& nums, int k) {
        constexpr int kMax = 1024;
        const int n = nums.size();
        // counts[i] := the counter that maps at the i-th position
        vector<unordered_map<int, int>> counts(k);
        // dp[i][j] := the minimum number of elements to change s.t. XOR(nums[i..k -
        // 1]) is j
        vector<vector<int>> dp(k, vector<int>(kMax, n));

        for (int i = 0; i < n; ++i)
            ++counts[i % k][nums[i]];

        auto countAt = [n, k](int i) -> int { return n / k + (n % k > i ? 1 : 0); };

        // Initialize the DP array.
        for (int j = 0; j < kMax; ++j)
            dp[k - 1][j] =
                countAt(k - 1) - (counts[k - 1].contains(j) ? counts[k - 1][j] : 0);

        for (int i = k - 2; i >= 0; --i) {
            // The worst-case scenario is changing all the i-th position numbers to a
            // non-existent value in the current bucket.
            const int changeAll = countAt(i) + ranges::min(dp[i + 1]);
            for (int j = 0; j < kMax; ++j) {
                dp[i][j] = changeAll;
                for (const auto& [num, freq] : counts[i]) {
                    // the cost to change every number in the i-th position to `num`
                    const int cost = countAt(i) - freq;
                    dp[i][j] = min(dp[i][j], dp[i + 1][j ^ num] + cost);
                }
            }
        }

        return dp[0][0];
    }
};
```

854. 1788.cpp

Path: LeetCode-main/solutions/1788. Maximize the Beauty of the Garden/1788.cpp

```
class Solution {
public:
    int maximumBeauty(vector<int>& flowers) {
        int ans = INT_MIN;
        int prefix = 0;
        unordered_map<int, int> flowerToPrefix;

        for (const int flower : flowers) {
            if (const auto it = flowerToPrefix.find(flower);
                it != flowerToPrefix.cend())
                ans = max(ans, prefix - it->second + flower * 2);
            prefix += max(0, flower);
            if (!flowerToPrefix.contains(flower))
                flowerToPrefix[flower] = prefix;
        }

        return ans;
    }
};
```

855. 179.cpp

Path: LeetCode-main/solutions/179. Largest Number/179.cpp

```
class Solution {
public:
    string largestNumber(vector<int>& nums) {
        string ans;

        ranges::sort(nums, [](int a, int b) {
            return to_string(a) + to_string(b) > to_string(b) + to_string(a);
        });

        for (const int num : nums)
            ans += to_string(num);

        return ans[0] == '0' ? "0" : ans;
    }
};
```


856. 1790.cpp

Path: LeetCode-main/solutions/1790. Check if One String Swap Can Make Strings Equal/1790.cpp

```
class Solution {
public:
    // Similar to 859. Buddy Strings
    bool areAlmostEqual(string s1, string s2) {
        vector<int> diffIndices;
        for (int i = 0; i < s1.length(); ++i)
            if (s1[i] != s2[i])
                diffIndices.push_back(i);
        return diffIndices.empty() || (diffIndices.size() == 2 &&
            s1[diffIndices[0]] == s2[diffIndices[1]] &&
            s1[diffIndices[1]] == s2[diffIndices[0]]);
    }
};
```

857. 1791.cpp

Path: LeetCode-main/solutions/1791. Find Center of Star Graph/1791.cpp

```
class Solution {
public:
    int findCenter(vector<vector<int>>& edges) {
        return edges[0][0] == edges[1][0] || edges[0][0] == edges[1][1]
            ? edges[0][0]
            : edges[0][1];
    }
};
```

858. 1792.cpp

Path: LeetCode-main/solutions/1792. Maximum Average Pass Ratio/1792.cpp

```
class Solution {
public:
    double maxAverageRatio(vector<vector<int>>& classes, int extraStudents) {
        // (extra pass ratio, pass, total)
        priority_queue<tuple<double, int, int>> maxHeap;

        for (const vector<int>& c : classes) {
            const int pass = c[0];
            const int total = c[1];
            maxHeap.emplace(extraPassRatio(pass, total), pass, total);
        }

        for (int i = 0; i < extraStudents; ++i) {
            const auto [_, pass, total] = maxHeap.top();
            maxHeap.pop();
            maxHeap.emplace(extraPassRatio(pass + 1, total + 1), pass + 1, total + 1);
        }

        double ratioSum = 0;

        while (!maxHeap.empty()) {
            const auto [_, pass, total] = maxHeap.top();
            maxHeap.pop();
            ratioSum += pass / static_cast<double>(total);
        }

        return ratioSum / classes.size();
    }

private:
    // Returns the extra pass ratio if a brilliant student joins.
    double extraPassRatio(int pass, int total) {
        return (pass + 1) / static_cast<double>(total + 1) -
            pass / static_cast<double>(total);
    }
};
```

859. 1793-2.cpp

Path: LeetCode-main/solutions/1793. Maximum Score of a Good Subarray/1793-2.cpp

```
class Solution {
public:
    int maximumScore(vector<int>& nums, int k) {
        const int n = nums.size();
        int ans = nums[k];
        int mn = nums[k];
        int i = k;
        int j = k;

        // Greedily expand the window and decrease the minimum as slow as possible.
        while (i > 0 || j < n - 1) {
            if (i == 0)
                ++j;
            else if (j == n - 1)
                --i;
            else if (nums[i - 1] < nums[j + 1])
                ++j;
            else // nums[i - 1] >= nums[j + 1]
                --i;
            mn = min({mn, nums[i], nums[j]});
            ans = max(ans, mn * (j - i + 1));
        }

        return ans;
    }
};
```

860. 1793.cpp

Path: LeetCode-main/solutions/1793. Maximum Score of a Good Subarray/1793.cpp

```
class Solution {
public:
    // Similar to 84. Largest Rectangle in Histogram
    int maximumScore(vector<int>& nums, int k) {
        int ans = 0;
        stack<int> stack;

        for (int i = 0; i <= nums.size(); ++i) {
            while (!stack.empty() &&
                (i == nums.size() || nums[stack.top()] > nums[i])) {
                const int h = nums[stack.top()];
                stack.pop();
                const int w = stack.empty() ? i : i - stack.top() - 1;
                if ((stack.empty() || stack.top() + 1 <= k) && i - 1 >= k)
                    ans = max(ans, h * w);
            }
            stack.push(i);
        }

        return ans;
    }
};
```

861. 1794.cpp

Path: LeetCode-main/solutions/1794. Count Pairs of Equal Substrings With Minimum Difference/1794.cpp

```
class Solution {
public:
    int countQuadruples(const string& s1, const string& s2) {
        // To minimize j - a, the length of the substring should be 1. This is
        // because for substrings with a size greater than 1, a will decrease,
        // causing j - a to become larger.
        int ans = 0;
        int diff = INT_MAX; // diff := j - a
        vector<int> firstJ(26, -1);
        vector<int> lastA(26, -1);

        for (int j = s1.length() - 1; j >= 0; --j)
            firstJ[s1[j] - 'a'] = j;

        for (int a = 0; a < s2.length(); ++a)
            lastA[s2[a] - 'a'] = a;

        for (int i = 0; i < 26; ++i) {
            if (firstJ[i] == -1 || lastA[i] == -1)
                continue;
            if (firstJ[i] - lastA[i] < diff) {
                diff = firstJ[i] - lastA[i];
                ans = 0;
            }
            if (firstJ[i] - lastA[i] == diff)
                ++ans;
        }

        return ans;
    }
};
```

862. 1796.cpp

Path: LeetCode-main/solutions/1796. Second Largest Digit in a String/1796.cpp

```
class Solution {
public:
    int secondHighest(string s) {
        int maxDigit = -1;
        int secondMaxDigit = -1;

        for (const char c : s)
            if (isdigit(c)) {
                const int digit = c - '0';
                if (digit > maxDigit) {
                    secondMaxDigit = maxDigit;
                    maxDigit = digit;
                } else if (maxDigit > digit && digit > secondMaxDigit) {
                    secondMaxDigit = digit;
                }
            }

        return secondMaxDigit;
    }
};
```

863. 1797.cpp

Path: LeetCode-main/solutions/1797. Design Authentication Manager/1797.cpp

```
class AuthenticationManager {
public:
    AuthenticationManager(int timeToLive) : timeToLive(timeToLive) {}

    void generate(string tokenId, int currentTime) {
        tokenIdToExpiryTime[tokenId] = currentTime;
        times.emplace(currentTime);
    }

    void renew(string tokenId, int currentTime) {
        if (const auto it = tokenIdToExpiryTime.find(tokenId);
            it == tokenIdToExpiryTime.cend() ||
            currentTime >= it->second + timeToLive)
            return;
        times.erase(tokenIdToExpiryTime[tokenId]);
        tokenIdToExpiryTime[tokenId] = currentTime;
        times.emplace(currentTime);
    }

    int countUnexpiredTokens(int currentTime) {
        const auto it = times.lower_bound(currentTime - timeToLive + 1);
        // Remove expired tokens.
        times.erase(times.begin(), it);
        return times.size();
    }

private:
    const int timeToLive;
    unordered_map<string, int> tokenIdToExpiryTime;
    set<int> times;
};
```


864. 1798.cpp

Path: LeetCode-main/solutions/1798. Maximum Number of Consecutive Values You Can Make/1798.cpp

```
class Solution {
public:
    int getMaximumConsecutive(vector<int>& coins) {
        int ans = 1; // the next value we want to make

        ranges::sort(coins);

        for (const int coin : coins) {
            if (coin > ans)
                return ans;
            ans += coin;
        }

        return ans;
    }
};
```

865. 1799.cpp

Path: LeetCode-main/solutions/1799. Maximize Score After N Operations/1799.cpp

```
class Solution {
public:
    int maxScore(vector<int>& nums) {
        const int n = nums.size() / 2;
        vector<vector<int>> mem(n + 1, vector<int>(1 << n * 2));
        return maxScore(nums, 1, 0, mem);
    }

private:
    // Returns the maximum score you can receive after performing the k to n
    // operations, where `mask` is the bitmask of the chosen numbers.
    int maxScore(const vector<int>& nums, int k, int mask,
                vector<vector<int>>& mem) {
        if (k == mem.size())
            return 0;
        if (mem[k][mask] > 0)
            return mem[k][mask];

        for (int i = 0; i < nums.size(); ++i)
            for (int j = i + 1; j < nums.size(); ++j) {
                const int chosenMask = 1 << i | 1 << j;
                if ((mask & chosenMask) == 0)
                    mem[k][mask] = max(mem[k][mask],
                                        k * __gcd(nums[i], nums[j]) +
                                        maxScore(nums, k + 1, mask | chosenMask, mem));
            }

        return mem[k][mask];
    }
};
```

866. 18.cpp

Path: LeetCode-main/solutions/18. 4Sum/18.cpp

```
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        vector<vector<int>> ans;
        vector<int> path;
        ranges::sort(nums);
        nSum(nums, 4, target, 0, nums.size() - 1, path, ans);
        return ans;
    }

private:
    // Finds n numbers that add up to the target in [l, r].
    void nSum(const vector<int>& nums, long n, long target, int l, int r,
              vector<int>& path, vector<vector<int>>& ans) {
        if (r - l + 1 < n || target < nums[l] * n || target > nums[r] * n)
            return;
        if (n == 2) {
            // Similar to the sub procedure in 15. 3Sum
            while (l < r) {
                const int sum = nums[l] + nums[r];
                if (sum == target) {
                    path.push_back(nums[l]);
                    path.push_back(nums[r]);
                    ans.push_back(path);
                    path.pop_back();
                    path.pop_back();
                    ++l;
                    --r;
                    while (l < r && nums[l] == nums[l - 1])
                        ++l;
                    while (l < r && nums[r] == nums[r + 1])
                        --r;
                } else if (sum < target) {
                    ++l;
                } else {
                    --r;
                }
            }
        }
        return;
    }

    for (int i = l; i <= r; ++i) {
        if (i > l && nums[i] == nums[i - 1])
            continue;
        path.push_back(nums[i]);
        nSum(nums, n - 1, target - nums[i], i + 1, r, path, ans);
        path.pop_back();
    }
};
```

867. 1800.cpp

Path: LeetCode-main/solutions/1800. Maximum Ascending Subarray Sum/1800.cpp

```
class Solution {
public:
    int maxAscendingSum(vector<int>& nums) {
        int ans = 0;
        int sum = nums[0];

        for (int i = 1; i < nums.size(); ++i)
            if (nums[i] > nums[i - 1]) {
                sum += nums[i];
            } else {
                ans = max(ans, sum);
                sum = nums[i];
            }

        return max(ans, sum);
    }
};
```

868. 1801.cpp

Path: LeetCode-main/solutions/1801. Number of Orders in the Backlog/1801.cpp

```
class Solution {
public:
    int getNumberOfBacklogOrders(vector<vector<int>>& orders) {
        constexpr int kMod = 1'000'000'007;
        int ans = 0;
        priority_queue<vector<int>> buysMaxHeap;
        priority_queue<vector<int>, vector<vector<int>>, greater<>> sellsMinHeap;

        for (const vector<int>& order : orders) {
            if (order[2] == 0)
                buysMaxHeap.push(order);
            else
                sellsMinHeap.push(order);
            while (!buysMaxHeap.empty() && !sellsMinHeap.empty() &&
                buysMaxHeap.top()[0] >= sellsMinHeap.top()[0]) {
                const int minAmount = min(buysMaxHeap.top()[1], sellsMinHeap.top()[1]);
                vector<int> buysMaxHeapTop = buysMaxHeap.top();
                buysMaxHeap.pop();
                buysMaxHeapTop[1] -= minAmount;
                if (buysMaxHeapTop[1] > 0)
                    buysMaxHeap.push(buysMaxHeapTop);

                vector<int> sellsMinHeapTop = sellsMinHeap.top();
                sellsMinHeap.pop();
                sellsMinHeapTop[1] -= minAmount;
                if (sellsMinHeapTop[1] > 0)
                    sellsMinHeap.push(sellsMinHeapTop);
            }
        }

        while (!buysMaxHeap.empty()) {
            ans += buysMaxHeap.top()[1], buysMaxHeap.pop();
            ans %= kMod;
        }

        while (!sellsMinHeap.empty()) {
            ans += sellsMinHeap.top()[1], sellsMinHeap.pop();
            ans %= kMod;
        }

        return ans;
    }
};
```

869. 1802.cpp

Path: LeetCode-main/solutions/1802. Maximum Value at a Given Index in a Bounded Array/1802.cpp

```
class Solution {
public:
    int maxValue(int n, int index, int maxSum) {
        maxSum -= n;

        int l = 0;
        int r = maxSum;

        // Find the first value x s.t. if A[index] = x, then sum(A) >= maxSum.
        while (l < r) {
            const int m = (l + r) / 2;
            if (getSum(n, index, m) >= maxSum)
                r = m;
            else
                l = m + 1;
        }

        return getSum(n, index, l) > maxSum ? l : l + 1;
    }

private:
    // Returns the minimum sum if nums[index] = x.
    long getSum(int n, int index, int x) {
        long l = min(index, x - 1);
        long r = min(n - index, x);
        long lSum = ((x - 1) + (x - 1 - l + 1)) * l / 2;
        long rSum = (x + (x - r + 1)) * r / 2;
        return lSum + rSum;
    }
};
```

870. 1803.cpp

Path: LeetCode-main/solutions/1803. Count Pairs With XOR in a Range/1803.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    int count = 0;
    TrieNode() : children(2) {}
};

class Solution {
public:
    int countPairs(vector<int>& nums, int low, int high) {
        int ans = 0;

        for (const int num : nums) {
            ans += getCount(num, high + 1) - getCount(num, low);
            insert(num);
        }

        return ans;
    }

private:
    static constexpr int kHeight = 14;
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(int num) {
        shared_ptr<TrieNode> node = root;
        for (int i = kHeight; i >= 0; --i) {
            const int bit = num >> i & 1;
            if (node->children[bit] == nullptr)
                node->children[bit] = make_shared<TrieNode>();
            node = node->children[bit];
            ++node->count;
        }
    }

    // Returns the number of numbers < limit.
    int getCount(int num, int limit) {
        int count = 0;
        shared_ptr<TrieNode> node = root;
        for (int i = kHeight; i >= 0; --i) {
            const int bit = num >> i & 1;
            const int bitLimit = limit >> i & 1;
            if (bitLimit == 1) {
                if (node->children[bit] != nullptr)
                    count += node->children[bit]->count;
                node = node->children[bit ^ 1];
            } else {
                node = node->children[bit];
            }
            if (node == nullptr)
                break;
        }
        return count;
    }
};
```

871. 1804.cpp

Path: LeetCode-main/solutions/1804. Implement Trie II (Prefix Tree)/1804.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    int prefixCount = 0;
    int wordCount = 0;
    TrieNode() : children(26) {}
};

class Trie {
public:
    void insert(string word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
            ++node->prefixCount;
        }
        ++node->wordCount;
    }

    int countWordsEqualTo(string word) {
        shared_ptr<TrieNode> node = find(word);
        return node ? node->wordCount : 0;
    }

    int countWordsStartingWith(string prefix) {
        shared_ptr<TrieNode> node = find(prefix);
        return node ? node->prefixCount : 0;
    }

    void erase(string word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            node = node->children[i];
            --node->prefixCount;
        }
        --node->wordCount;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    shared_ptr<TrieNode> find(const string& s) {
        shared_ptr<TrieNode> node = root;
        for (const char c : s) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                return nullptr;
            node = node->children[i];
        }
        return node;
    }
};
```


872. 1805.cpp

Path: LeetCode-main/solutions/1805. Number of Different Integers in a String/1805.cpp

```
class Solution {
public:
    int numDifferentIntegers(string word) {
        unordered_set<string> nums;
        string curr;

        for (const char c : word)
            if (isdigit(c)) {
                curr += c;
            } else if (curr.length() > 0) {
                nums.insert(removeLeadingZeros(curr));
                curr = "";
            }

        if (curr.length() > 0)
            nums.insert(removeLeadingZeros(curr));
        return nums.size();
    }

private:
    string removeLeadingZeros(const string& s) {
        const int index = s.find_first_not_of('0');
        return index == string::npos ? "0" : s.substr(index);
    }
};
```

873. 1806.cpp

Path: LeetCode-main/solutions/1806. Minimum Number of Operations to Reinitialize a Permutation/1806.cpp

```
class Solution {
public:
    int reinitializePermutation(int n) {
        int ans = 0;
        int i = 1;

        do {
            if (i < n / 2)
                i = i * 2;
            else
                i = (i - n / 2) * 2 + 1;
            ++ans;
        } while (i != 1);

        return ans;
    }
};
```

874. 1807.cpp

Path: LeetCode-main/solutions/1807. Evaluate the Bracket Pairs of a String/1807.cpp

```
class Solution {
public:
    string evaluate(string s, vector<vector<string>>& knowledge) {
        string ans;
        unordered_map<string, string> map;

        for (const vector<string>& list : knowledge)
            map["(" + list[0] + ")"] = list[1];

        for (int i = 0; i < s.length(); ++i) {
            const char c = s[i];
            if (c == '(') {
                const int j = s.find_first_of(')', i);
                const string& key = s.substr(i, j - i + 1);
                ans += map.contains(key) ? map[key] : "?";
                i = j;
            } else {
                ans += c;
            }
        }

        return ans;
    }
};
```

875. 1808.cpp

Path: LeetCode-main/solutions/1808. Maximize Number of Nice Divisors/1808.cpp

```
class Solution {
public:
    int maxNiceDivisors(int primeFactors) {
        if (primeFactors <= 3)
            return primeFactors;
        if (primeFactors % 3 == 0)
            return modPow(3, primeFactors / 3) % kMod;
        if (primeFactors % 3 == 1)
            return 4L * modPow(3, (primeFactors - 4) / 3) % kMod;
        return 2L * modPow(3, (primeFactors - 2) / 3) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

876. 1810.cpp

Path: LeetCode-main/solutions/1810. Minimum Path Cost in a Hidden Grid/1810.cpp

```
/**
 * // This is the GridMaster's API interface.
 * // You should not implement it, or speculate about its implementation
 * class GridMaster {
 * public:
 *     bool canMove(char direction);
 *     int std::move(char direction);
 *     boolean isTarget();
 * };
 */

class Solution {
public:
    int findShortestPath(GridMaster& master) {
        constexpr int m = 100;
        constexpr int startX = m;
        constexpr int startY = m;
        vector<int> target{m * 2, m * 2};
        vector<vector<int>> grid(m * 2, vector<int>(m * 2, -1));
        vector<vector<bool>> seen(m * 2, vector<bool>(m * 2));

        // Build the grid information by DFS.
        dfs(master, grid, startX, startY, target);

        priority_queue<vector<int>, vector<vector<int>>, greater<>> minHeap;
        minHeap.push({0, startX, startY});

        // Find the steps by BFS.
        while (!minHeap.empty()) {
            const vector<int> tuple = minHeap.top();
            const int cost = tuple[0];
            const int i = tuple[1];
            const int j = tuple[2];
            minHeap.pop();
            if (i == target[0] && j == target[1])
                return cost;
            if (seen[i][j])
                continue;
            seen[i][j] = true;
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == 2 * m || y < 0 || y == 2 * m)
                    continue;
                if (seen[x][y] || grid[x][y] == -1)
                    continue;
                const int nextCost = cost + grid[x][y];
                minHeap.push({nextCost, x, y});
            }
        }

        return -1;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
    static constexpr char charTable[4] = {'R', 'D', 'L', 'U'};

    void dfs(GridMaster& master, vector<vector<int>>& grid, int i, int j,
            vector<int>& target) {
        if (master.isTarget()) {
            target[0] = i;
            target[1] = j;
        }

        for (int k = 0; k < 4; ++k) {
            const int x = i + kDirs[k][0];
            const int y = j + kDirs[k][1];
            const char d = charTable[k];
            const char undoD = charTable[(k + 2) % 4];
            if (master.canMove(d) && grid[x][y] == -1) {
                grid[x][y] = master.move(d);
                dfs(master, grid, x, y, target);
                master.move(undoD);
            }
        }
    }
};
```

877. 1812.cpp

Path: LeetCode-main/solutions/1812. Determine Color of a Chessboard Square/1812.cpp

```
class Solution {  
public:  
    bool squareIsWhite(string coordinates) {  
        const char letter = coordinates[0];  
        const char digit = coordinates[1];  
        return letter % 2 != digit % 2;  
    }  
};
```

878. 1813.cpp

Path: LeetCode-main/solutions/1813. Sentence Similarity III/1813.cpp

```
class Solution {
public:
    bool areSentencesSimilar(string sentence1, string sentence2) {
        if (sentence1.length() == sentence2.length())
            return sentence1 == sentence2;

        vector<string> words1 = split(sentence1);
        vector<string> words2 = split(sentence2);
        const int m = words1.size();
        const int n = words2.size();
        if (m > n)
            return areSentencesSimilar(sentence2, sentence1);

        int i = 0; // words1's index
        while (i < m && words1[i] == words2[i])
            ++i;
        while (i < m && words1[i] == words2[i + n - m])
            ++i;

        return i == m;
    }

private:
    vector<string> split(const string& sentence) {
        vector<string> words;
        istringstream iss(sentence);

        for (string s; iss >> s;)
            words.push_back(s);

        return words;
    }
};
```

879. 1814.cpp

Path: LeetCode-main/solutions/1814. Count Nice Pairs in an Array/1814.cpp

```
class Solution {
public:
    int countNicePairs(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        unordered_map<int, long> count;

        for (const int num : nums)
            ++count[num - rev(num)];

        for (const auto& [_, freq] : count) {
            ans += freq * (freq - 1) / 2;
            ans %= kMod;
        }

        return ans;
    }

private:
    int rev(int n) {
        int x = 0;
        while (n > 0) {
            x = x * 10 + (n % 10);
            n /= 10;
        }
        return x;
    }
};
```


880. 1815.cpp

Path: LeetCode-main/solutions/1815. Maximum Number of Groups Getting Fresh Donuts/1815.cpp

```
class Solution {
public:
    int maxHappyGroups(int batchSize, vector<int>& groups) {
        int happy = 0;
        vector<int> freq(batchSize);

        for (int g : groups) {
            g %= batchSize;
            if (g == 0) {
                ++happy;
            } else if (freq[batchSize - g]) {
                --freq[batchSize - g];
                ++happy;
            } else {
                ++freq[g];
            }
        }

        return happy + dp(freq, 0, batchSize);
    }

private:
    map<vector<int>, int> mem;

    // Returns the maximum number of partitions can be formed.
    int dp(const vector<int>& freq, int remainder, const int& batchSize) {
        if (const auto it = mem.find(freq); it != mem.cend())
            return it->second;

        int ans = 0;

        if (ranges::any_of(freq, [](int f) { return f != 0; })) {
            for (int i = 0; i < freq.size(); ++i)
                if (freq[i]) {
                    vector<int> newFreq(freq);
                    --newFreq[i];
                    ans = max(ans, dp(newFreq, (remainder + i) % batchSize, batchSize));
                }
            if (remainder == 0)
                ++ans;
        }

        return mem[freq] = ans;
    }
};
```

881. 1816.cpp

Path: LeetCode-main/solutions/1816. Truncate Sentence/1816.cpp

```
class Solution {
public:
    string truncateSentence(string s, int k) {
        for (int i = 0; i < s.length(); ++i)
            if (s[i] == ' ' && --k == 0)
                return s.substr(0, i);
        return s;
    }
};
```

882. 1817.cpp

Path: LeetCode-main/solutions/1817. Finding the Users Active Minutes/1817.cpp

```
class Solution {
public:
    vector<int> findingUsersActiveMinutes(vector<vector<int>>& logs, int k) {
        vector<int> ans(k);
        unordered_map<int, unordered_set<int>> idToTimes;

        for (const vector<int>& log : logs)
            idToTimes[log[0]].insert(log[1]);

        for (const auto& [_, mins] : idToTimes)
            ++ans[mins.size() - 1];

        return ans;
    }
};
```

883. 1818.cpp

Path: LeetCode-main/solutions/1818. Minimum Absolute Sum Difference/1818.cpp

```
class Solution {
public:
    int minAbsoluteSumDiff(vector<int>& nums1, vector<int>& nums2) {
        constexpr int kMod = 1'000'000'007;
        long sumDiff = 0;
        long maxDecrement = 0;
        set<int> sorted(nums1.begin(), nums1.end());

        for (int i = 0; i < nums1.size(); ++i) {
            const long currDiff = abs(nums1[i] - nums2[i]);
            sumDiff += currDiff;
            const auto it = sorted.lower_bound(nums2[i]);
            if (it != sorted.begin())
                maxDecrement = max(maxDecrement, currDiff - abs(*prev(it) - nums2[i]));
            if (it != sorted.end())
                maxDecrement = max(maxDecrement, currDiff - abs(*it - nums2[i]));
        }

        return (sumDiff - maxDecrement) % kMod;
    }
};
```

884. 1819.cpp

Path: LeetCode-main/solutions/1819. Number of Different Subsequences GCDs/1819.cpp

```
class Solution {
public:
    int countDifferentSubsequenceGCDs(vector<int>& nums) {
        const int maxNum = ranges::max(nums);
        int ans = 0;
        // factor[i] := the GCD of numbers having factor i
        vector<int> factor(maxNum + 1);

        for (const int num : nums)
            for (int i = 1; i * i <= num; ++i)
                if (num % i == 0) {
                    const int j = num / i;
                    factor[i] = __gcd(factor[i], num);
                    factor[j] = __gcd(factor[j], num);
                }

        for (int i = 1; i <= maxNum; ++i)
            if (factor[i] == i)
                ++ans;

        return ans;
    }
};
```

885. 1820.cpp

Path: LeetCode-main/solutions/1820. Maximum Number of Accepted Invitations/1820.cpp

```
class Solution {
public:
    int maximumInvitations(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        int ans = 0;
        vector<int> mates(n, -1); // mates[i] := the i-th girl's mate

        for (int i = 0; i < m; ++i)
            if (canInvite(grid, i, vector<bool>(n), mates))
                ++ans;

        return ans;
    }

private:
    // Returns true if the i-th boy can make an invitation.
    bool canInvite(const vector<vector<int>>& grid, int i, vector<bool>&& seen,
                   vector<int>& mates) {
        // The i-th boy asks each girl.
        for (int j = 0; j < seen.size(); ++j) {
            if (!grid[i][j] || seen[j])
                continue;
            seen[j] = true;
            if (mates[j] == -1 || canInvite(grid, mates[j], std::move(seen), mates)) {
                mates[j] = i; // Match the j-th girl with i-th boy.
                return true;
            }
        }
        return false;
    }
};
```

886. 1822.cpp

Path: LeetCode-main/solutions/1822. Sign of the Product of an Array/1822.cpp

```
class Solution {
public:
    int arraySign(vector<int>& nums) {
        int sign = 1;

        for (const int num : nums) {
            if (num == 0)
                return 0;
            if (num < 0)
                sign = -sign;
        }

        return sign;
    }
};
```

887. 1823-2.cpp

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823-2.cpp

```
class Solution {
public:
    int findTheWinner(int n, int k) {
        // Converts back to 1-indexed.
        return f(n, k) + 1;
    }

    // e.g. n = 4, k = 2.
    // By using 0-indexed notation, we have the following circle:
    //
    // 0 -> 1 -> 2 -> 3 -> 0
    //      x
    //      0 -> 1 -> 2 -> 0
    //
    // After the first round, 1 is removed.
    // So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
    // Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
    // By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
private:
    int f(int n, int k) {
        if (n == 1)
            return 0;
        return (f(n - 1, k) + k) % n;
    }
};
```


888. 1823-3.cpp

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823-3.cpp

```
class Solution {
public:
    int findTheWinner(int n, int k) {
        // Converts back to 1-indexed.
        return f(n, k) + 1;
    }

    // e.g. n = 4, k = 2.
    // By using 0-indexed notation, we have the following circle:
    //
    // 0 -> 1 -> 2 -> 3 -> 0
    //      x
    //      0 -> 1 -> 2 -> 0
    //
    // After the first round, 1 is removed.
    // So, 2 becomes 0, 3 becomes 1, and 0 becomes 2.
    // Let's denote that oldIndex = f(n, k) and newIndex = f(n - 1, k).
    // By observation, we know f(n, k) = (f(n - 1, k) + k) % n.
private:
    int f(int n, int k) {
        int ans = 0; // f(1, k)
        // Computes f(i, k) based on f(i - 1, k).
        for (int i = 2; i <= n; ++i)
            ans = (ans + k) % i;
        return ans;
    }
};
```

889. 1823.cpp

Path: LeetCode-main/solutions/1823. Find the Winner of the Circular Game/1823.cpp

```
class Solution {
public:
    int findTheWinner(int n, int k) {
        // friends[i] := true if i-th friend is left
        vector<bool> friends(n);

        int friendCount = n;
        int fp = 0; // friends' index

        while (friendCount > 1) {
            for (int i = 0; i < k; ++i, ++fp)
                while (friends[fp % n]) // The friend is not there.
                    ++fp; // Point to the next one.
            friends[(fp - 1) % n] = true;
            --friendCount;
        }

        const auto it =
            find_if(friends.begin(), friends.end(), [](int f) { return !f; });
        return distance(friends.begin(), it) + 1;
    }
};
```

890. 1824.cpp

Path: LeetCode-main/solutions/1824. Minimum Sideway Jumps/1824.cpp

```
class Solution {
public:
    int minSideJumps(vector<int>& obstacles) {
        constexpr int kInf = 1'000'000;
        // dp[i] := the minimum jump to reach the i-th lane
        vector<int> dp{kInf, 1, 0, 1};

        for (const int obstacle : obstacles) {
            if (obstacle > 0)
                dp[obstacle] = kInf;
            for (int i = 1; i <= 3; ++i) // the current
                if (i != obstacle)
                    for (int j = 1; j <= 3; ++j) // the previous
                        dp[i] = min({dp[i], dp[j] + (i == j ? 0 : 1)});
        }

        return ranges::min(dp);
    }
};
```

891. 1825-2.cpp

Path: LeetCode-main/solutions/1825. Finding MK Average/1825-2.cpp

```
struct MyMap {
    map<int, int> map;
    int size = 0;
    long sum = 0;
};

class MKAverage {
public:
    MKAverage(int m, int k) : m(m), k(k), kMidSize(m - 2 * k) {}

    void addElement(int num) {
        q.push(num);
        add(num);

        if (q.size() > m) {
            const int removed = q.front();
            q.pop();
            remove(removed);
        }
    }

    int calculateMKAverage() {
        return q.size() == m ? mid.sum / kMidSize : -1;
    }

private:
    const int m;
    const int k;
    const int kMidSize;
    queue<int> q;
    MyMap top;
    MyMap mid;
    MyMap bot;

    void add(int num) {
        add(bot, num);
        if (bot.size > k)
            add(mid, remove(bot, bot.map.rbegin()->first));
        if (mid.size > kMidSize)
            add(top, remove(mid, mid.map.rbegin()->first));
    }

    void remove(int num) {
        if (bot.map.contains(num))
            remove(bot, num);
        else if (mid.map.contains(num))
            remove(mid, num);
        else
            remove(top, num);

        if (bot.size < k)
            add(bot, remove(mid, mid.map.begin()->first));
        if (mid.size < kMidSize)
            add(mid, remove(top, top.map.begin()->first));
    }

    void add(MyMap& m, int num) {
        ++m.map[num];
        ++m.size;
        m.sum += num;
    }

    int remove(MyMap& m, int num) {
        if (--m.map[num] == 0)
            m.map.erase(num);
        --m.size;
        m.sum -= num;
        return num;
    }
};
```

892. 1825.cpp

Path: LeetCode-main/solutions/1825. Finding MK Average/1825.cpp

```
class MKAverage {
public:
    MKAverage(int m, int k) : m(m), k(k) {}

    void addElement(int num) {
        q.push(num);
        add(mid, num);
        midSum += num;

        if (q.size() > m) {
            const int removed = q.front();
            q.pop();
            if (top.contains(removed)) {
                remove(top, removed);
                --topSize;
            } else if (mid.contains(removed)) {
                remove(mid, removed);
                midSum -= removed;
            } else {
                remove(bot, removed);
                --botSize;
            }
        }

        // Move item(s) from `mid` to `top` to fill k slots.
        while (!mid.empty() && topSize < k) {
            midSum -= mid.rbegin()->first;
            add(top, remove(mid, mid.rbegin()->first));
            ++topSize;
        }

        // Rebalance `mid` and `top`.
        while (!mid.empty() && mid.rbegin()->first > top.begin()->first) {
            midSum -= mid.rbegin()->first;
            midSum += top.begin()->first;
            add(top, remove(mid, mid.rbegin()->first));
            add(mid, remove(top, top.begin()->first));
        }

        // Move item(s) from `mid` to `bot` to fill k slots.
        while (!mid.empty() && botSize < k) {
            midSum -= mid.begin()->first;
            add(bot, remove(mid, mid.begin()->first));
            ++botSize;
        }

        // Move item(s) from `mid` to `bot` to fill k slots.
        while (!mid.empty() && mid.begin()->first < bot.rbegin()->first) {
            midSum -= mid.begin()->first;
            midSum += bot.rbegin()->first;
            add(bot, remove(mid, mid.begin()->first));
            add(mid, remove(bot, bot.rbegin()->first));
        }
    }

    int calculateMKAverage() {
        return q.size() == m ? midSum / (m - 2 * k) : -1;
    }

private:
    const int m;
    const int k;
    queue<int> q;
    map<int, int> top;
    map<int, int> mid;
    map<int, int> bot;
    int topSize = 0;
    int botSize = 0;
    long midSum = 0;

    void add(map<int, int>& map, int num) {
        ++map[num];
    }

    int remove(map<int, int>& map, int num) {
        if (--map[num] == 0)
            map.erase(num);
        return num;
    }
};
```

893. 1826.cpp

Path: LeetCode-main/solutions/1826. Faulty Sensor/1826.cpp

```
class Solution {
public:
    int badSensor(vector<int>& sensor1, vector<int>& sensor2) {
        const bool oneDefect = canReplace(sensor2, sensor1);
        const bool twoDefect = canReplace(sensor1, sensor2);
        if (oneDefect && twoDefect)
            return -1;
        if (!oneDefect && !twoDefect)
            return -1;
        return oneDefect ? 1 : 2;
    }

private:
    bool canReplace(const vector<int>& A, const vector<int>& B) {
        int i = 0; // A's index
        int j = 0; // B's index
        int droppedValue = -1;

        while (i < A.size())
            if (A[i] == B[j]) {
                ++i;
                ++j;
            } else {
                droppedValue = A[i];
                ++i;
            }

        return j == B.size() - 1 && B[j] != droppedValue;
    }
};
```

894. 1827.cpp

Path: LeetCode-main/solutions/1827. Minimum Operations to Make the Array Increasing/1827.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums) {
        int ans = 0;
        int last = 0;

        for (const int num : nums) {
            ans += max(0, last - num + 1);
            last = max(num, last + 1);
        }

        return ans;
    }
};
```

895. 1828.cpp

Path: LeetCode-main/solutions/1828. Queries on Number of Points Inside a Circle/1828.cpp

```
class Solution {
public:
    vector<int> countPoints(vector<vector<int>>& points,
                           vector<vector<int>>& queries) {
        vector<int> ans;

        for (const vector<int>& query : queries) {
            const int xj = query[0];
            const int yj = query[1];
            const int rj = query[2];
            int count = 0;
            for (const vector<int>& point : points) {
                const int xi = point[0];
                const int yi = point[1];
                if (squared(xi - xj) + squared(yi - yj) <= squared(rj))
                    ++count;
            }
            ans.push_back(count);
        }

        return ans;
    }

private:
    int squared(int x) {
        return x * x;
    }
};
```


896. 1829.cpp

Path: LeetCode-main/solutions/1829. Maximum XOR for Each Query/1829.cpp

```
class Solution {
public:
    vector<int> getMaximumXor(vector<int>& nums, int maximumBit) {
        const int mx = (1 << maximumBit) - 1;
        vector<int> ans;
        int xors = 0;

        for (const int num : nums) {
            xors ^= num;
            ans.push_back(xors ^ mx);
        }

        ranges::reverse(ans);
        return ans;
    }
};
```

897. 1830.cpp

Path: LeetCode-main/solutions/1830. Minimum Number of Operations to Make String Sorted/1830.cpp

```
class Solution {
public:
    int makeStringSorted(string s) {
        const int n = s.length();
        const auto [fact, invFact] = getFactAndInvFact(n);
        int ans = 0;
        vector<int> count(26);

        for (int i = n - 1; i >= 0; --i) {
            const int order = s[i] - 'a';
            ++count[order];
            long perm = accumulate(count.begin(), count.begin() + order, 0) *
                           fact[n - 1 - i] % kMod;
            for (int j = 0; j < 26; ++j)
                perm = perm * invFact[count[j]] % kMod;
            ans = (ans + perm) % kMod;
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    pair<vector<long>, vector<long>> getFactAndInvFact(int n) {
        vector<long> fact(n + 1);
        vector<long> invFact(n + 1);
        vector<long> inv(n + 1);
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = kMod - kMod / i * inv[kMod % i] % kMod;
            fact[i] = fact[i - 1] * i % kMod;
            invFact[i] = invFact[i - 1] * inv[i] % kMod;
        }
        return {fact, invFact};
    }
};
```

898. 1832-2.cpp

Path: LeetCode-main/solutions/1832. Check if the Sentence Is Pangram/1832-2.cpp

```
class Solution {
public:
    bool checkIfPangram(string sentence) {
        int seen = 0;

        for (const char c : sentence)
            seen |= 1 << c - 'a';

        return seen == (1 << 26) - 1;
    }
};
```

899. 1832.cpp

Path: LeetCode-main/solutions/1832. Check if the Sentence Is Pangram/1832.cpp

```
class Solution {
public:
    bool checkIfPangram(string sentence) {
        return unordered_set(sentence.begin(), sentence.end()).size() == 26;
    }
};
```

900. 1833.cpp

Path: LeetCode-main/solutions/1833. Maximum Ice Cream Bars/1833.cpp

```
class Solution {
public:
    int maxIceCream(vector<int>& costs, int coins) {
        ranges::sort(costs);

        for (int i = 0; i < costs.size(); ++i)
            if (coins >= costs[i])
                coins -= costs[i];
            else
                return i;

        return costs.size();
    }
};
```

901. 1834.cpp

Path: LeetCode-main/solutions/1834. Single-Threaded CPU/1834.cpp

```
class Solution {
public:
    vector<int> getOrder(vector<vector<int>>& tasks) {
        const int n = tasks.size();

        // Add index information.
        for (int i = 0; i < n; ++i)
            tasks[i].push_back(i);

        vector<int> ans;
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> minHeap;
        int i = 0; // tasks' index
        long time = 0; // the current time

        ranges::sort(tasks);
        while (i < n || !minHeap.empty()) {
            if (minHeap.empty())
                time = max(time, static_cast<long>(tasks[i][0]));
            while (i < n && time >= tasks[i][0]) {
                minHeap.emplace(tasks[i][1], tasks[i][2]);
                ++i;
            }
            const auto [procTime, index] = minHeap.top();
            minHeap.pop();
            time += procTime;
            ans.push_back(index);
        }

        return ans;
    }
};
```

902. 1835.cpp

Path: LeetCode-main/solutions/1835. Find XOR Sum of All Pairs Bitwise AND/1835.cpp

```
class Solution {
public:
    int getXORSum(vector<int>& arr1, vector<int>& arr2) {
        const int xors1 = accumulate(arr1.begin(), arr1.end(), 0, bit_xor<>());
        const int xors2 = accumulate(arr2.begin(), arr2.end(), 0, bit_xor<>());
        return xors1 & xors2;
    }
};
```

903. 1836.cpp

Path: LeetCode-main/solutions/1836. Remove Duplicates From an Unsorted Linked List/1836.cpp

```
class Solution {
public:
    ListNode* deleteDuplicatesUnsorted(ListNode* head) {
        ListNode dummy(0, head);
        unordered_map<int, int> count;

        for (ListNode* curr = head; curr; curr = curr->next)
            ++count[curr->val];

        ListNode* curr = &dummy;

        while (curr != nullptr) {
            while (curr->next && count.contains(curr->next->val) &&
                count[curr->next->val] > 1)
                curr->next = curr->next->next;
            curr = curr->next;
        }

        return dummy.next;
    }
};
```


904. 1837.cpp

Path: LeetCode-main/solutions/1837. Sum of Digits in Base K/1837.cpp

```
class Solution {
public:
    int sumBase(int n, int k) {
        int ans = 0;

        while (n > 0) {
            ans += n % k;
            n /= k;
        }

        return ans;
    }
};
```

905. 1838.cpp

Path: LeetCode-main/solutions/1838. Frequency of the Most Frequent Element/1838.cpp

```
class Solution {
public:
    int maxFrequency(vector<int>& nums, int k) {
        int ans = 0;
        long sum = 0;

        ranges::sort(nums);

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            sum += nums[r];
            while (sum + k < static_cast<long>(nums[r]) * (r - l + 1))
                sum -= nums[l++];
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

906. 1839.cpp

Path: LeetCode-main/solutions/1839. Longest Substring Of All Vowels in Order/1839.cpp

```
class Solution {
public:
    int longestBeautifulSubstring(string word) {
        int ans = 0;
        int count = 1;

        for (int l = 0, r = 1; r < word.length(); ++r) {
            const char curr = word[r];
            const char prev = word[r - 1];
            if (curr >= prev) {
                if (curr > prev)
                    ++count;
                if (count == 5)
                    ans = max(ans, r - l + 1);
            } else {
                count = 1;
                l = r;
            }
        }

        return ans;
    }
};
```

907. 1840.cpp

Path: LeetCode-main/solutions/1840. Maximum Building Height/1840.cpp

```
class Solution {
public:
    int maxBuilding(int n, vector<vector<int>>& restrictions) {
        vector<vector<int>> A(restrictions);

        A.push_back({1, 0});
        A.push_back({n, n - 1});
        ranges::sort(A);

        for (int i = 1; i < A.size(); ++i) {
            const int dist = A[i][0] - A[i - 1][0];
            A[i][1] = min(A[i][1], A[i - 1][1] + dist);
        }

        for (int i = A.size() - 2; i >= 0; --i) {
            const int dist = A[i + 1][0] - A[i][0];
            A[i][1] = min(A[i][1], A[i + 1][1] + dist);
        }

        int ans = 0;

        for (int i = 1; i < A.size(); ++i) {
            const int l = A[i - 1][0];
            const int r = A[i][0];
            const int hL = A[i - 1][1];
            const int hR = A[i][1];
            ans = max(ans, max(hL, hR) + (r - l - abs(hL - hR)) / 2);
        }

        return ans;
    }
};
```

908. 1842.cpp

Path: LeetCode-main/solutions/1842. Next Palindrome Using Same Digits/1842.cpp

```
class Solution {
public:
    string nextPalindrome(string num) {
        const int n = num.length();
        vector<int> arr(n / 2);

        for (int i = 0; i < arr.size(); ++i)
            arr[i] = num[i] - '0';

        if (!ranges::next_permutation(arr).found)
            return "";

        string s;

        for (const int a : arr)
            s += '0' + a;

        if (n % 2 == 1)
            return s + num[n / 2] + reversed(s);
        return s + reversed(s);
    }

private:
    string reversed(const string& s) {
        return {s.rbegin(), s.rend()};
    }
};
```

909. 1844.cpp

Path: LeetCode-main/solutions/1844. Replace All Digits with Characters/1844.cpp

```
class Solution {
public:
    string replaceDigits(string s) {
        for (int i = 1; i < s.length(); i += 2)
            s[i] += s[i - 1] - '0';
        return s;
    }
};
```

910. 1845.cpp

Path: LeetCode-main/solutions/1845. Seat Reservation Manager/1845.cpp

```
class SeatManager {
public:
    SeatManager(int n) {}

    int reserve() {
        if (minHeap.empty())
            return ++num;

        const int minNum = minHeap.top();
        minHeap.pop();
        return minNum;
    }

    void unreserve(int seatNumber) {
        minHeap.push(seatNumber);
    }

private:
    priority_queue<int, vector<int>, greater<>> minHeap;
    int num = 0;
};
```

911. 1846.cpp

Path: LeetCode-main/solutions/1846. Maximum Element After Decreasing and Rearranging/1846.cpp

```
class Solution {
public:
    int maximumElementAfterDecrementingAndRearranging(vector<int>& arr) {
        ranges::sort(arr);
        arr[0] = 1;

        for (int i = 1; i < arr.size(); ++i)
            arr[i] = min(arr[i], arr[i - 1] + 1);

        return arr.back();
    }
};
```


912. 1847.cpp

Path: LeetCode-main/solutions/1847. Closest Room/1847.cpp

```
class Solution {
public:
    vector<int> closestRoom(vector<vector<int>>& rooms,
                           vector<vector<int>>& queries) {
        vector<int> ans(queries.size());
        set<int> roomIds;

        for (int i = 0; i < queries.size(); ++i)
            queries[i].push_back(i);

        ranges::sort(rooms, ranges::greater{},
                     [](const vector<int>& room) { return room[1]; });
        ranges::sort(queries, ranges::greater{},
                     [](const vector<int>& query) { return query[1]; });

        int i = 0; // rooms' index
        for (const vector<int>& query : queries) {
            while (i < rooms.size() && rooms[i][1] >= query[1])
                roomIds.insert(rooms[i++][0]);
            ans[query[2]] = searchClosestRoomId(roomIds, query[0]);
        }

        return ans;
    }

private:
    int searchClosestRoomId(set<int>& roomIds, int preferred) {
        const auto it = roomIds.lower_bound(preferred);
        const int id1 = it == roomIds.cbegin() ? -1 : *(prev(it));
        const int id2 = it == roomIds.cend() ? -1 : *it;
        if (id1 == -1)
            return id2;
        if (id2 == -1)
            return id1;
        if (abs(preferred - id1) <= abs(preferred - id2))
            return id1;
        return id2;
    }
};
```

913. 1848.cpp

Path: LeetCode-main/solutions/1848. Minimum Distance to the Target Element/1848.cpp

```
class Solution {
public:
    int getMinDistance(vector<int>& nums, int target, int start) {
        int ans = INT_MAX;

        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == target)
                ans = min(ans, abs(i - start));

        return ans;
    }
};
```

914. 1849.cpp

Path: LeetCode-main/solutions/1849. Splitting a String Into Descending Consecutive Values/1849.cpp

```
class Solution {
public:
    bool splitString(string s) {
        return isValid(s, 0, -1, 0);
    }

private:
    bool isValid(const string& s, int start, long prev, int segment) {
        if (start == s.length() && segment > 1)
            return true;

        long curr = 0;
        for (int i = start; i < s.length(); ++i) {
            curr = curr * 10 + s[i] - '0';
            if (curr > 999999999L)
                return false;
            if ((prev == -1 || curr == prev - 1) &&
                isValid(s, i + 1, curr, segment + 1)) {
                return true;
            }
        }

        return false;
    }
};
```

915. 1850.cpp

Path: LeetCode-main/solutions/1850. Minimum Adjacent Swaps to Reach the Kth Smallest Number/1850.cpp

```
class Solution {
public:
    int getMinSwaps(string num, int k) {
        string perm = num;

        while (k-- > 0)
            next_permutation(perm.begin(), perm.end());

        return countSteps(num, perm);
    }

private:
    int countSteps(const string& A, string& B) {
        int count = 0;

        for (int i = 0, j = 0; i < A.length(); ++i) {
            j = i;
            while (A[i] != B[j])
                ++j;
            while (i < j) {
                swap(B[j], B[j - 1]);
                --j;
                ++count;
            }
        }

        return count;
    }
};
```

916. 1851.cpp

Path: LeetCode-main/solutions/1851. Minimum Interval to Include Each Query/1851.cpp

```
struct T {
    int size;
    int right;
};

class Solution {
public:
    vector<int> minInterval(vector<vector<int>>& intervals,
                           vector<int>& queries) {
        vector<int> ans(queries.size(), -1);
        auto compare = [](const T& a, const T& b) { return a.size > b.size; };
        priority_queue<T, vector<T>, decltype(compare)> minHeap(compare);
        vector<vector<int>> qs;

        for (int i = 0; i < queries.size(); ++i)
            qs.push_back({queries[i], i});

        ranges::sort(intervals);
        ranges::sort(qs);

        int i = 0; // intervals' index
        for (const vector<int>& q : qs) {
            while (i < intervals.size() && intervals[i][0] <= q[0]) {
                minHeap.emplace(intervals[i][1] - intervals[i][0] + 1, intervals[i][1]);
                ++i;
            }
            while (!minHeap.empty() && minHeap.top().right < q[0])
                minHeap.pop();
            if (!minHeap.empty())
                ans[q[1]] = minHeap.top().size;
        }

        return ans;
    }
};
```

917. 1852.cpp

Path: LeetCode-main/solutions/1852. Distinct Numbers in Each Subarray/1852.cpp

```
class Solution {
public:
    vector<int> distinctNumbers(vector<int>& nums, int k) {
        vector<int> ans;
        int distinct = 0;
        unordered_map<int, int> count;

        for (int i = 0; i < nums.size(); ++i) {
            if (++count[nums[i]] == 1)
                ++distinct;
            if (i >= k && --count[nums[i - k]] == 0)
                --distinct;
            if (i >= k - 1)
                ans.push_back(distinct);
        }

        return ans;
    }
};
```

918. 1854.cpp

Path: LeetCode-main/solutions/1854. Maximum Population Year/1854.cpp

```
class Solution {
public:
    int maximumPopulation(vector<vector<int>>& logs) {
        constexpr int kMinYear = 1950;
        constexpr int kMaxYear = 2050;
        int ans = 0;
        int maxPopulation = 0;
        int runningPopulation = 0;
        // population[i] := the population of year i
        vector<int> population(kMaxYear + 1);

        for (const vector<int>& log : logs) {
            const int birth = log[0];
            const int death = log[1];
            ++population[birth];
            --population[death];
        }

        for (int year = kMinYear; year <= kMaxYear; ++year) {
            runningPopulation += population[year];
            if (runningPopulation > maxPopulation) {
                maxPopulation = runningPopulation;
                ans = year;
            }
        }

        return ans;
    }
};
```

919. 1855-2.cpp

Path: LeetCode-main/solutions/1855. Maximum Distance Between a Pair of Values/1855-2.cpp

```
class Solution {
public:
    int maxDistance(vector<int>& nums1, vector<int>& nums2) {
        int i = 0;
        int j = 0;

        for (; i < nums1.size() && j < nums2.size(); ++j)
            if (nums1[i] > nums2[j])
                ++i;

        return i == j ? 0 : j - i - 1;
    }
};
```


920. 1855.cpp

Path: LeetCode-main/solutions/1855. Maximum Distance Between a Pair of Values/1855.cpp

```
class Solution {
public:
    int maxDistance(vector<int>& nums1, vector<int>& nums2) {
        int ans = 0;
        int i = 0;
        int j = 0;

        while (i < nums1.size() && j < nums2.size())
            if (nums1[i] > nums2[j])
                ++i;
            else
                ans = max(ans, j++ - i);

        return ans;
    }
};
```

921. 1856.cpp

Path: LeetCode-main/solutions/1856. Maximum Subarray Min-Product/1856.cpp

```
class Solution {
public:
    int maxSumMinProduct(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        stack<int> stack;
        vector<long> prefix(nums.size() + 1);

        for (int i = 0; i < nums.size(); ++i)
            prefix[i + 1] = prefix[i] + nums[i];

        for (int i = 0; i <= nums.size(); ++i) {
            while (!stack.empty() &&
                (i == nums.size() || nums[stack.top()] > nums[i])) {
                const int minVal = nums[stack.top()];
                stack.pop();
                const long sum =
                    stack.empty() ? prefix[i] : prefix[i] - prefix[stack.top() + 1];
                ans = max(ans, minVal * sum);
            }
            stack.push(i);
        }

        return ans % kMod;
    }
};
```

922. 1857.cpp

Path: LeetCode-main/solutions/1857. Largest Color Value in a Directed Graph/1857.cpp

```
class Solution {
public:
    int largestPathValue(string colors, vector<vector<int>>& edges) {
        const int n = colors.length();
        int ans = 0;
        int processed = 0;
        vector<vector<int>> graph(n);
        vector<int> inDegrees(n);
        queue<int> q;
        vector<vector<int>> count(n, vector<int>(26));

        // Build the graph.
        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        while (!q.empty()) {
            const int out = q.front();
            q.pop();
            ++processed;
            ans = max(ans, ++count[out][colors[out] - 'a']);
            for (const int in : graph[out]) {
                for (int i = 0; i < 26; ++i)
                    count[in][i] = max(count[in][i], count[out][i]);
                if (--inDegrees[in] == 0)
                    q.push(in);
            }
        }

        return processed == n ? ans : -1;
    }
};
```

923. 1858.cpp

Path: LeetCode-main/solutions/1858. Longest Word With All Prefixes/1858.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    bool isWord = false;
    TrieNode() : children(26) {}
};

class Solution {
public:
    string longestWord(vector<string>& words) {
        string ans;

        for (const string& word : words)
            insert(word);

        for (const string& word : words) {
            if (!allPrefixed(word))
                continue;
            if (ans.length() < word.length() ||
                (ans.length() == word.length() && ans > word))
                ans = word;
        }

        return ans;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
        node->isWord = true;
    }

    bool allPrefixed(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            node = node->children[i];
            if (!node->isWord)
                return false;
        }
        return true;
    }
};
```

924. 1859.cpp

Path: LeetCode-main/solutions/1859. Sorting the Sentence/1859.cpp

```
class Solution {
public:
    string sortSentence(string s) {
        vector<string> words;
        istringstream iss(s);
        string word;

        while (iss >> word)
            words.push_back(word);

        ranges::sort(words, ranges::less(),
                    [](const string& word) { return word.back(); });

        string ans = trim(words[0]);

        for (int i = 1; i < words.size(); ++i)
            ans += " " + trim(words[i]);

        return ans;
    }

private:
    string trim(const string& s) {
        return s.substr(0, s.length() - 1);
    }
};
```

925. 186.cpp

Path: LeetCode-main/solutions/186. Reverse Words in a String II/186.cpp

```
class Solution {
public:
    void reverseWords(vector<char>& s) {
        ranges::reverse(s); // Reverse the whole string.
        reverseWords(s, s.size()); // Reverse each word.
    }

private:
    void reverseWords(vector<char>& s, int n) {
        int i = 0;
        int j = 0;

        while (i < n) {
            while (i < j || i < n && s[i] == ' ') // Skip the spaces.
                ++i;
            while (j < i || j < n && s[j] != ' ') // Skip the spaces.
                ++j;
            reverse(s.begin() + i, s.begin() + j); // Reverse the word.
        }
    }
};
```

926. 1860.cpp

Path: LeetCode-main/solutions/1860. Incremental Memory Leak/1860.cpp

```
class Solution {
public:
    vector<int> memLeak(int memory1, int memory2) {
        int i = 1;

        while (memory1 >= i || memory2 >= i) {
            if (memory1 >= memory2)
                memory1 -= i;
            else
                memory2 -= i;
            ++i;
        }

        return {i, memory1, memory2};
    }
};
```

927. 1861.cpp

Path: LeetCode-main/solutions/1861. Rotating the Box/1861.cpp

```
class Solution {
public:
    vector<vector<char>> rotateTheBox(vector<vector<char>>& box) {
        const int m = box.size();
        const int n = box[0].size();
        vector<vector<char>> ans(n, vector<char>(m, '.'));

        for (int i = 0; i < m; ++i)
            for (int j = n - 1, k = n - 1; j >= 0; --j)
                if (box[i][j] != '.') {
                    if (box[i][j] == '*')
                        k = j;
                    ans[k--][m - i - 1] = box[i][j];
                }

        return ans;
    }
};
```


928. 1862.cpp

Path: LeetCode-main/solutions/1862. Sum of Floored Pairs/1862.cpp

```
class Solution {
public:
    int sumOfFlooredPairs(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        const int kMax = ranges::max(nums);
        long ans = 0;
        // count[i] := the number of `nums` <= i
        vector<int> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        for (int i = 1; i <= kMax; ++i)
            count[i] += count[i - 1];

        for (int i = 1; i <= kMax; ++i)
            if (count[i] > count[i - 1]) {
                long sum = 0;
                for (int j = 1; i * j <= kMax; ++j) {
                    const int lo = i * j - 1;
                    const int hi = i * (j + 1) - 1;
                    sum += (count[min(hi, kMax)] - count[lo]) * j;
                }
                ans += sum * (count[i] - count[i - 1]);
                ans %= kMod;
            }

        return ans;
    }
};
```

929. 1863-2.cpp

Path: LeetCode-main/solutions/1863. Sum of All Subset XOR Totals/1863-2.cpp

```
class Solution {
public:
    int subsetXORSum(vector<int>& nums) {
        return accumulate(nums.begin(), nums.end(), 0, bit_or<>())
            << nums.size() - 1;
    }
};
```

930. 1863.cpp

Path: LeetCode-main/solutions/1863. Sum of All Subset XOR Totals/1863.cpp

```
class Solution {
public:
    int subsetXORSum(vector<int>& nums) {
        return dfs(nums, 0, 0);
    }

private:
    int dfs(const vector<int>& nums, int i, int xors) {
        if (i == nums.size())
            return xors;

        const int x = dfs(nums, i + 1, xors);
        const int y = dfs(nums, i + 1, nums[i] ^ xors);
        return x + y;
    }
};
```

931. 1864.cpp

Path: LeetCode-main/solutions/1864. Minimum Number of Swaps to Make the Binary String Alternating/1864.cpp

```
class Solution {
public:
    int minSwaps(string s) {
        const int ones = ranges::count(s, '1');
        const int zeros = s.length() - ones;
        if (abs(ones - zeros) > 1)
            return -1;
        if (ones > zeros)
            return countSwaps(s, '1');
        if (zeros > ones)
            return countSwaps(s, '0');
        return min(countSwaps(s, '1'), countSwaps(s, '0'));
    }

private:
    int countSwaps(const string& s, char curr) {
        int swaps = 0;
        for (const char c : s) {
            if (c != curr)
                ++swaps;
            curr ^= 1;
        }
        return swaps / 2;
    }
};
```

932. 1865.cpp

Path: LeetCode-main/solutions/1865. Finding Pairs With a Certain Sum/1865.cpp

```
class FindSumPairs {
public:
    FindSumPairs(vector<int>& nums1, vector<int>& nums2)
        : nums1(nums1), nums2(nums2) {
        for (const int num : nums2)
            ++count2[num];
    }

    void add(int index, int val) {
        --count2[nums2[index]];
        nums2[index] += val;
        ++count2[nums2[index]];
    }

    int count(int tot) {
        int ans = 0;
        for (const int num : nums1) {
            const int target = tot - num;
            if (const auto it = count2.find(target); it != count2.cend())
                ans += it->second;
        }
        return ans;
    }

private:
    vector<int> nums1;
    vector<int> nums2;
    unordered_map<int, int> count2;
};
```

933. 1866.cpp

Path: LeetCode-main/solutions/1866. Number of Ways to Rearrange Sticks With K Sticks Visible/1866.cpp

```
class Solution {
public:
    long rearrangeSticks(int n, int k) {
        if (n == k)
            return 1;
        if (k == 0)
            return 0;
        if (dp[n][k])
            return dp[n][k];
        return dp[n][k] = (rearrangeSticks(n - 1, k - 1) +
                           rearrangeSticks(n - 1, k) * (n - 1)) %
                           kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;
    vector<vector<int>> dp = vector<vector<int>>(1001, vector<int>(1001));
};
```

934. 1868.cpp

Path: LeetCode-main/solutions/1868. Product of Two Run-Length Encoded Arrays/1868.cpp

```
class Solution {
public:
    vector<vector<int>> findRLEArray(vector<vector<int>>& encoded1,
                                   vector<vector<int>>& encoded2) {

        vector<vector<int>> ans;
        int i = 0; // encoded1's index
        int j = 0; // encoded2's index

        while (i < encoded1.size() && j < encoded2.size()) {
            const int mult = encoded1[i][0] * encoded2[j][0];
            const int minFreq = min(encoded1[i][1], encoded2[j][1]);
            if (!ans.empty() && mult == ans.back()[0])
                ans.back()[1] += minFreq;
            else
                ans.push_back({mult, minFreq});
            encoded1[i][1] -= minFreq;
            encoded2[j][1] -= minFreq;
            if (encoded1[i][1] == 0)
                ++i;
            if (encoded2[j][1] == 0)
                ++j;
        }

        return ans;
    }
};
```

935. 1869.cpp

Path: LeetCode-main/solutions/1869. Longer Contiguous Segments of Ones than Zeros/1869.cpp

```
class Solution {
public:
    bool checkZeroOnes(string s) {
        int longestOnes = 0;
        int longestZeros = 0;
        int currentOnes = 0;
        int currentZeros = 0;

        for (const char c : s)
            if (c == '0') {
                currentOnes = 0;
                longestZeros = max(longestZeros, ++currentZeros);
            } else {
                currentZeros = 0;
                longestOnes = max(longestOnes, ++currentOnes);
            }

        return longestOnes > longestZeros;
    }
};
```


936. 187.cpp

Path: LeetCode-main/solutions/187. Repeated DNA Sequences/187.cpp

```
class Solution {
public:
    vector<string> findRepeatedDnaSequences(string s) {
        unordered_set<string> ans;
        unordered_set<string_view> seen;
        const string_view sv(s);

        for (int i = 0; i + 10 <= s.length(); ++i) {
            if (seen.contains(sv.substr(i, 10)))
                ans.insert(s.substr(i, 10));
            seen.insert(sv.substr(i, 10));
        }

        return {ans.begin(), ans.end()};
    }
};
```

937. 1870.cpp

Path: LeetCode-main/solutions/1870. Minimum Speed to Arrive on Time/1870.cpp

```
class Solution {
public:
    int minSpeedOnTime(vector<int>& dist, double hour) {
        int ans = -1;
        int l = 1;
        int r = 10'000'000;

        while (l <= r) {
            const int m = (l + r) / 2;
            if (time(dist, hour, m) > hour) {
                l = m + 1;
            } else {
                ans = m;
                r = m - 1;
            }
        }

        return ans;
    }

private:
    double time(const vector<int>& dist, double hour, int speed) {
        double sum = 0;
        for (int i = 0; i < dist.size() - 1; ++i)
            sum += ceil((double)dist[i] / speed);
        return sum + (double)dist.back() / speed;
    }
};
```

938. 1871.cpp

Path: LeetCode-main/solutions/1871. Jump Game VII/1871.cpp

```
class Solution {
public:
    bool canReach(string s, int minJump, int maxJump) {
        int count = 0;
        vector<bool> dp(s.length());
        dp[0] = true;

        for (int i = minJump; i < s.length(); ++i) {
            count += dp[i - minJump];
            if (i - maxJump > 0)
                count -= dp[i - maxJump - 1];
            dp[i] = count && s[i] == '0';
        }

        return dp.back();
    }
};
```

939. 1872.cpp

Path: LeetCode-main/solutions/1872. Stone Game VIII/1872.cpp

```
class Solution {
public:
    int stoneGameVIII(vector<int>& stones) {
        const int n = stones.size();
        vector<int> prefix(n);
        // dp[i] := the maximum score difference the current player can get when the
        // game starts at i, i.e. stones[0..i] are merged into the value prefix[i]
        vector<int> dp(n, INT_MIN);

        partial_sum(stones.begin(), stones.end(), prefix.begin());

        // Must take all when there're only two stones left.
        dp[n - 2] = prefix.back();

        for (int i = n - 3; i >= 0; --i)
            dp[i] = max(dp[i + 1], prefix[i + 1] - dp[i + 1]);

        return dp[0];
    }
};
```

940. 1874.cpp

Path: LeetCode-main/solutions/1874. Minimize Product Sum of Two Arrays/1874.cpp

```
class Solution {
public:
    int minProductSum(vector<int>& nums1, vector<int>& nums2) {
        int ans = 0;

        ranges::sort(nums1);
        ranges::sort(nums2, greater<>());

        for (int i = 0; i < nums1.size(); ++i)
            ans += nums1[i] * nums2[i];

        return ans;
    }
};
```

941. 1876.cpp

Path: LeetCode-main/solutions/1876. Substrings of Size Three with Distinct Characters/1876.cpp

```
class Solution {
public:
    int countGoodSubstrings(string s) {
        int ans = 0;

        for (int i = 0; i + 2 < s.length(); ++i) {
            const char a = s[i];
            const char b = s[i + 1];
            const char c = s[i + 2];
            if (a == b || a == c || b == c)
                continue;
            ++ans;
        }
        return ans;
    }
};
```

942. 1877.cpp

Path: LeetCode-main/solutions/1877. Minimize Maximum Pair Sum in Array/1877.cpp

```
class Solution {
public:
    int minPairSum(vector<int>& nums) {
        int ans = 0;

        ranges::sort(nums);

        for (int i = 0, j = nums.size() - 1; i < j;)
            ans = max(ans, nums[i++] + nums[j--]);

        return ans;
    }
};
```

943. 1878.cpp

Path: LeetCode-main/solutions/1878. Get Biggest Three Rhombus Sums in a Grid/1878.cpp

```
class Solution {
public:
    vector<int> getBiggestThree(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        set<int> sums;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int sz = 0; i + sz < m && i - sz >= 0 && j + 2 * sz < n; ++sz) {
                    const int sum = sz == 0 ? grid[i][j] : getSum(grid, i, j, sz);
                    sums.insert(sum);
                    if (sums.size() > 3)
                        sums.erase(sums.begin());
                }

        return vector<int>(sums.rbegin(), sums.rend());
    }

private:
    // Returns the sum of the rhombus, where the top grid is (i, j) and the edge
    // size is `sz`.
    int getSum(const vector<vector<int>>& grid, int i, int j, int sz) {
        int x = i;
        int y = j;
        int sum = 0;

        // Go left down.
        for (int k = 0; k < sz; ++k)
            sum += grid[--x][++y];

        // Go right down.
        for (int k = 0; k < sz; ++k)
            sum += grid[++x][++y];

        // Go right up.
        for (int k = 0; k < sz; ++k)
            sum += grid[++x][--y];

        // Go left up.
        for (int k = 0; k < sz; ++k)
            sum += grid[--x][--y];

        return sum;
    }
};
```


944. 1879.cpp

Path: LeetCode-main/solutions/1879. Minimum XOR Sum of Two Arrays/1879.cpp

```
class Solution {
public:
    int minimumXORSum(vector<int>& nums1, vector<int>& nums2) {
        vector<int> mem(1 << nums2.size(), INT_MAX);
        return minimumXORSum(nums1, nums2, 0, mem);
    }

private:
    int minimumXORSum(const vector<int>& nums1, const vector<int>& nums2,
                     unsigned mask, vector<int>& mem) {
        const int i = popcount(mask);
        if (i == nums1.size())
            return 0;
        if (mem[mask] < INT_MAX)
            return mem[mask];

        for (int j = 0; j < nums2.size(); ++j)
            if ((mask >> j & 1) == 0)
                mem[mask] =
                    min(mem[mask], (nums1[i] ^ nums2[j]) +
                        minimumXORSum(nums1, nums2, mask | 1 << j, mem));

        return mem[mask];
    }
};
```

945. 188.cpp

Path: LeetCode-main/solutions/188. Best Time to Buy and Sell Stock IV/188.cpp

```
class Solution {
public:
    int maxProfit(int k, vector<int>& prices) {
        if (k >= prices.size() / 2) {
            int sell = 0;
            int hold = INT_MIN;

            for (const int price : prices) {
                sell = max(sell, hold + price);
                hold = max(hold, sell - price);
            }

            return sell;
        }

        vector<int> sell(k + 1);
        vector<int> hold(k + 1, INT_MIN);

        for (const int price : prices)
            for (int i = k; i > 0; --i) {
                sell[i] = max(sell[i], hold[i] + price);
                hold[i] = max(hold[i], sell[i - 1] - price);
            }

        return sell[k];
    }
};
```

946. 1880.cpp

Path: LeetCode-main/solutions/1880. Check if Word Equals Summation of Two Words/1880.cpp

```
class Solution {
public:
    bool isSumEqual(string firstWord, string secondWord, string targetWord) {
        const int first = getNumber(firstWord);
        const int second = getNumber(secondWord);
        const int target = getNumber(targetWord);
        return first + second == target;
    }

private:
    int getNumber(const string& word) {
        int num = 0;
        for (const char c : word)
            num = num * 10 + (c - 'a');
        return num;
    }
};
```

947. 1881.cpp

Path: LeetCode-main/solutions/1881. Maximum Value after Insertion/1881.cpp

```
class Solution {
public:
    string maxValue(string n, int x) {
        bool isNegative = n[0] == '-';

        for (int i = 0; i < n.length(); ++i)
            if (!isNegative && n[i] - '0' < x || isNegative && n[i] - '0' > x)
                return n.substr(0, i) + (char)('0' + x) + n.substr(i);

        return n + (char)('0' + x);
    }
};
```

948. 1882.cpp

Path: LeetCode-main/solutions/1882. Process Tasks Using Servers/1882.cpp

```
struct T {
    int weight;
    int index;
    int freeTime;
};

class Solution {
public:
    vector<int> assignTasks(vector<int>& servers, vector<int>& tasks) {
        const int n = servers.size();
        const int m = tasks.size();
        vector<int> ans(m);
        auto compareFree = [](const T& a, const T& b) {
            return a.weight == b.weight ? a.index > b.index : a.weight > b.weight;
        };
        auto compareUsed = [](const T& a, const T& b) {
            if (a.freeTime != b.freeTime)
                return a.freeTime > b.freeTime;
            if (a.weight != b.weight)
                return a.weight > b.weight;
            return a.index > b.index;
        };
        priority_queue<T, vector<T>, decltype(compareFree)> free(compareFree);
        priority_queue<T, vector<T>, decltype(compareUsed)> used(compareUsed);

        for (int i = 0; i < n; ++i)
            free.emplace(servers[i], i, 0);

        for (int i = 0; i < m; ++i) { // i := the current time
            const int executionTime = tasks[i];
            // Pop all the servers that'll be free at time i.
            while (!used.empty() && used.top().freeTime <= i) {
                const T curr = used.top();
                used.pop();
                free.push(curr);
            }
            if (free.empty()) {
                T server = used.top();
                used.pop();
                ans[i] = server.index;
                server.freeTime += executionTime;
                used.push(server);
            } else {
                T server = free.top();
                free.pop();
                ans[i] = server.index;
                server.freeTime = i + executionTime;
                used.push(server);
            }
        }

        return ans;
    }
};
```

949. 1883.cpp

Path: LeetCode-main/solutions/1883. Minimum Skips to Arrive at Meeting On Time/1883.cpp

```
class Solution {
public:
    int minSkips(vector<int>& dist, int speed, int hoursBefore) {
        constexpr double kInf = 10'000'000;
        constexpr double kEps = 1e-9;
        const int n = dist.size();
        // dp[i][j] := the minimum time, where i is the number of roads we traversed
        // so far and j is the number of skips we did
        vector<vector<double>> dp(n + 1, vector<double>(n + 1, kInf));
        dp[0][0] = 0;

        for (int i = 1; i <= n; ++i) {
            const double d = dist[i - 1];
            dp[i][0] = ceil(dp[i - 1][0] + d / speed - kEps);
            for (int j = 1; j <= i; ++j)
                dp[i][j] = min(dp[i - 1][j - 1] + d / speed,
                               ceil(dp[i - 1][j] + d / speed - kEps));
        }

        for (int j = 0; j <= n; ++j)
            if (dp[n][j] <= hoursBefore)
                return j;

        return -1;
    }
};
```

950. 1884.cpp

Path: LeetCode-main/solutions/1884. Egg Drop With 2 Eggs and N Floors/1884.cpp

```
class Solution {
public:
    int twoEggDrop(int n) {
        return superEggDrop(2, n);
    }

private:
    // Same as 887. Super Egg Drop
    int superEggDrop(int k, int n) {
        vector<vector<int>>> mem(k + 1, vector<int>(n + 1, -1));
        return drop(k, n, mem);
    }

    // Returns the minimum number of moves to know f with k eggs and n floors.
    int drop(int k, int n, vector<vector<int>>& mem) {
        if (k == 0) // no eggs -> done
            return 0;
        if (k == 1) // one egg -> drop from 1-th floor to n-th floor
            return n;
        if (n == 0) // no floor -> done
            return 0;
        if (n == 1) // one floor -> drop from that floor
            return 1;
        if (mem[k][n] != -1)
            return mem[k][n];

        // broken[i] := drop(k - 1, i - 1) is increasing with i
        // unbroken[i] := drop(k, n - i) is decreasing with i
        // mem[k][n] := 1 + min(max(broken[i], unbroken[i])), 1 <= i <= n
        // Find the first index i s.t broken[i] >= unbroken[i], which minimizes
        // max(broken[i], unbroken[i]).

        int l = 1;
        int r = n + 1;

        while (l < r) {
            const int m = (l + r) / 2;
            const int broken = drop(k - 1, m - 1, mem);
            const int unbroken = drop(k, n - m, mem);
            if (broken >= unbroken)
                r = m;
            else
                l = m + 1;
        }

        return mem[k][n] = 1 + drop(k - 1, l - 1, mem);
    }
};
```

951. 1885.cpp

Path: LeetCode-main/solutions/1885. Count Pairs in Two Arrays/1885.cpp

```
class Solution {
public:
    long long countPairs(vector<int>& nums1, vector<int>& nums2) {
        long ans = 0;
        vector<int> arr(nums1.size());

        for (int i = 0; i < arr.size(); ++i)
            arr[i] = nums1[i] - nums2[i];

        ranges::sort(arr);

        for (int i = 0; i < arr.size(); ++i) {
            const auto it = lower_bound(arr.begin() + i + 1, arr.end(), -arr[i] + 1);
            ans += arr.cend() - it;
        }

        return ans;
    }
};
```


952. 1886.cpp

Path: LeetCode-main/solutions/1886. Determine Whether Matrix Can Be Obtained By Rotation/1886.cpp

```
class Solution {
public:
    bool findRotation(vector<vector<int>>& mat, vector<vector<int>>& target) {
        for (int i = 0; i < 4; ++i) {
            if (mat == target)
                return true;
            rotate(mat);
        }
        return false;
    }

private:
    void rotate(vector<vector<int>>& mat) {
        ranges::reverse(mat);
        for (int i = 0; i < mat.size(); ++i)
            for (int j = i + 1; j < mat.size(); ++j)
                swap(mat[i][j], mat[j][i]);
    }
};
```

953. 1887.cpp

Path: LeetCode-main/solutions/1887. Reduction Operations to Make the Array Elements Equal/1887.cpp

```
class Solution {
public:
    int reductionOperations(vector<int>& nums) {
        int ans = 0;

        ranges::sort(nums);

        for (int i = nums.size() - 1; i > 0; --i)
            if (nums[i] != nums[i - 1])
                ans += nums.size() - i;

        return ans;
    }
};
```

954. 1888.cpp

Path: LeetCode-main/solutions/1888. Minimum Number of Flips to Make the Binary String Alternating/1888.cpp

```
class Solution {
public:
    int minFlips(string s) {
        const int n = s.length();
        // count[0][0] := the number of 0s in the even indices
        // count[0][1] := the number of 0s in the odd indices
        // count[1][0] := the number of 1s in the even indices
        // count[1][1] := the number of 1s in the odd indices
        vector<vector<int>> count(2, vector<int>(2));

        for (int i = 0; i < n; ++i)
            ++count[s[i] - '0'][i % 2];

        // min(make all 0s in the even indices + make all 1s in the odd indices,
        //      make all 1s in the even indices + make all 0s in the odd indices)
        int ans = min(count[1][0] + count[0][1], count[0][0] + count[1][1]);

        for (int i = 0; i < n; ++i) {
            --count[s[i] - '0'][i % 2];
            ++count[s[i] - '0'][(n + i) % 2];
            ans = min({ans, count[1][0] + count[0][1], count[0][0] + count[1][1]});
        }

        return ans;
    }
};
```

955. 1889.cpp

Path: LeetCode-main/solutions/1889. Minimum Space Wasted From Packaging/1889.cpp

```
class Solution {
public:
    int minWastedSpace(vector<int>& packages, vector<vector<int>>& boxes) {
        constexpr int kMod = 1'000'000'007;
        constexpr long kInf = 100'000'000'000;
        const long packagesSum = accumulate(packages.begin(), packages.end(), 0L);
        long minBoxesSum = kInf;

        ranges::sort(packages);

        for (vector<int>& box : boxes) {
            ranges::sort(box);
            if (box.back() < packages.back())
                continue;
            long accu = 0;
            long i = 0;
            for (const int b : box) {
                const long j = firstGreaterEqual(packages, b + 1);
                accu += b * (j - i);
                i = j;
            }
            minBoxesSum = min(minBoxesSum, accu);
        }

        return minBoxesSum == kInf ? -1 : (minBoxesSum - packagesSum) % kMod;
    }

private:
    int firstGreaterEqual(const vector<int>& arr, int target) {
        return ranges::lower_bound(arr, target) - arr.begin();
    }
};
```

956. 189.cpp

Path: LeetCode-main/solutions/189. Rotate Array/189.cpp

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k %= nums.size();
        reverse(nums, 0, nums.size() - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, nums.size() - 1);
    }

private:
    void reverse(vector<int>& nums, int l, int r) {
        while (l < r)
            swap(nums[l++], nums[r--]);
    }
};
```

957. 1891.cpp

Path: LeetCode-main/solutions/1891. Cutting Ribbons/1891.cpp

```
class Solution {
public:
    int maxLength(vector<int>& ribbons, int k) {
        int l = 1;
        int r = accumulate(ribbons.begin(), ribbons.end(), 0L) / k + 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (!isCutPossible(ribbons, m, k))
                r = m;
            else
                l = m + 1;
        }

        return l - 1;
    }

private:
    bool isCutPossible(const vector<int>& ribbons, int length, int k) {
        int count = 0;
        for (const int ribbon : ribbons)
            count += ribbon / length;
        return count >= k;
    }
};
```

958. 1893-2.cpp

Path: LeetCode-main/solutions/1893. Check if All the Integers in a Range Are Covered/1893-2.cpp

```
class Solution {
public:
    bool isCovered(vector<vector<int>>& ranges, int left, int right) {
        vector<int> seen(52);

        for (const vector<int>& range : ranges) {
            ++seen[range[0]];
            --seen[range[1] + 1];
        }

        for (int i = 1; i < 52; ++i)
            seen[i] += seen[i - 1];

        for (int i = left; i <= right; ++i)
            if (!seen[i])
                return false;

        return true;
    }
};
```

959. 1893.cpp

Path: LeetCode-main/solutions/1893. Check if All the Integers in a Range Are Covered/1893.cpp

```
class Solution {
public:
    bool isCovered(vector<vector<int>>& ranges, int left, int right) {
        for (int i = left; i <= right; ++i) {
            bool seen = false;
            for (const vector<int>& range : ranges)
                if (i >= range[0] && i <= range[1]) {
                    seen = true;
                    break;
                }
            if (!seen)
                return false;
        }
        return true;
    }
};
```


960. 1894.cpp

Path: LeetCode-main/solutions/1894. Find the Student that Will Replace the Chalk/1894.cpp

```
class Solution {
public:
    int chalkReplacer(vector<int>& chalk, int k) {
        k %= accumulate(chalk.begin(), chalk.end(), 0L);
        if (k == 0)
            return 0;

        for (int i = 0; i < chalk.size(); ++i) {
            k -= chalk[i];
            if (k < 0)
                return i;
        }

        throw;
    }
};
```

961. 1895.cpp

Path: LeetCode-main/solutions/1895. Largest Magic Square/1895.cpp

```
class Solution {
public:
    int largestMagicSquare(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        // prefixRow[i][j] := the sum of the first j numbers in the i-th row
        vector<vector<int>> prefixRow(m, vector<int>(n + 1));
        // prefixCol[i][j] := the sum of the first j numbers in the i-th column
        vector<vector<int>> prefixCol(n, vector<int>(m + 1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                prefixRow[i][j + 1] = prefixRow[i][j] + grid[i][j];
                prefixCol[j][i + 1] = prefixCol[j][i] + grid[i][j];
            }

        for (int k = min(m, n); k >= 2; --k)
            if (containsMagicSquare(grid, prefixRow, prefixCol, k))
                return k;

        return 1;
    }

private:
    // Returns true if the grid contains any magic square of size k x k.
    bool containsMagicSquare(const vector<vector<int>>& grid,
                            const vector<vector<int>>& prefixRow,
                            const vector<vector<int>>& prefixCol, int k) {
        for (int i = 0; i + k - 1 < grid.size(); ++i)
            for (int j = 0; j + k - 1 < grid[0].size(); ++j)
                if (isMagicSquare(grid, prefixRow, prefixCol, i, j, k))
                    return true;
        return false;
    }

    // Returns true if grid[i..i + k][j..j + k] is a magic square.
    bool isMagicSquare(const vector<vector<int>>& grid,
                      const vector<vector<int>>& prefixRow,
                      const vector<vector<int>>& prefixCol, int i, int j,
                      int k) {
        int diag = 0;
        int antiDiag = 0;
        for (int d = 0; d < k; ++d) {
            diag += grid[i + d][j + d];
            antiDiag += grid[i + d][j + k - 1 - d];
        }
        if (diag != antiDiag)
            return false;
        for (int d = 0; d < k; ++d) {
            if (getSum(prefixRow, i + d, j, j + k - 1) != diag)
                return false;
            if (getSum(prefixCol, j + d, i, i + k - 1) != diag)
                return false;
        }
        return true;
    }

    // Returns sum(grid[i][l..r]) or sum(grid[l..r][i]).
    int getSum(const vector<vector<int>>& prefix, int i, int l, int r) {
        return prefix[i][r + 1] - prefix[i][l];
    }
};
```

962. 1896.cpp

Path: LeetCode-main/solutions/1896. Minimum Cost to Change the Final Value of Expression/1896.cpp

```
class Solution {
public:
    int minOperationsToFlip(string expression) {
        // [(the expression, the cost to toggle the expression)]
        stack<pair<char, int>> stack;
        pair<char, int> lastPair;

        for (const char e : expression) {
            if (e == '(' || e == '&' || e == '|') {
                // These aren't expressions, so the cost is meaningless.
                stack.push({e, 0});
                continue;
            }
            if (e == ')') {
                lastPair = stack.top();
                stack.pop();
                stack.pop(); // Pop '('.
            } else { // e == '0' || e == '1'
                // Store the '0' or '1'. The cost to change their values is just 1,
                // whether it's changing '0' to '1' or '1' to '0'.
                lastPair = {e, 1};
            }
            if (!stack.empty() &&
                (stack.top().first == '&' || stack.top().first == '|')) {
                const char op = stack.top().first;
                stack.pop();
                const auto [a, costA] = stack.top();
                stack.pop();
                const auto [b, costB] = lastPair;
                // Determine the cost to toggle op(a, b).
                if (op == '&') {
                    if (a == '0' && b == '0')
                        // Change '&' to '|' and a|b to '1'.
                        lastPair = {'0', 1 + min(costA, costB)};
                    else if (a == '0' && b == '1')
                        // Change '&' to '|'.
                        lastPair = {'0', 1};
                    else if (a == '1' && b == '0')
                        // Change '&' to '|'.
                        lastPair = {'0', 1};
                    else // a == '1' and b == '1'
                        // Change a|b to '0'.
                        lastPair = {'1', min(costA, costB)};
                } else { // op == '|'
                    if (a == '0' && b == '0')
                        // Change a|b to '1'.
                        lastPair = {'0', min(costA, costB)};
                    else if (a == '0' && b == '1')
                        // Change '|' to '&'.
                        lastPair = {'1', 1};
                    else if (a == '1' && b == '0')
                        // Change '|' to '&'.
                        lastPair = {'1', 1};
                    else // a == '1' and b == '1'
                        // Change '|' to '&' and a|b to '0'.
                        lastPair = {'1', 1 + min(costA, costB)};
                }
            }
            stack.push(lastPair);
        }
        return stack.top().second;
    }
};
```

963. 1897.cpp

Path: LeetCode-main/solutions/1897. Redistribute Characters to Make All Strings Equal/1897.cpp

```
class Solution {
public:
    bool makeEqual(vector<string>& words) {
        vector<int> count(26);

        for (const string& word : words)
            for (const char c : word)
                ++count[c - 'a'];

        return ranges::all_of(count,
                               [&](const int c) { return c % words.size() == 0; });
    }
};
```

964. 1898.cpp

Path: LeetCode-main/solutions/1898. Maximum Number of Removable Characters/1898.cpp

```
class Solution {
public:
    int maximumRemovals(string s, string p, vector<int>& removable) {
        int l = 0;
        int r = removable.size() + 1;

        while (l < r) {
            const int m = (l + r) / 2;
            const string removed = remove(s, removable, m);
            if (isSubsequence(p, removed))
                l = m + 1;
            else
                r = m;
        }

        return l - 1;
    }

private:
    string remove(const string& s, const vector<int>& removable, int k) {
        string removed(s);
        for (int i = 0; i < k; ++i)
            removed[removable[i]] = '*';
        return removed;
    }

    bool isSubsequence(const string& p, const string& s) {
        int i = 0; // p's index
        for (int j = 0; j < s.length(); ++j)
            if (p[i] == s[j])
                if (++i == p.length())
                    return true;
        return false;
    }
};
```

965. 1899.cpp

Path: LeetCode-main/solutions/1899. Merge Triplets to Form Target Triplet/1899.cpp

```
class Solution {
public:
    bool mergeTriplets(vector<vector<int>>& triplets, vector<int>& target) {
        vector<int> merged(target.size());

        for (const vector<int>& triplet : triplets)
            if (equal(triplet.begin(), triplet.end(), target.begin(),
                    [](int a, int b) { return a <= b; }))
                transform(triplet.begin(), triplet.end(), merged.begin(),
                        merged.begin(), [](int a, int b) { return max(a, b); });

        return merged == target;
    }
};
```

966. 19.cpp

Path: LeetCode-main/solutions/19. Remove Nth Node From End of List/19.cpp

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (n-- > 0)
            fast = fast->next;
        if (fast == nullptr)
            return head->next;

        while (fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next;
        }
        slow->next = slow->next->next;

        return head;
    }
};
```

967. 190.cpp

Path: LeetCode-main/solutions/190. Reverse Bits/190.cpp

```
class Solution {
public:
    uint32_t reverseBits(uint32_t n) {
        uint32_t ans = 0;

        for (int i = 0; i < 32; ++i)
            if (n >> i & 1)
                ans |= 1 << 31 - i;

        return ans;
    }
};
```


968. 1900.cpp

Path: LeetCode-main/solutions/1900. The Earliest and Latest Rounds Where Players Compete/1900.cpp

```
class Solution {
public:
    vector<int> earliestAndLatest(int n, int firstPlayer, int secondPlayer) {
        using P = pair<int, int>;
        vector<vector<vector<P>>> mem(n + 1,
                                     vector<vector<P>>>(n + 1, vector<P>(n + 1)));
        const auto [a, b] = solve(firstPlayer, n - secondPlayer + 1, n, mem);
        return {a, b};
    }

private:
    // Returns the (earliest, latest) pair, the first player is the l-th player
    // from the front, the second player is the r-th player from the end, and
    // there're k people.
    pair<int, int> solve(int l, int r, int k,
                        vector<vector<vector<pair<int, int>>>& mem) {
        if (l == r)
            return {1, 1};
        if (l > r)
            swap(l, r);
        if (mem[l][r][k] != pair<int, int>{0, 0})
            return mem[l][r][k];

        int a = INT_MAX;
        int b = INT_MIN;

        // Enumerate all the possible positions.
        for (int i = 1; i <= l; ++i)
            for (int j = l - i + 1; j <= r - i; ++j) {
                if (i + j > (k + 1) / 2 || i + j < l + r - k / 2)
                    continue;
                const auto [x, y] = solve(i, j, (k + 1) / 2, mem);
                a = min(a, x + 1);
                b = max(b, y + 1);
            }

        return mem[l][r][k] = {a, b};
    }
};
```

969. 1901.cpp

Path: LeetCode-main/solutions/1901. Find a Peak Element II/1901.cpp

```
class Solution {
public:
    vector<int> findPeakGrid(vector<vector<int>>& mat) {
        int l = 0;
        int r = mat.size() - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (ranges::max(mat[m]) >= ranges::max(mat[m + 1]))
                r = m;
            else
                l = m + 1;
        }

        return {l, getMaxIndex(mat[l])};
    }

private:
    int getMaxIndex(const vector<int>& arr) {
        pair<int, int> res{0, arr[0]};
        for (int i = 1; i < arr.size(); ++i)
            if (arr[i] > res.second)
                res = {i, arr[i]};
        return res.first;
    }
};
```

970. 1902.cpp

Path: LeetCode-main/solutions/1902. Depth of BST Given Insertion Order/1902.cpp

```
class Solution {
public:
    int maxDepthBST(vector<int>& order) {
        int ans = 1;
        map<int, int> valToDepth;

        for (const int val : order) {
            const auto l = valToDepth.upper_bound(val);
            const auto r = valToDepth.lower_bound(val);
            const int leftDepth = l == valToDepth.cbegin() ? 0 : prev(l)->second;
            const int rightDepth = r == valToDepth.cend() ? 0 : r->second;
            const int depth = max(leftDepth, rightDepth) + 1;
            ans = max(ans, depth);
            valToDepth[val] = depth;
        }

        return ans;
    }
};
```

971. 1903.cpp

Path: LeetCode-main/solutions/1903. Largest Odd Number in String/1903.cpp

```
class Solution {
public:
    string largestOddNumber(string num) {
        for (int i = num.length() - 1; i >= 0; --i)
            if ((num[i] - '0') % 2 == 1)
                return num.substr(0, i + 1);
        return "";
    }
};
```

972. 1904.cpp

Path: LeetCode-main/solutions/1904. The Number of Full Rounds You Have Played/1904.cpp

```
class Solution {
public:
    int numberOfRounds(string loginTime, string logoutTime) {
        const int start = getMinutes(loginTime);
        int finish = getMinutes(logoutTime);
        if (start > finish)
            finish += 60 * 24;
        return max(0, finish / 15 - (start + 14) / 15);
    }

private:
    int getMinutes(const string& time) {
        const int h = stoi(time.substr(0, 2));
        const int m = stoi(time.substr(3));
        return 60 * h + m;
    }
};
```

973. 1905.cpp

Path: LeetCode-main/solutions/1905. Count Sub Islands/1905.cpp

```
class Solution {
public:
    int countSubIslands(vector<vector<int>>& grid1, vector<vector<int>>& grid2) {
        int ans = 0;

        for (int i = 0; i < grid2.size(); ++i)
            for (int j = 0; j < grid2[0].size(); ++j)
                if (grid2[i][j] == 1)
                    ans += dfs(grid1, grid2, i, j);

        return ans;
    }

private:
    int dfs(const vector<vector<int>>& grid1, vector<vector<int>>& grid2, int i,
            int j) {
        if (i < 0 || i == grid1.size() || j < 0 || j == grid2[0].size())
            return 1;
        if (grid2[i][j] != 1)
            return 1;

        grid2[i][j] = 2; // Mark 2 as visited.

        return dfs(grid1, grid2, i + 1, j) & dfs(grid1, grid2, i - 1, j) &
            dfs(grid1, grid2, i, j + 1) & dfs(grid1, grid2, i, j - 1) &
            grid1[i][j];
    }
};
```

974. 1906.cpp

Path: LeetCode-main/solutions/1906. Minimum Absolute Difference Queries/1906.cpp

```
class Solution {
public:
    vector<int> minDifference(vector<int>& nums, vector<vector<int>>& queries) {
        vector<vector<int>> numToIndices(101);

        for (int i = 0; i < nums.size(); ++i)
            numToIndices[nums[i]].push_back(i);

        if (numToIndices[nums[0]].size() == nums.size())
            return vector<int>(queries.size(), -1);

        vector<int> ans;

        for (const vector<int>& query : queries) {
            const int l = query[0];
            const int r = query[1];
            int prevNum = -1;
            int minDiff = 101;
            for (int num = 1; num <= 100; ++num) {
                const auto& indices = numToIndices[num];
                const auto it = ranges::lower_bound(indices, l);
                if (it == indices.cend() || *it > r)
                    continue;
                if (prevNum != -1)
                    minDiff = min(minDiff, num - prevNum);
                prevNum = num;
            }
            ans.push_back(minDiff == 101 ? -1 : minDiff);
        }

        return ans;
    }
};
```

975. 1908.cpp

Path: LeetCode-main/solutions/1908. Game of Nim/1908.cpp

```
class Solution {
public:
    bool nimGame(vector<int>& piles) {
        return accumulate(piles.begin(), piles.end(), 0, bit_xor<>()) > 0;
    }
};
```


976. 1909.cpp

Path: LeetCode-main/solutions/1909. Remove One Element to Make the Array Strictly Increasing/1909.cpp

```
class Solution {
public:
    bool canBeIncreasing(vector<int>& nums) {
        bool removed = false;

        for (int i = 1; i < nums.size(); ++i)
            if (nums[i - 1] >= nums[i]) {
                if (removed)
                    return false;
                removed = true; // Remove nums[i - 1].
                if (i > 1 && nums[i - 2] >= nums[i])
                    nums[i] = nums[i - 1]; // Remove nums[i] instead.
            }

        return true;
    }
};
```

977. 191-2.cpp

Path: LeetCode-main/solutions/191. Number of 1 Bits/191-2.cpp

```
class Solution {  
public:  
    int hammingweight(uint32_t n) {  
        return popcount(n);  
    }  
};
```

978. 191.cpp

Path: LeetCode-main/solutions/191. Number of 1 Bits/191.cpp

```
class Solution {
public:
    int hammingweight(uint32_t n) {
        int ans = 0;

        for (int i = 0; i < 32; ++i)
            if ((n >> i) & 1)
                ++ans;

        return ans;
    }
};
```

979. 1910.cpp

Path: LeetCode-main/solutions/1910. Remove All Occurrences of a Substring/1910.cpp

```
class Solution {
public:
    string removeOccurrences(string s, string part) {
        const int n = s.length();
        const int k = part.length();

        string t(n, ' ');
        int j = 0; // t's index

        for (int i = 0; i < n; ++i) {
            t[j++] = s[i];
            if (j >= k && t.substr(j - k, k) == part)
                j -= k;
        }

        return t.substr(0, j);
    }
};
```

980. 1911.cpp

Path: LeetCode-main/solutions/1911. Maximum Alternating Subsequence Sum/1911.cpp

```
class Solution {
public:
    long long maxAlternatingSum(vector<int>& nums) {
        long even = 0; // the maximum alternating sum ending in an even index
        long odd = 0;  // the maximum alternating sum ending in an odd index

        for (const int num : nums) {
            even = max(even, odd + num);
            odd = even - num;
        }

        return even;
    }
};
```

981. 1912.cpp

Path: LeetCode-main/solutions/1912. Design Movie Rental System/1912.cpp

```
class MovieRentingSystem {
public:
    MovieRentingSystem(int n, vector<vector<int>>& entries) {
        for (const vector<int>& e : entries) {
            const int shop = e[0];
            const int movie = e[1];
            const int price = e[2];
            unrented[movie].insert({price, shop});
            shopAndMovieToPrice[{shop, movie}] = price;
        }
    }

    vector<int> search(int movie) {
        vector<int> ans;
        int i = 0;

        for (const auto& [price, shop] : unrented[movie]) {
            ans.push_back(shop);
            if (++i >= 5)
                break;
        }

        return ans;
    }

    void rent(int shop, int movie) {
        const int price = shopAndMovieToPrice[{shop, movie}];
        unrented[movie].erase({price, shop});
        rented.insert({price, {shop, movie}});
    }

    void drop(int shop, int movie) {
        const int price = shopAndMovieToPrice[{shop, movie}];
        unrented[movie].insert({price, shop});
        rented.erase({price, {shop, movie}});
    }

    vector<vector<int>> report() {
        vector<vector<int>> ans;
        int i = 0;

        for (const auto& [_, shopAndMovie] : rented) {
            ans.push_back({shopAndMovie.first, shopAndMovie.second});
            if (++i >= 5)
                break;
        }

        return ans;
    }

private:
    struct PairHash {
        size_t operator()(const pair<int, int>& p) const {
            return p.first ^ p.second;
        }
    };

    // {movie: (price, shop)}
    unordered_map<int, set<pair<int, int>>> unrented;

    // {(shop, movie): price}
    unordered_map<pair<int, int>, int, PairHash> shopAndMovieToPrice;

    // (price, (shop, movie))
    set<pair<int, pair<int, int>>> rented;
};
```

982. 1913.cpp

Path: LeetCode-main/solutions/1913. Maximum Product Difference Between Two Pairs/1913.cpp

```
class Solution {
public:
    int maxProductDifference(vector<int>& nums) {
        int max1 = INT_MIN;
        int max2 = INT_MIN;
        int min1 = INT_MAX;
        int min2 = INT_MAX;

        for (const int num : nums) {
            if (num > max1) {
                max2 = max1;
                max1 = num;
            } else if (num > max2) {
                max2 = num;
            }
            if (num < min1) {
                min2 = min1;
                min1 = num;
            } else if (num < min2) {
                min2 = num;
            }
        }

        return max1 * max2 - min1 * min2;
    }
};
```

983. 1914.cpp

Path: LeetCode-main/solutions/1914. Cyclically Rotating a Grid/1914.cpp

```
class Solution {
public:
    vector<vector<int>> rotateGrid(vector<vector<int>>& grid, int k) {
        const int m = grid.size();
        const int n = grid[0].size();
        int t = 0;          // the top
        int l = 0;          // the left
        int b = m - 1;      // the bottom
        int r = n - 1;      // the right

        while (t < b && l < r) {
            const int elementInThisLayer = 2 * (b - t + 1) + 2 * (r - l + 1) - 4;
            const int netRotations = k % elementInThisLayer;
            for (int rotate = 0; rotate < netRotations; ++rotate) {
                const int topLeft = grid[t][l];
                for (int j = l; j < r; ++j)
                    grid[t][j] = grid[t][j + 1];
                for (int i = t; i < b; ++i)
                    grid[i][r] = grid[i + 1][r];
                for (int j = r; j > l; --j)
                    grid[b][j] = grid[b][j - 1];
                for (int i = b; i > t; --i)
                    grid[i][l] = grid[i - 1][l];
                grid[t + 1][l] = topLeft;
            }
            ++t;
            ++l;
            --b;
            --r;
        }

        return grid;
    }
};
```


984. 1915.cpp

Path: LeetCode-main/solutions/1915. Number of Wonderful Substrings/1915.cpp

```
class Solution {
public:
    long long wonderfulSubstrings(string word) {
        long ans = 0;
        int prefix = 0;          // the binary prefix
        vector<int> count(1024);  // the binary prefix count
        count[0] = 1;           // the empty string ""

        for (const char c : word) {
            prefix ^= 1 << c - 'a';
            // All the letters occur even number of times.
            ans += count[prefix];
            // ('a' + i) occurs odd number of times.
            for (int i = 0; i < 10; ++i)
                ans += count[prefix ^ 1 << i];
            ++count[prefix];
        }

        return ans;
    }
};
```

985. 1918.cpp

Path: LeetCode-main/solutions/1918. Kth Smallest Subarray Sum/1918.cpp

```
class Solution {
public:
    int kthSmallestSubarraySum(vector<int>& nums, int k) {
        int l = 0;
        int r = accumulate(nums.begin(), nums.end(), 0);

        while (l < r) {
            const int m = (l + r) / 2;
            if (numSubarrayLessThan(nums, m) >= k)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    int numSubarrayLessThan(const vector<int>& nums, int m) {
        int res = 0;
        int sum = 0;
        for (int l = 0, r = 0; r < nums.size(); ++r) {
            sum += nums[r];
            while (sum > m)
                sum -= nums[l++];
            res += r - l + 1;
        }
        return res;
    }
};
```

986. 1920.cpp

Path: LeetCode-main/solutions/1920. Build Array from Permutation/1920.cpp

```
class Solution {
public:
    vector<int> buildArray(vector<int>& nums) {
        const int n = nums.size();

        for (int i = 0; i < n; ++i)
            nums[i] += n * (nums[nums[i]] % n);

        for (int i = 0; i < n; ++i)
            nums[i] /= n;

        return nums;
    }
};
```

987. 1921.cpp

Path: LeetCode-main/solutions/1921. Eliminate Maximum Number of Monsters/1921.cpp

```
class Solution {
public:
    int eliminateMaximum(vector<int>& dist, vector<int>& speed) {
        const int n = dist.size();
        vector<int> arrivalTime(n);

        for (int i = 0; i < n; ++i)
            arrivalTime[i] = (dist[i] - 1) / speed[i];

        ranges::sort(arrivalTime);

        for (int i = 0; i < n; ++i)
            if (i > arrivalTime[i])
                return i;

        return n;
    }
};
```

988. 1922.cpp

Path: LeetCode-main/solutions/1922. Count Good Numbers/1922.cpp

```
class Solution {
public:
    int countGoodNumbers(long long n) {
        return modPow(4 * 5, n / 2) * (n % 2 == 0 ? 1 : 5) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % kMod;
        return modPow(x * x % kMod, n / 2);
    }
};
```

989. 1923.cpp

Path: LeetCode-main/solutions/1923. Longest Common Subpath/1923.cpp

```
class Solution {
public:
    int longestCommonSubpath(int n, vector<vector<int>>& paths) {
        int l = 0;
        int r = paths[0].size();

        while (l < r) {
            const int m = l + (r - l + 1) / 2;
            if (checkCommonSubpath(paths, m))
                l = m;
            else
                r = m - 1;
        }

        return l;
    }

    static constexpr long kBase = 165'131;
    static constexpr long kHash = 8'417'508'174'513;

    // Returns true if there's a common subpath of length m for all the paths.
    bool checkCommonSubpath(const vector<vector<int>>& paths, int m) {
        vector<unordered_set<long>> hashSets;

        // Calculate the hash values for subpaths of length m for every path.
        for (const vector<int>& path : paths)
            hashSets.push_back(rabinKarp(path, m));

        // Check if there is a common subpath of length m.
        for (const long subpathHash : hashSets[0])
            if (ranges::all_of(hashSets,
                               [subpathHash](const unordered_set<long>& hashSet) {
                                   return hashSet.contains(subpathHash);
                               }))
                return true;

        return false;
    }

    // Returns the hash values for subpaths of length m in the path.
    unordered_set<long> rabinKarp(const vector<int>& path, int m) {
        unordered_set<long> hashes;
        long maxPower = 1;
        long hash = 0;
        for (int i = 0; i < path.size(); ++i) {
            hash = (hash * kBase + path[i]) % kHash;
            if (i >= m)
                hash = (hash - path[i - m] * maxPower % kHash + kHash) % kHash;
            else
                maxPower = maxPower * kBase % kHash;
            if (i >= m - 1)
                hashes.insert(hash);
        }
        return hashes;
    }
};
```

990. 1924.cpp

Path: LeetCode-main/solutions/1924. Erect the Fence II/1924.cpp

```
struct Point {
    double x;
    double y;
    Point(double x, double y) : x(x), y(y) {}
};

struct Disk {
    Point center;
    double radius;
    Disk(const Point& center, double radius) : center(center), radius(radius) {}
};

class Solution {
public:
    vector<double> outerTrees(vector<vector<int>>& trees) {
        vector<Point> points;
        for (int i = 0; i < trees.size(); ++i)
            points.emplace_back(trees[i][0], trees[i][1]);
        Disk disk = welzl(points, 0, {});
        return {disk.center.x, disk.center.y, disk.radius};
    }

private:
    // Returns the smallest disk that encloses points[i..n].
    //
    // https://en.wikipedia.org/wiki/Smallest-disk_problem#Welzl's_algorithm
    Disk welzl(const vector<Point>& points, int i, vector<Point> planePoints) {
        if (i == points.size() || planePoints.size() == 3)
            return trivial(planePoints);
        Disk disk = welzl(points, i + 1, planePoints);
        if (inside(disk, points[i]))
            return disk;
        return welzl(points, i + 1, addPlanePoint(planePoints, points[i]));
    }

    vector<Point> addPlanePoint(const vector<Point>& planePoints,
                               const Point& point) {
        vector<Point> newPlanePoints(planePoints);
        newPlanePoints.push_back(point);
        return newPlanePoints;
    }

    // Returns the smallest disk that encloses `planePoints`.
    Disk trivial(const vector<Point>& planePoints) {
        if (planePoints.empty())
            return Disk(Point(0, 0), 0);
        if (planePoints.size() == 1)
            return Disk(Point(planePoints[0].x, planePoints[0].y), 0);
        if (planePoints.size() == 2)
            return getDisk(planePoints[0], planePoints[1]);

        Disk disk01 = getDisk(planePoints[0], planePoints[1]);
        if (inside(disk01, planePoints[2]))
            return disk01;

        Disk disk02 = getDisk(planePoints[0], planePoints[2]);
        if (inside(disk02, planePoints[1]))
            return disk02;

        Disk disk12 = getDisk(planePoints[1], planePoints[2]);
        if (inside(disk12, planePoints[0]))
            return disk12;

        return getDisk(planePoints[0], planePoints[1], planePoints[2]);
    }

    // Returns the smallest disk that encloses the points A and B.
    Disk getDisk(const Point& A, const Point& B) {
        const double x = (A.x + B.x) / 2;
        const double y = (A.y + B.y) / 2;
        return Disk(Point(x, y), distance(A, B) / 2);
    }

    // Returns the smallest disk that encloses the points A, B, and C.
    Disk getDisk(const Point& A, const Point& B, const Point& C) {
        // Calculate midpoints.
        Point mAB((A.x + B.x) / 2, (A.y + B.y) / 2);
        Point mBC((B.x + C.x) / 2, (B.y + C.y) / 2);

        // Calculate the slopes and the perpendicular slopes.
        const double slopeAB = (B.y - A.y) / (B.x - A.x);
```

```

const double slopeBC = (C.y - B.y) / (C.x - B.x);
const double perpSlopeAB = -1 / slopeAB;
const double perpSlopeBC = -1 / slopeBC;

// Calculate the center.
const double x =
    (perpSlopeBC * mBC.x - perpSlopeAB * mAB.x + mAB.y - mBC.y) /
    (perpSlopeBC - perpSlopeAB);
const double y = perpSlopeAB * (x - mAB.x) + mAB.y;
Point center(x, y);
return Disk(center, distance(center, A));
}

// Returns true if the point is inside the disk.
bool inside(Disk disk, Point point) {
    return disk.radius > 0 && distance(disk.center, point) <= disk.radius;
}

double distance(Point A, Point B) {
    const double dx = A.x - B.x;
    const double dy = A.y - B.y;
    return sqrt(dx * dx + dy * dy);
}
};

```


992. 1926.cpp

Path: LeetCode-main/solutions/1926. Nearest Exit from Entrance in Maze/1926.cpp

```
class Solution {
public:
    int nearestExit(vector<vector<char>>& maze, vector<int>& entrance) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = maze.size();
        const int n = maze[0].size();
        queue<pair<int, int>> q{{{entrance[0], entrance[1]}}};
        vector<vector<bool>> seen(m, vector<bool>(n));
        seen[entrance[0]][entrance[1]] = true;

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (seen[x][y] || maze[x][y] == '+')
                        continue;
                    if (x == 0 || x == m - 1 || y == 0 || y == n - 1)
                        return step;
                    q.emplace(x, y);
                    seen[x][y] = true;
                }
            }
        return -1;
    }
};
```

993. 1927.cpp

Path: LeetCode-main/solutions/1927. Sum Game/1927.cpp

```
class Solution {
public:
    bool sumGame(string num) {
        const int n = num.length();
        double ans = 0.0;

        for (int i = 0; i < n / 2; ++i)
            ans += getExpectation(num[i]);

        for (int i = n / 2; i < n; ++i)
            ans -= getExpectation(num[i]);

        return ans != 0.0;
    }

private:
    double getExpectation(char c) {
        return c == '?' ? 4.5 : c - '0';
    }
};
```

994. 1928.cpp

Path: LeetCode-main/solutions/1928. Minimum Cost to Reach Destination in Time/1928.cpp

```
class Solution {
public:
    int minCost(int maxTime, vector<vector<int>>& edges,
                vector<int>& passingFees) {
        const int n = passingFees.size();
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int w = edge[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        return dijkstra(graph, 0, n - 1, maxTime, passingFees);
    }

private:
    int dijkstra(const vector<vector<pair<int, int>>>& graph, int src, int dst,
                 int maxTime, const vector<int>& passingFees) {
        // cost[i] := the minimum cost to reach the i-th city
        vector<int> cost(graph.size(), INT_MAX);
        // dist[i] := the minimum time to reach the i-th city
        vector<int> dist(graph.size(), maxTime + 1);

        cost[src] = passingFees[src];
        dist[src] = 0;
        using T = tuple<int, int, int>; // (cost[u], dist[u], u)
        priority_queue<T, vector<T>, greater<>> minHeap;
        minHeap.emplace(cost[src], dist[src], src);

        while (!minHeap.empty()) {
            const auto [currCost, d, u] = minHeap.top();
            minHeap.pop();
            if (u == dst)
                return cost[dst];
            if (d > dist[u] && currCost > cost[u])
                continue;
            for (const auto& [v, w] : graph[u]) {
                if (d + w > maxTime)
                    continue;
                // Go from u -> v.
                if (currCost + passingFees[v] < cost[v]) {
                    cost[v] = currCost + passingFees[v];
                    dist[v] = d + w;
                    minHeap.emplace(cost[v], dist[v], v);
                } else if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.emplace(currCost + passingFees[v], dist[v], v);
                }
            }
        }

        return -1;
    }
};
```

995. 1929.cpp

Path: LeetCode-main/solutions/1929. Concatenation of Array/1929.cpp

```
class Solution {
public:
    vector<int> getConcatenation(vector<int>& nums) {
        const int n = nums.size();

        for (int i = 0; i < n; ++i)
            nums.push_back(nums[i]);

        return nums;
    }
};
```

996. 1930.cpp

Path: LeetCode-main/solutions/1930. Unique Length-3 Palindromic Subsequences/1930.cpp

```

class Solution {
public:
    int countPalindromicSubsequence(string s) {
        int ans = 0;
        vector<int> first(26, s.length());
        vector<int> last(26, -1);

        for (int i = 0; i < s.length(); ++i) {
            const int index = s[i] - 'a';
            first[index] = min(first[index], i);
            last[index] = i;
        }

        for (int i = 0; i < 26; ++i)
            if (first[i] < last[i])
                ans += unordered_set<int>(s.begin() + first[i] + 1, s.begin() + last[i])
                    .size();

        return ans;
    }
};

```

997. 1931.cpp

Path: LeetCode-main/solutions/1931. Painting a Grid With Three Different Colors/1931.cpp

```
class Solution {
public:
    int colorTheGrid(int m, int n) {
        this->m = m;
        this->n = n;
        return dp(0, 0, 0, 0);
    }

private:
    static constexpr int kMod = 1'000'000'007;
    int m;
    int n;
    vector<vector<int>> mem = vector<vector<int>>(1000, vector<int>(1024));

    int dp(int r, int c, int prevColMask, int currColMask) {
        if (c == n)
            return 1;
        if (mem[c][prevColMask])
            return mem[c][prevColMask];
        if (r == m)
            return dp(0, c + 1, currColMask, 0);

        int ans = 0;

        // 1 := red, 2 := green, 3 := blue
        for (int color = 1; color <= 3; ++color) {
            if (getColor(prevColMask, r) == color)
                continue;
            if (r > 0 && getColor(currColMask, r - 1) == color)
                continue;
            ans += dp(r + 1, c, prevColMask, setColor(currColMask, r, color));
            ans %= kMod;
        }

        if (r == 0)
            mem[c][prevColMask] = ans;

        return ans;
    }

    // e.g. 01 10 11 11 11
    //       R  G  B  B  B
    // getColor(011011111, 3) -> G
    int getColor(int mask, int r) {
        return mask >> r * 2 & 3;
    }

    int setColor(int mask, int r, int color) {
        return mask | color << r * 2;
    }
};
```

998. 1932.cpp

Path: LeetCode-main/solutions/1932. Merge BSTs to Create Single BST/1932.cpp

```
class Solution {
public:
    TreeNode* canMerge(vector<TreeNode*>& trees) {
        unordered_map<int, TreeNode*> valToNode; // {val: node}
        unordered_map<int, int> count;           // {val: freq}

        for (TreeNode* tree : trees) {
            valToNode[tree->val] = tree;
            ++count[tree->val];
            if (tree->left)
                ++count[tree->left->val];
            if (tree->right)
                ++count[tree->right->val];
        }

        for (TreeNode* tree : trees)
            if (count[tree->val] == 1) {
                if (isValidBST(tree, nullptr, nullptr, valToNode) &&
                    valToNode.size() <= 1)
                    return tree;
                return nullptr;
            }

        return nullptr;
    }

private:
    bool isValidBST(TreeNode* tree, TreeNode* minNode, TreeNode* maxNode,
                    unordered_map<int, TreeNode*>& valToNode) {
        if (tree == nullptr)
            return true;
        if (minNode && tree->val <= minNode->val)
            return false;
        if (maxNode && tree->val >= maxNode->val)
            return false;
        if (!tree->left && !tree->right && valToNode.contains(tree->val)) {
            const int val = tree->val;
            tree->left = valToNode[val]->left;
            tree->right = valToNode[val]->right;
            valToNode.erase(val);
        }

        return isValidBST(tree->left, minNode, tree, valToNode) &&
            isValidBST(tree->right, tree, maxNode, valToNode);
    }
};
```


999. 1933.cpp

Path: LeetCode-main/solutions/1933. Check if String Is Decomposable Into Value-Equal Substrings/1933.cpp

```
class Solution {
public:
    bool isDecomposable(string s) {
        int twos = 0;
        int groupLength = 0;
        char letter = '@'; // the running letter

        for (const char c : s)
            if (c == letter) {
                ++groupLength;
            } else {
                if (groupLength % 3 == 1)
                    return false;
                if (groupLength % 3 == 2 && ++twos > 1)
                    return false;
                groupLength = 1;
                letter = c;
            }

        // Check the final group.
        if (groupLength % 3 == 1)
            return false;
        if (groupLength % 3 == 2 && ++twos > 1)
            return false;
        return twos == 1;
    }
};
```

1000. 1935.cpp

Path: LeetCode-main/solutions/1935. Maximum Number of Words You Can Type/1935.cpp

```
class Solution {
public:
    int canBeTypedWords(string text, string brokenLetters) {
        int ans = 0;
        istringstream iss(text);
        vector<bool> broken(26);

        for (const char c : brokenLetters)
            broken[c - 'a'] = true;

        for (string word; iss >> word;)
            ans += canBeTyped(word, broken);

        return ans;
    }

private:
    bool canBeTyped(const string& word, const vector<bool>& broken) {
        for (const char c : word)
            if (broken[c - 'a'])
                return false;
        return true;
    }
};
```

1001. 1936.cpp

Path: LeetCode-main/solutions/1936. Add Minimum Number of Rungs/1936.cpp

```
class Solution {
public:
    int addRungs(vector<int>& rungs, int dist) {
        int ans = 0;
        int prev = 0;

        for (const int rung : rungs) {
            ans += (rung - prev - 1) / dist;
            prev = rung;
        }

        return ans;
    }
};
```

1002. 1937.cpp

Path: LeetCode-main/solutions/1937. Maximum Number of Points with Cost/1937.cpp

```
class Solution {
public:
    long long maxPoints(vector<vector<int>>& points) {
        const int n = points[0].size();
        // dp[j] := the maximum number of points you can have if points[i][j] is the
        // most recent cell you picked
        vector<long> dp(n);

        for (const vector<int>& row : points) {
            vector<long> leftToRight(n);
            long runningMax = 0;

            for (int j = 0; j < n; ++j) {
                runningMax = max(runningMax - 1, dp[j]);
                leftToRight[j] = runningMax;
            }

            vector<long> rightToLeft(n);
            runningMax = 0;

            for (int j = n - 1; j >= 0; --j) {
                runningMax = max(runningMax - 1, dp[j]);
                rightToLeft[j] = runningMax;
            }

            for (int j = 0; j < n; ++j)
                dp[j] = max(leftToRight[j], rightToLeft[j]) + row[j];
        }

        return ranges::max(dp);
    }
};
```

1003. 1938.cpp

Path: LeetCode-main/solutions/1938. Maximum Genetic Difference Query/1938.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    int count = 0;
    TrieNode() : children(2) {}
};

class Trie {
public:
    void update(int num, int val) {
        shared_ptr<TrieNode> node = root;
        for (int i = kHeight; i >= 0; --i) {
            const int bit = (num >> i) & 1;
            if (node->children[bit] == nullptr)
                node->children[bit] = make_shared<TrieNode>();
            node = node->children[bit];
            node->count += val;
        }
    }

    int query(int num) {
        int ans = 0;
        shared_ptr<TrieNode> node = root;
        for (int i = kHeight; i >= 0; --i) {
            const int bit = (num >> i) & 1;
            const int targetBit = bit ^ 1;
            if (node->children[targetBit] && node->children[targetBit]->count) {
                ans += 1 << i;
                node = node->children[targetBit];
            } else {
                node = node->children[targetBit ^ 1];
            }
        }
        return ans;
    }

private:
    static constexpr int kHeight = 17;
    shared_ptr<TrieNode> root = make_shared<TrieNode>();
};

class Solution {
public:
    vector<int> maxGeneticDifference(vector<int>& parents,
                                     vector<vector<int>>& queries) {
        const int n = parents.size();
        vector<int> ans(queries.size());
        int rootVal = -1;
        vector<vector<int>> tree(n);
        // {node: {index, val}}
        unordered_map<int, vector<pair<int, int>>> nodeToQueries;
        Trie trie;

        for (int i = 0; i < parents.size(); ++i)
            if (parents[i] == -1)
                rootVal = i;
            else
                tree[parents[i]].push_back(i);

        for (int i = 0; i < queries.size(); ++i) {
            const int node = queries[i][0];
            const int val = queries[i][1];
            nodeToQueries[node].emplace_back(i, val);
        }

        dfs(rootVal, trie, tree, nodeToQueries, ans);
        return ans;
    }

private:
    void dfs(int node, Trie& trie, const vector<vector<int>>& tree,
             const unordered_map<int, vector<pair<int, int>>& nodeToQueries,
             vector<int>& ans) {
        trie.update(node, 1);

        if (const auto it = nodeToQueries.find(node); it != nodeToQueries.cend())
            for (const auto& [i, val] : it->second)
                ans[i] = trie.query(val);

        for (const int child : tree[node])
            dfs(child, trie, tree, nodeToQueries, ans);
    }
};
```

```
        trie.update(node, -1);  
    }  
};
```

1004. 1940.cpp

Path: LeetCode-main/solutions/1940. Longest Common Subsequence Between Sorted Arrays/1940.cpp

```
class Solution {
public:
    vector<int> longestCommonSubsequence(vector<vector<int>>& arrays) {
        constexpr int kMax = 100;
        vector<int> ans;
        vector<int> count(kMax + 1);

        for (const vector<int>& array : arrays)
            for (const int a : array)
                if (++count[a] == arrays.size())
                    ans.push_back(a);

        return ans;
    }
};
```

1005. 1941.cpp

Path: LeetCode-main/solutions/1941. Check if All Characters Have Equal Number of Occurrences/1941.cpp

```
class Solution {
public:
    bool areOccurrencesEqual(string s) {
        vector<int> count(26);
        for (const char c : s)
            ++count[c - 'a'];
        return equalFreq(count, count[s[0] - 'a']);
    }

private:
    bool equalFreq(const vector<int>& count, int theFreq) {
        return ranges::all_of(
            count, [theFreq](int freq) { return freq == 0 || freq == theFreq; });
    }
};
```


1006. 1942.cpp

Path: LeetCode-main/solutions/1942. The Number of the Smallest Unoccupied Chair/1942.cpp

```
class Solution {
public:
    int smallestChair(vector<vector<int>>& times, int targetFriend) {
        int nextUnsatChair = 0;
        priority_queue<int, vector<int>, greater<>> emptyChairs;
        using P = pair<int, int>; // (leaving, chair)
        priority_queue<P, vector<P>, greater<>> occupied;

        for (int i = 0; i < times.size(); ++i)
            times[i].push_back(i);

        ranges::sort(times);

        for (const vector<int>& time : times) {
            const int arrival = time[0];
            const int leaving = time[1];
            const int i = time[2];
            while (!occupied.empty() && occupied.top().first <= arrival)
                emptyChairs.push(occupied.top().second), occupied.pop();
            if (i == targetFriend)
                return emptyChairs.empty() ? nextUnsatChair : emptyChairs.top();
            if (emptyChairs.empty())
                occupied.emplace(leaving, nextUnsatChair++);
            else
                occupied.emplace(leaving, emptyChairs.top()), emptyChairs.pop();
        }

        throw;
    }
};
```

1007. 1943.cpp

Path: LeetCode-main/solutions/1943. Describe the Painting/1943.cpp

```
class Solution {
public:
    vector<vector<long long>> splitPainting(vector<vector<int>>& segments) {
        vector<vector<long long>> ans;
        int prevIndex = 0;
        long runningMix = 0;
        map<int, long> line;

        for (const vector<int>& segment : segments) {
            const int start = segment[0];
            const int end = segment[1];
            const int color = segment[2];
            line[start] += color;
            line[end] -= color;
        }

        for (const auto& [i, mix] : line) {
            if (runningMix > 0)
                ans.push_back({prevIndex, i, runningMix});
            runningMix += mix;
            prevIndex = i;
        }

        return ans;
    }
};
```

1008. 1944.cpp

Path: LeetCode-main/solutions/1944. Number of Visible People in a Queue/1944.cpp

```
class Solution {
public:
    vector<int> canSeePersonsCount(vector<int>& heights) {
        const int n = heights.size();
        vector<int> ans(n);
        stack<int> stack;

        for (int i = 0; i < n; ++i) {
            while (!stack.empty() && heights[stack.top()] <= heights[i])
                ++ans[stack.top()], stack.pop();
            if (!stack.empty())
                ++ans[stack.top()];
            stack.push(i);
        }

        return ans;
    }
};
```

1009. 1945.cpp

Path: LeetCode-main/solutions/1945. Sum of Digits of String After Convert/1945.cpp

```
class Solution {
public:
    int getLucky(string s, int k) {
        int ans = convert(s);
        for (int i = 1; i < k; ++i)
            ans = getDigitSum(ans);
        return ans;
    }

private:
    int convert(string s) {
        int sum = 0;
        for (const char c : s) {
            const int val = c - 'a' + 1;
            // Do one transform to prevent integer overflow.
            sum += val < 10 ? val : (val % 10 + val / 10);
        }
        return sum;
    }

    int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
};
```

1010. 1946.cpp

Path: LeetCode-main/solutions/1946. Largest Number After Mutating Substring/1946.cpp

```
class Solution {
public:
    string maximumNumber(string num, vector<int>& change) {
        bool mutated = false;

        for (char& c : num) {
            const int d = c - '0';
            c = '0' + max(d, change[d]);
            if (mutated && d > change[d])
                return num;
            if (d < change[d])
                mutated = true;
        }
        return num;
    }
};
```

1011. 1947.cpp

Path: LeetCode-main/solutions/1947. Maximum Compatibility Score Sum/1947.cpp

```
class Solution {
public:
    int maxCompatibilitySum(vector<vector<int>>& students,
                           vector<vector<int>>& mentors) {
        int ans = 0;
        dfs(students, mentors, 0, /*score=*/0, vector<bool>(students.size()), ans);
        return ans;
    }

private:
    void dfs(const vector<vector<int>>& students,
             const vector<vector<int>>& mentors, int i, int scoreSum,
             vector<bool>&& used, int& ans) {
        if (i == students.size()) {
            ans = max(ans, scoreSum);
            return;
        }
        for (int j = 0; j < students.size(); ++j) {
            if (used[j])
                continue;
            used[j] = true; // The `mentors[j]` is used.
            dfs(students, mentors, i + 1,
                scoreSum + getScore(students[i], mentors[j]), std::move(used), ans);
            used[j] = false;
        }
    }

    int getScore(const vector<int>& student, const vector<int>& mentor) {
        int score = 0;
        for (int i = 0; i < student.size(); ++i)
            if (student[i] == mentor[i])
                ++score;
        return score;
    }
};
```

1012. 1948.cpp

Path: LeetCode-main/solutions/1948. Delete Duplicate Folders in System/1948.cpp

```
struct TrieNode {
    unordered_map<string, shared_ptr<TrieNode>> children;
    bool deleted = false;
};

class Solution {
public:
    vector<vector<string>> deleteDuplicateFolder(vector<vector<string>>& paths) {
        vector<vector<string>> ans;
        vector<string> path;
        unordered_map<string, vector<shared_ptr<TrieNode>>> subtreeToNodes;

        ranges::sort(paths);

        for (const vector<string>& path : paths) {
            shared_ptr<TrieNode> node = root;
            for (const string& s : path) {
                if (!node->children.contains(s))
                    node->children[s] = make_shared<TrieNode>();
                node = node->children[s];
            }
        }

        buildSubtreeToRoots(root, subtreeToNodes);

        for (const auto& [_, nodes] : subtreeToNodes)
            if (nodes.size() > 1)
                for (shared_ptr<TrieNode> node : nodes)
                    node->deleted = true;

        constructPath(root, path, ans);
        return ans;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    string buildSubtreeToRoots(
        shared_ptr<TrieNode> node,
        unordered_map<string, vector<shared_ptr<TrieNode>>>& subtreeToNodes) {
        string subtree = "(";
        for (const auto& [s, child] : node->children)
            subtree += s + buildSubtreeToRoots(child, subtreeToNodes);
        subtree += ")";
        if (subtree != "()")
            subtreeToNodes[subtree].push_back(node);
        return subtree;
    }

    void constructPath(shared_ptr<TrieNode> node, vector<string>& path,
                      vector<vector<string>>& ans) {
        for (const auto& [s, child] : node->children)
            if (!child->deleted) {
                path.push_back(s);
                constructPath(child, path, ans);
                path.pop_back();
            }
        if (!path.empty())
            ans.push_back(path);
    }
};
```

1013. 1950.cpp

Path: LeetCode-main/solutions/1950. Maximum of Minimum Values in All Subarrays/1950.cpp

```
class Solution {
public:
    // Similar to 1950. Maximum of Minimum Values in All Subarrays
    vector<int> findMaximums(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans(n);
        // prevMin[i] := the index k s.t.
        // nums[k] is the previous minimum in nums[0..n)
        vector<int> prevMin(n, -1);
        // nextMin[i] := the index k s.t.
        // nums[k] is the next minimum in nums[i + 1..n)
        vector<int> nextMin(n, n);
        stack<int> stack;

        for (int i = 0; i < n; ++i) {
            while (!stack.empty() && nums[stack.top()] > nums[i]) {
                const int index = stack.top();
                stack.pop();
                nextMin[index] = i;
            }
            if (!stack.empty())
                prevMin[i] = stack.top();
            stack.push(i);
        }

        // For each nums[i], let l = nextMin[i] + 1 and r = nextMin[i] - 1.
        // nums[i] is the minimum in nums[l..r].
        // So, the ans[r - l + 1] will be at least nums[i].
        for (int i = 0; i < n; ++i) {
            const int sz = nextMin[i] - prevMin[i] - 1;
            ans[sz - 1] = max(ans[sz - 1], nums[i]);
        }

        // ans[i] should always >= ans[i + 1..n).
        for (int i = n - 2; i >= 0; --i)
            ans[i] = max(ans[i], ans[i + 1]);

        return ans;
    }
};
```


1014. 1952.cpp

Path: LeetCode-main/solutions/1952. Three Divisors/1952.cpp

```
class Solution {
public:
    bool isThree(int n) {
        if (n == 1)
            return false;
        // The numbers with exactly three divisors are perfect squares of a prime
        // number.
        const int root = sqrt(n);
        return root * root == n && isPrime(root);
    }

private:
    bool isPrime(int num) {
        for (int i = 2; i <= sqrt(num); ++i)
            if (num % i == 0)
                return false;
        return true;
    }
};
```

1015. 1953.cpp

Path: LeetCode-main/solutions/1953. Maximum Number of Weeks for Which You Can Work/1953.cpp

```
class Solution {
public:
    long long numberOfWeeks(vector<int>& milestones) {
        // The best strategy is to pick "max, nonMax, max, nonMax, ...".
        const int mx = ranges::max(milestones);
        const long sum = accumulate(milestones.begin(), milestones.end(), 0L);
        const long nonMax = sum - mx;
        return min(sum, 2 * nonMax + 1);
    }
};
```

1016. 1954.cpp

Path: LeetCode-main/solutions/1954. Minimum Garden Perimeter to Collect Enough Apples/1954.cpp

```
class Solution {
public:
    long long minimumPerimeter(long long neededApples) {
        long l = 1;
        long r = 100'000; // \sqrt [3] {10^{15}}

        while (l < r) {
            const long m = (l + r) / 2;
            if (numApples(m) >= neededApples)
                r = m;
            else
                l = m + 1;
        }

        return l * 8;
    }

private:
    // Returns the number of apples at the k-th level.
    // k := the level making perimeter = 8k
    // p(k) := the number of apples at the k-th level on the perimeter
    // n(k) := the number of apples at the k-th level not on the perimeter
    //
    // p(1) = 1 + 2
    // p(2) = 3 + 2 + 3 + 4
    // p(3) = 5 + 4 + 3 + 4 + 5 + 6
    // p(4) = 7 + 6 + 5 + 4 + 5 + 6 + 7 + 8
    // p(k) = k + 2(k+1) + 2(k+2) + ... + 2(k+k-1) + 2k
    //       = k + 2k^2 + 2*k(k-1)/2
    //       = k + 2k^2 + k^2 - k = 3k^2
    //
    // n(k) = p(1) + p(2) + p(3) + ... + p(k)
    //       = 3*1^2 + 3*4^2 + 3*9^2 + ... + 3*k^2
    //       = 3 * (1 + 4 + 9 + ... + k^2)
    //       = 3 * k(k+1)(2k+1)/6 = k(k+1)(2k+1)/2
    // So, the number of apples at the k-th level should be
    // k(k+1)(2k+1)/2 * 4 = 2k(k+1)(2k+1)
    long numApples(long k) {
        return 2 * k * (k + 1) * (2 * k + 1);
    }
};
```

1017. 1955-2.cpp

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955-2.cpp

```
class Solution {
public:
    int countSpecialSubsequences(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        // dp[i][j] := the number of increasing subsequences of the first i numbers
        // that end in j
        vector<vector<long>> dp(n, vector<long>(3));

        if (nums[0] == 0)
            dp[0][0] = 1;

        for (int i = 1; i < n; ++i) {
            for (int ending = 0; ending < 3; ++ending)
                dp[i][ending] = dp[i - 1][ending];

            if (nums[i] == 0)
                // 1. The number of the previous subsequences that end in 0.
                // 2. Append a 0 to the previous subsequences that end in 0.
                // 3. Start a new subsequence from this 0.
                dp[i][0] = dp[i - 1][0] * 2 + 1;
            else if (nums[i] == 1)
                // 1. The number of the previous subsequences that end in 1.
                // 2. Append a 1 to the previous subsequences that end in 1.
                // 3. Append a 1 to the previous subsequences that end in 0.
                dp[i][1] = dp[i - 1][1] * 2 + dp[i - 1][0];
            else // nums[i] == 2
                // 1. The number of the previous subsequences that end in 2.
                // 2. Append a 2 to the previous subsequences that end in 2.
                // 3. Append a 2 to the previous subsequences that end in 1.
                dp[i][2] = dp[i - 1][2] * 2 + dp[i - 1][1];

            for (int ending = 0; ending < 3; ++ending)
                dp[i][ending] %= kMod;
        }

        return dp.back()[2];
    }
};
```

1018. 1955-3.cpp

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955-3.cpp

```
class Solution {
public:
    int countSpecialSubsequences(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        // dp[j] := the number of increasing subsequences of the numbers so far that
        // end in j
        vector<long> dp(3);

        if (nums[0] == 0)
            dp[0] = 1;

        for (int i = 1; i < n; ++i) {
            if (nums[i] == 0)
                dp[0] = dp[0] * 2 + 1;
            else if (nums[i] == 1)
                dp[1] = dp[1] * 2 + dp[0];
            else // nums[i] == 2
                dp[2] = dp[2] * 2 + dp[1];

            for (int ending = 0; ending < 3; ++ending)
                dp[ending] %= kMod;
        }

        return dp[2];
    }
};
```

1019. 1955.cpp

Path: LeetCode-main/solutions/1955. Count Number of Special Subsequences/1955.cpp

```
class Solution {
public:
    int countSpecialSubsequences(vector<int>& nums) {
        vector<vector<int>> mem(nums.size(), vector<int>(4, -1));
        return countSpecialSubsequences(nums, 0, -1, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of increasing subsequences of the first i numbers, where
    // the previous number is `prev`.
    int countSpecialSubsequences(const vector<int>& nums, int i, int prev,
                                vector<vector<int>>& mem) {
        if (i == nums.size())
            return prev == 2;
        const int j = prev + 1; // Map -1/0/1/2 to 0/1/2/3 respectively.
        if (mem[i][j] != -1)
            return mem[i][j];

        long res = 0;

        // Don't include `nums[i]`.
        res += countSpecialSubsequences(nums, i + 1, prev, mem);

        // Include `nums[i]`.
        if (nums[i] == prev)
            res += countSpecialSubsequences(nums, i + 1, prev, mem);
        if (prev == -1 && nums[i] == 0)
            res += countSpecialSubsequences(nums, i + 1, 0, mem);
        if (prev == 0 && nums[i] == 1)
            res += countSpecialSubsequences(nums, i + 1, 1, mem);
        if (prev == 1 && nums[i] == 2)
            res += countSpecialSubsequences(nums, i + 1, 2, mem);

        res %= kMod;
        return mem[i][j] = res;
    }
};
```

1020. 1956.cpp

Path: LeetCode-main/solutions/1956. Minimum Time For K Virus Variants to Spread/1956.cpp

```
class Solution {
public:
    int minDayskVariants(vector<vector<int>>& points, int k) {
        constexpr int kMax = 100;
        int ans = INT_MAX;

        for (int a = 1; a <= kMax; ++a)
            for (int b = 1; b <= kMax; ++b) {
                // Stores the k minimum distances of points that can reach (a, b).
                priority_queue<int> maxHeap;
                for (const vector<int>& point : points) {
                    const int x = point[0];
                    const int y = point[1];
                    maxHeap.push(abs(x - a) + abs(y - b));
                    if (maxHeap.size() > k)
                        maxHeap.pop();
                }
                ans = min(ans, maxHeap.top());
            }
        return ans;
    }
};
```

1021. 1957.cpp

Path: LeetCode-main/solutions/1957. Delete Characters to Make Fancy String/1957.cpp

```
class Solution {
public:
    string makeFancyString(string s) {
        string ans;
        for (const char c : s)
            if (ans.length() < 2 || ans.back() != c || ans[ans.size() - 2] != c)
                ans.push_back(c);
        return ans;
    }
};
```


1022. 1958.cpp

Path: LeetCode-main/solutions/1958. Check if Move is Legal/1958.cpp

```
class Solution {
public:
    bool checkMove(vector<vector<char>>& board, int rMove, int cMove,
                   char color) {
        constexpr int kDirs[8][2] = {{-1, -1}, {-1, 0}, {-1, 1}, {0, -1},
                                       {0, 1}, {1, -1}, {1, 0}, {1, 1}};

        for (const auto& [dx, dy] : kDirs) {
            int cellsCount = 2;
            int i = rMove + dx;
            int j = cMove + dy;
            while (0 <= i && i < 8 && 0 <= j && j < 8) {
                // There are no free cells in between.
                if (board[i][j] == '.')
                    break;
                // Need >= 3 cells.
                if (cellsCount == 2 && board[i][j] == color)
                    break;
                // >= 3 cells.
                if (board[i][j] == color)
                    return true;
                i += dx;
                j += dy;
                ++cellsCount;
            }
        }
        return false;
    }
};
```

1023. 1959.cpp

Path: LeetCode-main/solutions/1959. Minimum Total Space Wasted With K Resizing Operations/1959.cpp

```
class Solution {
public:
    int minSpaceWastedKResizing(vector<int>& nums, int k) {
        vector<vector<int>> mem(nums.size(), vector<int>(k + 1, -1));
        return minSpaceWasted(nums, 0, k, mem);
    }

private:
    static constexpr int kMax = 200'000'000;

    // Returns the minimum space wasted for nums[i..n) if you can resize k times.
    int minSpaceWasted(const vector<int>& nums, int i, int k,
        vector<vector<int>>& mem) {
        if (i == nums.size())
            return 0;
        if (k == -1)
            return kMax;
        if (mem[i][k] != -1)
            return mem[i][k];

        int res = kMax;
        int sum = 0;
        int maxNum = nums[i];

        for (int j = i; j < nums.size(); ++j) {
            sum += nums[j];
            maxNum = max(maxNum, nums[j]);
            const int wasted = maxNum * (j - i + 1) - sum;
            res = min(res, minSpaceWasted(nums, j + 1, k - 1, mem) + wasted);
        }

        return mem[i][k] = res;
    }
};
```

1024. 1960-2.cpp

Path: LeetCode-main/solutions/1960. Maximum Product of the Length of Two Palindromic Substrings/1960-2.cpp

```
class Solution {
public:
    long long maxProduct(string s) {
        const int n = s.length();
        long ans = 1;
        vector<long> pows{1}; // pows[i] := kBase^i % kHash
        // hashL[i] = the hash of the first i letters of s, where hashL[i] =
        // (26^(i - 1) * s[0] + 26^(i - 2) * s[1] + ... + s[i - 1]) % kHash
        vector<long> hashL{0};
        // hashR[i] = the hash of the last i letters of s, where hashR[i] =
        // (26^(i - 1) * s[-1] + 26^(i - 2) * s[-2] + ... + s[-i]) % kHash
        vector<long> hashR{0};
        // maxLeft[i] := the maximum odd length of palindromes in s[0..i]
        vector<int> maxLeft(n);
        // maxRight[i] := the maximum odd length of palindromes in s[i..n - 1]
        vector<int> maxRight(n);

        for (int i = 0; i < n; ++i)
            pows.push_back(pows.back() * kBase % kHash);

        for (int i = 0; i < n; ++i)
            hashL.push_back((hashL.back() * kBase + val(s[i])) % kHash);

        for (int i = n - 1; i >= 0; --i)
            hashR.push_back((hashR.back() * kBase + val(s[i])) % kHash);

        ranges::reverse(hashR);

        int maxLength = 1;
        for (int r = 0; r < n; ++r) {
            const int l = (r - maxLength - 2) + 1;
            if (l >= 0 && isPalindrome(l, r + 1, hashL, hashR, pows))
                maxLength += 2;
            maxLeft[r] = maxLength;
        }

        maxLength = 1;
        for (int l = n - 1; l >= 0; --l) {
            const int r = (l + maxLength + 2) - 1;
            if (r < n && isPalindrome(l, r + 1, hashL, hashR, pows))
                maxLength += 2;
            maxRight[l] = maxLength;
        }

        for (int i = 1; i < n; ++i)
            ans = max(ans, static_cast<long>(maxLeft[i - 1]) * maxRight[i]);

        return ans;
    }

private:
    static constexpr int kBase = 26;
    static constexpr int kHash = 1'000'000'007;

    static constexpr int val(char c) {
        return c - 'a';
    }

    // Returns true if s[l..r) is a palindrome.
    bool isPalindrome(int l, int r, const vector<long>& hashL,
                     const vector<long>& hashR, const vector<long>& pows) {
        return getLeftRollingHash(l, r, hashL, pows) ==
            getRightRollingHash(l, r, hashR, pows);
    }

    // Returns the left rolling hash of s[l..r).
    long getLeftRollingHash(int l, int r, const vector<long>& hashL,
                           const vector<long>& pows) {
        const long h = (hashL[r] - hashL[l] * pows[r - l]) % kHash;
        return h < 0 ? h + kHash : h;
    }

    // Returns the right rolling hash of s[l..r).
    long getRightRollingHash(int l, int r, const vector<long>& hashR,
                             const vector<long>& pows) {
        const long h = (hashR[l] - hashR[r] * pows[r - l]) % kHash;
        return h < 0 ? h + kHash : h;
    }
};
```

1025. 1960.cpp

Path: LeetCode-main/solutions/1960. Maximum Product of the Length of Two Palindromic Substrings/1960.cpp

```
class Solution {
public:
    long long maxProduct(string s) {
        const int n = s.length();
        long ans = 1;
        // maxLeft[i] := the maximum odd length of palindromes in s[0..i]
        vector<int> maxLeft = manacher(s, n);
        // maxRight[i] := the maximum odd length of palindromes in s[i..n - 1]
        vector<int> maxRight = manacher(string{s.rbegin(), s.rend()}, n);

        ranges::reverse(maxRight);

        for (int i = 1; i < n; ++i)
            ans = max(ans, static_cast<long>(maxLeft[i - 1]) * maxRight[i]);

        return ans;
    }

private:
    vector<int> manacher(const string& s, int n) {
        vector<int> maxExtends(n);
        vector<int> leftToRight(n, 1);
        int center = 0;

        for (int i = 0; i < n; ++i) {
            const int r = center + maxExtends[center] - 1;
            const int mirrorIndex = center - (i - center);
            int extend = i > r ? 1 : min(maxExtends[mirrorIndex], r - i + 1);
            while (i - extend >= 0 && i + extend < n &&
                    s[i - extend] == s[i + extend]) {
                leftToRight[i + extend] = 2 * extend + 1;
                ++extend;
            }
            maxExtends[i] = extend;
            if (i + maxExtends[i] >= r)
                center = i;
        }

        for (int i = 1; i < n; ++i)
            leftToRight[i] = max(leftToRight[i], leftToRight[i - 1]);

        return leftToRight;
    }
};
```

1026. 1961.cpp

Path: LeetCode-main/solutions/1961. Check If String Is a Prefix of Array/1961.cpp

```
class Solution {
public:
    bool isPrefixString(string s, vector<string>& words) {
        string prefix;
        for (const string& word : words) {
            prefix += word;
            if (prefix == s)
                return true;
        }
        return false;
    }
};
```

1027. 1962.cpp

Path: LeetCode-main/solutions/1962. Remove Stones to Minimize the Total/1962.cpp

```
class Solution {
public:
    int minStoneSum(vector<int>& piles, int k) {
        int ans = accumulate(piles.begin(), piles.end(), 0);
        priority_queue<int> maxHeap;

        for (const int pile : piles)
            maxHeap.push(pile);

        for (int i = 0; i < k; ++i) {
            const int maxPile = maxHeap.top();
            maxHeap.pop();
            maxHeap.push(maxPile - maxPile / 2);
            ans -= maxPile / 2;
        }

        return ans;
    }
};
```

1028. 1963.cpp

Path: LeetCode-main/solutions/1963. Minimum Number of Swaps to Make the String Balanced/1963.cpp

```
class Solution {
public:
    int minSwaps(string s) {
        // Cancel out all the matched pairs, then we'll be left with "]]]..[[[".
        // The answer is ceil(the number of unmatched pairs / 2).
        int unmatched = 0;

        for (const char c : s)
            if (c == '[')
                ++unmatched;
            else if (unmatched > 0) // c == ']' and there's a match.
                --unmatched;

        return (unmatched + 1) / 2;
    }
};
```

1029. 1964.cpp

Path: LeetCode-main/solutions/1964. Find the Longest Valid Obstacle Course at Each Position/1964.cpp

```
class Solution {
public:
    // Similar to 300. Longest Increasing Subsequence
    vector<int> longestObstacleCourseAtEachPosition(vector<int>& obstacles) {
        vector<int> ans;
        // tails[i] := the minimum tail of all the increasing subsequences having
        // length i + 1
        vector<int> tails;

        for (const int obstacle : obstacles)
            if (tails.empty() || obstacle >= tails.back()) {
                tails.push_back(obstacle);
                ans.push_back(tails.size());
            } else {
                const int index = firstGreater(tails, obstacle);
                tails[index] = obstacle;
                ans.push_back(index + 1);
            }

        return ans;
    }

private:
    int firstGreater(const vector<int>& arr, int target) {
        return ranges::upper_bound(arr, target) - arr.begin();
    }
};
```


1030. 1966.cpp

Path: LeetCode-main/solutions/1966. Binary Searchable Numbers in an Unsorted Array/1966.cpp

```
class Solution {
public:
    int binarySearchableNumbers(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        // prefixMaxs[i] := max(nums[0..i))
        vector<int> prefixMaxs(n);
        // suffixMins[i] := min(nums[i + 1..n))
        vector<int> suffixMins(n);

        // Fill in `prefixMaxs`.
        prefixMaxs[0] = INT_MIN;
        for (int i = 1; i < n; ++i)
            prefixMaxs[i] = max(prefixMaxs[i - 1], nums[i - 1]);

        // Fill in `suffixMins`.
        suffixMins[n - 1] = INT_MAX;
        for (int i = n - 2; i >= 0; --i)
            suffixMins[i] = min(suffixMins[i + 1], nums[i + 1]);

        for (int i = 0; i < n; ++i)
            if (prefixMaxs[i] < nums[i] && nums[i] < suffixMins[i])
                ++ans;

        return ans;
    }
};
```

1031. 1967.cpp

Path: LeetCode-main/solutions/1967. Number of Strings That Appear as Substrings in Word/1967.cpp

```
class Solution {
public:
    int numOfStrings(vector<string>& patterns, string word) {
        return ranges::count_if(patterns, [&word](const string& pattern) {
            return word.find(pattern) != word.npos;
        });
    }
};
```

1032. 1968.cpp

Path: LeetCode-main/solutions/1968. Array With Elements Not Equal to Average of Neighbors/1968.cpp

```
class Solution {
public:
    vector<int> rearrangeArray(vector<int>& nums) {
        ranges::sort(nums);
        for (int i = 1; i < nums.size(); i += 2)
            swap(nums[i], nums[i - 1]);
        return nums;
    }
};
```

1033. 1969.cpp

Path: LeetCode-main/solutions/1969. Minimum Non-Zero Product of the Array Elements/1969.cpp

```
class Solution {
public:
    int minNonZeroProduct(int p) {
        // Can always turn  $[1..2^p - 1]$  to  $[1, 1, \dots, 2^p - 2, 2^p - 2, 2^p - 1]$ .
        const long n = 1L << p;
        const long halfCount = n / 2 - 1;
        return modPow(n - 2, halfCount) * ((n - 1) % kMod) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1L;
        x %= kMod;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % kMod;
        return modPow(x * x, n / 2) % kMod;
    }
};
```

1034. 1970.cpp

Path: LeetCode-main/solutions/1970. Last Day Where You Can Still Cross/1970.cpp

```
class Solution {
public:
    int latestDayToCross(int row, int col, vector<vector<int>>& cells) {
        int ans = 0;
        int l = 1;
        int r = cells.size() - 1;

        while (l <= r) {
            const int m = (l + r) / 2;
            if (canWalk(m, row, col, cells)) {
                ans = m;
                l = m + 1;
            } else {
                r = m - 1;
            }
        }

        return ans;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    bool canWalk(int day, int row, int col, const vector<vector<int>>& cells) {
        vector<vector<int>> matrix(row, vector<int>(col));
        for (int i = 0; i < day; ++i) {
            const int x = cells[i][0] - 1;
            const int y = cells[i][1] - 1;
            matrix[x][y] = 1;
        }

        queue<pair<int, int>> q;

        for (int j = 0; j < col; ++j)
            if (matrix[0][j] == 0) {
                q.emplace(0, j);
                matrix[0][j] = 1;
            }

        while (!q.empty()) {
            const auto [i, j] = q.front();
            q.pop();
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == row || y < 0 || y == col)
                    continue;
                if (matrix[x][y] == 1)
                    continue;
                if (x == row - 1)
                    return true;
                q.emplace(x, y);
                matrix[x][y] = 1;
            }
        }

        return false;
    }
};
```

1035. 1971.cpp

Path: LeetCode-main/solutions/1971. Find if Path Exists in Graph/1971.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    bool validPath(int n, vector<vector<int>>& edges, int source,
                  int destination) {
        UnionFind uf(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            uf.unionByRank(u, v);
        }

        return uf.find(source) == uf.find(destination);
    }
};
```

1036. 1973.cpp

Path: LeetCode-main/solutions/1973. Count Nodes Equal to Sum of Descendants/1973.cpp

```
struct T {
    long sum;
    int count;
};

class Solution {
public:
    int equalToDescendants(TreeNode* root) {
        return dfs(root).count;
    }

private:
    T dfs(TreeNode* root) {
        if (root == nullptr)
            return T{.sum = 0, .count = 0};
        T left = dfs(root->left);
        T right = dfs(root->right);
        return T{.sum = root->val + left.sum + right.sum,
                .count = left.count + right.count +
                    (root->val == left.sum + right.sum ? 1 : 0)};
    }
};
```

1037. 1974.cpp

Path: LeetCode-main/solutions/1974. Minimum Time to Type Word Using Special Typewriter/1974.cpp

```
class Solution {
public:
    int minTimeToType(string word) {
        int moves = 0;
        char letter = 'a';

        for (const char c : word) {
            const int diff = abs(c - letter);
            moves += min(diff, 26 - diff);
            letter = c;
        }

        return moves + word.length();
    }
};
```


1038. 1975.cpp

Path: LeetCode-main/solutions/1975. Maximum Matrix Sum/1975.cpp

```
class Solution {
public:
    long long maxMatrixSum(vector<vector<int>>& matrix) {
        long absSum = 0;
        int minAbs = INT_MAX;
        // 0 := even number of negatives
        // 1 := odd number of negatives
        int oddNeg = 0;

        for (const vector<int>& row : matrix)
            for (const int num : row) {
                absSum += abs(num);
                minAbs = min(minAbs, abs(num));
                if (num < 0)
                    oddNeg ^= 1;
            }

        return absSum - oddNeg * minAbs * 2;
    }
};
```

1039. 1976.cpp

Path: LeetCode-main/solutions/1976. Number of Ways to Arrive at Destination/1976.cpp

```
class Solution {
public:
    int countPaths(int n, vector<vector<int>>& roads) {
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& road : roads) {
            const int u = road[0];
            const int v = road[1];
            const int w = road[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        return dijkstra(graph, 0, n - 1);
    }

private:
    // Similar to 1786. Number of Restricted Paths From First to Last Node
    int dijkstra(const vector<vector<pair<int, int>>>& graph, int src, int dst) {
        constexpr int kMod = 1'000'000'007;
        vector<long> ways(graph.size());
        vector<long> dist(graph.size(), LONG_MAX);

        ways[src] = 1;
        dist[src] = 0;
        using P = pair<long, int>; // (d, u)
        priority_queue<P, vector<P>, greater<>> minHeap;
        minHeap.emplace(dist[src], src);

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (d > dist[u])
                continue;
            for (const auto& [v, w] : graph[u])
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    ways[v] = ways[u];
                    minHeap.emplace(dist[v], v);
                } else if (d + w == dist[v]) {
                    ways[v] += ways[u];
                    ways[v] %= kMod;
                }
        }

        return ways[dst];
    }
};
```

1040. 1977.cpp

Path: LeetCode-main/solutions/1977. Number of Ways to Separate Numbers/1977.cpp

```
class Solution {
public:
    int numberOfCombinations(string num) {
        if (num[0] == '0')
            return 0;

        constexpr int kMod = 1'000'000'007;
        const int n = num.size();
        // dp[i][k] := the number of possible lists of integers ending in num[i]
        // with the length of the last number being 1..k
        vector<vector<long>> dp(n, vector<long>(n + 1));
        // lcs[i][j] := the number of the same digits in num[i..n) and num[j..n)
        vector<vector<int>> lcs(n + 1, vector<int>(n + 1));

        for (int i = n - 1; i >= 0; --i)
            for (int j = i + 1; j < n; ++j)
                if (num[i] == num[j])
                    lcs[i][j] = lcs[i + 1][j + 1] + 1;

        for (int i = 0; i < n; ++i)
            for (int k = 1; k <= i + 1; ++k) {
                dp[i][k] += dp[i][k - 1];
                dp[i][k] %= kMod;
                // The last number is num[s..i].
                const int s = i - k + 1;
                if (num[s] == '0')
                    // the number of possible lists of integers ending in num[i] with the
                    // length of the last number being k
                    continue;
                if (s == 0) {
                    // the whole string
                    dp[i][k] += 1;
                    continue;
                }
                if (s < k) {
                    // The length k is not enough, so add the number of possible lists of
                    // integers in num[0..s - 1].
                    dp[i][k] += dp[s - 1][s];
                    continue;
                }
                const int l = lcs[s - k][s];
                if (l >= k || num[s - k + l] <= num[s + l])
                    // Have enough length k and num[s - k..s - 1] <= num[j..i].
                    dp[i][k] += dp[s - 1][k];
                else
                    // Have enough length k but num[s - k..s - 1] > num[j..i].
                    dp[i][k] += dp[s - 1][k - 1];
            }

        return dp[n - 1][n] % kMod;
    }
};
```

1041. 1979.cpp

Path: LeetCode-main/solutions/1979. Find Greatest Common Divisor of Array/1979.cpp

```
class Solution {
public:
    int findGCD(vector<int>& nums) {
        return __gcd(ranges::min(nums), ranges::max(nums));
    }
};
```

1042. 198-2.cpp

Path: LeetCode-main/solutions/198. House Robber/198-2.cpp

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int prev1 = 0; // dp[i - 1]
        int prev2 = 0; // dp[i - 2]

        for (const int num : nums) {
            const int dp = max(prev1, prev2 + num);
            prev2 = prev1;
            prev1 = dp;
        }

        return prev1;
    }
};
```

1043. 198.cpp

Path: LeetCode-main/solutions/198. House Robber/198.cpp

```
class Solution {
public:
    int rob(vector<int>& nums) {
        if (nums.empty())
            return 0;
        if (nums.size() == 1)
            return nums[0];

        // dp[i] := the maximum money of robbing nums[0..i]
        vector<int> dp(nums.size());
        dp[0] = nums[0];
        dp[1] = max(nums[0], nums[1]);

        for (int i = 2; i < nums.size(); ++i)
            dp[i] = max(dp[i - 1], dp[i - 2] + nums[i]);

        return dp.back();
    }
};
```

1044. 1980-2.cpp

Path: LeetCode-main/solutions/1980. Find Unique Binary String/1980-2.cpp

```
class Solution {
public:
    string findDifferentBinaryString(vector<string>& nums) {
        string ans;

        // Flip the i-th bit for each nums[i] so that `ans` is unique.
        for (int i = 0; i < nums.size(); ++i)
            ans += nums[i][i] == '0' ? '1' : '0';

        return ans;
    }
};
```

1045. 1980.cpp

Path: LeetCode-main/solutions/1980. Find Unique Binary String/1980.cpp

```
class Solution {
public:
    string findDifferentBinaryString(vector<string>& nums) {
        const int bitSize = nums[0].length();
        const int maxNum = 1 << bitSize;
        unordered_set<int> numsSet;

        for (const string& num : nums)
            numsSet.insert(stoi(num, nullptr, 2));

        for (int num = 0; num < maxNum; ++num)
            if (!numsSet.contains(num))
                return std::bitset<16>(num).to_string().substr(16 - bitSize);

        throw;
    }
};
```


1046. 1981.cpp

Path: LeetCode-main/solutions/1981. Minimize the Difference Between Target and Chosen Elements/1981.cpp

```
class Solution {
public:
    int minimizeTheDifference(vector<vector<int>>& mat, int target) {
        const int minSum = getMinSum(mat);
        if (minSum >= target) // No need to consider any larger combination.
            return minSum - target;

        const int maxSum = getMaxSum(mat);
        vector<vector<int>> mem(mat.size(), vector<int>(maxSum + 1, -1));
        return minimizeTheDifference(mat, 0, 0, target, mem);
    }

private:
    int minimizeTheDifference(const vector<vector<int>>& mat, int i, int sum,
                             int target, vector<vector<int>>& mem) {
        if (i == mat.size())
            return abs(sum - target);
        if (mem[i][sum] != -1)
            return mem[i][sum];
        int res = INT_MAX;
        for (const int num : mat[i])
            res = min(res, minimizeTheDifference(mat, i + 1, sum + num, target, mem));
        return mem[i][sum] = res;
    }

    int getMinSum(const vector<vector<int>>& mat) {
        return accumulate(
            mat.begin(), mat.end(), 0,
            [](int acc, const vector<int>& row) { return acc + ranges::min(row); });
    }

    int getMaxSum(const vector<vector<int>>& mat) {
        return accumulate(
            mat.begin(), mat.end(), 0,
            [](int acc, const vector<int>& row) { return acc + ranges::max(row); });
    }
};
```

1047. 1982.cpp

Path: LeetCode-main/solutions/1982. Find Array Given Subset Sums/1982.cpp

```
class Solution {
public:
    vector<int> recoverArray(int n, vector<int>& sums) {
        ranges::sort(sums);
        return recover(sums);
    }

private:
    vector<int> recover(const vector<int>& sums) {
        if (sums.size() == 1) // sums[0] must be 0.
            return {};

        // Either num or -num must be in the final array.
        // num + sumsExcludingNum = sumsIncludingNum
        // -num + sumsIncludingNum = sumsExcludingNum
        unordered_map<int, int> count;
        for (const int sum : sums)
            ++count[sum];

        const int num = sums[1] - sums[0];
        vector<int> sumsExcludingNum;
        vector<int> sumsIncludingNum;
        bool chooseSumsIncludingNum = false;

        for (const int sum : sums) {
            if (count[sum] == 0)
                continue;
            --count[sum];
            --count[sum + num];
            sumsExcludingNum.push_back(sum);
            sumsIncludingNum.push_back(sum + num);
            if (sum + num == 0)
                chooseSumsIncludingNum = true;
        }

        // Choose `sumsExcludingNum` by default since we want to gradually strip
        // `num` from each sum in `sums` to have the final array. However, we should
        // always choose the group of sums with 0 since it's a must-have.
        vector<int> recovered =
            recover(chooseSumsIncludingNum ? sumsIncludingNum : sumsExcludingNum);
        recovered.push_back(chooseSumsIncludingNum ? -num : num);
        return recovered;
    }
};
```

1048. 1983.cpp

Path: LeetCode-main/solutions/1983. Widest Pair of Indices With Equal Range Sum/1983.cpp

```
class Solution {
public:
    int widestPairOfIndices(vector<int>& nums1, vector<int>& nums2) {
        int ans = 0;
        int prefix = 0;
        unordered_map<int, int> prefixToIndex{{0, -1}};

        for (int i = 0; i < nums1.size(); ++i) {
            prefix += nums1[i] - nums2[i];
            if (const auto it = prefixToIndex.find(prefix);
                it != prefixToIndex.cend())
                ans = max(ans, i - it->second);
            else
                prefixToIndex[prefix] = i;
        }

        return ans;
    }
};
```

1049. 1984.cpp

Path: LeetCode-main/solutions/1984. Minimum Difference Between Highest and Lowest of K Scores/1984.cpp

```
class Solution {
public:
    int minimumDifference(vector<int>& nums, int k) {
        ranges::sort(nums);

        int ans = nums[k - 1] - nums[0];

        for (int i = k; i < nums.size(); ++i)
            ans = min(ans, nums[i] - nums[i - k + 1]);

        return ans;
    }
};
```

1050. 1985.cpp

Path: LeetCode-main/solutions/1985. Find the Kth Largest Integer in the Array/1985.cpp

```
class Solution {
public:
    // Similar to 215. Kth Largest Element in an Array
    string kthLargestNumber(vector<string>& nums, int k) {
        auto compare = [](const string& a, const string& b) {
            return a.length() == b.length() ? a > b : a.length() > b.length();
        };
        priority_queue<string, vector<string>, decltype(compare)> minHeap(compare);

        for (const string& num : nums) {
            minHeap.push(num);
            if (minHeap.size() > k)
                minHeap.pop();
        }
        return minHeap.top();
    }
};
```

1051. 1986.cpp

Path: LeetCode-main/solutions/1986. Minimum Number of Work Sessions to Finish the Tasks/1986.cpp

```
class Solution {
public:
    int minSessions(vector<int>& tasks, int sessionTime) {
        for (int numSessions = 1; numSessions <= tasks.size(); ++numSessions)
            if (dfs(tasks, 0, vector<int>(numSessions), sessionTime))
                return numSessions;
        throw;
    }

    // Returns true if we can assign tasks[s..n) to `sessions`. Note that
    // `sessions` may be occupied by some tasks.
    bool dfs(const vector<int>& tasks, int s, vector<int>&& sessions,
             const int& sessionTime) {
        if (s == tasks.size())
            return true;

        for (int& session : sessions) {
            // Can't assign the tasks[s] to this session.
            if (session + tasks[s] > sessionTime)
                continue;
            // Assign the tasks[s] to this session.
            session += tasks[s];
            if (dfs(tasks, s + 1, std::move(sessions), sessionTime))
                return true;
            // Backtracking.
            session -= tasks[s];
            // If it's the first time we assign the tasks[s] to this session, then
            // future `session`s can't satisfy either.
            if (session == 0)
                return false;
        }

        return false;
    }
};
```

1052. 1987.cpp

Path: LeetCode-main/solutions/1987. Number of Unique Good Subsequences/1987.cpp

```
class Solution {
public:
    // Similar to 940. Distinct Subsequences II
    int numberOfUniqueGoodSubsequences(string binary) {
        constexpr int kMod = 1'000'000'007;
        // endsIn[i] := the number of subsequence that end in ('0' + i)
        vector<int> endsIn(2);

        for (const char c : binary) {
            endsIn[c - '0'] = (endsIn[0] + endsIn[1]) % kMod;
            // Don't count '0' since we want to avoid the leading zeros case.
            // However, we can always count '1'.
            if (c == '1')
                ++endsIn[1];
        }

        // Count '0' in the end.
        return (endsIn[0] + endsIn[1] +
                (binary.find('0') == string::npos ? 0 : 1)) %
            kMod;
    }
};
```

1053. 1989.cpp

Path: LeetCode-main/solutions/1989. Maximum Number of People That Can Be Caught in Tag/1989.cpp

```
class Solution {
public:
    int catchMaximumAmountofPeople(vector<int>& team, int dist) {
        int ans = 0;
        int i = 0; // 0s index
        int j = 0; // 1s index

        while (i < team.size() && j < team.size())
            if (i + dist < j || team[i] != 0) {
                // Find the next 0 that can be caught by 1.
                ++i;
            } else if (j + dist < i || team[j] != 1) {
                // Find the next 1 that can catch 0.
                ++j;
            } else {
                // team[j] catches team[i], so move both.
                ++ans;
                ++i;
                ++j;
            }

        return ans;
    }
};
```


1054. 199-2.cpp

Path: LeetCode-main/solutions/199. Binary Tree Right Side View/199-2.cpp

```
class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        vector<int> ans;
        dfs(root, 0, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, int depth, vector<int>& ans) {
        if (root == nullptr)
            return;

        if (depth == ans.size())
            ans.push_back(root->val);
        dfs(root->right, depth + 1, ans);
        dfs(root->left, depth + 1, ans);
    }
};
```

1055. 199.cpp

Path: LeetCode-main/solutions/199. Binary Tree Right Side View/199.cpp

```
class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<int> ans;
        queue<TreeNode*> q{{root}};

        while (!q.empty()) {
            const int size = q.size();
            for (int i = 0; i < size; ++i) {
                TreeNode* node = q.front();
                q.pop();
                if (i == size - 1)
                    ans.push_back(node->val);
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
        }

        return ans;
    }
};
```

1056. 1991.cpp

Path: LeetCode-main/solutions/1991. Find the Middle Index in Array/1991.cpp

```
class Solution {
public:
    int findMiddleIndex(vector<int>& nums) {
        int prefix = 0;
        int suffix = accumulate(nums.begin(), nums.end(), 0);

        for (int i = 0; i < nums.size(); ++i) {
            suffix -= nums[i];
            if (prefix == suffix)
                return i;
            prefix += nums[i];
        }

        return -1;
    }
};
```

1057. 1992.cpp

Path: LeetCode-main/solutions/1992. Find All Groups of Farmland/1992.cpp

```
class Solution {
public:
    vector<vector<int>> findFarmland(vector<vector<int>>& land) {
        vector<vector<int>> ans;

        for (int i = 0; i < land.size(); ++i)
            for (int j = 0; j < land[0].size(); ++j)
                if (land[i][j] == 1) {
                    int x = i;
                    int y = j;
                    dfs(land, i, j, x, y);
                    ans.push_back({i, j, x, y});
                }

        return ans;
    }

private:
    void dfs(vector<vector<int>>& land, int i, int j, int& x, int& y) {
        if (i < 0 || i == land.size() || j < 0 || j == land[0].size())
            return;
        if (land[i][j] != 1)
            return;
        land[i][j] = 2; // Mark as visited.
        x = max(x, i);
        y = max(y, j);
        dfs(land, i + 1, j, x, y);
        dfs(land, i, j + 1, x, y);
    }
};
```

1058. 1993.cpp

Path: LeetCode-main/solutions/1993. Operations on Tree/1993.cpp

```
struct Node {
    vector<int> children;
    int lockedBy = -1;
};

class LockingTree {
public:
    LockingTree(vector<int>& parent) : parent(parent) {
        nodes.resize(parent.size());
        for (int i = 1; i < parent.size(); ++i)
            nodes[parent[i]].children.push_back(i);
    }

    bool lock(int num, int user) {
        if (nodes[num].lockedBy != -1)
            return false;
        return nodes[num].lockedBy = user;
    }

    bool unlock(int num, int user) {
        if (nodes[num].lockedBy != user)
            return false;
        return nodes[num].lockedBy = -1;
    }

    bool upgrade(int num, int user) {
        if (nodes[num].lockedBy != -1)
            return false;
        if (!anyLockedDescendant(num))
            return false;

        // Walk up the hierarchy to ensure that there are no locked ancestors.
        for (int i = num; i != -1; i = parent[i])
            if (nodes[i].lockedBy != -1)
                return false;

        unlockDescendants(num);
        return nodes[num].lockedBy = user;
    }

private:
    const vector<int> parent;
    vector<Node> nodes;

    bool anyLockedDescendant(int i) {
        return nodes[i].lockedBy != -1 ||
            ranges::any_of(nodes[i].children, [this](const int child) {
                return anyLockedDescendant(child);
            });
    }

    void unlockDescendants(int i) {
        nodes[i].lockedBy = -1;
        for (const int child : nodes[i].children)
            unlockDescendants(child);
    }
};
```

1059. 1994.cpp

Path: LeetCode-main/solutions/1994. The Number of Good Subsets/1994.cpp

```
class Solution {
public:
    int numberOfGoodSubsets(vector<int>& nums) {
        const vector<int> primes{2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
        const int n = 1 << primes.size();
        const int maxNum = ranges::max(nums);
        vector<long> dp(n);
        vector<int> count(maxNum + 1);

        dp[0] = 1;

        for (const int num : nums)
            ++count[num];

        for (int num = 2; num <= maxNum; ++num) {
            if (count[num] == 0)
                continue;
            if (num % 4 == 0 || num % 9 == 0 || num % 25 == 0)
                continue;
            const int numPrimesMask = getPrimesMask(num, primes);
            for (int primesMask = 0; primesMask < n; ++primesMask) {
                if ((primesMask & numPrimesMask) > 0)
                    continue;
                const int nextPrimesMask = primesMask | numPrimesMask;
                dp[nextPrimesMask] += dp[primesMask] * count[num];
                dp[nextPrimesMask] %= kMod;
            }
        }

        return modPow(2, count[1]) *
            (accumulate(dp.begin() + 1, dp.end(), 0L) % kMod) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    int getPrimesMask(int num, const vector<int>& primes) {
        int primesMask = 0;
        for (int i = 0; i < primes.size(); ++i)
            if (num % primes[i] == 0)
                primesMask |= 1 << i;
        return primesMask;
    }

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

1060. 1995-2.cpp

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995-2.cpp

```
class Solution {
public:
    int countQuadruplets(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        unordered_map<int, int> count;

        for (int c = n - 1; c > 1; --c) { // `c` also represents `d`.
            for (int b = c - 1; b > 0; --b)
                for (int a = b - 1; a >= 0; --a)
                    if (const auto it = count.find(nums[a] + nums[b] + nums[c]);
                        it != count.cend())
                        ans += it->second;
                    ++count[nums[c]]; // c := d
        }

        return ans;
    }
};
```

1061. 1995-3.cpp

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995-3.cpp

```
class Solution {
public:
    int countQuadruplets(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        unordered_map<int, int> count;

        //    nums[a] + nums[b] + nums[c] == nums[d]
        // => nums[a] + nums[b] == nums[d] - nums[c]
        for (int b = n - 1; b > 0; --b) { // `b` also represents `c`.
            for (int a = b - 1; a >= 0; --a)
                if (const auto it = count.find(nums[a] + nums[b]); it != count.cend())
                    ans += it->second;
            for (int d = n - 1; d > b; --d)
                ++count[nums[d] - nums[b]]; // b := c
        }

        return ans;
    }
};
```


1062. 1995.cpp

Path: LeetCode-main/solutions/1995. Count Special Quadruplets/1995.cpp

```
class Solution {  
public:  
    int countQuadruplets(std::vector<int>& nums) {  
        const int n = nums.size();  
        int ans = 0;  
  
        for (int a = 0; a < n; ++a)  
            for (int b = a + 1; b < n; ++b)  
                for (int c = b + 1; c < n; ++c)  
                    for (int d = c + 1; d < n; ++d)  
                        if (nums[a] + nums[b] + nums[c] == nums[d])  
                            ++ans;  
  
        return ans;  
    }  
};
```

1063. 1996-2.cpp

Path: LeetCode-main/solutions/1996. The Number of Weak Characters in the Game/1996-2.cpp

```
class Solution {
public:
    int numberOfWeakCharacters(vector<vector<int>>& properties) {
        int ans = 0;
        const int maxAttack = (*ranges::max_element(
            properties, ranges::less{},
            [](const vector<int>& property) { return property[0]; }))[0];
        // maxDefenses[i] := the maximum defense for the i-th attack
        vector<int> maxDefenses(maxAttack + 2);

        for (const vector<int>& property : properties) {
            const int attack = property[0];
            const int defense = property[1];
            maxDefenses[attack] = max(maxDefenses[attack], defense);
        }

        for (int i = maxAttack; i >= 1; --i)
            maxDefenses[i] = max(maxDefenses[i], maxDefenses[i + 1]);

        for (const vector<int>& property : properties) {
            const int attack = property[0];
            const int defense = property[1];
            if (maxDefenses[attack + 1] > defense)
                ++ans;
        }

        return ans;
    }
};
```

1064. 1996.cpp

Path: LeetCode-main/solutions/1996. The Number of Weak Characters in the Game/1996.cpp

```
class Solution {
public:
    int numberOfWeakCharacters(vector<vector<int>>& properties) {
        ranges::sort(properties, ranges::less{}, [](const vector<int>& property) {
            const int attack = property[0];
            const int defense = property[1];
            return pair<int, int>{-attack, defense};
        });

        int ans = 0;
        int maxDefense = 0;

        for (const vector<int>& property : properties) {
            const int defense = property[1];
            if (defense < maxDefense)
                ++ans;
            maxDefense = max(maxDefense, defense);
        }

        return ans;
    }
};
```

1065. 1997.cpp

Path: LeetCode-main/solutions/1997. First Day Where You Have Been in All the Rooms/1997.cpp

```
class Solution {
public:
    int firstDayBeenInAllRooms(vector<int>& nextVisit) {
        constexpr int kMod = 1'000'000'007;
        const int n = nextVisit.size();
        // dp[i] := the number of days to visit room i for the first time
        vector<int> dp(n);

        // Whenever we visit i, visit times of room[0..i - 1] are all even.
        // Therefore, the rooms before i can be seen as reset and we can safely
        // reuse dp[0..i - 1] as first-time visit to get second-time visit.
        for (int i = 1; i < n; ++i)
            // The total days to visit room[i] is the sum of
            // * dp[i - 1]: 1st-time visit room[i - 1]
            // * 1: visit room[nextVisit[i - 1]]
            // * dp[i - 1] - dp[nextVisit[i - 1]]: 2-time visit room[i - 1]
            // * 1: visit room[i]
            dp[i] = (2L * dp[i - 1] - dp[nextVisit[i - 1]] + 2 + kMod) % kMod;

        return dp.back();
    }
};
```

1066. 1998.cpp

Path: LeetCode-main/solutions/1998. GCD Sort of an Array/1998.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    bool gcdSort(vector<int>& nums) {
        const int mx = ranges::max(nums);
        const vector<int> minPrimeFactors = sieveEratosthenes(mx + 1);
        UnionFind uf(mx + 1);

        for (const int num : nums)
            for (const int primeFactor : getPrimeFactors(num, minPrimeFactors))
                uf.unionByRank(num, primeFactor);

        vector<int> sortedNums(nums);
        ranges::sort(sortedNums);

        for (int i = 0; i < nums.size(); ++i)
            // Can't swap nums[i] with sortedNums[i].
            if (uf.find(nums[i]) != uf.find(sortedNums[i]))
                return false;

        return true;
    }

private:
    // Gets the minimum prime factor of i, where 1 < i <= n.
    vector<int> sieveEratosthenes(int n) {
        vector<int> minPrimeFactors(n + 1);
        iota(minPrimeFactors.begin() + 2, minPrimeFactors.end(), 2);
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    vector<int> getPrimeFactors(int num, const vector<int>& minPrimeFactors) {
        vector<int> primeFactors;
        while (num > 1) {
            const int divisor = minPrimeFactors[num];
            primeFactors.push_back(divisor);
            while (num % divisor == 0)
                num /= divisor;
        }
        return primeFactors;
    }
};
```

1067. 1999-2.cpp

Path: LeetCode-main/solutions/1999. Smallest Greater Multiple Made of Two Digits/1999-2.cpp

```
class Solution {
public:
    int findInteger(int k, int digit1, int digit2) {
        return findInteger(k, digit1, digit2, 0);
    }

private:
    int findInteger(int k, int d1, int d2, long x) {
        if (x > INT_MAX)
            return -1;
        if (x > k && x % k == 0)
            return x;
        // Skip if d1/d2 and x are zero.
        const int a = (x + d1 == 0) ? -1 : findInteger(k, d1, d2, x * 10 + d1);
        const int b = (x + d2 == 0) ? -1 : findInteger(k, d1, d2, x * 10 + d2);
        if (a == -1)
            return b;
        if (b == -1)
            return a;
        return min(a, b);
    }
};
```

1068. 1999.cpp

Path: LeetCode-main/solutions/1999. Smallest Greater Multiple Made of Two Digits/1999.cpp

```
class Solution {
public:
    int findInteger(int k, int digit1, int digit2) {
        const int minDigit = min(digit1, digit2);
        const int maxDigit = max(digit1, digit2);
        const vector<int> digits = minDigit == maxDigit
                                   ? vector<int>{minDigit}
                                   : vector<int>{minDigit, maxDigit};

        queue<int> q;

        for (const int digit : digits)
            q.push(digit);

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            if (u > k && u % k == 0)
                return u;
            if (u == 0)
                continue;
            for (const int digit : digits) {
                const long nextNum = u * 10L + digit;
                if (nextNum > INT_MAX)
                    continue;
                q.push(nextNum);
            }
        }

        return -1;
    }
};
```

1069. 2.cpp

Path: LeetCode-main/solutions/2. Add Two Numbers/2.cpp

```
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode dummy(0);
        ListNode* curr = &dummy;
        int carry = 0;

        while (l1 != nullptr || l2 != nullptr || carry > 0) {
            if (l1 != nullptr) {
                carry += l1->val;
                l1 = l1->next;
            }
            if (l2 != nullptr) {
                carry += l2->val;
                l2 = l2->next;
            }
            curr->next = new ListNode(carry % 10);
            carry /= 10;
            curr = curr->next;
        }

        return dummy.next;
    }
};
```


1070. 20.cpp

Path: LeetCode-main/solutions/20. Valid Parentheses/20.cpp

```
class Solution {
public:
    bool isValid(string s) {
        stack<char> stack;

        for (const char c : s)
            if (c == '(')
                stack.push(')');
            else if (c == '{')
                stack.push('}');
            else if (c == '[')
                stack.push(']');
            else if (stack.empty() || pop(stack) != c)
                return false;

        return stack.empty();
    }

private:
    int pop(stack<char>& stack) {
        const int c = stack.top();
        stack.pop();
        return c;
    }
};
```

1071. 200-2.cpp

Path: LeetCode-main/solutions/200. Number of Islands/200-2.cpp

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int ans = 0;

        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    ++ans;
                }

        return ans;
    }

private:
    void dfs(vector<vector<char>>& grid, int i, int j) {
        if (i < 0 || i == grid.size() || j < 0 || j == grid[0].size())
            return;
        if (grid[i][j] != '1')
            return;

        grid[i][j] = '2'; // Mark '2' as visited.
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }
};
```

1072. 200.cpp

Path: LeetCode-main/solutions/200. Number of Islands/200.cpp

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = grid.size();
        const int n = grid[0].size();
        int ans = 0;

        auto bfs = [&](int r, int c) {
            queue<pair<int, int>> q{{{r, c}}};
            grid[r][c] = '2'; // Mark '2' as visited.
            while (!q.empty()) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (grid[x][y] != '1')
                        continue;
                    q.emplace(x, y);
                    grid[x][y] = '2'; // Mark '2' as visited.
                }
            }
        };

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == '1') {
                    bfs(i, j);
                    ++ans;
                }

        return ans;
    }
};
```

1073. 2000.cpp

Path: LeetCode-main/solutions/2000. Reverse Prefix of Word/2000.cpp

```
class Solution {  
public:  
    string reversePrefix(string word, char ch) {  
        reverse(word.begin(), word.begin() + word.find(ch) + 1);  
        return word;  
    }  
};
```

1074. 2001-2.cpp

Path: LeetCode-main/solutions/2001. Number of Pairs of Interchangeable Rectangles/2001-2.cpp

```
class Solution {
public:
    long long interchangeableRectangles(vector<vector<int>>& rectangles) {
        long ans = 0;
        unordered_map<pair<int, int>, int, PairHash> ratioCount;

        for (const vector<int>& rectangle : rectangles) {
            const int width = rectangle[0];
            const int height = rectangle[1];
            const int d = __gcd(width, height);
            ++ratioCount[{width / d, height / d}];
        }

        for (const auto& [_, count] : ratioCount)
            ans += static_cast<long>(count) * (count - 1) / 2;

        return ans;
    }

private:
    struct PairHash {
        size_t operator()(const pair<int, int>& p) const {
            return p.first ^ p.second;
        }
    };
};
```

1075. 2001.cpp

Path: LeetCode-main/solutions/2001. Number of Pairs of Interchangeable Rectangles/2001.cpp

```
class Solution {
public:
    long long interchangeableRectangles(vector<vector<int>>& rectangles) {
        long ans = 0;
        unordered_map<double, int> ratioCount;

        for (const vector<int>& rectangle : rectangles) {
            const int width = rectangle[0];
            const int height = rectangle[1];
            ++ratioCount[1.0 * width / height];
        }

        for (const auto& [_, count] : ratioCount)
            ans += static_cast<long>(count) * (count - 1) / 2;

        return ans;
    }
};
```

1076. 2002.cpp

Path: LeetCode-main/solutions/2002. Maximum Product of the Length of Two Palindromic Subsequences/2002.cpp

```
class Solution {
public:
    int maxProduct(string s) {
        size_t ans = 0;
        dfs(s, 0, "", "", ans);
        return ans;
    }

private:
    void dfs(const string& s, int i, string&& s1, string&& s2, size_t& ans) {
        if (i == s.length()) {
            if (isPalindrome(s1) && isPalindrome(s2))
                ans = max(ans, s1.length() * s2.length());
            return;
        }

        s1.push_back(s[i]);
        dfs(s, i + 1, std::move(s1), std::move(s2), ans);
        s1.pop_back();

        s2.push_back(s[i]);
        dfs(s, i + 1, std::move(s1), std::move(s2), ans);
        s2.pop_back();

        dfs(s, i + 1, std::move(s1), std::move(s2), ans);
    }

    bool isPalindrome(const string& s) {
        int i = 0;
        int j = s.length() - 1;
        while (i < j) {
            if (s[i] != s[j])
                return false;
            ++i;
            --j;
        }
        return true;
    }
};
```

1077. 2003.cpp

Path: LeetCode-main/solutions/2003. Smallest Missing Genetic Value in Each Subtree/2003.cpp

```
class Solution {
public:
    vector<int> smallestMissingValueSubtree(vector<int>& parents,
                                          vector<int>& nums) {
        const int n = parents.size();
        vector<int> ans(n, 1);
        vector<vector<int>> tree(n);
        unordered_set<int> seen;
        int minMiss = 1;

        for (int i = 1; i < n; ++i)
            tree[parents[i]].push_back(i);

        int nodeThatsOne = getNode(nums);
        if (nodeThatsOne == -1)
            return ans;

        int u = nodeThatsOne;
        int prev = -1; // the u that just handled

        // Upward from `nodeThatsOne` to the root `u`.
        while (u != -1) {
            for (const int v : tree[u])
                if (v != prev)
                    dfs(v, tree, seen, nums);
            seen.insert(nums[u]);
            while (seen.contains(minMiss))
                ++minMiss;
            ans[u] = minMiss;
            prev = u;
            u = parents[u];
        }

        return ans;
    }

private:
    int getNode(const vector<int>& nums) {
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == 1)
                return i;
        return -1;
    }

    void dfs(int u, const vector<vector<int>>& tree, unordered_set<int>& seen,
            const vector<int>& nums) {
        seen.insert(nums[u]);
        for (const int v : tree[u])
            dfs(v, tree, seen, nums);
    }
};
```


1078. 2005.cpp

Path: LeetCode-main/solutions/2005. Subtree Removal Game with Fibonacci Tree/2005.cpp

```
class Solution {  
public:  
    bool findGameWinner(int n) {  
        return n % 6 != 1;  
    }  
};
```

1079. 2006.cpp

Path: LeetCode-main/solutions/2006. Count Number of Pairs With Absolute Difference K/2006.cpp

```
class Solution {
public:
    int countKDifference(vector<int>& nums, int k) {
        constexpr int kMax = 100;
        int ans = 0;
        vector<int> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        for (int i = k + 1; i <= kMax; ++i)
            ans += count[i] * count[i - k];

        return ans;
    }
};
```

1080. 2007.cpp

Path: LeetCode-main/solutions/2007. Find Original Array From Doubled Array/2007.cpp

```
class Solution {
public:
    vector<int> findOriginalArray(vector<int>& changed) {
        vector<int> ans;
        queue<int> q;

        ranges::sort(changed);

        for (const int num : changed)
            if (!q.empty() && num == q.front()) {
                q.pop();
            } else {
                q.push(num * 2);
                ans.push_back(num);
            }

        return q.empty() ? ans : vector<int>();
    }
};
```

1081. 2008-2.cpp

Path: LeetCode-main/solutions/2008. Maximum Earnings From Taxi/2008-2.cpp

```
class Solution {
public:
    long long maxTaxiEarnings(int n, vector<vector<int>>& rides) {
        vector<vector<pair<int, int>>> endToStartAndEarns(n + 1);
        // dp[i] := the maximum dollars you can earn ending in i
        vector<long> dp(n + 1);

        for (const vector<int>& ride : rides) {
            const int start = ride[0];
            const int end = ride[1];
            const int tip = ride[2];
            const int earn = end - start + tip;
            endToStartAndEarns[end].emplace_back(start, earn);
        }

        for (int i = 1; i <= n; ++i) {
            dp[i] = dp[i - 1];
            for (const auto& [start, earn] : endToStartAndEarns[i])
                dp[i] = max(dp[i], dp[start] + earn);
        }

        return dp[n];
    }
};
```

1082. 2008.cpp

Path: LeetCode-main/solutions/2008. Maximum Earnings From Taxi/2008.cpp

```
class Solution {
public:
    long long maxTaxiEarnings(int n, vector<vector<int>>& rides) {
        vector<vector<pair<int, int>>> startToEndAndEarns(n);
        // dp[i] := the maximum dollars you can earn starting at i
        vector<long> dp(n + 1);

        for (const vector<int>& ride : rides) {
            const int start = ride[0];
            const int end = ride[1];
            const int tip = ride[2];
            const int earn = end - start + tip;
            startToEndAndEarns[start].emplace_back(end, earn);
        }

        for (int i = n - 1; i >= 1; --i) {
            dp[i] = dp[i + 1];
            for (const auto& [end, earn] : startToEndAndEarns[i])
                dp[i] = max(dp[i], dp[end] + earn);
        }

        return dp[1];
    }
};
```

1083. 2009.cpp

Path: LeetCode-main/solutions/2009. Minimum Number of Operations to Make Array Continuous/2009.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums) {
        const int n = nums.size();
        int ans = n;

        ranges::sort(nums);
        nums.erase(unique(nums.begin(), nums.end()), nums.end());

        for (int i = 0; i < nums.size(); ++i) {
            const int start = nums[i];
            const int end = start + n - 1;
            const int index = firstGreater(nums, end);
            const int uniqueLength = index - i;
            ans = min(ans, n - uniqueLength);
        }

        return ans;
    }

private:
    int firstGreater(const vector<int>& arr, int target) {
        return ranges::upper_bound(arr, target) - arr.begin();
    }
};
```

1084. 201.cpp

Path: LeetCode-main/solutions/201. Bitwise AND of Numbers Range/201.cpp

```
class Solution {
public:
    int rangeBitwiseAnd(int m, int n) {
        int shiftBits = 0;

        while (m != n) {
            m >>= 1;
            n >>= 1;
            ++shiftBits;
        }

        return m << shiftBits;
    }
};
```

1085. 2011.cpp

Path: LeetCode-main/solutions/2011. Final Value of Variable After Performing Operations/2011.cpp

```
class Solution {
public:
    int finalValueAfterOperations(vector<string>& operations) {
        int ans = 0;

        for (const string& op : operations)
            ans += op[1] == '+' ? 1 : -1;

        return ans;
    }
};
```


1086. 2012.cpp

Path: LeetCode-main/solutions/2012. Sum of Beauty in the Array/2012.cpp

```
class Solution {
public:
    int sumOfBeauties(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        vector<int> minOfRight(n);
        minOfRight.back() = nums.back();

        for (int i = n - 2; i >= 2; --i)
            minOfRight[i] = min(nums[i], minOfRight[i + 1]);

        int maxOfLeft = nums[0];

        for (int i = 1; i <= n - 2; ++i) {
            if (maxOfLeft < nums[i] && nums[i] < minOfRight[i + 1])
                ans += 2;
            else if (nums[i - 1] < nums[i] && nums[i] < nums[i + 1])
                ans += 1;
            maxOfLeft = max(maxOfLeft, nums[i]);
        }

        return ans;
    }
};
```

1087. 2013.cpp

Path: LeetCode-main/solutions/2013. Detect Squares/2013.cpp

```
class DetectSquares {
public:
    void add(vector<int> point) {
        ++pointCount[getHash(point[0], point[1])];
    }

    int count(vector<int> point) {
        const int x1 = point[0];
        const int y1 = point[1];
        int ans = 0;

        for (const auto& [hash, count] : pointCount) {
            const int x3 = hash >> 10;
            const int y3 = hash & 1023;
            if (x1 != x3 && abs(x1 - x3) == abs(y1 - y3)) {
                const int p = getHash(x1, y3);
                const int q = getHash(x3, y1);
                if (pointCount.contains(p) && pointCount.contains(q))
                    ans += count * pointCount[p] * pointCount[q];
            }
        }

        return ans;
    }

private:
    unordered_map<int, int> pointCount;

    int getHash(int i, int j) {
        return i << 10 | j;
    }
};
```

1088. 2014.cpp

Path: LeetCode-main/solutions/2014. Longest Subsequence Repeated k Times/2014.cpp

```
class Solution {
public:
    string longestSubsequenceRepeatedK(string s, int k) {
        string ans;
        vector<int> count(26);
        vector<char> possibleChars;
        // Stores subsequences, where the length grows by 1 each time.
        queue<string> q{{""}};

        for (const char c : s)
            ++count[c - 'a'];

        for (char c = 'a'; c <= 'z'; ++c)
            if (count[c - 'a'] >= k)
                possibleChars.push_back(c);

        while (!q.empty()) {
            const string currSubseq = q.front();
            q.pop();
            if (currSubseq.length() * k > s.length())
                return ans;
            for (const char c : possibleChars) {
                const string& newSubseq = currSubseq + c;
                if (isSubsequence(newSubseq, s, k)) {
                    q.push(newSubseq);
                    ans = newSubseq;
                }
            }
        }

        return ans;
    }

private:
    bool isSubsequence(const string& subseq, string& s, int k) {
        int i = 0; // subseq's index
        for (const char c : s)
            if (c == subseq[i])
                if (++i == subseq.length()) {
                    if (--k == 0)
                        return true;
                    i = 0;
                }
        return false;
    }
};
```

1089. 2015.cpp

Path: LeetCode-main/solutions/2015. Average Height of Buildings in Each Segment/2015.cpp

```
class Solution {
public:
    vector<vector<int>> averageHeightOfBuildings(vector<vector<int>>& buildings) {
        vector<vector<int>> ans;
        vector<pair<int, int>> events;

        for (const vector<int>& b : buildings) {
            const int start = b[0];
            const int end = b[1];
            const int height = b[2];
            events.emplace_back(start, height);
            events.emplace_back(end, -height);
        }

        ranges::sort(events);

        int prev = 0;
        int count = 0;
        int sumHeight = 0;

        for (const auto& [curr, height] : events) {
            if (sumHeight > 0 && curr > prev) {
                const int avgHeight = sumHeight / count;
                if (!ans.empty() && ans.back()[1] == prev && avgHeight == ans.back()[2])
                    ans.back()[1] = curr;
                else
                    ans.push_back({prev, curr, avgHeight});
            }
            sumHeight += height;
            count += height > 0 ? 1 : -1;
            prev = curr;
        }

        return ans;
    }
};
```

1090. 2016.cpp

Path: LeetCode-main/solutions/2016. Maximum Difference Between Increasing Elements/2016.cpp

```
class Solution {
public:
    int maximumDifference(vector<int>& nums) {
        int ans = -1;
        int mn = nums[0];

        for (int i = 1; i < nums.size(); ++i) {
            if (nums[i] > mn)
                ans = max(ans, nums[i] - mn);
            mn = min(mn, nums[i]);
        }

        return ans;
    }
};
```

1091. 2017.cpp

Path: LeetCode-main/solutions/2017. Grid Game/2017.cpp

```
class Solution {
public:
    long long gridGame(vector<vector<int>>& grid) {
        const int n = grid[0].size();
        long ans = LONG_MAX;
        long sumRow0 = accumulate(grid[0].begin(), grid[0].end(), 0L);
        long sumRow1 = 0;

        for (int i = 0; i < n; ++i) {
            sumRow0 -= grid[0][i];
            ans = min(ans, max(sumRow0, sumRow1));
            sumRow1 += grid[1][i];
        }

        return ans;
    }
};
```

1092. 2018.cpp

Path: LeetCode-main/solutions/2018. Check if Word Can Be Placed In Crossword/2018.cpp

```
class Solution {
public:
    bool placeWordInCrossword(vector<vector<char>>& board, string word) {
        for (const vector<vector<char>>& state : {board, getRotated(board)}) {
            for (const vector<char>& chars : state)
                for (const string& token : getTokens(join(chars)))
                    for (const string& letters :
                        {word, string{word.rbegin(), word.rend()}})
                        if (letters.length() == token.length())
                            if (canFit(letters, token))
                                return true;
        }
        return false;
    }

private:
    vector<vector<char>> getRotated(const vector<vector<char>>& board) {
        const int m = board.size();
        const int n = board[0].size();
        vector<vector<char>> rotated(n, vector<char>(m));
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                rotated[j][i] = board[i][j];
        return rotated;
    }

    vector<string> getTokens(const string& row) {
        vector<string> tokens;
        int start = 0;
        int end;
        string token;
        do {
            end = row.find('#', start);
            token = row.substr(start, end - start);
            if (!token.empty())
                tokens.push_back(token);
            start = end + 1;
        } while (end != string::npos);
        return tokens;
    }

    string join(const vector<char>& chars) {
        string joined;
        for (const char c : chars)
            joined += c;
        return joined;
    }

    bool canFit(const string& letters, const string& token) {
        for (int i = 0; i < letters.length(); ++i)
            if (token[i] != ' ' && token[i] != letters[i])
                return false;
        return true;
    }
};
```

1093. 2019.cpp

Path: LeetCode-main/solutions/2019. The Score of Students Solving Math Expression/2019.cpp

```
class Solution {
public:
    int scoreOfStudents(string s, vector<int>& answers) {
        const int n = s.length() / 2 + 1;
        const unordered_map<char, function<int(int, int)>> func{
            {'+', plus<int>()}, {'*', multiplies<int>()}};
        int ans = 0;
        vector<vector<unordered_set<int>>> dp(n, vector<unordered_set<int>>(n));
        unordered_map<int, int> count;

        for (int i = 0; i < n; ++i)
            dp[i][i].insert(s[i * 2] - '0');

        for (int d = 1; d < n; ++d)
            for (int i = 0; i + d < n; ++i) {
                const int j = i + d;
                for (int k = i; k < j; ++k) {
                    const char op = s[k * 2 + 1];
                    for (const int a : dp[i][k])
                        for (const int b : dp[k + 1][j]) {
                            const int res = func.at(op)(a, b);
                            if (res <= 1000)
                                dp[i][j].insert(res);
                        }
                }
            }

        const int correctAnswer = eval(s);

        for (const int answer : answers)
            ++count[answer];

        for (const auto& [answer, freq] : count)
            if (answer == correctAnswer)
                ans += 5 * freq;
            else if (dp[0][n - 1].contains(answer))
                ans += 2 * freq;

        return ans;
    }

private:
    int eval(const string& s) {
        int ans = 0;
        int prevNum = 0;
        int currNum = 0;
        char op = '+';

        for (int i = 0; i < s.length(); ++i) {
            const char c = s[i];
            if (isdigit(c))
                currNum = currNum * 10 + (c - '0');
            if (!isdigit(c) || i == s.length() - 1) {
                if (op == '+') {
                    ans += prevNum;
                    prevNum = currNum;
                } else if (op == '*') {
                    prevNum = prevNum * currNum;
                }
                op = c;
                currNum = 0;
            }
        }

        return ans + prevNum;
    }
};
```


1094. 202.cpp

Path: LeetCode-main/solutions/202. Happy Number/202.cpp

```
class Solution {
public:
    bool isHappy(int n) {
        int slow = squaredSum(n);
        int fast = squaredSum(squaredSum(n));

        while (slow != fast) {
            slow = squaredSum(slow);
            fast = squaredSum(squaredSum(fast));
        }

        return slow == 1;
    }

private:
    int squaredSum(int n) {
        int sum = 0;
        while (n > 0) {
            sum += pow(n % 10, 2);
            n /= 10;
        }
        return sum;
    };
};
```

1095. 2021.cpp

Path: LeetCode-main/solutions/2021. Brightest Position on Street/2021.cpp

```
class Solution {
public:
    int brightestPosition(vector<vector<int>>& lights) {
        int ans = INT_MAX;
        int maxBrightness = -1;
        int currBrightness = 0;
        map<int, int> line;

        for (const vector<int>& light : lights) {
            const int position = light[0];
            const int range = light[1];
            ++line[position - range];
            --line[position + range + 1];
        }

        for (const auto& [pos, brightness] : line) {
            currBrightness += brightness;
            if (currBrightness > maxBrightness) {
                maxBrightness = currBrightness;
                ans = pos;
            }
        }

        return ans;
    }
};
```

1096. 2022.cpp

Path: LeetCode-main/solutions/2022. Convert 1D Array Into 2D Array/2022.cpp

```
class Solution {
public:
    vector<vector<int>> construct2DArray(vector<int>& original, int m, int n) {
        if (original.size() != m * n)
            return {};

        vector<vector<int>> ans(m, vector<int>(n));

        for (int i = 0; i < original.size(); ++i)
            ans[i / n][i % n] = original[i];

        return ans;
    }
};
```

1097. 2023.cpp

Path: LeetCode-main/solutions/2023. Number of Pairs of Strings With Concatenation Equal to Target/2023.cpp

```
class Solution {
public:
    int numOfPairs(vector<string>& nums, string target) {
        const int n = target.length();
        int ans = 0;
        unordered_map<string, int> count;

        for (const string& num : nums) {
            const int k = num.length();
            if (k >= n)
                continue;
            if (target.substr(0, k) == num)
                ans += count[target.substr(k)];
            if (target.substr(n - k) == num)
                ans += count[target.substr(0, n - k)];
            ++count[num];
        }

        return ans;
    }
};
```

1098. 2024.cpp

Path: LeetCode-main/solutions/2024. Maximize the Confusion of an Exam/2024.cpp

```
class Solution {
public:
    int maxConsecutiveAnswers(string answerKey, int k) {
        int ans = 0;
        int maxCount = 0;
        vector<int> count(2);

        for (int l = 0, r = 0; r < answerKey.length(); ++r) {
            maxCount = max(maxCount, ++count[answerKey[r] == 'T']);
            while (maxCount + k < r - l + 1)
                --count[answerKey[l++] == 'T'];
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

1099. 2025.cpp

Path: LeetCode-main/solutions/2025. Maximum Number of Ways to Partition an Array/2025.cpp

```
class Solution {
public:
    int waysToPartition(vector<int>& nums, int k) {
        const int n = nums.size();
        const long sum = accumulate(nums.begin(), nums.end(), 0L);
        long prefix = 0;
        // count of sum(A[0..k)) - sum(A[k..n)) for k in [0..i)
        unordered_map<long, int> l;
        // count of sum(A[0..k)) - sum(A[k..n)) for k in [i..n)
        unordered_map<long, int> r;

        for (int pivot = 1; pivot < n; ++pivot) {
            prefix += nums[pivot - 1];
            const long suffix = sum - prefix;
            ++r[prefix - suffix];
        }

        int ans = r[0];
        prefix = 0;

        for (const int num : nums) {
            ans = max(ans, l[k - num] + r[num - k]);
            prefix += num;
            const long suffix = sum - prefix;
            const long diff = prefix - suffix;
            --r[diff];
            ++l[diff];
        }

        return ans;
    }
};
```

1100. 2027.cpp

Path: LeetCode-main/solutions/2027. Minimum Moves to Convert String/2027.cpp

```
class Solution {
public:
    int minimumMoves(string s) {
        int ans = 0;

        for (int i = 0; i < s.length(); )
            if (s[i] == '0') {
                ++i;
            } else {
                ++ans;
                i += 3;
            }

        return ans;
    }
};
```

1101. 2028.cpp

Path: LeetCode-main/solutions/2028. Find Missing Observations/2028.cpp

```
class Solution {
public:
    vector<int> missingRolls(vector<int>& rolls, int mean, int n) {
        const int targetSum = (rolls.size() + n) * mean;
        int missingSum = targetSum - accumulate(rolls.begin(), rolls.end(), 0);
        if (missingSum > n * 6 || missingSum < n)
            return {};

        vector<int> ans(n, missingSum / n);
        missingSum %= n;
        for (int i = 0; i < missingSum; ++i)
            ++ans[i];

        return ans;
    }
};
```


1102. 2029.cpp

Path: LeetCode-main/solutions/2029. Stone Game IX/2029.cpp

```
class Solution {
public:
    bool stoneGameIX(vector<int>& stones) {
        vector<int> count(3);

        for (const int stone : stones)
            ++count[stone % 3];

        if (count[0] % 2 == 0)
            return min(count[1], count[2]) > 0;
        return abs(count[1] - count[2]) > 2;
    }
};
```

1103. 203.cpp

Path: LeetCode-main/solutions/203. Remove Linked List Elements/203.cpp

```
class Solution {
public:
    ListNode* removeElements(ListNode* head, int val) {
        ListNode dummy(0, head);
        ListNode* prev = &dummy;

        for (; head; head = head->next)
            if (head->val != val) {
                prev->next = head;
                prev = prev->next;
            }
        prev->next = nullptr; // In case that the last value equals `val`.

        return dummy.next;
    }
};
```

1104. 2030.cpp

Path: LeetCode-main/solutions/2030. Smallest K-Length Subsequence With Occurrences of a Letter/2030.cpp

```
class Solution {
public:
    string smallestSubsequence(string s, int k, char letter, int repetition) {
        string ans;
        vector<char> stack;
        int required = repetition;
        int nLetters = ranges::count(s, letter);

        for (int i = 0; i < s.length(); ++i) {
            const char c = s[i];
            while (!stack.empty() && stack.back() > c &&
                stack.size() + s.length() - i - 1 >= k &&
                (stack.back() != letter || nLetters > required)) {
                const char popped = stack.back();
                stack.pop_back();
                if (popped == letter)
                    ++required;
            }
            if (stack.size() < k)
                if (c == letter) {
                    stack.push_back(c);
                    --required;
                } else if (k > stack.size() + required) {
                    stack.push_back(c);
                }
            if (c == letter)
                --nLetters;
        }

        for (const char c : stack)
            ans += c;

        return ans;
    }
};
```

1105. 2031.cpp

Path: LeetCode-main/solutions/2031. Count Subarrays With More Ones Than Zeros/2031.cpp

```
class FenwickTree {
public:
    FenwickTree(int n) : sums(n + 1) {}

    void add(int i, int delta) {
        while (i < sums.size()) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    int get(int i) const {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

private:
    vector<int> sums;

    static inline int lowbit(int i) {
        return i & -i;
    }
};

class Solution {
public:
    int subarraysWithMoreZerosThanOnes(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        int ans = 0;
        int prefix = 0;
        // Map [-n, n] to [1, 2 * n + 1].
        FenwickTree tree(2 * n + 1);
        tree.add(remap(0, n), 1);

        for (const int num : nums) {
            prefix += num == 0 ? -1 : 1;
            // If prefix[j] > prefix[i], where 0 <= i < j < |prefix|, that means that
            // there are more ones than zeros in nums[i + 1, j].
            ans += tree.get(remap(prefix - 1, n));
            ans %= kMod;
            tree.add(remap(prefix, n), 1);
        }

        return ans;
    }

private:
    int remap(int i, int n) {
        return i + n + 1;
    }
};
```

1106. 2032.cpp

Path: LeetCode-main/solutions/2032. Two Out of Three/2032.cpp

```
class Solution {
public:
    vector<int> twoOutOfThree(vector<int>& nums1, vector<int>& nums2,
                             vector<int>& nums3) {
        vector<int> ans;
        vector<int> count(101);

        for (const vector<int>& nums : {nums1, nums2, nums3})
            update(count, nums);

        for (int i = 1; i <= 100; ++i)
            if (count[i] >= 2)
                ans.push_back(i);

        return ans;
    }

private:
    void update(vector<int>& count, const vector<int>& nums) {
        for (const int num : unordered_set<int>(nums.begin(), nums.end()))
            ++count[num];
    }
};
```

1107. 2033.cpp

Path: LeetCode-main/solutions/2033. Minimum Operations to Make a Uni-Value Grid/2033.cpp

```
class Solution {
public:
    int minOperations(vector<vector<int>>& grid, int x) {
        vector<int> arr;
        for (const vector<int>& row : grid)
            arr.insert(arr.end(), row.begin(), row.end());
        if (ranges::any_of(arr, [&](int a) { return (a - arr[0]) % x; }))
            return -1;

        int ans = 0;

        nth_element(arr.begin(), arr.begin() + arr.size() / 2, arr.end());

        for (const int a : arr)
            ans += abs(a - arr[arr.size() / 2]) / x;

        return ans;
    }
};
```

1108. 2034-2.cpp

Path: LeetCode-main/solutions/2034. Stock Price Fluctuation/2034-2.cpp

```
class StockPrice {
public:
    void update(int timestamp, int price) {
        if (const auto it = timestampToPrice.find(timestamp);
            it != timestampToPrice.cend()) {
            const int prevPrice = it->second;
            prices.erase(prices.equal_range(prevPrice).first);
        }
        timestampToPrice[timestamp] = price;
        prices.insert(price);
    }

    int current() {
        return timestampToPrice.rbegin()->second;
    }

    int maximum() {
        return *prices.rbegin();
    }

    int minimum() {
        return *prices.begin();
    }

private:
    map<int, int> timestampToPrice;
    multiset<int> prices;
};
```

1109. 2034.cpp

Path: LeetCode-main/solutions/2034. Stock Price Fluctuation/2034.cpp

```
class StockPrice {
public:
    void update(int timestamp, int price) {
        if (timestampToPrice.contains(timestamp)) {
            const int prevPrice = timestampToPrice[timestamp];
            if (--pricesCount[prevPrice] == 0)
                pricesCount.erase(prevPrice);
        }
        timestampToPrice[timestamp] = price;
        ++pricesCount[price];
    }

    int current() {
        return timestampToPrice.rbegin()->second;
    }

    int maximum() {
        return pricesCount.rbegin()->first;
    }

    int minimum() {
        return pricesCount.begin()->first;
    }

private:
    map<int, int> timestampToPrice;
    map<int, int> pricesCount;
};
```


1110. 2035.cpp

Path: LeetCode-main/solutions/2035. Partition Array Into Two Arrays to Minimize Sum Difference/2035.cpp

```
class Solution {
public:
    int minimumDifference(vector<int>& nums) {
        const int n = nums.size() / 2;
        const int sum = accumulate(nums.begin(), nums.end(), 0);
        const int goal = sum / 2;
        const vector<int> lNums(nums.begin(), nums.begin() + n);
        const vector<int> rNums(nums.begin() + n, nums.end());
        int ans = INT_MAX;
        vector<vector<int>> lSums(n + 1);
        vector<vector<int>> rSums(n + 1);

        dfs(lNums, 0, 0, 0, lSums);
        dfs(rNums, 0, 0, 0, rSums);

        for (int lCount = 0; lCount <= n; ++lCount) {
            auto& l = lSums[lCount];
            auto& r = rSums[n - lCount];
            ranges::sort(r);
            for (const int lSum : l) {
                const int i = firstGreaterEqual(r, goal - lSum);
                if (i < r.size()) {
                    const int sumPartOne = sum - lSum - r[i];
                    const int sumPartTwo = sum - sumPartOne;
                    ans = min(ans, abs(sumPartOne - sumPartTwo));
                }
                if (i > 0) {
                    const int sumPartOne = sum - lSum - r[i - 1];
                    const int sumPartTwo = sum - sumPartOne;
                    ans = min(ans, abs(sumPartOne - sumPartTwo));
                }
            }
        }

        return ans;
    }

private:
    void dfs(const vector<int>& arr, int i, int count, int path,
             vector<vector<int>>& sums) {
        if (i == arr.size()) {
            sums[count].push_back(path);
            return;
        }
        dfs(arr, i + 1, count + 1, path + arr[i], sums);
        dfs(arr, i + 1, count, path, sums);
    }

    int firstGreaterEqual(const vector<int>& arr, int target) {
        return ranges::lower_bound(arr, target) - arr.begin();
    }
};
```

1111. 2036.cpp

Path: LeetCode-main/solutions/2036. Maximum Alternating Subarray Sum/2036.cpp

```
class Solution {
public:
    long long maximumAlternatingSubarraySum(vector<int>& nums) {
        long ans = INT_MIN;
        long even = 0; // the subarray sum starting from an even index
        long odd = 0; // the subarray sum starting from an odd index

        for (int i = 0; i < nums.size(); ++i) {
            if (i % 2 == 0) // Must pick.
                even += nums[i];
            else // Start a fresh subarray or subtract `nums[i]`.
                even = max(0L, even - nums[i]);
            ans = max(ans, even);
        }

        for (int i = 1; i < nums.size(); ++i) {
            if (i % 2 == 1) // Must pick.
                odd += nums[i];
            else // Start a fresh subarray or subtract `nums[i]`.
                odd = max(0L, odd - nums[i]);
            ans = max(ans, odd);
        }

        return ans;
    }
};
```

1112. 2037.cpp

Path: LeetCode-main/solutions/2037. Minimum Number of Moves to Seat Everyone/2037.cpp

```
class Solution {
public:
    int minMovesToSeat(vector<int>& seats, vector<int>& students) {
        int ans = 0;

        ranges::sort(seats);
        ranges::sort(students);

        for (int i = 0; i < seats.size(); ++i)
            ans += abs(seats[i] - students[i]);

        return ans;
    }
};
```

1113. 2038.cpp

Path: LeetCode-main/solutions/2038. Remove Colored Pieces if Both Neighbors are the Same Color/2038.cpp

```
class Solution {
public:
    bool winnerOfGame(string colors) {
        int countAAA = 0;
        int countBBB = 0;

        for (int i = 1; i + 1 < colors.length(); ++i)
            if (colors[i - 1] == colors[i] && colors[i] == colors[i + 1])
                if (colors[i] == 'A')
                    ++countAAA;
                else
                    ++countBBB;

        return countAAA > countBBB;
    }
};
```

1114. 2039.cpp

Path: LeetCode-main/solutions/2039. The Time When the Network Becomes Idle/2039.cpp

```
class Solution {
public:
    int networkBecomesIdle(vector<vector<int>>& edges, vector<int>& patience) {
        const int n = patience.size();
        int ans = 0;
        vector<vector<int>> graph(n);
        queue<int> q{{0}};
        vector<int> dist(n, INT_MAX); // dist[i] := the distance between i and 0
        dist[0] = 0;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        while (!q.empty())
            for (int sz = q.size(); sz > 0; --sz) {
                const int u = q.front();
                q.pop();
                for (const int v : graph[u])
                    if (dist[v] == INT_MAX) {
                        dist[v] = dist[u] + 1;
                        q.push(v);
                    }
            }

        for (int i = 1; i < n; ++i) {
            const int numResending = (dist[i] * 2 - 1) / patience[i];
            const int lastResendingTime = patience[i] * numResending;
            const int lastArrivingTime = lastResendingTime + dist[i] * 2;
            ans = max(ans, lastArrivingTime);
        }

        return ans + 1;
    }
};
```

1115. 204.cpp

Path: LeetCode-main/solutions/204. Count Primes/204.cpp

```
class Solution {
public:
    int countPrimes(int n) {
        if (n <= 2)
            return 0;
        const vector<bool> isPrime = sieveEratosthenes(n);
        return ranges::count(isPrime, true);
    }

private:
    vector<bool> sieveEratosthenes(int n) {
        vector<bool> isPrime(n, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
};
```

1116. 2040.cpp

Path: LeetCode-main/solutions/2040. Kth Smallest Product of Two Sorted Arrays/2040.cpp

```
class Solution {
public:
    long long kthSmallestProduct(vector<int>& nums1, vector<int>& nums2,
                                long long k) {
        vector<int> A1;
        vector<int> A2;
        vector<int> B1;
        vector<int> B2;

        seperate(nums1, A1, A2);
        seperate(nums2, B1, B2);

        const long negCount = A1.size() * B2.size() + A2.size() * B1.size();
        int sign = 1;

        if (k > negCount) {
            k -= negCount; // Find the (k - negCount)-th positive.
        } else {
            k = negCount - k + 1; // Find the (negCount - k + 1)-th abs(negative).
            sign = -1;
            swap(B1, B2);
        }

        long l = 0;
        long r = 1e10;

        while (l < r) {
            const long m = (l + r) / 2;
            if (numProductNoGreaterThan(A1, B1, m) +
                numProductNoGreaterThan(A2, B2, m) >=
                    k)
                r = m;
            else
                l = m + 1;
        }

        return sign * l;
    }

private:
    void seperate(const vector<int>& arr, vector<int>& A1, vector<int>& A2) {
        for (const int a : arr)
            if (a < 0)
                A1.push_back(-a);
            else
                A2.push_back(a);
        ranges::reverse(A1); // Reverse to sort ascending
    }

    long numProductNoGreaterThan(const vector<int>& A, const vector<int>& B,
                                long m) {
        long count = 0;
        int j = B.size() - 1;
        // For each a, find the first index j s.t. a * B[j] <= m
        // So numProductNoGreaterThan m for this row will be j + 1
        for (const long a : A) {
            while (j >= 0 && a * B[j] > m)
                --j;
            count += j + 1;
        }
        return count;
    }
};
```

1117. 2042.cpp

Path: LeetCode-main/solutions/2042. Check if Numbers Are Ascending in a Sentence/2042.cpp

```
class Solution {
public:
    bool areNumbersAscending(string s) {
        int prev = 0;
        istringstream iss(s);

        for (string token; iss >> token;)
            if (isdigit(token[0])) {
                const int num = stoi(token);
                if (num <= prev)
                    return false;
                prev = num;
            }
        return true;
    }
};
```


1118. 2043.cpp

Path: LeetCode-main/solutions/2043. Simple Bank System/2043.cpp

```
class Bank {
public:
    Bank(vector<long long>& balance) : balance(std::move(balance)) {}

    bool transfer(int account1, int account2, long long money) {
        if (!isValid(account2))
            return false;
        return withdraw(account1, money) && deposit(account2, money);
    }

    bool deposit(int account, long long money) {
        if (!isValid(account))
            return false;
        balance[account - 1] += money;
        return true;
    }

    bool withdraw(int account, long long money) {
        if (!isValid(account))
            return false;
        if (balance[account - 1] < money)
            return false;
        balance[account - 1] -= money;
        return true;
    }

private:
    vector<long long> balance;

    bool isValid(int account) {
        return 1 <= account && account <= balance.size();
    }
};
```

1119. 2044.cpp

Path: LeetCode-main/solutions/2044. Count Number of Maximum Bitwise-OR Subsets/2044.cpp

```
class Solution {
public:
    int countMaxOrSubsets(vector<int>& nums) {
        const int ors = accumulate(nums.begin(), nums.end(), 0, bit_or<>());
        int ans = 0;
        dfs(nums, 0, 0, ors, ans);
        return ans;
    }

private:
    void dfs(const vector<int>& nums, int i, int path, const int& ors, int& ans) {
        if (i == nums.size()) {
            if (path == ors)
                ++ans;
            return;
        }

        dfs(nums, i + 1, path, ors, ans);
        dfs(nums, i + 1, path | nums[i], ors, ans);
    }
};
```

1120. 2045.cpp

Path: LeetCode-main/solutions/2045. Second Minimum Time to Reach Destination/2045.cpp

```
class Solution {
public:
    int secondMinimum(int n, vector<vector<int>>& edges, int time, int change) {
        vector<vector<int>> graph(n + 1);
        queue<pair<int, int>> q{{{1, 0}}};
        // minTime[u][0] := the first minimum time to reach the node u
        // minTime[u][1] := the second minimum time to reach the node u
        vector<vector<int>> minTime(n + 1, vector<int>(2, INT_MAX));
        minTime[1][0] = 0;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        while (!q.empty()) {
            const auto [u, prevTime] = q.front();
            q.pop();
            // Start from green.
            // If `numChangeSignal` is odd, now red.
            // If `numChangeSignal` is even, now green.
            const int numChangeSignal = prevTime / change;
            const int waitTime =
                numChangeSignal % 2 == 0 ? 0 : change - prevTime % change;
            const int newTime = prevTime + waitTime + time;
            for (const int v : graph[u])
                if (newTime < minTime[v][0]) {
                    minTime[v][0] = newTime;
                    q.emplace(v, newTime);
                } else if (minTime[v][0] < newTime && newTime < minTime[v][1]) {
                    if (v == n)
                        return newTime;
                    minTime[v][1] = newTime;
                    q.emplace(v, newTime);
                }
        }

        throw;
    }
};
```

1121. 2046.cpp

Path: LeetCode-main/solutions/2046. Sort Linked List Already Sorted Using Absolute Values/2046.cpp

```
class Solution {
public:
    ListNode* sortLinkedList(ListNode* head) {
        ListNode* prev = head;
        ListNode* curr = head->next;

        while (curr != nullptr)
            if (curr->val < 0) {
                prev->next = curr->next;
                curr->next = head;
                head = curr;
                curr = prev->next;
            } else {
                prev = curr;
                curr = curr->next;
            }
        return head;
    }
};
```

1122. 2047-2.cpp

Path: LeetCode-main/solutions/2047. Number of Valid Words in a Sentence/2047-2.cpp

```
class Solution {
public:
    int countValidWords(string sentence) {
        const regex re("[a-z]*([a-z]-[a-z])?[a-z]*[!.,]?");
        int ans = 0;
        istringstream iss(sentence);

        for (string token; iss >> token;)
            if (regex_match(token, re))
                ++ans;

        return ans;
    }
};
```

1123. 2047.cpp

Path: LeetCode-main/solutions/2047. Number of Valid Words in a Sentence/2047.cpp

```
class Solution {
public:
    int countValidWords(string sentence) {
        int ans = 0;
        istringstream iss(sentence);

        for (string token; iss >> token;)
            if (isValid(token))
                ++ans;

        return ans;
    }

private:
    bool isValid(const string& token) {
        int countHyphen = 0;
        for (int i = 0; i < token.length(); ++i) {
            const char c = token[i];
            if (isdigit(c))
                return false;
            if (c == '-') {
                if (i == 0 || !isalpha(token[i - 1]))
                    return false;
                if (i + 1 == token.length() || !isalpha(token[i + 1]))
                    return false;
                if (++countHyphen > 1)
                    return false;
            } else if (c == '!' || c == '.' || c == ',') {
                if (i != token.length() - 1)
                    return false;
            }
        }
        return true;
    }
};
```

1124. 2048.cpp

Path: LeetCode-main/solutions/2048. Next Greater Numerically Balanced Number/2048.cpp

```
class Solution {
public:
    int nextBeautifulNumber(int n) {
        while (!isBalance(++n))
            ;
        return n;
    }

private:
    bool isBalance(int num) {
        vector<int> count(10);
        while (num > 0) {
            if (num % 10 == 0)
                return false;
            ++count[num % 10];
            num /= 10;
        }
        for (int i = 1; i < 10; ++i)
            if (count[i] && count[i] != i)
                return false;
        return true;
    }
};
```

1125. 2049.cpp

Path: LeetCode-main/solutions/2049. Count Nodes With the Highest Score/2049.cpp

```
class Solution {
public:
    int countHighestScoreNodes(vector<int>& parents) {
        int ans = 0;
        vector<vector<int>> tree(parents.size());

        for (int i = 0; i < parents.size(); ++i) {
            if (parents[i] == -1)
                continue;
            tree[parents[i]].push_back(i);
        }

        dfs(tree, 0, 0, ans);
        return ans;
    }

private:
    int dfs(const vector<vector<int>>& tree, int u, long&& maxScore, int& ans) {
        int count = 1;
        long score = 1;
        for (const int v : tree[u]) {
            const int childCount = dfs(tree, v, std::move(maxScore), ans);
            count += childCount;
            score *= childCount;
        }
        const int aboveCount = tree.size() - count;
        score *= max(aboveCount, 1);
        if (score > maxScore) {
            maxScore = score;
            ans = 1;
        } else if (score == maxScore) {
            ++ans;
        }
        return count;
    }
};
```


1126. 205.cpp

Path: LeetCode-main/solutions/205. Isomorphic Strings/205.cpp

```
class Solution {
public:
    bool isIsomorphic(string s, string t) {
        vector<int> charToIndex_s(128);
        vector<int> charToIndex_t(128);

        for (int i = 0; i < s.length(); ++i) {
            if (charToIndex_s[s[i]] != charToIndex_t[t[i]])
                return false;
            charToIndex_s[s[i]] = i + 1;
            charToIndex_t[t[i]] = i + 1;
        }

        return true;
    }
};
```

1127. 2050.cpp

Path: LeetCode-main/solutions/2050. Parallel Courses III/2050.cpp

```
class Solution {
public:
    int minimumTime(int n, vector<vector<int>>& relations, vector<int>& time) {
        vector<vector<int>> graph(n);
        vector<int> inDegrees(n);
        queue<int> q;
        vector<int> dist(time);

        // Build the graph.
        for (const vector<int>& r : relations) {
            const int u = r[0] - 1;
            const int v = r[1] - 1;
            graph[u].push_back(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            for (const int v : graph[u]) {
                dist[v] = max(dist[v], dist[u] + time[v]);
                if (--inDegrees[v] == 0)
                    q.push(v);
            }
        }

        return ranges::max(dist);
    }
};
```

1128. 2052.cpp

Path: LeetCode-main/solutions/2052. Minimum Cost to Separate Sentence Into Rows/2052.cpp

```
class Solution {
public:
    int minimumCost(string sentence, int k) {
        if (sentence.length() <= k)
            return 0;

        vector<string> words = getWords(sentence);

        // dp[i] := the minimum cost of the first i words
        vector<int> dp(words.size() + 1);

        for (int i = 1; i <= words.size(); ++i) {
            int n = words[i - 1].length(); // the length of the current row
            dp[i] = dp[i - 1] + (k - n) * (k - n);
            // Gradually add words[j - 1], words[j - 2], ....
            for (int j = i - 1; j > 0; --j) {
                n += words[j - 1].length() + 1;
                if (n > k)
                    break;
                dp[i] = min(dp[i], dp[j - 1] + (k - n) * (k - n));
            }
        }

        int lastRowLen = words.back().length();
        int i = words.size() - 2;

        while (i > 0 && lastRowLen + words[i].length() + 1 <= k)
            lastRowLen += words[i--].length() + 1;

        return *min_element(dp.begin() + i + 1, dp.end());
    }

private:
    vector<string> getWords(const string& sentence) {
        vector<string> words;
        istringstream iss(sentence);
        for (string token; iss >> token;)
            words.push_back(token);
        return words;
    }
};
```

1129. 2053.cpp

Path: LeetCode-main/solutions/2053. Kth Distinct String in an Array/2053.cpp

```
class Solution {
public:
    string kthDistinct(vector<string>& arr, int k) {
        unordered_map<string, int> count;

        for (const string& a : arr)
            ++count[a];

        for (const string& a : arr)
            if (count[a] == 1 && --k == 0)
                return a;

        return "";
    }
};
```

1130. 2054.cpp

Path: LeetCode-main/solutions/2054. Two Best Non-Overlapping Events/2054.cpp

```
struct Event {
    int time;
    int value;
    bool isStart;
};

class Solution {
public:
    int maxTwoEvents(vector<vector<int>>& events) {
        int ans = 0;
        int maxValue = 0;
        vector<Event> evts;

        for (const vector<int>& event : events) {
            const int start = event[0];
            const int end = event[1];
            const int value = event[2];
            evts.emplace_back(start, value, true);
            evts.emplace_back(end + 1, value, false);
        }

        ranges::sort(evts, ranges::less{}, [](const Event& evt) {
            return pair<int, bool>{evt.time, evt.isStart};
        });

        for (const auto& [_, value, isStart] : evts)
            if (isStart)
                ans = max(ans, value + maxValue);
            else
                maxValue = max(maxValue, value);

        return ans;
    }
};
```

1131. 2055-2.cpp

Path: LeetCode-main/solutions/2055. Plates Between Candles/2055-2.cpp

```
class Solution {
public:
    vector<int> platesBetweenCandles(string s, vector<vector<int>>& queries) {
        const int n = s.length();
        vector<int> ans;
        vector<int> closestLeftCandle(n);
        vector<int> closestRightCandle(n);
        vector<int> candleCount(
            n); // candleCount[i] := the number of candles in s[0..i]
        int candle = -1;
        int count = 0;

        for (int i = 0; i < n; ++i) {
            if (s[i] == '|') {
                candle = i;
                ++count;
            }
            closestLeftCandle[i] = candle;
            candleCount[i] = count;
        }

        candle = -1;
        for (int i = n - 1; i >= 0; --i) {
            if (s[i] == '|')
                candle = i;
            closestRightCandle[i] = candle;
        }

        for (const vector<int>& query : queries) {
            const int left = query[0];
            const int right = query[1];
            const int l = closestRightCandle[left];
            const int r = closestLeftCandle[right];
            if (l == -1 || r == -1 || l > r) {
                ans.push_back(0);
            } else {
                const int lengthBetweenCandles = r - l + 1;
                const int numCandles = candleCount[r] - candleCount[l] + 1;
                ans.push_back(lengthBetweenCandles - numCandles);
            }
        }

        return ans;
    }
};
```

1132. 2055.cpp

Path: LeetCode-main/solutions/2055. Plates Between Candles/2055.cpp

```
class Solution {
public:
    vector<int> platesBetweenCandles(string s, vector<vector<int>>& queries) {
        vector<int> ans;
        vector<int> indices; // indices of '|'

        for (int i = 0; i < s.length(); ++i)
            if (s[i] == '|')
                indices.push_back(i);

        for (const vector<int>& query : queries) {
            const int left = query[0];
            const int right = query[1];
            const int l = ranges::lower_bound(indices, left) - indices.begin();
            const int r = ranges::upper_bound(indices, right) - indices.begin() - 1;
            if (l < r) {
                const int lengthBetweenCandles = indices[r] - indices[l] + 1;
                const int numCandles = r - l + 1;
                ans.push_back(lengthBetweenCandles - numCandles);
            } else {
                ans.push_back(0);
            }
        }

        return ans;
    }
};
```

1133. 2056.cpp

Path: LeetCode-main/solutions/2056. Number of Valid Move Combinations On Chessboard/2056.cpp

```
class Solution {
public:
    int countCombinations(vector<string>& pieces,
                        vector<vector<int>>& positions) {
        const int n = pieces.size();
        unordered_set<long long> hashedBoards;
        // Stores all possible move combinations for `pieces`. Each element is a
        // vector of moves, one for each piece in the input order. e.g., if pieces =
        // ["rook", "bishop"], one element might be [[1,0], [1,1]], representing a
        // rook moving right and a bishop moving diagonally up-right.
        vector<vector<pair<int, int>>> pieceMovesList;

        getPieceMovesList(pieces, 0, {}, pieceMovesList);

        for (const vector<pair<int, int>>& pieceMoves : pieceMovesList)
            dfs(positions, n, pieceMoves, (1 << n) - 1, hashedBoards);

        return hashedBoards.size();
    }

private:
    const unordered_map<string, vector<pair<int, int>>> kPieceToMoves{
        {"rook", {{1, 0}, {-1, 0}, {0, 1}, {0, -1}}},
        {"bishop", {{1, 1}, {1, -1}, {-1, 1}, {-1, -1}}},
        {"queen", {{1, 0}, {-1, 0}, {0, 1}, {0, -1}, {1, 1}, {1, -1}, {-1, 1}, {-1, -1}}};

    // Generates all possible combinations of moves.
    void getPieceMovesList(const vector<string>& pieces, int i,
                        vector<pair<int, int>>&& path,
                        vector<vector<pair<int, int>>>& pieceMovesList) {
        if (i == pieces.size()) {
            pieceMovesList.push_back(path);
            return;
        }
        for (const pair<int, int>& move : kPieceToMoves.at(pieces[i])) {
            path.push_back(move);
            getPieceMovesList(pieces, i + 1, std::move(path), pieceMovesList);
            path.pop_back();
        }
    }

    // Performs a depth-first search to explore all possible board states.
    void dfs(const vector<vector<int>>& board, int n,
            const vector<pair<int, int>>& pieceMoves, int activeMask,
            unordered_set<long long>& hashedBoards) {
        if (activeMask == 0)
            return;
        hashedBoards.insert(getHash(board));
        for (int nextActiveMask = 1; nextActiveMask < 1 << n; ++nextActiveMask) {
            if ((activeMask & nextActiveMask) != nextActiveMask)
                continue;

            // Copy the board.
            vector<vector<int>> nextBoard = board;

            // Move the pieces that are active in this turn.
            for (int i = 0; i < n; ++i)
                if (nextActiveMask >> i & 1) {
                    nextBoard[i][0] += pieceMoves[i].first;
                    nextBoard[i][1] += pieceMoves[i].second;
                }

            // No two or more pieces occupy the same square.
            if (getUniqueSize(nextBoard) < n)
                continue;

            // Every piece needs to be in the boundary.
            if (ranges::all_of(nextBoard, [](const vector<int>& pos) {
                return 1 <= pos[0] && pos[0] <= 8 && 1 <= pos[1] && pos[1] <= 8;
            }))
                dfs(nextBoard, n, pieceMoves, nextActiveMask, hashedBoards);
        }
    }

    long long getHash(const vector<vector<int>>& board) {
        long long hash = 0;
        for (const vector<int>& pos : board)
            hash = (hash * 64) + ((pos[0] - 1) << 3) + (pos[1] - 1);
        return hash;
    }
}
```



```
}  
  
int getUniqueSize(const vector<vector<int>>& board) {  
    unordered_set<int> unique;  
    for (const vector<int>& pos : board)  
        unique.insert(pos[0] * 8 + pos[1]);  
    return unique.size();  
}  
};
```

1134. 2057.cpp

Path: LeetCode-main/solutions/2057. Smallest Index With Equal Value/2057.cpp

```
class Solution {
public:
    int smallestEqual(vector<int>& nums) {
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == i % 10)
                return i;
        return -1;
    }
};
```

1135. 2058.cpp

Path: LeetCode-main/solutions/2058. Find the Minimum and Maximum Number of Nodes Between Critical Points/2058.cpp

```
class Solution {
public:
    vector<int> nodesBetweenCriticalPoints(ListNode* head) {
        int minDistance = INT_MAX;
        int firstMaIndex = -1;
        int prevMaIndex = -1;
        int index = 1;
        ListNode* prev = head;          // Point to the index 0.
        ListNode* curr = head->next;    // Point to the index 1.

        while (curr->next != nullptr) {
            if (curr->val > prev->val && curr->val > curr->next->val ||
                curr->val < prev->val && curr->val < curr->next->val) {
                if (firstMaIndex == -1) // Only assign once.
                    firstMaIndex = index;
                if (prevMaIndex != -1)
                    minDistance = min(minDistance, index - prevMaIndex);
                prevMaIndex = index;
            }
            prev = curr;
            curr = curr->next;
            ++index;
        }

        if (minDistance == INT_MAX)
            return {-1, -1};
        return {minDistance, prevMaIndex - firstMaIndex};
    }
};
```

1136. 2059.cpp

Path: LeetCode-main/solutions/2059. Minimum Operations to Convert Number/2059.cpp

```
class Solution {
public:
    int minimumOperations(vector<int>& nums, int start, int goal) {
        queue<int> q{{start}};
        vector<bool> seen(1001);
        seen[start] = true;

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const int x = q.front();
                q.pop();
                for (const int num : nums) {
                    for (const int res : {x + num, x - num, x ^ num}) {
                        if (res == goal)
                            return step;
                        if (res < 0 || res > 1000 || seen[res])
                            continue;
                        seen[res] = true;
                        q.push(res);
                    }
                }
            }
        return -1;
    }
};
```

1137. 206-2.cpp

Path: LeetCode-main/solutions/206. Reverse Linked List/206-2.cpp

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* prev = nullptr;

        while (head != nullptr) {
            ListNode* next = head->next;
            head->next = prev;
            prev = head;
            head = next;
        }

        return prev;
    }
};
```

1138. 206.cpp

Path: LeetCode-main/solutions/206. Reverse Linked List/206.cpp

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if (!head || !head->next)
            return head;

        ListNode* newHead = reverseList(head->next);
        head->next->next = head;
        head->next = nullptr;
        return newHead;
    }
};
```

1139. 2060.cpp

Path: LeetCode-main/solutions/2060. Check if an Original String Exists Given Two Encoded Strings/2060.cpp

```
class Solution {
public:
    bool possiblyEquals(string s1, string s2) {
        vector<vector<unordered_map<int, bool>>> mem(
            s1.length() + 1, vector<unordered_map<int, bool>>(s2.length() + 1));
        return f(s1, s2, 0, 0, 0, mem);
    }

private:
    // Returns true if s1[i..n) matches s2[j..n), accounting for the padding
    // difference. Here, `paddingDiff` represents the signed padding. A positive
    // `paddingDiff` indicates that s1 has an additional number of offset bytes
    // compared to s2.
    bool f(const string& s1, const string& s2, int i, int j, int paddingDiff,
        vector<vector<unordered_map<int, bool>>>& mem) {
        if (const auto it = mem[i][j].find(paddingDiff); it != mem[i][j].cend())
            return it->second;
        if (i == s1.length() && j == s2.length())
            return paddingDiff == 0;
        if (i < s1.length() && isdigit(s1[i])) {
            // Add padding on s1.
            const int nextLetterIndex = getNextLetterIndex(s1, i);
            for (const int num : getNums(s1.substr(i, nextLetterIndex - i)))
                if (f(s1, s2, nextLetterIndex, j, paddingDiff + num, mem))
                    return true;
        } else if (j < s2.length() && isdigit(s2[j])) {
            // Add padding on s2.
            const int nextLetterIndex = getNextLetterIndex(s2, j);
            for (const int num : getNums(s2.substr(j, nextLetterIndex - j)))
                if (f(s1, s2, i, nextLetterIndex, paddingDiff - num, mem))
                    return true;
        } else if (paddingDiff > 0) {
            // `s1` has more padding, so j needs to catch up.
            if (j < s2.length())
                return f(s1, s2, i, j + 1, paddingDiff - 1, mem);
        } else if (paddingDiff < 0) {
            // `s2` has more padding, so i needs to catch up.
            if (i < s1.length())
                return f(s1, s2, i + 1, j, paddingDiff + 1, mem);
        } else { // paddingDiff == 0
            // There's no padding difference, so consume the next letter.
            if (i < s1.length() && j < s2.length() && s1[i] == s2[j])
                return f(s1, s2, i + 1, j + 1, 0, mem);
        }
        return mem[i][j][paddingDiff] = false;
    }

    int getNextLetterIndex(const string& s, int i) {
        int j = i;
        while (i < s.length() && isdigit(s[j]))
            ++j;
        return j;
    }

    vector<int> getNums(const string& s) {
        vector<int> nums{stoi(s)};
        if (s.length() == 2) {
            nums.push_back(stoi(s.substr(0, 1)) + stoi(s.substr(1, 1)));
        } else if (s.length() == 3) {
            nums.push_back(stoi(s.substr(0, 1)) + stoi(s.substr(1, 2)));
            nums.push_back(stoi(s.substr(0, 2)) + stoi(s.substr(2, 1)));
            nums.push_back(stoi(s.substr(0, 1)) + stoi(s.substr(1, 1)) +
                stoi(s.substr(2, 1)));
        }
        return nums;
    }
};
```

1140. 2061.cpp

Path: LeetCode-main/solutions/2061. Number of Spaces Cleaning Robot Cleaned/2061.cpp

```
class Solution {
public:
    int numberOfCleanRooms(vector<vector<int>>& room) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = room.size();
        const int n = room[0].size();
        int ans = 1;
        int i = 0;
        int j = 0;
        int state = 0; // 0 := right, 1 := down, 2 := left, 3 := up
        vector<vector<int>> seen(m, vector<int>(n)); // seen[i][j] := bitmask
        seen[i][j] |= 1 << state;
        room[i][j] = 2; // 2 := cleaned

        while (true) {
            const int x = i + kDirs[state][0];
            const int y = j + kDirs[state][1];
            if (x < 0 || x == m || y < 0 || y == n || room[x][y] == 1) {
                // Turn 90 degrees clockwise.
                state = (state + 1) % 4;
            } else {
                // Walk to (x, y).
                if (room[x][y] == 0) {
                    ++ans;
                    room[x][y] = 2;
                }
                i = x;
                j = y;
            }
            if (seen[i][j] >> state & 1)
                return ans;
            seen[i][j] |= (1 << state);
        }
    }
};
```


1141. 2062.cpp

Path: LeetCode-main/solutions/2062. Count Vowel Substrings of a String/2062.cpp

```
class Solution {
public:
    int countVowelSubstrings(string word) {
        return countVowelSubstringsAtMost(word, 5) -
            countVowelSubstringsAtMost(word, 4);
    }

private:
    int countVowelSubstringsAtMost(const string& s, int goal) {
        int ans = 0;
        int k = goal;
        vector<int> count(26);

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (!isVowel(s[r])) { // Fresh start.
                l = r + 1;
                k = goal;
                count = vector<int>(26);
                continue;
            }
            if (++count[s[r] - 'a'] == 1)
                --k;
            while (k == -1)
                if (--count[s[l++ - 'a'] == 0)
                    ++k;
            ans += r - l + 1; // s[l..r], s[l + 1..r], ..., s[r]
        }

        return ans;
    }

    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiou";
        return kVowels.find(c) != string_view::npos;
    }
};
```

1142. 2063-2.cpp

Path: LeetCode-main/solutions/2063. Vowels of All Substrings/2063-2.cpp

```
class Solution {
public:
    long long countVowels(string word) {
        long ans = 0;

        for (int i = 0; i < word.length(); ++i)
            if (isVowel(word[i]))
                ans += (i + 1) * (word.length() - i);

        return ans;
    }

private:
    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiou";
        return kVowels.find(c) != string_view::npos;
    }
};
```

1143. 2063.cpp

Path: LeetCode-main/solutions/2063. Vowels of All Substrings/2063.cpp

```
class Solution {
public:
    long long countVowels(string word) {
        // dp[i] := the sum of the number of vowels of word[0..i), ...,
        // word[i - 1..i)
        vector<long> dp(word.length() + 1);

        for (int i = 1; i <= word.length(); ++i) {
            dp[i] = dp[i - 1];
            if (isVowel(word[i - 1]))
                dp[i] += i;
        }

        return accumulate(dp.begin(), dp.end(), 0L);
    }

private:
    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiou";
        return kVowels.find(c) != string_view::npos;
    }
};
```

1144. 2064.cpp

Path: LeetCode-main/solutions/2064. Minimized Maximum of Products Distributed to Any Store/2064.cpp

```
class Solution {
public:
    int minimizedMaximum(int n, vector<int>& quantities) {
        int l = 1;
        int r = ranges::max(quantities);

        while (l < r) {
            const int m = (l + r) / 2;
            if (numStores(quantities, m) <= n)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    int numStores(const vector<int>& quantities, int m) {
        // ceil(q / m)
        return accumulate(quantities.begin(), quantities.end(), 0,
            [&](int acc, int q) { return acc + (q - 1) / m + 1; });
    }
};
```

1145. 2065.cpp

Path: LeetCode-main/solutions/2065. Maximum Path Quality of a Graph/2065.cpp

```
class Solution {
public:
    int maximalPathQuality(vector<int>& values, vector<vector<int>>& edges,
                          int maxTime) {
        const int n = values.size();
        int ans = 0;
        vector<vector<pair<int, int>>> graph(n);
        vector<int> seen(n);
        seen[0] = 1;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int time = edge[2];
            graph[u].emplace_back(v, time);
            graph[v].emplace_back(u, time);
        }

        dfs(graph, values, 0, values[0], maxTime, seen, ans);
        return ans;
    }

private:
    void dfs(const vector<vector<pair<int, int>>>& graph,
             const vector<int>& values, int u, int quality, int remainingTime,
             vector<int>& seen, int& ans) {
        if (u == 0)
            ans = max(ans, quality);
        for (const auto& [v, time] : graph[u]) {
            if (time > remainingTime)
                continue;
            const int newQuality = quality + values[v] * (seen[v] == 0);
            ++seen[v];
            dfs(graph, values, v, newQuality, remainingTime - time, seen, ans);
            --seen[v];
        }
    }
};
```

1146. 2067.cpp

Path: LeetCode-main/solutions/2067. Number of Equal Count Substrings/2067.cpp

```
class Solution {
public:
    int equalCountSubstrings(string s, int count) {
        const int maxUnique = unordered_set<int>(s.begin(), s.end()).size();
        int ans = 0;

        for (int unique = 1; unique <= maxUnique; ++unique) {
            const int windowSize = unique * count;
            vector<int> lettersCount(26);
            int uniqueCount = 0;
            for (int i = 0; i < s.length(); ++i) {
                if (++lettersCount[s[i] - 'a'] == count)
                    ++uniqueCount;
                if (i >= windowSize &&
                    --lettersCount[s[i - windowSize] - 'a'] == count - 1)
                    --uniqueCount;
                ans += uniqueCount == unique;
            }
        }

        return ans;
    }
};
```

1147. 2068.cpp

Path: LeetCode-main/solutions/2068. Check Whether Two Strings are Almost Equivalent/2068.cpp

```
class Solution {
public:
    bool checkAlmostEquivalent(string word1, string word2) {
        vector<int> count(26);

        for (const char c : word1)
            ++count[c - 'a'];

        for (const char c : word2)
            --count[c - 'a'];

        return ranges::all_of(count, [](int freq) { return abs(freq) <= 3; });
    }
};
```

1148. 2069.cpp

Path: LeetCode-main/solutions/2069. Walking Robot Simulation II/2069.cpp

```
class Robot {
public:
    Robot(int width, int height) {
        pos.push_back({0, 0}, "South");
        for (int i = 1; i < width; ++i)
            pos.push_back({i, 0}, "East");
        for (int j = 1; j < height; ++j)
            pos.push_back({width - 1, j}, "North");
        for (int i = width - 2; i >= 0; --i)
            pos.push_back({i, height - 1}, "West");
        for (int j = height - 2; j > 0; --j)
            pos.push_back({0, j}, "South");
    }

    void step(int num) {
        isOrigin = false;
        i = (i + num) % pos.size();
    }

    vector<int> getPos() {
        return pos[i].first;
    }

    string getDir() {
        return isOrigin ? "East" : pos[i].second;
    }

private:
    bool isOrigin = true;
    int i = 0;
    vector<pair<vector<int>, string>> pos;
};
```


1149. 207.cpp

Path: LeetCode-main/solutions/207. Course Schedule/207.cpp

```
enum class State { kInit, kVisiting, kVisited };

class Solution {
public:
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> graph(numCourses);
        vector<State> states(numCourses);

        for (const vector<int>& prerequisite : prerequisites) {
            const int u = prerequisite[1];
            const int v = prerequisite[0];
            graph[u].push_back(v);
        }

        for (int i = 0; i < numCourses; ++i)
            if (hasCycle(graph, i, states))
                return false;

        return true;
    }

private:
    bool hasCycle(const vector<vector<int>>& graph, int u,
                  vector<State>& states) {
        if (states[u] == State::kVisiting)
            return true;
        if (states[u] == State::kVisited)
            return false;
        states[u] = State::kVisiting;
        for (const int v : graph[u])
            if (hasCycle(graph, v, states))
                return true;
        states[u] = State::kVisited;
        return false;
    }
};
```

1150. 2070.cpp

Path: LeetCode-main/solutions/2070. Most Beautiful Item for Each Query/2070.cpp

```
class Solution {
public:
    vector<int> maximumBeauty(vector<vector<int>>& items, vector<int>& queries) {
        vector<int> ans;
        vector<int> prices;
        vector<int> maxBeautySoFar(items.size() + 1);

        ranges::sort(items);

        for (const vector<int>& item : items)
            prices.push_back(item[0]);

        for (int i = 0; i < items.size(); ++i)
            maxBeautySoFar[i + 1] = max(maxBeautySoFar[i], items[i][1]);

        for (const int query : queries) {
            const int i = ranges::upper_bound(prices, query) - prices.begin();
            ans.push_back(maxBeautySoFar[i]);
        }

        return ans;
    }
};
```

1151. 2071.cpp

Path: LeetCode-main/solutions/2071. Maximum Number of Tasks You Can Assign/2071.cpp

```
class Solution {
public:
    int maxTaskAssign(vector<int>& tasks, vector<int>& workers, int pills,
                     int strength) {
        int ans = 0;
        int l = 0;
        int r = min(tasks.size(), workers.size());

        ranges::sort(tasks);
        ranges::sort(workers);

        // Returns true if we can finish k tasks.
        auto canComplete = [&](int k, int pillsLeft) {
            // k strongest workers
            map<int, int> sortedWorkers;
            for (int i = workers.size() - k; i < workers.size(); ++i)
                ++sortedWorkers[workers[i]];

            // Out of the k smallest tasks, start from the biggest one.
            for (int i = k - 1; i >= 0; --i) {
                // Find the first worker that has strength >= tasks[i].
                auto it = sortedWorkers.lower_bound(tasks[i]);
                if (it != sortedWorkers.end()) {
                    if (--(it->second) == 0)
                        sortedWorkers.erase(it);
                } else if (pillsLeft > 0) {
                    // Find the first worker that has strength >= tasks[i] - strength.
                    it = sortedWorkers.lower_bound(tasks[i] - strength);
                    if (it != sortedWorkers.end()) {
                        if (--(it->second) == 0)
                            sortedWorkers.erase(it);
                        --pillsLeft;
                    } else {
                        return false;
                    }
                } else {
                    return false;
                }
            }

            return true;
        };

        while (l <= r) {
            const int m = (l + r) / 2;
            if (canComplete(m, pills)) {
                ans = m;
                l = m + 1;
            } else {
                r = m - 1;
            }
        }

        return ans;
    }
};
```

1152. 2073.cpp

Path: LeetCode-main/solutions/2073. Time Needed to Buy Tickets/2073.cpp

```
class Solution {
public:
    int timeRequiredToBuy(vector<int>& tickets, int k) {
        int ans = 0;

        for (int i = 0; i < tickets.size(); ++i)
            if (i <= k)
                ans += min(tickets[i], tickets[k]);
            else
                ans += min(tickets[i], tickets[k] - 1);

        return ans;
    }
};
```

1153. 2074.cpp

Path: LeetCode-main/solutions/2074. Reverse Nodes in Even Length Groups/2074.cpp

```
class Solution {
public:
    ListNode* reverseEvenLengthGroups(ListNode* head) {
        // prev -> (head -> ... -> tail) -> next -> ...
        ListNode dummy(0, head);
        ListNode* prev = &dummy;
        ListNode* tail = head;
        ListNode* next = head->next;
        int groupLength = 1;

        while (true) {
            if (groupLength % 2 == 1) {
                prev->next = head;
                prev = tail;
            } else {
                tail->next = nullptr;
                prev->next = reverse(head);
                // Prev -> (tail -> ... -> head) -> next -> ...
                head->next = next;
                prev = head;
            }
            if (next == nullptr)
                break;
            head = next;
            const auto [theTail, theLength] = getTailAndLength(head, groupLength + 1);
            tail = theTail;
            next = tail->next;
            groupLength = theLength;
        }

        return dummy.next;
    }

private:
    pair<ListNode*, int> getTailAndLength(ListNode* head, int groupLength) {
        int length = 1;
        ListNode* tail = head;
        while (length < groupLength && tail->next) {
            tail = tail->next;
            ++length;
        }
        return {tail, length};
    }

    ListNode* reverse(ListNode* head) {
        ListNode* prev = nullptr;
        while (head != nullptr) {
            ListNode* next = head->next;
            head->next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
};
```

1154. 2076.cpp

Path: LeetCode-main/solutions/2076. Process Restricted Friend Requests/2076.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    vector<bool> friendRequests(int n, vector<vector<int>>& restrictions,
                                vector<vector<int>>& requests) {
        vector<bool> ans;
        UnionFind uf(n);

        for (const vector<int>& request : requests) {
            const int i = uf.find(request[0]);
            const int j = uf.find(request[1]);
            bool isValid = true;
            if (i != j)
                for (const vector<int>& restriction : restrictions) {
                    const int x = uf.find(restriction[0]);
                    const int y = uf.find(restriction[1]);
                    if (i == x && j == y || i == y && j == x) {
                        isValid = false;
                        break;
                    }
                }
            ans.push_back(isValid);
            if (isValid)
                uf.unionByRank(i, j);
        }

        return ans;
    }
};
```

1155. 2078.cpp

Path: LeetCode-main/solutions/2078. Two Furthest Houses With Different Colors/2078.cpp

```
class Solution {
public:
    int maxDistance(vector<int>& colors) {
        // The maximum distance always includes either the first or the last house.
        const int n = colors.size();
        int i = 0;        // the leftmost index, where colors[i] != colors[-1]
        int j = n - 1;    // the rightmost index, where colors[j] != colors[0]
        while (colors[i] == colors.back())
            ++i;
        while (colors[j] == colors.front())
            --j;
        return max(n - 1 - i, j);
    }
};
```

1156. 208.cpp

Path: LeetCode-main/solutions/208. Implement Trie (Prefix Tree)/208.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    bool isWord = false;
    TrieNode() : children(26) {}
};

class Trie {
public:
    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
        node->isWord = true;
    }

    bool search(const string& word) {
        shared_ptr<TrieNode> node = find(word);
        return node && node->isWord;
    }

    bool startsWith(const string& prefix) {
        return find(prefix) != nullptr;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    shared_ptr<TrieNode> find(const string& prefix) {
        shared_ptr<TrieNode> node = root;
        for (const char c : prefix) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                return nullptr;
            node = node->children[i];
        }
        return node;
    }
};
```


1157. 2083.cpp

Path: LeetCode-main/solutions/2083. Substrings That Begin and End With the Same Letter/2083.cpp

```
class Solution {
public:
    long long numberOfSubstrings(string s) {
        long ans = 0;
        vector<int> count(26);

        for (const char c : s)
            ans += ++count[c - 'a'];

        return ans;
    }
};
```

1158. 2085.cpp

Path: LeetCode-main/solutions/2085. Count Common Words With One Occurrence/2085.cpp

```
class Solution {
public:
    int countWords(vector<string>& words1, vector<string>& words2) {
        unordered_map<string, int> count;

        for (const string& word : words1)
            ++count[word];

        for (const string& word : words2)
            if (const auto it = count.find(word);
                it != count.cend() && it->second < 2)
                --it->second;

        return ranges::count_if(
            count, [](const pair<string, int>& c) { return c.second == 0; });
    }
};
```

1159. 2086.cpp

Path: LeetCode-main/solutions/2086. Minimum Number of Buckets Required to Collect Rainwater from Houses/2086.cpp

```
class Solution {
public:
    int minimumBuckets(string street) {
        for (int i = 0; i < street.length(); ++i)
            if (street[i] == 'H') {
                if (i > 0 && street[i - 1] == 'B')
                    continue;
                if (i + 1 < street.length() && street[i + 1] == '.')
                    // Always prefer place a bucket in (i + 1) because it enhances the
                    // possibility to collect the upcoming houses.
                    street[i + 1] = 'B';
                else if (i > 0 && street[i - 1] == '.')
                    street[i - 1] = 'B';
                else
                    return -1;
            }
        return ranges::count(street, 'B');
    }
};
```

1160. 2087.cpp

Path: LeetCode-main/solutions/2087. Minimum Cost Homecoming of a Robot in a Grid/2087.cpp

```
class Solution {
public:
    int minCost(vector<int>& startPos, vector<int>& homePos,
               vector<int>& rowCosts, vector<int>& colCosts) {
        int ans = 0;
        int i = startPos[0];
        int j = startPos[1];
        int x = homePos[0];
        int y = homePos[1];

        while (i != x)
            ans += i < x ? rowCosts[++i] : rowCosts[--i];
        while (j != y)
            ans += j < y ? colCosts[++j] : colCosts[--j];

        return ans;
    }
};
```

1161. 2088.cpp

Path: LeetCode-main/solutions/2088. Count Fertile Pyramids in a Land/2088.cpp

```
class Solution {
public:
    int countPyramids(vector<vector<int>>& grid) {
        int ans = count(grid);
        ranges::reverse(grid);
        ans += count(grid);
        return ans;
    }

private:
    // dp[i][j] := the maximum height of the pyramid for which it is the apex
    int count(vector<vector<int>> dp) {
        int ans = 0;
        for (int i = dp.size() - 2; i >= 0; --i)
            for (int j = 1; j + 1 < dp[0].size(); ++j)
                if (dp[i][j] == 1) {
                    dp[i][j] =
                        min({dp[i + 1][j - 1], dp[i + 1][j], dp[i + 1][j + 1]}) + 1;
                    ans += dp[i][j] - 1;
                }
        return ans;
    }
};
```

1162. 2089.cpp

Path: LeetCode-main/solutions/2089. Find Target Indices After Sorting Array/2089.cpp

```
class Solution {
public:
    vector<int> targetIndices(vector<int>& nums, int target) {
        vector<int> ans;
        const int count = ranges::count(nums, target);
        int lessThan =
            ranges::count_if(nums, [&](int num) { return num < target; });

        for (int i = 0; i < count; ++i)
            ans.push_back(lessThan++);

        return ans;
    }
};
```

1163. 209.cpp

Path: LeetCode-main/solutions/209. Minimum Size Subarray Sum/209.cpp

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int ans = INT_MAX;
        int sum = 0;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            sum += nums[r];
            while (sum >= target) {
                ans = min(ans, r - l + 1);
                sum -= nums[l++];
            }
        }

        return ans == INT_MAX ? 0 : ans;
    }
};
```

1164. 2090.cpp

Path: LeetCode-main/solutions/2090. K Radius Subarray Averages/2090.cpp

```
class Solution {
public:
    vector<int> getAverages(vector<int>& nums, int k) {
        const int n = nums.size();
        const int size = 2 * k + 1;
        vector<int> ans(n, -1);
        if (size > n)
            return ans;

        long sum = accumulate(nums.begin(), nums.begin() + size, 0L);

        for (int i = k; i + k < n; ++i) {
            ans[i] = sum / size;
            if (i + k + 1 < n)
                sum += nums[i + k + 1] - nums[i - k];
        }

        return ans;
    }
};
```


1165. 2091.cpp

Path: LeetCode-main/solutions/2091. Removing Minimum and Maximum From Array/2091.cpp

```
class Solution {
public:
    int minimumDeletions(vector<int>& nums) {
        const int n = nums.size();
        int a = ranges::min_element(nums) - nums.begin();
        int b = ranges::max_element(nums) - nums.begin();
        if (a > b)
            swap(a, b);
        return min({a + 1 + n - b, b + 1, n - a});
    }
};
```

1166. 2092.cpp

Path: LeetCode-main/solutions/2092. Find All People With Secret/2092.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    bool connected(int u, int v) {
        return find(u) == find(v);
    }

    void reset(int u) {
        id[u] = u;
    }

private:
    vector<int> id;
    vector<int> rank;

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }
};

class Solution {
public:
    vector<int> findAllPeople(int n, vector<vector<int>>& meetings,
                              int firstPerson) {
        vector<int> ans;
        UnionFind uf(n);
        map<int, vector<pair<int, int>>> timeToPairs;

        uf.unionByRank(0, firstPerson);

        for (const vector<int>& meeting : meetings) {
            const int x = meeting[0];
            const int y = meeting[1];
            const int time = meeting[2];
            timeToPairs[time].push_back({x, y});
        }

        for (const auto& [_, pairs] : timeToPairs) {
            unordered_set<int> peopleUnioned;
            for (const auto& [x, y] : pairs) {
                uf.unionByRank(x, y);
                peopleUnioned.insert(x);
                peopleUnioned.insert(y);
            }
            for (const int person : peopleUnioned)
                if (!uf.connected(person, 0))
                    uf.reset(person);
        }

        for (int i = 0; i < n; ++i)
            if (uf.connected(i, 0))
                ans.push_back(i);

        return ans;
    }
};
```

1167. 2093.cpp

Path: LeetCode-main/solutions/2093. Minimum Cost to Reach City With Discounts/2093.cpp

```
struct T {
    int u;
    int d;
    int leftDiscounts;
};

class Solution {
public:
    int minimumCost(int n, vector<vector<int>>& highways, int discounts) {
        vector<vector<pair<int, int>>> graph(n);
        auto compare = [](const T& a, const T& b) { return a.d > b.d; };
        priority_queue<T, vector<T>, decltype(compare)> minHeap(compare);
        unordered_map<int, int> minDiscounts;

        for (const vector<int>& h : highways) {
            const int city1 = h[0];
            const int city2 = h[1];
            const int toll = h[2];
            graph[city1].emplace_back(city2, toll);
            graph[city2].emplace_back(city1, toll);
        }

        minHeap.emplace(0, 0, discounts);

        while (!minHeap.empty()) {
            const auto [u, d, leftDiscounts] = minHeap.top();
            minHeap.pop();
            if (u == n - 1)
                return d;
            if (const auto it = minDiscounts.find(u);
                it != minDiscounts.cend() && it->second >= leftDiscounts)
                continue;
            minDiscounts[u] = leftDiscounts;
            for (const auto& [v, w] : graph[u]) {
                minHeap.emplace(v, d + w, leftDiscounts);
                if (leftDiscounts > 0)
                    minHeap.emplace(v, d + w / 2, leftDiscounts - 1);
            }
        }

        return -1;
    }
};
```

1168. 2094.cpp

Path: LeetCode-main/solutions/2094. Finding 3-Digit Even Numbers/2094.cpp

```
class Solution {
public:
    vector<int> findEvenNumbers(vector<int>& digits) {
        vector<int> ans;
        vector<int> count(10);

        for (const int digit : digits)
            ++count[digit];

        // Try to construct `abc`.
        for (int a = 1; a <= 9; ++a)
            for (int b = 0; b <= 9; ++b)
                for (int c = 0; c <= 8; c += 2)
                    if (count[a] > 0 && count[b] > (b == a) &&
                        count[c] > (c == a) + (c == b))
                        ans.push_back(a * 100 + b * 10 + c);

        return ans;
    }
};
```

1169. 2095.cpp

Path: LeetCode-main/solutions/2095. Delete the Middle Node of a Linked List/2095.cpp

```
class Solution {
public:
    ListNode* deleteMiddle(ListNode* head) {
        ListNode dummy(0, head);
        ListNode* slow = &dummy;
        ListNode* fast = &dummy;

        while (fast->next != nullptr && fast->next->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }

        // Delete the middle node.
        slow->next = slow->next->next;
        return dummy.next;
    }
};
```

1170. 2096-2.cpp

Path: LeetCode-main/solutions/2096. Step-By-Step Directions From a Binary Tree Node to Another/2096-2.cpp

```
class Solution {
public:
    string getDirections(TreeNode* root, int startValue, int destValue) {
        string pathToStart;
        string pathToDest;

        dfs(root, startValue, pathToStart);
        dfs(root, destValue, pathToDest);

        while (!pathToStart.empty() && !pathToDest.empty() &&
            pathToStart.back() == pathToDest.back()) {
            pathToStart.pop_back();
            pathToDest.pop_back();
        }

        return string(pathToStart.length(), 'U') +
            string{pathToDest.rbegin(), pathToDest.rend()};
    }

private:
    // Builds the string in reverse order to avoid creating a new copy.
    bool dfs(TreeNode* root, int val, string& path) {
        if (root->val == val)
            return true;
        if (root->left && dfs(root->left, val, path))
            path.push_back('L');
        else if (root->right && dfs(root->right, val, path))
            path.push_back('R');
        return !path.empty();
    }
};
```

1171. 2096.cpp

Path: LeetCode-main/solutions/2096. Step-By-Step Directions From a Binary Tree Node to Another/2096.cpp

```
class Solution {
public:
    string getDirections(TreeNode* root, int startValue, int destValue) {
        string path;
        string pathToStart;
        string pathToDest;
        // Only this subtree matters.
        dfs(lca(root, startValue, destValue), startValue, destValue, path,
            pathToStart, pathToDest);
        return string(pathToStart.length(), 'U') + pathToDest;
    }

private:
    TreeNode* lca(TreeNode* root, int p, int q) {
        if (root == nullptr || root->val == p || root->val == q)
            return root;
        TreeNode* left = lca(root->left, p, q);
        TreeNode* right = lca(root->right, p, q);
        if (left != nullptr && right != nullptr)
            return root;
        return left == nullptr ? right : left;
    }

    void dfs(TreeNode* root, int p, int q, string& path, string& pathToStart,
        string& pathToDest) {
        if (root == nullptr)
            return;
        if (root->val == p)
            pathToStart = path;
        if (root->val == q)
            pathToDest = path;
        path.push_back('L');
        dfs(root->left, p, q, path, pathToStart, pathToDest);
        path.pop_back();
        path.push_back('R');
        dfs(root->right, p, q, path, pathToStart, pathToDest);
        path.pop_back();
    }
};
```

1172. 2097.cpp

Path: LeetCode-main/solutions/2097. Valid Arrangement of Pairs/2097.cpp

```
class Solution {
public:
    vector<vector<int>> validArrangement(vector<vector<int>>& pairs) {
        vector<vector<int>> ans;
        unordered_map<int, stack<int>> graph;
        unordered_map<int, int> outDegree;
        unordered_map<int, int> inDegrees;

        for (const vector<int>& pair : pairs) {
            const int start = pair[0];
            const int end = pair[1];
            graph[start].push(end);
            ++outDegree[start];
            ++inDegrees[end];
        }

        const int startNode = getStartNode(graph, outDegree, inDegrees, pairs);
        euler(graph, startNode, ans);
        ranges::reverse(ans);
        return ans;
    }

private:
    int getStartNode(const unordered_map<int, stack<int>>& graph,
                    unordered_map<int, int>& outDegree,
                    unordered_map<int, int>& inDegrees,
                    const vector<vector<int>>& pairs) {
        for (const auto& [u, _] : graph)
            if (outDegree[u] - inDegrees[u] == 1)
                return u;
        return pairs[0][0]; // Arbitrarily choose a node.
    }

    void euler(unordered_map<int, stack<int>>& graph, int u,
              vector<vector<int>>& ans) {
        auto& stack = graph[u];
        while (!stack.empty()) {
            const int v = stack.top();
            stack.pop();
            euler(graph, v, ans);
            ans.push_back({u, v});
        }
    }
};
```


1173. 2098.cpp

Path: LeetCode-main/solutions/2098. Subsequence of Size K With the Largest Even Sum/2098.cpp

```
class Solution {
public:
    long long largestEvenSum(vector<int>& nums, int k) {
        ranges::sort(nums);
        long sum = accumulate(nums.end() - k, nums.end(), 0L);
        if (sum % 2 == 0)
            return sum;

        int minOdd = -1;
        int minEven = -1;
        int maxOdd = -1;
        int maxEven = -1;

        for (int i = nums.size() - 1; i + k >= nums.size(); --i)
            if (nums[i] % 2 == 1)
                minOdd = nums[i];
            else
                minEven = nums[i];

        for (int i = 0; i + k < nums.size(); ++i)
            if (nums[i] % 2 == 1)
                maxOdd = nums[i];
            else
                maxEven = nums[i];

        long ans = -1;

        if (maxEven >= 0 && minOdd >= 0)
            ans = max(ans, sum + maxEven - minOdd);
        if (maxOdd >= 0 && minEven >= 0)
            ans = max(ans, sum + maxOdd - minEven);
        return ans;
    }
};
```

1174. 2099.cpp

Path: LeetCode-main/solutions/2099. Find Subsequence of Length K With the Largest Sum/2099.cpp

```
class Solution {
public:
    vector<int> maxSubsequence(vector<int>& nums, int k) {
        vector<int> ans;
        vector<int> arr(nums);
        nth_element(arr.begin(), arr.end() - k, arr.end());
        const int threshold = arr[arr.size() - k];
        const int larger =
            ranges::count_if(nums, [&](int num) { return num > threshold; });
        int equal = k - larger;

        for (const int num : nums)
            if (num > threshold) {
                ans.push_back(num);
            } else if (num == threshold && equal) {
                ans.push_back(num);
                --equal;
            }

        return ans;
    }
};
```

1175. 21.cpp

Path: LeetCode-main/solutions/21. Merge Two Sorted Lists/21.cpp

```
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if (!list1 || !list2)
            return list1 ? list1 : list2;
        if (list1->val > list2->val)
            swap(list1, list2);
        list1->next = mergeTwoLists(list1->next, list2);
        return list1;
    }
};
```

1176. 210-2.cpp

Path: LeetCode-main/solutions/210. Course Schedule II/210-2.cpp

```
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<int> ans;
        vector<vector<int>> graph(numCourses);
        vector<int> inDegrees(numCourses);
        queue<int> q;

        // Build the graph.
        for (const vector<int>& prerequisite : prerequisites) {
            const int u = prerequisite[1];
            const int v = prerequisite[0];
            graph[u].push_back(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        for (int i = 0; i < numCourses; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            ans.push_back(u);
            for (const int v : graph[u])
                if (--inDegrees[v] == 0)
                    q.push(v);
        }

        return ans.size() == numCourses ? ans : vector<int>();
    }
};
```

1177. 210.cpp

Path: LeetCode-main/solutions/210. Course Schedule II/210.cpp

```
enum class State { kInit, kVisiting, kVisited };

class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<int> ans;
        vector<vector<int>> graph(numCourses);
        vector<State> states(numCourses);

        for (const vector<int>& prerequisite : prerequisites) {
            const int u = prerequisite[1];
            const int v = prerequisite[0];
            graph[u].push_back(v);
        }

        for (int i = 0; i < numCourses; ++i)
            if (hasCycle(graph, i, states, ans))
                return {};

        ranges::reverse(ans);
        return ans;
    }

private:
    bool hasCycle(const vector<vector<int>>& graph, int u, vector<State>& states,
                  vector<int>& ans) {
        if (states[u] == State::kVisiting)
            return true;
        if (states[u] == State::kVisited)
            return false;
        states[u] = State::kVisiting;
        for (const int v : graph[u])
            if (hasCycle(graph, v, states, ans))
                return true;
        states[u] = State::kVisited;
        ans.push_back(u);
        return false;
    }
};
```

1178. 2100.cpp

Path: LeetCode-main/solutions/2100. Find Good Days to Rob the Bank/2100.cpp

```
class Solution {
public:
    vector<int> goodDaysToRobBank(vector<int>& security, int time) {
        const int n = security.size();
        vector<int> ans;
        // dec[i] := the number of continuous decreasing numbers before i
        vector<int> dec(n);
        // inc[i] := the number of continuous increasing numbers after i
        vector<int> inc(n);

        for (int i = 1; i < n; ++i)
            if (security[i - 1] >= security[i])
                dec[i] = dec[i - 1] + 1;

        for (int i = n - 2; i >= 0; --i)
            if (security[i] <= security[i + 1])
                inc[i] = inc[i + 1] + 1;

        for (int i = 0; i < n; ++i)
            if (dec[i] >= time && inc[i] >= time)
                ans.push_back(i);

        return ans;
    }
};
```

1179. 2101.cpp

Path: LeetCode-main/solutions/2101. Detonate the Maximum Bombs/2101.cpp

```
class Solution {
public:
    int maximumDetonation(vector<vector<int>>& bombs) {
        const int n = bombs.size();
        size_t ans = 0;
        vector<vector<int>> graph(n);

        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == j)
                    continue;
                const long ri = bombs[i][2];
                if (ri * ri >= squaredDist(bombs, i, j))
                    graph[i].push_back(j);
            }
        }

        for (int i = 0; i < n; ++i) {
            unordered_set<int> seen{i};
            dfs(graph, i, seen);
            ans = max(ans, seen.size());
        }

        return ans;
    }

private:
    void dfs(const vector<vector<int>>& graph, int u, unordered_set<int>& seen) {
        for (const int v : graph[u]) {
            if (seen.contains(v))
                continue;
            seen.insert(v);
            dfs(graph, v, seen);
        }
    }

    long squaredDist(const vector<vector<int>>& bombs, int i, int j) {
        return static_cast<long>(bombs[i][0] - bombs[j][0]) *
            (bombs[i][0] - bombs[j][0]) +
            static_cast<long>(bombs[i][1] - bombs[j][1]) *
            (bombs[i][1] - bombs[j][1]);
    }
};
```

1180. 2102.cpp

Path: LeetCode-main/solutions/2102. Sequentially Ordinal Rank Tracker/2102.cpp

```
struct Location {
    string name;
    int score;
    Location(const string& name, int score)
        : name(std::move(name)), score(score) {}
};

class SORTracker {
public:
    void add(const string& name, int score) {
        l.emplace(name, score);
        if (l.size() > k + 1) {
            const Location location = l.top();
            l.pop();
            r.emplace(location.name, location.score);
        }
    }

    string get() {
        const string name = l.top().name;
        if (!r.empty()) {
            const Location location = r.top();
            r.pop();
            l.emplace(location.name, location.score);
        }
        ++k;
        return name;
    }

private:
    struct CompareLeftMinHeap {
        bool operator()(const Location& a, const Location& b) {
            return a.score == b.score ? a.name < b.name : a.score > b.score;
        }
    };

    struct CompareRightMaxHeap {
        bool operator()(const Location& a, const Location& b) {
            return a.score == b.score ? a.name > b.name : a.score < b.score;
        }
    };

    priority_queue<Location, vector<Location>, CompareLeftMinHeap> l;
    priority_queue<Location, vector<Location>, CompareRightMaxHeap> r;
    int k = 0;
};
```


1181. 2103.cpp

Path: LeetCode-main/solutions/2103. Rings and Rods/2103.cpp

```
class Solution {
public:
    int countPoints(string rings) {
        vector<int> colors(10);

        for (int i = 0; i < rings.length(); i += 2) {
            const int c = rings[i];
            const int color = c == 'R' ? 1 : c == 'G' ? 2 : 4;
            colors[rings[i + 1] - '0'] |= color;
        }

        return ranges::count(colors, 7);
    }
};
```

1182. 2104.cpp

Path: LeetCode-main/solutions/2104. Sum of Subarray Ranges/2104.cpp

```
class Solution {
public:
    long long subArrayRanges(vector<int>& nums) {
        const auto [prevGt, nextGt] = getPrevNext(nums, less<>());
        const auto [prevLt, nextLt] = getPrevNext(nums, greater<>());
        long ans = 0;

        for (int i = 0; i < nums.size(); ++i) {
            ans += static_cast<long>(nums[i]) * (i - prevGt[i]) * (nextGt[i] - i);
            ans -= static_cast<long>(nums[i]) * (i - prevLt[i]) * (nextLt[i] - i);
        }

        return ans;
    }

private:
    // Returns `prev` and `next`, that store the indices of the nearest numbers
    // that are smaller or larger than the current number depending on `op`.
    pair<vector<int>, vector<int>> getPrevNext(
        const vector<int>& nums, const function<bool(int, int)>& op) {
        const int n = nums.size();
        vector<int> prev(n, -1);
        vector<int> next(n, n);
        stack<int> stack;
        for (int i = 0; i < n; ++i) {
            while (!stack.empty() && op(nums[stack.top()], nums[i])) {
                const int index = stack.top();
                stack.pop();
                next[index] = i;
            }
            if (!stack.empty())
                prev[i] = stack.top();
            stack.push(i);
        }
        return {prev, next};
    }
};
```

1183. 2105.cpp

Path: LeetCode-main/solutions/2105. Watering Plants II/2105.cpp

```
class Solution {
public:
    int minimumRefill(vector<int>& plants, int capacityA, int capacityB) {
        int ans = 0;
        int i = 0;
        int j = plants.size() - 1;
        int canA = capacityA;
        int canB = capacityB;

        while (i < j) {
            ans += (canA < plants[i]) + (canB < plants[j]);
            if (canA < plants[i])
                canA = capacityA;
            if (canB < plants[j])
                canB = capacityB;
            canA -= plants[i++];
            canB -= plants[j--];
        }

        return ans + (i == j && max(canA, canB) < plants[i]);
    }
};
```

1184. 2106.cpp

Path: LeetCode-main/solutions/2106. Maximum Fruits Harvested After at Most K Steps/2106.cpp

```
class Solution {
public:
    int maxTotalFruits(vector<vector<int>>& fruits, int startPos, int k) {
        const int maxRight = max(startPos, fruits.back()[0]);
        int ans = 0;
        vector<int> amounts(1 + maxRight);
        vector<int> prefix(2 + maxRight);

        for (const vector<int>& f : fruits)
            amounts[f[0]] = f[1];

        partial_sum(amounts.begin(), amounts.end(), prefix.begin() + 1);

        auto getFruits = [&](int leftSteps, int rightSteps) {
            const int l = max(0, startPos - leftSteps);
            const int r = min(maxRight, startPos + rightSteps);
            return prefix[r + 1] - prefix[l];
        };

        // Go right first.
        const int maxRightSteps = min(maxRight - startPos, k);
        for (int rightSteps = 0; rightSteps <= maxRightSteps; ++rightSteps) {
            const int leftSteps = max(0, k - 2 * rightSteps); // Turn left
            ans = max(ans, getFruits(leftSteps, rightSteps));
        }

        // Go left first.
        const int maxLeftSteps = min(startPos, k);
        for (int leftSteps = 0; leftSteps <= maxLeftSteps; ++leftSteps) {
            const int rightSteps = max(0, k - 2 * leftSteps); // Turn right
            ans = max(ans, getFruits(leftSteps, rightSteps));
        }

        return ans;
    }
};
```

1185. 2107.cpp

Path: LeetCode-main/solutions/2107. Number of Unique Flavors After Sharing K Candies/2107.cpp

```
class Solution {
public:
    int shareCandies(vector<int>& candies, int k) {
        int ans = 0;
        unordered_map<int, int> count;

        for (const int candy : candies)
            ++count[candy];

        int unique = count.size();

        for (int i = 0; i < candies.size(); ++i) {
            if (--count[candies[i]] == 0) {
                count.erase(candies[i]);
                --unique;
            }
            if (i >= k && ++count[candies[i - k]] == 1)
                ++unique;
            if (i >= k - 1)
                ans = max(ans, unique);
        }

        return ans;
    }
};
```

1186. 2108.cpp

Path: LeetCode-main/solutions/2108. Find First Palindromic String in the Array/2108.cpp

```
class Solution {
public:
    string firstPalindrome(vector<string>& words) {
        for (const string& word : words)
            if (isPalindrome(word))
                return word;
        return "";
    }

private:
    bool isPalindrome(const string& s) {
        int i = 0;
        int j = s.length() - 1;
        while (i < j)
            if (s[i++] != s[j--])
                return false;
        return true;
    }
};
```

1187. 2109.cpp

Path: LeetCode-main/solutions/2109. Adding Spaces to a String/2109.cpp

```
class Solution {  
public:  
    string addSpaces(string s, vector<int>& spaces) {}  
};
```

1188. 211.cpp

Path: LeetCode-main/solutions/211. Add and Search Word - Data structure design/211.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    bool isWord = false;
    TrieNode() : children(26) {}
};

class WordDictionary {
public:
    void addWord(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
        node->isWord = true;
    }

    bool search(const string& word) {
        return dfs(word, 0, root);
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    bool dfs(const string& word, int s, shared_ptr<TrieNode> node) {
        if (s == word.length())
            return node->isWord;
        if (word[s] != '.') {
            shared_ptr<TrieNode> next = node->children[word[s] - 'a'];
            return next ? dfs(word, s + 1, next) : false;
        }

        // If word[s] == '.', then search all the 26 children.
        for (int i = 0; i < 26; ++i)
            if (node->children[i] && dfs(word, s + 1, node->children[i]))
                return true;

        return false;
    }
};
```


1189. 2110.cpp

Path: LeetCode-main/solutions/2110. Number of Smooth Descent Periods of a Stock/2110.cpp

```
class Solution {
public:
    long long getDescentPeriods(vector<int>& prices) {
        long ans = 1; // prices[0]
        int dp = 1;

        for (int i = 1; i < prices.size(); ++i) {
            if (prices[i] == prices[i - 1] - 1)
                ++dp;
            else
                dp = 1;
            ans += dp;
        }
        return ans;
    }
};
```

1190. 2111.cpp

Path: LeetCode-main/solutions/2111. Minimum Operations to Make the Array K-Increasing/2111.cpp

```
class Solution {
public:
    int kIncreasing(vector<int>& arr, int k) {
        int ans = 0;

        for (int i = 0; i < k; ++i) {
            vector<int> arr;
            for (int j = i; j < arr.size(); j += k)
                arr.push_back(arr[j]);
            ans += numReplaced(arr);
        }

        return ans;
    }

private:
    int numReplaced(const vector<int>& arr) {
        vector<int> tails;
        for (const int a : arr)
            if (tails.empty() || tails.back() <= a)
                tails.push_back(a);
            else
                tails[firstGreater(tails, a)] = a;
        return arr.size() - tails.size();
    }

    int firstGreater(const vector<int>& arr, int target) {
        return ranges::upper_bound(arr, target) - arr.begin();
    }
};
```

1191. 2113.cpp

Path: LeetCode-main/solutions/2113. Elements in Array After Removing and Replacing Elements/2113.cpp

```
class Solution {
public:
    vector<int> elementInNums(vector<int>& nums, vector<vector<int>>& queries) {
        vector<int> ans;

        for (const vector<int>& query : queries) {
            const int time = query[0];
            const int index = query[1];
            ans.push_back(f(nums, time % (2 * nums.size()), index));
        }

        return ans;
    }

private:
    int f(const vector<int>& arr, int time, int index) {
        const int n = arr.size();
        if (time < n) { // [0, 1, 2] -> [1, 2] -> [2]
            index += time;
            return index >= n ? -1 : arr[index];
        } else { // [] -> [0] -> [0, 1]
            return index >= time - n ? -1 : arr[index];
        }
    }
};
```

1192. 2114.cpp

Path: LeetCode-main/solutions/2114. Maximum Number of Words Found in Sentences/2114.cpp

```
class Solution {
public:
    int mostWordsFound(vector<string>& sentences) {
        long maxSpaceCount = 0;

        for (const string& s : sentences)
            maxSpaceCount = max(maxSpaceCount, ranges::count(s, ' '));

        return maxSpaceCount + 1;
    }
};
```

1193. 2115.cpp

Path: LeetCode-main/solutions/2115. Find All Possible Recipes from Given Supplies/2115.cpp

```
class Solution {
public:
    vector<string> findAllRecipes(vector<string>& recipes,
                                vector<vector<string>>& ingredients,
                                vector<string>& supplies) {

        vector<string> ans;
        unordered_set<string> suppliesSet(supplies.begin(), supplies.end());
        unordered_map<string, vector<string>>> graph;
        unordered_map<string, int> inDegrees;
        queue<string> q;

        // Build the graph.
        for (int i = 0; i < recipes.size(); ++i)
            for (const string& ingredient : ingredients[i])
                if (!suppliesSet.contains(ingredient)) {
                    graph[ingredient].push_back(recipes[i]);
                    ++inDegrees[recipes[i]];
                }

        // Perform topological sorting.
        for (const string& recipe : recipes)
            if (!inDegrees.contains(recipe))
                q.push(recipe);

        while (!q.empty()) {
            const string u = q.front();
            q.pop();
            ans.push_back(u);
            if (!graph.contains(u))
                continue;
            for (const string& v : graph[u])
                if (--inDegrees[v] == 0)
                    q.push(v);
        }

        return ans;
    }
};
```

1194. 2116.cpp

Path: LeetCode-main/solutions/2116. Check if a Parentheses String Can Be Valid/2116.cpp

```
class Solution {
public:
    bool canBeValid(string s, string locked) {
        if (s.length() % 2 == 1)
            return false;

        const bool leftToRightIsOkay = check(s, locked, true);
        ranges::reverse(s);
        ranges::reverse(locked);
        const bool rightToLeftIsOkay = check(s, locked, false);
        return leftToRightIsOkay && rightToLeftIsOkay;
    }

private:
    bool check(const string& s, const string& locked, bool isForward) {
        int changeable = 0;
        int l = 0;
        int r = 0;

        for (int i = 0; i < s.length(); ++i) {
            const char c = s[i];
            const char lock = locked[i];
            if (lock == '0')
                ++changeable;
            else if (c == '(')
                ++l;
            else // c == ')'
                ++r;
            if (isForward && changeable + l - r < 0)
                return false;
            if (!isForward && changeable + r - l < 0)
                return false;
        }

        return true;
    }
};
```

1195. 2117.cpp

Path: LeetCode-main/solutions/2117. Abbreviating the Product of a Range/2117.cpp

```
class Solution {
public:
    string abbreviateProduct(int left, int right) {
        constexpr long maxSuf = 100'000'000'000;
        double prod = 1.0;
        long suf = 1;
        int countDigits = 0;
        int countZeros = 0;

        for (int num = left; num <= right; ++num) {
            prod *= num;
            while (prod >= 1.0) {
                prod /= 10;
                ++countDigits;
            }
            suf *= num;
            while (suf % 10 == 0) {
                suf /= 10;
                ++countZeros;
            }
            if (suf > maxSuf)
                suf %= maxSuf;
        }

        if (countDigits - countZeros <= 10) {
            const long tens = pow(10, countDigits - countZeros);
            return to_string(static_cast<long>(prod * tens + 0.5)) + 'e' +
                to_string(countZeros);
        }

        const string pre = to_string(static_cast<long>(prod * pow(10, 5)));
        string sufStr = to_string(suf);
        sufStr = sufStr.substr(sufStr.length() - 5);
        return pre + "..." + sufStr + 'e' + to_string(countZeros);
    }
};
```

1196. 2119-2.cpp

Path: LeetCode-main/solutions/2119. A Number After a Double Reversal/2119-2.cpp

```
class Solution {  
public:  
    bool isSameAfterReversals(int num) {  
        return num == 0 || num % 10;  
    }  
};
```


1197. 2119.cpp

Path: LeetCode-main/solutions/2119. A Number After a Double Reversal/2119.cpp

```
class Solution {
public:
    bool isSameAfterReversals(int num) {
        const int reversed1 = getReversed(num);
        const int reversed2 = getReversed(reversed1);
        return reversed2 == num;
    }

private:
    int getReversed(int num) {
        int reversed = 0;
        while (num > 0) {
            reversed = reversed * 10 + num % 10;
            num /= 10;
        }
        return reversed;
    }
};
```

1198. 212.cpp

Path: LeetCode-main/solutions/212. Word Search II/212.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    const string* word = nullptr;
    TrieNode() : children(26) {}
};

class Solution {
public:
    vector<string> findWords(vector<vector<char>>& board, vector<string>& words) {
        vector<string> ans;

        for (const string& word : words)
            insert(word);

        for (int i = 0; i < board.size(); ++i)
            for (int j = 0; j < board[0].size(); ++j)
                dfs(board, i, j, root, ans);

        return ans;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
        node->word = &word;
    }

    void dfs(vector<vector<char>>& board, int i, int j, shared_ptr<TrieNode> node,
            vector<string>& ans) {
        if (i < 0 || i == board.size() || j < 0 || j == board[0].size())
            return;
        if (board[i][j] == '*')
            return;

        const char c = board[i][j];
        shared_ptr<TrieNode> child = node->children[c - 'a'];
        if (child == nullptr)
            return;
        if (child->word != nullptr) {
            ans.push_back(*child->word);
            child->word = nullptr;
        }

        board[i][j] = '*';
        dfs(board, i + 1, j, child, ans);
        dfs(board, i - 1, j, child, ans);
        dfs(board, i, j + 1, child, ans);
        dfs(board, i, j - 1, child, ans);
        board[i][j] = c;
    }
};
```

1199. 2120.cpp

Path: LeetCode-main/solutions/2120. Execution of All Suffix Instructions Staying in a Grid/2120.cpp

```
class Solution {
public:
    vector<int> executeInstructions(int n, vector<int>& startPos, string s) {
        const int m = s.length();
        const int uMost = startPos[0] + 1;
        const int dMost = n - startPos[0];
        const int lMost = startPos[1] + 1;
        const int rMost = n - startPos[1];
        const unordered_map<char, pair<int, int>> moves{
            {'L', {0, -1}},
            {'R', {0, 1}},
            {'U', {-1, 0}},
            {'D', {1, 0}},
        };

        vector<int> ans(m);
        unordered_map<int, int> reachX{{0, m}};
        unordered_map<int, int> reachY{{0, m}};
        int x = 0;
        int y = 0;

        for (int i = m - 1; i >= 0; --i) {
            const auto& [dx, dy] = moves.at(s[i]);
            x -= dx;
            y -= dy;
            reachX[x] = i;
            reachY[y] = i;
            int out = INT_MAX;
            if (const auto it = reachX.find(x - uMost); it != reachX.cend())
                out = min(out, it->second);
            if (const auto it = reachX.find(x + dMost); it != reachX.cend())
                out = min(out, it->second);
            if (const auto it = reachY.find(y - lMost); it != reachY.cend())
                out = min(out, it->second);
            if (const auto it = reachY.find(y + rMost); it != reachY.cend())
                out = min(out, it->second);
            ans[i] = out == INT_MAX ? m - i : out - i - 1;
        }

        return ans;
    }
};
```

1200. 2121.cpp

Path: LeetCode-main/solutions/2121. Intervals Between Identical Elements/2121.cpp

```
class Solution {
public:
    vector<long long> getDistances(vector<int>& arr) {
        const int n = arr.size();
        vector<long long> ans(n);
        vector<long long> prefix(n);
        vector<long long> suffix(n);
        unordered_map<int, vector<int>> numToIndices;

        for (int i = 0; i < n; ++i)
            numToIndices[arr[i]].push_back(i);

        for (const auto& [_, indices] : numToIndices) {
            for (int i = 1; i < indices.size(); ++i) {
                const int currIndex = indices[i];
                const int prevIndex = indices[i - 1];
                prefix[currIndex] += prefix[prevIndex] + i * (currIndex - prevIndex);
            }
            for (int i = indices.size() - 2; i >= 0; --i) {
                const int currIndex = indices[i];
                const int prevIndex = indices[i + 1];
                suffix[currIndex] += suffix[prevIndex] +
                    (indices.size() - i - 1) * (prevIndex - currIndex);
            }
        }

        for (int i = 0; i < n; ++i)
            ans[i] = prefix[i] + suffix[i];

        return ans;
    }
};
```

1201. 2122.cpp

Path: LeetCode-main/solutions/2122. Recover the Original Array/2122.cpp

```
class Solution {
public:
    vector<int> recoverArray(vector<int>& nums) {
        const int n = nums.size();
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        ranges::sort(nums);

        for (int i = 1; i < n; ++i) {
            const int x = nums[i] - nums[0]; // 2 * k
            if (x <= 0 || x % 2 == 1)
                continue;
            vector<int> arr = getArray(nums, x, count);
            if (!arr.empty())
                return arr;
        }

        throw;
    }

private:
    vector<int> getArray(const vector<int>& nums, int x,
                        unordered_map<int, int> count) {
        vector<int> arr;
        for (const int num : nums) {
            if (const auto it = count.find(num);
                it == count.cend() || it->second == 0)
                continue;
            if (const auto it = count.find(num + x);
                it == count.cend() || it->second == 0)
                return {};
            --count[num];
            --count[num + x];
            arr.push_back(num + x / 2);
        }
        return arr;
    }
};
```

1202. 2123.cpp

Path: LeetCode-main/solutions/2123. Minimum Operations to Remove Adjacent Ones in Matrix/2123.cpp

```
class Solution {
public:
    int minimumOperations(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        int ans = 0;
        vector<vector<int>> seen(m, vector<int>(n));
        vector<vector<int>> match(m, vector<int>(n, -1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1 && match[i][j] == -1) {
                    const int sessionId = i * n + j;
                    seen[i][j] = sessionId;
                    ans += dfs(grid, i, j, sessionId, seen, match);
                }

        return ans;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    int dfs(const vector<vector<int>>& grid, int i, int j, int sessionId,
            vector<vector<int>>& seen, vector<vector<int>>& match) {
        const int m = grid.size();
        const int n = grid[0].size();

        for (const auto& [dx, dy] : kDirs) {
            const int x = i + dx;
            const int y = j + dy;
            if (x < 0 || x == m || y < 0 || y == n)
                continue;
            if (grid[x][y] == 0 || seen[x][y] == sessionId)
                continue;
            seen[x][y] = sessionId;
            if (match[x][y] == -1 ||
                dfs(grid, match[x][y] / n, match[x][y] % n, sessionId, seen, match)) {
                match[x][y] = i * n + j;
                match[i][j] = x * n + y;
                return 1;
            }
        }

        return 0;
    }
};
```

1203. 2124.cpp

Path: LeetCode-main/solutions/2124. Check if All A's Appears Before All B's/2124.cpp

```
class Solution {  
public:  
    bool checkString(string s) {  
        return s.find("ba") == string::npos;  
    }  
};
```

1204. 2125.cpp

Path: LeetCode-main/solutions/2125. Number of Laser Beams in a Bank/2125.cpp

```
class Solution {
public:
    int numberOfBeams(vector<string>& bank) {
        int ans = 0;
        int prevOnes = 0;

        for (const string& row : bank) {
            const int ones = ranges::count(row, '1');
            if (ones > 0) {
                ans += prevOnes * ones;
                prevOnes = ones;
            }
        }

        return ans;
    }
};
```


1205. 2126.cpp

Path: LeetCode-main/solutions/2126. Destroying Asteroids/2126.cpp

```
class Solution {
public:
    bool asteroidsDestroyed(int mass, vector<int>& asteroids) {
        ranges::sort(asteroids);

        long m = mass;

        for (const int asteroid : asteroids)
            if (m >= asteroid)
                m += asteroid;
            else
                return false;

        return true;
    }
};
```

1206. 2127.cpp

Path: LeetCode-main/solutions/2127. Maximum Employees to Be Invited to a Meeting/2127.cpp

```
enum class State { kInit, kVisiting, kVisited };

class Solution {
public:
    int maximumInvitations(vector<int>& favorite) {
        const int n = favorite.size();
        int sumComponentsLength = 0; // the component: a -> b -> c <-> x <- y
        vector<vector<int>> graph(n);
        vector<int> inDegrees(n);
        vector<int> maxChainLength(n, 1);
        queue<int> q;

        // Build the graph.
        for (int i = 0; i < n; ++i) {
            graph[i].push_back(favorite[i]);
            ++inDegrees[favorite[i]];
        }

        // Perform topological sorting.
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            for (const int v : graph[u]) {
                if (--inDegrees[v] == 0)
                    q.push(v);
                maxChainLength[v] = max(maxChainLength[v], 1 + maxChainLength[u]);
            }
        }

        for (int i = 0; i < n; ++i)
            if (favorite[favorite[i]] == i)
                // i <-> favorite[i] (the cycle's length = 2)
                sumComponentsLength += maxChainLength[i] + maxChainLength[favorite[i]];

        int maxCycleLength = 0; // the cycle : a -> b -> c -> a
        vector<int> parent(n, -1);
        vector<bool> seen(n);
        vector<State> states(n);

        for (int i = 0; i < n; ++i)
            if (!seen[i])
                findCycle(graph, i, parent, seen, states, maxCycleLength);

        return max(sumComponentsLength / 2, maxCycleLength);
    }

private:
    void findCycle(const vector<vector<int>>& graph, int u, vector<int>& parent,
                  vector<bool>& seen, vector<State>& states,
                  int& maxCycleLength) {
        seen[u] = true;
        states[u] = State::kVisiting;

        for (const int v : graph[u]) {
            if (!seen[v]) {
                parent[v] = u;
                findCycle(graph, v, parent, seen, states, maxCycleLength);
            } else if (states[v] == State::kVisiting) {
                // Find the cycle's length.
                int curr = u;
                int cycleLength = 1;
                while (curr != v) {
                    curr = parent[curr];
                    ++cycleLength;
                }
                maxCycleLength = max(maxCycleLength, cycleLength);
            }
        }

        states[u] = State::kVisited;
    }
};
```

1207. 2128.cpp

Path: LeetCode-main/solutions/2128. Remove All Ones With Row and Column Flips/2128.cpp

```
class Solution {
public:
    bool removeOnes(vector<vector<int>>& grid) {
        const vector<int> revRow = getRevRow(grid[0]);
        return ranges::all_of(
            grid, [&](const auto& row) { return row == grid[0] || row == revRow; });
    }

private:
    vector<int> getRevRow(vector<int>& row) {
        vector<int> revRow;
        for (const int a : row)
            revRow.push_back(a ^ 1);
        return revRow;
    }
};
```

1208. 2129.cpp

Path: LeetCode-main/solutions/2129. Capitalize the Title/2129.cpp

```
class Solution {
public:
    string capitalizeTitle(string title) {
        ranges::transform(title, title.begin(), ::tolower);

        int i = 0; // Point to the start of a word.
        int j = 0; // Point to the end of a word.

        while (j < title.length()) {
            while (j < title.length() && title[j] != ' ')
                ++j;
            if (j - i > 2)
                title[i] = toupper(title[i]);
            i = j + 1;
            ++j; // Skip the spaces.
        }

        return title;
    }
};
```

1209. 213.cpp

Path: LeetCode-main/solutions/213. House Robber II/213.cpp

```
class Solution {
public:
    int rob(vector<int>& nums) {
        if (nums.empty())
            return 0;
        if (nums.size() == 1)
            return nums[0];

        auto rob = [&](int l, int r) {
            int prev1 = 0; // dp[i - 1]
            int prev2 = 0; // dp[i - 2]

            for (int i = l; i <= r; ++i) {
                const int dp = max(prev1, prev2 + nums[i]);
                prev2 = prev1;
                prev1 = dp;
            }

            return prev1;
        };

        return max(rob(0, nums.size() - 2), rob(1, nums.size() - 1));
    }
};
```

1210. 2130.cpp

Path: LeetCode-main/solutions/2130. Maximum Twin Sum of a Linked List/2130.cpp

```
class Solution {
public:
    int pairSum(ListNode* head) {
        int ans = 0;
        ListNode* slow = head;
        ListNode* fast = head;

        // `slow` points to the start of the second half.
        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }

        // `tail` points to the end of the reversed second half.
        ListNode* tail = reverseList(slow);

        while (tail != nullptr) {
            ans = max(ans, head->val + tail->val);
            head = head->next;
            tail = tail->next;
        }

        return ans;
    }

private:
    ListNode* reverseList(ListNode* head) {
        ListNode* prev = nullptr;
        while (head != nullptr) {
            auto next = head->next;
            head->next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
};
```

1211. 2131.cpp

Path: LeetCode-main/solutions/2131. Longest Palindrome by Concatenating Two Letter Words/2131.cpp

```
class Solution {
public:
    int longestPalindrome(vector<string>& words) {
        int ans = 0;
        vector<vector<int>> count(26, vector<int>(26));

        for (const string& word : words) {
            const int i = word[0] - 'a';
            const int j = word[1] - 'a';
            if (count[j][i]) {
                ans += 4;
                --count[j][i];
            } else {
                ++count[i][j];
            }
        }

        for (int i = 0; i < 26; ++i)
            if (count[i][i])
                return ans + 2;

        return ans;
    }
};
```

1212. 2132.cpp

Path: LeetCode-main/solutions/2132. Stamping the Grid/2132.cpp

```
class Solution {
public:
    bool possibleToStamp(vector<vector<int>>& grid, int stampHeight,
                        int stampWidth) {
        const int m = grid.size();
        const int n = grid[0].size();
        // A[i][j] := the number of 1s in grid[0..i][0..j]
        vector<vector<int>> A(m + 1, vector<int>(n + 1));
        // B[i][j] := the number of ways to stamp the submatrix in [0..i][0..j]
        vector<vector<int>> B(m + 1, vector<int>(n + 1));
        // fit[i][j] := true if the stamps can fit with the right-bottom at (i, j)
        vector<vector<bool>> fit(m, vector<bool>(n));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                A[i + 1][j + 1] = A[i + 1][j] + A[i][j + 1] - A[i][j] + grid[i][j];
                if (i + 1 >= stampHeight && j + 1 >= stampWidth) {
                    const int x = i - stampHeight + 1;
                    const int y = j - stampWidth + 1;
                    if (A[i + 1][j + 1] - A[x][j + 1] - A[i + 1][y] + A[x][y] == 0)
                        fit[i][j] = true;
                }
            }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                B[i + 1][j + 1] = B[i + 1][j] + B[i][j + 1] - B[i][j] + fit[i][j];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (!grid[i][j]) {
                    const int x = min(i + stampHeight, m);
                    const int y = min(j + stampWidth, n);
                    if (B[x][y] - B[i][y] - B[x][j] + B[i][j] == 0)
                        return false;
                }

        return true;
    }
};
```


1213. 2133.cpp

Path: LeetCode-main/solutions/2133. Check if Every Row and Column Contains All Numbers/2133.cpp

```
class Solution {
public:
    bool checkValid(vector<vector<int>>& matrix) {
        const int n = matrix.size();

        for (int i = 0; i < n; ++i) {
            bitset<101> row;
            bitset<101> col;
            for (int j = 0; j < n; ++j) {
                row[matrix[i][j]] = true;
                col[matrix[j][i]] = true;
            }
            if (min(row.contains(), col.contains()) < n)
                return false;
        }

        return true;
    }
};
```

1214. 2134.cpp

Path: LeetCode-main/solutions/2134. Minimum Swaps to Group All 1's Together II/2134.cpp

```
class Solution {
public:
    int minSwaps(vector<int>& nums) {
        const int n = nums.size();
        const int k = ranges::count(nums, 1);
        int ones = 0; // the number of ones in the window
        int maxOnes = 0; // the maximum number of ones in the window

        for (int i = 0; i < n * 2; ++i) {
            if (i >= k && nums[(i - k) % n])
                --ones;
            if (nums[i % n])
                ++ones;
            maxOnes = max(maxOnes, ones);
        }

        return k - maxOnes;
    }
};
```

1215. 2135.cpp

Path: LeetCode-main/solutions/2135. Count Words Obtained After Adding a Letter/2135.cpp

```
class Solution {
public:
    int wordCount(vector<string>& startWords, vector<string>& targetWords) {
        int ans = 0;
        unordered_set<int> seen;

        for (const string& w : startWords)
            seen.insert(getMask(w));

        for (const string& w : targetWords) {
            const int mask = getMask(w);
            for (const char c : w)
                // Toggle one character.
                if (seen.contains(mask ^ 1 << c - 'a')) {
                    ++ans;
                    break;
                }
        }

        return ans;
    }

private:
    int getMask(const string& s) {
        int mask = 0;
        for (const char c : s)
            mask ^= 1 << c - 'a';
        return mask;
    }
};
```

1216. 2136.cpp

Path: LeetCode-main/solutions/2136. Earliest Possible Day of Full Bloom/2136.cpp

```
struct Seed {
    int p;
    int g;
};

class Solution {
public:
    int earliestFullBloom(vector<int>& plantTime, vector<int>& growTime) {
        int ans = 0;
        int time = 0;
        vector<Seed> seeds;

        for (int i = 0; i < plantTime.size(); ++i)
            seeds.emplace_back(plantTime[i], growTime[i]);

        ranges::sort(seeds, ranges::greater{},
                    [](const Seed& seed) { return seed.g; });

        for (const auto& [p, g] : seeds) {
            time += p;
            ans = max(ans, time + g);
        }

        return ans;
    }
};
```

1217. 2137.cpp

Path: LeetCode-main/solutions/2137. Pour Water Between Buckets to Make Water Levels Equal/2137.cpp

```
class Solution {
public:
    double equalizeWater(vector<int>& buckets, int loss) {
        constexpr double kErr = 1e-5;
        const double kPercentage = (100 - loss) / (double)100;
        double l = 0.0;
        double r = ranges::max(buckets);

        while (r - l > kErr) {
            const double m = (l + r) / 2;
            if (canFill(buckets, m, kPercentage))
                l = m;
            else
                r = m;
        }

        return l;
    }

private:
    bool canFill(const vector<int>& buckets, double target, double kPercentage) {
        double extra = 0.0;
        double need = 0.0;
        for (const int bucket : buckets)
            if (bucket > target)
                extra += bucket - target;
            else
                need += target - bucket;
        return extra * kPercentage >= need;
    }
};
```

1218. 2138.cpp

Path: LeetCode-main/solutions/2138. Divide a String Into Groups of Size k/2138.cpp

```
class Solution {
public:
    vector<string> divideString(string s, int k, char fill) {
        vector<string> ans;

        for (int i = 0; i < s.length(); i += k)
            ans.push_back(i + k > s.length()
                          ? s.substr(i) + string(i + k - s.length(), fill)
                          : s.substr(i, k));

        return ans;
    }
};
```

1219. 2139.cpp

Path: LeetCode-main/solutions/2139. Minimum Moves to Reach Target Score/2139.cpp

```
class Solution {
public:
    int minMoves(int target, int maxDoubles) {
        int steps = 0;

        while (target > 1 && maxDoubles) {
            if (target % 2 == 1) {
                --target;
            } else {
                target /= 2;
                --maxDoubles;
            }
            ++steps;
        }
        return steps + target - 1;
    }
};
```

1220. 214.cpp

Path: LeetCode-main/solutions/214. Shortest Palindrome/214.cpp

```
class Solution {
public:
    string shortestPalindrome(string s) {
        const string t = {s.rbegin(), s.rend()};
        const string_view sv_s(s);
        const string_view sv_t(t);

        for (int i = 0; i < s.length(); ++i)
            if (sv_s.substr(0, s.length() - i) == sv_t.substr(i))
                return t.substr(0, i) + s;

        return t + s;
    }
};
```


1221. 2140.cpp

Path: LeetCode-main/solutions/2140. Solving Questions With Brainpower/2140.cpp

```
class Solution {
public:
    long long mostPoints(vector<vector<int>>& questions) {
        const int n = questions.size();
        // dp[i] := the maximum points starting from questions[i]
        vector<long> dp(n + 1);

        for (int i = n - 1; i >= 0; --i) {
            const int points = questions[i][0];
            const int brainpower = questions[i][1];
            const int nextIndex = i + brainpower + 1;
            const long nextPoints = nextIndex < n ? dp[nextIndex] : 0;
            dp[i] = max(points + nextPoints, dp[i + 1]);
        }

        return dp[0];
    }
};
```

1222. 2141.cpp

Path: LeetCode-main/solutions/2141. Maximum Running Time of N Computers/2141.cpp

```
class Solution {
public:
    long long maxRunTime(int n, vector<int>& batteries) {
        long sum = accumulate(batteries.begin(), batteries.end(), 0L);

        ranges::sort(batteries);

        // The maximum battery is greater than the average, so it can last forever.
        // Reduce the problem from size n to size n - 1.
        while (batteries.back() > sum / n) {
            sum -= batteries.back(), batteries.pop_back();
            --n;
        }

        // If the maximum battery <= average running time, it won't be waste, and so
        // do smaller batteries.
        return sum / n;
    }
};
```

1223. 2143.cpp

Path: LeetCode-main/solutions/2143. Choose Numbers From Two Arrays in Range/2143.cpp

```
class Solution {
public:
    int countSubranges(vector<int>& nums1, vector<int>& nums2) {
        constexpr int kMod = 1'000'000'007;
        int ans = 0;
        // {sum, count}, add if choose from nums1, minus if choose from nums2
        unordered_map<int, int> dp;

        for (int i = 0; i < nums1.size(); ++i) {
            // edge case: nums1[i] == nums2[i] == 0, so can't put them in the
            // initializer list.
            unordered_map<int, int> newDp;
            ++newDp[nums1[i]];
            ++newDp[-nums2[i]];

            for (const auto& [prevSum, count] : dp) {
                // Choose nums1[i].
                newDp[prevSum + nums1[i]] += count;
                newDp[prevSum + nums1[i]] %= kMod;
                // Choose nums2[i].
                newDp[prevSum - nums2[i]] += count;
                newDp[prevSum - nums2[i]] %= kMod;
            }

            dp = std::move(newDp);
            if (const auto it = dp.find(0); it != dp.cend()) {
                ans += it->second;
                ans %= kMod;
            }
        }

        return ans;
    }
};
```

1224. 2144.cpp

Path: LeetCode-main/solutions/2144. Minimum Cost of Buying Candies With Discount/2144.cpp

```
class Solution {
public:
    int minimumCost(vector<int>& cost) {
        int ans = 0;

        ranges::sort(cost, greater<>());

        for (int i = 0; i < cost.size(); ++i)
            if (i % 3 != 2)
                ans += cost[i];

        return ans;
    }
};
```

1225. 2145-2.cpp

Path: LeetCode-main/solutions/2145. Count the Hidden Sequences/2145-2.cpp

```
class Solution {
public:
    int numberOfArrays(vector<int>& differences, int lower, int upper) {
        long prefix = 0;
        long mn = 0; // Starts from 0.
        long mx = 0; // Starts from 0.

        for (const int d : differences) {
            prefix += d;
            mn = min(mn, prefix);
            mx = max(mx, prefix);
        }

        return max(0L, (upper - lower) - (mx - mn) + 1);
    }
};
```

1226. 2145.cpp

Path: LeetCode-main/solutions/2145. Count the Hidden Sequences/2145.cpp

```
class Solution {
public:
    int numberOfArrays(vector<int>& differences, int lower, int upper) {
        // Starts from 0, so prefix[0] = 0.
        // Changing prefix[0] to any other number doesn't affect the answer.
        vector<long> prefix(differences.size() + 1);

        for (int i = 0; i < differences.size(); ++i)
            prefix[i + 1] += prefix[i] + differences[i];

        const long mx = ranges::max(prefix);
        const long mn = ranges::min(prefix);
        return max(0L, (upper - lower) - (mx - mn) + 1);
    }
};
```

1227. 2146.cpp

Path: LeetCode-main/solutions/2146. K Highest Ranked Items Within a Price Range/2146.cpp

```
class Solution {
public:
    vector<vector<int>> highestRankedKItems(vector<vector<int>>& grid,
                                           vector<int>& pricing,
                                           vector<int>& start, int k) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = grid.size();
        const int n = grid[0].size();
        const int low = pricing[0];
        const int high = pricing[1];
        const int row = start[0];
        const int col = start[1];
        vector<vector<int>> ans;

        if (low <= grid[row][col] && grid[row][col] <= high) {
            ans.push_back({row, col});
            if (k == 1)
                return ans;
        }

        queue<pair<int, int>> q{{{row, col}}};
        vector<vector<bool>> seen(m, vector<bool>(n));
        seen[row][col] = true; // Mark as visited.

        while (!q.empty()) {
            vector<vector<int>> neighbors;
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (!grid[x][y] || seen[x][y])
                        continue;
                    if (low <= grid[x][y] && grid[x][y] <= high)
                        neighbors.push_back({x, y});
                    q.emplace(x, y);
                    seen[x][y] = true;
                }
            }
            ranges::sort(neighbors, [&](const vector<int>& a, const vector<int>& b) {
                const int x1 = a[0];
                const int y1 = a[1];
                const int x2 = b[0];
                const int y2 = b[1];
                if (grid[x1][y1] != grid[x2][y2])
                    return grid[x1][y1] < grid[x2][y2];
                return x1 == x2 ? y1 < y2 : x1 < x2;
            });
            for (const vector<int>& neighbor : neighbors) {
                if (ans.size() < k)
                    ans.push_back(neighbor);
                if (ans.size() == k)
                    return ans;
            }
        }

        return ans;
    }
};
```

1228. 2147.cpp

Path: LeetCode-main/solutions/2147. Number of Ways to Divide a Long Corridor/2147.cpp

```
class Solution {
public:
    int numberOfWays(string corridor) {
        constexpr int kMod = 1'000'000'007;
        long ans = 1;
        int prevSeat = -1;
        int numSeats = 0;

        for (int i = 0; i < corridor.length(); ++i) {
            if (corridor[i] == 'S') {
                if (++numSeats > 2 && numSeats % 2 == 1)
                    ans = ans * (i - prevSeat) % kMod;
                prevSeat = i;
            }
        }

        return numSeats > 1 && numSeats % 2 == 0 ? ans : 0;
    }
};
```


1229. 2148.cpp

Path: LeetCode-main/solutions/2148. Count Elements With Strictly Smaller and Greater Elements/2148.cpp

```
class Solution {
public:
    int countElements(vector<int>& nums) {
        const int mn = ranges::min(nums);
        const int mx = ranges::max(nums);
        return ranges::count_if(
            nums, [&](const int num) { return mn < num && num < mx; });
    }
};
```

1230. 2149.cpp

Path: LeetCode-main/solutions/2149. Rearrange Array Elements by Sign/2149.cpp

```
class Solution {
public:
    vector<int> rearrangeArray(vector<int>& nums) {
        vector<int> ans;
        vector<int> pos;
        vector<int> neg;

        for (const int num : nums)
            (num > 0 ? pos : neg).push_back(num);

        for (int i = 0; i < pos.size(); ++i) {
            ans.push_back(pos[i]);
            ans.push_back(neg[i]);
        }

        return ans;
    }
};
```

1231. 215-2.cpp

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215-2.cpp

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        return quickSelect(nums, 0, nums.size() - 1, k);
    }

private:
    int quickSelect(vector<int>& nums, int l, int r, int k) {
        const int pivot = nums[r];

        int nextSwapped = l;
        for (int i = l; i < r; ++i)
            if (nums[i] >= pivot)
                swap(nums[nextSwapped++], nums[i]);
        swap(nums[nextSwapped], nums[r]);

        const int count = nextSwapped - l + 1; // the number of `nums` >= pivot
        if (count == k)
            return nums[nextSwapped];
        if (count > k)
            return quickSelect(nums, l, nextSwapped - 1, k);
        return quickSelect(nums, nextSwapped + 1, r, k - count);
    }
};
```

1232. 215-3.cpp

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215-3.cpp

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        return quickSelect(nums, 0, nums.size() - 1, k);
    }

private:
    int quickSelect(vector<int>& nums, int l, int r, int k) {
        const int randIndex = rand() % (r - l + 1) + l;
        swap(nums[randIndex], nums[r]);
        const int pivot = nums[r];

        int nextSwapped = l;
        for (int i = l; i < r; ++i)
            if (nums[i] >= pivot)
                swap(nums[nextSwapped++], nums[i]);
        swap(nums[nextSwapped], nums[r]);

        const int count = nextSwapped - l + 1; // the number of `nums` >= pivot
        if (count == k)
            return nums[nextSwapped];
        if (count > k)
            return quickSelect(nums, l, nextSwapped - 1, k);
        return quickSelect(nums, nextSwapped + 1, r, k - count);
    }
};
```

1233. 215.cpp

Path: LeetCode-main/solutions/215. Kth Largest Element in an Array/215.cpp

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const int num : nums) {
            minHeap.push(num);
            if (minHeap.size() > k)
                minHeap.pop();
        }

        return minHeap.top();
    }
};
```

1234. 2150.cpp

Path: LeetCode-main/solutions/2150. Find All Lonely Numbers in the Array/2150.cpp

```
class Solution {
public:
    vector<int> findLonely(vector<int>& nums) {
        vector<int> ans;
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        for (const auto& [num, freq] : count)
            if (freq == 1 && !count.contains(num - 1) && !count.contains(num + 1))
                ans.push_back(num);

        return ans;
    }
};
```

1235. 2151-2.cpp

Path: LeetCode-main/solutions/2151. Maximum Good People Based on Statements/2151-2.cpp

```
class Solution {
public:
    int maximumGood(vector<vector<int>>& statements) {
        const int maxMask = 1 << statements.size();
        int ans = 0;

        for (unsigned mask = 0; mask < maxMask; ++mask)
            if (isValid(statements, mask))
                ans = max(ans, popcount(mask));

        return ans;
    }

private:
    bool isValid(const vector<vector<int>>& statements, int mask) {
        for (int i = 0; i < statements.size(); ++i) {
            if ((mask >> i & 1) == 0) // The i-th person is bad, so no need to check.
                continue;
            for (int j = 0; j < statements.size(); ++j) {
                if (statements[i][j] == 2)
                    continue;
                if (statements[i][j] != (mask >> j & 1))
                    return false;
            }
        }
        return true;
    }
};
```

1236. 2151.cpp

Path: LeetCode-main/solutions/2151. Maximum Good People Based on Statements/2151.cpp

```
class Solution {
public:
    int maximumGood(vector<vector<int>>& statements) {
        int ans = 0;
        dfs(statements, {}, 0, 0, ans);
        return ans;
    }

private:
    void dfs(const vector<vector<int>>& statements, vector<int>&& good, int i,
            int count, int& ans) {
        if (i == statements.size()) {
            if (isValid(statements, good))
                ans = max(ans, count);
            return;
        }

        good.push_back(0); // Assume the i-th person is bad.
        dfs(statements, std::move(good), i + 1, count, ans);
        good.back() = 1; // Assume the i-th person is good.
        dfs(statements, std::move(good), i + 1, count + 1, ans);
        good.pop_back();
    }

    bool isValid(const vector<vector<int>>& statements, const vector<int>& good) {
        for (int i = 0; i < good.size(); ++i) {
            if (!good[i]) // The i-th person is bad, so no need to check.
                continue;
            for (int j = 0; j < statements.size(); ++j) {
                if (statements[i][j] == 2)
                    continue;
                if (statements[i][j] != good[j])
                    return false;
            }
        }
        return true;
    }
};
```


1237. 2152.cpp

Path: LeetCode-main/solutions/2152. Minimum Number of Lines to Cover Points/2152.cpp

```
class Solution {
public:
    int minimumLines(vector<vector<int>>& points) {
        const int allCovered = (1 << points.size()) - 1;
        return dfs(points, 0, allCovered, vector<int>(allCovered, -1));
    }

private:
    int dfs(const vector<vector<int>>& points, int covered, int allCovered,
            vector<int>&& mem) {
        if (covered == allCovered)
            return 0;
        if (mem[covered] != -1)
            return mem[covered];

        const int n = points.size();
        int ans = n / 2 + (n & 1);

        for (int i = 0; i < n; ++i) {
            if (covered >> i & 1)
                continue;
            for (int j = 0; j < n; ++j) {
                if (i == j)
                    continue;
                // Connect the points[i] with the points[j].
                int newCovered = covered | 1 << i | 1 << j;
                // Mark the points covered by this line.
                const pair<int, int> slope = getSlope(points[i], points[j]);
                for (int k = 0; k < n; ++k)
                    if (getSlope(points[i], points[k]) == slope)
                        newCovered |= 1 << k;
                ans = min(ans, 1 + dfs(points, newCovered, allCovered, std::move(mem)));
            }
        }

        return mem[covered] = ans;
    }

    pair<int, int> getSlope(const vector<int>& p, const vector<int>& q) {
        const int dx = p[0] - q[0];
        const int dy = p[1] - q[1];
        if (dx == 0)
            return {0, p[0]};
        if (dy == 0)
            return {p[1], 0};
        const int d = __gcd(dx, dy);
        const int x = dx / d;
        const int y = dy / d;
        if (x > 0)
            return {x, y};
        return {-x, -y};
    }
};
```

1238. 2155-2.cpp

Path: LeetCode-main/solutions/2154. Keep Multiplying Found Values by Two/2155-2.cpp

```
class Solution {
public:
    int findFinalValue(vector<int>& nums, int original) {
        vector<bool> seen(1001);

        for (const int num : nums)
            seen[num] = true;

        while (original < 1001 && seen[original])
            original *= 2;

        return original;
    }
};
```

1239. 2155.cpp

Path: LeetCode-main/solutions/2154. Keep Multiplying Found Values by Two/2155.cpp

```
class Solution {
public:
    int findFinalValue(vector<int>& nums, int original) {
        unordered_set<int> numsSet{nums.begin(), nums.end()};
        while (numsSet.contains(original))
            original *= 2;
        return original;
    }
};
```

1240. 2155.cpp

Path: LeetCode-main/solutions/2155. All Divisions With the Highest Score of a Binary Array/2155.cpp

```
class Solution {
public:
    vector<int> maxScoreIndices(vector<int>& nums) {
        const int zeros = ranges::count(nums, 0);
        const int ones = nums.size() - zeros;
        vector<int> ans{0}; // the division at index 0
        int leftZeros = 0;
        int leftOnes = 0;
        int maxScore = ones; // `leftZeros` + `rightOnes`

        for (int i = 0; i < nums.size(); ++i) {
            leftZeros += nums[i] == 0;
            leftOnes += nums[i] == 1;
            const int rightOnes = ones - leftOnes;
            const int score = leftZeros + rightOnes;
            if (maxScore == score) {
                ans.push_back(i + 1);
            } else if (maxScore < score) {
                maxScore = score;
                ans = {i + 1};
            }
        }

        return ans;
    }
};
```

1241. 2156.cpp

Path: LeetCode-main/solutions/2156. Find Substring With Given Hash Value/2156.cpp

```
class Solution {
public:
    string subStrHash(string s, int power, int modulo, int k, int hashValue) {
        long maxPower = 1;
        long hash = 0;
        int bestLeft = -1;

        auto val = [](char c) -> int { return c - 'a' + 1; };

        for (int i = s.length() - 1; i >= 0; --i) {
            hash = (hash * power + val(s[i])) % modulo;
            if (i + k < s.length())
                hash = (hash - val(s[i + k]) * maxPower % modulo + modulo) % modulo;
            else
                maxPower = maxPower * power % modulo;
            if (hash == hashValue)
                bestLeft = i;
        }

        return s.substr(bestLeft, k);
    }
};
```

1242. 2157.cpp

Path: LeetCode-main/solutions/2157. Groups of Strings/2157.cpp

```
class UnionFind {
public:
    UnionFind(int n) : count(n), id(n), sz(n, 1) {
        iota(id.begin(), id.end(), 0);
    }

    void unionBySize(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (sz[i] < sz[j]) {
            sz[j] += sz[i];
            id[i] = j;
        } else {
            sz[i] += sz[j];
            id[j] = i;
        }
        --count;
    }

    int getCount() const {
        return count;
    }

    int getMaxSize() const {
        return ranges::max(sz);
    }

private:
    int count;
    vector<int> id;
    vector<int> sz;

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }
};

class Solution {
public:
    vector<int> groupStrings(vector<string>& words) {
        UnionFind uf(words.size());
        unordered_map<int, int> maskToIndex;
        unordered_map<int, int> deletedMaskToIndex;

        for (int i = 0; i < words.size(); ++i) {
            const int mask = getMask(words[i]);
            for (int j = 0; j < 26; ++j)
                if (mask >> j & 1) {
                    // Going to delete this bit.
                    const int m = mask ^ 1 << j;
                    if (const auto it = maskToIndex.find(m); it != maskToIndex.cend())
                        uf.unionBySize(i, it->second);
                    if (const auto it = deletedMaskToIndex.find(m);
                        it != deletedMaskToIndex.cend())
                        uf.unionBySize(i, it->second);
                    else
                        deletedMaskToIndex[m] = i;
                } else {
                    // Going to add this bit.
                    const int m = mask | 1 << j;
                    if (const auto it = maskToIndex.find(m); it != maskToIndex.cend())
                        uf.unionBySize(i, it->second);
                }
            maskToIndex[mask] = i;
        }

        return {uf.getCount(), uf.getMaxSize()};
    }

private:
    int getMask(const string& s) {
        int mask = 0;
        for (const char c : s)
            mask |= 1 << c - 'a';
        return mask;
    }
};
```

1243. 2158-2.cpp

Path: LeetCode-main/solutions/2158. Amount of New Area Painted Each Day/2158-2.cpp

```
class Solution {
public:
    vector<int> amountPainted(vector<vector<int>>& paint) {
        map<int, int> intervals;
        vector<int> result;

        for (const vector<int>& p : paint) {
            int eraseLen = 0;
            auto it = intervals.lower_bound(p[0]);
            if (it != intervals.begin() && prev(it)->second >= p[0])
                --it;
            while (it != intervals.end() && p[1] >= it->first) {
                eraseLen += it->second - it->first;
                p[0] = min(p[0], it->first);
                p[1] = max(p[1], it->second);
                it = intervals.erase(it);
            }
            intervals.insert({p[0], p[1]});
            result.push_back(p[1] - p[0] - eraseLen);
        }

        return result;
    }
};
```

1244. 2158.cpp

Path: LeetCode-main/solutions/2158. Amount of New Area Painted Each Day/2158.cpp

```
enum class Type { kEntering, kLeaving };

struct Event {
    int day;
    int index;
    Type type;
    Event(int day, int index, Type type) : day(day), index(index), type(type) {}
};

class Solution {
public:
    vector<int> amountPainted(vector<vector<int>>& paint) {
        const int n = paint.size();
        const int minDay = (*ranges::min_element(
            paint, ranges::less{}, [](const vector<int>& p) { return p[0]; }))[0];
        const int maxDay = (*ranges::max_element(
            paint, ranges::less{}, [](const vector<int>& p) { return p[1]; }))[1];
        vector<int> ans(n);
        vector<Event> events;
        // Stores the indices of paints that are available now.
        set<int> runningIndices;

        for (int i = 0; i < n; ++i) {
            const int start = paint[i][0];
            const int end = paint[i][1];
            events.emplace_back(start, i, Type::kEntering); // 1 := entering
            events.emplace_back(end, i, Type::kLeaving);    // -1 := leaving
        }

        ranges::sort(events,
            [](const auto& a, const auto& b) { return a.day < b.day; });

        int i = 0; // events' index
        for (int day = minDay; day < maxDay; ++day) {
            while (i < events.size() && events[i].day == day) {
                if (events[i].type == Type::kEntering)
                    runningIndices.insert(events[i].index);
                else
                    runningIndices.erase(events[i].index);
                ++i;
            }
            if (!runningIndices.empty())
                ++ans[*runningIndices.begin()];
        }

        return ans;
    }
};
```


1245. 216.cpp

Path: LeetCode-main/solutions/216. Combination Sum III/216.cpp

```
class Solution {
public:
    vector<vector<int>> combinationSum3(int k, int n) {
        vector<vector<int>> ans;
        dfs(k, n, 1, {}, ans);
        return ans;
    }

private:
    void dfs(int k, int n, int s, vector<int>&& path, vector<vector<int>>& ans) {
        if (k == 0 && n == 0) {
            ans.push_back(path);
            return;
        }
        if (k == 0 || n <= 0)
            return;

        for (int i = s; i <= 9; ++i) {
            path.push_back(i);
            dfs(k - 1, n - i, i + 1, std::move(path), ans);
            path.pop_back();
        }
    }
};
```

1246. 2160.cpp

Path: LeetCode-main/solutions/2160. Minimum Sum of Four Digit Number After Splitting Digits/2160.cpp

```
class Solution {
public:
    int minimumSum(int num) {
        string s = to_string(num);
        ranges::sort(s);
        return stoi(s.substr(0, 1) + s.substr(2, 1)) +
            stoi(s.substr(1, 1) + s.substr(3, 1));
    }
};
```

1247. 2161.cpp

Path: LeetCode-main/solutions/2161. Partition Array According to Given Pivot/2161.cpp

```
class Solution {
public:
    vector<int> pivotArray(vector<int>& nums, int pivot) {
        vector<int> ans;

        for (const int num : nums)
            if (num < pivot)
                ans.push_back(num);

        for (const int num : nums)
            if (num == pivot)
                ans.push_back(num);

        for (const int num : nums)
            if (num > pivot)
                ans.push_back(num);

        return ans;
    }
};
```

1248. 2162.cpp

Path: LeetCode-main/solutions/2162. Minimum Cost to Set Cooking Time/2162.cpp

```
class Solution {
public:
    int minCostSetTime(int startAt, int moveCost, int pushCost,
                      int targetSeconds) {
        int ans = INT_MAX;
        int mins = targetSeconds > 5999 ? 99 : targetSeconds / 60;
        int secs = targetSeconds - mins * 60;

        auto getCost = [&](int mins, int secs) -> int {
            int cost = 0;
            char curr = '0' + startAt;
            for (const char c : to_string(mins * 100 + secs))
                if (c == curr) {
                    cost += pushCost;
                } else {
                    cost += moveCost + pushCost;
                    curr = c;
                }
            return cost;
        };

        while (secs < 100) {
            ans = min(ans, getCost(mins, secs));
            --mins;
            secs += 60;
        }

        return ans;
    }
};
```

1249. 2163.cpp

Path: LeetCode-main/solutions/2163. Minimum Difference in Sums After Removal of Elements/2163.cpp

```
class Solution {
public:
    long long minimumDifference(vector<int>& nums) {
        const int n = nums.size() / 3;
        long ans = LONG_MAX;
        long leftSum = 0;
        long rightSum = 0;
        // The left part should be as small as possible.
        priority_queue<int> maxHeap;
        // The right part should be as big as possible.
        priority_queue<int, vector<int>, greater<>> minHeap;
        // minLeftSum[i] := the minimum of the sum of n nums in nums[0..i]
        vector<long> minLeftSum(nums.size());

        for (int i = 0; i < 2 * n; ++i) {
            maxHeap.push(nums[i]);
            leftSum += nums[i];
            if (maxHeap.size() == n + 1)
                leftSum -= maxHeap.top(), maxHeap.pop();
            if (maxHeap.size() == n)
                minLeftSum[i] = leftSum;
        }

        for (int i = nums.size() - 1; i >= n; --i) {
            minHeap.push(nums[i]);
            rightSum += nums[i];
            if (minHeap.size() == n + 1)
                rightSum -= minHeap.top(), minHeap.pop();
            if (minHeap.size() == n)
                ans = min(ans, minLeftSum[i - 1] - rightSum);
        }

        return ans;
    }
};
```

1250. 2164.cpp

Path: LeetCode-main/solutions/2164. Sort Even and Odd Indices Independently/2164.cpp

```
class Solution {
public:
    vector<int> sortEvenOdd(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans(n);
        vector<int> evenCount(101);
        vector<int> oddCount(101);

        for (int i = 0; i < n; ++i)
            if (i % 2 == 1)
                ++oddCount[nums[i]];
            else
                ++evenCount[nums[i]];

        int ansIndex = 0;
        for (int i = 1; i < 101; ++i)
            while (evenCount[i]--) {
                ans[ansIndex] = i;
                ansIndex += 2;
            }

        ansIndex = 1;
        for (int i = 100; i > 0; --i)
            while (oddCount[i]--) {
                ans[ansIndex] = i;
                ansIndex += 2;
            }

        return ans;
    }
};
```

1251. 2165.cpp

Path: LeetCode-main/solutions/2165. Smallest Value of the Rearranged Number/2165.cpp

```
class Solution {
public:
    long long smallestNumber(long long num) {
        string s = to_string(abs(num));
        ranges::sort(s, [&](int a, int b) { return num < 0 ? a > b : a < b; });
        if (num > 0)
            swap(s[0], s[s.find_first_not_of('0')]);
        return stoll(s) * (num < 0 ? -1 : 1);
    }
};
```

1252. 2166.cpp

Path: LeetCode-main/solutions/2166. Design Bitset/2166.cpp

```
class Bitset {
public:
    Bitset(int size) : s(size, '0'), r(size, '1') {}

    void fix(int idx) {
        if (s[idx] == '0')
            ++cnt;
        s[idx] = '1';
        r[idx] = '0';
    }

    void unfix(int idx) {
        if (s[idx] == '1')
            --cnt;
        s[idx] = '0';
        r[idx] = '1';
    }

    void flip() {
        swap(s, r);
        cnt = s.length() - cnt;
    }

    bool all() {
        return cnt == s.length();
    }

    bool one() {
        return cnt;
    }

    int count() {
        return cnt;
    }

    string toString() {
        return s;
    }

private:
    string s; // the original
    string r; // the reversed
    int cnt = 0;
};
```


1253. 2167-2.cpp

Path: LeetCode-main/solutions/2167. Minimum Time to Remove All Cars Containing Illegal Goods/2167-2.cpp

```
class Solution {
public:
    int minimumTime(string s) {
        const int n = s.length();
        int ans = n;
        int left = 0; // the minimum time to remove the illegal cars so far

        for (int i = 0; i < n; ++i) {
            left = min(left + (s[i] - '0') * 2, i + 1);
            ans = min(ans, left + n - 1 - i);
        }

        return ans;
    }
};
```

1254. 2167.cpp

Path: LeetCode-main/solutions/2167. Minimum Time to Remove All Cars Containing Illegal Goods/2167.cpp

```
class Solution {
public:
    int minimumTime(string s) {
        const int n = s.length();
        // left[i] := the minimum time to remove the illegal cars of s[0..i]
        vector<int> left(n);
        left[0] = s[0] - '0';
        // dp[i] := the minimum time to remove the illegal cars of s[0..i] optimally
        // + the time to remove the illegal cars of s[i + 1..n) consecutively
        // Note that the way to remove the illegal cars in the right part
        // doesn't need to be optimal since:
        //   `left | illegal cars | n - 1 - k` will be covered in
        //   `left' | n - 1 - i` later.
        vector<int> dp(n, n);
        dp[0] = left[0] + n - 1;

        for (int i = 1; i < n; ++i) {
            left[i] = min(left[i - 1] + (s[i] - '0') * 2, i + 1);
            dp[i] = min(dp[i], left[i] + n - 1 - i);
        }

        return ranges::min(dp);
    }
};
```

1255. 2168.cpp

Path: LeetCode-main/solutions/2168. Unique Substrings With Equal Digit Frequency/2168.cpp

```
class Solution {
public:
    int equalDigitFrequency(string s) {
        vector<vector<int>> counts; // counts[i] := the counter map of s[0..i]
        vector<int> count(10);
        vector<long> pows{1}; // pows[i] := kBase^i % kHash
        // hash[i] = the hash of the first i letters of s, where hash[i] =
        // (26^(i - 1) * s[0] + 26^(i - 2) * s[1] + ... + s[i - 1]) % kHash
        vector<long> hash{0};
        unordered_set<int> seen;

        for (const char c : s) {
            ++count[c - '0'];
            counts.push_back(count);
            pows.push_back(pows.back() * kBase % kHash);
            hash.push_back((hash.back() * kBase + val(c)) % kHash);
        }

        for (int i = 0; i < s.length(); ++i)
            for (int j = i; j < s.length(); ++j)
                if (isSameFreq(counts, i, j))
                    seen.insert(getRollingHash(i, j + 1, hash, pows));

        return seen.size();
    }

private:
    static constexpr int kMax = 1001;
    static constexpr int kBase = 11;
    static constexpr int kHash = 1'000'000'007;

    static constexpr int val(char c) {
        return c - '0' + 1;
    }

    // Returns true if s[i..j] has the same digit frequency.
    bool isSameFreq(const vector<vector<int>>& counts, int i, int j) {
        vector<int> count = counts[j];
        if (i > 0)
            for (int num = 0; num < 10; ++num)
                count[num] -= counts[i - 1][num];
        return equalFreq(count);
    }

    bool equalFreq(const vector<int>& count) {
        int minfreq = kMax;
        int maxfreq = 0;
        for (const int freq : count)
            if (freq > 0) {
                minfreq = min(minfreq, freq);
                maxfreq = max(maxfreq, freq);
            }
        return minfreq == maxfreq;
    }

    // Returns the rolling hash of s[l..r).
    int getRollingHash(int l, int r, const vector<long>& hash,
                      const vector<long>& pows) {
        const long h = (hash[r] - hash[l] * pows[r - l]) % kHash;
        return h < 0 ? h + kHash : h;
    }
};
```

1256. 2169.cpp

Path: LeetCode-main/solutions/2169. Count Operations to Obtain Zero/2169.cpp

```
class Solution {
public:
    int countOperations(int num1, int num2) {
        int ans = 0;

        while (num1 && num2) {
            if (num1 < num2)
                swap(num1, num2);
            ans += num1 / num2;
            num1 %= num2;
        }

        return ans;
    }
};
```

1257. 217.cpp

Path: LeetCode-main/solutions/217. Contains Duplicate/217.cpp

```
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        unordered_set<int> seen;

        for (const int num : nums)
            if (!seen.insert(num).second)
                return true;

        return false;
    }
};
```

1258. 2170.cpp

Path: LeetCode-main/solutions/2170. Minimum Operations to Make the Array Alternating/2170.cpp

```
struct T {
    unordered_map<int, int> count;
    int mx = 0;
    int secondMax = 0;
    int maxFreq = 0;
    int secondMaxFreq = 0;
};

class Solution {
public:
    int minimumOperations(vector<int>& nums) {
        // 0 := odd indices, 1 := even indices
        vector<T> ts(2);

        for (int i = 0; i < nums.size(); ++i) {
            T& t = ts[i % 2];
            const int freq = ++t.count[nums[i]];
            if (freq > t.maxFreq) {
                t.maxFreq = freq;
                t.mx = nums[i];
            } else if (freq > t.secondMaxFreq) {
                t.secondMaxFreq = freq;
                t.secondMax = nums[i];
            }
        }

        if (ts[0].mx == ts[1].mx)
            return nums.size() - max(ts[0].maxFreq + ts[1].secondMaxFreq,
                                     ts[1].maxFreq + ts[0].secondMaxFreq);
        return nums.size() - (ts[0].maxFreq + ts[1].maxFreq);
    }
};
```

1259. 2171.cpp

Path: LeetCode-main/solutions/2171. Removing Minimum Number of Magic Beans/2171.cpp

```
class Solution {
public:
    long long minimumRemoval(vector<int>& beans) {
        const long n = beans.size();
        const long sum = accumulate(beans.begin(), beans.end(), 0L);
        long ans = LONG_MAX;

        ranges::sort(beans);

        for (int i = 0; i < n; ++i)
            ans = min(ans, sum - (n - i) * beans[i]);

        return ans;
    }
};
```

1260. 2172.cpp

Path: LeetCode-main/solutions/2172. Maximum AND Sum of Array/2172.cpp

```
class Solution {
public:
    int maximumANDSum(vector<int>& nums, int numSlots) {
        const int n = 2 * numSlots;
        const int nSelected = 1 << n;
        // dp[i] := the maximum value, where i is the bitmask of the selected
        // numbers
        vector<int> dp(nSelected);

        nums.resize(n);

        for (unsigned mask = 1; mask < nSelected; ++mask) {
            const int selected = popcount(mask);
            const int slot = (selected + 1) / 2; // (1, 2) -> 1, (3, 4) -> 2
            for (int i = 0; i < n; ++i)
                if (mask >> i & 1) // Assign `nums[i]` to the `slot`-th slot.
                    dp[mask] = max(dp[mask], dp[mask ^ 1 << i] + (slot & nums[i]));
        }

        return dp.back();
    }
};
```


1261. 2174-2.cpp

Path: LeetCode-main/solutions/2174. Remove All Ones With Row and Column Flips II/2174-2.cpp

```
class Solution {
public:
    int removeOnes(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        const int maxMask = 1 << m * n;
        // dp[i] := the minimum number of operations to remove all 1s from the grid,
        // where `i` is the bitmask of the state of the grid
        vector<int> dp(maxMask, INT_MAX / 2);
        dp[0] = 0;

        for (int mask = 0; mask < maxMask; ++mask)
            for (int i = 0; i < m; ++i)
                for (int j = 0; j < n; ++j)
                    if (grid[i][j] == 1) {
                        int newMask = mask;
                        // Set the cells in the same row with 0.
                        for (int k = 0; k < n; ++k)
                            newMask &= ~(1 << i * n + k);
                        // Set the cells in the same column with 0.
                        for (int k = 0; k < m; ++k)
                            newMask &= ~(1 << k * n + j);
                        dp[mask] = min(dp[mask], 1 + dp[newMask]);
                    }

        return dp[encode(grid, m, n)];
    }

private:
    int encode(const vector<vector<int>>& grid, int m, int n) {
        int encoded = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j])
                    encoded |= 1 << i * n + j;
        return encoded;
    }
};
```

1262. 2174.cpp

Path: LeetCode-main/solutions/2174. Remove All Ones With Row and Column Flips II/2174.cpp

```
class Solution {
public:
    int removeOnes(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<int> mem(1 << m * n, INT_MAX);
        return removeOnes(encode(grid, m, n), m, n, mem);
    }

private:
    // Returns the minimum number of operations to remove all 1s from the grid,
    // where `mask` is the bitmask of the state of the grid.
    int removeOnes(int mask, int m, int n, vector<int>& mem) {
        if (mask == 0)
            return 0;
        if (mem[mask] < INT_MAX)
            return mem[mask];

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (mask >> i * n + j & 1) { // grid[i][j] == 1
                    int newMask = mask;
                    // Set the cells in the same row with 0.
                    for (int k = 0; k < n; ++k)
                        newMask &= ~(1 << i * n + k);
                    // Set the cells in the same column with 0.
                    for (int k = 0; k < m; ++k)
                        newMask &= ~(1 << k * n + j);
                    mem[mask] = min(mem[mask], 1 + removeOnes(newMask, m, n, mem));
                }

        return mem[mask];
    }

    int encode(const vector<vector<int>>& grid, int m, int n) {
        int encoded = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j])
                    encoded |= 1 << i * n + j;
        return encoded;
    }
};
```

1263. 2176.cpp

Path: LeetCode-main/solutions/2176. Count Equal and Divisible Pairs in an Array/2176.cpp

```
class Solution {
public:
    int countPairs(vector<int>& nums, int k) {
        int ans = 0;
        unordered_map<int, vector<int>>> numToIndices;

        for (int i = 0; i < nums.size(); ++i)
            numToIndices[nums[i]].push_back(i);

        for (const auto& [_, indices] : numToIndices) {
            unordered_map<int, int> gcds;
            for (const int i : indices) {
                const int gcd_i = gcd(i, k);
                for (const auto& [gcd_j, count] : gcds)
                    if (gcd_i * gcd_j % k == 0)
                        ans += count;
                ++gcds[gcd_i];
            }
        }

        return ans;
    }
};
```

1264. 2177.cpp

Path: LeetCode-main/solutions/2177. Find Three Consecutive Integers That Sum to a Given Number/2177.cpp

```
class Solution {
public:
    vector<long long> sumOfThree(long long num) {
        if (num % 3)
            return {};
        const long x = num / 3;
        return {x - 1, x, x + 1};
    }
};
```

1265. 2178.cpp

Path: LeetCode-main/solutions/2178. Maximum Split of Positive Even Integers/2178.cpp

```
class Solution {
public:
    vector<long long> maximumEvenSplit(long long finalSum) {
        if (finalSum % 2 == 1)
            return {};

        vector<long long> ans;
        long needSum = finalSum;
        long even = 2;

        while (needSum - even >= even + 2) {
            ans.push_back(even);
            needSum -= even;
            even += 2;
        }

        ans.push_back(needSum);
        return ans;
    }
};
```

1266. 2179.cpp

Path: LeetCode-main/solutions/2179. Count Good Triplets in an Array/2179.cpp

```
class FenwickTree {
public:
    FenwickTree(int n) : sums(n + 1) {}

    void add(int i, int delta) {
        while (i < sums.size()) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    int get(int i) const {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

private:
    vector<int> sums;

    static inline int lowbit(int i) {
        return i & -i;
    }
};

class Solution {
public:
    long long goodTriplets(vector<int>& nums1, vector<int>& nums2) {
        const int n = nums1.size();
        long ans = 0;
        unordered_map<int, int> numToIndex;
        vector<int> arr;
        // leftSmaller[i] := the number of arr[j] < arr[i], where 0 <= j < i
        vector<int> leftSmaller(n);
        // rightLarger[i] := the number of arr[j] > arr[i], where i < j < n
        vector<int> rightLarger(n);
        FenwickTree tree1(n); // Calculates `leftSmaller`.
        FenwickTree tree2(n); // Calculates `rightLarger`.

        for (int i = 0; i < n; ++i)
            numToIndex[nums1[i]] = i;

        // Remap each number in `nums2` to the according index in `nums1` as `arr`.
        // So the problem is to find the number of increasing tripets in `arr`.
        for (const int num : nums2)
            arr.push_back(numToIndex[num]);

        for (int i = 0; i < n; ++i) {
            leftSmaller[i] = tree1.get(arr[i]);
            tree1.add(arr[i] + 1, 1);
        }

        for (int i = n - 1; i >= 0; --i) {
            rightLarger[i] = tree2.get(n) - tree2.get(arr[i]);
            tree2.add(arr[i] + 1, 1);
        }

        for (int i = 0; i < n; ++i)
            ans += static_cast<long>(leftSmaller[i]) * rightLarger[i];

        return ans;
    }
};
```

1267. 218-2.cpp

Path: LeetCode-main/solutions/218. The Skyline Problem/218-2.cpp

```
class Solution {
public:
    vector<vector<int>> getSkyline(vector<vector<int>>& buildings) {
        vector<vector<int>> ans;
        vector<vector<int>> events; // [(Li, Hi) | (Ri, -Hi)]

        for (const vector<int>& b : buildings) {
            events.push_back({b[0], b[2]});
            events.push_back({b[1], -b[2]}); // Minus means leaving.
        }

        ranges::sort(events, ranges::less{}, [](const vector<int>& event) {
            return pair<int, int>{event[0], -event[1]};
        });

        for (const vector<int>& event : events) {
            const int x = event[0];
            const int h = abs(event[1]);
            const int isEntering = event[1] > 0;
            if (isEntering) {
                if (h > maxHeight())
                    ans.push_back({x, h});
                set.insert(h);
            } else {
                set.erase(set.equal_range(h).first);
                if (h > maxHeight())
                    ans.push_back({x, maxHeight()});
            }
        }

        return ans;
    }

private:
    multiset<int> set;

    int maxHeight() const {
        return set.empty() ? 0 : *set.rbegin();
    }
};
```

1268. 218.cpp

Path: LeetCode-main/solutions/218. The Skyline Problem/218.cpp

```
class Solution {
public:
    vector<vector<int>> getSkyline(const vector<vector<int>>& buildings) {
        const int n = buildings.size();
        if (n == 0)
            return {};
        if (n == 1) {
            const int left = buildings[0][0];
            const int right = buildings[0][1];
            const int height = buildings[0][2];
            return {{left, height}, {right, 0}};
        }

        const vector<vector<int>> left =
            getSkyline({buildings.begin(), buildings.begin() + n / 2});
        const vector<vector<int>> right =
            getSkyline({buildings.begin() + n / 2, buildings.end()});
        return merge(left, right);
    }

private:
    vector<vector<int>> merge(const vector<vector<int>>& left,
                           const vector<vector<int>>& right) {
        vector<vector<int>> ans;
        int i = 0; // left's index
        int j = 0; // right's index
        int leftY = 0;
        int rightY = 0;

        while (i < left.size() && j < right.size())
            // Choose the point with the smaller x.
            if (left[i][0] < right[j][0]) {
                leftY = left[i][1]; // Update the ongoing `leftY`.
                addPoint(ans, left[i][0], max(left[i+1][1], rightY));
            } else {
                rightY = right[j][1]; // Update the ongoing `rightY`.
                addPoint(ans, right[j][0], max(right[j+1][1], leftY));
            }

        while (i < left.size())
            addPoint(ans, left[i][0], left[i+1][1]);

        while (j < right.size())
            addPoint(ans, right[j][0], right[j+1][1]);

        return ans;
    }

    void addPoint(vector<vector<int>>& ans, int x, int y) {
        if (!ans.empty() && ans.back()[0] == x) {
            ans.back()[1] = y;
            return;
        }
        if (!ans.empty() && ans.back()[1] == y)
            return;
        ans.push_back({x, y});
    }
};
```


1269. 2180.cpp

Path: LeetCode-main/solutions/2180. Count Integers With Even Digit Sum/2180.cpp

```
class Solution {
public:
    int countEven(int num) {
        return (num - getDigitSum(num) % 2) / 2;
    }

private:
    int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
};
```

1270. 2181-2.cpp

Path: LeetCode-main/solutions/2181. Merge Nodes in Between Zeros/2181-2.cpp

```
class Solution {
public:
    ListNode* mergeNodes(ListNode* head) {
        ListNode* curr = head->next;

        while (curr != nullptr) {
            ListNode* running = curr;
            int sum = 0;
            while (running->val > 0) {
                sum += running->val;
                running = running->next;
            }
            curr->val = sum;
            curr->next = running->next;
            curr = running->next;
        }

        return head->next;
    }
};
```

1271. 2181.cpp

Path: LeetCode-main/solutions/2181. Merge Nodes in Between Zeros/2181.cpp

```
class Solution {
public:
    ListNode* mergeNodes(ListNode* head) {
        if (head == nullptr)
            return nullptr;
        if (!head->next->val) {
            ListNode* node = new ListNode(head->val);
            node->next = mergeNodes(head->next->next);
            return node;
        }

        ListNode* next = mergeNodes(head->next);
        next->val += head->val;
        return next;
    }
};
```

1272. 2182.cpp

Path: LeetCode-main/solutions/2182. Construct String With Repeat Limit/2182.cpp

```
class Solution {
public:
    string repeatLimitedString(string s, int repeatLimit) {
        string ans;
        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        while (true) {
            const bool addOne = !ans.empty() && shouldAddOne(ans, count);
            const int i = getLargestChar(ans, count);
            if (i == -1)
                break;
            const int repeats = addOne ? 1 : min(count[i], repeatLimit);
            ans += string(repeats, 'a' + i);
            count[i] -= repeats;
        }

        return ans;
    }

private:
    bool shouldAddOne(const string& ans, const vector<int>& count) {
        for (int i = 25; i >= 0; --i)
            if (count[i])
                return ans.back() == 'a' + i;
        return false;
    }

    int getLargestChar(const string& ans, const vector<int>& count) {
        for (int i = 25; i >= 0; --i)
            if (count[i] && (ans.empty() || ans.back() != 'a' + i))
                return i;
        return -1;
    }
};
```

1273. 2183.cpp

Path: LeetCode-main/solutions/2183. Count Array Pairs Divisible by K/2183.cpp

```
class Solution {
public:
    long long countPairs(vector<int>& nums, int k) {
        long ans = 0;
        unordered_map<int, int> gcds;

        for (const int num : nums) {
            const int gcd_i = gcd(num, k);
            for (const auto& [gcd_j, count] : gcds)
                if (static_cast<long>(gcd_i) * gcd_j % k == 0)
                    ans += count;
            ++gcds[gcd_i];
        }

        return ans;
    }
};
```

1274. 2184.cpp

Path: LeetCode-main/solutions/2184. Number of Ways to Build Sturdy Brick Wall/2184.cpp

```
class Solution {
public:
    int buildWall(int height, int width, vector<int>& bricks) {
        constexpr int kMod = 1'000'000'007;
        // Stores the valid rows in bitmask.
        vector<int> rows;
        buildRows(width, bricks, 0, rows);

        const int n = rows.size();
        // dp[i] := the number of ways to build `h` height walls with rows[i] in the
        // bottom
        vector<long> dp(n, 1);
        // graph[i] := the valid neighbors of rows[i]
        vector<vector<int>> graph(n);

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (!(rows[i] & rows[j]))
                    graph[i].push_back(j);

        for (int h = 2; h <= height; ++h) {
            vector<long> newDp(n);
            for (int i = 0; i < n; ++i)
                for (const int v : graph[i]) {
                    newDp[i] += dp[v];
                    newDp[i] %= kMod;
                }
            dp = std::move(newDp);
        }

        return accumulate(dp.begin(), dp.end(), 0L) % kMod;
    }

private:
    void buildRows(int width, const vector<int>& bricks, int path,
                  vector<int>& rows) {
        for (const int brick : bricks)
            if (brick == width)
                rows.push_back(path);
            else if (brick < width) {
                const int newWidth = width - brick;
                buildRows(newWidth, bricks, path | 1 << newWidth, rows);
            }
    }
};
```

1275. 2185.cpp

Path: LeetCode-main/solutions/2185. Counting Words With a Given Prefix/2185.cpp

```
class Solution {
public:
    int prefixCount(vector<string>& words, string pref) {
        return ranges::count_if(
            words, [&](const string& word) { return word.find(pref) == 0; });
    }
};
```

1276. 2186.cpp

Path: LeetCode-main/solutions/2186. Minimum Number of Steps to Make Two Strings Anagram II/2186.cpp

```
class Solution {
public:
    int minSteps(string s, string t) {
        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        for (const char c : t)
            --count[c - 'a'];

        return accumulate(count.begin(), count.end(), 0,
            [](int acc, int c) { return acc + abs(c); });
    }
};
```


1277. 2187.cpp

Path: LeetCode-main/solutions/2187. Minimum Time to Complete Trips/2187.cpp

```
class Solution {
public:
    long long minimumTime(vector<int>& time, int totalTrips) {
        long l = 1;
        long r = static_cast<long>(ranges::min(time)) * totalTrips;

        while (l < r) {
            const long m = (l + r) / 2;
            if (numTrips(time, m) >= totalTrips)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

    long numTrips(const vector<int>& times, long m) {
        return accumulate(times.begin(), times.end(), 0L,
            [&](long acc, int t) { return acc + m / t; });
    }
};
```

1278. 2188-2.cpp

Path: LeetCode-main/solutions/2188. Minimum Time to Finish the Race/2188-2.cpp

```
class Solution {
public:
    int minimumFinishTime(vector<vector<int>>& tires, int changeTime,
                          int numLaps) {
        // dp[i] := the minimum time of the first i laps
        vector<int> dp(numLaps + 1, INT_MAX);

        for (const vector<int>& t : tires)
            for (int seconds = t[0], secondsSum = t[0], lapCount = 1;
                 seconds < t[0] + changeTime && lapCount < dp.size();
                 seconds *= t[1], ++lapCount, secondsSum += seconds)
                dp[lapCount] = min(secondsSum, dp[lapCount]);

        for (int i = 2; i <= numLaps; ++i)
            for (int j = 1; j * 2 <= i; ++j)
                dp[i] = min(dp[i], dp[j] + dp[i - j] + changeTime);

        return dp[numLaps];
    }
};
```

1279. 2188.cpp

Path: LeetCode-main/solutions/2188. Minimum Time to Finish the Race/2188.cpp

```
class Solution {
public:
    int minimumFinishTime(vector<vector<int>>& tires, int changeTime,
                          int numLaps) {
        // singleTire[i] := the minimum time to finish i laps without changing tire
        vector<int> singleTire(numLaps + 1, INT_MAX / 2);
        // dp[i] := the minimum time to finish i laps
        vector<int> dp(numLaps + 1, INT_MAX / 2);

        for (int i = 0; i < tires.size(); ++i) {
            const int f = tires[i][0];
            const int r = tires[i][1];
            int sumSecs = 0;
            int rPower = 1;
            for (int j = 1; j <= numLaps; ++j) {
                // the time to use the same tire for the next lap >=
                // the time to change a new tire + f
                if ((long)f * rPower >= changeTime + f)
                    break;
                sumSecs += f * rPower;
                rPower *= r;
                singleTire[j] = min(singleTire[j], sumSecs);
            }
        }

        dp[0] = 0;
        for (int i = 1; i <= numLaps; ++i)
            for (int j = 1; j <= i; ++j)
                dp[i] = min(dp[i], dp[i - j] + changeTime + singleTire[j]);

        return dp[numLaps] - changeTime;
    }
};
```

1280. 2189.cpp

Path: LeetCode-main/solutions/2189. Number of Ways to Build House of Cards/2189.cpp

```
class Solution {
public:
    int houseOfCards(int n) {
        // dp[i] := the number of valid result for i cards
        vector<int> dp(n + 1);
        dp[0] = 1;

        for (int baseCards = 2; baseCards <= n; baseCards += 3)
            for (int i = n; i >= baseCards; --i)
                // Use `baseCards` as the base, so we're left with `i - baseCards`
                // cards.
                dp[i] += dp[i - baseCards];

        return dp[n];
    }
};
```

1281. 219.cpp

Path: LeetCode-main/solutions/219. Contains Duplicate II/219.cpp

```
class Solution {
public:
    bool containsNearbyDuplicate(vector<int>& nums, int k) {
        unordered_set<int> seen;

        for (int i = 0; i < nums.size(); ++i) {
            if (!seen.insert(nums[i]).second)
                return true;
            if (i >= k)
                seen.erase(nums[i - k]);
        }

        return false;
    }
};
```

1282. 2190.cpp

Path: LeetCode-main/solutions/2190. Most Frequent Number Following Key In an Array/2190.cpp

```
class Solution {
public:
    int mostFrequent(vector<int>& nums, int key) {
        vector<int> count(1001);

        for (int i = 0; i + 1 < nums.size(); ++i)
            if (nums[i] == key)
                ++count[nums[i + 1]];

        return ranges::max_element(count) - count.begin();
    }
};
```

1283. 2191-2.cpp

Path: LeetCode-main/solutions/2191. Sort the Jumbled Numbers/2191-2.cpp

```
class Solution {
public:
    vector<int> sortJumbled(vector<int>& mapping, vector<int>& nums) {
        vector<int> ans;
        map<int, vector<int>> mappedToOriginalNums;

        for (const int num : nums)
            mappedToOriginalNums[getMapped(num, mapping)].push_back(num);

        for (const auto& [_, originalNums] : mappedToOriginalNums)
            ans.insert(ans.end(), originalNums.begin(), originalNums.end());

        return ans;
    }

private:
    int getMapped(int num, const vector<int>& mapping) {
        string mapped;
        for (const char c : to_string(num))
            mapped += to_string(mapping[c - '0']);
        return stoi(mapped);
    }
};
```

1284. 2191.cpp

Path: LeetCode-main/solutions/2191. Sort the Jumbled Numbers/2191.cpp

```
class Solution {
public:
    vector<int> sortJumbled(vector<int>& mapping, vector<int>& nums) {
        vector<int> ans;
        vector<tuple<int, int, int>> A; // (mapped, index, num)

        for (int i = 0; i < nums.size(); ++i)
            A.emplace_back(getMapped(nums[i], mapping), i, nums[i]);

        ranges::sort(A);

        for (const auto& [_, i, num] : A)
            ans.push_back(num);

        return ans;
    }

private:
    int getMapped(int num, const vector<int>& mapping) {
        string mapped;
        for (const char c : to_string(num))
            mapped += to_string(mapping[c - '0']);
        return stoi(mapped);
    }
};
```


1285. 2192-2.cpp

Path: LeetCode-main/solutions/2192. All Ancestors of a Node in a Directed Acyclic Graph/2192-2.cpp

```
class Solution {
public:
    vector<vector<int>> getAncestors(int n, vector<vector<int>>& edges) {
        vector<vector<int>> ans;
        vector<vector<int>> graph(n);
        vector<set<int>> ancestors(n); // {u: {ancestors}}
        vector<int> inDegrees(n);
        queue<int> q;

        // Build the graph.
        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        while (!q.empty())
            for (int sz = q.size(); sz > 0; --sz) {
                const int u = q.front();
                q.pop();
                for (const int v : graph[u]) {
                    ancestors[v].insert(u);
                    std::copy(ancestors[u].begin(), ancestors[u].end(),
                            std::inserter(ancestors[v], ancestors[v].end()));
                    if (--inDegrees[v] == 0)
                        q.push(v);
                }
            }

        for (const set<int>& nodes : ancestors)
            ans.push_back({nodes.begin(), nodes.end()});

        return ans;
    }
};
```

1286. 2192.cpp

Path: LeetCode-main/solutions/2192. All Ancestors of a Node in a Directed Acyclic Graph/2192.cpp

```
class Solution {
public:
    vector<vector<int>> getAncestors(int n, vector<vector<int>>& edges) {
        vector<vector<int>> ans(n);
        vector<vector<int>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
        }

        for (int i = 0; i < n; ++i)
            dfs(graph, i, i, vector<bool>(n), ans);

        return ans;
    }

private:
    void dfs(const vector<vector<int>>& graph, int u, int ancestor,
             vector<bool>&& seen, vector<vector<int>>& ans) {
        seen[u] = true;
        for (const int v : graph[u]) {
            if (seen[v])
                continue;
            ans[v].push_back(ancestor);
            dfs(graph, v, ancestor, std::move(seen), ans);
        }
    }
};
```

1287. 2193.cpp

Path: LeetCode-main/solutions/2193. Minimum Number of Moves to Make Palindrome/2193.cpp

```
class Solution {
public:
    int minMovesToMakePalindrome(string s) {
        int ans = 0;

        while (s.length() > 1) {
            // Greedily match the last digit.
            const int i = s.find(s.back());
            if (i == s.length() - 1) {
                // s[i] is the middle letter.
                ans += i / 2;
            } else {
                s.erase(i, 1);
                ans += i; // Swap the matched letter to the left.
            }
            s.pop_back();
        }

        return ans;
    }
};
```

1288. 2194.cpp

Path: LeetCode-main/solutions/2194. Cells in a Range on an Excel Sheet/2194.cpp

```
class Solution {
public:
    vector<string> cellsInRange(string s) {
        vector<string> ans;
        const char startCol = s[0];
        const char endCol = s[3];
        const char startRow = s[1];
        const char endRow = s[4];

        for (char col = startCol; col <= endCol; ++col)
            for (char row = startRow; row <= endRow; ++row)
                ans.push_back(string(1, col) + row);

        return ans;
    }
};
```

1289. 2195.cpp

Path: LeetCode-main/solutions/2195. Append K Integers With Minimal Sum/2195.cpp

```
class Solution {
public:
    long long minimalKSum(vector<int>& nums, int k) {
        long ans = 0;
        nums.push_back(0);
        ranges::sort(nums);

        for (int i = 0; i + 1 < nums.size(); ++i) {
            if (nums[i] == nums[i + 1])
                continue;
            const int l = nums[i] + 1;
            const int r = min(nums[i] + k, nums[i + 1] - 1);
            ans += static_cast<long>(l + r) * (r - l + 1) / 2;
            k -= r - l + 1;
            if (k == 0)
                return ans;
        }

        if (k > 0) {
            const int l = nums.back() + 1;
            const int r = nums.back() + k;
            ans += static_cast<long>(l + r) * (r - l + 1) / 2;
        }

        return ans;
    }
};
```

1290. 2196.cpp

Path: LeetCode-main/solutions/2196. Create Binary Tree From Descriptions/2196.cpp

```
class Solution {
public:
    TreeNode* createBinaryTree(vector<vector<int>>& descriptions) {
        unordered_map<TreeNode*, TreeNode*> childToParent;
        unordered_map<int, TreeNode*> valToNode;

        for (const vector<int>& d : descriptions) {
            const int p = d[0];
            const int c = d[1];
            const bool isLeft = d[2];
            TreeNode* parent = valToNode.contains(p)
                ? valToNode[p]
                : (valToNode[p] = new TreeNode(p));
            TreeNode* child = valToNode.contains(c)
                ? valToNode[c]
                : (valToNode[c] = new TreeNode(c));
            childToParent[child] = parent;
            if (isLeft)
                parent->left = child;
            else
                parent->right = child;
        }

        // Pick a random node and traverse upwardly.
        TreeNode* root = childToParent.begin()->second;
        while (childToParent.contains(root))
            root = childToParent[root];
        return root;
    }
};
```

1291. 2197.cpp

Path: LeetCode-main/solutions/2197. Replace Non-Coprime Numbers in Array/2197.cpp

```
class Solution {
public:
    vector<int> replaceNonCoprimes(vector<int>& nums) {
        vector<int> ans;

        for (int num : nums) {
            while (!ans.empty() && std::gcd(ans.back(), num) > 1)
                num = std::lcm(ans.back(), num), ans.pop_back();
            ans.push_back(num);
        }

        return ans;
    }
};
```

1292. 2198.cpp

Path: LeetCode-main/solutions/2198. Number of Single Divisor Triplets/2198.cpp

```
class Solution {
public:
    long long singleDivisorTriplet(vector<int>& nums) {
        constexpr int kMax = 100;
        long ans = 0;
        vector<int> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        for (int a = 1; a <= kMax; ++a)
            for (int b = a; count[a] > 0 && b <= kMax; ++b)
                for (int c = b; count[b] > 0 && c <= kMax; ++c) {
                    const int sum = a + b + c;
                    if (divisible(sum, a) + divisible(sum, b) + divisible(sum, c) != 1)
                        continue;
                    if (a == b)
                        ans += static_cast<long>(count[a] * (count[a] - 1) / 2 * count[c]);
                    else if (b == c)
                        ans += static_cast<long>(count[b] * (count[b] - 1) / 2 * count[a]);
                    else
                        ans += static_cast<long>(count[a] * count[b] * count[c]);
                }

        return ans * 6;
    }

private:
    int divisible(int sum, int num) {
        return sum % num == 0;
    }
};
```


1293. 22.cpp

Path: LeetCode-main/solutions/22. Generate Parentheses/22.cpp

```
class Solution {
public:
    vector<string> generateParenthesis(int n) {
        vector<string> ans;
        dfs(n, n, "", ans);
        return ans;
    }

private:
    void dfs(int l, int r, string&& path, vector<string>& ans) {
        if (l == 0 && r == 0) {
            ans.push_back(path);
            return;
        }

        if (l > 0) {
            path.push_back('(');
            dfs(l - 1, r, std::move(path), ans);
            path.pop_back();
        }
        if (l < r) {
            path.push_back(')');
            dfs(l, r - 1, std::move(path), ans);
            path.pop_back();
        }
    }
};
```

1294. 220-2.cpp

Path: LeetCode-main/solutions/220. Contains Duplicate III/220-2.cpp

```
class Solution {
public:
    bool containsNearbyAlmostDuplicate(vector<int>& nums, int indexDiff,
                                      int valueDiff) {
        if (nums.empty() || indexDiff <= 0 || valueDiff < 0)
            return false;

        const long mn = ranges::min(nums);
        const long diff = valueDiff + 1L; // In case that `valueDiff` equals 0.
        // Use long because the corner case INT_MAX - (-1) will overflow.
        unordered_map<long, long> bucket;

        for (int i = 0; i < nums.size(); ++i) {
            const long num = nums[i];
            const long key = getKey(num, mn, diff);
            if (bucket.contains(key)) // the current bucket
                return true;
            if (bucket.contains(key - 1) &&
                num - bucket[key - 1] < diff) // the left adjacent bucket
                return true;
            if (bucket.contains(key + 1) &&
                bucket[key + 1] - num < diff) // the right adjacent bucket
                return true;
            bucket[key] = num;
            if (i >= indexDiff)
                bucket.erase(getKey(nums[i - indexDiff], mn, diff));
        }

        return false;
    }

private:
    int getKey(long num, long mn, long diff) {
        return (num - mn) / diff;
    }
};
```

1295. 220.cpp

Path: LeetCode-main/solutions/220. Contains Duplicate III/220.cpp

```
class Solution {
public:
    bool containsNearbyAlmostDuplicate(vector<int>& nums, int indexDiff,
                                       int valueDiff) {
        set<long> window;

        for (int i = 0; i < nums.size(); ++i) {
            if (const auto it =
                window.lower_bound(static_cast<long>(nums[i]) - valueDiff);
                it != window.cend() && *it - nums[i] <= valueDiff)
                return true;
            window.insert(nums[i]);
            if (i >= indexDiff)
                window.erase(nums[i - indexDiff]);
        }

        return false;
    }
};
```

1296. 2200.cpp

Path: LeetCode-main/solutions/2200. Find All K-Distant Indices in an Array/2200.cpp

```
class Solution {
public:
    vector<int> findKDistantIndices(vector<int>& nums, int key, int k) {
        const int n = nums.size();
        vector<int> ans;

        for (int i = 0, j = 0; i < n; ++i) {
            // the first index j s.t. nums[j] == key and j >= i - k
            while (j < n && (nums[j] != key || j < i - k))
                ++j;
            if (j == n)
                break;
            if (abs(i - j) <= k)
                ans.push_back(i);
        }

        return ans;
    }
};
```

1297. 2201.cpp

Path: LeetCode-main/solutions/2201. Count Artifacts That Can Be Extracted/2201.cpp

```
class Solution {
public:
    int digArtifacts(int n, vector<vector<int>>& artifacts,
                    vector<vector<int>>& dig) {
        unordered_set<int> digged;

        for (const vector<int>& d : dig)
            digged.insert(hash(d[0], d[1]));

        return ranges::count_if(
            artifacts, [&](const auto& a) { return canExtract(a, digged); });
    }

private:
    int hash(int i, int j) {
        return i << 16 | j;
    }

    bool canExtract(const vector<int>& a, const unordered_set<int>& digged) {
        for (int i = a[0]; i <= a[2]; ++i)
            for (int j = a[1]; j <= a[3]; ++j)
                if (!digged.contains(hash(i, j)))
                    return false;
        return true;
    }
};
```

1298. 2202.cpp

Path: LeetCode-main/solutions/2202. Maximize the Topmost Element After K Moves/2202.cpp

```
class Solution {
public:
    int maximumTop(vector<int>& nums, int k) {
        const int n = nums.size();
        // After taking k elements, if there's something left, then return nums[k].
        // Otherwise, return -1.
        if (k == 0 || k == 1)
            return n == k ? -1 : nums[k];
        // Remove then add even number of times.
        if (n == 1)
            return k % 2 == 0 ? nums[0] : -1;
        // Take min(n, k - 1) elements and put the largest one back.
        const int mx = *max_element(nums.begin(), nums.begin() + min(n, k - 1));
        if (k >= n)
            return mx;
        return max(mx, nums[k]);
    }
};
```

1299. 2203.cpp

Path: LeetCode-main/solutions/2203. Minimum Weighted Subgraph With the Required Paths/2203.cpp

```
class Solution {
public:
    long long minimumWeight(int n, vector<vector<int>>& edges, int src1, int src2,
                           int dest) {
        vector<vector<pair<int, int>>> graph(n);
        vector<vector<pair<int, int>>> reversedGraph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int w = edge[2];
            graph[u].emplace_back(v, w);
            reversedGraph[v].emplace_back(u, w);
        }

        const vector<long> fromSrc1 = dijkstra(graph, src1);
        const vector<long> fromSrc2 = dijkstra(graph, src2);
        const vector<long> fromDest = dijkstra(reversedGraph, dest);
        long ans = kMax;

        for (int i = 0; i < n; ++i) {
            if (fromSrc1[i] == kMax || fromSrc2[i] == kMax || fromDest[i] == kMax)
                continue;
            ans = min(ans, fromSrc1[i] + fromSrc2[i] + fromDest[i]);
        }

        return ans == kMax ? -1 : ans;
    }

private:
    static constexpr long kMax = 10'000'000'000;

    vector<long> dijkstra(const vector<vector<pair<int, int>>>& graph, int src) {
        vector<long> dist(graph.size(), kMax);

        dist[src] = 0;
        using P = pair<long, int>; // (d, u)
        priority_queue<P, vector<P>, greater<>> minHeap;
        minHeap.emplace(dist[src], src);

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (d > dist[u])
                continue;
            for (const auto& [v, w] : graph[u])
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.emplace(dist[v], v);
                }
        }

        return dist;
    }
};
```

1300. 2204.cpp

Path: LeetCode-main/solutions/2204. Distance to a Cycle in Undirected Graph/2204.cpp

```
class Solution {
public:
    vector<int> distanceToCycle(int n, vector<vector<int>>& edges) {
        vector<int> ans(n);
        vector<vector<int>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        // rank[i] := the minimum node that node i can reach with forward edges
        // Initialize with NO_RANK = -2 to indicate not visited.
        vector<int> cycle;
        getRank(graph, 0, 0, vector<int>(n, NO_RANK), cycle);

        queue<int> q;
        vector<bool> seen(n);
        for (const int u : cycle) {
            q.push(u);
            seen[u] = true;
        }

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const int u = q.front();
                q.pop();
                for (const int v : graph[u]) {
                    if (seen[v])
                        continue;
                    q.push(v);
                    seen[v] = true;
                    ans[v] = step;
                }
            }

        return ans;
    }

private:
    static constexpr int NO_RANK = -2;

    // The minRank that u can reach with forward edges
    int getRank(const vector<vector<int>>& graph, int u, int currRank,
                vector<int>&& rank, vector<int>& cycle) {
        if (rank[u] != NO_RANK) // The rank is already determined
            return rank[u];

        rank[u] = currRank;
        int minRank = currRank;

        for (const int v : graph[u]) {
            // Visited || parent (that's why NO_RANK = -2 instead of -1)
            if (rank[v] == rank.size() || rank[v] == currRank - 1)
                continue;
            const int nextRank =
                getRank(graph, v, currRank + 1, std::move(rank), cycle);
            // NextRank should > currRank if there's no cycle
            if (nextRank <= currRank)
                cycle.push_back(v);
            minRank = min(minRank, nextRank);
        }

        rank[u] = rank.size(); // Mark as visited.
        return minRank;
    }
};
```


1301. 2206.cpp

Path: LeetCode-main/solutions/2206. Divide Array Into Equal Pairs/2206.cpp

```
class Solution {
public:
    bool divideArray(vector<int>& nums) {
        vector<int> count(501);

        for (const int num : nums)
            ++count[num];

        return ranges::all_of(count, [](int c) { return c % 2 == 0; });
    }
};
```

1302. 2207.cpp

Path: LeetCode-main/solutions/2207. Maximize Number of Subsequences in a String/2207.cpp

```
class Solution {
public:
    long long maximumSubsequenceCount(string text, string pattern) {
        long ans = 0;
        int count0 = 0; // the count of the letter pattern[0]
        int count1 = 0; // the count of the letter pattern[1]

        for (const char c : text) {
            if (c == pattern[1]) {
                ans += count0;
                ++count1;
            }
            if (c == pattern[0])
                ++count0;
        }

        // It is optimal to add pattern[0] at the beginning or add pattern[1] at the
        // end of the text.
        return ans + max(count0, count1);
    }
};
```

1303. 2208.cpp

Path: LeetCode-main/solutions/2208. Minimum Operations to Halve Array Sum/2208.cpp

```
class Solution {
public:
    int halveArray(vector<int>& nums) {
        const double halfSum = accumulate(nums.begin(), nums.end(), 0.) / 2;
        int ans = 0;
        double runningSum = 0;
        priority_queue<double> maxHeap{nums.begin(), nums.end()};

        while (runningSum < halfSum) {
            const double maxValue = maxHeap.top() / 2;
            runningSum += maxValue, maxHeap.pop();
            maxHeap.push(maxValue);
            ++ans;
        }

        return ans;
    }
};
```

1304. 2209-2.cpp

Path: LeetCode-main/solutions/2209. Minimum White Tiles After Covering With Carpets/2209-2.cpp

```
class Solution {
public:
    int minimumWhiteTiles(string floor, int numCarpets, int carpetLen) {
        const int n = floor.length();
        // dp[i][j] := the minimum number of visible white tiles of floor[i..n)
        // after covering at most j carpets
        vector<vector<int>> dp(n + 1, vector<int>(numCarpets + 1));

        for (int i = n - 1; i >= 0; --i)
            dp[i][0] = floor[i] - '0' + dp[i + 1][0];

        for (int i = n - 1; i >= 0; --i)
            for (int j = 1; j <= numCarpets; ++j) {
                const int cover = i + carpetLen < n ? dp[i + carpetLen][j - 1] : 0;
                const int skip = floor[i] - '0' + dp[i + 1][j];
                dp[i][j] = min(cover, skip);
            }

        return dp[0][numCarpets];
    }
};
```

1305. 2209.cpp

Path: LeetCode-main/solutions/2209. Minimum White Tiles After Covering With Carpets/2209.cpp

```
class Solution {
public:
    int minimumWhiteTiles(string floor, int numCarpets, int carpetLen) {
        vector<vector<int>> mem(floor.length() + 1,
                               vector<int>(numCarpets + 1, kMax));
        return minimumWhiteTiles(floor, 0, numCarpets, carpetLen, mem);
    }

private:
    static constexpr int kMax = 1000;

    // Returns the minimum number of visible white tiles of floor[i..n) after
    // covering at most j carpets.
    int minimumWhiteTiles(const string& floor, int i, int j, int carpetLen,
                          vector<vector<int>>& mem) {
        if (j < 0)
            return kMax;
        if (i >= floor.length())
            return 0;
        if (mem[i][j] != kMax)
            return mem[i][j];
        return mem[i][j] = min(
            minimumWhiteTiles(floor, i + carpetLen, j - 1, carpetLen, mem),
            minimumWhiteTiles(floor, i + 1, j, carpetLen, mem) + floor[i] -
            '0');
    }
};
```

1306. 221-2.cpp

Path: LeetCode-main/solutions/221. Maximal Square/221-2.cpp

```
class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        const int m = matrix.size();
        const int n = matrix[0].size();
        vector<int> dp(n);
        int maxLength = 0;
        int prev = 0; // dp[i - 1][j - 1]

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                const int cache = dp[j];
                if (i == 0 || j == 0 || matrix[i][j] == '0')
                    dp[j] = matrix[i][j] == '1' ? 1 : 0;
                else
                    dp[j] = min({prev, dp[j], dp[j - 1]}) + 1;
                maxLength = max(maxLength, dp[j]);
                prev = cache;
            }

        return maxLength * maxLength;
    }
};
```

1307. 221.cpp

Path: LeetCode-main/solutions/221. Maximal Square/221.cpp

```
class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        const int m = matrix.size();
        const int n = matrix[0].size();
        vector<vector<int>> dp(m, vector<int>(n));
        int maxLength = 0;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (i == 0 || j == 0 || matrix[i][j] == '0')
                    dp[i][j] = matrix[i][j] == '1' ? 1 : 0;
                else
                    dp[i][j] = min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]}) + 1;
                maxLength = max(maxLength, dp[i][j]);
            }

        return maxLength * maxLength;
    }
};
```

1308. 2210.cpp

Path: LeetCode-main/solutions/2210. Count Hills and Valleys in an Array/2210.cpp

```
class Solution {
public:
    int countHillValley(vector<int>& nums) {
        int ans = 0;
        int left = nums[0];

        for (int i = 1; i + 1 < nums.size(); ++i)
            if (left < nums[i] && nums[i] > nums[i + 1] || // the hill
                left > nums[i] && nums[i] < nums[i + 1]) { // the valley
                ++ans;
                left = nums[i];
            }

        return ans;
    }
};
```


1309. 2211.cpp

Path: LeetCode-main/solutions/2211. Count Collisions on a Road/2211.cpp

```
class Solution {
public:
    int countCollisions(string directions) {
        int l = 0;
        int r = directions.length() - 1;

        while (l < directions.length() && directions[l] == 'L')
            ++l;

        while (r >= 0 && directions[r] == 'R')
            --r;

        return count_if(directions.begin() + l, directions.begin() + r + 1,
            [](char c) { return c != 'S'; });
    }
};
```

1310. 2212.cpp

Path: LeetCode-main/solutions/2212. Maximum Points in an Archery Competition/2212.cpp

```
class Solution {
public:
    vector<int> maximumBobPoints(int numArrows, vector<int>& aliceArrows) {
        constexpr int kFullMask = (1 << 12) - 1;
        int maxPoint = 0;
        int maxMask = 0;

        for (int mask = 0; mask < kFullMask; ++mask) {
            const auto& [shorable, point] =
                getShorableAndPoint(mask, numArrows, aliceArrows);
            if (shorable && point > maxPoint) {
                maxPoint = point;
                maxMask = mask;
            }
        }

        return getBobsArrows(maxMask, numArrows, aliceArrows);
    }

private:
    pair<bool, int> getShorableAndPoint(int mask, int leftArrows,
                                       const vector<int>& aliceArrows) {
        int point = 0;
        for (int i = 0; i < 12; ++i)
            if (mask >> i & 1) {
                leftArrows -= aliceArrows[i] + 1;
                point += i;
            }
        return {leftArrows >= 0, point};
    }

    vector<int> getBobsArrows(int mask, int leftArrows,
                              const vector<int>& aliceArrows) {
        vector<int> bobsArrows(12);
        for (int i = 0; i < 12; ++i)
            if (mask >> i & 1) {
                bobsArrows[i] = aliceArrows[i] + 1;
                leftArrows -= aliceArrows[i] + 1;
            }
        bobsArrows[0] = leftArrows;
        return bobsArrows;
    }
};
```

1311. 2213.cpp

Path: LeetCode-main/solutions/2213. Longest Substring of One Repeating Character/2213.cpp

```
struct SegmentTreeNode {
    int lo;
    int hi;
    char maxLetter;
    char prefixLetter;
    char suffixLetter;
    int maxLength;
    int prefixLength;
    int suffixLength;
    SegmentTreeNode* left;
    SegmentTreeNode* right;
    SegmentTreeNode(int lo, int hi, char maxLetter, char prefixLetter,
                    char suffixLetter, int maxLength, int prefixLength,
                    int suffixLength, SegmentTreeNode* left = nullptr,
                    SegmentTreeNode* right = nullptr)
        : lo(lo),
          hi(hi),
          maxLetter(maxLetter),
          prefixLetter(prefixLetter),
          suffixLetter(suffixLetter),
          maxLength(maxLength),
          prefixLength(prefixLength),
          suffixLength(suffixLength),
          left(left),
          right(right) {}
    ~SegmentTreeNode() {
        delete left;
        delete right;
        left = nullptr;
        right = nullptr;
    }
};

class SegmentTree {
public:
    explicit SegmentTree(const string& s) : root(build(s, 0, s.length() - 1)) {}
    ~SegmentTree() {
        delete root;
    }

    void update(int i, char val) {
        root = update(root, i, val);
    }

    int getMaxLength() {
        return root->maxLength;
    }

private:
    SegmentTreeNode* root;

    SegmentTreeNode* build(const string& s, int lo, int hi) const {
        if (lo == hi)
            return new SegmentTreeNode(lo, hi, s[lo], s[lo], s[lo], 1, 1, 1);
        const int mid = (lo + hi) / 2;
        SegmentTreeNode* left = build(s, lo, mid);
        SegmentTreeNode* right = build(s, mid + 1, hi);
        return merge(left, right);
    }

    SegmentTreeNode* update(SegmentTreeNode* root, int i, char c) {
        if (root->lo == i && root->hi == i) {
            root->maxLetter = c;
            root->prefixLetter = c;
            root->suffixLetter = c;
            return root;
        }
        const int mid = (root->lo + root->hi) / 2;
        if (i <= mid) {
            SegmentTreeNode* updatedLeft = update(root->left, i, c);
            return root = merge(updatedLeft, root->right);
        } else {
            SegmentTreeNode* updatedRight = update(root->right, i, c);
            return root = merge(root->left, updatedRight);
        }
    }

    SegmentTreeNode* merge(SegmentTreeNode* left, SegmentTreeNode* right) const {
        // Get `maxLetter` and `maxLength`.
        char maxLetter = ' ';
    }
```

```

int maxLength = 0;
if (left->maxLength > right->maxLength) {
    maxLetter = left->maxLetter;
    maxLength = left->maxLength;
} else {
    maxLetter = right->maxLetter;
    maxLength = right->maxLength;
}
if (left->suffixLetter == right->prefixLetter &&
    left->suffixLength + right->prefixLength > maxLength) {
    maxLetter = left->suffixLetter;
    maxLength = left->suffixLength + right->prefixLength;
}

// Get `prefixLetter` and `prefixLength`.
char prefixLetter = left->prefixLetter;
int prefixLength = left->prefixLength;
if (left->lo + prefixLength == right->lo &&
    left->prefixLetter == right->prefixLetter)
    prefixLength += right->prefixLength;

// Get `suffixLetter` and `suffixLength`.
char suffixLetter = right->suffixLetter;
int suffixLength = right->suffixLength;
if (right->hi - suffixLength == left->hi &&
    right->suffixLetter == left->suffixLetter)
    suffixLength += left->suffixLength;
return new SegmentTreeNode(left->lo, right->hi, maxLetter, prefixLetter,
                           suffixLetter, maxLength, prefixLength,
                           suffixLength, left, right);
}
};

class Solution {
public:
    vector<int> longestRepeating(string s, string queryLetteracters,
                               vector<int>& queryIndices) {
        vector<int> ans;
        SegmentTree tree(s);

        for (int i = 0; i < queryIndices.size(); ++i) {
            tree.update(queryIndices[i], queryLetteracters[i]);
            ans.push_back(tree.getMaxLength());
        }

        return ans;
    }
};

```

1312. 2214.cpp

Path: LeetCode-main/solutions/2214. Minimum Health to Beat Game/2214.cpp

```
class Solution {
public:
    long long minimumHealth(vector<int>& damage, int armor) {
        const long sum = accumulate(damage.begin(), damage.end(), 0L);
        const int maxDamage = ranges::max(damage);
        return 1 + sum - min(maxDamage, armor);
    }
};
```

1313. 2215.cpp

Path: LeetCode-main/solutions/2215. Find the Difference of Two Arrays/2215.cpp

```
class Solution {
public:
    vector<vector<int>> findDifference(vector<int>& nums1, vector<int>& nums2) {
        const unordered_set<int> set1{nums1.begin(), nums1.end()};
        const unordered_set<int> set2{nums2.begin(), nums2.end()};
        vector<vector<int>> ans(2);

        for (const int num : set1)
            if (!set2.contains(num))
                ans[0].push_back(num);

        for (const int num : set2)
            if (!set1.contains(num))
                ans[1].push_back(num);

        return ans;
    }
};
```

1314. 2216.cpp

Path: LeetCode-main/solutions/2216. Minimum Deletions to Make Array Beautiful/2216.cpp

```
class Solution {
public:
    int minDeletion(vector<int>& nums) {
        int ans = 0;

        for (int i = 0; i + 1 < nums.size(); ++i)
            // i - ans := the index after deletion
            if (nums[i] == nums[i + 1] && (i - ans) % 2 == 0)
                ++ans;

        return ans + ((nums.size() - ans) & 1);
    }
};
```

1315. 2217.cpp

Path: LeetCode-main/solutions/2217. Find Palindrome With Fixed Length/2217.cpp

```
class Solution {
public:
    vector<long long> kthPalindrome(vector<int>& queries, int intLength) {
        const int start = pow(10, (intLength + 1) / 2 - 1);
        const int end = pow(10, (intLength + 1) / 2);
        const int mul = pow(10, intLength / 2);
        vector<long long> ans;

        for (const int query : queries)
            if (start + query > end)
                ans.push_back(-1);
            else
                ans.push_back(getKthPalindrome(query, start, mul, intLength));

        return ans;
    }

private:
    long getKthPalindrome(int query, int start, int mul, int intLength) {
        const long prefix = start + query - 1;
        return prefix * mul + reverse(intLength % 2 == 0 ? prefix : prefix / 10);
    }

    long reverse(int num) {
        long res = 0;
        while (num > 0) {
            res = res * 10 + num % 10;
            num /= 10;
        }
        return res;
    }
};
```


1316. 2218.cpp

Path: LeetCode-main/solutions/2218. Maximum Value of K Coins From Piles/2218.cpp

```
class Solution {
public:
    int maxValueOfCoins(vector<vector<int>>& piles, int k) {
        vector<vector<int>> mem(piles.size(), vector<int>(k + 1));
        return maxValueOfCoins(piles, 0, k, mem);
    }

private:
    // Returns the maximum value of picking k coins from piles[i..n)
    int maxValueOfCoins(const vector<vector<int>>& piles, int i, size_t k,
                        vector<vector<int>>& mem) {
        if (i == piles.size() || k == 0)
            return 0;
        if (mem[i][k])
            return mem[i][k];

        // Pick no coins from the current pile.
        int res = maxValueOfCoins(piles, i + 1, k, mem);
        int val = 0; // the coins picked from the current pile

        // Try to pick 1, 2, ..., k coins from the current pile.
        for (int j = 0; j < min(piles[i].size(), k); ++j) {
            val += piles[i][j];
            res = max(res, val + maxValueOfCoins(piles, i + 1, k - j - 1, mem));
        }

        return mem[i][k] = res;
    }
};
```

1317. 2219.cpp

Path: LeetCode-main/solutions/2219. Maximum Sum Score of Array/2219.cpp

```
class Solution {
public:
    long long maximumSumScore(vector<int>& nums) {
        long ans = LONG_MIN;
        long prefix = 0;
        long sum = accumulate(nums.begin(), nums.end(), 0L);

        for (const int num : nums) {
            prefix += num;
            ans = max({ans, prefix, sum - prefix + num});
        }

        return ans;
    }
};
```

1318. 222-2.cpp

Path: LeetCode-main/solutions/222. Count Complete Tree Nodes/222-2.cpp

```
class Solution {
public:
    int countNodes(TreeNode* root) {
        if (root == nullptr)
            return 0;

        TreeNode* left = root;
        TreeNode* right = root;
        int heightL = 0;
        int heightR = 0;

        while (left != nullptr) {
            ++heightL;
            left = left->left;
        }

        while (right != nullptr) {
            ++heightR;
            right = right->right;
        }

        if (heightL == heightR) // `root` is a complete tree.
            return pow(2, heightL) - 1;
        return 1 + countNodes(root->left) + countNodes(root->right);
    }
};
```

1319. 222.cpp

Path: LeetCode-main/solutions/222. Count Complete Tree Nodes/222.cpp

```
class Solution {
public:
    int countNodes(TreeNode* root) {
        if (root == nullptr)
            return 0;
        return 1 + countNodes(root->left) + countNodes(root->right);
    }
};
```

1320. 2220.cpp

Path: LeetCode-main/solutions/2220. Minimum Bit Flips to Convert Number/2220.cpp

```
class Solution {
public:
    int minBitFlips(unsigned start, unsigned goal) {
        return popcount(start ^ goal);
    }
};
```

1321. 2221.cpp

Path: LeetCode-main/solutions/2221. Find Triangular Sum of an Array/2221.cpp

```
class Solution {
public:
    int triangularSum(vector<int>& nums) {
        for (int sz = nums.size(); sz > 0; --sz)
            for (int i = 0; i + 1 < sz; ++i)
                nums[i] = (nums[i] + nums[i + 1]) % 10;
        return nums[0];
    }
};
```

1322. 2222.cpp

Path: LeetCode-main/solutions/2222. Number of Ways to Select Buildings/2222.cpp

```
class Solution {
public:
    long long numberOfWays(string s) {
        long ans = 0;
        // before[i] := the number of i before the current digit
        vector<int> before(2);
        // after[i] := the number of i after the current digit
        vector<int> after(2);
        after[0] = ranges::count(s, '0');
        after[1] = s.length() - after[0];

        for (const char c : s) {
            const int num = c - '0';
            --after[num];
            if (num == 0)
                ans += before[1] * after[1];
            else
                ans += before[0] * after[0];
            ++before[num];
        }

        return ans;
    }
};
```

1323. 2223.cpp

Path: LeetCode-main/solutions/2223. Sum of Scores of Built Strings/2223.cpp

```
class Solution {
public:
    long long sumScores(string s) {
        const int n = s.length();
        // https://cp-algorithms.com/string/z-function.html#implementation
        vector<int> z(n);
        // [l, r] := the indices of the rightmost segment match
        int l = 0;
        int r = 0;

        for (int i = 1; i < n; ++i) {
            if (i < r)
                z[i] = min(r - i, z[i - l]);
            while (i + z[i] < n && s[z[i]] == s[i + z[i]])
                ++z[i];
            if (i + z[i] > r) {
                l = i;
                r = i + z[i];
            }
        }

        return accumulate(z.begin(), z.end(), 0L) + n;
    }
};
```


1324. 2224.cpp

Path: LeetCode-main/solutions/2224. Minimum Number of Operations to Convert Time/2224.cpp

```
class Solution {
public:
    int convertTime(string current, string correct) {
        const vector<int> ops{60, 15, 5, 1};
        int diff = getMinutes(correct) - getMinutes(current);
        int ans = 0;

        for (const int op : ops) {
            ans += diff / op;
            diff %= op;
        }

        return ans;
    }

private:
    int getMinutes(const string& s) {
        return stoi(s.substr(0, 2)) * 60 + stoi(s.substr(3));
    }
};
```

1325. 2225.cpp

Path: LeetCode-main/solutions/2225. Find Players With Zero or One Losses/2225.cpp

```
class Solution {
public:
    vector<vector<int>> findWinners(vector<vector<int>>& matches) {
        vector<vector<int>> ans(2);
        map<int, int> lossesCount;

        for (const vector<int>& m : matches) {
            const int winner = m[0];
            const int loser = m[1];
            if (!lossesCount.contains(winner))
                lossesCount[winner] = 0;
            ++lossesCount[loser];
        }

        for (const auto& [player, nLosses] : lossesCount)
            if (nLosses < 2)
                ans[nLosses].push_back(player);

        return ans;
    }
};
```

1326. 2226.cpp

Path: LeetCode-main/solutions/2226. Maximum Candies Allocated to K Children/2226.cpp

```
class Solution {
public:
    int maximumCandies(vector<int>& candies, long long k) {
        int l = 1;
        int r = accumulate(candies.begin(), candies.end(), 0LL) / k;

        while (l < r) {
            const int m = (l + r) / 2;
            if (numChildren(candies, m) < k)
                r = m;
            else
                l = m + 1;
        }

        return numChildren(candies, l) >= k ? l : l - 1;
    }

private:
    long numChildren(const vector<int>& candies, int m) {
        return accumulate(candies.begin(), candies.end(), 0LL,
            [&](long acc, int c) { return acc + c / m; });
    };
};
```

1327. 2227-2.cpp

Path: LeetCode-main/solutions/2227. Encrypt and Decrypt Strings/2227-2.cpp

```
class Encrypter {
public:
    Encrypter(vector<char>& keys, vector<string>& values,
              vector<string>& dictionary) {
        for (int i = 0; i < keys.size(); ++i)
            keyToValue[keys[i]] = values[i];

        for (const string& word : dictionary)
            ++encryptedCount[encrypt(word)];
    }

    string encrypt(string word1) {
        string ans;
        for (const char c : word1)
            ans += keyToValue[c];
        return ans;
    }

    int decrypt(string word2) {
        return encryptedCount[word2];
    }

private:
    unordered_map<char, string> keyToValue;
    unordered_map<string, int> encryptedCount;
};
```

1328. 2227.cpp

Path: LeetCode-main/solutions/2227. Encrypt and Decrypt Strings/2227.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    bool isWord = false;
    TrieNode() : children(26) {}
};

class Encrypter {
public:
    Encrypter(vector<char>& keys, vector<string>& values,
              vector<string>& dictionary) {
        for (int i = 0; i < keys.size(); ++i) {
            const char key = keys[i];
            const string& value = values[i];
            keyToValue[key] = value;
            valueToKeys[value].push_back(key);
        }
        for (const string& word : dictionary)
            insert(word);
    }

    string encrypt(string word1) {
        string ans;
        for (const char c : word1)
            ans += keyToValue[c];
        return ans;
    }

    int decrypt(string word2) {
        return find(word2, 0, root);
    }

private:
    unordered_map<char, string> keyToValue;
    unordered_map<string, vector<char>> valueToKeys;
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
        node->isWord = true;
    }

    int find(const string& word, int i, shared_ptr<TrieNode> node) {
        const string value = word.substr(i, 2);
        if (!valueToKeys.contains(value))
            return 0;

        int ans = 0;
        if (i + 2 == word.length()) {
            for (const char key : valueToKeys[value]) {
                shared_ptr<TrieNode>& child = node->children[key - 'a'];
                if (child != nullptr && child->isWord)
                    ++ans;
            }
            return ans;
        }

        for (const char key : valueToKeys[value]) {
            if (!node->children[key - 'a'])
                continue;
            ans += find(word, i + 2, node->children[key - 'a']);
        }

        return ans;
    }
};
```

1329. 2229.cpp

Path: LeetCode-main/solutions/2229. Check if an Array Is Consecutive/2229.cpp

```
class Solution {
public:
    bool isConsecutive(vector<int>& nums) {
        const int n = nums.size();
        const int mx = ranges::max(nums);
        const int mn = ranges::min(nums);
        return mx - mn + 1 == n &&
            unordered_set<int>{nums.begin(), nums.end()}.size() == n;
    }
};
```

1330. 223.cpp

Path: LeetCode-main/solutions/223. Rectangle Area/223.cpp

```
class Solution {
public:
    int computeArea(long A, long B, long C, long D, //
                    long E, long F, long G, long H) {
        const long x = max(A, E) < min(C, G) ? (min(C, G) - max(A, E)) : 0;
        const long y = max(B, F) < min(D, H) ? (min(D, H) - max(B, F)) : 0;
        return (C - A) * (D - B) + (G - E) * (H - F) - x * y;
    }
};
```

1331. 2231.cpp

Path: LeetCode-main/solutions/2231. Largest Number After Digit Swaps by Parity/2231.cpp

```
class Solution {
public:
    int largestInteger(int num) {
        const string s = to_string(num);
        int ans = 0;
        // maxHeap[0] := the odd digits
        // maxHeap[1] := the even digits
        vector<priority_queue<int>> maxHeap(2);

        for (const char c : s) {
            const int digit = c - '0';
            maxHeap[digit % 2].push(digit);
        }

        for (const char c : s) {
            const int i = c - '0' & 1;
            ans *= 10;
            ans += maxHeap[i].top(), maxHeap[i].pop();
        }

        return ans;
    }
};
```


1332. 2232.cpp

Path: LeetCode-main/solutions/2232. Minimize Result by Adding Parentheses to Expression/2232.cpp

```
class Solution {
public:
    string minimizeResult(string expression) {
        const int plusIndex = expression.find('+');
        const string left = expression.substr(0, plusIndex);
        const string right = expression.substr(plusIndex + 1);
        string ans;
        int mn = INT_MAX;

        // the expression -> a * (b + c) * d
        for (int i = 0; i < left.length(); ++i)
            for (int j = 0; j < right.length(); ++j) {
                const int a = i == 0 ? 1 : stoi(left.substr(0, i));
                const int b = stoi(left.substr(i));
                const int c = stoi(right.substr(0, j + 1));
                const int d = j == right.length() - 1 ? 1 : stoi(right.substr(j + 1));
                const int val = a * (b + c) * d;
                if (val < mn) {
                    mn = val;
                    ans = (i == 0 ? "" : to_string(a)) + '(' + to_string(b) + '+' +
                        to_string(c) + ')' +
                        (j == right.length() - 1 ? "" : to_string(d));
                }
            }
        return ans;
    }
};
```

1333. 2233.cpp

Path: LeetCode-main/solutions/2233. Maximum Product After K Increments/2233.cpp

```
class Solution {
public:
    int maximumProduct(vector<int>& nums, int k) {
        constexpr int kMod = 1'000'000'007;
        long ans = 1;
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const int num : nums)
            minHeap.push(num);

        for (int i = 0; i < k; ++i) {
            const int minNum = minHeap.top();
            minHeap.pop();
            minHeap.push(minNum + 1);
        }

        while (!minHeap.empty()) {
            ans *= minHeap.top(), minHeap.pop();
            ans %= kMod;
        }

        return ans;
    }
};
```

1334. 2234.cpp

Path: LeetCode-main/solutions/2234. Maximum Total Beauty of the Gardens/2234.cpp

```
class Solution {
public:
    long long maximumBeauty(vector<int>& flowers, long long newFlowers,
                           int target, int full, int partial) {
        const int n = flowers.size();

        // If a garden is already complete, clamp it to the target.
        for (int& flower : flowers)
            flower = min(flower, target);
        ranges::sort(flowers);

        // All gardens are complete, so nothing we can do.
        if (flowers[0] == target)
            return static_cast<long>(n) * full;

        // Having many new flowers maximizes the beauty value.
        if (newFlowers >= static_cast<long>(n) * target -
            accumulate(flowers.begin(), flowers.end(), 0L))
            return max(static_cast<long>(n) * full,
                (n - 1L) * full + (target - 1L) * partial);

        long ans = 0;
        long leftFlowers = newFlowers;
        // cost[i] := the cost to make flowers[0..i] the same
        vector<long> cost(n);

        for (int i = 1; i < n; ++i)
            // Plant (flowers[i] - flowers[i - 1]) flowers for flowers[0..i - 1].
            cost[i] =
                cost[i - 1] + static_cast<long>(i) * (flowers[i] - flowers[i - 1]);

        int i = n - 1; // flowers' index (flowers[i + 1..n) are complete)
        while (flowers[i] == target)
            --i;

        for (; leftFlowers >= 0; --i) {
            // To maximize the minimum number of incomplete flowers, we find the first
            // index j that we can't make flowers[0..j] equal to flowers[j], then we
            // know we can make flowers[0..j - 1] equal to flowers[j - 1]. In the
            // meantime, evenly increase each of them to seek a bigger minimum value.
            const int j = firstGreater(cost, i, leftFlowers);
            const long minIncomplete =
                flowers[j - 1] + (leftFlowers - cost[j - 1]) / j;
            ans = max(ans, (n - 1L - i) * full + minIncomplete * partial);
            leftFlowers -= max(0, target - flowers[i]);
        }

        return ans;
    }

private:
    int firstGreater(const vector<long>& A, int maxIndex, long target) {
        return upper_bound(A.begin(), A.begin() + maxIndex + 1, target) - A.begin();
    }
};
```

1335. 2235.cpp

Path: LeetCode-main/solutions/2235. Add Two Integers/2235.cpp

```
class Solution {  
public:  
    int sum(int num1, int num2) {  
        return num1 + num2;  
    }  
};
```

1336. 2236.cpp

Path: LeetCode-main/solutions/2236. Root Equals Sum of Children/2236.cpp

```
class Solution {  
public:  
    bool checkTree(TreeNode* root) {  
        return root->val == root->left->val + root->right->val;  
    }  
};
```

1337. 2237.cpp

Path: LeetCode-main/solutions/2237. Count Positions on Street With Required Brightness/2237.cpp

```
class Solution {
public:
    int meetRequirement(int n, vector<vector<int>>& lights,
                        vector<int>& requirement) {
        int ans = 0;
        int currBrightness = 0;
        vector<int> change(n + 1);

        for (const vector<int>& light : lights) {
            const int position = light[0];
            const int range = light[1];
            ++change[max(0, position - range)];
            --change[min(n, position + range + 1)];
        }

        for (int i = 0; i < n; ++i) {
            currBrightness += change[i];
            if (currBrightness >= requirement[i])
                ++ans;
        }

        return ans;
    }
};
```

1338. 2239.cpp

Path: LeetCode-main/solutions/2239. Find Closest Number to Zero/2239.cpp

```
class Solution {
public:
    int findClosestNumber(vector<int>& nums) {
        return *ranges::min_element(nums, [](const int a, const int b) {
            return abs(a) == abs(b) ? a > b : abs(a) < abs(b);
        });
    }
};
```

1339. 224.cpp

Path: LeetCode-main/solutions/224. Basic Calculator/224.cpp

```
class Solution {
public:
    int calculate(string s) {
        int ans = 0;
        int num = 0;
        int sign = 1;
        stack<int> stack{{sign}}; // stack.top() := the current environment's sign

        for (const char c : s)
            if (isdigit(c))
                num = num * 10 + (c - '0');
            else if (c == '(')
                stack.push(sign);
            else if (c == ')')
                stack.pop();
            else if (c == '+' || c == '-') {
                ans += sign * num;
                sign = (c == '+' ? 1 : -1) * stack.top();
                num = 0;
            }

        return ans + sign * num;
    }
};
```


1340. 2240.cpp

Path: LeetCode-main/solutions/2240. Number of Ways to Buy Pens and Pencils/2240.cpp

```
class Solution {
public:
    long long waysToBuyPensPencils(int total, int cost1, int cost2) {
        long ans = 0;
        const int maxPen = total / cost1;

        for (int i = 0; i <= maxPen; ++i)
            ans += (total - i * cost1) / cost2 + 1;

        return ans;
    }
};
```

1341. 2241.cpp

Path: LeetCode-main/solutions/2241. Design an ATM Machine/2241.cpp

```
class ATM {
public:
    ATM() : bank(5) {}

    void deposit(vector<int> banknotesCount) {
        for (int i = 0; i < 5; ++i)
            bank[i] += banknotesCount[i];
    }

    vector<int> withdraw(int amount) {
        vector<int> withdrew(5);

        for (int i = 4; i >= 0; --i) {
            withdrew[i] = min(bank[i], static_cast<long>(amount) / banknotes[i]);
            amount -= withdrew[i] * banknotes[i];
        }

        if (amount > 0)
            return {-1};

        for (int i = 0; i < 5; ++i)
            bank[i] -= withdrew[i];
        return withdrew;
    }

private:
    vector<int> banknotes{20, 50, 100, 200, 500};
    vector<long> bank;
};
```

1342. 2242.cpp

Path: LeetCode-main/solutions/2242. Maximum Score of a Node Sequence/2242.cpp

```
class Solution {
public:
    int maximumScore(vector<int>& scores, vector<vector<int>>& edges) {
        int ans = -1;
        vector<set<pair<int, int>>> graph(scores.size()); // {(score, node)}

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].emplace(scores[v], v);
            graph[v].emplace(scores[u], u);
            if (graph[u].size() > 3)
                graph[u].erase(graph[u].begin());
            if (graph[v].size() > 3)
                graph[v].erase(graph[v].begin());
        }

        // To find the target sequence: a - u - v - b, enumerate each edge (u, v),
        // and find a (u's child) and b (v's child). That's why we find the 3
        // children that have the highest scores because one of the 3 children is
        // guaranteed to be valid.
        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            for (const auto& [scoreA, a] : graph[u])
                for (const auto& [scoreB, b] : graph[v])
                    if (a != b && a != v && b != u)
                        ans = max(ans, scoreA + scores[u] + scores[v] + scoreB);
        }

        return ans;
    }
};
```

1343. 2243.cpp

Path: LeetCode-main/solutions/2243. Calculate Digit Sum of a String/2243.cpp

```
class Solution {
public:
    string digitSum(string s, int k) {
        while (s.length() > k) {
            string next;
            for (int i = 0; i < s.length(); ++i)
                next += to_string(accumulate(
                    s.begin() + i, s.begin() + min(static_cast<int>(s.length()), i + k),
                    0, [](int acc, char c) { return acc + c - '0'; }));
            s = std::move(next);
        }
        return s;
    }
};
```

1344. 2244.cpp

Path: LeetCode-main/solutions/2244. Minimum Rounds to Complete All Tasks/2244.cpp

```
class Solution {
public:
    int minimumRounds(vector<int>& tasks) {
        int ans = 0;
        unordered_map<int, int> count;

        for (const int task : tasks)
            ++count[task];

        // freq = 1 -> it's impossible
        // freq = 2 -> needs 1 round
        // freq = 3 -> needs 1 round
        // freq = 3k -> needs k rounds
        // freq = 3k + 1 = 3 * (k - 1) + 2 * 2 -> needs k + 1 rounds
        // freq = 3k + 2 = 3 * k + 2 * 1 -> needs k + 1 rounds
        for (const auto& [_, freq] : count)
            if (freq == 1)
                return -1;
            else
                ans += (freq + 2) / 3;

        return ans;
    }
};
```

1345. 2245.cpp

Path: LeetCode-main/solutions/2245. Maximum Trailing Zeros in a Cornered Path/2245.cpp

```
class Solution {
public:
    int maxTrailingZeros(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        // leftPrefix2[i][j] := the number of 2 in grid[i][0..j]
        // leftPrefix5[i][j] := the number of 5 in grid[i][0..j]
        // topPrefix2[i][j] := the number of 2 in grid[0..i][j]
        // topPrefix5[i][j] := the number of 5 in grid[0..i][j]
        vector<vector<int>> leftPrefix2(m, vector<int>(n));
        vector<vector<int>> leftPrefix5(m, vector<int>(n));
        vector<vector<int>> topPrefix2(m, vector<int>(n));
        vector<vector<int>> topPrefix5(m, vector<int>(n));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                leftPrefix2[i][j] = getCount(grid[i][j], 2);
                leftPrefix5[i][j] = getCount(grid[i][j], 5);
                if (j > 0) {
                    leftPrefix2[i][j] += leftPrefix2[i][j - 1];
                    leftPrefix5[i][j] += leftPrefix5[i][j - 1];
                }
            }

        for (int j = 0; j < n; ++j)
            for (int i = 0; i < m; ++i) {
                topPrefix2[i][j] = getCount(grid[i][j], 2);
                topPrefix5[i][j] = getCount(grid[i][j], 5);
                if (i > 0) {
                    topPrefix2[i][j] += topPrefix2[i - 1][j];
                    topPrefix5[i][j] += topPrefix5[i - 1][j];
                }
            }

        int ans = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                const int curr2 = getCount(grid[i][j], 2);
                const int curr5 = getCount(grid[i][j], 5);
                const int l2 = leftPrefix2[i][j];
                const int l5 = leftPrefix5[i][j];
                const int r2 = leftPrefix2[i][n - 1] - (j ? leftPrefix2[i][j - 1] : 0);
                const int r5 = leftPrefix5[i][n - 1] - (j ? leftPrefix5[i][j - 1] : 0);
                const int t2 = topPrefix2[i][j];
                const int t5 = topPrefix5[i][j];
                const int d2 = topPrefix2[m - 1][j] - (i ? topPrefix2[i - 1][j] : 0);
                const int d5 = topPrefix5[m - 1][j] - (i ? topPrefix5[i - 1][j] : 0);
                ans = max({ans, min(l2 + t2 - curr2, l5 + t5 - curr5),
                           min(r2 + t2 - curr2, r5 + t5 - curr5),
                           min(l2 + d2 - curr2, l5 + d5 - curr5),
                           min(r2 + d2 - curr2, r5 + d5 - curr5)}});
            }

        return ans;
    }

private:
    int getCount(int num, int factor) {
        int count = 0;
        while (num % factor == 0) {
            num /= factor;
            ++count;
        }
        return count;
    }
};
```

1346. 2246.cpp

Path: LeetCode-main/solutions/2246. Longest Path With Different Adjacent Characters/2246.cpp

```
class Solution {
public:
    int longestPath(vector<int>& parent, string s) {
        const int n = parent.size();
        int ans = 1;
        vector<vector<int>> graph(n);

        for (int i = 1; i < n; ++i)
            graph[parent[i]].push_back(i);

        longestPathDownFrom(graph, 0, s, ans);
        return ans;
    }

private:
    int longestPathDownFrom(const vector<vector<int>>& graph, int u,
                           const string& s, int& ans) {
        int max1 = 0;
        int max2 = 0;

        for (const int v : graph[u]) {
            const int res = longestPathDownFrom(graph, v, s, ans);
            if (s[u] == s[v])
                continue;
            if (res > max1) {
                max2 = max1;
                max1 = res;
            } else if (res > max2) {
                max2 = res;
            }
        }

        ans = max(ans, 1 + max1 + max2);
        return 1 + max1;
    }
};
```

1347. 2247.cpp

Path: LeetCode-main/solutions/2247. Maximum Cost of Trip With K Highways/2247.cpp

```
class Solution {
public:
    int maximumCost(int n, vector<vector<int>>& highways, int k) {
        if (k + 1 > n)
            return -1;

        int ans = -1;
        vector<vector<int>> mem(n, vector<int>(1 << n, -1));
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& h : highways) {
            const int u = h[0];
            const int v = h[1];
            const int w = h[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        for (int i = 0; i < n; ++i)
            ans = max(ans, maximumCost(graph, i, 1 << i, k, mem));

        return ans;
    }

private:
    // Returns the maximum cost of trip starting from u, where `mask` is the
    // bitmask of the visited cities.
    int maximumCost(const vector<vector<pair<int, int>>>& graph, int u,
                    unsigned mask, int k, vector<vector<int>>& mem) {
        if (popcount(mask) == k + 1)
            return 0;
        if (mem[u][mask] != -1)
            return mem[u][mask];

        int res = -1;
        for (const auto& [v, w] : graph[u]) {
            if (mask >> v & 1)
                continue;
            const int nextCost = maximumCost(graph, v, mask | 1 << v, k, mem);
            if (nextCost != -1)
                res = max(res, w + nextCost);
        }

        return mem[u][mask] = res;
    }
};
```


1348. 2248.cpp

Path: LeetCode-main/solutions/2248. Intersection of Multiple Arrays/2248.cpp

```
class Solution {
public:
    vector<int> intersection(vector<vector<int>>& nums) {
        constexpr int kMax = 10000;
        vector<int> ans;
        vector<int> count(kMax + 1);

        for (const vector<int>& row : nums)
            for (const int a : row)
                ++count[a];

        for (int i = 1; i <= kMax; ++i)
            if (count[i] == nums.size())
                ans.push_back(i);

        return ans;
    }
};
```

1349. 2249-2.cpp

Path: LeetCode-main/solutions/2249. Count Lattice Points Inside a Circle/2249-2.cpp

```
class Solution {
public:
    int countLatticePoints(vector<vector<int>>& circles) {
        set<pair<int, int>> points;

        // dx := relative to x
        // dy := relative to y
        // So, dx^2 + dy^2 = r^2.
        for (const vector<int>& c : circles) {
            const int x = c[0];
            const int y = c[1];
            const int r = c[2];
            for (int dx = -r; dx <= r; ++dx) {
                const int yMax = sqrt(r * r - dx * dx);
                for (int dy = -yMax; dy <= yMax; ++dy)
                    points.emplace(x + dx, y + dy);
            }
        }
        return points.size();
    }
};
```

1350. 2249.cpp

Path: LeetCode-main/solutions/2249. Count Lattice Points Inside a Circle/2249.cpp

```
class Solution {
public:
    int countLatticePoints(vector<vector<int>>& circles) {
        int ans = 0;

        for (int x = 0; x < 201; ++x)
            for (int y = 0; y < 201; ++y)
                ans += ranges::any_of(circles, [x, y](const auto& c) {
                    return (c[0] - x) * (c[0] - x) + (c[1] - y) * (c[1] - y) <=
                           c[2] * c[2];
                });

        return ans;
    }
};
```

1351. 225.cpp

Path: LeetCode-main/solutions/225. Implement Stack using Queues/225.cpp

```
class MyStack {
public:
    void push(int x) {
        q.push(x);
        for (int i = 0; i < q.size() - 1; ++i) {
            q.push(q.front());
            q.pop();
        }
    }

    int pop() {
        const int val = q.front();
        q.pop();
        return val;
    }

    int top() {
        return q.front();
    }

    bool empty() {
        return q.empty();
    }

private:
    queue<int> q;
};
```

1352. 2250.cpp

Path: LeetCode-main/solutions/2250. Count Number of Rectangles Containing Each Point/2250.cpp

```
class Solution {
public:
    vector<int> countRectangles(vector<vector<int>>& rectangles,
                               vector<vector<int>>& points) {
        vector<int> ans;
        vector<vector<int>> yToXs(101);

        for (const vector<int>& r : rectangles)
            yToXs[r[1]].push_back(r[0]);

        for (auto& xs : yToXs)
            ranges::sort(xs);

        for (const vector<int>& p : points) {
            int count = 0;
            for (int y = p[1]; y < 101; ++y) {
                const vector<int>& xs = yToXs[y];
                count += xs.end() - ranges::lower_bound(xs, p[0]);
            }
            ans.push_back(count);
        }

        return ans;
    }
};
```

1353. 2251.cpp

Path: LeetCode-main/solutions/2251. Number of Flowers in Full Bloom/2251.cpp

```
class Solution {
public:
    vector<int> fullBloomFlowers(vector<vector<int>>& flowers,
                                vector<int>& persons) {
        vector<int> ans;
        vector<int> starts;
        vector<int> ends;

        for (const vector<int>& flower : flowers) {
            starts.push_back(flower[0]);
            ends.push_back(flower[1]);
        }

        ranges::sort(starts);
        ranges::sort(ends);

        for (const int p : persons) {
            const int started = ranges::upper_bound(starts, p) - starts.begin();
            const int ended = ranges::lower_bound(ends, p) - ends.begin();
            ans.push_back(started - ended);
        }

        return ans;
    }
};
```

1354. 2254.cpp

Path: LeetCode-main/solutions/2254. Design Video Sharing Platform/2254.cpp

```
class VideoSharingPlatform {
public:
    int upload(string video) {
        const int videoId = getVideoId();
        videoIdToVideo[videoId] = video;
        return videoId;
    }

    void remove(int videoId) {
        if (videoIdToVideo.contains(videoId)) {
            usedIds.push(videoId);
            videoIdToVideo.erase(videoId);
            videoIdToViews.erase(videoId);
            videoIdToLikes.erase(videoId);
            videoIdToDislikes.erase(videoId);
        }
    }

    string watch(int videoId, int startMinute, int endMinute) {
        const auto it = videoIdToVideo.find(videoId);
        if (it == videoIdToVideo.cend())
            return "-1";
        ++videoIdToViews[videoId];
        const string video = it->second;
        const int duration =
            min(endMinute, static_cast<int>(video.length()) - 1) - startMinute + 1;
        return video.substr(startMinute, duration);
    }

    void like(int videoId) {
        if (videoIdToVideo.contains(videoId))
            ++videoIdToLikes[videoId];
    }

    void dislike(int videoId) {
        if (videoIdToVideo.contains(videoId))
            ++videoIdToDislikes[videoId];
    }

    vector<int> getLikesAndDislikes(int videoId) {
        return videoIdToVideo.contains(videoId)
            ? vector<int>{videoIdToLikes[videoId],
                          videoIdToDislikes[videoId]}
            : vector<int>{-1};
    }

    int getViews(int videoId) {
        return videoIdToVideo.contains(videoId) ? videoIdToViews[videoId] : -1;
    }

private:
    int currVideoId = 0;
    priority_queue<int, vector<int>, greater<>> usedIds;
    unordered_map<int, string> videoIdToVideo;
    unordered_map<int, int> videoIdToViews;
    unordered_map<int, int> videoIdToLikes;
    unordered_map<int, int> videoIdToDislikes;

    int getVideoId() {
        if (usedIds.empty())
            return currVideoId++;
        const int minUsedId = usedIds.top();
        usedIds.pop();
        return minUsedId;
    }
};
```

1355. 2255.cpp

Path: LeetCode-main/solutions/2255. Count Prefixes of a Given String/2255.cpp

```
class Solution {
public:
    int countPrefixes(vector<string>& words, string s) {
        return ranges::count_if(
            words, [&](const string& word) { return s.find(word) == 0; });
    }
};
```


1356. 2256.cpp

Path: LeetCode-main/solutions/2256. Minimum Average Difference/2256.cpp

```
class Solution {
public:
    int minimumAverageDifference(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        int minDiff = INT_MAX;
        long prefix = 0;
        long suffix = accumulate(nums.begin(), nums.end(), 0L);

        for (int i = 0; i < nums.size(); ++i) {
            prefix += nums[i];
            suffix -= nums[i];
            const int prefixAvg = prefix / (i + 1);
            const int suffixAvg = (i == n - 1) ? 0 : suffix / (n - 1 - i);
            const int diff = abs(prefixAvg - suffixAvg);
            if (diff < minDiff) {
                ans = i;
                minDiff = diff;
            }
        }

        return ans;
    }
};
```

1357. 2257.cpp

Path: LeetCode-main/solutions/2257. Count Unguarded Cells in the Grid/2257.cpp

```
class Solution {
public:
    int countUnguarded(int m, int n, vector<vector<int>>& guards,
                      vector<vector<int>>& walls) {
        int ans = 0;
        vector<vector<char>> grid(m, vector<char>(n));
        vector<vector<char>> left(m, vector<char>(n));
        vector<vector<char>> right(m, vector<char>(n));
        vector<vector<char>> up(m, vector<char>(n));
        vector<vector<char>> down(m, vector<char>(n));

        for (const vector<int>& guard : guards)
            grid[guard[0]][guard[1]] = 'G';

        for (const vector<int>& wall : walls)
            grid[wall[0]][wall[1]] = 'W';

        for (int i = 0; i < m; ++i) {
            char lastCell = 0;
            for (int j = 0; j < n; ++j)
                recordOrFill(grid[i][j], lastCell, left[i][j]);
            lastCell = 0;
            for (int j = n - 1; j >= 0; --j)
                recordOrFill(grid[i][j], lastCell, right[i][j]);
        }

        for (int j = 0; j < n; ++j) {
            char lastCell = 0;
            for (int i = 0; i < m; ++i)
                recordOrFill(grid[i][j], lastCell, up[i][j]);
            lastCell = 0;
            for (int i = m - 1; i >= 0; --i)
                recordOrFill(grid[i][j], lastCell, down[i][j]);
        }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 0 && left[i][j] != 'G' && right[i][j] != 'G' &&
                    up[i][j] != 'G' && down[i][j] != 'G')
                    ++ans;

        return ans;
    }

private:
    void recordOrFill(char currCell, char& lastCell, char& infoCell) {
        if (currCell == 'G' || currCell == 'W')
            lastCell = currCell;
        else
            infoCell = lastCell;
    }
};
```

1358. 2258.cpp

Path: LeetCode-main/solutions/2258. Escape the Spreading Fire/2258.cpp

```
class Solution {
public:
    int maximumMinutes(vector<vector<int>>& grid) {
        const int kMax = grid.size() * grid[0].size();
        vector<vector<int>> fireMinute(grid.size(),
                                     vector<int>(grid[0].size(), -1));
        buildFireGrid(grid, fireMinute);

        int ans = -1;
        int l = 0;
        int r = kMax;

        while (l <= r) {
            const int m = (l + r) / 2;
            if (canStayFor(grid, fireMinute, m)) {
                ans = m;
                l = m + 1;
            } else {
                r = m - 1;
            }
        }

        return ans == kMax ? 1'000'000'000 : ans;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    void buildFireGrid(const vector<vector<int>>& grid,
                      vector<vector<int>>& fireMinute) {
        queue<pair<int, int>> q;

        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] == 1) { // the fire
                    q.emplace(i, j);
                    fireMinute[i][j] = 0;
                }

        for (int minuteFromFire = 1; !q.empty(); ++minuteFromFire)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == grid.size() || y < 0 || y == grid[0].size())
                        continue;
                    if (grid[x][y] == 2) // the wall
                        continue;
                    if (fireMinute[x][y] != -1)
                        continue;
                    fireMinute[x][y] = minuteFromFire;
                    q.emplace(x, y);
                }
            }
    }

    bool canStayFor(const vector<vector<int>>& grid,
                   const vector<vector<int>>& fireMinute, int minute) {
        queue<pair<int, int>> q{{0, 0}}; // the start position
        vector<vector<bool>> seen(grid.size(), vector<bool>(grid[0].size()));
        seen[0][0] = true;

        while (!q.empty()) {
            ++minute;
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == grid.size() || y < 0 || y == grid[0].size())
                        continue;
                    if (grid[x][y] == 2) // the wall
                        continue;
                    if (x == grid.size() - 1 && y == grid[0].size() - 1) {
                        if (fireMinute[x][y] != -1 && fireMinute[x][y] < minute)
                            continue;
                        return true;
                    }
                }
            }
        }
    }
};
```

```
    }
    if (fireMinute[x][y] != -1 && fireMinute[x][y] <= minute)
        continue;
    if (seen[x][y])
        continue;
    q.emplace(x, y);
    seen[x][y] = true;
}
}
}
return false;
}
};
```

1359. 2259.cpp

Path: LeetCode-main/solutions/2259. Remove Digit From Number to Maximize Result/2259.cpp

```
class Solution {
public:
    string removeDigit(string number, char digit) {
        for (int i = 0; i + 1 < number.length(); ++i)
            if (number[i] == digit && digit < number[i + 1])
                return number.erase(i, 1);
        return number.erase(number.rfind(digit), 1);
    }
};
```

1360. 226.cpp

Path: LeetCode-main/solutions/226. Invert Binary Tree/226.cpp

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (root == nullptr)
            return nullptr;

        TreeNode* const left = root->left;
        TreeNode* const right = root->right;
        root->left = invertTree(right);
        root->right = invertTree(left);
        return root;
    }
};
```

1361. 2260.cpp

Path: LeetCode-main/solutions/2260. Minimum Consecutive Cards to Pick Up/2260.cpp

```
class Solution {
public:
    int minimumCardPickup(vector<int>& cards) {
        int ans = INT_MAX;
        unordered_map<int, int> lastSeen;

        for (int i = 0; i < cards.size(); ++i) {
            if (const auto it = lastSeen.find(cards[i]); it != lastSeen.cend())
                ans = min(ans, i - it->second + 1);
            lastSeen[cards[i]] = i;
        }

        return ans == INT_MAX ? -1 : ans;
    }
};
```

1362. 2261.cpp

Path: LeetCode-main/solutions/2261. K Divisible Elements Subarrays/2261.cpp

```
struct TrieNode {
    unordered_map<int, shared_ptr<TrieNode>> children;
    int count = 0;
};

class Solution {
public:
    int countDistinct(vector<int>& nums, int k, int p) {
        int ans = 0;
        for (int i = 0; i < nums.size(); ++i)
            insert(root, nums, i, k, p, ans);
        return ans;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(shared_ptr<TrieNode> node, const vector<int>& nums, int i, int k,
                int p, int& ans) {
        if (i == nums.size() || k - (nums[i] % p == 0) < 0)
            return;
        if (!node->children.contains(nums[i])) {
            node->children[nums[i]] = make_shared<TrieNode>();
            ++ans;
        }
        insert(node->children[nums[i]], nums, i + 1, k - (nums[i] % p == 0), p,
              ans);
    }
};
```


1363. 2262-2.cpp

Path: LeetCode-main/solutions/2262. Total Appeal of A String/2262-2.cpp

```
class Solution {
public:
    long long appealSum(string s) {
        const int n = s.length();
        long ans = 0;
        vector<int> lastSeen(26, -1);

        for (int i = 0; i < n; ++i) {
            const int c = s[i] - 'a';
            ans += (i - lastSeen[c]) * (n - i);
            lastSeen[c] = i;
        }

        return ans;
    }
};
```

1364. 2262.cpp

Path: LeetCode-main/solutions/2262. Total Appeal of A String/2262.cpp

```
class Solution {
public:
    long long appealSum(string s) {
        long ans = 0;
        // the total appeal of all substrings ending in the index so far
        int dp = 0;
        vector<int> lastSeen(26, -1);

        for (int i = 0; i < s.length(); ++i) {
            // the total appeal of all substrings ending in s[i]
            // = the total appeal of all substrings ending in s[i - 1]
            // + the number of substrings ending in s[i] that contain only this s[i]
            const int c = s[i] - 'a';
            dp += i - lastSeen[c];
            ans += dp;
            lastSeen[c] = i;
        }

        return ans;
    }
};
```

1365. 2263.cpp

Path: LeetCode-main/solutions/2263. Make Array Non-decreasing or Non-increasing/2263.cpp

```
class Solution {
public:
    int convertArray(vector<int>& nums) {
        return min(cost(nums), cost(negative(nums)));
    }

private:
    int cost(const vector<int>& nums) {
        int ans = 0;
        priority_queue<int> maxHeap;

        // Greedily make `nums` non-decreasing.
        for (const int num : nums) {
            if (!maxHeap.empty() && maxHeap.top() > num) {
                ans += maxHeap.top() - num, maxHeap.pop();
                maxHeap.push(num);
            }
            maxHeap.push(num);
        }

        return ans;
    }

    vector<int> negative(const vector<int>& nums) {
        vector<int> arr(nums);
        for (int& a : arr)
            a *= -1;
        return arr;
    }
};
```

1366. 2264.cpp

Path: LeetCode-main/solutions/2264. Largest 3-Same-Digit Number in String/2264.cpp

```
class Solution {
public:
    string largestGoodInteger(string num) {
        string ans;

        for (int i = 2; i < num.length(); ++i)
            if (num[i - 2] == num[i - 1] && num[i - 1] == num[i])
                ans = max(ans, num.substr(i - 2, 3));

        return ans;
    }
};
```

1367. 2265.cpp

Path: LeetCode-main/solutions/2265. Count Nodes Equal to Average of Subtree/2265.cpp

```
class Solution {
public:
    int averageOfSubtree(TreeNode* root) {
        int ans = 0;
        dfs(root, ans);
        return ans;
    }

private:
    pair<int, int> dfs(TreeNode* root, int& ans) {
        if (root == nullptr)
            return {0, 0};
        const auto [leftSum, leftCount] = dfs(root->left, ans);
        const auto [rightSum, rightCount] = dfs(root->right, ans);
        const int sum = root->val + leftSum + rightSum;
        const int count = 1 + leftCount + rightCount;
        if (sum / count == root->val)
            ++ans;
        return {sum, count};
    }
};
```

1368. 2266.cpp

Path: LeetCode-main/solutions/2266. Count Number of Texts/2266.cpp

```
class Solution {
public:
    int countTexts(string pressedKeys) {
        constexpr int kMod = 1'000'000'007;
        const int n = pressedKeys.length();
        // dp[i] := the number of possible text messages of pressedKeys[i..n)
        vector<long> dp(n + 1);
        dp[n] = 1; // ""

        for (int i = n - 1; i >= 0; --i) {
            dp[i] = dp[i + 1];
            if (isSame(pressedKeys, i, 2))
                dp[i] += dp[i + 2];
            if (isSame(pressedKeys, i, 3))
                dp[i] += dp[i + 3];
            if ((pressedKeys[i] == '7' || pressedKeys[i] == '9') &&
                isSame(pressedKeys, i, 4))
                dp[i] += dp[i + 4];
            dp[i] %= kMod;
        }

        return dp[0];
    }

private:
    // Returns true if s[i..i + k) are the same digits.
    bool isSame(const string& s, int i, int k) {
        if (i + k > s.length())
            return false;
        for (int j = i + 1; j < i + k; ++j)
            if (s[j] != s[i])
                return false;
        return true;
    }
};
```

1369. 2267.cpp

Path: LeetCode-main/solutions/2267. Check if There Is a Valid Parentheses String Path/2267.cpp

```
class Solution {
public:
    bool hasValidPath(vector<vector<char>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<vector<int>>> mem(
            m, vector<vector<int>>(n, vector<int>(m + n, -1)));
        return hasValidPath(grid, 0, 0, 0, mem);
    }

private:
    // Returns true if there's a path from grid[i][j] to grid[m - 1][n - 1], where
    // the number of '(' - the number of ')' == k.
    bool hasValidPath(const vector<vector<char>>& grid, int i, int j, int k,
        vector<vector<vector<int>>>& mem) {
        if (i == grid.size() || j == grid[0].size())
            return false;
        k += grid[i][j] == '(' ? 1 : -1;
        if (k < 0)
            return false;
        if (i == grid.size() - 1 && j == grid[0].size() - 1)
            return k == 0;
        if (mem[i][j][k] != -1)
            return mem[i][j][k];
        return mem[i][j][k] = hasValidPath(grid, i + 1, j, k, mem) |
            hasValidPath(grid, i, j + 1, k, mem);
    }
};
```

1370. 2268.cpp

Path: LeetCode-main/solutions/2268. Minimum Number of Keypresses/2268.cpp

```
class Solution {
public:
    int minimumKeypresses(string s) {
        int ans = 0;
        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        ranges::sort(count, greater<>());

        for (int i = 0; i < 26; ++i)
            ans += count[i] * (i / 9 + 1);

        return ans;
    }
};
```


1371. 2269.cpp

Path: LeetCode-main/solutions/2269. Find the K-Beauty of a Number/2269.cpp

```
class Solution {
public:
    int divisorSubstrings(int num, int k) {
        const string s = to_string(num);
        int ans = 0;

        for (int i = 0; i + k <= s.length(); ++i) {
            const int x = stoi(s.substr(i, k));
            if (x != 0 && num % x == 0)
                ++ans;
        }

        return ans;
    }
};
```

1372. 227-2.cpp

Path: LeetCode-main/solutions/227. Basic Calculator II/227-2.cpp

```
class Solution {
public:
    int calculate(string s) {
        int ans = 0;
        int prevNum = 0;
        int currNum = 0;
        char op = '+';

        for (int i = 0; i < s.length(); ++i) {
            const char c = s[i];
            if (isdigit(c))
                currNum = currNum * 10 + (c - '0');
            if (!isdigit(c) && !isspace(c) || i == s.length() - 1) {
                if (op == '+' || op == '-') {
                    ans += prevNum;
                    prevNum = op == '+' ? currNum : -currNum;
                } else if (op == '*') {
                    prevNum *= currNum;
                } else if (op == '/') {
                    prevNum /= currNum;
                }
                op = c;
                currNum = 0;
            }
        }
        return ans + prevNum;
    }
};
```

1373. 227.cpp

Path: LeetCode-main/solutions/227. Basic Calculator II/227.cpp

```
class Solution {
public:
    int calculate(string s) {
        stack<int> nums;
        stack<char> ops;

        for (int i = 0; i < s.length(); ++i) {
            const char c = s[i];
            if (isdigit(c)) {
                int num = c - '0';
                while (i + 1 < s.length() && isdigit(s[i + 1])) {
                    num = num * 10 + (s[i + 1] - '0');
                    ++i;
                }
                nums.push(num);
            } else if (c == '+' || c == '-' || c == '*' || c == '/') {
                while (!ops.empty() && compare(ops.top(), c))
                    nums.push(calculate(pop(ops), pop(nums), pop(nums)));
                ops.push(c);
            }
        }

        while (!ops.empty())
            nums.push(calculate(pop(ops), pop(nums), pop(nums)));

        return nums.top();
    }

private:
    int calculate(char op, int b, int a) {
        switch (op) {
            case '+':
                return a + b;
            case '-':
                return a - b;
            case '*':
                return a * b;
            case '/':
                return a / b;
        }
        throw;
    }

    // Returns true if priority(op1) >= priority(op2).
    bool compare(char op1, char op2) {
        return op1 == '*' || op1 == '/' || op2 == '+' || op2 == '-';
    }

    char pop(stack<char>& ops) {
        const char op = ops.top();
        ops.pop();
        return op;
    }

    int pop(stack<int>& nums) {
        const int num = nums.top();
        nums.pop();
        return num;
    }
};
```

1374. 2270.cpp

Path: LeetCode-main/solutions/2270. Number of Ways to Split Array/2270.cpp

```
class Solution {
public:
    int waysToSplitArray(vector<int>& nums) {
        int ans = 0;
        long prefix = 0;
        long suffix = accumulate(nums.begin(), nums.end(), 0L);

        for (int i = 0; i < nums.size() - 1; ++i) {
            prefix += nums[i];
            suffix -= nums[i];
            if (prefix >= suffix)
                ++ans;
        }

        return ans;
    }
};
```

1375. 2271.cpp

Path: LeetCode-main/solutions/2271. Maximum White Tiles Covered by a Carpet/2271.cpp

```
class Solution {
public:
    int maximumWhiteTiles(vector<vector<int>>& tiles, int carpetLen) {
        if (ranges::any_of(tiles, [&](const auto& tile) {
            return tile[1] - tile[0] + 1 >= carpetLen;
        }))
            return carpetLen;

        int ans = 0;
        vector<int> starts;
        vector<int> prefix(tiles.size() + 1);

        ranges::sort(tiles);

        for (const vector<int>& tile : tiles)
            starts.push_back(tile[0]);

        for (int i = 0; i < tiles.size(); ++i) {
            const int length = tiles[i][1] - tiles[i][0] + 1;
            prefix[i + 1] = prefix[i] + length;
        }

        for (int i = 0; i < tiles.size(); ++i) {
            const int s = tiles[i][0];
            const int carpetEnd = s + carpetLen - 1;
            const int endIndex =
                ranges::upper_bound(starts, carpetEnd) - starts.begin() - 1;
            const int notCover = max(0, tiles[endIndex][1] - carpetEnd);
            ans = max(ans, prefix[endIndex + 1] - prefix[i] - notCover);
        }

        return ans;
    }
};
```

1376. 2272.cpp

Path: LeetCode-main/solutions/2272. Substring With Largest Variance/2272.cpp

```
class Solution {
public:
    int largestVariance(string s) {
        int ans = 0;

        for (char a = 'a'; a <= 'z'; ++a)
            for (char b = 'a'; b <= 'z'; ++b)
                if (a != b)
                    ans = max(ans, kadane(s, a, b));

        return ans;
    }

private:
    // a := the letter with the higher frequency
    // b := the letter with the lower frequency
    int kadane(const string& s, char a, char b) {
        int ans = 0;
        int countA = 0;
        int countB = 0;
        bool canExtendPrevB = false;

        for (const char c : s) {
            if (c != a && c != b)
                continue;
            if (c == a)
                ++countA;
            else
                ++countB;
            if (countB > 0) {
                // An interval should contain at least one b.
                ans = max(ans, countA - countB);
            } else if (countB == 0 && canExtendPrevB) {
                // edge case: consider the previous b.
                ans = max(ans, countA - 1);
            }
            // Reset if the number of b > the number of a.
            if (countB > countA) {
                countA = 0;
                countB = 0;
                canExtendPrevB = true;
            }
        }

        return ans;
    }
};
```

1377. 2273.cpp

Path: LeetCode-main/solutions/2273. Find Resultant Array After Removing Anagrams/2273.cpp

```
class Solution {
public:
    vector<string> removeAnagrams(vector<string>& words) {
        vector<string> ans;

        for (int i = 0; i < words.size(); i) {
            int j = i + 1;
            while (j < words.size() && isAnagram(words[i], words[j]))
                ++j;
            ans.push_back(words[i]);
            i = j;
        }

        return ans;
    }

private:
    bool isAnagram(const string& a, const string& b) {
        if (a.length() != b.length())
            return false;

        vector<int> count(26);

        for (const char c : a)
            ++count[c - 'a'];

        for (const char c : b)
            --count[c - 'a'];

        return ranges::all_of(count, [](const int c) { return c == 0; });
    }
};
```

1378. 2274.cpp

Path: LeetCode-main/solutions/2274. Maximum Consecutive Floors Without Special Floors/2274.cpp

```
class Solution {
public:
    int maxConsecutive(int bottom, int top, vector<int>& special) {
        int ans = 0;

        ranges::sort(special);

        for (int i = 1; i < special.size(); ++i)
            ans = max(ans, special[i] - special[i - 1] - 1);

        return max({ans, special.front() - bottom, top - special.back()});
    }
};
```


1379. 2275.cpp

Path: LeetCode-main/solutions/2275. Largest Combination With Bitwise AND Greater Than Zero/2275.cpp

```
class Solution {
public:
    int largestCombination(vector<int>& candidates) {
        constexpr int kMaxBit = 24;
        int ans = 0;

        for (int i = 0; i < kMaxBit; ++i) {
            int count = 0;
            for (const int candidate : candidates)
                if (candidate >> i & 1)
                    ++count;
            ans = max(ans, count);
        }

        return ans;
    }
};
```

1380. 2276.cpp

Path: LeetCode-main/solutions/2276. Count Integers in Intervals/2276.cpp

```
class CountIntervals {
public:
    void add(int left, int right) {
        while (isOverlapped(left, right)) {
            auto it = prev(intervals.upper_bound(right));
            const int l = it->first;
            const int r = it->second;
            left = min(left, l);
            right = max(right, r);
            intervals.erase(l);
            cnt -= r - l + 1;
        }

        intervals[left] = right;
        cnt += right - left + 1;
    }

    int count() {
        return cnt;
    }

private:
    map<int, int> intervals;
    int cnt = 0;

    bool isOverlapped(int left, int right) {
        auto it = intervals.upper_bound(right);
        return it != intervals.begin() && prev(it)->second >= left;
    }
};
```

1381. 2277.cpp

Path: LeetCode-main/solutions/2277. Closest Node to Path in Tree/2277.cpp

```
class Solution {
public:
    vector<int> closestNode(int n, vector<vector<int>>& edges,
                           vector<vector<int>>& query) {
        vector<int> ans;
        vector<vector<int>> tree(n);
        vector<vector<int>> dist(n, vector<int>(n, -1));

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        for (int i = 0; i < n; ++i)
            fillDist(tree, i, i, 0, dist);

        for (const vector<int>& query : queries) {
            const int start = query[0];
            const int end = query[1];
            const int node = query[2];
            ans.push_back(findClosest(tree, dist, start, end, node, start));
        }

        return ans;
    }

private:
    void fillDist(const vector<vector<int>>& tree, int start, int u, int d,
                 vector<vector<int>>& dist) {
        dist[start][u] = d;
        for (const int v : tree[u])
            if (dist[start][v] == -1)
                fillDist(tree, start, v, d + 1, dist);
    }

    int findClosest(const vector<vector<int>>& tree,
                   const vector<vector<int>>& dist, int u, int end, int node,
                   int ans) {
        for (const int v : tree[u])
            if (dist[v][end] < dist[u][end])
                return findClosest(tree, dist, v, end, node,
                                   dist[ans][node] < dist[v][node] ? ans : v);
        return ans;
    }
};
```

1382. 2278.cpp

Path: LeetCode-main/solutions/2278. Percentage of Letter in String/2278.cpp

```
class Solution {  
public:  
    int percentageLetter(string s, char letter) {  
        return 100 * ranges::count(s, letter) / s.length();  
    }  
};
```

1383. 2279.cpp

Path: LeetCode-main/solutions/2279. Maximum Bags With Full Capacity of Rocks/2279.cpp

```
class Solution {
public:
    int maximumBags(vector<int>& capacity, vector<int>& rocks,
                    int additionalRocks) {
        const int n = capacity.size();
        vector<int> diff(n);

        for (int i = 0; i < n; ++i)
            diff[i] = capacity[i] - rocks[i];

        ranges::sort(diff);

        for (int i = 0; i < n; ++i) {
            if (diff[i] > additionalRocks)
                return i;
            additionalRocks -= diff[i];
        }

        return n;
    }
};
```

1384. 228.cpp

Path: LeetCode-main/solutions/228. Summary Ranges/228.cpp

```
class Solution {
public:
    vector<string> summaryRanges(vector<int>& nums) {
        vector<string> ans;

        for (int i = 0; i < nums.size(); ++i) {
            const int begin = nums[i];
            while (i + 1 < nums.size() && nums[i] == nums[i + 1] - 1)
                ++i;
            const int end = nums[i];
            if (begin == end)
                ans.push_back(to_string(begin));
            else
                ans.push_back(to_string(begin) + "->" + to_string(end));
        }

        return ans;
    }
};
```

1385. 2280.cpp

Path: LeetCode-main/solutions/2280. Minimum Lines to Represent a Line Chart/2280.cpp

```
class Solution {
public:
    int minimumLines(vector<vector<int>>& stockPrices) {
        int ans = 0;

        ranges::sort(stockPrices);

        for (int i = 2; i < stockPrices.size(); ++i) {
            const pair<int, int> a = getSlope(stockPrices[i - 2], stockPrices[i - 1]);
            const pair<int, int> b = getSlope(stockPrices[i - 1], stockPrices[i]);
            if (a != b)
                ++ans;
        }

        return ans + (stockPrices.size() > 1);
    }

private:
    pair<int, int> getSlope(const vector<int>& p, const vector<int>& q) {
        const int dx = p[0] - q[0];
        const int dy = p[1] - q[1];
        if (dx == 0)
            return {0, p[0]};
        if (dy == 0)
            return {p[1], 0};
        const int d = __gcd(dx, dy);
        return {dx / d, dy / d};
    }
};
```

1386. 2281.cpp

Path: LeetCode-main/solutions/2281. Sum of Total Strength of Wizards/2281.cpp

```
class Solution {
public:
    int totalStrength(vector<int>& strength) {
        constexpr int kMod = 1'000'000'007;
        const int n = strength.size();
        vector<long> prefix(n);
        vector<long> prefixOfPrefix(n + 1);
        // left[i] := the next index on the left (if any) s.t.
        // nums[left[i]] <= nums[i]
        vector<int> left(n, -1);
        // right[i] := the next index on the right (if any) s.t.
        // nums[right[i]] < nums[i]
        vector<int> right(n, n);
        stack<int> stack;

        for (int i = 0; i < n; ++i)
            prefix[i] = i == 0 ? strength[0] : (strength[i] + prefix[i - 1]) % kMod;

        for (int i = 0; i < n; ++i)
            prefixOfPrefix[i + 1] = (prefixOfPrefix[i] + prefix[i]) % kMod;

        for (int i = n - 1; i >= 0; --i) {
            while (!stack.empty() && strength[stack.top()] >= strength[i])
                left[stack.top()] = i, stack.pop();
            stack.push(i);
        }

        stack = std::stack<int>();

        for (int i = 0; i < n; ++i) {
            while (!stack.empty() && strength[stack.top()] > strength[i])
                right[stack.top()] = i, stack.pop();
            stack.push(i);
        }

        long ans = 0;

        // For each strength[i] as minimum, calculate sum.
        for (int i = 0; i < n; ++i) {
            const int l = left[i];
            const int r = right[i];
            const long leftSum = prefixOfPrefix[i] - prefixOfPrefix[max(0, l)];
            const long rightSum = prefixOfPrefix[r] - prefixOfPrefix[i];
            const int leftLen = i - l;
            const int rightLen = r - i;
            ans += strength[i] *
                (rightSum * leftLen % kMod - leftSum * rightLen % kMod + kMod) %
                kMod;
            ans %= kMod;
        }

        return ans;
    }
};
```


1387. 2282.cpp

Path: LeetCode-main/solutions/2282. Number of People That Can Be Seen in a Grid/2282.cpp

```
class Solution {
public:
    vector<vector<int>> seePeople(vector<vector<int>>& heights) {
        const int m = heights.size();
        const int n = heights[0].size();
        vector<vector<int>> ans(m, vector<int>(n));

        for (int i = 0; i < m; ++i) {
            stack<int> stack;
            for (int j = 0; j < n; ++j) {
                bool hasEqualHeight = false;
                while (!stack.empty() && heights[i][stack.top()] <= heights[i][j]) {
                    if (heights[i][stack.top()] == heights[i][j])
                        // edge case: [4, 2, 1, 1, 3]
                        hasEqualHeight = true;
                    ++ans[i][stack.top()], stack.pop();
                }
                if (!stack.empty() && !hasEqualHeight)
                    ++ans[i][stack.top()];
                stack.push(j);
            }
        }

        for (int j = 0; j < n; ++j) {
            stack<int> stack;
            for (int i = 0; i < m; ++i) {
                bool hasEqualHeight = false;
                while (!stack.empty() && heights[stack.top()][j] <= heights[i][j]) {
                    if (heights[stack.top()][j] == heights[i][j])
                        hasEqualHeight = true;
                    ++ans[stack.top()][j], stack.pop();
                }
                if (!stack.empty() && !hasEqualHeight)
                    ++ans[stack.top()][j];
                stack.push(i);
            }
        }

        return ans;
    }
};
```

1388. 2283.cpp

Path: LeetCode-main/solutions/2283. Check if Number Has Equal Digit Count and Digit Value/2283.cpp

```
class Solution {
public:
    bool digitCount(string num) {
        vector<int> count(10);

        for (const char c : num)
            ++count[c - '0'];

        for (int i = 0; i < num.length(); ++i)
            if (count[i] != num[i] - '0')
                return false;

        return true;
    }
};
```

1389. 2284.cpp

Path: LeetCode-main/solutions/2284. Sender With Largest Word Count/2284.cpp

```
class Solution {
public:
    string largestWordCount(vector<string>& messages, vector<string>& senders) {
        const int n = messages.size();
        string ans;
        int maxWordsSent = 0;
        unordered_map<string, int> count; // {sender, the number of words sent}

        for (int i = 0; i < n; ++i) {
            const string& message = messages[i];
            const string& sender = senders[i];
            const int wordsCount = ranges::count(message, ' ') + 1;
            count[sender] += wordsCount;
            const int numWordsSent = count[sender];
            if (numWordsSent > maxWordsSent) {
                ans = sender;
                maxWordsSent = numWordsSent;
            } else if (numWordsSent == maxWordsSent && sender > ans) {
                ans = sender;
            }
        }

        return ans;
    }
};
```

1390. 2285.cpp

Path: LeetCode-main/solutions/2285. Maximum Total Importance of Roads/2285.cpp

```
class Solution {
public:
    long long maximumImportance(int n, vector<vector<int>>& roads) {
        long ans = 0;
        vector<long> count(n);

        for (const vector<int>& road : roads) {
            const int u = road[0];
            const int v = road[1];
            ++count[u];
            ++count[v];
        }

        ranges::sort(count);

        for (int i = 0; i < n; ++i)
            ans += (i + 1) * count[i];

        return ans;
    }
};
```

1391. 2286.cpp

Path: LeetCode-main/solutions/2286. Booking Concert Tickets in Groups/2286.cpp

```
struct SegmentTreeNode {
    int lo;
    int hi;
    std::unique_ptr<SegmentTreeNode> left;
    std::unique_ptr<SegmentTreeNode> right;
    int mx;
    long sum;
    SegmentTreeNode(int lo, int hi, std::unique_ptr<SegmentTreeNode>&& left,
                    std::unique_ptr<SegmentTreeNode>&& right, int mx, long sum)
        : lo(lo),
          hi(hi),
          left(std::move(left)),
          right(std::move(right)),
          mx(mx),
          sum(sum) {}
};

class SegmentTree {
public:
    explicit SegmentTree(int n, int m) : m(m), root(std::move(build(0, n - 1))) {}

    vector<int> maxRange(int k, int maxRow) {
        return maxRange(root, k, maxRow);
    }

    long sumRange(int maxRow) {
        return sumRange(root, 0, maxRow);
    }

    // Subtracts k from the seats row.
    void subtract(int row, int k) {
        subtract(root, row, k);
    }

private:
    const int m;
    std::unique_ptr<SegmentTreeNode> root;

    std::unique_ptr<SegmentTreeNode> build(int l, int r) {
        if (l == r)
            return make_unique<SegmentTreeNode>(l, r, nullptr, nullptr, m, m);
        const int mid = (l + r) / 2;
        std::unique_ptr<SegmentTreeNode> left = build(l, mid);
        std::unique_ptr<SegmentTreeNode> right = build(mid + 1, r);
        return make_unique<SegmentTreeNode>(l, r, std::move(left), std::move(right),
                                            max(left->mx, right->mx),
                                            left->sum + right->sum);
    }

    vector<int> maxRange(std::unique_ptr<SegmentTreeNode>& root, int k,
                        int maxRow) {
        if (root->lo == root->hi) {
            if (root->sum < k || root->lo > maxRow)
                return {};
            return {root->lo, m - static_cast<int>(root->sum)}; // {row, col}
        }
        // Greedily search the left subtree
        if (root->left->mx >= k)
            return maxRange(root->left, k, maxRow);
        return maxRange(root->right, k, maxRow);
    }

    long sumRange(std::unique_ptr<SegmentTreeNode>& root, int i, int j) {
        if (root->lo == i && root->hi == j)
            return root->sum;
        const int mid = (root->lo + root->hi) / 2;
        if (j <= mid)
            return sumRange(root->left, i, j);
        if (i > mid)
            return sumRange(root->right, i, j);
        return sumRange(root->left, i, mid) + sumRange(root->right, mid + 1, j);
    }

    void subtract(std::unique_ptr<SegmentTreeNode>& root, int row, int k) {
        if (root == nullptr)
            return;
        if (root->lo == root->hi && root->hi == row) {
            root->mx -= k;
            root->sum -= k;
            return;
        }
    }
};
```

```

    }
    const int mid = (root->lo + root->hi) / 2;
    if (row <= mid)
        subtract(root->left, row, k);
    else
        subtract(root->right, row, k);
    root->mx = max(root->left->mx, root->right->mx);
    root->sum = root->left->sum + root->right->sum;
}
};

class BookMyShow {
public:
    BookMyShow(int n, int m) : tree(n, m), seats(n, m) {}

    vector<int> gather(int k, int maxRow) {
        const vector<int> res = tree.maxRange(k, maxRow);
        if (res.size() == 2) {
            const int row = res[0];
            tree.subtract(row, k);
            seats[row] -= k;
        }
        return res;
    }

    bool scatter(int k, int maxRow) {
        if (tree.sumRange(maxRow) < k)
            return false;

        while (k > 0)
            if (seats[minVacantRow] >= k) {
                tree.subtract(minVacantRow, k);
                seats[minVacantRow] -= k;
                k = 0;
            } else {
                tree.subtract(minVacantRow, seats[minVacantRow]);
                k -= seats[minVacantRow];
                seats[minVacantRow] = 0;
                ++minVacantRow;
            }

        return true;
    }

private:
    SegmentTree tree;
    vector<int> seats; // the remaining seats at each row
    int minVacantRow = 0;
};

```

1392. 2287.cpp

Path: LeetCode-main/solutions/2287. Rearrange Characters to Make Target String/2287.cpp

```
class Solution {
public:
    int rearrangeCharacters(string s, string target) {
        int ans = s.length();
        vector<int> counts(128);
        vector<int> countT(128);

        for (const char c : s)
            ++counts[c];

        for (const char c : target)
            ++countT[c];

        for (const char c : target)
            ans = min(ans, counts[c] / countT[c]);

        return ans;
    }
};
```

1393. 2288.cpp

Path: LeetCode-main/solutions/2288. Apply Discount to Prices/2288.cpp

```
class Solution {
public:
    string discountPrices(string sentence, int discount) {
        constexpr int kPrecision = 2;
        string ans;
        istringstream iss(sentence);

        for (string word; iss >> word;)
            if (word[0] == '$' && word.length() > 1) {
                const string digits = word.substr(1);
                if (ranges::all_of(digits,
                                   [](const char digit) { return isdigit(digit); })) {
                    const double val = stold(digits) * (100 - discount) / 100;
                    const string s = to_string(val);
                    const string trimmed = s.substr(0, s.find(".") + kPrecision + 1);
                    ans += "$" + trimmed + " ";
                } else {
                    ans += word + " ";
                }
            } else {
                ans += word + " ";
            }

        ans.pop_back();
        return ans;
    }
};
```


1394. 2289.cpp

Path: LeetCode-main/solutions/2289. Steps to Make Array Non-decreasing/2289.cpp

```
class Solution {
public:
    int totalSteps(vector<int>& nums) {
        // dp[i] := the number of steps to remove nums[i]
        vector<int> dp(nums.size());
        stack<int> stack;

        for (int i = 0; i < nums.size(); ++i) {
            int step = 1;
            while (!stack.empty() && nums[stack.top()] <= nums[i])
                step = max(step, dp[stack.top()] + 1), stack.pop();
            if (!stack.empty())
                dp[i] = step;
            stack.push(i);
        }

        return ranges::max(dp);
    }
};
```

1395. 229.cpp

Path: LeetCode-main/solutions/229. Majority Element II/229.cpp

```
class Solution {
public:
    vector<int> majorityElement(vector<int>& nums) {
        vector<int> ans;
        int candidate1 = 0;
        int candidate2 = 1;    // any number different from candidate1
        int countSoFar1 = 0;    // the number of candidate1 so far
        int countSoFar2 = 0;    // the number of candidate2 so far

        for (const int num : nums)
            if (num == candidate1) {
                ++countSoFar1;
            } else if (num == candidate2) {
                ++countSoFar2;
            } else if (countSoFar1 == 0) { // Assign the new candidate.
                candidate1 = num;
                ++countSoFar1;
            } else if (countSoFar2 == 0) { // Assign the new candidate.
                candidate2 = num;
                ++countSoFar2;
            } else { // Meet a new number, so pair with the previous counts.
                --countSoFar1;
                --countSoFar2;
            }

        const int count1 = ranges::count(nums, candidate1);
        const int count2 = ranges::count(nums, candidate2);

        if (count1 > nums.size() / 3)
            ans.push_back(candidate1);
        if (count2 > nums.size() / 3)
            ans.push_back(candidate2);
        return ans;
    }
};
```

1396. 2290.cpp

Path: LeetCode-main/solutions/2290. Minimum Obstacle Removal to Reach Corner/2290.cpp

```
class Solution {
public:
    int minimumObstacles(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        using T = tuple<int, int, int>; // (d, i, j)
        priority_queue<T, vector<T>, greater<>> minHeap;
        vector<vector<int>> dist(m, vector<int>(n, INT_MAX));

        minHeap.emplace(grid[0][0], 0, 0);
        dist[0][0] = grid[0][0];

        while (!minHeap.empty()) {
            const auto [d, i, j] = minHeap.top();
            minHeap.pop();
            if (i == m - 1 && j == n - 1)
                return d;
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                const int newDist = d + grid[i][j];
                if (newDist < dist[x][y]) {
                    dist[x][y] = newDist;
                    minHeap.emplace(newDist, x, y);
                }
            }
        }
        return dist[m - 1][n - 1];
    }
};
```

1397. 2291-2.cpp

Path: LeetCode-main/solutions/2291. Maximum Profit From Trading Stocks/2291-2.cpp

```
class Solution {
public:
    int maximumProfit(vector<int>& present, vector<int>& future, int budget) {
        const int n = present.size();
        // dp[i] := the maximum profit of buying present so far with i budget
        vector<int> dp(budget + 1);

        for (int i = 0; i < n; ++i)
            for (int j = budget; j >= present[i]; --j)
                dp[j] = max(dp[j], future[i] - present[i] + dp[j - present[i]]);

        return dp[budget];
    }
};
```

1398. 2291.cpp

Path: LeetCode-main/solutions/2291. Maximum Profit From Trading Stocks/2291.cpp

```
class Solution {
public:
    int maximumProfit(vector<int>& present, vector<int>& future, int budget) {
        const int n = present.size();
        // dp[i][j] := the maximum profit of buying present[0..i) with j budget
        vector<vector<int>> dp(n + 1, vector<int>(budget + 1));

        for (int i = 1; i <= n; ++i) {
            const int profit = future[i - 1] - present[i - 1];
            for (int j = 0; j <= budget; ++j)
                if (j < present[i - 1])
                    dp[i][j] = dp[i - 1][j];
                else
                    dp[i][j] = max(dp[i - 1][j], profit + dp[i - 1][j - present[i - 1]]);
        }

        return dp[n][budget];
    }
};
```

1399. 2293.cpp

Path: LeetCode-main/solutions/2293. Min Max Game/2293.cpp

```
class Solution {
public:
    int minMaxGame(vector<int>& nums) {
        if (nums.size() == 1)
            return nums[0];

        vector<int> nextNums;
        for (int i = 0; i < nums.size() / 2; ++i)
            nextNums.push_back(i % 2 == 0 ? min(nums[2 * i], nums[2 * i + 1])
                                           : max(nums[2 * i], nums[2 * i + 1]));
        return minMaxGame(nextNums);
    }
};
```

1400. 2294.cpp

Path: LeetCode-main/solutions/2294. Partition Array Such That Maximum Difference Is K/2294.cpp

```
class Solution {
public:
    int partitionArray(vector<int>& nums, int k) {
        ranges::sort(nums);

        int ans = 1;
        int mn = nums[0];

        for (int i = 1; i < nums.size(); ++i)
            if (mn + k < nums[i]) {
                ++ans;
                mn = nums[i];
            }

        return ans;
    }
};
```

1401. 2295.cpp

Path: LeetCode-main/solutions/2295. Replace Elements in an Array/2295.cpp

```
class Solution {
public:
    vector<int> arrayChange(vector<int>& nums, vector<vector<int>>& operations) {
        unordered_map<int, int> numToIndex;

        for (int i = 0; i < nums.size(); ++i)
            numToIndex[nums[i]] = i;

        for (const vector<int>& o : operations) {
            const int original = o[0];
            const int replaced = o[1];
            const int index = numToIndex[original];
            nums[index] = replaced;
            numToIndex.erase(original);
            numToIndex[replaced] = index;
        }

        return nums;
    }
};
```


1402. 2296.cpp

Path: LeetCode-main/solutions/2296. Design a Text Editor/2296.cpp

```
class TextEditor {
public:
    void addText(string text) {
        s += text;
    }

    int deleteText(int k) {
        const int numDeleted = min(k, static_cast<int>(s.length()));
        for (int i = 0; i < numDeleted; ++i)
            s.pop_back();
        return numDeleted;
    }

    string cursorLeft(int k) {
        while (!s.empty() && k--)
            stack.push(s.back()), s.pop_back();
        return getString();
    }

    string cursorRight(int k) {
        while (!stack.empty() && k--)
            s += stack.top(), stack.pop();
        return getString();
    }

    string getString() {
        if (s.length() < 10)
            return s;
        return s.substr(s.length() - 10);
    }

private:
    string s;
    stack<char> stack;
};
```

1403. 2297.cpp

Path: LeetCode-main/solutions/2297. Jump Game IX/2297.cpp

```
class Solution {
public:
    long long minCost(vector<int>& nums, vector<int>& costs) {
        const int n = nums.size();
        // dp[i] := the minimum cost to jump to i
        vector<long> dp(n, LONG_MAX);
        stack<int> maxStack;
        stack<int> minStack;

        dp[0] = 0;

        for (int i = 0; i < n; ++i) {
            while (!maxStack.empty() && nums[i] >= nums[maxStack.top()])
                dp[i] = min(dp[i], dp[maxStack.top()] + costs[i]), maxStack.pop();
            while (!minStack.empty() && nums[i] < nums[minStack.top()])
                dp[i] = min(dp[i], dp[minStack.top()] + costs[i]), minStack.pop();
            maxStack.push(i);
            minStack.push(i);
        }

        return dp.back();
    }
};
```

1404. 2299.cpp

Path: LeetCode-main/solutions/2299. Strong Password Checker II/2299.cpp

```
class Solution {
public:
    bool strongPasswordCheckerII(string password) {
        if (password.length() < 8)
            return false;

        const bool hasLowerCase =
            ranges::any_of(password, [](const char c) { return islower(c); });
        if (!hasLowerCase)
            return false;

        const bool hasUpperCase =
            ranges::any_of(password, [](const char c) { return isupper(c); });
        if (!hasUpperCase)
            return false;

        const bool hasDigit =
            ranges::any_of(password, [](const char c) { return isdigit(c); });
        if (!hasDigit)
            return false;

        const bool hasSpecial = ranges::any_of(password, [](const char c) {
            return string("!@#$%^&*()-+").find(c) != string::npos;
        });
        if (!hasSpecial)
            return false;

        for (int i = 1; i < password.length(); ++i)
            if (password[i] == password[i - 1])
                return false;
        return true;
    }
};
```

1405. 23.cpp

Path: LeetCode-main/solutions/23. Merge k Sorted Lists/23.cpp

```
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        ListNode dummy(0);
        ListNode* curr = &dummy;
        auto compare = [](ListNode* a, ListNode* b) { return a->val > b->val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(compare)> minHeap(
            compare);

        for (ListNode* list : lists)
            if (list != nullptr)
                minHeap.push(list);

        while (!minHeap.empty()) {
            ListNode* minNode = minHeap.top();
            minHeap.pop();
            if (minNode->next)
                minHeap.push(minNode->next);
            curr->next = minNode;
            curr = curr->next;
        }

        return dummy.next;
    }
};
```

1406. 230-2.cpp

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230-2.cpp

```
class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        int ans = -1;
        int rank = 0;
        traverse(root, k, rank, ans);
        return ans;
    }

private:
    void traverse(TreeNode* root, int k, int& rank, int& ans) {
        if (root == nullptr)
            return;

        traverse(root->left, k, rank, ans);
        if (++rank == k) {
            ans = root->val;
            return;
        }
        traverse(root->right, k, rank, ans);
    }
};
```

1407. 230-3.cpp

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230-3.cpp

```
class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        stack<TreeNode*> stack;

        while (root != nullptr) {
            stack.push(root);
            root = root->left;
        }

        for (int i = 0; i < k - 1; ++i) {
            root = stack.top(); stack.pop();
            root = root->right;
            while (root != nullptr) {
                stack.push(root);
                root = root->left;
            }
        }

        return stack.top()->val;
    }
};
```

1408. 230.cpp

Path: LeetCode-main/solutions/230. Kth Smallest Element in a BST/230.cpp

```
class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        const int leftCount = countNodes(root->left);

        if (leftCount == k - 1)
            return root->val;
        if (leftCount >= k)
            return kthSmallest(root->left, k);
        return kthSmallest(root->right, k - 1 - leftCount); // leftCount < k
    }

private:
    int countNodes(TreeNode* root) {
        if (root == nullptr)
            return 0;
        return 1 + countNodes(root->left) + countNodes(root->right);
    }
};
```

1409. 2300.cpp

Path: LeetCode-main/solutions/2300. Successful Pairs of Spells and Potions/2300.cpp

```
class Solution {
public:
    vector<int> successfulPairs(vector<int>& spells, vector<int>& potions,
                              long long success) {
        vector<int> ans;
        ranges::sort(potions);

        for (const int spell : spells)
            ans.push_back(potions.size() -
                          firstIndexSuccess(spell, potions, success));

        return ans;
    }

private:
    // Returns the first index i s.t. spell * potions[i] >= success.
    int firstIndexSuccess(int spell, const vector<int>& potions, long success) {
        int l = 0;
        int r = potions.size();
        while (l < r) {
            const int m = (l + r) / 2;
            if (static_cast<long>(spell) * potions[m] >= success)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
};
```


1410. 2301.cpp

Path: LeetCode-main/solutions/2301. Match Substring After Replacement/2301.cpp

```
class Solution {
public:
    bool matchReplacement(string s, string sub, vector<vector<char>>& mappings) {
        vector<vector<bool>> isMapped(128, vector<bool>(128));

        for (const vector<char>& m : mappings) {
            const char old = m[0];
            const char _new = m[1];
            isMapped[old][_new] = true;
        }

        for (int i = 0; i < s.length(); ++i)
            if (canTransform(s, i, sub, isMapped))
                return true;

        return false;
    }

private:
    bool canTransform(const string& s, int start, const string& sub,
                     const vector<vector<bool>>& isMapped) {
        if (start + sub.length() > s.length())
            return false;

        for (int i = 0; i < sub.length(); ++i) {
            const char a = sub[i];
            const char b = s[start + i];
            if (a != b && !isMapped[a][b])
                return false;
        }

        return true;
    }
};
```

1411. 2302.cpp

Path: LeetCode-main/solutions/2302. Count Subarrays With Score Less Than K/2302.cpp

```
class Solution {
public:
    long long countSubarrays(vector<int>& nums, long long k) {
        long ans = 0;
        long sum = 0;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            sum += nums[r];
            while (sum * (r - l + 1) >= k)
                sum -= nums[l++];
            ans += r - l + 1;
        }

        return ans;
    }
};
```

1412. 2303.cpp

Path: LeetCode-main/solutions/2303. Calculate Amount Paid in Taxes/2303.cpp

```
class Solution {
public:
    double calculateTax(vector<vector<int>>& brackets, int income) {
        double ans = 0;
        int prev = 0;

        for (const vector<int>& b : brackets) {
            const int upper = b[0];
            const int percent = b[1];
            if (income < upper)
                return ans + (income - prev) * percent / 100.0;
            ans += (upper - prev) * percent / 100.0;
            prev = upper;
        }

        return ans;
    }
};
```

1413. 2304.cpp

Path: LeetCode-main/solutions/2304. Minimum Path Cost in a Grid/2304.cpp

```
class Solution {
public:
    int minPathCost(vector<vector<int>>& grid, vector<vector<int>>& moveCost) {
        const int m = grid.size();
        const int n = grid[0].size();
        // dp[i][j] := the minimum cost to reach grid[i][j]
        vector<vector<int>> dp(m, vector<int>(n, INT_MAX));
        dp[0] = grid[0];

        for (int i = 1; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int k = 0; k < n; ++k)
                    dp[i][j] = min(dp[i][j], dp[i - 1][k] + moveCost[grid[i - 1][k]][j] +
                                     grid[i][j]);

        return ranges::min(dp.back());
    }
};
```

1414. 2305.cpp

Path: LeetCode-main/solutions/2305. Fair Distribution of Cookies/2305.cpp

```
class Solution {
public:
    int distributeCookies(vector<int>& cookies, int k) {
        int ans = INT_MAX;
        dfs(cookies, 0, k, vector<int>(k), ans);
        return ans;
    }

private:
    void dfs(const vector<int>& cookies, int s, int k, vector<int>&& children,
             int& ans) {
        if (s == cookies.size()) {
            ans = min(ans, ranges::max(children));
            return;
        }

        for (int i = 0; i < k; ++i) {
            children[i] += cookies[s];
            dfs(cookies, s + 1, k, std::move(children), ans);
            children[i] -= cookies[s];
        }
    }
};
```

1415. 2306.cpp

Path: LeetCode-main/solutions/2306. Naming a Company/2306.cpp

```
class Solution {
public:
    long long distinctNames(vector<string>& ideas) {
        long ans = 0;
        // suffixes[i] := the set of strings omitting the first letter, where the
        // first letter is ('a' + i)
        vector<unordered_set<string>> suffixes(26);

        for (const string& idea : ideas)
            suffixes[idea[0] - 'a'].insert(idea.substr(1));

        for (int i = 0; i < 25; ++i)
            for (int j = i + 1; j < 26; ++j) {
                int count = 0;
                for (const string& suffix : suffixes[i])
                    if (suffixes[j].contains(suffix))
                        ++count;
                ans += 2 * (suffixes[i].size() - count) * (suffixes[j].size() - count);
            }

        return ans;
    }
};
```

1416. 2307.cpp

Path: LeetCode-main/solutions/2307. Check for Contradictions in Equations/2307.cpp

```
class Solution {
public:
    bool checkContradictions(vector<vector<string>>& equations,
                             vector<double>& values) {
        // Convert `string` to `int` for a better performance.
        unordered_map<string, int> strToInt;

        for (const vector<string>& equation : equations) {
            const string& u = equation[0];
            const string& v = equation[1];
            if (!strToInt.contains(u))
                strToInt[u] = strToInt.size();
            if (!strToInt.contains(v))
                strToInt[v] = strToInt.size();
        }

        vector<vector<pair<int, double>>> graph(strToInt.size());
        vector<double> seen(graph.size());

        for (int i = 0; i < equations.size(); ++i) {
            const int u = strToInt.at(equations[i][0]);
            const int v = strToInt.at(equations[i][1]);
            graph[u].emplace_back(v, values[i]);
            graph[v].emplace_back(u, 1 / values[i]);
        }

        for (int i = 0; i < graph.size(); ++i)
            if (!seen[i] && dfs(graph, i, seen, 1.0))
                return true;

        return false;
    }

private:
    bool dfs(const vector<vector<pair<int, double>>>& graph, int u,
             vector<double>& seen, double val) {
        if (seen[u])
            return abs(val / seen[u] - 1) > 1e-5;

        seen[u] = val;
        for (const auto& [v, w] : graph[u])
            if (dfs(graph, v, seen, val / w))
                return true;

        return false;
    }
};
```

1417. 2309.cpp

Path: LeetCode-main/solutions/2309. Greatest English Letter in Upper and Lower Case/2309.cpp

```
class Solution {
public:
    string greatestLetter(string s) {
        vector<bool> seen(128);

        for (const char c : s)
            seen[c] = true;

        for (int i = 25; i >= 0; --i)
            if (seen['a' + i] && seen['A' + i])
                return string(1, 'A' + i);

        return "";
    }
};
```


1418. 231.cpp

Path: LeetCode-main/solutions/231. Power of Two/231.cpp

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        return n >= 0 && __builtin_popcount(n) == 1;
    }
};
```

1419. 2310.cpp

Path: LeetCode-main/solutions/2310. Sum of Numbers With Units Digit K/2310.cpp

```
class Solution {
public:
    int minimumNumbers(int num, int k) {
        if (num == 0)
            return 0;

        // Assume the size of the set is n, and the numbers in the set are X1, X2,
        // ..., Xn. Since the units digit of each number is k,  $X_1 + X_2 + \dots + X_n =$ 
        //  $N * k + 10 * (x_1 + x_2 + \dots + x_n) = \text{num}$ . Therefore, the goal is to find
        // the n s.t.  $n * k \% 10 = \text{num} \% 10$ 
        for (int i = 1; i <= 10 && i * k <= num; ++i)
            if (i * k \% 10 == num \% 10)
                return i;

        return -1;
    }
};
```

1420. 2311.cpp

Path: LeetCode-main/solutions/2311. Longest Binary Subsequence Less Than or Equal to K/2311.cpp

```
class Solution {
public:
    int longestSubsequence(string s, int k) {
        int oneCount = 0;
        int num = 0;
        int pow = 1;

        // Take as many 1s as possible from the right.
        for (int i = s.length() - 1; i >= 0 && num + pow <= k; --i) {
            if (s[i] == '1') {
                ++oneCount;
                num += pow;
            }
            pow *= 2;
        }

        return ranges::count(s, '0') + oneCount;
    }
};
```

1421. 2312.cpp

Path: LeetCode-main/solutions/2312. Selling Pieces of Wood/2312.cpp

```
class Solution {
public:
    long long sellingWood(int m, int n, vector<vector<int>>& prices) {
        // dp[i][j] := the maximum money of cutting i x j piece of wood
        vector<vector<long>> dp(m + 1, vector<long>(n + 1));

        for (const vector<int>& p : prices) {
            const int h = p[0];
            const int w = p[1];
            const int price = p[2];
            dp[h][w] = price;
        }

        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j) {
                for (int h = 1; h <= i / 2; ++h)
                    dp[i][j] = max(dp[i][j], dp[h][j] + dp[i - h][j]);
                for (int w = 1; w <= j / 2; ++w)
                    dp[i][j] = max(dp[i][j], dp[i][w] + dp[i][j - w]);
            }

        return dp[m][n];
    }
};
```

1422. 2313.cpp

Path: LeetCode-main/solutions/2313. Minimum Flips in Binary Tree to Get Result/2313.cpp

```
class Solution {
public:
    int minimumFlips(TreeNode* root, bool result) {
        return dp(root, result);
    }

private:
    struct PairHash {
        template <class T1, class T2>
        std::size_t operator()(const std::pair<T1, T2>& p) const {
            return std::hash<T1>{}(p.first) ^ std::hash<T2>{}(p.second);
        }
    };

    unordered_map<pair<TreeNode*, bool>, int, PairHash> mem;

    // Returns the minimum flips to make the subtree become the target.
    int dp(TreeNode* root, bool target) {
        const pair<TreeNode*, bool> key{root, target};
        if (const auto it = mem.find(key); it != mem.cend())
            return it->second;
        if (root->val == 0 || root->val == 1) // the leaf
            return root->val == target ? 0 : 1;
        if (root->val == 5) // NOT
            return dp(root->left == nullptr ? root->right : root->left, !target);

        vector<pair<int, int>> nextTargets;
        if (root->val == 2) // OR
            nextTargets = target ? vector<pair<int, int>>{{0, 1}, {1, 0}, {1, 1}}
                                : vector<pair<int, int>>{{0, 0}};
        else if (root->val == 3) // AND
            nextTargets = target ? vector<pair<int, int>>{{1, 1}}
                                : vector<pair<int, int>>{{0, 0}, {0, 1}, {1, 0}};
        else // root.val == 4 (XOR)
            nextTargets = target ? vector<pair<int, int>>{{0, 1}, {1, 0}}
                                : vector<pair<int, int>>{{0, 0}, {1, 1}};

        int ans = INT_MAX;
        for (const auto& [leftTarget, rightTarget] : nextTargets)
            ans = min(ans, dp(root->left, leftTarget) + dp(root->right, rightTarget));
        return mem[key] = ans;
    }
};
```

1423. 2315.cpp

Path: LeetCode-main/solutions/2315. Count Asterisks/2315.cpp

```
class Solution {
public:
    int countAsterisks(string s) {
        int ans = 0;
        int bars = 0;

        for (const char c : s) {
            if (c == '|')
                ++bars;
            else if (c == '*' && bars % 2 == 0)
                ++ans;
        }

        return ans;
    }
};
```

1424. 2316.cpp

Path: LeetCode-main/solutions/2316. Count Unreachable Pairs of Nodes in an Undirected Graph/2316.cpp

```
class Solution {
public:
    long long countPairs(int n, vector<vector<int>>& edges) {
        long ans = 0;
        vector<vector<int>> graph(n);
        vector<bool> seen(n);
        int unreached = n;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        for (int i = 0; i < n; ++i) {
            const int reached = dfs(graph, i, seen);
            unreached -= reached;
            ans += static_cast<long>(unreached) * reached;
        }
        return ans;
    }

private:
    int dfs(const vector<vector<int>>& graph, int u, vector<bool>& seen) {
        if (seen[u])
            return 0;
        seen[u] = true;
        return accumulate(graph[u].begin(), graph[u].end(), 1, [&](int acc, int v) {
            return acc + dfs(graph, v, seen);
        });
    }
};
```

1425. 2317.cpp

Path: LeetCode-main/solutions/2317. Maximum XOR After Operations/2317.cpp

```
class Solution {
public:
    int maximumXOR(vector<int>& nums) {
        // 1. nums[i] & (nums[i] ^ x) enables you to turn 1-bit to 0-bit from
        //    nums[i] since x is arbitrary.
        // 2. The i-th bit of the XOR of all the elements is 1 if the i-th bit is 1
        //    for an odd number of elements.
        // 3. Therefore, the question is equivalent to: if you can convert any digit
        //    from 1 to 0 for any number, what is the maximum for XOR(nums[i]).
        // 4. The maximum we can get is of course to make every digit of the answer
        //    to be 1 if possible
        // 5. Therefore, OR(nums[i]) is an approach.
        return reduce(nums.begin(), nums.end(), 0, bit_or());
    }
};
```


1426. 2318.cpp

Path: LeetCode-main/solutions/2318. Number of Distinct Roll Sequences/2318.cpp

```
class Solution {
public:
    int distinctSequences(int n) {
        vector<vector<vector<int>>> mem(n + 1,
                                     vector<vector<int>>(7, vector<int>(7)));
        return distinctSequences(n, 0, 0, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of distinct sequences for n dices with `prev` and
    // `prevPrev`.
    int distinctSequences(int n, int prev, int prevPrev,
                        vector<vector<vector<int>>>& mem) {
        if (n == 0)
            return 1;
        if (mem[n][prev][prevPrev] > 0)
            return mem[n][prev][prevPrev];

        for (int dice = 1; dice <= 6; ++dice)
            if (dice != prev && dice != prevPrev &&
                (prev == 0 || gcd(dice, prev) == 1)) {
                mem[n][prev][prevPrev] += distinctSequences(n - 1, dice, prev, mem);
                mem[n][prev][prevPrev] %= kMod;
            }

        return mem[n][prev][prevPrev];
    }
};
```

1427. 2319.cpp

Path: LeetCode-main/solutions/2319. Check if Matrix Is X-Matrix/2319.cpp

```
class Solution {
public:
    bool checkXMatrix(vector<vector<int>>& grid) {
        const int n = grid.size();

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (i == j || i + j == n - 1) { // in diagonal
                    if (grid[i][j] == 0)
                        return false;
                } else if (grid[i][j]) { // not in diagonal
                    return false;
                }

        return true;
    }
};
```

1428. 232.cpp

Path: LeetCode-main/solutions/232. Implement Queue using Stacks/232.cpp

```
class MyQueue {
public:
    void push(int x) {
        input.push(x);
    }

    int pop() {
        peek();
        const int val = output.top();
        output.pop();
        return val;
    }

    int peek() {
        if (output.empty())
            while (!input.empty())
                output.push(input.top()), input.pop();
        return output.top();
    }

    bool empty() {
        return input.empty() && output.empty();
    }

private:
    stack<int> input;
    stack<int> output;
};
```

1429. 2320.cpp

Path: LeetCode-main/solutions/2320. Count Number of Ways to Place Houses/2320.cpp

```
class Solution {
public:
    int countHousePlacements(int n) {
        constexpr int kMod = 1'000'000'007;
        int house = 1; // the number of ways ending in a house
        int space = 1; // the number of ways ending in a space
        int total = house + space;

        for (int i = 2; i <= n; ++i) {
            house = space;
            space = total;
            total = (house + space) % kMod;
        }

        return static_cast<long>(total) * total % kMod;
    }
};
```

1430. 2321.cpp

Path: LeetCode-main/solutions/2321. Maximum Score Of Spliced Array/2321.cpp

```
class Solution {
public:
    int maximumSplicedArray(vector<int>& nums1, vector<int>& nums2) {
        return max(kadane(nums1, nums2), kadane(nums2, nums1));
    }

private:
    // Returns the maximum gain of swapping some numbers in `nums1` with some
    // numbers in `nums2`.
    int kadane(const vector<int>& nums1, const vector<int>& nums2) {
        int gain = 0;
        int maxGain = 0;

        for (int i = 0; i < nums1.size(); ++i) {
            gain = max(0, gain + nums2[i] - nums1[i]);
            maxGain = max(maxGain, gain);
        }

        return maxGain + accumulate(nums1.begin(), nums1.end(), 0);
    }
};
```

1431. 2322.cpp

Path: LeetCode-main/solutions/2322. Minimum Score After Removals on a Tree/2322.cpp

```
class Solution {
public:
    int minimumScore(vector<int>& nums, vector<vector<int>>& edges) {
        const int n = nums.size();
        const int xors = reduce(nums.begin(), nums.end(), 0, bit_xor());
        vector<int> subXors(nums);
        vector<vector<int>> tree(n);
        vector<unordered_set<int>> children(n);

        for (int i = 0; i < n; ++i)
            children[i].insert(i);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        dfs(tree, 0, -1, subXors, children);

        int ans = INT_MAX;

        for (int i = 0; i < edges.size(); ++i) {
            int a = edges[i][0];
            int b = edges[i][1];
            if (children[a].contains(b))
                swap(a, b);
            for (int j = 0; j < i; ++j) {
                int c = edges[j][0];
                int d = edges[j][1];
                if (children[c].contains(d))
                    swap(c, d);
                vector<int> cands;
                if (a != c && children[a].contains(c))
                    cands = {subXors[c], subXors[a] ^ subXors[c], xors ^ subXors[a]};
                else if (a != c && children[c].contains(a))
                    cands = {subXors[a], subXors[c] ^ subXors[a], xors ^ subXors[c]};
                else
                    cands = {subXors[a], subXors[c], xors ^ subXors[a] ^ subXors[c]};
                ans = min(ans, ranges::max(cands) - ranges::min(cands));
            }
        }

        return ans;
    }

private:
    pair<int, unordered_set<int>> dfs(const vector<vector<int>>& tree, int u,
                                     int prev, vector<int>& subXors,
                                     vector<unordered_set<int>>& children) {
        for (const int v : tree[u]) {
            if (v == prev)
                continue;
            const auto& [vXor, vChildren] = dfs(tree, v, u, subXors, children);
            subXors[u] ^= vXor;
            children[u].insert(vChildren.begin(), vChildren.end());
        }
        return {subXors[u], children[u]};
    }
};
```

1432. 2323.cpp

Path: LeetCode-main/solutions/2323. Find Minimum Time to Finish All Jobs II/2323.cpp

```
class Solution {
public:
    int minimumTime(vector<int>& jobs, vector<int>& workers) {
        int ans = 0;

        ranges::sort(jobs);
        ranges::sort(workers);

        for (int i = 0; i < jobs.size(); ++i)
            ans = max(ans, (jobs[i] - 1) / workers[i] + 1);

        return ans;
    }
};
```

1433. 2325.cpp

Path: LeetCode-main/solutions/2325. Decode the Message/2325.cpp

```
class Solution {
public:
    string decodeMessage(string key, string message) {
        vector<char> keyToActual(128);
        keyToActual[' ' ] = ' ';
        char currChar = 'a';

        for (const char c : key)
            keyToActual[c] = keyToActual[c] ? currChar++ : c;

        transform(message.cbegin(), message.cend(), message.begin(),
            [&](char c) { return keyToActual[c]; });

        return message;
    }
};
```


1434. 2326.cpp

Path: LeetCode-main/solutions/2326. Spiral Matrix IV/2326.cpp

```
class Solution {
public:
    vector<vector<int>> spiralMatrix(int m, int n, ListNode* head) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        vector<vector<int>> ans(m, vector<int>(n, -1));
        int x = 0; // the current x position
        int y = 0; // the current y position
        int d = 0;

        for (ListNode* curr = head; curr; curr = curr->next) {
            ans[x][y] = curr->val;
            if (x + kDirs[d][0] < 0 || x + kDirs[d][0] == m || y + kDirs[d][1] < 0 ||
                y + kDirs[d][1] == n || ans[x + kDirs[d][0]][y + kDirs[d][1]] != -1)
                d = (d + 1) % 4;
            x += kDirs[d][0];
            y += kDirs[d][1];
        }

        return ans;
    }
};
```

1435. 2327.cpp

Path: LeetCode-main/solutions/2327. Number of People Aware of a Secret/2327.cpp

```
class Solution {
public:
    int peopleAwareOfSecret(int n, int delay, int forget) {
        constexpr int kMod = 1'000'000'007;
        long share = 0;
        // dp[i] := the number of people know the secret at day i
        vector<int> dp(n); // Maps day i to i + 1.
        dp[0] = 1;

        for (int i = 1; i < n; ++i) {
            if (i - delay >= 0)
                share += dp[i - delay];
            if (i - forget >= 0)
                share -= dp[i - forget];
            share += kMod;
            share %= kMod;
            dp[i] = share;
        }

        // People before day `n - forget - 1` already forget the secret.
        return accumulate(dp.end() - forget, dp.end(), 0,
            [&](int acc, int d) { return (acc + d) % kMod; });
    }
};
```

1436. 2328.cpp

Path: LeetCode-main/solutions/2328. Number of Increasing Paths in a Grid/2328.cpp

```
class Solution {
public:
    int countPaths(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        int ans = 0;
        vector<vector<int>> mem(m, vector<int>(n, -1));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                ans += countPaths(grid, i, j, mem);
                ans %= kMod;
            }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns the number of increasing paths starting from (i, j).
    int countPaths(const vector<vector<int>>& grid, int i, int j,
        vector<vector<int>>& mem) {
        if (mem[i][j] != -1)
            return mem[i][j];

        mem[i][j] = 1; // The current cell contributes 1 length.

        for (const auto& [dx, dy] : kDirs) {
            const int x = i + dx;
            const int y = j + dy;
            if (x < 0 || x == grid.size() || y < 0 || y == grid[0].size())
                continue;
            if (grid[x][y] <= grid[i][j])
                continue;
            mem[i][j] += countPaths(grid, x, y, mem);
            mem[i][j] %= kMod;
        }

        return mem[i][j];
    }
};
```

1437. 233.cpp

Path: LeetCode-main/solutions/233. Number of Digit One/233.cpp

```
class Solution {
public:
    int countDigitOne(int n) {
        int ans = 0;

        for (long pow10 = 1; pow10 <= n; pow10 *= 10) {
            const long divisor = pow10 * 10;
            const int quotient = n / divisor;
            const int remainder = n % divisor;
            if (quotient > 0)
                ans += quotient * pow10;
            if (remainder >= pow10)
                ans += min(remainder - pow10 + 1, pow10);
        }

        return ans;
    }
};
```

1438. 2330.cpp

Path: LeetCode-main/solutions/2330. Valid Palindrome IV/2330.cpp

```
class Solution {
public:
    bool makePalindrome(string s) {
        int change = 0;
        int l = 0;
        int r = s.length() - 1;

        while (l < r) {
            if (s[l] != s[r] && ++change > 2)
                return false;
            ++l;
            --r;
        }

        return true;
    }
};
```

1439. 2331.cpp

Path: LeetCode-main/solutions/2331. Evaluate Boolean Binary Tree/2331.cpp

```
class Solution {
public:
    bool evaluateTree(TreeNode* root) {
        if (root->val < 2)
            return root->val;
        if (root->val == 2) // OR
            return evaluateTree(root->left) || evaluateTree(root->right);
        // AND
        return evaluateTree(root->left) && evaluateTree(root->right);
    }
};
```

1440. 2332.cpp

Path: LeetCode-main/solutions/2332. The Latest Time to Catch a Bus/2332.cpp

```
class Solution {
public:
    int latestTimeCatchTheBus(vector<int>& buses, vector<int>& passengers,
                             int capacity) {
        ranges::sort(buses);
        ranges::sort(passengers);

        if (passengers.front() > buses.back())
            return buses.back();

        int ans = passengers[0] - 1;
        int i = 0; // buses' index
        int j = 0; // passengers' index

        while (i < buses.size()) {
            // Greedily make passengers catch `buses[i]`.
            int arrived = 0;
            while (arrived < capacity && j < passengers.size() &&
                  passengers[j] <= buses[i]) {
                if (j > 0 && passengers[j] != passengers[j - 1] + 1)
                    ans = passengers[j] - 1;
                ++j;
                ++arrived;
            }
            // There's room for `buses[i]` to carry a passenger arriving at
            // `buses[i]`.
            if (arrived < capacity && j > 0 && passengers[j - 1] != buses[i])
                ans = buses[i];
            ++i;
        }
        return ans;
    }
};
```

1441. 2333.cpp

Path: LeetCode-main/solutions/2333. Minimum Sum of Squared Difference/2333.cpp

```
class Solution {
public:
    long long minSumSquareDiff(vector<int>& nums1, vector<int>& nums2, int k1,
                               int k2) {
        const vector<int> diff = getDiff(nums1, nums2);
        int k = k1 + k2;
        if (accumulate(diff.begin(), diff.end(), 0L) <= k)
            return 0;

        unordered_map<int, int> count;
        priority_queue<pair<int, int>> maxHeap; // (num, freq)

        for (const int d : diff)
            if (d != 0)
                ++count[d];

        for (const auto& [num, freq] : count)
            maxHeap.emplace(num, freq);

        while (k > 0) {
            const auto [maxNum, maxNumFreq] = maxHeap.top();
            maxHeap.pop();
            // Buck decrease in this turn
            const int numDecreased = min(k, maxNumFreq);
            k -= numDecreased;
            if (maxNumFreq > numDecreased)
                maxHeap.emplace(maxNum, maxNumFreq - numDecreased);
            if (!maxHeap.empty() && maxHeap.top().first + 1 == maxNum) {
                const auto [secondMaxNum, secondMaxNumFreq] = maxHeap.top();
                maxHeap.pop();
                maxHeap.emplace(secondMaxNum, secondMaxNumFreq + numDecreased);
            } else if (maxNum > 1) {
                maxHeap.emplace(maxNum - 1, numDecreased);
            }
        }

        long ans = 0;
        while (!maxHeap.empty()) {
            const auto [num, freq] = maxHeap.top();
            maxHeap.pop();
            ans += static_cast<long>(num) * num * freq;
        }

        return ans;
    }

private:
    vector<int> getDiff(const vector<int>& nums1, const vector<int>& nums2) {
        vector<int> diff;
        for (int i = 0; i < nums1.size(); ++i)
            diff.push_back(abs(nums1[i] - nums2[i]));
        return diff;
    }
};
```


1442. 2334.cpp

Path: LeetCode-main/solutions/2334. Subarray With Elements Greater Than Varying Threshold/2334.cpp

```
class Solution {
public:
    // Similar to 907. Sum of Subarray Minimums
    int validSubarraySize(vector<int>& nums, int threshold) {
        const int n = nums.size();
        // prev[i] := the index k s.t. nums[k] is the previous minimum in nums[0..n)
        vector<int> prev(n, -1);
        // next[i] := the index k s.t. nums[k] is the next minimum in nums[i + 1..n)
        vector<int> next(n, n);
        stack<int> stack;

        for (int i = 0; i < n; ++i) {
            while (!stack.empty() && nums[stack.top()] > nums[i]) {
                const int index = stack.top();
                stack.pop();
                next[index] = i;
            }
            if (!stack.empty())
                prev[i] = stack.top();
            stack.push(i);
        }

        for (int i = 0; i < n; ++i) {
            // the number of `nums` in subarray containing nums[i] >= nums[i]
            const int k = (i - prev[i]) + (next[i] - i) - 1;
            if (nums[i] > threshold / static_cast<double>(k))
                return k;
        }

        return -1;
    }
};
```

1443. 2335.cpp

Path: LeetCode-main/solutions/2335. Minimum Amount of Time to Fill Cups/2335.cpp

```
class Solution {
public:
    int fillCups(vector<int>& amount) {
        const int mx = ranges::max(amount);
        const int sum = accumulate(amount.begin(), amount.end(), 0);
        return max(mx, (sum + 1) / 2);
    }
};
```

1444. 2336.cpp

Path: LeetCode-main/solutions/2336. Smallest Number in Infinite Set/2336.cpp

```
class SmallestInfiniteSet {
public:
    int popSmallest() {
        if (added.empty())
            return curr++;
        const int mn = *added.begin();
        added.erase(added.begin());
        return mn;
    }

    void addBack(int num) {
        if (num < curr)
            added.insert(num);
    }

private:
    int curr = 1;
    set<int> added;
};
```

1445. 2337.cpp

Path: LeetCode-main/solutions/2337. Move Pieces to Obtain a String/2337.cpp

```
class Solution {
public:
    bool canChange(string start, string target) {
        const int n = start.length();
        int i = 0; // start's index
        int j = 0; // target's index

        while (i <= n && j <= n) {
            while (i < n && start[i] == '_')
                ++i;
            while (j < n && target[j] == '_')
                ++j;
            if (i == n || j == n)
                return i == n && j == n;
            if (start[i] != target[j])
                return false;
            if (start[i] == 'R' && i > j)
                return false;
            if (start[i] == 'L' && i < j)
                return false;
            ++i;
            ++j;
        }
        return true;
    }
};
```

1446. 2338.cpp

Path: LeetCode-main/solutions/2338. Count the Number of Ideal Arrays/2338.cpp

```
class Solution {
public:
    int idealArrays(int n, int maxValue) {
        // Since  $2^{14} > 10^4$ , the longest strictly increasing array is [1, 2, 4,
        // ...,  $2^{13}$ ]
        const int maxLength = min(14, n);
        const vector<vector<int>> factors = getFactors(maxValue);
        // dp[i][j] := the number of strictly increasing ideal arrays of length i
        // ending in j
        // dp[i][j] := sum(dp[i - 1][k]), where  $j \% k == 0$ 
        // dp[i][0] := sum(dp[i][j]) where  $1 \leq j \leq \text{maxValue}$ 
        vector<vector<long>> dp(maxLength + 1, vector<long>(maxValue + 1));
        vector<vector<long>> mem(n, vector<long>(maxLength, -1));
        long ans = 0;

        for (int j = 1; j <= maxValue; ++j)
            dp[1][j] = 1;

        for (int i = 2; i <= maxLength; ++i)
            for (int j = 1; j <= maxValue; ++j)
                for (const int k : factors[j]) {
                    dp[i][j] += dp[i - 1][k];
                    dp[i][j] %= kMod;
                }

        for (int i = 1; i <= maxLength; ++i)
            for (int j = 1; j <= maxValue; ++j) {
                dp[i][0] += dp[i][j];
                dp[i][0] %= kMod;
            }

        for (int i = 1; i <= maxLength; ++i) {
            // nCk(n - 1, i - 1) := the number of ways to create an ideal array of
            // length n from a strictly increasing array of length i
            ans += dp[i][0] * nCk(n - 1, i - 1, mem);
            ans %= kMod;
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    vector<vector<int>> getFactors(int maxValue) {
        vector<vector<int>> factors(maxValue + 1);
        for (int i = 1; i <= maxValue; ++i)
            // Start from i * 2 because of strictly increasing.
            for (int j = i * 2; j <= maxValue; j += i)
                factors[j].push_back(i);
        return factors;
    }

    long nCk(int n, int k, vector<vector<long>>& mem) {
        if (k == 0)
            return 1;
        if (n == k)
            return 1;
        if (mem[n][k] != -1)
            return mem[n][k];
        return mem[n][k] = (nCk(n - 1, k, mem) + nCk(n - 1, k - 1, mem)) % kMod;
    }
};
```

1447. 234.cpp

Path: LeetCode-main/solutions/234. Palindrome Linked List/234.cpp

```
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }

        if (fast != nullptr)
            slow = slow->next;
        slow = reverseList(slow);

        while (slow != nullptr) {
            if (slow->val != head->val)
                return false;
            slow = slow->next;
            head = head->next;
        }

        return true;
    }

private:
    ListNode* reverseList(ListNode* head) {
        ListNode* prev = nullptr;
        while (head != nullptr) {
            ListNode* next = head->next;
            head->next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }
};
```

1448. 2340.cpp

Path: LeetCode-main/solutions/2340. Minimum Adjacent Swaps to Make a Valid Array/2340.cpp

```
class Solution {
public:
    int minimumSwaps(vector<int>& nums) {
        const int minIndex = getLeftmostMinIndex(nums);
        const int maxIndex = getRightmostMaxIndex(nums);
        const int swaps = minIndex + (nums.size() - 1 - maxIndex);
        return minIndex <= maxIndex ? swaps : swaps - 1;
    }

private:
    int getLeftmostMinIndex(const vector<int>& nums) {
        int mn = nums.front();
        int minIndex = 0;
        for (int i = 1; i < nums.size(); ++i)
            if (nums[i] < mn) {
                mn = nums[i];
                minIndex = i;
            }
        return minIndex;
    }

    int getRightmostMaxIndex(const vector<int>& nums) {
        int mx = nums.back();
        int maxIndex = nums.size() - 1;
        for (int i = nums.size() - 2; i >= 0; --i)
            if (nums[i] > mx) {
                mx = nums[i];
                maxIndex = i;
            }
        return maxIndex;
    }
};
```

1449. 2341.cpp

Path: LeetCode-main/solutions/2341. Maximum Number of Pairs in Array/2341.cpp

```
class Solution {
public:
    vector<int> numberOfPairs(vector<int>& nums) {
        constexpr int kMax = 100;
        vector<int> ans(2);
        vector<int> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        for (int i = 0; i <= kMax; ++i) {
            ans[0] += count[i] / 2;
            ans[1] += count[i] & 1;
        }

        return ans;
    }
};
```


1450. 2342.cpp

Path: LeetCode-main/solutions/2342. Max Sum of a Pair With Equal Sum of Digits/2342.cpp

```
class Solution {
public:
    int maximumSum(vector<int>& nums) {
        constexpr int kMax = 9 * 9; // 999,999,999
        int ans = -1;
        vector<vector<int>> count(kMax + 1);

        for (const int num : nums)
            count[getDigitSum(num)].push_back(num);

        for (vector<int>& groupNums : count) {
            if (groupNums.size() < 2)
                continue;
            ranges::sort(groupNums, greater<>());
            ans = max(ans, groupNums[0] + groupNums[1]);
        }

        return ans;
    }

private:
    int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
};
```

1451. 2343.cpp

Path: LeetCode-main/solutions/2343. Query Kth Smallest Trimmed Number/2343.cpp

```
class Solution {
public:
    vector<int> smallestTrimmedNumbers(vector<string>& nums,
                                      vector<vector<int>>& queries) {
        vector<int> ans;
        for (const vector<int>& query : queries)
            ans.push_back(getKSmallestIndex(nums, query[0], query[1]));
        return ans;
    }

private:
    int getKSmallestIndex(const vector<string>& nums, int k, int trim) {
        const int startIndex = nums[0].length() - trim;
        vector<pair<string, int>> trimmed; // (num, index)

        for (int i = 0; i < nums.size(); ++i) {
            const string& num = nums[i];
            trimmed.push_back({num.substr(startIndex), i});
        }

        ranges::sort(trimmed);
        return trimmed[k - 1].second;
    }
};
```

1452. 2344.cpp

Path: LeetCode-main/solutions/2344. Minimum Deletions to Make Array Divisible/2344.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums, vector<int>& numsDivide) {
        const int gcd = getGCD(numsDivide);

        ranges::sort(nums);

        for (int i = 0; i < nums.size(); ++i)
            if (gcd % nums[i] == 0)
                return i;

        return -1;
    }

private:
    int getGCD(const vector<int>& nums) {
        int gcd = nums[0];
        for (const int num : nums)
            gcd = __gcd(gcd, num);
        return gcd;
    }
};
```

1453. 2345.cpp

Path: LeetCode-main/solutions/2345. Finding the Number of Visible Mountains/2345.cpp

```
class Solution {
public:
    int visibleMountains(vector<vector<int>>& peaks) {
        int ans = 0;
        int maxRightFoot = 0;

        ranges::sort(peaks, ranges::less{}, [](const vector<int>& peak) {
            return pair<int, int>(peak[0] - peak[1], -peak[0]);
        });

        for (int i = 0; i < peaks.size(); ++i) {
            const bool overlapWithNext =
                i + 1 < peaks.size() && peaks[i] == peaks[i + 1];
            const int currRightFoot = peaks[i][0] + peaks[i][1];
            if (currRightFoot > maxRightFoot) {
                if (!overlapWithNext)
                    ++ans;
                maxRightFoot = currRightFoot;
            }
        }

        return ans;
    }
};
```

1454. 2347.cpp

Path: LeetCode-main/solutions/2347. Best Poker Hand/2347.cpp

```
class Solution {
public:
    string bestHand(vector<int>& ranks, vector<char>& suits) {
        if (ranges::all_of(suits,
                           [&suits](const char suit) { return suit == suits[0]; }))
            return "Flush";

        constexpr int kMax = 13;
        vector<int> count(kMax + 1);

        for (const int rank : ranks)
            ++count[rank];

        const int mx = ranges::max(count);
        if (mx > 2)
            return "Three of a Kind";
        if (mx == 2)
            return "Pair";
        return "High Card";
    }
};
```

1455. 2348.cpp

Path: LeetCode-main/solutions/2348. Number of Zero-Filled Subarrays/2348.cpp

```
class Solution {
public:
    long long zeroFilledSubarray(vector<int>& nums) {
        long ans = 0;
        int indexBeforeZero = -1;

        for (int i = 0; i < nums.size(); ++i)
            if (nums[i])
                indexBeforeZero = i;
            else
                ans += i - indexBeforeZero;

        return ans;
    }
};
```

1456. 2349.cpp

Path: LeetCode-main/solutions/2349. Design a Number Container System/2349.cpp

```
class NumberContainers {
public:
    void change(int index, int number) {
        const auto it = indexToNumber.find(index);
        if (it != indexToNumber.cend()) {
            const int originalNumber = it->second;
            auto& indices = numberToIndices[originalNumber];
            indices.erase(index);
            if (indices.empty())
                numberToIndices.erase(originalNumber);
        }
        numberToIndices[number].insert(index);
    }

    int find(int number) {
        const auto it = numberToIndices.find(number);
        if (it == numberToIndices.cend())
            return -1;
        const auto& indices = it->second;
        return *indices.begin();
    }

private:
    unordered_map<int, int> indexToNumber;
    unordered_map<int, set<int>> numberToIndices;
};
```

1457. 235.cpp

Path: LeetCode-main/solutions/235. Lowest Common Ancestor of a Binary Search Tree/235.cpp

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root->val > max(p->val, q->val))
            return lowestCommonAncestor(root->left, p, q);
        if (root->val < min(p->val, q->val))
            return lowestCommonAncestor(root->right, p, q);
        return root;
    }
};
```


1458. 2350.cpp

Path: LeetCode-main/solutions/2350. Shortest Impossible Sequence of Rolls/2350.cpp

```
class Solution {
public:
    int shortestSequence(vector<int>& rolls, int k) {
        int ans = 1; // the the next target length
        unordered_set<int> seen;

        for (const int roll : rolls) {
            seen.insert(roll);
            if (seen.size() == k) {
                // Have all combinations that form `ans` length, and we are going to
                // extend the sequence to `ans + 1` length.
                ++ans;
                seen.clear();
            }
        }

        return ans;
    }
};
```

1459. 2351.cpp

Path: LeetCode-main/solutions/2351. First Letter to Appear Twice/2351.cpp

```
class Solution {
public:
    char repeatedCharacter(string s) {
        vector<bool> seen(26);

        for (const char c : s) {
            if (seen[c - 'a'])
                return c;
            seen[c - 'a'] = true;
        }

        throw;
    }
};
```

1460. 2352.cpp

Path: LeetCode-main/solutions/2352. Equal Row and Column Pairs/2352.cpp

```
class Solution {
public:
    int equalPairs(vector<vector<int>>& grid) {
        const int n = grid.size();
        int ans = 0;

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                int k = 0;
                for (; k < n; ++k)
                    if (grid[i][k] != grid[k][j])
                        break;
                if (k == n) // R[i] == C[j]
                    ++ans;
            }

        return ans;
    }
};
```

1461. 2353.cpp

Path: LeetCode-main/solutions/2353. Design a Food Rating System/2353.cpp

```
class FoodRatings {
public:
    FoodRatings(vector<string>& foods, vector<string>& cuisines,
                vector<int>& ratings) {
        for (int i = 0; i < foods.size(); ++i) {
            cuisineToRatingAndFoods[cuisines[i]].insert({-ratings[i], foods[i]});
            foodToCuisine[foods[i]] = cuisines[i];
            foodToRating[foods[i]] = ratings[i];
        }
    }

    void changeRating(string food, int newRating) {
        const string cuisine = foodToCuisine[food];
        const int oldRating = foodToRating[food];
        auto& ratingAndFoods = cuisineToRatingAndFoods[cuisine];
        ratingAndFoods.erase({-oldRating, food});
        ratingAndFoods.insert({-newRating, food});
        foodToRating[food] = newRating;
    }

    string highestRated(string cuisine) {
        return cuisineToRatingAndFoods[cuisine].begin()->second;
    }

private:
    // {cuisine: {(-rating, food)}} stores negative rating for smarter comparison
    unordered_map<string, set<pair<int, string>>> cuisineToRatingAndFoods;
    unordered_map<string, string> foodToCuisine;
    unordered_map<string, int> foodToRating;
};
```

1462. 2354.cpp

Path: LeetCode-main/solutions/2354. Number of Excellent Pairs/2354.cpp

```
class Solution {
public:
    long long countExcellentPairs(vector<int>& nums, int k) {
        constexpr int kMaxBit = 30;
        // bits(num1 | num2) + bits(num1 & num2) = bits(num1) + bits(num2)
        long ans = 0;
        vector<long> count(kMaxBit);

        for (const unsigned num : unordered_set<int>(nums.begin(), nums.end()))
            ++count[popcount(num)];

        for (int i = 0; i < kMaxBit; ++i)
            for (int j = 0; j < kMaxBit; ++j)
                if (i + j >= k)
                    ans += count[i] * count[j];

        return ans;
    }
};
```

1463. 2355.cpp

Path: LeetCode-main/solutions/2355. Maximum Number of Books You Can Take/2355.cpp

```
class Solution {
public:
    long long maximumBooks(vector<int>& books) {
        // dp[i] := the maximum number of books we can take from books[0..i] with
        // taking all of books[i]
        vector<long> dp(books.size());
        stack<int> stack; // the possible indices we can reach

        for (int i = 0; i < books.size(); ++i) {
            // We may take all of books[j], where books[j] < books[i] - (i - j).
            while (!stack.empty() &&
                books[stack.top()] >= books[i] - (i - stack.top()))
                stack.pop();
            // We can now take books[j + 1..i].
            const int j = stack.empty() ? -1 : stack.top();
            const int lastTook = books[i] - (i - j) + 1;
            if (lastTook > 1)
                // books[i] + (books[i] - 1) + ... + (books[i] - (i - j) + 1)
                dp[i] = static_cast<long>(books[i] + lastTook) * (i - j) / 2;
            else
                // 1 + 2 + ... + books[i]
                dp[i] = static_cast<long>(books[i]) * (books[i] + 1) / 2;
            if (j >= 0)
                dp[i] += dp[j];
            stack.push(i);
        }

        return ranges::max(dp);
    }
};
```

1464. 2357.cpp

Path: LeetCode-main/solutions/2357. Make Array Zero by Subtracting Equal Amounts/2357.cpp

```
class Solution {
public:
    int minimumOperations(vector<int>& nums) {
        unordered_set<int> seen(nums.begin(), nums.end());
        return seen.size() - seen.contains(0);
    }
};
```

1465. 2358.cpp

Path: LeetCode-main/solutions/2358. Maximum Number of Groups Entering a Competition/2358.cpp

```
class Solution {
public:
    int maximumGroups(vector<int>& grades) {
        // Sort grades, then we can separate the students into groups of sizes 1, 2,
        // 3, ..., k, s.t. the i-th group < the (i + 1)-th group for both sum and
        // size. So, we can rephrase the problem into:
        // Find the maximum k s.t.  $1 + 2 + 3 + \dots + k \leq n$ 

        //  $1 + 2 + 3 + \dots + k \leq n$ 
        //  $k(k + 1) / 2 \leq n$ 
        //  $k^2 + k \leq 2n$ 
        //  $(k + 0.5)^2 - 0.25 \leq 2n$ 
        //  $(k + 0.5)^2 \leq 2n + 0.25$ 
        //  $k \leq \sqrt{2n + 0.25} - 0.5$ 
        return sqrt(grades.size() * 2 + 0.25) - 0.5;
    }
};
```


1466. 2359.cpp

Path: LeetCode-main/solutions/2359. Find Closest Node to Given Two Nodes/2359.cpp

```
class Solution {
public:
    int closestMeetingNode(vector<int>& edges, int node1, int node2) {
        constexpr int kMax = 10000;
        const vector<int> dist1 = getDist(edges, node1);
        const vector<int> dist2 = getDist(edges, node2);
        int minDist = kMax;
        int ans = -1;

        for (int i = 0; i < edges.size(); ++i)
            if (min(dist1[i], dist2[i]) >= 0) {
                const int maxDist = max(dist1[i], dist2[i]);
                if (maxDist < minDist) {
                    minDist = maxDist;
                    ans = i;
                }
            }

        return ans;
    }

private:
    vector<int> getDist(const vector<int>& edges, int u) {
        vector<int> dist(edges.size(), -1);
        int d = 0;
        while (u != -1 && dist[u] == -1) {
            dist[u] = d++;
            u = edges[u];
        }
        return dist;
    }
};
```

1467. 236-2.cpp

Path: LeetCode-main/solutions/236. Lowest Common Ancestor of a Binary Tree/236-2.cpp

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        queue
```

1468. 236.cpp

Path: LeetCode-main/solutions/236. Lowest Common Ancestor of a Binary Tree/236.cpp

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root == nullptr || root == p || root == q)
            return root;
        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);
        if (left != nullptr && right != nullptr)
            return root;
        return left == nullptr ? right : left;
    }
};
```

1469. 2360.cpp

Path: LeetCode-main/solutions/2360. Longest Cycle in a Graph/2360.cpp

```
class Solution {
public:
    int longestCycle(vector<int>& edges) {
        int ans = -1;
        int time = 1;
        vector<int> timeVisited(edges.size());

        for (int i = 0; i < edges.size(); ++i) {
            if (timeVisited[i])
                continue;
            const int startTime = time;
            int u = i;
            while (u != -1 && !timeVisited[u]) {
                timeVisited[u] = time++;
                u = edges[u]; // Move to the next node.
            }
            if (u != -1 && timeVisited[u] >= startTime)
                ans = max(ans, time - timeVisited[u]);
        }

        return ans;
    }
};
```

1470. 2361.cpp

Path: LeetCode-main/solutions/2361. Minimum Costs Using the Train Line/2361.cpp

```
class Solution {
public:
    vector<long long> minimumCosts(vector<int>& regular, vector<int>& express,
                                   int expressCost) {
        const int n = regular.size();
        vector<long long> ans(n);
        // the minimum cost to reach the current stop in a regular route
        long dpReg = 0;
        // the minimum cost to reach the current stop in an express route
        long dpExp = expressCost;

        for (int i = 0; i < n; ++i) {
            const long prevReg = dpReg;
            const long prevExp = dpExp;
            dpReg = min(prevReg + regular[i], prevExp + 0 + regular[i]);
            dpExp = min(prevReg + expressCost + express[i], prevExp + express[i]);
            ans[i] = min(dpReg, dpExp);
        }

        return ans;
    }
};
```

1471. 2363.cpp

Path: LeetCode-main/solutions/2363. Merge Similar Items/2363.cpp

```
class Solution {
public:
    vector<vector<int>> mergeSimilarItems(vector<vector<int>>& items1,
                                         vector<vector<int>>& items2) {
        constexpr int kMax = 1000;
        vector<vector<int>> ans;
        vector<int> count(kMax + 1);

        for (const vector<int>& item : items1)
            count[item[0]] += item[1];

        for (const vector<int>& item : items2)
            count[item[0]] += item[1];

        for (int i = 1; i <= kMax; ++i)
            if (count[i])
                ans.push_back({i, count[i]});

        return ans;
    }
};
```

1472. 2364.cpp

Path: LeetCode-main/solutions/2364. Count Number of Bad Pairs/2364.cpp

```
class Solution {
public:
    long long countBadPairs(vector<int>& nums) {
        long ans = 0;
        unordered_map<int, int> count; // (nums[i] - i)

        for (int i = 0; i < nums.size(); ++i)
            // count[nums[i] - i] := the number of good pairs
            // i - count[nums[i] - i] := the number of bad pairs
            ans += i - count[nums[i] - i]++;

        return ans;
    }
};
```

1473. 2365.cpp

Path: LeetCode-main/solutions/2365. Task Scheduler II/2365.cpp

```
class Solution {
public:
    long long taskSchedulerII(vector<int>& tasks, int space) {
        unordered_map<int, long> taskToNextAvailable;
        long ans = 0;

        for (const int task : tasks) {
            ans = max(ans + 1, taskToNextAvailable[task]);
            taskToNextAvailable[task] = ans + space + 1;
        }

        return ans;
    }
};
```


1474. 2366.cpp

Path: LeetCode-main/solutions/2366. Minimum Replacements to Sort the Array/2366.cpp

```
class Solution {
public:
    long long minimumReplacement(vector<int>& nums) {
        long ans = 0;
        int mx = nums.back();

        for (int i = nums.size() - 2; i >= 0; --i) {
            const int ops = (nums[i] - 1) / mx;
            ans += ops;
            mx = nums[i] / (ops + 1);
        }

        return ans;
    }
};
```

1475. 2367.cpp

Path: LeetCode-main/solutions/2367. Number of Arithmetic Triplets/2367.cpp

```
class Solution {
public:
    int arithmeticTriplets(vector<int>& nums, int diff) {
        constexpr int kMax = 200;
        int ans = 0;
        vector<bool> count(kMax + 1);

        for (const int num : nums) {
            if (num >= 2 * diff && count[num - diff] && count[num - 2 * diff])
                ++ans;
            count[num] = true;
        }

        return ans;
    }
};
```

1476. 2368.cpp

Path: LeetCode-main/solutions/2368. Reachable Nodes With Restrictions/2368.cpp

```
class Solution {
public:
    int reachableNodes(int n, vector<vector<int>>& edges,
                      vector<int>& restricted) {
        vector<vector<int>> tree(n);
        vector<bool> seen(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        for (const int r : restricted)
            seen[r] = true;

        return dfs(tree, 0, seen);
    }

private:
    int dfs(const vector<vector<int>>& tree, int u, vector<bool>& seen) {
        if (seen[u])
            return 0;

        seen[u] = true;
        int ans = 1;

        for (const int v : tree[u])
            ans += dfs(tree, v, seen);

        return ans;
    }
};
```

1477. 2369.cpp

Path: LeetCode-main/solutions/2369. Check if There is a Valid Partition For The Array/2369.cpp

```
class Solution {
public:
    bool validPartition(vector<int>& nums) {
        const int n = nums.size();
        // dp[i] := true if there's a valid partition for the first i numbers
        vector<bool> dp(n + 1);
        dp[0] = true;
        dp[2] = nums[0] == nums[1];

        for (int i = 3; i <= n; ++i)
            dp[i] = (dp[i - 2] && nums[i - 2] == nums[i - 1]) ||
                    (dp[i - 3] &&
                     ((nums[i - 3] == nums[i - 2] && nums[i - 2] == nums[i - 1]) ||
                      (nums[i - 3] + 1 == nums[i - 2] &&
                       nums[i - 2] + 1 == nums[i - 1]))));

        return dp[n];
    }
};
```

1478. 237.cpp

Path: LeetCode-main/solutions/237. Delete Node in a Linked List/237.cpp

```
class Solution {
public:
    void deleteNode(ListNode* node) {
        node->val = node->next->val;
        node->next = node->next->next;
    }
};
```

1479. 2370.cpp

Path: LeetCode-main/solutions/2370. Longest Ideal Subsequence/2370.cpp

```
class Solution {
public:
    int longestIdealString(string s, int k) {
        // dp[i] := the longest subsequence that ends in ('a' + i)
        vector<int> dp(26);

        for (const char c : s) {
            const int i = c - 'a';
            dp[i] = 1 + getMaxReachable(dp, i, k);
        }

        return ranges::max(dp);
    }

private:
    int getMaxReachable(const vector<int>& dp, int i, int k) {
        const int first = max(0, i - k);
        const int last = min(25, i + k);
        int maxReachable = 0;
        for (int j = first; j <= last; ++j)
            maxReachable = max(maxReachable, dp[j]);
        return maxReachable;
    }
};
```

1480. 2371.cpp

Path: LeetCode-main/solutions/2371. Minimize Maximum Value in a Grid/2371.cpp

```
class Solution {
public:
    vector<vector<int>> minScore(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<int>> ans(m, vector<int>(n));
        vector<array<int, 3>> valAndIndices;
        vector<int> rows(m); // rows[i] := the maximum used number so far
        vector<int> cols(n); // cols[j] := the maximum used number so far

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                valAndIndices.push_back({grid[i][j], i, j});

        ranges::sort(valAndIndices);

        for (const auto& [_ , i, j] : valAndIndices) {
            const int nextAvailable = max(rows[i], cols[j]) + 1;
            ans[i][j] = nextAvailable;
            rows[i] = nextAvailable;
            cols[j] = nextAvailable;
        }

        return ans;
    }
};
```

1481. 2373.cpp

Path: LeetCode-main/solutions/2373. Largest Local Values in a Matrix/2373.cpp

```
class Solution {
public:
    vector<vector<int>> largestLocal(vector<vector<int>>& grid) {
        const int n = grid.size();
        vector<vector<int>> ans(n - 2, vector<int>(n - 2));

        for (int i = 0; i < n - 2; ++i)
            for (int j = 0; j < n - 2; ++j)
                for (int x = i; x < i + 3; ++x)
                    for (int y = j; y < j + 3; ++y)
                        ans[i][j] = max(ans[i][j], grid[x][y]);

        return ans;
    }
};
```


1482. 2374.cpp

Path: LeetCode-main/solutions/2374. Node With Highest Edge Score/2374.cpp

```
class Solution {
public:
    int edgeScore(vector<int>& edges) {
        vector<long> scores(edges.size());
        for (int i = 0; i < edges.size(); ++i)
            scores[edges[i]] += i;
        return ranges::max_element(scores) - scores.begin();
    }
};
```

1483. 2375.cpp

Path: LeetCode-main/solutions/2375. Construct Smallest Number From DI String/2375.cpp

```
class Solution {
public:
    string smallestNumber(string pattern) {
        string ans;
        stack<char> stack{{'1'}};

        for (const char c : pattern) {
            char maxSorFar = stack.top();
            if (c == 'I')
                while (!stack.empty()) {
                    maxSorFar = max(maxSorFar, stack.top());
                    ans += stack.top(), stack.pop();
                }
            stack.push(maxSorFar + 1);
        }

        while (!stack.empty())
            ans += stack.top(), stack.pop();

        return ans;
    }
};
```

1484. 2376.cpp

Path: LeetCode-main/solutions/2376. Count Special Integers/2376.cpp

```
class Solution {
public:
    // Same as 1012. Numbers With Repeated Digits
    int countSpecialNumbers(int n) {
        const int digitSize = log10(n) + 1;
        vector<vector<vector<int>>> mem(
            digitSize + 1, vector<vector<int>>(1 << 10, vector<int>(2, -1)));
        return count(to_string(n), 0, 0, true, mem) - 1; // - 0;
    }

private:
    // Returns the number of special integers, considering the i-th digit, where
    // `used` is the bitmask of the used digits, and `tight` indicates if the
    // current digit is tightly bound.
    int count(const string& s, int i, int used, bool tight,
              vector<vector<vector<int>>>& mem) {
        if (i == s.length())
            return 1;
        if (mem[i][used][tight] != -1)
            return mem[i][used][tight];

        int res = 0;
        const int maxDigit = tight ? s[i] - '0' : 9;

        for (int d = 0; d <= maxDigit; ++d) {
            // `d` is used.
            if (used >> d & 1)
                continue;
            // Use `d` now.
            const bool nextTight = tight && (d == maxDigit);
            if (used == 0 && d == 0) // Don't count leading 0s as used.
                res += count(s, i + 1, used, nextTight, mem);
            else
                res += count(s, i + 1, used | 1 << d, nextTight, mem);
        }

        return mem[i][used][tight] = res;
    }
};
```

1485. 2378.cpp

Path: LeetCode-main/solutions/2378. Choose Edges to Maximize Score in a Tree/2378.cpp

```
class Solution {
public:
    long long maxScore(vector<vector<int>>& edges) {
        const int n = edges.size();
        vector<vector<pair<int, int>>> graph(n);

        for (int i = 0; i < n; ++i) {
            const int parent = edges[i][0];
            const int weight = edges[i][1];
            if (parent != -1)
                graph[parent].emplace_back(i, weight);
        }

        const auto [takeRoot, notTakeRoot] = dfs(graph, 0);
        return max(takeRoot, notTakeRoot);
    }

private:
    // Returns (the maximum sum at u if we take one u->v edge,
    //           the maximum sum at u if we don't take any child edge).
    pair<long, long> dfs(const vector<vector<pair<int, int>>& graph, int u) {
        long bestEdge = 0;
        long notTakeU = 0;

        for (const auto& [v, w] : graph[u]) {
            const auto [takeV, notTakeV] = dfs(graph, v);
            bestEdge = max(bestEdge, w + notTakeV - takeV);
            notTakeU += takeV;
        }

        return {bestEdge + notTakeU, notTakeU};
    }
};
```

1486. 2379.cpp

Path: LeetCode-main/solutions/2379. Minimum Recolors to Get K Consecutive Black Blocks/2379.cpp

```
class Solution {
public:
    int minimumRecolors(string blocks, int k) {
        int countB = 0;
        int maxCountB = 0;

        for (int i = 0; i < blocks.length(); ++i) {
            if (blocks[i] == 'B')
                ++countB;
            if (i >= k && blocks[i - k] == 'B')
                --countB;
            maxCountB = max(maxCountB, countB);
        }

        return k - maxCountB;
    }
};
```

1487. 238-2.cpp

Path: LeetCode-main/solutions/238. Product of Array Except Self/238-2.cpp

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans(n, 1);

        // Use ans as the prefix product array.
        for (int i = 1; i < n; ++i)
            ans[i] = ans[i - 1] * nums[i - 1];

        int suffix = 1; // suffix product
        for (int i = n - 1; i >= 0; --i) {
            ans[i] *= suffix;
            suffix *= nums[i];
        }

        return ans;
    }
};
```

1488. 238.cpp

Path: LeetCode-main/solutions/238. Product of Array Except Self/238.cpp

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans(n); // Can also use `nums` as the ans array.
        vector<int> prefix(n, 1); // prefix product
        vector<int> suffix(n, 1); // suffix product

        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i - 1] * nums[i - 1];

        for (int i = n - 2; i >= 0; --i)
            suffix[i] = suffix[i + 1] * nums[i + 1];

        for (int i = 0; i < n; ++i)
            ans[i] = prefix[i] * suffix[i];

        return ans;
    }
};
```

1489. 2380.cpp

Path: LeetCode-main/solutions/2380. Time Needed to Rearrange a Binary String/2380.cpp

```
class Solution {
public:
    int secondsToRemoveOccurrences(string s) {
        int ans = 0;
        int zeros = 0;

        for (const char c : s)
            if (c == '0')
                ++zeros;
            else if (zeros > 0) // c == '1'
                ans = max(ans + 1, zeros);

        return ans;
    }
};
```


1490. 2381.cpp

Path: LeetCode-main/solutions/2381. Shifting Letters II/2381.cpp

```
class Solution {
public:
    string shiftingLetters(string s, vector<vector<int>>& shifts) {
        int currShift = 0;
        vector<int> line(s.length() + 1);

        for (const vector<int>& shift : shifts) {
            const int start = shift[0];
            const int end = shift[1];
            const int direction = shift[2];
            const int diff = direction ? 1 : -1;
            line[start] += diff;
            line[end + 1] -= diff;
        }

        for (int i = 0; i < s.length(); ++i) {
            currShift = (currShift + line[i]) % 26;
            const int num = (s[i] - 'a' + currShift + 26) % 26;
            s[i] = 'a' + num;
        }

        return s;
    }
};
```

1491. 2382.cpp

Path: LeetCode-main/solutions/2382. Maximum Segment Sum After Removals/2382.cpp

```
class Solution {
public:
    vector<long long> maximumSegmentSum(vector<int>& nums,
                                       vector<int>& removeQueries) {
        const int n = nums.size();
        long maxSum = 0;
        vector<long long> ans(n);
        // For the segment [l, r], record its sum in sum[l] and sum[r]
        vector<long long> sum(n);
        // For the segment [l, r], record its count in count[l] and count[r]
        vector<int> count(n);

        for (int i = n - 1; i >= 0; --i) {
            ans[i] = maxSum;
            const int j = removeQueries[i];

            // Calculate `segmentSum`.
            const long leftSum = j > 0 ? sum[j - 1] : 0;
            const long rightSum = j + 1 < n ? sum[j + 1] : 0;
            const long segmentSum = nums[j] + leftSum + rightSum;

            // Calculate `segmentCount`.
            const int leftCount = j > 0 ? count[j - 1] : 0;
            const int rightCount = j + 1 < n ? count[j + 1] : 0;
            const int segmentCount = 1 + leftCount + rightCount;

            // Update the sum and count of the segment [l, r].
            const int l = j - leftCount;
            const int r = j + rightCount;
            sum[l] = segmentSum;
            sum[r] = segmentSum;
            count[l] = segmentCount;
            count[r] = segmentCount;
            maxSum = max(maxSum, segmentSum);
        }

        return ans;
    }
};
```

1492. 2383.cpp

Path: LeetCode-main/solutions/2383. Minimum Hours of Training to Win a Competition/2383.cpp

```
class Solution {
public:
    int minNumberOfHours(int initialEnergy, int initialExperience,
                        vector<int>& energy, vector<int>& experience) {
        return getRequiredEnergy(initialEnergy, energy) +
               getRequiredExperience(initialExperience, experience);
    }

private:
    int getRequiredEnergy(int initialEnergy, const vector<int>& energy) {
        return max(0,
                   accumulate(energy.begin(), energy.end(), 0) + 1 - initialEnergy);
    }

    int getRequiredExperience(int currentExperience,
                             const vector<int>& experience) {
        int requiredExperience = 0;
        for (const int e : experience) {
            if (e >= currentExperience) {
                requiredExperience += e + 1 - currentExperience;
                currentExperience += e + 1 - currentExperience;
            }
            currentExperience += e;
        }
        return requiredExperience;
    }
};
```

1493. 2384.cpp

Path: LeetCode-main/solutions/2384. Largest Palindromic Number/2384.cpp

```
class Solution {
public:
    string largestPalindromic(string num) {
        unordered_map<char, int> count;

        for (const char c : num)
            ++count[c];

        const string firstHalf = getFirstHalf(count);
        const string mid = getMid(count);
        const string ans = firstHalf + mid + reversed(firstHalf);
        return ans.empty() ? "0" : ans;
    }

private:
    string getFirstHalf(const unordered_map<char, int>& count) {
        string firstHalf;
        for (char c = '9'; c >= '0'; --c) {
            const auto it = count.find(c);
            if (it == count.cend())
                continue;
            const int freq = it->second;
            firstHalf += string(freq / 2, c);
        }
        const int index = firstHalf.find_first_not_of('0');
        return index == string::npos ? "" : firstHalf.substr(index);
    }

    string getMid(const unordered_map<char, int>& count) {
        for (char c = '9'; c >= '0'; --c) {
            const auto it = count.find(c);
            if (it == count.cend())
                continue;
            const int freq = it->second;
            if (freq % 2 == 1)
                return string(1, c);
        }
        return "";
    }

    string reversed(const string& s) {
        return {s.rbegin(), s.rend()};
    }
};
```

1494. 2385.cpp

Path: LeetCode-main/solutions/2385. Amount of Time for Binary Tree to Be Infected/2385.cpp

```
class Solution {
public:
    int amountOfTime(TreeNode* root, int start) {
        int ans = -1;
        const unordered_map<int, vector<int>> graph = getGraph(root);
        queue<int> q{{start}};
        unordered_set<int> seen{start};

        for (; !q.empty(); ++ans) {
            for (int sz = q.size(); sz > 0; --sz) {
                const int u = q.front();
                q.pop();
                if (!graph.contains(u))
                    continue;
                for (const int v : graph.at(u)) {
                    if (seen.contains(v))
                        continue;
                    q.push(v);
                    seen.insert(v);
                }
            }
        }

        return ans;
    }

private:
    unordered_map<int, vector<int>> getGraph(TreeNode* root) {
        unordered_map<int, vector<int>> graph;
        queue<pair<TreeNode*, int>> q{{root, -1}}; // (node, parent)

        while (!q.empty()) {
            const auto [node, parent] = q.front();
            q.pop();
            if (parent != -1) {
                graph[parent].push_back(node->val);
                graph[node->val].push_back(parent);
            }
            if (node->left)
                q.emplace(node->left, node->val);
            if (node->right)
                q.emplace(node->right, node->val);
        }

        return graph;
    }
};
```

1495. 2386.cpp

Path: LeetCode-main/solutions/2386. Find the K-Sum of an Array/2386.cpp

```
class Solution {
public:
    long long kSum(vector<int>& nums, int k) {
        const long long maxSum = getMaxSum(nums);
        const vector<int> absNums = getAbsNums(nums);
        long long ans = maxSum;
        // (the next maximum sum, the next index i)
        using P = pair<long long, int>;
        priority_queue<P> maxHeap;
        maxHeap.emplace(maxSum - absNums[0], 0);

        for (int j = 0; j < k - 1; ++j) {
            const auto [nextMaxSum, i] = maxHeap.top();
            maxHeap.pop();
            ans = nextMaxSum;
            if (i + 1 < absNums.size()) {
                maxHeap.emplace(nextMaxSum - absNums[i + 1], i + 1);
                maxHeap.emplace(nextMaxSum - absNums[i + 1] + absNums[i], i + 1);
            }
        }

        return ans;
    }

private:
    long long getMaxSum(const vector<int>& nums) {
        long long maxSum = 0;
        for (const int num : nums)
            if (num > 0)
                maxSum += num;
        return maxSum;
    }

    vector<int> getAbsNums(const vector<int>& nums) {
        vector<int> absNums;
        for (const int num : nums)
            absNums.push_back(abs(num));
        ranges::sort(absNums);
        return absNums;
    }
};
```

1496. 2387.cpp

Path: LeetCode-main/solutions/2387. Median of a Row Wise Sorted Matrix/2387.cpp

```
class Solution {
public:
    int matrixMedian(vector<vector<int>>& grid) {
        const int noGreaterThanMedianCount = grid.size() * grid[0].size() / 2 + 1;
        int l = 1;
        int r = 1'000'000;

        while (l < r) {
            const int m = (l + r) / 2;
            if (numsNoGreaterThan(grid, m) >= noGreaterThanMedianCount)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    int numsNoGreaterThan(const vector<vector<int>>& grid, int m) {
        int count = 0;
        for (const vector<int>& row : grid)
            count += ranges::upper_bound(row, m) - row.begin();
        return count;
    }
};
```

1497. 2389.cpp

Path: LeetCode-main/solutions/2389. Longest Subsequence With Limited Sum/2389.cpp

```
class Solution {
public:
    vector<int> answerQueries(vector<int>& nums, vector<int>& queries) {
        vector<int> ans;

        ranges::sort(nums);

        for (const int query : queries)
            ans.push_back(numOfElementsLessThan(nums, query));

        return ans;
    }

private:
    int numOfElementsLessThan(const vector<int>& nums, int query) {
        int sum = 0;
        for (int i = 0; i < nums.size(); ++i) {
            sum += nums[i];
            if (sum > query)
                return i;
        }
        return nums.size();
    }
};
```


1498. 239.cpp

Path: LeetCode-main/solutions/239. Sliding Window Maximum/239.cpp

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        vector<int> ans;
        deque<int> maxQ;

        for (int i = 0; i < nums.size(); ++i) {
            while (!maxQ.empty() && maxQ.back() < nums[i])
                maxQ.pop_back();
            maxQ.push_back(nums[i]);
            if (i >= k && nums[i - k] == maxQ.front()) // out-of-bounds
                maxQ.pop_front();
            if (i >= k - 1)
                ans.push_back(maxQ.front());
        }

        return ans;
    }
};
```

1499. 2390.cpp

Path: LeetCode-main/solutions/2390. Removing Stars From a String/2390.cpp

```
class Solution {
public:
    string removeStars(string s) {
        string ans;
        for (const char c : s)
            if (c == '*')
                ans.pop_back();
            else
                ans.push_back(c);
        return ans;
    }
};
```

1500. 2391.cpp

Path: LeetCode-main/solutions/2391. Minimum Amount of Time to Collect Garbage/2391.cpp

```
class Solution {
public:
    int garbageCollection(vector<string>& garbage, vector<int>& travel) {
        vector<int> prefix(travel.size());
        partial_sum(travel.begin(), travel.end(), prefix.begin());
        const int timeM = getTime(garbage, prefix, 'M');
        const int timeP = getTime(garbage, prefix, 'P');
        const int timeG = getTime(garbage, prefix, 'G');
        return timeM + timeP + timeG;
    }

private:
    int getTime(const vector<string>& garbage, const vector<int>& prefix,
                char c) {
        int characterCount = 0;
        int lastIndex = -1;
        for (int i = 0; i < garbage.size(); ++i) {
            const string& s = garbage[i];
            if (ranges::any_of(s, [c](const char g) { return g == c; }))
                lastIndex = i;
            characterCount += std::ranges::count(s, c);
        }
        return characterCount + (lastIndex <= 0 ? 0 : prefix[lastIndex - 1]);
    }
};
```

1501. 2392.cpp

Path: LeetCode-main/solutions/2392. Build a Matrix With Conditions/2392.cpp

```
class Solution {
public:
    vector<vector<int>> buildMatrix(int k, vector<vector<int>>& rowConditions,
                                   vector<vector<int>>& colConditions) {
        const vector<int> rowOrder = topologicalSort(rowConditions, k);
        if (rowOrder.empty())
            return {};

        const vector<int> colOrder = topologicalSort(colConditions, k);
        if (colOrder.empty())
            return {};

        vector<vector<int>> ans(k, vector<int>(k));
        vector<int> nodeToRowIndex(k + 1);

        for (int i = 0; i < k; ++i)
            nodeToRowIndex[rowOrder[i]] = i;

        for (int j = 0; j < k; ++j) {
            const int node = colOrder[j];
            const int i = nodeToRowIndex[node];
            ans[i][j] = node;
        }

        return ans;
    }

private:
    vector<int> topologicalSort(const vector<vector<int>>& conditions, int n) {
        vector<int> order;
        vector<vector<int>> graph(n + 1);
        vector<int> inDegrees(n + 1);
        queue<int> q;

        // Build the graph.
        for (const vector<int>& condition : conditions) {
            const int u = condition[0];
            const int v = condition[1];
            graph[u].push_back(v);
            ++inDegrees[v];
        }

        // Perform topological sorting.
        for (int i = 1; i <= n; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            order.push_back(u);
            for (const int v : graph[u])
                if (--inDegrees[v] == 0)
                    q.push(v);
        }

        return order.size() == n ? order : vector<int>();
    }
};
```

1502. 2393.cpp

Path: LeetCode-main/solutions/2393. Count Strictly Increasing Subarrays/2393.cpp

```
class Solution {
public:
    long long countSubarrays(vector<int>& nums) {
        long long ans = 0;

        for (int i = 0, j = -1; i < nums.size(); ++i) {
            if (i > 0 && nums[i] <= nums[i - 1])
                j = i - 1;
            ans += i - j;
        }

        return ans;
    }
};
```

1503. 2395.cpp

Path: LeetCode-main/solutions/2395. Find Subarrays With Equal Sum/2395.cpp

```
class Solution {
public:
    bool findSubarrays(vector<int>& nums) {
        unordered_set<int> seen;

        for (int i = 1; i < nums.size(); ++i)
            if (!seen.insert(nums[i - 1] + nums[i]).second)
                return true;

        return false;
    }
};
```

1504. 2396.cpp

Path: LeetCode-main/solutions/2396. Strictly Palindromic Number/2396.cpp

```
class Solution {  
public:  
    bool isStrictlyPalindromic(int n) {  
        return false;  
    }  
};
```

1505. 2397.cpp

Path: LeetCode-main/solutions/2397. Maximum Rows Covered by Columns/2397.cpp

```
class Solution {
public:
    int maximumRows(vector<vector<int>>& matrix, int numSelect) {
        int ans = 0;
        dfs(matrix, /*colIndex=*/0, numSelect, /*mask=*/0, ans);
        return ans;
    }

private:
    void dfs(const vector<vector<int>>& matrix, int colIndex, int leftColsCount,
             int mask, int& ans) {
        if (leftColsCount == 0) {
            ans = max(ans, getAllZerosRowCount(matrix, mask));
            return;
        }
        if (colIndex == matrix[0].size())
            return;

        // Choose this column.
        dfs(matrix, colIndex + 1, leftColsCount - 1, mask | 1 << colIndex, ans);
        // Don't choose this column.
        dfs(matrix, colIndex + 1, leftColsCount, mask, ans);
    }

    int getAllZerosRowCount(const vector<vector<int>>& matrix, int mask) {
        int count = 0;
        for (const vector<int>& row : matrix) {
            bool isAllZeros = true;
            for (int i = 0; i < row.size(); ++i) {
                if (row[i] == 1 && (mask >> i & 1) == 0) {
                    isAllZeros = false;
                    break;
                }
            }
            if (isAllZeros)
                ++count;
        }
        return count;
    }
};
```


1506. 2398.cpp

Path: LeetCode-main/solutions/2398. Maximum Number of Robots Within Budget/2398.cpp

```
class Solution {
public:
    int maximumRobots(vector<int>& chargeTimes, vector<int>& runningCosts,
                      long long budget) {
        long cost = 0;
        deque<int> maxQ; // Stores `chargeTimes[i]`.

        int j = 0; // window's range := [i..j], so k = i - j + 1
        for (int i = 0; i < chargeTimes.size(); ++i) {
            cost += runningCosts[i];
            while (!maxQ.empty() && maxQ.back() < chargeTimes[i])
                maxQ.pop_back();
            maxQ.push_back(chargeTimes[i]);
            if (maxQ.front() + (i - j + 1) * cost > budget) {
                if (maxQ.front() == chargeTimes[j])
                    maxQ.pop_front();
                cost -= runningCosts[j++];
            }
        }
        return chargeTimes.size() - j;
    }
};
```

1507. 2399.cpp

Path: LeetCode-main/solutions/2399. Check Distances Between Same Letters/2399.cpp

```
class Solution {
public:
    bool checkDistances(string s, vector<int>& distance) {
        vector<int> firstSeenIndex(26, -1);

        for (int i = 0; i < s.length(); ++i) {
            const int j = s[i] - 'a';
            int& prevIndex = firstSeenIndex[j];
            if (prevIndex != -1 && i - prevIndex - 1 != distance[j])
                return false;
            prevIndex = i;
        }

        return true;
    }
};
```

1508. 24.cpp

Path: LeetCode-main/solutions/24. Swap Nodes in Pairs/24.cpp

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        const int length = getLength(head);
        ListNode dummy(0, head);
        ListNode* prev = &dummy;
        ListNode* curr = head;

        for (int i = 0; i < length / 2; ++i) {
            ListNode* next = curr->next;
            curr->next = next->next;
            next->next = prev->next;
            prev->next = next;
            prev = curr;
            curr = curr->next;
        }

        return dummy.next;
    }

private:
    int getLength(ListNode* head) {
        int length = 0;
        for (ListNode* curr = head; curr; curr = curr->next)
            ++length;
        return length;
    }
};
```

1509. 240.cpp

Path: LeetCode-main/solutions/240. Search a 2D Matrix II/240.cpp

```
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int r = 0;
        int c = matrix[0].size() - 1;

        while (r < matrix.size() && c >= 0) {
            if (matrix[r][c] == target)
                return true;
            if (matrix[r][c] > target)
                --c;
            else
                ++r;
        }
        return false;
    }
};
```

1510. 2400.cpp

Path: LeetCode-main/solutions/2400. Number of Ways to Reach a Position After Exactly k Steps/2400.cpp

```
class Solution {
public:
    int numberOfWays(int startPos, int endPos, int k) {
        // leftStep + rightStep = k
        // rightStep - leftStep = endPos - startPos
        //      2 * rightStep = k + endPos - startPos
        //      rightStep = (k + endPos - startPos) / 2
        const int val = k + endPos - startPos;
        if (val < 0 || val % 2 == 1)
            return 0;
        const int rightStep = val / 2;
        const int leftStep = k - rightStep;
        if (leftStep < 0)
            return 0;
        return nck(leftStep + rightStep, min(leftStep, rightStep));
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // C(n, k) = C(n - 1, k) + C(n - 1, k - 1)
    int nck(int n, int k) {
        // dp[i] := C(n so far, i)
        vector<int> dp(k + 1);
        dp[0] = 1;

        while (n-- > 0) // Calculate n times.
            for (int j = k; j > 0; --j) {
                dp[j] += dp[j - 1];
                dp[j] %= kMod;
            }

        return dp[k];
    }
};
```

1511. 2401.cpp

Path: LeetCode-main/solutions/2401. Longest Nice Subarray/2401.cpp

```
class Solution {
public:
    int longestNiceSubarray(vector<int>& nums) {
        int ans = 0;
        int used = 0;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            while (used & nums[r])
                used ^= nums[l++];
            used |= nums[r];
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

1512. 2402.cpp

Path: LeetCode-main/solutions/2402. Meeting Rooms III/2402.cpp

```
struct T {
    long endTime;
    int roomId;
};

class Solution {
public:
    int mostBooked(int n, vector<vector<int>>& meetings) {
        vector<int> count(n);

        ranges::sort(meetings);

        auto compare = [](const T& a, const T& b) {
            return a.endTime == b.endTime ? a.roomId > b.roomId
                : a.endTime > b.endTime;
        };
        priority_queue<T, vector<T>, decltype(compare)> occupied(compare);
        priority_queue<int, vector<int>, greater<>> availableRoomIds;

        for (int i = 0; i < n; ++i)
            availableRoomIds.push(i);

        for (const vector<int>& meeting : meetings) {
            const int start = meeting[0];
            const int end = meeting[1];
            // Push meetings ending before this `meeting` in occupied to the
            // `availableRoomIds`.
            while (!occupied.empty() && occupied.top().endTime <= start)
                availableRoomIds.push(occupied.top().roomId), occupied.pop();
            if (availableRoomIds.empty()) {
                const auto [newStart, roomId] = occupied.top();
                occupied.pop();
                ++count[roomId];
                occupied.push({newStart + (end - start), roomId});
            } else {
                const int roomId = availableRoomIds.top();
                availableRoomIds.pop();
                ++count[roomId];
                occupied.push({end, roomId});
            }
        }

        return ranges::max_element(count) - count.begin();
    }
};
```

1513. 2403.cpp

Path: LeetCode-main/solutions/2403. Minimum Time to Kill All Monsters/2403.cpp

```
class Solution {
public:
    long long minimumTime(vector<int>& power) {
        const int n = power.size();
        const int maxMask = 1 << n;
        // dp[i] := the minimum number of days needed to defeat the monsters, where
        // i is the bitmask of the monsters
        vector<long> dp(maxMask, LONG_MAX);
        dp[0] = 0;

        for (unsigned mask = 1; mask < maxMask; ++mask) {
            const int currentGain = popcount(mask);
            for (int i = 0; i < n; ++i)
                if (mask >> i & 1)
                    dp[mask] =
                        min(dp[mask],
                            dp[mask & ~(1 << i)] +
                            static_cast<int>(ceil(power[i] * 1.0 / currentGain)));
        }

        return dp.back();
    }
};
```


1514. 2404.cpp

Path: LeetCode-main/solutions/2404. Most Frequent Even Element/2404.cpp

```
class Solution {
public:
    int mostFrequentEven(vector<int>& nums) {
        int ans = -1;
        unordered_map<int, int> count;

        for (const int num : nums) {
            if (num % 2 == 1)
                continue;
            const int newCount = ++count[num];
            const int maxCount = count[ans];
            if (newCount > maxCount || newCount == maxCount && num < ans)
                ans = num;
        }

        return ans;
    }
};
```

1515. 2405.cpp

Path: LeetCode-main/solutions/2405. Optimal Partition of String/2405.cpp

```
class Solution {
public:
    int partitionString(string s) {
        int ans = 1;
        int used = 0;

        for (const char c : s) {
            const int i = c - 'a';
            if (used >> i & 1) {
                used = 1 << i;
                ++ans;
            } else {
                used |= 1 << i;
            }
        }

        return ans;
    }
};
```

1516. 2406.cpp

Path: LeetCode-main/solutions/2406. Divide Intervals Into Minimum Number of Groups/2406.cpp

```
class Solution {
public:
    // Similar to 253. Meeting Rooms II
    int minGroups(vector<vector<int>>& intervals) {
        // Stores `right`s.
        priority_queue<int, vector<int>, greater<>> minHeap;

        ranges::sort(intervals);

        for (const vector<int>& interval : intervals) {
            // There's no overlap, so we can reuse the same group.
            if (!minHeap.empty() && interval[0] > minHeap.top())
                minHeap.pop();
            minHeap.push(interval[1]);
        }

        return minHeap.size();
    }
};
```

1517. 2407.cpp

Path: LeetCode-main/solutions/2407. Longest Increasing Subsequence II/2407.cpp

```
struct SegmentTreeNode {
    int lo;
    int hi;
    int maxLength;
    std::unique_ptr<SegmentTreeNode> left;
    std::unique_ptr<SegmentTreeNode> right;
    // maxLength := the maximum length of LIS ending in [lo..hi]
    SegmentTreeNode(int lo, int hi, int maxLength,
                    std::unique_ptr<SegmentTreeNode> left = nullptr,
                    std::unique_ptr<SegmentTreeNode> right = nullptr)
        : lo(lo),
          hi(hi),
          maxLength(maxLength),
          left(std::move(left)),
          right(std::move(right)) {}
};

class SegmentTree {
public:
    explicit SegmentTree() : root(make_unique<SegmentTreeNode>(0, 1e5 + 1, 0)) {}

    void updateRange(int i, int j, int maxLength) {
        update(root, i, j, maxLength);
    }

    // Returns the maximum length of LIS ending in [i..j].
    int queryRange(int i, int j) {
        return query(root, i, j);
    }

private:
    std::unique_ptr<SegmentTreeNode> root;

    void update(std::unique_ptr<SegmentTreeNode>& root, int i, int j,
                int maxLength) {
        if (root->lo == i && root->hi == j) {
            root->maxLength = maxLength;
            root->left = nullptr;
            root->right = nullptr;
            return;
        }
        const int mid = root->lo + (root->hi - root->lo) / 2;
        if (root->left == nullptr) {
            root->left = make_unique<SegmentTreeNode>(root->lo, mid, root->maxLength);
            root->right =
                make_unique<SegmentTreeNode>(mid + 1, root->hi, root->maxLength);
        }
        if (j <= mid)
            update(root->left, i, j, maxLength);
        else if (i > mid)
            update(root->right, i, j, maxLength);
        else {
            update(root->left, i, mid, maxLength);
            update(root->right, mid + 1, j, maxLength);
        }
        root->maxLength = merge(root->left->maxLength, root->right->maxLength);
    }

    int query(std::unique_ptr<SegmentTreeNode>& root, int i, int j) {
        if (root->left == nullptr)
            return root->maxLength;
        if (root->lo == i && root->hi == j)
            return root->maxLength;
        const int mid = root->lo + (root->hi - root->lo) / 2;
        if (j <= mid)
            return query(root->left, i, j);
        if (i > mid)
            return query(root->right, i, j);
        return merge(query(root->left, i, mid), query(root->right, mid + 1, j));
    }

    int merge(int left, int right) const {
        return max(left, right);
    }
};

class Solution {
public:
    int lengthOfLIS(vector<int>& nums, int k) {
        int ans = 1;
    }
};
```

```
SegmentTree tree;

for (const int num : nums) {
    const int left = max(1, num - k);
    const int right = num - 1;
    // the maximum length of LIS ending in [left..right] + the current number
    const int maxLength = tree.queryRange(left, right) + 1;
    ans = max(ans, maxLength);
    tree.updateRange(num, num, maxLength);
}

return ans;
};
```

1518. 2408.cpp

Path: LeetCode-main/solutions/2408. Design SQL/2408.cpp

```
class SQL {
public:
    void insertRow(string name, vector<string> row) {
        db[name].push_back(row);
    }

    void deleteRow(string name, int rowId) {}

    string selectCell(string name, int rowId, int columnId) {
        return db[name][rowId - 1][columnId - 1];
    }

private:
    unordered_map<string, vector<vector<string>>> db;
};
```

1519. 2409.cpp

Path: LeetCode-main/solutions/2409. Count Days Spent Together/2409.cpp

```
class Solution {
public:
    int countDaysTogether(string arriveAlice, string leaveAlice, string arriveBob,
                        string leaveBob) {
        const int arriveA = toDays(arriveAlice);
        const int leaveA = toDays(leaveAlice);
        const int arriveB = toDays(arriveBob);
        const int leaveB = toDays(leaveBob);
        int ans = 0;

        for (int day = 1; day <= 365; ++day)
            if (arriveA <= day && day <= leaveA && arriveB <= day && day <= leaveB)
                ++ans;

        return ans;
    }

private:
    const vector<int> days{0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    int toDays(const string& s) {
        const int month = (s[0] - '0') * 10 + (s[1] - '0');
        const int day = (s[3] - '0') * 10 + (s[4] - '0');
        int prevDays = 0;
        for (int m = 1; m < month; ++m)
            prevDays += days[m];
        return prevDays + day;
    }
};
```

1520. 241.cpp

Path: LeetCode-main/solutions/241. Different Ways to Add Parentheses/241.cpp

```
class Solution {
public:
    vector<int> diffWaysToCompute(string expression) {
        return ways(expression, {});
    }

private:
    vector<int> ways(const string& s, unordered_map<string, vector<int>>&& mem) {
        if (const auto it = mem.find(s); it != mem.cend())
            return it->second;

        vector<int> ans;

        for (int i = 0; i < s.length(); ++i)
            if (ispunct(s[i]))
                for (const int a : ways(s.substr(0, i), std::move(mem)))
                    for (const int b : ways(s.substr(i + 1), std::move(mem)))
                        if (s[i] == '+')
                            ans.push_back(a + b);
                        else if (s[i] == '-')
                            ans.push_back(a - b);
                        else
                            ans.push_back(a * b);

        return mem[s] = (ans.empty() ? vector<int>{stoi(s)} : ans);
    }
};
```


1521. 2410.cpp

Path: LeetCode-main/solutions/2410. Maximum Matching of Players With Trainers/2410.cpp

```
class Solution {
public:
    int matchPlayersAndTrainers(vector<int>& players, vector<int>& trainers) {
        int ans = 0;

        ranges::sort(players);
        ranges::sort(trainers);

        for (int i = 0; i < trainers.size(); ++i)
            if (players[ans] <= trainers[i] && ++ans == players.size())
                return ans;

        return ans;
    }
};
```

1522. 2411.cpp

Path: LeetCode-main/solutions/2411. Smallest Subarrays With Maximum Bitwise OR/2411.cpp

```
class Solution {
public:
    vector<int> smallestSubarrays(vector<int>& nums) {
        constexpr int kMaxBit = 30;
        vector<int> ans(nums.size(), 1);
        // closest[j] := the closest index i s.t. the j-th bit of nums[i] is 1
        vector<int> closest(kMaxBit);

        for (int i = nums.size() - 1; i >= 0; --i)
            for (int j = 0; j < kMaxBit; ++j) {
                if (nums[i] >> j & 1)
                    closest[j] = i;
                ans[i] = max(ans[i], closest[j] - i + 1);
            }

        return ans;
    }
};
```

1523. 2412.cpp

Path: LeetCode-main/solutions/2412. Minimum Money Required Before Transactions/2412.cpp

```
class Solution {
public:
    long long minimumMoney(vector<vector<int>>& transactions) {
        long ans = 0;
        long losses = 0;

        // Before picking the final transaction, perform any transaction that raises
        // the required money.
        for (const vector<int>& t : transactions) {
            const int cost = t[0];
            const int cashback = t[1];
            losses += max(0, cost - cashback);
        }

        // Now, pick a transaction to be the final one.
        for (const vector<int>& t : transactions) {
            const int cost = t[0];
            const int cashback = t[1];
            if (cost > cashback)
                // The losses except this transaction: losses - (cost - cashback), so
                // add the cost of this transaction = losses - (cost - cashback) + cost.
                ans = max(ans, losses + cashback);
            else
                // The losses except this transaction: losses, so add the cost of this
                // transaction = losses + cost.
                ans = max(ans, losses + cost);
        }

        return ans;
    }
};
```

1524. 2413.cpp

Path: LeetCode-main/solutions/2413. Smallest Even Multiple/2413.cpp

```
class Solution {  
public:  
    int smallestEvenMultiple(int n) {  
        return n * (n % 2 + 1);  
    }  
};
```

1525. 2414.cpp

Path: LeetCode-main/solutions/2414. Length of the Longest Alphabetical Continuous Substring/2414.cpp

```
class Solution {
public:
    int longestContinuousSubstring(string s) {
        int ans = 1;
        int runningLen = 1;

        for (int i = 1; i < s.length(); ++i)
            if (s[i] == s[i - 1] + 1)
                ans = max(ans, ++runningLen);
            else
                runningLen = 1;

        return ans;
    }
};
```

1526. 2415.cpp

Path: LeetCode-main/solutions/2415. Reverse Odd Levels of Binary Tree/2415.cpp

```
class Solution {
public:
    TreeNode* reverseOddLevels(TreeNode* root) {
        dfs(root->left, root->right, true);
        return root;
    }

private:
    void dfs(TreeNode* left, TreeNode* right, bool isOddLevel) {
        if (left == nullptr)
            return;
        if (isOddLevel)
            swap(left->val, right->val);
        dfs(left->left, right->right, !isOddLevel);
        dfs(left->right, right->left, !isOddLevel);
    }
};
```

1527. 2416.cpp

Path: LeetCode-main/solutions/2416. Sum of Prefix Scores of Strings/2416.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    int count = 0;
    TrieNode() : children(26) {}
};

class Solution {
public:
    vector<int> sumPrefixScores(vector<string>& words) {
        vector<int> ans;

        for (const string& word : words)
            insert(word);

        for (const string& word : words)
            ans.push_back(getScore(word));

        return ans;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
            ++node->count;
        }
    }

    int getScore(const string& word) {
        shared_ptr<TrieNode> node = root;
        int score = 0;
        for (const char c : word) {
            node = node->children[c - 'a'];
            score += node->count;
        }
        return score;
    }
};
```

1528. 2417.cpp

Path: LeetCode-main/solutions/2417. Closest Fair Integer/2417.cpp

```
class Solution {
public:
    int closestFair(int n) {
        const int digitsCount = to_string(n).length();
        return digitsCount % 2 == 0 ? getEvenDigits(n) : getOddDigits(digitsCount);
    }

private:
    long getOddDigits(int digitsCount) {
        const int zeros = (digitsCount + 1) / 2;
        const int ones = (digitsCount - 1) / 2;
        return stoi('1' + string(zeros, '0') + string(ones, '1'));
    }

    int getEvenDigits(int n) {
        const int digitsCount = to_string(n).length();
        const long maxNum = stol('1' + string(digitsCount, '0'));
        for (long num = n; num < maxNum; ++num)
            if (isValidNum(num))
                return num;
        return getOddDigits(digitsCount + 1);
    }

    bool isValidNum(long num) {
        int count = 0;
        for (const char c : to_string(num))
            count += (c - '0') % 2 == 0 ? 1 : -1;
        return count == 0;
    }
};
```


1529. 2418.cpp

Path: LeetCode-main/solutions/2418. Sort the People/2418.cpp

```
class Solution {
public:
    vector<string> sortPeople(vector<string>& names, vector<int>& heights) {
        vector<string> ans;
        vector<pair<int, string>> heightAndNames;

        for (int i = 0; i < names.size(); ++i)
            heightAndNames.emplace_back(heights[i], names[i]);

        ranges::sort(heightAndNames, greater<>());

        for (const auto& [_, name] : heightAndNames)
            ans.push_back(name);

        return ans;
    }
};
```

1530. 2419.cpp

Path: LeetCode-main/solutions/2419. Longest Subarray With Maximum Bitwise AND/2419.cpp

```
class Solution {
public:
    int longestSubarray(vector<int>& nums) {
        int ans = 0;
        int maxIndex = 0;
        int sameNumLength = 0;

        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == nums[maxIndex]) {
                ans = max(ans, ++sameNumLength);
            } else if (nums[i] > nums[maxIndex]) {
                maxIndex = i;
                sameNumLength = 1;
                ans = 1;
            } else {
                sameNumLength = 0;
            }

        return ans;
    }
};
```

1531. 242.cpp

Path: LeetCode-main/solutions/242. Valid Anagram/242.cpp

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length())
            return false;

        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        for (const char c : t) {
            if (count[c - 'a'] == 0)
                return false;
            --count[c - 'a'];
        }

        return true;
    }
};
```

1532. 2420.cpp

Path: LeetCode-main/solutions/2420. Find All Good Indices/2420.cpp

```
class Solution {
public:
    // Same as 2100. Find Good Days to Rob the Bank
    vector<int> goodIndices(vector<int>& nums, int k) {
        const int n = nums.size();
        vector<int> ans;
        // dec[i] := 1 + the number of continuous decreasing numbers before i
        vector<int> dec(n, 1);
        // inc[i] := 1 + the number of continuous increasing numbers after i
        vector<int> inc(n, 1);

        for (int i = 1; i < n; ++i)
            if (nums[i - 1] >= nums[i])
                dec[i] = dec[i - 1] + 1;

        for (int i = n - 2; i >= 0; --i)
            if (nums[i] <= nums[i + 1])
                inc[i] = inc[i + 1] + 1;

        for (int i = k; i < n - k; ++i)
            if (dec[i - 1] >= k && inc[i + 1] >= k)
                ans.push_back(i);

        return ans;
    }
};
```

1533. 2421.cpp

Path: LeetCode-main/solutions/2421. Number of Good Paths/2421.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    int numberOfGoodPaths(vector<int>& vals, vector<vector<int>>& edges) {
        const int n = vals.size();
        int ans = 0;
        UnionFind uf(n);
        vector<vector<int>> tree(n);
        map<int, vector<int>> valToNodes;

        for (int i = 0; i < vals.size(); ++i)
            valToNodes[vals[i]].push_back(i);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            if (vals[v] <= vals[u])
                tree[u].push_back(v);
            if (vals[u] <= vals[v])
                tree[v].push_back(u);
        }

        for (const auto& [val, nodes] : valToNodes) {
            for (const int u : nodes)
                for (const int v : tree[u])
                    uf.unionByRank(u, v);
            unordered_map<int, int> rootCount;
            for (const int u : nodes)
                ++rootCount[uf.find(u)];
            // For each group,  $C(\text{count}, 2) := \text{count} * (\text{count} - 1) / 2$ 
            for (const auto& [_, count] : rootCount)
                ans += count * (count - 1) / 2;
        }

        return ans;
    }
};
```

1534. 2422.cpp

Path: LeetCode-main/solutions/2422. Merge Operations to Turn Array Into a Palindrome/2422.cpp

```
class Solution {
public:
    int minimumOperations(vector<int>& nums) {
        int ans = 0;
        int l = 0;
        int r = nums.size() - 1;
        long leftSum = nums.front();
        long rightSum = nums.back();

        while (l < r)
            if (leftSum < rightSum) {
                leftSum += nums[++l];
                ++ans;
            } else if (leftSum > rightSum) {
                rightSum += nums[--r];
                ++ans;
            } else { // leftSum == rightSum
                leftSum = nums[++l];
                rightSum = nums[--r];
            }

        return ans;
    }
};
```

1535. 2423.cpp

Path: LeetCode-main/solutions/2423. Remove Letter To Equalize Frequency/2423.cpp

```
class Solution {
public:
    bool equalFrequency(string word) {
        vector<int> count(26);

        for (const char c : word)
            ++count[c - 'a'];

        // Try to remove each letter, then check if the frequency of all the letters
        // in `word` are equal.
        for (const char c : word) {
            --count[c - 'a'];
            if (equalFreq(count))
                return true;
            ++count[c - 'a'];
        }

        return false;
    }

private:
    static constexpr int kMax = 101;

    bool equalFreq(const vector<int>& count) {
        int minfreq = kMax;
        int maxfreq = 0;
        for (const int freq : count)
            if (freq > 0) {
                minfreq = min(minfreq, freq);
                maxfreq = max(maxfreq, freq);
            }
        return minfreq == maxfreq;
    }
};
```

1536. 2424.cpp

Path: LeetCode-main/solutions/2424. Longest Uploaded Prefix/2424.cpp

```
class LUPrefix {
public:
    LUPrefix(int n) {}

    void upload(int video) {
        seen.insert(video);
        while (seen.contains(longestPrefix + 1))
            ++longestPrefix;
    }

    int longest() {
        return longestPrefix;
    }

private:
    unordered_set<int> seen;
    int longestPrefix = 0;
};
```


1537. 2425.cpp

Path: LeetCode-main/solutions/2425. Bitwise XOR of All Pairings/2425.cpp

```
class Solution {
public:
    int xorAllNums(vector<int>& nums1, vector<int>& nums2) {
        // If the size of nums1 is m and the size of nums2 is n, then each number in
        // nums1 is repeated n times and each number in nums2 is repeated m times.
        const int xors1 = accumulate(nums1.begin(), nums1.end(), 0, bit_xor<>());
        const int xors2 = accumulate(nums2.begin(), nums2.end(), 0, bit_xor<>());
        return (nums1.size() % 2 * xors2) ^ (nums2.size() % 2 * xors1);
    }
};
```

1538. 2426.cpp

Path: LeetCode-main/solutions/2426. Number of Pairs Satisfying Inequality/2426.cpp

```
class Solution {
public:
    long long numberOfPairs(vector<int>& nums1, vector<int>& nums2, int diff) {
        // nums1[i] - nums1[j] <= nums2[i] - nums2[j] + diff
        // nums1[i] - nums2[i] <= nums1[j] - nums2[j] + diff
        // Define arr[i] := nums1[i] - nums2[i] -> arr[i] <= arr[j] + diff
        vector<int> arr;

        for (int i = 0; i < nums1.size(); ++i)
            arr.push_back(nums1[i] - nums2[i]);

        long ans = 0;
        mergeSort(arr, 0, arr.size() - 1, diff, ans);
        return ans;
    }

private:
    void mergeSort(vector<int>& arr, int l, int r, int diff, long& ans) {
        if (l >= r)
            return;

        const int m = (l + r) / 2;
        mergeSort(arr, l, m, diff, ans);
        mergeSort(arr, m + 1, r, diff, ans);
        merge(arr, l, m, r, diff, ans);
    }

    void merge(vector<int>& arr, int l, int m, int r, int diff, long& ans) {
        const int lo = m + 1;
        int hi = m + 1; // the first index s.t. arr[i] <= arr[hi] + diff

        // For each index i in the range [l, m], add `r - hi + 1` to `ans`.
        for (int i = l; i <= m; ++i) {
            while (hi <= r && arr[i] > arr[hi] + diff)
                ++hi;
            ans += r - hi + 1;
        }

        vector<int> sorted(r - l + 1);
        int k = 0; // sorted's index
        int i = l; // left's index
        int j = m + 1; // right's index

        while (i <= m && j <= r)
            if (arr[i] < arr[j])
                sorted[k++] = arr[i++];
            else
                sorted[k++] = arr[j++];

        // Put the possible remaining left part into the sorted array.
        while (i <= m)
            sorted[k++] = arr[i++];

        // Put the possible remaining right part into the sorted array.
        while (j <= r)
            sorted[k++] = arr[j++];

        copy(sorted.begin(), sorted.end(), arr.begin() + l);
    }
};
```

1539. 2427.cpp

Path: LeetCode-main/solutions/2427. Number of Common Factors/2427.cpp

```
class Solution {
public:
    int commonFactors(int a, int b) {
        int ans = 1;
        const int gcd = __gcd(a, b);
        for (int i = 2; i <= gcd; ++i)
            if (a % i == 0 && b % i == 0)
                ++ans;
        return ans;
    }
};
```

1540. 2428.cpp

Path: LeetCode-main/solutions/2428. Maximum Sum of an Hourglass/2428.cpp

```
class Solution {
public:
    int maxSum(vector<vector<int>>& grid) {
        int ans = 0;

        for (int i = 1; i + 1 < grid.size(); ++i)
            for (int j = 1; j + 1 < grid[0].size(); ++j)
                ans =
                    max(ans, grid[i - 1][j - 1] + grid[i - 1][j] + grid[i - 1][j + 1] +
                        grid[i][j] + grid[i + 1][j - 1] + grid[i + 1][j] +
                        grid[i + 1][j + 1]);

        return ans;
    }
};
```

1541. 2429.cpp

Path: LeetCode-main/solutions/2429. Minimize XOR/2429.cpp

```
class Solution {
public:
    int minimizeXor(unsigned num1, unsigned num2) {
        constexpr int kMaxBit = 30;
        int bits = popcount(num2);
        // Can turn off all the bits in `num1`.
        if (popcount(num1) == bits)
            return num1;

        int ans = 0;

        // Turn off the MSB if we have `bits` quota.
        for (int i = kMaxBit; i >= 0; --i)
            if (num1 >> i & 1) {
                ans |= 1 << i;
                if (--bits == 0)
                    return ans;
            }

        // Turn on the LSB if we still have `bits`.
        for (int i = 0; i < kMaxBit; ++i)
            if ((num1 >> i & 1) == 0) {
                ans |= 1 << i;
                if (--bits == 0)
                    return ans;
            }

        return ans;
    }
};
```

1542. 243.cpp

Path: LeetCode-main/solutions/243. Shortest Word Distance/243.cpp

```
class Solution {
public:
    int shortestDistance(vector<string>& wordsDict, string word1, string word2) {
        int ans = wordsDict.size();
        int index1 = -1; // wordsDict[index1] == word1
        int index2 = -1; // wordsDict[index2] == word2

        for (int i = 0; i < wordsDict.size(); ++i) {
            if (wordsDict[i] == word1) {
                index1 = i;
                if (index2 != -1)
                    ans = min(ans, index1 - index2);
            }
            if (wordsDict[i] == word2) {
                index2 = i;
                if (index1 != -1)
                    ans = min(ans, index2 - index1);
            }
        }

        return ans;
    }
};
```

1543. 2430.cpp

Path: LeetCode-main/solutions/2430. Maximum Deletions on a String/2430.cpp

```
class Solution {
public:
    int deleteString(string s) {
        const int n = s.length();
        // lcs[i][j] := the number of the same letters of s[i..n) and s[j..n)
        vector<vector<int>> lcs(n + 1, vector<int>(n + 1));
        // dp[i] := the maximum number of operations needed to delete s[i..n)
        vector<int> dp(n, 1);

        for (int i = n - 1; i >= 0; --i)
            for (int j = i + 1; j < n; ++j) {
                if (s[i] == s[j])
                    lcs[i][j] = lcs[i + 1][j + 1] + 1;
                if (lcs[i][j] >= j - i)
                    dp[i] = max(dp[i], dp[j] + 1);
            }

        return dp[0];
    }
};
```

1544. 2431-2.cpp

Path: LeetCode-main/solutions/2431. Maximize Total Tastiness of Purchased Fruits/2431-2.cpp

```
class Solution {
public:
    int maxTastiness(vector<int>& price, vector<int>& tastiness, int maxAmount,
                    int maxCoupons) {
        // dp[j][k] := the maximum tastiness of price so far with j amount of money
        // and k coupons
        vector<vector<int>> dp(maxAmount + 1, vector<int>(maxCoupons + 1));

        for (int i = 0; i < price.size(); ++i)
            for (int j = maxAmount; j >= price[i] / 2; --j)
                for (int k = maxCoupons; k >= 0; --k) {
                    const int buyWithCoupon =
                        k == 0 ? 0 : dp[j - price[i] / 2][k - 1] + tastiness[i];
                    const int buyWithoutCoupon =
                        j < price[i] ? 0 : dp[j - price[i]][k] + tastiness[i];
                    dp[j][k] = max({dp[j][k], buyWithCoupon, buyWithoutCoupon});
                }

        return dp[maxAmount][maxCoupons];
    }
};
```


1545. 2431.cpp

Path: LeetCode-main/solutions/2431. Maximize Total Tastiness of Purchased Fruits/2431.cpp

```
class Solution {
public:
    int maxTastiness(vector<int>& price, vector<int>& tastiness, int maxAmount,
                    int maxCoupons) {
        const int n = price.size();
        // dp[i][j][k] := the maximum tastiness of the first i price with j amount
        // of money and k coupons
        vector<vector<vector<int>>> dp(
            n + 1, vector<vector<int>>(maxAmount + 1, vector<int>(maxCoupons + 1)));

        for (int i = 1; i <= n; ++i) {
            // 1-indexed
            const int currPrice = price[i - 1];
            const int currTastiness = tastiness[i - 1];
            for (int amount = 0; amount <= maxAmount; ++amount) {
                for (int coupon = 0; coupon <= maxCoupons; ++coupon) {
                    // 1. Don't buy, the tastiness will be the same as the first i - 1
                    // price.
                    dp[i][amount][coupon] = dp[i - 1][amount][coupon];

                    // 2. Buy without coupon if have enough money.
                    if (amount >= currPrice)
                        dp[i][amount][coupon] =
                            max(dp[i][amount][coupon],
                                dp[i - 1][amount - currPrice][coupon] + currTastiness);

                    // 3. Buy with coupon if have coupon and enough money.
                    if (coupon > 0 && amount >= currPrice / 2)
                        dp[i][amount][coupon] = max(
                            dp[i][amount][coupon],
                            dp[i - 1][amount - currPrice / 2][coupon - 1] + currTastiness);
                }
            }
        }

        return dp[n][maxAmount][maxCoupons];
    }
};
```

1546. 2432.cpp

Path: LeetCode-main/solutions/2432. The Employee That Worked on the Longest Task/2432.cpp

```
class Solution {
public:
    int hardestWorker(int n, vector<vector<int>>& logs) {
        int ans = logs[0][0];
        int maxWorkingTime = logs[0][1];

        for (int i = 1; i < logs.size(); ++i) {
            const int id = logs[i][0];
            const int workingTime = logs[i][1] - logs[i - 1][1];
            if (workingTime > maxWorkingTime) {
                ans = id;
                maxWorkingTime = workingTime;
            } else if (workingTime == maxWorkingTime) {
                ans = min(ans, id);
            }
        }

        return ans;
    }
};
```

1547. 2433.cpp

Path: LeetCode-main/solutions/2433. Find The Original Array of Prefix Xor/2433.cpp

```
class Solution {
public:
    vector<int> findArray(vector<int>& pref) {
        vector<int> ans(pref.size());

        ans[0] = pref[0];
        for (int i = 1; i < ans.size(); ++i)
            ans[i] = pref[i] ^ pref[i - 1];

        return ans;
    }
};
```

1548. 2434.cpp

Path: LeetCode-main/solutions/2434. Using a Robot to Print the Lexicographically Smallest String/2434.cpp

```
class Solution {
public:
    string robotWithString(string s) {
        string ans;
        vector<int> count(26);
        stack<char> stack;

        for (const char c : s)
            ++count[c - 'a'];

        for (const char c : s) {
            stack.push(c);
            --count[c - 'a'];
            const char minChar = getMinChar(count);
            while (!stack.empty() && stack.top() <= minChar)
                ans += stack.top(), stack.pop();
        }

        while (!stack.empty())
            ans += stack.top(), stack.pop();

        return ans;
    }

private:
    char getMinChar(const vector<int>& count) {
        for (int i = 0; i < 26; ++i)
            if (count[i])
                return 'a' + i;
        return 'a';
    }
};
```

1549. 2435-2.cpp

Path: LeetCode-main/solutions/2435. Paths in Matrix Whose Sum Is Divisible by K/2435-2.cpp

```
class Solution {
public:
    int numberOfPaths(vector<vector<int>>& grid, int k) {
        constexpr int kMod = 1'000'000'007;
        const int m = grid.size();
        const int n = grid[0].size();
        // dp[i][j][sum] : = the number of paths to(i, j), where the sum / k == sum
        vector<vector<vector<int>>> dp(m, vector<vector<int>>(n, vector<int>(k)));
        dp[0][0][grid[0][0] % k] = 1;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                for (int sum = 0; sum < k; ++sum) {
                    const int newSum = (sum + grid[i][j]) % k;
                    if (i > 0)
                        dp[i][j][newSum] += dp[i - 1][j][sum];
                    if (j > 0)
                        dp[i][j][newSum] += dp[i][j - 1][sum];
                    dp[i][j][newSum] %= kMod;
                }

        return dp[m - 1][n - 1][0];
    }
};
```

1550. 2435.cpp

Path: LeetCode-main/solutions/2435. Paths in Matrix Whose Sum Is Divisible by K/2435.cpp

```
class Solution {
public:
    int numberOfPaths(vector<vector<int>>& grid, int k) {
        vector<vector<vector<int>>> mem(
            grid.size(), vector<vector<int>>>(grid[0].size(), vector<int>(k, -1)));
        return numberOfPaths(grid, 0, 0, 0, k, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of paths to (i, j), where the sum / k == `sum`.
    int numberOfPaths(const vector<vector<int>>& grid, int i, int j, int sum,
        int k, vector<vector<vector<int>>>& mem) {
        if (i == grid.size() || j == grid[0].size())
            return 0;
        if (i == grid.size() - 1 && j == grid[0].size() - 1)
            return (sum + grid[i][j]) % k == 0;
        if (mem[i][j][sum] != -1)
            return mem[i][j][sum];
        const int newSum = (sum + grid[i][j]) % k;
        return mem[i][j][sum] = (numberOfPaths(grid, i + 1, j, newSum, k, mem) +
            numberOfPaths(grid, i, j + 1, newSum, k, mem)) %
            kMod;
    }
};
```

1551. 2436.cpp

Path: LeetCode-main/solutions/2436. Minimum Split Into Subarrays With GCD Greater Than One/2436.cpp

```
class Solution {
public:
    int minimumSplits(vector<int>& nums) {
        int ans = 1;
        int gcd = nums[0];

        for (const int num : nums) {
            const int newGcd = __gcd(gcd, num);
            if (newGcd > 1) {
                gcd = newGcd;
            } else {
                gcd = num;
                ++ans;
            }
        }

        return ans;
    }
};
```

1552. 2437.cpp

Path: LeetCode-main/solutions/2437. Number of Valid Clock Times/2437.cpp

```
class Solution {
public:
    int countTime(string time) {
        int ans = 1;
        if (time[3] == '?')
            ans *= 6;
        if (time[4] == '?')
            ans *= 10;

        if (time[0] == '?' && time[1] == '?')
            return ans * 24;
        if (time[0] == '?')
            return time[1] < '4' ? ans * 3 : ans * 2;
        if (time[1] == '?')
            return time[0] == '2' ? ans * 4 : ans * 10;
        return ans;
    }
};
```


1553. 2438.cpp

Path: LeetCode-main/solutions/2438. Range Product Queries of Powers/2438.cpp

```
class Solution {
public:
    vector<int> productQueries(int n, vector<vector<int>>& queries) {
        constexpr int kMod = 1'000'000'007;
        constexpr int kMaxBit = 30;
        vector<int> ans;
        vector<int> pows;

        for (int i = 0; i < kMaxBit; ++i)
            if (n >> i & 1)
                pows.push_back(1 << i);

        for (const vector<int>& query : queries) {
            const int left = query[0];
            const int right = query[1];
            long prod = 1;
            for (int i = left; i <= right; ++i) {
                prod *= pows[i];
                prod %= kMod;
            }
            ans.push_back(prod);
        }

        return ans;
    }
};
```

1554. 2439.cpp

Path: LeetCode-main/solutions/2439. Minimize Maximum of Array/2439.cpp

```
class Solution {
public:
    int minimizeArrayValue(vector<int>& nums) {
        long ans = 0;
        long prefix = 0;

        for (int i = 0; i < nums.size(); ++i) {
            prefix += nums[i];
            const long prefixAvg = ceil(prefix / static_cast<double>(i + 1));
            ans = max(ans, prefixAvg);
        }

        return ans;
    }
};
```

1555. 244.cpp

Path: LeetCode-main/solutions/244. Shortest Word Distance II/244.cpp

```
class WordDistance {
public:
    WordDistance(vector<string>& words) {
        for (int i = 0; i < words.size(); ++i)
            wordToIndices[words[i]].push_back(i);
    }

    int shortest(string word1, string word2) {
        const vector<int> indices1 = wordToIndices[word1];
        const vector<int> indices2 = wordToIndices[word2];
        int ans = INT_MAX;

        for (int i = 0, j = 0; i < indices1.size() && j < indices2.size(); ) {
            ans = min(ans, abs(indices1[i] - indices2[j]));
            if (indices1[i] < indices2[j])
                ++i;
            else
                ++j;
        }

        return ans;
    }

private:
    unordered_map<string, vector<int>> wordToIndices;
};
```

1556. 2440.cpp

Path: LeetCode-main/solutions/2440. Create Components With Same Value/2440.cpp

```
class Solution {
public:
    int componentValue(vector<int>& nums, vector<vector<int>>& edges) {
        const int n = nums.size();
        const int sum = accumulate(nums.begin(), nums.end(), 0);
        vector<vector<int>>> tree(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        for (int i = n; i > 1; --i)
            // Split the tree into i parts, i.e. delete (i - 1) edges.
            if (sum % i == 0 && dfs(nums, tree, 0, sum / i, vector<bool>(n)) == 0)
                return i - 1;

        return 0;
    }

private:
    static constexpr int kMax = 1'000'000'000;

    // Returns the sum of the subtree rooted at u subtracting the sum of the
    // deleted subtrees.
    int dfs(const vector<int>& nums, const vector<vector<int>>& tree, int u,
            int target, vector<bool>&& seen) {
        int sum = nums[u];
        seen[u] = true;

        for (const int v : tree[u]) {
            if (seen[v])
                continue;
            sum += dfs(nums, tree, v, target, std::move(seen));
            if (sum > target)
                return kMax;
        }

        // Delete the tree that has sum == target.
        if (sum == target)
            return 0;
        return sum;
    }
};
```

1557. 2441.cpp

Path: LeetCode-main/solutions/2441. Largest Positive Integer That Exists With Its Negative/2441.cpp

```
class Solution {
public:
    int findMaxK(vector<int>& nums) {
        int ans = -1;
        unordered_set<int> seen;

        for (const int num : nums)
            if (seen.contains(-num))
                ans = max(ans, abs(num));
            else
                seen.insert(num);

        return ans;
    }
};
```

1558. 2442.cpp

Path: LeetCode-main/solutions/2442. Count Number of Distinct Integers After Reverse Operations/2442.cpp

```
class Solution {
public:
    int countDistinctIntegers(vector<int>& nums) {
        unordered_set<int> numsSet{nums.begin(), nums.end()};

        for (const int num : nums)
            numsSet.insert(reversed(num));

        return numsSet.size();
    }

private:
    int reversed(int num) {
        int ans = 0;
        while (num > 0) {
            ans = ans * 10 + num % 10;
            num /= 10;
        }
        return ans;
    }
};
```

1559. 2443.cpp

Path: LeetCode-main/solutions/2443. Sum of Number and Its Reverse/2443.cpp

```
class Solution {
public:
    bool sumOfNumberAndReverse(int num) {
        for (int i = num / 2; i <= num; ++i)
            if (num == i + reversed(i))
                return true;
        return false;
    }

private:
    int reversed(int num) {
        int ans = 0;
        while (num > 0) {
            ans = ans * 10 + num % 10;
            num /= 10;
        }
        return ans;
    }
};
```

1560. 2444.cpp

Path: LeetCode-main/solutions/2444. Count Subarrays With Fixed Bounds/2444.cpp

```
class Solution {
public:
    long long countSubarrays(vector<int>& nums, int minK, int maxK) {
        long ans = 0;
        int j = -1;
        int prevMinKIndex = -1;
        int prevMaxKIndex = -1;

        for (int i = 0; i < nums.size(); ++i) {
            if (nums[i] < minK || nums[i] > maxK)
                j = i;
            if (nums[i] == minK)
                prevMinKIndex = i;
            if (nums[i] == maxK)
                prevMaxKIndex = i;
            // Any index k in [j + 1, min(prevMinKIndex, prevMaxKIndex)] can be the
            // start of the subarray s.t. nums[k..i] satisfies the conditions.
            ans += max(0, min(prevMinKIndex, prevMaxKIndex) - j);
        }

        return ans;
    }
};
```


1561. 2445.cpp

Path: LeetCode-main/solutions/2445. Number of Nodes With Value One/2445.cpp

```
class Solution {
public:
    int numberOfNodes(int n, vector<int>& queries) {
        // flipped[i] := true if we should flip all the values in the subtree rooted
        // at i
        vector<bool> flipped(n + 1);

        for (const int query : queries)
            flipped[query] = flipped[query] ^ true;

        return dfs(1, 0, n, flipped);
    }

private:
    int dfs(int label, int value, int n, const vector<bool>& flipped) {
        if (label > n)
            return 0;
        value ^= flipped[label];
        return value + //
            dfs(label * 2, value, n, flipped) +
            dfs(label * 2 + 1, value, n, flipped);
    }
};
```

1562. 2446.cpp

Path: LeetCode-main/solutions/2446. Determine if Two Events Have Conflict/2446.cpp

```
class Solution {
public:
    bool haveConflict(vector<string>& event1, vector<string>& event2) {
        return event1[0] <= event2[1] && event2[0] <= event1[1];
    }
};
```

1563. 2447.cpp

Path: LeetCode-main/solutions/2447. Number of Subarrays With GCD Equal to K/2447.cpp

```
class Solution {
public:
    int subarrayGCD(vector<int>& nums, int k) {
        int ans = 0;
        unordered_map<int, int> gcds;

        for (const int num : nums)
            if (num % k == 0) {
                unordered_map<int, int> nextGcds{{num, 1}};
                for (const auto& [prevGcd, count] : gcds)
                    nextGcds[gcd(prevGcd, num)] += count;
                ans += nextGcds[k];
                gcds = std::move(nextGcds);
            } else {
                // The GCD streak stops, so fresh start from the next number.
                gcds.clear();
            }

        return ans;
    }
};
```

1564. 2448.cpp

Path: LeetCode-main/solutions/2448. Minimum Cost to Make Array Equal/2448.cpp

```
class Solution {
public:
    long long minCost(vector<int>& nums, vector<int>& cost) {
        long ans = 0;
        int l = ranges::min(nums);
        int r = ranges::max(nums);

        while (l < r) {
            const int m = (l + r) / 2;
            const long cost1 = getCost(nums, cost, m);
            const long cost2 = getCost(nums, cost, m + 1);
            ans = min(cost1, cost2);
            if (cost1 < cost2)
                r = m;
            else
                l = m + 1;
        }

        return ans;
    }

private:
    long getCost(const vector<int>& nums, const vector<int>& cost, int target) {
        long res = 0;
        for (int i = 0; i < nums.size(); ++i)
            res += static_cast<long>(abs(nums[i] - target)) * cost[i];
        return res;
    }
};
```

1565. 2449.cpp

Path: LeetCode-main/solutions/2449. Minimum Number of Operations to Make Arrays Similar/2449.cpp

```
class Solution {
public:
    long long makeSimilar(vector<int>& nums, vector<int>& target) {
        long ans = 0;
        vector<vector<int>> A(2); // A[0] := even nums, A[1] := odd nums
        vector<vector<int>> B(2); // B[0] := even target, B[1] := odd target

        for (const int num : nums)
            A[num % 2].push_back(num);

        for (const int num : target)
            B[num % 2].push_back(num);

        ranges::sort(A[0]);
        ranges::sort(A[1]);
        ranges::sort(B[0]);
        ranges::sort(B[1]);

        for (int i = 0; i < 2; ++i)
            for (int j = 0; j < A[i].size(); ++j)
                ans += abs(A[i][j] - B[i][j]) / 2;

        return ans / 2;
    }
};
```

1566. 245.cpp

Path: LeetCode-main/solutions/245. Shortest Word Distance III/245.cpp

```
class Solution {
public:
    int shortestWordDistance(vector<string>& words, string word1, string word2) {
        const bool isSame = word1 == word2;
        int ans = INT_MAX;
        // If word1 == word2, index1 is the newest index.
        int index1 = words.size();
        // If word1 == word2, index2 is the previous index.
        int index2 = -words.size();

        for (int i = 0; i < words.size(); ++i) {
            if (words[i] == word1) {
                if (isSame)
                    index2 = index1;
                index1 = i;
            } else if (words[i] == word2) {
                index2 = i;
            }
            ans = min(ans, abs(index1 - index2));
        }

        return ans;
    }
};
```

1567. 2450.cpp

Path: LeetCode-main/solutions/2450. Number of Distinct Binary Strings After Applying Operations/2450.cpp

```
class Solution {
public:
    int countDistinctStrings(string s, int k) {
        // Since the content of `s` doesn't matter, for each i in [0, n - k], we can
        // flip s[i..i + k] or don't flip it. Therefore, there's  $2^{(n - k + 1)}$  ways.
        return modPow(2, s.length() - k + 1);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

1568. 2451.cpp

Path: LeetCode-main/solutions/2451. Odd String Difference/2451.cpp

```
class Solution {
public:
    string oddString(vector<string>& words) {
        const vector<pair<string, string>> wordAndDiffStrs =
            getWordAndDiffStrs(words);
        unordered_map<string, int> diffStrCount;

        for (const auto& [_, diffStr] : wordAndDiffStrs)
            ++diffStrCount[diffStr];

        for (const auto& [word, diffStr] : wordAndDiffStrs)
            if (diffStrCount[diffStr] == 1)
                return word;

        throw;
    }

private:
    // Returns the pairs of the word and its corresponding difference string.
    // e.g. [("adc", "3#-1#"), ("wzy", "3#-1#"), ("abc", "1#1#")]
    vector<pair<string, string>> getWordAndDiffStrs(const vector<string>& words) {
        vector<pair<string, string>> wordAndDiffStrs;
        for (const string& word : words)
            wordAndDiffStrs.emplace_back(word, getDiffStr(word));
        return wordAndDiffStrs;
    }

    // Returns the difference string of `s`.
    // e.g. getDiffStr("adc") -> "3#-1#"
    string getDiffStr(const string& s) {
        string diffStr;
        for (int i = 1; i < s.length(); ++i)
            diffStr += to_string(s[i] - s[i - 1]) + "#";
        return diffStr;
    }
};
```


1569. 2452.cpp

Path: LeetCode-main/solutions/2452. Words Within Two Edits of Dictionary/2452.cpp

```
class Solution {
public:
    vector<string> twoEditWords(vector<string>& queries,
                               vector<string>& dictionary) {
        vector<string> ans;

        for (const string& query : queries)
            for (const string& word : dictionary)
                if (inner_product(query.begin(), query.end(), word.begin(), 0, plus<>(),
                                   not_equal_to<char>()) < 3) {
                    ans.push_back(q);
                    break;
                }
        return ans;
    }
};
```

1570. 2453-2.cpp

Path: LeetCode-main/solutions/2453. Destroy Sequential Targets/2453-2.cpp

```
class Solution {
public:
    int destroyTargets(vector<int>& nums, int space) {
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num % space];

        return *ranges::max_element(nums, [&count, space](int a, int b) {
            const int countA = count[a % space];
            const int countB = count[b % space];
            return countA == countB ? a > b : countA < countB;
        });
    }
};
```

1571. 2453.cpp

Path: LeetCode-main/solutions/2453. Destroy Sequential Targets/2453.cpp

```
class Solution {
public:
    int destroyTargets(vector<int>& nums, int space) {
        int ans = INT_MAX;
        int maxCount = 0;
        unordered_map<int, int> count;

        for (const int num : nums)
            maxCount = max(maxCount, ++count[num % space]);

        for (const int num : nums)
            if (count[num % space] == maxCount)
                ans = min(ans, num);

        return ans;
    }
};
```

1572. 2454.cpp

Path: LeetCode-main/solutions/2454. Next Greater Element IV/2454.cpp

```
class Solution {
public:
    vector<int> secondGreaterElement(vector<int>& nums) {
        vector<int> ans(nums.size(), -1);
        // a decreasing stack that stores indices that met the first greater number
        stack<int> prevStack;
        // a decreasing stack that stores indices
        stack<int> currStack;

        for (int i = 0; i < nums.size(); ++i) {
            // Indices in `prevStack` meet the second greater number.
            while (!prevStack.empty() && nums[prevStack.top()] < nums[i])
                ans[prevStack.top()] = nums[i], prevStack.pop();
            // Push indices that meet the first greater number from `currStack` to
            // `prevStack`. We need a temporary array to make the indices in the
            // `prevStack` increasing.
            stack<int> decreasingIndices;
            while (!currStack.empty() && nums[currStack.top()] < nums[i])
                decreasingIndices.push(currStack.top()), currStack.pop();
            while (!decreasingIndices.empty())
                prevStack.push(decreasingIndices.top()), decreasingIndices.pop();
            currStack.push(i);
        }

        return ans;
    }
};
```

1573. 2455.cpp

Path: LeetCode-main/solutions/2455. Average Value of Even Numbers That Are Divisible by Three/2455.cpp

```
class Solution {
public:
    int averageValue(vector<int>& nums) {
        int sum = 0;
        int count = 0;

        for (const int num : nums)
            if (num % 6 == 0) {
                sum += num;
                ++count;
            }

        return count == 0 ? 0 : sum / count;
    }
};
```

1574. 2456.cpp

Path: LeetCode-main/solutions/2456. Most Popular Video Creator/2456.cpp

```
struct Creator {
    long popularity; // the popularity sum
    string videoId; // the video id that has the maximum view
    int maxView;    // the maximum view of the creator
};

class Solution {
public:
    vector<vector<string>> mostPopularCreator(vector<string>& creators,
                                              vector<string>& ids,
                                              vector<int>& views) {

        vector<vector<string>> ans;
        unordered_map<string, Creator> nameToCreator;
        long maxPopularity = 0;

        for (int i = 0; i < creators.size(); ++i) {
            if (!nameToCreator.contains(creators[i])) {
                nameToCreator[creators[i]] = Creator{
                    .popularity = views[i],
                    .videoId = ids[i],
                    .maxView = views[i],
                };
                maxPopularity = max(maxPopularity, static_cast<long>(views[i]));
                continue;
            }
            Creator& creator = nameToCreator[creators[i]];
            creator.popularity += views[i];
            maxPopularity = max(maxPopularity, static_cast<long>(creator.popularity));
            if (creator.maxView < views[i] ||
                creator.maxView == views[i] && creator.videoId > ids[i]) {
                creator.videoId = ids[i];
                creator.maxView = views[i];
            }
        }

        for (const auto& [name, creator] : nameToCreator)
            if (creator.popularity == maxPopularity)
                ans.push_back({name, creator.videoId});

        return ans;
    }
};
```

1575. 2457.cpp

Path: LeetCode-main/solutions/2457. Minimum Addition to Make Integer Beautiful/2457.cpp

```
class Solution {
public:
    long long makeIntegerBeautiful(long long n, int target) {
        long ans = 0;
        long power = 1;

        // e.g. n = 123. After turning off the last bit by adding 7, n = 130.
        // Effectively, we can think n as 13. That's why we do n = (n / 10) + 1.
        while (sum(n) > target) {
            // the cost to turn off the last digit
            ans += power * (10 - n % 10);
            n = n / 10 + 1;
            power *= 10;
        }

        return ans;
    }

private:
    int sum(long n) {
        int res = 0;
        while (n > 0) {
            res += n % 10;
            n /= 10;
        }
        return res;
    }
};
```

1576. 2458.cpp

Path: LeetCode-main/solutions/2458. Height of Binary Tree After Subtree Removal Queries/2458.cpp

```
class Solution {
public:
    vector<int> treeQueries(TreeNode* root, vector<int>& queries) {
        vector<int> ans;

        dfs(root, 0, 0);

        for (const int query : queries)
            ans.push_back(valToMaxHeight[query]);

        return ans;
    }

private:
    // valToMaxHeight[val] := the maximum height without the node with `val`
    unordered_map<int, int> valToMaxHeight;
    // valToHeight[val] := the height of the node with `val`
    unordered_map<int, int> valToHeight;

    int height(TreeNode* root) {
        if (root == nullptr)
            return 0;
        if (const auto it = valToHeight.find(root->val); it != valToHeight.cend())
            return it->second;
        return valToHeight[root->val] =
            (1 + max(height(root->left), height(root->right)));
    }

    // maxHeight := the maximum height without the current node `root`
    void dfs(TreeNode* root, int depth, int maxHeight) {
        if (root == nullptr)
            return;
        valToMaxHeight[root->val] = maxHeight;
        dfs(root->left, depth + 1, max(maxHeight, depth + height(root->right)));
        dfs(root->right, depth + 1, max(maxHeight, depth + height(root->left)));
    }
};
```


1577. 246.cpp

Path: LeetCode-main/solutions/246. Strobogrammatic Number/246.cpp

```
class Solution {
public:
    bool isStrobogrammatic(string num) {
        const vector<char> rotated{'0', '1', 'x', 'x', 'x',
                                   'x', '9', 'x', '8', '6'};

        int l = 0;
        int r = num.length() - 1;

        while (l <= r)
            if (num[l++] != rotated[num[r--] - '0'])
                return false;

        return true;
    }
};
```

1578. 2460-2.cpp

Path: LeetCode-main/solutions/2460. Apply Operations to an Array/2460-2.cpp

```
class Solution {
public:
    vector<int> applyOperations(vector<int>& nums) {
        for (int i = 0, j = 0; i < nums.size(); ++i) {
            if (i + 1 < nums.size() && nums[i] == nums[i + 1]) {
                nums[i] *= 2;
                nums[i + 1] = 0;
            }
            if (nums[i] > 0)
                swap(nums[i], nums[j++]);
        }
        return nums;
    }
};
```

1579. 2460.cpp

Path: LeetCode-main/solutions/2460. Apply Operations to an Array/2460.cpp

```
class Solution {
public:
    vector<int> applyOperations(vector<int>& nums) {
        vector<int> ans(nums.size());

        for (int i = 0; i + 1 < nums.size(); ++i)
            if (nums[i] == nums[i + 1]) {
                nums[i] *= 2;
                nums[i + 1] = 0;
            }

        int i = 0;
        for (const int num : nums)
            if (num > 0)
                ans[i++] = num;

        return ans;
    }
};
```

1580. 2461.cpp

Path: LeetCode-main/solutions/2461. Maximum Sum of Distinct Subarrays With Length K/2461.cpp

```
class Solution {
public:
    long long maximumSubarraySum(vector<int>& nums, int k) {
        long ans = 0;
        long sum = 0;
        int distinct = 0;
        unordered_map<int, int> count;

        for (int i = 0; i < nums.size(); ++i) {
            sum += nums[i];
            if (++count[nums[i]] == 1)
                ++distinct;
            if (i >= k) {
                if (--count[nums[i - k]] == 0)
                    --distinct;
                sum -= nums[i - k];
            }
            if (i >= k - 1 && distinct == k)
                ans = max(ans, sum);
        }
        return ans;
    }
};
```

1581. 2462.cpp

Path: LeetCode-main/solutions/2462. Total Cost to Hire K Workers/2462.cpp

```
class Solution {
public:
    long long totalCost(vector<int>& costs, int k, int candidates) {
        long ans = 0;
        int i = 0;
        int j = costs.size() - 1;
        priority_queue<int, vector<int>, greater<>> minHeapL;
        priority_queue<int, vector<int>, greater<>> minHeapR;

        for (int hired = 0; hired < k; ++hired) {
            while (minHeapL.size() < candidates && i <= j)
                minHeapL.push(costs[i++]);
            while (minHeapR.size() < candidates && i <= j)
                minHeapR.push(costs[j--]);
            if (minHeapL.empty())
                ans += minHeapR.top(), minHeapR.pop();
            else if (minHeapR.empty())
                ans += minHeapL.top(), minHeapL.pop();
            // Both `minHeapL` and `minHeapR` are not empty.
            else if (minHeapL.top() <= minHeapR.top())
                ans += minHeapL.top(), minHeapL.pop();
            else
                ans += minHeapR.top(), minHeapR.pop();
        }

        return ans;
    }
};
```

1582. 2463.cpp

Path: LeetCode-main/solutions/2463. Minimum Total Distance Traveled/2463.cpp

```
class Solution {
public:
    long long minimumTotalDistance(vector<int>& robot,
                                   vector<vector<int>>& factory) {
        ranges::sort(robot);
        ranges::sort(factory);
        vector<vector<vector<long>>> mem(
            robot.size(),
            vector<vector<long>>(factory.size(), vector<long>(robot.size())));
        return minimumTotalDistance(robot, factory, 0, 0, 0, mem);
    }

private:
    // Returns the minimum distance to fix robot[i..n) with factory[j..n), where
    // factory[j] already fixed k robots.
    long minimumTotalDistance(const vector<int>& robot,
                              const vector<vector<int>>& factory, int i, int j,
                              int k, vector<vector<vector<long>>>& mem) {
        if (i == robot.size())
            return 0;
        if (j == factory.size())
            return LONG_MAX;
        if (mem[i][j][k] > 0)
            return mem[i][j][k];
        const long skipFactory =
            minimumTotalDistance(robot, factory, i, j + 1, 0, mem);
        const int position = factory[j][0];
        const int limit = factory[j][1];
        const long useFactory =
            limit > k ? minimumTotalDistance(robot, factory, i + 1, j, k + 1, mem) +
                        abs(robot[i] - position)
                      : LONG_MAX / 2;
        return mem[i][j][k] = min(skipFactory, useFactory);
    }
};
```

1583. 2464.cpp

Path: LeetCode-main/solutions/2464. Minimum Subarrays in a Valid Split/2464.cpp

```
class Solution {
public:
    int validSubarraySplit(vector<int>& nums) {
        constexpr int kMax = INT_MAX / 2;
        // dp[i] := the minimum number of subarrays to validly split nums[0..i]
        vector<int> dp(nums.size(), kMax);

        for (int i = 0; i < nums.size(); ++i)
            for (int j = 0; j <= i; ++j)
                if (__gcd(nums[j], nums[i]) > 1)
                    dp[i] = min(dp[i], j == 0 ? 1 : dp[j - 1] + 1);

        return dp.back() == kMax ? -1 : dp.back();
    }
};
```

1584. 2465.cpp

Path: LeetCode-main/solutions/2465. Number of Distinct Averages/2465.cpp

```
class Solution {
public:
    int distinctAverages(vector<int>& nums) {
        const int n = nums.size();
        unordered_set<int> sums;

        ranges::sort(nums);

        for (int i = 0; i < n / 2; ++i)
            sums.insert(nums[i] + nums[n - 1 - i]);

        return sums.size();
    }
};
```


1585. 2466.cpp

Path: LeetCode-main/solutions/2466. Count Ways To Build Good Strings/2466.cpp

```
class Solution {
public:
    int countGoodStrings(int low, int high, int zero, int one) {
        constexpr int kMod = 1'000'000'007;
        int ans = 0;
        // dp[i] := the number of good strings with length i
        vector<int> dp(high + 1);
        dp[0] = 1;

        for (int i = 1; i <= high; ++i) {
            if (i >= zero)
                dp[i] = (dp[i] + dp[i - zero]) % kMod;
            if (i >= one)
                dp[i] = (dp[i] + dp[i - one]) % kMod;
            if (i >= low)
                ans = (ans + dp[i]) % kMod;
        }

        return ans;
    }
};
```

1586. 2467.cpp

Path: LeetCode-main/solutions/2467. Most Profitable Path in a Tree/2467.cpp

```
class Solution {
public:
    int mostProfitablePath(vector<vector<int>>& edges, int bob,
                          vector<int>& amount) {
        const int n = amount.size();
        vector<vector<int>> tree(n);
        vector<int> parent(n);
        vector<int> aliceDist(n, -1);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        dfs(tree, 0, -1, 0, parent, aliceDist);

        // Modify the amount along the path from node Bob to node 0.
        // For each node,
        // 1. If Bob reaches earlier than Alice does, change the amount to 0.
        // 2. If Bob and Alice reach simultaneously, divide the amount by 2.
        for (int u = bob, bobDist = 0; u != 0; u = parent[u], ++bobDist)
            if (bobDist < aliceDist[u])
                amount[u] = 0;
            else if (bobDist == aliceDist[u])
                amount[u] /= 2;

        return getMoney(tree, 0, -1, amount);
    }

private:
    // Fills `parent` and `dist`.
    void dfs(const vector<vector<int>>& tree, int u, int prev, int d,
            vector<int>& parent, vector<int>& dist) {
        parent[u] = prev;
        dist[u] = d;
        for (const int v : tree[u]) {
            if (dist[v] == -1)
                dfs(tree, v, u, d + 1, parent, dist);
        }
    }

    int getMoney(const vector<vector<int>>& tree, int u, int prev,
                const vector<int>& amount) {
        // a leaf node
        if (tree[u].size() == 1 && tree[u][0] == prev)
            return amount[u];

        int maxPath = INT_MIN;
        for (const int v : tree[u])
            if (v != prev)
                maxPath = max(maxPath, getMoney(tree, v, u, amount));

        return amount[u] + maxPath;
    }
};
```

1587. 2468.cpp

Path: LeetCode-main/solutions/2468. Split Message Based on Limit/2468.cpp

```
class Solution {
public:
    vector<string> splitMessage(string message, int limit) {
        const int kMessageLength = message.length();
        int b = 1;
        // the total length of a: initialized with the length of "1"
        int aLength = sz(1);

        // the total length of b := b * sz(b)
        // The total length of "</>" := b * 3
        while (b * limit < b * (sz(b) + 3) + aLength + kMessageLength) {
            // If the length of the last suffix "<b/b>" := sz(b) * 2 + 3 >= limit,
            // then it's impossible that the length of "*<b/b>" <= limit.
            if (sz(b) * 2 + 3 >= limit)
                return {};
            aLength += sz(++b);
        }

        vector<string> ans;

        for (int i = 0, a = 1; a <= b; ++a) {
            // the length of "<a/b>" := sz(a) + sz(b) + 3
            const int j = limit - (sz(a) + sz(b) + 3);
            ans.push_back(message.substr(i, j) + "<" + to_string(a) + "/" +
                          to_string(b) + ">");
            i += j;
        }

        return ans;
    }

private:
    int sz(int num) {
        return to_string(num).length();
    }
};
```

1588. 2469.cpp

Path: LeetCode-main/solutions/2469. Convert the Temperature/2469.cpp

```
class Solution {  
public:  
    vector<double> convertTemperature(double celsius) {  
        return {celsius + 273.15, celsius * 1.8 + 32};  
    }  
};
```

1589. 247.cpp

Path: LeetCode-main/solutions/247. Strobogrammatic Number II/247.cpp

```
class Solution {
public:
    vector<string> findStrobogrammatic(int n) {
        return helper(n, n);
    }

private:
    vector<string> helper(int n, int k) {
        if (n == 0)
            return {""};
        if (n == 1)
            return {"0", "1", "8"};

        vector<string> ans;

        for (const string& inner : helper(n - 2, k)) {
            if (n < k)
                ans.push_back("0" + inner + "0");
            ans.push_back("1" + inner + "1");
            ans.push_back("6" + inner + "9");
            ans.push_back("8" + inner + "8");
            ans.push_back("9" + inner + "6");
        }

        return ans;
    }
};
```

1590. 2470.cpp

Path: LeetCode-main/solutions/2470. Number of Subarrays With LCM Equal to K/2470.cpp

```
class Solution {
public:
    int subarrayLCM(vector<int>& nums, int k) {
        int ans = 0;

        for (int i = 0; i < nums.size(); ++i) {
            int runningLcm = nums[i];
            for (int j = i; j < nums.size(); ++j) {
                runningLcm = std::lcm(runningLcm, nums[j]);
                if (runningLcm > k)
                    break;
                if (runningLcm == k)
                    ++ans;
            }
        }

        return ans;
    }
};
```

1591. 2471.cpp

Path: LeetCode-main/solutions/2471. Minimum Number of Operations to Sort a Binary Tree by Level/2471.cpp

```
class Solution {
public:
    int minimumOperations(TreeNode* root) {
        int ans = 0;
        queue
```

1592. 2472.cpp

Path: LeetCode-main/solutions/2472. Maximum Number of Non-overlapping Palindrome Substrings/2472.cpp

```
class Solution {
public:
    int maxPalindromes(string s, int k) {
        const int n = s.length();
        // dp[i] := the maximum number of substrings in the first i chars of s
        vector<int> dp(n + 1);

        // If a palindrome is a substring of another palindrome, then considering
        // the longer palindrome won't increase the number of non-overlapping
        // palindromes. So, we only need to consider the shorter one. Also,
        // considering palindromes with both k length and k + 1 length ensures that
        // we look for both even and odd length palindromes.
        for (int i = k; i <= n; ++i) {
            dp[i] = dp[i - 1];
            // Consider palindrome with length k.
            if (isPalindrome(s, i - k, i - 1))
                dp[i] = max(dp[i], 1 + dp[i - k]);
            // Consider palindrome with length k + 1.
            if (isPalindrome(s, i - k - 1, i - 1))
                dp[i] = max(dp[i], 1 + dp[i - k - 1]);
        }

        return dp[n];
    }

private:
    // Returns true if s[i..j] is a palindrome.
    bool isPalindrome(const string& s, int l, int r) {
        if (l < 0)
            return false;
        while (l < r)
            if (s[l++] != s[r--])
                return false;
        return true;
    }
};
```


1593. 2473.cpp

Path: LeetCode-main/solutions/2473. Minimum Cost to Buy Apples/2473.cpp

```
class Solution {
public:
    vector<long long> minCost(int n, vector<vector<int>>& roads,
                             vector<int>& appleCost, int k) {
        vector<long long> ans;
        vector<vector<pair<int, long>>> graph(n);

        for (const vector<int>& road : roads) {
            const int u = road[0] - 1;
            const int v = road[1] - 1;
            const int w = road[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        for (int i = 0; i < n; ++i)
            ans.push_back(dijkstra(graph, i, appleCost, k));

        return ans;
    }

private:
    long dijkstra(const vector<vector<pair<int, long>>>& graph, int src,
                  const vector<int>& appleCost, int k) {
        long ans = LONG_MAX;
        vector<long> dist(graph.size(), LONG_MAX);

        dist[src] = 0;
        using P = pair<long, int>; // (d, u)
        priority_queue<P, vector<P>, greater<>> minHeap;
        minHeap.emplace(dist[src], src);

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (d > dist[u])
                continue;
            ans = min(ans, appleCost[u] + (k + 1) * d);
            for (const auto& [v, w] : graph[u])
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.emplace(dist[v], v);
                }
        }

        return ans;
    }
};
```

1594. 2475.cpp

Path: LeetCode-main/solutions/2475. Number of Unequal Triplets in Array/2475.cpp

```
// Assume that we have 4 kinds of numbers a, b, c, and d in the count map.
//
// What we want is:
// cnt[a] * cnt[b] * cnt[c]
// cnt[a] * cnt[b] * cnt[d]
// cnt[a] * cnt[c] * cnt[d]
// cnt[b] * cnt[c] * cnt[d]
//
// The above combinations can be reduced as:
//
// prev | curr | next
// -----
// (0) * cnt[a] * (cnt[b] + cnt[c] + cnt[d])
// (cnt[a]) * cnt[b] * (cnt[c] + cnt[d])
// (cnt[a] + cnt[b]) * cnt[c] * (cnt[d])
// (cnt[a] + cnt[b] + cnt[c]) * cnt[d] * (0)

class Solution {
public:
    int unequalTriplets(vector<int>& nums) {
        int ans = 0;
        int prev = 0;
        int next = nums.size();
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        for (const auto& [_, freq] : count) {
            next -= freq;
            ans += prev * freq * next;
            prev += freq;
        }

        return ans;
    }
};
```

1595. 2476.cpp

Path: LeetCode-main/solutions/2476. Closest Nodes Queries in a Binary Search Tree/2476.cpp

```
class Solution {
public:
    vector<vector<int>> closestNodes(TreeNode* root, vector<int>& queries) {
        vector<vector<int>> ans;
        vector<int> sortedVals;

        inorder(root, sortedVals);

        for (const int query : queries) {
            const auto it = ranges::lower_bound(sortedVals, query);
            // query is presented in the tree, so just use {query, query}.
            if (it != sortedVals.cend() && *it == query)
                ans.push_back({query, query});
            // query isn't presented in the tree, so find the closest one if possible.
            else
                ans.push_back({it == sortedVals.cbegin() ? -1 : *prev(it),
                               it == sortedVals.cend() ? -1 : *it});
        }

        return ans;
    }

private:
    // Walks the BST to collect the sorted numbers.
    void inorder(TreeNode* root, vector<int>& sortedVals) {
        if (root == nullptr)
            return;
        inorder(root->left, sortedVals);
        sortedVals.push_back(root->val);
        inorder(root->right, sortedVals);
    }
};
```

1596. 2477.cpp

Path: LeetCode-main/solutions/2477. Minimum Fuel Cost to Report to the Capital/2477.cpp

```
class Solution {
public:
    long long minimumFuelCost(vector<vector<int>>& roads, int seats) {
        long ans = 0;
        vector<vector<int>> tree(roads.size() + 1);

        for (const vector<int>& road : roads) {
            const int u = road[0];
            const int v = road[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        dfs(tree, 0, -1, seats, ans);
        return ans;
    }

private:
    int dfs(const vector<vector<int>>& tree, int u, int prev, int seats,
            long& ans) {
        int people = 1;
        for (const int v : tree[u])
            if (v != prev)
                people += dfs(tree, v, u, seats, ans);
        if (u > 0)
            // the number of cars needed = ceil(people / seats)
            ans += (people + seats - 1) / seats;
        return people;
    }
};
```

1597. 2478.cpp

Path: LeetCode-main/solutions/2478. Number of Beautiful Partitions/2478.cpp

```
class Solution {
public:
    int beautifulPartitions(string s, int k, int minLength) {
        if (!isPrime(s.front()) || isPrime(s.back()))
            return 0;
        vector<vector<int>> mem(s.length(), vector<int>(k, -1));
        return beautifulPartitions(s, minLength, k - 1, minLength, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of beautiful partitions of s[i..n) with k bars (|) left.
    int beautifulPartitions(const string& s, int i, int k, int minLength,
                           vector<vector<int>>& mem) {
        if (i <= s.length() && k == 0)
            return 1;
        if (i >= s.length())
            return 0;
        if (mem[i][k] != -1)
            return mem[i][k];

        // Don't split between s[i - 1] and s[i].
        int res = beautifulPartitions(s, i + 1, k, minLength, mem) % kMod;

        // Split between s[i - 1] and s[i].
        if (isPrime(s[i]) && !isPrime(s[i - 1]))
            res += beautifulPartitions(s, i + minLength, k - 1, minLength, mem);

        return mem[i][k] = res % kMod;
    }

    bool isPrime(char c) {
        return c == '2' || c == '3' || c == '5' || c == '7';
    }
};
```

1598. 2479.cpp

Path: LeetCode-main/solutions/2479. Maximum XOR of Two Non-Overlapping Subtrees/2479.cpp

```
struct TrieNode {
    vector<shared_ptr<TrieNode>> children;
    TrieNode() : children(2) {}
};

class BitTrie {
public:
    BitTrie(int maxBit) : maxBit(maxBit) {}

    void insert(long num) {
        shared_ptr<TrieNode> node = root;
        for (int i = maxBit; i >= 0; --i) {
            const int bit = num >> i & 1;
            if (node->children[bit] == nullptr)
                node->children[bit] = make_shared<TrieNode>();
            node = node->children[bit];
        }
    }

    long getMaxXor(long num) {
        long maxXor = 0;
        shared_ptr<TrieNode> node = root;
        for (int i = maxBit; i >= 0; --i) {
            const int bit = num >> i & 1;
            const int toggleBit = bit ^ 1;
            if (node->children[toggleBit] != nullptr) {
                maxXor = maxXor | 1L << i;
                node = node->children[toggleBit];
            } else if (node->children[bit] != nullptr) {
                node = node->children[bit];
            } else { // There's nothing in the Bit Trie.
                return 0;
            }
        }
        return maxXor;
    }

private:
    const int maxBit;
    shared_ptr<TrieNode> root = make_shared<TrieNode>();
};

class Solution {
public:
    long long maxXor(int n, vector<vector<int>>& edges, vector<int>& values) {
        long ans = 0;
        vector<vector<int>> tree(n);
        vector<long> treeSums(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        getTreeSum(tree, 0, -1, treeSums, values);
        const long maxSubTreeSum =
            *max_element(treeSums.begin() + 1, treeSums.end());
        const int maxBit = static_cast<int>(log2(maxSubTreeSum));
        // Similar to 421. Maximum XOR of Two Numbers in an Array
        dfs(tree, 0, -1, treeSums, BitTrie(maxBit), ans);
        return ans;
    }

private:
    // Gets the tree sum rooted at node u.
    long getTreeSum(const vector<vector<int>>& tree, int u, int prev,
        vector<long>& treeSums, const vector<int>& values) {
        long treeSum = values[u];
        for (const int v : tree[u])
            if (v != prev)
                treeSum += getTreeSum(tree, v, u, treeSums, values);
        treeSums[u] = treeSum;
        return treeSum;
    }

    void dfs(const vector<vector<int>>& tree, int u, int prev,
        const vector<long>& treeSums, BitTrie&& bitTrie, long& ans) {
        for (const int v : tree[u]) {
```

```
    if (v == prev)
        continue;
    // Preorder to get the ans.
    ans = max(ans, bitTrie.getMaxXor(treeSums[v]));
    // Recursively call on the subtree rooted at node v.
    dfs(tree, v, u, treeSums, std::move(bitTrie), ans);
    // Postorder insert the tree sum rooted at node v.
    bitTrie.insert(treeSums[v]);
}
};
```

1599. 248.cpp

Path: LeetCode-main/solutions/248. Strobogrammatic Number III/248.cpp

```
class Solution {
public:
    int strobogrammaticInRange(string low, string high) {
        int ans = 0;

        for (int n = low.length(); n <= high.length(); ++n) {
            string s(n, ' ');
            dfs(low, high, s, 0, n - 1, ans);
        }

        return ans;
    }

private:
    const vector<pair<char, char>> pairs{
        {'0', '0'}, {'1', '1'}, {'6', '9'}, {'8', '8'}, {'9', '6'}};

    void dfs(const string& low, const string& high, string& s, int l, int r,
             int& ans) {
        if (l > r) {
            if (s.length() == low.length() && s < low)
                return;
            if (s.length() == high.length() && s > high)
                return;
            ++ans;
            return;
        }

        for (const auto& [leftDigit, rightDigit] : pairs) {
            if (l == r && leftDigit != rightDigit)
                continue;
            s[l] = leftDigit;
            s[r] = rightDigit;
            if (s.length() > 1 && s[0] == '0')
                continue;
            dfs(low, high, s, l + 1, r - 1, ans);
        }
    }
};
```


1600. 2481.cpp

Path: LeetCode-main/solutions/2481. Minimum Cuts to Divide a Circle/2481.cpp

```
class Solution {
public:
    int numberOfCuts(int n) {
        if (n == 1)
            return 0;
        return n % 2 == 0 ? n / 2 : n;
    }
};
```

1601. 2482.cpp

Path: LeetCode-main/solutions/2482. Difference Between Ones and Zeros in Row and Column/2482.cpp

```
class Solution {
public:
    vector<vector<int>> onesMinusZeros(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<int>> ans(m, vector<int>(n));
        vector<int> onesRow;
        vector<int> onesCol;

        for (const vector<int>& row : grid)
            onesRow.push_back(ranges::count(row, 1));

        for (int j = 0; j < n; ++j) {
            int ones = 0;
            for (int i = 0; i < m; ++i)
                ones += grid[i][j];
            onesCol.push_back(ones);
        }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                ans[i][j] =
                    onesRow[i] + onesCol[j] - (n - onesRow[i]) - (m - onesCol[j]);

        return ans;
    }
};
```

1602. 2483.cpp

Path: LeetCode-main/solutions/2483. Minimum Penalty for a Shop/2483.cpp

```
class Solution {
public:
    int bestClosingTime(string customers) {
        // Instead of computing the minimum penalty, we can compute the maximum
        // profit.
        int ans = 0;
        int profit = 0;
        int maxProfit = 0;

        for (int i = 0; i < customers.length(); ++i) {
            profit += customers[i] == 'Y' ? 1 : -1;
            if (profit > maxProfit) {
                maxProfit = profit;
                ans = i + 1;
            }
        }

        return ans;
    }
};
```

1603. 2484.cpp

Path: LeetCode-main/solutions/2484. Count Palindromic Subsequences/2484.cpp

```
class Solution {
public:
    int countPalindromes(string s) {
        constexpr int kMod = 1'000'000'007;
        constexpr int kPatternSize = 5;
        long ans = 0;

        for (char a = '0'; a <= '9'; ++a)
            for (char b = '0'; b <= '9'; ++b) {
                const vector<char> pattern{a, b, '.', b, a};
                // dp[i] := the number of subsequences of pattern[i..n) in s, where
                // pattern[2] can be any character
                vector<long> dp(kPatternSize + 1);
                dp.back() = 1;
                for (const char c : s)
                    for (int i = 0; i < kPatternSize; ++i)
                        if (pattern[i] == '.' || pattern[i] == c)
                            dp[i] += dp[i + 1];
                ans += dp[0];
                ans %= kMod;
            }
        return ans;
    }
};
```

1604. 2485.cpp

Path: LeetCode-main/solutions/2485. Find the Pivot Integer/2485.cpp

```
class Solution {
public:
    int pivotInteger(int n) {
        // 1 + 2 + ... + x = x + ... + n
        // (1 + x) * x / 2 = (x + n) * (n - x + 1) / 2
        // x + x^2 = nx - x^2 + x + n^2 - nx + n
        // 2 * x^2 = n^2 + n
        // x = sqrt((n^2 + n) / 2)
        const int y = (n * n + n) / 2;
        const int x = sqrt(y);
        return x * x == y ? x : -1;
    }
};
```

1605. 2486.cpp

Path: LeetCode-main/solutions/2486. Append Characters to String to Make Subsequence/2486.cpp

```
class Solution {
public:
    int appendCharacters(string s, string t) {
        int i = 0; // t's index

        for (const char c : s)
            if (c == t[i])
                if (++i == t.length())
                    return 0;

        return t.length() - i;
    }
};
```

1606. 2487.cpp

Path: LeetCode-main/solutions/2487. Remove Nodes From Linked List/2487.cpp

```
class Solution {
public:
    ListNode* removeNodes(ListNode* head) {
        if (head == nullptr)
            return nullptr;
        head->next = removeNodes(head->next);
        return head->next != nullptr && head->val < head->next->val ? head->next
            : head;
    }
};
```

1607. 2488.cpp

Path: LeetCode-main/solutions/2488. Count Subarrays With Median K/2488.cpp

```
class Solution {
public:
    int countSubarrays(vector<int>& nums, int k) {
        const int kIndex = find(nums.begin(), nums.end(), k) - nums.begin();
        int ans = 0;
        unordered_map<int, int> count;

        for (int i = kIndex, balance = 0; i >= 0; --i) {
            if (nums[i] < k)
                --balance;
            else if (nums[i] > k)
                ++balance;
            ++count[balance];
        }

        for (int i = kIndex, balance = 0; i < nums.size(); ++i) {
            if (nums[i] < k)
                --balance;
            else if (nums[i] > k)
                ++balance;
            // The subarray that has balance == 0 or 1 having median equal to k.
            // So, add count[0 - balance] and count[1 - balance] to `ans`.
            ans += count[-balance] + count[1 - balance];
        }

        return ans;
    }
};
```


1608. 2489.cpp

Path: LeetCode-main/solutions/2489. Number of Substrings With Fixed Ratio/2489.cpp

```
class Solution {
public:
    long long fixedRatio(string s, int num1, int num2) {
        // Let x := the number of 0s and y := the number of 1s in the subarray.
        // We want x : y = num1 : num2, so our goal is to find number of subarrays
        // with x * num2 - y * num1 = 0. To achieve this, we can use a prefix count
        // map to record the count of the running x * num2 - y * num1. If the
        // running x * num2 - y * num1 = prefix, then add count[prefix] to the
        // `ans`.
        long ans = 0;
        long prefix = 0;
        unordered_map<long, int> prefixCount{{0, 1}};

        for (const char c : s) {
            if (c == '0')
                prefix += num2;
            else // c == '1'
                prefix -= num1;
            ans += prefixCount[prefix];
            ++prefixCount[prefix];
        }

        return ans;
    }
};
```

1609. 249.cpp

Path: LeetCode-main/solutions/249. Group Shifted Strings/249.cpp

```
class Solution {
public:
    vector<vector<string>> groupStrings(vector<string>& strings) {
        vector<vector<string>> ans;
        unordered_map<string, vector<string>> keyToStrings;

        for (const string& s : strings)
            keyToStrings[getKey(s)].push_back(s);

        for (const auto& [_, strings] : keyToStrings)
            ans.push_back(strings);

        return ans;
    }

private:
    // Returns the key of 's' by pairwise calculation of differences.
    // e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
    string getKey(const string& s) {
        string key;

        for (int i = 1; i < s.length(); ++i) {
            const int diff = (s[i] - s[i - 1] + 26) % 26;
            key += to_string(diff) + ",";
        }

        return key;
    }
};
```

1610. 2490.cpp

Path: LeetCode-main/solutions/2490. Circular Sentence/2490.cpp

```
class Solution {
public:
    bool isCircularSentence(string sentence) {
        for (int i = 0; i < sentence.length(); ++i)
            if (sentence[i] == ' ' && sentence[i - 1] != sentence[i + 1])
                return false;
        return sentence.front() == sentence.back();
    }
};
```

1611. 2491.cpp

Path: LeetCode-main/solutions/2491. Divide Players Into Teams of Equal Skill/2491.cpp

```
class Solution {
public:
    long long dividePlayers(vector<int>& skill) {
        const int n = skill.size();
        const int teamSkill = accumulate(skill.begin(), skill.end(), 0) / (n / 2);
        long ans = 0;
        unordered_map<int, int> count;

        for (const int s : skill)
            ++count[s];

        for (const auto& [s, freq] : count) {
            const int requiredSkill = teamSkill - s;
            if (const auto it = count.find(requiredSkill);
                it == count.cend() || it->second != freq)
                return -1;
            ans += static_cast<long>(s) * requiredSkill * freq;
        }

        return ans / 2;
    }
};
```

1612. 2492.cpp

Path: LeetCode-main/solutions/2492. Minimum Score of a Path Between Two Cities/2492.cpp

```
class Solution {
public:
    int minScore(int n, vector<vector<int>>& roads) {
        int ans = INT_MAX;
        vector<vector<pair<int, int>>> graph(n); // graph[u] := [(v, distance)]
        queue<int> q{{0}};
        vector<bool> seen(n);
        seen[0] = true;

        for (const vector<int>& r : roads) {
            const int u = r[0] - 1;
            const int v = r[1] - 1;
            const int distance = r[2];
            graph[u].emplace_back(v, distance);
            graph[v].emplace_back(u, distance);
        }

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            for (const auto& [v, d] : graph[u]) {
                ans = min(ans, d);
                if (seen[v])
                    continue;
                q.push(v);
                seen[v] = true;
            }
        }

        return ans;
    }
};
```

1613. 2493.cpp

Path: LeetCode-main/solutions/2493. Divide Nodes Into the Maximum Number of Groups/2493.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

private:
    vector<int> id;
    vector<int> rank;
};

class Solution {
public:
    int magnificentSets(int n, vector<vector<int>>& edges) {
        vector<vector<int>> graph(n);
        UnionFind uf(n);
        unordered_map<int, int> rootToGroupSize;

        for (const vector<int>& edge : edges) {
            const int u = edge[0] - 1;
            const int v = edge[1] - 1;
            graph[u].push_back(v);
            graph[v].push_back(u);
            uf.unionByRank(u, v);
        }

        for (int i = 0; i < n; ++i) {
            const int newGroupSize = bfs(graph, i);
            if (newGroupSize == -1)
                return -1;
            const int root = uf.find(i);
            auto& groupSize = rootToGroupSize[root];
            groupSize = max(groupSize, newGroupSize);
        }

        int ans = 0;
        for (const auto& [_ , groupSize] : rootToGroupSize)
            ans += groupSize;

        return ans;
    }

private:
    int bfs(const vector<vector<int>>& graph, int u) {
        int step = 0;
        queue<int> q{{u}};
        unordered_map<int, int> nodeToStep{{u, 1}};

        while (!q.empty()) {
            ++step;
            for (int sz = q.size(); sz > 0; --sz) {
                const int u = q.front();
                q.pop();
                for (const int v : graph[u]) {
                    if (!nodeToStep.contains(v)) {
                        q.push(v);
                        nodeToStep[v] = step + 1;
                    } else if (nodeToStep[v] == step) {
                        // There is an odd number of edges in the cycle.
                        return -1;
                    }
                }
            }
        }
    }
};
```

```
    }  
  }  
}  
return step;  
};
```

1614. 2495.cpp

Path: LeetCode-main/solutions/2495. Number of Subarrays Having Even Product/2495.cpp

```
class Solution {
public:
    long long evenProduct(vector<int>& nums) {
        long ans = 0;
        int numsBeforeEven = 0; // inclusively

        // e.g. nums = [1, 0, 1, 1, 0].
        // After meeting the first 0, set `numsBeforeEven` to 2. So, the number
        // between index 1 to index 3 (the one before next 0) will contribute 2 to
        // `ans`.
        for (int i = 0; i < nums.size(); ++i) {
            if (nums[i] % 2 == 0)
                numsBeforeEven = i + 1;
            ans += numsBeforeEven;
        }

        return ans;
    }
};
```


1615. 2496.cpp

Path: LeetCode-main/solutions/2496. Maximum Value of a String in an Array/2496.cpp

```
class Solution {
public:
    int maximumValue(vector<string>& strs) {
        int ans = 0;
        for (const string& s : strs)
            ans = max(ans, ranges::any_of(s, [](char c) { return isalpha(c); })
                ? static_cast<int>(s.length())
                : stoi(s));
        return ans;
    }
};
```

1616. 2497.cpp

Path: LeetCode-main/solutions/2497. Maximum Star Sum of a Graph/2497.cpp

```
class Solution {
public:
    int maxStarSum(vector<int>& vals, vector<vector<int>>& edges, int k) {
        const int n = vals.size();
        int ans = INT_MIN;
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].emplace_back(v, vals[v]);
            graph[v].emplace_back(u, vals[u]);
        }

        for (int i = 0; i < n; ++i) {
            priority_queue<int> maxHeap;
            for (const auto& [_, val] : graph[i])
                if (val > 0)
                    maxHeap.push(val);
            int starSum = vals[i];
            for (int j = 0; j < k && !maxHeap.empty(); ++j)
                starSum += maxHeap.top(), maxHeap.pop();
            ans = max(ans, starSum);
        }

        return ans;
    }
};
```

1617. 2498.cpp

Path: LeetCode-main/solutions/2498. Frog Jump II/2498.cpp

```
class Solution {
public:
    int maxJump(vector<int>& stones) {
        // Let's denote the forwarding path as F and the backwarding path as B.
        // "F1 B2 B1 F2" is no better than "F1 B2 F2 B1" since the distance between
        // F1 and F2 increase, resulting a larger `ans`.
        int ans = stones[1] - stones[0]; // If there're only two stones.
        for (int i = 2; i < stones.size(); ++i)
            ans = max(ans, stones[i] - stones[i - 2]);
        return ans;
    }
};
```

1618. 2499.cpp

Path: LeetCode-main/solutions/2499. Minimum Total Cost to Make Arrays Unequal/2499.cpp

```
class Solution {
public:
    long long minimumTotalCost(vector<int>& nums1, vector<int>& nums2) {
        const int n = nums1.size();
        long ans = 0;
        int maxFreq = 0;
        int maxFreqNum = 0;
        int shouldBeSwapped = 0;
        vector<int> conflictedNumCount(n + 1);

        // Collect the indices i s.t. nums1[i] == nums2[i] and record their
        // `maxFreq` and `maxFreqNum`.
        for (int i = 0; i < n; ++i)
            if (nums1[i] == nums2[i]) {
                const int conflictedNum = nums1[i];
                if (++conflictedNumCount[conflictedNum] > maxFreq) {
                    maxFreq = conflictedNumCount[conflictedNum];
                    maxFreqNum = conflictedNum;
                }
                ++shouldBeSwapped;
                ans += i;
            }

        // Collect the indices with nums1[i] != nums2[i] that contribute less cost.
        // This can be greedily achieved by iterating from 0 to n - 1.
        for (int i = 0; i < n; ++i) {
            // Since we have over `maxFreq * 2` spaces, `maxFreqNum` can be
            // successfully distributed, so no need to collect extra spaces.
            if (maxFreq * 2 <= shouldBeSwapped)
                break;
            if (nums1[i] == nums2[i])
                continue;
            // The numbers == `maxFreqNum` worsen the result since they increase the
            // `maxFreq`.
            if (nums1[i] == maxFreqNum || nums2[i] == maxFreqNum)
                continue;
            ++shouldBeSwapped;
            ans += i;
        }

        return maxFreq * 2 > shouldBeSwapped ? -1 : ans;
    }
};
```

1619. 25-2.cpp

Path: LeetCode-main/solutions/25. Reverse Nodes in k-Group/25-2.cpp

```
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        if (!head || k == 1)
            return head;

        const int length = getLength(head);
        ListNode dummy(0, head);
        ListNode* prev = &dummy;
        ListNode* curr = head;

        for (int i = 0; i < length / k; ++i) {
            for (int j = 0; j < k - 1; ++j) {
                ListNode* next = curr->next;
                curr->next = next->next;
                next->next = prev->next;
                prev->next = next;
            }
            prev = curr;
            curr = curr->next;
        }

        return dummy.next;
    }

private:
    int getLength(ListNode* head) {
        int length = 0;
        for (ListNode* curr = head; curr; curr = curr->next)
            ++length;
        return length;
    }
};
```

1620. 25.cpp

Path: LeetCode-main/solutions/25. Reverse Nodes in k-Group/25.cpp

```
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        if (head == nullptr)
            return nullptr;

        ListNode* tail = head;

        for (int i = 0; i < k; ++i) {
            // There are less than k nodes in the list, do nothing.
            if (tail == nullptr)
                return head;
            tail = tail->next;
        }

        ListNode* newHead = reverse(head, tail);
        head->next = reverseKGroup(tail, k);
        return newHead;
    }

private:
    // Reverses [head, tail).
    ListNode* reverse(ListNode* head, ListNode* tail) {
        ListNode* prev = nullptr;
        ListNode* curr = head;
        while (curr != tail) {
            ListNode* next = curr->next;
            curr->next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }
};
```

1621. 250.cpp

Path: LeetCode-main/solutions/250. Count Unival Subtrees/250.cpp

```
class Solution {
public:
    int countUnivalSubtrees(TreeNode* root) {
        int ans = 0;
        isUnival(root, INT_MAX, ans);
        return ans;
    }

private:
    bool isUnival(TreeNode* root, int val, int& ans) {
        if (root == nullptr)
            return true;

        if (isUnival(root->left, root->val, ans) &
            isUnival(root->right, root->val, ans)) {
            ++ans;
            return root->val == val;
        }

        return false;
    }
};
```

1622. 2500.cpp

Path: LeetCode-main/solutions/2500. Delete Greatest Value in Each Row/2500.cpp

```
class Solution {
public:
    int deleteGreatestValue(vector<vector<int>>& grid) {
        int ans = 0;

        for (vector<int>& row : grid)
            ranges::sort(row);

        for (int j = 0; j < grid[0].size(); ++j) {
            int maxOfColumn = 0;
            for (int i = 0; i < grid.size(); ++i)
                maxOfColumn = max(maxOfColumn, grid[i][j]);
            ans += maxOfColumn;
        }

        return ans;
    }
};
```


1623. 2501.cpp

Path: LeetCode-main/solutions/2501. Longest Square Streak in an Array/2501.cpp

```
class Solution {
public:
    int longestSquareStreak(vector<int>& nums) {
        nums.erase(std::unique(nums.begin(), nums.end()), nums.end());
        ranges::sort(nums, greater<>());

        const int maxNum = ranges::max(nums);
        // dp[i] := the longest square streak starts with i
        vector<int> dp(maxNum + 1);

        for (const int num : nums) {
            dp[num] = 1;
            const long squaredNum = static_cast<long>(num) * num;
            if (squaredNum <= maxNum)
                dp[num] += dp[squaredNum];
        }

        const int ans = ranges::max(dp);
        return ans < 2 ? -1 : ans;
    }
};
```

1624. 2502.cpp

Path: LeetCode-main/solutions/2502. Design Memory Allocator/2502.cpp

```
class Allocator {
public:
    Allocator(int n) : memory(n), mIDToIndices(1001) {}

    int allocate(int size, int mID) {
        int consecutiveFree = 0;
        for (int i = 0; i < memory.size(); ++i) {
            consecutiveFree = memory[i] == 0 ? consecutiveFree + 1 : 0;
            if (consecutiveFree == size) {
                for (int j = i - consecutiveFree + 1; j <= i; ++j) {
                    memory[j] = mID;
                    mIDToIndices[mID].push_back(j);
                }
                return i - consecutiveFree + 1;
            }
        }
        return -1;
    }

    int free(int mID) {
        vector<int>& indices = mIDToIndices[mID];
        const int freedUnits = indices.size();
        for (const int index : indices)
            memory[index] = 0;
        indices.clear();
        return freedUnits;
    }

private:
    vector<int> memory;
    vector<vector<int>> mIDToIndices;
};
```

1625. 2503.cpp

Path: LeetCode-main/solutions/2503. Maximum Number of Points From Grid Queries/2503.cpp

```
struct IndexedQuery {
    int queryIndex;
    int query;
};

struct T {
    int i;
    int j;
    int val; // grid[i][j]
};

class Solution {
public:
    vector<int> maxPoints(vector<vector<int>>& grid, vector<int>& queries) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = grid.size();
        const int n = grid[0].size();
        vector<int> ans(queries.size());
        auto compare = [](const T& a, const T& b) { return a.val > b.val; };
        priority_queue<T, vector<T>, decltype(compare)> minHeap(compare);
        vector<vector<bool>> seen(m, vector<bool>(n));

        minHeap.emplace(0, 0, grid[0][0]);
        seen[0][0] = true;
        int accumulate = 0;

        for (const auto& [queryIndex, query] : getIndexedQueries(queries)) {
            while (!minHeap.empty()) {
                const auto [i, j, val] = minHeap.top();
                minHeap.pop();
                if (val >= query) {
                    // The smallest neighbor is still larger than `query`, so no need to
                    // keep exploring. Re-push (i, j, grid[i][j]) back to the `minHeap`.
                    minHeap.emplace(i, j, val);
                    break;
                }
                ++accumulate;
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (seen[x][y])
                        continue;
                    minHeap.emplace(x, y, grid[x][y]);
                    seen[x][y] = true;
                }
            }
            ans[queryIndex] = accumulate;
        }

        return ans;
    }

private:
    vector<IndexedQuery> getIndexedQueries(const vector<int>& queries) {
        vector<IndexedQuery> indexedQueries;
        for (int i = 0; i < queries.size(); ++i)
            indexedQueries.push_back({i, queries[i]});
        ranges::sort(
            indexedQueries, ranges::less{},
            [](const IndexedQuery& indexedQuery) { return indexedQuery.query; });
        return indexedQueries;
    }
};
```

1626. 2505.cpp

Path: LeetCode-main/solutions/2505. Bitwise OR of All Subsequence Sums/2505.cpp

```
class Solution {
public:
    long long subsequenceSumOr(vector<int>& nums) {
        long ans = 0;
        long prefix = 0;

        for (const int num : nums) {
            prefix += num;
            ans |= num | prefix;
        }

        return ans;
    }
};
```

1627. 2506.cpp

Path: LeetCode-main/solutions/2506. Count Pairs Of Similar Strings/2506.cpp

```
class Solution {
public:
    int similarPairs(vector<string>& words) {
        int ans = 0;
        vector<int> masks;

        for (const string& word : words)
            masks.push_back(getMask(word));

        for (int i = 0; i < masks.size(); ++i)
            for (int j = i + 1; j < masks.size(); ++j)
                if (masks[i] == masks[j])
                    ++ans;

        return ans;
    }

private:
    int getMask(const string& word) {
        int mask = 0;
        for (const char c : word)
            mask |= 1 << c - 'a';
        return mask;
    }
};
```

1628. 2507.cpp

Path: LeetCode-main/solutions/2507. Smallest Value After Replacing With Sum of Prime Factors/2507.cpp

```
class Solution {
public:
    int smallestValue(int n) {
        int primeSum = getPrimeSum(n);
        while (n != primeSum) {
            n = primeSum;
            primeSum = getPrimeSum(n);
        }
        return n;
    }

private:
    int getPrimeSum(int n) {
        int primeSum = 0;
        for (int i = 2; i <= n; ++i)
            while (n % i == 0) {
                n /= i;
                primeSum += i;
            }
        return primeSum;
    }
};
```

1629. 2508.cpp

Path: LeetCode-main/solutions/2508. Add Edges to Make Degrees of All Nodes Even/2508.cpp

```
class Solution {
public:
    bool isPossible(int n, vector<vector<int>>& edges) {
        vector<unordered_set<int>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0] - 1;
            const int v = edge[1] - 1;
            graph[u].insert(v);
            graph[v].insert(u);
        }

        const vector<int> oddNodes = getOddNodes(graph);
        if (oddNodes.empty())
            return true;
        if (oddNodes.size() == 2) {
            const int a = oddNodes[0];
            const int b = oddNodes[1];
            for (int i = 0; i < n; ++i)
                // Can connect i with a and i with b.
                if (!graph[i].contains(a) && !graph[i].contains(b))
                    return true;
        }
        if (oddNodes.size() == 4) {
            const int a = oddNodes[0];
            const int b = oddNodes[1];
            const int c = oddNodes[2];
            const int d = oddNodes[3];
            return (!graph[a].contains(b) && !graph[c].contains(d)) ||
                (!graph[a].contains(c) && !graph[b].contains(d)) ||
                (!graph[a].contains(d) && !graph[b].contains(c));
        }
        return false;
    }

private:
    vector<int> getOddNodes(const vector<unordered_set<int>>& graph) {
        vector<int> oddNodes;
        for (int i = 0; i < graph.size(); ++i)
            if (graph[i].size() % 2 == 1)
                oddNodes.push_back(i);
        return oddNodes;
    }
};
```

1630. 2509.cpp

Path: LeetCode-main/solutions/2509. Cycle Length Queries in a Tree/2509.cpp

```
class Solution {
public:
    vector<int> cycleLengthQueries(int n, vector<vector<int>>& queries) {
        vector<int> ans;

        for (const vector<int>& query : queries) {
            ans.push_back(1);
            int a = query[0];
            int b = query[1];
            while (a != b) {
                if (a > b)
                    a /= 2;
                else
                    b /= 2;
                ++ans.back();
            }
        }
        return ans;
    }
};
```


1631. 251-2.cpp

Path: LeetCode-main/solutions/251. Flatten 2D Vector/251-2.cpp

```
class Vector2D {
public:
    Vector2D(vector<vector<int>>& v) {
        i = v.begin();
        iEnd = v.end();
    }

    int next() {
        moveIterator();
        return (*i)[j++];
    }

    bool hasNext() {
        moveIterator();
        return i != iEnd;
    }

private:
    // (*i)[j] := the current pointed value
    vector<vector<int>>::iterator i, iEnd;
    int j = 0;

    void moveIterator() {
        while (i != iEnd && j == (*i).size())
            ++i, j = 0;
    }
};
```

1632. 251.cpp

Path: LeetCode-main/solutions/251. Flatten 2D Vector/251.cpp

```
class Vector2D {
public:
    Vector2D(vector<vector<int>>& vec) {
        for (const vector<int>& arr : vec)
            for (const int a : arr)
                this->vec.push_back(a);
    }

    int next() {
        return vec[i++];
    }

    bool hasNext() {
        return i < vec.size();
    }

private:
    vector<int> vec;
    int i = 0;
};
```

1633. 2510.cpp

Path: LeetCode-main/solutions/2510. Check if There is a Path With Equal Number of 0's And 1's/2510.cpp

```
class Solution {
public:
    bool isThereAPath(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        // Map negative (the number of 0s - the number of 1s) to non-negative one.
        const int cells = m + n - 1;
        if (cells % 2 == 1)
            return false;
        vector<vector<vector<int>>> mem(
            m, vector<vector<int>>(n, vector<int>(cells * 2 + 1, -1)));
        return isThereAPath(grid, 0, 0, 0, cells, mem);
    }

private:
    // Returns 1 if there's a path to grid[i][j]
    // s.t. `sum` = (the number of 0s - the number of 1s).
    bool isThereAPath(const vector<vector<int>>& grid, int i, int j, int sum,
        const int& cells, vector<vector<vector<int>>>& mem) {
        if (i == grid.size() || j == grid[0].size())
            return false;
        sum += grid[i][j] == 0 ? 1 : -1;
        if (i == grid.size() - 1 && j == grid[0].size() - 1)
            return sum == 0;
        const int k = cells + sum;
        if (mem[i][j][k] != -1)
            return mem[i][j][k];
        return mem[i][j][k] = isThereAPath(grid, i + 1, j, sum, cells, mem) ||
            isThereAPath(grid, i, j + 1, sum, cells, mem);
    }
};
```

1634. 2511.cpp

Path: LeetCode-main/solutions/2511. Maximum Enemy Forts That Can Be Captured/2511.cpp

```
class Solution {
public:
    int captureForts(vector<int>& forts) {
        int ans = 0;

        for (int i = 0, j = 0; i < forts.size(); ++i)
            if (forts[i] != 0) { // -1 or 1
                if (forts[i] == -forts[j])
                    ans = max(ans, i - j - 1);
                j = i;
            }

        return ans;
    }
};
```

1635. 2512.cpp

Path: LeetCode-main/solutions/2512. Reward Top K Students/2512.cpp

```
class Solution {
public:
    vector<int> topStudents(vector<string>& positive_feedback,
                           vector<string>& negative_feedback,
                           vector<string>& report, vector<int>& student_id,
                           int k) {
        vector<int> ans;
        vector<pair<int, int>> scoreAndIds;
        unordered_set<string> pos{positive_feedback.begin(),
                                   positive_feedback.end()};
        unordered_set<string> neg{negative_feedback.begin(),
                                   negative_feedback.end()};

        for (int i = 0; i < report.size(); ++i) {
            int score = 0;
            istringstream iss(report[i]);
            for (string word; iss >> word;) {
                if (pos.contains(word))
                    score += 3;
                if (neg.contains(word))
                    score -= 1;
            }
            scoreAndIds.emplace_back(-score, student_id[i]);
        }

        partial_sort(scoreAndIds.begin(), scoreAndIds.begin() + k,
                     scoreAndIds.end());
        transform(
            scoreAndIds.begin(), scoreAndIds.begin() + k, back_inserter(ans),
            [](const pair<int, int>& scoreAndId) { return scoreAndId.second; });
        return ans;
    }
};
```

1636. 2513.cpp

Path: LeetCode-main/solutions/2513. Minimize the Maximum of Two Arrays/2513.cpp

```
class Solution {
public:
    int minimizeSet(int divisor1, int divisor2, int uniqueCnt1, int uniqueCnt2) {
        const long divisorLcm = std::lcm(static_cast<long>(divisor1), divisor2);
        long l = 0;
        long r = INT_MAX;

        while (l < r) {
            const long m = (l + r) / 2;
            if (isPossible(m, divisorLcm, divisor1, divisor2, uniqueCnt1, uniqueCnt2))
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    // Returns true if we can take uniqueCnt1 integers from [1..m] to arr1 and
    // take uniqueCnt2 integers from [1..m] to arr2.
    bool isPossible(long m, long divisorLcm, int divisor1, int divisor2,
                    int uniqueCnt1, int uniqueCnt2) {
        const long cnt1 = m - m / divisor1;
        const long cnt2 = m - m / divisor2;
        const long totalCnt = m - m / divisorLcm;
        return cnt1 >= uniqueCnt1 && cnt2 >= uniqueCnt2 &&
            totalCnt >= uniqueCnt1 + uniqueCnt2;
    }
};
```

1637. 2514.cpp

Path: LeetCode-main/solutions/2514. Count Anagrams/2514.cpp

```
class Solution {
public:
    int countAnagrams(string s) {
        const int n = s.length();
        const auto [fact, invFact] = getFactAndInvFact(n);
        int ans = 1;
        istringstream iss(s);

        for (string word; iss >> word;) {
            ans = ans * fact[word.length()] % kMod;
            vector<int> count(26);
            for (const char c : word)
                ++count[c - 'a'];
            for (const int freq : count)
                ans = ans * invFact[freq] % kMod;
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    pair<vector<long>, vector<long>> getFactAndInvFact(int n) {
        vector<long> fact(n + 1);
        vector<long> invFact(n + 1);
        vector<long> inv(n + 1);
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = kMod - kMod / i * inv[kMod % i] % kMod;
            fact[i] = fact[i - 1] * i % kMod;
            invFact[i] = invFact[i - 1] * inv[i] % kMod;
        }
        return {fact, invFact};
    }
};
```

1638. 2515.cpp

Path: LeetCode-main/solutions/2515. Shortest Distance to Target String in a Circular Array/2515.cpp

```
class Solution {
public:
    int closetTarget(vector<string>& words, string target, int startIndex) {
        const int n = words.size();

        for (int i = 0; i < n; ++i) {
            if (words[(startIndex + i + n) % n] == target)
                return i;
            if (words[(startIndex - i + n) % n] == target)
                return i;
        }

        return -1;
    }
};
```


1639. 2516.cpp

Path: LeetCode-main/solutions/2516. Take K of Each Character From Left and Right/2516.cpp

```
class Solution {
public:
    int takeCharacters(string s, int k) {
        const int n = s.length();
        int ans = n;
        vector<int> count(3);

        for (const char c : s)
            ++count[c - 'a'];

        if (count[0] < k || count[1] < k || count[2] < k)
            return -1;

        for (int l = 0, r = 0; r < n; ++r) {
            --count[s[r] - 'a'];
            while (count[s[r] - 'a'] < k)
                ++count[s[l++]] - 'a'];
            ans = min(ans, n - (r - l + 1));
        }

        return ans;
    }
};
```

1640. 2517.cpp

Path: LeetCode-main/solutions/2517. Maximum Tastiness of Candy Basket/2517.cpp

```
class Solution {
public:
    int maximumTastiness(vector<int>& price, int k) {
        ranges::sort(price);

        int l = 0;
        int r = ranges::max(price) - ranges::min(price) + 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (numBaskets(price, m) < k)
                r = m;
            else
                l = m + 1;
        }

        return l - 1;
    }

private:
    // Returns the number of baskets we can pick for m tastiness.
    int numBaskets(const vector<int>& price, int m) {
        int baskets = 0;
        int prevPrice = -m;
        for (const int p : price)
            if (p >= prevPrice + m) {
                prevPrice = p;
                ++baskets;
            }
        return baskets;
    }
};
```

1641. 2518.cpp

Path: LeetCode-main/solutions/2518. Number of Great Partitions/2518.cpp

```
class Solution {
public:
    int countPartitions(vector<int>& nums, int k) {
        const long sum = accumulate(nums.begin(), nums.end(), 0L);
        long ans = modPow(2, nums.size());
        vector<long> dp(k + 1);
        dp[0] = 1;

        for (const int num : nums)
            for (int i = k; i >= num; --i) {
                dp[i] += dp[i - num];
                dp[i] %= kMod;
            }

        // Subtract the cases that're not satisfied.
        for (int i = 0; i < k; ++i)
            if (sum - i < k) // Both group1 and group2 < k.
                ans -= dp[i];
            else
                ans -= dp[i] * 2;

        return (ans % kMod + kMod) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

1642. 2519.cpp

Path: LeetCode-main/solutions/2519. Count the Number of K-Big Indices/2519.cpp

```
class FenwickTree {
public:
    FenwickTree(int n) : sums(n + 1) {}

    void add(int i, int delta) {
        while (i < sums.size()) {
            sums[i] += delta;
            i += lowbit(i);
        }
    }

    int get(int i) const {
        int sum = 0;
        while (i > 0) {
            sum += sums[i];
            i -= lowbit(i);
        }
        return sum;
    }

private:
    vector<int> sums;

    static inline int lowbit(int i) {
        return i & -i;
    }
};

class Solution {
public:
    int kBigIndices(vector<int>& nums, int k) {
        const int n = nums.size();
        int ans = 0;
        FenwickTree leftTree(n);
        FenwickTree rightTree(n);
        // left[i] := the number of `nums` < nums[i] with index < i
        vector<int> left(n);
        // right[i] := the number of `nums` < nums[i] with index > i
        vector<int> right(n);

        for (int i = 0; i < n; ++i) {
            left[i] = leftTree.get(nums[i] - 1);
            leftTree.add(nums[i], 1);
        }

        for (int i = n - 1; i >= 0; --i) {
            right[i] = rightTree.get(nums[i] - 1);
            rightTree.add(nums[i], 1);
        }

        for (int i = 0; i < n; ++i)
            if (left[i] >= k && right[i] >= k)
                ++ans;

        return ans;
    }
};
```

1643. 252.cpp

Path: LeetCode-main/solutions/252. Meeting Rooms/252.cpp

```
class Solution {
public:
    bool canAttendMeetings(vector<vector<int>>& intervals) {
        ranges::sort(intervals);

        for (int i = 1; i < intervals.size(); ++i)
            if (intervals[i - 1][1] > intervals[i][0])
                return false;

        return true;
    }
};
```

1644. 2520.cpp

Path: LeetCode-main/solutions/2520. Count the Digits That Divide a Number/2520.cpp

```
class Solution {
public:
    int countDigits(int num) {
        int ans = 0;

        for (int n = num; n > 0; n /= 10)
            if (num % (n % 10) == 0)
                ++ans;

        return ans;
    }
};
```


1646. 2522.cpp

Path: LeetCode-main/solutions/2522. Partition String Into Substrings With Values at Most K/2522.cpp

```
class Solution {
public:
    int minimumPartition(string s, int k) {
        int ans = 1;
        long curr = 0;

        for (const char c : s) {
            curr = curr * 10 + c - '0';
            if (curr > k) {
                curr = c - '0';
                ++ans;
            }
            if (curr > k)
                return -1;
        }
        return ans;
    }
};
```


1647. 2523.cpp

Path: LeetCode-main/solutions/2523. Closest Prime Numbers in Range/2523.cpp

```
class Solution {
public:
    vector<int> closestPrimes(int left, int right) {
        const vector<bool> isPrime = sieveEratosthenes(right + 1);
        vector<int> primes;

        for (int i = left; i <= right; ++i)
            if (isPrime[i])
                primes.push_back(i);

        if (primes.size() < 2)
            return {-1, -1};

        int minDiff = INT_MAX;
        int num1 = -1;
        int num2 = -1;

        for (int i = 1; i < primes.size(); ++i) {
            const int diff = primes[i] - primes[i - 1];
            if (diff < minDiff) {
                minDiff = diff;
                num1 = primes[i - 1];
                num2 = primes[i];
            }
        }

        return {num1, num2};
    }

private:
    vector<bool> sieveEratosthenes(int n) {
        vector<bool> isPrime(n, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
};
```

1648. 2524.cpp

Path: LeetCode-main/solutions/2524. Maximum Frequency Score of a Subarray/2524.cpp

```
class Solution {
public:
    int maxFrequencyScore(vector<int>& nums, int k) {
        unordered_map<int, int> count;

        for (int i = 0; i < k; ++i)
            ++count[nums[i]];

        int sum = getInitialSum(count);
        int ans = sum;

        for (int i = k; i < nums.size(); ++i) {
            // Remove the leftmost number that's out-of-window.
            const int leftNum = nums[i - k];
            sum = (sum - modPow(leftNum, count[leftNum]) + kMod) % kMod;
            // After decreasing its frequency, if it's still > 0, then add it back.
            if (--count[leftNum] > 0)
                sum = (sum + modPow(leftNum, count[leftNum])) % kMod;
            // Otherwise, remove it from the count map.
            else
                count.erase(leftNum);
            // Add the current number. Similarly, remove the current score like above.
            const int rightNum = nums[i];
            if (count[rightNum] > 0)
                sum = (sum - modPow(rightNum, count[rightNum]) + kMod) % kMod;
            sum = (sum + modPow(rightNum, ++count[rightNum])) % kMod;
            ans = max(ans, sum);
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    int getInitialSum(const unordered_map<int, int>& count) {
        long sum = 0;
        for (const auto& [num, freq] : count)
            sum = (sum + modPow(num, freq)) % kMod;
        return sum;
    }

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

1649. 2525.cpp

Path: LeetCode-main/solutions/2525. Categorize Box According to Criteria/2525.cpp

```
class Solution {
public:
    string categorizeBox(int length, int width, int height, int mass) {
        const bool isBulky =
            length >= 10000 || width >= 10000 || height >= 10000 ||
            static_cast<long>(length) * width * height >= 1'000'000'000;
        const bool isHeavy = mass >= 100;
        if (isBulky && isHeavy)
            return "Both";
        if (isBulky)
            return "Bulky";
        if (isHeavy)
            return "Heavy";
        return "Neither";
    }
};
```

1650. 2526.cpp

Path: LeetCode-main/solutions/2526. Find Consecutive Integers from a Data Stream/2526.cpp

```
class DataStream {
public:
    DataStream(int value, int k) : value(value), k(k) {}

    bool consec(int num) {
        if (q.size() == k) {
            if (q.front() == value)
                --count;
            q.pop();
        }
        if (num == value)
            ++count;
        q.push(num);
        return count == k;
    }

private:
    const int value;
    const int k;
    queue<int> q;
    int count = 0;
};
```

1651. 2527.cpp

Path: LeetCode-main/solutions/2527. Find Xor-Beauty of Array/2527.cpp

```
class Solution {
public:
    int xorBeauty(vector<int>& nums) {
        return accumulate(nums.begin(), nums.end(), 0, bit_xor<>());
    }
};
```

1652. 2528.cpp

Path: LeetCode-main/solutions/2528. Maximize the Minimum Powered City/2528.cpp

```
class Solution {
public:
    long long maxPower(vector<int>& stations, int r, int k) {
        long left = ranges::min(stations);
        long right = accumulate(stations.begin(), stations.end(), 0L) + k + 1;

        while (left < right) {
            const long mid = (left + right) / 2;
            if (check(stations, r, k, mid))
                left = mid + 1;
            else
                right = mid;
        }

        return left - 1;
    }

private:
    // Returns true if each city can have at least `minPower`.
    bool check(vector<int> stations, int r, int additionalStations,
               long minPower) {
        const int n = stations.size();
        // Initilaize `power` as the 0-th city's power - stations[r].
        long power = accumulate(stations.begin(), stations.begin() + r, 0L);

        for (int i = 0; i < n; ++i) {
            if (i + r < n)
                power += stations[i + r]; // `power` = sum(stations[i - r..i + r]).
            if (power < minPower) {
                const long requiredPower = minPower - power;
                // There're not enough stations to plant.
                if (requiredPower > additionalStations)
                    return false;
                // Greedily plant `requiredPower` power stations in the farthest place
                // to cover as many cities as possible.
                stations[min(n - 1, i + r)] += requiredPower;
                additionalStations -= requiredPower;
                power += requiredPower;
            }
            if (i - r >= 0)
                power -= stations[i - r];
        }

        return true;
    }
};
```

1653. 2529.cpp

Path: LeetCode-main/solutions/2529. Maximum Count of Positive Integer and Negative Integer/2529.cpp

```
class Solution {
public:
    int maximumCount(vector<int>& nums) {
        return max(ranges::count_if(nums, [](int num) { return num > 0; })),
               ranges::count_if(nums, [](int num) { return num < 0; }));
    }
};
```

1654. 253-2.cpp

Path: LeetCode-main/solutions/253. Meeting Rooms II/253-2.cpp

```
class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        const int n = intervals.size();
        int ans = 0;
        vector<int> starts;
        vector<int> ends;

        for (const vector<int>& interval : intervals) {
            starts.push_back(interval[0]);
            ends.push_back(interval[1]);
        }

        ranges::sort(starts);
        ranges::sort(ends);

        for (int i = 0, j = 0; i < n; ++i)
            if (starts[i] < ends[j])
                ++ans;
            else
                ++j;

        return ans;
    }
};
```


1655. 253.cpp

Path: LeetCode-main/solutions/253. Meeting Rooms II/253.cpp

```
class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        // Store the end times of each room.
        priority_queue<int, vector<int>, greater<>> minHeap;

        ranges::sort(intervals);

        for (const vector<int>& interval : intervals) {
            // There's no overlap, so we can reuse the same room.
            if (!minHeap.empty() && interval[0] >= minHeap.top())
                minHeap.pop();
            minHeap.push(interval[1]);
        }

        return minHeap.size();
    }
};
```

1656. 2530.cpp

Path: LeetCode-main/solutions/2530. Maximal Score After Applying K Operations/2530.cpp

```
class Solution {
public:
    long long maxKelements(vector<int>& nums, int k) {
        long ans = 0;
        priority_queue<int> maxHeap;

        for (const int num : nums)
            maxHeap.push(num);

        for (int i = 0; i < k; ++i) {
            const int num = maxHeap.top();
            maxHeap.pop();
            ans += num;
            maxHeap.push((num + 2) / 3);
        }

        return ans;
    }
};
```

1657. 2531.cpp

Path: LeetCode-main/solutions/2531. Make Number of Distinct Characters Equal/2531.cpp

```
class Solution {
public:
    bool isItPossible(string word1, string word2) {
        const vector<int> count1 = getCount(word1);
        const vector<int> count2 = getCount(word2);
        const int distinct1 = getDistinct(count1);
        const int distinct2 = getDistinct(count2);

        for (int i = 0; i < 26; ++i)
            for (int j = 0; j < 26; ++j) {
                if (count1[i] == 0 || count2[j] == 0)
                    continue;
                if (i == j) {
                    // Swapping the same letters won't change the number of distinct
                    // letters in each string, so just check if `distinct1 == distinct2`.
                    if (distinct1 == distinct2)
                        return true;
                    continue;
                }
                // The calculation is meaningful only when i != j.
                // Swap ('a' + i) in word1 with ('a' + j) in word2.
                const int distinctAfterSwap1 =
                    distinct1 - (count1[i] == 1) + (count1[j] == 0);
                const int distinctAfterSwap2 =
                    distinct2 - (count2[j] == 1) + (count2[i] == 0);
                if (distinctAfterSwap1 == distinctAfterSwap2)
                    return true;
            }

        return false;
    }

private:
    vector<int> getCount(const string& s) {
        vector<int> count(26);
        for (const char c : s)
            ++count[c - 'a'];
        return count;
    }

    int getDistinct(const vector<int>& count) {
        return ranges::count_if(count, [](const int c) { return c > 0; });
    }
};
```

1658. 2532.cpp

Path: LeetCode-main/solutions/2532. Time to Cross a Bridge/2532.cpp

```
class Solution {
public:
    int findCrossingTime(int n, int k, vector<vector<int>>& time) {
        int ans = 0;
        using P = pair<int, int>;
        // (leftToRight + rightToLeft, i)
        priority_queue<P> leftBridgeQueue;
        priority_queue<P> rightBridgeQueue;
        // (time to be idle, i)
        priority_queue<P, vector<P>, greater<>> leftWorkers;
        priority_queue<P, vector<P>, greater<>> rightWorkers;

        for (int i = 0; i < k; ++i)
            leftBridgeQueue.emplace(
                /*leftToRight*/ time[i][0] + /*rightToLeft*/ time[i][2], i);

        while (n > 0 || !rightBridgeQueue.empty() || !rightWorkers.empty()) {
            // Idle left workers get on the left bridge.
            while (!leftWorkers.empty() && leftWorkers.top().first <= ans) {
                const int i = leftWorkers.top().second;
                leftWorkers.pop();
                leftBridgeQueue.emplace(
                    /*leftToRight*/ time[i][0] + /*rightToLeft*/ time[i][2], i);
            }
            // Idle right workers get on the right bridge.
            while (!rightWorkers.empty() && rightWorkers.top().first <= ans) {
                const int i = rightWorkers.top().second;
                rightWorkers.pop();
                rightBridgeQueue.emplace(
                    /*leftToRight*/ time[i][0] + /*rightToLeft*/ time[i][2], i);
            }

            if (!rightBridgeQueue.empty()) {
                // If the bridge is free, the worker waiting on the right side of the
                // bridge gets to cross the bridge. If more than one worker is waiting
                // on the right side, the one with the lowest efficiency crosses first.
                const int i = rightBridgeQueue.top().second;
                rightBridgeQueue.pop();
                ans += /*rightToLeft*/ time[i][2];
                leftWorkers.emplace(ans + /*putNew*/ time[i][3], i);
            } else if (!leftBridgeQueue.empty() && n > 0) {
                // If the bridge is free and no worker is waiting on the right side, and
                // at least one box remains at the old warehouse, the worker on the left
                // side of the river gets to cross the bridge. If more than one worker
                // is waiting on the left side, the one with the lowest efficiency
                // crosses first.
                const int i = leftBridgeQueue.top().second;
                leftBridgeQueue.pop();
                ans += /*leftToRight*/ time[i][0];
                rightWorkers.emplace(ans + /*pickOld*/ time[i][1], i);
                --n;
            } else {
                // Advance the time of the last crossing worker.
                ans = min(
                    !leftWorkers.empty() && n > 0 ? leftWorkers.top().first : INT_MAX,
                    !rightWorkers.empty() ? rightWorkers.top().first : INT_MAX);
            }
        }

        return ans;
    }
};
```

1659. 2533.cpp

Path: LeetCode-main/solutions/2533. Number of Good Binary Strings/2533.cpp

```
class Solution {
public:
    int goodBinaryStrings(int minLength, int maxLength, int oneGroup,
                           int zeroGroup) {
        constexpr int kMod = 1'000'000'007;
        // dp[i] := the number of good binary strings with length i
        vector<int> dp(maxLength + 1);
        dp[0] = 1; // ""

        for (int i = 0; i <= maxLength; ++i)
            // There are good binary strings with length i, so we can append
            // consecutive 0s or 1s after it.
            if (dp[i] > 0) {
                const int appendZeros = i + zeroGroup;
                if (appendZeros <= maxLength) {
                    dp[appendZeros] += dp[i];
                    dp[appendZeros] %= kMod;
                }
                const int appendOnes = i + oneGroup;
                if (appendOnes <= maxLength) {
                    dp[appendOnes] += dp[i];
                    dp[appendOnes] %= kMod;
                }
            }

        return accumulate(dp.begin() + minLength, dp.end(), 0L) % kMod;
    }
};
```

1660. 2534.cpp

Path: LeetCode-main/solutions/2534. Time Taken to Cross the Door/2534.cpp

```
class Solution {
public:
    vector<int> timeTaken(vector<int>& arrival, vector<int>& state) {
        const int n = arrival.size();
        vector<int> ans(n);
        vector<queue<int>> qs(2); // qs[0] := enter queue, qs[1] := exit queue
        int time = 0;
        int d = 1; // 0 := enter, 1 := exit

        for (int i = 0; i < n; ++i) {
            popQueues(time, d, arrival[i], qs, ans);
            // If the door was not used in the previous second, then the person who
            // wants to exit goes first.
            if (arrival[i] > time) {
                time = arrival[i]; // Forward `time` to now.
                d = 1;
            }
            qs[state[i]].push(i);
        }

        popQueues(time, d, 200'000, qs, ans);
        return ans;
    }

private:
    void popQueues(int& time, int& d, int arrivalTime, vector<queue<int>>& qs,
                   vector<int>& ans) {
        while (arrivalTime > time && (!qs[0].empty() || !qs[1].empty())) {
            if (qs[d].empty())
                d ^= 1;
            ans[qs[d].front()] = time++, qs[d].pop();
        }
    }
};
```

1661. 2535.cpp

Path: LeetCode-main/solutions/2535. Difference Between Element Sum and Digit Sum of an Array/2535.cpp

```
class Solution {
public:
    int differenceOfSum(vector<int>& nums) {
        const int elementSum = accumulate(nums.begin(), nums.end(), 0);
        const int allDigitSum = getAllDigitSum(nums);
        return abs(elementSum - allDigitSum);
    }

private:
    int getAllDigitSum(const vector<int>& nums) {
        int allDigitSum = 0;
        for (const int num : nums)
            allDigitSum += getDigitSum(num);
        return allDigitSum;
    }

    int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
};
```

1662. 2536.cpp

Path: LeetCode-main/solutions/2536. Increment Submatrices by One/2536.cpp

```
class Solution {
public:
    vector<vector<int>> rangeAddQueries(int n, vector<vector<int>>& queries) {
        vector<vector<int>> ans(n, vector<int>(n));
        vector<vector<int>> prefix(n, vector<int>(n + 1));

        for (const vector<int>& query : queries) {
            const int row1 = query[0];
            const int col1 = query[1];
            const int row2 = query[2];
            const int col2 = query[3];
            for (int i = row1; i <= row2; ++i) {
                ++prefix[i][col1];
                --prefix[i][col2 + 1];
            }
        }

        for (int i = 0; i < n; ++i) {
            int sum = 0;
            for (int j = 0; j < n; ++j) {
                sum += prefix[i][j];
                ans[i][j] = sum;
            }
        }

        return ans;
    }
};
```


1663. 2537.cpp

Path: LeetCode-main/solutions/2537. Count the Number of Good Subarrays/2537.cpp

```
class Solution {
public:
    long long countGood(vector<int>& nums, int k) {
        long ans = 0;
        int pairs = 0;
        unordered_map<int, int> count;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            // Since there're count[r] nums[r]s, including nums[r] to the window will
            // increase the number of good subarrays by count[r].
            pairs += count[nums[r]]++;
            while (pairs >= k)
                pairs -= --count[nums[l++]];
            // nums[0..r], nums[1..r], ..., nums[l - 1..r] are good subarrays, so add
            // l to `ans`.
            ans += l;
        }

        return ans;
    }
};
```

1664. 2538.cpp

Path: LeetCode-main/solutions/2538. Difference Between Maximum and Minimum Price Sum/2538.cpp

```
class Solution {
public:
    long long maxOutput(int n, vector<vector<int>>& edges, vector<int>& price) {
        int ans = 0;
        vector<vector<int>> tree(n);
        // maxSums[i] := the maximum the sum of path rooted at i
        vector<int> maxSums(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }

        // Precalculate `maxSums`.
        maxSum(tree, 0, /*prev=*/-1, maxSums, price);
        reroot(tree, 0, /*prev=*/-1, /*parentSum=*/0, maxSums, price, ans);
        return ans;
    }

private:
    int maxSum(const vector<vector<int>>& tree, int u, int prev,
               vector<int>& maxSums, const vector<int>& price) {
        int maxChildSum = 0;
        for (const int v : tree[u])
            if (v != prev)
                maxChildSum = max(maxChildSum, maxSum(tree, v, u, maxSums, price));
        return maxSums[u] = price[u] + maxChildSum;
    }

    void reroot(const vector<vector<int>>& tree, int u, int prev, int parentSum,
               const vector<int>& maxSums, const vector<int>& price, int& ans) {
        // Get the top two subtree sums and the top one node index.
        int maxSubtreeSum1 = 0;
        int maxSubtreeSum2 = 0;
        int maxNode = -1;
        for (const int v : tree[u]) {
            if (v == prev)
                continue;
            if (maxSums[v] > maxSubtreeSum1) {
                maxSubtreeSum2 = maxSubtreeSum1;
                maxSubtreeSum1 = maxSums[v];
                maxNode = v;
            } else if (maxSums[v] > maxSubtreeSum2) {
                maxSubtreeSum2 = maxSums[v];
            }
        }

        if (tree[u].size() == 1)
            ans = max({ans, parentSum, maxSubtreeSum1});

        for (const int v : tree[u]) {
            if (v == prev)
                continue;
            const int nextParentSum =
                (v == maxNode ? price[u] + max(parentSum, maxSubtreeSum2)
                 : price[u] + max(parentSum, maxSubtreeSum1));
            reroot(tree, v, u, nextParentSum, maxSums, price, ans);
        }
    }
};
```

1665. 2539.cpp

Path: LeetCode-main/solutions/2539. Count the Number of Good Subsequences/2539.cpp

```
class Solution {
public:
    int countGoodSubsequences(string s) {
        // For each frequency f in [1, max(freq)], start with "" and calculate how
        // many subsequences can be constructed with each letter's frequency = f.
        //
        // e.g. s = "abb", so f = max(freq) = 2.
        //
        // For f = 1, with 1 way to build "", choose any 'a' to construct a good
        // subseq, so # of good subsequences = 1 + 1 * (1, 1) = 2 ("", "a"). Next,
        // add 'b' and # of good subsequences = 2 + 2 * (2, 1) = 6 ("", "a", "b1",
        // "b2", "ab1", "ab2"). So, the number of good subsequences for f = 1 is 5
        // since we need to exclude "".
        //
        // For f = 2, with 1 way to build "", choose any two 'b's to construct a
        // good subseq, so # of good subsequences = 1 + 1 * (2, 2) is 2 ("", "bb").
        // So, the number of good subsequences for f = 2 = 1 since we need to
        // exclude "".
        //
        // Therefore, the number of good subsequences for "aab" = 5 + 1 = 6.
        int ans = 0;
        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        const int maxFreq = ranges::max(count);
        const auto [fact, invFact] = getFactAndInvFact(maxFreq);

        for (int freq = 1; freq <= maxFreq; ++freq) {
            long numSubseqs = 1; // ""
            for (const int charFreq : count)
                if (charFreq >= freq)
                    numSubseqs = (numSubseqs + //
                                numSubseqs * nCk(charFreq, freq, fact, invFact)) %
                                kMod;
            ans += numSubseqs - 1; // Minus "".
            ans %= kMod;
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    pair<vector<long>, vector<long>> getFactAndInvFact(int n) {
        vector<long> fact(n + 1);
        vector<long> invFact(n + 1);
        vector<long> inv(n + 1);
        fact[0] = invFact[0] = 1;
        inv[0] = inv[1] = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2)
                inv[i] = kMod - kMod / i * inv[kMod % i] % kMod;
            fact[i] = fact[i - 1] * i % kMod;
            invFact[i] = invFact[i - 1] * inv[i] % kMod;
        }
        return {fact, invFact};
    }

    int nCk(int n, int k, const vector<long>& fact, const vector<long>& invFact) {
        return fact[n] * invFact[k] % kMod * invFact[n - k] % kMod;
    }
};
```

1666. 254.cpp

Path: LeetCode-main/solutions/254. Factor Combinations/254.cpp

```
class Solution {
public:
    vector<vector<int>> getFactors(int n) {
        vector<vector<int>> ans;
        dfs(n, 2, {}, ans); // The minimum factor is 2.
        return ans;
    }

private:
    void dfs(int n, int s, vector<int>&& path, vector<vector<int>>& ans) {
        if (n <= 1) {
            if (path.size() > 1)
                ans.push_back(path);
            return;
        }

        for (int i = s; i <= n; ++i)
            if (n % i == 0) {
                path.push_back(i);
                dfs(n / i, i, std::move(path), ans);
                path.pop_back();
            }
    }
};
```

1667. 2540.cpp

Path: LeetCode-main/solutions/2540. Minimum Common Value/2540.cpp

```
class Solution {
public:
    int getCommon(vector<int>& nums1, vector<int>& nums2) {
        int i = 0; // nums1's index
        int j = 0; // nums2's index

        while (i < nums1.size() && j < nums2.size()) {
            if (nums1[i] == nums2[j])
                return nums1[i];
            if (nums1[i] < nums2[j])
                ++i;
            else
                ++j;
        }
        return -1;
    }
};
```

1668. 2541.cpp

Path: LeetCode-main/solutions/2541. Minimum Operations to Make Array Equal II/2541.cpp

```
class Solution {
public:
    long long minOperations(vector<int>& nums1, vector<int>& nums2, int k) {
        if (k == 0)
            return nums1 == nums2 ? 0 : -1;

        long ans = 0;
        // the number of increments - the number of decrements
        long opsDiff = 0;

        for (int i = 0; i < nums1.size(); ++i) {
            const int diff = nums1[i] - nums2[i];
            if (diff == 0)
                continue;
            if (diff % k != 0)
                return -1;
            const int ops = diff / k;
            opsDiff += ops;
            ans += abs(ops);
        }

        return opsDiff == 0 ? ans / 2 : -1;
    }
};
```

1669. 2542.cpp

Path: LeetCode-main/solutions/2542. Maximum Subsequence Score/2542.cpp

```
class Solution {
public:
    // Same as 1383. Maximum Performance of a Team
    long long maxScore(vector<int>& nums1, vector<int>& nums2, int k) {
        long ans = 0;
        long sum = 0;
        // (nums2[i], nums1[i]) sorted by nums2[i] in descending order
        vector<pair<int, int>> A;
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (int i = 0; i < nums1.size(); ++i)
            A.emplace_back(nums2[i], nums1[i]);

        ranges::sort(A, greater<>());

        for (const auto& [num2, num1] : A) {
            minHeap.push(num1);
            sum += num1;
            if (minHeap.size() > k)
                sum -= minHeap.top(), minHeap.pop();
            if (minHeap.size() == k)
                ans = max(ans, sum * num2);
        }

        return ans;
    }
};
```

1670. 2543.cpp

Path: LeetCode-main/solutions/2543. Check if Point Is Reachable/2543.cpp

```
class Solution {
public:
    bool isReachable(unsigned targetX, unsigned targetY) {
        return popcount(gcd(targetX, targetY)) == 1;
    }
};
```


1671. 2544.cpp

Path: LeetCode-main/solutions/2544. Alternating Digit Sum/2544.cpp

```
class Solution {
public:
    int alternateDigitSum(int n) {
        int ans = 0;
        int sign = 1;

        for (; n > 0; n /= 10) {
            sign *= -1;
            ans += sign * n % 10;
        }

        return sign * ans;
    }
};
```

1672. 2545.cpp

Path: LeetCode-main/solutions/2545. Sort the Students by Their Kth Score/2545.cpp

```
class Solution {
public:
    vector<vector<int>> sortTheStudents(vector<vector<int>>& score, int k) {
        ranges::sort(score, [k](const vector<int>& a, const vector<int>& b) {
            return a[k] > b[k];
        });
        return score;
    }
};
```

1673. 2546.cpp

Path: LeetCode-main/solutions/2546. Apply Bitwise Operations to Make Strings Equal/2546.cpp

```
class Solution {
public:
    bool makeStringsEqual(string s, string target) {
        return (s.find('1') != string::npos) == (target.find('1') != string::npos);
    }
};
```

1674. 2547.cpp

Path: LeetCode-main/solutions/2547. Minimum Cost to Split an Array/2547.cpp

```
class Solution {
public:
    int minCost(vector<int>& nums, int k) {
        constexpr int kMax = 1001;
        const int n = nums.size();
        // trimmedLength[i][j] := trimmed(nums[i..j]).length
        vector<vector<int>> trimmedLength(n, vector<int>(n));
        // dp[i] := the minimum cost to split nums[i..n)
        vector<int> dp(n + 1, INT_MAX / 2);

        for (int i = 0; i < n; ++i) {
            int length = 0;
            vector<int> count(kMax);
            for (int j = i; j < n; ++j) {
                if (++count[nums[j]] == 2)
                    length += 2;
                else if (count[nums[j]] > 2)
                    ++length;
                trimmedLength[i][j] = length;
            }
        }

        dp[n] = 0;

        for (int i = n - 1; i >= 0; --i)
            for (int j = i; j < n; ++j)
                dp[i] = min(dp[i], k + trimmedLength[i][j] + dp[j + 1]);

        return dp[0];
    }
};
```

1675. 2548.cpp

Path: LeetCode-main/solutions/2548. Maximum Price to Fill a Bag/2548.cpp

```
class Solution {
public:
    double maxPrice(vector<vector<int>>& items, int capacity) {
        double ans = 0;

        // Sort items based on price/weight.
        ranges::sort(items, ranges::greater{}, [](const vector<int>& item) {
            return static_cast<double>(item[0]) / item[1];
        });

        for (const vector<int>& item : items) {
            const int price = item[0];
            const int weight = item[1];
            // The bag is filled.
            if (capacity <= weight)
                return ans + price * capacity / static_cast<double>(weight);
            ans += price;
            capacity -= weight;
        }

        return -1;
    }
};
```

1676. 2549.cpp

Path: LeetCode-main/solutions/2549. Count Distinct Numbers on Board/2549.cpp

```
class Solution {  
public:  
    int distinctIntegers(int n) {  
        return max(n - 1, 1);  
    }  
};
```

1677. 255-2.cpp

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255-2.cpp

```
class Solution {
public:
    bool verifyPreorder(vector<int>& preorder) {
        int low = INT_MIN;
        stack<int> stack;

        for (const int p : preorder) {
            if (p < low)
                return false;
            while (!stack.empty() && stack.top() < p)
                low = stack.top(), stack.pop();
            stack.push(p);
        }
        return true;
    }
};
```

1678. 255-3.cpp

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255-3.cpp

```
class Solution {
public:
    bool verifyPreorder(vector<int>& preorder) {
        int low = INT_MIN;
        int i = -1;

        for (const int p : preorder) {
            if (p < low)
                return false;
            while (i >= 0 && preorder[i] < p)
                low = preorder[i--];
            preorder[++i] = p;
        }

        return true;
    }
};
```


1679. 255.cpp

Path: LeetCode-main/solutions/255. Verify Preorder Sequence in Binary Search Tree/255.cpp

```
class Solution {
public:
    bool verifyPreorder(vector<int>& preorder) {
        int i = 0;
        dfs(preorder, i, INT_MIN, INT_MAX);
        return i == preorder.size();
    }

private:
    void dfs(const vector<int>& preorder, int& i, int min, int max) {
        if (i == preorder.size())
            return;
        if (preorder[i] < min || preorder[i] > max)
            return;

        const int val = preorder[i++];
        dfs(preorder, i, min, val);
        dfs(preorder, i, val, max);
    }
};
```

1680. 2550.cpp

Path: LeetCode-main/solutions/2550. Count Collisions of Monkeys on a Polygon/2550.cpp

```
class Solution {
public:
    int monkeyMove(int n) {
        const int res = modPow(2, n) - 2;
        return res < 0 ? res + kMod : res;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

1681. 2551.cpp

Path: LeetCode-main/solutions/2551. Put Marbles in Bags/2551.cpp

```
class Solution {
public:
    long long putMarbles(vector<int>& weights, int k) {
        // To distribute marbles into k bags, there will be k - 1 cuts. If there's a
        // cut after weights[i], then weights[i] and weights[i + 1] will be added to
        // the cost. Also, no matter how we cut, weights[0] and weights[n - 1] will
        // be counted. So, the goal is to find the max/min k - 1 weights[i] +
        // weights[i + 1].
        vector<int> arr; // weights[i] + weights[i + 1]
        long mn = 0;
        long mx = 0;

        for (int i = 0; i + 1 < weights.size(); ++i)
            arr.push_back(weights[i] + weights[i + 1]);

        ranges::sort(arr);

        for (int i = 0; i < k - 1; ++i) {
            mn += arr[i];
            mx += arr[arr.size() - 1 - i];
        }

        return mx - mn;
    }
};
```

1682. 2552.cpp

Path: LeetCode-main/solutions/2552. Count Increasing Quadruplets/2552.cpp

```
class Solution {
public:
    long long countQuadruplets(vector<int>& nums) {
        long ans = 0;
        // dp[j] := the number of triplets (i, j, k) where i < j < k and nums[i] <
        // nums[k] < nums[j]. Keep this information for l to use later.
        vector<int> dp(nums.size());

        // k can be treated as l.
        for (int k = 2; k < nums.size(); ++k)
            // j can be treated as i.
            for (int j = 0, numLessThanK = 0; j < k; ++j)
                if (nums[j] < nums[k]) {
                    ++numLessThanK; // nums[i] < nums[k]
                    // nums[j] < nums[l], so we should add dp[j] since we find a new
                    // quadruplets for (i, j, k, l).
                    ans += dp[j];
                } else if (nums[j] > nums[k]) {
                    dp[j] += numLessThanK;
                }

        return ans;
    }
};
```

1683. 2553.cpp

Path: LeetCode-main/solutions/2553. Separate the Digits in an Array/2553.cpp

```
class Solution {
public:
    vector<int> separateDigits(vector<int>& nums) {
        vector<int> ans;

        for (const int num : nums)
            for (const char c : to_string(num))
                ans.push_back(c - '0');

        return ans;
    }
};
```

1684. 2554.cpp

Path: LeetCode-main/solutions/2554. Maximum Number of Integers to Choose From a Range I/2554.cpp

```
class Solution {
public:
    int maxCount(vector<int>& banned, int n, int maxSum) {
        int ans = 0;
        int sum = 0;
        const unordered_set<int> bannedSet{banned.begin(), banned.end()};

        for (int i = 1; i <= n; ++i)
            if (!bannedSet.contains(i) && sum + i <= maxSum) {
                ++ans;
                sum += i;
            }

        return ans;
    }
};
```

1685. 2555.cpp

Path: LeetCode-main/solutions/2555. Maximize Win From Two Segments/2555.cpp

```
class Solution {
public:
    int maximizeWin(vector<int>& prizePositions, int k) {
        int ans = 0;
        // dp[i] := the maximum number of prizes to choose the first i
        // `prizePositions`
        vector<int> dp(prizePositions.size() + 1);

        for (int i = 0, j = 0; i < prizePositions.size(); ++i) {
            while (prizePositions[i] - prizePositions[j] > k)
                ++j;
            const int covered = i - j + 1;
            dp[i + 1] = max(dp[i], covered);
            ans = max(ans, dp[j] + covered);
        }

        return ans;
    }
};
```

1686. 2556.cpp

Path: LeetCode-main/solutions/2556. Disconnect Path in a Binary Matrix by at Most One Flip/2556.cpp

```
class Solution {
public:
    bool isPossibleToCutPath(vector<vector<int>>& grid) {
        if (!hasPath(grid, 0, 0))
            return true;
        // Reassign (0, 0) as 1.
        grid[0][0] = 1;
        return !hasPath(grid, 0, 0);
    }

private:
    // Returns true if there's a path from (0, 0) to (m - 1, n - 1).
    // Also marks the visited path as 0 except (m - 1, n - 1).
    bool hasPath(vector<vector<int>>& grid, int i, int j) {
        if (i == grid.size() || j == grid[0].size())
            return false;
        if (i == grid.size() - 1 && j == grid[0].size() - 1)
            return true;
        if (grid[i][j] == 0)
            return false;

        grid[i][j] = 0;
        // Go down first. Since we use OR logic, we'll only mark one path.
        return hasPath(grid, i + 1, j) || hasPath(grid, i, j + 1);
    }
};
```


1687. 2557.cpp

Path: LeetCode-main/solutions/2557. Maximum Number of Integers to Choose From a Range II/2557.cpp

```
class Solution {
public:
    int maxCount(vector<int>& banned, int n, long long maxSum) {
        const unordered_set<int> bannedSet{banned.begin(), banned.end()};
        int l = 1;
        int r = n;

        while (l < r) {
            const int m = (l + r + 1) / 2;
            if (getSum(bannedSet, m) > maxSum)
                r = m - 1;
            else
                l = m;
        }

        return l - ranges::count_if(bannedSet, [l](const int b) { return b <= l; });
    }

private:
    // Returns sum([1..m]) - sum(bannedSet).
    long getSum(const unordered_set<int>& bannedSet, long m) {
        long sum = m * (m + 1) / 2; // sum([1..m])
        for (const int b : bannedSet)
            if (b <= m)
                sum -= b;
        return sum;
    }
};
```

1688. 2558.cpp

Path: LeetCode-main/solutions/2558. Take Gifts From the Richest Pile/2558.cpp

```
class Solution {
public:
    long long pickGifts(vector<int>& gifts, int k) {
        long ans = 0;
        priority_queue<int> maxHeap;

        for (const int gift : gifts)
            maxHeap.push(gift);

        for (int i = 0; i < k; ++i) {
            const int squaredMax = sqrt(maxHeap.top());
            maxHeap.pop();
            maxHeap.push(squaredMax);
        }

        while (!maxHeap.empty())
            ans += maxHeap.top(), maxHeap.pop();

        return ans;
    }
};
```

1689. 2559.cpp

Path: LeetCode-main/solutions/2559. Count Vowel Strings in Ranges/2559.cpp

```
class Solution {
public:
    vector<int> vowelStrings(vector<string>& words,
                             vector<vector<int>>& queries) {
        vector<int> ans;
        // prefix[i] := the number of the first i words that start with and end in a
        // vowel
        vector<int> prefix(words.size() + 1);

        for (int i = 0; i < words.size(); ++i)
            prefix[i + 1] += prefix[i] + startsAndEndsWithVowel(words[i]);

        for (const vector<int>& query : queries) {
            const int l = query[0];
            const int r = query[1];
            ans.push_back(prefix[r + 1] - prefix[l]);
        }

        return ans;
    }

private:
    bool startsAndEndsWithVowel(const string& word) {
        return isVowel(word.front()) && isVowel(word.back());
    }

    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiou";
        return kVowels.find(c) != string_view::npos;
    }
};
```

1690. 256.cpp

Path: LeetCode-main/solutions/256. Paint House/256.cpp

```
class Solution {
public:
    int minCost(vector<vector<int>>& costs) {
        for (int i = 1; i < costs.size(); ++i) {
            costs[i][0] += min(costs[i - 1][1], costs[i - 1][2]);
            costs[i][1] += min(costs[i - 1][0], costs[i - 1][2]);
            costs[i][2] += min(costs[i - 1][0], costs[i - 1][1]);
        }

        return ranges::min(costs.back());
    }
};
```

1691. 2560.cpp

Path: LeetCode-main/solutions/2560. House Robber IV/2560.cpp

```
class Solution {
public:
    int minCapability(vector<int>& nums, int k) {
        int l = ranges::min(nums);
        int r = ranges::max(nums);

        while (l < r) {
            const int m = (l + r) / 2;
            if (numStolenHouses(nums, m) >= k)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    int numStolenHouses(const vector<int>& nums, int capacity) {
        int stolenHouses = 0;
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] <= capacity) {
                ++stolenHouses;
                ++i;
            }
        return stolenHouses;
    }
};
```

1692. 2561.cpp

Path: LeetCode-main/solutions/2561. Rearranging Fruits/2561.cpp

```
class Solution {
public:
    long long minCost(vector<int>& basket1, vector<int>& basket2) {
        vector<int> swapped;
        unordered_map<int, int> count;

        for (const int b : basket1)
            ++count[b];

        for (const int b : basket2)
            --count[b];

        for (const auto& [num, freq] : count) {
            if (freq % 2 != 0)
                return -1;
            for (int i = 0; i < abs(freq) / 2; ++i)
                swapped.push_back(num);
        }

        const int minNum = min(ranges::min(basket1), ranges::min(basket2));
        const auto midIt = swapped.begin() + swapped.size() / 2;
        nth_element(swapped.begin(), midIt, swapped.end());
        return accumulate(swapped.begin(), midIt, 0L, [minNum](long acc, int num) {
            return acc + min(2 * minNum, num);
        });
    }
};
```

1693. 2562.cpp

Path: LeetCode-main/solutions/2562. Find the Array Concatenation Value/2562.cpp

```
class Solution {
public:
    long long findTheArrayConcVal(vector<int>& nums) {
        long ans = 0;

        for (int i = 0, j = nums.size() - 1; i <= j; ++i, --j) {
            ans += nums[j];
            if (i < j)
                ans += nums[i] * pow(10, static_cast<int>(log10(nums[j])) + 1);
        }

        return ans;
    }
};
```

1694. 2563.cpp

Path: LeetCode-main/solutions/2563. Count the Number of Fair Pairs/2563.cpp

```
class Solution {
public:
    long long countFairPairs(vector<int>& nums, int lower, int upper) {
        // nums[i] + nums[j] == nums[j] + nums[i], so the condition that i < j
        // degrades to i != j and we can sort the array.
        ranges::sort(nums);
        return countLess(nums, upper) - countLess(nums, lower - 1);
    }

private:
    long countLess(const vector<int>& nums, int sum) {
        long res = 0;
        for (int i = 0, j = nums.size() - 1; i < j; ++i) {
            while (i < j && nums[i] + nums[j] > sum)
                --j;
            res += j - i;
        }
        return res;
    }
};
```


1695. 2564.cpp

Path: LeetCode-main/solutions/2564. Substring XOR Queries/2564.cpp

```
class Solution {
public:
    vector<vector<int>> substringXorQueries(string s,
                                           vector<vector<int>>& queries) {
        constexpr int kMaxBit = 30;
        vector<vector<int>> ans;
        // {val: (left, right)} := s[left..right]'s decimal value = val
        unordered_map<int, pair<int, int>> valToLeftAndRight;

        for (int left = 0; left < s.length(); ++left) {
            int val = 0;
            if (s[left] == '0') {
                // edge case: Save the index of the first 0.
                if (!valToLeftAndRight.contains(0))
                    valToLeftAndRight[0] = {left, left};
                continue;
            }
            const int maxRight = min(static_cast<int>(s.length()), left + kMaxBit);
            for (int right = left; right < maxRight; ++right) {
                val = val * 2 + s[right] - '0';
                if (!valToLeftAndRight.contains(val))
                    valToLeftAndRight[val] = {left, right};
            }
        }

        for (const vector<int>& query : queries) {
            const int first = query[0];
            const int second = query[1];
            const int val = first ^ second;
            const auto it = valToLeftAndRight.find(val);
            if (it == valToLeftAndRight.cend()) {
                ans.push_back({-1, -1});
            } else {
                const auto [left, right] = it->second;
                ans.push_back({left, right});
            }
        }

        return ans;
    }
};
```

1696. 2566.cpp

Path: LeetCode-main/solutions/2566. Maximum Difference by Remapping a Digit/2566.cpp

```
class Solution {
public:
    int minMaxDifference(int num) {
        const string s = to_string(num);
        const char to9 = s[firstNotNineIndex(s)];
        const char to0 = s[0];
        return getMax(s, to9) - getMin(s, to0);
    }

private:
    int firstNotNineIndex(const string& s) {
        for (int i = 0; i < s.length(); ++i)
            if (s[i] != '9')
                return i;
        return 0;
    }

    int getMax(string s, char to9) {
        for (char& c : s)
            if (c == to9)
                c = '9';
        return stoi(s);
    }

    int getMin(string s, char to0) {
        for (char& c : s)
            if (c == to0)
                c = '0';
        return stoi(s);
    }
};
```


1698. 2568.cpp

Path: LeetCode-main/solutions/2568. Minimum Impossible OR/2568.cpp

```
class Solution {
public:
    int minImpossibleOR(vector<int>& nums) {
        int ans = 1;
        const unordered_set<int> numsSet{nums.begin(), nums.end()};

        while (numsSet.contains(ans))
            ans <<= 1;

        return ans;
    }
};
```

1699. 2569.cpp

Path: LeetCode-main/solutions/2569. Handling Sum Queries After Update/2569.cpp

```
class LazySegmentTree {
public:
    explicit LazySegmentTree(const vector<int>& nums) {
        const int n = nums.size();
        tree.resize(4 * n);
        lazy.resize(4 * n);
        build(nums, 0, 0, n - 1);
    }

    // i := index of the current node
    // [start, end] := range of the current node
    // [l, r] := range of the query
    void updateRange(int i, int start, int end, int l, int r) {
        if (lazy[i])
            propagate(i, start, end);
        if (start > r || end < l)
            return;
        if (start >= l && end <= r) {
            flip(i, start, end);
            return;
        }
        const int mid = (start + end) / 2;
        updateRange(i * 2 + 1, start, mid, l, r);
        updateRange(i * 2 + 2, mid + 1, end, l, r);
        tree[i] = tree[2 * i + 1] + tree[2 * i + 2];
    }

    int getTreeSum() const {
        return tree[0];
    }

private:
    vector<int> tree;
    vector<bool> lazy;

    void build(const vector<int>& nums, int i, int start, int end) {
        if (start == end) {
            tree[i] = nums[start];
            return;
        }
        const int mid = (start + end) / 2;
        build(nums, 2 * i + 1, start, mid);
        build(nums, 2 * i + 2, mid + 1, end);
        tree[i] = tree[2 * i + 1] + tree[2 * i + 2];
    }

    void propagate(int i, int start, int end) {
        flip(i, start, end);
        lazy[i] = false;
    }

    void flip(int i, int start, int end) {
        tree[i] = (end - start + 1) - tree[i]; // Flip 0/1.
        if (start < end) {
            lazy[2 * i + 1] = !lazy[2 * i + 1];
            lazy[2 * i + 2] = !lazy[2 * i + 2];
        }
    }
};

class Solution {
public:
    vector<long long> handleQuery(vector<int>& nums1, vector<int>& nums2,
                                  vector<vector<int>>& queries) {
        vector<long long> ans;
        LazySegmentTree tree(nums1);
        long sumNums2 = accumulate(nums2.begin(), nums2.end(), 0L);

        for (const vector<int>& query : queries) {
            const int type = query[0];
            const int l = query[1];
            const int r = query[2];
            if (type == 1) {
                tree.updateRange(0, 0, nums1.size() - 1, l, r);
            } else if (type == 2) {
                sumNums2 += static_cast<long>(l) * tree.getTreeSum();
            } else { // type == 3
                ans.push_back(sumNums2);
            }
        }
    }
};
```

```
        return ans;  
    }  
};
```

1700. 257.cpp

Path: LeetCode-main/solutions/257. Binary Tree Paths/257.cpp

```
class Solution {
public:
    vector<string> binaryTreePaths(TreeNode* root) {
        vector<string> ans;
        dfs(root, {}, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, vector<string>&& path, vector<string>& ans) {
        if (root == nullptr)
            return;
        if (root->left == nullptr && root->right == nullptr) {
            ans.push_back(join(path) + to_string(root->val));
            return;
        }

        path.push_back(to_string(root->val) + "->");
        dfs(root->left, std::move(path), ans);
        dfs(root->right, std::move(path), ans);
        path.pop_back();
    }

    string join(const vector<string>& path) {
        string joined;
        for (const string& s : path)
            joined += s;
        return joined;
    }
};
```

1701. 2570.cpp

Path: LeetCode-main/solutions/2570. Merge Two 2D Arrays by Summing Values/2570.cpp

```
class Solution {
public:
    vector<vector<int>> mergeArrays(vector<vector<int>>& nums1,
                                   vector<vector<int>>& nums2) {
        constexpr int kMax = 1000;
        vector<vector<int>> ans;
        vector<int> count(kMax + 1);

        addCount(nums1, count);
        addCount(nums2, count);

        for (int i = 1; i <= kMax; ++i)
            if (count[i] > 0)
                ans.push_back({i, count[i]});

        return ans;
    }

private:
    void addCount(const vector<vector<int>>& nums, vector<int>& count) {
        for (const vector<int>& idAndVal : nums) {
            const int id = idAndVal[0];
            const int val = idAndVal[1];
            count[id] += val;
        }
    }
};
```


1702. 2571.cpp

Path: LeetCode-main/solutions/2571. Minimum Operations to Reduce an Integer to 0/2571.cpp

```
class Solution {
public:
    int minOperations(int n) {
        // The strategy is that when the end of n is
        // 1. consecutive 1s, add 1 ( $2^0$ ).
        // 2. single 1, subtract 1 ( $2^0$ ).
        // 3. 0, subtract  $2^k$  to omit the last 1. Equivalently,  $n \gg 1$ .
        //
        // e.g.
        //
        //      n = 0b101
        // n -=  $2^0$  -> 0b100
        // n -=  $2^2$  -> 0b0
        //      n = 0b1011
        // n +=  $2^0$  -> 0b1100
        // n -=  $2^2$  -> 0b1000
        // n -=  $2^3$  -> 0b0
        int ans = 0;

        while (n > 0)
            if ((n & 3) == 3) {
                ++n;
                ++ans;
            } else if (n % 2 == 1) {
                --n;
                ++ans;
            } else {
                n >>= 1;
            }

        return ans;
    }
};
```

1703. 2572.cpp

Path: LeetCode-main/solutions/2572. Count the Number of Square-Free Subsets/2572.cpp

```
class Solution {
public:
    int squareFreeSubsets(vector<int>& nums) {
        vector<vector<int>> mem(nums.size(),
                                vector<int>(1 << (kPrimesCount + 1), -1));

        vector<int> masks;

        for (const int num : nums)
            masks.push_back(getMask(num));

        // -1 means that we take no number.
        // `used` is initialized to 1 so that -1 & 1 = 1 instead of 0.
        return (squareFreeSubsets(masks, 0, /*used=*/1, mem) - 1 + kMod) % kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;
    static constexpr int kPrimesCount = 10;
    static constexpr int primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};

    int squareFreeSubsets(const vector<int>& masks, int i, int used,
                          vector<vector<int>>& mem) {
        if (i == masks.size())
            return 1;
        if (mem[i][used] != -1)
            return mem[i][used];
        const int pick = (masks[i] & used) == 0
            ? squareFreeSubsets(masks, i + 1, used | masks[i], mem)
            : 0;
        const int skip = squareFreeSubsets(masks, i + 1, used, mem);
        return mem[i][used] = (pick + skip) % kMod;
    }

    // e.g. num = 10 = 2 * 5, so mask = 0b101 -> 0b1010 (append a 0)
    //       num = 15 = 3 * 5, so mask = 0b110 -> 0b1100 (append a 0)
    //       num = 25 = 5 * 5, so mask = 0b-1 -> 0b1..1 (invalid)
    int getMask(int num) {
        int mask = 0;
        for (int i = 0; i < sizeof(primes) / sizeof(int); ++i) {
            int rootCount = 0;
            while (num % primes[i] == 0) {
                num /= primes[i];
                ++rootCount;
            }
            if (rootCount >= 2)
                return -1;
            if (rootCount == 1)
                mask |= 1 << i;
        }
        return mask << 1;
    }
};
```

1704. 2573.cpp

Path: LeetCode-main/solutions/2573. Find the String with LCP/2573.cpp

```
class Solution {
public:
    string findTheString(vector<vector<int>>& lcp) {
        const int n = lcp.size();
        constexpr char nonLetter = 'a' - 1;
        char c = nonLetter;
        vector<char> word(n, nonLetter);

        for (int i = 0; i < n; ++i) {
            if (word[i] != nonLetter) // There's a candidate already.
                continue;
            if (++c > 'z') // Run out of letters, so return "".
                return "";
            // No need to consider [0..i - 1] since they were considered.
            for (int j = i; j < n; ++j)
                if (lcp[i][j] > 0)
                    word[j] = c;
        }

        // Check if `word` is valid.
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                const int nextLcp = i + 1 < n && j + 1 < n ? lcp[i + 1][j + 1] : 0;
                const int currLcp = word[i] == word[j] ? 1 + nextLcp : 0;
                if (lcp[i][j] != currLcp)
                    return "";
            }

        string ans;
        for (const char c : word)
            ans += c;
        return ans;
    }
};
```

1705. 2574-2.cpp

Path: LeetCode-main/solutions/2574. Left and Right Sum Differences/2574-2.cpp

```
class Solution {
public:
    vector<int> leftRigthDifference(vector<int>& nums) {
        vector<int> ans;
        int leftSum = 0;
        int rightSum = accumulate(nums.begin(), nums.end(), 0);

        for (const int num : nums) {
            rightSum -= num;
            ans.push_back(abs(leftSum - rightSum));
            leftSum += num;
        }

        return ans;
    }
};
```

1706. 2574.cpp

Path: LeetCode-main/solutions/2574. Left and Right Sum Differences/2574.cpp

```
class Solution {
public:
    vector<int> leftRigthDifference(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans(n);
        vector<int> leftSum(n);
        vector<int> rightSum(n);
        int prefix = 0;
        int suffix = 0;

        for (int i = 0; i < n; ++i) {
            if (i > 0)
                prefix += nums[i - 1];
            leftSum[i] = prefix;
        }

        for (int i = n - 1; i >= 0; --i) {
            if (i + 1 < n)
                suffix += nums[i + 1];
            rightSum[i] = suffix;
        }

        for (int i = 0; i < n; ++i)
            ans[i] = abs(leftSum[i] - rightSum[i]);

        return ans;
    }
};
```

1707. 2575.cpp

Path: LeetCode-main/solutions/2575. Find the Divisibility Array of a String/2575.cpp

```
class Solution {
public:
    vector<int> divisibilityArray(string word, int m) {
        vector<int> ans;
        long prevRemainder = 0;

        for (const char c : word) {
            const long remainder = (prevRemainder * 10 + (c - '0')) % m;
            ans.push_back(remainder == 0 ? 1 : 0);
            prevRemainder = remainder;
        }

        return ans;
    }
};
```

1708. 2576-2.cpp

Path: LeetCode-main/solutions/2576. Find the Maximum Number of Marked Indices/2576-2.cpp

```
class Solution {
public:
    int maxNumOfMarkedIndices(vector<int>& nums) {
        ranges::sort(nums);

        int i = 0;
        for (int j = nums.size() / 2; j < nums.size(); ++j)
            if (2 * nums[i] <= nums[j] && ++i == nums.size() / 2)
                break;

        return i * 2;
    }
};
```

1709. 2576.cpp

Path: LeetCode-main/solutions/2576. Find the Maximum Number of Marked Indices/2576.cpp

```
class Solution {
public:
    int maxNumOfMarkedIndices(vector<int>& nums) {
        ranges::sort(nums);

        int l = 0;
        int r = nums.size() / 2 + 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (isPossible(nums, m))
                l = m + 1;
            else
                r = m;
        }

        return (l - 1) * 2;
    }

private:
    bool isPossible(const vector<int>& nums, int m) {
        for (int i = 0; i < m; ++i)
            if (2 * nums[i] > nums[nums.size() - m + i])
                return false;
        return true;
    }
};
```


1710. 2577.cpp

Path: LeetCode-main/solutions/2577. Minimum Time to Visit a Cell In a Grid/2577.cpp

```
class Solution {
public:
    int minimumTime(vector<vector<int>>& grid) {
        if (grid[0][1] > 1 && grid[1][0] > 1)
            return -1;

        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = grid.size();
        const int n = grid[0].size();
        using T = tuple<int, int, int>; // (time, i, j)
        priority_queue<T, vector<T>, greater<>> minHeap;
        vector<vector<bool>> seen(m, vector<bool>(n));

        minHeap.emplace(0, 0, 0);
        seen[0][0] = true;

        while (!minHeap.empty()) {
            const auto [time, i, j] = minHeap.top();
            minHeap.pop();
            if (i == m - 1 && j == n - 1)
                return time;
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                const int extrawait = (grid[x][y] - time) % 2 == 0 ? 1 : 0;
                const int nextTime = max(time + 1, grid[x][y] + extrawait);
                minHeap.emplace(nextTime, x, y);
                seen[x][y] = true;
            }
        }

        throw;
    }
};
```

1711. 2578.cpp

Path: LeetCode-main/solutions/2578. Split With Minimum Sum/2578.cpp

```
class Solution {
public:
    int splitNum(int num) {
        string num1;
        string num2;
        string s = to_string(num);

        ranges::sort(s);

        for (int i = 0; i < s.length(); ++i)
            if (i % 2 == 0)
                num1 += s[i];
            else
                num2 += s[i];

        return stoi(num1) + stoi(num2);
    }
};
```

1712. 2579.cpp

Path: LeetCode-main/solutions/2579. Count Total Number of Colored Cells/2579.cpp

```
class Solution {  
public:  
    long long coloredCells(int n) {  
        return static_cast<long>(n) * n + static_cast<long>(n - 1) * (n - 1);  
    }  
};
```

1713. 258.cpp

Path: LeetCode-main/solutions/258. Add Digits/258.cpp

```
class Solution {  
public:  
    int addDigits(int num) {  
        return 1 + (num - 1) % 9;  
    }  
};
```

1714. 2580.cpp

Path: LeetCode-main/solutions/2580. Count Ways to Group Overlapping Ranges/2580.cpp

```
class Solution {
public:
    int countWays(vector<vector<int>>& ranges) {
        constexpr int kMod = 1'000'000'007;
        int ans = 1;
        int prevEnd = -1;

        ranges::sort(ranges);

        for (const vector<int>& range : ranges) {
            const int start = range[0];
            const int end = range[1];
            if (start > prevEnd)
                ans = ans * 2 % kMod;
            prevEnd = max(prevEnd, end);
        }

        return ans;
    }
};
```

1715. 2581.cpp

Path: LeetCode-main/solutions/2581. Count Number of Possible Root Nodes/2581.cpp

```
class Solution {
public:
    int rootCount(vector<vector<int>>& edges, vector<vector<int>>& guesses,
                  int k) {
        int ans = 0;
        const int n = edges.size() + 1;
        vector<vector<int>> graph(n);
        vector<unordered_set<int>> guessGraph(n);
        vector<int> parent(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        for (const vector<int>& guess : guesses) {
            const int u = guess[0];
            const int v = guess[1];
            guessGraph[u].insert(v);
        }

        // Precalculate `parent`.
        dfs(graph, 0, /*prev=*/-1, parent);

        // Calculate `correctGuess` for tree rooted at 0.
        int correctGuess = 0;
        for (int i = 1; i < n; ++i)
            if (guessGraph[parent[i]].contains(i))
                ++correctGuess;

        reroot(graph, 0, /*prev=*/-1, correctGuess, guessGraph, k, ans);
        return ans;
    }

private:
    void dfs(const vector<vector<int>>& graph, int u, int prev,
            vector<int>& parent) {
        parent[u] = prev;
        for (const int v : graph[u])
            if (v != prev)
                dfs(graph, v, u, parent);
    }

    void reroot(const vector<vector<int>>& graph, int u, int prev,
               int correctGuess, const vector<unordered_set<int>>& guessGraph,
               const int& k, int& ans) {
        if (u != 0) {
            // The tree is rooted at u, so a guess edge (u, prev) will match the new
            // `parent` relationship.
            if (guessGraph[u].contains(prev))
                ++correctGuess;
            // A guess edge (prev, u) matching the old `parent` relationship will no
            // longer be true.
            if (guessGraph[prev].contains(u))
                --correctGuess;
        }
        if (correctGuess >= k)
            ++ans;
        for (const int v : graph[u])
            if (v != prev)
                reroot(graph, v, u, correctGuess, guessGraph, k, ans);
    }
};
```

1716. 2582.cpp

Path: LeetCode-main/solutions/2582. Pass the Pillow/2582.cpp

```
class Solution {
public:
    int passThePillow(int n, int time) {
        // Repeat every (n - 1) * 2 seconds.
        time %= (n - 1) * 2;
        if (time < n) // Go forward from 1.
            return 1 + time;
        return n - (time - (n - 1)); // Go backward from n.
    }
};
```

1717. 2583.cpp

Path: LeetCode-main/solutions/2583. Kth Largest Sum in a Binary Tree/2583.cpp

```
class Solution {
public:
    long long kthLargestLevelSum(TreeNode* root, int k) {
        vector<long> levelSums;
        dfs(root, 0, levelSums);
        if (levelSums.size() < k)
            return -1;

        nth_element(levelSums.begin(), levelSums.begin() + k - 1, levelSums.end(),
                    greater<>());
        return levelSums[k - 1];
    }

private:
    void dfs(TreeNode* root, int level, vector<long>& levelSums) {
        if (root == nullptr)
            return;
        if (levelSums.size() == level)
            levelSums.push_back(0);
        levelSums[level] += root->val;
        dfs(root->left, level + 1, levelSums);
        dfs(root->right, level + 1, levelSums);
    }
};
```


1718. 2584.cpp

Path: LeetCode-main/solutions/2584. Split the Array to Make Coprime Products/2584.cpp

```
class Solution {
public:
    int findValidSplit(vector<int>& nums) {
        unordered_map<int, int> leftPrimeFactors;
        unordered_map<int, int> rightPrimeFactors;

        for (const int num : nums)
            for (const int primeFactor : getPrimeFactors(num))
                ++rightPrimeFactors[primeFactor];

        for (int i = 0; i < nums.size() - 1; ++i) {
            for (const int primeFactor : getPrimeFactors(nums[i])) {
                if (--rightPrimeFactors[primeFactor] == 0) {
                    // rightPrimeFactors[primeFactor] == 0, so no need to track
                    // leftPrimeFactors[primeFactor].
                    rightPrimeFactors.erase(primeFactor);
                    leftPrimeFactors.erase(primeFactor);
                } else {
                    // Otherwise, need to track leftPrimeFactors[primeFactor].
                    ++leftPrimeFactors[primeFactor];
                }
            }
            if (leftPrimeFactors.empty())
                return i;
        }

        return -1;
    }

private:
    // Gets the prime factors under sqrt(10^6).
    vector<int> getPrimeFactors(int num) {
        vector<int> primeFactors;
        for (int divisor = 2; divisor <= min(1000, num); ++divisor)
            if (num % divisor == 0) {
                primeFactors.push_back(divisor);
                while (num % divisor == 0)
                    num /= divisor;
            }
        // Handle the case that `num` contains a prime factor > 1000.
        if (num > 1)
            primeFactors.push_back(num);
        return primeFactors;
    }
};
```

1719. 2585-2.cpp

Path: LeetCode-main/solutions/2585. Number of Ways to Earn Points/2585-2.cpp

```
class Solution {
public:
    int waysToReachTarget(int target, vector<vector<int>>& types) {
        constexpr int kMod = 1'000'000'007;
        // dp[j] := the number of ways to earn j points with the types so far
        vector<int> dp(target + 1);
        dp[0] = 1;

        for (const vector<int>& type : types) {
            const int count = type[0];
            const int mark = type[1];
            for (int j = target; j >= 0; --j)
                for (int solved = 1; solved <= count; ++solved)
                    if (j - solved * mark >= 0) {
                        dp[j] += dp[j - solved * mark];
                        dp[j] %= kMod;
                    }
        }
        return dp[target];
    }
};
```

1720. 2585.cpp

Path: LeetCode-main/solutions/2585. Number of Ways to Earn Points/2585.cpp

```
class Solution {
public:
    int waysToReachTarget(int target, vector<vector<int>>& types) {
        constexpr int kMod = 1'000'000'007;
        // dp[i][j] := the number of ways to earn j points with the first i types
        vector<vector<int>> dp(types.size() + 1, vector<int>(target + 1));
        dp[0][0] = 1;

        for (int i = 1; i <= types.size(); ++i) {
            const int count = types[i - 1][0];
            const int mark = types[i - 1][1];
            for (int j = 0; j <= target; ++j)
                for (int solved = 0; solved <= count; ++solved)
                    if (j - solved * mark >= 0) {
                        dp[i][j] += dp[i - 1][j - solved * mark];
                        dp[i][j] %= kMod;
                    }
        }

        return dp[types.size()][target];
    }
};
```

1721. 2586.cpp

Path: LeetCode-main/solutions/2586. Count the Number of Vowel Strings in Range/2586.cpp

```
class Solution {
public:
    int vowelStrings(vector<string>& words, int left, int right) {
        return count_if(words.begin() + left, words.begin() + right + 1,
            [this](const string& word) {
                return isVowel(word.front()) && isVowel(word.back());
            });
    }

private:
    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiou";
        return kVowels.find(c) != string_view::npos;
    }
};
```

1722. 2587.cpp

Path: LeetCode-main/solutions/2587. Rearrange Array to Maximize Prefix Score/2587.cpp

```
class Solution {
public:
    int maxScore(vector<int>& nums) {
        long prefix = 0;

        ranges::sort(nums, greater<>());

        for (int i = 0; i < nums.size(); ++i) {
            prefix += nums[i];
            if (prefix <= 0)
                return i;
        }

        return nums.size();
    }
};
```

1723. 2588.cpp

Path: LeetCode-main/solutions/2588. Count the Number of Beautiful Subarrays/2588.cpp

```
class Solution {
public:
    long long beautifulSubarrays(vector<int>& nums) {
        // A subarray is beautiful if xor(subarray) = 0.
        long ans = 0;
        int prefix = 0;
        unordered_map<int, int> prefixCount{{0, 1}};

        for (const int num : nums) {
            prefix ^= num;
            ans += prefixCount[prefix]++;
        }

        return ans;
    }
};
```

1724. 2589.cpp

Path: LeetCode-main/solutions/2589. Minimum Time to Complete All Tasks/2589.cpp

```
class Solution {
public:
    int findMinimumTime(vector<vector<int>>& tasks) {
        constexpr int kMax = 2000;
        vector<bool> running(kMax + 1);

        // Sort tasks by end.
        sort(
            tasks.begin(), tasks.end(),
            [](const vector<int>& a, const vector<int>& b) { return a[1] < b[1]; });

        for (const vector<int>& task : tasks) {
            const int start = task[0];
            const int end = task[1];
            const int duration = task[2];
            int neededDuration = duration - count(running.begin() + start,
                                                  running.begin() + end + 1, true);

            // Greedily run the task as late as possible so that later tasks can run
            // simultaneously.
            for (int i = end; neededDuration > 0; --i) {
                if (!running[i]) {
                    running[i] = true;
                    --neededDuration;
                }
            }
        }

        return ranges::count(running, true);
    }
};
```

1725. 259.cpp

Path: LeetCode-main/solutions/259. 3Sum Smaller/259.cpp

```
class Solution {
public:
    int threeSumSmaller(vector<int>& nums, int target) {
        if (nums.size() < 3)
            return 0;

        int ans = 0;

        ranges::sort(nums);

        for (int i = 0; i + 2 < nums.size(); ++i) {
            int l = i + 1;
            int r = nums.size() - 1;
            while (l < r)
                if (nums[i] + nums[l] + nums[r] < target) {
                    // (nums[i], nums[l], nums[r])
                    // (nums[i], nums[l], nums[r - 1])
                    // ...,
                    // (nums[i], nums[l], nums[l + 1])
                    ans += r - l;
                    ++l;
                } else {
                    --r;
                }
        }

        return ans;
    }
};
```


1726. 2590.cpp

Path: LeetCode-main/solutions/2590. Design a Todo List/2590.cpp

```
struct Task {
    // int taskId;
    string taskDescription;
    int dueDate;
    unordered_set<string> tags;

    bool operator<(const Task& other) const {
        return dueDate < other.dueDate;
    }
};

class TodoList {
public:
    int addTask(int userId, string taskDescription, int dueDate,
               vector<string> tags) {
        vector<string> res;
        int taskId = ++taskIdCounter;
        Task task{taskDescription, dueDate, tags};
        taskIds.insert(taskId);
        return taskId;
    }

    vector<string> getAllTasks(int userId) {
        vector<string> res;
        for (const Task& task : getTasksSortedByDueDate(userId))
            res.push_back(task.taskDescription);
        return res;
    }

    vector<string> getTasksForTag(int userId, string tag) {
        vector<string> res;
        for (const Task& task : getTasksSortedByDueDate(userId))
            if (task.tags.contains(tag))
                res.push_back(task.taskDescription);
        return res;
    }

    void completeTask(int userId, int taskId) {
        if (!taskIds.contains(taskId))
            return;
        const auto it = taskIdToTasks.find(userId);
        if (it == taskIdToTasks.cend())
            return;
        auto& tasks = it->second;
        if (!tasks.contains(taskId))
            return;
        tasks.erase(taskId);
    }

private:
    int taskIdCounter = 0;
    unordered_set<int> taskIds;
    // {userId: {taskId: tasks}}
    unordered_map<int, unordered_map<int, Task>> taskIdToTasks;

    vector<Task> getTasksSortedByDueDate(int userId) {
        const auto it = taskIdToTasks.find(userId);
        if (it == taskIdToTasks.cend())
            return {};
        set<Task> tasks;
        for (const auto& [_, task] : it->second)
            tasks.insert(task);
        return {tasks.begin(), tasks.end()};
    }
};
```

1727. 2591.cpp

Path: LeetCode-main/solutions/2591. Distribute Money to Maximum Children/2591.cpp

```
class Solution {
public:
    int distMoney(int money, int children) {
        // Everyone must receive at least 1 dollar.
        money -= children;
        if (money < 0)
            return -1;

        const int count7 = money / 7;
        const int remaining = money % 7;

        // Distribute 8 dollars to every child.
        if (count7 == children && remaining == 0)
            return count7;

        // Need to move 1 dollar from the last child with 4 dollars to one of other
        // children. That's why we need to subtract 1.
        if (count7 == children - 1 && remaining == 3)
            return count7 - 1;

        // Though there might be child with 4 dollars, since count7 < children - 1,
        // we have "extra" spot to move money to if needed.
        return min(children - 1, count7);
    }
};
```

1728. 2592.cpp

Path: LeetCode-main/solutions/2592. Maximize Greatness of an Array/2592.cpp

```
class Solution {
public:
    int maximizeGreatness(vector<int>& nums) {
        int ans = 0;

        ranges::sort(nums);

        for (const int num : nums)
            if (num > nums[ans])
                ++ans;

        return ans;
    }
};
```

1729. 2593.cpp

Path: LeetCode-main/solutions/2593. Find Score of an Array After Marking All Elements/2593.cpp

```
class Solution {
public:
    long long findScore(vector<int>& nums) {
        long ans = 0;
        set<pair<int, int>> numAndIndexes;
        vector<bool> seen(nums.size());

        for (int i = 0; i < nums.size(); ++i)
            numAndIndexes.insert({nums[i], i});

        for (const auto& [num, i] : numAndIndexes) {
            if (seen[i])
                continue;
            if (i > 0)
                seen[i - 1] = true;
            if (i + 1 < nums.size())
                seen[i + 1] = true;
            seen[i] = true;
            ans += num;
        }

        return ans;
    }
};
```

1730. 2594.cpp

Path: LeetCode-main/solutions/2594. Minimum Time to Repair Cars/2594.cpp

```
class Solution {
public:
    long long repairCars(vector<int>& ranks, int cars) {
        long l = 0;
        long r = static_cast<long>(ranges::min(ranks)) * cars * cars;

        while (l < r) {
            const long m = (l + r) / 2;
            if (numCarsFixed(ranks, m) >= cars)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    long numCarsFixed(const vector<int>& ranks, long minutes) {
        long carsFixed = 0;
        //    r * n^2 = minutes
        // -> n = sqrt(minutes / r)
        for (const int rank : ranks)
            carsFixed += sqrt(minutes / rank);
        return carsFixed;
    }
};
```

1731. 2595.cpp

Path: LeetCode-main/solutions/2595. Number of Even and Odd Bits/2595.cpp

```
class Solution {
public:
    vector<int> evenOddBit(int n) {
        vector<int> ans(2);
        int i = 0; // 0 := even, 1 := odd

        while (n > 0) {
            ans[i] += n & 1;
            n >>= 1;
            i ^= 1;
        }

        return ans;
    }
};
```

1732. 2596.cpp

Path: LeetCode-main/solutions/2596. Check Knight Tour Configuration/2596.cpp

```
class Solution {
public:
    bool checkValidGrid(vector<vector<int>>& grid) {
        if (grid[0][0] != 0)
            return false;

        const int n = grid.size();
        int i = 0;
        int j = 0;

        for (int target = 1; target < n * n; ++target) {
            const auto [x, y] = nextGrid(grid, i, j, target);
            if (x == -1 && y == -1)
                return false;
            // Move (x, y) to (i, j).
            i = x;
            j = y;
        }

        return true;
    }

private:
    static constexpr int kDirs[8][2] = {{-2, 1}, {-1, 2}, {1, 2}, {2, 1},
                                         {2, -1}, {1, -2}, {-1, -2}, {-2, -1}};

    // Returns (x, y), where grid[x][y] == target if (i, j) can reach target.
    pair<int, int> nextGrid(const vector<vector<int>>& grid, int i, int j,
                           int target) {
        const int n = grid.size();
        for (const auto& [dx, dy] : kDirs) {
            const int x = i + dx;
            const int y = j + dy;
            if (x < 0 || x >= n || y < 0 || y >= n)
                continue;
            if (grid[x][y] == target)
                return {x, y};
        }
        return {-1, -1};
    }
};
```

1733. 2597.cpp

Path: LeetCode-main/solutions/2597. The Number of Beautiful Subsets/2597.cpp

```
// e.g. nums = [2, 3, 4, 4], k = 2
//
// subset[0] = [2, 4, 4']
// subset[1] = [1]
// count = {2: 1, 4: 2, 1: 1}
//
// Initially, skip = len([]) = 0, pick = len([]) = 0
//
// * For values in subset[0]:
//   After 2:
//     skip = skip + pick = len([]) = 0
//     pick = (2^count[2] - 1) * (1 + skip + pick)
//           = len([[2]]) * len([])
//           = len([[2]]) = 1
//   After 4:
//     skip = skip + pick = len([[2]]) = 1
//     pick = (2^count[4] - 1) * (1 + skip + pick)
//           = len([[4], [4']], [4, 4']]) * len([])
//           = len([[4], [4']], [4, 4']]) = 3
//
// * For values in subset[1]:
//   After 1:
//     skip = skip + pick
//           = len([[2], [4], [4']], [4, 4']]) = 4
//     pick = (2^count[1] - 1) * (1 + skip + pick)
//           = len([[1]]) * len([[2], [4], [4']], [4, 4']])
//           = len([[1], [1, 2], [1, 4], [1, 4']], [1, 4, 4']]) = 5
//
// So, ans = skip + pick = 9

class Solution {
public:
    int beautifulSubsets(vector<int>& nums, int k) {
        constexpr int kMax = 1000;
        vector<int> count(kMax + 1);
        unordered_map<int, set<int>> modToSubset;

        for (const int num : nums) {
            ++count[num];
            modToSubset[num % k].insert(num);
        }

        int prevNum = -k;
        int skip = 0;
        int pick = 0;

        for (const auto& [_, subset] : modToSubset)
            for (const int num : subset) {
                const int nonEmptyCount = pow(2, count[num]) - 1;
                const int cacheSkip = skip;
                skip += pick;
                pick =
                    nonEmptyCount * (1 + cacheSkip + (num - prevNum == k ? 0 : pick));
                prevNum = num;
            }

        return skip + pick;
    }
};
```


1734. 2598.cpp

Path: LeetCode-main/solutions/2598. Smallest Missing Non-negative Integer After Operations/2598.cpp

```
class Solution {
public:
    int findSmallestInteger(vector<int>& nums, int value) {
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[(num % value + value) % value];

        for (int i = 0; i < nums.size(); ++i)
            if (--count[i % value] < 0)
                return i;

        return nums.size();
    }
};
```

1735. 2599.cpp

Path: LeetCode-main/solutions/2599. Make the Prefix Sum Non-negative/2599.cpp

```
class Solution {
public:
    int makePrefSumNonNegative(vector<int>& nums) {
        int ans = 0;
        long prefix = 0;
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const int num : nums) {
            prefix += num;
            if (num < 0)
                minHeap.push(num);
            while (prefix < 0) {
                prefix -= minHeap.top(), minHeap.pop();
                ++ans;
            }
        }
        return ans;
    }
};
```

1736. 26.cpp

Path: LeetCode-main/solutions/26. Remove Duplicates from Sorted Array/26.cpp

```
class Solution {
public:
    int removeDuplicates(vector<int>& nums) {
        int i = 0;

        for (const int num : nums)
            if (i < 1 || num > nums[i - 1])
                nums[i++] = num;

        return i;
    }
};
```

1737. 260.cpp

Path: LeetCode-main/solutions/260. Single Number III/260.cpp

```
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {
        const int xors = accumulate(nums.begin(), nums.end(), 0, bit_xor<>());
        const int lowbit = xors & -xors;
        vector<int> ans(2);

        // Seperate `nums` into two groups by `lowbit`.
        for (const int num : nums)
            if (num & lowbit)
                ans[0] ^= num;
            else
                ans[1] ^= num;

        return ans;
    }
};
```

1738. 2600.cpp

Path: LeetCode-main/solutions/2600. K Items With the Maximum Sum/2600.cpp

```
class Solution {
public:
    int kItemsWithMaximumSum(int numOnes, int numZeros, int numNegOnes, int k) {
        if (k <= numOnes)
            return k;
        if (k <= numOnes + numZeros)
            return numOnes;
        return numOnes - (k - numOnes - numZeros);
    }
};
```

1739. 2601.cpp

Path: LeetCode-main/solutions/2601. Prime Subtraction Operation/2601.cpp

```
class Solution {
public:
    bool primeSubOperation(vector<int>& nums) {
        constexpr int kMax = 1000;
        const vector<int> primes = sieveEratosthenes(kMax);

        int prevNum = 0;
        for (int num : nums) {
            // Make nums[i] the smallest as possible and still > nums[i - 1].
            const auto it = ranges::lower_bound(primes, num - prevNum);
            if (it != primes.begin())
                num -= *prev(it);
            if (num <= prevNum)
                return false;
            prevNum = num;
        }

        return true;
    }

    vector<int> sieveEratosthenes(int n) {
        vector<int> primes;
        vector<bool> isPrime(n, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        for (int i = 2; i < n; ++i)
            if (isPrime[i])
                primes.push_back(i);
        return primes;
    }
};
```

1740. 2602.cpp

Path: LeetCode-main/solutions/2602. Minimum Operations to Make All Array Elements Equal/2602.cpp

```
class Solution {
public:
    vector<long long> minOperations(vector<int>& nums, vector<int>& queries) {
        const int n = nums.size();
        vector<long long> ans;
        vector<long long> prefix{0};

        ranges::sort(nums);

        for (int i = 0; i < n; ++i)
            prefix.push_back(prefix.back() + nums[i]);

        for (const long query : queries) {
            const int i = ranges::upper_bound(nums, query) - nums.begin();
            // Since nums[0..i) <= query, nums[i..n) > `query`, we should
            // - increase each num in nums[0..i) to `query` and
            // - decrease each num in nums[i..n) to `query`.
            const long inc = query * i - prefix[i];
            const long dec = prefix[n] - prefix[i] - query * (n - i);
            ans.push_back(inc + dec);
        }

        return ans;
    }
};
```

1741. 2603.cpp

Path: LeetCode-main/solutions/2603. Collect Coins in a Tree/2603.cpp

```
class Solution {
public:
    int collectTheCoins(vector<int>& coins, vector<vector<int>>& edges) {
        const int n = coins.size();
        vector<unordered_set<int>> tree(n);
        queue<int> leavesToBeRemoved;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            tree[u].insert(v);
            tree[v].insert(u);
        }

        for (int i = 0; i < n; ++i) {
            int u = i;
            // Remove the leaves that don't have coins.
            while (tree[u].size() == 1 && coins[u] == 0) {
                const int v = *tree[u].begin();
                tree[u].clear();
                tree[v].erase(u);
                u = v; // Walk up to its parent.
            }
            // After trimming leaves without coins, leaves with coins may satisfy
            // `leavesToBeRemoved`.
            if (tree[u].size() == 1)
                leavesToBeRemoved.push(u);
        }

        // Remove each remaining leaf node and its parent. The remaining nodes are
        // the ones that must be visited.
        for (int i = 0; i < 2; ++i)
            for (int sz = leavesToBeRemoved.size(); sz > 0; --sz) {
                const int u = leavesToBeRemoved.front();
                leavesToBeRemoved.pop();
                if (!tree[u].empty()) {
                    const int v = *tree[u].begin();
                    tree[u].clear();
                    tree[v].erase(u);
                    if (tree[v].size() == 1)
                        leavesToBeRemoved.push(v);
                }
            }

        return accumulate(tree.begin(), tree.end(), 0,
            [](int acc, const unordered_set<int>& children) {
                return acc + children.size();
            });
    }
};
```


1742. 2604.cpp

Path: LeetCode-main/solutions/2604. Minimum Time to Eat All Grains/2604.cpp

```
class Solution {
public:
    int minimumTime(vector<int>& hens, vector<int>& grains) {
        ranges::sort(hens);
        ranges::sort(grains);

        const int maxPosition = max(ranges::max(hens), ranges::max(grains));
        const int minPosition = min(ranges::min(hens), ranges::min(grains));
        int l = 0;
        int r = 1.5 * (maxPosition - minPosition);

        while (l < r) {
            const int m = (l + static_cast<long>(r)) / 2;
            if (canEat(hens, grains, m))
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    // Returns true if `hens` can eat all `grains` within `time`.
    bool canEat(const vector<int>& hens, const vector<int>& grains, int time) {
        int i = 0; // grains[i] := next grain to be ate
        for (const int hen : hens) {
            int moves = time;
            if (grains[i] < hen) {
                // `hen` needs go back to eat `grains[i]`.
                const int leftMoves = hen - grains[i];
                if (leftMoves > time)
                    return false;
                const int leftThenRight = time - 2 * leftMoves;
                const int rightThenLeft = (time - leftMoves) / 2;
                moves = max({0, leftThenRight, rightThenLeft});
            }
            i = ranges::upper_bound(grains, hen + moves) - grains.begin();
            if (i == grains.size())
                return true;
        }
        return false;
    }
};
```

1743. 2605.cpp

Path: LeetCode-main/solutions/2605. Form Smallest Number From Two Digit Arrays/2605.cpp

```
class Solution {
public:
    int minNumber(vector<int>& nums1, vector<int>& nums2) {
        int ans = 89; // the largest num we can have
        for (const int a : nums1)
            for (const int b : nums2)
                ans = min(ans, a == b ? a : min(a, b) * 10 + max(a, b));
        return ans;
    }
};
```

1744. 2606.cpp

Path: LeetCode-main/solutions/2606. Find the Substring With Maximum Cost/2606.cpp

```
class Solution {
public:
    int maximumCostSubstring(string s, string chars, vector<int>& vals) {
        int ans = 0;
        int cost = 0;
        vector<int> costs(26); // costs[i] := the cost of 'a' + i

        iota(costs.begin(), costs.end(), 1);

        for (int i = 0; i < chars.size(); ++i)
            costs[chars[i] - 'a'] = vals[i];

        for (const char c : s) {
            cost = max(0, cost + costs[c - 'a']);
            ans = max(ans, cost);
        }

        return ans;
    }
};
```

1745. 2607.cpp

Path: LeetCode-main/solutions/2607. Make K-Subarray Sums Equal/2607.cpp

```
class Solution {
public:
    long long makeSubKSumEqual(vector<int>& arr, int k) {
        // If the sum of each subarray of length k is equal, then `arr` must have a
        // repeated pattern of size k. e.g. arr = [1, 2, 3, ...] and k = 3, to have
        // sum([1, 2, 3]) == sum([2, 3, x]), x must be 1. Therefore, arr[i] ==
        // arr[(i + k) % n] for every i.
        const int n = arr.size();
        long ans = 0;
        vector<bool> seen(n);

        for (int i = 0; i < n; ++i) {
            vector<int> groups;
            int j = i;
            while (!seen[j]) {
                groups.push_back(arr[j]);
                seen[j] = true;
                j = (j + k) % n;
            }
            nth_element(groups.begin(), groups.begin() + groups.size() / 2,
                        groups.end());
            for (const int num : groups)
                ans += abs(num - groups[groups.size() / 2]);
        }

        return ans;
    }
};
```

1746. 2608.cpp

Path: LeetCode-main/solutions/2608. Shortest Cycle in a Graph/2608.cpp

```
class Solution {
public:
    int findShortestCycle(int n, vector<vector<int>>& edges) {
        int ans = kInf;
        vector<vector<int>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        for (int i = 0; i < n; ++i)
            ans = min(ans, bfs(graph, i));

        return ans == kInf ? -1 : ans;
    }

private:
    static constexpr int kInf = 1001;

    // Returns the length of the minimum cycle by starting BFS from node `i`.
    // Returns `kInf` if there's no cycle.
    int bfs(const vector<vector<int>>& graph, int i) {
        vector<int> dist(graph.size(), kInf);
        queue<int> q{{i}};
        dist[i] = 0;
        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            for (const int v : graph[u]) {
                if (dist[v] == kInf) {
                    dist[v] = dist[u] + 1;
                    q.push(v);
                } else if (dist[v] + 1 != dist[u]) { // v is not a parent u.
                    return dist[v] + dist[u] + 1;
                }
            }
        }
        return kInf;
    }
};
```

1747. 2609-2.cpp

Path: LeetCode-main/solutions/2609. Find the Longest Balanced Substring of a Binary String/2609-2.cpp

```
class Solution {
public:
    int findTheLongestBalancedSubstring(string s) {
        int ans = 0;
        int zeros = 0;
        int ones = 0;

        for (const char c : s) {
            if (c == '0') {
                zeros = ones > 0 ? 1 : zeros + 1;
                ones = 0;
            } else { // c == '1'
                ++ones;
            }
            if (zeros >= ones)
                ans = max(ans, ones);
        }
        return ans * 2;
    }
};
```

1748. 2609.cpp

Path: LeetCode-main/solutions/2609. Find the Longest Balanced Substring of a Binary String/2609.cpp

```
class Solution {
public:
    int findTheLongestBalancedSubstring(string s) {
        int ans = 0;

        for (int i = 0; i < s.length(); i) {
            int zeros = 0;
            int ones = 0;
            while (i < s.length() && s[i] == '0') {
                ++zeros;
                ++i;
            }
            while (i < s.length() && s[i] == '1') {
                ++ones;
                ++i;
            }
            ans = max(ans, min(zeros, ones));
        }

        return ans * 2;
    }
};
```

1749. 261-2.cpp

Path: LeetCode-main/solutions/261. Graph Valid Tree/261-2.cpp

```
class UnionFind {
public:
    UnionFind(int n) : count(n), id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        --count;
    }

    int getCount() const {
        return count;
    }

private:
    int count;
    vector<int> id;
    vector<int> rank;

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }
};

class Solution {
public:
    bool validTree(int n, vector<vector<int>>& edges) {
        if (n == 0 || edges.size() != n - 1)
            return false;

        UnionFind uf(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            uf.unionByRank(u, v);
        }

        return uf.getCount() == 1;
    }
};
```


1750. 261.cpp

Path: LeetCode-main/solutions/261. Graph Valid Tree/261.cpp

```
class Solution {
public:
    bool validTree(int n, vector<vector<int>>& edges) {
        if (n == 0 || edges.size() != n - 1)
            return false;

        vector<vector<int>> graph(n);
        queue<int> q{{0}};
        unordered_set<int> seen{{0}};

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            for (const int v : graph[u])
                if (!seen.contains(v)) {
                    q.push(v);
                    seen.insert(v);
                }
        }

        return seen.size() == n;
    }
};
```

1751. 2610.cpp

Path: LeetCode-main/solutions/2610. Convert an Array Into a 2D Array With Conditions/2610.cpp

```
class Solution {
public:
    vector<vector<int>> findMatrix(vector<int>& nums) {
        // The number of rows we need equals the maximum frequency.
        vector<vector<int>> ans;
        vector<int> count(nums.size() + 1);

        for (const int num : nums) {
            // Construct `ans` on demand.
            if (++count[num] > ans.size())
                ans.push_back({});
            ans[count[num] - 1].push_back(num);
        }

        return ans;
    }
};
```

1752. 2611.cpp

Path: LeetCode-main/solutions/2611. Mice and Cheese/2611.cpp

```
class Solution {
public:
    int miceAndCheese(vector<int>& reward1, vector<int>& reward2, int k) {
        // diffs[i] := reward1[i] - reward2[i].
        vector<int> diffs;

        for (int i = 0; i < reward1.size(); ++i)
            diffs.push_back(reward1[i] - reward2[i]);

        nth_element(diffs.begin(), diffs.begin() + k, diffs.end(), greater<>());
        return accumulate(reward2.begin(), reward2.end(), 0) +
            accumulate(diffs.begin(), diffs.begin() + k, 0);
    }
};
```

1753. 2612.cpp

Path: LeetCode-main/solutions/2612. Minimum Reverse Operations/2612.cpp

```
class Solution {
public:
    vector<int> minReverseOperations(int n, int p, vector<int>& banned, int k) {
        const unordered_set<int> bannedSet{banned.begin(), banned.end()};
        vector<int> ans(n, -1);
        // unseen[i] := the unseen numbers that % 2 == i
        vector<set<int>> unseen(2);

        for (int num = 0; num < n; ++num)
            if (num != p && !bannedSet.contains(num))
                unseen[num % 2].insert(num);

        // Perform BFS from `p`.
        queue<int> q{{p}};
        ans[p] = 0;

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            const int lo = max(u - k + 1, k - 1 - u);
            const int hi = min(u + k - 1, n - 1 - (u - (n - k)));
            // Choose the correct set of numbers.
            set<int>& nums = unseen[lo % 2];
            for (auto it = nums.lower_bound(lo); it != nums.end() && *it <= hi;) {
                ans[*it] = ans[u] + 1;
                q.push(*it);
                it = nums.erase(it);
            }
        }

        return ans;
    }
};
```

1754. 2613.cpp

Path: LeetCode-main/solutions/2613. Beautiful Pairs/2613.cpp

```
class SegmentTree {
public:
    explicit SegmentTree(const vector<int>& nums)
        : maxNum(nums.size()), nums(std::move(nums)), tree(4 * maxNum, -1) {}

    // Updates the value of the node to be the index of the smaller number between
    // the old value stored at the node and the new value `index`, where `i` is
    // the index of the element in the `nums` array.
    void update(int i, int numIndex) {
        update(0, 0, maxNum, i, numIndex);
    }

    // Returns the index k s.t. `nums[k]` is maximum number in the range [i, j].
    int query(int i, int j) const {
        return query(0, 0, maxNum, i, j);
    }

private:
    // default node value for non-overlapping queries
    static constexpr int kDefaultValue = -1;
    const int maxNum;
    const vector<int> nums; // nums2 + nums1 or nums2 - nums1
    vector<int> tree;       // tree[i] := the minimum index stored in this node

    void update(int treeIndex, int lo, int hi, int i, int numIndex) {
        if (lo == hi) {
            tree[treeIndex] = merge(tree[treeIndex], numIndex);
            return;
        }
        const int mid = (lo + hi) / 2;
        if (i <= mid)
            update(2 * treeIndex + 1, lo, mid, i, numIndex);
        else
            update(2 * treeIndex + 2, mid + 1, hi, i, numIndex);
        tree[treeIndex] = merge(tree[2 * treeIndex + 1], tree[2 * treeIndex + 2]);
    }

    int query(int treeIndex, int lo, int hi, int i, int j) const {
        if (i <= lo && hi <= j) // [lo, hi] lies completely inside [i, j].
            return tree[treeIndex];
        if (j < lo || hi < i) // [lo, hi] lies completely outside [i, j].
            return kDefaultValue;
        const int mid = (lo + hi) / 2;
        return merge(query(2 * treeIndex + 1, lo, mid, i, j),
                     query(2 * treeIndex + 2, mid + 1, hi, i, j));
    }

    // Returns the better index. Index i is better than index j if
    // (nums[i] > nums[j]) or (nums[i] == nums[j] && i < j).
    int merge(const int& i, const int& j) const {
        if (i == -1)
            return j;
        if (j == -1)
            return i;
        if (nums[i] > nums[j])
            return i;
        if (nums[j] > nums[i])
            return j;
        return min(i, j);
    }
};

class Solution {
public:
    vector<int> beautifulPair(vector<int>& nums1, vector<int>& nums2) {
        // The goal is to find the lexicographically smallest beautiful pair (i, j)
        // s.t. |nums1[i] - nums1[j]| + |nums2[i] - nums2[j]| is the smallest.
        //
        // Sort `nums2` and store the order in `indices`.
        // To minimize |nums1[i] - nums1[j]| + |nums2[i] - nums2[j]|, since we
        // already have sorted `nums2`, we can always have nums2[i] > nums2[j] while
        // iterating the array. So we only need to consider the following 2 cases:
        //
        // * nums1[i] >= nums1[j]:
        //   The value will be nums1[i] - nums1[j] + nums2[i] - nums2[j]
        //   = (nums2[i] + nums1[i]) - (nums2[j] + nums1[j])
        //   Just find max (nums2[j] + nums1[j]) s.t. 0 <= nums1[j] <= nums1[i].
        //
        // * nums1[i] <= nums1[j]:
        //   The value will be nums1[j] - nums1[i] + nums2[i] - nums2[j]
```

```

//          = (nums2[i] - nums1[i]) - (nums2[j] - nums1[j])
// Just find max (nums2[j] - nums1[j]) s.t. nums1[i] <= nums1[j] <= n.
constexpr int kInf = 1'000'000'000;
const int n = nums1.size();
vector<int> ans(2, n);
vector<int> nums2PlusNums1;
vector<int> nums2MinusNums1;
vector<int> indices;
int minBeauty = INT_MAX;

for (int i = 0; i < n; ++i) {
    nums2PlusNums1.push_back(nums2[i] + nums1[i]);
    nums2MinusNums1.push_back(nums2[i] - nums1[i]);
    indices.push_back(i);
}

ranges::sort(indices,
              [&nums2](int i, int j) { return nums2[i] < nums2[j]; });

SegmentTree tree1(nums2PlusNums1);
SegmentTree tree2(nums2MinusNums1);

for (const int i : indices) {
    const int num = nums1[i];
    // For case nums1[i] >= nums1[j], find index j s.t. (nums2[j] + nums1[j])
    // is the maximum in the range [0, nums1[i]].
    int j = tree1.query(0, num);
    if (j >= 0)
        updateAns(nums2PlusNums1, i, j, minBeauty, ans);
    tree1.update(num, i);
    // For case nums1[i] <= nums1[j], find index j s.t. (nums2[j] - nums1[j])
    // is the maximum in the range [nums1[i], n].
    j = tree2.query(num, n);
    if (j >= 0)
        updateAns(nums2MinusNums1, i, j, minBeauty, ans);
    tree2.update(num, i);
}

return ans;
}

private:
void updateAns(const vector<int>& nums, int i, int j, int& minBeauty,
               vector<int>& ans) {
    // beauty := |nums1[i] - nums1[j]| + |nums2[i] - nums2[j]|
    const int beauty = nums[i] - nums[j];
    const vector<int> nextAns = {min(i, j), max(i, j)};
    if (beauty < minBeauty) {
        minBeauty = beauty;
        ans = nextAns;
    } else if (beauty == minBeauty) {
        ans = min(ans, nextAns);
    }
}
};

```

1755. 2614.cpp

Path: LeetCode-main/solutions/2614. Prime In Diagonal/2614.cpp

```
class Solution {
public:
    int diagonalPrime(vector<vector<int>>& nums) {
        int ans = 0;
        for (int i = 0; i < nums.size(); ++i) {
            const int a = nums[i][i];
            const int b = nums[i][nums.size() - i - 1];
            if (isPrime(a))
                ans = max(ans, a);
            if (isPrime(b))
                ans = max(ans, b);
        }
        return ans;
    }

private:
    bool isPrime(int n) {
        if (n <= 1)
            return false;
        for (int i = 2; i * i <= n; ++i)
            if (n % i == 0)
                return false;
        return true;
    }
};
```

1756. 2615.cpp

Path: LeetCode-main/solutions/2615. Sum of Distances/2615.cpp

```
class Solution {
public:
    vector<long long> distance(vector<int>& nums) {
        vector<long long> ans(nums.size());
        unordered_map<int, vector<int>> numToIndices;

        for (int i = 0; i < nums.size(); ++i)
            numToIndices[nums[i]].push_back(i);

        for (const auto& [_, indices] : numToIndices) {
            const int n = indices.size();
            if (n == 1)
                continue;
            long sumSoFar = accumulate(indices.begin(), indices.end(), 0L);
            int prevIndex = 0;
            for (int i = 0; i < n; ++i) {
                sumSoFar += (i - 1) * (indices[i] - prevIndex);
                sumSoFar -= (n - 1 - i) * (indices[i] - prevIndex);
                ans[indices[i]] = sumSoFar;
                prevIndex = indices[i];
            }
        }

        return ans;
    }
};
```


1757. 2616.cpp

Path: LeetCode-main/solutions/2616. Minimize the Maximum Difference of Pairs/2616.cpp

```
class Solution {
public:
    int minimizeMax(vector<int>& nums, int p) {
        ranges::sort(nums);

        int l = 0;
        int r = nums.back() - nums.front();

        while (l < r) {
            const int m = (l + r) / 2;
            if (numPairs(nums, m) >= p)
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    // Returns the number of pairs that can be obtained if the difference between
    // each pair <= `maxDiff`.
    int numPairs(const vector<int>& nums, int maxDiff) {
        int pairs = 0;
        for (int i = 1; i < nums.size(); ++i)
            // Greedily pair nums[i] with nums[i - 1].
            if (nums[i] - nums[i - 1] <= maxDiff) {
                ++pairs;
                ++i;
            }
        return pairs;
    }
};
```

1758. 2617.cpp

Path: LeetCode-main/solutions/2617. Minimum Number of Visited Cells in a Grid/2617.cpp

```
class SegmentTree {
public:
    explicit SegmentTree(int n, int kInf) : kInf(kInf), n(n), tree(4 * n, kInf) {}

    // Updates nums[i] to val.
    void update(int i, int val) {
        update(0, 0, n - 1, i, val);
    }

    // Returns min(nums[i..j]).
    int query(int i, int j) const {
        return query(0, 0, n - 1, i, j);
    }

private:
    const int kInf;    // the invalid value
    const int n;       // the size of the input array
    vector<int> tree;   // the segment tree

    void update(int treeIndex, int lo, int hi, int i, int val) {
        if (lo == hi) {
            tree[treeIndex] = val;
            return;
        }
        const int mid = (lo + hi) / 2;
        if (i <= mid)
            update(2 * treeIndex + 1, lo, mid, i, val);
        else
            update(2 * treeIndex + 2, mid + 1, hi, i, val);
        tree[treeIndex] = merge(tree[2 * treeIndex + 1], tree[2 * treeIndex + 2]);
    }

    int query(int treeIndex, int lo, int hi, int i, int j) const {
        if (i <= lo && hi <= j) // [lo, hi] lies completely inside [i, j].
            return tree[treeIndex];
        if (j < lo || hi < i) // [lo, hi] lies completely outside [i, j].
            return kInf;
        const int mid = (lo + hi) / 2;
        return merge(query(treeIndex * 2 + 1, lo, mid, i, j),
                     query(treeIndex * 2 + 2, mid + 1, hi, i, j));
    }

    int merge(int left, int right) const {
        return min(left, right);
    }
};

class Solution {
public:
    int minimumVisitedCells(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        const int kInf = (m + n) * 2 - 1;
        vector<SegmentTree> rows(m, SegmentTree(n, kInf));
        vector<SegmentTree> cols(n, SegmentTree(m, kInf));

        // The min # cells to visit (m - 1, n - 1) from (m - 1, n - 1) is 1.
        rows[m - 1].update(n - 1, 1);
        cols[n - 1].update(m - 1, 1);

        for (int i = m - 1; i >= 0; --i)
            for (int j = n - 1; j >= 0; --j) {
                // There's no k s.t. j < k <= 0 + j.
                // There's no k s.t. i < k <= 0 + i.
                if (grid[i][j] == 0)
                    continue;
                const int moveRight = rows[i].query(j + 1, grid[i][j] + j);
                const int moveDown = cols[j].query(i + 1, grid[i][j] + i);
                const int minMove = min(kInf, min(moveRight, moveDown) + 1);
                rows[i].update(j, minMove);
                cols[j].update(i, minMove);
            }

        const int res = rows[0].query(0, 0);
        return res == kInf ? -1 : res;
    }
};
```

1759. 263.cpp

Path: LeetCode-main/solutions/263. Ugly Number/263.cpp

```
class Solution {
public:
    bool isUgly(int n) {
        if (n == 0)
            return false;

        for (const int prime : {2, 3, 5})
            while (n % prime == 0)
                n /= prime;

        return n == 1;
    }
};
```

1760. 2638.cpp

Path: LeetCode-main/solutions/2638. Count the Number of K-Free Subsets/2638.cpp

```
class Solution {
public:
    // Similar to 2597. The Number of Beautiful Subsets
    long long countTheNumOfKFreeSubsets(vector<int>& nums, int k) {
        unordered_map<int, set<int>> modToSubset;

        for (const int num : nums)
            modToSubset[num % k].insert(num);

        int prevNum = -k;
        long skip = 0;
        long pick = 0;

        for (const auto& [_, subset] : modToSubset)
            for (const int num : subset) {
                const long cacheSkip = skip;
                skip += pick;
                pick = 1 + cacheSkip + (num - prevNum == k ? 0 : pick);
                prevNum = num;
            }

        return 1 + skip + pick;
    }
};
```

1761. 2639.cpp

Path: LeetCode-main/solutions/2639. Find the Width of Columns of a Grid/2639.cpp

```
class Solution {
public:
    vector<int> findColumnWidth(vector<vector<int>>& grid) {
        vector<int> ans;

        for (int j = 0; j < grid[0].size(); ++j) {
            ans.push_back(0);
            for (int i = 0; i < grid.size(); ++i)
                ans[j] = max(ans[j], static_cast<int>(to_string(grid[i][j]).length()));
        }

        return ans;
    }
};
```

1762. 264.cpp

Path: LeetCode-main/solutions/264. Ugly Number II/264.cpp

```
class Solution {
public:
    int nthUglyNumber(int n) {
        vector<int> uglyNums{1};
        int i2 = 0;
        int i3 = 0;
        int i5 = 0;

        while (uglyNums.size() < n) {
            const int next2 = uglyNums[i2] * 2;
            const int next3 = uglyNums[i3] * 3;
            const int next5 = uglyNums[i5] * 5;
            const int next = min({next2, next3, next5});
            if (next == next2)
                ++i2;
            if (next == next3)
                ++i3;
            if (next == next5)
                ++i5;
            uglyNums.push_back(next);
        }
        return uglyNums.back();
    }
};
```

1763. 2640.cpp

Path: LeetCode-main/solutions/2640. Find the Score of All Prefixes of an Array/2640.cpp

```
class Solution {
public:
    vector<long long> findPrefixScore(vector<int>& nums) {
        vector<long long> ans;
        long prefix = 0;
        int mx = 0;

        for (const int num : nums) {
            mx = max(mx, num);
            prefix += num + mx;
            ans.push_back(prefix);
        }

        return ans;
    }
};
```

1764. 2641.cpp

Path: LeetCode-main/solutions/2641. Cousins in Binary Tree II/2641.cpp

```
class Solution {
public:
    TreeNode* replaceValueInTree(TreeNode* root) {
        vector<int> levelSums;
        dfs(root, 0, levelSums);
        return replace(root, 0, new TreeNode(0), levelSums);
    }

private:
    void dfs(TreeNode* root, int level, vector<int>& levelSums) {
        if (root == nullptr)
            return;
        if (levelSums.size() == level)
            levelSums.push_back(0);
        levelSums[level] += root->val;
        dfs(root->left, level + 1, levelSums);
        dfs(root->right, level + 1, levelSums);
    }

    TreeNode* replace(TreeNode* root, int level, TreeNode* curr,
                      const vector<int>& levelSums) {
        const int nextLevel = level + 1;
        const int nextLevelCousinsSum =
            nextLevel >= levelSums.size()
            ? 0
            : levelSums[nextLevel] -
              (root->left == nullptr ? 0 : root->left->val) -
              (root->right == nullptr ? 0 : root->right->val);
        if (root->left != nullptr) {
            curr->left = new TreeNode(nextLevelCousinsSum);
            replace(root->left, level + 1, curr->left, levelSums);
        }
        if (root->right != nullptr) {
            curr->right = new TreeNode(nextLevelCousinsSum);
            replace(root->right, level + 1, curr->right, levelSums);
        }
        return curr;
    }
};
```


1765. 2642.cpp

Path: LeetCode-main/solutions/2642. Design Graph With Shortest Path Calculator/2642.cpp

```
class Graph {
public:
    Graph(int n, vector<vector<int>>& edges) {
        graph.resize(n);
        for (const vector<int>& edge : edges)
            addEdge(edge);
    }

    void addEdge(vector<int> edge) {
        const int u = edge[0];
        const int v = edge[1];
        const int w = edge[2];
        graph[u].emplace_back(v, w);
    }

    int shortestPath(int node1, int node2) {
        vector<int> dist(graph.size(), INT_MAX);
        using P = pair<int, int>; // (d, u)
        priority_queue<P, vector<P>, greater<>> minHeap;

        dist[node1] = 0;
        minHeap.emplace(dist[node1], node1);

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (u == node2)
                return d;
            for (const auto& [v, w] : graph[u])
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.emplace(dist[v], v);
                }
        }

        return -1;
    }

private:
    vector<vector<pair<int, int>>> graph;
};
```

1766. 2643.cpp

Path: LeetCode-main/solutions/2643. Row With Maximum Ones/2643.cpp

```
class Solution {
public:
    vector<int> rowAndMaximumOnes(vector<vector<int>>& mat) {
        vector<int> ans{0, 0};

        for (int i = 0; i < mat.size(); ++i) {
            const int ones = ranges::count(mat[i], 1);
            if (ones > ans[1]) {
                ans[0] = i;
                ans[1] = ones;
            }
        }

        return ans;
    }
};
```

1767. 2644.cpp

Path: LeetCode-main/solutions/2644. Find the Maximum Divisibility Score/2644.cpp

```
class Solution {
public:
    int maxDivScore(vector<int>& nums, vector<int>& divisors) {
        int ans = -1;
        int maxScore = -1;

        for (const int divisor : divisors) {
            const int score = ranges::count_if(
                nums, [divisor](const int num) { return num % divisor == 0; });
            if (score > maxScore) {
                ans = divisor;
                maxScore = score;
            } else if (score == maxScore) {
                ans = min(ans, divisor);
            }
        }

        return ans;
    }
};
```

1768. 2645.cpp

Path: LeetCode-main/solutions/2645. Minimum Additions to Make Valid String/2645.cpp

```
class Solution {
public:
    int addMinimum(string word) {
        const string letters = "abc";
        int ans = 0;
        int i = 0;

        while (i < word.length())
            for (const char c : letters) {
                if (i < word.length() && word[i] == c)
                    ++i;
                else
                    ++ans;
            }
        return ans;
    }
};
```

1769. 2646.cpp

Path: LeetCode-main/solutions/2646. Minimize the Total Price of the Trips/2646.cpp

```
class Solution {
public:
    int minimumTotalPrice(int n, vector<vector<int>>& edges, vector<int>& price,
                          vector<vector<int>>& trips) {
        vector<vector<int>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        // count[i] := the number of times node i is traversed
        vector<int> count(n);

        for (const vector<int>& trip : trips) {
            const int start = trip[0];
            const int end = trip[1];
            vector<int> path;
            dfsCount(graph, start, /*prev=*/-1, end, count, path);
        }

        vector<vector<int>> mem(n, vector<int>(2, -1));
        return dfs(graph, 0, -1, price, count, false, mem);
    }

private:
    void dfsCount(const vector<vector<int>>& graph, int u, int prev, int end,
                  vector<int>& count, vector<int>& path) {
        path.push_back(u);
        if (u == end) {
            for (const int i : path)
                ++count[i];
            return;
        }
        for (const int v : graph[u])
            if (v != prev)
                dfsCount(graph, v, u, end, count, path);
        path.pop_back();
    }

    // Returns the minimum price sum for the i-th node, where its parent is
    // halved parent or not halved not.
    int dfs(const vector<vector<int>>& graph, int u, int prev,
            const vector<int>& price, const vector<int>& count, int parentHalved,
            vector<vector<int>>& mem) {
        if (mem[u][parentHalved] != -1)
            return mem[u][parentHalved];

        int sumWithFullNode = price[u] * count[u];
        for (const int v : graph[u])
            if (v != prev)
                sumWithFullNode += dfs(graph, v, u, price, count, false, mem);

        if (parentHalved) // Can't halve this node if its parent was halved.
            return mem[u][parentHalved] = sumWithFullNode;

        int sumWithHalvedNode = (price[u] / 2) * count[u];
        for (const int v : graph[u])
            if (v != prev)
                sumWithHalvedNode += dfs(graph, v, u, price, count, true, mem);

        return mem[u][parentHalved] = min(sumWithFullNode, sumWithHalvedNode);
    }
};
```

1770. 2647.cpp

Path: LeetCode-main/solutions/2647. Color the Triangle Red/2647.cpp

```
class Solution {
public:
    vector<vector<int>> colorRed(int n) {
        vector<vector<int>> ans;
        const int tipSize = n % 4;

        // The tip of the triangle is always painted red.
        if (tipSize >= 1)
            ans.push_back({1, 1});

        // Paint the rightmost and the leftmost elements at the following rows.
        for (int i = 2; i <= tipSize; ++i) {
            ans.push_back({i, 1});
            ans.push_back({i, 2 * i - 1});
        }

        // Paint the 4-row chunks.
        for (int i = tipSize + 1; i < n; i += 4) {
            // Fill the first row of the chunk.
            ans.push_back({i, 1});
            // Fill the second row.
            for (int j = 1; j <= i; ++j)
                ans.push_back({i + 1, 2 * j + 1});
            // Fill the third row.
            ans.push_back({i + 2, 2});
            // Fill the fourth row.
            for (int j = 0; j <= i + 2; ++j)
                ans.push_back({i + 3, 2 * j + 1});
        }

        return ans;
    }
};
```

1771. 265.cpp

Path: LeetCode-main/solutions/265. Paint House II/265.cpp

```
class Solution {
public:
    int minCostII(vector<vector<int>>& costs) {
        int prevIndex = -1; // the previous minimum index
        int prevMin1 = 0;   // the minimum cost so far
        int prevMin2 = 0;   // the second minimum cost so far

        for (const vector<int>& cost : costs) { // O(n)
            // the painted index that will achieve the minimum cost after painting the
            // current house
            int index = -1;
            // the minimum cost after painting the current house
            int min1 = INT_MAX;
            // the second minimum cost after painting the current house
            int min2 = INT_MAX;
            for (int i = 0; i < cost.size(); ++i) { // O(k)
                const int theCost = cost[i] + (i == prevIndex ? prevMin2 : prevMin1);
                if (theCost < min1) {
                    index = i;
                    min2 = min1;
                    min1 = theCost;
                } else if (theCost < min2) { // min1 <= theCost < min2
                    min2 = theCost;
                }
            }
            prevIndex = index;
            prevMin1 = min1;
            prevMin2 = min2;
        }
        return prevMin1;
    }
};
```

1772. 2651.cpp

Path: LeetCode-main/solutions/2651. Calculate Delayed Arrival Time/2651.cpp

```
class Solution {
public:
    int findDelayedArrivalTime(int arrivalTime, int delayedTime) {
        return (arrivalTime + delayedTime) % 24;
    }
};
```


1773. 2652-2.cpp

Path: LeetCode-main/solutions/2652. Sum Multiples/2652-2.cpp

```
class Solution {
public:
    int sumOfMultiples(int n) {
        // Returns the sum of multiples of value in [1, n].
        auto sumOfMultiples = [n](int value) {
            const int lo = value;
            const int hi = n / value * value;
            const int count = (hi - lo) / value + 1;
            return (lo + hi) * count / 2;
        };
        return sumOfMultiples(3) + sumOfMultiples(5) + sumOfMultiples(7) -
            (sumOfMultiples(15) + sumOfMultiples(21) + sumOfMultiples(35)) +
            sumOfMultiples(105);
    }
};
```

1774. 2652.cpp

Path: LeetCode-main/solutions/2652. Sum Multiples/2652.cpp

```
class Solution {
public:
    int sumOfMultiples(int n) {
        int ans = 0;
        for (int i = 1; i <= n; ++i)
            if (i % 3 == 0 || i % 5 == 0 || i % 7 == 0)
                ans += i;
        return ans;
    }
};
```

1775. 2653.cpp

Path: LeetCode-main/solutions/2653. Sliding Subarray Beauty/2653.cpp

```
class Solution {
public:
    vector<int> getSubarrayBeauty(vector<int>& nums, int k, int x) {
        vector<int> ans;
        vector<int> count(50); // count[i] := the frequency of (i + 50)

        for (int i = 0; i < nums.size(); ++i) {
            if (nums[i] < 0)
                ++count[nums[i] + 50];
            if (i - k >= 0 && nums[i - k] < 0)
                --count[nums[i - k] + 50];
            if (i + 1 >= k)
                ans.push_back(getXthSmallestNum(count, x));
        }

        return ans;
    }

private:
    int getXthSmallestNum(const vector<int>& count, int x) {
        int prefix = 0;
        for (int i = 0; i < 50; ++i) {
            prefix += count[i];
            if (prefix >= x)
                return i - 50;
        }
        return 0;
    }
};
```

1776. 2654.cpp

Path: LeetCode-main/solutions/2654. Minimum Number of Operations to Make All Array Elements Equal to 1/2654.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums) {
        const int n = nums.size();
        const int ones = ranges::count(nums, 1);
        if (ones > 0)
            return n - ones;

        // the minimum operations to make the shortest subarray with a gcd == 1
        int minOps = INT_MAX;

        for (int i = 0; i < n; ++i) {
            int g = nums[i];
            for (int j = i + 1; j < n; ++j) {
                g = __gcd(g, nums[j]);
                if (g == 1) { // gcd(nums[i..j]) == 1
                    minOps = min(minOps, j - i);
                    break;
                }
            }
        }

        // After making the shortest subarray with `minOps`, need additional n - 1
        // operations to make the other numbers to 1.
        return minOps == INT_MAX ? -1 : minOps + n - 1;
    }
};
```

1777. 2655.cpp

Path: LeetCode-main/solutions/2655. Find Maximal Uncovered Ranges/2655.cpp

```
class Solution {
public:
    vector<vector<int>> findMaximalUncoveredRanges(int n,
                                                    vector<vector<int>>& ranges) {
        vector<vector<int>> ans;
        int start = 0;

        ranges::sort(ranges);

        for (const vector<int>& range : ranges) {
            const int l = range[0];
            const int r = range[1];
            if (start < l)
                ans.push_back({start, l - 1});
            if (start <= r)
                start = r + 1;
        }

        if (start < n)
            ans.push_back({start, n - 1});

        return ans;
    }
};
```

1778. 2656.cpp

Path: LeetCode-main/solutions/2656. Maximum Sum With Exactly K Elements/2656.cpp

```
class Solution {
public:
    int maximizeSum(vector<int>& nums, int k) {
        // If x = max(nums), ans = x + (x + 1) + .. + (x + k - 1).
        const int x = ranges::max(nums);
        return x * k + k * (k - 1) / 2;
    }
};
```

1779. 2657.cpp

Path: LeetCode-main/solutions/2657. Find the Prefix Common Array of Two Arrays/2657.cpp

```
class Solution {
public:
    vector<int> findThePrefixCommonArray(vector<int>& A, vector<int>& B) {
        const int n = A.size();
        int prefixCommon = 0;
        vector<int> ans;
        vector<int> count(n + 1);

        for (int i = 0; i < A.size(); ++i) {
            if (++count[A[i]] == 2)
                ++prefixCommon;
            if (++count[B[i]] == 2)
                ++prefixCommon;
            ans.push_back(prefixCommon);
        }

        return ans;
    }
};
```

1780. 2658.cpp

Path: LeetCode-main/solutions/2658. Maximum Number of Fish in a Grid/2658.cpp

```
class Solution {
public:
    int findMaxFish(vector<vector<int>>& grid) {
        int ans = 0;

        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] > 0)
                    ans = max(ans, dfs(grid, i, j));

        return ans;
    }

private:
    int dfs(vector<vector<int>>& grid, int i, int j) {
        if (i < 0 || i == grid.size() || j < 0 || j == grid[0].size())
            return 0;
        if (grid[i][j] == 0)
            return 0;
        int caughtFish = grid[i][j];
        grid[i][j] = 0; // Mark 0 as visited
        return caughtFish +
            dfs(grid, i + 1, j) + dfs(grid, i - 1, j) + //
            dfs(grid, i, j + 1) + dfs(grid, i, j - 1);
    }
};
```


1781. 2659.cpp

Path: LeetCode-main/solutions/2659. Make Array Empty/2659.cpp

```
class Solution {
public:
    long long countOperationsToEmptyArray(vector<int>& nums) {
        const int n = nums.size();
        long ans = n;
        unordered_map<int, int> numToIndex;

        for (int i = 0; i < n; ++i)
            numToIndex[nums[i]] = i;

        ranges::sort(nums);

        for (int i = 1; i < n; ++i)
            // On the i-th step we've already removed the i - 1 smallest numbers and
            // can ignore them. If an element nums[i] has smaller index in origin
            // array than nums[i - 1], we should rotate the whole left array n - i
            // times to set nums[i] element on the first position.
            if (numToIndex[nums[i]] < numToIndex[nums[i - 1]])
                ans += n - i;

        return ans;
    }
};
```

1782. 266.cpp

Path: LeetCode-main/solutions/266. Palindrome Permutation/266.cpp

```
class Solution {
public:
    bool canPermutePalindrome(string s) {
        unordered_set<char> seen;

        for (const char c : s)
            if (!seen.insert(c).second)
                seen.erase(c);

        return seen.size() <= 1;
    }
};
```

1783. 2660.cpp

Path: LeetCode-main/solutions/2660. Determine the Winner of a Bowling Game/2660.cpp

```
class Solution {
public:
    int isWinner(vector<int>& player1, vector<int>& player2) {
        const int score1 = getScore(player1);
        const int score2 = getScore(player2);
        if (score1 > score2)
            return 1;
        if (score2 > score1)
            return 2;
        return 0;
    }

private:
    int getScore(const vector<int>& player) {
        constexpr int kInvalid = -3;
        int score = 0;
        int last10 = kInvalid;
        for (int i = 0; i < player.size(); ++i) {
            score += i - last10 > 2 ? player[i] : player[i] * 2;
            if (player[i] == 10)
                last10 = i;
        }
        return score;
    }
};
```

1784. 2661.cpp

Path: LeetCode-main/solutions/2661. First Completely Painted Row or Column/2661.cpp

```
class Solution {
public:
    int firstCompleteIndex(vector<int>& arr, vector<vector<int>>& mat) {
        const int m = mat.size();
        const int n = mat[0].size();
        // rows[i] := the number of painted grid in the i-th row
        vector<int> rows(m);
        // cols[j] := the number of painted grid in the j-th column
        vector<int> cols(n);
        // numToRow[num] := the i-th row of `num` in `mat`
        vector<int> numToRow(m * n + 1);
        // numToCol[num] := the j-th column of `num` in `mat`
        vector<int> numToCol(m * n + 1);

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                numToRow[mat[i][j]] = i;
                numToCol[mat[i][j]] = j;
            }

        for (int i = 0; i < arr.size(); ++i) {
            if (++rows[numToRow[arr[i]]] == n)
                return i;
            if (++cols[numToCol[arr[i]]] == m)
                return i;
        }

        throw;
    }
};
```

1785. 2662.cpp

Path: LeetCode-main/solutions/2662. Minimum Cost of a Path With Special Roads/2662.cpp

```
class Solution {
public:
    int minimumCost(vector<int>& start, vector<int>& target,
                    vector<vector<int>>& specialRoads) {
        return dijkstra(specialRoads, start[0], start[1], target[0], target[1]);
    }

private:
    int dijkstra(const vector<vector<int>>& specialRoads, int srcX, int srcY,
                 int dstX, int dstY) {
        const int n = specialRoads.size();
        // dist[i] := the minimum distance of (srcX, srcY) to
        // specialRoads[i](x2, y2)
        vector<int> dist(specialRoads.size(), INT_MAX);
        using P = pair<int, int>; // (d, u), where u := the i-th specialRoads
        priority_queue<P, vector<P>, greater<>> minHeap;

        // (srcX, srcY) -> (x1, y1) to cost -> (x2, y2)
        for (int u = 0; u < n; ++u) {
            const int x1 = specialRoads[u][0];
            const int y1 = specialRoads[u][1];
            const int cost = specialRoads[u][4];
            const int d = abs(x1 - srcX) + abs(y1 - srcY) + cost;
            dist[u] = d;
            minHeap.emplace(dist[u], u);
        }

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (d > dist[u])
                continue;
            const int ux2 = specialRoads[u][2];
            const int uy2 = specialRoads[u][3];
            for (int v = 0; v < n; ++v) {
                if (v == u)
                    continue;
                const int vx1 = specialRoads[v][0];
                const int vy1 = specialRoads[v][1];
                const int vcost = specialRoads[v][4];
                // (ux2, uy2) -> (vx1, vy1) to vcost -> (vx2, vy2)
                const int newDist = d + abs(vx1 - ux2) + abs(vy1 - uy2) + vcost;
                if (newDist < dist[v]) {
                    dist[v] = newDist;
                    minHeap.emplace(dist[v], v);
                }
            }
        }

        int ans = abs(dstX - srcX) + abs(dstY - srcY);
        for (int u = 0; u < n; ++u) {
            const int x2 = specialRoads[u][2];
            const int y2 = specialRoads[u][3];
            // (srcX, srcY) -> (x2, y2) -> (dstX, dstY).
            ans = min(ans, dist[u] + abs(dstX - x2) + abs(dstY - y2));
        }
        return ans;
    }
};
```

1786. 2663.cpp

Path: LeetCode-main/solutions/2663. Lexicographically Smallest Beautiful String/2663.cpp

```
class Solution {
public:
    string smallestBeautifulString(string s, int k) {
        for (int i = s.length() - 1; i >= 0; --i) {
            do {
                ++s[i];
            } while (containsPalindrome(s, i));
            if (s[i] < 'a' + k)
                // If s[i] is among the first k letters, then change the letters after
                // s[i] to the smallest ones that don't form any palindrome substring.
                return changeSuffix(s, i + 1);
        }

        return "";
    }

private:
    // Returns true if s[0..i] contains any palindrome.
    bool containsPalindrome(const string& s, int i) {
        return (i > 0 && s[i] == s[i - 1]) || (i > 1 && s[i] == s[i - 2]);
    }

    // Returns a string, where replacing s[i..n) with the smallest possible
    // letters don't form any palindrome substring.
    string changeSuffix(string& s, int i) {
        for (int j = i; j < s.length(); ++j)
            for (s[j] = 'a'; containsPalindrome(s, j); ++s[j])
                ;
        return s;
    }
};
```

1787. 2664.cpp

Path: LeetCode-main/solutions/2664. The Knight's Tour/2664.cpp

```
class Solution {
public:
    vector<vector<int>> tourOfKnight(int m, int n, int r, int c) {
        vector<vector<int>> ans(m, vector<int>(n, -1));
        dfs(m, n, r, c, 0, ans);
        return ans;
    }

private:
    static constexpr int kDirs[8][2] = {{1, 2}, {2, 1}, {2, -1}, {1, -2},
                                           {-1, -2}, {-2, -1}, {-2, 1}, {-1, 2}};

    bool dfs(int m, int n, int i, int j, int step, vector<vector<int>>& ans) {
        if (step == m * n)
            return true;
        if (i < 0 || i >= m || j < 0 || j >= n)
            return false;
        if (ans[i][j] != -1)
            return false;
        ans[i][j] = step;
        for (const auto& [dx, dy] : kDirs)
            if (dfs(m, n, i + dx, j + dy, step + 1, ans))
                return true;
        ans[i][j] = -1;
        return false;
    }
};
```

1788. 267.cpp

Path: LeetCode-main/solutions/267. Palindrome Permutation II/267.cpp

```
class Solution {
public:
    vector<string> generatePalindromes(string s) {
        int odd = 0;
        unordered_map<char, int> count;

        for (const char c : s)
            ++count[c];

        // Count odd ones.
        for (const auto& [_, value] : count)
            if (value % 2 == 1)
                ++odd;

        // Can't form any palindrome.
        if (odd > 1)
            return {};

        vector<string> ans;
        vector<char> candidates;
        string mid;

        // Get the mid and the candidates characters.
        for (const auto& [key, value] : count) {
            if (value % 2 == 1)
                mid += key;
            for (int i = 0; i < value / 2; ++i)
                candidates.push_back(key);
        }

        // Backtrack to generate the ans strings.
        dfs(candidates, mid, vector<bool>(candidates.size()), "", ans);
        return ans;
    }

private:
    // Generates all the unique palindromes from the candidates.
    void dfs(const vector<char>& candidates, const string& mid,
            vector<bool>&& used, string&& path, vector<string>& ans) {
        if (path.length() == candidates.size()) {
            string secondHalf = path;
            ranges::reverse(secondHalf);
            ans.push_back(path + mid + secondHalf);
            return;
        }

        for (int i = 0; i < candidates.size(); ++i) {
            if (used[i])
                continue;
            if (i > 0 && candidates[i] == candidates[i - 1] && !used[i - 1])
                continue;
            used[i] = true;
            path.push_back(candidates[i]);
            dfs(candidates, mid, std::move(used), std::move(path), ans);
            path.pop_back();
            used[i] = false;
        }
    }
};
```


1789. 2670.cpp

Path: LeetCode-main/solutions/2670. Find the Distinct Difference Array/2670.cpp

```
class Solution {
public:
    vector<int> distinctDifferenceArray(vector<int>& nums) {
        constexpr int kMax = 50;
        vector<int> ans;
        vector<int> prefixCount(kMax + 1);
        vector<int> suffixCount(kMax + 1);
        int distinctPrefix = 0;
        int distinctSuffix = 0;

        for (const int num : nums)
            if (++suffixCount[num] == 1)
                ++distinctSuffix;

        for (const int num : nums) {
            if (++prefixCount[num] == 1)
                ++distinctPrefix;
            if (--suffixCount[num] == 0)
                --distinctSuffix;
            ans.push_back(distinctPrefix - distinctSuffix);
        }

        return ans;
    }
};
```

1790. 2671.cpp

Path: LeetCode-main/solutions/2671. Frequency Tracker/2671.cpp

```
class FrequencyTracker {
public:
    void add(int number) {
        if (count[number] > 0)
            --freqCount[count[number]];
        ++count[number];
        ++freqCount[count[number]];
    }

    void deleteOne(int number) {
        if (count[number] == 0)
            return;
        --freqCount[count[number]];
        --count[number];
        ++freqCount[count[number]];
    }

    bool hasFrequency(int frequency) {
        return freqCount[frequency] > 0;
    }

private:
    unordered_map<int, int> count;
    unordered_map<int, int> freqCount;
};
```

1791. 2672.cpp

Path: LeetCode-main/solutions/2672. Number of Adjacent Elements With the Same Color/2672.cpp

```
class Solution {
public:
    vector<int> colorTheArray(int n, vector<vector<int>>& queries) {
        vector<int> ans;
        vector<int> arr(n);
        int sameColors = 0;

        for (const vector<int>& query : queries) {
            const int i = query[0];
            const int color = query[1];
            if (i + 1 < n) {
                if (arr[i + 1] > 0 && arr[i + 1] == arr[i])
                    --sameColors;
                if (arr[i + 1] == color)
                    ++sameColors;
            }
            if (i > 0) {
                if (arr[i - 1] > 0 && arr[i - 1] == arr[i])
                    --sameColors;
                if (arr[i - 1] == color)
                    ++sameColors;
            }
            arr[i] = color;
            ans.push_back(sameColors);
        }

        return ans;
    }
};
```

1792. 2673.cpp

Path: LeetCode-main/solutions/2673. Make Costs of Paths Equal in a Binary Tree/2673.cpp

```
class Solution {
public:
    int minIncrements(int n, vector<int>& cost) {
        int ans = 0;

        for (int i = n / 2 - 1; i >= 0; --i) {
            const int l = i * 2 + 1;
            const int r = i * 2 + 2;
            ans += abs(cost[l] - cost[r]);
            // Record the information in the parent from the children. So, there's
            // need to actually update the values in the children.
            cost[i] += max(cost[l], cost[r]);
        }

        return ans;
    }
};
```

1793. 2674.cpp

Path: LeetCode-main/solutions/2674. Split a Circular Linked List/2674.cpp

```
class Solution {
public:
    vector<ListNode*> splitCircularLinkedList(ListNode* list) {
        ListNode* slow = list;
        ListNode* fast = list;

        // Point `slow` to the last node in the first half.
        while (fast->next != list && fast->next->next != list) {
            slow = slow->next;
            fast = fast->next->next;
        }

        // Circle back the second half.
        ListNode* secondHead = slow->next;
        if (fast->next == list)
            fast->next = secondHead;
        else
            fast->next->next = secondHead;

        // Circle back the first half.
        slow->next = list;

        return {list, secondHead};
    }
};
```

1794. 2678.cpp

Path: LeetCode-main/solutions/2678. Number of Senior Citizens/2678.cpp

```
class Solution {
public:
    int countSeniors(vector<string>& details) {
        return ranges::count_if(details, [](const string& detail) {
            return stoi(detail.substr(11, 2)) > 60;
        });
    }
};
```

1795. 2679.cpp

Path: LeetCode-main/solutions/2679. Sum in a Matrix/2679.cpp

```
class Solution {
public:
    int matrixSum(vector<vector<int>>& nums) {
        int ans = 0;

        for (vector<int>& row : nums)
            ranges::sort(row);

        for (int j = 0; j < nums[0].size(); ++j) {
            int mx = 0;
            for (int i = 0; i < nums.size(); ++i)
                mx = max(mx, nums[i][j]);
            ans += mx;
        }

        return ans;
    }
};
```

1796. 268.cpp

Path: LeetCode-main/solutions/268. Missing Number/268.cpp

```
class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int ans = nums.size();

        for (int i = 0; i < nums.size(); ++i)
            ans ^= i ^ nums[i];

        return ans;
    }
};
```


1797. 2680.cpp

Path: LeetCode-main/solutions/2680. Maximum OR/2680.cpp

```
class Solution {
public:
    long long maximumOr(vector<int>& nums, int k) {
        const int n = nums.size();
        long ans = 0;
        // prefix[i] := nums[0] | nums[1] | ... | nums[i - 1]
        vector<long> prefix(n);
        // suffix[i] := nums[i + 1] | nums[i + 2] | ... | nums[n - 1]
        vector<long> suffix(n);

        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i - 1] | nums[i - 1];

        for (int i = n - 2; i >= 0; --i)
            suffix[i] = suffix[i + 1] | nums[i + 1];

        // For each num, greedily shift it left by k bits.
        for (int i = 0; i < n; ++i)
            ans = max(ans, prefix[i] | static_cast<long>(nums[i]) << k | suffix[i]);

        return ans;
    }
};
```

1798. 2681.cpp

Path: LeetCode-main/solutions/2681. Power of Heroes/2681.cpp

```
class Solution {
public:
    int sumOfPower(vector<int>& nums) {
        constexpr int kMod = 1'000'000'007;
        long ans = 0;
        long sum = 0;

        ranges::sort(nums);

        for (const int num : nums) {
            ans = (ans + (num + sum) * num % kMod * num % kMod) % kMod;
            sum = (sum * 2 + num) % kMod;
        }

        return ans;
    }
};
```

1799. 2682.cpp

Path: LeetCode-main/solutions/2682. Find the Losers of the Circular Game/2682.cpp

```
class Solution {
public:
    vector<int> circularGameLosers(int n, int k) {
        vector<int> ans;
        vector<bool> seen(n);

        for (int friendIndex = 0, turn = 1; !seen[friendIndex];) {
            seen[friendIndex] = true;
            friendIndex += turn++ * k;
            friendIndex %= n;
        }

        for (int friendIndex = 0; friendIndex < n; ++friendIndex)
            if (!seen[friendIndex])
                ans.push_back(friendIndex + 1);

        return ans;
    }
};
```

1800. 2683.cpp

Path: LeetCode-main/solutions/2683. Neighboring Bitwise XOR/2683.cpp

```
class Solution {
public:
    bool doesValidArrayExist(vector<int>& derived) {
        //      original = [0[0], 0[1], ..., 0[n - 1]]
        //      derived = [0[0]^0[1], 0[1]^0[2], ..., 0[n - 1]^0[0]]
        // XOR(derived) = 0
        return accumulate(derived.begin(), derived.end(), 0, bit_xor<>()) == 0;
    }
};
```

1801. 2684.cpp

Path: LeetCode-main/solutions/2684. Maximum Number of Moves in a Grid/2684.cpp

```
class Solution {
public:
    int maxMoves(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        int ans = 0;
        // dp[i][j] := the maximum number of moves you can perform from (i, j)
        vector<vector<int>> dp(m, vector<int>(n));

        for (int j = n - 2; j >= 0; --j)
            for (int i = 0; i < m; ++i) {
                if (grid[i][j + 1] > grid[i][j])
                    dp[i][j] = 1 + dp[i][j + 1];
                if (i > 0 && grid[i - 1][j + 1] > grid[i][j])
                    dp[i][j] = max(dp[i][j], 1 + dp[i - 1][j + 1]);
                if (i + 1 < m && grid[i + 1][j + 1] > grid[i][j])
                    dp[i][j] = max(dp[i][j], 1 + dp[i + 1][j + 1]);
            }

        for (int i = 0; i < m; ++i)
            ans = max(ans, dp[i][0]);

        return ans;
    }
};
```

1802. 2685.cpp

Path: LeetCode-main/solutions/2685. Count the Number of Complete Components/2685.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n), nodeCount(n, 1), edgeCount(n) {
        iota(id.begin(), id.end(), 0);
    }

    void unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        ++edgeCount[i];
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
            edgeCount[j] += edgeCount[i];
            nodeCount[j] += nodeCount[i];
        } else if (rank[i] > rank[j]) {
            id[j] = i;
            edgeCount[i] += edgeCount[j];
            nodeCount[i] += nodeCount[j];
        } else {
            id[i] = j;
            edgeCount[j] += edgeCount[i];
            nodeCount[j] += nodeCount[i];
            ++rank[j];
        }
    }

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }

    bool isComplete(int u) {
        return nodeCount[u] * (nodeCount[u] - 1) / 2 == edgeCount[u];
    }

private:
    vector<int> id;
    vector<int> rank;
    vector<int> nodeCount;
    vector<int> edgeCount;
};

class Solution {
public:
    int countCompleteComponents(int n, vector<vector<int>>& edges) {
        int ans = 0;
        UnionFind uf(n);
        unordered_set<int> parents;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            uf.unionByRank(u, v);
        }

        for (int i = 0; i < n; ++i) {
            const int parent = uf.find(i);
            if (parents.insert(parent).second && uf.isComplete(parent))
                ++ans;
        }

        return ans;
    }
};
```

1803. 2689-2.cpp

Path: LeetCode-main/solutions/2689. Extract Kth Character From The Rope Tree/2689-2.cpp

```
class Solution {
public:
    char getKthCharacter(RopeTreeNode* root, int k) {
        while (root->len > 0) {
            const int leftLen = root->left == nullptr
                ? 0
                : max(root->left->len,
                    static_cast<int>(root->left->val.length()));

            if (leftLen >= k) {
                root = root->left;
            } else {
                root = root->right;
                k -= leftLen;
            }
        }
        return root->val[k - 1];
    }
};
```

1804. 2689.cpp

Path: LeetCode-main/solutions/2689. Extract Kth Character From The Rope Tree/2689.cpp

```
class Solution {
public:
    char getKthCharacter(RopeTreeNode* root, int k) {
        if (root->len == 0)
            return root->val[k - 1];
        const int leftLen =
            root->left == nullptr
                ? 0
                : max(root->left->len, static_cast<int>(root->left->val.length()));
        return leftLen >= k ? getKthCharacter(root->left, k)
                           : getKthCharacter(root->right, k - leftLen);
    }
};
```


1805. 269.cpp

Path: LeetCode-main/solutions/269. Alien Dictionary/269.cpp

```
class Solution {
public:
    string alienOrder(vector<string>& words) {
        unordered_map<char, unordered_set<char>> graph;
        vector<int> inDegrees(26);
        buildGraph(graph, words, inDegrees);
        return topology(graph, inDegrees);
    }

private:
    void buildGraph(unordered_map<char, unordered_set<char>>& graph,
                   const vector<string>& words, vector<int>& inDegrees) {
        // Create a node for each character in each word.
        for (const string& word : words)
            for (const char c : word)
                if (!graph.contains(c))
                    graph[c] = unordered_set<char>();

        for (int i = 1; i < words.size(); ++i) {
            const string& first = words[i - 1];
            const string& second = words[i];
            const int length = min(first.length(), second.length());
            for (int j = 0; j < length; ++j) {
                const char u = first[j];
                const char v = second[j];
                if (u != v) {
                    if (!graph[u].contains(v)) {
                        graph[u].insert(v);
                        ++inDegrees[v - 'a'];
                    }
                    break; // The order of characters after this are meaningless.
                }
                // e.g. first = "ab", second = "a" -> invalid
                if (j == length - 1 && first.length() > second.length()) {
                    graph.clear();
                    return;
                }
            }
        }
    }

    string topology(unordered_map<char, unordered_set<char>>& graph,
                   vector<int>& inDegrees) {
        string s;
        queue<char> q;

        for (const auto& [c, _] : graph)
            if (inDegrees[c - 'a'] == 0)
                q.push(c);

        while (!q.empty()) {
            const char u = q.front();
            q.pop();
            s += u;
            for (const char v : graph[u])
                if (--inDegrees[v - 'a'] == 0)
                    q.push(v);
        }

        // e.g. words = ["z", "x", "y", "x"]
        return s.length() == graph.size() ? s : "";
    }
};
```

1806. 2696.cpp

Path: LeetCode-main/solutions/2696. Minimum String Length After Removing Substrings/2696.cpp

```
class Solution {
public:
    int minLength(string s) {
        stack<char> stack;

        for (const char c : s)
            if (c == 'B' && match(stack, 'A'))
                stack.pop();
            else if (c == 'D' && match(stack, 'C'))
                stack.pop();
            else
                stack.push(c);

        return stack.size();
    }

private:
    bool match(const stack<char>& stack, int c) {
        return !stack.empty() && stack.top() == c;
    }
};
```

1807. 2697.cpp

Path: LeetCode-main/solutions/2697. Lexicographically Smallest Palindrome/2697.cpp

```
class Solution {
public:
    string makeSmallestPalindrome(string s) {
        for (int i = 0, j = s.length() - 1; i < j; ++i, --j) {
            const int minChar = min(s[i], s[j]);
            s[i] = minChar;
            s[j] = minChar;
        }
        return s;
    }
};
```

1808. 2698.cpp

Path: LeetCode-main/solutions/2698. Find the Punishment Number of an Integer/2698.cpp

```
class Solution {
public:
    int punishmentNumber(int n) {
        int ans = 0;
        for (int i = 1; i <= n; ++i)
            if (isPossible(0, 0, to_string(i * i), 0, i))
                ans += i * i;
        return ans;
    }

private:
    // Returns true if the sum of any split of `numChars` equals to the target.
    bool isPossible(int accumulate, int running, const string& numChars, int s,
                    int target) {
        if (s == numChars.length())
            return target == accumulate + running;
        const int d = numChars[s] - '0';
        return isPossible(accumulate, running * 10 + d, numChars, s + 1, target) ||
               isPossible(accumulate + running, d, numChars, s + 1, target);
    }
};
```

1809. 2699.cpp

Path: LeetCode-main/solutions/2699. Modify Graph Edge Weights/2699.cpp

```
class Solution {
public:
    vector<vector<int>> modifiedGraphEdges(int n, vector<vector<int>>& edges,
                                         int source, int destination,
                                         int target) {

        constexpr int kMax = 2'000'000'000;
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int w = edge[2];
            if (w == -1)
                continue;
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        int distToDestination = dijkstra(graph, source, destination);
        if (distToDestination < target)
            return {};
        if (distToDestination == target) {
            // Change the weights of negative edges to an impossible value.
            for (vector<int>& edge : edges)
                if (edge[2] == -1)
                    edge[2] = kMax;
            return edges;
        }

        for (int i = 0; i < edges.size(); ++i) {
            const int u = edges[i][0];
            const int v = edges[i][1];
            int& w = edges[i][2];
            if (w != -1)
                continue;
            w = 1;
            graph[u].emplace_back(v, 1);
            graph[v].emplace_back(u, 1);
            distToDestination = dijkstra(graph, source, destination);
            if (distToDestination <= target) {
                w += target - distToDestination;
                // Change the weights of negative edges to an impossible value.
                for (int j = i + 1; j < edges.size(); ++j)
                    if (edges[j][2] == -1)
                        edges[j][2] = kMax;
                return edges;
            }
        }

        return {};
    }

private:
    int dijkstra(const vector<vector<pair<int, int>>& graph, int src, int dst) {
        vector<int> dist(graph.size(), INT_MAX);

        dist[src] = 0;
        using P = pair<int, int>; // (d, u)
        priority_queue<P, vector<P>, greater<>> minHeap;
        minHeap.emplace(dist[src], src);

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (d > dist[u])
                continue;
            for (const auto& [v, w] : graph[u])
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.emplace(dist[v], v);
                }
        }

        return dist[dst];
    }
};
```

1810. 27.cpp

Path: LeetCode-main/solutions/27. Remove Element/27.cpp

```
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int i = 0;

        for (const int num : nums)
            if (num != val)
                nums[i++] = num;

        return i;
    }
};
```

1811. 270.cpp

Path: LeetCode-main/solutions/270. Closest Binary Search Tree Value/270.cpp

```
class Solution {
public:
    int closestValue(TreeNode* root, double target) {
        // If target < root->val, search the left subtree.
        if (target < root->val && root->left) {
            const int left = closestValue(root->left, target);
            if (abs(left - target) <= abs(root->val - target))
                return left;
        }

        // If target > root->val, search the right subtree.
        if (target > root->val && root->right) {
            const int right = closestValue(root->right, target);
            if (abs(right - target) < abs(root->val - target))
                return right;
        }

        return root->val;
    }
};
```

1812. 2702.cpp

Path: LeetCode-main/solutions/2702. Minimum Operations to Make Numbers Non-positive/2702.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums, int x, int y) {
        int l = 0;
        int r = ranges::max(nums);

        while (l < r) {
            const int m = (l + r) / 2;
            if (isPossible(nums, x, y, m))
                r = m;
            else
                l = m + 1;
        }

        return l;
    }

private:
    // Returns true if it's possible to make all `nums` <= 0 using m operations.
    bool isPossible(const vector<int>& nums, int x, int y, int m) {
        long additionalOps = 0;
        // If we want m operations, first decrease all the numbers by y * m. Then,
        // we have m operations to select indices to decrease them by x - y.
        for (const int num : nums)
            additionalOps +=
                max(0L, (num - static_cast<long>(y) * m + x - y - 1) / (x - y));
        return additionalOps <= m;
    }
};
```


1813. 2706.cpp

Path: LeetCode-main/solutions/2706. Buy Two Chocolates/2706.cpp

```
class Solution {
public:
    int buyChoco(vector<int>& prices, int money) {
        int min1 = INT_MAX;
        int min2 = INT_MAX;

        for (const int price : prices)
            if (price <= min1) {
                min2 = min1;
                min1 = price;
            } else if (price < min2) {
                min2 = price;
            }

        const int minCost = min1 + min2;
        return minCost > money ? money : money - minCost;
    }
};
```

1814. 2707.cpp

Path: LeetCode-main/solutions/2707. Extra Characters in a String/2707.cpp

```
class Solution {
public:
    // Similar to 139. Word Break
    int minExtraChar(string s, vector<string>& dictionary) {
        const int n = s.length();
        const unordered_set<string> dictionarySet{dictionary.begin(),
                                                    dictionary.end()};

        // dp[i] := the minimum extra letters if breaking up s[0..i) optimally
        vector<int> dp(n + 1, n);
        dp[0] = 0;

        for (int i = 1; i <= n; ++i)
            for (int j = 0; j < i; ++j)
                // s[j..i) is in `dictionarySet`.
                if (dictionarySet.contains(s.substr(j, i - j)))
                    dp[i] = min(dp[i], dp[j]);
                // s[j..i) are extra letters.
                else
                    dp[i] = min(dp[i], dp[j] + i - j);

        return dp[n];
    }
};
```

1815. 2708.cpp

Path: LeetCode-main/solutions/2708. Maximum Strength of a Group/2708.cpp

```
class Solution {
public:
    long long maxStrength(vector<int>& nums) {
        long posProd = 1;
        long negProd = 1;
        int maxNeg = INT_MIN;
        int negCount = 0;
        bool hasPos = false;
        bool hasZero = false;

        for (const int num : nums)
            if (num > 0) {
                posProd *= num;
                hasPos = true;
            } else if (num < 0) {
                negProd *= num;
                maxNeg = max(maxNeg, num);
                ++negCount;
            } else { // num == 0
                hasZero = true;
            }

        if (negCount == 0 && !hasPos)
            return 0;
        if (negCount % 2 == 0)
            return negProd * posProd;
        if (negCount >= 3)
            return negProd / maxNeg * posProd;
        if (hasPos)
            return posProd;
        if (hasZero)
            return 0;
        return maxNeg;
    }
};
```

1816. 2709.cpp

Path: LeetCode-main/solutions/2709. Greatest Common Divisor Traversal/2709.cpp

```
class UnionFind {
public:
    UnionFind(int n) : id(n), sz(n, 1) {
        iota(id.begin(), id.end(), 0);
    }

    void unionBySize(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return;
        if (sz[i] < sz[j]) {
            sz[j] += sz[i];
            id[i] = j;
        } else {
            sz[i] += sz[j];
            id[j] = i;
        }
    }

    int getSize(int i) {
        return sz[i];
    }

private:
    vector<int> id;
    vector<int> sz;

    int find(int u) {
        return id[u] == u ? u : id[u] = find(id[u]);
    }
};

class Solution {
public:
    bool canTraverseAllPairs(vector<int>& nums) {
        const int n = nums.size();
        const int mx = ranges::max(nums);
        const vector<int> minPrimeFactors = sieveEratosthenes(mx + 1);
        unordered_map<int, int> primeToFirstIndex;
        UnionFind uf(n);

        for (int i = 0; i < n; ++i)
            for (const int primeFactor : getPrimeFactors(nums[i], minPrimeFactors))
                // `primeFactor` already appeared in the previous indices.
                if (const auto it = primeToFirstIndex.find(primeFactor);
                    it != primeToFirstIndex.cend())
                    uf.unionBySize(it->second, i);
                else
                    primeToFirstIndex[primeFactor] = i;

        for (int i = 0; i < n; ++i)
            if (uf.getSize(i) == n)
                return true;

        return false;
    }

private:
    // Gets the minimum prime factor of i, where 1 < i <= n.
    vector<int> sieveEratosthenes(int n) {
        vector<int> minPrimeFactors(n + 1);
        iota(minPrimeFactors.begin() + 2, minPrimeFactors.end(), 2);
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    vector<int> getPrimeFactors(int num, const vector<int>& minPrimeFactors) {
        vector<int> primeFactors;
        while (num > 1) {
            const int divisor = minPrimeFactors[num];
            primeFactors.push_back(divisor);
            while (num % divisor == 0)
                num /= divisor;
        }
        return primeFactors;
    }
}
```


1817. 271.cpp

Path: LeetCode-main/solutions/271. Encode and Decode Strings/271.cpp

```
class Codec {
public:
    // Encodes a list of strings to a single string.
    string encode(vector<string>& strs) {
        string encoded;

        for (const string& s : strs)
            encoded += to_string(s.length()) + '/' + s;

        return encoded;
    }

    // Decodes a single string to a list of strings.
    vector<string> decode(string s) {
        vector<string> decoded;

        for (int i = 0; i < s.length(); i) {
            const int slash = s.find('/', i);
            const int length = stoi(s.substr(i, slash - i));
            i = slash + length + 1;
            decoded.push_back(s.substr(slash + 1, i - slash - 1));
        }

        return decoded;
    }
};
```

1818. 2710.cpp

Path: LeetCode-main/solutions/2710. Remove Trailing Zeros From a String/2710.cpp

```
class Solution {  
public:  
    string removeTrailingZeros(string num) {  
        return regex_replace(num, regex("0+$"), "");  
    }  
};
```

1819. 2711.cpp

Path: LeetCode-main/solutions/2711. Difference of Number of Distinct Values on Diagonals/2711.cpp

```
class Solution {
public:
    vector<vector<int>> differenceOfDistinctValues(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<int>> ans(m, vector<int>(n));

        for (int i = 0; i < m; ++i)
            fillInDiagonal(grid, i, 0, ans);

        for (int j = 1; j < n; ++j)
            fillInDiagonal(grid, 0, j, ans);

        return ans;
    }

private:
    void fillInDiagonal(const vector<vector<int>>& grid, int i, int j,
                       vector<vector<int>>& ans) {
        unordered_set<int> topLeft;
        unordered_set<int> bottomRight;

        // Fill in the diagonal from the top-left to the bottom-right.
        while (i < grid.size() && j < grid[0].size()) {
            ans[i][j] = topLeft.size();
            // Post-addition, so this information can be utilized in subsequent cells.
            topLeft.insert(grid[i++][j++]);
        }

        --i;
        --j;

        // Fill in the diagonal from the bottom-right to the top-left.
        while (i >= 0 && j >= 0) {
            ans[i][j] = abs(ans[i][j] - static_cast<int>(bottomRight.size()));
            // Post-addition, so this information can be utilized in subsequent cells.
            bottomRight.insert(grid[i--][j--]);
        }
    }
};
```


1820. 2712.cpp

Path: LeetCode-main/solutions/2712. Minimum Cost to Make All Characters Equal/2712.cpp

```
class Solution {
public:
    long long minimumCost(string s) {
        const int n = s.length();
        long ans = 0;

        for (int i = 1; i < n; ++i)
            if (s[i] != s[i - 1])
                // Invert s[0..i - 1] or s[i..n - 1].
                ans += min(i, n - i);

        return ans;
    }
};
```

1821. 2713.cpp

Path: LeetCode-main/solutions/2713. Maximum Strictly Increasing Cells in a Matrix/2713.cpp

```
class Solution {
public:
    int maxIncreasingCells(vector<vector<int>>& mat) {
        const int m = mat.size();
        const int n = mat[0].size();
        // rows[i] := the maximum path length for the i-th row
        vector<int> rows(m);
        // cols[j] := the maximum path length for the j-th column
        vector<int> cols(n);
        unordered_map<int, vector<pair<int, int>>> valToIndices;
        // maxPathLength[i][j] := the maximum path length from mat[i][j]
        vector<vector<int>> maxPathLength(m, vector<int>(n));
        // Sort all the unique values in the matrix in non-increasing order.
        set<int, greater<>> decreasingSet;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                valToIndices[mat[i][j]].emplace_back(i, j);
                decreasingSet.insert(mat[i][j]);
            }

        for (const int val : decreasingSet) {
            for (const auto& [i, j] : valToIndices[val])
                maxPathLength[i][j] = max(rows[i], cols[j]) + 1;
            for (const auto& [i, j] : valToIndices[val]) {
                rows[i] = max(rows[i], maxPathLength[i][j]);
                cols[j] = max(cols[j], maxPathLength[i][j]);
            }
        }

        return max(ranges::max(rows), ranges::max(cols));
    }
};
```

1822. 2714.cpp

Path: LeetCode-main/solutions/2714. Find Shortest Path with K Hops/2714.cpp

```
class Solution {
public:
    // Similar to 787. Cheapest Flights Within K Stops
    int shortestPathWithHops(int n, vector<vector<int>>& edges, int s, int d,
                           int k) {
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int w = edge[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        return dijkstra(graph, s, d, k);
    }

private:
    int dijkstra(const vector<vector<pair<int, int>>>& graph, int src, int dst,
                int k) {
        vector<vector<int>> dist(graph.size(), vector<int>(k + 1, INT_MAX));

        dist[src][k] = 0;
        using T = tuple<int, int, int>; // (d, u, hops)
        priority_queue<T, vector<T>, greater<>> minHeap;
        minHeap.emplace(dist[src][k], src, k);

        while (!minHeap.empty()) {
            const auto [d, u, hops] = minHeap.top();
            minHeap.pop();
            if (u == dst)
                return d;
            if (dist[u][hops] > d)
                continue;
            for (const auto& [v, w] : graph[u]) {
                // Go from u -> v with w cost.
                if (d + w < dist[v][hops]) {
                    dist[v][hops] = d + w;
                    minHeap.emplace(dist[v][hops], v, hops);
                }
                // Hop from u -> v with 0 cost.
                if (hops > 0 && d < dist[v][hops - 1]) {
                    dist[v][hops - 1] = d;
                    minHeap.emplace(dist[v][hops - 1], v, hops - 1);
                }
            }
        }

        throw;
    }
};
```

1823. 2716.cpp

Path: LeetCode-main/solutions/2716. Minimize String Length/2716.cpp

```
class Solution {  
public:  
    int minimizedStringLength(string s) {  
        return unordered_set(s.begin(), s.end()).size();  
    }  
};
```

1824. 2717.cpp

Path: LeetCode-main/solutions/2717. Semi-Ordered Permutation/2717.cpp

```
class Solution {
public:
    int semiOrderedPermutation(vector<int>& nums) {
        const int n = nums.size();
        const int index1 = find(nums.begin(), nums.end(), 1) - nums.begin();
        const int indexN = find(nums.begin(), nums.end(), n) - nums.begin();
        return index1 + (n - 1 - indexN) - (index1 > indexN ? 1 : 0);
    }
};
```

1825. 2718.cpp

Path: LeetCode-main/solutions/2718. Sum of Matrix After Queries/2718.cpp

```
class Solution {
public:
    long long matrixSumQueries(int n, vector<vector<int>>& queries) {
        long ans = 0;
        // seen[0] := row, seen[1] := col
        vector<vector<bool>> seen(2, vector<bool>(n));
        // notSet[0] = row, notSet[1] := col
        vector<int> notSet(2, n);

        // Later queries dominate.
        for (int i = queries.size() - 1; i >= 0; --i) {
            const int type = queries[i][0];
            const int index = queries[i][1];
            const int val = queries[i][2];
            if (!seen[type][index]) {
                ans += val * notSet[type ^ 1];
                seen[type][index] = true;
                --notSet[type];
            }
        }
        return ans;
    }
};
```

1826. 2719.cpp

Path: LeetCode-main/solutions/2719. Count of Integers/2719.cpp

```
class Solution {
public:
    int count(string num1, string num2, int min_sum, int max_sum) {
        const string num1WithLeadingZeros =
            string(num2.length() - num1.length(), '0') + num1;
        vector<vector<vector<vector<int>>>>> mem(
            num2.length(),
            vector<vector<vector<int>>>>(
                max_sum + 1, vector<vector<int>>>(2, vector<int>(2, -1))));
        return (count(num1WithLeadingZeros, num2, 0, max_sum, true, true, mem) -
            count(num1WithLeadingZeros, num2, 0, min_sum - 1, true, true, mem) +
            kMod) %
            kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of valid integers, considering the i-th digit, where
    // `sum` is the maximum digit sum, `tight1` indicates if the current digit
    // is tightly bound for `num1` and `tight2` indicates if the current digit
    // is tightly bound for `num2`
    int count(const string& num1, const string& num2, int i, int sum, bool tight1,
        bool tight2, vector<vector<vector<vector<int>>>>& mem) {
        if (sum < 0)
            return 0;
        if (i == num2.length())
            return 1;
        if (mem[i][sum][tight1][tight2] != -1)
            return mem[i][sum][tight1][tight2];

        int res = 0;

        const int minDigit = tight1 ? num1[i] - '0' : 0;
        const int maxDigit = tight2 ? num2[i] - '0' : 9;
        for (int d = minDigit; d <= maxDigit; ++d) {
            const bool nextTight1 = tight1 && (d == minDigit);
            const bool nextTight2 = tight2 && (d == maxDigit);
            res += count(num1, num2, i + 1, sum - d, nextTight1, nextTight2, mem);
            res %= kMod;
        }

        return mem[i][sum][tight1][tight2] = res;
    }
};
```

1827. 272.cpp

Path: LeetCode-main/solutions/272. Closest Binary Search Tree Value II/272.cpp

```
class Solution {
public:
    vector<int> closestKValues(TreeNode* root, double target, int k) {
        deque<int> dq;

        inorder(root, dq);

        while (dq.size() > k)
            if (abs(dq.front() - target) > abs(dq.back() - target))
                dq.pop_front();
            else
                dq.pop_back();

        return {dq.begin(), dq.end()};
    }

private:
    void inorder(TreeNode* root, deque<int>& dq) {
        if (root == nullptr)
            return;

        inorder(root->left, dq);
        dq.push_back(root->val);
        inorder(root->right, dq);
    }
};
```


1828. 2728.cpp

Path: LeetCode-main/solutions/2728. Count Houses in a Circular Street/2728.cpp

```
/**
 * Definition for a street.
 * class Street {
 * public:
 *     Street(vector<int> doors);
 *     void openDoor();
 *     void closeDoor();
 *     bool isDoorOpen();
 *     void moveRight();
 *     void moveLeft();
 * };
 */

class Solution {
public:
    int houseCount(Street* street, int k) {
        for (int i = 0; i < k; ++i) {
            if (street->isDoorOpen())
                street->closeDoor();
            street->moveRight();
        }

        for (int ans = 0;; ++ans) {
            if (street->isDoorOpen())
                return ans;
            street->openDoor();
            street->moveRight();
        }

        throw;
    }
};
```

1829. 2729.cpp

Path: LeetCode-main/solutions/2729. Check if The Number is Fascinating/2729.cpp

```
class Solution {
public:
    bool isFascinating(int n) {
        string s = to_string(n) + to_string(2 * n) + to_string(3 * n);
        ranges::sort(s);
        return s == "123456789";
    }
};
```

1830. 273.cpp

Path: LeetCode-main/solutions/273. Integer to English Words/273.cpp

```
class Solution {
public:
    string numberToWords(int num) {
        if (num == 0)
            return "Zero";
        return helper(num);
    }

private:
    const vector<string> belowTwenty{
        "", "One", "Two", "Three", "Four",
        "Five", "Six", "Seven", "Eight", "Nine",
        "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen",
        "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"};
    const vector<string> tens{"", "", "Twenty", "Thirty", "Forty",
        "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"};

    string helper(int num) {
        string s;

        if (num < 20)
            s = belowTwenty.at(num);
        else if (num < 100)
            s = tens.at(num / 10) + " " + belowTwenty.at(num % 10);
        else if (num < 1000)
            s = helper(num / 100) + " Hundred " + helper(num % 100);
        else if (num < 1000000)
            s = helper(num / 1000) + " Thousand " + helper(num % 1000);
        else if (num < 1000000000)
            s = helper(num / 1000000) + " Million " + helper(num % 1000000);
        else
            s = helper(num / 1000000000) + " Billion " + helper(num % 1000000000);

        trim(s);
        return s;
    }

    void trim(string& s) {
        s.erase(0, s.find_first_not_of(' '));
        s.erase(s.find_last_not_of(' ') + 1);
    }
};
```

1831. 2730.cpp

Path: LeetCode-main/solutions/2730. Find the Longest Semi-Repetitive Substring/2730.cpp

```
class Solution {
public:
    int longestSemiRepetitiveSubstring(string s) {
        int ans = 1;
        int prevStart = 0;
        int start = 0;

        for (int i = 1; i < s.length(); ++i) {
            if (s[i] == s[i - 1]) {
                if (prevStart > 0)
                    start = prevStart;
                prevStart = i;
            }
            ans = max(ans, i - start + 1);
        }

        return ans;
    }
};
```

1832. 2731.cpp

Path: LeetCode-main/solutions/2731. Movement of Robots/2731.cpp

```
class Solution {
public:
    int sumDistance(vector<int>& nums, string s, int d) {
        constexpr int kMod = 1'000'000'007;
        const int n = nums.size();
        int ans = 0;
        int prefix = 0;
        vector<int> pos;

        for (int i = 0; i < nums.size(); ++i)
            if (s[i] == 'L')
                pos.push_back(nums[i] - d);
            else
                pos.push_back(nums[i] + d);

        ranges::sort(pos);

        for (int i = 0; i < n; ++i) {
            ans =
                ((ans + static_cast<long>(i) * pos[i] - prefix) % kMod + kMod) % kMod;
            prefix = ((0L + prefix + pos[i]) % kMod + kMod) % kMod;
        }

        return ans;
    }
};
```

1833. 2732.cpp

Path: LeetCode-main/solutions/2732. Find a Good Subset of the Matrix/2732.cpp

```
class Solution {
public:
    vector<int> goodSubsetofBinaryMatrix(vector<vector<int>>& grid) {
        constexpr int kMaxBit = 30;
        unordered_map<int, int> maskToIndex;

        for (int i = 0; i < grid.size(); ++i) {
            const int mask = getMask(grid[i]);
            if (mask == 0)
                return {i};
            for (int prevMask = 1; prevMask < kMaxBit; ++prevMask)
                if ((mask & prevMask) == 0 && maskToIndex.contains(prevMask))
                    return {maskToIndex[prevMask], i};
            maskToIndex[mask] = i;
        }

        return {};
    }

private:
    int getMask(const vector<int>& row) {
        int mask = 0;
        for (int i = 0; i < row.size(); ++i)
            if (row[i] == 1)
                mask |= 1 << i;
        return mask;
    }
};
```

1834. 2733.cpp

Path: LeetCode-main/solutions/2733. Neither Minimum nor Maximum/2733.cpp

```
class Solution {
public:
    int findNonMinOrMax(vector<int>& nums) {
        if (nums.size() < 3)
            return -1;
        sort(nums.begin(), nums.begin() + 3);
        return nums[1];
    }
};
```

1835. 2734.cpp

Path: LeetCode-main/solutions/2734. Lexicographically Smallest String After Substring Operation/2734.cpp

```
class Solution {
public:
    string smallestString(string s) {
        const int n = s.length();
        int i = 0;

        while (i < n && s[i] == 'a')
            ++i;
        if (i == n) {
            s[n - 1] = 'z';
            return s;
        }

        while (i < n && s[i] != 'a')
            --s[i++];

        return s;
    }
};
```


1836. 2735.cpp

Path: LeetCode-main/solutions/2735. Collecting Chocolates/2735.cpp

```
class Solution {
public:
    long long minCost(vector<int>& nums, long long x) {
        const int n = nums.size();
        long ans = LONG_MAX;
        // minCost[i] := the minimum cost to collect the i-th type
        vector<int> minCost(n, INT_MAX);

        for (int rotate = 0; rotate < n; ++rotate) {
            for (int i = 0; i < n; ++i)
                minCost[i] = min(minCost[i], nums[(i - rotate + n) % n]);
            ans =
                min(ans, accumulate(minCost.begin(), minCost.end(), 0L) + rotate * x);
        }
        return ans;
    }
};
```

1837. 2736.cpp

Path: LeetCode-main/solutions/2736. Maximum Sum Queries/2736.cpp

```
struct Pair {
    int x;
    int y;
};

struct IndexedQuery {
    int queryIndex;
    int minX;
    int minY;
};

class Solution {
public:
    vector<int> maximumSumQueries(vector<int>& nums1, vector<int>& nums2,
                                vector<vector<int>>& queries) {
        const vector<Pair> pairs = getPairs(nums1, nums2);
        vector<int> ans(queries.size());
        vector<pair<int, int>> stack; // [(y, x + y)]

        int pairsIndex = 0;
        for (const auto& [queryIndex, minX, minY] : getIndexedQueries(queries)) {
            while (pairsIndex < pairs.size() && pairs[pairsIndex].x >= minX) {
                const auto [x, y] = pairs[pairsIndex++];
                // x + y is a better candidate. Given that x is decreasing, the
                // condition "x + y >= stack.back().second" suggests that y is
                // relatively larger, thereby making it a better candidate.
                while (!stack.empty() && x + y >= stack.back().second)
                    stack.pop_back();
                if (stack.empty() || y > stack.back().first)
                    stack.emplace_back(y, x + y);
            }
            const auto it = ranges::lower_bound(stack, pair<int, int>{minY, INT_MIN});
            ans[queryIndex] = it == stack.end() ? -1 : it->second;
        }

        return ans;
    }

private:
    vector<Pair> getPairs(const vector<int>& nums1, const vector<int>& nums2) {
        vector<Pair> pairs;
        for (int i = 0; i < nums1.size(); ++i)
            pairs.push_back({nums1[i], nums2[i]});
        ranges::sort(pairs, ranges::greater{},
                    [](const Pair& pair) { return pair.x; });
        return pairs;
    }

    vector<IndexedQuery> getIndexedQueries(const vector<vector<int>>& queries) {
        vector<IndexedQuery> indexedQueries;
        for (int i = 0; i < queries.size(); ++i)
            indexedQueries.push_back({i, queries[i][0], queries[i][1]});
        ranges::sort(indexedQueries,
                    [](const IndexedQuery& a, const IndexedQuery& b) {
                        return a.minX > b.minX;
                    });
        return indexedQueries;
    }
};
```

1838. 2737.cpp

Path: LeetCode-main/solutions/2737. Find the Closest Marked Node/2737.cpp

```
class Solution {
public:
    int minimumDistance(int n, vector<vector<int>>& edges, int s,
                        vector<int>& marked) {
        int ans = INT_MAX;
        vector<vector<pair<int, int>>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int w = edge[2];
            graph[u].emplace_back(v, w);
        }

        const vector<int> dist = dijkstra(graph, s);

        for (const int u : marked)
            ans = min(ans, dist[u]);

        return ans == INT_MAX ? -1 : ans;
    }

private:
    vector<int> dijkstra(const vector<vector<pair<int, int>>>& graph, int src) {
        vector<int> dist(graph.size(), INT_MAX);

        dist[src] = 0;
        using P = pair<int, int>; // (d, u)
        priority_queue<P, vector<P>, greater<>> minHeap;
        minHeap.emplace(dist[src], src);

        while (!minHeap.empty()) {
            const auto [d, u] = minHeap.top();
            minHeap.pop();
            if (d > dist[u])
                continue;
            for (const auto& [v, w] : graph[u])
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    minHeap.emplace(dist[v], v);
                }
        }

        return dist;
    }
};
```

1839. 2739.cpp

Path: LeetCode-main/solutions/2739. Total Distance Traveled/2739.cpp

```
class Solution {
public:
    int distanceTraveled(int mainTank, int additionalTank) {
        // M M M M M A M M M M A
        // 1 [2 3 4 5] 6 [7 8 9 10] 11
        return (mainTank + min((mainTank - 1) / 4, additionalTank)) * 10;
    }
};
```

1840. 274-2.cpp

Path: LeetCode-main/solutions/274. H-Index/274-2.cpp

```
class Solution {
public:
    int hIndex(vector<int>& citations) {
        const int n = citations.size();

        ranges::sort(citations);

        for (int i = 0; i < n; ++i)
            if (citations[i] >= n - i)
                return n - i;

        return 0;
    }
};
```

1841. 274.cpp

Path: LeetCode-main/solutions/274. H-Index/274.cpp

```
class Solution {
public:
    int hIndex(vector<int>& citations) {
        const int n = citations.size();
        int accumulate = 0;
        vector<int> count(n + 1);

        for (const int citation : citations)
            ++count[min(citation, n)];

        // To find the maximum h-index, loop from the back to the front.
        // i := the candidate's h-index
        for (int i = n; i >= 0; --i) {
            accumulate += count[i];
            if (accumulate >= i)
                return i;
        }

        throw;
    }
};
```

1842. 2740.cpp

Path: LeetCode-main/solutions/2740. Find the Value of the Partition/2740.cpp

```
class Solution {
public:
    int findValueOfPartition(vector<int>& nums) {
        int ans = INT_MAX;

        sort(begin(nums), end(nums));

        for (int i = 1; i < nums.size(); ++i)
            ans = min(ans, nums[i] - nums[i - 1]);

        return ans;
    }
};
```

1843. 2741.cpp

Path: LeetCode-main/solutions/2741. Special Permutations/2741.cpp

```
class Solution {
public:
    int specialPerm(vector<int>& nums) {
        const int n = nums.size();
        const int maxMask = 1 << n;
        int ans = 0;
        vector<vector<int>> mem(n, vector<int>(maxMask));

        for (int i = 0; i < n; ++i) {
            ans += specialPerm(nums, i, 1 << i, maxMask, mem);
            ans %= kMod;
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of special permutations, where the previous number is
    // nums[i] and `mask` is the bitmask of the used numbers.
    int specialPerm(const vector<int>& nums, int prev, int mask,
                    const int& maxMask, vector<vector<int>>& mem) {
        if (mask == maxMask - 1)
            return 1;
        if (mem[prev][mask] > 0)
            return mem[prev][mask];

        int res = 0;

        for (int i = 0; i < nums.size(); ++i) {
            if (mask >> i & 1)
                continue;
            if (nums[i] % nums[prev] == 0 || nums[prev] % nums[i] == 0) {
                res += specialPerm(nums, i, mask | 1 << i, maxMask, mem);
                res %= kMod;
            }
        }

        return mem[prev][mask] = res;
    }
};
```


1844. 2742-2.cpp

Path: LeetCode-main/solutions/2742. Painting the Walls/2742-2.cpp

```
class Solution {
public:
    int paintWalls(vector<int>& cost, vector<int>& time) {
        constexpr int kMax = 500'000'000;
        const int n = cost.size();
        // dp[i] := the minimum cost to paint i walls by the painters so far
        vector<int> dp(n + 1, kMax);
        dp[0] = 0;

        for (int i = 0; i < n; ++i)
            for (int walls = n; walls > 0; --walls)
                dp[walls] = min(dp[walls], dp[max(walls - time[i] - 1, 0)] + cost[i]);

        return dp[n];
    }
};
```

1845. 2742.cpp

Path: LeetCode-main/solutions/2742. Painting the Walls/2742.cpp

```
class Solution {
public:
    int paintWalls(vector<int>& cost, vector<int>& time) {
        const int n = cost.size();
        vector<vector<int>> mem(n, vector<int>(n + 1));
        return paintWalls(cost, time, 0, time.size(), mem);
    }

private:
    static constexpr int kMax = 500'000'000;

    // Returns the minimum cost to paint j walls by painters[i..n).
    int paintWalls(const vector<int>& cost, const vector<int>& time, int i,
                   int walls, vector<vector<int>>& mem) {
        if (walls <= 0)
            return 0;
        if (i == cost.size())
            return kMax;
        if (mem[i][walls] > 0)
            return mem[i][walls];
        const int pick =
            cost[i] + paintWalls(cost, time, i + 1, walls - time[i] - 1, mem);
        const int skip = paintWalls(cost, time, i + 1, walls, mem);
        return mem[i][walls] = min(pick, skip);
    }
};
```

1846. 2743.cpp

Path: LeetCode-main/solutions/2743. Count Substrings Without Repeating Character/2743.cpp

```
class Solution {
public:
    int numberOfSpecialSubstrings(string s) {
        int ans = 0;
        vector<int> count(26);

        for (int l = 0, r = 0; r < s.length(); ++r) {
            ++count[s[r] - 'a'];
            while (count[s[r] - 'a'] == 2)
                --count[s[l++ - 'a'];
            ans += r - l + 1;
        }

        return ans;
    }
};
```

1847. 2744.cpp

Path: LeetCode-main/solutions/2744. Find Maximum Number of String Pairs/2744.cpp

```
class Solution {
public:
    int maximumNumberOfStringPairs(vector<string>& words) {
        int ans = 0;
        vector<bool> seen(26 * 26);

        for (const string& word : words) {
            if (seen[val(word[1]) * 26 + val(word[0])])
                ++ans;
            seen[val(word[0]) * 26 + val(word[1])] = true;
        }

        return ans;
    }

private:
    constexpr int val(char c) {
        return c - 'a';
    }
};
```

1848. 2745.cpp

Path: LeetCode-main/solutions/2745. Construct the Longest New String/2745.cpp

```
class Solution {
public:
    int longestString(int x, int y, int z) {
        // "AB" can always be easily appended within the string.
        // Alternating "AA" and "BB" can be appended, creating a pattern like "AABB"
        // If x == y, we repeat the pattern "AABBAABB...AABB".
        // If x != y, the pattern becomes "AABBAABB...AABBAA" or "BBAABBAABB...AABB"
        const int mn = min(x, y);
        if (x == y)
            return (mn * 2 + z) * 2;
        return (mn * 2 + 1 + z) * 2;
    }
};
```

1849. 2746.cpp

Path: LeetCode-main/solutions/2746. Decremental String Concatenation/2746.cpp

```

class Solution {
public:
    int minimizeConcatenatedLength(vector<string>& words) {
        vector<vector<vector<int>>> mem(words.size(),
                                     vector<vector<int>>(26, vector<int>(26)));
        return words[0].length() + minimizeConcatenatedLength(words, 1,
                                                                words[0].front(),
                                                                words[0].back(), mem);
    }

private:
    // Returns the minimum concatenated length of the first i words starting with
    // `first` and ending in `last`.
    int minimizeConcatenatedLength(const vector<string>& words, int i, char first,
                                   char last, vector<vector<vector<int>>>& mem) {
        if (i == words.size())
            return 0;
        const int j = first - 'a';
        const int k = last - 'a';
        if (mem[i][j][k] > 0)
            return mem[i][j][k];
        const char nextFirst = words[i].front();
        const char nextLast = words[i].back();
        return mem[i][j][k] = //
            words[i].length() +
            min(
                // join(words[i - 1], words[i])
                minimizeConcatenatedLength(words, i + 1, first, nextLast, mem) -
                    (last == nextFirst ? 1 : 0),
                // join(words[i], words[i - 1])
                minimizeConcatenatedLength(words, i + 1, nextFirst, last, mem) -
                    (first == nextLast ? 1 : 0));
    }
};

```

1850. 2747.cpp

Path: LeetCode-main/solutions/2747. Count Zero Request Servers/2747.cpp

```
struct IndexedQuery {
    int queryIndex;
    int query;
};

class Solution {
public:
    vector<int> countServers(int n, vector<vector<int>>& logs, int x,
                           vector<int>& queries) {
        vector<int> ans(queries.size());
        vector<int> count(n + 1);

        ranges::sort(logs, ranges::less{},
                     [](const vector<int>& log) { return log[1]; });

        int i = 0;
        int j = 0;
        int servers = 0;

        // For each query, we care about logs[i..j].
        for (const auto& [queryIndex, query] : getIndexedQueries(queries)) {
            for (; j < logs.size() && logs[j][1] <= query; ++j)
                if (++count[logs[j][0]] == 1)
                    ++servers;
            for (; i < logs.size() && logs[i][1] < query - x; ++i)
                if (--count[logs[i][0]] == 0)
                    --servers;
            ans[queryIndex] = n - servers;
        }

        return ans;
    }

private:
    vector<IndexedQuery> getIndexedQueries(const vector<int>& queries) {
        vector<IndexedQuery> indexedQueries;
        for (int i = 0; i < queries.size(); ++i)
            indexedQueries.push_back({i, queries[i]});
        ranges::sort(indexedQueries,
                     [](const IndexedQuery& a, const IndexedQuery& b) {
                         return a.query < b.query;
                     });
        return indexedQueries;
    }
};
```

1851. 2748.cpp

Path: LeetCode-main/solutions/2748. Number of Beautiful Pairs/2748.cpp

```
class Solution {
public:
    int countBeautifulPairs(vector<int>& nums) {
        int ans = 0;

        for (int i = 0; i < nums.size(); ++i)
            for (int j = i + 1; j < nums.size(); ++j)
                if (__gcd(firstDigit(nums[i]), lastDigit(nums[j])) == 1)
                    ++ans;

        return ans;
    }

private:
    int firstDigit(int num) {
        return to_string(num)[0] - '0';
    }

    int lastDigit(int num) {
        return num % 10;
    }
};
```


1852. 2749.cpp

Path: LeetCode-main/solutions/2749. Minimum Operations to Make the Integer Zero/2749.cpp

```
class Solution {
public:
    int makeTheIntegerZero(int num1, int num2) {
        // If k operations are used,  $\text{num1} - [(\text{num2} + 2^{i_1}) + (\text{num2} + 2^{i_2}) + \dots + (\text{num2} + 2^{i_k})] = 0$ . So,  $\text{num1} - k * \text{num2} = (2^{i_1} + 2^{i_2} + \dots + 2^{i_k})$ , where  $i_1, i_2, \dots, i_k$  are in the range  $[0, 60]$ .
        // Note that for any number x, we can use "x's bit count" operations to make x equal to 0. Additionally, we can also use x operations to deduct x by  $2^0$  (x times), which also results in 0.

        for (long ops = 0; ops <= 60; ++ops) {
            const long target = num1 - ops * num2;
            if (__builtin_popcountl(target) <= ops && ops <= target)
                return ops;
        }

        return -1;
    }
};
```

1853. 275.cpp

Path: LeetCode-main/solutions/275. H-Index II/275.cpp

```
class Solution {
public:
    int hIndex(vector<int>& citations) {
        const int n = citations.size();
        int l = 0;
        int r = n;

        while (l < r) {
            const int m = (l + r) / 2;
            if (citations[m] + m >= n)
                r = m;
            else
                l = m + 1;
        }
        return n - l;
    }
};
```

1854. 2750.cpp

Path: LeetCode-main/solutions/2750. Ways to Split Array Into Good Subarrays/2750.cpp

```
class Solution {
public:
    int numberOfGoodSubarraySplits(vector<int>& nums) {
        if (ranges::count(nums, 1) == 0)
            return 0;

        constexpr int kMod = 1'000'000'007;
        int prev = -1; // the previous index of 1
        int ans = 1;

        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == 1) {
                if (prev != -1)
                    ans = static_cast<long>(ans) * (i - prev) % kMod;
                prev = i;
            }

        return ans;
    }
};
```

1855. 2751.cpp

Path: LeetCode-main/solutions/2751. Robot Collisions/2751.cpp

```
struct Robot {
    int index;
    int position;
    int health;
    char direction;
};

class Solution {
public:
    vector<int> survivedRobotsHealths(vector<int>& positions,
                                      vector<int>& healths, string directions) {

        vector<int> ans;
        vector<Robot> robots;
        vector<Robot> stack; // the runnig robots

        for (int i = 0; i < positions.size(); ++i)
            robots.push_back(Robot{i, positions[i], healths[i], directions[i]});

        ranges::sort(robots, ranges::less{},
                    [](const Robot& robot) { return robot.position; });

        for (Robot& robot : robots) {
            if (robot.direction == 'R') {
                stack.push_back(robot);
                continue;
            }
            // Collide with robots going right if any.
            while (!stack.empty() && stack.back().direction == 'R' &&
                  robot.health > 0) {
                if (stack.back().health == robot.health) {
                    stack.pop_back();
                    robot.health = 0;
                } else if (stack.back().health < robot.health) {
                    stack.pop_back();
                    robot.health -= 1;
                } else { // stack.back().health > robot.health
                    stack.back().health -= 1;
                    robot.health = 0;
                }
            }
            if (robot.health > 0)
                stack.push_back(robot);
        }

        ranges::sort(stack, ranges::less{},
                    [](const Robot& robot) { return robot.index; });

        for (const Robot& robot : stack)
            ans.push_back(robot.health);

        return ans;
    }
};
```

1856. 2753.cpp

Path: LeetCode-main/solutions/2753. Count Houses in a Circular Street II/2753.cpp

```
/**
 * Definition for a street.
 * class Street {
 * public:
 *     Street(vector<int> doors);
 *     void closeDoor();
 *     bool isDoorOpen();
 *     void moveRight();
 * };
 */
class Solution {
public:
    int houseCount(Street* street, int k) {
        int ans = 0;

        // Go to the first open door.
        while (!street->isDoorOpen())
            street->moveRight();

        street->moveRight();

        for (int count = 1; count <= k; ++count) {
            // Each time we encounter an open door, there's a possibility that it's
            // the first open door we intentionally left open.
            if (street->isDoorOpen()) {
                ans = count;
                street->closeDoor();
            }
            street->moveRight();
        }

        return ans;
    }
};
```

1857. 276.cpp

Path: LeetCode-main/solutions/276. Paint Fence/276.cpp

```
class Solution {
public:
    int numWays(int n, int k) {
        if (n == 0)
            return 0;
        if (n == 1)
            return k;
        if (n == 2)
            return k * k;

        // dp[i] := the number of ways to paint n posts with k colors
        vector<int> dp(n + 1);
        dp[0] = 0;
        dp[1] = k;
        dp[2] = k * k;

        for (int i = 3; i <= n; ++i)
            dp[i] = (dp[i - 1] + dp[i - 2]) * (k - 1);

        return dp[n];
    }
};
```

1858. 2760.cpp

Path: LeetCode-main/solutions/2760. Longest Even Odd Subarray With Threshold/2760.cpp

```
class Solution {
public:
    int longestAlternatingSubarray(vector<int>& nums, int threshold) {
        int ans = 0;
        int dp = 0;

        for (int i = 0; i < nums.size(); ++i) {
            if (nums[i] > threshold)
                dp = 0;
            else if (i > 0 && dp > 0 && isOddEven(nums[i - 1], nums[i]))
                // Increase the size of the subarray.
                ++dp;
            else
                // Start a new subarray if the start is valid.
                dp = nums[i] % 2 == 0 ? 1 : 0;
            ans = max(ans, dp);
        }

        return ans;
    }

private:
    bool isOddEven(int a, int b) {
        return a % 2 != b % 2;
    }
};
```

1859. 2761.cpp

Path: LeetCode-main/solutions/2761. Prime Pairs With Target Sum/2761.cpp

```
class Solution {
public:
    vector<vector<int>> findPrimePairs(int n) {
        const vector<bool> isPrime = sieveEratosthenes(n + 1);
        vector<vector<int>> ans;

        for (int i = 2; i <= n / 2; ++i)
            if (isPrime[i] && isPrime[n - i])
                ans.push_back({i, n - i});

        return ans;
    }

private:
    vector<bool> sieveEratosthenes(int n) {
        vector<bool> isPrime(n, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
};
```


1860. 2762.cpp

Path: LeetCode-main/solutions/2762. Continuous Subarrays/2762.cpp

```
class Solution {
public:
    long long continuousSubarrays(vector<int>& nums) {
        long ans = 1; // [nums[0]]
        int left = nums[0] - 2;
        int right = nums[0] + 2;
        int l = 0;

        // nums[l..r] is a valid window with range in [left, right].
        for (int r = 1; r < nums.size(); r++) {
            if (left <= nums[r] && nums[r] <= right) {
                left = max(left, nums[r] - 2);
                right = min(right, nums[r] + 2);
            } else {
                // nums[r] is out-of-bounds, so reconstruct the window.
                left = nums[r] - 2;
                right = nums[r] + 2;
                l = r;
                // If we consistently move leftward in each iteration, it implies that
                // the entire left subarray satisfies the given condition. For every
                // subarray with l in the range [0, r], the condition is met, preventing
                // the code from reaching the final "else" condition. Instead, it stops
                // at the "if" condition.
                while (nums[r] - 2 <= nums[l] && nums[l] <= nums[r] + 2) {
                    left = max(left, nums[l] - 2);
                    right = min(right, nums[l] + 2);
                    --l;
                }
                ++l;
            }
            // nums[l..r], nums[l + 1..r], ..., nums[r]
            ans += r - l + 1;
        }

        return ans;
    }
};
```

1861. 2763-2.cpp

Path: LeetCode-main/solutions/2763. Sum of Imbalance Numbers of All Subarrays/2763-2.cpp

```
class Solution {
public:
    // If sorted(nums)[i + 1] - sorted(nums)[i] > 1, then there's a gap. Instead
    // of determining the number of gaps in each subarray, let's find out how many
    // subarrays contain each gap.
    int sumImbalanceNumbers(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;
        // Note that to avoid double counting, only `left` needs to check nums[i].
        // This adjustment ensures that i represents the position of the leftmost
        // element of nums[i] within the subarray.

        // left[i] := the maximum index l s.t. nums[l] = nums[i] or nums[i] + 1
        vector<int> left(n);
        // right[i] := the minimum index r s.t. nums[r] = nums[i]
        vector<int> right(n);

        vector<int> numToIndex(n + 2, -1);
        for (int i = 0; i < n; ++i) {
            left[i] = max(numToIndex[nums[i]], numToIndex[nums[i] + 1]);
            numToIndex[nums[i]] = i;
        }

        fill(numToIndex.begin(), numToIndex.end(), n);
        for (int i = n - 1; i >= 0; --i) {
            right[i] = numToIndex[nums[i] + 1];
            numToIndex[nums[i]] = i;
        }

        // The gap above nums[i] persists until encountering nums[i] or nums[i] + 1.
        // Consider subarrays nums[l..r] with l <= i <= r, where l in [left[i], i]
        // and r in [i, right[i] - 1]. There are (i - left[i]) * (right[i] - i)
        // subarrays satisfying this condition.
        for (int i = 0; i < n; ++i)
            ans += (i - left[i]) * (right[i] - i);

        // Subtract n * (n + 1) / 2 to account for the overcounting of elements
        // initially assumed to have a gap. This adjustment is necessary as the
        // maximum element of every subarray does not have a gap.
        return ans - n * (n + 1) / 2;
    }
};
```

1862. 2764.cpp

Path: LeetCode-main/solutions/2764. is Array a Preorder of Some ■ Binary Tree/2764.cpp

```
class Solution {
public:
    bool isPreorder(vector<vector<int>>& nodes) {
        stack<int> stack; // Stores `id`s.

        for (const vector<int>& node : nodes) {
            const int id = node[0];
            const int parentId = node[1];
            if (parentId == -1) {
                stack.push(id);
                continue;
            }
            while (!stack.empty() && stack.top() != parentId)
                stack.pop();
            if (stack.empty())
                return false;
            stack.push(id);
        }
        return true;
    }
};
```

1863. 2765.cpp

Path: LeetCode-main/solutions/2765. Longest Alternating Subarray/2765.cpp

```
class Solution {
public:
    int alternatingSubarray(vector<int>& nums) {
        int ans = 1;
        int dp = 1;

        for (int i = 1; i < nums.size(); ++i) {
            const int targetDiff = dp % 2 == 0 ? -1 : 1;
            // Append nums[i] to the current alternating subarray.
            if (nums[i] - nums[i - 1] == targetDiff)
                ++dp;
            // Reset the alternating subarray to nums[i - 1..i].
            else if (nums[i] - nums[i - 1] == 1)
                dp = 2;
            // Reset the alternating subarray to nums[i].
            else
                dp = 1;
            ans = max(ans, dp);
        }

        return ans == 1 ? -1 : ans;
    }
};
```

1864. 2766.cpp

Path: LeetCode-main/solutions/2766. Relocate Marbles/2766.cpp

```
class Solution {
public:
    vector<int> relocateMarbles(vector<int>& nums, vector<int>& moveFrom,
                               vector<int>& moveTo) {
        set<int> numsSet{nums.begin(), nums.end()};

        for (int i = 0; i < moveFrom.size(); ++i) {
            numsSet.erase(numsSet.find(moveFrom[i]));
            numsSet.insert(moveTo[i]);
        }

        return {numsSet.begin(), numsSet.end()};
    }
};
```

1865. 2767.cpp

Path: LeetCode-main/solutions/2767. Partition String Into Minimum Beautiful Substrings/2767.cpp

```
class Solution {
public:
    int minimumBeautifulSubstrings(string s) {
        const int n = s.length();
        // dp[i] := the minimum number of beautiful substrings for the first i chars
        vector<int> dp(n + 1, n + 1);
        dp[0] = 0;

        for (int i = 1; i <= n; ++i) {
            if (s[i - 1] == '0')
                continue;
            int num = 0; // num of s[i - 1..j - 1]
            for (int j = i; j <= n; ++j) {
                num = (num << 1) + s[j - 1] - '0';
                if (isPowerOfFive(num))
                    dp[j] = min(dp[j], dp[i - 1] + 1);
            }
        }

        return dp[n] == n + 1 ? -1 : dp[n];
    }

private:
    bool isPowerOfFive(int num) {
        while (num % 5 == 0)
            num /= 5;
        return num == 1;
    }
};
```

1866. 2768.cpp

Path: LeetCode-main/solutions/2768. Number of Black Blocks/2768.cpp

```
class Solution {
public:
    vector<long long> countBlackBlocks(int m, int n,
                                      vector<vector<int>>& coordinates) {
        vector<long long> ans(5);
        // count[i * n + j] := the number of black cells in
        // (i - 1, j - 1), (i - 1, j), (i, j - 1), (i, j)
        unordered_map<long, int> count;

        for (const vector<int>& coordinate : coordinates) {
            const int x = coordinate[0];
            const int y = coordinate[1];
            for (long i = x; i < x + 2; ++i)
                for (long j = y; j < y + 2; ++j)
                    // 2 x 2 submatrix with right-bottom conner being (i, j) contains the
                    // current black cell (x, y).
                    if (i - 1 >= 0 && i < m && j - 1 >= 0 && j < n)
                        ++count[i * n + j];
        }

        for (const auto& [_, freq] : count)
            ++ans[freq];

        ans[0] = (m - 1L) * (n - 1) - accumulate(ans.begin(), ans.end(), 0L);
        return ans;
    }
};
```

1867. 2769.cpp

Path: LeetCode-main/solutions/2769. Find the Maximum Achievable Number/2769.cpp

```
class Solution {  
public:  
    int theMaximumAchievableX(int num, int t) {  
        return num + 2 * t;  
    }  
};
```


1868. 277.cpp

Path: LeetCode-main/solutions/277. Find the Celebrity/277.cpp

```
class Solution {
public:
    int findCelebrity(int n) {
        int candidate = 0;

        // Everyone knows the celebrity.
        for (int i = 1; i < n; ++i)
            if (knows(candidate, i))
                candidate = i;

        // The candidate knows nobody and everyone knows the celebrity.
        for (int i = 0; i < n; ++i) {
            if (i < candidate && knows(candidate, i) || !knows(i, candidate))
                return -1;
            if (i > candidate && !knows(i, candidate))
                return -1;
        }

        return candidate;
    }
};
```

1869. 2770.cpp

Path: LeetCode-main/solutions/2770. Maximum Number of Jumps to Reach the Last Index/2770.cpp

```
class Solution {
public:
    int maximumJumps(vector<int>& nums, int target) {
        const int n = nums.size();
        // dp[i] := the maximum number of jumps to reach i from 0
        vector<int> dp(n, -1);
        dp[0] = 0;

        for (int j = 1; j < n; ++j)
            for (int i = 0; i < j; ++i)
                if (dp[i] != -1 && abs(nums[j] - nums[i]) <= target)
                    dp[j] = max(dp[j], dp[i] + 1);

        return dp[n - 1];
    }
};
```

1870. 2771.cpp

Path: LeetCode-main/solutions/2771. Longest Non-decreasing Subarray From Two Arrays/2771.cpp

```
class Solution {
public:
    int maxNonDecreasingLength(vector<int>& nums1, vector<int>& nums2) {
        int ans = 1;
        int dp1 = 1; // the longest subarray that ends in nums1[i] so far
        int dp2 = 1; // the longest subarray that ends in nums2[i] so far

        for (int i = 1; i < nums1.size(); ++i) {
            const int dp11 = nums1[i - 1] <= nums1[i] ? dp1 + 1 : 1;
            const int dp21 = nums2[i - 1] <= nums1[i] ? dp2 + 1 : 1;
            const int dp12 = nums1[i - 1] <= nums2[i] ? dp1 + 1 : 1;
            const int dp22 = nums2[i - 1] <= nums2[i] ? dp2 + 1 : 1;
            dp1 = max(dp11, dp21);
            dp2 = max(dp12, dp22);
            ans = max({ans, dp1, dp2});
        }

        return ans;
    }
};
```

1871. 2772.cpp

Path: LeetCode-main/solutions/2772. Apply Operations to Make All Array Elements Equal to Zero/2772.cpp

```
class Solution {
public:
    bool checkArray(vector<int>& nums, int k) {
        if (k == 1)
            return true;

        int needDecrease = 0;
        // Store nums[i - k + 1..i] with decreasing nums[i - k + 1].
        deque<int> dq;

        for (int i = 0; i < nums.size(); ++i) {
            if (i >= k) {
                needDecrease -= dq.front();
                dq.pop_front();
            }
            if (nums[i] < needDecrease)
                return false;
            const int decreasedNum = nums[i] - needDecrease;
            dq.push_back(decreasedNum);
            needDecrease += decreasedNum;
        }

        return dq.back() == 0;
    }
};
```

1872. 2773.cpp

Path: LeetCode-main/solutions/2773. Height of Special Binary Tree/2773.cpp

```
class Solution {
public:
    int heightOfTree(TreeNode* root) {
        if (root == nullptr)
            return 0;
        // a leaf node
        if (root->left != nullptr && root->left->right == root)
            return 0;
        return 1 + max(heightOfTree(root->left), heightOfTree(root->right));
    }
};
```

1873. 2778.cpp

Path: LeetCode-main/solutions/2778. Sum of Squares of Special Elements/2778.cpp

```
class Solution {
public:
    int sumOfSquares(vector<int>& nums) {
        const int n = nums.size();
        int ans = 0;

        for (int i = 0; i < n; ++i)
            if (n % (i + 1) == 0)
                ans += nums[i] * nums[i];

        return ans;
    }
};
```

1874. 2779-2.cpp

Path: LeetCode-main/solutions/2779. Maximum Beauty of an Array After Applying Operation/2779-2.cpp

```
class Solution {
public:
    int maximumBeauty(vector<int>& nums, int k) {
        // l and r track the maximum window instead of the valid window.
        int l = 0;
        int r = 0;

        ranges::sort(nums);

        for (r = 0; r < nums.size(); ++r)
            if (nums[r] - nums[l] > 2 * k)
                ++l;

        return r - l;
    }
};
```

1875. 2779.cpp

Path: LeetCode-main/solutions/2779. Maximum Beauty of an Array After Applying Operation/2779.cpp

```
class Solution {
public:
    int maximumBeauty(vector<int>& nums, int k) {
        int ans = 0;

        ranges::sort(nums);

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            while (nums[r] - nums[l] > 2 * k)
                ++l;
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```


1876. 278.cpp

Path: LeetCode-main/solutions/278. First Bad Version/278.cpp

```
bool isBadVersion(int version);

class Solution {
public:
    int firstBadVersion(int n) {
        int l = 1;
        int r = n;

        while (l < r) {
            const int m = l + (r - l) / 2;
            if (isBadVersion(m))
                r = m;
            else
                l = m + 1;
        }

        return l;
    }
};
```

1877. 2780.cpp

Path: LeetCode-main/solutions/2780. Minimum Index of a Valid Split/2780.cpp

```
class Solution {
public:
    int minimumIndex(vector<int>& nums) {
        const int n = nums.size();
        unordered_map<int, int> count1;
        unordered_map<int, int> count2;

        for (const int num : nums)
            ++count2[num];

        for (int i = 0; i < n; ++i) {
            const int freq1 = ++count1[nums[i]];
            const int freq2 = --count2[nums[i]];
            if (freq1 * 2 > i + 1 && freq2 * 2 > n - 1 - i)
                return i;
        }
        return -1;
    }
};
```

1878. 2781-2.cpp

Path: LeetCode-main/solutions/2781. Length of the Longest Valid Substring/2781-2.cpp

```
struct TrieNode {
public:
    unordered_map<char, shared_ptr<TrieNode>> children;
    bool isWord = false;
};

class Trie {
public:
    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            if (node->children[c] == nullptr)
                node->children[c] = make_shared<TrieNode>();
            node = node->children[c];
        }
        node->isWord = true;
    }

    bool search(const string& word, int l, int r) {
        shared_ptr<TrieNode> node = root;
        for (int j = l; j <= r; ++j) {
            const char c = word[j];
            if (node->children[c] == nullptr)
                return false;
            node = node->children[c];
        }
        return node->isWord;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();
};

class Solution {
public:
    int longestValidSubstring(string word, vector<string>& forbidden) {
        int ans = 0;
        Trie trie;

        for (const string& s : forbidden)
            trie.insert(s);

        // r is the rightmost index to make word[l..r] a valid substring.
        int r = word.length() - 1;
        for (int l = word.length() - 1; l >= 0; --l) {
            for (int end = l; end < min(l + 10, r + 1); ++end)
                if (trie.search(word, l, end)) {
                    r = end - 1;
                    break;
                }
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

1879. 2781.cpp

Path: LeetCode-main/solutions/2781. Length of the Longest Valid Substring/2781.cpp

```
class Solution {
public:
    int longestValidSubstring(string word, vector<string>& forbidden) {
        int ans = 0;
        unordered_set<string> forbiddenSet{forbidden.begin(), forbidden.end()};

        // r is the rightmost index to make word[l..r] a valid substring.
        int r = word.length() - 1;
        for (int l = word.length() - 1; l >= 0; --l) {
            for (int end = l; end < min(l + 10, r + 1); ++end)
                if (forbiddenSet.contains(word.substr(l, end - l + 1))) {
                    r = end - 1;
                    break;
                }
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

1880. 2782.cpp

Path: LeetCode-main/solutions/2782. Number of Unique Categories/2782.cpp

```
/**
 * Definition for a category handler.
 * class CategoryHandler {
 * public:
 *     CategoryHandler(vector<int> categories);
 *     bool haveSameCategory(int a, int b);
 * };
 */

class Solution {
public:
    int numberOfCategories(int n, CategoryHandler* categoryHandler) {
        int ans = 0;

        for (int i = 0; i < n; ++i)
            if (!haveSameCategoryPreviously(i, n, categoryHandler))
                ++ans;

        return ans;
    }

private:
    bool haveSameCategoryPreviously(int i, int n,
                                    CategoryHandler* categoryHandler) {
        for (int j = 0; j < i; ++j)
            if (categoryHandler->haveSameCategory(i, j))
                return true;
        return false;
    }
};
```

1881. 2784.cpp

Path: LeetCode-main/solutions/2784. Check if Array is Good/2784.cpp

```
class Solution {
public:
    bool isGood(vector<int>& nums) {
        constexpr int kMax = 200;
        const int n = nums.size() - 1;
        vector<int> count(kMax + 1);

        for (const int num : nums)
            ++count[num];

        return all_of(count.begin() + 1, count.begin() + n, [](int c) {
            return c == 1;
        }) && count[n] == 2;
    }
};
```

1882. 2785.cpp

Path: LeetCode-main/solutions/2785. Sort Vowels in a String/2785.cpp

```
class Solution {
public:
    string sortVowels(string s) {
        string ans;
        vector<char> vowels;

        for (const char c : s)
            if (isVowel(c))
                vowels.push_back(c);

        ranges::sort(vowels);

        int i = 0; // vowels' index
        for (const char c : s)
            ans += isVowel(c) ? vowels[i++] : c;

        return ans;
    }

private:
    bool isVowel(char c) {
        static constexpr string_view kVowels = "aeiouAEIOU";
        return kVowels.find(c) != string_view::npos;
    }
};
```

1883. 2786.cpp

Path: LeetCode-main/solutions/2786. Visit Array Positions to Maximize Score/2786.cpp

```
class Solution {
public:
    long long maxScore(vector<int>& nums, int x) {
        // Note that we always need to take nums[0], so the initial definition might
        // not hold true.

        // dp0 := the maximum score so far with `nums` ending in an even number
        long dp0 = nums[0] - (nums[0] % 2 == 1 ? x : 0);
        // dp1 := the maximum score so far with `nums` ending in an odd number
        long dp1 = nums[0] - (nums[0] % 2 == 0 ? x : 0);

        for (int i = 1; i < nums.size(); ++i)
            if (nums[i] % 2 == 0)
                dp0 = nums[i] + max(dp0, dp1 - x);
            else
                dp1 = nums[i] + max(dp1, dp0 - x);

        return max(dp0, dp1);
    }
};
```


1884. 2787.cpp

Path: LeetCode-main/solutions/2787. Ways to Express an Integer as Sum of Powers/2787.cpp

```
class Solution {
public:
    int numberOfWays(int n, int x) {
        constexpr int kMod = 1'000'000'007;
        // dp[i] := the number of ways to express i
        vector<int> dp(n + 1);
        int ax; // a^x

        dp[0] = 1;

        for (int a = 1; (ax = pow(a, x)) <= n; ++a)
            for (int i = n; i >= ax; --i) {
                dp[i] += dp[i - ax];
                dp[i] %= kMod;
            }

        return dp[n];
    }
};
```

1885. 2788.cpp

Path: LeetCode-main/solutions/2788. Split Strings by Separator/2788.cpp

```
class Solution {
public:
    vector<string> splitWordsBySeparator(vector<string>& words, char separator) {
        vector<string> ans;

        for (const string& word : words) {
            istringstream iss(word);
            for (string word; getline(iss, word, separator);)
                if (!word.empty())
                    ans.push_back(word);
        }

        return ans;
    }
};
```

1886. 2789.cpp

Path: LeetCode-main/solutions/2789. Largest Element in an Array after Merge Operations/2789.cpp

```
class Solution {
public:
    long long maxArrayValue(vector<int>& nums) {
        long ans = nums.back();

        for (int i = nums.size() - 2; i >= 0; --i)
            if (nums[i] > ans)
                ans = nums[i];
            else
                ans += nums[i];

        return ans;
    }
};
```

1887. 279.cpp

Path: LeetCode-main/solutions/279. Perfect Squares/279.cpp

```
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, n); // 1^2 x n
        dp[0] = 0;                // no way
        dp[1] = 1;                // 1^2

        for (int i = 2; i <= n; ++i)
            for (int j = 1; j * j <= i; ++j)
                dp[i] = min(dp[i], dp[i - j * j] + 1);

        return dp[n];
    }
};
```

1888. 2790.cpp

Path: LeetCode-main/solutions/2790. Maximum Number of Groups With Increasing Length/2790.cpp

```
class Solution {
public:
    int maxIncreasingGroups(vector<int>& usageLimits) {
        int ans = 1; // the next target length
        long availableLimits = 0;

        ranges::sort(usageLimits);

        for (const int usageLimit : usageLimits) {
            availableLimits += usageLimit;
            // Can create groups 1, 2, ..., ans.
            if (availableLimits >= static_cast<long>(ans) * (ans + 1) / 2)
                ++ans;
        }
        return ans - 1;
    }
};
```

1889. 2791.cpp

Path: LeetCode-main/solutions/2791. Count Paths That Can Form a Palindrome in a Tree/2791.cpp

```
class Solution {
public:
    long long countPalindromePaths(vector<int>& parent, string s) {
        // A valid (u, v) has at most 1 letter with odd frequency on its path. The
        // frequency of a letter on the u-v path is equal to the sum of its
        // frequencies on the root-u and root-v paths subtract twice of its
        // frequency on the root-LCA(u, v) path. Considering only the parity
        // (even/odd), the part involving root-LCA(u, v) can be ignored, making it
        // possible to calculate both parts easily using a simple DFS.
        vector<vector<int>>> tree(parent.size());

        for (int i = 1; i < parent.size(); ++i)
            tree[parent[i]].push_back(i);

        return dfs(tree, 0, 0, s, {{0, 1}});
    }

private:
    // mask := 26 bits that represent the parity of each character in the alphabet
    // on the path from node 0 to node u
    long dfs(const vector<vector<int>>&& tree, int u, int mask, const string& s,
             unordered_map<int, int>&& maskToCount) {
        long res = 0;
        if (u > 0) {
            mask ^= 1 << (s[u] - 'a');
            // Consider any u-v path with 1 bit set.
            for (int i = 0; i < 26; ++i)
                if (const auto it = maskToCount.find(mask ^ (1 << i));
                    it != maskToCount.cend())
                    res += it->second;
            // Consider u-v path with 0 bit set.
            res += maskToCount[mask ^ 0]++;
        }
        for (const int v : tree[u])
            res += dfs(tree, v, mask, s, std::move(maskToCount));
        return res;
    }
};
```

1890. 2792.cpp

Path: LeetCode-main/solutions/2792. Count Nodes That Are Great Enough/2792.cpp

```
class Solution {
public:
    int countGreatEnoughNodes(TreeNode* root, int k) {
        int ans = 0;
        dfs(root, k, ans);
        return ans;
    }

private:
    multiset<int> dfs(TreeNode* root, int k, int& ans) {
        if (root == nullptr)
            return {};

        multiset<int> kSmallest = dfs(root->left, k, ans);
        multiset<int> kSmallestRight = dfs(root->right, k, ans);
        kSmallest.merge(kSmallestRight);

        if (kSmallest.size() > k)
            kSmallest.erase(next(kSmallest.begin(), k), kSmallest.end());
        if (kSmallest.size() == k && root->val > *kSmallest.rbegin())
            ++ans;

        kSmallest.insert(root->val);
        return kSmallest;
    }
};
```

1891. 2798.cpp

Path: LeetCode-main/solutions/2798. Number of Employees Who Met the Target/2798.cpp

```
class Solution {
public:
    int numberOfEmployeesWhoMetTarget(vector<int>& hours, int target) {
        return ranges::count_if(hours,
                                [target](int hour) { return hour >= target; });
    }
};
```


1892. 2799.cpp

Path: LeetCode-main/solutions/2799. Count Complete Subarrays in an Array/2799.cpp

```
class Solution {
public:
    int countCompleteSubarrays(vector<int>& nums) {
        constexpr int kMax = 2000;
        const int totalDistinct =
            unordered_set<int>(nums.begin(), nums.end()).size();
        int ans = 0;
        int distinct = 0;
        vector<int> count(kMax + 1);

        int l = 0;
        for (const int num : nums) {
            if (++count[num] == 1)
                ++distinct;
            while (distinct == totalDistinct)
                if (--count[nums[l++]] == 0)
                    --distinct;
            // Assume nums[r] = num,
            // nums[0..r], nums[1..r], ..., nums[l - 1..r] have k different values.
            ans += l;
        }
        return ans;
    }
};
```

1893. 28.cpp

Path: LeetCode-main/solutions/28. Implement strStr()/28.cpp

```
class Solution {
public:
    int strStr(string haystack, string needle) {
        const int m = haystack.length();
        const int n = needle.length();

        for (int i = 0; i < m - n + 1; i++)
            if (haystack.substr(i, n) == needle)
                return i;

        return -1;
    }
};
```

1894. 280.cpp

Path: LeetCode-main/solutions/280. Wiggle Sort/280.cpp

```
class Solution {
public:
    void wiggleSort(vector<int>& nums) {
        // 1. If i is even, then nums[i] <= nums[i - 1].
        // 2. If i is odd, then nums[i] >= nums[i - 1].
        for (int i = 1; i < nums.size(); ++i)
            if (i % 2 == 0 && nums[i] > nums[i - 1] ||
                i % 2 == 1 && nums[i] < nums[i - 1])
                swap(nums[i], nums[i - 1]);
    }
};
```

1895. 2800.cpp

Path: LeetCode-main/solutions/2800. Shortest String That Contains Three Strings/2800.cpp

```

class Solution {
public:
    string minimumString(string a, string b, string c) {
        const string abc = merge(a, merge(b, c));
        const string acb = merge(a, merge(c, b));
        const string bac = merge(b, merge(a, c));
        const string bca = merge(b, merge(c, a));
        const string cab = merge(c, merge(a, b));
        const string cba = merge(c, merge(b, a));
        return getMin({abc, acb, bac, bca, cab, cba});
    }

private:
    // Merges a and b.
    string merge(const string& a, const string& b) {
        if (b.find(a) != string::npos) // a is a substring of b.
            return b;
        for (int i = 0; i < a.length(); ++i) {
            const string aSuffix = a.substr(i);
            const string bPrefix = b.substr(0, min(b.length(), aSuffix.length()));
            if (aSuffix == bPrefix)
                return a + b.substr(bPrefix.length());
        }
        return a + b;
    }

    // Returns the lexicographically smallest string.
    string getMin(const vector<string>& words) {
        string res = words[0];
        for (int i = 1; i < words.size(); ++i)
            res = getMin(res, words[i]);
        return res;
    }

    // Returns the lexicographically smaller string.
    string getMin(const string& a, const string& b) {
        return (a.length() < b.length() || (a.length() == b.length() && a < b)) ? a : b;
    }
};

```

1896. 2801.cpp

Path: LeetCode-main/solutions/2801. Count Stepping Numbers in Range/2801.cpp

```
class Solution {
public:
    int countSteppingNumbers(string low, string high) {
        const string lowWithLeadingZeros =
            string(high.length() - low.length(), '0') + low;
        vector<vector<vector<vector<int>>>> mem(
            high.length(), vector<vector<vector<int>>>>(
                11, vector<vector<int>>>(2, vector<int>(2, -1))));
        return count(lowWithLeadingZeros, high, 0, 10, /*isLeadingZero=*/true, true,
            true, mem);
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns the number of valid integers, considering the i-th digit, where
    // `prevDigit` is the previous digit, `tight1` indicates if the current
    // digit is tightly bound for `low`, and `tight2` indicates if the current
    // digit is tightly bound for `high`.
    int count(const string& low, const string& high, int i, int prevDigit,
        bool isLeadingZero, bool tight1, bool tight2,
        vector<vector<vector<vector<int>>>>& mem) {
        if (i == high.length())
            return 1;
        if (mem[i][prevDigit][tight1][tight2] != -1)
            return mem[i][prevDigit][tight1][tight2];

        int res = 0;
        const int minDigit = tight1 ? low[i] - '0' : 0;
        const int maxDigit = tight2 ? high[i] - '0' : 9;

        for (int d = minDigit; d <= maxDigit; ++d) {
            const bool nextTight1 = tight1 && (d == minDigit);
            const bool nextTight2 = tight2 && (d == maxDigit);
            if (isLeadingZero)
                // Can place any digit in [minDigit, maxDigit].
                res += count(low, high, i + 1, d, isLeadingZero && d == 0, nextTight1,
                    nextTight2, mem);
            else if (abs(d - prevDigit) == 1)
                // Can only place prevDigit - 1 or prevDigit + 1.
                res += count(low, high, i + 1, d, false, nextTight1, nextTight2, mem);
            res %= kMod;
        }

        return mem[i][prevDigit][tight1][tight2] = res;
    }
};
```

1897. 2802.cpp

Path: LeetCode-main/solutions/2802. Find The K-th Lucky Number/2802.cpp

```
class Solution {
public:
    string kthLuckyNumber(int k) {
        string ans;

        for (int i = k + 1; i > 0; i /= 2)
            ans += i % 2 == 0 ? '4' : '7';

        ranges::reverse(ans);
        return ans.substr(1);
    }
};
```

1898. 2806.cpp

Path: LeetCode-main/solutions/2806. Account Balance After Rounded Purchase/2806.cpp

```
class Solution {  
public:  
    int accountBalanceAfterPurchase(int purchaseAmount) {  
        return 100 - ((purchaseAmount + 5) / 10) * 10;  
    }  
};
```

1899. 2807.cpp

Path: LeetCode-main/solutions/2807. Insert Greatest Common Divisors in Linked List/2807.cpp

```
class Solution {
public:
    ListNode* insertGreatestCommonDivisors(ListNode* head) {
        for (ListNode* curr = head; curr->next != nullptr;) {
            ListNode* inserted =
                new ListNode(__gcd(curr->val, curr->next->val), curr->next);
            curr->next = inserted;
            curr = inserted->next;
        }
        return head;
    }
};
```


1900. 2808.cpp

Path: LeetCode-main/solutions/2808. Minimum Seconds to Equalize a Circular Array/2808.cpp

```
class Solution {
public:
    int minimumSeconds(vector<int>& nums) {
        const int n = nums.size();
        int ans = n;
        unordered_map<int, vector<int>> numToIndices;

        for (int i = 0; i < n; ++i)
            numToIndices[nums[i]].push_back(i);

        for (const auto& [_ , indices] : numToIndices) {
            int seconds = getSeconds(indices.front() + n, indices.back());
            for (int i = 1; i < indices.size(); ++i)
                seconds = max(seconds, getSeconds(indices[i], indices[i - 1]));
            ans = min(ans, seconds);
        }

        return ans;
    }

private:
    // Returns the number of seconds required to make nums[i..j] the same.
    int getSeconds(int i, int j) {
        return (i - j) / 2;
    }
};
```

1901. 2809-2.cpp

Path: LeetCode-main/solutions/2809. Minimum Time to Make Array Sum At Most x/2809-2.cpp

```
class Solution {
public:
    int minimumTime(vector<int>& nums1, vector<int>& nums2, int x) {
        const int n = nums1.size();
        const int sum1 = accumulate(nums1.begin(), nums1.end(), 0);
        const int sum2 = accumulate(nums2.begin(), nums2.end(), 0);
        // dp[j] := the maximum reduced value if we do j operations on the numbers
        // so far
        vector<int> dp(n + 1);
        vector<pair<int, int>> sortedNums;

        for (int i = 0; i < n; ++i)
            sortedNums.emplace_back(nums2[i], nums1[i]);

        ranges::sort(sortedNums);

        for (int i = 1; i <= n; ++i) {
            const auto [num2, num1] = sortedNums[i - 1];
            for (int j = i; j > 0; --j)
                dp[j] = max(
                    // the maximum reduced value if we do j operations on the first
                    // i - 1 numbers
                    dp[j],
                    // the maximum reduced value if we do j - 1 operations on the first
                    // i - 1 numbers + making the i-th number of `nums1` to 0 at the
                    // j-th operation
                    dp[j - 1] + num2 * j + num1);
        }

        for (int op = 0; op <= n; ++op)
            if (sum1 + sum2 * op - dp[op] <= x)
                return op;

        return -1;
    }
};
```

1902. 2809.cpp

Path: LeetCode-main/solutions/2809. Minimum Time to Make Array Sum At Most x/2809.cpp

```
class Solution {
public:
    int minimumTime(vector<int>& nums1, vector<int>& nums2, int x) {
        const int n = nums1.size();
        const int sum1 = accumulate(nums1.begin(), nums1.end(), 0);
        const int sum2 = accumulate(nums2.begin(), nums2.end(), 0);
        // dp[i][j] := the maximum reduced value if we do j operations on the first
        // i numbers
        vector<vector<int>> dp(n + 1, vector<int>(n + 1));
        vector<pair<int, int>> sortedNums;

        for (int i = 0; i < n; ++i)
            sortedNums.emplace_back(nums2[i], nums1[i]);

        ranges::sort(sortedNums);

        for (int i = 1; i <= n; ++i) {
            const auto [num2, num1] = sortedNums[i - 1];
            for (int j = 1; j <= i; ++j)
                dp[i][j] = max(
                    // the maximum reduced value if we do j operations on the first
                    // i - 1 numbers
                    dp[i - 1][j],
                    // the maximum reduced value if we do j - 1 operations on the first
                    // i - 1 numbers + making the i-th number of `nums1` to 0 at the
                    // j-th operation
                    dp[i - 1][j - 1] + num2 * j + num1);
        }

        for (int op = 0; op <= n; ++op)
            if (sum1 + sum2 * op - dp[n][op] <= x)
                return op;

        return -1;
    }
};
```

1903. 281.cpp

Path: LeetCode-main/solutions/281. Zigzag Iterator/281.cpp

```
class ZigzagIterator {
public:
    ZigzagIterator(vector<int>& v1, vector<int>& v2) {
        if (!v1.empty())
            q.emplace(v1.begin(), v1.end());
        if (!v2.empty())
            q.emplace(v2.begin(), v2.end());
    }

    int next() {
        const auto [it, endIt] = q.front();
        q.pop();
        if (it + 1 != endIt)
            q.emplace(it + 1, endIt);
        return *it;
    }

    bool hasNext() {
        return !q.empty();
    }

private:
    // {{ it, endIt }}
    queue<pair<vector<int>::iterator, vector<int>::iterator>> q;
};
```

1904. 2810.cpp

Path: LeetCode-main/solutions/2810. Faulty Keyboard/2810.cpp

```
class Solution {
public:
    string finalString(string s) {
        deque<char> dq;
        bool inversed = false;

        for (const char c : s)
            if (c == 'i')
                inversed = !inversed;
            else if (inversed)
                dq.push_front(c);
            else
                dq.push_back(c);

        return inversed ? string{dq.rbegin(), dq.rend()}
            : string{dq.begin(), dq.end()};
    }
};
```

1905. 2811.cpp

Path: LeetCode-main/solutions/2811. Check if it is Possible to Split Array/2811.cpp

```
class Solution {
public:
    bool canSplitArray(vector<int>& nums, int m) {
        if (nums.size() < 3)
            return true;

        for (int i = 1; i < nums.size(); ++i)
            if (nums[i] + nums[i - 1] >= m)
                return true;

        return false;
    }
};
```

1906. 2812.cpp

Path: LeetCode-main/solutions/2812. Find the Safest Path in a Grid/2812.cpp

```
class Solution {
public:
    int maximumSafenessFactor(vector<vector<int>>& grid) {
        const vector<vector<int>> distToThief = getDistToThief(grid);
        int l = 0;
        int r = grid.size() * 2;

        while (l < r) {
            const int m = (l + r) / 2;
            if (hasValidPath(distToThief, m))
                l = m + 1;
            else
                r = m;
        }

        return l - 1;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    bool hasValidPath(const vector<vector<int>>& distToThief, int safeness) {
        if (distToThief[0][0] < safeness)
            return false;

        const int n = distToThief.size();
        queue<pair<int, int>> q{{{0, 0}}};
        vector<vector<bool>> seen(n, vector<bool>(n));
        seen[0][0] = true;

        while (!q.empty()) {
            const auto [i, j] = q.front();
            q.pop();
            if (distToThief[i][j] < safeness)
                continue;
            if (i == n - 1 && j == n - 1)
                return true;
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == n || y < 0 || y == n)
                    continue;
                if (seen[x][y])
                    continue;
                q.emplace(x, y);
                seen[x][y] = true;
            }
        }

        return false;
    }

    vector<vector<int>> getDistToThief(const vector<vector<int>>& grid) {
        const int n = grid.size();
        vector<vector<int>> distToThief(n, vector<int>(n));
        queue<pair<int, int>> q;
        vector<vector<bool>> seen(n, vector<bool>(n));

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1) {
                    q.emplace(i, j);
                    seen[i][j] = true;
                }

        for (int dist = 0; !q.empty(); ++dist) {
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                distToThief[i][j] = dist;
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == n || y < 0 || y == n)
                        continue;
                    if (seen[x][y])
                        continue;
                    q.emplace(x, y);
                    seen[x][y] = true;
                }
            }
        }
    }
}
```

```
    }  
    return distToThief;  
}  
};
```


1907. 2813.cpp

Path: LeetCode-main/solutions/2813. Maximum Elegance of a K-Length Subsequence/2813.cpp

```
class Solution {
public:
    long long findMaximumElegance(vector<vector<int>>& items, int k) {
        long ans = 0;
        long totalProfit = 0;
        unordered_set<int> seenCategories;
        stack<int> decreasingDuplicateProfits;

        ranges::sort(items, greater<>());

        for (int i = 0; i < k; i++) {
            const int profit = items[i][0];
            const int category = items[i][1];
            totalProfit += profit;
            if (seenCategories.contains(category))
                decreasingDuplicateProfits.push(profit);
            else
                seenCategories.insert(category);
        }

        ans = totalProfit +
            static_cast<long>(seenCategories.size()) * seenCategories.size();

        for (int i = k; i < items.size(); ++i) {
            const int profit = items[i][0];
            const int category = items[i][1];
            if (!seenCategories.contains(category) &&
                !decreasingDuplicateProfits.empty()) {
                // If this is a new category we haven't seen before, it's worth
                // considering taking it and replacing the one with the least profit
                // since it will increase the distinct_categories and potentially result
                // in a larger total_profit + distinct_categories^2.
                totalProfit -= decreasingDuplicateProfits.top();
                decreasingDuplicateProfits.pop();
                totalProfit += profit;
                seenCategories.insert(category);
                ans = max(ans,
                    static_cast<long>(totalProfit +
                        static_cast<long>(seenCategories.size()) *
                        seenCategories.size()));
            }
        }

        return ans;
    }
};
```

1908. 2814.cpp

Path: LeetCode-main/solutions/2814. Minimum Time Takes to Reach Destination Without Drowning/2814.cpp

```
class Solution {
public:
    int minimumSeconds(vector<vector<string>>& land) {
        const int m = land.size();
        const int n = land[0].size();
        const vector<vector<int>> floodDist = getFloodDist(land);
        queue<pair<int, int>> q;
        vector<vector<bool>> seen(m, vector<bool>(n));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (land[i][j] == "S") {
                    q.emplace(i, j);
                    seen[i][j] = true;
                }

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (land[x][y] == "D")
                        return step;
                    if (floodDist[x][y] <= step || land[x][y] == "X" || seen[x][y])
                        continue;
                    q.emplace(x, y);
                    seen[x][y] = true;
                }
            }

        return -1;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    vector<vector<int>> getFloodDist(const vector<vector<string>>& land) {
        const int m = land.size();
        const int n = land[0].size();
        vector<vector<int>> dist(m, vector<int>(n, INT_MAX));
        queue<pair<int, int>> q;
        vector<vector<bool>> seen(m, vector<bool>(n));

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (land[i][j] == "*") {
                    q.emplace(i, j);
                    seen[i][j] = true;
                }

        for (int d = 0; !q.empty(); ++d)
            for (int sz = q.size(); sz > 0; --sz) {
                const auto [i, j] = q.front();
                q.pop();
                dist[i][j] = d;
                for (const auto& [dx, dy] : kDirs) {
                    const int x = i + dx;
                    const int y = j + dy;
                    if (x < 0 || x == m || y < 0 || y == n)
                        continue;
                    if (land[x][y] == "X" || land[x][y] == "D" || seen[x][y])
                        continue;
                    q.emplace(x, y);
                    seen[x][y] = true;
                }
            }

        return dist;
    }
};
```

1909. 2815.cpp

Path: LeetCode-main/solutions/2815. Max Pair Sum in an Array/2815.cpp

```
class Solution {
public:
    int maxSum(vector<int>& nums) {
        int ans = 0;
        // maxNum[i] := the maximum num we met so far with the maximum digit i
        vector<int> maxNum(10);

        for (const int num : nums) {
            const int d = getMaxDigit(num);
            if (maxNum[d] > 0)
                ans = max(ans, num + maxNum[d]);
            maxNum[d] = max(maxNum[d], num);
        }

        return ans == 0 ? -1 : ans;
    }

private:
    int getMaxDigit(int num) {
        int maxDigit = 0;
        while (num > 0) {
            maxDigit = max(maxDigit, num % 10);
            num /= 10;
        }
        return maxDigit;
    }
};
```

1910. 2816-2.cpp

Path: LeetCode-main/solutions/2816. Double a Number Represented as a Linked List/2816-2.cpp

```
class Solution {
public:
    ListNode* doubleIt(ListNode* head) {
        if (head->val >= 5)
            head = new ListNode(0, head);

        for (ListNode* curr = head; curr != nullptr; curr = curr->next) {
            curr->val *= 2;
            curr->val %= 10;
            if (curr->next && curr->next->val >= 5)
                ++curr->val;
        }

        return head;
    }
};
```

1911. 2816.cpp

Path: LeetCode-main/solutions/2816. Double a Number Represented as a Linked List/2816.cpp

```
class Solution {
public:
    ListNode* doubleIt(ListNode* head) {
        if (getCarry(head) == 1)
            return new ListNode(1, head);
        return head;
    }

private:
    int getCarry(ListNode* node) {
        int val = node->val * 2;
        if (node->next != nullptr)
            val += getCarry(node->next);

        node->val = val % 10;
        return val / 10;
    }
};
```

1912. 2817.cpp

Path: LeetCode-main/solutions/2817. Minimum Absolute Difference Between Elements With Constraint/2817.cpp

```
class Solution {
public:
    int minAbsoluteDifference(vector<int>& nums, int x) {
        int ans = INT_MAX;
        set<int> seen;

        for (int i = x; i < nums.size(); ++i) {
            seen.insert(nums[i - x]);
            // `upper_bound` works as well.
            const auto it = seen.lower_bound(nums[i]);
            if (it != seen.cend())
                ans = min(ans, *it - nums[i]);
            if (it != seen.cbegin())
                ans = min(ans, nums[i] - *prev(it));
        }

        return ans;
    }
};
```

1913. 2818.cpp

Path: LeetCode-main/solutions/2818. Apply Operations to Maximize Score/2818.cpp

```
class Solution {
public:
    int maximumScore(vector<int>& nums, int k) {
        const int n = nums.size();
        const int mx = ranges::max(nums);
        const vector<int> minPrimeFactors = sieveEratosthenes(mx + 1);
        const vector<int> primeScores = getPrimeScores(nums, minPrimeFactors);
        int ans = 1;
        // left[i] := the next index on the left (if any) s.t.
        // primeScores[left[i]] >= primeScores[i]
        vector<int> left(n, -1);
        // right[i] := the next index on the right (if any) s.t.
        // primeScores[right[i]] > primeScores[i]
        vector<int> right(n, n);
        stack<int> stack;

        // Find the next indices on the left where `primeScores` are greater or
        // equal.
        for (int i = n - 1; i >= 0; --i) {
            while (!stack.empty() && primeScores[stack.top()] <= primeScores[i])
                left[stack.top()] = i, stack.pop();
            stack.push(i);
        }

        stack = std::stack<int>();

        // Find the next indices on the right where `primeScores` are greater.
        for (int i = 0; i < n; ++i) {
            while (!stack.empty() && primeScores[stack.top()] < primeScores[i])
                right[stack.top()] = i, stack.pop();
            stack.push(i);
        }

        vector<pair<int, int>> numAndIndexes;

        for (int i = 0; i < n; ++i)
            numAndIndexes.emplace_back(nums[i], i);

        ranges::sort(numAndIndexes,
            [&](const pair<int, int>& a, const pair<int, int>& b) {
                return a.first == b.first ? a.second < b.second : a.first > b.first;
            });

        for (const auto& [num, i] : numAndIndexes) {
            // nums[i] is the maximum value in the range [left[i] + 1, right[i] - 1]
            // So, there are (i - left[i]) * (right[i] - 1) ranges where nums[i] will
            // be chosen.
            const long rangeCount = static_cast<long>(i - left[i]) * (right[i] - i);
            const long actualCount = min(rangeCount, static_cast<long>(k));
            k -= actualCount;
            ans = static_cast<long>(ans) * modPow(num, actualCount) % kMod;
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }

    // Gets the minimum prime factor of i, where 1 < i <= n.
    vector<int> sieveEratosthenes(int n) {
        vector<int> minPrimeFactors(n + 1);
        iota(minPrimeFactors.begin() + 2, minPrimeFactors.end(), 2);
        for (int i = 2; i * i < n; ++i)
            if (minPrimeFactors[i] == i) // `i` is prime.
                for (int j = i * i; j < n; j += i)
                    minPrimeFactors[j] = min(minPrimeFactors[j], i);
        return minPrimeFactors;
    }

    vector<int> getPrimeScores(const vector<int>& nums,
        const vector<int>& minPrimeFactors) {
```

```

    vector<int> primeScores;
    for (const int num : nums)
        primeScores.push_back(getPrimeScore(num, minPrimeFactors));
    return primeScores;
}

int getPrimeScore(int num, const vector<int>& minPrimeFactors) {
    unordered_set<int> primeFactors;
    while (num > 1) {
        const int divisor = minPrimeFactors[num];
        primeFactors.insert(divisor);
        while (num % divisor == 0)
            num /= divisor;
    }
    return primeFactors.size();
}
};

```


1914. 2819.cpp

Path: LeetCode-main/solutions/2819. Minimum Relative Loss After Buying Chocolates/2819.cpp

```
class Solution {
public:
    vector<long long> minimumRelativeLosses(vector<int>& prices,
                                           vector<vector<int>>& queries) {
        const int n = prices.size();
        vector<long long> ans;
        vector<long long> prefix{0};

        ranges::sort(prices);

        for (const int price : prices)
            prefix.push_back(prefix.back() + price);

        for (const vector<int>& query : queries) {
            const int k = query[0];
            const int m = query[1];
            const int countFront = getCountFront(k, m, prices);
            const int countBack = m - countFront;
            ans.push_back(getRelativeLoss(countFront, countBack, k, prefix));
        }

        return ans;
    }

private:
    // Returns `countFront` for query (k, m) s.t. picking the first `countFront`
    // and the last `m - countFront` chocolates is optimal.
    //
    // Define loss[i] := the relative loss of picking `prices[i]`.
    // 1. For prices[i] <= k, Bob pays prices[i] while Alice pays 0.
    //    Thus, loss[i] = prices[i] - 0 = prices[i].
    // 2. For prices[i] > k, Bob pays k while Alice pays prices[i] - k.
    //    Thus, loss[i] = k - (prices[i] - k) = 2 * k - prices[i].
    // By observation, we deduce that it is always better to pick from the front
    // or the back since loss[i] is increasing for 1. and is decreasing for 2.
    //
    // Assume that picking `left` chocolates from the left and `right = m - left`
    // chocolates from the right is optimal. Therefore, we are selecting
    // chocolates from `prices[0..left - 1]` and `prices[n - right..n - 1]`.
    //
    // To determine the optimal `left` in each iteration, we simply compare
    // `loss[left]` with `loss[n - right]`; if `loss[left] < loss[n - right]`,
    // it's worth increasing `left`.
    int getCountFront(int k, int m, const vector<int>& prices) {
        const int n = prices.size();
        const int countNoGreaterThank =
            ranges::upper_bound(prices, k) - prices.begin();
        int l = 0;
        int r = min(countNoGreaterThank, m);

        while (l < r) {
            const int mid = (l + r) / 2;
            const int right = m - mid;
            // Picking prices[mid] is better than picking prices[n - right].
            if (prices[mid] < 2L * k - prices[n - right])
                l = mid + 1;
            else
                r = mid;
        }

        return l;
    }

    // Returns the relative loss of picking `countFront` and `countBack`
    // chocolates.
    long getRelativeLoss(int countFront, int countBack, int k,
                        const vector<long long>& prefix) {
        const long lossFront = prefix[countFront];
        const long lossBack =
            2L * k * countBack -
            (prefix.back() - prefix[prefix.size() - 1 - countBack]);
        return lossFront + lossBack;
    }
};
```

1915. 282.cpp

Path: LeetCode-main/solutions/282. Expression Add Operators/282.cpp

```
class Solution {
public:
    vector<string> addOperators(string num, int target) {
        vector<string> ans;
        dfs(num, target, 0, 0, 0, {}, ans);
        return ans;
    }

private:
    string join(const vector<string>& path) {
        string joined;
        for (const string& s : path)
            joined += s;
        return joined;
    }

    void dfs(const string& num, int target, int start, long prev, long eval,
            vector<string>&& path, vector<string>& ans) {
        if (start == num.length()) {
            if (eval == target)
                ans.push_back(join(path));
            return;
        }

        for (int i = start; i < num.length(); ++i) {
            if (i > start && num[start] == '0')
                return;
            const string& s = num.substr(start, i - start + 1);
            const long curr = stol(s);
            if (start == 0) {
                path.push_back(s);
                dfs(num, target, i + 1, curr, curr, std::move(path), ans);
                path.pop_back();
            } else {
                for (const string& op : {"+", "-", "*"}) {
                    path.push_back(op + s);
                    if (op == "+")
                        dfs(num, target, i + 1, curr, eval + curr, std::move(path), ans);
                    else if (op == "-")
                        dfs(num, target, i + 1, -curr, eval - curr, std::move(path), ans);
                    else
                        dfs(num, target, i + 1, prev * curr, eval - prev + prev * curr,
                                std::move(path), ans);
                    path.pop_back();
                }
            }
        }
    }
};
```


1917. 2825.cpp

Path: LeetCode-main/solutions/2825. Make String a Subsequence Using Cyclic Increments/2825.cpp

```
class Solution {
public:
    bool canMakeSubsequence(string str1, string str2) {
        int i = 0; // str2's index

        for (const char c : str1)
            if (c == str2[i] || ('a' + ((c - 'a' + 1) % 26)) == str2[i])
                if (++i == str2.length())
                    return true;

        return false;
    }
};
```

1918. 2826.cpp

Path: LeetCode-main/solutions/2826. Sorting Three Groups/2826.cpp

```
class Solution {
public:
    int minimumOperations(vector<int>& nums) {
        // dp[i] := the longest non-decreasing subsequence so far with numbers in
        // [1..i]
        vector<int> dp(4);

        for (const int num : nums) {
            ++dp[num];
            dp[2] = max(dp[2], dp[1]);
            dp[3] = max(dp[3], dp[2]);
        }

        return nums.size() - dp[3];
    }
};
```

1919. 2827.cpp

Path: LeetCode-main/solutions/2827. Number of Beautiful Integers in the Range/2827.cpp

```
class Solution {
public:
    int numberOfBeautifulIntegers(int low, int high, int k) {
        const string lowString = to_string(low);
        const string highString = to_string(high);
        const string lowWithLeadingZeros =
            string(highString.length() - lowString.length(), '0') + lowString;
        vector<vector<vector<vector<vector<int>>>>>>> mem(
            highString.length(),
            vector<vector<vector<vector<vector<int>>>>>>>(
                10, vector<vector<vector<vector<int>>>>>(
                    10, vector<vector<vector<int>>>>(
                        k, vector<vector<int>>>(2, vector<int>(2, -1))))));
        return count(lowWithLeadingZeros, highString, k, 0, 0, 0, 0, true, true,
            true, mem);
    }

private:
    // Returns the number of beautiful integers, considering the i-th digit with
    // counts of even `even` digits and odd `odd` digits, where the current number
    // modulo k equals remainder, `tight1` indicates if the current digit is
    // tightly bound for `low` and `tight2` indicates if the current digit is
    // tightly bound for `high`
    int count(const string& low, const string& high, int k, int i, int even,
        int odd, int remainder, bool isLeadingZero, bool tight1,
        bool tight2,
        vector<vector<vector<vector<vector<vector<int>>>>>>>& mem) {
        if (i == high.length())
            return !isLeadingZero && even == odd && remainder == 0;
        if (mem[i][even][odd][remainder][tight1][tight2] != -1)
            return mem[i][even][odd][remainder][tight1][tight2];

        int res = 0;
        const int minDigit = tight1 ? low[i] - '0' : 0;
        const int maxDigit = tight2 ? high[i] - '0' : 9;

        for (int d = minDigit; d <= maxDigit; ++d) {
            const int nextEven = even + ((!isLeadingZero || d > 0) && d % 2 == 0);
            const int nextOdd = odd + (d % 2 == 1);
            const int nextRemainder = (remainder * 10 + d) % k;
            const bool nextTight1 = tight1 && (d == minDigit);
            const bool nextTight2 = tight2 && (d == maxDigit);
            res += count(low, high, k, i + 1, nextEven, nextOdd, nextRemainder,
                isLeadingZero && d == 0, nextTight1, nextTight2, mem);
        }

        return mem[i][even][odd][remainder][tight1][tight2] = res;
    }
};
```

1920. 2828.cpp

Path: LeetCode-main/solutions/2828. Check if a String Is an Acronym of Words/2828.cpp

```
class Solution {
public:
    bool isAcronym(vector<string>& words, string s) {
        if (words.size() != s.length())
            return false;

        for (int i = 0; i < words.size(); ++i)
            if (words[i][0] != s[i])
                return false;

        return true;
    }
};
```

1921. 2829.cpp

Path: LeetCode-main/solutions/2829. Determine the Minimum Sum of a k-avoiding Array/2829.cpp

```
class Solution {
public:
    int minimumSum(int n, int k) {
        // These are the unique pairs that sum up to k:
        // (1, k - 1), (2, k - 2), ..., (ceil(k / 2), floor(k / 2)).
        // Our optimal strategy is to select 1, 2, ..., floor(k / 2), and then
        // choose k, k + 1, ... if necessary, as selecting any number in the range
        // [ceil(k / 2), k - 1] will result in a pair summing up to k.
        const int mid = k / 2; // floor(k / 2)
        if (n <= mid)
            return trapezoid(1, n);
        return trapezoid(1, mid) + trapezoid(k, k + (n - mid - 1));
    }

private:
    // Returns sum(a..b).
    int trapezoid(int a, int b) {
        return (a + b) * (b - a + 1) / 2;
    }
};
```


1922. 283.cpp

Path: LeetCode-main/solutions/283. Move Zeroes/283.cpp

```
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int i = 0;
        for (const int num : nums)
            if (num != 0)
                nums[i++] = num;

        while (i < nums.size())
            nums[i++] = 0;
    }
};
```

1923. 2830.cpp

Path: LeetCode-main/solutions/2830. Maximize the Profit as the Salesman/2830.cpp

```
class Solution {
public:
    int maximizeTheProfit(int n, vector<vector<int>>& offers) {
        // dp[i] := the maximum amount of gold of selling the first i houses
        vector<int> dp(n + 1);
        vector<vector<pair<int, int>>> endToStartAndGolds(n);

        for (const vector<int>& offer : offers) {
            const int start = offer[0];
            const int end = offer[1];
            const int gold = offer[2];
            endToStartAndGolds[end].emplace_back(start, gold);
        }

        for (int end = 1; end <= n; ++end) {
            // Get at least the same gold as selling the first `end - 1` houses.
            dp[end] = dp[end - 1];
            for (const auto& [start, gold] : endToStartAndGolds[end - 1])
                dp[end] = max(dp[end], dp[start] + gold);
        }

        return dp[n];
    }
};
```

1924. 2831-2.cpp

Path: LeetCode-main/solutions/2831. Find the Longest Equal Subarray/2831-2.cpp

```
class Solution {
public:
    int longestEqualSubarray(vector<int>& nums, int k) {
        int ans = 0;
        unordered_map<int, int> count;

        // l and r track the maximum window instead of the valid window.
        for (int l = 0, r = 0; r < nums.size(); ++r) {
            ans = max(ans, ++count[nums[r]]);
            if (r - l + 1 - k > ans)
                --count[nums[l++]];
        }

        return ans;
    }
};
```

1925. 2831.cpp

Path: LeetCode-main/solutions/2831. Find the Longest Equal Subarray/2831.cpp

```
class Solution {
public:
    int longestEqualSubarray(vector<int>& nums, int k) {
        int ans = 0;
        unordered_map<int, int> count;

        for (int l = 0, r = 0; r < nums.size(); ++r) {
            ans = max(ans, ++count[nums[r]]);
            while (r - l + 1 - k > ans)
                --count[nums[l++]];
        }

        return ans;
    }
};
```

1926. 2832.cpp

Path: LeetCode-main/solutions/2832. Maximal Range That Each Element Is Maximum in It/2832.cpp

```
class Solution {
public:
    vector<int> maximumLengthOfRanges(vector<int>& nums) {
        vector<int> ans(nums.size());
        stack<int> stack; // a decreasing stack

        for (int i = 0; i <= nums.size(); ++i) {
            while (!stack.empty() &&
                (i == nums.size() || nums[stack.top()] < nums[i])) {
                const int index = stack.top();
                stack.pop();
                const int left = stack.empty() ? -1 : stack.top();
                ans[index] = i - left - 1;
            }
            stack.push(i);
        }

        return ans;
    }
};
```

1927. 2833.cpp

Path: LeetCode-main/solutions/2833. Furthest Point From Origin/2833.cpp

```
class Solution {
public:
    int furthestDistanceFromOrigin(string moves) {
        int countL = 0;
        int countR = 0;
        int countUnderline = 0;

        for (const char c : moves)
            if (c == 'L')
                ++countL;
            else if (c == 'R')
                ++countR;
            else // c == '_'
                ++countUnderline;

        return abs(countL - countR) + countUnderline;
    }
};
```

1928. 2834.cpp

Path: LeetCode-main/solutions/2834. Find the Minimum Possible Sum of a Beautiful Array/2834.cpp

```
class Solution {
public:
    // Same as 2829. Determine the Minimum Sum of a k-avoiding Array
    int minimumPossibleSum(int n, int target) {
        // These are the unique pairs that sum up to target (k):
        // (1, k - 1), (2, k - 2), ..., (ceil(k / 2), floor(k / 2)).
        // Our optimal strategy is to select 1, 2, ..., floor(k / 2), and then
        // choose k, k + 1, ... if necessary, as selecting any number in the range
        // [ceil(k / 2), k - 1] will result in a pair summing up to k.
        const int mid = target / 2;
        if (n <= mid)
            return trapezoid(1, n);
        return (trapezoid(1, mid) + trapezoid(target, target + (n - mid - 1))) %
            kMod;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    // Returns sum(a..b).
    long trapezoid(long a, long b) {
        return (a + b) * (b - a + 1) / 2;
    }
};
```

1929. 2835.cpp

Path: LeetCode-main/solutions/2835. Minimum Operations to Form Subsequence With Target Sum/2835.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums, int target) {
        constexpr int kNoMissingBit = 31;
        constexpr int maxBit = 31;
        int ans = 0;
        int minMissingBit = kNoMissingBit;
        // count[i] := the number of occurrences of 2^i
        vector<int> count(maxBit + 1);

        for (const int num : nums)
            ++count[static_cast<int>(log2(num))];

        for (int bit = 0; bit < maxBit; ++bit) {
            // Check if `bit` is in the target.
            if (target >> bit & 1) {
                // If there are available bits, use one bit.
                if (count[bit] > 0)
                    --count[bit];
                else
                    minMissingBit = min(minMissingBit, bit);
            }
            // If we previously missed a bit and there are available bits.
            if (minMissingBit != kNoMissingBit && count[bit] > 0) {
                --count[bit];
                // Count the operations to break `bit` into `minMissingBit`.
                ans += bit - minMissingBit;
                minMissingBit = kNoMissingBit;
            }
            // Combining smaller numbers costs nothing.
            count[bit + 1] += count[bit] / 2;
        }

        // Check if all target bits have been covered, otherwise return -1.
        return minMissingBit == kNoMissingBit ? ans : -1;
    }
};
```


1930. 2836.cpp

Path: LeetCode-main/solutions/2836. Maximize Value of Function in a Ball Passing Game/2836.cpp

```
class Solution {
public:
    long long getMaxFunctionValue(vector<int>& receiver, long long k) {
        const int n = receiver.size();
        const int m = log2(k) + 1;
        long ans = 0;
        // jump[i][j] := the the node you reach after jumping  $2^j$  steps from i
        vector<vector<int>> jump(n, vector<int>(m));
        // sum[i][j] := the sum of the first  $2^j$  nodes you reach when jumping from i
        vector<vector<long>> sum(n, vector<long>(m));

        for (int i = 0; i < n; ++i) {
            jump[i][0] = receiver[i];
            sum[i][0] = receiver[i];
        }

        // Calculate binary lifting.
        for (int j = 1; j < m; ++j)
            for (int i = 0; i < n; ++i) {
                const int midNode = jump[i][j - 1];
                // the the node you reach after jumping  $2^j$  steps from i
                // = the node you reach after jumping  $2^{(j-1)}$  steps from i
                // + the node you reach after jumping another  $2^{(j-1)}$  steps
                jump[i][j] = jump[midNode][j - 1];
                // the sum of the first  $2^j$  nodes you reach when jumping from i
                // = the sum of the first  $2^{(j-1)}$  nodes you reach when jumping from i
                // + the sum of another  $2^{(j-1)}$  nodes you reach
                sum[i][j] = sum[i][j - 1] + sum[midNode][j - 1];
            }

        for (int i = 0; i < n; ++i) {
            long currSum = i;
            int currPos = i;
            for (int j = 0; j < m; ++j)
                if (k >> j & 1) {
                    currSum += sum[currPos][j];
                    currPos = jump[currPos][j];
                }
            ans = max(ans, currSum);
        }

        return ans;
    }
};
```

1931. 2838.cpp

Path: LeetCode-main/solutions/2838. Maximum Coins Heroes Can Collect/2838.cpp

```
class Solution {
public:
    vector<long long> maximumCoins(vector<int>& heroes, vector<int>& monsters,
                                   vector<int>& coins) {
        const vector<pair<int, int>> monsterAndCoins =
            getSortedMonsterAndCoins(monsters, coins);
        vector<long long> ans;
        vector<long long> coinsPrefix{0};

        for (const auto& [_, coin] : monsterAndCoins)
            coinsPrefix.push_back(coinsPrefix.back() + coin);

        for (const int hero : heroes)
            ans.push_back(coinsPrefix[firstGreaterEqual(monsterAndCoins, hero)]);

        return ans;
    }

private:
    vector<pair<int, int>> getSortedMonsterAndCoins(const vector<int>& monsters,
                                                    const vector<int>& coins) {
        vector<pair<int, int>> monsterAndCoins;
        for (int i = 0; i < monsters.size(); ++i)
            monsterAndCoins.emplace_back(monsters[i], coins[i]);
        ranges::sort(monsterAndCoins);
        return monsterAndCoins;
    }

    int firstGreaterEqual(const vector<pair<int, int>>& monsterAndCoins,
                          int hero) {
        int l = 0;
        int r = monsterAndCoins.size();
        while (l < r) {
            const int m = (l + r) / 2;
            if (monsterAndCoins[m].first > hero)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
};
```

1932. 2839.cpp

Path: LeetCode-main/solutions/2839. Check if Strings Can be Made Equal With Operations I/2839.cpp

```
class Solution {
public:
    bool canBeEqual(string s1, string s2) {
        for (const string& a : swappedStrings(s1))
            for (const string& b : swappedStrings(s2))
                if (a == b)
                    return true;
        return false;
    }

private:
    vector<string> swappedStrings(const string& s) {
        vector<char> chars(s.begin(), s.end());
        return {s, string({chars[2], chars[1], chars[0], chars[3]}),
                string({chars[0], chars[3], chars[2], chars[1]}),
                string({chars[2], chars[3], chars[0], chars[1]}));
    }
};
```

1933. 284.cpp

Path: LeetCode-main/solutions/284. Peeking Iterator/284.cpp

```
class PeekingIterator : public Iterator {
public:
    PeekingIterator(const vector<int>& nums) : Iterator(nums) {}

    // Returns the next element in the iteration without advancing the iterator.
    int peek() {
        // Iterator(*this) makes a copy of current iterator, then call next on the
        // Copied iterator to get the next value without affecting current iterator
        return Iterator(*this).next();
    }

    // hasNext() and next() should behave the same as in the Iterator interface.
    // Override them if needed.
    int next() {
        return Iterator::next();
    }

    bool hasNext() const {
        return Iterator::hasNext();
    }
};
```

1934. 2840.cpp

Path: LeetCode-main/solutions/2840. Check if Strings Can be Made Equal With Operations II/2840.cpp

```
class Solution {
public:
    bool checkStrings(string s1, string s2) {
        vector<vector<int>> count(2, vector<int>(26));

        for (int i = 0; i < s1.length(); ++i) {
            ++count[i % 2][s1[i] - 'a'];
            --count[i % 2][s2[i] - 'a'];
        }

        for (int i = 0; i < 26; ++i)
            if (count[0][i] > 0 || count[1][i] > 0)
                return false;

        return true;
    }
};
```

1935. 2841.cpp

Path: LeetCode-main/solutions/2841. Maximum Sum of Almost Unique Subarray/2841.cpp

```
class Solution {
public:
    long long maxSum(vector<int>& nums, int m, int k) {
        long ans = 0;
        long sum = 0;
        unordered_map<int, int> count;

        for (int i = 0; i < nums.size(); ++i) {
            sum += nums[i];
            ++count[nums[i]];
            if (i >= k) {
                const int numToRemove = nums[i - k];
                sum -= numToRemove;
                if (--count[numToRemove] == 0)
                    count.erase(numToRemove);
            }
            if (count.size() >= m)
                ans = max(ans, sum);
        }

        return ans;
    }
};
```

1936. 2842.cpp

Path: LeetCode-main/solutions/2842. Count K-Subsequences of a String With Maximum Beauty Solved/2842.cpp

```
class Solution {
public:
    int countKSubsequencesWithMaxBeauty(string s, int k) {
        unordered_map<char, int> count;
        for (const char c : s)
            ++count[c];
        if (count.size() < k)
            return 0;

        long ans = 1;

        for (const auto& [fc, numOfChars] : getFreqCountPairs(count)) {
            if (numOfChars >= k) {
                ans *= nCk(numOfChars, k);
                ans %= kMod;
                return ans * modPow(fc, k) % kMod;
            }
            ans *= modPow(fc, numOfChars);
            ans %= kMod;
            k -= numOfChars;
        }

        return ans;
    }

private:
    static constexpr int kMod = 1'000'000'007;

    vector<pair<int, int>> getFreqCountPairs(
        const unordered_map<char, int>& count) {
        unordered_map<int, int> freqCount;
        for (const auto& [_, value] : count)
            ++freqCount[value];
        vector<pair<int, int>> freqCountPairs;
        for (const auto& [fc, numOfChars] : freqCount)
            freqCountPairs.emplace_back(fc, numOfChars);
        ranges::sort(freqCountPairs, ranges::greater{},
            [](const pair<int, int>& freqCountPair) {
                return freqCountPair.first;
            });
        return freqCountPairs;
    }

    long nCk(int n, int k) {
        long res = 1;
        for (int i = 1; i <= k; ++i)
            res = res * (n - i + 1) / i;
        return res;
    }

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x % kMod, (n - 1)) % kMod;
        return modPow(x * x % kMod, (n / 2)) % kMod;
    }
};
```

1937. 2843.cpp

Path: LeetCode-main/solutions/2843. Count Symmetric Integers/2843.cpp

```
class Solution {
public:
    int countSymmetricIntegers(int low, int high) {
        int ans = 0;

        for (int num = low; num <= high; ++num)
            if (isSymmetricInteger(num))
                ++ans;

        return ans;
    }

private:
    bool isSymmetricInteger(int num) {
        if (num >= 10 && num <= 99)
            return num / 10 == num % 10;
        if (num >= 1000 && num <= 9999) {
            const int left = num / 100;
            const int right = num % 100;
            return left / 10 + left % 10 == right / 10 + right % 10;
        }
        return false;
    }
};
```


1938. 2844.cpp

Path: LeetCode-main/solutions/2844. Minimum Operations to Make a Special Number/2844.cpp

```
class Solution {
public:
    int minimumOperations(string num) {
        const int n = num.length();
        bool seenFive = false;
        bool seenZero = false;

        for (int i = n - 1; i >= 0; --i) {
            if (seenZero && num[i] == '0') // '00'
                return n - i - 2;
            if (seenZero && num[i] == '5') // '50'
                return n - i - 2;
            if (seenFive && num[i] == '2') // '25'
                return n - i - 2;
            if (seenFive && num[i] == '7') // '75'
                return n - i - 2;
            seenZero = seenZero || num[i] == '0';
            seenFive = seenFive || num[i] == '5';
        }

        return seenZero ? n - 1 : n;
    }
};
```

1939. 2845.cpp

Path: LeetCode-main/solutions/2845. Count of Interesting Subarrays/2845.cpp

```
class Solution {
public:
    long countInterestingSubarrays(vector<int>& nums, int modulo, int k) {
        long ans = 0;
        int prefix = 0; // (number of nums[i] % modulo == k so far) % modulo
        unordered_map<int, int> prefixCount{{0, 1}};

        for (const int num : nums) {
            if (num % modulo == k)
                prefix = (prefix + 1) % modulo;
            ans += prefixCount[(prefix - k + modulo) % modulo];
            ++prefixCount[prefix];
        }

        return ans;
    }
};
```

1940. 2846.cpp

Path: LeetCode-main/solutions/2846. Minimum Edge Weight Equilibrium Queries in a Tree/2846.cpp

```
class Solution {
public:
    vector<int> minOperationsQueries(int n, vector<vector<int>>& edges,
                                   vector<vector<int>>& queries) {
        constexpr int kMax = 26;
        const int m = ceil(log2(n));
        vector<int> ans;
        vector<vector<pair<int, int>>> graph(n);
        // jump[i][j] := the 2^j-th ancestor of i
        vector<vector<int>>> jump(n, vector<int>(m));
        // depth[i] := the depth of i
        vector<int> depth(n);
        // count[i][j] := the count of j from root to i, where 1 <= j <= 26
        vector<vector<int>>> count(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            const int w = edge[2];
            graph[u].emplace_back(v, w);
            graph[v].emplace_back(u, w);
        }

        count[0] = vector<int>(kMax + 1);
        dfs(graph, 0, /*prev=*/-1, jump, depth, count);

        for (int j = 1; j < m; ++j)
            for (int i = 0; i < n; ++i)
                jump[i][j] = jump[jump[i][j - 1]][j - 1];

        for (const vector<int>& query : queries) {
            const int u = query[0];
            const int v = query[1];
            const int lca = getLCA(u, v, jump, depth);
            // the number of edges between (u, v).
            const int numEdges = depth[u] + depth[v] - 2 * depth[lca];
            // the maximum frequency of edges between (u, v)
            int maxFreq = 0;
            for (int j = 1; j <= kMax; ++j)
                maxFreq = max(maxFreq, count[u][j] + count[v][j] - 2 * count[lca][j]);
            ans.push_back(numEdges - maxFreq);
        }

        return ans;
    }

private:
    void dfs(const vector<vector<pair<int, int>>>& graph, int u, int prev,
            vector<vector<int>>>& jump, vector<int>& depth,
            vector<vector<int>>>& count) {
        for (const auto& [v, w] : graph[u]) {
            if (v == prev)
                continue;
            jump[v][0] = u;
            depth[v] = depth[u] + 1;
            count[v] = count[u];
            ++count[v][w];
            dfs(graph, v, u, jump, depth, count);
        }
    }

    // Returns the lca(u, v) by binary jump.
    int getLCA(int u, int v, const vector<vector<int>>>& jump,
               const vector<int>& depth) {
        // v is always deeper than u.
        if (depth[u] > depth[v])
            return getLCA(v, u, jump, depth);
        // Jump v to the same height of u.
        for (int j = 0; j < jump[0].size(); ++j)
            if (depth[v] - depth[u] >= j & 1)
                v = jump[v][j];
        if (u == v)
            return u;
        // Jump u and v to the node right below the lca.
        for (int j = jump[0].size() - 1; j >= 0; --j)
            if (jump[u][j] != jump[v][j]) {
                u = jump[u][j];
                v = jump[v][j];
            }
        return jump[v][0];
    }
};
```


1941. 2847.cpp

Path: LeetCode-main/solutions/2847. Smallest Number With Given Digit Product/2847.cpp

```
class Solution {
public:
    string smallestNumber(long long n) {
        if (n <= 9)
            return to_string(n);

        string ans;

        for (int divisor = 9; divisor > 1; --divisor)
            while (n % divisor == 0) {
                ans += to_string(divisor);
                n /= divisor;
            }

        return n > 1 ? "-1" : string{ans.rbegin(), ans.rend()};
    }
};
```

1942. 2848.cpp

Path: LeetCode-main/solutions/2848. Points That Intersect With Cars/2848.cpp

```
class Solution {
public:
    int numberOfPoints(vector<vector<int>>& nums) {
        constexpr int kMax = 100;
        int ans = 0;
        int runningSum = 0;
        vector<int> count(kMax + 2);

        for (const vector<int>& num : nums) {
            const int start = num[0];
            const int end = num[1];
            ++count[start];
            --count[end + 1];
        }

        for (int i = 1; i <= kMax; ++i) {
            runningSum += count[i];
            if (runningSum > 0)
                ++ans;
        }

        return ans;
    }
};
```

1943. 2849.cpp

Path: LeetCode-main/solutions/2849. Determine if a Cell Is Reachable at a Given Time/2849.cpp

```
class Solution {
public:
    bool isReachableAtTime(int sx, int sy, int fx, int fy, int t) {
        const int minStep = max(abs(sx - fx), abs(sy - fy));
        return minStep == 0 ? t != 1 : minStep <= t;
    }
};
```

1944. 285.cpp

Path: LeetCode-main/solutions/285. Inorder Successor in BST/285.cpp

```
class Solution {
public:
    TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {
        if (root == nullptr)
            return nullptr;
        if (root->val <= p->val)
            return inorderSuccessor(root->right, p);

        TreeNode* left = inorderSuccessor(root->left, p);
        return left ? left : root;
    }
};
```


1946. 2851.cpp

Path: LeetCode-main/solutions/2851. String Transformation/2851.cpp

```
class Solution {
public:
    // This dynamic programming table dp[k][i] represents the number of ways to
    // rearrange the string s after k steps such that it starts with s[i].
    // A string can be rotated from 1 to n - 1 times. The transition rule is
    // dp[k][i] = sum(dp[k - 1][j]) for all j != i. For example, when n = 4 and
    // k = 3, the table looks like this:
    //
    // -----
    // |           | i = 0 | i = 1 | i = 2 | i = 3 | sum = (n - 1)^k |
    // -----
    // | k = 0 |   1   |   0   |   0   |   0   |         1         |
    // | k = 1 |   0   |   1   |   1   |   1   |         3         |
    // | k = 2 |   3   |   2   |   2   |   2   |         9         |
    // | k = 3 |   6   |   7   |   7   |   7   |        27         |
    // -----
    //
    // By observation, we have
    // * dp[k][!0] = ((n - 1)^k - (-1)^k) / n
    // * dp[k][0] = dp[k][!0] + (-1)^k
    int numberOfWays(string s, string t, long long k) {
        const int n = s.length();
        const int negOnePowK = (k % 2 == 0 ? 1 : -1); // (-1)^k
        const vector<int> z = zFunction(s + t + t);
        const vector<int> indices = getIndices(z, n);
        vector<int> dp(2); // dp[0] := dp[k][0]; dp[1] := dp[k][!0]
        dp[1] = (modPow(n - 1, k) - negOnePowK + kMod) % kMod *
                modPow(n, kMod - 2) % kMod;
        dp[0] = (dp[1] + negOnePowK + kMod) % kMod;
        return accumulate(indices.begin(), indices.end(), 0L,
                           [&](long acc, int index) {
                               return (acc + dp[index == 0 ? 0 : 1]) % kMod;
                           });
    }
private:
    static constexpr int kMod = 1'000'000'007;

    long modPow(long x, long n) {
        if (n == 0)
            return 1;
        if (n % 2 == 1)
            return x * modPow(x, n - 1) % kMod;
        return modPow(x * x % kMod, n / 2);
    }

    // Returns the z array, where z[i] is the length of the longest prefix of
    // s[i..n) which is also a prefix of s.
    //
    // https://cp-algorithms.com/string/z-function.html#implementation
    vector<int> zFunction(const string& s) {
        const int n = s.length();
        vector<int> z(n);
        int l = 0;
        int r = 0;
        for (int i = 1; i < n; ++i) {
            if (i < r)
                z[i] = min(r - i, z[i - l]);
            while (i + z[i] < n && s[z[i]] == s[i + z[i]])
                ++z[i];
            if (i + z[i] > r) {
                l = i;
                r = i + z[i];
            }
        }
        return z;
    }

    // Returns the indices in `s` s.t. for each `i` in the returned indices,
    // `s[i..n) + s[0..i) = t`.
    vector<int> getIndices(const vector<int>& z, int n) {
        vector<int> indices;
        for (int i = n; i < n + n; ++i)
            if (z[i] >= n)
                indices.push_back(i - n);
        return indices;
    }
};
```

1947. 2852.cpp

Path: LeetCode-main/solutions/2852. Sum of Remoteness of All Cells/2852.cpp

```
class Solution {
public:
    long long sumRemoteness(vector<vector<int>>& grid) {
        const long sum = accumulate(grid.begin(), grid.end(), 0L,
            [](long acc, const vector<int>& row) {
                return acc +
                    accumulate(row.begin(), row.end(), 0L,
                        [](long acc, int num) { return acc + max(0, num); });
            });

        long ans = 0;

        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] > 0) {
                    const auto [count, componentSum] = dfs(grid, i, j);
                    ans += (sum - componentSum) * count;
                }

        return ans;
    }

private:
    static constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Returns the (count, componentSum) of the connected component that contains
    // (x, y).
    pair<int, long> dfs(vector<vector<int>>& grid, int i, int j) {
        if (i < 0 || i == grid.size() || j < 0 || j == grid[0].size())
            return {0, 0};
        if (grid[i][j] == -1)
            return {0, 0};

        int count = 1;
        long componentSum = grid[i][j];
        grid[i][j] = -1; // Mark as visited.;

        for (const auto& [dx, dy] : kDirs) {
            const int x = i + dx;
            const int y = j + dy;
            const auto [nextCount, nextComponentSum] = dfs(grid, x, y);
            count += nextCount;
            componentSum += nextComponentSum;
        }

        return {count, componentSum};
    }
};
```

1948. 2855.cpp

Path: LeetCode-main/solutions/2855. Minimum Right Shifts to Sort the Array/2855.cpp

```
class Solution {
public:
    int minimumRightShifts(vector<int>& nums) {
        int count = 0;
        int pivot = -1;

        for (int i = 0; i + 1 < nums.size(); i++)
            if (nums[i] > nums[i + 1]) {
                ++count;
                pivot = i;
            }

        if (count == 0)
            return 0;
        if (count > 1 || nums.back() > nums.front())
            return -1;
        return nums.size() - 1 - pivot;
    }
};
```

1949. 2856.cpp

Path: LeetCode-main/solutions/2856. Minimum Array Length After Pair Removals/2856.cpp

```
class Solution {
public:
    int minLengthAfterRemovals(vector<int>& nums) {
        const int n = nums.size();
        unordered_map<int, int> count;
        int maxFreq = 0;

        for (const int num : nums)
            ++count[num];

        for (const auto& [_, freq] : count)
            maxFreq = max(maxFreq, freq);

        // The number with the maximum frequency cancel all the other numbers.
        if (maxFreq <= n / 2)
            return n % 2;
        // The number with the maximum frequency cancel all the remaining numbers.
        return maxFreq - (n - maxFreq);
    }
};
```

1950. 2857.cpp

Path: LeetCode-main/solutions/2857. Count Pairs of Points With Distance k/2857.cpp

```
class Solution {
public:
    int countPairs(vector<vector<int>>& coordinates, int k) {
        int ans = 0;

        for (int x = 0; x <= k; ++x) {
            const int y = k - x;
            unordered_map<pair<int, int>, int, PairHash> count;
            for (const vector<int>& point : coordinates) {
                const int xi = point[0];
                const int yi = point[1];
                if (const auto it = count.find({xi ^ x, yi ^ y}); it != count.cend())
                    ans += it->second;
                ++count[{xi, yi}];
            }
        }

        return ans;
    }

private:
    struct PairHash {
        size_t operator()(const pair<int, int>& p) const {
            return p.first ^ p.second;
        }
    };
};
```

1951. 2858.cpp

Path: LeetCode-main/solutions/2858. Minimum Edge Reversals So Every Node Is Reachable/2858.cpp

```
class Solution {
public:
    vector<int> minEdgeReversals(int n, vector<vector<int>>& edges) {
        vector<int> ans(n);
        vector<vector<pair<int, bool>>> graph(n);
        vector<bool> seen(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].emplace_back(v, /*isForward=*/true);
            graph[v].emplace_back(u, /*isForward=*/false);
        }

        vector<int> mem(n, -1);
        ans[0] = minEdgeReversals(graph, 0, seen, mem);
        seen = vector<bool>(n);
        dfs(graph, 0, seen, ans);
        return ans;
    }

private:
    // Returns the minimum number of edge reversals so node u can reach every
    // node in its subtree.
    int minEdgeReversals(const vector<vector<pair<int, bool>>>& graph, int u,
                        vector<bool>& seen, vector<int>& mem) {
        if (mem[u] != -1)
            return mem[u];
        int res = 0;
        seen[u] = true;
        for (const auto& [v, isForward] : graph[u]) {
            if (seen[v])
                continue;
            seen[v] = true;
            res += minEdgeReversals(graph, v, seen, mem) + (isForward ? 0 : 1);
        }
        return mem[u] = res;
    }

    void dfs(const vector<vector<pair<int, bool>>>& graph, int u,
            vector<bool>& seen, vector<int>& ans) {
        seen[u] = true;
        for (const auto& [v, isForward] : graph[u]) {
            if (seen[v])
                continue;
            seen[v] = true;
            ans[v] = ans[u] + (isForward ? 1 : -1);
            dfs(graph, v, seen, ans);
        }
    }
};
```

1952. 2859.cpp

Path: LeetCode-main/solutions/2859. Sum of Values at Indices With K Set Bits/2859.cpp

```
class Solution {
public:
    int sumIndicesWithKSetBits(vector<int>& nums, int k) {
        int ans = 0;
        for (unsigned i = 0; i < nums.size(); ++i)
            if (popcount(i) == k)
                ans += nums[i];
        return ans;
    }
};
```


1953. 286.cpp

Path: LeetCode-main/solutions/286. Walls and Gates/286.cpp

```
class Solution {
public:
    void wallsAndGates(vector<vector<int>>& rooms) {
        constexpr int kDirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int m = rooms.size();
        const int n = rooms[0].size();
        queue<pair<int, int>> q;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (rooms[i][j] == 0)
                    q.emplace(i, j);

        while (!q.empty()) {
            const auto [i, j] = q.front();
            q.pop();
            for (const auto& [dx, dy] : kDirs) {
                const int x = i + dx;
                const int y = j + dy;
                if (x < 0 || x == m || y < 0 || y == n)
                    continue;
                if (rooms[x][y] != INT_MAX)
                    continue;
                rooms[x][y] = rooms[i][j] + 1;
                q.emplace(x, y);
            }
        }
    }
};
```

1954. 2860.cpp

Path: LeetCode-main/solutions/2860. Happy Students/2860.cpp

```
class Solution {
public:
    int countWays(vector<int>& nums) {
        nums.push_back(-1);
        nums.push_back(INT_MAX);
        ranges::sort(nums);

        int ans = 0;

        // i := the number of the selected numbers
        for (int i = 0; i + 1 < nums.size(); ++i)
            if (nums[i] < i && i < nums[i + 1])
                ++ans;

        return ans;
    }
};
```

1955. 2861.cpp

Path: LeetCode-main/solutions/2861. Maximum Number of Alloys/2861.cpp

```
class Solution {
public:
    int maxNumberOfAlloys(int n, int k, int budget,
                        vector<vector<int>>& composition, vector<int>& stock,
                        vector<int>& cost) {

        int l = 1;
        int r = 1'000'000'000;

        while (l < r) {
            const int m = (l + r) / 2;
            if (isPossible(n, budget, composition, stock, cost, m))
                l = m + 1;
            else
                r = m;
        }

        return l - 1;
    }

private:
    // Returns true if it's possible to create `m` alloys by using any machine.
    bool isPossible(int n, int budget, const vector<vector<int>>& composition,
                    const vector<int>& stock, const vector<int>& costs, int m) {
        // Try all the possible machines.
        for (const vector<int>& machine : composition) {
            long requiredMoney = 0;
            for (int j = 0; j < n; ++j) {
                const long requiredUnits =
                    max(0L, static_cast<long>(machine[j]) * m - stock[j]);
                requiredMoney += static_cast<long>(requiredUnits) * costs[j];
            }
            if (requiredMoney <= budget)
                return true;
        }
        return false;
    }
};
```

1956. 2862-2.cpp

Path: LeetCode-main/solutions/2862. Maximum Element-Sum of a Complete Subset of Indices/2862-2.cpp

```
class Solution {
public:
    long long maximumSum(vector<int>& nums) {
        long ans = 0;

        for (int oddPower = 1; oddPower <= nums.size(); ++oddPower) {
            long sum = 0;
            for (int num = 1; num * num * oddPower <= nums.size(); ++num)
                sum += nums[oddPower * num * num - 1];
            ans = max(ans, sum);
        }

        return ans;
    }
};
```

1957. 2862.cpp

Path: LeetCode-main/solutions/2862. Maximum Element-Sum of a Complete Subset of Indices/2862.cpp

```
class Solution {
public:
    long long maximumSum(vector<int>& nums) {
        long ans = 0;
        unordered_map<int, long> oddPowerToSum;

        for (int i = 0; i < nums.size(); ++i) {
            const int oddPower = divideSquares(i + 1);
            ans = max(ans, oddPowerToSum[oddPower] += nums[i]);
        }

        return ans;
    }

private:
    int divideSquares(int val) {
        for (int num = 2; num * num <= val; ++num)
            while (val % (num * num) == 0)
                val /= num * num;
        return val;
    }
};
```

1958. 2863.cpp

Path: LeetCode-main/solutions/2863. Maximum Length of Semi-Decreasing Subarrays/2863.cpp

```
class Solution {
public:
    int maxSubarrayLength(vector<int>& nums) {
        int ans = 0;
        stack<int> stack;

        for (int i = nums.size() - 1; i >= 0; --i)
            // If nums[stack.top()] <= nums[i], stack.top() is better than i.
            // So, no need to push it.
            if (stack.empty() || nums[stack.top()] > nums[i])
                stack.push(i);

        for (int i = 0; i < nums.size(); ++i)
            while (!stack.empty() && num > nums[stack.top()])
                ans = max(ans, stack.top() - i + 1), stack.pop();

        return ans;
    }
};
```

1959. 2864.cpp

Path: LeetCode-main/solutions/2864. Maximum Odd Binary Number/2864.cpp

```
class Solution {
public:
    string maximumOddBinaryNumber(string s) {
        const int zeros = ranges::count(s, '0');
        const int ones = s.length() - zeros;
        return string(ones - 1, '1') + string(zeros, '0') + '1';
    }
};
```

1960. 2865.cpp

Path: LeetCode-main/solutions/2865. Beautiful Towers I/2865.cpp

```
class Solution {
public:
    long long maximumSumOfHeights(vector<int>& heights) {
        const int n = heights.size();
        vector<long> maxSum(n); // maxSum[i] := the maximum sum with peak i

        stack<int> stack{{-1}};
        long sum = 0;

        for (int i = 0; i < n; ++i) {
            sum = process(stack, heights, i, sum);
            maxSum[i] = sum;
        }

        stack = std::stack<int>{{n}};
        sum = 0;

        for (int i = n - 1; i >= 0; --i) {
            sum = process(stack, heights, i, sum);
            maxSum[i] += sum - heights[i];
        }

        return ranges::max(maxSum);
    }

private:
    long process(stack<int>& stack, const vector<int>& heights, int i, long sum) {
        while (stack.size() > 1 && heights[stack.top()] > heights[i]) {
            const int j = stack.top();
            stack.pop();
            // The last abs(j - stack.top()) heights are `heights[j]`.
            sum -= static_cast<long>(abs(j - stack.top())) * heights[j];
        }
        // Put abs(i - stack.top()) * heights[i] in `heights`.
        sum += static_cast<long>(abs(i - stack.top())) * heights[i];
        stack.push(i);
        return sum;
    }
};
```


1961. 2866.cpp

Path: LeetCode-main/solutions/2866. Beautiful Towers II/2866.cpp

```
class Solution {
public:
    // Same as 2865. Beautiful Towers I
    long long maximumSumOfHeights(vector<int>& maxHeights) {
        const int n = maxHeights.size();
        vector<long> maxSum(n); // maxSum[i] := the maximum sum with peak i

        stack<int> stack{{-1}};
        long sum = 0;

        for (int i = 0; i < n; ++i) {
            sum = process(stack, maxHeights, i, sum);
            maxSum[i] = sum;
        }

        stack = std::stack<int>{{n}};
        sum = 0;

        for (int i = n - 1; i >= 0; --i) {
            sum = process(stack, maxHeights, i, sum);
            maxSum[i] += sum - maxHeights[i];
        }

        return ranges::max(maxSum);
    }

private:
    long process(stack<int>& stack, const vector<int>& maxHeights, int i,
                long sum) {
        while (stack.size() > 1 && maxHeights[stack.top()] > maxHeights[i]) {
            const int j = stack.top();
            stack.pop();
            // The last abs(j - stack.top()) maxHeights are `maxHeights[j]`.
            sum -= static_cast<long>(abs(j - stack.top())) * maxHeights[j];
        }
        // Put abs(i - stack.top()) * maxHeights[i] in `maxHeights`.
        sum += static_cast<long>(abs(i - stack.top())) * maxHeights[i];
        stack.push(i);
        return sum;
    }
};
```

1962. 2867.cpp

Path: LeetCode-main/solutions/2867. Count Valid Paths in a Tree/2867.cpp

```
class Solution {
public:
    long long countPaths(int n, vector<vector<int>>& edges) {
        long ans = 0;
        const vector<bool> isPrime = sieveEratosthenes(n + 1);
        vector<vector<int>> graph(n + 1);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        dfs(graph, 1, /*prev=*/-1, isPrime, ans);
        return ans;
    }

private:
    pair<long, long> dfs(const vector<vector<int>>& graph, int u, int prev,
                        const vector<bool>& isPrime, long& ans) {
        long countZeroPrimePath = !isPrime[u];
        long countOnePrimePath = isPrime[u];

        for (const int v : graph[u]) {
            if (v == prev)
                continue;
            const auto& [countZeroPrimeChildPath, countOnePrimeChildPath] =
                dfs(graph, v, u, isPrime, ans);
            ans += countZeroPrimePath * countOnePrimeChildPath +
                countOnePrimePath * countZeroPrimeChildPath;
            if (isPrime[u]) {
                countOnePrimePath += countZeroPrimeChildPath;
            } else {
                countZeroPrimePath += countZeroPrimeChildPath;
                countOnePrimePath += countOnePrimeChildPath;
            }
        }

        return {countZeroPrimePath, countOnePrimePath};
    }

    vector<bool> sieveEratosthenes(int n) {
        vector<bool> isPrime(n, true);
        isPrime[0] = false;
        isPrime[1] = false;
        for (int i = 2; i * i < n; ++i)
            if (isPrime[i])
                for (int j = i * i; j < n; j += i)
                    isPrime[j] = false;
        return isPrime;
    }
};
```

1963. 2868.cpp

Path: LeetCode-main/solutions/2868. The Wording Game/2868.cpp

```
class Solution {
public:
    bool canAliceWin(vector<string>& a, vector<string>& b) {
        // words[0][i] := the biggest word starting with ('a' + i) for Alice
        // words[1][i] := the biggest word starting with ('a' + i) for Bob
        vector<vector<string>> words(2, vector<string>(26));

        // For each letter, only the biggest word is useful.
        for (const string& word : a)
            words[0][word[0] - 'a'] = word;

        for (const string& word : b)
            words[1][word[0] - 'a'] = word;

        // Find Alice's smallest word.
        int i = 0;
        while (words[0][i].empty())
            ++i;

        // Iterate through each letter until we find a winner.
        // Start with Alice's turn (0), so it's Bob's turn (1) now.
        for (int turn = 1; true; turn = turn ^ 1)
            // If the current player has a word that having the letter that is greater
            // than the opponent's word, choose it.
            if (!words[turn][i].empty() && words[turn][i] > words[turn ^ 1][i]) {
                // Choose the current words[turn][i].
            } else if (!words[turn][i + 1].empty()) {
                // Choose the next words[turn][i + 1].
                ++i;
            } else {
                // Game over. If it's Bob's turn, Alice wins, and vice versa.
                return turn == 1;
            }
    }
};
```

1964. 2869.cpp

Path: LeetCode-main/solutions/2869. Minimum Operations to Collect Elements/2869.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums, int k) {
        unordered_set<int> seen;

        for (int i = nums.size() - 1; i >= 0; --i) {
            if (nums[i] > k)
                continue;
            seen.insert(nums[i]);
            if (seen.size() == k)
                return nums.size() - i;
        }

        throw;
    }
};
```

1965. 287.cpp

Path: LeetCode-main/solutions/287. Find the Duplicate Number/287.cpp

```
class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int slow = nums[nums[0]];
        int fast = nums[nums[nums[0]]];

        while (slow != fast) {
            slow = nums[slow];
            fast = nums[nums[fast]];
        }

        slow = nums[0];

        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }

        return slow;
    }
};
```

1966. 2870.cpp

Path: LeetCode-main/solutions/2870. Minimum Number of Operations to Make Array Empty/2870.cpp

```
class Solution {
public:
    int minOperations(vector<int>& nums) {
        int ans = 0;
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        for (const auto& [_, freq] : count) {
            // If freq == 3k, need k operations.
            // If freq == 3k + 1 = 3*(k - 1) + 2*2, need k + 1 operations.
            // If freq == 3k + 2, need k + 1 operations.
            if (freq == 1)
                return -1;
            ans += (freq + 2) / 3;
        }

        return ans;
    }
};
```

1967. 2871.cpp

Path: LeetCode-main/solutions/2871. Split Array Into Maximum Number of Subarrays/2871.cpp

```
class Solution {
public:
    int maxSubarrays(vector<int>& nums) {
        int ans = 0;
        int score = 0;

        for (const int num : nums) {
            score = score == 0 ? num : score & num;
            if (score == 0)
                ++ans;
        }

        return max(1, ans);
    }
};
```

1968. 2872.cpp

Path: LeetCode-main/solutions/2872. Maximum Number of K-Divisible Components/2872.cpp

```
class Solution {
public:
    int maxKDivisibleComponents(int n, vector<vector<int>>& edges,
                                vector<int>& values, int k) {
        int ans = 0;
        vector<vector<int>> graph(n);

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        dfs(graph, 0, /*prev=*/-1, values, k, ans);
        return ans;
    }

private:
    long dfs(const vector<vector<int>>& graph, int u, int prev,
             const vector<int>& values, int k, int& ans) {
        long treeSum = values[u];

        for (const int v : graph[u])
            if (v != prev)
                treeSum += dfs(graph, v, u, values, k, ans);

        if (treeSum % k == 0)
            ++ans;
        return treeSum;
    }
};
```


1969. 2873.cpp

Path: LeetCode-main/solutions/2873. Maximum Value of an Ordered Triplet I/2873.cpp

```
class Solution {
public:
    long long maximumTripletValue(vector<int>& nums) {
        long ans = 0;
        int maxDiff = 0; // max(nums[i] - nums[j])
        int maxNum = 0; // max(nums[i])

        for (const int num : nums) {
            ans = max(ans, static_cast<long>(maxDiff) * num); // num := nums[k]
            maxDiff = max(maxDiff, maxNum - num); // num := nums[j]
            maxNum = max(maxNum, num); // num := nums[i]
        }

        return ans;
    }
};
```

1970. 2874.cpp

Path: LeetCode-main/solutions/2874. Maximum Value of an Ordered Triplet II/2874.cpp

```
class Solution {
public:
    // Same as 2873. Maximum Value of an Ordered Triplet I
    long long maximumTripletValue(vector<int>& nums) {
        long ans = 0;
        int maxDiff = 0; // max(nums[i] - nums[j])
        int maxNum = 0; // max(nums[i])

        for (const int num : nums) {
            ans = max(ans, static_cast<long>(maxDiff) * num); // num := nums[k]
            maxDiff = max(maxDiff, maxNum - num); // num := nums[j]
            maxNum = max(maxNum, num); // num := nums[i]
        }

        return ans;
    }
};
```

1971. 2875.cpp

Path: LeetCode-main/solutions/2875. Minimum Size Subarray in Infinite Array/2875.cpp

```
class Solution {
public:
    int minSizeSubarray(vector<int>& nums, int target) {
        const long sum = accumulate(nums.begin(), nums.end(), 0L);
        const int n = nums.size();
        const int remainingTarget = target % sum;
        const int repeatLength = (target / sum) * n;
        if (remainingTarget == 0)
            return repeatLength;

        int suffixPlusPrefixLength = n;
        long prefix = 0;
        unordered_map<long, int> prefixToIndex{{0, -1}};

        for (int i = 0; i < 2 * n; ++i) {
            prefix += nums[i % n];
            if (const auto it = prefixToIndex.find(prefix - remainingTarget);
                it != prefixToIndex.cend())
                suffixPlusPrefixLength = min(suffixPlusPrefixLength, i - it->second);
            prefixToIndex[prefix] = i;
        }

        return suffixPlusPrefixLength == n ? -1
            : suffixPlusPrefixLength + repeatLength;
    }
};
```

1972. 2876.cpp

Path: LeetCode-main/solutions/2876. Count Visited Nodes in a Directed Graph/2876.cpp

```
class Solution {
public:
    vector<int> countVisitedNodes(vector<int>& edges) {
        const int n = edges.size();
        vector<int> ans(n);
        vector<int> inDegrees(n);
        vector<bool> seen(n);
        queue<int> q;
        stack<int> stack;

        for (const int v : edges)
            ++inDegrees[v];

        // Perform topological sorting.
        for (int i = 0; i < n; ++i)
            if (inDegrees[i] == 0)
                q.push(i);

        // Push non-cyclic nodes to stack.
        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            if (--inDegrees[edges[u]] == 0)
                q.push(edges[u]);
            stack.push(u);
            seen[u] = true;
        }

        // Fill the length of cyclic nodes.
        for (int i = 0; i < n; ++i)
            if (!seen[i])
                fillCycle(edges, i, seen, ans);

        // Fill the length of non-cyclic nodes.
        while (!stack.empty()) {
            const int u = stack.top();
            stack.pop();
            ans[u] = ans[edges[u]] + 1;
        }

        return ans;
    }

private:
    void fillCycle(const vector<int>& edges, int start, vector<bool>& seen,
                  vector<int>& ans) {
        int cycleLength = 0;
        for (int u = start; !seen[u]; u = edges[u]) {
            ++cycleLength;
            seen[u] = true;
        }
        ans[start] = cycleLength;
        for (int u = edges[start]; u != start; u = edges[u])
            ans[u] = cycleLength;
    }
};
```

1973. 288.cpp

Path: LeetCode-main/solutions/288. Unique Word Abbreviation/288.cpp

```
class ValidWordAbbr {
public:
    ValidWordAbbr(vector<string>& dictionary) {
        dict = unordered_set(dictionary.begin(), dictionary.end());

        for (const string& word : dict) {
            const string& abbr = getAbbr(word);
            abbrUnique[abbr] = !abbrUnique.contains(abbr);
        }

        bool isUnique(string word) {
            const string& abbr = getAbbr(word);
            return !abbrUnique.contains(abbr) ||
                abbrUnique[abbr] && dict.contains(word);
        }

private:
    unordered_set<string> dict;
    unordered_map<string, bool> abbrUnique; // T := unique, F := not unique

    string getAbbr(const string& s) {
        const int n = s.length();
        if (n <= 2)
            return s;
        return s[0] + to_string(n - 2) + s.back();
    }
};
```

1974. 289.cpp

Path: LeetCode-main/solutions/289. Game of Life/289.cpp

```
class Solution {
public:
    void gameOfLife(vector<vector<int>>& board) {
        const int m = board.size();
        const int n = board[0].size();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                int ones = 0;
                for (int x = max(0, i - 1); x < min(m, i + 2); ++x)
                    for (int y = max(0, j - 1); y < min(n, j + 2); ++y)
                        ones += board[x][y] & 1;
                // Any live cell with two or three live neighbors lives on to the next
                // generation.
                if (board[i][j] == 1 && (ones == 3 || ones == 4))
                    board[i][j] |= 0b10;
                // Any dead cell with exactly three live neighbors becomes a live cell,
                // as if by reproduction.
                if (board[i][j] == 0 && ones == 3)
                    board[i][j] |= 0b10;
            }

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                board[i][j] >>= 1;
    }
};
```

1975. 2892.cpp

Path: LeetCode-main/solutions/2892. Minimizing Array After Replacing Pairs With Their Product/2892.cpp

```
class Solution {
public:
    int minArrayLength(vector<int>& nums, int k) {
        int count = 0;
        long prod = -1;

        for (const int num : nums) {
            if (num == 0)
                return 1;
            if (prod != -1 && prod * num <= k) {
                prod *= num;
            } else {
                prod = num;
                ++count;
            }
        }

        return count;
    }
};
```

1976. 2894.cpp

Path: LeetCode-main/solutions/2894. Divisible and Non-divisible Sums Difference/2894.cpp

```
class Solution {
public:
    int differenceOfSums(int n, int m) {
        const int sum = (1 + n) * n / 2;
        const int num2 = getDivisibleSum(n, m);
        const int num1 = sum - num2;
        return num1 - num2;
    }

private:
    // Returns the sum of all the integers in [1, n] that are divisible by m.
    int getDivisibleSum(int n, int m) {
        const int last = n / m * m;
        if (last == 0)
            return 0;
        const int first = m;
        const int count = (last - first) / m + 1;
        return (first + last) * count / 2;
    }
};
```


1977. 2895.cpp

Path: LeetCode-main/solutions/2895. Minimum Processing Time/2895.cpp

```
class Solution {
public:
    int minProcessingTime(vector<int>& processorTime, vector<int>& tasks) {
        int ans = 0;
        ranges::sort(processorTime);
        ranges::sort(tasks, greater<>());

        // It's optimal to run each 4 longer tasks with a smaller processor time.
        // Therefore, for each processor, take the maximum of the sum of the
        // processor time and the largest assigned tasks[i].
        for (int i = 0; i < processorTime.size(); ++i)
            ans = max(ans, processorTime[i] + tasks[i * 4]);

        return ans;
    }
};
```

1978. 2896-2.cpp

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896-2.cpp

```
class Solution {
public:
    int minOperations(string s1, string s2, int x) {
        const vector<int> diffIndices = getDiffIndices(s1, s2);
        if (diffIndices.empty())
            return 0;
        // It's impossible to make two strings equal if there are an odd number of
        // differences.
        if (diffIndices.size() % 2 == 1)
            return -1;

        vector<double> dp(diffIndices.size() + 1, DBL_MAX);
        dp.back() = 0;
        dp[diffIndices.size() - 1] = x / 2.0;

        for (int i = diffIndices.size() - 2; i >= 0; --i)
            dp[i] = min(dp[i + 1] + x / 2.0,
                        dp[i + 2] + diffIndices[i + 1] - diffIndices[i]);

        return dp[0];
    }

private:
    vector<int> getDiffIndices(const string& s1, const string& s2) {
        vector<int> diffIndices;
        for (int i = 0; i < s1.length(); ++i)
            if (s1[i] != s2[i])
                diffIndices.push_back(i);
        return diffIndices;
    }
};
```

1979. 2896-3.cpp

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896-3.cpp

```
class Solution {
public:
    int minOperations(string s1, string s2, int x) {
        const vector<int> diffIndices = getDiffIndices(s1, s2);
        if (diffIndices.empty())
            return 0;
        // It's impossible to make two strings equal if there are an odd number of
        // differences.
        if (diffIndices.size() % 2 == 1)
            return -1;

        double dp = 0;
        double dpNext = x / 2.0;
        double dpNextNext = 0;

        for (int i = diffIndices.size() - 2; i >= 0; --i) {
            dp = min(dpNext + x / 2.0,
                    dpNextNext + diffIndices[i + 1] - diffIndices[i]);
            dpNextNext = dpNext;
            dpNext = dp;
        }

        return dp;
    }

private:
    vector<int> getDiffIndices(const string& s1, const string& s2) {
        vector<int> diffIndices;
        for (int i = 0; i < s1.length(); ++i)
            if (s1[i] != s2[i])
                diffIndices.push_back(i);
        return diffIndices;
    }
};
```

1980. 2896.cpp

Path: LeetCode-main/solutions/2896. Apply Operations to Make Two Strings Equal/2896.cpp

```
class Solution {
public:
    int minOperations(string s1, string s2, int x) {
        const vector<int> diffIndices = getDiffIndices(s1, s2);
        if (diffIndices.empty())
            return 0;
        // It's impossible to make two strings equal if there are an odd number of
        // differences.
        if (diffIndices.size() % 2 == 1)
            return -1;
        vector<double> mem(diffIndices.size(), -1.0);
        return minOperations(diffIndices, 0, x, mem);
    }

private:
    // Returns the minimum cost to correct diffIndices[i..n).
    double minOperations(const vector<int>& diffIndices, int i, double x,
                        vector<double>& mem) {
        if (i == diffIndices.size())
            return 0;
        if (i == diffIndices.size() - 1)
            return x / 2;
        if (mem[i] != -1.0)
            return mem[i];
        return mem[i] = min(minOperations(diffIndices, i + 1, x, mem) + x / 2,
                            minOperations(diffIndices, i + 2, x, mem) +
                                diffIndices[i + 1] - diffIndices[i]);
    }

    vector<int> getDiffIndices(const string& s1, const string& s2) {
        vector<int> diffIndices;
        for (int i = 0; i < s1.length(); ++i)
            if (s1[i] != s2[i])
                diffIndices.push_back(i);
        return diffIndices;
    }
};
```

1981. 2897.cpp

Path: LeetCode-main/solutions/2897. Apply Operations on Array to Maximize Sum of Squares/2897.cpp

```
class Solution {
public:
    int maxSum(vector<int>& nums, int k) {
        constexpr int kMod = 1'000'000'007;
        constexpr int kMaxBit = 30;
        int ans = 0;
        // minIndices[i] := the minimum index in `optimalNums` that the i-th bit
        // should be moved to
        vector<int> minIndices(kMaxBit);
        vector<int> optimalNums(nums.size());

        for (const int num : nums)
            for (int i = 0; i < kMaxBit; ++i)
                if (num >> i & 1)
                    optimalNums[minIndices[i]++] |= 1 << i;

        for (int i = 0; i < k; ++i)
            ans = (ans + static_cast<long>(optimalNums[i]) * optimalNums[i]) % kMod;

        return ans;
    }
};
```


1983. 2899.cpp

Path: LeetCode-main/solutions/2899. Last Visited Integers/2899.cpp

```
class Solution {
public:
    vector<int> lastVisitedIntegers(vector<string>& words) {
        vector<int> ans;
        vector<int> nums;
        int k = 0;

        for (const string& word : words)
            if (word == "prev") {
                ++k;
                ans.push_back(k > nums.size() ? -1 : nums[nums.size() - k]);
            } else {
                k = 0;
                nums.push_back(stoi(word));
            }

        return ans;
    }
};
```

1984. 29.cpp

Path: LeetCode-main/solutions/29. Divide Two Integers/29.cpp

```
class Solution {
public:
    int divide(int dividend, int divisor) {
        // -2^{31} / -1 = 2^{31} will overflow, so return 2^{31} - 1.
        if (dividend == INT_MIN && divisor == -1)
            return INT_MAX;

        const int sign = dividend > 0 ^ divisor > 0 ? -1 : 1;
        long dvd = labs(dividend);
        long dvs = labs(divisor);

        while (dvd >= dvs) {
            long k = 1;
            while (k * 2 * dvs <= dvd)
                k *= 2;
            dvd -= k * dvs;
            ans += k;
        }

        return sign * ans;
    }
};
```


1985. 290.cpp

Path: LeetCode-main/solutions/290. Word Pattern/290.cpp

```
class Solution {
public:
    bool wordPattern(string pattern, string str) {
        const int n = pattern.length();
        istringstream iss(str);
        vector<int> charToIndex(128);
        unordered_map<string, int> stringToIndex;

        int i = 0;
        for (string word; iss >> word; ++i) {
            if (i == n) // out-of-bounds
                return false;
            if (charToIndex[pattern[i]] != stringToIndex[word])
                return false;
            charToIndex[pattern[i]] = i + 1;
            stringToIndex[word] = i + 1;
        }
        return i == n;
    }
};
```

1986. 2900.cpp

Path: LeetCode-main/solutions/2900. Longest Unequal Adjacent Groups Subsequence I/2900.cpp

```
class Solution {
public:
    vector<string> getWordsInLongestSubsequence(int n, vector<string>& words,
                                              vector<int>& groups) {
        vector<string> ans;
        int groupId = -1;

        for (int i = 0; i < n; ++i)
            if (groups[i] != groupId) {
                groupId = groups[i];
                ans.push_back(words[i]);
            }

        return ans;
    }
};
```

1987. 2901.cpp

Path: LeetCode-main/solutions/2901. Longest Unequal Adjacent Groups Subsequence II/2901.cpp

```
class Solution {
public:
    vector<string> getWordsInLongestSubsequence(int n, vector<string>& words,
                                              vector<int>& groups) {
        vector<string> ans;
        // dp[i] := the length of the longest subsequence ending in `words[i]`
        vector<int> dp(n, 1);
        // prev[i] := the best index of words[i]
        vector<int> prev(n, -1);

        for (int i = 1; i < n; ++i)
            for (int j = 0; j < i; ++j) {
                if (groups[i] == groups[j])
                    continue;
                if (words[i].length() != words[j].length())
                    continue;
                if (hammingDist(words[i], words[j]) != 1)
                    continue;
                if (dp[i] < dp[j] + 1) {
                    dp[i] = dp[j] + 1;
                    prev[i] = j;
                }
            }

        // Find the last index of the subsequence.
        int index = ranges::max_element(dp) - dp.begin();
        while (index != -1) {
            ans.push_back(words[index]);
            index = prev[index];
        }

        ranges::reverse(ans);
        return ans;
    }

private:
    int hammingDist(const string& s1, const string& s2) {
        int dist = 0;
        for (int i = 0; i < s1.length(); ++i)
            if (s1[i] != s2[i])
                ++dist;
        return dist;
    }
};
```

1988. 2902.cpp

Path: LeetCode-main/solutions/2902. Count of Sub-Multisets With Bounded Sum/2902.cpp

```
class Solution {
public:
    int countSubMultisets(vector<int>& nums, int l, int r) {
        constexpr int kMod = 1'000'000'007;
        // dp[i] := the number of submultisets of `nums` with sum i
        vector<long> dp(r + 1);
        dp[0] = 1;
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        const int zeros = count[0];
        count.erase(0);

        for (const auto& [num, freq] : count) {
            // stride[i] := dp[i] + dp[i - num] + dp[i - 2 * num] + ...
            vector<long> stride = dp;
            for (int i = num; i <= r; ++i)
                stride[i] += stride[i - num];
            for (int i = r; i > 0; --i)
                if (i >= num * (freq + 1))
                    // dp[i] + dp[i - num] + dp[i - freq * num]
                    dp[i] = (stride[i] - stride[i - num * (freq + 1)]) % kMod;
                else
                    dp[i] = stride[i] % kMod;
        }

        long ans = 0;
        for (int i = l; i <= r; ++i)
            ans = (ans + dp[i]) % kMod;
        return ((zeros + 1) * ans) % kMod;
    }
};
```

1989. 2903.cpp

Path: LeetCode-main/solutions/2903. Find Indices With Index and Value Difference I/2903.cpp

```
class Solution {
public:
    vector<int> findIndices(vector<int>& nums, int indexDifference,
                           int valueDifference) {
        // nums[minIndex] := the minimum number with enough index different from the
        // current number
        int minIndex = 0;
        // nums[maxIndex] := the maximum number with enough index different from the
        // current number
        int maxIndex = 0;

        for (int i = indexDifference; i < nums.size(); ++i) {
            if (nums[i - indexDifference] < nums[minIndex])
                minIndex = i - indexDifference;
            if (nums[i - indexDifference] > nums[maxIndex])
                maxIndex = i - indexDifference;
            if (nums[i] - nums[minIndex] >= valueDifference)
                return {i, minIndex};
            if (nums[maxIndex] - nums[i] >= valueDifference)
                return {i, maxIndex};
        }

        return {-1, -1};
    }
};
```

1990. 2904.cpp

Path: LeetCode-main/solutions/2904. Shortest and Lexicographically Smallest Beautiful String/2904.cpp

```
class Solution {
public:
    // Same as 76. Minimum Window Substring
    string shortestBeautifulSubstring(string s, int k) {
        int bestLeft = -1;
        int minLength = s.length() + 1;
        int ones = 0;

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (s[r] == '1')
                ++ones;
            while (ones == k) {
                if (r - l + 1 < minLength) {
                    bestLeft = l;
                    minLength = r - l + 1;
                } else if (r - l + 1 == minLength &&
                           s.compare(l, minLength, s, bestLeft, minLength) < 0) {
                    bestLeft = l;
                }
                if (s[l++] == '1')
                    --ones;
            }
        }

        return bestLeft == -1 ? "" : s.substr(bestLeft, minLength);
    }
};
```

1991. 2905.cpp

Path: LeetCode-main/solutions/2905. Find Indices With Index and Value Difference II/2905.cpp

```
class Solution {
public:
    vector<int> findIndices(vector<int>& nums, int indexDifference,
                           int valueDifference) {
        // nums[minIndex] := the minimum number with enough index different from the
        // current number
        int minIndex = 0;
        // nums[maxIndex] := the maximum number with enough index different from the
        // current number
        int maxIndex = 0;

        for (int i = indexDifference; i < nums.size(); ++i) {
            if (nums[i - indexDifference] < nums[minIndex])
                minIndex = i - indexDifference;
            if (nums[i - indexDifference] > nums[maxIndex])
                maxIndex = i - indexDifference;
            if (nums[i] - nums[minIndex] >= valueDifference)
                return {i, minIndex};
            if (nums[maxIndex] - nums[i] >= valueDifference)
                return {i, maxIndex};
        }

        return {-1, -1};
    }
};
```

1992. 2906.cpp

Path: LeetCode-main/solutions/2906. Construct Product Matrix/2906.cpp

```
class Solution {
public:
    vector<vector<int>> constructProductMatrix(vector<vector<int>>& grid) {
        constexpr int kMod = 12345;
        const int m = grid.size();
        const int n = grid[0].size();
        vector<vector<int>> ans(m, vector<int>(n));
        vector<int> prefix{1};
        int suffix = 1;

        for (const vector<int>& row : grid)
            for (const int num : row)
                prefix.push_back(static_cast<long>(prefix.back()) * num % kMod);

        for (int i = m - 1; i >= 0; --i)
            for (int j = n - 1; j >= 0; --j) {
                ans[i][j] = prefix[i * n + j] * suffix % kMod;
                suffix = static_cast<long>(suffix) * grid[i][j] % kMod;
            }

        return ans;
    }
};
```


1993. 2907.cpp

Path: LeetCode-main/solutions/2907. Maximum Profitable Triplets With Increasing Prices I/2907.cpp

```
class FenwickTree {
public:
    FenwickTree(int n) : vals(n + 1) {}

    void maximize(int i, int val) {
        while (i < vals.size()) {
            vals[i] = max(vals[i], val);
            i += lowbit(i);
        }
    }

    int get(int i) const {
        int res = 0;
        while (i > 0) {
            res = max(res, vals[i]);
            i -= lowbit(i);
        }
        return res;
    }

private:
    vector<int> vals;

    static int lowbit(int i) {
        return i & -i;
    }
};

class Solution {
public:
    int maxProfit(vector<int>& prices, vector<int>& profits) {
        const int maxPrice = ranges::max(prices);
        int ans = -1;
        FenwickTree maxProfitTree1(maxPrice);
        FenwickTree maxProfitTree2(maxPrice);

        for (int i = 0; i < prices.size(); ++i) {
            const int price = prices[i];
            const int profit = profits[i];
            // max(proftis[i])
            const int maxProfit1 = maxProfitTree1.get(price - 1);
            // max(proftis[i]) + max(profits[j])
            const int maxProfit2 = maxProfitTree2.get(price - 1);
            maxProfitTree1.maximize(price, profit);
            if (maxProfit1 > 0)
                maxProfitTree2.maximize(price, profit + maxProfit1);
            if (maxProfit2 > 0)
                ans = max(ans, profit + maxProfit2);
        }

        return ans;
    }
};
```

1994. 2908.cpp

Path: LeetCode-main/solutions/2908. Minimum Sum of Mountain Triplets I/2908.cpp

```
class Solution {
public:
    int minimumSum(vector<int>& nums) {
        const int n = nums.size();
        int ans = INT_MAX;
        vector<int> minPrefix(n);
        vector<int> minSuffix(n);

        partial_sum(nums.begin(), nums.end(), minPrefix.begin(),
                    [](int x, int y) { return min(x, y); });
        partial_sum(nums.rbegin(), nums.rend(), minSuffix.begin(),
                    [](int x, int y) { return min(x, y); });
        ranges::reverse(minSuffix);

        for (int i = 0; i < n; ++i)
            if (nums[i] > minPrefix[i] && nums[i] > minSuffix[i])
                ans = min(ans, nums[i] + minPrefix[i] + minSuffix[i]);

        return ans == INT_MAX ? -1 : ans;
    }
};
```

1995. 2909.cpp

Path: LeetCode-main/solutions/2909. Minimum Sum of Mountain Triplets II/2909.cpp

```
class Solution {
public:
    // Same as 2908. Minimum Sum of Mountain Triplets I
    int minimumSum(vector<int>& nums) {
        const int n = nums.size();
        int ans = INT_MAX;
        vector<int> minPrefix(n);
        vector<int> minSuffix(n);

        partial_sum(nums.begin(), nums.end(), minPrefix.begin(),
                    [](int x, int y) { return min(x, y); });
        partial_sum(nums.rbegin(), nums.rend(), minSuffix.begin(),
                    [](int x, int y) { return min(x, y); });
        ranges::reverse(minSuffix);

        for (int i = 0; i < n; ++i)
            if (nums[i] > minPrefix[i] && nums[i] > minSuffix[i])
                ans = min(ans, nums[i] + minPrefix[i] + minSuffix[i]);

        return ans == INT_MAX ? -1 : ans;
    }
};
```

1996. 291.cpp

Path: LeetCode-main/solutions/291. Word Pattern II/291.cpp

```
class Solution {
public:
    bool wordPatternMatch(string pattern, string s) {
        return isMatch(pattern, 0, s, 0, unordered_map<char, string>(),
                        unordered_set<string>());
    }

private:
    bool isMatch(const string& pattern, int i, const string& s, int j,
                unordered_map<char, string>&& charToString,
                unordered_set<string>&& seen) {
        if (i == pattern.length() && j == s.length())
            return true;
        if (i == pattern.length() || j == s.length())
            return false;

        const char c = pattern[i];

        if (const auto it = charToString.find(c); it != charToString.cend()) {
            const string& t = it->second;
            // See if we can match t with s[j..n).
            if (s.substr(j).find(t) == string::npos)
                return false;

            // If there's a match, continue to match the rest.
            return isMatch(pattern, i + 1, s, j + t.length(), std::move(charToString),
                          std::move(seen));
        }

        for (int k = j; k < s.length(); ++k) {
            const string& t = s.substr(j, k - j + 1);

            // This string is mapped by another character.
            if (seen.contains(t))
                continue;

            charToString[c] = t;
            seen.insert(t);

            if (isMatch(pattern, i + 1, s, k + 1, std::move(charToString),
                      std::move(seen)))
                return true;

            // Backtrack.
            charToString.erase(c);
            seen.erase(t);
        }

        return false;
    }
};
```

1997. 2910.cpp

Path: LeetCode-main/solutions/2910. Minimum Number of Groups to Create a Valid Assignment/2910.cpp

```
class Solution {
public:
    int minGroupsForValidAssignment(vector<int>& nums) {
        unordered_map<int, int> count;
        int minFreq = nums.size();

        for (const int num : nums)
            ++count[num];

        for (const auto& [_, freq] : count)
            minFreq = min(minFreq, freq);

        for (int groupSize = minFreq; groupSize >= 1; --groupSize) {
            const int numGroups = getNumGroups(count, groupSize);
            if (numGroups > 0)
                return numGroups;
        }

        throw;
    }

private:
    // Returns the number of groups if each group's size is `groupSize` or
    // `groupSize + 1`.
    int getNumGroups(unordered_map<int, int>& count, int groupSize) {
        int numGroups = 0;
        for (const auto& [_, freq] : count) {
            const int a = freq / (groupSize + 1);
            const int b = freq % (groupSize + 1);
            if (b == 0) {
                numGroups += a;
            } else if (groupSize - b <= a) {
                // Assign 1 number from `groupSize - b` out of `a` groups to this group,
                // so we'll have `a - (groupSize - b)` groups of size `groupSize + 1`
                // and `groupSize - b + 1` groups of size `groupSize`. In total, we have
                // `a + 1` groups.
                numGroups += a + 1;
            } else {
                return 0;
            }
        }
        return numGroups;
    }
};
```

1998. 2911.cpp

Path: LeetCode-main/solutions/2911. Minimum Changes to Make K Semi-palindromes/2911.cpp

```
class Solution {
public:
    int minimumChanges(string s, int k) {
        const int n = s.length();
        // factors[i] := factors of i
        const vector<vector<int>> factors = getFactors(n);
        // cost[i][j] := changes to make s[i..j] a semi-palindrome
        const vector<vector<int>> cost = getCost(s, n, factors);
        // dp[i][j] := the minimum changes to split s[i:] into j valid parts
        vector<vector<int>> dp(n + 1, vector<int>(k + 1, n));

        dp[n][0] = 0;

        for (int i = n - 1; i >= 0; --i)
            for (int j = 1; j <= k; ++j)
                for (int l = i + 1; l < n; ++l)
                    dp[i][j] = min(dp[i][j], dp[l + 1][j - 1] + cost[i][l]);

        return dp[0][k];
    }

private:
    vector<vector<int>> getFactors(int n) {
        vector<vector<int>> factors(n + 1);
        for (int i = 1; i <= n; ++i)
            factors[i].push_back(1);
        for (int d = 2; d < n; ++d)
            for (int i = d * 2; i <= n; i += d)
                factors[i].push_back(d);
        return factors;
    }

    vector<vector<int>> getCost(const string& s, int n,
                             const vector<vector<int>>& factors) {
        vector<vector<int>> cost(n, vector<int>(n));
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j) {
                const int length = j - i + 1;
                int minCost = length;
                for (const int d : factors[length])
                    minCost = min(minCost, getCost(s, i, j, d));
                cost[i][j] = minCost;
            }
        return cost;
    }

    // Returns the cost to make s[i..j] a semi-palindrome of `d`.
    int getCost(const string& s, int i, int j, int d) {
        int cost = 0;
        for (int offset = 0; offset < d; ++offset) {
            int l = i + offset;
            int r = j - d + 1 + offset;
            while (l < r) {
                if (s[l] != s[r])
                    ++cost;
                l += d;
                r -= d;
            }
        }
        return cost;
    }
};
```

1999. 2912-2.cpp

Path: LeetCode-main/solutions/2912. Number of Ways to Reach Destination in the Grid/2912-2.cpp

```
class Solution {
public:
    int numberOfWays(int n, int m, int k, vector<int>& source,
                    vector<int>& dest) {
        constexpr int kMod = 1'000'000'007;
        // the number of ways of `source` to `dest` using steps so far
        int ans = source[0] == dest[0] && source[1] == dest[1];
        // the number of ways of `source` to dest's row using steps so far
        int row = source[0] == dest[0] && source[1] != dest[1];
        // the number of ways of `source` to dest's col using steps so far
        int col = source[0] != dest[0] && source[1] == dest[1];
        // the number of ways of `source` to others using steps so far
        int others = source[0] != dest[0] && source[1] != dest[1];

        for (int i = 0; i < k; ++i) {
            const int nextAns = (row + col) % kMod;
            const int nextRow = static_cast<int>((ans * (m - 1L) + // -self
                                                    row * (m - 2L) + //-self, -center
                                                    others) %
                                                    kMod);
            const int nextCol = static_cast<int>((ans * (n - 1L) + // -self
                                                    col * (n - 2L) + // -self, -center
                                                    others) %
                                                    kMod);
            const int nextOthers =
                static_cast<int>((row * (n - 1L) + // -self
                                   col * (m - 1L) + // -self
                                   others * (m + n - 1 - 3L)) % // -self, -row, -col
                                   kMod);
            ans = nextAns;
            row = nextRow;
            col = nextCol;
            others = nextOthers;
        }
        return ans;
    }
};
```

2000. 2912.cpp

Path: LeetCode-main/solutions/2912. Number of Ways to Reach Destination in the Grid/2912.cpp

```
class Solution {
public:
    int numberOfWays(int n, int m, int k, vector<int>& source,
                    vector<int>& dest) {
        constexpr int kMod = 1'000'000'007;
        // dp[i][0] := the number of ways of `source` to `dest` using i
        // steps dp[i][1] := the the number of ways of `source` to dest's row
        // using i steps dp[i][2] := the the number of ways of `source` to
        // dest's col using i steps dp[i][3] := the the number of ways of
        // `source` to others using i steps
        vector<vector<int>> dp(k + 1, vector<int>(4));
        if (source == dest)
            dp[0][0] = 1;
        else if (source[0] == dest[0])
            dp[0][1] = 1;
        else if (source[1] == dest[1])
            dp[0][2] = 1;
        else
            dp[0][3] = 1;

        for (int i = 1; i <= k; i++) {
            dp[i][0] = (dp[i - 1][1] + dp[i - 1][2]) % kMod;
            dp[i][1] = ((dp[i - 1][0] * (m - 1L) + // -self
                        dp[i - 1][1] * (m - 2L) + // -self, -center
                        dp[i - 1][3]) %
                       kMod);
            dp[i][2] = ((dp[i - 1][0] * (n - 1L) + // -self
                        dp[i - 1][2] * (n - 2L) + // -self, -center
                        dp[i - 1][3]) %
                       kMod);
            dp[i][3] = ((dp[i - 1][1] * (n - 1L) + // -self
                        dp[i - 1][2] * (m - 1L) + // -self
                        dp[i - 1][3] * (m + n - 1 - 3L)) // -self, -row, -col
                       % kMod);
        }

        return dp[k][0];
    }
};
```


... 1495 more files omitted to avoid huge PDF ...